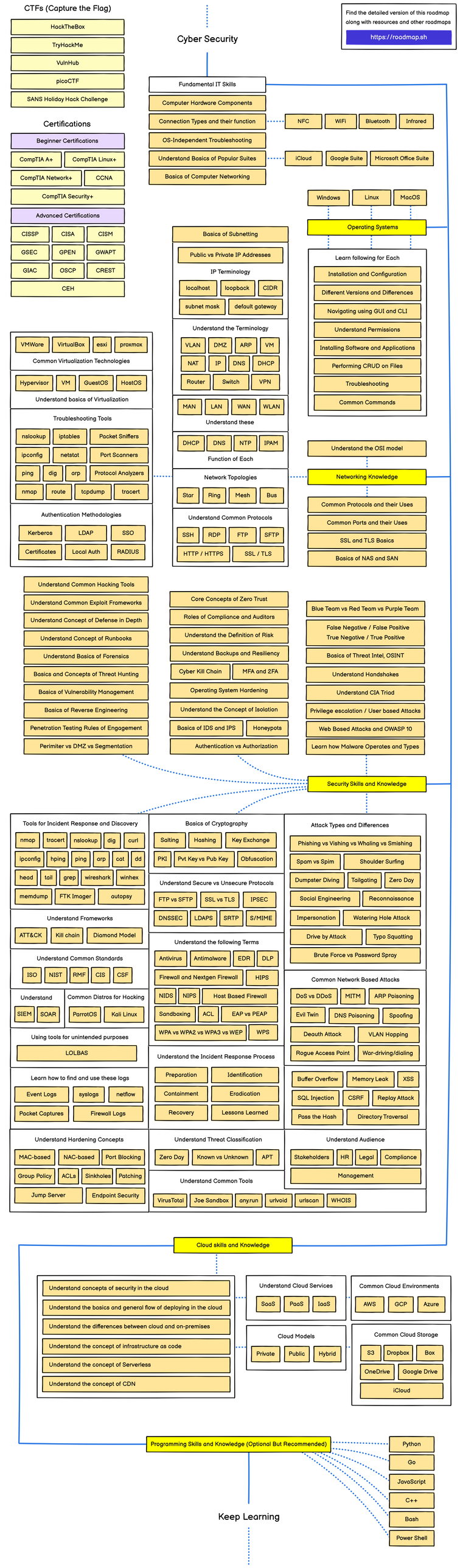

CTFs (Capture the Flag)
HackTheBox
TryHackMe
VulnHub
picoCTF
SANS Holiday Hack Challenge
Certifications
Beginner Certifications
CompTIA A+
CompTIA Linux+
CompTIA Network+
CCNA
CompTIA Security+
Advanced Certifications
CISSP
CISA
CISM
GSEC
GPEN
GWAPT
GIAC
OSCP
CREST
CEH
Cyber Security
Find the detailed version of this roadmap along with resources and other roadmaps
https://roadmap.sh
Fundamental IT Skills
Computer Hardware Components
Connection Types and their function
NFC
WiFi
Bluetooth
Infrared
OS-Independent Troubleshooting
Understand Basics of Popular Suites
iCloud
Google Suite
Microsoft Office Suite
Basics of Computer Networking
Windows
Linux
MacOS
Operating Systems
Learn following for Each
Installation and Configuration
Different Versions and Differences
Navigating using GUI and CLI
Understand Permissions
Installing Software and Applications
Performing CRUD on Files
Troubleshooting
Common Commands
Basics of Subnetting
Public vs Private IP Addresses
IP Terminology
localhost
loopback
CIDR
subnet mask
default gateway
Understand the Terminology
VLAN
DMZ
ARP
VM
NAT
IP
DNS
DHCP
Router
Switch
VPN
MAN
LAN
WAN
WLAN
Understand these
DHCP
DNS
NTP
IPAM
Function of Each
Network Topologies
Star
Ring
Mesh
Bus
Understand Common Protocols
SSH
RDP
FTP
SFTP
HTTP / HTTPS
SSL / TLS
VMWare
VirtualBox
esxi
proxmox
Common Virtualization Technologies
Hypervisor
VM
GuestOS
HostOS
Understand basics of Virtualization
Troubleshooting Tools
nslookup
iptables
Packet Sniffers
ipconfig
netstat
Port Scanners
ping
dig
arp
Protocol Analyzers
nmap
route
tcpdump
tracert
Authentication Methodologies
Kerberos
LDAP
SSO
Certificates
Local Auth
RADIUS
Understand the OSI model
Networking Knowledge
Common Protocols and their Uses
Common Ports and their Uses
SSL and TLS Basics
Basics of NAS and SAN
Understand Common Hacking Tools
Understand Common Exploit Frameworks
Understand Concept of Defense in Depth
Understand Concept of Runbooks
Understand Basics of Forensics
Basics and Concepts of Threat Hunting
Basics of Vulnerability Management
Basics of Reverse Engineering
Penetration Testing Rules of Engagement
Perimiter vs DMZ vs Segmentation
Core Concepts of Zero Trust
Roles of Compliance and Auditors
Understand the Definition of Risk
Understand Backups and Resiliency
Cyber Kill Chain
MFA and 2FA
Operating System Hardening
Understand the Concept of Isolation
Basics of IDS and IPS
Honeypots
Authentication vs Authorization
Blue Team vs Red Team vs Purple Team
False Negative / False Positive
True Negative / True Positive
Basics of Threat Intel, OSINT
Understand Handshakes
Understand CIA Triad
Privilege escalation / User based Attacks
Web Based Attacks and OWASP 10
Learn how Malware Operates and Types
Security Skills and Knowledge
Tools for Incident Response and Discovery
nmap
tracert
nslookup
dig
curl
ipconfig
hping
ping
arp
cat
dd
head
tail
grep
wireshark
winhex
memdump
FTK Imager
autopsy
Understand Frameworks
ATT&CK
Kill chain
Diamond Model
Understand Common Standards
ISO
NIST
RMF
CIS
CSF
Understand
SIEM
SOAR
Common Distros for Hacking
ParrotOS
Kali Linux
Using tools for unintended purposes
LOLBAS
Learn how to find and use these logs
Event Logs
syslogs
netflow
Packet Captures
Firewall Logs
Basics of Cryptography
Salting
Hashing
Key Exchange
PKI
Pvt Key vs Pub Key
Obfuscation
Understand Secure vs Unsecure Protocols
FTP vs SFTP
SSL vs TLS
IPSEC
DNSSEC
LDAPS
SRTP
S/MIME
Understand the following Terms
Antivirus
Antimalware
EDR
DLP
Firewall and Nextgen Firewall
HIPS
NIDS
NIPS
Host Based Firewall
Sandboxing
ACL
EAP vs PEAP
WPA vs WPA2 vs WPA3 vs WEP
WPS
Understand the Incident Response Process
Preparation
Identification
Containment
Eradication
Recovery
Lessons Learned
Attack Types and Differences
Phishing vs Vishing vs Whaling vs Smishing
Spam vs Spim
Shoulder Surfing
Dumpster Diving
Tailgating
Zero Day
Social Engineering
Reconnaissance
Impersonation
Watering Hole Attack
Drive by Attack
Typo Squatting
Brute Force vs Password Spray
Common Network Based Attacks
DoS vs DDoS
MITM
ARP Poisoning
Evil Twin
DNS Poisoning
Spoofing
Deauth Attack
VLAN Hopping
Rogue Access Point
War-driving/dialing
Buffer Overflow
Memory Leak
XSS
SQL Injection
CSRF
Replay Attack
Pass the Hash
Directory Traversal
Understand Hardening Concepts
MAC-based
NAC-based
Port Blocking
Group Policy
ACLs
Sinkholes
Patching
Jump Server
Endpoint Security
Understand Threat Classification
Zero Day
Known vs Unknown
APT
Understand Common Tools
VirusTotal
Joe Sandbox
any.run
urlvoid
urlscan
WHOIS
Understand Audience
Stakeholders
HR
Legal
Compliance
Management
Cloud skills and Knowledge
Understand concepts of security in the cloud
Understand the basics and general flow of deploying in the cloud
Understand the differences between cloud and on-premises
Understand the concept of infrastructure as code
Understand the concept of Serverless
Understand the concept of CDN
Understand Cloud Services
SaaS
PaaS
IaaS
Cloud Models
Private
Public
Hybrid
Common Cloud Environments
AWS
GCP
Azure
Common Cloud Storage
S3
Dropbox
Box
OneDrive
Google Drive
iCloud
Programming Skills and Knowledge (Optional But Recommended)
Python
Go
JavaScript
C++
Bash
Power Shell
Keep Learning

LEARNING MADE EASY

3rd Edition

Beginning Programming with Python®

for dummies®
A Wiley Brand

Get started with your first Python application

Troubleshoot and resolve errors in your code

Code anywhere and use magic functions to do it

John Paul Mueller
Dreams about teaching real pythons to write code

Beginning Programming with Python®

Beginning Programming with Python®

3rd Edition

by John Paul Mueller

for dummies®
A Wiley Brand

Beginning Programming with Python® For Dummies®, 3rd Edition

Published by: **John Wiley & Sons, Inc.,** 111 River Street, Hoboken, NJ 07030-5774, www.wiley.com

Copyright © 2023 by John Wiley & Sons, Inc., Hoboken, New Jersey

Media and software compilation copyright © 2023 by John Wiley & Sons, Inc. All rights reserved.

Published simultaneously in Canada

No part of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning or otherwise, except as permitted under Sections 107 or 108 of the 1976 United States Copyright Act, without the prior written permission of the Publisher. Requests to the Publisher for permission should be addressed to the Permissions Department, John Wiley & Sons, Inc., 111 River Street, Hoboken, NJ 07030, (201) 748-6011, fax (201) 748-6008, or online at http://www.wiley.com/go/permissions.

Trademarks: Wiley, For Dummies, the Dummies Man logo, Dummies.com, Making Everything Easier, and related trade dress are trademarks or registered trademarks of John Wiley & Sons, Inc. and may not be used without written permission. Python is a registered trademark of Python Software Foundation Corporation. All other trademarks are the property of their respective owners. John Wiley & Sons, Inc. is not associated with any product or vendor mentioned in this book.

LIMIT OF LIABILITY/DISCLAIMER OF WARRANTY: WHILE THE PUBLISHER AND AUTHORS HAVE USED THEIR BEST EFFORTS IN PREPARING THIS WORK, THEY MAKE NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE ACCURACY OR COMPLETENESS OF THE CONTENTS OF THIS WORK AND SPECIFICALLY DISCLAIM ALL WARRANTIES, INCLUDING WITHOUT LIMITATION ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. NO WARRANTY MAY BE CREATED OR EXTENDED BY SALES REPRESENTATIVES, WRITTEN SALES MATERIALS OR PROMOTIONAL STATEMENTS FOR THIS WORK. THE FACT THAT AN ORGANIZATION, WEBSITE, OR PRODUCT IS REFERRED TO IN THIS WORK AS A CITATION AND/OR POTENTIAL SOURCE OF FURTHER INFORMATION DOES NOT MEAN THAT THE PUBLISHER AND AUTHORS ENDORSE THE INFORMATION OR SERVICES THE ORGANIZATION, WEBSITE, OR PRODUCT MAY PROVIDE OR RECOMMENDATIONS IT MAY MAKE. THIS WORK IS SOLD WITH THE UNDERSTANDING THAT THE PUBLISHER IS NOT ENGAGED IN RENDERING PROFESSIONAL SERVICES. THE ADVICE AND STRATEGIES CONTAINED HEREIN MAY NOT BE SUITABLE FOR YOUR SITUATION. YOU SHOULD CONSULT WITH A SPECIALIST WHERE APPROPRIATE. FURTHER, READERS SHOULD BE AWARE THAT WEBSITES LISTED IN THIS WORK MAY HAVE CHANGED OR DISAPPEARED BETWEEN WHEN THIS WORK WAS WRITTEN AND WHEN IT IS READ. NEITHER THE PUBLISHER NOR AUTHORS SHALL BE LIABLE FOR ANY LOSS OF PROFIT OR ANY OTHER COMMERCIAL DAMAGES, INCLUDING BUT NOT LIMITED TO SPECIAL, INCIDENTAL, CONSEQUENTIAL, OR OTHER DAMAGES.

For general information on our other products and services, please contact our Customer Care Department within the U.S. at 877-762-2974, outside the U.S. at 317-572-3993, or fax 317-572-4002. For technical support, please visit https://hub.wiley.com/community/support/dummies.

Wiley publishes in a variety of print and electronic formats and by print-on-demand. Some material included with standard print versions of this book may not be included in e-books or in print-on-demand. If this book refers to media such as a CD or DVD that is not included in the version you purchased, you may download this material at http://booksupport.wiley.com. For more information about Wiley products, visit www.wiley.com.

Library of Congress Control Number: 2022948492

ISBN: 978-1-119-91377-1 (pbk); ISBN 978-1-119-91378-8 (ebk); ISBN 978-1-119-91379-5 (ebk)

Contents at a Glance

Introduction 1

Part 1: Getting Started with Python 7

CHAPTER 1: Talking to Your Computer 9
CHAPTER 2: Working with Google Colab 23
CHAPTER 3: Interacting with Python 41
CHAPTER 4: Writing Your First Application 57
CHAPTER 5: Performing Magic 79

Part 2: Talking the Talk 93

CHAPTER 6: Storing and Modifying Information 95
CHAPTER 7: Managing Information 107
CHAPTER 8: Making Decisions 127
CHAPTER 9: Performing Repetitive Tasks 143
CHAPTER 10: Dealing with Errors 157

Part 3: Performing Common Tasks 187

CHAPTER 11: Interacting with Packages 189
CHAPTER 12: Working with Strings 215
CHAPTER 13: Managing Lists 239
CHAPTER 14: Collecting All Sorts of Data 257
CHAPTER 15: Creating and Using Classes 279

Part 4: Performing Advanced Tasks 301

CHAPTER 16: Storing Data in Files 303
CHAPTER 17: Sending an Email 321

Part 5: The Part of Tens 337

CHAPTER 18: Ten Amazing Programming Resources 339
CHAPTER 19: Ten Ways to Make a Living with Python 349
CHAPTER 20: Ten Tools That Enhance Your Python Experience 357
CHAPTER 21: Ten (Plus) Libraries You Need to Know About 369

Index 379

Table of Contents

INTRODUCTION 1
About This Book 1
Foolish Assumptions 2
Icons Used in This Book 3
Beyond the Book 4
Where to Go from Here 5

PART 1: GETTING STARTED WITH PYTHON 7

CHAPTER 1: Talking to Your Computer 9
Understanding Why You Want to Talk to Your Computer 10
Knowing that an Application Is a Form of Communication 11
Thinking about procedures you use daily 11
Writing procedures down 12
Seeing applications as being like any other procedure 13
Making your computer do funny things 13
Defining What an Application Is 13
Understanding that computers use a special language 14
Helping humans speak to the computer 14
Understanding Why Python Is So Cool 16
Unearthing the reasons for using Python 16
Deciding how you can personally benefit from Python 17
Discovering which organizations use Python 18
Finding useful Python applications 19
Comparing Python to other languages 20

CHAPTER 2: Working with Google Colab 23
Defining Google Colab 24
Understanding what Google Colab does 24
Working with Google Colab features 25
Working with Notebooks 29
Creating a new notebook 29
Opening existing notebooks 30
Saving notebooks using GitHub 32
Getting the gist of things 33
Working with Drive 34
Performing Common Tasks 35
Creating code cells 35
Creating text cells 37
Creating special cells 38

Editing cells 38
Moving cells 39
Using Hardware Acceleration 39
Executing the Code 39
Getting Help 40

CHAPTER 3: **Interacting with Python** 41
Typing a Command 42
Telling the computer what to do 43
Telling the computer you're done 43
Seeing the result 44
Getting Python's Help 45
Entering into help mode 46
Asking for help 48
Leaving help mode 50
Obtaining help directly 50
Finding Out More about Functions and Objects 52
Yelling "Hello There" doesn't help: Use dir() instead 52
What are those double underscores all about? 53
Drilling, drilling, drilling down into classes 54
Playing the Part of Inspector 55
Gaining access to inspect 55
Using inspect 55
Is reflection really like looking in a mirror? 56

CHAPTER 4: **Writing Your First Application** 57
Understanding Why IDEs Are Important 58
Creating better code 58
Debugging functionality 59
Defining why notebooks are useful 59
Creating the Application 60
Developing the code 60
Adding documentation cells 61
Other cell content 62
Playing around with scratch cells 63
Interacting with form fields 64
Running the Application 66
Seeing the result 67
Viewing the executed code history 68
Understanding the Use of Indentation 68
Adding Comments 70
Understanding comments 70
Creating multiline comments 71
Using comments to leave yourself reminders 72
Using comments to keep code from executing 73

Making Your Notebook Informative, Descriptive, and Pretty 74
Working with text cells 74
Adding section headers 76
Interacting with the table of contents 76
Renaming a notebook. 76
Closing and Halting a Notepad 77

CHAPTER 5: **Performing Magic** 79
Understanding the Concept of a Magic Command 80
What Kind of Magic Do You Want to Perform? 82
Working with line magic commands. 82
Working with cell magic commands 83
Learning the Magic Commands 83
Getting magic command details 83
An overview of line magic commands 85
An overview of cell magic commands 90

PART 2: TALKING THE TALK 93

CHAPTER 6: **Storing and Modifying Information** 95
Storing Information. 96
Seeing variables as storage boxes 96
Using the right box to store the data 96
Defining the Essential Python Data Types 97
Putting information into variables 97
Understanding the numeric types 97
Understanding Boolean values 102
Understanding strings 103
Working with Dates and Times 104

CHAPTER 7: **Managing Information** 107
Controlling How Python Views Data 108
Making comparisons. 108
Understanding how computers make comparisons 109
Working with Operators 109
Defining the operators 109
Understanding operator precedence. 116
Creating and Using Functions 117
Viewing functions as code packages. 117
Understanding code reusability 117
Defining a function 118
Accessing functions. 119
Sending information to functions 119
Returning information from functions 123
Comparing function output 124
Getting User Input. 125

CHAPTER 8: **Making Decisions** 127
Making Simple Decisions by Using the if Statement 128
Understanding the if statement 128
Using the if statement in an application 129
Choosing Alternatives by Using the if. . .else Statement 133
Understanding the if. . .else statement 133
Using the if. . .else statement in an application 134
Using the if. . .elif statement in an application 135
Using Nested Decision Statements. 138
Using multiple if or if. . .else statements 138
Combining other types of decisions 140

CHAPTER 9: **Performing Repetitive Tasks** 143
Processing Data Using the for Statement 144
Understanding the for statement 145
Creating a basic for loop. 145
Controlling execution with the break statement 146
Controlling execution with the continue statement. 148
Doing nothing with the pass statement 149
Validating input with the else statement 150
Processing Data by Using the while Statement 152
Understanding the while statement 152
Using the while statement in an application 153
Nesting Loop Statements 154

CHAPTER 10: **Dealing with Errors** 157
Knowing Why Python Doesn't Understand You 158
Considering the Sources of Errors 159
Classifying when errors occur 160
Distinguishing error types 162
Catching Exceptions 164
Basic exception handling 165
Handling more specific to less specific exceptions 176
Nested exception handling 178
Raising Exceptions. 180
Raising exceptions during exceptional conditions 181
Passing error information to the caller 181
Deciding to Say "Oops" in Your Own Way: Custom Exceptions 182
Using the finally Clause 184

PART 3: PERFORMING COMMON TASKS 187

CHAPTER 11: **Interacting with Packages** 189
Creating Code Groupings 190
I'm confused! Understanding modules versus packages 190
Creating your first package 191

Understanding the package types 194
Considering the package cache 195
Importing Packages 196
Using the import statement 197
Using the from. . .import statement 198
Using the import. . .as statement 200
Finding Packages 201
Locating packages on disk 202
Locating packages online 204
Downloading Packages from Other Sources 204
Opening the Anaconda Prompt 205
Working with conda packages 205
And just why is conda missing in Colab? 208
Installing packages by using pip 209
Installing packages using the %pip magics 209
Viewing the Package Content 210
Viewing Package Documentation 211
Using !pydoc to access PyDoc 211
Typing a search term 212

CHAPTER 12: **Working with Strings** 215
Understanding That Strings Are Different 216
Defining a character by using numbers 216
Using characters to create strings 217
Creating Stings with Special Characters 218
Selecting Individual Characters 221
Slicing and Dicing Strings 223
Locating a Value in a String 227
Using String Interpolation 229
Employing the % (modulo) approach 229
Working with the format() function 232
Simplifying things using f-string 236
Creating and using string templates 237

CHAPTER 13: **Managing Lists** 239
Organizing Information in an Application 240
Defining organization using lists 240
Understanding how computers view lists 241
Creating Lists 242
Accessing Lists 243
Looping through Lists 244
Modifying Lists 245
Searching Lists 249

Sorting Lists 250
Printing Lists 251
Working with the Counter Object 253

CHAPTER 14: Collecting All Sorts of Data 257
Understanding Collections 258
Working with Tuples 259
Working with Dictionaries 262
Creating and using a dictionary 263
Working with nested dictionaries 266
Replacing the switch statement with a dictionary 268
Creating Stacks Using Lists 271
Working with queues 273
Working with deques 275

CHAPTER 15: Creating and Using Classes 279
Considering the Parts of a Class 280
Creating the class definition 280
Considering the built-in class attributes 281
Working with methods 282
Working with constructors 285
Working with variables 287
Using methods with variable argument lists 289
Overloading operators 291
Creating and Using an External Class 293
Developing the external class 293
Using MyClass in an application 295
Extending Classes to Make New Classes 296
Building the child class 297
Testing class inheritance in an application 298

PART 4: PERFORMING ADVANCED TASKS 301

CHAPTER 16: Storing Data in Files 303
Understanding How Permanent Storage Works 304
Creating Content for Permanent Storage 306
Creating a File 309
Reading File Content 313
Updating File Content 315
Deleting a File 319

CHAPTER 17: Sending an Email 321
Understanding What Happens When You Send Email 322
Viewing email as you do a letter 323
Defining the parts of the envelope 324

Defining the parts of the letter 329
Putting everything together for text messages. 334
Working with an HTML message 335

PART 5: THE PART OF TENS 337

CHAPTER 18: Ten Amazing Programming Resources 339
Working with the Python Documentation Online. 340
Discovering Details Using a Tutorial 341
Performing Web Programming by Using Python 342
Locating Useful (versus Useless) Modules. 343
Creating Applications Faster by Using an IDE 343
Checking Your Syntax with Greater Ease 344
Using XML to Your Advantage 345
Getting Past the Common Python Newbie Errors 346
Understanding Unicode 346
Making Your Python Application Fast. 347

CHAPTER 19: Ten Ways to Make a Living with Python. 349
Working in QA 350
Becoming the IT Staff for a Smaller Organization 351
Performing Specialty Scripting for Applications 352
Administering a Network 353
Teaching Programming Skills. 353
Helping People Decide on Location 354
Performing Data Mining 354
Interacting with Embedded Systems 355
Carrying Out Scientific Tasks 355
Performing Real-Time Analysis of Data 356

CHAPTER 20: Ten Tools That Enhance Your Python Experience 357
Tracking Bugs with Roundup Issue Tracker. 358
Creating a Virtual Environment by Using VirtualEnv 359
Installing Your Application by Using PyInstaller 361
Building Developer Documentation by Using pdoc 362
Developing Application Code by Using Komodo Edit. 363
Debugging Your Application by Using pydbgr. 363
Entering an Interactive Environment by Using IPython. 364
Testing Python Applications by Using PyUnit 365
Tidying Your Code by Using Isort 365
Providing Version Control by Using Mercurial 366

CHAPTER 21: **Ten (Plus) Libraries You Need to Know About** 369
Developing a Secure Environment by Using CryptLib 370
Interacting with Databases by Using SQLAlchemy 371
Seeing the World by Using Google Maps 371
Adding a Graphical User Interface by Using TkInter 372
Providing a Nice Tabular Data Presentation by Using PrettyTable . . . 373
Enhancing Your Application with Sound by Using PyAudio 373
Manipulating Images by Using PyQtGraph 374
Locating Your Information by Using Whoosh 375
Creating an Interoperable Java Environment by Using JPype 376
Accessing Local Network Resources by Using Twisted Matrix 377
Accessing Internet Resources by Using Libraries 377

INDEX 379

Introduction

Python is an example of a language that does everything right within the domain of things that it's designed to do. This isn't just me saying it, either: Programmers have voted by using Python enough that it's now the first-ranked language in the world (see https://www.tiobe.com/tiobe-index/ for details). The amazing thing about Python is that you really can write an application on one platform and use it on every other platform that you need to support. In contrast to other programming languages that promised to provide platform independence, Python really does make that independence possible. In this case, the promise is as good as the result you get.

Python emphasizes code readability and a concise syntax that lets you write applications using fewer lines of code than other programming languages require. You can also use a coding style that meets your needs, given that Python supports the functional, imperative, object-oriented, and procedural coding styles (see Chapter 3 for details). In addition, because of the way Python works, you find it used in all sorts of fields that are filled with nonprogrammers. *Beginning Programming with Python For Dummies,* 3rd Edition is designed to help everyone, including nonprogrammers, get up and running with Python quickly.

Some people view Python as a scripted language, but it really is so much more. (Chapter 19 gives you just an inkling of the occupations that rely on Python to make things work.) However, Python does lend itself to educational and other uses for which other programming languages can fall short. In fact, this book uses both Google Colab and Jupyter Notebook for examples, which rely on the highly readable literate programming paradigm advanced by Stanford computer scientist Donald Knuth (see Chapter 4 for details). Your examples end up looking like highly readable reports that almost anyone can understand with ease.

About This Book

Beginning Programming with Python For Dummies, 3rd Edition is all about getting up and running with Python quickly. You want to learn the language fast so that you can become productive in using it to perform your real job, which could be anything. With the goal in mind of making things simple in every environment, this book emphasizes a code anywhere approach. If you want to code on your

smart phone (not really recommended unless you like to squint a lot), you can do so as long as your smart phone has a browser that can access Google Colab. Likewise, coding while watching a TV equipped with a keyboard is possible, but not necessarily recommended because of the distractions involved. Besides, trying to write code that you can see only in that small square in the corner of the screen would be very tough. Highly recommended is your desktop, laptop, or tablet.

Unlike most books on the topic, this one starts you right at the beginning by showing you what makes Python different from other languages and how it can help you perform useful work in a job other than programming. As a result, you gain an understanding of what you need to do from the start, using hands-on examples and spending a good deal of time performing actually useful tasks. By the time you finish working through the examples in this book, you'll be writing simple programs and performing tasks such as sending an email using Python. No, you won't be an expert, but you will be able to use Python to meet specific needs in the job environment. To make absorbing the concepts even easier, this book uses the following conventions:

- Text that you're meant to type just as it appears in the book is **bold**. The exception is when you're working through a step list: Because each step is bold, the text to type is not bold.
- When you see words in *italics* as part of a typing sequence, you need to replace that value with something that works for you. For example, if you see "Type ***Your Name*** and press Enter," you need to replace *Your Name* with your actual name.
- Web addresses and programming code appear in `monofont`. If you're reading a digital version of this book on a device connected to the Internet, note that you can click the web address to visit that website, like this: `www.dummies.com`.
- When you need to type command sequences, you see them separated by a special arrow, like this: File ➪ New File. In this case, you go to the File menu first and then select the New File entry on that menu. The result is that you see a new file created.

Foolish Assumptions

You might find it difficult to believe that I've assumed anything about you — after all, I haven't even met you yet! Although most assumptions are indeed foolish, I made these assumptions to provide a starting point for the book.

Familiarity with the platform you want to use is important because the book doesn't provide any guidance in this regard. To provide you with maximum information about Python, this book doesn't discuss any platform-specific issues. You really do need to know how to install applications (when working with a desktop system), use applications, work with your browser, and generally work with your chosen platform before you begin working with this book.

This book also assumes that you can locate information on the Internet. Sprinkled throughout are numerous references to online material that will enhance your learning experience. However, these added sources are useful only if you actually find and use them.

Icons Used in This Book

As you read this book, you see icons in the margins that indicate material of interest (or not, as the case may be). This section briefly describes each icon in this book.

TIP

Tips are nice because they help you save time or perform some task without a lot of extra work. The tips in this book are time-saving techniques or pointers to resources that you should try in order to get the maximum benefit from Python.

WARNING

I don't want to sound like an angry parent or some kind of maniac, but you should avoid doing anything marked with a Warning icon. Otherwise, you could find that your program only serves to confuse users, who will then refuse to work with it.

TECHNICAL STUFF

Whenever you see this icon, think advanced tip or technique. You might find these tidbits of useful information just too boring for words, or they could contain the solution you need to get a program running. Skip these bits of information whenever you like.

REMEMBER

If you don't get anything else out of a particular chapter or section, remember the material marked by this icon. This text usually contains an essential process or a bit of information that you must know to write Python programs successfully.

Beyond the Book

This book isn't the end of your Python programming experience — it's really just the beginning. I provide online content to make this book more flexible and better able to meet your needs. That way, as I receive email from you, I can do things like address questions and tell you how updates to either Python or its associated libraries affect book content. In fact, you gain access to all these cool additions:

- **Cheat sheet:** You remember using crib notes in school to make a better mark on a test, don't you? You do? Well, a cheat sheet is sort of like that. It provides you with some special notes about tasks that you can do with Python that not every other developer knows. You can find the cheat sheet for this book by going to `www.dummies.com` and searching for *Beginning Programming with Python For Dummies,* 3rd Edition Cheat Sheet. It contains really neat information like how to perform magic when using Python.
- **Updates:** Sometimes changes happen. For example, I might not have seen an upcoming change when I looked into my crystal ball during the writing of this book. In the past, that simply meant the book would become outdated and less useful, but you can now find updates to the book by going to `www.dummies.com` and searching for this book's title.

  In addition to these updates, check out the blog posts with answers to reader questions and demonstrations of useful book-related techniques at `http://blog.johnmuellerbooks.com/`.
- **Companion files:** Hey! Who really wants to type all the code in the book? Most readers would prefer to spend their time actually working through coding examples, rather than typing. Fortunately for you, the source code is available for download, so all you need to do is read the book to learn Python coding techniques. Each of the book examples even tells you precisely which example project to use. You can find these files by going to `www.dummies.com` and searching for this book's title. You can also find the downloadable source on my website at `http://www.johnmuellerbooks.com/source-code/`; just click the Download button for *Beginning Programming with Python For Dummies,* 3rd Edition. Be sure to unzip the file using the instructions at `https://support.microsoft.com/en-us/windows/zip-and-unzip-files-8d28fa72-f2f9-712f-67df-f80cf89fd4e5` before attempting to use the source code, even if you can see it in Windows Explorer.

Where to Go from Here

It's time to start your Programming with Python adventure! If you're a complete programming novice, you should start with Chapter 1 and progress through the book at a pace that allows you to absorb as much of the material as possible.

If you're a novice who's in an absolute rush to get going with Python as quickly as possible, you could skip to Chapter 2 with the understanding that you may find some topics a bit confusing later. Skipping to Chapter 3 is possible if you want to start working with Python immediately and have access to Google Colab or a Jupyter Notebook installation.

Readers who have some exposure to Python can save time by moving directly to Chapter 4. This chapter gets you started working with notebooks so that you have a better idea of how to work with Google Colab or Jupyter Notebook for the examples in the remainder of the book. Make sure you also read through Chapter 5, which tells you how to perform magic in notebooks.

Assuming that you already have access to either Google Colab or Jupyter Notebook and know how to use your IDE of choice, you can move directly to Chapter 6. You can always go back to earlier chapters as necessary when you have questions. However, it's important that you understand how each example works before moving to the next one. Every example has important lessons for you, and you could miss vital content if you start skipping too much information.

1 Getting Started with Python

IN THIS PART . . .

Defining the association between Python and applications

Using Google Colab to work with Python

Performing essential tasks using Python

Creating your first application

Performing feats of magic

IN THIS CHAPTER

- Talking to your computer
- Creating programs to talk to your computer
- Understanding programs and their creation
- Considering why you want to use Python

Chapter 1

Talking to Your Computer

Having a conversation with your computer might sound like the script of a science fiction movie. After all, the members of the *Enterprise* on *Star Trek* regularly talked with their computer. In fact, the computer often talked back. However, with the rise of Apple's Siri (`https://www.apple.com/siri/`), Amazon's Echo (`https://www.amazon.com/dp/B07XKF5RM3/`) and other interactive software (`https://windowsreport.com/talking-pc-software/`), perhaps you really don't find a conversation so unbelievable.

REMEMBER

Asking the computer for information is one thing, but providing it with instructions is quite another. This chapter considers why you want to instruct your computer about anything and what benefit you gain from it. You also discover the need for a special language when performing this kind of communication and why you want to use Python to accomplish it. However, the main thing to get out of this chapter is that programming is simply a kind of communication that is akin to other forms of communication you already have with your computer.

Understanding Why You Want to Talk to Your Computer

Talking to a machine may seem quite odd at first (then again, people do talk to cats, dogs, cars, toasters, and other odd assorted things), but it's necessary because a computer can't read your mind — yet. Mind-reading computers are getting closer, as described in the article at `https://www.psychnewsdaily.com/this-computer-can-read-your-mind-and-render-your-thoughts-as-pictures/`. Even if the computer did read your mind, it would still be communicating with you. Nothing can occur without an exchange of information between the machine and you. Activities such as

- Reading your email
- Writing about your vacation
- Finding the greatest gift in the world

are all examples of communication that occurs between a computer and you. That the computer further communicates with other machines or people to address requests that you make simply extends the basic idea that communication is necessary to produce any result.

In most cases, the communication takes place in a manner that is nearly invisible to you unless you really think about it. For example, when you visit a chat room (sometimes called *spaces* now; see `https://workspaceupdates.googleblog.com/2021/09/google-chat-rooms-are-now-spaces.html`) online, you might think that you're communicating with another person. However, you're communicating with your computer, your computer is communicating with the other person's computer through the chat room (whatever it consists of), and the other person's computer is communicating with that person (or possibly an AI). Figure 1-1 gives you an idea of what is actually taking place.

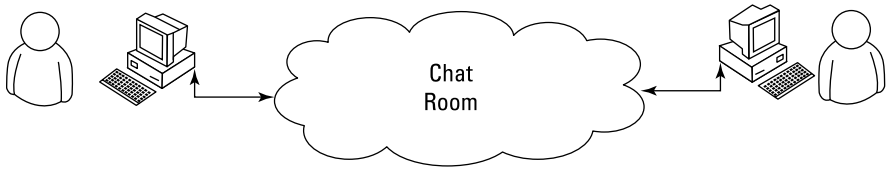


FIGURE 1-1: Communication with your computer may be invisible unless you really think about it.

Notice the cloud in the center of Figure 1-1. The cloud could contain anything, but you know that it at least contains other computers running other applications. These computers make it possible for your friend and you to chat. Now, think

about how easy the whole process seems when you're using the chat application. Even though all these things are going on in the background, it seems as if you're simply chatting with your friend, and the process itself is invisible.

Knowing that an Application Is a Form of Communication

Computer communication occurs through the use of applications. You use one application to answer your email, another to purchase goods, and still another to create a presentation. An *application* (sometimes called an *app*) provides the means to express human ideas to the computer in a manner the computer can understand and defines the tools needed to shape the data used for the communication in specific ways. Data used to express the content of a presentation is different from data used to purchase a present for your mother. The way you view, use, and understand the data is different for each task, so you must use different applications to interact with the data in a manner that both the computer and you can understand.

You can obtain applications to meet just about any general need you can conceive of today. In fact, you probably have access to applications for which you haven't even thought about a purpose yet. Programmers have been busy creating millions of applications of all types for many years now, so it may be hard to understand what you can accomplish by creating some new method for talking with your computer through an application. The answer comes down to thinking about the data and how you want to interact with it. Some data simply isn't common enough to have attracted the attention of a programmer, or you may need the data in a format that no application currently supports, so you don't have any way to tell the computer about it unless you create a custom application to do it. The following sections describe applications from the perspective of working with unique data in a manner that is special in some way.

Thinking about procedures you use daily

A *procedure* is simply a set of steps you follow to perform a task. For example, when making toast, you might use a procedure like this:

1. Get the bread and butter from the refrigerator.
2. Open the bread bag and take out two pieces of bread.
3. Remove the cover from the toaster.

4. Place each piece of bread in its own slot.
5. Push the toaster lever down to start toasting the bread.
6. Wait for the toasting process to complete.
7. Remove toast from the toaster.
8. Place toast on a plate.
9. Butter the toast.

Your procedure might vary from the one presented here, but it's unlikely that you'd butter the toast before placing it in the toaster. Of course, you do actually have to remove the bread from the wrapper before you toast it (placing the bread, wrapper and all, into the toaster would likely produce undesirable results). Most people never actually think about the procedure for making toast. However, you use a procedure like this one even though you don't think about it.

REMEMBER

Computers can't perform tasks without a procedure. You must tell the computer which steps to perform, the order in which to perform them, and any exceptions to the rule that could cause failure. All this information (and more) appears within an application. In short, an application is simply a written procedure that you use to tell the computer what to do, when to do it, and how to do it. Because you've been using procedures all your life, all you really need to do is apply the knowledge you already possess to what a computer needs to know about specific tasks.

Writing procedures down

When I was in grade school, our teacher asked us to write a paper about making toast. After we turned in our papers, she brought in a toaster and some loaves of bread. Each paper was read and demonstrated. None of our procedures worked as expected, but they all produced humorous results. In my case, I forgot to tell the teacher to remove the bread from the wrapper, so she dutifully tried to stuff the piece of bread, wrapper and all, into the toaster. The lesson stuck with me. Writing about procedures can be quite hard because we know precisely want we want to do, but often we leave steps out — we assume that the other person also knows precisely what we want to do.

Writing procedures down isn't really sufficient, though — you also need to test the procedure by asking someone who isn't familiar with the task to perform it using your procedure. When working with computers, the computer is your perfect test subject.

Seeing applications as being like any other procedure

A computer acts like the grade school teacher in my example in the previous section. When you write an application, you're writing a procedure that defines a series of steps that the computer should perform to accomplish whatever task you have in mind. If you leave out a step, the results won't be what you expected. The computer won't know what you mean or that you intended for it to perform certain tasks automatically. The only thing the computer knows is that you have provided it with a specific procedure and it needs to perform that procedure.

Making your computer do funny things

People eventually get used to the procedures you create. They automatically compensate for deficiencies in the procedure or make notes about things that were left out. In other words, people compensate for problems with the procedures that you write.

REMEMBER

When you begin writing computer programs, you'll get frustrated because computers perform tasks precisely and read your instructions literally. For example, if you tell the computer that a certain value should equal 5, the computer will look for a value of exactly 5. A human might see 4.9 and know that the value is good enough, but a computer doesn't see things that way. It sees a value of 4.9 and decides that it doesn't equal 5 exactly. In short, computers are inflexible, unintuitive, and unimaginative. When you write a procedure for a computer, the computer will do precisely as you ask absolutely every time and never modify your procedure or decide that you really meant for it to do something else. In some cases (not many), the results can actually be quite humorous (such as that time the computer began reciting a limerick that you meant to keep private). A sense of humor is helpful in computer programming.

Defining What an Application Is

As previously mentioned, applications provide the means to define and express human ideas in a manner that a computer can understand. To accomplish this goal, the application relies on one or more procedures that tell the computer how to perform the tasks related to the manipulation of data and its presentation. What you see onscreen is the text from your word processor, but to see that information, the computer requires procedures for retrieving the data from disk, putting it into a form you can understand, and then presenting it to you. The following sections define the specifics of an application in more detail.

Understanding that computers use a special language

Human language is complex and difficult to understand. Even applications such as Siri and Alexa have serious limits in understanding what you're saying. Over the years, computers have gained the capability to input human speech as data and to interpret certain spoken words as commands, but computers still don't understand human speech. What the computer does is match voice patterns to data it understands and then match that data to specific commands.

Given what you know from previous sections of this chapter, computers could never rely on human speech to understand the procedures you write. Computers always take things literally, so you'd end up with completely unpredictable results if you were to use human language to write applications. That's why humans use special languages, called *programming languages,* to communicate with computers. These special languages make it possible to write procedures that are both specific and completely understandable by both humans and computers.

TECHNICAL STUFF

Computers don't actually speak any language. They use binary codes to flip switches internally and to perform math calculations. Computers don't even understand letters — they understand only numbers. A special application turns the computer-specific language you use to write a procedure into binary codes. For the purposes of this book, you really don't need to worry too much about the low-level specifics of how computers work at the binary level. However, it's interesting to know that computers speak math and numbers, not really a language at all.

Helping humans speak to the computer

It's important to keep the purpose of an application in mind as you write it. An application is there to help humans speak to the computer in a certain way. Every application works with some type of data that is input, stored, manipulated, and output so that the humans using the application obtain a desired result. Whether the application is a game or a spreadsheet, the basic idea is the same. Computers work with data provided by humans to obtain a desired result.

When you create an application, you're providing a new method for humans to speak to the computer. The new approach you create will make it possible for other humans to view data in new ways. The communication between human and computer should be easy enough that the application actually disappears from view. Think about the kinds of applications you've used in the past. The best applications are the ones that let you focus on whatever data you're interacting with. For example, a game application is considered immersive only if you can

focus on the planet you're trying to save or the ship you're trying to fly, rather than the application that lets you do these things.

TIP

One of the best ways to start thinking about how you want to create an application is to look at other applications. Writing down what you like and dislike about other applications is a useful way to start discovering how you want your applications to look and work. Here are some questions you can ask yourself as you work with the applications:

- What do I find distracting about the application?
- Which features were easy to use?
- Which features were hard to use?
- How did the application make it easy to interact with my data?
- How would I make the data easier to work with?
- What do I hope to achieve with my application that this application doesn't provide?

Professional developers ask many other questions as part of creating an application, but these are good starter questions because they begin to help you think about applications as a means to help humans speak with computers. If you've ever found yourself frustrated by an application you used, you already know how other people will feel if you don't ask the appropriate questions when you create your application. Communication is the most important element of any application you create.

You can also start to think about the ways in which you work. Start writing procedures for the things you do. It's a good idea to take the process one step at a time and write everything you can think of about that step. When you get finished, ask someone else to try your procedure to see how it actually works. You might be surprised to learn that even with a lot of effort, you can easily forget to include steps.

WARNING

The world's worst application usually begins with a programmer who doesn't know what the application is supposed to do, why it's special, what need it addresses, or whom it is for. When you decide to create an application, make sure that you know why you're creating it and what you hope to achieve. Just having a plan in place really helps make programming fun. You can work on your new application and see your goals accomplished one at a time until you have a completed application to use and show off to your friends (all of whom will think you're really cool for creating it).

Understanding Why Python Is So Cool

Many programming languages are available today. In fact, a student can spend an entire semester in college studying computer languages and still not hear about them all. (I did just that during my college days.) You'd think that programmers would be happy with all these programming languages and just choose one to talk to the computer, but they keep inventing more.

REMEMBER

Programmers keep creating new languages for good reason. Each language has something special to offer — something it does exceptionally well. In addition, as computer technology evolves, so do the programming languages in order to keep up. Because creating an application is all about efficient communication, many programmers know multiple programming languages so that they can choose just the right language for a particular task. One language might work better to obtain data from a database, and another might create user interface elements especially well.

As with every other programming language, Python does some things exceptionally well, and you need to know what they are before you begin using it. You might be amazed by the really cool things you can do with Python. Knowing a programming language's strengths and weaknesses helps you use it better as well as avoid frustration by not using the language for things it doesn't do well. The following sections help you make these sorts of decisions about Python.

Unearthing the reasons for using Python

When Guido van Rossum (`https://gvanrossum.github.io/`) decided to create Python, the main objective was to develop a programming language that would make programmers efficient and productive. With that in mind, here are the reasons that you want to use Python when creating an application:

- **Less application development time:** Python code is usually 2–10 times shorter than comparable code written in languages like C/C++ and Java, which means that you spend less time writing your application and more time using it.
- **Ease of reading:** A programming language is like any other language — you need to be able to read it to understand what it does. Python code tends to be easier to read than the code written in other languages, which means you spend less time interpreting it and more time making essential changes.
- **Reduced learning time:** The creators of Python wanted to make a programming language with fewer odd rules that make the language hard to learn. After all, programmers want to create applications, not learn obscure and difficult languages.

TIP

Although Python is a popular language, it's not always the most popular language out there (depending on the site you use for comparison). However, it currently ranks first on sites such as TIOBE (`https://www.tiobe.com/tiobe-index/`), an organization that tracks usage statistics (among other things). Another good place to look is Statistics Times (`https://statisticstimes.com/tech/top-computer-languages.php`), which also ranks Python as the number one language today.

If you're looking for a language solely for the purpose of obtaining a job, Python is a great choice, but Java, C/C++, or C# might be better choices, depending on the kind of job you want to get. Visual Basic is also a great choice, even if it isn't currently quite as popular as Python. Make sure to choose a language you like and one that will address your application-development needs, but also choose on the basis of what you intend to accomplish. You may be surprised to learn that many colleges use Python to teach coding, and it has become the most popular language in that venue (see `https://www.pythoncentral.io/how-is-python-used-in-education/` for details).

Deciding how you can personally benefit from Python

Ultimately, you can use any programming language to write any sort of application you want. If you use the wrong programming language for the job, the process will be slow, error prone, bug ridden, and you'll absolutely hate it — but you can get the job done. Of course, most of us would rather avoid horribly painful experiences, so you need to know what sorts of applications people typically use Python to create. Here's a list of the most common uses for Python (although people do use it for other purposes):

- **Creating rough application examples:** Developers often need to create a *prototype,* a rough example of an application, before getting the resources to create the actual application. Python emphasizes productivity, so you can use it to create prototypes of an application quickly.
- **Scripting browser-based applications:** Even though JavaScript is probably the most popular language used for browser-based application scripting, Python is a close second. Python offers functionality that JavaScript doesn't provide (see the comparison at `https://www.educba.com/python-vs-javascript/` for details) and its high efficiency makes it possible to create browser-based applications faster (a real plus in today's fast-paced world).
- **Designing mathematic, scientific, and engineering applications:** Interestingly enough, Python provides access to some really cool libraries that make it easier to create math, scientific, and engineering applications. The two most popular libraries are NumPy (`https://numpy.org/`) and SciPy

(https://scipy.org/). These libraries greatly reduce the time you spend writing specialized code to perform common math, scientific, and engineering tasks.

- **Working with XML:** The eXtensible Markup Language (XML) is the basis of most data storage needs on the Internet and many desktop applications today. Unlike most languages, where XML is just sort of bolted on, Python makes it a first-class citizen. If you need to work with a web service, the main method for exchanging information on the Internet (or any other XML-intensive application), Python is a great choice.
- **Interacting with databases:** Business relies heavily on databases. Python isn't quite a query language, like the Structured Query Language (SQL) or Language INtegrated Query (LINQ), but it does do a great job of interacting with databases. It makes creating connections and manipulating data relatively painless.
- **Developing user interfaces:** Python isn't like some languages like C# where you have a built-in designer and can drag and drop items from a toolbox onto the user interface. However, it does have an extensive array of graphical user interface (GUI) frameworks — extensions that make graphics a lot easier to create (see https://wiki.python.org/moin/GuiProgramming for details). Some of these frameworks do come with designers that make the user interface creation process easier. The point is that Python isn't devoted to just one method of creating a user interface — you can use the method that best suits your needs.

Discovering which organizations use Python

Python really is quite good at the tasks that it was designed to perform. In fact, that's why a lot of large organizations use Python to perform at least some application-creation (development) tasks. You want a programming language that has good support from these large organizations because these organizations tend to spend money to make the language better. Table 1-1 lists some of the large organizations that use Python the most.

TIP

These are just a few of the many organizations that use Python extensively. You can find a more complete list of organizations at https://www.python.org/about/success/. The number of success stories has become so large that even this list probably isn't complete and the people supporting it have had to create categories to better organize it.

TABLE 1-1 Large Organizations That Use Python

Vendor	URL	Uses Python For . . .
Alice Educational Software – Carnegie Mellon University	`https://www.alice.org/`	Educational applications
Fermilab	`https://www.fnal.gov/`	Scientific applications
Go.com	`http://go.com/`	Browser-based applications
Google	`https://www.google.com/`	Search engine
Industrial Light & Magic	`https://www.ilm.com/`	Just about every programming need
Lawrence Livermore National Library	`https://www.llnl.gov/`	Scientific applications
National Space and Aeronautics Administration (NASA)	(`https://www.nasa.gov/`)	Scientific applications
New York Stock Exchange	`https://www.nyse.com/index`	Browser-based applications
Redhat	`https://www.redhat.com/en`	Linux installation tools
Yahoo!	`https://www.yahoo.com/`	Parts of Yahoo! Mail
YouTube	`https://www.youtube.com/`	Graphics engine

Finding useful Python applications

You might have an application written in Python sitting on your machine right now and not even know it. Python is used in a vast array of applications on the market today. The applications range from utilities that run at the console to full-fledged CAD/CAM suites. Some applications run on mobile devices, while others run on the large services employed by enterprises. In short, there is no limit to what you can do with Python, but it really does help to see what others have done. You can find a number of places online that list applications written in Python, but the best place to look is `https://wiki.python.org/moin/Applications`.

As a Python programmer, you'll also want to know that Python development tools are available to make your life easier. A *development tool* provides some level of automation in writing the procedures needed to tell the computer what to do. Having more development tools means that you have to perform less work in order to obtain a working application. Developers love to share their lists of favorite tools, but you can find a great list of tools broken into categories at `https://www.python.org/about/apps/`.

Comparing Python to other languages

Comparing one language to another is somewhat dangerous because the selection of a language is just as much a matter of taste and personal preference as it is any sort of quantifiable scientific fact. So before I'm attacked by the rabid protectors of the languages that follow, it's important to realize that I also use a number of languages and find at least some level of overlap among them all. There is no best language in the world, simply the language that works best for a particular application. With this idea in mind, the following sections provide an overview comparison of Python to other languages. (You can find comparisons to other languages at `https://wiki.python.org/moin/LanguageComparisons`.)

C#

A lot of people claim that Microsoft simply copied Java to create C#. That said, C# does have some advantages (and disadvantages) when compared to Java. The main (undisputed) intent behind C# is to create a better kind of C/C++ language — one that is easier to learn and use. However, we're here to talk about C# and Python. When compared to C#, Python has these advantages:

- Significantly easier to learn
- Smaller (more concise) code
- Supported fully as open source
- Better multiplatform support
- Easily allows use of multiple development environments
- Easier to extend using Java and C/C++
- Enhanced scientific and engineering support

Java

For years, programmers looked for a language that they could use to write an application just once and have it run anywhere. Java is designed to work well on any platform. It relies on some tricks that you'll discover later in the book to accomplish this magic. For now, all you really need to know is that Java was so successful at running well everywhere that other languages have sought to emulate it (with varying levels of success). Even so, Python has some important advantages over Java, as shown in the following list:

- Significantly easier to learn
- Smaller (more concise) code

- Enhanced variables (storage boxes in computer memory) that can hold different kinds of data based on the application's needs while running (dynamic typing)
- Faster development times

Perl

Perl was originally an acronym for Practical Extraction and Report Language. Today, people simply call it Perl and let it go at that. However, Perl still shows its roots in that it excels at obtaining data from a database and presenting it in report format. Of course, Perl has been extended to do a lot more than that — you can use it to write all sorts of applications. (I've even used it for a web service application.) In a comparison with Python, you'll find that Python has these advantages over Perl:

- Simpler to learn
- Easier to read
- Enhanced protection for data
- Better Java integration
- Fewer platform-specific biases

R

Data scientists often have a tough time choosing between R and Python because both languages are adept at statistical analysis and the sorts of graphing that data scientists need to understand data patterns. Both languages are also open source and support a large range of platforms. However, R is a bit more specialized than Python and tends to cater to the academic market. Consequently, Python has these advantages over R:

- Emphasizes productivity and code readability
- Is designed for use by enterprises
- Offers easier debugging
- Uses consistent coding techniques
- Has greater flexibility
- Is easier to learn

Haskell

Pure functional languages, such as Haskell, have some significant advantages because they work extremely well in multithreaded application due to the fact that pure functions always return the same value for the same inputs. There are no side effects. Many developers also see functional languages as easier to debug and less likely to produce bugs. Python is an impure functional language, which means it doesn't strictly adhere to all of the functional language requisites. However, Python has these advantages over Haskell:

- Is easier to learn
- Greater selection of libraries
- Easier to use for prototyping and modeling
- Viewed as a better server-side language
- Has greater flexibility

IN THIS CHAPTER

- Using Google Colab as an editor
- Performing the common tasks to create code files
- Making your code run faster as you execute it
- Getting help when you need it

Chapter 2
Working with Google Colab

Back in the Stone Age of computing, developers used punch cards to write their applications and then waited, sometimes a week, to get time on the computer to run an application that might instantly fail with an error. Programmers in that age were a frustrated bunch, prone to grimacing a lot. So moving to Integrated Development Environments (IDEs) (special editors that you use to write and run code) on large desktop machines seemed like a real achievement, but things became even better with time. Now you can write your applications anywhere and at any time with browser-based IDEs like Colabortory. Colaboratory (`https://colab.research.google.com/notebooks/welcome.ipynb`), or Colab for short, is a Google cloud-based service that lets you write Python code using a notebook-like environment, rather than the usual desktop IDE. (Jupyter Notebook, `https://jupyter.org/`, provides an environment similar to Colab on the desktop if you don't have an Internet connection.)

You don't have to install anything on your system to use Colab. The benefit of this approach is that you can work with code in small pieces and obtain nearly instant results from any work you do (no more frustrated grimacing). A notebook format also lends itself to output in a report format that works well for presentations and reports. The first section of this chapter helps you work through some Colab basics and understand how Colab differs from a standard IDE (and why this difference imparts significant benefit when learning how to program).

You can use Colab to perform specific tasks in a cell-oriented paradigm. The next sections of the chapter present a range of task-related topics that start with the use of notebooks. Of course, you also want to perform other sorts of tasks, such as creating various cell types and use them to create notebooks that have a report-like appearance with functional code.

Part of working with Colab is knowing how to run the example code, making it run as quickly as possible. Two sections of the chapter are dedicated to using hardware acceleration and running the example code in various ways.

Finally, this chapter can't address every aspect of Colab, so the last section of the chapter serves as a handy resource for locating the most reliable information about Colab.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 02; Colab Examples.ipynb` file of the downloadable source. See the Introduction for details on how to find these source files.

Defining Google Colab

Colab helps you write, document, and test Python code using a single application. It's designed to mimic a desktop application called Jupyter Notebook (`https://jupyter.org/`), so you can use Jupyter Notebook in place of Colab to run the applications in this book. In fact, it's somewhat difficult to tell the two applications apart in the functionality they provide. Google Colab is the cloud version of Notebook, and the Welcome page makes this fact apparent. It even uses IPython (the previous name for Jupyter) Notebook (`.ipynb`) files for the site.

Even though the two applications are similar and both use `.ipynb` files, they do have some differences that you need to know about. The previous edition of this book used Jupyter Notebook, but Colab offers the ability to compute anywhere on any device that sports a browser, so this edition of the book focuses on Colab instead. The following sections help you understand the Colab differences.

Understanding what Google Colab does

You can use Colab to perform many tasks, but for the purpose of this book, you use it to write and run code, create its associated documentation, and display graphics. The downloadable source for this book is designed to run on Colab, but you can also use it with Jupyter Notebook if you want.

Jupyter Notebook is a localized application in that you use local resources with it. You could potentially use other sources, but doing so could prove inconvenient or impossible in some cases. For example, according to `https://docs.github.com/repositories/working-with-files/using-files/working-with-non-code-files`, your Notebook files will appear as static HTML pages when you use a GitHub repository. (*GitHub* is a cloud-based storage technology specifically oriented to working with code.) In fact, some features won't work at all.

TIP

Colab enables you to fully interact with your Notebook files using GitHub as a repository, and Colab supports a number of other online storage options as well, so you can regard Colab as your online partner in creating Python code. The GitHub repository for this book is at `https://github.com/JohnPaulMueller/BPP4D3E`.

The other reason you really need to know about Colab is that you can use it with your alternative device. During the writing process, some of the example code was tested on an Android-based tablet (an ASUS ZenPad 3S 10). The target tablet has Chrome installed and executes the code well enough to follow the examples. All this said, you likely won't want to try to write code using a tablet of that size (it has a 9.7-inch diagonal screen) — the text was incredibly small, for one thing, and the lack of a keyboard could be a problem, too. The point is that you don't absolutely have to have a Windows, Linux, or macOS system to try the code, but the alternatives might not provide quite the performance you expect.

REMEMBER

Google Colab generally doesn't work with browsers other than Chrome (the browser used in this chapter), Firefox, or Safari (although initial tests with Microsoft Edge have also been encouraging). In most cases, you see an error message, such as `This site may not work in your browser. Please use a supported browser`, and no other display if you try to start Colab in a browser that it doesn't support. The included More Info link takes you to `https://research.google.com/colaboratory/faq.html#browsers`, where you can learn more information.

Working with Google Colab features

Google Colab provides access to a number of features through the menu system. One of these features, hardware acceleration, appears in the "Using Hardware Acceleration" section, later in this chapter. The features in this section all appear on the Tools menu.

Locating commands

The Tools➪ Command Palette option displays a list of commands you can execute, as shown in Figure 2-1. Some of these commands also have shortcut keys, such as Ctrl+Alt+M for adding a comment to a cell. All these commands help you to perform tasks associated with Notebook content, such as adding forms.

SOME FIREFOX ODDITIES

Even with online help, you may still find that your copy of Firefox displays a `SecurityError: The operation is insecure.` error message. The initial error dialog box points to some unrelated issue, such as cookies, but you see this error message when you click Details. Simply dismissing the dialog box by clicking OK makes Colab appear to be working because it displays your code, but you won't see results from running the code.

As a first step toward fixing this problem, make sure that your copy of Firefox is current; older versions don't provide the required support. After you've updated your copy, setting the `network.websocket.allowInsecureFromHTTPS` preference using `About:Config` to `True` should resolve the problem, but sometimes it doesn't. In this case, verify that Firefox actually does allow third-party cookies by selecting both the Always option for Accept Third Party Cookies and Site Data entry and the Remember History option in the History section on the Privacy & Security tab of the Options dialog box. Restart Firefox after each change and then try Colab again. If none of these fixes works, you must use Chrome to work with Colab on your system.

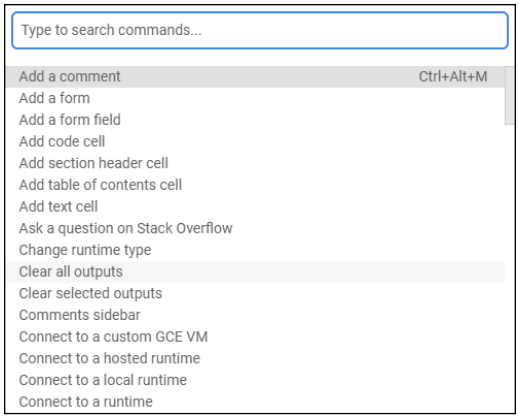


FIGURE 2-1: Using Colab commands makes configuring your Notebook easy.

Configuring settings

The Tools➪Settings option displays the Settings dialog box, shown in Figure 2-2. The four common and one optional (not shown) settings tabs perform these tasks:

- **Site:** Configures how the site works. The most interesting setting is the theme. Selecting Adaptive lets Colab choose the interface colors based on lighting conditions. You can also configure display and access settings on this tab.

- **Editor:** Determines how text appears onscreen and how the interface works. For example, you can set the key bindings to work like those in *Vim* (a text editor included on Unix and Linux systems, generally as the vi utility, `https://www.vim.org/`) if desired. You can also select font size, spaces for each level of indentation, and a plethora of other settings.
- **Colab Pro:** Provides an advertisement for Colab Pro (`https://colab.research.google.com/signup`), which gives you some significant benefits like faster GPUs, longer runtimes, and more memory — all of which let you get more work done in a shorter time.
- **GitHub (optional):** Shows that your GitHub account is connected to Colab when this tab appears. The various options let you remove GitHub access and determine whether you can see private repositories when opening files. Figure 2-2 doesn't show this particular tab because you won't normally see it until you grant GitHub access using the instructions found at `https://medium.com/analytics-vidhya/how-to-use-google-colab-with-github-via-google-drive-68efb23a42d`. You can find some additional insights at `https://stackoverflow.com/questions/67553747/how-do-i-link-a-github-repository-to-a-google-colab-notebook`.
- **Miscellaneous:** Contains fun settings. You can choose from three visual effects: adding thunder and lightning using the Power Level setting; letting a Corgi run across the top of the display; and allowing a kitten to run across the top of the display. You can choose any mix of these visual effects.

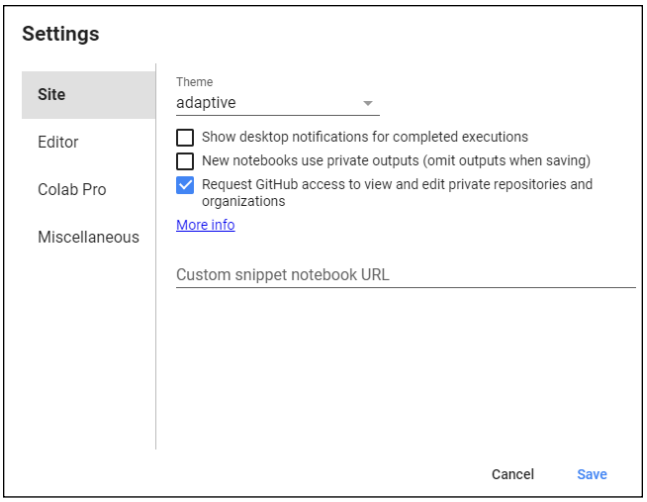


FIGURE 2-2: The Settings dialog box helps you configure the Colab IDE.

Customizing keyboard shortcuts

If you don't like the default keyboard shortcuts, you can customize them to match your needs. To do so, choose Tools➪Keyboard Shortcuts, and you see the Keyboard Preferences dialog box, shown in Figure 2-3. If you see Set Shortcut, it means that the command doesn't currently have a shortcut, so you can add one if desired. Here's how you work with shortcuts:

- To add or change a shortcut, place the cursor in the box next to the command and press the shortcut key you want to use for that command.
- To remove a shortcut, press Delete.

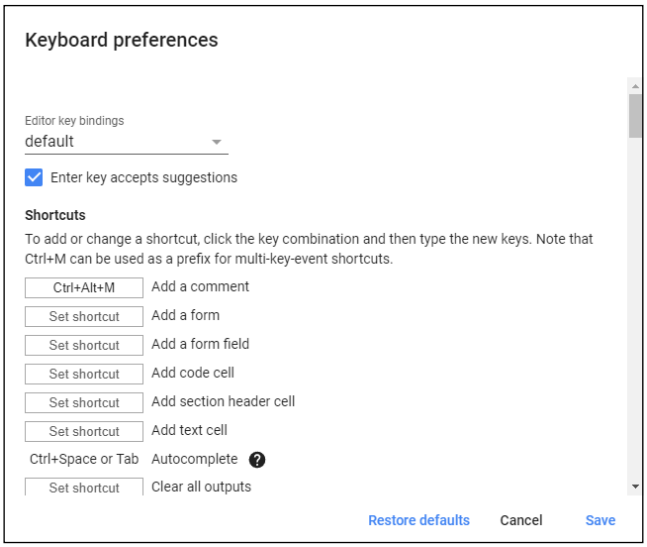


FIGURE 2-3: Customize shortcut keys for speed of access to commands.

Comparing files

Sometimes you need to compare two files to see how they differ. When you select Tools➪Diff Notebooks, Colab opens a new browser tab and shows you two notebooks side by side, as shown in Figure 2-4. These are files selected by random from your Google Drive. To select the files you actually want to work with, click the down arrow next to the file path in each pane. The differences appear onscreen.

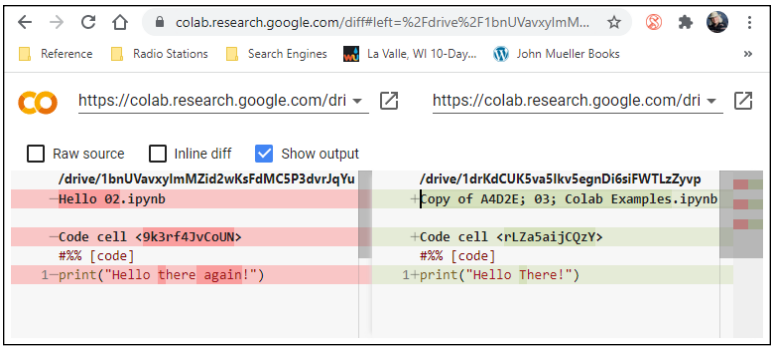


FIGURE 2-4: Colab lets you compare two files to see how they differ.

Working with Notebooks

The notebook forms the basis for interactions with Colab. In fact, Colab is built on notebooks, as previously mentioned. When you place the mouse on certain parts of the Welcome page at `https://colab.research.google.com/notebooks/welcome.ipynb`, you see opportunities for interacting with the page by adding either code or text entries (which you can use for notes as needed). These entries are active, so you can interact with them. You can also move cells around and copy the resulting material to your Google Drive. Of course, although interacting with the Welcome page is both unexpected and fun, the real purpose of this chapter is to demonstrate how to interact with Colab notebooks. The following sections describe how to perform basic notebook-related tasks with Colab.

Creating a new notebook

To create a new notebook, choose File➪ New Notebook. You see a new Python 3 notebook, like the one shown in Figure 2-5.

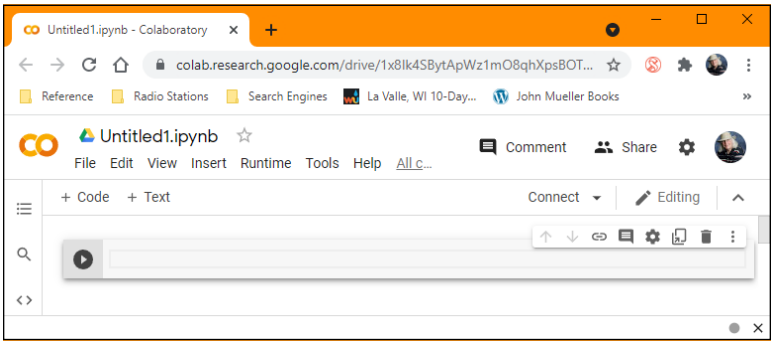


FIGURE 2-5: Create a new Python 3 Notebook.

The notebook shown in Figure 2-5 lets you change the filename by clicking it. To run the code in a particular cell, you click the right-pointing arrow on the left side of that cell. After you run the code, you must choose the next cell directly.

Opening existing notebooks

You can open existing notebooks found in local storage, on Google Drive, or on GitHub. You can also open any of the Colab examples or upload files from sources that you can access, such as a network drive on your system. In all cases, you begin by choosing File➪Open Notebook. You see the dialog box shown in Figure 2-6.

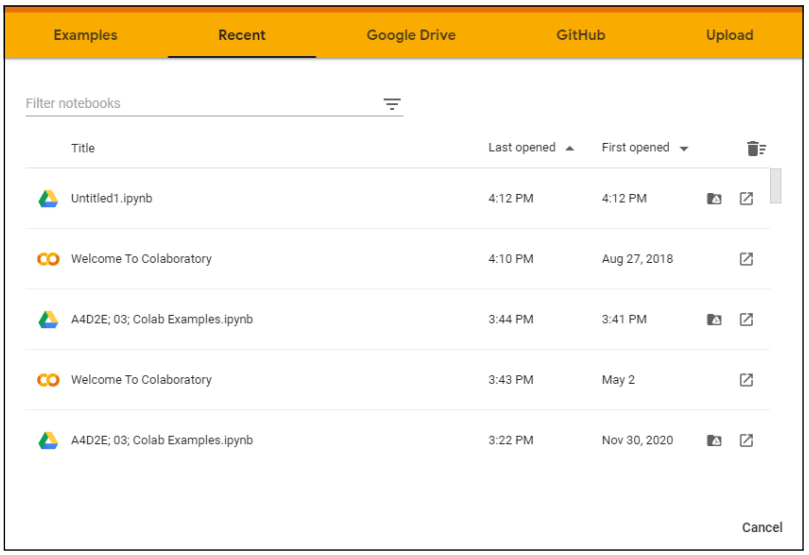


FIGURE 2-6: Use this dialog box to open existing notebooks.

The default view shows all the files you opened recently, regardless of location. The files appear in alphabetical order. You can filter the number of items displayed by typing a string into the Filter Notebooks field. Across the top are other options for opening notebooks.

TIP

Even if you're not logged in, you can still access the Colab example projects. These projects help you understand Colab but won't allow you to do anything with your own projects. Even so, you can still experiment with Colab without logging into Google first. The following sections tell you more details about these options.

Using Google Drive for existing notebooks

Google Drive is the default location for many operations in Colab, and you can always choose it as a destination. When working with Google Drive, you see a list of files similar to those shown in Figure 2-6. To open a particular file, you click its link in the dialog box. The file opens in the current tab of your browser.

Using GitHub for existing notebooks

When working with GitHub, you initially need to provide the location of the source code online by modifying the Repository field, as shown in Figure 2-7. If the project contains more than one branch (path of code changes), you must also select an option in the Branch field. The location must point to a public project or a private project owned by you; you can't use Colab to access other people's private projects.

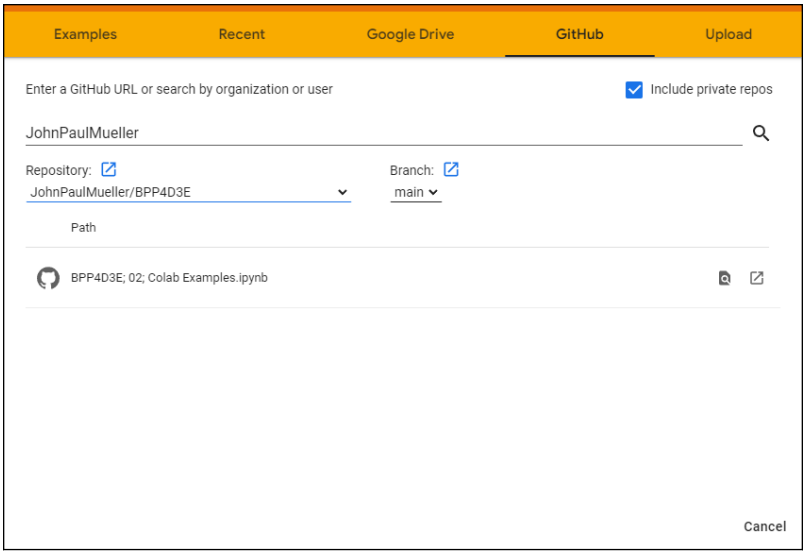


FIGURE 2-7: When using GitHub, you must provide the location of the source code.

After you make the connection to GitHub, you see two lists: repositories, which are containers for code related to a particular project; and branches, a particular implementation of the code. Selecting a repository and branch displays a list of notebook files that you can load into Colab. Simply click the required link, and it loads as if you were using a *Google Drive* (`https://drive.google.com/`), which is another type of online storage.

Using local storage for existing notebooks

If you want to use the downloadable source for this book, or any local source for that matter, you select the Upload tab of the dialog box. In the center is a single

button, Choose File. Clicking this button opens the File Open dialog box for your browser. You locate the file you want to upload, just as you normally would for opening any file.

REMEMBER

Selecting a file and clicking Open uploads the file to Google Drive. If you make changes to the file, those changes appear on Google Drive, not on your local drive. Depending on your browser, you usually see a new window open with the code loaded. However, you may also simply see a success message, in which case you must now open the file using the same technique as you would when using Google Drive. In some cases, your browser asks whether you want to leave the current page. You should tell the browser to do so.

TIP

The File➪ Upload Notebook command also uploads a file to Google Drive. In fact, uploading a notebook works like uploading any other kind of file, and you see the same dialog box. If you want to upload other kinds of files, using the File➪ Upload Notebook command is likely faster.

Saving notebooks using GitHub

Colab provides a significant number of options for saving your notebook. However, none of these options works with your local drive. After you upload content from your local drive to Google Drive or GitHub, Colab manages the content in the cloud and not on your local drive. To save updates to your local drive, you must download the file to your local drive by choosing one of the options on the File➪ Download menu (either an `.ipynb` or a `.py` file format). The following sections review the cloud-based options for saving notebooks.

Using Drive to save notebooks

The default location for storing your data is Google Drive. When you choose File➪ Save, the content you create goes to the root directory of your Google Drive. If you want to save the content to a different folder, you need to select that folder in Google Drive.

REMEMBER

Colab tracks the versions of your project as you perform saves. However, as these revisions age, Colab removes them. To save a version that won't age, you use the File➪ Save and Pin Revision command. To see the revisions for your project, choose File➪ Revision History.

You can also save a copy of your project by choosing File➪ Save a Copy In Drive. The copy receives the word *Copy* as part of its name. Of course, you can rename it later. Colab stores the copy in the current Google Drive folder.

Using GitHub to save notebooks

GitHub provides an alternative to Google Drive for saving content. It offers an organized method of sharing code for the purpose of discussion, review, and distribution. You can find GitHub at `https://github.com/`. The source code for this book appears at `https://github.com/JohnPaulMueller/BPP4D3E`, so you can access it easily from Colab.

REMEMBER

You may use only public repositories and private repositories that you own when working with GitHub from Colab, even though GitHub also supports private repositories. To save a file to GitHub, choose File➪ Save a Copy in GitHub. If you aren't already signed into GitHub, Colab displays a window that requests your sign-in information. After you sign in, you see a dialog box similar to the one shown in Figure 2-8.

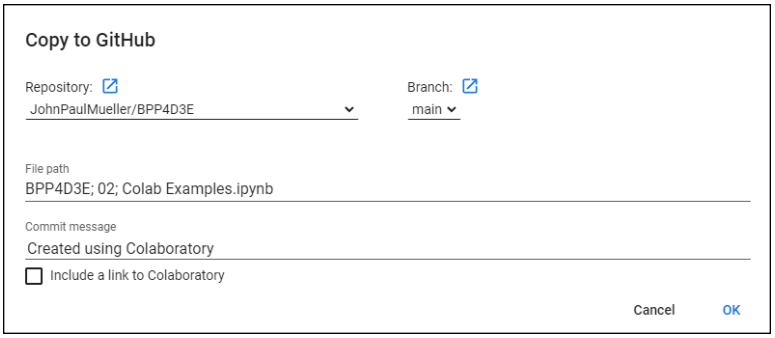


FIGURE 2-8: Using GitHub means storing your data in a repository.

If your account doesn't currently have a repository, you must either create a new repository or choose an existing repository in which to store your data. After you save the file, it appears in the GitHub repository of your choice. The repository can include a link to open the data in Colab by default when you check the Include a Link to Colaboratory option.

Getting the gist of things

You use GitHub gists as a means of sharing single files or other resources with other people. Some people use them for full projects as well, but the idea is that you have a concept you want to share — something that isn't quite fully formed and doesn't necessarily represent a usable application. You can read more about gists at `https://docs.github.com/en/get-started/writing-on-github/editing-and-sharing-content-with-gists/creating-gists`.

WARNING

Gists come in two forms, public and secret. A secret gist isn't private; anyone who has the URL can see it. The only difference between a public and a secret gist is that someone can't use Discover (`https://gist.github.com/discover`) to locate the gist. If you really need to keep your code private, you need to use a GitHub private repository instead of a gist.

When you choose File➪ Save a Copy as a GitHub gist, Colab opens a new copy of the file for you and places it on GitHub gist as a secret gist. The saved file automatically includes an Open in Colab button (badge) so that you can open the file in Colab as needed. The problem is that you really don't know much about the gist (not even where Colab put it), so here are some helpful pointers:

- **Locate:** To locate your gist, open Discover (`https://gist.github.com/discover`), type the name of the file in the Search field, and press Enter. You see a list of matching files, one of which is going to be the one you saved. It's also possible to locate your gists by accessing them on your gist page, such as `https://gist.github.com/JohnPaulMueller`.
- **Delete:** You may not want to keep the file around because it contains secrets, or perhaps you're just embarrassed about it. Open the file and click the Delete button that appears at the top of the browser page. GitHub displays a dialog box asking whether you're positive that you want to delete the file. Click OK to complete the action.
- **Publicize:** After your masterpiece is complete, you may want to share it with the world. To perform this task, open the file and click Edit at the top of the page. GitHub will display the editable form of the file for you. Click Make Public at the top of the page.

  WARNING

  Making your gist public is a one-way trip. After the gist is public, you can't make it private again, so your only option is to delete the gist if you want to keep it out of the public view.
- **Share:** You have multiple options for sharing your gist: Embed, Share, Clone via HTTPS, and Clone via SSH. To share a gist, choose the option you want to use from the drop-down box (Embed is the default) and then click the Copy button to the right of the option selection. You can now paste the link to your gist anywhere you need to.
- **Download:** If you need a local copy of your gist, click the Download ZIP button. GitHub will download a `.zip` file to your local drive that contains the gist.

Working with Drive

When you upload a file or obtain it from another source, such as GitHub, you can choose to save a copy of it to Drive by choosing File➪ Save a Copy in Drive. When

you choose this option, Colab opens another copy of the file in a window that says Copy Of and then the original name of the file. When you open Google Drive, you see a new folder named Colab Notebooks with the copy of the file in it. You can double-click this entry to open the notebook in Colab later.

Performing Common Tasks

Most tasks in Colab and Notebook work the same. Each has both code cells and noncode cells, and you can create code cells in both Colab and Notebook by using the options on the Insert menu. Likewise, both environments have noncode cells that come in three forms:

- Text
- Section header
- Form field, which comes in these types:
  - Drop-down
  - Input
  - Slider
  - Markdown

Noncode cells in Colab work somewhat differently from the Markdown cells found in Notebook, but the idea is the same. Two interesting additions in Colab that aren't found in Notepad are the scratch code cell, which allows you to experiment with code in real time, and code snippets, which are canned code for performing specific tasks (you just insert them where needed).

You can also edit and move cells. One important difference between the two environments is that you can't change a cell type in Colab but you can in Notebook. A cell that you create as a section header can't suddenly transform into a code cell. The following sections offer a brief overview of the various features.

Creating code cells

The first cell that Colab creates for you is a code cell. Colab and Notebook share the same features with regard to code, so code you write in Colab also works in Notepad (and vice versa). However, off to the side of the cell, you see a menu of extras that you can use with Colab (see Figure 2-9); these aren't present in Notebook.

FIGURE 2-9: Colab code cells contain a few extras not found in Notebook.

You use the icons shown in Figure 2-9 to augment your Colab code experience. Here's what these features do (in order of appearance, left to right, in the figure):

- **Move cell up:** Moves the cell up one position in the cell ordering.
- **Move cell down:** Moves the cell down one position in the cell ordering.
- **Link to cell:** Displays a dialog box containing a link you can use to access a specific cell within the notebook. You can embed this link anywhere on a web page or within a notebook to allow someone to access that specific cell. The person still sees the entire notebook but doesn't have to search for the cell you want to discuss.
- **(Optional) Add a comment (assuming that you have the right to make a comment):** Creates a comment balloon to the right of the cell. This is not the same as a code comment, which exists inline with the code, but this kind of comment affects the entire cell. You can edit, delete, or resolve comments. A resolved comment is one that received attention and is no longer applicable.
- **Open editor settings:** Opens the same dialog box shown in Figure 2-2 and discussed in the "Working with Google Colab features" section, earlier in this chapter. This option appears when you are editing a cell.
- **Mirror cell in tab:** Mirrors the currently selected cell in a Cell pane that appears on the right side of the window. You can scroll wherever you want within the code in the left pane and keep this code accessible. The right-pointing arrow lets you execute the cell at any time after making changes in left-pane code. A pair of double-pointing arrows lets you move the focus back to the selected code in the left pane with a single click. You can also move the cell code to a scratch cell, where you can play with it without modifying your original code. You can have more than one Cell pane. You simply select the one you want and move between them as needed, which lets you move easily from place to place in your code. Close a Cell pane by clicking the *X* next to the word *Cell.*
- **Delete Cell:** Removes the cell from the notebook.
- **Vertical ellipses:** Contains a number of additional features in a menu (not all of which may appear because they depend on the file you have opened, the tasks you have performed, and your rights to work with the file content):
  - **Select Cell:** Highlights the entire content of the current cell, including the output, which is a convenient way to copy the material for use elsewhere.

- **Copy Cell:** Copies the content of the currently selected cell to the Clipboard.
- **Cut Cell:** Deletes the content of the currently selected cell and places it on the Clipboard.
- **Clear Output:** Removes the output from the cell. You must run the code again to regenerate the output.
- **View Output Fullscreen:** Displays the output (not the entire cell or any other part of the notebook) in full-screen mode on the host device. This option is useful when displaying a significant amount of content, or when a detailed view of graphics helps explain a topic. Press Esc to exit full-screen mode.
- **Add a Form:** Inserts a form into the cell to the right of the code. You use forms to provide a graphical input for parameters. Forms don't appear in Notebook, but because of how you create them, they won't prevent you from running the code in Notebook. You can read more about forms at `https://colab.research.google.com/notebooks/forms.ipynb`.

Code cells also tell you about the code and its execution. The little icon next to the output displays information about the execution when you hover your mouse over it. Clicking the icon clears the output. You must run the code again to regenerate the output.

Creating text cells

Text cells work much like Markdown cells in Notebook. However, Figure 2-10 shows that you receive additional help in formatting the text using a graphical interface. The markdown is the same, but you have the option of allowing the GUI to help you create the markdown. For example, in this case, to create the hash sign (#) for a heading, you click the double *T* icon that appears first in the list. Clicking the double *T* icon again would increase the header level. To the right, you see how the text will appear in the notebook.

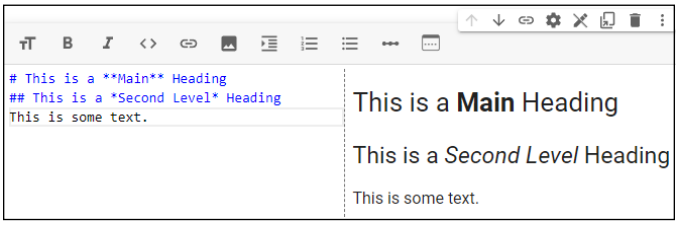


FIGURE 2-10: Use the GUI to make formatting your text easier.

Notice the menu to the right of the text cell. This menu contains many of the same options that a code cell does. For example, you can create a list of links to help people access specific parts of your notebook through an index. In contrast to Notebook, you can't execute text cells to resolve the markup they contain.

TIP

You can find details of Colab markdown at `https://colab.research.google.com/notebooks/markdown_guide.ipynb`. The tutorial at `https://www.datacamp.com/community/tutorials/markdown-in-jupyter-notebook` is helpful in discovering how to use both Colab and Notebook markdown.

Creating special cells

The special cells that Colab provides are variations of the text cell. These special cells, which you access using the Insert menu option, make creating the required cells faster. However, you shouldn't use these special cells if you need to maintain compatibility between Colab and Notebook. The following sections describe each of these special cell types.

Working with headings

When you choose Insert➪ Section Header Cell, you see a new cell created below the currently selected cell that has the appropriate header level-1 entry in it. You can increase the heading level by clicking the double *T* icon. The GUI looks the same as the one in Figure 2-10, shown previously, so you have all the standard formatting features for your text.

Working with a table of contents

An interesting addition to Colab is the automatic generation of a table of contents for your notebook. To use this feature, click the Table of Contents icon on the left side of the window. The table of contents contains one entry for each heading you provide in your notebook. The entries are automatically organized according to level, so you see the hierarchy of your code. Clicking an entry automatically takes you to that location in your code.

Editing cells

Both Colab and Notebook have Edit menus that contain the options you expect, such as the ability to cut, copy, and paste cells. The two products also have some interesting differences. For example, Notebook allows you to split and merge cells. Colab contains an option to show or hide the code as a toggle. These differences give each product a slightly different flavor but don't really change your ability to use each one to create and modify Python code.

Moving cells

The same technique you use for moving cells in Notebook also works with Colab. The only difference is that Colab relies exclusively on toolbar buttons (refer to in Figure 2-9); Notebook also has cell movement options on the Edit menu. To move a cell, select it and then click the Move Cell Up or Move Cell Down buttons as needed.

Using Hardware Acceleration

Even though you won't need it for the examples in this book, Colab does offer hardware acceleration in the form of a Graphics Processing Unit (GPU) or Tensor Processing Unit (TPU). Both of these special processors offer the ability to process multiple sets of data in parallel at high speed. When working with big data in a machine learning or deep learning environment, a GPU or TPU can make a huge difference in the time required to accomplish a task. The main difference between a GPU and a TPU is that a GPU appears as part of most high-end display adapters today and can double for rendering complex graphics, while a TPU is a custom processor designed by Google specifically for machine learning and deep learning tasks. (There are other differences, but they aren't important for this book.)

GPU and TPU support are disabled by default in Colab. To enable GPU or TPU support, choose Runtime➪Change Runtime Type. A Notebook Settings dialog box appears. In this dialog box is the Hardware Accelerator drop-down list, from which you can choose None (the default), GPU, or TPU.

Executing the Code

For your code to be useful, you need to run it at some point. Previous sections have mentioned the right-pointing arrow that appears in the current cell. Clicking it runs just the current cell. Of course, you have other options than clicking the right-pointing arrow, and all these options appear on the Runtime menu (the Cell menu in Notebook). The following list summarizes these options:

- **Running the current cell:** Instead of clicking the right-pointing arrow, you can also choose Runtime➪Run the Focused Cell to execute the code in the current cell.
- **Running other cells:** Colab provides options on the Runtime menu for executing the code in the next cell, the previous cell, or a selection of cells.

Simply choose the option that matches the cell or set of cells you want to execute.

- **Running all the cells:** In some cases, you want to execute all the code in a notebook. In this case, choose Runtime ➪ Run All. Execution starts at the top of the notebook, in the first cell containing code, and continues to the last cell that contains code in the notebook. You can stop execution at any time by choosing Runtime ➪ Interrupt Execution.

TIP

Choosing Runtime ➪ Manage Sessions displays a dialog box containing a list of all the sessions that are currently executing for your account on Colab. You can use this dialog box to determine when the code in that notebook last executed and how much memory the notebook consumes. Click the trash can icon (also shown as a TERMINATE link) to end execution for a particular notebook.

Getting Help

The most obvious place to obtain help with Colab is from the Colab Help menu. The menu doesn't have a general help link, but you can find it at `https://colab.research.google.com/notebooks/welcome.ipynb` (which requires you to log into the Colab site). This menu does contain all the usual entries (some of which may not appear on your browser):

- **Frequently Asked Questions (FAQs):** Takes you to a page with questions that other people have asked.
- **View Release Notes:** Takes you to a page that contains the release notes for the current Colab version, along with all of the versions before it.
- **Search Code Snippets:** Opens a pane showing common tasks, such as working with a camera, in which you can search for example code that may meet your needs with a little modification. Clicking the Insert button inserts the code at the current cursor location in the cell that has focus. Each of the entries also shows an example of the code.
- **Report a Bug:** Takes you to a page where you can report Colab errors.
- **Ask a Question on Stack Overflow:** Displays a new browser tab, where you can ask questions from other users. You see a login screen if you haven't already logged in to Stack Overflow.
- **Send Feedback:** Displays a dialog box with links for locations where you can obtain additional information. If you really do want to send feedback, you click the Continue Anyway link at the bottom of the dialog box.

IN THIS CHAPTER

- Using commands to perform tasks
- Obtaining help about Python
- Discovering functions and objects
- Employing the inspector

Chapter 3
Interacting with Python

Ultimately, any application you create interacts with the computer and the data it contains. The focus is on data because without data, there isn't a good reason to have an application. Any application you use (even one as simple as Solitaire) manipulates data in some way. In fact, the acronym CRUD sums up what most applications do:

- Create
- Read
- Update
- Delete

If you remember CRUD, you'll be able to summarize what most applications do with the data your computer contains (and some applications really are quite cruddy). However, before your application accesses the computer, you have to interact with a programming language that creates a list of tasks to perform in a language the computer understands. That's the purpose of this chapter. You begin interacting with Python. Python takes the list of steps you want to perform on the computer's data and changes those steps into bits the computer understands. The chapter helps you understand how to obtain help about data manipulation methods, work with functions and objects that interact with data, and use the Python inspect module to perform detective work on your data.

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 03; Working with Commands.ipynb` file of the downloadable source. See the Introduction for details on how to find these source files.

Typing a Command

When you tame your lion (or cat or dog), you teach it commands such as sit, stay, and don't eat me. Each command instructs the animal to perform a task or do something else interesting. Likewise, using commands in Python makes it possible to perform application tasks, test ideas that you have for writing your application, and discover more about Python. Using the commands let you gain hands-on experience with how Python actually works.

UNDERSTANDING THE IMPORTANCE OF THE README FILE

Many applications include a README file (not necessarily named README). The README file usually provides updated information that didn't make it into the documentation before the application was put into a production status. Unfortunately, most people ignore the README file and some don't even know it exists. As a result, people who should know something interesting about their shiny new product never find out.

The version of Python you use has a Release Notes file (just because the developers didn't want to call it a README file) that contains essential information about Python. If you download and install a copy of Python on your local machine, this Release Notes file is named `NEWS.txt` and it appears in the main Python folder. If you use Colab and don't download anything, you can find the same information online. For example, the Python 3.10.3 Release Notes appear at `https://docs.python.org/release/3.10.3/whatsnew/changelog.html`. You can find Release Notes links for other version of Python at `https://www.python.org/downloads/`. When you open this file, you find all sorts of really interesting information, most of which centers on upgrades to Python that you really need to know about.

Opening and reading the Release Notes file (named `NEWS.txt` or `changelog.html` because people were apparently ignoring the other file) will help you become a Python genius. People will be amazed that you really do know something interesting about Python and will ask you all sorts of questions (deferring to your wisdom). Of course, you could always just sit there, thinking that the README is just too much effort to read.

Telling the computer what to do

Python, like every other programming language in existence, relies on commands. A *command* is simply a step in a procedure. In Chapter 1, you see how "Get the bread and butter from the refrigerator" is a step in a procedure for making toast. When working with Python, a command, such as `print()`, is simply the same thing: a step in a procedure.

To tell the computer what to do, you issue one or more commands that Python understands. Python translates these commands into instructions that the computer understands, and then you see the result. A command such as `print()` can display the results onscreen so that you get an instant result. However, Python supports all sorts of commands, many of which don't display any results onscreen but still do something important.

As the book progresses, you use commands to perform all sorts of tasks. Each of these tasks will help you accomplish a goal, just as the steps in a procedure do. When it seems as if all the Python commands become far too complex, simply remember to look at them as steps in a procedure. Even human procedures become complex at times, but if you take them one step at a time, you begin to see how they work. Python commands are the same way. Don't get overwhelmed by them; instead, look at them one at a time and focus on just that step in your procedure.

Telling the computer you're done

At some point, the procedure you create ends. When you make toast, the procedure ends when you finish buttering the toast. Computer procedures work precisely the same way. They have a starting and an ending point. When typing commands, the ending point for a particular step is the Enter key. You press Enter to tell the computer that you're done typing the command.

TIP

Pressing Enter doesn't actually run the command; you need to click Run Cell in Colab or click Run in Jupyter Notebook. As an alternative, you can also type a single command in a code cell and then press Ctrl+Enter in either IDE to run it.

As the book progresses, you find that Python provides a number of ways to signify that a step, group of steps, or even an entire application is complete. No matter how the task is accomplished, computer programs always have a distinct starting and stopping point.

Seeing the result

You now know that a command is a step in a procedure and that each command has a distinct starting and ending point. In addition, groups of commands and entire applications also have a distinct starting and ending point. So, take a look at how this works. The following procedure helps you see the result of using a command:

1. **Start a copy of either Google Colab or Jupyter Notebook and then create a new Python 3 Notebook as described in the "Creating a new notebook" section of Chapter 2.**

   You see a code cell where you can type a command.

2. **Type** print("This is a line of text.") **in the code cell.**

   Notice that nothing happens. Yes, you typed a command, but you haven't signified that the command is complete.

3. **Press Ctrl+Enter.**

   The command is complete, so you see a result like the one shown in Figure 3-1.

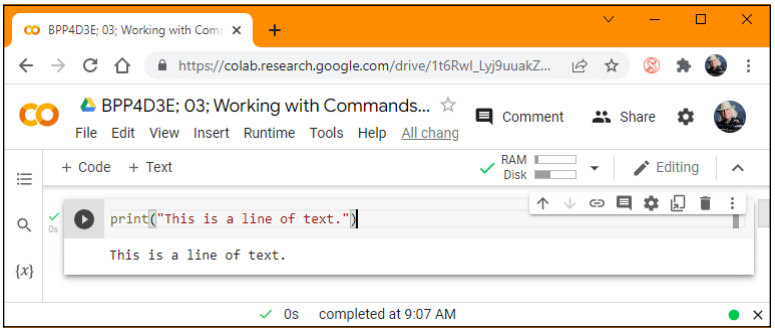


FIGURE 3-1: Issuing commands tells Python what to tell the computer to do.

This exercise shows you how things work within Python. Each command that you type performs some task, but only after you tell Python that the command is complete in some way. The `print()` command displays data onscreen. In this case, you supplied text to display. Notice that the output shown in Figure 3-1 comes immediately after the command because this is an interactive environment — one in which you see the result of any given command immediately after Python performs it. Later, as you start creating applications, you notice that sometimes a result doesn't appear immediately because the application environment delays it. Even so, the command is executed by Python immediately after the application tells Python that the command is complete.

PYTHON'S CODING STYLES

Most programming languages are dedicated to using just one coding style, which reduces flexibility for the programmer. However, Python is different. You can use a number of coding styles to achieve differing effects with Python. The four commonly used Python coding styles are

- **Functional:** Every statement is a kind of math equation. This style lends itself well to use in parallel processing activities.
- **Imperative:** Computations occur as changes to program state. This style is most used for manipulating data structures.
- **Object-oriented:** This is the style commonly used with other languages to simplify the coding environment by using objects to model the real world. Python doesn't fully implement this coding style because it doesn't support features like data hiding, but you can still use this approach to a significant degree. You see this style used later in the book.
- **Procedural:** All the code you've written so far (and much of the initial code in this book) is procedural, meaning that tasks proceed a step at a time. This style is most used for iteration, sequencing, selection, and modularization. It's the simplest form of coding you can use.

Even though this book doesn't cover all these coding styles (and others that Python supports), it's useful to know that you aren't trapped using a particular coding style. Because Python supports multiple coding styles and you can mix and match those styles in a single application, you have the advantage of being able to use Python in the manner that works best for a particular need. You can read more about the coding styles at `https://newrelic.com/blog/nerd-life/python-programming-styles`.

Getting Python's Help

Python is a computer language, not a human language. As a result, you won't speak it fluently at first. If you think about it for a moment, it makes sense that you won't speak Python fluently (and as with most human languages, you won't know every command even after you do become fluent). Having to discover Python commands a little at a time is the same thing that happens when you learn to speak another human language. If you normally speak English and try to say something in German, you find that you must have some sort of guide to help you along. Otherwise, anything you say is gibberish and people will look at you quite oddly. Even if you manage to say something that makes sense, it may not be what you want. You might go to a restaurant and order hot hubcaps for dinner when what you really wanted was a steak.

Likewise, when you try to speak Python, you need a guide to help you. Fortunately, Python is quite accommodating and provides immediate help to keep you from ordering something you really don't want. The help provided inside Python works at two levels:

- **Help mode,** in which you can browse the available commands
- **Direct help,** in which you ask about a specific command

There isn't a correct way to use help — just the method that works best for you at a particular time. The following sections describe how to obtain help.

Entering into help mode

One of the ways in which Jupyter Notebook excels over Google Colab is that Jupyter Notebook provides a good language reference as part of the Help menu. In fact, you can also find references for NumPy, SciPy, MatPlotLib, SymPy, and pandas. However, it's also important to realize that you have access to other forms of help that are supported by both Jupyter Notebook and Colab in the form of the four commands shown in the following list:

- `copyright()`
- `credits()`
- `license()`
- `help()`

All four commands provide you with help, of a sort, about Python. For example, the `copyright()` command tells you about who holds the right to copy, license, or otherwise distribute Python. The `credits()` command tells you who put Python together. The `license()` command describes the usage agreement between you and the copyright holder. Unlike `copyright()` and `credits()`, `license()` requires that you either press Enter to obtain more licensing information or type **q** and press Enter to exit the licensing screen, as shown in Figure 3-2, when working with Colab. (Jupyter Notebook simply prints the following: `See https://www.python.org/psf/license/`.) Notice that the Run Cell icon to the left of the cell continues to remain active as long as there is more licensing information to see.

The command you most want to know about is simply `help()`. To enter help mode, type **help()** and click Run Cell. Notice that you must include the parentheses after the command. Every Python command has parentheses associated with it. After you enter this command, Python goes into help mode and you see a display similar to the one shown in Figure 3-3.

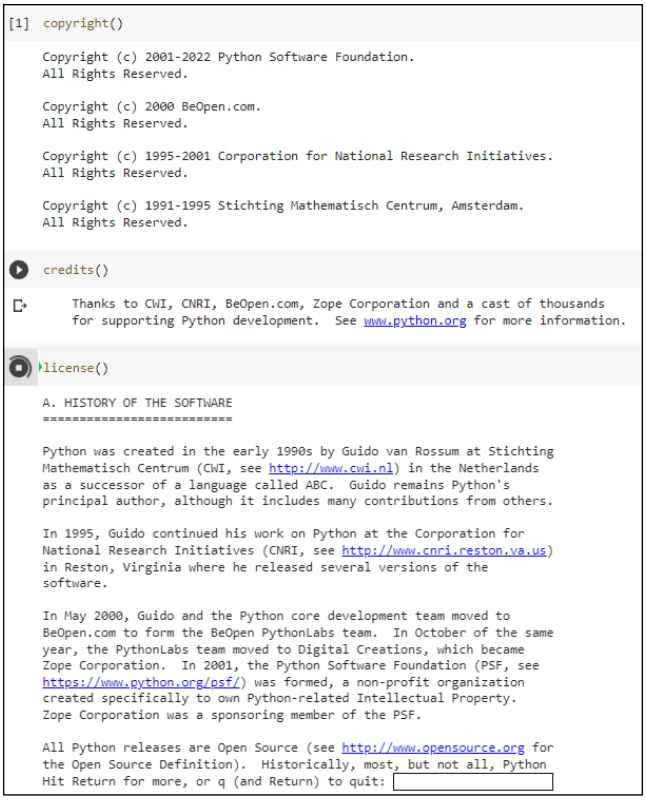


FIGURE 3-2: Some commands require further input from you to complete.

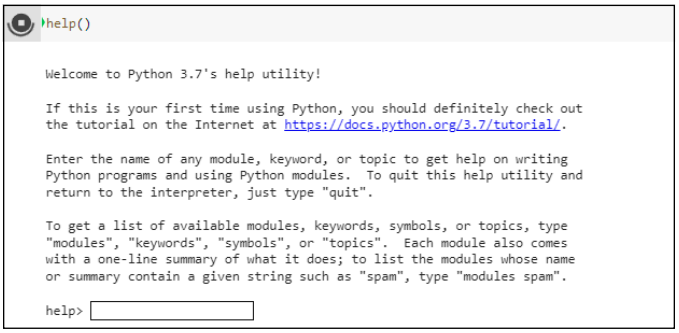


FIGURE 3-3: You ask Python about other commands in help mode.

REMEMBER

You can always tell that you're in help mode by the `help>` prompt that you see in the cell and that the Run Cell icon remains active. As long as you see the `help>` prompt, you know that you're in help mode.

Asking for help

To obtain help, you need to know what question to ask. The initial help message that you see when you go into help mode (refer to Figure 3-3) provides some helpful tips about the kinds of questions you can ask. If you want to explore Python, the four basic topics are

- `modules`
- `keywords`
- `symbols`
- `topics`

The first two topics won't tell you much for now. You won't need the `modules` topic until Chapter 11. The `keywords` topic will begin proving useful in Chapter 5. However, the `symbols` and `topics` keywords are already useful because they help you understand where to begin your Python adventure. When you type **symbols** and press Enter, you see a list of symbols used in Python. To see what topics are available, type **topics** and press Enter. You see a list of topics similar to those shown in Figure 3-4.

```
help> topics

Here is a list of available topics.  Enter any topic name to get more help.

ASSERTION           DELETION            LOOPING             SHIFTING
ASSIGNMENT          DICTIONARIES        MAPPINGMETHODS      SLICINGS
ATTRIBUTEMETHODS    DICTIONARYLITERALS  MAPPINGS            SPECIALATTRIBUTES
ATTRIBUTES          DYNAMICFEATURES     METHODS             SPECIALIDENTIFIERS
AUGMENTEDASSIGNMENT ELLIPSIS            MODULES             SPECIALMETHODS
BASICMETHODS        EXCEPTIONS          NAMESPACES          STRINGMETHODS
BINARY              EXECUTION           NONE                STRINGS
BITWISE             EXPRESSIONS         NUMBERMETHODS       SUBSCRIPTS
BOOLEAN             FLOAT               NUMBERS             TRACEBACKS
CALLABLEMETHODS     FORMATTING          OBJECTS             TRUTHVALUE
CALLS               FRAMEOBJECTS        OPERATORS           TUPLELITERALS
CLASSES             FRAMES              PACKAGES            TUPLES
CODEOBJECTS         FUNCTIONS           POWER               TYPEOBJECTS
COMPARISON          IDENTIFIERS         PRECEDENCE          TYPES
COMPLEX             IMPORTING           PRIVATENAMES        UNARY
CONDITIONAL         INTEGER             RETURNING           UNICODE
CONTEXTMANAGERS     LISTLITERALS        SCOPING
CONVERSIONS         LISTS               SEQUENCEMETHODS
DEBUGGING           LITERALS            SEQUENCES

help>
```

FIGURE 3-4: The `topics` help topic provides you with a starting point for your Python adventure.

REMEMBER

Chapter 7 begins the discussion of `symbols` when you explore the use of operators in Python. When you see a topic that you like, such as `FUNCTIONS`, simply type that topic and press Enter. To see how this works, type **FUNCTIONS** and press Enter (you must type the word in uppercase — don't worry, Python won't think you're shouting). You see help information similar to that shown in Figure 3-5.

```
help> FUNCTIONS
Functions
*********

Function objects are created by function definitions.  The only
operation on a function object is to call it: "func(argument-list)".

There are really two flavors of function objects: built-in functions
and user-defined functions.  Both support the same operation (to call
the function), but the implementation is different, hence the
different object types.

See Function definitions for more information.

Related help topics: def, TYPES

help>
```

FIGURE 3-5: You must use uppercase when requesting topic information.

As you work through examples in the book, you use commands that look interesting, and you might want more information about them. For example, in the "Seeing the result" section of this chapter, you use the `print()` command. To see more information about the `print()` command, type **print** and press Enter (notice that you don't include the parentheses this time because you're requesting help about `print()`, not actually using the command). Figure 3-6 shows typical help information for the `print()` command.

```
help> print
Help on built-in function print in module builtins:

print(...)
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

    Prints the values to a stream, or to sys.stdout by default.
    Optional keyword arguments:
    file:  a file-like object (stream); defaults to the current sys.stdout.
    sep:   string inserted between values, default a space.
    end:   string appended after the last value, default a newline.
    flush: whether to forcibly flush the stream.

help>
```

FIGURE 3-6: Request command help information by typing the command using whatever case it actually uses.

TIP

Unfortunately, reading the help information probably doesn't help much yet because you need to know more about Python. However, you can ask for more information. For example, you might wonder what `sys.stdout` means — and the help topic certainly doesn't tell you anything about it. Type **sys.stdout** and press Enter. You see the help information shown in Figure 3-7.

You may still not find the information as helpful as you need, but at least you know a little more. In this case, help has a lot to say and it can't all fit on one screen (Figure 3-7 is a truncated view). To see more of the information, you'll need to scroll up or down in the cell.

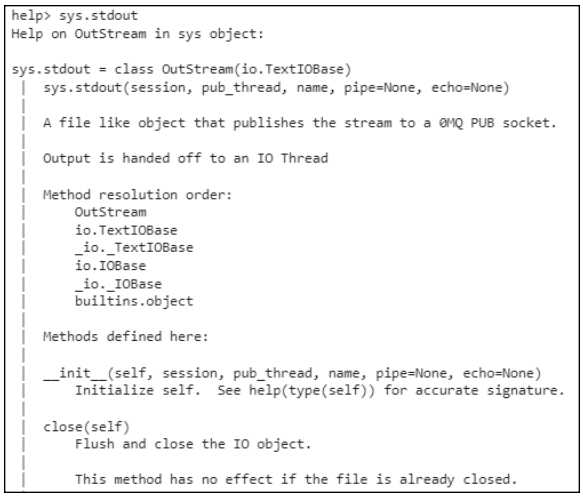


FIGURE 3-7: You can ask for help on the help you receive.

Leaving help mode

At some point, you need to leave help mode to perform useful work. All you have to do is press Enter without typing anything. When you press Enter, you see a message about leaving help, and then the Run Cell icon changes to the standard Python prompt, as shown in Figure 3-8.

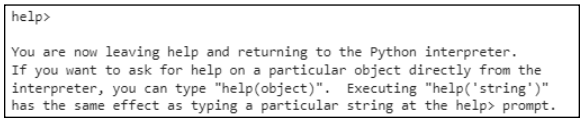


FIGURE 3-8: Exit help mode by pressing Enter without typing anything.

Obtaining help directly

Entering help mode isn't necessary unless you want to browse, which is always a good idea, or unless you don't actually know what you need to find. If you have a good idea of what you need, all you need to do is ask for help directly (a really nice thing for Python to do). So, instead of fiddling with help mode, you simply type the word *help,* followed by a left parenthesis and single quote, whatever you want to find, another single quote, and the right parenthesis. For example, if you want to know more about the `print()` command, you type **help('print')** and click Run Cell. Figure 3-9 shows typical output when you access help this way.

You can browse directly within a cell, too. For example, when you type **help('topics')** and press Enter, you see a list of topics like the one that appears in Figure 3-10. You can compare this list with the one shown in Figure 3-4. The two lists are

identical, even though you typed one while in help mode and the other while at the Python prompt.

```
help('print')

Help on built-in function print in module builtins:

print(...)
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

    Prints the values to a stream, or to sys.stdout by default.
    Optional keyword arguments:
    file:  a file-like object (stream); defaults to the current sys.stdout.
    sep:   string inserted between values, default a space.
    end:   string appended after the last value, default a newline.
    flush: whether to forcibly flush the stream.
```

FIGURE 3-9: Python lets you obtain help whenever you need it without leaving the Python prompt.

```
help('topics')


Here is a list of available topics.  Enter any topic name to get more help.

ASSERTION            DELETION             LOOPING              SHIFTING
ASSIGNMENT           DICTIONARIES         MAPPINGMETHODS       SLICINGS
ATTRIBUTEMETHODS     DICTIONARYLITERALS   MAPPINGS             SPECIALATTRIBUTES
ATTRIBUTES           DYNAMICFEATURES      METHODS              SPECIALIDENTIFIERS
AUGMENTEDASSIGNMENT  ELLIPSIS            MODULES              SPECIALMETHODS
BASICMETHODS         EXCEPTIONS           NAMESPACES           STRINGMETHODS
BINARY               EXECUTION            NONE                 STRINGS
BITWISE              EXPRESSIONS          NUMBERMETHODS        SUBSCRIPTS
BOOLEAN              FLOAT                NUMBERS              TRACEBACKS
CALLABLEMETHODS      FORMATTING           OBJECTS              TRUTHVALUE
CALLS                FRAMEOBJECTS         OPERATORS            TUPLELITERALS
CLASSES              FRAMES               PACKAGES             TUPLES
CODEOBJECTS          FUNCTIONS            POWER                TYPEOBJECTS
COMPARISON           IDENTIFIERS          PRECEDENCE           TYPES
COMPLEX              IMPORTING            PRIVATENAMES         UNARY
CONDITIONAL          INTEGER              RETURNING            UNICODE
CONTEXTMANAGERS      LISTLITERALS         SCOPING
CONVERSIONS          LISTS                SEQUENCEMETHODS
DEBUGGING            LITERALS             SEQUENCES
```

FIGURE 3-10: You can browse at the Python prompt if you really want to.

TIP

You might wonder why Python has a help mode at all if you can get the same results directly as a cell command. The answer is convenience. It's easier to browse in the help mode. In addition, even though you don't do a lot of extra typing as a separate cell command, you do perform less typing while in help mode. Help mode also provides additional helps, such as by listing topics that you can type, as shown previously in Figure 3-4. So you have all kinds of good reasons to enter help mode when you plan to ask Python a lot of help questions.

REMEMBER

No matter where you ask for help, you need to observe the correct capitalization of help topics. For example, if you want general information about functions, you must type **help('FUNCTIONS')** and not `help('Functions')` or `help('functions')`. When you use the wrong capitalization, Python will tell you that it doesn't know what you mean or that it couldn't find the help topic. It won't know to tell you that you used the wrong capitalization. Someday computers will know what you meant to type, rather than what you did type, but that hasn't happened yet.

Finding Out More about Functions and Objects

The help feature in Python offers an overview of the various functions and objects found in an application. However, you often need to know more than what help provides. In this case, you can use a type of alternative approach to help as described in the following sections.

Yelling "Hello There" doesn't help: Use dir() instead

There is a need to determine the details of what an application requires without expending a lot of effort. Of course, you could try simply asking, starting with "Hello There!" as a form of greeting, but extensive testing shows that speaking to your computer won't work. The `dir()` function gives you a means of discovering new things about functions and objects in Python. It also provides more information in some respects to using `help()` and is more flexible to boot. Using the `dir()` function by itself displays a listing of functions, classes, and objects that are currently available and are similar to this list (your list could vary depending on what you have loaded into Python):

```
['In', 'Out', '_', '__', '___', '__builtin__',
 '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__', '_dh', '_i', '_i1', '_ih',
 '_ii', '_iii', '_oh', '_sh', 'exit', 'get_ipython',
 'quit']
```

REMEMBER

Note that this list is reformatted into horizontal form to save space in the book. You'll actually see it in a vertical format that can be a little hard to read.

As you import packages, classes, and functions and execute code, the list will change to reflect the new additions. For example, when you execute the following code:

```
myNum = 2056
dir(myNum)
```

you see a list of the functions you can use with `myNum`, as shown here:

```
['__abs__', '__add__', '__and__', '__bool__',
 '__ceil__', '__class__', '__delattr__', '__dir__',
```

```
'__divmod__', '__doc__', '__eq__', '__float__',
'__floor__', '__floordiv__', '__format__', '__ge__',
'__getattribute__', '__getnewargs__', '__gt__',
'__hash__', '__index__', '__init__',
'__init_subclass__', '__int__', '__invert__', '__le__',
'__lshift__', '__lt__', '__mod__', '__mul__', '__ne__',
'__neg__', '__new__', '__or__', '__pos__', '__pow__',
'__radd__', '__rand__', '__rdivmod__', '__reduce__',
'__reduce_ex__', '__repr__', '__rfloordiv__',
'__rlshift__', '__rmod__', '__rmul__', '__ror__',
'__round__', '__rpow__', '__rrshift__', '__rshift__',
'__rsub__', '__rtruediv__', '__rxor__', '__setattr__',
'__sizeof__', '__str__', '__sub__', '__subclasshook__',
'__truediv__', '__trunc__', '__xor__',
'as_integer_ratio', 'bit_length', 'conjugate',
'denominator', 'from_bytes', 'imag', 'numerator',
'real', 'to_bytes']
```

That's quite a list of things you can do with `myNum`! What this list tells you is the details about `myNum`, an object that you created. So, you could type **myNum.bit_length()** and press Enter to see that the bit length for this particular variable is `12`. Using `dir()` can help jog your memory or make you aware of new features you might not have known about.

What are those double underscores all about?

Python lacks the concept of private variables, classes, functions, and so on. Some developers feel that this makes the object-oriented programming (OOP) features in Python incomplete (see `https://www.geeksforgeeks.org/data-hiding-in-python/` for a conversation about the topic). The creator of Python, Guido van Rossum, put it this way, "We are all adults. Feel free to shoot yourself in the foot if you must." So, Python lacks true data hiding and the benefits and problems that data hiding provides. What it has instead is double underlines (`__`), which indicates that the member is private and you shouldn't use it, but many developers do anyway.

Some Python implementations enforce the double underline as private, even though this functionality isn't part of the specification. For example, `myNum.__eq__(2056)` returns `True` if `myNum` actually does contain the value `2056`, but the `__eq__()` function is listed as private so what you should use in your code instead is `myNum == 2056`.

Drilling, drilling, drilling down into classes

In any OOP implementation, classes inherit from other classes, so you end up with an object hierarchy. Drilling down into this hierarchy can tell you a lot about how to interact with Python objects in ways that you might not have considered in the past. For example, when working with `myNum`, you can use the `myNum.to_bytes(4, byteorder = 'big')` call to display `myNum` as a series of bytes: `b'\x00\x00\x08\x08'`. The output is another object. Consequently, you can use `dir(myNum.to_bytes(4, byteorder = 'big'))` call to find out more about how you can use this new object:

```
['__add__', '__class__', '__contains__', '__delattr__',
 '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
 '__getattribute__', '__getitem__', '__getnewargs__',
 '__gt__', '__hash__', '__init__', '__init_subclass__',
 '__iter__', '__le__', '__len__', '__lt__', '__mod__',
 '__mul__', '__ne__', '__new__', '__reduce__',
 '__reduce_ex__', '__repr__', '__rmod__', '__rmul__',
 '__setattr__', '__sizeof__', '__str__',
 '__subclasshook__', 'capitalize', 'center', 'count',
 'decode', 'endswith', 'expandtabs', 'find', 'fromhex',
 'hex', 'index', 'isalnum', 'isalpha', 'isascii',
 'isdigit', 'islower', 'isspace', 'istitle', 'isupper',
 'join', 'ljust', 'lower', 'lstrip', 'maketrans',
 'partition', 'replace', 'rfind', 'rindex', 'rjust',
 'rpartition', 'rsplit', 'rstrip', 'split',
 'splitlines', 'startswith', 'strip', 'swapcase',
 'title', 'translate', 'upper', 'zfill']
```

Oddly enough, you can replace some of the bytes in the value with different bytes using `myNum.to_bytes(4, byteorder = 'big').replace(b'\x00', b'\x01')`, which provides an output of: `b'\x01\x01\x08\x08'`. However, you might not have known you could do this without drilling down into the original `myNum` object using `dir()`.

If you really want to get fancy, you can start combining bits of knowledge you obtain while drilling down into objects. For example, `myNum.from_bytes(myNum.to_bytes(4, byteorder = 'big').replace(b'\x00', b'\x01'), 'big')` produces a new `myNum` value of `16844808`. That's a really large number, but it also shows that you can work with Python to produce a wide assortment of interesting effects just by playing with `dir()`.

Playing the Part of Inspector

The `inspect` module (see `https://docs.python.org/3/library/inspect.html`) allows you to get even more in depth than the `dir()` function by playing the role of a detective. You can query any object, class, or variable about its details in a detailed manner. The following sections give you the briefest overview of what you can do because this module can become really involved. As with all tools, you need to spend time with `inspect` before you become proficient at using it.

Gaining access to inspect

The `inspect` module isn't included with the standard Python setup, so you must import it into the environment so that you can use it by adding the following line of code:

```
import inspect
```

REMEMBER

This code imports the entire `inspect` module. If you want to use a limited number of features, you can also use other Python methods of importation, such as

```
from inspect import isfunction
```

Instead of importing everything, this approach imports just the `isfunction()` method.

Using inspect

There are a lot of different ways to use `inspect`. For example, you can use it to ask direct questions such as `inspect.ismodule(inspect)`, which returns `True` because inspect is a module. It's also possible to use `inspect` on any object, including those you create. The query `inspect.getmembers(myNum)` returns a considerable amount of information, as shown here (in part; the actual list is much longer):

```
[('__abs__', <method-wrapper '__abs__' of int object at
    0x7f71e52ea4b0>),
 ('__add__', <method-wrapper '__add__' of int object at
    0x7f71e52ea4b0>),
 ('__and__', <method-wrapper '__and__' of int object at
    0x7f71e52ea4b0>),
 ('__bool__', <method-wrapper '__bool__' of int object
    at 0x7f71e52ea4b0>),
 ('__ceil__', <function int.__ceil__>),
 ('__class__', int),
```

```
 ('__delattr__',
    <method-wrapper '__delattr__' of int object at
    0x7f71e52ea4b0>),
 ('__dir__', <function int.__dir__>),
 ('__divmod__', <method-wrapper '__divmod__' of int
    object at 0x7f71e52ea4b0>),

...

 ('bit_length', <function int.bit_length>),
 ('conjugate', <function int.conjugate>),
 ('denominator', 1),
 ('from_bytes', <function int.from_bytes>),
 ('imag', 0),
 ('numerator', 2056),
 ('real', 2056),
 ('to_bytes', <function int.to_bytes>)]
```

TIP

Most of the added information tells you what a particular member is, such as a method or a function. In some cases, you also get the member value. For example, the `denominator()` function output is `1`.

It may take some time for you to go through all the `inspect` features, but it's worth the effort. For example, you can quickly check whether a function is built in. A call to `inspect.isbuiltin(dir)` returns `True` because the `dir()` function is part of the default Python functionality.

Is reflection really like looking in a mirror?

The process used by the `inspect` module to obtain detailed information about your code is called *reflection*, but it has nothing to do with looking in a mirror. Instead, reflection looks back through the compiled code in an effort to reconstruct the original element information. Another term, *introspection*, means essentially the same thing when it comes to viewing code, and some people would say that the `inspect` module performs introspection.

TECHNICAL STUFF

In addition to viewing element information, reflection can also allow you to modify the underlying objects, but this is an extremely advanced technique that most developers don't really need. For the purposes of this book, looking back through code to find out more about it is called *reflection*, just to keep things simple (and to keep the terminology on par with what other languages use). You can discover more about the differences between introspection and reflection at `https://betterprogramming.pub/python-reflection-and-introspection-97b348be54d8` and see some advanced used of reflection at `https://www.geeksforgeeks.org/reflection-in-python/`.

IN THIS CHAPTER

- Writing and running the first application
- Formatting your application code
- Using comments effectively
- Developing manageable applications

Chapter 4
Writing Your First Application

Many people view application development as some sort of magic practiced by wizards called geeks who wave their keyboard to produce software both great and small. However, the truth is a lot more mundane.

Application development follows a number of processes. It's more than a strict procedure, but is most definitely not magic of any sort. As Arthur C. Clark once noted, "Any sufficiently advanced technology is indistinguishable from magic." This chapter is all about removing the magic from the picture and introducing you to the technology. By the time you're finished with this chapter, you, too, will be able to develop a simple application (and you won't use magic to do it).

As with any other task, people use tools to write applications. In the case of Python, you don't have to use a tool, but using a tool makes the task so much easier that you really will want to use one. In this chapter, you use a commonly available Integrated Development Environment (IDE) named Google Colab (discussed in Chapter 2). An *IDE* is a special kind of application that makes writing, testing, and debugging code significantly easier. You can also use Jupyter Notebook for desktop development, which appears as part of the Anaconda tool collection (`https://www.anaconda.com/products/distribution`) or you can get Jupyter Notebook as a separate download at `https://jupyter.org/install`.

TIP

A vast number of other tools are available for you to use when writing Python applications. This book doesn't tell you much about them because *Conda* (a command-line tool provided with Jupyter Notebook) performs every task needed and it's readily available free of charge. However, as your skills increase, you might find the features in other tools such as Komodo Edit (`https://www.activestate.com/products/komodo-ide/downloads/edit/`) more to your liking. You can find a great list of these tools at `https://wiki.python.org/moin/IntegratedDevelopmentEnvironments`.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 04; Comments.ipynb`, `BPP4D3E; 04; Indentation.ipynb`, and `BPP4D3E; 04; Sample.ipynb` files of the downloadable source. See the Introduction for details on how to find these source files.

Understanding Why IDEs Are Important

A good question to ask is, why do you need an IDE to work with Python? It does come with a command-line tool that you can open by typing **python** and pressing Enter at a command line or terminal window for desktop setups. (You can make things easier by ensuring that the folder in which Python is installed appears as part of your system's `PATH` environment variable or by installing Anaconda, which comes with its own Anaconda prompt.) For that matter, Python actually comes with a limited IDE called Integrated Development and Learning Environment (IDLE) that appears in the `Lib\idlelib` folder of your Python installation (again, for desktop setups, but please, no more email about just how dead the desktop really is; I get it). Many people probably question the need for anything more during the learning process and possibly to develop full-fledged applications.

REMEMBER

Unfortunately, the tools that come with Python are interesting and even helpful in getting started, but they won't help you create useful applications with any ease. In addition, accessing these tools means creating a desktop setup that may not provide the flexibility you require because now you can't program on your tablet (or smart phone, if you're really adept). If you choose to work with Python long term, you really need a better tool for the reasons described in the following sections.

Creating better code

A good IDE contains a certain amount of intelligence. For example, the IDE can suggest alternatives when you type the incorrect keyword, or it can tell you that a certain line of code simply won't work as written. The more intelligence that an

IDE contains, the less hard you have to work to write better code. Writing better code is essential because no one wants to spend hours looking for errors, called *bugs*.

TIP

IDEs vary greatly in the level and kind of intelligence they provide, which is why so many IDEs exist. You may find the level of help obtained from one IDE to be insufficient to your needs, but another IDE hovers over you like a mother hen. Every developer has different needs and, therefore, different IDE requirements. The point is to obtain an IDE that helps you write clean, efficient code quickly and easily.

Debugging functionality

Finding bugs (errors) in your code is a process called *debugging*. Even the most expert developer in the world spends time debugging. Writing perfect code on the first pass is nearly impossible, especially when squinting to see the text on your smart phone. When you do, it's cause for celebration because it won't happen often. Consequently, the debugging capabilities of your IDE are critical. Unfortunately, the debugging capabilities of the native Python tools are almost nonexistent. If you spend any time at all debugging, you quickly find the native tools annoying because of what they don't tell you about your code.

TIP

The best IDEs double as training tools. Given enough features, an IDE can help you explore code written by true experts. Tracing through applications using a debugger is a time-honored method of learning new skills and honing the skills you already possess. A seemingly small advance in knowledge can often become a huge savings in time later. When looking for an IDE, don't just look at debugging features as a means to remove errors — see them also as a means to learn new things about Python.

Defining why notebooks are useful

Most IDEs look like fancy text editors, and that's precisely what they are. Yes, you get all sorts of intelligent features, hints, tips, code coloring, and so on, but at the end of the day, they're all text editors. There's nothing wrong with text editors, and this chapter isn't telling you anything of the sort. However, given that Python developers often focus on scientific applications that require something better than pure text presentation, using notebooks instead can be helpful.

REMEMBER

A *notebook* differs from a text editor in that it focuses on a technique advanced by Stanford computer scientist Donald Knuth called literate programming. You use *literate programming* to create a kind of presentation of code, notes, math equations, and graphics. In short, you wind up with a scientist's notebook full of

everything needed to understand the code completely. You commonly see literate programming techniques used in high-priced packages such as Mathematica and MATLAB (see *MATLAB For Dummies*, 2nd Edition, by John Paul Mueller and Jim Sizemore [Wiley] for details). Notebook development excels at

- Demonstration
- Collaboration
- Research
- Teaching objectives
- Presentation

This book uses Colab and Jupyter Notebook because these IDEs provide you with a great Python coding experience, as well as help you discover the enormous potential of literate programming techniques. If you spend a lot of time performing scientific tasks, Colab and products like it are essential. In addition, both Colab and Jupyter Notebook are free, so you get the benefits of the literate programming style without the cost of other packages.

REMEMBER

When working through this book, you see Colab used to reference Google Colab specifically, Jupyter Notebook used to reference Jupyter Notebook specifically, and Notebook (by itself) to refer to both Colab and Jupyter Notebook generically. There are differences between the two products that you need to know about, and the book points them out for you.

Creating the Application

You've actually created your first Python application by using the steps in the "Seeing the result" section of Chapter 3. The `print()` method may not seem like much, but you use it quite often. However, the literate programming approach provided by Colab requires a little more knowledge than you currently have. The following sections don't tell you everything about this approach, but they do help you gain an understanding of what literate programming can provide in the way of functionality. However, before you begin, make sure you have the `BPP4D3E; 04; Sample.ipynb` file open for use because you need it to explore Notebook.

Developing the code

Real applications do something more than print a message using the `print()` method. Starting with Part 2 of the book, you develop the skills required to do

something really interesting. For now though, it's time to do something a little better than just printing a message, even if you don't quite understand everything that the code does. (You definitely will later in the book.) Create a new notebook (see the "Creating a new notebook" section of Chapter 2 if you need some help in this regard) and type the following code into a code cell:

```
def addSomething(value1, value2):
    return value1 + value2

print(addSomething(1, 2))
```

When you click the Run Cell icon (or click the Run button in Jupyter Notebook), you see the result of `3` magically appear. Of course, the delight in seeing code that you typed run is momentary. A second impression might be that this is an interesting group of gibberish letters that really don't mean much to most humans. So it's a good time to break this code down a little and add comments to it so that you can remember what it does. Add the following comments (in bold), as shown here:

```
# Define a function to encapsulate task code that accepts
# two pieces of information in boxes called variables
# named value1 and value2.
def addSomething(value1, value2):

    # Add the two values together and return the result
    # to the caller.
    return value1 + value2

# Call the addSomething() function with two numbers to
# add and then use print() to display the result.
print(addSomething(1, 2))
```

Comments always begin with a hashtag (`#`) so that the Python interpreter doesn't mistake them for code. It's amazing what a difference a few comments can make. Of course, you still might not understand the code completely, but you have a better idea of how it works, which is why you always comment your code.

Adding documentation cells

Cells come in a number of different forms. This book doesn't use them all. However, knowing how to use the documentation cells can come in handy. Select the first cell (the one with the code you just created in it). Click the +Text button in Colab or choose Insert ➪ Insert Cell Above in Jupyter Notebook. You see a new cell added to the notebook. (When working with Jupyter Notebook, you need to select Markdown from the drop-down list that currently contains Code in it.) Type **# Creating the Application** because this is the parent heading for this section. The `#`

symbol is called *markup*, which is a method of telling Notebook how to format text (the "Working with text cells" section of the chapter talks about using markup in more detail). Select the next cell in Colab or click Run in Jupyter Notebook. You see the text change into a heading. Click the Move Cell Up button in Colab as needed to place the heading correctly, as shown in Figure 4-1.

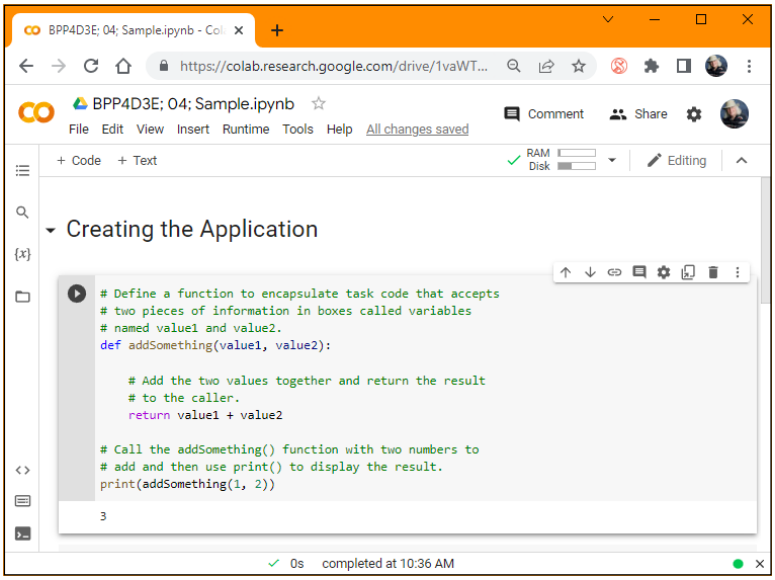


FIGURE 4-1: Adding headings helps you separate and document your code.

About now, you may be thinking that these special cells act just like HTML pages, and you'd be right. Use the same process you did for adding the first heading to add a second heading by typing **## Developing the code**. As you can see in Figure 4-2, the number of hashes (#) you add to the text affects the heading level, but the hashes don't show up in the actual heading.

TIP

Notice that you have also documented precisely where the code appears in the book. The other source code examples in the book follow this same strategy so that you can easily match the source code to the section that describes it.

Other cell content

This chapter (and book) doesn't demonstrate all the kinds of cell content that you can see by using Colab and Jupyter Notebook. However, you can add things like graphics to your notebooks, too. When the time comes, you can output (print) your notebook as a report and use it in presentations of all sorts. The literate programming technique is different from what you may have used in the past, but it has definite advantages, as you see in upcoming chapters.

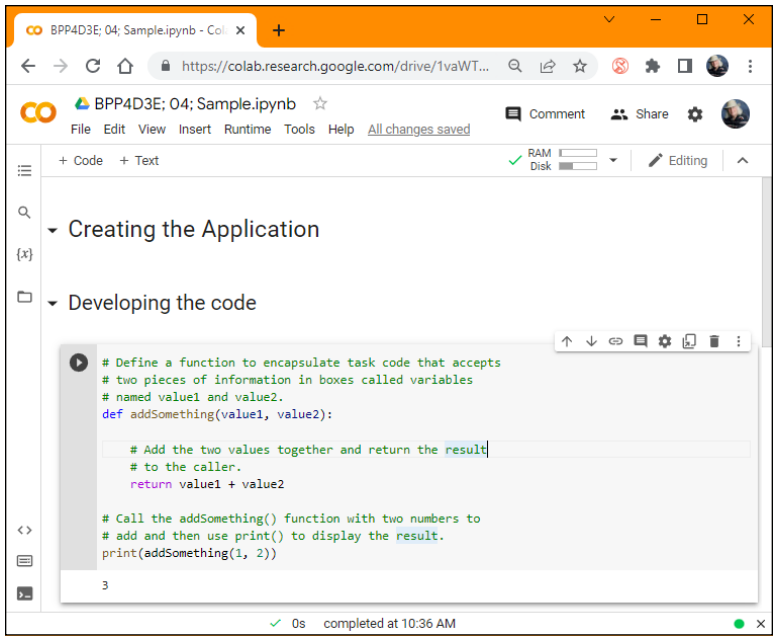


FIGURE 4-2: Using heading levels provides emphasis for cell content.

Playing around with scratch cells

There are times when you want to experiment without making code a part of your main application. You may also need to access a shell while working with Colab. For example, the Linux `ls` command (`https://linuxize.com/post/how-to-list-files-in-linux-using-the-ls-command/`) displays a list of files and directories found in the location where you're executing the Colab code. In this case, you can create a scratch cell, like the one shown in Figure 4-3, by choosing Insert➪ Scratch Code Cell. This feature is available only in Colab and not in Jupyter Notebook.

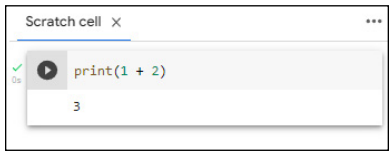


FIGURE 4-3: A scratch cell provides a place to experiment and run shell commands.

Clicking the ellipsis in the upper-right corner displays tasks you can perform with the scratch cell. Here are some of the most important options:

- **Copy Cell:** Copies the cell contents to the Clipboard where you can paste the information to another application.

- **Clear Output:** Clears the cell output before you run the scratch code again.
- **View Output Fullscreen:** Displays the cell data full screen so that you can see all of it at one time outside the Colab panes. Press Esc to exit the full-screen mode.
- **Move Tab to Next Pane:** Places the scratch cell in the next available pane. Normally, a scratch cell opens in its own pane, which may take up too much screen real estate in some cases.
- **Move Tab to Previous Pane:** Places the scratch cell in the previous pane, which may be the pane originally used to hold it.
- **Close Tab:** Closes the selected scratch cell tab without affecting any other tabs.
- **Close All Tabs:** Closes all the scratch cell tabs without affecting any other Colab cells. Note that you may also see options for closing all the tabs to the left of the current tab and all the tabs to the right of the current tab.
- **Change Page Layout:** Determines the appearance of Colab with regard to scratch cells. The default is to use two columns, but you can also use a single column view where the scratch cell tabs appear across the top with a separate tab labeled Notebook that holds your code. It's also possible to place the scratch cells below the last line in the current Notebook pane.

TIP

You can have more than one scratch cell at a time. Every time you choose Insert➪Scratch Code Cell, Colab creates another scratch cell tab for you. You can close the tabs in any order desired.

Interacting with form fields

Form fields let the user enter values that a program can then use as input. Just as Jupyter Notebook doesn't provide scratch cells, it also doesn't provide access to form fields. So even though adding form fields to your Colab notebook won't prevent the code from also running in Jupyter Notebook, you also won't see the functionality that form fields provide in Jupyter Notebook.

WARNING

Using form fields adds the potential for incompatibility to your notebooks. Unless you have a good reason to add GUI elements to your notebook, it's usually best to skip the form fields and use standard Python data-entry methods instead, like the `input()` function shown here:

```
name = input("Type your name: ")
print(f"Your name is: {name}")
```

Choosing a form field type

If you do decide to use form fields in your program, you choose Insert➪Add A Form Field. You see the Add New Form Field dialog box, shown in Figure 4-4. The Form Field Type field contains a list of GUI elements you can add, which include `input`, `dropdown`, `slider`, and `markdown`.

FIGURE 4-4: The Add New Form Field dialog box allows you to add GUI inputs to your notebook.

The form field type that you choose determines how the Add New Form Field dialog box appears. For example, if you choose `dropdown`, you need to add a list of drop-down options. A `slider` requires that you add minimum and maximum values, and the size of the steps between values.

TIP

The markdown option is interesting because it allows you to add what amounts to a text cell within a code cell. You type **Colab markdown** (`https://colab.research.google.com/notebooks/markdown_guide.ipynb`) in the Markdown field. This means that you really can mix code and markdown together to create a better looking notebook.

Defining the variable type and name

To use a form field, all you need to do is fill in the blanks. Follow these steps for this example:

1. **Choose input in the Form Field Type field.**

   You see the dialog box shown in Figure 4-4.

2. **Choose string in the Variable Type field.**

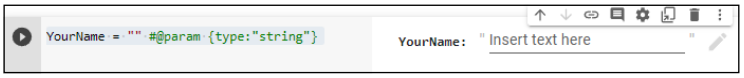
3. **Type** YourName **in the Variable Name field; then click Save.**

   You see the combination of code and GUI presentation shown in Figure 4-5.

TIP

Displaying both code and GUI presentation may seem counterproductive because you want a nice display for your notebook. To get rid of the code, choose the ellipsis in the cell toolbar and then select Form ➪ Hide Code from the menu.

FIGURE 4-5: Adding a form field to your notebook displays a combination of code and GUI presentation.



Using the form field

One of the interesting aspects of form fields is that they exist as part of a code cell, so you can add code directly below the form field entry to interact with the form field. Type **print(f"Your name is: {YourName}")** directly below the `YourName = "" #@param {type:"string"}` entry in the cell. Type your name in the form field, and you see that the code changes as well. Now, click Run Cell and you see that the code interacts with the form field as, shown in Figure 4-6.

FIGURE 4-6: Design code to interact with the form fields you create.

REMEMBER

By linking a number of form fields together in the same cell, you can create the appearance of a form used in any other application. The user fills out the form and clicks Run Cell to see the result. When you hide the code area, the GUI presentation looks seamless, and because it resides in a notebook, you have access to the presentation features that a notebook can provide as well.

Running the Application

In general, an application consists of the current notebook and any linked resources. It's not an application in the traditional sense because you can run it just one cell at a time, but it does help to view the application as being the entire notebook.

Seeing the result

To this point, you have seen applications run one cell at a time. To see the entire result without having to click Run Cell in each and every cell like an annoying little habit that drives sane people a little batty, you can use the following options:

- **Runtime ➪ Run All:** Runs all the cells without clearing the output first.
- **Runtime ➪ Run Before:** Runs all the cells above the cell that currently has focus.
- **Selected cells:** Select multiple cells using Shift+Up Arrow or Shift+Down Arrow; then choose Runtime ➪ Run Selection to run just the selected cells.
- **Runtime ➪ Run After:** Runs all the cells below the cell that currently has focus.
- **Runtime ➪ Restart and Run All:** Runs all the cells after restarting the runtime and clearing the output.

The Jupyter Notebook options are similar, but different enough to warrant separate coverage.

- **Selected cells:** Select multiple cells using Shift+Down Arrow or Shift+Up Arrow; then choose Cell ➪ Run Cells to run just the selected cells. You can also choose Cell ➪ Run Cells and Select Below to run the selected cells and choose the next cell as the focused cell. If there is no next cell, Jupyter Notebook creates one for you. Choosing Cell ➪ Run Cells and Insert Below runs the selected cells and then inserts a new cell as the focused cell even if there is an existing cell to use.

  When using this method, ensure that you have the cell selected and not the text within the cell. Jupyter Notebook highlights the selected cells for you in a light color.

TIP

- **Cell ➪ Run All:** Runs all the cells without clearing the output first.
- **Cell ➪ Run All Above:** Runs all the cells above the cell that currently has focus.
- **Cell ➪ Run All Below:** Runs all the cells below the cell that currently has focus.
- **Kernel ➪ Restart & Clear Output:** Clears the output from the previous runs without running the code. This is an important feature if you want to run just some of the cells without having previous results interfere with the current test.
- **Kernel ➪ Restart & Run All:** Runs all the cells after restarting the runtime and clearing the output.

Viewing the executed code history

It's helpful to see the history of code that you've run while creating an application. Doing so can help you locate errors in logic or even execution order. Sometimes, viewing the code history simply reminds you to restart the runtime or kernel because things have gotten messy with variables dangling about (staying when you don't need them any longer) and causing problems. To see the code history in Colab, choose View➪ Executed Code History. Figure 4-7 shows a typical execution history for the example found in the previous sections.

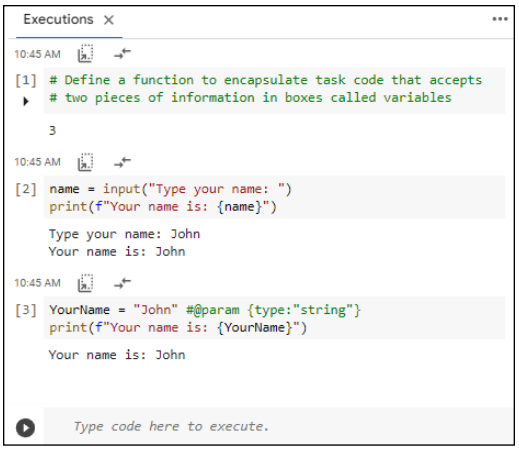


FIGURE 4-7: Viewing the executed code history can tell you a lot about your code.

The top of each entry contains the time that you executed the code. Next to the time stamp is an icon you can click to copy the code to a scratch cell for further analysis. The last icon in the time stamp row scrolls the display to the code in question and selects it so that you don't have to manually look for it.

TIP

At the bottom of the window is a cell where you can try ideas as you browse the executed code. It acts like a scratch cell, and you use it like any other cell.

Understanding the Use of Indentation

As you work through the examples in this book, you see that certain lines are indented. In fact, the examples also provide a fair amount of white space (such as extra lines between lines of code). The indentation is also functional. Python uses indentation to define code regions such as code associated with functions and loops. So, adding indentation is an important part of your Python program.

The various uses of indentation will become more familiar as you work your way through the examples in the book. However, you should know at the outset why indentation is used and how it gets put in place. So it's time for another example. The following steps help you create a new example that uses indentation to make the relationship between application elements a lot more apparent and easier to figure out later.

1. **Open a new notebook by selecting New Notebook in the Colab notebook opening dialog box, New ➪ Python3 in Jupyter Notebook, or by choosing File ➪ New Notebook (optionally choosing the correct notebook type) if you already have a notebook open.**

   Colab or Jupyter Notebook creates a new notebook for you.

2. **Type** print("This is a really long line of text that will" +**.**

   You see the text displayed normally onscreen, just as you expect. The plus sign (+) tells Python that there is additional text to display. (Don't add the period at the end of the sentence.) Adding text from multiple lines together into a single long piece of text is called *concatenation*. You learn more about using this feature later in the book, so you don't need to worry about it now.

3. **Press Enter.**

   The insertion point doesn't go back to the beginning of the line, as you might expect. Instead, it ends up directly under the first double quote, as shown in Figure 4-8. This feature is called automatic indention and it's one of the features that differentiates a regular text editor from one designed to write code.

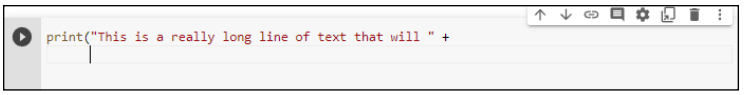


FIGURE 4-8: The Edit window automatically indents some types of text.

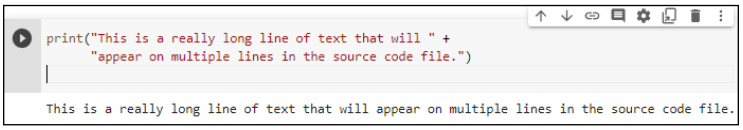
4. **Type** "appear on multiple lines in the source code file.") **and press Enter.**

   Notice that the insertion point goes back to the beginning of the line. When Notebook senses that you have reached the end of the code, it automatically outdents the text to its original position.

5. **Click Run.**

   You see the output shown in Figure 4-9. Even though the text appears on multiple lines in the source code file, it appears on just one line in the output.

FIGURE 4-9: Use concatenation to make multiple lines of text appear on a single line in the output.



Adding Comments

People create notes for themselves all the time. When you need to buy groceries, you look through your cabinets, determine what you need, and write it down on a list or dictate it into your phone. When you get to the store, you review your list to remember what you need. Using notes comes in handy for all sorts of needs, such as tracking the course of a conversation between business partners or remembering the essential points of a lecture. Humans need notes to jog their memories. Comments in source code are just another form of note. You add them to the code so that you can remember what task the code performs later. The following sections describe comments in more detail.

Understanding comments

Computers need some special way to determine that the text you're writing is a comment, not code to execute. Python provides two methods of defining text as a comment and not as code. The first method is the single-line comment. It uses the number sign (#), like this:

```
# This is a comment.
print("Hello from Python!") #This is also a comment.
```

HEADINGS VERSUS COMMENTS

You may find headings and comments a bit confusing at first. Headings appear in separate cells; comments appear with the source code. They serve different purposes. Headings serve to tell you about an entire code grouping, and individual comments tell you about individual code steps or even lines of code. Even though you use both of them for documentation, each serves a unique purpose. Comments are generally more detailed than headings.

REMEMBER

A single-line comment can appear on a line by itself or it can appear after executable code. It appears on only one line. You typically use a single-line comment for short descriptive text, such as an explanation of a particular bit of code. Notebook shows comments in a distinctive color (usually blue or green) and in italics.

Creating multiline comments

Python doesn't actually support a multiline comment directly, but you can create one using a triple-quoted string (called a *docstring*). A multiline comment both starts and ends with three double quotes (""") or three single quotes (''') like this:

```
"""
   Application: Comments.py
   Written by: John
   Purpose: Shows how to use comments.
"""
```

REMEMBER

Unlike single-line comments, multiline docstrings used as comments normally appear in red and in roman type. The interpreter does read them, but you can still use them for comments. These lines aren't executed. Python won't display an error message when they appear in your code. However, Notebook treats them differently, as shown in Figure 4-10.

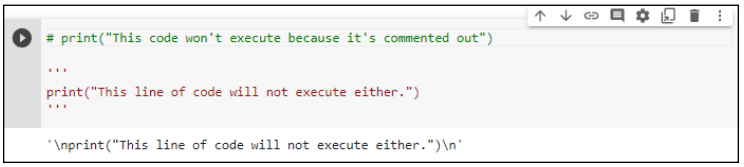


FIGURE 4-10: Multiline comments do work, but they also provide output.

Note that the actual Python comments, those preceded by a hash (#) in cell 1, don't generate any output. The triple-quote docstrings, however, do generate output. If you plan to output your notebook as a report, you need to avoid using triple-quoted docstrings. (Some IDEs, such as IDLE, ignore the triple-quoted strings completely.)

TECHNICAL STUFF

You can also assign a docstring to a variable, which isn't possible with a comment. The code and output shown in Figure 4-11 shows what a docstring looks like as output when assigned to a variable. Notice that the docstring isn't printed in this case because it appears as a variable assignment. The docstring formatting does remain in place, though, unlike the concatenation example shown in the "Understanding the Use of Indentation" section, earlier in this chapter.

FIGURE 4-11: Use docstrings for strings that require multiple lines.

You typically use multiline docstrings for longer explanations of who created an application, why it was created, and what tasks it performs. Of course, there aren't any hard rules on precisely how you use comments. The main goal is to tell the computer precisely what is and isn't a comment so that it doesn't become confused.

Using comments to leave yourself reminders

A lot of people don't really understand comments — they don't quite know what to do with notes in code. Keep in mind that you might write a piece of code today and then not look at it for years. You need notes to jog your memory so that you remember what task the code performs and why you wrote it. In fact, here are some common reasons to use comments in your code:

- Reminding yourself about what the code does and why you wrote it
- Recording who wrote the code when involved in a multiperson project (usually along with other information, such as how to contact the person)
- Telling others how to maintain your code
- Making your code accessible to other developers
- Listing ideas for future updates
- Providing a list of documentation sources you used to write the code
- Maintaining a list of improvements you've made

You can use comments in a lot of other ways, too, but these are the most common ways. Look at the way comments are used in the examples in the book, especially as you get to later chapters where the code becomes more complex. As your code becomes more complex, you need to add more comments and make the comments pertinent to what you need to remember about it.

Using comments to keep code from executing

Developers also sometimes use the commenting feature to keep lines of code from executing (referred to as *commenting out*). You might need to do this in order to determine whether a line of code is causing your application to fail. As with any other comment, you can use either single line commenting or multiline commenting. However, when using multiline commenting, you do see the code that isn't executing as part of the output (and it can actually be helpful to see where the code affects the output). Figure 4-12 shows an example of code commenting techniques.

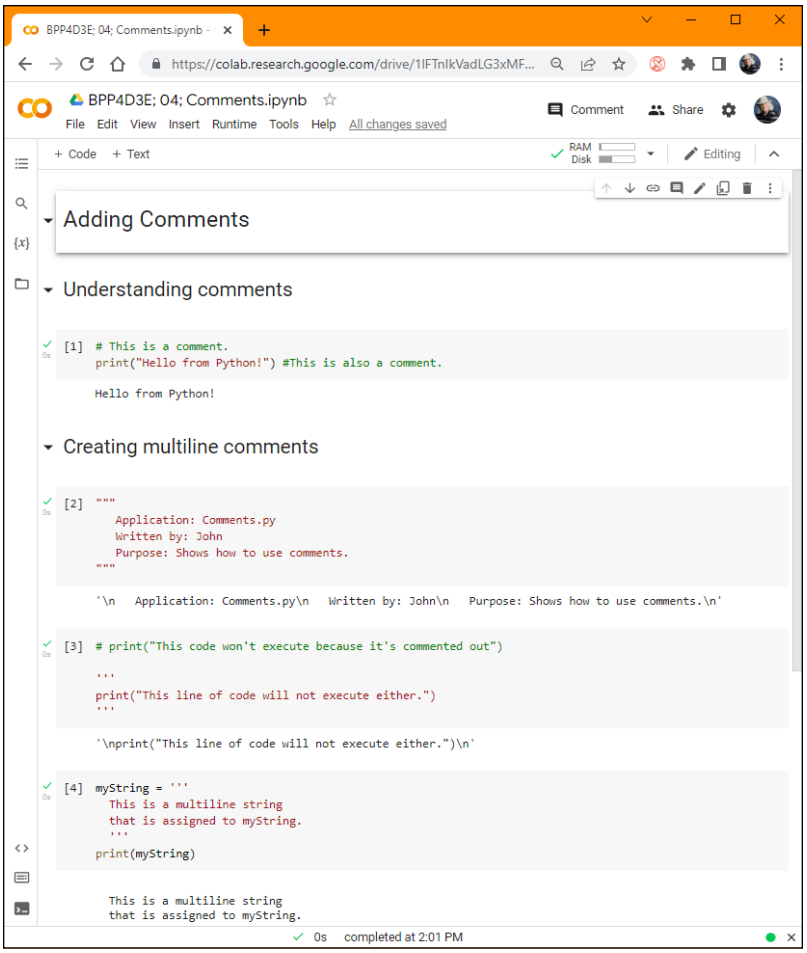


FIGURE 4-12: Use comments to keep code from executing.

Making Your Notebook Informative, Descriptive, and Pretty

After putting a lot of time and effort into creating headings and code for your notebook, you don't want to falter at the end and not add the fit and finish items that will truly make your notebook stand out from any competition. Fortunately, you don't need a degree in media to make notebooks interesting and even pretty. Adding pizzazz does require a little time, but the payoff is worth it when you make a presentation. The following sections offer some ideas on how to dress your notebook up.

Working with text cells

So far, you have worked with text cells that contain headings. You've also been exposed to some of the other markup options you can employ in a heading, such as adding italics. Boring! When you open a text cell in Colab, you see a toolbar containing all sorts of interesting options, like those found in Figure 4-13.

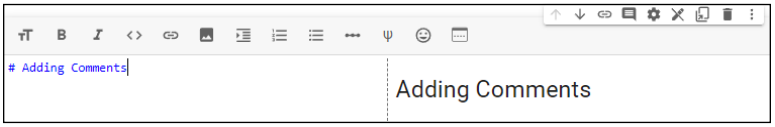


FIGURE 4-13: The text cell toolbar contains a variety of interesting options.

Here is what those buttons do:

- **Toggle Heading:** Cycles through the various heading levels so that you can see how the text appears in various sizes.
- **Bold:** Makes the selected text appear in bold type.
- **Italics:** Makes the selected text appear in italics type.
- **Format as Code:** Sets the selected text to appear as code rather than regular text. You can use this option for things like calling syntax or examples of usage.
- **Insert Link:** Adds a URL to the text cell so that the user can click it and see other online resources. This is also a handy way to create reference notes for yourself as you write the code so that you can simply go back to the website that contains the really good coding idea.

- **Insert Image:** Places an image at the current cursor position, making it possible to add graphical documentation to your notebook in addition to text. This is also a dandy way to personalize your notebook so that you can see a loved one as you type.
- **Indent:** Adds an indent to the current text. You can also use this feature to create multiple-level bulleted and numbered lists.
- **Add Numbered List:** Creates a new numeric list at the current cursor position.

TIP

You can add bulleted sublists to a numeric list and vice versa simply by using the indent feature. For example, Figure 4-14 shows what a nested list looks like. Notice the use of the > symbol to increase the indent level. Also notice that the bullet type changes to make the indent level.

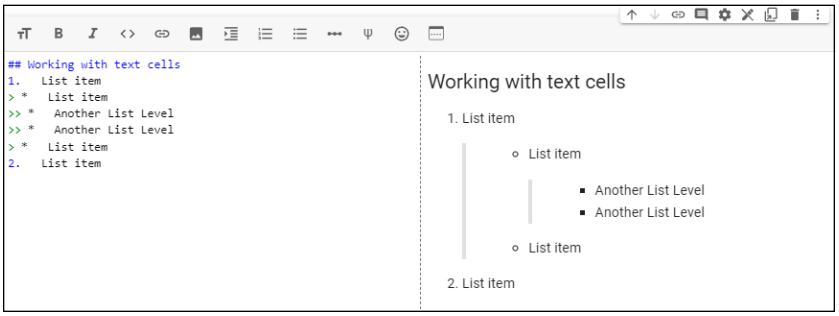


FIGURE 4-14: Use indents to create nested lists for your documentation.

- **Add Bulleted List:** Creates a new bulleted list at the current cursor position.
- **Add Horizontal Rule:** Places a line across the text cell to separate one group of material from another.
- **LaTeX:** Allows use of LaTeX markup (`https://www.latex-project.org/`) in the text cell.
- **Insert Emoji:** Places the selected emoji at the current cursor location. When you click this option, what you see is a list of standard emojis. Note that emoji support comes with Colab, but you may have to install it on Jupyter Notebook as described at `https://sawanrai777.medium.com/have-fun-with-emojis-in-jupyter-notebook-f74db7f45210`. Note that you can copy and paste emojis from websites and other locations into your code, but place them within a string to ensure correct interpretation by the IDE.

Adding section headers

A section heading is simply a text cell with a heading already defined in it. You can promote or demote the section heading level using fewer or more # characters in front of the heading. The only difference is that Colab does some of the work for you. Add a new section header to your notebook by choosing Insert➪Section Header Cell.

Interacting with the table of contents

The table of contents is a feature that is supported by Colab and not Jupyter Notebook. To see the Table of Contents, choose View➪Table of Contents or click the Table of Contents button on the left toolbar. Figure 4-15 shows typical output.

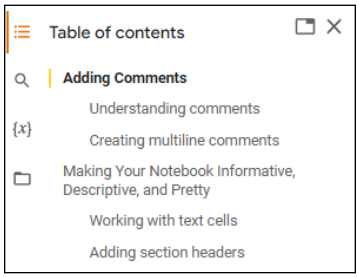


FIGURE 4-15: The Table of Contents feature provides an outline of your notebook.

To use the Table of Contents, simply click the heading that contains the material you want to see. The currently selected heading appears in bold so that you know where you're currently at in the notebook.

REMEMBER

Figure 4-15 points out the need to use separate text cells for each heading level in your notebook, rather than combine all the headers for a particular area into one text cell (even though it would look acceptable that way in the document). If you don't place each heading level in its own text cell, the feature won't work as intended and you'll get really frustrated trying to make it behave itself.

Renaming a notebook

The notebooks you create should have descriptive names. Otherwise, they become too difficult to find after you've accumulated enough of them. It's sort of like having a closet at home that's stocked floor to ceiling with unmarked boxes (or worse yet, boxes that have the wrong content marked on them). The only way to find something is to take the boxes out one at a time and open them to see what's inside. To rename a notebook, just click the filename at the top of the notebook such as `BPP4D3E; 04; Comments.ipynb` and type a new name for it.

Closing and Halting a Notepad

There is no need (or option) to close and halt notebooks when working with Colab. However, when working with Jupyter Notebook, you have a local server (called a *kernel*), and you need to interact with that server to ensure that all the changes you make to a notebook are recorded. With this idea in mind, the following list contains the menu options you use to interact with the server to ensure that your changes are saved and that the server itself is halted when you want to end a session.

- **File ➪ Save and Checkpoint:** Saves any changes you made to disk and creates a checkpoint so that you can restore a previous version of your file. However, Jupyter Notebook saves only one previous version, so your undo options are somewhat limited.
- **File ➪ Revert to Checkpoint:** Presents a list of checkpoint files for this notebook. There is usually just one such file, and selecting it will restore the file from the previous checkpoint.
- **File ➪ Close and Halt:** Closes the file and halts the server for this notebook. The overall server functionality is unaffected. To completely shut the server down, you must click Quit in the main Jupyter Notebook screen (the one where you select which file to open).
- **Kernel ➪ Reconnect:** Allows recovery after a software glitch if the server breaks the connection with the current notebook. You can tell that the connection is broken because you see a No Kernel message at the right side of the menu bar.
- **Kernel ➪ Shutdown:** Shuts the kernel down without closing the file. You see a No Kernel message at the right side of the menu bar when the shutdown is complete.
- **Kernel ➪ Change Kernel:** Selects an alternative kernel to use when running the current notebook. You might install multiple versions of Python, for example, and need to test the code with each one. (The "Seeing the result" section of the chapter talks about various restart options you can use to ensure you have a clean environment to use between tests.)

IN THIS CHAPTER

- Defining what is meant by magic
- Performing magic of your own
- Discovering the magic words

Chapter 5
Performing Magic

Most everyone likes magic. You see it in films, books, magazines, online, video games, and a seemingly endless assortment of other sources. Watching a magician perform tricks is nothing short of amazing. Unlike other programming languages, Python comes with its own built-in set of *magic* commands. A magic command is a special kind of function that does something to the programming environment, rather than work as code. These magical words can make you look just like a magician on a stage; others will wonder how you created a certain effect, and the secret of your magic commands will remain yours alone. The first part of this chapter discusses the ins and outs of magic commands in a little more detail.

Of course, you'll want to start performing magic commands immediately, so the second part of the chapter gets you started with a few beginner tricks designed to tantalize without being too hard to use. Some magic commands can become involved, just like those on the stage (but your code won't ever disappear and no one will get locked up in a tank of water).

The third part of this chapter provides an overview of the magic commands keywords that you need to know. Each entry comes with a short description so that you know which magic commands will fulfill needs you have with your notebooks. Spending time working with the magic commands is the only way to learn them well.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 05; Magics.ipynb` file of the downloadable source. See the Introduction for details on how to find these source files.

Understanding the Concept of a Magic Command

A magic command really isn't magic. The magic is in the output. For example, instead of displaying graphic output in a separate window, you can choose to display it within the cell, as if by magic (because the cells appear to hold only text). Or you can use magic to check the performance of your application, and do so without all the usual added code that such performance checks require.

A *magic command* begins with either a percent sign (%) or double percent sign (%%). Those with a % sign work within the environment, and those with a %% sign work at the cell level. For example, if you want to obtain a list of magic commands, type **%lsmagic** and then run the command in Colab or Notebook to see them, as shown in Figure 5-1. (You may see the output all on one line, in which case you can apply the fix described in the "Adding magic to the magic commands" sidebar.)

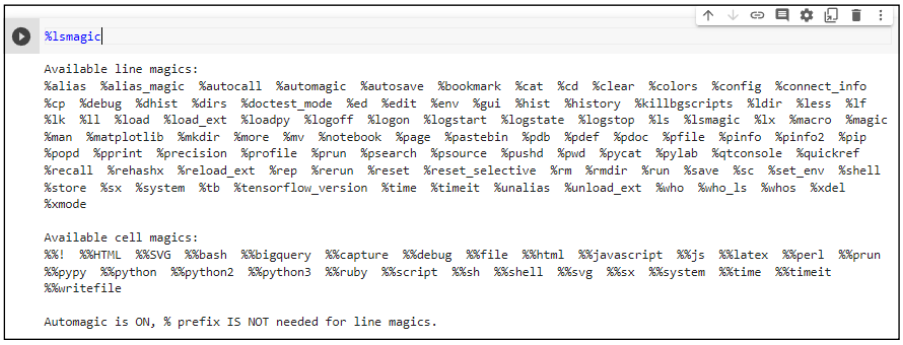


FIGURE 5-1: The `%lsmagic` command displays a list of magic commands for you.

ADDING MAGIC TO THE MAGIC COMMANDS

Depending on the IDE and browser you use (Colab tends to have this problem; Notebook doesn't), you may notice that certain commands display a very long line of text without any text wrap, which means you must scroll back and forth to see the entire line. Of course, this can prove frustrating, especially if you're already having a bad day. Rather than throw your shoe at the monitor, however, you can add the following piece of code to the beginning of your notebook and run it in a separate cell from your other code:

```
from IPython.display import HTML, display

def set_css():
  display(HTML('''
  <style>
    pre {
        white-space: pre-wrap;
    }
  </style>
  '''))
get_ipython().events.register('pre_run_cell', set_css)
```

TECHNICAL STUFF

It's not essential to understand this code to make use of it, although understanding it would be nice. This code tells the IDE to provide text wrapping using a specific Cascading Style Sheets (CSS) style, `pre {white-space: pre-wrap;}`, as documented at `https://developer.mozilla.org/en-US/docs/Web/CSS/white-space`. You access this style by using the `IPython.display.HTML` class documented at `https://ipython.readthedocs.io/en/stable/api/generated/IPython.display.html`. You can see other `IPython.display.HTML` examples at `https://www.programcreek.com/python/example/83957/IPython.display.HTML`.

The call to `get_ipython()` obtains a copy of the interactive shell object that you then use to register a new event handler named `pre_run_cell` (see `https://ipython.readthedocs.io/en/stable/api/generated/IPython.core.getipython.html` and `https://ipython.readthedocs.io/en/stable/config/callbacks.html` for details). Every time you run a cell, the event handler adds the bit of CSS code defined by the `set_css()` function to ensure that text lines will wrap for you. You can see other `get_ipython()` call examples at `https://www.programcreek.com/python/example/50972/IPython.get_ipython`. The point is that the code fixes the text wrapping problem for you.

Each of these magic commands is part of the `IPython` module that is automatically loaded when you load Python, no matter where or how you load it. So when you work at the command line (something you won't do in this book), you can still use many of the magic commands. Of course, you have to have Python installed on your machine to test it by typing **ipython** and pressing Enter at a command prompt or in a terminal window. (Note that this is the interactive Python (`ipython`) command, not the standard Python (`python`) command.) Some magic commands don't have a use at the command line, such as `%autosave`, because the environment is different. The point is that Python provides magic commands no matter where you work with it.

What Kind of Magic Do You Want to Perform?

Magic functions are all about the programming environment rather than the application code. When you issue a magic command, what you're doing is accessing the underlying interactive shell in a manner that doesn't require you to write code similar to that found in the "Adding Magic to the Magic Commands" sidebar in the chapter. All this code is already encapsulated for you in the form of the magic functions. There are two kinds of magic commands: line and cell, as described in the sections that follow.

Working with line magic commands

Line magic commands affect a single line in your code. When you start reviewing them in the "An overview of line magic commands" section of the chapter, you see that this limitation makes sense. For example, if you perform a directory listing using the `%ls` function, you don't want to have it keep affecting every other cell in the application. All you really want to see are the items (files and other directories) in the current directory.

Any line magic functions that affect settings do continue to modify the environment. For example, the settings you use with the `%matplotlib` function remain in effect until you change them again. However, like any other line magic commands, the `%matplotlib` affects only the current line where the setting is made. That the setting remains permanent is a matter of application state, rather than having the `%matplotlib` function remain active.

REMEMBER

It's important to note that the magic functions work directly with the interactive shell and aren't actually part of your application. For example, you can't write `myDir = %ls` and expect that `myDir` will contain a directory listing after you execute the command. What will happen instead is that you see a directory listing and `myDir` remains empty (a call to `print(myDir)` will produce an output of `None`).

Working with cell magic commands

Cell magic commands affect an entire cell in your code and some affect all cells after the cell in which the magic command appears. For example, the `%%debug` function affects only the current cell, but it does so in a manner that affects the application as a whole because it activates the built-in debugger. You can use this debugger to locate errors in your code much faster and with better accuracy than using the `print()` function, which is how most developers debug their Python code. The article at `https://towardsdatascience.com/4-little-known-magic-commands-that-boost-your-productivity-af1ba4268c57` tells you more about the built-in debugger (a feature that few people even know about).

Depending on how you use `%%prun` and `%%time` functions, they can affect the application as a whole because you're using them to monitor performance. In fact, `%%prun` actually shows all the external calls involved in running your code (the listing can become quite long). So it's important to view cell magic commands as running in the current cell, but possibly affecting other cells as well.

Learning the Magic Commands

Like any good magician, you need to learn the magic words before you can perform magic commands. The following sections provide you with the information needed to make creating magic in your notebooks a reality.

Getting magic command details

As previously mentioned, you can use the `%lsmagic` command to obtain a simple listing of available magic commands. Tables 5-1 and 5-2 list the most common magic commands and their purpose. To obtain a full detail listing, type **%quickref** and click Run Cell in Colab or Notebook to display output like that shown in Figure 5-2. You can also find a full listing of the magic commands at `https://damontallen.github.io/IPython-quick-ref-sheets/`.

WATCHING FOR ENVIRONMENTAL DIFFERENCES

Most of the magic commands you read about in this chapter will work in most environments. However, some magic commands don't work with Colab, and others don't work with Jupyter Notebook. Unfortunately, the support you see for magic commands changes all the time, so your best option is to try a magic command in the environment in which you want to use it and see if it works. For example, `%cp --help` works fine in Colab, but doesn't work in Jupyter Notebook (at least, not now). You may also get different results from some commands based on environment because the environment helps to determine the information you receive. A directory listing for Colab will definitely differ from a directory listing in Jupyter Notebook. You also see differences based on platform — output from a Windows platform will differ from the output on a Linux platform.

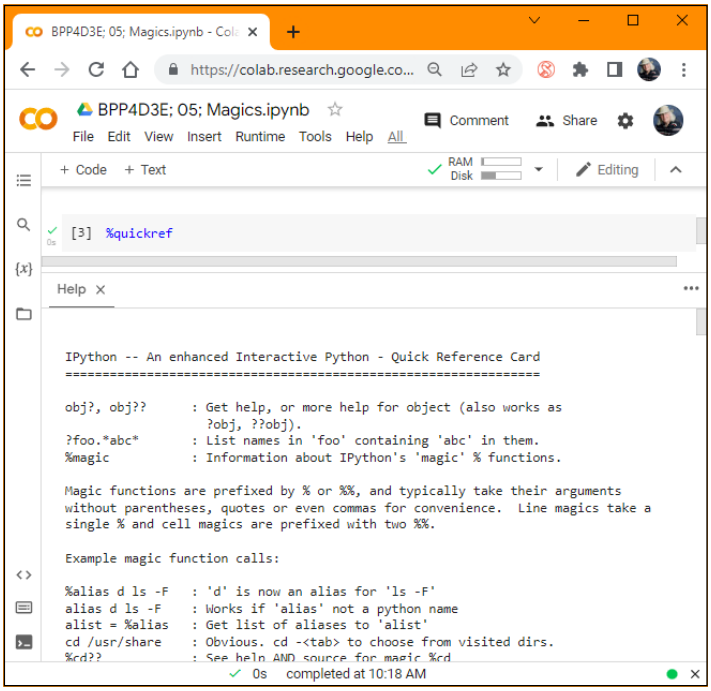


FIGURE 5-2: The `%quickref` function displays more detailed magic command information for you.

TIP

In some cases, `%quickref` shows `::` for an output, rather than a description, to indicate that this particular magic command doesn't have a specific output. For example, when working with `%debug` or `%%debug`, the output depends on the code that you debug. In other cases, `%quickref` does provide help text, but it doesn't really tell you how to work with the magic command, such as with `%%prun`, where

you must supply the code to run as part of the cell, as shown in Figure 5-3. In this case, the output from the code appears as part of the cell as usual, but the performance information appears in a help screen.

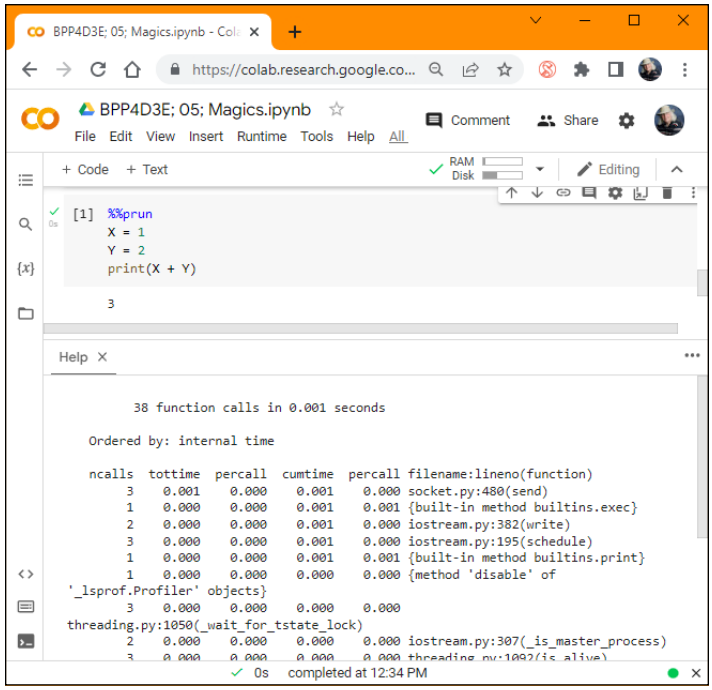


FIGURE 5-3: The `%quickref` function may not tell you everything you need to know.

It's possible to obtain even more detailed information about a particular magic command by adding `--help` to the function. For example, `%cp --help` tells you about the arguments used with the `%cp` function. However, this feature doesn't work with all magic commands.

An overview of line magic commands

Table 5-1 shows a listing of common line magic commands and their purpose. The table also tells you when you can use the line magic command by itself. For example, when you use the `%alias` magic command alone, you obtain a listing of the currently defined aliases for system commands as a list of tuples, such as `('ldir', 'ls -F -o --color %l | grep /$')`, where `ldir` is the alias for `ls -F -o --color %l | grep /$`. Typing `ldir` is much shorter, of course, but it accomplishes so much. Note that Table 5-1 doesn't include a few of the most advanced magic commands, such as `%gui`, or those that seem incompatible or somewhat useless, such a `%dirs`. In addition, you won't find deprecated (that is, no longer supported) magic commands such as `%profile` and `%sc`.

TABLE 5-1 Common IPython Line Magic Commands

Magic Command	Type Alone Provides Status?	Description
`%alias`	Yes	Assigns or displays an alias for a system command.
`%alias_magic`	No	Assigns an alias for a magic command. You must supply the new name you want to use and the name of the magic command that it represents. It's also possible to specify –c for cell magic commands and –l for line magic commands.
`%autocall`	Yes	Enables you to call functions without including the parentheses. The settings are Off, Smart (default), and Full. The Smart setting applies the parentheses only if you include an argument with the call.
`%automagic`	Yes	Enables you to call the line magic commands without including the percent (%) sign. The settings are False (default) and True.
`%autosave`	Yes	Displays or modifies the intervals between automatic Notebook saves. The default setting is every 120 seconds.
`%bookmark`	No	Makes it possible to create a bookmark in your code so that you don't have to remember where you're at between sessions. You must supply the bookmark name. Use the `%cd –b` function to go to a bookmark in your code.
`%cd`	Yes	Changes directory to a new storage location. You can also use this command to move through the directory history or to change directories to a bookmark. When used by itself, you see the current directory on disk.
`%clear`	No	Clears the terminal window (may not be available in all environments).
`%cls`	No	Clears the screen (may not be available in all environments).
`%colors`	No	Specifies the colors used to display text associated with prompts, the information system, and exception handlers. You can choose between NoColor (black and white), Linux (default), and LightBG.
`%config`	Yes	Enables you to configure IPython.
`%connect_info`	Yes	Displays the connection information for the notebook's interactive environment.

Magic Command	Type Alone Provides Status?	Description
%conda	No	Runs the specified conda command, assuming that you have conda installed on your system (may not be available in all environments).
%cp	No	Copies a file from a source to a destination (may not be available in all environments).
%debug and %%debug.	No	Places the interactive environment into debug mode.
%dhist	Yes	Displays a list of directories visited during the current session.
%dirs, %popd, %pushd, %pwd	Yes	Pushes the current directory on the directory stack (%pushd), pops a saved directory from the directory stack (%popd), shows the list of directories on the directory stack (%dirs), and shows the current working directory (%pwd).
%doctest_mode	No	Toggles the interactive environment between doctest mode and standard environment behavior. In doctest mode, the environment behaves more like the plain Python environment in reporting errors and outputting information, which makes it easier to copy and paste the information into doctests (see https://docs.python.org/3/library/doctest.html and https://www.digitalocean.com/community/tutorials/how-to-write-doctests-in-python for additional details).
%env and %set_env	Yes	Displays (%env) and sets (%set_env) the environment variables for the current machine. When the name of an environment variable is provided, the output shows just the information for that environment variable.
%file	No	Outputs the name of the file that contains the source code for the object. (May not be available in all environments.)
%hist and %history	Yes	Displays a list of magic commands issued during the current session.
%install_ext	No	Installs the specified extension. (May not be available in all environments.)
%ldir	Yes	Provides a directory listing of the current directory (no files). It's also possible to add the path to a directory that you want to search.

(continued)

TABLE 5-1 *(continued)*

Magic Command	Type Alone Provides Status?	Description
`%load` and `loadpy`	No	Loads application code from another source, such as an online example.
`%load_ext`	No	Loads a Python extension using its module name.
`%logoff`, `%logon`, `%logstart`, `%log–state`, and `%logstop`	Yes	Starts, stops, queries, and pauses Python logging of application activity. The `%logstart` and `%logstop` functions start and stop the log. Use `%logoff` and `%logon` to pause and continue the log. The `%logstate` function shows the current logging status.
`%lsmagic`	Yes	Displays a list of the currently available magic commands.
`%magic`	Yes	Displays a help screen showing information about the magic commands.
`%matplotlib`	Yes	Sets the back-end processor used for plots. Using the inline value displays the plot within the cell for an IPython Notebook file. The possible values are: `'gtk'`, `'gtk3'`, `'inline'`, `'nbagg'`, `'osx'`, `'qt'`, `'qt4'`, `'qt5'`, `'tk'`, and `'wx'`.
`%mkdir`	No	Creates a new directory in the location specified.
`%notebook`	No	Exports the current environment history to the specified notebook file that you can import and review.
`%paste`	No	Pastes the content of the Clipboard into the IPython environment (may not be available in all environments).
`%pdef`	No	Shows how to call the object (assuming that the object is callable).
`%pdoc`	No	Displays the `docstring` for an object.
`%pinfo`	No	Displays detailed information about the object (often more than provided by help alone).
`%pinfo2`	No	Displays extra detailed information about the object (when available).
`%pip`	No	Runs the specified pip command assuming that you have pip (package installer for Python) installed on your system (may not be available in all environments).
`%pprint`	No	Toggles pretty printing (how output appears) of content on and off.

Magic Command	Type Alone Provides Status?	Description
`%precision`	No	Sets the precision for pretty printing of floating-point values.
`%prun`	Yes	Profiles the specified function that appears on the same line as the magic command, rather than the code found in a cell as the `%%prun` function does. When run by itself, you see the calls used to run `%prun`, making it easier to separate the `%prun` calls from those made by your code.
`%psearch`	No	Performs a pattern search for desired objects in the environment. For example, using `%psearch d*` would search for all objects beginning with the letter `d`.
`%pylab`	Yes	Provides additional support for plotting using MatPlotLib as described at `https://www.tutorialspoint.com/matplotlib/matplotlib_pylab_module.htm`.
`%quickref`	Yes	Displays a reference sheet for all the available magic commands on a particular system (see the "Getting magic command details" section of the chapter for details).
`%reload_ext`	No	Reloads a previously installed extension.
`%reset` and `%reset_selective`	No	Resets the namespace, removing all the variables when used alone. It's also possible to specify which variables to reset. Clearing the variables allows execution of code in a clean environment but means that you must recreate any lost variables.
`%rm` and `%rmdir`	No	Removes a directory from the specified location. Note that the directory must be empty before you can remove it.
`%run`	No	Loads the specified Python module and runs it.
`%source`	No	Displays the source code for the object, assuming that the source is available (may not be available in all environments).
`%store`	No	Provides the means to store variables long term in Python. The article at `https://www.blopig.com/blog/2020/05/storing-variables-in-jupyter-notebooks-using-store-magic/` provides details on how to use this magic command effectively.
`%time` and `%timeit`	No	Calculates the best performance time for an instruction.
`%unalias`	No	Removes a previously created alias from the list.

(continued)

TABLE 5-1 *(continued)*

Magic Command	Type Alone Provides Status?	Description
`%unload_ext`	No	Unloads the specified extension.
%who, %who_ls, and %whos	Yes	Provides a listing of currently defined variables. The `%who` and `%who_ls` functions provide overviews. The `%whos` function provides the most detailed output.
`%xdel`	No	Deletes a variable, even when the variable has several aliases. The variable is also removed from the output history.
`%xmode`	No	Toggles between exception handling verbosity levels: Minimal, Plain, Context, and Verbose. By setting the verbosity level correctly, you can usually reduce the time needed to find errors in your application.

An overview of cell magic commands

Cell magic commands tend to provide greater functionality in many ways than line magic commands do. You can perform tasks that might seem impossible, but they actually do work (making them quite magical indeed). For example, you may want to create an HTML presentation as part of your notebook, which would seem impossible because notebooks don't execute HTML very well, or do they? Using the following code lets you execute HTML within your notebook to get just the right presentation:

```
%%HTML

<p>This is a list.</p>
<ul>
  <li>Morning</li>
  <li>Noon</li>
  <li>Night</li>
</ul>
```

When you execute the cell, you get precisely the output you intend. Table 5-2 tells you more about the magical nature of cell magic commands. Unlike line magic commands, cell magic commands don't provide status information.

TABLE 5-2 Common IPython Cell Magic Commands

Magic Comand	Description
%%!, %%bash, %sx, %%sx, %system, and %%system	Performs a list of system-specific commands. The commands must match the underlying operating system. When working with Colab, this means using Linux commands, such as ls. When working with Notebook, it may mean using Windows commands, such as dir. Even though ls and dir are equivalent, you can't mix and match them, which makes this particular kind of magic command brittle (prone to breaking). The %%bash function is available only on Linux systems and provides the additional functionality detailed at https://tldp.org/LDP/Bash-Beginners-Guide/html/Bash-Beginners-Guide.html for bash users.
%%capture	Records the stdout, stderr, and IPython's rich display() calls for later use. The resulting object acts as a macro that you can play back as needed. The article at https://notebook.community/lifeinoppo/littlefishlet-scode/RES/REF/python_sourcecode/ipython-master/examples/IPython%20Kernel/Capturing%20Output provides an example of how to use this feature.
%%file and %%writefile	Writes the contents of a cell to the specified file.
%%HTML and %%html	Runs the specified HTML script immediately following the magic command in the same cell.
%%javascript and %%js	Runs the specified JavaScript script immediately following the magic command in the same cell.
%%latex	Renders the cell as a block of LaTeX (https://www.latex-project.org/). The level of LaTeX support depends on the environment you're using. For example, Jupyter Notebook supports only the level described at https://docs.mathjax.org/en/v2.5-latest/tex.html. The tutorial at https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes tells you more about how to work with LaTeX. You can also use LaTeX directly in markup cells as described in the "Working with text cells" section of Chapter 4.
%%prun	Performance tests the code found in the remainder of the cell and provides detailed statistics about it. This magic command is different from the %prun function, so don't confuse the two.
%%SVG and %%svg	Renders the cell as a Scalable Vector Graphics (SVG) literal, so you can create images as part of your notebook. The tutorial at https://www.w3schools.com/graphics/svg_intro.asp tells you more about working with SVG.
%%time and %%timeit	Calculates the best performance time for all the instructions in a cell, apart from the one placed on the same cell line as the cell magic (which could therefore be an initialization instruction).

2 Talking the Talk

IN THIS PART . . .

Working with various kinds of data

Changing how Python views data

Choosing paths in applications

Performing tasks more than once

Dealing with exceptional situations

IN THIS CHAPTER

- Understanding data storage
- Considering the kinds of data storage
- Adding dates and times to applications

Chapter 6
Storing and Modifying Information

Chapter 3 introduces you to CRUD, that is, Create, Read, Update, and Delete — not that Chapter 3 contains cruddy material. This acronym provides an easy method to remember precisely what tasks all computer programs perform with information you want to manage. Of course, geeks use a special term for information — data, but either information or data works fine for this book.

To make information useful, you have to have some means of storing it permanently. Otherwise, every time you turned the computer off, all your information would be gone and the computer would provide limited value. In addition, Python must provide some rules for modifying information. The alternative is to have applications running amok, changing information in any and every conceivable manner. This chapter is about controlling information — defining how information is stored permanently and manipulated by applications you create.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 06; Storing And Modifying Information.ipynb` file of the downloadable source. See the Introduction for details on how to find these source files.

Storing Information

An application requires fast access to information or else it will take a long time to complete tasks. As a result, applications store information in memory. However, memory is temporary. When you turn off the machine, the information must be stored in some permanent form, such as on your hard drive, a Universal Serial Bus (USB) flash drive, a Secure Digital (SD) card, or on the Internet using a cloud-based solution. In addition, you must also consider the form of the information, such as whether it's a number or text. The following sections discuss the issue of storing information as part of an application in more detail.

Seeing variables as storage boxes

When working with applications, you store information in variables. A *variable* is a kind of storage box. Whenever you want to work with the information, you access it using the variable. If you have new information you want to store, you put it in a variable. Changing information means accessing the variable first and then storing the new value in the variable. Just as you store things in boxes in the real world, so you store things in variables when working with applications.

REMEMBER

Computers are actually pretty tidy. Each variable stores just one piece of information. Using this technique makes it easy to find the particular piece of information you need — unlike in your closet, where things from ancient Egypt could be hidden. Even though the examples you work with in previous chapters don't use variables, most applications rely heavily on variables to make working with information easier.

Using the right box to store the data

People tend to store things in the wrong sort of box. For example, you might find a pair of shoes in a garment bag and a supply of pens in a shoebox. However, Python likes to be neat. As a result, you find numbers stored in one sort of variable and text stored in an entirely different kind of variable. Yes, you use variables in both cases, but the variable is designed to store a particular kind of information. Using specialized variables makes it possible to work with the information inside in particular ways. You don't need to worry about the details just yet — just keep in mind that each kind of information is stored in a special kind of variable.

TECHNICAL STUFF

Python uses specialized variables to store information to make things easy for the programmer and to ensure that the information remains safe. However, computers don't actually know about information types. All that the computer knows about are 0s and 1s, which is the absence or presence of a voltage. At a higher level, computers do work with numbers, but that's the extent of what computers do.

Numbers, letters, dates, times, and any other kind of information you can think about all come down to 0s and 1s in the computer system. For example, the letter *A* is actually stored as 01000001 or the number 65. The computer has no concept of the letter *A* or of a date such as 8/31/2022.

Defining the Essential Python Data Types

Every programming language defines variables that hold specific kinds of information, and Python is no exception. The specific kind of variable is called a *data type.* Knowing the data type of a variable is important because it tells you what kind of information you find inside. In addition, when you want to store information in a variable, you need a variable of the correct data type to do it. Python doesn't allow you to store text in a variable designed to hold numeric information. Doing so would damage the text and cause problems with the application. You can generally classify Python data types as numeric, string, and Boolean, although there really isn't any limit on just how you can view them. The following sections describe each of the standard Python data types within these classifications.

Putting information into variables

To place a value into any variable, you make an assignment using the assignment operator (=). Chapter 7 discusses the whole range of basic Python operators in more detail, but you need to know how to use this particular operator to some extent now. For example, to place the number 5 into a variable named `myVar`, you type **myVar = 5**. Even though Python doesn't provide any additional information to you, you can always type the variable name and click Run Cell to see the value it contains, or use a `print()` function as shown in Figure 6-1.

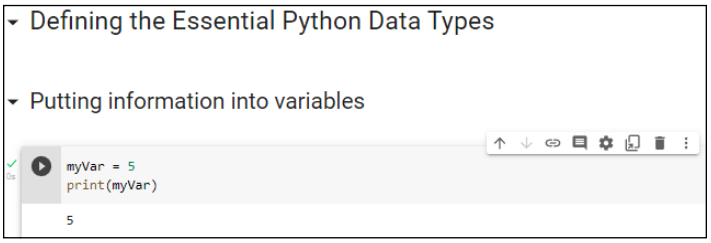


FIGURE 6-1: Use the assignment operator to place information into a variable.

Understanding the numeric types

Humans tend to think about numbers in general terms. We view 1 and 1.0 as being the same number — one of them simply has a decimal point. However, as far as

we're concerned, the two numbers are equal and we could easily use them interchangeably. Python views them as being different kinds of numbers because each form requires a different kind of processing. The following sections describe the integer, floating-point, and complex number classes of data types that Python supports.

Integers

Any whole number is an *integer.* For example, the value 1 is a whole number, so it's an integer. On the other hand, 1.0 isn't a whole number; it has a decimal part to it, so it's not an integer. Integers are represented by the `int` data type.

TECHNICAL STUFF

Unlike many programming languages, Python doesn't have an `int` capacity limit — you can make an `int` any size you need it. In case you want to understand how this unlimited size feature works, the site at `https://jakevdp.github.io/PythonDataScienceHandbook/02.01-understanding-data-types.html` provides additional details, but you really don't need to know how things work under the hood to work with Python. This unlimited size feature can cause problems with working with third-party libraries, such as NumPy (`https://numpy.org/doc/stable/user/basics.types.html`), where there are different integer data types with specific sizes, so you need to be aware of this issue when writing an application using third-party libraries.

When working with the `int` type, you have access to a number of interesting features. Many of them appear later in the book, but one feature is the ability to use different numeric bases:

- **Base 2:** Uses only 0 and 1 as numbers.
- **Base 8:** Uses the numbers 0 through 7.
- **Base 10:** Uses the usual numeric system.
- **Base 16:** Is also called *hex* (or *hexadecimal*) and uses the numbers 0 through 9 and the letters A through F to create 16 different possible values.

To tell Python when to use bases other than base 10, you add a 0 and a special letter to the number. For example, `0b100` is the value one-zero-zero in base 2. Here are the letters you normally use:

- **b:** Base 2
- **o:** Base 8
- **x:** Base 16

You can also convert numeric values to other bases by using the `bin()`, `oct()`, and `hex()` functions. So, putting everything together, you can see how to convert between bases using the functions shown in Figure 6-2. Try the function shown in the figure yourself so that you can see how the various bases work. Using a different base actually makes things easier in many situations, and you'll encounter some of those situations later in the book. For now, all you really need to know is that integers support different numeric bases.

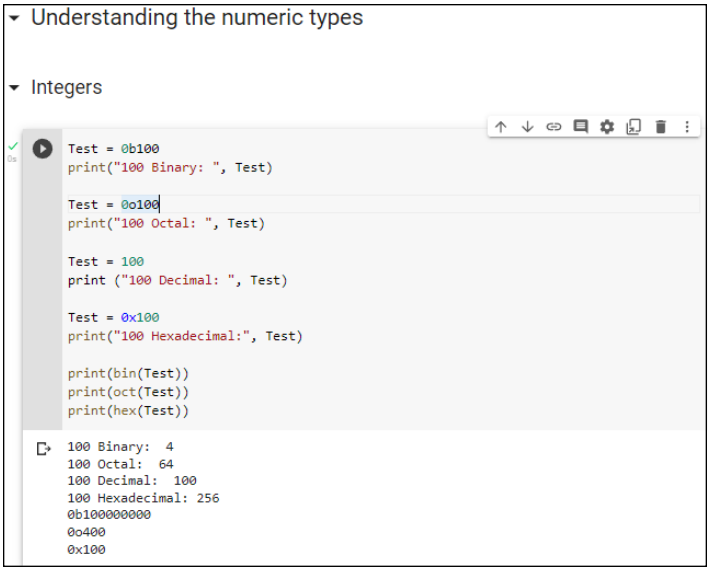


FIGURE 6-2: Integers have many interesting features, including the capability to use different numeric bases.

Floating-point values

Any number that includes a decimal portion is a floating-point value. For example, 1.0 has a decimal part, so it's a floating-point value. Many people get confused about whole numbers and floating-point numbers, but the difference is easy to remember. If you see a decimal in the number, then it's a floating-point value. Python stores floating-point values in the `float` data type.

REMEMBER

Floating-point values have an advantage over integer values in that you can store values with a fractional portion in them. Unlike integers, floating-point variables do have a storage capacity because of the way in which they're stored using IEEE 754 format (see `https://www.geeksforgeeks.org/ieee-standard-754-floating-point-numbers/` for details). In their case, the maximum value that a variable can contain is $\pm 1.7976931348623157 \times 10^{308}$ and the minimum value that a variable can contain is $\pm 2.2250738585072014 \times 10^{-308}$ on most platforms.

UNDERSTANDING THE NEED FOR MULTIPLE NUMBER TYPES

A lot of new developers (and even some older ones) have a hard time understanding why there is a need for more than one numeric type. After all, humans can use just one kind of number. To understand the need for multiple number types, you have to understand a little about how a computer works with numbers.

An integer is stored in the computer as simply a series of bits that the computer reads directly. A value of 0100 in binary equates to a value of 4 in decimal. On the other hand, numbers that have decimal points are stored in an entirely different manner. Think back to all those classes you slept through on exponents in school — they actually come in handy sometimes. A floating-point number is stored as a sign bit (plus or minus), *mantissa* (the fractional part of the number), and *exponent* (the power of 2). (Some texts use the term *significand* in place of mantissa — the terms are interchangeable.) To obtain the floating-point value, you use the equation:

```
Value = Sign Bit, Mantissa * 2^Exponent
```

At one time, computers all used different floating-point representations, but they all use the IEEE-754 standard (`https://standards.ieee.org/ieee/754/6210/`) now. You can read about this standard at `https://www.geeksforgeeks.org/ieee-standard-754-floating-point-numbers/`. A full explanation of precisely how floating-point numbers work is outside the scope of this book, but you can read a fairly understandable description at `https://www.cprogramming.com/tutorial/floating_point/understanding_floating_point_representation.html`. Nothing helps you understand a concept like playing with the values. You can find a really interesting floating-point number converter at `https://www.h-schmidt.net/FloatConverter/IEEE754.html`, where you can click the individual bits (to turn them off or on) and see the floating-point number that results.

As you might imagine, floating-point numbers tend to consume more space in memory because of their complexity. In addition, they use an entirely different area of the processor — one that works more slowly than the part used for integer math. Finally, integers are precise, as contrasted to floating-point numbers, which can't precisely represent some numbers, so you get an approximation instead. The bottom line is that decimals are unavoidable in the real world, so you need floating-point numbers. Computer systems have many trade-offs, and this one is unavoidable.

When working with floating-point values, you can assign the information to the variable in a number of ways. The two most common methods are to provide the number directly and to use scientific notation. When using scientific notation, an *e* separates the number from its exponent. Figure 6-3 shows both methods of making an assignment. Notice that using a negative exponent results in a fractional value.

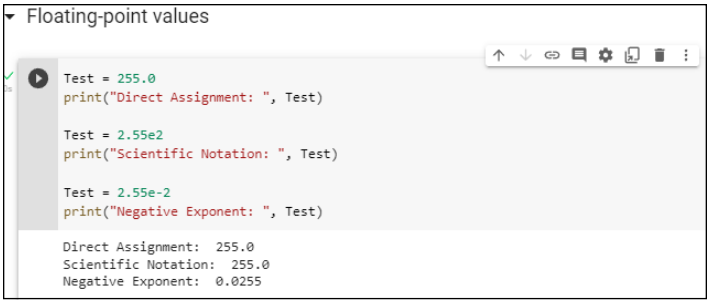


FIGURE 6-3: Floating-point values provide multiple assignment techniques.

Complex numbers

You may or may not remember complex numbers from school. A *complex number* consists of a real number and an imaginary number that are paired together. Just in case you've completely forgotten about complex numbers, you can read about them at `https://www.mathsisfun.com/numbers/complex-numbers.html`. Real-world uses for complex numbers include:

- Electrical engineering
- Fluid dynamics
- Quantum mechanics
- Computer graphics
- Dynamic systems

Complex numbers have other uses, too, but this list should give you some ideas. In general, if you aren't involved in any of these disciplines, you probably won't ever encounter complex numbers. However, Python is one of the few languages that provides a built-in data type to support them. As you progress through the book, you find other ways in which Python lends itself especially well to science and engineering.

The imaginary part of a complex number always appears with a *j* after it in Python. So, if you want to create a complex number with 3 as the real part and 4 as the imaginary part, you make an assignment like this:

```
myComplex = 3 + 4j
```

If you want to see the real part of the variable, you simply type **print(myComplex.real)**. Likewise, if you want to see the imaginary part of the variable, you type **print(myComplex.imag)**.

Understanding Boolean values

It may seem amazing, but computers always give you a straight answer! A computer will never provide "maybe" as output. Every answer you get is either `True` or `False`. In fact, there is an entire branch of mathematics called Boolean algebra (`https://www.tutorialspoint.com/computer_logical_organization/boolean_algebra.htm`) that was originally defined by George Boole (`https://plato.stanford.edu/entries/boole/`), a super-geek of his time, that computers rely upon to make decisions. Contrary to common belief, Boolean algebra has existed since 1854 — long before the time of computers.

When using Boolean values in Python, you rely on the `bool` type. A variable of this type can contain only two values: `True` or `False`. You can assign a value by using the `True` or `False` keywords, or you can create an expression that defines a logical idea that equates to true or false. For example, you could say, `myBool = 1 > 2`, which would equate to `False` because `1` is most definitely not greater than `2`. You see the `bool` type used extensively in the book, so don't worry about understanding this concept right now.

DETERMINING A VARIABLE'S TYPE

Sometimes you might want to know the variable type. Perhaps the type isn't obvious from the code or you've received the information from a source whose code isn't accessible. Whenever you want to see the type of a variable, use the `type()` method. For example, if you start by placing a value of 5 in `myInt` by typing **myInt = 5** and clicking Run Cell, you can find the type of `myInt` by typing **type(myInt)** and pressing Enter. The output will be `<class 'int'>`, which means that `myInt` contains an `int` value.

Understanding strings

Of all the data types, strings are the most easily understood by humans and not understood at all by computers. If you have read the previous chapters in this book, you have already seen strings used quite often. For example, all the example code in Chapter 4 relies on strings. A *string* is simply any grouping of characters you place within double or single quotes. For example, `myString = "Python is a great language."` assigns a string of characters to `myString`.

The computer doesn't see letters at all. Every letter you use is represented by a number in memory. For example, the letter *A* is actually the number 65. To see this for yourself, type **print(ord("A"))** and click Run Cell. You see 65 as output. You can convert any single letter to its numeric equivalent by using the `ord()` function.

Because the computer doesn't really understand strings, but strings are so useful in writing applications, you sometimes need to convert a string to a number. You can use the `int()` and `float()` functions to perform this conversion. For example, if you type **myInt = int("123")** and click Run Cell, you create an `int` named `myInt` that contains the value `123`. Figure 6-4 shows how you can perform this task and validate the content and type of `myInt`.

FIGURE 6-4: Converting a string to a number is easy by using the `int()` and `float()` functions.

You can convert numbers to a string as well by using the `str()` function. For example, if you type **myStr = str(1234.56)** and click Run Cell, you create a string containing the value `"1234.56"` and assign it to `myStr`. Figure 6-5 shows this type of conversion and the test you can perform on it. The point is that you can go back and forth between strings and numbers with great ease. Later chapters demonstrate how these conversions make a lot of seemingly impossible tasks quite doable.

```
myStr = str(1234.56)
print(myStr)
print(type(myStr))

1234.56
<class 'str'>
```

FIGURE 6-5: You can convert numbers to strings as well.

Working with Dates and Times

Dates and times are items that most people work with quite a bit. Society bases almost everything on the date and time that a task needs to be or was completed. We make appointments and plan events for specific dates and times. Most of our day revolves around the clock. Because of the time-oriented nature of humans, it's a good idea to look at how Python deals with interacting with dates and time (especially storing these values for later use). As with everything else, computers understand only numbers — the date and time don't really exist.

REMEMBER

To work with dates and times, you need to perform a special task in Python. When writing computer books, chicken-and-egg scenarios always arise, and this is one of them. To use dates and times, you must issue a special `import datetime` function. Technically, this act is called *importing a package,* and you learn more about it in Chapter 11. Don't worry how the function works right now — just use it whenever you want to do something with date and time.

Computers do have clocks inside them, but the clocks are for the humans using the computer. Yes, some software also depends on the clock, but again, the emphasis is on human needs rather than anything the computer might require. To get the current time, you can simply type **print(datetime.datetime.now())** and click Run Cell. You see the full date and time information as found on your computer's clock (see Figure 6-6).

Working with Dates and Times

```
import datetime
print(datetime.datetime.now())

2022-04-05 21:05:02.198893
```

FIGURE 6-6: Get the current date and time by using the now( ) function.

You may have noticed that the date and time are a little hard to read in the existing format. Say that you want to get just the current date, in a readable format. It's time to combine a few things you discovered in previous sections to accomplish that task. Type **print(str(datetime.datetime.now().date()))** and click Run Cell. Figure 6-7 shows that you now have something a little more usable.

FIGURE 6-7: Make the date and time more readable by using the `str()` function.

Interestingly enough, Python also has a `time()` function (`https://docs.python.org/3/library/time.html`), which you can use to obtain the current time. You can obtain separate values for each of the components that make up date and time by using the `day`, `month`, `year`, `hour`, `minute`, `second`, and `microsecond` values. Later chapters help you understand how to use these various date and time features to keep application users informed about the current date and time on their system.

IN THIS CHAPTER

- Understanding the Python view of data
- Using operators to assign, modify, and compare data
- Organizing code using functions
- Interacting with the user

Chapter 7

Managing Information

Whether you use the term *information* or *data* to refer to the content that applications manage, the fact is that you must provide some means of working with it or your application really doesn't have a purpose. Throughout the rest of the book, you see *information* and *data* used interchangeably because they really are the same thing, and in real-world situations, you'll encounter them both, so getting used to both is a good idea. No matter which term you use, you need some means of assigning data to variables, modifying the content of those variables to achieve specific goals, and comparing the result you receive with desired results.

Also essential is to start working through methods of keeping your code understandable. Yes, you could write your application as a really long procedure, but trying to understand such a procedure is incredibly hard, and you'd find yourself repeating some steps because they must be done more than once. Functions are one way for you to package code so that you can more easily understand and reuse as needed.

Applications also need to interact with the user. Yes, some perfectly usable applications exist that don't really interact with the user, but they're rare. To provide a useful service, most applications interact with the user to discover how the user wants to manage data.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 07; Managing Information.ipynb` file of the downloadable source. See the Introduction for details on how to find these source files.

Controlling How Python Views Data

As discussed in Chapter 6, all data on your computer is stored as 0s and 1s. The computer doesn't understand the concept of letters, Boolean values, dates, times, or any other kind of information except numbers. In addition, a computer's capability to work with numbers is both inflexible and relatively simplistic. When you work with a string in Python, you depend on Python to translate the concept of a string into a form the computer can understand. The storage containers that your application creates and uses in the form of *variables* tell Python how to treat the 0s and 1s that the computer has stored. The Python view of data isn't the same as your view of data or the computer's view of data — Python acts as an intermediary to make your applications functional.

REMEMBER

To manage data within an application, the application must control the way in which Python views the data. The use of operators, packaging methods such as functions, and the introduction of user input all help applications control data. All these techniques rely, in part, on making comparisons. Determining what to do next means understanding what state the data is in now as compared to some other state.

Making comparisons

Python uses *operators* to make comparisons. In fact, operators play a major role in manipulating data as well. The upcoming "Working with Operators" section discusses how operators work and how you can use them in applications to control data in various ways. Later chapters use operators extensively as you discover techniques for creating applications that can make decisions, perform tasks repetitively, and interact with the user in interesting ways.

In some cases, you use some fancy methods to perform *comparisons* (the strict measuring of one value or one result against another) in an application. For example, you can compare the output of two functions (as described in the "Comparing function output" section, later in this chapter). With Python, you can perform comparisons at a number of levels so that you can manage data without a problem in your application. Using these techniques hides detail so that you can focus on the point of the comparison and define how to react to that comparison. Your choice of techniques to perform comparisons affects the manner in which Python views the data and determines the sorts of things you can do to manage the data after the comparison is made. Applications require comparisons to interact with data correctly.

Understanding how computers make comparisons

Computers don't understand packaging, such as functions, or any of the other structures that you create with Python. All this packaging is for your benefit, not the computer's. However, computers do directly support the concept of operators at the processor level. Most Python operators have a direct corollary with a command that the computer understands directly. For example, when you ask whether one number is greater than another number, the computer can actually perform this computation directly, using its hardware.

REMEMBER

Some comparisons aren't direct. Computers work only with numbers. So, when you ask Python to compare two strings, what Python actually does is compare the numeric value of each character in the string. For example, the letter *A* is actually the number 65 in the computer. A lowercase letter *a* has a different numeric value — 97. Consequently, the computer sees ABC as different from abc because the numeric values of their individual letters are different.

Working with Operators

Operators are the basis for both control and management of data within applications. You use operators to define how one piece of data is compared to another and to modify the information within a single variable. In fact, operators are essential to performing any sort of math-related task and to assigning data to variables in the first place.

REMEMBER

When using an operator, you must supply either a variable or an expression. You already know that a variable is a kind of storage box used to hold data. An *expression* is an equation or formula that provides a description of a mathematical concept. In most cases, the result of evaluating an expression is a Boolean (true or false) value. The following sections describe operators in detail because you use them everywhere throughout the rest of the book.

Defining the operators

An *operator* accepts one or more inputs in the form of variables or expressions, performs a task (such as comparison or addition), and then provides an output consistent with that task. Operators are classified partially by their effect and partially by the number of elements they require. For example, a unary operator works with a single variable or expression; a binary operator requires two; a ternary operator requires three. Python doesn't support anything above three because that would really be ridiculous (and possibly confusing).

REMEMBER

The elements provided as input to an operator are called *operands.* The operand on the left side of the operator is called the left operand, while the operand on the right side of the operator is called the right operand (ternary operators are special, as you find out in the "Understanding Python's one ternary operator" sidebar, so don't worry about it for now). The following list shows the categories of operators that you use within Python:

- Unary
- Arithmetic
- Relational
- Logical
- Bitwise
- Assignment
- Membership
- Identity

Each of these categories performs a specific task. For example, the arithmetic operators perform math-based tasks, while relational operators perform comparisons. The following sections describe the operators based on the category in which they appear.

UNDERSTANDING PYTHON'S ONE TERNARY OPERATOR

A ternary operator requires three elements. Python supports just one such operator, and you use it to determine the truth value of an expression. This ternary operator takes the following form (it apparently has no actual name, but you can call it the if. . .else operator if desired):

```
TrueValue if Expression else FalseValue
```

When the `Expression` is true, the operator outputs `TrueValue`. When the expression is false, it outputs `FalseValue`. As an example, if you type

```
"Hello" if True else "Goodbye"
```

the operator outputs a response of `'Hello'`. However, if you type

```
"Hello" if False else "Goodbye"
```

the operator outputs a response of `'Goodbye'`. This is a handy operator for times when you need to make a quick decision and don't want to write a lot of code to do it.

One of the advantages of using Python is that it normally has more than one way to do things. Python has an alternative form of this ternary operator — an even shorter shortcut. It takes the following form:

```
(FalseValue, TrueValue)[Expression]
```

As before, when `Expression` is true, the operator outputs `TrueValue`; otherwise, it outputs `FalseValue`. Notice that the `TrueValue` and `FalseValue` elements are reversed in this case. An example of this version is

```
("Hello", "Goodbye")[True]
```

In this case, the output of the operator is `'Goodbye'` because that's the value in the `TrueValue` position. Of the two forms, the first is a little clearer, while the second is shorter.

Unary

The unary operators shown in Table 7-1 require a single variable or expression as input. You often use these operators as part of a decision-making process.

TABLE 7-1 Python Unary Operators

Operator	Description	Example
~	Inverts the bits in a number so that all the 0 bits become 1 bits and vice versa.	~4 results in a value of –5
–	Negates the original value so that positive becomes negative and vice versa.	–(–4) results in 4 and –4 results in –4
+	Is provided purely for the sake of completeness. This operator returns the same value that you provide as input.	+4 results in a value of 4

Arithmetic

The complex math tasks that Python performs are often based on much simpler math tasks, such as addition, as shown in Table 7-2. You can always create your own libraries of math functions using these simple operators as well.

TABLE 7-2 Python Arithmetic Operators

Operator	Description	Example
+	Adds two values together	5 + 2 = 7
–	Subtracts the right operand from the left operand	5 – 2 = 3
*	Multiplies the right operand by the left operand	5 * 2 = 10
/	Divides the left operand by the right operand	5 / 2 = 2.5
%	Divides the left operand by the right operand and returns the remainder	5 % 2 = 1
**	Calculates the exponential value of the left operand raised to the power of the right operand	5 ** 2 = 25
//	Performs integer division, in which the left operand is divided by the right operand and only the whole number is returned (also called floor division)	5 // 2 = 2

Relational

The relational operators compare one value to another and tell you when the relationship you've provided is true as shown in Table 7-3. For example, 1 is less than 2, but 1 is never greater than 2.

Logical

The logical operators shown in Table 7-4 combine the true or false value of variables or expressions so that you can determine their resultant truth value. You use the logical operators to create Boolean expressions that help determine whether to perform tasks.

Bitwise

The bitwise operators shown in Table 7-5 interact with the individual bits in a number. For example, the number 6 is actually 0b0110 in binary. A bitwise operator would interact with each bit within the number in a specific way. When working with a logical bitwise operator, a value of 0 counts as false and a value of 1 counts as true.

TABLE 7-3 Python Relational Operators

Operator	Description	Example
==	Determines whether two values are equal. Notice that the relational operator uses two equals signs. A mistake many developers make is using just one equals sign, which results in one value being assigned to another.	1 == 2 is False
!=	Determines whether two values are not equal. Some older versions of Python allowed you to use the <> operator in place of the != operator. Using the <> operator results in an error in current versions of Python.	1 != 2 is True
>	Verifies that the left operand value is greater than the right operand value.	1 > 2 is False
<	Verifies that the left operand value is less than the right operand value.	1 < 2 is True
>=	Verifies that the left operand value is greater than or equal to the right operand value.	1 >= 2 is False
<=	Verifies that the left operand value is less than or equal to the right operand value.	1 <= 2 is True

TABLE 7-4 Python Logical Operators

Operator	Description	Example
and	Determines whether both operands are true.	True and True is True True and False is False False and True is False False and False is False
or	Determines when one of two operands is true.	True or True is True True or False is True False or True is True False or False is False
not	Negates the truth value of a single operand. A true value becomes false and a false value becomes true.	not True is False not False is True

TABLE 7-5 Python Bitwise Operators

Operator	Description	Example
& (And)	Determines whether both individual bits within two operands are true and sets the resulting bit to true when they are.	0b1100 & 0b0110 = 0b0100
| (Or)	Determines whether either of the individual bits within two operands is true and sets the resulting bit to true when one of them is.	0b1100 | 0b0110 = 0b1110
^ (Exclusive or)	Determines whether just one of the individual bits within two operands is true and sets the resulting bit to true when one is. When both bits are true or both bits are false, the result is false.	0b1100 ^ 0b0110 = 0b1010
~ (One's complement)	Calculates the one's complement value of a number.	~0b1100 = –0b1101 ~0b0110 = –0b0111
<< (Left shift)	Shifts the bits in the left operand left by the value of the right operand. All new bits are set to 0 and all bits that flow off the end are lost.	0b00110011 << 2 = 0b11001100
>> (Right shift)	Shifts the bits in the left operand right by the value of the right operand. All new bits are set to 0 and all bits that flow off the end are lost.	0b00110011 >> 2 = 0b00001100

Assignment

The assignment operators shown in Table 7-6 place data within a variable, `MyVar`, which has a starting value of `5`. The simple assignment operator appears in previous chapters of the book, but Python offers a number of other interesting assignment operators that you can use. These other (compound) assignment operators can perform mathematical tasks during the assignment process, which makes it possible to combine assignment with a math operation.

Membership

The membership operators shown in Table 7-7 detect the appearance of a value within a list or sequence and then output the truth value of that appearance. Think of the membership operators as you would a search routine for a database. You enter a value that you think should appear in the database, and the search routine finds it for you or reports that the value doesn't exist in the database.

Identity

The identity operators shown in Table 7-8 determine whether a value or expression is of a certain class or type. You use identity operators to ensure that you're actually working with the sort of information that you think you are.

TABLE 7-6 Python Assignment Operators

Operator	Description	Example
=	Assigns the value found in the right operand to the left operand.	MyVar = 5 results in MyVar containing 5
+=	Adds the value found in the right operand to the value found in the left operand and places the result in the left operand.	MyVar += 2 results in MyVar containing 7
−=	Subtracts the value found in the right operand from the value found in the left operand and places the result in the left operand.	MyVar −= 2 results in MyVar containing 3
*=	Multiplies the value found in the right operand by the value found in the left operand and places the result in the left operand.	MyVar *= 2 results in MyVar containing 10
/=	Divides the value found in the left operand by the value found in the right operand and places the result in the left operand.	MyVar /= 2 results in MyVar containing 2.5
%=	Divides the value found in the left operand by the value found in the right operand and places the remainder in the left operand.	MyVar %= 2 results in MyVar containing 1
**=	Determines the exponential value found in the left operand when raised to the power of the value found in the right operand and places the result in the left operand.	MyVar **= 2 results in MyVar containing 25
//=	Divides the value found in the left operand by the value found in the right operand and places the integer (whole number) result in the left operand.	MyVar //= 2 results in MyVar containing 2

TABLE 7-7 Python Membership Operators

Operator	Description	Example
in	Determines whether the value in the left operand appears in the sequence found in the right operand.	"Hello" in "Hello Goodbye" is True
not in	Determines whether the value in the left operand is missing from the sequence found in the right operand.	"Hello" not in "Hello Goodbye" is False

TABLE 7-8 Python Identity Operators

Operator	Description	Example
is	Evaluates to true when the type of the value or expression in the right operand points to the same type in the left operand.	type(2) is int is True
is not	Evaluates to true when the type of the value or expression in the right operand points to a different type than the value or expression in the left operand.	type(2) is not int is False

Understanding operator precedence

When you create simple statements that contain just one operator, the order of determining the output of that operator is also simple. However, when you start working with multiple operators, it becomes necessary to determine which operator to evaluate first. For example, you should know whether 1 + 2 * 3 evaluates to 7 (where the multiplication is done first) or 9 (where the addition is done first). An order of operator precedence tells you that the answer is 7 unless you use parentheses to override the default order. In this case, (1 + 2) * 3 would evaluate to 9 because the parentheses have a higher order of precedence than multiplication does. Table 7-9 defines the order of operator precedence for Python.

TABLE 7-9 Python Operator Precedence

Operator(s)	Description
`()`	You use parentheses to group expressions and to override the default precedence so that you can force an operation of lower precedence (such as addition) to take precedence over an operation of higher precedence (such as multiplication).
`**`	Exponentiation raises the value of the left operand to the power of the right operand.
`~ + -`	Unary operators interact with a single variable or expression.
`* / % //`	Multiply, divide, modulo, and floor division.
`+ -`	Addition and subtraction.
`>> <<`	Right and left bitwise shift.
`&`	Bitwise AND.
`^ |`	Bitwise exclusive OR and standard OR.
`<= < > >=`	Comparison operators.
`== !=`	Equality operators.
`= %= /= //= -= += *= **=`	Assignment operators.
`is` `is not`	Identity operators.
`in` `not in`	Membership operators.
`not or and`	Logical operators.

Creating and Using Functions

To manage information properly, you need to organize the tools used to perform the required tasks. Each line of code that you create performs a specific task, and you combine these lines of code to achieve a desired result. Sometimes you need to repeat the instructions with different data, and in some cases your code becomes so long that keeping track of what each part does is hard. Functions serve as organization tools that keep your code neat and tidy. In addition, functions make it easy to reuse the instructions you've created as needed with different data. This section of the chapter tells you all about functions.

Viewing functions as code packages

You go to your closet, open the door, and everything spills out. In fact, it's an avalanche, and you're lucky that you've survived. That bowling ball in the top shelf could have done some severe damage! However, you're armed with storage boxes and soon you have everything in the closet in neatly organized boxes. After you're done, you can find anything you want in the closet without fear of injury. Functions are just like that: They take messy code and place it in packages that make it easy to see what you have and understand how it works.

Understanding code reusability

You go to your closet, take out new pants and shirt and put them on. At the end of the day, you take everything off and throw it in the trash. Hmmm . . . That really isn't what most people do. Most people take the clothes off, wash them, and then put them back into the closet for reuse. Functions are reusable, too. No one wants to keep repeating the same task; it becomes monotonous and boring. When you create a function, you define a package of code that you can use over and over to perform the same task. All you need to do is tell the computer to perform a specific task by telling it which function to use.

REMEMBER

When you work with functions, the code that needs services from the function is named the *caller*, and it calls upon the function to perform tasks for it. Much of the information you see about functions refers to the caller. The caller must supply information to the function, and the function returns information to the caller.

At one time, computer programs didn't include the concept of code reusability. As a result, developers had to keep reinventing the same code. It didn't take long for someone to come up with the idea of functions, though, and the concept has evolved over the years until functions have become quite flexible. You can make

functions do anything you want. Code reusability is a necessary part of applications to

- Reduce development time
- Reduce programmer error
- Increase application reliability
- Allow entire groups to benefit from the work of one programmer
- Make code easier to understand
- Improve application efficiency

Defining a function

Creating a function doesn't require much work. The following steps show you the process of creating a function that you can later access:

1. **Create a new notebook in Notebook.**

   The book uses the filename `BPP4D3E; 07; Managing Information.ipynb`, which is where you find all the source code for this chapter. See the Introduction for information on using the downloadable source.

2. **Type** def Hello(): **and press Enter.**

   This step tells Python to define a function named Hello. The parentheses are important because they define any requirements for using the function. (There aren't any requirements in this case.) The colon at the end tells Python that you're done defining the way in which people will access the function. Notice that the insertion pointer is now indented, as shown in Figure 7-1. This indentation is a reminder that you must give the function a task to perform.

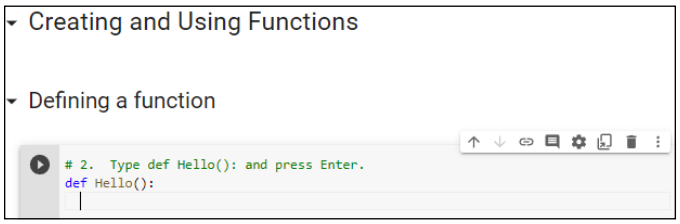


FIGURE 7-1: Define the name of your function.

3. **Type** print("This is my first Python function!") **and press Enter.**

   You should notice something special. The insertion pointer is still indented because Notebook is waiting for you to provide the next step in the function.

4. **Click Run Cell.**

   The function is now complete.

Even though this is a really simple function, it demonstrates the pattern you use when creating any Python function. You define a name, provide any requirements for using the function (none in this case), and provide a series of steps for the function to perform. A function ends when the insertion point is at the left side or you move to the next cell.

Accessing functions

After you define a function, you probably want to use it to perform useful work. Of course, this means knowing how to access the function. In the previous section, you create a new function named `Hello()`. To access this function, you type **Hello()** and click Run Cell. Figure 7-2 shows the output you see when you execute this function.

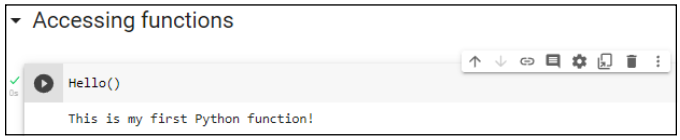


FIGURE 7-2: Whenever you type the function's name, you get the output the function provides.

Every function you create provides a similar pattern of usage. You type the function name, an open parenthesis, any required input, and a closed parenthesis; then you click Run Cell. In this case, you have no input, so all you type is **Hello()**.

Sending information to functions

The `Hello()` example in the previous section is nice because you don't have to keep typing that long string every time you want to say `Hello()`. However, it's also quite limited because you can use it to say only one thing. Functions should allow you to do more than just one thing. Otherwise, you end up writing a lot of functions that vary by the data they use rather than the functionality they provide. Using arguments helps you create functions that are flexible and can use a wealth of data.

Understanding arguments

The term *argument* doesn't mean that you're going to have a fight with the function; it means that you supply information to the function to use in processing a request. Perhaps a better word for it would be input, but the term *input* has been used for so many other purposes that developers decided to use something a bit different: argument. An argument makes it possible for you to send data to the function so that the function can use it when performing a task, making your function more flexible.

The `Hello()` function is currently inflexible because it prints just one string. Adding an argument to the function can make it a lot more flexible because you can send strings to the function to say anything you want. To see how arguments work, create a new function in the notebook. This version of `Hello()`, `Hello2()`, requires an argument:

```
def Hello2(Greeting):
    print(Greeting)
```

Notice that the parentheses are no longer empty. They contain a word, `Greeting`, which is the argument for `Hello2()`. The `Greeting` argument is a variable that you can pass to `print()` to see it onscreen.

REMEMBER

Make sure you click Run Cell after you create `Hello2()` or you will get a `NameError` message when you try to use the function in the next section.

Sending required arguments

You have a new function, `Hello2()`. This function requires that you provide an argument to use it. At least, that's what you've heard so far. Type **Hello2()** and click Run Cell. You see an error message, as shown in Figure 7-3, telling you that `Hello2()` requires an argument.

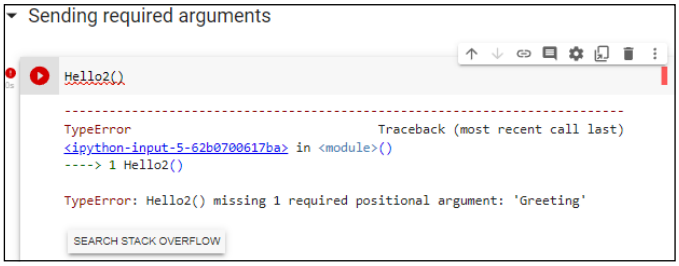


FIGURE 7-3: You must supply an argument or you get an error message.

Not only does Python tell you that the argument is missing, it tells you the name of the argument as well. Creating a function the way you have done so far means that you must supply an argument. Type **Hello2("This is an interesting function.")** and click Run Cell. (Note that if you're using Jupyter Notebook, you automatically advance to the next cell after clicking Run Cell.) This time, you see the expected output. However, you still don't know whether `Hello2()` is flexible enough to print multiple messages. Press Enter (as needed) to start another line, type **Hello2("Another message. . .")**, and click Run Cell. You see the expected output again, so `Hello2()` is indeed an improvement over `Hello()`.

You might easily assume that `Greeting` will accept only a string from the tests you have performed so far. Create a new cell (if necessary). Type **Hello2(1234)**, click Run Cell, and you see 1234 as the output. Likewise, type **Hello2(5 + 5)** in a new cell, and click Run Cell. This time you see the result of the expression, which is 10.

Sending arguments by name

As your functions become more complex and the methods to use them do as well, you may want to provide a little more control over precisely how you call the function and provide arguments to it. Up until now, you have *positional arguments*, which means that you have supplied values in the order in which they appear in the argument list for the function definition. However, Python also has a method for sending arguments by name. In this case, you supply the name of the argument followed by an equals sign (=) and the argument value. To see how this works, type the following function in the notebook and run it:

```
def AddIt(Value1, Value2):
    print(Value1, " + ", Value2, " = ", (Value1 + Value2))
```

REMEMBER

Notice that the `print()` function argument includes a list of items to print and that those items are separated by commas. In addition, the arguments are of different types. Python makes it easy to mix and match arguments in this manner.

Time to test `AddIt()`. Of course, you want to try the function using positional arguments first, so type **AddIt(2, 3)** in a new cell and click Run Cell. You see the expected output of `2 + 3 = 5`. Now type **AddIt(Value2 = 3, Value1 = 2)** and click Run Cell. Again, you receive the output `2 + 3 = 5` even though the positions of the arguments have been reversed.

Giving function arguments a default value

Whether you make the call using positional arguments or named arguments, the functions to this point have required that you supply a value. Sometimes a

function can use default values when a common value is available. Default values make the function easier to use and less likely to cause errors when a developer doesn't provide an input. To create a default value, you simply follow the argument name with an equals sign and the default value. To see how this works, type the following function in a new cell of the notebook and click Run Cell:

```
def Hello3(Greeting = "No Value Supplied"):
    print(Greeting)
```

This is yet another version of the original `Hello()` and updated `Hello2()` functions, but `Hello3()` automatically compensates for individuals who don't supply a value. When someone tries to call `Hello3()` without an argument, it doesn't raise an error. Type **Hello3()** and click Run Cell to see for yourself. Type **Hello3("This is a string.")** and click Run Cell to see a normal response. Lest you think the function is now unable to use other kinds of data, type **Hello3(5)** and click Run Cell; then **Hello3(2 + 7)** and click Run Cell. Figure 7-4 shows the output from all these tests.

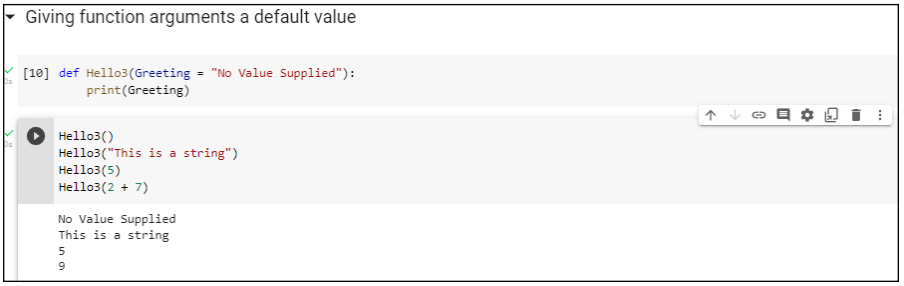


FIGURE 7-4: Supply default arguments when possible to make your functions easier to use.

Creating functions with a variable number of arguments

In most cases, you know precisely how many arguments to provide with your function. It pays to work toward this goal whenever you can because functions with a fixed number of arguments are easier to troubleshoot later. However, sometimes you simply can't determine how many arguments the function will receive at the outset. For example, when you create a Python application that works at the command line, the user might provide no arguments, the maximum number of arguments (assuming there is more than one), or any number of arguments in between.

Fortunately, Python provides a technique for sending a variable number of arguments to a function. You simply create an argument that has an asterisk in front of it, such as `*VarArgs`. The usual technique is to provide a second argument that contains the number of arguments passed as an input. However, you can

eliminate the second argument and simply use the `len()` function to obtain the number of arguments passed to the function. Here is an example of a function that can print a variable number of elements. (Don't worry too much if you don't understand it completely now — you haven't seen some of these techniques used before.)

```
def Hello4(ArgCount, *VarArgs):
    print("You passed ", ArgCount, " arguments.")
    for Arg in VarArgs:
       print(Arg)
```

This example uses something called a `for` loop (described in Chapter 9). For now, all you really need to know is that it takes the arguments out of `VarArgs` one at a time, places the individual argument into `Arg`, and then prints `Arg` using `print()`. After you type the code, click Run Cell.

In a new cell, type **Hello4(1, "A Test String.")** and click Run Cell. You should see the number of arguments and the test string as output — nothing too exciting there. In the next new cell type **Hello4(3, "One", "Two", "Three")** and click Run Cell. As shown in Figure 7-5, the function handles a variable number of arguments without problem.

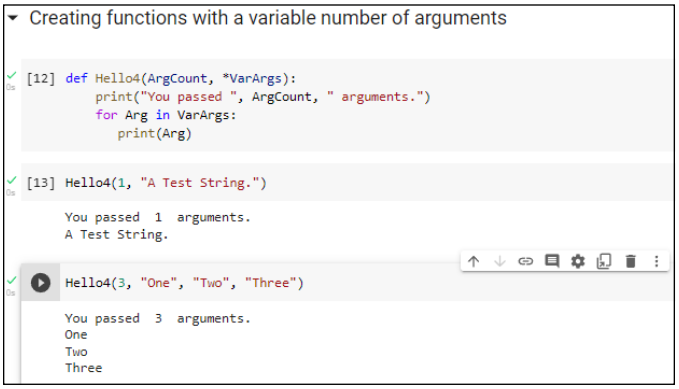


FIGURE 7-5: Variable argument functions can make your applications more flexible.

Returning information from functions

Functions can display data directly or they can return the data to the caller so that the caller can do something more with it. In some cases, a function displays data directly as well as returns data to the caller, but more commonly, a function either displays the data directly or returns it to the caller. Just how a function works depends on the task the function is supposed to perform; a function that performs a math-related task is more likely to return the data to the caller than other functions.

To return data to a caller, a function needs to include the keyword `return`, followed by the data to return. You have no limit on what you can return to a caller. Here are some types of data that you commonly see returned by a function to a caller:

- **Values:** Any value is acceptable. You can return numbers, such as 1 or 2.5; strings, such as "Hello There!"; or Boolean values, such as True or False.
- **Variables:** The content of any variable works just as well as a direct value. The caller receives whatever data is stored in the variable.
- **Expressions:** Many developers use expressions as a shortcut. For example, you can simply return A + B rather than perform the calculation, place the result in a variable, and then return the variable to the caller. Using the expression is faster and accomplishes the same task.
- **Results from other functions:** You can actually return data from another function as part of the return of your function.

It's time to see how return values work. Type the following code into the notebook and click Run Cell:

```
def DoAdd(Value1, Value2):
    return Value1 + Value2
```

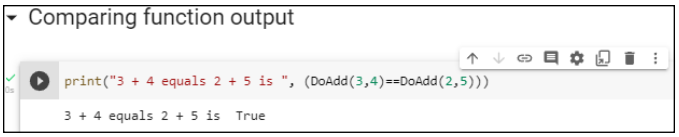
This function accepts two values as input and then returns the sum of those two values. Yes, you could probably perform this task without using a function, but this is how many functions start. To test this function, type **print("The sum of 3 + 4 is ", DoAdd(3, 4))** in a new cell and click Run Cell. You see `The sum of 3 + 4 is 7`.

Comparing function output

You use functions with return values in a number of ways. For example, the previous section of this chapter shows how you can use functions to provide input for another function. You use functions to perform all sorts of tasks. One of the ways to use functions is for comparison purposes. You can actually create expressions from them that define a logical output.

To see how this might work, use the `DoAdd()` function from the previous section. Type **print("3 + 4 equals 2 + 5 is ", (DoAdd(3, 4) == DoAdd(2, 5)))** and click Run Cell. You see the truth value of the statement that 3 + 4 equals 2 + 5, as shown in Figure 7-6.

FIGURE 7-6: Use your functions to perform a wide variety of tasks.



Getting User Input

Very few applications exist in their own world — that is, apart from the user. In fact, most applications interact with users in a major way because computers are designed to serve user needs. To interact with a user, an application must provide some means of obtaining user input. The most commonly used technique for obtaining input is also relatively easy to implement; you use the `input()` function.

REMEMBER

The `input()` function always outputs a string. Even if a user types a number, the output from the `input()` function is a string. This means that if you are expecting a number, you need to convert it after receiving the input. The `input()` function also lets you provide a string prompt. This prompt is displayed to tell the user what to provide in the way of information. The following example shows how to use the `input()` function in a simple way:

```
Name = input("Tell me your name: ")
print("Hello ", Name)
```

In this case, the `input()` function asks the user for a name. After the user types a name and presses Enter, the example outputs a customized greeting to the user. Try running this example. Figure 7-7 shows typical results when you input John as the username.

FIGURE 7-7: Provide a username and see a greeting as output.

You can use `input()` for other kinds of data; all you need is the correct conversion function. The code in the following example provides one technique for performing such a conversion:

```
ANumber = float(input("Type a number: "))
print("You typed: ", ANumber)
```

When you run this example, the application asks for a numeric input. The call to `float()` converts the input to a number. After the conversion, `print()` outputs the result. When you run the example using a value such as 5.5, you obtain the desired result.

WARNING

Understand that data conversion isn't without risk. If you attempt to type something other than a number, you get an error message, as shown in Figure 7-8. Chapter 10 helps you understand how to detect and fix errors before they cause the program to crash.

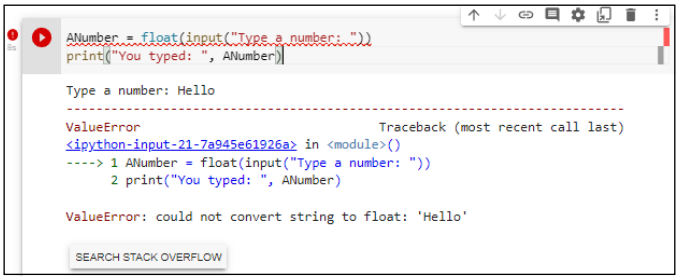


FIGURE 7-8: Data conversion changes the input type to whatever you need, but could cause errors.

IN THIS CHAPTER

- Using the `if` statement to make simple decisions
- Making advanced decisions using `if...else`
- Nesting statements

Chapter 8
Making Decisions

The ability to make a decision, to take one path or another, is an essential element of performing useful work. Math gives the computer the capability to obtain useful information. Decisions enable you to do something with the information after obtaining it. Without the capability to make decisions, a computer would be useless. So any language you use will include the capability to make decisions in some manner. This chapter explores the techniques that Python uses to make decisions.

REMEMBER

Think through the process you use when making a decision. You obtain the actual value of something, compare it to a desired value, and then act accordingly. For example, when you see a signal light and see that it's red, you compare the red light to the desired green light, decide that the light isn't green, and then stop (well, you stop if you aren't being chased by someone). Most people don't take time to consider the process they use because they use it so many times every day. Decision making comes naturally to humans, but computers must perform the following tasks every time:

1. Obtain the actual or current value of something.
2. Compare the actual or current value to a desired value.
3. Perform an action that corresponds to the desired outcome of the comparison.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the BPP4D3E; 08; Making Decisions.ipynb file of the downloadable source. See the Introduction for details on how to find these source files.

Making Simple Decisions by Using the if Statement

The `if` statement is the easiest method for making a decision in Python. It simply states that if something is true, Python should perform the steps that follow. The following sections tell you how you can use the `if` statement to make decisions of various sorts in Python. You may be surprised at what this simple statement can do for you.

Understanding the if statement

You use if statements regularly in everyday life. For example, you may say to yourself, "If it's Wednesday, I'll eat tuna salad for lunch." The Python `if` statement is a little less verbose, but it follows precisely the same pattern. Say you create a variable, `TestMe`, and place a value of `6` in it, like this:

```
TestMe = 6
```

You can then ask the computer to check for a value of `6` in `TestMe`, like this:

```
if TestMe == 6:
   print("TestMe does equal 6!")
```

Every Python `if` statement begins, oddly enough, with the word *if*. When Python sees `if`, it knows that you want it to make a decision. After the word *if* comes a condition. A *condition* simply states what sort of comparison you want Python to make. In this case, you want Python to determine whether `TestMe` contains the value `6`.

REMEMBER

Notice that the condition uses the relational equality operator, `==`, and not the assignment operator, `=`. A common mistake that developers make is to use the assignment operator rather than the equality operator. You can see a list of relational operators in Chapter 7.

The condition always ends with a colon (:). If you don't provide a colon, Python doesn't know that the condition has ended and will continue to look for additional conditions on which to base its decision. After the colon come any tasks you want Python to perform. In this case, Python prints a statement saying that `TestMe` is equal to `6`.

Using the if statement in an application

You can use the `if` statement in a number of ways in Python. However, you immediately need to know about three common ways to use it:

- Use a single condition to execute a single statement when the condition is true.
- Use a single condition to execute multiple statements when the condition is true.
- Combine multiple conditions into a single decision and execute one or more statements when the combined condition is true.

The following sections explore these three possibilities and provide you with examples of their use. You see additional examples of how to use the `if` statement throughout the book because it's such an important method of making decisions.

Working with relational operators

A *relational operator* determines how a value on the left side of an expression compares to the value on the right side of an expression. After it makes the determination, it outputs a value of `true` or `false` that reflects the truth value of the expression. For example, `6 == 6` is `true`, while `5 == 6` is `false`. Table 7-3 in Chapter 7 lists the relational operators. The following steps show how to create and use an `if` statement.

1. **Open a new notebook.**

   You can also use the downloadable source file, `BPP4D3E; 08; Making Decisions.ipynb`.

2. **Type** `TestMe = 6` **and press Enter.**

   This step assigns a value of 6 to `TestMe`. Notice that it uses the assignment operator and not the equality operator.

3. **Type `if TestMe == 6:` and press Enter.**

   This step creates an `if` statement that tests the value of `TestMe` by using the equality operator. You should notice two features of Notebook at this point:

   - The word *if* is highlighted in a different color than the rest of the statement.
   - The next line is automatically indented.

4. **Type `print("TestMe does equal 6!")`.**

   Notice that the word *print* appears in a special color because it's a function name. In addition, the text appears in another color to show you that it's a string value. Color coding makes it much easier to see how Python works.

5. **Click Run Cell.**

   Notebook executes the `if` statement, as shown in Figure 8-1. Because `TestMe` contains a value of 6, the `if` statement works as expected.

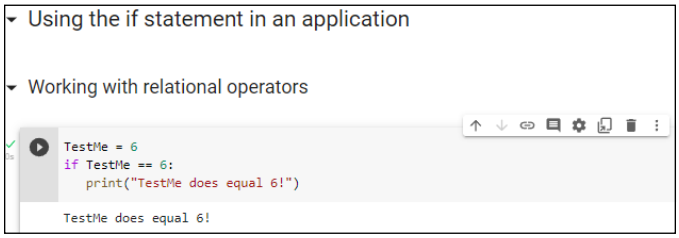


FIGURE 8-1: Simple `if` statements can help your application know what to do in certain conditions.

Performing multiple tasks

Sometimes you want to perform more than one task after making a decision. Python relies on indentation to determine when to stop executing tasks as part of an `if` statement. As long as the next line is indented, it's part of the `if` statement. When the next line is outdented, it becomes the first line of code outside the `if` block. A *code block* consists of a statement and the tasks associated with that statement. The same term is used no matter what kind of statement you're working with, but in this case, you're working with an `if` statement that is part of a code block. The following steps show how to use indentation to execute multiple steps as part of an `if` statement.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
TestMe = 6
if TestMe == 6:
   print("TestMe does equal 6!")
   print("All done!")
```

Notice that the shell continues to indent lines as long as you continue to type code. Each line you type is part of the current `if` statement code block.

REMEMBER

2. **Click Run Cell.**

   Python executes the entire code block. You see the output shown in Figure 8-2.

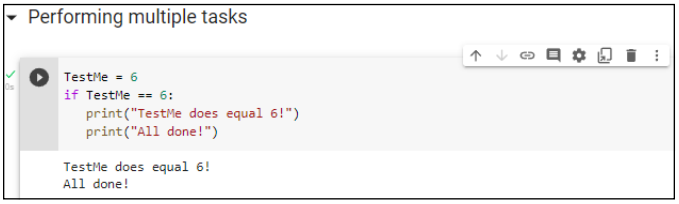


FIGURE 8-2: A code block can contain multiple lines of code — one for each task.

Making multiple comparisons by using logical operators

So far, the examples have all shown a single comparison. Real life often requires that you make multiple comparisons to account for multiple requirements. For example, when baking cookies, if the timer has gone off and the edges are brown, it's time to take the cookies out of the oven.

REMEMBER

To make multiple comparisons, you create multiple conditions by using relational operators and combine them by using logical operators (see Table 7-4 in Chapter 7). A *logical operator* describes how to combine conditions. For example, you might say `x == 6 and y == 7` as two conditions for performing one or more tasks. The `and` keyword is a logical operator that states that both conditions must be true.

One of the most common uses for making multiple comparisons is to determine when a value is within a certain range. In fact, *range checking*, the act of determining whether data is between two values, is an important part of making your application secure and user friendly. The following steps help you see how to perform this task.

1. **Type the following code into a new cell of the notebook — pressing Enter after each line:**

```
Value = int(input("Type a number between 1 and 10: "))
if (Value > 0) and (Value <= 10):
   print("You typed: ", Value)
```

   The example begins by obtaining an input value. You have no idea what the user has typed other than that it's a value of some sort. The use of the `int()` function means that the user must type a whole number (one without a

decimal portion). Otherwise, the application will raise an *exception* (an error indication; Chapter 10 describes exceptions). This first check ensures that the input is at least of the correct type.

The `if` statement contains two conditions. The first states that `Value` must be greater than `0`. You could also present this condition as `Value >= 1`. The second condition states that `Value` must be less than or equal to 10. Only when `Value` meets both of these conditions will the `if` statement succeed and print the value the user typed.

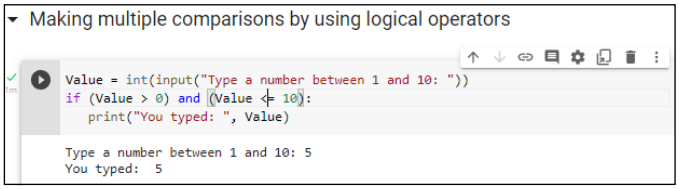
2. **Click Run Cell.**

   Python prompts you to type a number between 1 and 10.

3. **Type** 5 **and press Enter.**

   The application determines that the number is in the right range and outputs the message shown in Figure 8-3.

FIGURE 8-3: The application verifies the value is in the right range and outputs a message.



4. **Select the cell again. Repeat Steps 2 and 3, but type** 22 **instead of 5.**

   The application doesn't output anything because the number is in the wrong range. Whenever you type a value that's outside the programmed range, the statements that are part of the `if` block aren't executed.

REMEMBER

5. **Select the cell again. Repeat Steps 2 and 3, but type** 5.5 **instead of 5.**

   Python displays the error message shown in Figure 8-4. Even though you may think of 5.5 and 5 as both being numbers, Python sees the first number as a floating-point value and the second as an integer.

6. **Repeat Steps 2 and 3, but type** Hello **instead of 5.**

   Python displays about the same error message as before. Python doesn't differentiate between types of wrong input. It knows only that the input type is incorrect and therefore unusable.

TIP

The best applications use various kinds of range checking to ensure that the application behaves in a predictable manner. The more predictable an application becomes, the less the user thinks about the application and the more time the user spends on performing useful work. Productive users tend to be a lot happier than those who constantly fight with their applications.

```
Value = int(input("Type a number between 1 and 10: "))
if (Value > 0) and (Value <= 10):
   print("You typed: ", Value)

Type a number between 1 and 10: 5.5
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-2-c9bf963d9691> in <module>()
----> 1 Value = int(input("Type a number between 1 and 10: "))
      2 if (Value > 0) and (Value <= 10):
      3    print("You typed: ", Value)

ValueError: invalid literal for int() with base 10: '5.5'

SEARCH STACK OVERFLOW
```

FIGURE 8-4: Typing the wrong type of information results in an error message.

Choosing Alternatives by Using the if. . .else Statement

Many of the decisions you make in an application fall into a category of choosing one of two options based on conditions. For example, when looking at a signal light, you choose one of two options: Press on the brake to stop or press the accelerator to continue (the assumption is that you won't run the red light). The option you choose depends on the conditions. A green light signals that you can continue on through the light; a red light tells you to stop. The following sections describe how Python makes choosing between two alternatives possible.

Understanding the if. . .else statement

With Python, you choose one of two alternatives by using the `else` clause of the `if` statement. A *clause* is an addition to a code block that modifies the way in which it works. Most code blocks support multiple clauses. In this case, the `else` clause enables you to perform an alternative task, which increases the usefulness of the `if` statement. Most developers refer to the form of the `if` statement that has the `else` clause included as the `if...else` statement, with the ellipsis implying that something happens between `if` and `else`.

WARNING

Sometimes developers encounter problems with the `if...else` statement because they forget that the `else` clause always executes when the conditions for the `if` statement aren't met. Be sure to think about the consequences of always executing a set of tasks when the conditions are false. Sometimes doing so can lead to unintended consequences.

Using the if. . .else statement in an application

The example in the previous section is a little less helpful than it could be when the user enters a value that's outside the intended range. Even entering data of the wrong type produces an error message, but entering the correct type of data outside the range tells the user nothing. In this example, you discover the means for correcting this problem by using an `else` clause. The following steps demonstrate just one reason to provide an alternative action when the condition for an `if` statement is false:

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
Value = int(input("Type a number between 1 and 10: "))
if (Value > 0) and (Value <= 10):
   print("You typed: ", Value)
else:
   print("The value you typed is incorrect!")
```

   As before, the example obtains input from the user and then determines whether that input is in the correct range. However, in this case, the `else` clause provides an alternative output message when the user enters data outside the desired range.

TIP

   Notice that the `else` clause ends with a colon, just as the `if` statement does. Most clauses that you use with Python statements have a colon associated with them so that Python knows when the clause has ended. If you receive a coding error for your application, make sure that you check for the presence of the colon as needed.

2. **Click Run Cell.**

   Python prompts you to type a number between 1 and 10.

3. **Type** 5 **and press Enter.**

   The application determines that the number is in the right range and outputs the message shown previously in Figure 8-3.

4. **Repeat Steps 2 and 3, but type** 22 **instead of 5.**

   This time the application outputs the error message shown in Figure 8-5. The user now knows that the input is incorrect and knows to try entering it again.

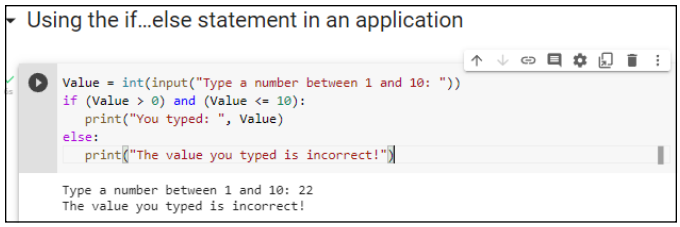


FIGURE 8-5: Providing feedback for incorrect input is always a good idea.

Using the if. . .elif statement in an application

You go to a restaurant and look at the menu. The restaurant offers eggs, pancakes, waffles, and oatmeal for breakfast. After you choose one of the items, the server brings it to you. Creating a menu selection requires something like an `if...else` statement, but with a little extra oomph. In this case, you use the `elif` clause to create another set of conditions. The `elif` clause is a combination of the `else` clause and a separate `if` statement. The following steps describe how to use the `if...elif` statement to create a menu.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
print("1. Red")
print("2. Orange")
print("3. Yellow")
print("4. Green")
print("5. Blue")
print("6. Purple")
Choice = int(input("Select your favorite color: "))
if (Choice == 1):
   print("You chose Red!")
elif (Choice == 2):
   print("You chose Orange!")
elif (Choice == 3):
   print("You chose Yellow!")
elif (Choice == 4):
   print("You chose Green!")
elif (Choice == 5):
   print("You chose Blue!")
elif (Choice == 6):
   print("You chose Purple!")
else:
   print("You made an invalid choice!")
```

The example begins by displaying a menu. The user sees a list of choices for the application. It then asks the user to make a selection, which it places inside `Choice`. The use of the `int()` function ensures that the user can't type anything other than a number.

After the user makes a choice, the application looks for it in the list of potential values. In each case, `Choice` is compared against a particular value to create a condition for that value. When the user types 1, the application outputs the message `You chose Red!`. If none of the options is correct, the `else` clause is executed by default to tell the user that the input choice is invalid.

2. **Click Run Cell.**

   Python displays the menu. The application asks you to select your favorite color.

3. **Type** 1 **and press Enter.**

   The application displays the appropriate output message, as shown in Figure 8-6.

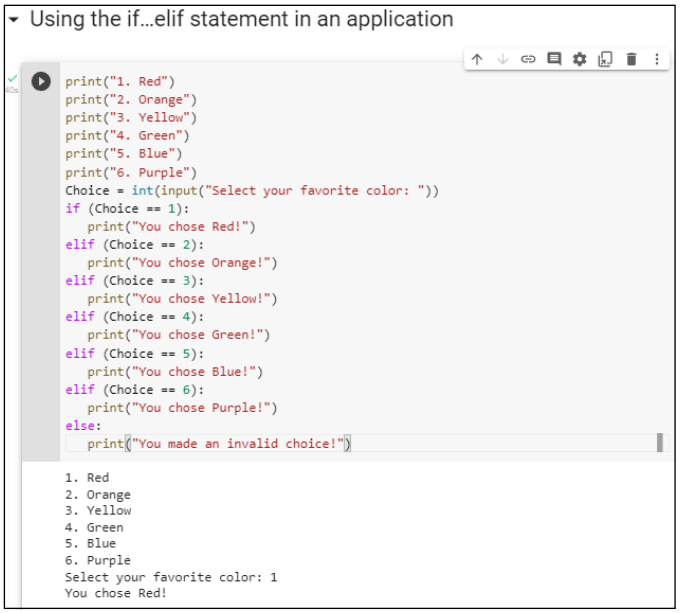


FIGURE 8-6: Menus let you choose one option from a list of options.

4. **Repeat Steps 3 and 4, but type** 5 **instead of 1.**

   The application displays a different output message — the one associated with the requested color.

5. **Repeat Steps 3 and 4, but type** 8 **instead of 1.**

   The application tells you that you made an invalid choice.

6. **Repeat Steps 3 and 4, but type** Red **instead of 1.**

   The application displays the expected error message, as shown in Figure 8-7. Any application you create should be able to detect errors and incorrect inputs. Chapter 10 shows you how to handle errors so that they're user friendly.

```
print("1. Red")
print("2. Orange")
print("3. Yellow")
print("4. Green")
print("5. Blue")
print("6. Purple")
Choice = int(input("Select your favorite color: "))
if (Choice == 1):
   print("You chose Red!")
elif (Choice == 2):
   print("You chose Orange!")
elif (Choice == 3):
   print("You chose Yellow!")
elif (Choice == 4):
   print("You chose Green!")
elif (Choice == 5):
   print("You chose Blue!")
elif (Choice == 6):
   print("You chose Purple!")
else:
   print("You made an invalid choice!")
```

```
1. Red
2. Orange
3. Yellow
4. Green
5. Blue
6. Purple
Select your favorite color: Red
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-3-e5261cca43c4> in <module>()
      5 print("5. Blue")
      6 print("6. Purple")
----> 7 Choice = int(input("Select your favorite color: "))
      8 if (Choice == 1):
      9    print("You chose Red!")

ValueError: invalid literal for int() with base 10: 'Red'

SEARCH STACK OVERFLOW
```

FIGURE 8-7: Every application you create should include some means of detecting errant input.

NO SWITCH STATEMENT?

If you've worked with other languages, you might notice that Python lacks a switch statement (if you haven't, there is no need to worry about it with Python). Developers commonly use the switch statement in other languages to create menu-based applications. The `if...elif` statement is generally used for the same purpose in Python.

However, the `if...elif` statement doesn't provide quite the same functionality as a switch statement because it doesn't enforce the use of a single variable for comparison purposes. As a result, some developers rely on Python's dictionary functionality to stand in for the switch statement. Chapter 14 describes how to work with dictionaries. You can also use a class approach for a switch substitute, which you can read about at `https://favtutor.com/blogs/python-switch-case`. The reason the class approach isn't covered in the book is that it's harder to implement, harder to read, and harder to debug than using the dictionary approach.

Using Nested Decision Statements

The decision-making process often happens in levels. For example, when you go to the restaurant and choose eggs for breakfast, you have made a first-level decision. Now the server asks you what type of toast you want with your eggs. The server wouldn't ask this question if you had ordered pancakes, so the selection of toast becomes a second-level decision. When the breakfast arrives, you decide whether you want to use jelly on your toast. This is a third-level decision. If you had selected a kind of toast that doesn't work well with jelly, you might not have had to make this decision at all. This process of making decisions in levels, with each level reliant on the decision made at the previous level, is called *nesting.* Developers often use nesting techniques to create applications that can make complex decisions based on various inputs. The following sections describe several kinds of nesting you can use within Python to make complex decisions.

Using multiple if or if. . .else statements

The most commonly used multiple selection technique is a combination of `if` and `if...else` statements. This form of selection is often called a *selection tree* because of its resemblance to the branches of a tree. In this case, you follow a particular path to obtain a desired result. The following steps show how to create a selection tree:

1. **Type the following code into the notebook — pressing Enter after each line:**

```
One = int(input("Type a number between 1 and 10: "))
Two = int(input("Type a number between 1 and 10: "))
if (One >= 1) and (One <= 10):
   if (Two >= 1) and (Two <= 10):
      print("Your secret number is: ", One * Two)
   else:
      print("Incorrect second value!")
else:
   print("Incorrect first value!")
```

This is simply an extension of the example you see in the "Using the if. . .else statement in an application" section of the chapter. However, notice that the indentation is different. The second `if. . .else` statement is indented within the first `if. . .else` statement. The indentation tells Python that this is a second-level statement.

2. **Click Run Cell.**

You see a Python Shell window open with a prompt to type a number between 1 and 10.

3. **Type** 5 **and press Enter.**

The shell asks for another number between 1 and 10.

4. **Type** 2 **and press Enter.**

You see the combination (product) of the two numbers as output, as shown in Figure 8-8.

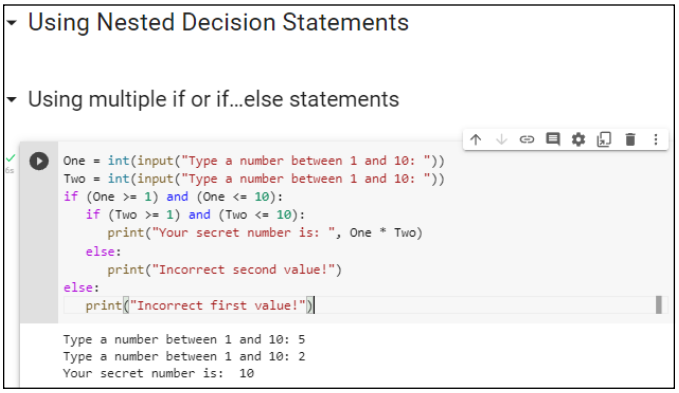


FIGURE 8-8: Adding multiple levels lets you perform tasks with greater complexity.

This example has the same input features as the previous `if...else` example. For example, if you attempt to provide a value that's outside the requested range, you see an error message. The error message is tailored for either the first or second input value so that the user knows which value was incorrect.

REMEMBER

Providing specific error messages is always useful because users tend to become confused and frustrated otherwise. In addition, a specific error message helps you find errors in your application much faster.

Combining other types of decisions

You can use any combination of `if`, `if...else`, and `if...elif` statements to produce a desired outcome. You can nest the code blocks as many levels deep as needed to perform the required checks. For example, Listing 8-1 shows what you might accomplish for a breakfast menu.

LISTING 8-1: **Creating a Breakfast Menu**

```
print("1. Eggs")
print("2. Pancakes")
print("3. Waffles")
print("4. Oatmeal")
MainChoice = int(input("Choose a breakfast item: "))
if (MainChoice == 2):
   Meal = "Pancakes"
elif (MainChoice == 3):
   Meal = "Waffles"
if (MainChoice == 1):
   print("1. Wheat Toast")
   print("2. Sour Dough")
   print("3. Rye Toast")
   print("4. Pancakes")
   Bread = int(input("Choose a type of bread: "))
   if (Bread == 1):
      print("You chose eggs with wheat toast.")
   elif (Bread == 2):
      print("You chose eggs with sour dough.")
   elif (Bread == 3):
      print("You chose eggs with rye toast.")
   elif (Bread == 4):
      print("You chose eggs with pancakes.")
   else:
      print("We have eggs, but not that kind of bread.")
```

```
elif (MainChoice == 2) or (MainChoice == 3):
   print("1. Syrup")
   print("2. Strawberries")
   print("3. Powdered Sugar")
   Topping = int(input("Choose a topping: "))
   if (Topping == 1):
      print ("You chose " + Meal + " with syrup.")
   elif (Topping == 2):
      print ("You chose " + Meal + " with strawberries.")
   elif (Topping == 3):
      print ("You chose " + Meal + " with powdered sugar.")
   else:
      print ("We have " + Meal + ", but not that topping.")
elif (MainChoice == 4):
   print("You chose oatmeal.")
else:
   print("We don't serve that breakfast item!")
```

This example has some interesting features. For one thing, you might assume that an `if...elif` statement always requires an `else` clause. This example shows a situation that doesn't require such a clause. You use an `if...elif` statement to ensure that `Meal` contains the correct value, but you have no other options to consider.

The selection technique is the same as you saw for the previous examples. A user enters a number in the correct range to obtain a desired result. Three of the selections require a secondary choice, so you see the menu for that choice. For example, when ordering eggs, it isn't necessary to choose a topping, but you do want a topping for pancakes or waffles.

Notice that this example also combines variables and text in a specific way. Because a topping can apply equally to waffles or pancakes, you need some method for defining precisely which meal is being served as part of the output. The `Meal` variable that the application defines earlier is used as part of the output after the topping choice is made.

The best way to understand this example is to play with it. Try various menu combinations to see how the application works.

IN THIS CHAPTER

- Performing a task a specific number of times
- Performing a task until completion
- Placing one task loop within another

Chapter 9
Performing Repetitive Tasks

All the examples in the book so far have performed a series of steps just one time and then stopped. However, the real world doesn't work this way. Many of the tasks that humans perform are repetitious. For example, the doctor might state that you need to exercise more and tell you to do 100 push-ups each day. If you just do one push-up, you won't get much benefit from the exercise and you definitely won't be following the doctor's orders. Of course, because you know precisely how many push-ups to do, you can perform the task a specific number of times. Python allows the same sort of repetition by using the `for` statement.

Unfortunately, you don't always know how many times to perform a task. For example, consider needing to check a stack of coins for one of extreme rarity. Taking just the first coin from the top, examining it, and deciding that it either is or isn't the rare coin doesn't complete the task. Instead, you must examine each coin in turn, looking for the rare coin. Your stack may contain more than one. Only after you have looked at every coin in the stack can you say that the task is complete. However, because you don't know how many coins are in the stack, you don't know how many times to perform the task at the outset. You only know the task is done when the stack is gone. Python performs this kind of repetition by using the `while` statement.

REMEMBER

Most programming languages call any sort of repeating sequence of events a *loop*. The idea is to picture the repetition as a circle, with the code going round and round executing tasks until the loop ends. Loops are an essential part of application elements such as menus. In fact, writing most modern applications without using loops would be impossible.

In some cases, you must create loops within loops. For example, to create a multiplication table, you use a loop within a loop. The inner loop calculates the column values and the outer loop moves between rows. You see such an example later in the chapter, so don't worry too much about understanding precisely how such things work right now.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 09; Performing Repetitive Tasks.ipynb` file of the downloadable source. See the Introduction for details on how to find these source files.

Processing Data Using the for Statement

The first looping code block that most developers encounter is the `for` statement. It's hard to imagine creating a conventional programming language that lacks such a statement. In this case, the loop executes a fixed number of times, and you know the number of times it will execute before the loop even begins. Because everything about a `for` loop is known at the outset, `for` loops tend to be the easiest kind of loop to use. However, in order to use one, you need to know how many times to execute the loop. The following sections describe the `for` loop in greater detail.

RECURSION: ANOTHER WAY TO PERFORM TASKS REPETITIVELY

Recursion provides a method of repeating tasks (a type of loop implementation) by using a function that calls itself until it solves a particular problem by simplifying the original problem. It's sort of like talking to yourself until you've unraveled a problem to its simplest case, and then you notice that people are looking at you quite strangely. Functional programming languages such as Haskell get along just fine without `for` or `while` loops by using recursion; see `https://sarakhandaker.medium.com/how-to-code-with-no-loops-8ed815624aae` for details. Python also supports recursion as an advanced technique that isn't covered in this book, but you can read about it at `https://realpython.com/python-recursion/`. For the most part, when working with Python, you want to use `for` and `while` loops to keep things simple and easy to debug.

Understanding the for statement

A `for` loop begins with a `for` statement. The `for` statement describes how to perform the loop. The Python `for` loop works through a sequence of some type. It doesn't matter whether the sequence is a series of letters in a string or items within a collection. You can even specify a range of values to use by specifying the `range()` function. Here's a simple `for` statement.

```
for Letter in "Howdy!":
```

The statement begins with the keyword `for`. The next item is a variable that holds a single element of a sequence. In this case, the variable name is `Letter`. The `in` keyword tells Python that the sequence comes next. In this case, the sequence is the string `"Howdy"`. The `for` statement always ends with a colon, just as the decision-making statements described in Chapter 8 do.

Indented under the `for` statement are the tasks you want performed within the `for` loop. Python considers every following indented statement part of the code block that composes the `for` loop. Again, the `for` loop works just like the decision-making statements in Chapter 8.

Creating a basic for loop

The best way to see how a `for` loop actually works is to create one. In this case, the example uses a string for the sequence. The `for` loop processes each of the characters in the string in turn until it runs out of characters.

1. **Open a new notebook.**

   You can also use the downloadable source file, `BPP4D3E; 09; Performing Repetitive Tasks.ipynb`.

2. **Type the following code into the notebook — pressing Enter after each line:**

   ```
   LetterNum = 1
   for Letter in "Howdy!":
      print("Letter ", LetterNum, " is ", Letter)
      LetterNum += 1
   ```

   The example begins by creating a variable, `LetterNum`, to track the number of letters that have been processed. Every time the loop completes, `LetterNum` is updated (incremented) by `1`.

The `for` statement works through the sequence of letters in the string `"Howdy!"`. It places each letter, in turn, in `Letter`. The code that follows displays the current `LetterNum` value and its associated character found in `Letter`.

3. **Click Run Cell.**

   The application displays the letter sequence along with the letter number, as shown in Figure 9-1.

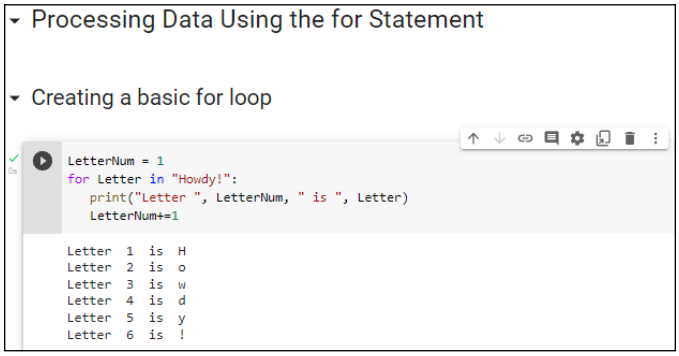


FIGURE 9-1: Use the `for` loop to process the characters in a string one at a time.

Controlling execution with the break statement

Life is often about exceptions to the rule. For example, you might want an assembly line to produce a number of clocks. However, at some point, the assembly line runs out of a needed part. If the part isn't available, the assembly line must stop in the middle of the processing cycle. The count hasn't completed, but the line must be stopped anyway until the missing part is restocked.

Interruptions also occur in computers. You might be streaming data from an online source when a network glitch occurs and breaks the connection; the stream temporarily runs dry, so the application runs out of things to do even though the set number of tasks isn't completed.

REMEMBER

The `break` clause makes breaking out of a loop possible. However, you don't simply place the `break` clause in your code — you surround it with an `if` statement that defines the condition for issuing a `break`. The statement might say something like this: If the data stream runs dry, then break out of the loop.

In this example, you see what happens when the count reaches a certain level when processing a string. The example is a little contrived in the interest of

keeping things simple, but it reflects what could happen in the real world when a data element is too long to process (possibly indicating an error condition).

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
Value = input("Type less than 6 characters: ")
LetterNum = 1
for Letter in Value:
   print("Letter ", LetterNum, " is ", Letter)
   LetterNum += 1
   if LetterNum > 6:
      print("The string is too long!")
      break
```

   This example builds on the one found in the previous section. However, it lets the user provide a variable-length string. When the string is longer than six characters, the application stops processing it.

   The `if` statement contains the conditional code. When `LetterNum` is greater than 6, it means that the string is too long. Notice the second level of indentation used for the `if` statement. In this case, the user sees an error message stating that the string is too long, and then the code executes a `break` to end the loop.

2. **Click Run Cell.**

   Python displays a prompt asking for input.

3. **Type** `Hello` **and press Enter.**

   The application lists each character in the string.

4. **Perform Steps 2 and 3 again, but type** I am too long. **instead of Hello.**

   The application displays the expected error message and stops processing the string at character 6, as shown in Figure 9-2.

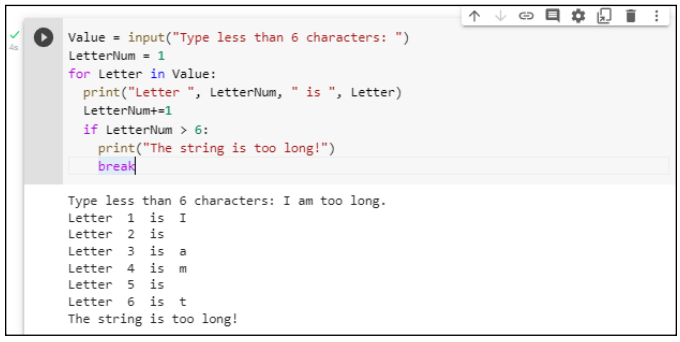


FIGURE 9-2: Long strings are truncated to ensure that they remain a certain size.

TIP

This example adds *length checking* to your repertoire of application data error checks. Chapter 8 shows how to perform range checks, which ensure that a value meets specific value limits. The length check is necessary to ensure that data, especially strings, aren't going to overrun the size of data fields. In addition, a small input size makes it harder for intruders to perform certain types of hacks (such as script injection, as described at `https://www.stackhawk.com/blog/command-injection-python/`) on your system, which makes your system more secure.

Controlling execution with the continue statement

Sometimes you want to check every element in a sequence, but don't want to process certain elements. For example, you might decide that you want to process all the information for every car in a database except brown cars. Perhaps you simply don't need the information about that particular color of car. The `break` clause simply ends the loop, so you can't use it in this situation. Otherwise, you won't see the remaining elements in the sequence.

REMEMBER

The `break` clause alternative that many developers use is the `continue` clause. As with the `break` clause, the `continue` clause appears as part of an `if` statement. However, processing continues with the next element in the sequence rather than ending completely.

The following steps help you see how the `continue` clause differs from the `break` clause. In this case, the code refuses to process the letter *w*, but will process every other letter in the alphabet.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
LetterNum = 1
for Letter in "Howdy!":
   if Letter == "w":
      continue
      print("Encountered w, not processed.")
   print("Letter ", LetterNum, " is ", Letter)
   LetterNum += 1
```

This example is based on the one found in the "Creating a basic for loop" section, earlier in this chapter. However, this example adds an `if` statement with the `continue` clause in the `if` code block.

WARNING

Notice the `print()` function call that is part of the `if` code block. You never see this string printed because the current loop iteration ends immediately. This kind of unreachable code is often called *dead code* by developers because it just sort of sits there like a couch potato doing nothing at all. Avoid dead code because it complicates the debugging process and makes code harder to read and understand.

2. **Click Run Cell.**

   Python displays the letter sequence along with the letter number, as shown in Figure 9-3. However, notice the effect of the `continue` clause — the letter *w* isn't processed.

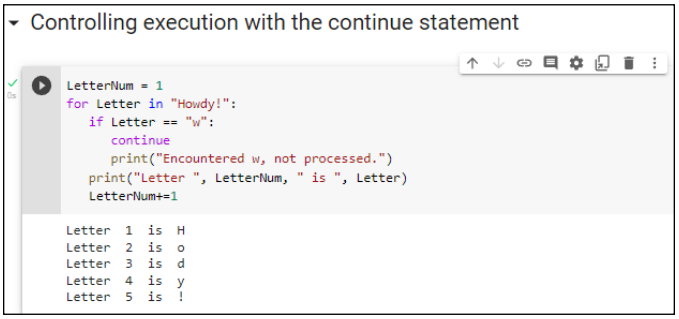


FIGURE 9-3: Use the `continue` clause to avoid processing specific elements.

Doing nothing with the pass statement

The Python language includes something not commonly found in other languages: a `pass` clause. The `pass` clause tells Python to do nothing. You might wonder why a language would include a clause that does nothing at all. Mostly, the `pass` clause is used when you're developing the logic for your application and you don't quite know what you're going to do inside an `if`, `for`, `while`, or other statement that isn't complete without code to execute. The `pass` clause allows you to execute your code so that you can see the logic work without having to deal with error messages. The following steps use an example that is similar to the one found in the previous section, "Controlling execution with the continue statement," except that it uses a `pass` clause instead.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
LetterNum = 1
for Letter in "Howdy!":
   if Letter == "w":
      pass # Add some code here later to process w
   print("Letter ", LetterNum, " is ", Letter)
   LetterNum += 1
```

2. **Click Run Cell.**

   The output is similar to that shown in Figure 9-1. The `pass` clause simply makes the `if` statement complete in this case.

TIP

Notice how the `pass` clause makes it possible for this example to run even though the `if` statement would normally be considered incomplete. Using the pass clause provides you with these advantages during development:

- Allows you to work on the overall application first, without worrying about details
- Enables you to run the code in a scaffolded form so that you can verify its overall operation
- Reduces development time because you don't get involved in the detailed work of writing the application too soon

Validating input with the else statement

Python has another loop clause that you won't find with other languages: `else`. The `else` clause makes executing code possible even if you have no elements to process in a sequence. For example, you might need to convey to the user that there simply isn't anything to do. The `else` clause is always partnered with an `if` statement that's part of the loop and matched with a `break` clause in this case, so it's an extension of the example in the "Controlling execution with the break statement" section of the chapter. In fact, that's what the following example does.

1. **Type the following code into the notebook — pressing Enter after each line:**

```
Value = input("Type less than 6 characters: ")
LetterNum = 1
for Letter in Value:
   print("Letter ", LetterNum, " is ", Letter)
   LetterNum += 1
   if (Letter == "" or Letter == Value[-1]):
      break
else:
   print("The string is blank.")
```

REMEMBER

Notice the indentation levels in this case. The `if` statement is indented below the `for` statement, but the `else` clause appears at the same level of indentation as the `for` statement. Using the correct indentation is essential in this situation.

The `if` statement also has to check for two conditions in this example:

- The string is empty because the user pressed Enter.
- The processing has reached the end of the string and is complete. (You find the end of a string by using `Value[-1]` in this case; the "Selecting Individual Characters" section of Chapter 12 tells you more about string indexes.)

2. **Click Run Cell.**

   Python displays a prompt asking for input.

3. **Type** Hello **and press Enter.**

   The application lists each character in the string, as shown previously in Figure 9-1.

4. **Repeat Steps 2 and 3. However, simply press Enter instead of entering any sort of text.**

   You see the alternative message shown in Figure 9-4 that tells you the string is blank.

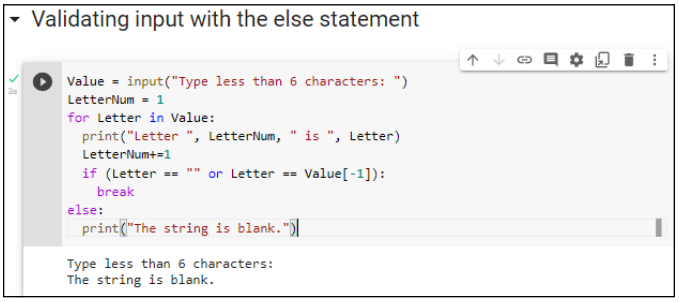


FIGURE 9-4: The `else` clause enables you to perform tasks based on an empty sequence.

WARNING

You can easily misuse the `else` clause because an empty sequence doesn't always signify a simple lack of input. An empty sequence can also signal an application error or other conditions that need to be handled differently from a simple omission of data. Make sure you understand how the application works with data to ensure that the `else` clause doesn't end up hiding potential error conditions, rather than making them visible so that they can be fixed.

Processing Data by Using the while Statement

You use the `while` statement for situations when you're not sure how much data the application will have to process. Instead of instructing Python to process a static number of items, you use the `while` statement to tell Python to continue processing items until it runs out of items. This kind of loop is useful when you need to perform tasks such as downloading files of unknown size or streaming data from a source such as a radio station. Any situation in which you can't define at the outset how much data the application will process is a good candidate for the `while` statement, which the following sections describe more fully.

Understanding the while statement

The `while` statement works with a condition rather than a sequence. The condition states that the `while` statement should perform a task until the condition is no longer true. For example, imagine a deli with a number of customers standing in front of the counter. The salesperson continues to service customers until no more customers are left in line. The line could (and probably will) grow as the other customers are handled, so it's impossible to know at the outset how many customers will be served. All the salesperson knows is that continuing to serve customers until no more are left is important. Here is how a `while` statement might look:

```
while Sum < 5:
```

The statement begins with the `while` keyword. It then adds a condition. In this case, a variable, `Sum`, must be less than `5` for the loop to continue. Nothing specifies the current value of `Sum`, nor does the code define how the value of `Sum` will change. The only thing that is known when Python executes the statement is that `Sum` must be less than `5` for the loop to continue performing tasks. The statement ends with a colon and the tasks are indented below the statement.

WARNING

Because the `while` statement doesn't perform a series of tasks a set number of times, creating an *endless loop* is possible, meaning that the loop never ends. For example, say that `Sum` is set to `0` when the loop begins, and the ending condition is `while Sum < 5`. If the value of `Sum` never increases, the loop will continue executing forever (or at least until the computer is shut down). Endless loops can cause all sorts of bizarre problems on systems, such as slowdowns and even computer freezes, so it's best to avoid them. You must always provide a method for the loop to end when using a `while` loop (contrasted with the `for` loop, in which the

end of the sequence determines the end of the loop). So, when working with the `while` statement, you must perform three tasks:

1. Create the environment for the condition (such as setting `Sum` to `0`).
2. State the condition within the `while` statement (such as `Sum < 5`).
3. Update the condition as needed to ensure that the loop eventually ends (such as adding `Sum+=1` to the `while` code block).

REMEMBER

As with the `for` statement, you can modify the default behavior of the `while` statement. In fact, you have access to the same four clauses to modify the `while` statement behavior:

- `break`: Ends the current loop.
- `continue`: Immediately ends processing of the current element, but continues on with the next element.
- `pass`: Acts as a placeholder for future code.
- `else`: Provides an alternative processing technique when conditions aren't met for the loop.

Using the while statement in an application

You can use the `while` statement in many ways, but this first example is straightforward. It simply displays a count based on the starting and ending condition of a variable named `Sum`. The following steps help you create and test the example code.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
Sum = 0
while Sum < 5:
   print(Sum)
   Sum += 1
```

The example code demonstrates the three tasks you must perform when working with a `while` loop in a straightforward manner. It begins by setting `Sum` to `0`, which is the first step of setting the condition environment. The condition itself appears as part of the `while` statement. The end of the `while` code block accomplishes the third step. Of course, the code displays the current value of `Sum` before it updates the value of `Sum`.

REMEMBER

A `while` statement provides flexibility that you don't get with a `for` statement. This example shows a relatively straightforward way to update `Sum`. However, you can use any update method required to meet the goals of the application. Nothing says that you have to update `Sum` in a specific manner. In addition, the condition can be as complex as you want it to be. For example, you can track the current value of three or four variables if you want. Of course, the more complex you make the condition, the more likely you are to create an endless loop, so you have a practical limit as to how complex you should make the `while` loop condition.

2. **Click Run Cell.**

   Python executes the `while` loop and displays the numeric sequence shown in Figure 9-5.

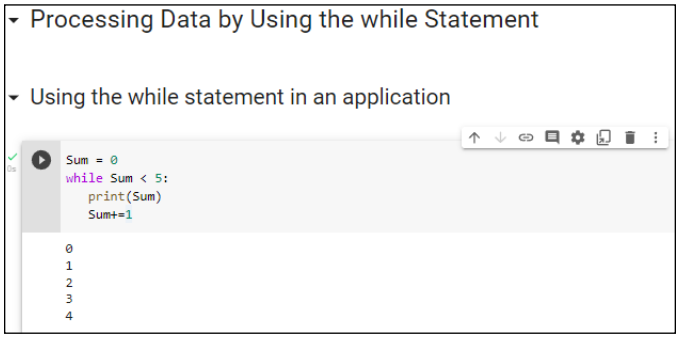


FIGURE 9-5: The simple `while` loop displays a sequence of numbers.

Nesting Loop Statements

In some cases, you can use either a `for` loop or a `while` loop to achieve the same effect. The loop statements work differently, but the effect is the same. In this example, you create a multiplication table generator by nesting a `while` loop within a `for` loop. Because you want the output to look nice, you use a little formatting as well. (Chapter 12 gives you the details.)

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
X = 1
Y = 1
print ('{:>4}'.format(' '), end= ' ')
for X in range(1, 11):
   print('{:>4}'.format(X), end=' ')
```

```
print()
for X in range(1,11):
   print('{:>4}'.format(X), end=' ')
   while Y <= 10:
      print('{:>4}'.format(X * Y), end=' ')
      Y += 1
   print()
   Y=1
```

This example begins by creating two variables, `X` and `Y`, to hold the row and column values of the table. `X` is the row variable and `Y` is the column variable.

To make the table readable, this example must create a heading at the top and another along the side. When users see a `1` at the top and a `1` at the side, and follow these values to where they intersect in the table, they can see the value of the two numbers when multiplied.

The first `print()` statement adds a space (because nothing appears in the corner of the table; see Figure 9-6 to more easily follow this discussion). All the formatting statement says is to create a space 4 characters wide and place a space within it. The `{:>4}` part of the code determines the size of the column. The `format(' ')` function determines what appears in that space. The end attribute of the `print()` statement changes the ending character from a carriage return to a simple space.

The first `for` loop displays the numbers 1 through 10 at the top of the table. The `range()` function creates the sequence of numbers for you. When using the `range()` function, you specify the starting value, which is `1` in this case, and one more than the ending value, which is `11` in this case.

At this point, the cursor is sitting at the end of the heading row. To move it to the next line, the code issues a `print()` call with no other information.

Even though the next bit of code looks quite complex, you can figure it out if you look at it a line at a time. The multiplication table shows the values from `1 * 1` to `10 * 10`, so you need ten rows and ten columns to display the information. The `for` statement tells Python to create ten rows.

Look again at Figure 9-6 to note the row heading. The first `print()` call displays the row's label on the left. Of course, you have to format this information, and the code uses a space of four characters that end with a space, rather than a carriage return, in order to continue printing information in that row.

The `while` loop comes next. This loop prints the columns in an individual row. The column values are the multiplied values of `X * Y`. Again, the output is formatted to take up four spaces. The `while` loop ends when `Y` is updated to the next value by using `Y+=1`.

Now you're back into the `for` loop. The `print()` statement ends the current row. In addition, `Y` must be reset to `1` so that it's ready for the beginning of the next row, which begins with `1`.

2. **Click Run Cell.**

   You see the multiplication table shown in Figure 9-6.

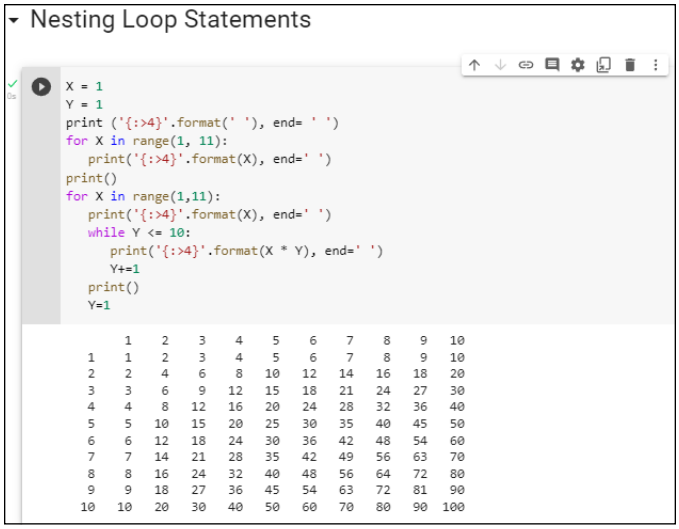


FIGURE 9-6: The multiplication table is pleasing to the eye thanks to its formatting.

IN THIS CHAPTER

- Understanding error sources
- Handling error conditions
- Specifying that an error has occurred
- Developing your own error indicators
- Performing tasks even after an error occurs

Chapter 10
Dealing with Errors

Most application code of any complexity has errors in it. When your application suddenly freezes for no apparent reason, that's an error. Seeing one of those obscure message dialog boxes is another kind of error. However, errors can occur that don't provide you with any sort of notification. An application might perform the wrong computation on a series of numbers you provide, resulting in incorrect output that you may never know about unless someone tells you that something is wrong or you check for the issue yourself. Errors need not be consistent, either. You may see them on some occasions and not on others. For example, an error can occur only when the weather is bad or the network is overloaded. In short, errors occur in all sorts of situations and for all sorts of reasons. The big thing to remember is that the computer hasn't personally targeted you and has nothing against you — computers don't have emotions. This chapter is designed to alleviate any computer-related angst you might have about errors.

It shouldn't surprise you that errors occur. Applications are written by humans, and humans make mistakes. Most developers call application errors *exceptions*, meaning that they're the exception to the rule. Because exceptions do occur in applications, you need to detect and do something about them whenever possible. The act of detecting and processing an exception is called *error handling* or *exception handling*. To properly detect errors, you need to know about error sources and why errors occur in the first place. When you do detect the error, you must process it by *catching* the exception. Catching an exception means examining it and possibly doing something about it. So, another part of this chapter is about discovering how to perform exception handling in your own application.

Sometimes your code detects an error in the application. When this happens, you need to *raise* or *throw* an exception. You see both terms used for the same thing, which simply means that your code encountered an error it couldn't handle, so it passed the error information onto another piece of code to *handle* (interpret, process, and, with luck, fix the exception). In some cases, you use custom error message objects to pass on the information. Even though Python has a wealth of generic message objects that cover most situations, some situations are special. For example, you might want to provide special support for a database application, and Python doesn't normally cover that contingency with a generic message object. You need to know when to handle exceptions locally, when to send them to the code that called your code, and when to create special exceptions so that every part of the application knows how to handle the exception — all of which are topics covered by this chapter.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 10; Dealing with Errors.ipynb` file of the downloadable source. See the Introduction for details on how to find these source files.

Knowing Why Python Doesn't Understand You

Developers often get frustrated with programming languages and computers because they seemingly go out of their way to cause communication problems. Of course, programming languages and computers are both inanimate — they don't "want" anything. Programming languages and computers also don't think; they literally accept whatever the developer says. Therein lies the problem.

REMEMBER

Neither Python nor the computer will "know what you mean" when you type instructions as code. Both follow whatever instructions you provide to the letter and literally as you provide them. You may not have meant to tell Python to delete a data file unless some absurd condition occurred. However, if you don't make the conditions clear, Python will delete the file whether the condition exists or not. When an error of this sort happens, people commonly say that the application has a *bug* in it. Bugs are simply coding errors that you can remove by using a debugger. (A *debugger* is a special kind of tool that lets you stop or pause application execution, examine the content of variables, and generally dissect the application to see what makes it tick.)

Errors occur in many cases when the developer makes assumptions that simply aren't true. Of course, this includes assumptions about the application user, who probably doesn't care about the extreme level of care you took when crafting your application. The user will enter bad data. Again, Python won't know or care that

the data is bad and will process it even when your intent was to disallow the bad input. Python doesn't understand the concepts of good or bad data; it simply processes incoming data according to any rules you set, which means that you must set rules to protect users from themselves.

Python isn't proactive or creative — those qualities exist only in the developer. When a network error occurs or the user does something unexpected, Python doesn't create a solution to fix the problem. It only processes code. If you don't provide code to handle the error, the application is likely to fail and crash ungracefully — possibly taking all the user's data with it. Of course, the developer can't anticipate every potential error situation, either, which is why most complex applications have errors in them — errors of omission, in this case.

WARNING

Some developers out there think they can create bulletproof code, despite the absurdity of thinking that such code is even possible. Smart developers assume that some number of bugs will get through the code-screening process, that nature and users will continue to perform unexpected actions, and that even the smartest developer can't anticipate every possible error condition. Always assume that your application is subject to errors that will cause exceptions; that way, you'll have the mindset required to actually make your application more reliable. Keeping Murphy's Law, "If anything can go wrong, it will" in mind will help more than you think. (See more about Murphy's laws at `https://people.howstuffworks.com/murphys-law.htm`.) In addition, you must consider that some low-probability events can have a high impact on your software. They're called black-swan events (`https://betterprogramming.pub/the-black-swan-events-in-distributed-systems-d6a5d51adddf`), and you need to be prepared for them as well.

Considering the Sources of Errors

You might be able to divine the potential sources of error in your application by reading tea leaves, but that's hardly an efficient way to do things. Errors actually fall into well-defined categories that help you predict (to some degree) when and where they'll occur. By thinking about these categories as you work through your application, you're far more likely to discover potential error sources before they occur and cause potential damage. The two principle categories are

- Errors that occur at a specific time
- Errors that are of a specific type

The following sections discuss these two categories in greater detail. The overall concept is that you need to think about error classifications in order to start finding and fixing potential errors in your application before they become a problem.

Classifying when errors occur

Errors occur at specific times. The two major time frames are

- Compile time
- Runtime

No matter when an error occurs, it causes your application to misbehave. The following sections describe each time frame.

Compile time

A compile time error occurs when you ask Python to run the application. Before Python can run the application, it must interpret the code and put it into a form that the computer can understand. A computer relies on machine code that is specific to that processor and architecture. If the instructions you write are malformed or lack needed information, Python can't perform the required conversion. It presents an error that you must fix before the application can run.

Fortunately, compile-time errors are the easiest to spot and fix. Because the application won't run with a compile-time error in place, the user never sees this error category. You fix this sort of error as you write your code.

TIP

The appearance of a compile-time error should tell you that other typos or omissions could exist in the code. It always pays to check the surrounding code to ensure that no other potential problems exist that might not show up as part of the compile cycle.

CONSIDERING THE EFFECTS OF LIBRARY/ PACKAGE UPDATES ON YOUR CODE

Python relies on a huge number of third-party libraries to perform tasks. A *library* is code that someone else puts together and then you use it with your application so that you don't have to write so much code. These libraries are put into a convenient form called a *package,* as sort of a gift from one programmer to another. As of this writing, there are 3,409,411 released packages listed at `https://pypi.org/` for use with the Python programming language. There are 659 packages available for use with email projects (see Chapter 17) alone. So before you write extensive code to perform a specific task, consider the fact that someone else has probably already written the code for you.

Libraries are usually a good addition to your programming toolbox because someone else debugs them for you, so you have fewer errors to worry about. However, it's this whole debugging thing that causes some problems. The code that worked fine with library version 1.0 may not work with library version 2.0 because the updated library fixed problems that your code depends on to work. A fix of this sort is called a *breaking change* because it breaks existing code and causes it to malfunction. Consequently, your code can work one day, but not the next, when something you install updates the libraries to the new version for you without your knowledge. Yes, other programmers can ruin your day by doing the right thing and fixing bugs in their code. Chapter 11 tells you about importing packages (libraries) into your code and using them.

TECHNICAL STUFF

Some update errors also occur in the Python language itself. For example, when working with Python 2, you could create a statement that uses the statement version of `print` like this: `print "Hello World"`. In Python 3, you must use the function version of `print()`, like this: `print("Hello World")`. The update information at `https://docs.python.org/3/whatsnew/3.0.html` tells you more about this change (and others).

Runtime

A runtime error occurs after Python compiles the code you write and the computer begins to execute it. Runtime errors come in several different types, and some are harder to find than others. You know you have a runtime error when the application suddenly stops running and displays an exception dialog box or when the user complains about erroneous output (or at least instability).

REMEMBER

Not all runtime errors produce an exception. Some runtime errors cause instability (the application freezes), errant output, or data damage. Runtime errors can affect other applications or create unforeseen damage to the platform on which the application is running. In short, runtime errors can cause you quite a bit of grief, depending on precisely the kind of error you're dealing with at the time.

Many runtime errors are caused by errant code. For example, you can misspell the name of a variable, preventing Python from placing information in the correct variable during execution. Leaving out an optional (it's not absolutely required in most cases) but necessary (it is required in this specific instance) argument when calling a method can also cause problems. These are examples of *errors of commission*, which are specific errors associated with your code. In general, you can find these kinds of errors by using a debugger or by simply reading your code line by line to check for errors.

Runtime errors can also be caused by external sources not associated with your code (see the "Considering the effects of library/package updates on your code" sidebar for additional information about third-party code). For example, the user can input incorrect information that the application isn't expecting, causing an exception. A network error can make a required resource inaccessible. Sometimes even the computer hardware has a glitch that causes a nonrepeatable application error. These are all examples of *errors of omission*, from which the application might recover if your application has error-trapping code in place. It's important that you consider both kinds of runtime errors — errors of commission and omission — when building your application.

Distinguishing error types

You can distinguish errors by type, that is, by how they're made. Knowing the error types helps you understand where to look in an application for potential problems. Exceptions work like many other things in life. For example, you know that electronic devices don't work without power. So, when you try to turn your television on and it doesn't do anything, you might look to ensure that the power cord is firmly seated in the socket.

TIP

Understanding the error types helps you locate errors faster, earlier, and more consistently, resulting in fewer misdiagnoses. The best developers know that fixing errors while an application is in development is always easier than fixing it when the application is in production because users are inherently impatient and want errors fixed immediately and correctly. In addition, fixing an error earlier in the development cycle is always easier than fixing it when the application nears completion because less code exists to review.

The trick is to know where to look. With this in mind, Python (and most other programming languages) breaks errors into the following types:

- Syntactical
- Semantic
- Logical

The following sections examine each of these error types in more detail. I've arranged the sections in order of difficulty, starting with the easiest to find. A syntactical error is generally the easiest; a logical error is generally the hardest.

Syntactical

Whenever you make a typo of some sort, you create a syntactical error. Some Python syntactical errors are quite easy to find because the application simply

doesn't run. The interpreter may even point out the error for you by highlighting the errant code and displaying an error message. However, some syntactical errors are quite hard to find. Python is case sensitive, so you may use the wrong case for a variable in one place and find that the variable isn't quite working as you thought it would. Finding the one place where you used the wrong capitalization can be quite challenging.

REMEMBER

Most syntactical errors occur at compile time and the interpreter points them out for you. Fixing the error is made easy because the interpreter generally tells you what to fix, and with considerable accuracy. Even when the interpreter doesn't find the problem, syntactical errors prevent the application from running correctly, so any errors the interpreter doesn't find show up during the testing phase. Few syntactical errors should make it into production as long as you perform adequate application testing.

Semantic

When you create a loop that executes one too many times, you don't generally receive any sort of error information from the application. The application will happily run because it thinks that it's doing everything correctly, but that one additional loop can cause all sorts of data errors. When you create an error of this sort in your code, it's called a *semantic error.*

REMEMBER

Semantic errors occur because the meaning behind a series of steps used to perform a task is wrong — the result is incorrect even though the code apparently runs precisely as it should. Semantic errors are tough to find, and you sometimes need some sort of debugger to find them. (Chapter 20 provides a discussion of tools that you can use with Python to perform tasks such as debugging applications. You can also find blog posts about debugging on my blog at `http://blog.johnmuellerbooks.com`.)

Logical

Some developers don't create a division between semantic and logical errors, but they are different. A semantic error occurs when the code is essentially correct but the implementation is wrong (such as having a loop execute once too often). Logical errors occur when the developer's thinking is faulty. In many cases, this sort of error happens when the developer uses a relational or logical operator incorrectly. However, logical errors can happen in all sorts of other ways, too. For example, a developer might think that data is always stored on the local hard drive, which means that the application may behave in an unusual manner when it attempts to load data from a network drive instead.

REMEMBER

Logical errors are quite hard to fix because the problem isn't with the actual code, yet the code itself is incorrectly defined. The thought process that went into creating the code is faulty; therefore, the developer who created the error is less likely to find it. Smart developers use a second pair of eyes to help spot logical errors. Having a formal application specification also helps because the logic behind the tasks the application performs is usually given a formal review.

Catching Exceptions

Generally speaking, a user should never see an exception dialog box. Your application should always catch the exception and handle it before the user sees it. Of course, the real world is different — users do see unexpected exceptions from time to time. However, catching every potential exception is still the goal when developing an application. The following sections describe how to catch exceptions and handle them.

UNDERSTANDING THE BUILT-IN EXCEPTIONS

Python comes with a host of built-in exceptions — far more than you might think possible. You can see a list of these exceptions at `https://docs.python.org/3.10/library/exceptions.html`. The documentation breaks the exception list down into categories (see the exception hierarchy at `https://docs.python.org/3.10/library/exceptions.html#exception-hierarchy`). Here is a brief overview of the Python exception categories that you work with regularly:

- **Base classes:** The base classes provide the essential building blocks (such as the `Exception` exception) for other exceptions. However, you might actually see some of these exceptions, such as the `ArithmeticError` exception, when working with an application.
- **Concrete exceptions:** Applications can experience hard errors — errors that are hard to overcome because there really isn't a good way to handle them or they signal an event that the application must handle. For example, when a system runs out of memory, Python generates a `MemoryError` exception. Recovering from this error is hard because it isn't always possible to release memory from other uses. When the user presses an interrupt key (such as Ctrl+C or Delete), Python generates a `KeyboardInterrupt` exception. The application must handle this exception before proceeding with any other tasks.

- **OS exceptions:** The operating system can generate errors that Python then passes along to your application. For example, if your application tries to open a file that doesn't exist, the operating system generates a `FileNotFoundError` exception.
- **Warnings:** Python tries to warn you about unexpected events or actions that could result in errors later. For example, if you try to inappropriately use a resource, such as an icon, Python generates a `ResourceWarning` exception. You want to remember that this particular category is a warning and not an actual error: Ignoring it can cause you woe later, but you can ignore it.

Basic exception handling

To handle exceptions, you must tell Python that you want to do so and then provide code to perform the handling tasks. You have a number of ways in which you can perform this task. The following sections start with the simplest method first and then move on to more complex methods that offer added flexibility.

Handling a single exception

In Chapter 8, the various examples have a terrible habit of spitting out exceptions when the user inputs unexpected values. Part of the solution is to provide range checking. However, range checking doesn't overcome the problem of a user typing text such as `Hello` in place of an expected numeric value. Exception handling provides a more complex solution to the problem, as described in the following steps.

1. **Open a new notebook.**

   You can also use the downloadable source file, `BPP4D3E; 10; Dealing with Errors.ipynb`.

2. **Type the following code into the notebook — pressing Enter after each line:**

```
try:
   Value = int(input("Type a number between 1 and 10: "))
except ValueError:
   print("You must type a number between 1 and 10!")
else:
   if (Value > 0) and (Value <= 10):
      print("You typed: ", Value)
   else:
      print("The value you typed is incorrect!")
```

The code within the `try` block has its `ValueError` exception handled. In this case, handling the exception means getting input from the user by using the `int(input())` calls. If an exception occurs outside this block, the code doesn't handle it. With reliability in mind, the temptation might be to enclose all the executable code in a `try` block so that every exception would be handled. However, you want to keep your exception handling small and specific to make locating the problem easier.

REMEMBER

The `except` block looks for a specific exception in this case: `ValueError`. When the user creates a `ValueError` exception by typing `Hello` instead of a numeric value, this particular exception block is executed. If the user were to generate some other exception, this `except` block wouldn't handle it.

The `else` block contains all the code that is executed when the `try` block code is successful (doesn't generate an exception). The remainder of the code is in this block because you don't want to execute it unless the user does provide valid input. When the user provides a whole number as input, the code can then range check it to ensure that it's correct.

3. **Click Run Cell.**

   Python asks you to type a number between 1 and 10.

4. **Type** Hello **and press Enter.**

   The application displays an error message, as shown in Figure 10-1.

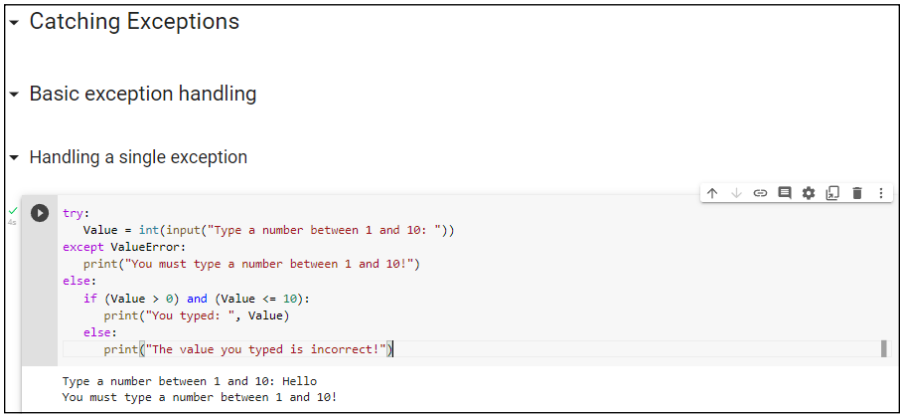


FIGURE 10-1: Typing the wrong input type generates an error message instead of an exception.

5. **Perform Steps 3 and 4 again, but type** 5.5 **instead of Hello.**

   The application generates the same error message, as shown in Figure 10-1.

6. **Perform Steps 3 and 4 again, but type** 22 **instead of Hello.**

   The application outputs the expected range error message, as shown in Figure 10-2. Exception handling doesn't weed out range errors. You must still check for them separately.

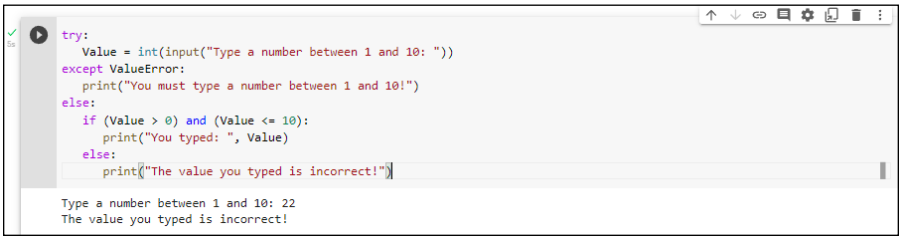


FIGURE 10-2: Exception handling doesn't ensure that the value is in the correct range.

7. **Perform Steps 3 and 4 again, but type** 7 **instead of Hello.**

   This time, the application finally reports that you've provided a correct value of 7. Even though it seems like a lot of work to perform this level of checking, you can't really be certain that your application is working correctly without it.

Using the except clause without an exception

You can create an exception handling block in Python that's generic because it doesn't look for a specific exception. In most cases, you want to provide a specific exception when performing exception handling for these reasons:

- To avoid hiding an exception you didn't consider when designing the application
- To ensure that others know precisely which exceptions your application will handle
- To handle the exceptions correctly by using specific code for that exception

However, sometimes you may need a generic exception-handling capability, such as when you're working with third-party libraries or interacting with an external service. The following steps demonstrate how to use an `except` clause without a specific exception attached to it.

1. **Type the following code into a new cell of the notebook — pressing Enter after each line:**

```
try:
   Value = int(input("Type a number between 1 and 10: "))
```

```
except:
   print("This is the generic error!")
except ValueError:
   print("You must type a number between 1 and 10!")
else:
   if (Value > 0) and (Value <= 10):
      print("You typed: ", Value)
   else:
      print("The value you typed is incorrect!")
```

The only difference between this example and the previous example is that the `except` clause doesn't have the `ValueError` exception specifically associated with it. The result is that this `except` clause will also catch any other exception that occurs.

2. **Click Run Cell.**

You see the error message shown in Figure 10-3. Python automatically detects that you have placed the exception handlers in the wrong order. (You discover more about this issue in the "Handling more specific to less specific exceptions" section, later in the chapter.) Notice that the errant lines have red, squiggly lines under them in Colab so that you can see the error with less effort.

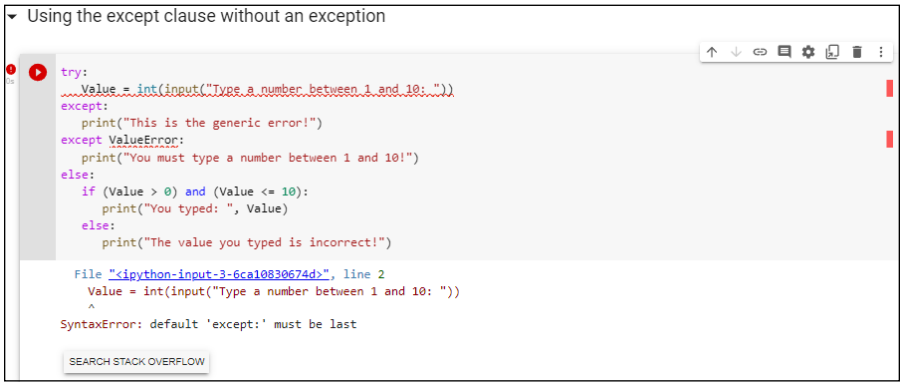


FIGURE 10-3: The exception handlers are in the wrong order.

Type the following code in a new cell to reverse the order of the two exceptions so that they appear like this:

```
try:
   Value = int(input("Type a number between 1 and 10: "))
except ValueError:
   print("You must type a number between 1 and 10!")
except:
   print("This is the generic error!")
```

```
else:
   if (Value > 0) and (Value <= 10):
      print("You typed: ", Value)
   else:
      print("The value you typed is incorrect!")
```

3. **Click Run Cell.**

   Python asks you to type a number between 1 and 10.

4. **Type** Hello **and press Enter.**

   The application displays an error message (refer to Figure 10-1). When the exceptions are in the right order, the code detects specific errors first and then uses less specific handlers only when necessary.

5. **Click Run Cell.**

   Python asks you to type a number between 1 and 10.

6. **Choose Runtime ➪ Interrupt Execution (when using Colab) or Kernel ➪ Interrupt (when using Jupyter Notebook).**

   This act is akin to pressing Ctrl+C or Cmd+C in other IDEs. However, nothing actually appears to happen. If you're working with Jupyter Notebook, you can look at the server window to see a `Kernel Interrupted` message. In addition, you see the generic error message. Python is still waiting for you to enter a value in the `input()` textbox.

7. **Select the input() textbox (if necessary), type** 5.5**, and press Enter.**

   You still see the generic error message as before, now shown in Figure 10-4 because Notebook is reacting to the interrupt, rather than the incorrect input; the reason is that the interrupt came first. Python queues errors in the order in which it receives them. Consequently, you may find that an application outputs what appears to be the wrong error message at times.

FIGURE 10-4: Generic exception handling traps the KeyboardInterrupt exception.

8. **Perform Steps 3 and 4 again, but type** 5.5 **instead of Hello.**

   The application generates the same error message as before (again, refer to Figure 10-1). In this case, no interrupt occurred, so you see the error message you expected.

Working with exception arguments

Most exceptions don't provide arguments (a list of values that you can check for additional information). The exception either occurs or it doesn't. However, a few exceptions do provide arguments, and you see them used later in the book. The arguments tell you more about the exception and provide details that you need to correct it.

For the sake of completeness, this chapter includes a simple example that generates an exception with an argument. You can safely skip the remainder of this section if desired because the information is covered in more detail later in the book.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
import sys
try:
   File = open("myfile.txt")
except IOError as e:
   print("Error opening file!\r\n" +
      "Error Number: {0}\r\n".format(e.errno) +
      "Error Text: {0}".format(e.strerror))
else:
   print("File opened as expected.")
   File.close()
```

   This example uses some advanced features. The `import` statement obtains code from another file. Chapter 11 tells you how to use this Python feature.

   The `open()` function opens a file and provides access to the file through the `File` variable. Chapter 16 tells you how file access works. Given that `myfile.txt` doesn't exist in the application directory, the operating system can't open it and will tell Python that the file doesn't exist.

   Trying to open a nonexistent file generates an `IOError` exception. This particular exception provides access to two arguments:

   - `errno`: Provides the operating system error number as an integer
   - `strerror`: Contains the error information as a human-readable string

The as clause places the exception information into a variable, e, that you can access as needed for additional information. The except block contains a print() call that formats the error information into an easily read error message.

If you should decide to create the myfile.txt file, the else clause executes. In this case, you see a message stating that the file opened normally. The code then closes the file without doing anything with it.

2. **Click Run Cell.**

The application displays the Error opening file information, as shown in Figure 10-5.

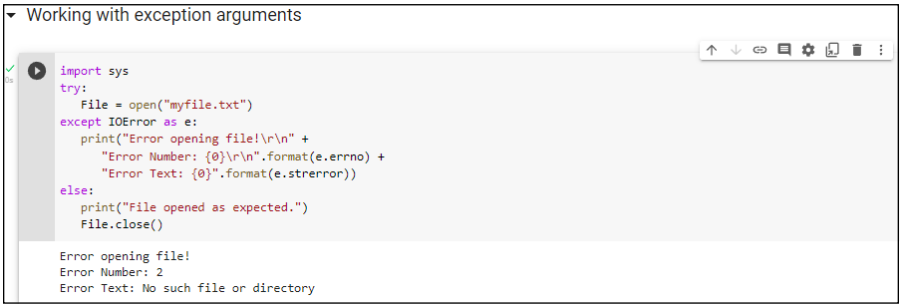


FIGURE 10-5: Attempting to open a nonexistent file never works.

OBTAINING A LIST OF EXCEPTION ARGUMENTS

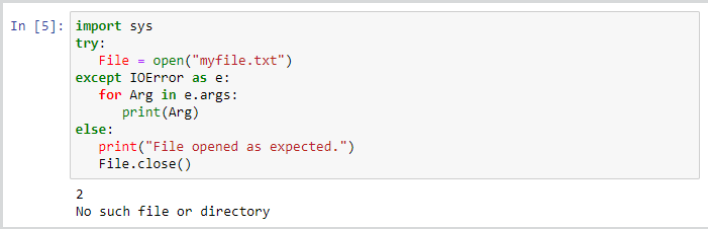
The list of arguments supplied with exceptions varies by exception and by what the sender provides. It isn't always easy to figure out what you can hope to obtain in the way of additional information. One way to handle the problem is to simply print everything by using code like this:

```
import sys
try:
   File = open("myfile.txt")
except IOError as e:
   for Arg in e.args:
      print(Arg)
else:
   print("File opened as expected.")
   File.close()
```

(continued)

(continued)

The `args` property always contains a list of the exception arguments in string format as shown in the following screenshot. You can use a simple `for` loop to print each of the arguments. The only problem with this approach is that you're missing the argument names, so you know the output information (which is obvious in this case), but you don't know what to call it.



A more complex method of dealing with the issue is to print both the names and the contents of the arguments. The following code displays both the names and the values of each of the arguments:

```
import sys
try:
   File = open("myfile.txt")
except IOError as e:
   for Entry in dir(e):
      if (not Entry.startswith("_")):
         try:
            print(Entry, " = ", e.__getattribute__(Entry))
         except AttributeError:
            print("Attribute ", Entry, " not accessible.")
else:
   print("File opened as expected.")
   File.close()
```

In this case, you begin by getting a listing of the attributes associated with the error argument object using the `dir()` function. The output of the `dir()` function is a list of strings containing the names of the attributes that you can print as shown in the following screenshot. Only those arguments that don't start with an underscore (_) contain useful information about the exception. However, even some of those entries are inaccessible, so you must encase the output code in a second `try ... except` block (see the "Nested exception handling" section, later in the chapter, for details).

```
In [6]: import sys
        try:
           File = open("myfile.txt")
        except IOError as e:
           for Entry in dir(e):
              if (not Entry.startswith("_")):
                 try:
                    print(Entry, " = ", e.__getattribute__(Entry))
                 except AttributeError:
                    print("Attribute ", Entry, " not accessible.")
        else:
           print("File opened as expected.")
           File.close()

        args  =  (2, 'No such file or directory')
        Attribute  characters_written  not accessible.
        errno  =  2
        filename  =  myfile.txt
        filename2  =  None
        strerror  =  No such file or directory
        winerror  =  None
        with_traceback  =  <built-in method with_traceback of FileNotFoundError object
        at 0x000001EA39EE0440>
```

The attribute name is easy because it's contained in `Entry`. To obtain the value associated with that attribute, you must use the `__getattribute()` function and supply the name of the attribute you want. When you run this code, you see both the name and the value of each of the attributes supplied with a particular error argument object. In this case, the actual output is as follows:

```
args  =  (2, 'No such file or directory')
Attribute  characters_written  not accessible.
errno  =  2
filename  =  myfile.txt
filename2  =  None
strerror  =  No such file or directory
with_traceback  =  <built-in method with_traceback of
   FileNotFoundError object at 0x7f52bf07f5f0>
```

Handling multiple exceptions with a single except clause

Most applications can generate multiple exceptions for a single line of code. This fact is demonstrated in the "Using the except clause without an exception" section of the chapter. How you handle the multiple exceptions depends on your goals for the application, the types of exceptions, and the relative skill of your users. Sometimes when working with a less skilled user, it's simply easier to say that the application experienced a nonrecoverable error and then log the details into a log file in the application directory or a central location.

REMEMBER

Using a single `except` clause to handle multiple exceptions works only when a common action fulfills the needs of all the exception types. Otherwise, you need to handle each exception individually. The following steps show how to handle multiple exceptions by using a single `except` clause.

1. **Type the following code into a new cell of the notebook — pressing Enter after each line:**

```
try:
   Value = int(input("Type a number between 1 and 10: "))
except (ValueError, KeyboardInterrupt):
   print("You must type a number between 1 and 10!")
else:
   if (Value > 0) and (Value <= 10):
      print("You typed: ", Value)
   else:
      print("The value you typed is incorrect!")
```

REMEMBER

Note that the `except` clause now sports both a `ValueError` and a `KeyboardInterrupt` exception. These exceptions appear within parentheses and are separated by commas.

2. **Click Run Cell.**

   Python asks you to type a number between 1 and 10.

3. **Type** Hello **and press Enter.**

   The application displays an error message (refer to Figure 10-1).

4. **Click Run Cell.**

   Python asks you to type a number between 1 and 10.

5. **Choose Runtime ➪ Interrupt Execution for Colab or Kernel ➪ Interrupt for Jupyter Notebook.**

   This act is akin to pressing Ctrl+C or Cmd+C in other IDEs.

6. **Click Run Cell, type** 5.5**, and press Enter.**

   The application displays an error message (refer to Figure 10-1).

7. **Perform Steps 2 and 3 again, but type** 7 **instead of Hello.**

   This time, the application finally reports that you've provided a correct value of 7.

Handling multiple exceptions with multiple except clauses

When working with multiple exceptions, it's usually a good idea to place each exception in its own `except` clause. This approach allows you to provide custom handling for each exception and makes it easier for the user to know precisely what went wrong. Of course, this approach is also a lot more work. The following steps demonstrate how to perform exception handling by using multiple `except` clauses.

1. **Type the following code into a new cell of the notebook — pressing Enter after each line:**

```
try:
   Value = int(input("Type a number between 1 and 10: "))
except ValueError:
   print("You must type a number between 1 and 10!")
except KeyboardInterrupt:
   print("You pressed Ctrl+C!")
else:
   if (Value > 0) and (Value <= 10):
      print("You typed: ", Value)
   else:
      print("The value you typed is incorrect!")
```

REMEMBER

Notice the use of multiple `except` clauses in this case. Each `except` clause handles a different exception. You can use a combination of techniques, with some `except` clauses handling just one exception and other `except` clauses handling multiple exceptions. Python lets you use the approach that works best for the error-handling situation.

2. **Click Run Cell.**

   Python asks you to type a number between 1 and 10.

3. **Type** Hello **and press Enter.**

   The application displays an error message (refer to Figure 10-1).

4. **Perform Steps 2 and 3 again, but type** 22 **instead of Hello.**

   The application outputs the expected range error message (refer to Figure 10-2).

5. **Perform Steps 2 and 3 again, but choose Runtime ➪ Interrupt Execution for Colab or Kernel ➪ Interrupt for Jupyter Notebook.**

   The application outputs a specific message, You pressed Ctrl+C, that tells the user what went wrong.

6. **Perform Steps 2 and 3 again, but type `7` instead of Hello.**

   This time, the application finally reports that you've provided a correct value of 7.

Handling more specific to less specific exceptions

One strategy for handling exceptions is to provide specific `except` clauses for all known exceptions and generic `except` clauses to handle unknown exceptions. You can see the exception hierarchy that Python uses at `https://docs.python.org/3/library/exceptions.html#exception-hierarchy`. When viewing this chart, `BaseException` is the uppermost exception. Most exceptions are derived from `Exception`. When working through math errors, you can use the generic `ArithmeticError` or a more specific `ZeroDivisionError` exception.

Python evaluates `except` clauses in the order in which they appear in the source code file. The first clause is examined first, the second clause is examined second, and so on. The following steps help you examine an example that demonstrates the importance of using the correct exception order. In this case, you perform tasks that result in math errors.

1. **Type the following code into a new cell of the notebook — pressing Enter after each line:**

```
try:
   Value1 = int(input("Type the first number: "))
   Value2 = int(input("Type the second number: "))
   Output = Value1 / Value2
except ValueError:
   print("You must type a whole number!")
except KeyboardInterrupt:
   print("You pressed Ctrl+C!")
except ArithmeticError:
   print("An undefined math error occurred.")
except ZeroDivisionError:
   print("Attempted to divide by zero!")
else:
   print(Output)
```

   The code begins by obtaining two inputs: `Value1` and `Value2`. The first two `except` clauses handle unexpected input. The second two `except` clauses handle math exceptions, such as dividing by zero. If everything goes well with the application, the `else` clause executes, which prints the result of the operation.

2. **Click Run Cell.**

   Python asks you to type the first number.

3. **Type** Hello **and press Enter.**

   As expected, Python displays the `ValueError` exception message. However, it always pays to check for potential problems.

4. **Click Run Cell again.**

   Python asks you to type the first number.

5. **Type** 8 **and press Enter.**

   The application asks you to enter the second number.

6. **Type** 0 **and press Enter.**

   You see the error message for the `ArithmeticError` exception, as shown in Figure 10-6. What you should actually see is the `ZeroDivisionError` exception because it's more specific than the `ArithmeticError` exception.

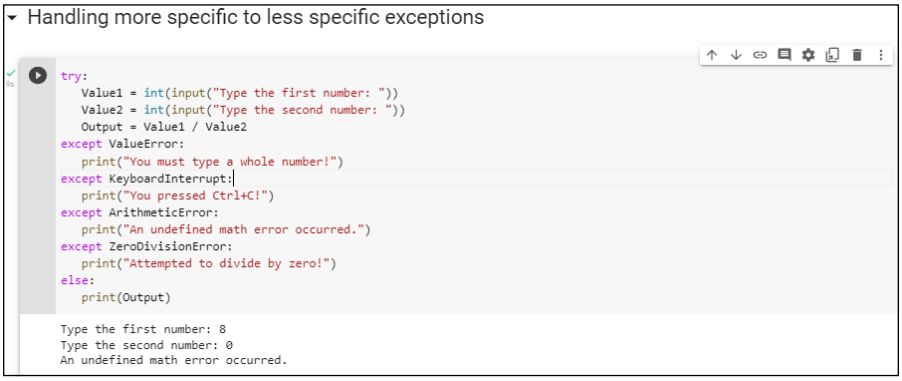


FIGURE 10-6: The order in which Python processes exceptions is important.

7. **Reverse the order of the two exceptions so that they look like this:**

```
except ZeroDivisionError:
   print("Attempted to divide by zero!")
except ArithmeticError:
   print("An undefined math error occurred.")
```

8. **Perform Steps 4 through 6 again.**

   This time, you see the `ZeroDivisionError` exception message because the exceptions appear in the correct order.

9. **Perform Steps 4 through 5 again, but type 2 for the second number instead of 0.**

   This time, the application finally reports an output value of 4.0.

REMEMBER

Notice that the output is a floating-point value. Division results in a floating-point value unless you specify that you want an integer output by using the floor division operator (//).

Nested exception handling

Sometimes you need to place one exception-handling routine within another in a process called *nesting*. When you nest exception-handling routines, Python tries to find an exception handler in the nested level first and then moves to the outer layers. You can nest exception-handling routines as deeply as needed to make your code safe.

One of the more common reasons to use a dual layer of exception-handling code is when you want to obtain input from a user and need to place the input code in a loop to ensure that you actually get the required information. The following steps demonstrate how this sort of code might work.

1. **Type the following code into a new cell of the notebook — pressing Enter after each line:**

```
TryAgain = True
while TryAgain:
   try:
      Value = int(input("Type a whole number. "))
   except ValueError:
      print("You must type a whole number!")
      try:
         DoOver = input("Try again (y/n)? ")
      except:
         print("OK, see you next time!")
         TryAgain = False
      else:
         if (str.upper(DoOver) == "N"):
            TryAgain = False
   except KeyboardInterrupt:
      print("You pressed Ctrl+C!")
      print("See you next time!")
      TryAgain = False
   else:
      print(Value)
      TryAgain = False
```

The code begins by creating an input loop. Using loops for this type of purpose is actually quite common in applications because you don't want the application to end every time an input error is made. This is a simplified loop, and normally you create a separate function to hold the code.

When the loop starts, the application asks the user to type a whole number. It can be any integer value. If the user types any non-integer value or presses Ctrl+C, Cmd+C, or another interrupt key combination, the exception-handling code takes over. Otherwise, the application prints the value that the user supplied and sets `TryAgain` to `False`, which causes the loop to end.

A `ValueError` exception can occur when the user makes a mistake. Because you don't know why the user input the wrong value, you have to ask if the user wants to try again. Of course, getting more input from the user could generate another exception. The inner `try...except` code block handles this secondary input.

TIP

Notice the use of the `str.upper()` function when getting character input from the user. This function makes it possible to receive `y` or `Y` as input and accept them both. Whenever you ask the user for character input, converting lowercase characters to uppercase is a good idea so that you can perform a single comparison (reducing the potential for error).

REMEMBER

The `KeyboardInterrupt` exception displays two messages and then exits automatically by setting `TryAgain` to `False`. The `KeyboardInterrupt` occurs only when the user presses a specific key combination designed to end the application. The user is unlikely to want to continue using the application at this point.

2. **Click Run Cell.**

   Python asks the user to input a whole number.

3. **Type** Hello **and press Enter.**

   The application displays an error message and asks whether you want to try again.

4. **Type** Y **and press Enter.**

   The application asks you to input a whole number again, as shown in Figure 10-7.

5. **Type** 5.5 **and press Enter.**

   The application again displays the error message and asks whether you want to try again.

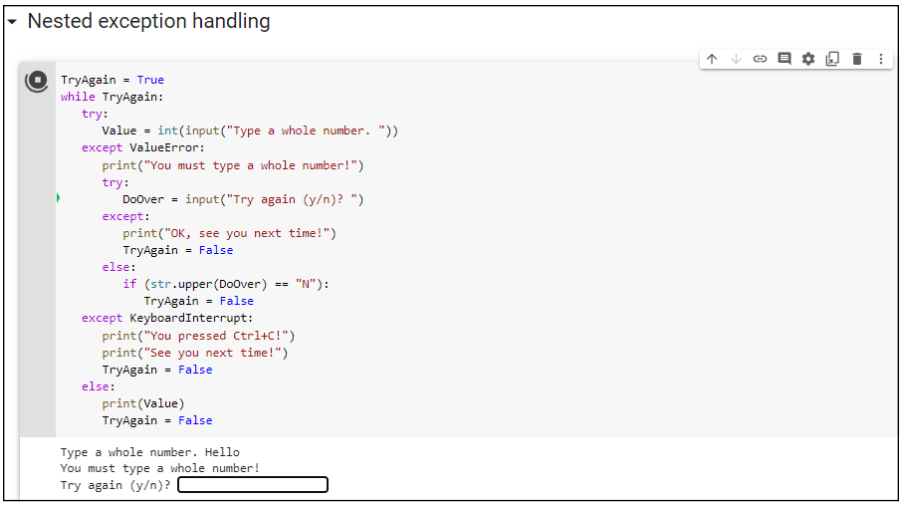


FIGURE 10-7: Using a loop means that the application can recover from the error.

6. **Choose Runtime ➪ Interrupt Execution for Colab or Kernel ➪ Interrupt for Jupyter Notebook to interrupt the application.**

   The application ends by displaying `OK, see you next time!`. Notice that the message is the one from the inner exception. The application never gets to the outer exception because the inner exception handler provides generic exception handling.

7. **Click Run Cell.**

   Python asks the user to input a whole number.

8. **Choose Kernel ➪ Interrupt to interrupt the application, type** 5.5, **and then press Enter.**

   The application ends by displaying `You pressed Ctrl+C!`. Notice that the message is the one from the outer exception. In Steps 6 and 8, the user ends the application by pressing an interrupt key. However, the application uses two different exception handlers to address the problem.

Raising Exceptions

So far, the examples in this chapter have reacted to exceptions. Something happens and the application provides error-handling support for that event. However, situations arise for which you may not know how to handle an error event during the application design process. Perhaps you can't even handle the error at a particular level and need to pass it up to some other level to handle. In short, in

some situations, your application must generate an exception. This act is called *raising* (or sometimes *throwing*) the exception. The following sections describe common scenarios in which you raise exceptions in specific ways.

Raising exceptions during exceptional conditions

The example in this section demonstrates how you raise a simple exception — that it doesn't require anything special. The following steps simply create the exception and then handle it immediately.

1. **Type the following code into a new cell of the notebook — pressing Enter after each line:**

```
try:
   raise ValueError
except ValueError:
   print("ValueError Exception!")
```

   You wouldn't ever actually create code that looks like this, but it shows you how raising an exception works at its most basic level. In this case, the `raise` call appears within a `try . . . except` block. A basic raise call simply provides the name of the exception to raise (or throw). You can also provide arguments as part of the output to provide additional information.

REMEMBER

   Notice that this `try . . . except` block lacks an `else` clause because there is nothing to do after the call. Although you rarely use a `try . . . except` block in this manner, you can. You may encounter situations like this one sometimes and need to remember that adding the `else` clause is purely optional. On the other hand, you must add at least one `except` clause.

2. **Click Run Cell.**

   Python displays the expected exception text, `ValueError Exception!`.

Passing error information to the caller

Python provides exceptionally flexible error handling in that you can pass information to the *caller* (the code that is calling your code) no matter which exception you use. Of course, the caller may not know that the information is available, which leads to a lot of discussion on the topic. If you're working with someone else's code and don't know whether additional information is available, you can always use the technique described in the "Obtaining a list of exception arguments" sidebar, earlier in this chapter, to find it.

You may have wondered whether you could provide better information when working with a `ValueError` exception than with an exception provided natively by Python. The following steps show that you can modify the output so that it does include helpful information.

1. **Type the following code into a new cell of the notebook — pressing Enter after each line:**

```
try:
   Ex = ValueError()
   Ex.strerror = "Value must be within 1 and 10."
   raise Ex
except ValueError as e:
   print("ValueError Exception!", e.strerror)
```

   The `ValueError` exception normally doesn't provide an attribute named `strerror` (a common name for string error), but you can add it simply by assigning a value to it as shown. When the example raises the exception, the `except` clause handles it as usual but obtains access to the attributes using `e`. You can then access the `e.strerror` member to obtain the added information.

2. **Click Run Cell.**

   Python displays an expanded `ValueError` exception, as shown in Figure 10-8.

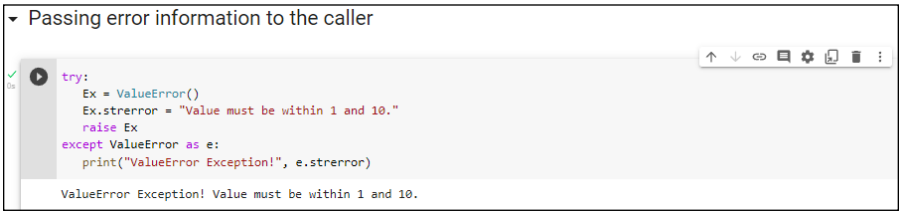


FIGURE 10-8: You can add error information to any exception.

Deciding to Say "Oops" in Your Own Way: Custom Exceptions

Python provides a wealth of standard exceptions that you should use whenever possible. These exceptions are incredibly flexible, and you can even modify them as needed (within reason) to meet specific needs. For example, the "Passing error information to the caller" section of this chapter demonstrates how to modify a `ValueError` exception to allow for additional data. However, sometimes you

simply must create a custom exception because none of the standard exceptions will work. Perhaps the exception name just doesn't tell the viewer the purpose that the exception serves. You may need a custom exception for specialized database work or when working with a service.

WARNING

The example in this section is going to seem a little complicated for now because you haven't worked with classes before. Chapter 15 introduces you to classes and helps you understand how they work. If you want to skip this section until after you read Chapter 15, you can do so without any problem.

The example in this section shows a quick method for creating your own exceptions. To perform this task, you must create a class that uses an existing exception as a starting point. To make things a little easier, this example creates an exception that builds upon the functionality provided by the `ValueError` exception. The advantage of using this approach rather than the one shown in the preceding section, "Passing error information to the caller," is that this approach tells anyone who follows you precisely what the addition to the `ValueError` exception is. It also makes the modified exception easier to use.

1. **Type the following code into the notebook — pressing Enter after each line:**

```
class CustomValueError(ValueError):
   def __init__(self, arg):
      self.strerror = arg
      self.args = {arg}
try:
   raise CustomValueError("Value must be within 1 and 10.")
except CustomValueError as e:
   print("CustomValueError Exception!", e.strerror)
```

This example essentially replicates the functionality of the example in the "Passing error information to the caller" section of the chapter. However, it places the same error in both `strerror` and `args` so that the developer has access to either (as would normally happen).

The code begins by creating the `CustomValueError` class that uses the `ValueError` exception class as a starting point. The `__init__()` function provides the means for creating a new instance of that class. Think of the class as a blueprint and the instance as the building created from the blueprint.

REMEMBER

Notice that the `strerror` attribute has the value assigned directly to it, but `args` receives it as an array. The `args` member normally contains an array of all the exception values as strings, so this is standard procedure, even when `args` contains just one value as it does now.

The code for using the exception is considerably easier than modifying `ValueError` directly. All you do is call `raise` with the name of the exception and the arguments you want to pass, all on one line.

2. **Click Run Cell.**

   The application displays a similar error message to that shown in Figure 10-8, except this time it notes that it's a custom error message by saying, `CustomValueError Exception! Value must be within 1 and 10.`

Using the finally Clause

Normally you want to handle any exception that occurs in a way that doesn't cause the application to crash. However, sometimes you can't do anything to fix the problem, and the application is most definitely going to crash. At this point, your goal is to cause the application to crash gracefully, which means closing files so that the user doesn't lose data and performing other tasks of that nature. Anything you can do to keep damage to data and the system to a minimum is an essential part of handling data for a crashing application.

The `finally` clause is part of the crashing-application strategy. You use this clause to perform any required last-minute tasks. Normally, the `finally` clause is quite short and uses only calls that are likely to succeed without further problem. It's essential to close the files, log the user off, and perform other required tasks, and then let the application crash before something terrible happens (such as a total system failure). With this necessity in mind, the following steps show a simple example of using the `finally` clause:

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
import sys
try:
   raise ValueError
   print("Raising an exception.")
except ValueError:
   print("ValueError Exception!")
   sys.exit()
finally:
   print("Taking care of last minute details.")
print("This code will never execute.")
```

In this example, the code raises a `ValueError` exception. The `except` clause executes as normal when this happens. The call to `sys.exit()` means that the application exits after the exception is handled. Perhaps the application can't recover in this particular instance, but the application normally ends, which is why the final `print()` function call won't ever execute.

REMEMBER

The `finally` clause code always executes. It doesn't matter whether the exception happens or not. The code you place in this block needs to be common code that you always want to execute. For example, when working with a file, you place the code to close the file into this block to ensure that the data isn't damaged by remaining in memory rather than going to disk.

2. **Click Run Cell.**

The application displays the `except` clause message and the `finally` clause message, as shown in Figure 10-9. The `sys.exit()` call prevents any other code from executing.

REMEMBER

Note that this isn't a normal exit, so Jupyter Notepad displays additional information for you in the form of a traceback. Colab doesn't display this information, but you can display it by using `%tb`. Simply type **%tb** in an empty cell and click Run Cell to see the information. When you use some other IDEs, the application may simply exit without displaying any additional information.

3. **Comment out the `raise ValueErrorcall` by preceding it with two pound signs, like this:**

```
## raise ValueError
```

FIGURE 10-9: Use the finally clause to ensure that specific actions take place before the application ends.

Removing the exception will demonstrate how the `finally` clause actually works. Using two pound signs will make the commented code stand out and easier to search for.

4. **Click Run Cell.**

   The application displays a series of messages, including the `finally` clause message, as shown in Figure 10-10. This part of the example shows that the `finally` clause always executes, so you need to use it carefully.

```
import sys
try:
   ##raise ValueError
   print("Raising an exception.")
except ValueError:
   print("ValueError Exception!")
   sys.exit()
finally:
   print("Taking care of last minute details.")
print("This code will never execute.")
```

```
Raising an exception.
Taking care of last minute details.
This code will never execute.
```

FIGURE 10-10: Be sure to remember that the finally clause always executes.

3 Performing Common Tasks

IN THIS PART . . .

Using packages instead of coding

Adding language to applications

Creating lists of things to do

Defining collections of objects

Working with classes

IN THIS CHAPTER

- Organizing your code
- Adding code from outside sources
- Locating and viewing code libraries
- Obtaining and reading library documentation

Chapter 11
Interacting with Packages

The examples in this book are small, but the functionality of the resulting applications is extremely limited as well. Even tiny real-world applications contain thousands of lines of code. In fact, applications that contain millions of lines of code are somewhat common. Imagine trying to work with a file large enough to contain millions of lines of code — you'd never find anything. In short, you need some method to organize code into small pieces that are easier to manage, much like the examples in this book. The Python solution is to place code in separate code groupings called *packages.* Commonly used groupings that contain source code for generic needs and more than one package are called *libraries.*

REMEMBER

Packages are contained in separate files. To use the package, you must tell Python to grab the code and read it into the current application. The process of obtaining code found in external files is called *importing.* You import a package or library to use the code it contains. A few examples in the book have already shown the `import` statement in use, but this chapter explains the `import` statement in detail so that you know how to use it.

The library code is self-contained and well documented (at least in most cases it is). Some developers might feel that they never need to look at the library code, and they're right to some degree — you never have to look at the library code in order to use it. You might want to view the library code, though, to ensure that you understand how the code works. In addition, the library code can teach you new

programming techniques that you might not otherwise discover. So, viewing the library code is optional, but it can be helpful.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 11; Interacting with Packages.ipynb` and `BPP4D3E; 11; Packages.ipynb` files of the downloadable source. See the Introduction for details on how to find these source files.

Creating Code Groupings

Code groupings in the form of packages and libraries are part of working with even the simplest program in Python. Some libraries are automatically imported for you because you use them for every application. Other libraries require that you import them individually. As part of the initial setup, Python created a pointer to the general-purpose libraries it uses. That's why you can simply add an `import` statement with the name of the library, and Python can find it. However, it pays to know how to locate the files on disk in case you ever need to update them or want to add your own packages and libraries to the list of files that Python can use.

The one thing you do need to know how to do is obtain and use the Python library documentation. This chapter shows you how to obtain and use the library documentation as part of the application-creation process. In the sections that follow, you discover more about the code groupings that Python uses, including how to interact with them at a basic level.

WARNING

The example in the "Creating your first package" section that follows works well only with Jupyter Notebook. Google Colab users can still follow along, but the act of working online makes this particular example (the only one in the book) quite a challenge to complete. The problems are that you would need to download the file locally, rename it because Colab mangles the name, and then upload it to the correct directory online, which can be insanely difficult to find. Consequently, the steps will focus on Jupyter Notebook use in this one particular case.

I'm confused! Understanding modules versus packages

TECHNICAL STUFF

In some sources, you may see *modules* used in place of packages. People often use the two terms interchangeably because there really isn't much of a difference between them. Most sources define a *module* as a single file containing code with an extension of `.py`. A *package* is a group of modules often organized in a

hierarchical fashion (although, hierarchical organization isn't a language requirement). Even though there are small differences between the two, you can view both as methods of putting code into an easily used container, so this chapter uses "package" throughout because that's the term you see most often in the official documentation. If you want a more detailed view of the differences, you can read about it at `https://learnpython.com/blog/python-modules-packages-libraries-frameworks/`.

Creating your first package

Grouping like pieces of code together is important to make the code easier to use, modify, and understand. As an application grows, managing the code found in a single file becomes harder and harder. At some point, the code becomes impossible to manage because the file has become too large for anyone to work with.

REMEMBER

The term *code* is used broadly in this particular case. Code groupings can include:

- Classes
- Functions
- Variables
- Runnable code

The collection of classes, functions, variables, and runnable code within a package is known as *attributes.* A package has attributes that you access by that attribute's name. Later sections in this chapter discuss precisely how package access works.

TECHNICAL STUFF

The runnable code can actually be written in a language other than Python. For example, it's somewhat common to find packages that are written in C/C++ instead of Python. The reason that some developers use runnable code is to make the Python application faster, less resource intensive, and better able to use a particular platform's resources. However, using runnable code comes with the downside of making your application less portable (able to run on other platforms) unless you have runnable code packages for each platform that you want to support. In addition, dual-language applications can be harder to maintain because you must have developers who can speak each of the computer languages used in the application.

The most common way to create a package is to define one or more separate files containing the code you want to group separately from the rest of the application. For example, you might want to create a print routine that an application uses in

a number of places. The print routine isn't designed to work on its own but is part of the application as a whole. You want to separate it because the application uses it in numerous places and you could potentially use the same code in another application. The ability to reuse code ranks high on the list of reasons to create packages.

To make things easier to understand, the examples in this chapter use a common package. The package doesn't do anything too amazing, but it demonstrates the principles of working with packages. Open a Python 3 Notebook project, name it `BPP4D3E; 11; Packages`, and create the code shown in Listing 11-1. After you complete this task, download the code as a new Python file named `BPP4D3E; 11; Packagesl.py` by choosing File ➪ Download As ➪ Python (.py) in Notebook.

LISTING 11-1: A Simple Demonstration Package

```
def SayHello(Name):
   print("Hello ", Name)
   return
def SayGoodbye(Name):
   print("Goodbye ", Name)
   return
```

WARNING

This example specifically uses a Notebook filename that contains special characters and spaces to make a point. Any package you create can't have spaces or special characters in the filename and still work correctly. Consequently, you must rename `BPP4D3E; 11; Packagesl.py` to `BPP4D3E_11_Packages.py` if you want it to work with the remainder of the example in this chapter.

You may need to copy the resulting file to your existing `BPP4D3E` folder, depending on where your browser normally downloads files. When done correctly, your Notebook dashboard should contain a copy of the file, as shown in Figure 11-1. Using the Notebook's Import feature makes things considerably easier. Notice that the name of the `.py` file is `BPP4D3E_11_Packages.py`.

The example code contains two simple functions named `SayHello()` and `SayGoodbye()`. In both cases, you supply a `Name` to print and the function prints it onscreen along with a greeting for you. At that point, the function returns control to the caller. Obviously, you normally create more complicated functions, but these functions work well for the purposes of this chapter.

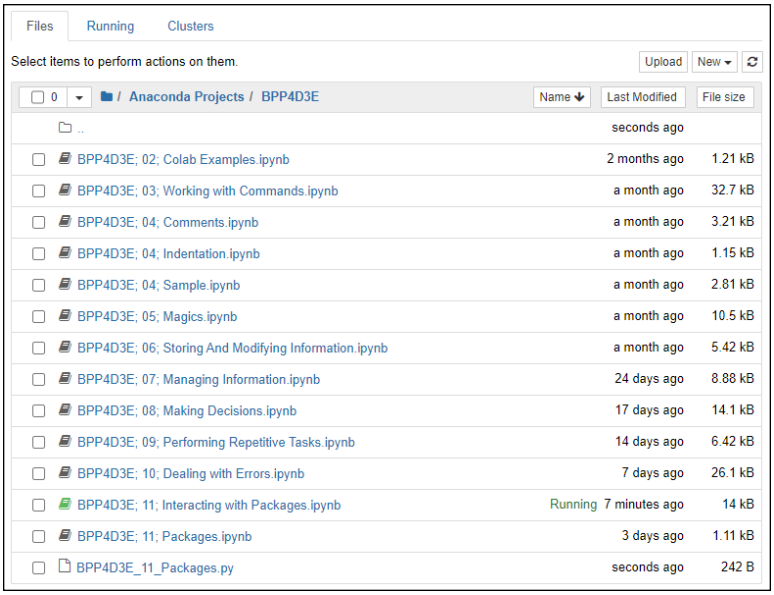


FIGURE 11-1: Make sure you place a copy of the package in your BPP4D3E folder.

INTERACTING WITH THE CURRENT PYTHON DIRECTORY

The directory that Python is using to access code affects which packages you can load. The Python library files are always included in the list of locations that Python can access, but Python knows nothing of the directory you use to hold your source code unless you tell it to look there. Of course, you need to know how to interact with the directory functions in order to tell Python where to look for specific bits of code.

1. **Open a new notebook.**

   You can also use the downloadable source file, `BPP4D3E; 11; Interacting with Packages.ipynb`.

2. **Type** `import os` **and press Enter.**

   This action imports the Python `os` library. You need to import this library to change the directory (the location Python sees on disk) to the working directory for this book.

3. **Type** `print(os.getcwd())` **and click Run Cell.**

   You see the current working directory (CWD) that Python uses to obtain local code.

(continued)

(continued)

4. **In a new cell, type** for entry in os.listdir(): print(entry) **and click Run Cell.**

   You see a listing of the directory entries. The listing lets you determine whether the file you need is in the CWD. If not, you need to change directories to a location that does contain the required file.

To change directories to a new location, you use the `os.chdir()` method and include the new location as a string, such as `os.chdir('C:\MyDir')`. However, you normally find with Notebook that the CWD does contain the files for your current project.

TIP

The code in this sidebar works with Google Colab as well as Jupyter Notebook, but the Google Colab results are less useful. The Colab Python executable files are normally located in the `/usr/local/bin/python` directory, which you can determine using the `!which python` command, where the `!` (bang) operator (`https://anaconda.zendesk.com/hc/en-us/articles/360023858254-Executing-Terminal-Commands-in-Jupyter-Notebooks`) provides access to the system shell, and the `which` command (`https://linuxize.com/post/linux-which-command/`) provides the location of a particular executable, `python` in this case.

Understanding the package types

The Python support system is immense. In fact, you'll likely never use more than a small fraction of it for even the most demanding applications. It's not that Python itself is all that huge; the language is actually somewhat concise compared to many other languages out there. The immensity comes from the Python system of packages and libraries that perform everything from intense scientific work, to AI, to space exploration, to biologic modeling, to anything else you can imagine and many things you can't. However, not all those packages are available all the time, so you need some idea of what sort of packages Python supports and where you might find them (in order of preference):

- **Built-in:** The built-in packages address most common needs. When working with Jupyter Notebook as part of *Anaconda* (a development package containing multiple tools, including conda, that you can find at `https://www.anaconda.com/`), you find them in the `\Users\<Username>\Adaconda3\Lib` folder on your system, and all you need to do to use them is `import` them into your application. Locating them in Google Colab is somewhat more difficult. You may need to do a little searching. However, if you're using Python 3.7, you could use this command (all on one line) in a Colab cell to obtain a listing of the built-in packages (see the "Interacting with the current python directory" sidebar for details on using `os` commands):

```
for entry in os.listdir("/usr/local/lib/python3.7/
  dist-packages"): print(entry)
```

- **Custom:** As demonstrated in this chapter, you can create your own packages and use them as needed. They appear on your hard drive, normally in the same directory as your project code, and you simply `import` them into your application.
- **Conda:** You can find a wealth of packages specifically designed for Anaconda (Jupyter Notebook). Many of these packages appear at `https://anaconda.org/mutirri/repo`. Before you can use these packages, you must install them by using the `conda` utility at the Anaconda command line, as described in the "Installing conda packages" section of this chapter. After you have the package installed, you use it as you would any built-in package. If you are using Colab, you won't have access to the `conda` utility, which is explained in the "And just why is conda missing in Colab?" section of the chapter.
- **Non-conda:** Just because a package isn't specifically designed for use with Anaconda doesn't mean that you can't use it. You can find a great wealth of packages from third parties that provide significant functionality. To install these packages, you use the `pip` utility, as described in the "Installing packages by using pip" and "Installing packages using the %pip magics" sections, later in this chapter. After you have the package installed, you may have to perform additional configuration as described by the party who created the package. Generally, when the package is configured, you use it as you would any built-in package.

Considering the package cache

This section doesn't apply to Google Colab, so you can skip it if you're working online or simply don't want to know about the package cache. Anaconda provides a package cache that resides outside the Python library. This package cache lets you easily interact with the Anaconda-specific packages by using the `conda` command-line utility. To see how you use this package cache:

1. **Open an Anaconda command prompt or terminal window.**

   You get access to this feature through the Anaconda Prompt entry in the Anaconda3 folder on your system.

2. **Type** conda list **and press Enter.**

   You see a list of the packages that you have installed now.

REMEMBER

Note that the output displays the package name as you would access it from within Anaconda, the package version, and the associated Python version. All this information is helpful in managing the packages. The following list provides the essential `conda` commands for managing your packages:

- `conda clean`: Removes packages that you aren't using.
- `conda config`: Configures the package cache setup.
- `conda create`: Defines a new `conda` environment that contains a specific list of packages, which makes it easier to manage the packages and can improve application speed.
- `conda help`: Displays a complete list of `conda` commands.
- `conda info`: Displays the `conda` configuration information, which includes details on where `conda` stores packages and where it looks for new packages.
- `conda install`: Installs one or more packages into the default or specified `conda` environment.
- `conda list`: Outputs a list of `conda` packages with varying levels of detail. You can specify which packages to list and in which environments to look.
- `conda remove`: Removes one or more packages from the package cache.
- `conda search`: Looks for specific packages by using the search criteria you provide.
- `conda update`: Updates some or all of the packages in the package cache.

TIP

These commands can do a lot more than you might think. Of course, it's impossible to memorize all that information, so you can rely on the `--help` command-line switch to obtain full details on using a particular command. For example, to learn more about `conda list`, type **conda list --help** and press Enter.

Importing Packages

To use a package, you must import it. Python places the package code inline with the rest of your application in memory — as if you had created one huge file. Neither file is changed on disk — they're still separate, but the way Python views the code is different.

REMEMBER

You have two ways to import packages. Each technique is used in specific circumstances:

- `import`: You use the `import` statement when you want to import an entire package. This is the most common method that developers use to import packages because it saves time and requires only one line of code. However, this approach also uses more memory resources than does the approach of selectively importing the attributes you need, which the next paragraph describes.
- `from...import`: You use the `from...import` statement when you want to selectively import individual package attributes. This method saves resources, but at the cost of complexity. In addition, if you try to use an attribute that you didn't import, Python registers an error. Yes, the package still contains the attribute, but Python can't see it because you didn't import it.

Now that you have a better idea of how to import packages, it's time to look at them in detail. The following sections help you work through importing packages using the two techniques available in Python.

Using the import statement

The `import` statement is the most common method for importing a package into Python. This approach is fast and ensures that the entire package is ready for use. The following steps get you started using the `import` statement:

1. **Open a new notebook, if you haven't already created one for the code in the "Interacting with the current python directory" sidebar.**

   You can also use the downloadable source file, `BPP4D3E; 11; Interacting with Packages.ipynb`.

2. **Change directories, if necessary, to the downloadable source code directory.**

   Generally, Notebook places you in the correct directory to use the source code files, so you won't need to perform this step. See the instructions found in the "Interacting with the current Python directory" sidebar.

3. **In a new cell, type** `import BPP4D3E_11_Packages` **and press Enter.**

   This instruction tells Python to import the contents of the `BPP4D3E_11_Packages.py` file that you created in the "Creating your first package" section of the chapter. The entire library is now ready for use.

WARNING

It's important to know that Python also creates a cache of the package in the `__pycache__` subdirectory of the `BPP4D3E` folder. If you look into your source code directory after you import `BPP4D3E_11_Packages` for the first time, you see the new `__pycache__` directory. If you want to make changes to your package, you must delete this directory. Otherwise, Python will continue to use the unchanged cache file instead of your updated source code file.

TECHNICAL STUFF

The cached filename includes the version of Python for which it is meant, so it's BPP4D3E_11_Packages.cpython–38.pyc in this case. The 38 in the filename means that this file is Python 3.8 specific. A .pyc file represents a compiled Python file, which is used to improve application speed.

4. **Type** dir(BPP4D3E_11_Packages) **and click Run Cell.**

   You see a listing of the package contents, which includes the SayHello() and SayGoodbye() functions, as shown in Figure 11-2. (A discussion of the other entries appears in the "Viewing the Package Content" section, later in this chapter.)

Using the import statement

```
In [3]: import BPP4D3E_11_Packages
        dir(BPP4D3E_11_Packages)

Out[3]: ['SayGoodbye',
         'SayHello',
         '__builtins__',
         '__cached__',
         '__doc__',
         '__file__',
         '__loader__',
         '__name__',
         '__package__',
         '__spec__']
```

FIGURE 11-2: A directory listing shows that Python imports both functions from the package.

5. **In a new cell, type** BPP4D3E_11_Packages.SayHello("Josh")**.**

REMEMBER

Notice that you must precede the attribute name, which is the SayHello() function in this case, with the package name, which is BPP4D3E_11_Packages. The two elements are separated by a period (also called a dot). Every call to a package that you import follows the same pattern.

6. **Type** BPP4D3E_11_Packages.SayGoodbye("Sally") **and click Run Cell.**

   The SayHello() and SayGoodbye() functions output the expected text.

Using the from. . .import statement

The `from...import` statement has the advantage of importing only the attributes you need from a package. This difference means that the package uses less memory and other system resources than using the `import` statement does. In addition, the `from...import` statement makes the package a little easier to use because some commands, such as `dir()`, show less information, or only the information that you actually need. The point is that you get only what you want and not anything else. The following steps demonstrate using the `from...import` statement. However, before you can import `BPP4D3E_11_Packages` selectively, you must remove it from the environment, which is the first part of the following process.

1. **Type the following code into a new cell in the Notebook:**

```
import sys
del sys.modules["BPP4D3E_11_Packages"]
del BPP4D3E_11_Packages
dir(BPP4D3E_11_Packages)
```

2. **Click Run Cell.**

   You see the error message shown in Figure 11-3. Listing the content of the BPP4D3E_11_Packages package isn't possible anymore because it's no longer loaded.

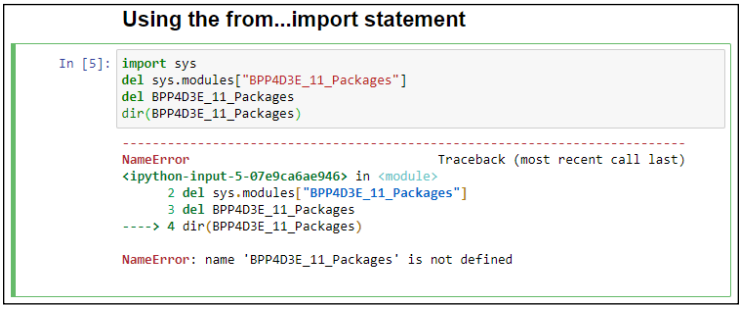


FIGURE 11-3: Removing a package from the environment requires two steps.

3. **In a new cell, type** from BPP4D3E_11_Packages import SayHello **and press Enter.**

   Python imports the SayHello() function from BPP4D3E_11_Packages. Only this specific function is now ready for use.

REMEMBER

To import both functions you need, you create a list of attributes to import; the names can be separated by commas, such as `from BPP4D3E_11_Packages import SayHello, SayGoodbye`.

4. **Type** dir(BPP4D3E_11_Packages) **and click Run Cell.**

   Python displays an error message, as shown previously in Figure 11-3. Python imports only the attributes that you specifically request. This means that the BPP4D3E_11_Packages package isn't in memory — only the attributes that you requested are in memory.

5. **In a new cell, type** dir(SayHello) **and click Run Cell.**

   You see a listing of attributes that are associated with the SayHello() function, as shown in Figure 11-4 (which is only a partial list). You don't need to know how these attributes work just now, but you'll use some of them later in the book.

FIGURE 11-4: Use the dir() function to obtain information about the specific attributes you import.

6. **In a new cell, type** SayHello("Angie") **and click Run Cell.**

   The `SayHello()` function outputs the expected text.

REMEMBER

When you import attributes by using the `from...import` statement, you don't need to precede the attribute name with a package name. This feature makes the attribute easier to access.

WARNING

Using the `from...import` statement can also cause problems. If attributes from two different packages have the same name, you can import only one of them using this approach. Using the `import` statement form prevents name collisions, which is important when you have a large number of attributes to import. In sum, you must exercise care when using the `from...import` statement.

7. **In a new cell, type** SayGoodbye("Harold") **and click Run Cell.**

   You imported only the `SayHello()` function, so Python knows nothing about `SayGoodbye()` and displays an error message. The selective nature of the `from...import` statement can cause problems when you assume that an attribute is present when it really isn't.

Using the import. . .as statement

Having to precede every attribute name with the `BPP4D3E_11_Packages` name will cause severe finger cramping after a while, and no one wants that! Of course, such

a long name can also cause steaming-brain syndrome as you attempt to remember what the package name is. Fortunately, Python is concerned about your health and provides an alternative to finger cramping and a steaming brain in the form of the `import...as` statement. The `as` part of the statement gives the package a shorter, easier-to-remember name that won't cause finger problems or that other thing. The following steps show how to use this approach.

1. **Type the following code into a new cell in the Notebook:**

```
import BPP4D3E_11_Packages as BPP
dir(BPP)
```

2. **Click Run Cell.**

   You see the same listing of package contents as that shown in Figure 11-2, but you had to type less code to get it.

3. **Type the following code into a new cell in the Notebook:**

```
BPP.SayHello("Albert")
BPP.SayGoodbye("Daphne")
```

4. **Click Run Cell.**

   The two functions output the expected text. However, you typed a lot less code to obtain the results.

Finding Packages

To use the code in a package, Python must be able to locate the package and load it into memory, so you need to know where the package resides. When working with Jupyter Notebook, the packages reside on your hard drive. If you don't find them on your hard drive, you can download what you need. Colab stores its files in the `/usr/local/lib/python3.7/dist-packages` folder (with the 3.7 indicating the Python version you're using) online. Anything on your local disk is inaccessible. The following sections discuss how to find packages that are already available to Python in more detail. (The "Downloading Packages from Other Sources" section, later in the chapter, tells how to obtain packages that you can't already access.)

USING ENVIRONMENT VARIABLES WITH PYTHON

Environment variables tell Python how to configure itself or where to look for resources. Using environment variables makes sense when you need to configure Python the same way on a regular basis. You can find a list of environment variables at `https://www.geeksforgeeks.org/environment-variables-in-python/` and how to interact with them at `https://www.askpython.com/python/environment-variables-in-python`. Of the environment variables, the one you use most often is `PYTHONPATH`, which provides a semicolon (;) separated list of directories to search for modules. This value is stored in the `sys.path` variable in Python.

Locating packages on disk

Some of the information in this section works only with Jupyter Notebook; it may not work with any browser-based IDE you use (such as Google Colab). The location information is stored as paths within Python. Whenever you request that Python import a package, Python looks at all the files in its list of paths to find it. The path information comes from three sources:

- **Environment variables:** Environment variables, such as `PYTHONPATH`, tell Python where to find packages on disk (see the "Using environment variables with Python" sidebar for details).
- **Current directory:** Earlier in this chapter, you discover that you can change the current Python directory so that it can locate any packages used by your application.
- **Default directories:** Even when you don't define any environment variables and the current directory doesn't yield any usable packages, Python can still find its own libraries in the set of default directories that are included as part of its own path information.

Knowing the current path information is helpful because the lack of a path can cause your application to fail. To obtain path information, type **for p in sys.path: print(p)** in a new cell and click Run Cell. You see a listing of the path information, as shown in Figure 11-5. Your listing may be different from the one shown in Figure 11-5, depending on your platform, the version of Python you have installed, and the Python features you have installed.

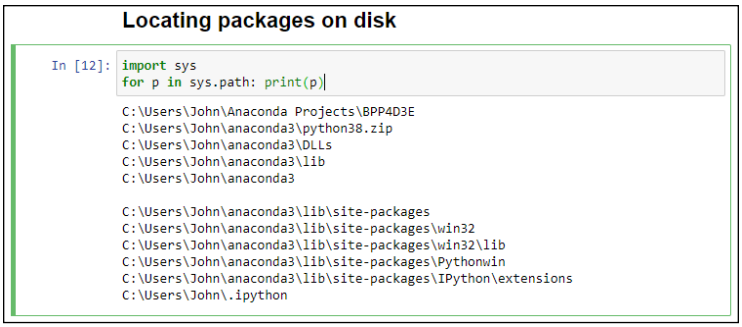


FIGURE 11-5: The `sys.path` attribute contains a listing of the individual paths for your system.

The `sys.path` attribute is reliable but may not always contain every path that Python can see. If you don't see a needed path, you can always check in another place that Python looks for information. The following steps show how to perform this task:

1. **Type the following code into a new cell in the Notebook:**

```
import os
os.environ['PYTHONPATH'].split(os.pathsep)
```

   When you have a `PYTHONPATH` environment variable defined, you see a list of one or more paths, such as `['C:\\BP4D']` or `['/env/python']`. However, if you don't have the environment variable defined, you see an error message instead.

   You must provide `split()` with a value to look for in splitting a list of items. The `os.pathsep` *constant* (a variable that has one, unchangeable, defined value) defines the path separator for the current platform so that you can use the same code on any platform that supports Python.

2. **Click Run Cell.**

   You see the listing of paths for your platform and setup.

TIP

You can also add and remove items from `sys.path`. For example, if you want to add the current working directory to the list of packages, you type **`sys.path.append(os.getcwd())`** in the Notebook cell and click Run Cell. When you list the `sys.path` contents again, you see that the new entry is added to the end of the list. Likewise, when you want to remove an entry, you type **`sys.path.remove(os.getcwd())`** in the Notebook cell and click Run Cell. The addition is present only during the current session.

WARNING

If you change an environment variable while the Jupyter Notebook server is running, you won't see any changes within the Notebook. To see the changes, you must restart Jupyter Notebook completely from scratch.

Locating packages online

When working with a browser-based IDE, you often see the results of using a virtual environment, the platform that the host has chosen to use, or simply find the information you need in a location other than what you expected. Consequently, the code in the "Locating packages on disk" section may work, but may not provide what you need. For example, to ensure that you can actually find the Python path, you need to use the `!echo $PYTHONPATH` command for Linux and macOS, or `!echo %PYTHONPATH%` for Windows (see the article at `https://www3.ntu.edu.sg/home/ehchua/programming/howto/Environment_Variables.html` for details on how environment variables work on different operating systems). Using the wrong form of the command for a particular host doesn't do anything terrible; you just won't see the path to Python (assuming that the host actually defines the `PYTHONPATH` environment variable).

In many situations, you must rely more heavily on magics (as discussed in Chapter 5) when working with packages and attempting to perform other tasks when using a browser-based IDE. For example, when working with Colab, every `!` operator command you use creates an entirely new subshell, which means that the changes made by each statement may be lost. Consequently, if you wanted to do something like change directories to load a package, you'd need to use the `%cd` magics command rather than the `!cd` operator command, even though it seems as if both should work. The result of the `!cd` operator command would go away as soon as the command completed, but the `%cd` magics command is permanent for a particular session. Note that the functions in the `os` package (as used in the previous section) are often effective when working with browser-based IDEs. For example, you may be able to use `os.chdir()`, rather than rely on the `%cd` magics.

Downloading Packages from Other Sources

Your copy of Python and the associated Jupyter Notebook component of Anaconda or Google Colab come with a wide assortment of packages that fulfill many common needs. In fact, for experimentation purposes, you seldom have to go beyond these packages because they are already installed as part of your IDE. Of course, someone is always thinking of some new way to do things, which requires new code and packages to store the code. In addition, some coding techniques are so esoteric that including the packages to support them with a default install would consume space that most people will never use. Consequently, you may have to install packages from online or other sources from time to time.

The two most common methods of obtaining new packages are to use the `conda` (which you find only in Jupyter Notebook) or `pip` (also known by the recursive acronym Pip Installs Packages) utilities. However, you may find packages that use other installation methods with varying degrees of success. You use `conda` and `pip` for different purposes:

- `conda`: Provides general-purpose package management for a wide range of languages with special needs in the conda environment.
- `pip`: Provides services specifically for Python in any environment. When you need a Python-specific package, look to `pip` first. For example, `pip` gives you access to the Python Package Index (PyPI) found at `https://pypi.org/`.

TIP

You can read more about the differences between `conda` and `pip` at `https://jakevdp.github.io/blog/2016/08/25/conda-myths-and-misconceptions/` and `https://docs.conda.io/projects/conda/en/latest/commands.html#conda-vs-pip-vs-virtualenv-commands`. The following sections discuss these two methods.

Opening the Anaconda Prompt

Before you can do much in the way of managing packages in Jupyter Notebook, you must open the Anaconda Prompt. The Anaconda Prompt is just like any other command prompt or terminal window, but it provides special configuration features to make working with the various command-line utilities supplied with Anaconda easier. To open the prompt, locate its icon in the Anaconda3 folder on your machine. For example, when using a Windows system, you can open the Anaconda Prompt by choosing Start ➪ All Programs ➪ Anaconda3 ➪ Anaconda Prompt. The Anaconda Prompt may take a moment or two to appear onscreen because of its configuration requirements.

Working with conda packages

When working with Jupyter Notebook, you can perform a wide range of tasks using `conda`, but some tasks are more common than others. The following sections describe how to perform five essential tasks using `conda`. You can obtain additional information about this utility at `https://docs.conda.io/projects/conda/en/latest/commands.html`. Typing **conda --help** at the Anaconda Prompt or terminal window and pressing Enter also yields an overview of help information.

Viewing conda packages

You can view `conda` packages in two ways. The first is to create a list of available packages, while the second is to search for a specific package. Listing helps you discover whether a package is already installed. Searching helps you discover the details about the installed package.

You can perform searching and listing in a general way to locate everything installed on a particular system. In this case, you use the commands by themselves:

```
conda list
conda search
```

The output of these commands is lengthy and might scroll right off the end of the screen buffer (making it impossible to scroll back and view all of the results). You can use `conda list | more` to display the output one page at a time on some platforms.

Note that the output shows the package name, version, and associated version of Python. You can use this output to determine whether a package is installed on your system. However, sometimes you need more, which requires a search. For example, say that you want to know what you have installed from the scikit-learn package for the Windows 64-bit platform. In this case, you type **conda search --platform win-64 scikit-learn** and press Enter.

TIP

A number of flags exist to greatly increase the amount of information you receive. For example, when you use the `--json` flag, you obtain details such as a complete list of dependencies for the package, whether the package is completely installed, and a URL containing the location of the packages online. You can learn more about `conda` searches at `https://docs.conda.io/projects/conda/en/latest/commands/search.html`.

Installing conda packages

The lists of `conda` packages appear at `https://docs.anaconda.com/anaconda/packages/pkg-docs/`. The lists are grouped by Python version number and platform. To determine whether a package is available for your version of Python on a particular platform, simply click the associated link. You can use `conda` to install a package using the `conda install` command documented at `https://docs.conda.io/projects/conda/en/latest/commands/install.html`. For example, to install SciPy, you type **conda install scipy** and press Enter at the Anaconda Prompt.

USING CONDA INFO

Even though the `conda info` command is normally associated with environment information, you can also use it to work with packages. To discover the specifics of a particular package, you just add the package name, such as `conda info numpy`. Unfortunately, using this command often results in information overload, so you need to shorten it a little. One way to do this is to add a version number after the package name separated by an equals (=) sign, such as `conda info numpy=1.18.5` for version 1.18.5 of the NumPy package.

In most cases, you don't receive any additional information using the `--verbose` switch with packages. However, using the `--json` switch can yield a little additional information, and this switch puts the information in a form that lets you easily manipulate the output using code, such as a script. The point is that you can use `conda info` to discover even more deep, dark secrets about your packages. You can learn more about `conda info` at `https://docs.conda.io/projects/conda/en/latest/commands/info.html`.

Updating conda packages

The packages you use to develop applications can become outdated with time. The developers who maintain them might add new features or apply bug fixes. The problem with updates is that they can cause your application to work incorrectly, or sometimes not at all if you're depending on a broken behavior. However, it's generally a good idea to keep packages updated if for no other reason than to apply security-related bug fixes. Of course, you need to know that the package requires updating. To find outdated packages, you use the `conda search --outdated --names-only` command, followed by the name of the package you want to check.

After you know what you need to update, you can use the `conda update` command to perform the task. For example, you might want to update the NumPy package, which means typing **conda update numpy** and pressing Enter. Few packages are stand-alone, so `conda` will present a list of items that you need to update along with NumPy. Type **y** and press Enter to proceed. Figure 11-6 shows a typical sequence of events during the update process.

WARNING

You do have the option of updating all packages at one time. Simply type **conda update --all** and press Enter to get started. However, you may find that interactions between packages make the update less successful than it could be if you performed the updates individually. In addition, the update can take a long time, so be sure to have plenty of coffee and a copy of *War and Peace* on hand. You can learn more about `conda` updates at `https://docs.conda.io/projects/conda/en/latest/commands/update.html`.

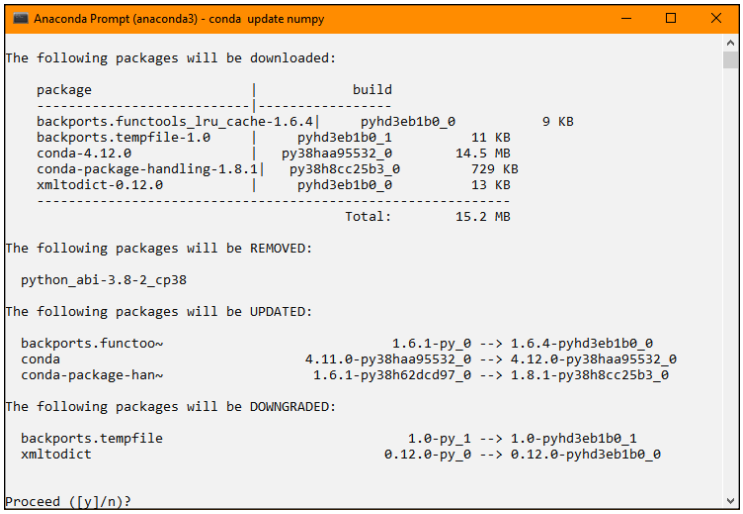


FIGURE 11-6: You see a lot of information during the update process.

Removing conda packages

At some point, you might decide that you no longer need a `conda` package. The only problem is that you don't know whether other packages depend on the package in question. Because package dependencies can become quite complex, and you want to be sure that your applications will continue to run, you need to check which other packages depend on this particular package. Unfortunately, the `conda info` command (described at `https://docs.conda.io/projects/conda/en/latest/commands/info.html`) tells you only about the package requirements — that is, what it depends on. Best practice is to not uninstall packages after you've installed them.

REMEMBER

However, assuming that you really must remove the package, you use the `conda remove` command described at `https://docs.conda.io/projects/conda/en/latest/commands/remove.html`. This command removes the package that you specify, along with any packages that depend on this package. In this case, best practice is to use the `--dry-run` command-line switch first to ensure that you really do want to remove the package. For example, you may decide that you want to remove NumPy. In this case, you type **conda remove --dry-run numpy** and press Enter. The command won't actually execute; `conda` simply shows what would happen if you actually did run the command.

And just why is conda missing in Colab?

Colab doesn't come with `conda` because it isn't part of the Anaconda suite of tools. Unfortunately, a number of tools recommend using `conda`, especially those associated with data science and statistics. Consequently, you may actually want to

install conda into your Colab environment. Installing conda as part of Colab can become a complex undertaking and is well outside the scope of this book. However, you can find help for installing conda in Colab online in articles like the one at https://towardsdatascience.com/conda-google-colab-75f7c867a522, and there is even a YouTube video showing the process at https://www.youtube.com/watch?v=Rm2ak31u1us. The video actually tells you how to access a script that can reduce the complexity of the task.

Installing packages by using pip

Oddly enough, working with pip is much like working with conda. They both need to perform essentially the same tasks, so if you know how to use one, you know how to use the other. The reference at https://pip.pypa.io/en/stable/cli/ shows that pip does support essentially the same commands (with a few wording differences). For example, if you want to find a list of outdated packages, you type **pip list --outdated** and press Enter. Here is a list of the common commands that pip supports:

- check: Verify that the installed packages have compatible dependencies.
- download: Download the specified packages for later installation.
- freeze: Output installed packages in requirements format.
- help: Display a help screen listing an overview of the commands.
- install: Install the specified packages.
- list: List the installed packages.
- search: Search online at PyPI for packages.
- show: Show information about the installed packages.
- uninstall: Uninstall the specified packages.

TIP

When working with either Jupyter Notebook or Google Colab, you can use the ! operator to work with pip. Consequently, typing **!pip list --outdated** in a cell and then clicking Run Cell produces a list of outdated packages on your system. The advantage of this approach is that you can create a notebook that contains information about your setup that you can refer to as needed when working on projects.

Installing packages using the %pip magics

There is a magics version of the pip command that is accessible from within Colab and Jupyter Notebook. It works just like the ! operator version, except you use %pip instead. You use this approach when you need to work with pip within the

current kernel so that the result of a `pip` command doesn't just evaporate when the command completes. This technique allows you to execute a number of `pip` commands in sequence and know that they'll work as expected.

TIP

It shouldn't surprise you to know that a `%conda` magics command exists for Jupyter Notebook users. As with `%pip`, `%conda` works the same as the command-line version.

Viewing the Package Content

Python gives you several different ways to view package content. The method that most developers use is to work with the `dir()` function, which tells you about the attributes that the package provides.

Look at Figure 11-2, earlier in the chapter. In addition to the `SayGoodbye()` and `SayHello()` function entries discussed previously, the list has other entries. These attributes are automatically generated by Python for you. These attributes perform the following tasks or contain the following information:

- `__builtins__`: Contains a listing of all the built-in attributes that are accessible from the package. Python adds these attributes automatically for you.
- `__cached__`: Tells you the name and location of the cached file that is associated with the package. The location information (path) is relative to the current Python directory.
- `__doc__`: Outputs help information for the package, assuming that you've actually filled it in. For example, if you type **print(os.__doc__)** and press Enter, Python will output the help information associated with the `os` library. (Using the `print()` function ensures that the output is nicely formatted.)
- `__file__`: Tells you the name and location of the package. The location information (path) is relative to the current Python directory.
- `__initializing__`: Determines whether the package is in the process of initializing itself. Normally this attribute returns a value of `False`. This attribute is useful when you need to wait until one package is done loading before you import another package that depends on it.
- `__loader__`: Outputs the loader information for this package. The *loader* is a piece of software that gets the package and puts it into memory so that Python can use it. This is one attribute you rarely (if ever) use.
- `__name__`: Tells you just the name of the package.

» `__package__`: This attribute is used internally by the import system to make it easier to load and manage packages. You don't need to worry about this particular attribute.

It may surprise you to find that you can drill down even further into the attributes. Type **dir(BPPD_11_Packages.SayHello)** and press Enter. You see the attributes associated with the `SayHello()` function.

Some of these entries, such as `__name__`, also appeared in the package listing. However, you might be curious about some of the other entries. For example, you might want to know what `__sizeof__` is all about. One way to get additional information is to type **help("___sizeof___")** and press Enter. You see some scanty (but useful) help information, such as "Size of object in memory, in bytes."

Python won't blow up if you try the attribute. Even if the notebook does experience problems, you can always restart the kernel (or simply restart the environment as a whole). Another way to check out a package is to simply try the attributes. For example, if you type **BPPD_11_Packages.SayHello.___sizeof___()** and press Enter, you see the size of the `SayHello()` function in bytes, which is 120 bytes in this case.

Viewing Package Documentation

You can use the `doc()` function whenever needed to get quick help. However, you have a better way to study the packages and libraries located in the Python path — the Python Package Documentation. This feature often appears as Package Docs in the Python folder on your system. It's also referred to as PyDoc. Whatever you call it, the Python Package Documentation makes life a lot easier for developers. The following sections describe how to work with this feature.

Using !pydoc to access PyDoc

You can access a command-line version of PyDoc from the Anaconda Prompt when working with Jupyter Notebook, but the best and easiest way to use PyDoc is within a notebook cell using the `!` operator. This approach works equally well in Colab and Jupyter Notebook, so you can use the same techniques even if you switch between the two IDEs. If you type **!pydoc** and click Run Cell, you see the output shown in Figure 11-7, which describes how to use PyDoc from within your IDE.

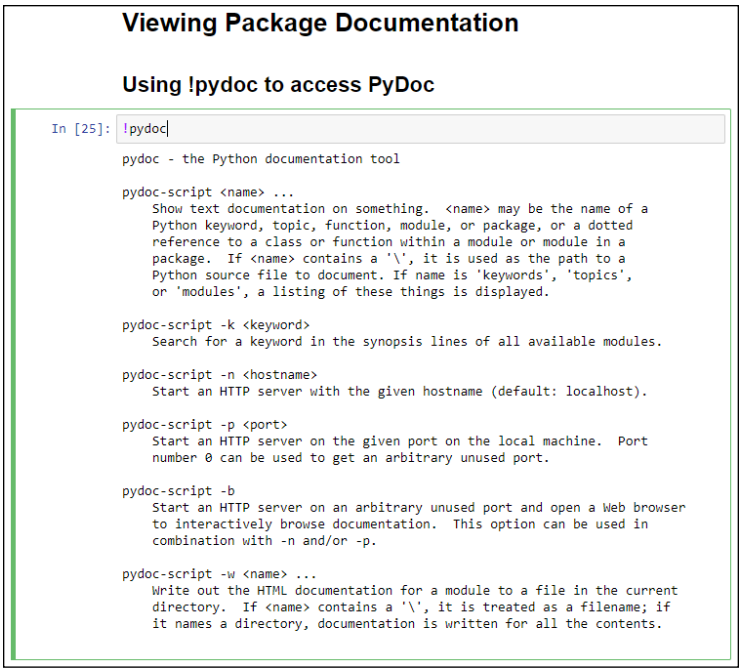


FIGURE 11-7: PyDoc makes it easy to get instructions on its use directly within a cell.

Typing a search term

To search for a particular Python keyword, function, or other attribute using PyDoc, simply add a search term after the `!pydoc` command. For example, if you want to learn more about the `print()` function, type **!pydoc print** and click Run Cell. You see output similar to that shown in Figure 11-8.

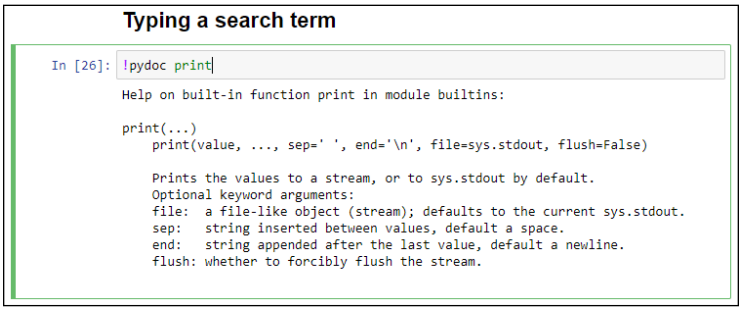


FIGURE 11-8: PyDoc will tell you all about the `print()` function if you ask.

When working with Jupyter Notebook, you can perform keyword searches using the `-k` switch. A *keyword search* helps you locate features that could be helpful, but that you don't necessarily know about when you start the search. Keyword searches don't work in Colab (you get an error instead). Note that keyword searches can take a long time to run and produce copious results. For example, type **!pydoc -k stream** and click Run Cell to see a long list of packages, functions, and other features that work with streams. (The size of this list will amaze you.) Figure 11-9 shows a sampling of what you might see when executing this command.

```
In [29]: !pydoc -k stream

_io - The io module provides the Python interfaces to stream handling. The
asyncio.streams
io - The io module provides the Python interfaces to stream handling. The
subprocess - Subprocesses with accessible I/O streams
anaconda_project.internal.streaming_popen
anaconda_project.internal.test.test_streaming_popen
astropy.io.fits.hdu.streaming
astropy.utils.xml.writer - Contains a class that makes it simple to stream out well-formed and
bleach._vendor.html5lib._inputstream
distributed.diagnostics.eventstream
distributed.diagnostics.progress_stream
distributed.diagnostics.task_stream
fsspec.compression - Helper functions for a standard streaming compression API
html5lib._inputstream
ipykernel.iostream - Wrappers for forwarding stdout/stderr over zmq
matplotlib.streamplot - Streamline plotting for 2D vector fields.
nacl.bindings.crypto_secretstream
nbconvert.preprocessors.coalescestreams - Preprocessor for merging consecutive stream outputs for eas
ier handling.
nbconvert.preprocessors.tests.test_coalescestreams - Tests for the coalescestreams preprocessor
nltk.test.unit.test_seekable_unicode_stream_reader - The following test performs a random series of r
eads, seeks, and
numpy.compat._inspect - Subset of inspect module from upstream python
pandas._config - pandas._config is considered explicitly upstream of everything else in pandas,
pandas.tests.test_downstream - Testing that we work in the downstream packages
pip._vendor.html5lib._inputstream
prompt_toolkit.input.vt100_parser - Parser for VT100 input stream.
pyls_jsonrpc.streams
scipy._lib.messagestream
scipy.io.matlab.miobase - Base classes for MATLAB file stream reading.
scipy.io.matlab.streams
scipy.io.matlab.tests.test_streams - Testing
tornado.iostream - Utility classes to write to and read from non-blocking files and sockets.
tornado.test.iostream_test
win32com.test.testStreams
win_unicode_console.streams
zmq.eventloop.zmqstream - A utility class to send to and recv from a non-blocking socket,
zmq.green.eventloop.zmqstream
zmq.tests.test_zmqstream
```

FIGURE 11-9: Keyword searches help you locate items that you may not know how to reference otherwise.

IN THIS CHAPTER

- Considering the string difference
- Working with special and single characters
- Manipulating and searching strings
- Modifying the appearance of string output

Chapter 12
Working with Strings

Your computer doesn't understand strings. It's a basic fact. Computers understand numbers, not letters. When you see a string on the computer screen, the computer actually sees a series of numbers. However, humans understand strings quite well, so applications need to be able to work with them. Fortunately, Python makes working with strings relatively easy. It translates the string you understand into the numbers the computer understands, and vice versa.

To make strings useful, you need to be able to manipulate them. Of course, that means taking strings apart and using just the pieces you need or searching the string for specific information. This chapter describes how you can build strings by using Python, dissect them as needed, and use just the parts you want after you find what's required. String manipulation is an important part of applications because humans depend on computers performing that sort of work for them (even though the computer has no idea of what a string is).

After you have the string you want, you need to present it to the user in an eye-pleasing manner. The computer doesn't really care how it presents the string, so often you get the information, but it lacks pizzazz. In fact, it may be downright difficult to read. Knowing how to format strings so that they look nice onscreen is important because users need to see information in a form they understand. By the time you complete this chapter, you know how to create, manipulate, and format strings so that the user sees precisely the right information.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 12; Working with Strings.ipynb` file of the downloadable source. See the Introduction for details on how to find these source files.

Understanding That Strings Are Different

Most aspiring developers (and even a few who have written code for a long time) really have a hard time understanding that computers truly do only understand 0s and 1s. Even larger numbers are made up of 0s and 1s. Comparisons take place with 0s and 1s. Data is moved by using 0s and 1s. In short, strings don't exist for the computer (and numbers just barely exist). Although grouping 0s and 1s to make numbers is relatively easy, strings are a lot harder because now you're talking about information that the computer must manipulate as numbers but present as characters.

REMEMBER

There are no strings in computer science. Strings are made up of characters, and individual characters are actually numeric values. When you work with strings in Python, what you're really doing is creating an assembly of characters that the computer sees as numeric values. That's why the following sections are so important. They help you understand why strings are so special. Understanding this material will save you a lot of headaches later.

Defining a character by using numbers

To create a character, you must first define a relationship between that character and a number. More important, everyone must agree that when a certain number appears in an application and is viewed as a character by that application, the number is translated into a specific character. One of the most common ways to perform this task is to use the American Standard Code for Information Interchange (ASCII). Python uses ASCII to translate the number 65 to the letter *A*. The chart at `https://www.asciitable.com/` shows the various numeric values and their character equivalents.

REMEMBER

Every character you use must have a different numeric value assigned to it. The letter *A* uses a value of 65. To create a lowercase *a*, you must assign a different number, which is 97. The computer views *A* and *a* as completely different characters, even though people view them as uppercase and lowercase versions of the same character.

The numeric values used in this chapter are in decimal. However, the computer still views them as 0s and 1s. For example, the letter *A* is really the value 01000001

and the letter *a* is really the value 01100001. When you see an *A* onscreen, the computer sees a binary value instead.

TECHNICAL STUFF

Having just one character set to deal with would be nice. However, not everyone could agree on a single set of numeric values to equate with specific characters. Part of the problem is that ASCII doesn't support characters used by other languages; also, it lacks the capability to translate special characters into an onscreen presentation. In fact, character sets abound. You can see a number of them at `http://www.i18nguy.com/unicode/codepages.html`. Click one of the character set entries to see how it assigns specific numeric values to each character. Most characters sets do use ASCII as a starting point.

Using characters to create strings

Python doesn't make you jump through hoops to create strings. However, the term *string* should actually give you a good idea of what happens. Think about beads or anything else you might string. You place one bead at a time onto the string. Eventually you end up with some type of ornamentation — perhaps a necklace or tree garland. The point is that these items are made up of individual beads.

The same concept used for necklaces made of beads holds true for strings in computers. When you see a sentence, you understand that the sentence is made up of individual characters that are strung together by the programming language you use. The language creates a structure that holds the individual characters together. So, the language, not the computer, knows that so many numbers in a row (each number being represented as a character) defines a string such as a sentence.

REMEMBER

You may wonder why it's important to even know how Python works with characters. The reason is that many of the functions and special features that Python provides work with individual characters, and you need to know that Python sees the individual characters. Even though you see a sentence, Python sees a specific number of characters.

Unlike most programming languages, strings in Python can use either single quotes or double quotes. For example, `"Hello There!"` with double quotes is a string, as is `'Hello There!'` with single quotes. Python also supports triple double and single quotes that let you create strings spanning multiple lines. The following steps help you create an example that demonstrates some of the string features that Python provides.

1. **Open a new notebook.**

   You can also use the downloadable source file, `BPP4D3E; 12; Working with Strings.ipynb`.

2. **Type the following code into the notebook — pressing Enter after each line:**

```
print('Hello There (Single Quote)!')
print("Hello There (Double Quote)!")
print("""This is a multiple line
string using triple double quotes.
You can also use triple single quotes.""")
```

Each of the three `print()` function calls demonstrates a different principle in working with strings. Equally acceptable is to enclose the string in either single or double quotes. When you use a triple quote (either single or double), the text can appear on multiple lines.

3. **Click Run Cell.**

Python outputs the expected text. Notice that the multiline text appears on three lines (see Figure 12-1), just as it does in the source code file, so this is a kind of formatting. You can use multiline formatting to ensure that the text breaks where you want it to onscreen.

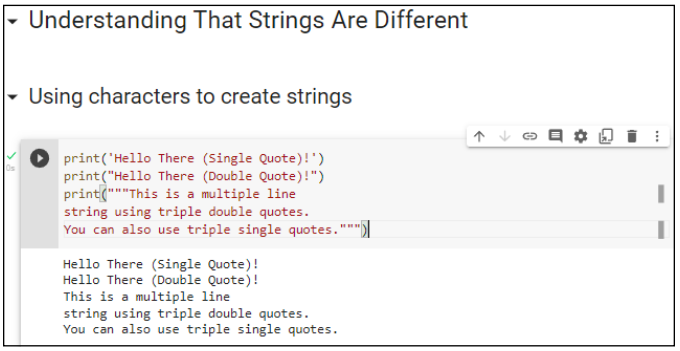


FIGURE 12-1: Strings consist of individual characters that are linked together.

Creating Stings with Special Characters

Some strings include special characters. These characters are different from the alphanumeric and punctuation characters that you're used to using. In fact, they fall into these categories:

- **Control:** An application requires some means of determining that a particular character isn't meant to be displayed but rather to control the display. All the control movements are based on the *insertion pointer,* the line you see when

you type text on the screen. For example, you don't see a tab character. The tab character provides a space between two elements, and the size of that space is controlled by a tab stop. Likewise, when you want to go to the next line, you use a carriage return (which returns the insertion pointer to the beginning of the line) and linefeed (which places the insertion pointer on the next line) combination.

TECHNICAL STUFF

To add spice to your life and make things completely confusing, some platforms use just a carriage return to advance to the next line, and other platforms use just the linefeed to advance to the next line. If you encounter problems in certain situations with too many lines or an unexpected result, try using just a carriage return or a linefeed in place of the combination of the two. The computer knows what it's doing; the humans don't.

- **Accented:** Characters that have accents, such as the acute (′), grave (`), circumflex (^), umlaut or diaeresis (¨), tilde (~), or ring (°), represent emphasis or special spoken sounds, in most cases. You must use special characters to create alphabetical characters with these accents included.
- **Drawing:** You can create rudimentary art with some characters. You can see examples of the box-drawing characters at `http://jrgraphix.net/r/Unicode/2500-257F`. Some people actually create art by using ASCII characters as well (`http://www.asciiworld.com/`).
- **Typographical:** A number of typographical characters, such as the pilcrow (¶), are used when displaying certain kinds of text onscreen, especially when the application acts as an editor.
- **Other:** Depending on the character set you use, the selection of characters is nearly endless. Need a smile? Well, there is a smiley face character for that need (☺). You can find a character for just about any need. The point is that you need some means of telling Python how to present these special characters.

A common need when working with strings, even strings from simple console applications, is control characters. With this in mind, Python provides escape sequences that you use to define control characters directly (and a special escape sequence for other characters).

REMEMBER

An *escape sequence* literally escapes the common meaning of a letter, such as *a*, and gives it a new meaning (such as the ASCII bell or beep). The combination of the backslash (\) and a letter (such as *a*) is commonly viewed as a single letter by developers — an *escape character* or *escape code*. Table 12-1 provides an overview of these escape sequences.

TABLE 12-1 Python Escape Sequences

Escape Sequence	Meaning
\newline	Ignored
\\	Backslash (\)
\'	Single quote (')
\"	Double quote (")
\a	ASCII Bell (BEL)
\b	ASCII Backspace (BS)
\f	ASCII Formfeed (FF)
\n	ASCII Linefeed (LF)
\r	ASCII Carriage Return (CR)
\t	ASCII Horizontal Tab (TAB)
\u*hhhh*	Unicode character (a specific kind of character set with broad appeal across the world) with a hexadecimal value that replaces *hhhh*
\v	ASCII Vertical Tab (VT)
ooo	ASCII character with octal numeric value that replaces *ooo*
\x*hh*	ASCII character with hexadecimal value that replaces *hh*

The best way to see how the escape sequences work is to try them. The following steps help you create an example that tests various escape sequences so that you can see them in action.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
print("Part of this text\r\nis on the next line.")
print("This is an A with a grave accent: \xC0.")
print("This is a drawing character: \u2562.")
print("This is a pilcrow: \266.")
print("This is a division sign: \xF7.")
```

The example code uses various techniques to achieve the same end — to create a special character. Of course, you can use control characters directly, as shown in the first line. Many special letters are accessible by using a hexadecimal number that has two digits (as in the second and fifth lines). However,

some require that you rely on Unicode numbers (which always require four digits), as shown in the third line. Octal values use three digits and have no special character associated with them, as shown in the fourth line.

2. **Click Run Cell.**

   Python outputs the expected text and special characters, as shown in Figure 12-2.

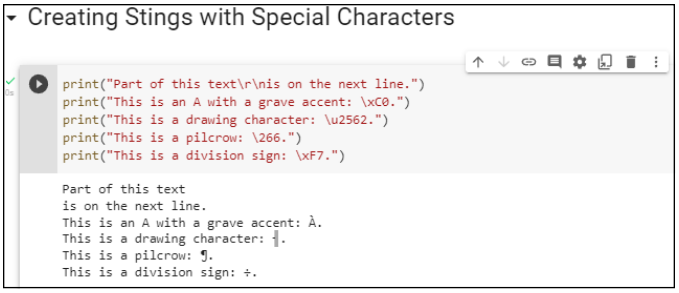


FIGURE 12-2: Use special characters as needed to present special information or to format the output.

WARNING

Notebook uses a standard character set across platforms, so you should see the same special characters no matter which platform you test. However, when creating your application, make sure to test it on various platforms to see how the application will react. A character set on one platform may use different numbers for special characters than another platform does. In addition, user selection of character sets could have an impact on how special characters displayed by your application appear. When working on applications that will see use with multiple languages, the chance of having problems between character sets is much higher. Always make sure that you test special character usage completely.

Selecting Individual Characters

Earlier in the chapter, you discover that strings are made up of individual characters. They are, in fact, just like beads on a necklace — with each bead being an individual element of the whole string.

Python makes it possible to access individual characters in a string. This is an important feature because you can use it to create new strings that contain only part of the original. In addition, you can combine strings to create new results.

The secret to this feature is the square bracket. You place a square bracket with a number in it after the name of the variable. Here's an example:

```
MyString = "Hello World"
print(MyString[0])
```

REMEMBER

In this case, the output of the code is the letter *H.* Python strings are zero-based, which means they start with the number *0* and proceed from there. For example, if you were to type `print(MyString[1])`, the output would be the letter *e.*

You can also obtain a range of characters from a string. Simply provide the beginning and ending letter count separated by a colon in the square brackets. For example, `print(MyString[6:11])` would output the word `World`. The output would begin with letter 6 and end with letter 10 (remember that the index is zero based). The following steps demonstrate some basic tasks that you can perform by using Python's character-selection technique.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line.**

```
String1 = "Hello World"
String2 = "Python is Fun!"
print(String1[0])
print(String1[0:5])
print(String1[:5])
print(String1[6:])
String3 = String1[:6] + String2[:6]
print(String3)
print(String2[:7]*5)
```

The example begins by creating two strings. It then demonstrates various methods for using the index on the first string. Notice that you can leave out the beginning or ending number in a range if you want to work with the remainder of that string.

The next step is to combine two substrings. In this case, the code combines the beginning of `String1` with the beginning of `String2` to create `String3`.

REMEMBER

The use of the + sign to combine two strings is called *concatenation.* This sign is one of the handier operators to remember when you're working with strings in an application.

The final step is to use a Python feature called *repetition.* You use repetition to make a number of copies of a string or substring.

2. **Click Run Cell.**

Python outputs a series of substrings and string combinations, as shown in Figure 12-3.

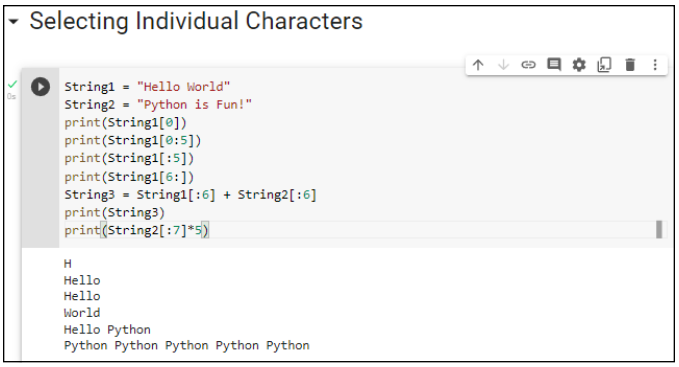


FIGURE 12-3: You can select individual pieces of a string.

Slicing and Dicing Strings

Working with ranges of characters provides some degree of flexibility, but it doesn't provide you with the capability to actually manipulate the string content or discover anything about it. For example, you might want to change the characters to uppercase or determine whether the string contains all letters. Fortunately, Python has functions that help you perform tasks of this sort. Here are the most commonly used functions:

- `capitalize()`: Capitalizes the first letter of a string.
- `center(`*`width`*`, `*`fillchar`*`=" ")`: Centers a string so that it fits within the number of spaces specified by *`width`*. If you supply a character for *`fillchar`*, the function uses that character. Otherwise, `center()` uses spaces to create a string of the desired width.
- `expandtabs(`*`tabsize`*`=8)`: Expands tabs in a string by replacing the tab with the number of spaces specified by *`tabsize`*. The function defaults to 8 spaces per tab when *`tabsize`* isn't provided.
- `isalnum()`: Returns `True` when the string has at least one character and all characters are alphanumeric (letters or numbers).
- `isalpha()`: Returns `True` when the string has at least one character and all characters are alphabetic (letters only).
- `isdecimal()`: Returns `True` when a string contains only decimal (base 10) characters. For example, `'100'` would return `True` because the string can represent a decimal value of 100. The string `'0xF'` would return `False` because it represents a base 16 number, not a base 10 number. Likewise, `'10.55'` would return `False` because this is a floating-point value, not a decimal number.

- `isdigit()`: Returns `True` when a string contains only digits (numbers and not letters).
- `islower()`: Returns `True` when a string has at least one alphabetic character and all alphabetic characters are in lowercase.
- `isnumeric()`: Returns `True` when a string contains only numeric characters.
- `isspace()`: Returns `True` when a string contains only whitespace characters (which includes spaces, tabs, carriage returns, linefeeds, form feeds, and vertical tabs, but not the backspace).
- `istitle()`: Returns `True` when a string is cased for use as a title, such as `'Hello World'`. However, the function requires that even little words have the title case. For example, `'Follow a Star'` returns `False`, even though it's properly cased, but `'Follow A Star'` returns `True`.
- `isupper()`: Returns `True` when a string has at least one alphabetic character and all alphabetic characters are in uppercase.
- `join(`*`seq`*`)`: Creates a string in which the base string is separated in turn by each character in *`seq`* in a repetitive fashion. For example, if you start with `MyString = "Hello"` and type `print(MyString.join("!*!"))`, the output is `!Hello*Hello!`.

  TIP

  The `join()` function is also used to join lists of characters. For example, `print('-'.join(['a', 'b', 'c']))` would result in an output of `a-b-c`. It can also be used with tuples, where `print(', '.join(('apples', 'cherries', 'and peaches')))` outputs apples, cherries, and peaches and sets, such as `print(', '.join({'apples', 'cherries', 'and peaches'}))`.
- `len(`*`str`*`)`: Obtains the length of *`str`*.
- `ljust(`*`width`*`, `*`fillchar`*`=" ")`: Left justifies a string so that it fits within the number of spaces specified by *`width`*. If you supply a character for *`fillchar`*, the function uses that character. Otherwise, `ljust()` uses spaces to create a string of the desired width.
- `lower()`: Converts all uppercase letters in a string to lowercase letters.
- `lstrip()`: Removes all leading whitespace characters in a string.
- `max(`*`str`*`)`: Returns the character that has the maximum numeric value in *`str`*. For example, `a` would have a larger numeric value than `A`.
- `min(`*`str`*`)`: Returns the character that has the minimum numeric value in *`str`*. For example, `A` would have a smaller numeric value than `a`.
- `rjust(`*`width`*`, `*`fillchar`*`=" ")`: Right justifies a string so that it fits within the number of spaces specified by *`width`*. If you supply a character for *`fillchar`*, the function uses that character. Otherwise, `rjust()` uses spaces to create a string of the desired width.

- `rstrip()`: Removes all trailing whitespace (right side of the string) characters in a string.
- `split(str=" ", num=string.count(str))`: Splits a string into substrings using the delimiter specified by *`str`* (when supplied). The default is to use a space as a delimiter. Consequently, if your string contains `'A Fine Day'`, the output would be three substrings consisting of `A`, `Fine`, and `Day`. You use *`num`* to define the number of substrings to return. The default is to return every substring that the function can produce.
- `splitlines(num=string.count('\n'))`: Splits a string that contains newline (`\n`) characters into individual strings. Each break occurs at the newline character. The output has the newline characters removed. You can use *`num`* to specify the number of strings to return.
- `strip()`: Removes all leading and trailing whitespace characters in a string.
- `swapcase()`: Inverts the case for each alphabetic character in a string.
- `title()`: Returns a string in which the initial letter in each word is in uppercase and all remaining letters in the word are in lowercase.
- `upper()`: Converts all lowercase letters in a string to uppercase letters.
- `zfill(width)`: Returns a string that is left-padded with zeros so that the resulting string is the size of *`width`*. This function is designed for use with strings containing numeric values. It retains the original sign information (if any) supplied with the number.

Playing with these functions a bit can help you understand them better. The following steps create an example that demonstrates some of the tasks you can perform by using these functions.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
MyString = " Hello World "
print(MyString.upper())
print(MyString.strip())
print(MyString.center(21, "*"))
print(MyString.strip().center(21, "*"))
print(MyString.isdigit())
print(MyString.istitle())
print(max(MyString))
print(MyString.split())
print(MyString.split()[0])
```

The code begins by creating `MyString`, which includes spaces before and after the text so that you can see how space-related functions work. The initial task is to convert all the characters to uppercase.

Removing extra space is a common task in application development. The `strip()` function performs this task well. The `center()` function lets you add padding to both the left and right side of a string so that it consumes a desired amount of space. When you combine the `strip()` and `center()` functions, the output is different from when you use the `center()` function alone.

REMEMBER

You can combine functions to produce a desired result. Python executes each of the functions one at a time from left to right. The order in which the functions appear will affect the output, and developers commonly make the mistake of putting the functions in the wrong order. If your output is different from what you expected, try changing the function order.

Some functions work on the string as an input rather than on the string instance. The `max()` function falls into this category. If you had typed `MyString.max()`, Python would have displayed an error. The bulleted list that appears earlier in this section shows which functions require this sort of string input by including a *`str`* input as part of the function declaration.

When working with functions that produce a list as an output, you can access an individual member by providing an index to it. The example shows how to use `split()` to split the string into substrings. It then shows how to access just the first substring in the list. You find out more about working with lists in Chapter 13.

2. **Click Run Cell.**

   Python outputs a number of modified strings, as shown in Figure 12-4.

FIGURE 12-4: Using functions makes string manipulation a lot more flexible.

Locating a Value in a String

Sometimes you need to locate specific information in a string. For example, you may want to know whether a string contains the word Hello in it. One of the essential purposes behind creating and maintaining data is to be able to search it later to locate specific bits of information. Strings are no different — they're most useful when you can find what you need quickly and without any problems. Python provides a number of functions for searching strings. Here are the most commonly used functions:

- `count(str, beg= 0, end=len(string))`: Counts how many times *str* occurs in a string. You can limit the search by specifying a beginning index using *beg* or an ending index using *end*.
- `endswith(suffix, beg=0, end=len(string))`: Returns `True` when a string ends with the characters specified by *suffix*. You can limit the check by specifying a beginning index using *beg* or an ending index using *end*.
- `find(str, beg=0, end=len(string))`: Determines whether *str* occurs in a string and outputs the index of the location. You can limit the search by specifying a beginning index using *beg* or an ending index using *end*. This function returns a value of `-1` when the target isn't present.
- `index(str, beg=0, end=len(string))`: Provides the same functionality as `find()`, but raises an exception when *str* isn't found.
- `replace(old, new [, max])`: Replaces all occurrences of the character sequence specified by *old* in a string with the character sequence specified by *new*. You can limit the number of replacements by specifying a value for *max*.
- `rfind(str, beg=0, end=len(string))`: Provides the same functionality as `find()`, but searches backward from the end of the string instead of the beginning.
- `rindex(str, beg=0, end=len(string))`: Provides the same functionality as `index()`, but searches backward from the end of the string instead of the beginning.
- `startswith(prefix, beg=0, end=len(string))`: Returns `True` when a string begins with the characters specified by *prefix*. You can limit the check by specifying a beginning index using *beg* or an ending index using *end*.

Finding the data that you need is an essential programming task — one that is required no matter what kind of application you create. The following steps help you create an example that demonstrates the use of search functionality within strings.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
SearchMe = "The apple is red and the berry is blue!"
print(SearchMe.find("is"))
print(SearchMe.rfind("is"))
print(SearchMe.count("is"))
print(SearchMe.startswith("The"))
print(SearchMe.endswith("The"))
print(SearchMe.replace("apple", "car")
    .replace("berry", "truck"))
```

   The example begins by creating `SearchMe`, a string with two instances of the word `is`. The two instances are important because they demonstrate how searches differ depending on where you start. When using `find()`, the example starts from the beginning of the string. By contrast, `rfind()` starts from the end of the string.

   Of course, you won't always know how many times a certain set of characters appears in a string. The `count()` function lets you determine this value.

   Depending on the kind of data you work with, sometimes the data is heavily formatted and you can use a particular pattern to your advantage. For example, you can determine whether a particular string (or substring) ends or begins with a specific sequence of characters. You could just as easily use this technique to look for a part number.

   The final bit of code replaces `apple` with `car` and `berry` with `truck`. Notice the technique used to place the code on two lines. In some cases, your code will need to appear on multiple lines to make it more readable.

2. **Click Run Cell.**

   Python displays the output shown in Figure 12-5. Notice especially that the searches returned different indexes based on where they started in the string. Using the correct function when performing searches is essential to ensure that you get the results you expected.

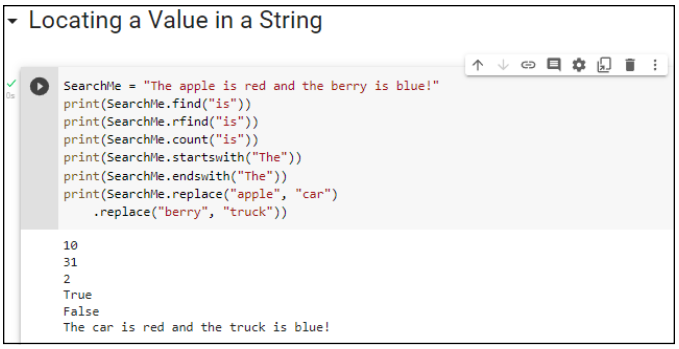


FIGURE 12-5: Using string search features correctly is essential.

Using String Interpolation

You can format strings in a number of ways using Python. The main emphasis of formatting is to present the string in a form that is both pleasing to the user and easy to understand. Formatting doesn't mean adding special fonts or effects in this case, but refers merely to the presentation of the data. For example, the user might want a fixed-point number rather than a decimal number as output. The following sections discuss and demonstrate various techniques for formatting strings, a process known as *string interpolation* (just so that programmers can sound fancy).

Employing the % (modulo) approach

In some cases, all you really need is simple information replacement in a string. The `%` (modulo) operator approach is the simplest form of formatting that Python provides, but it also sees less use in new applications because it's extremely limited and there are better methods (as explored in the sections that follow this one). Even so, if you're in a hurry and need something quite simple and easy to troubleshoot, the `%` operator may be the way to go.

When using the `%` operator, you place the `%` operator in every location where you want to see some information appear, along with the type of that information, as shown in the following table:

Operator	Description	Operator	Description	Operator	Description
%c	Character	%d	Decimal integer	%e	Float exponent (lowercase)
%E	Float exponent (uppercase)	%f	Float (lowercase)	%F	Float (uppercase)

Operator	Description	Operator	Description	Operator	Description
%g	General format (lowercase)	%G	General format (uppercase)	%i	Integer
%o	Octal integer	%s	String	%u	Unsigned integer
%x	Hexadecimal integer				

The % operator also allows the addition of some formatting information, as described in the next section of the chapter for the format() function, but this formatting information is generally limited to number of decimal places. The following steps show how to use the % operator in various ways:

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
print("%c is lowercase %c" %('a', 65))
print("%s are %s" %("Violets", "Blue"))
print("%d is %#x hexadecimal" %(60, 60))
```

In each case, you see a print() function that uses one of the % operator replacements, such as %c in the first case. Notice that when using %c you can use either a character, 'a', or an ASCII number, 65. The values you want to substitute for each of the % operator entries appear in a tuple (a series of values enclosed in paretheses and delimited by commas) with the % operator outside of it. The values are always replaced sequentially, and you must provide enough values for each of the % operator entries in the string.

The % operator handles characters differently from strings. When working with strings, you use the %s operator.

The third print() function call is for the integer value 60. Notice that you must provide 60 twice, once for each % operator entry. In this case, the values are output the first time as a decimal integer and the second time as a hexadecimal integer. The # in the %#x operator tells Python to place a 0x in front of the output. The # character is one of a number of *format flags:*

- #: Adds 0x in front of hexadecimal output.
- +: Adds a plus sign in front of positive values.
- –: Left justifies the output, rather than using the default right justification.
- " ": Inserts spaces before a numeric output.
- 0: Inserts zeros before the numeric output.

2. **Click Run Cell.**

Python outputs data in various forms:

```
a is lowercase A
Violets are Blue
60 is 0x3c hexadecimal
```

at the next step results in some examples of numeric formatting.

3. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
Value1 = 100
Value2 = float('inf')
print('% e is smaller than %E.' %(Value1, Value2))
print("%.3F is smaller than %.2f." %(Value1,
                                     Value2))
print("%05d" %Value1)
```

This part of the example focuses on an integer value of `100` and a `float` value of infinity. The code formats the two values in various ways. Notice the use of uppercase and lowercase `%` operators. In addition, there is a space between the `%` and the `e` in the third line, which adds a space in front of the output value. (Adding more spaces won't add more spaces to the output.) The fourth line has a `.3` in front of the `F` that controls the number of digits after the decimal point. The last line shows how to add zeros in front of the number.

4. **Click Run Cell.**

Python outputs data in various forms:

```
 1.000000e+02 is smaller than INF.
100.000 is smaller than inf.
00100
```

The final part of the example shows a special use for `%` operator formatting when working with a dictionary.

5. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
Employees = {
    'Monica': {
        'Occupation': "Designer",
        'Name': "Monica",
        'Department': "Engineering"
    },
```

```
    'Sam': {
        'Occupation': "Programmer",
        'Name': "Sam",
        'Department': "Software"
    }
}

for Item in Employees:
    print("%(Name)s is a %(Occupation)s in " \
        "%(Department)s." %Employees[Item])
```

This example uses a nested dictionary. Don't worry too much about the precise use of dictionaries for now; you discover how to use them in Chapter 14. For now, think about this code as being a kind of database. It contains two employees, `Monica` and `Sam`. There are entries for each employee: `Occupation`, `Name`, and `Department`.

A `for` loop comes next. It gets the name of an employee (not the actually employee information) from `Employees` and uses that name to access a particular employee record using `%Employees[Item]`. Now you have a single employee record, and you can access the information by name using the `%` and enclosing the information you want within a tuple, such as `%(Name)`. As always, you must tell Python what kind of data format to use, so the entire operator would be `%(Name)s`.

TIP

Notice the use of the line-continuation character, `\`, in this part of the example. Whenever your code becomes too long, you can use a line-continuation character to continue the code on the next line.

6. **Click Run Cell.**

   Python outputs data in various forms. Figure 12-6 shows the outputs for this entire example.

Working with the format() function

The `format()` function provides more functionality and flexibility than using the `%` operator shown in the previous section. You create a formatting specification as part of the string and then use the `format()` function to add data to that string. A format specification may be as simple as two curly brackets `{}` that specify a placeholder for data. You can number the placeholder to create special effects. For example, `{0}` would contain the first data element in a string. When the data elements are numbered, you can even repeat them so that the same data appears more than once in the string.

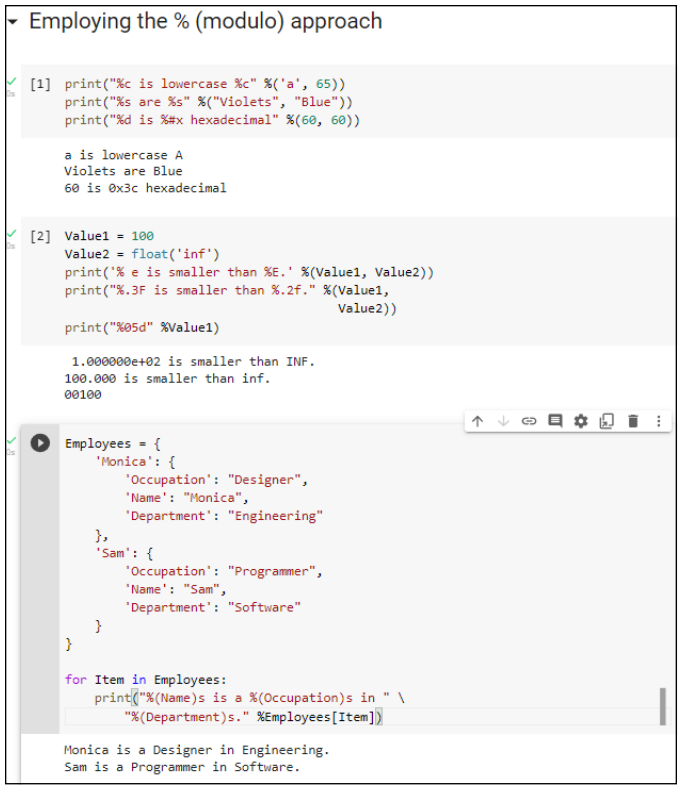


FIGURE 12-6: The % operator is good for simple output needs.

The formatting specification follows a colon. When you want to create just a formatting specification, the curly brackets contain just the colon and whatever formatting you want to use. For example, `{:f}` would create a fixed-point number as output. If you want to number the entries, the number that precedes the colon: `{0:f}` creates a fixed-point number output for data element one. The formatting specification follows this form, with the italicized elements serving as placeholders here:

```
[[fill]align][sign][#][0][width][,][.precision][type]
```

The specification at `https://docs.python.org/3/library/string.html` provides you with the in-depth details, but here's an overview of what the various entries mean:

- **fill:** Defines the fill character used when displaying data that is too small to fit within the assigned space.

- **align:** Specifies the alignment of data within the display space. You can use these alignments:
  - <: Left aligned
  - >: Right aligned
  - ^: Centered
  - =: Justified
- **sign:** Determines the use of signs for the output:
  - +: Positive numbers have a plus sign and negative numbers have a minus sign.
  - –: Negative numbers have a minus sign.
  - *<space>*: Positive numbers are preceded by a space and negative numbers have a minus sign.
- **#:** Specifies that the output should use the alternative display format for numbers. For example, hexadecimal numbers will have a 0x prefix added to them.
- **0:** Specifies that the output should be sign aware and padded with zeros as needed to provide consistent output.
- **width:** Determines the full width of the data field (even if the data won't fit in the space provided).
- **,:** Specifies that numeric data should have commas as a thousands separator.
- **.precision:** Determines the number of characters after the decimal point.
- **type:** Specifies the output type, even if the input type doesn't match. The types are split into three groups:
  - *String:* Use an `s` or nothing at all to specify a string.
  - *Integer:* The integer types are as follows: `b` (binary); `c` (character); `d` (decimal); `o` (octal); `x` (hexadecimal with lowercase letters); `X` (hexadecimal with uppercase letters); and `n` (locale-sensitive decimal that uses the appropriate characters for the thousands separator).
  - *Floating point:* The floating-point types are as follows: `e` (exponent using a lowercase e as a separator); `E` (exponent using an uppercase E as a separator); `f` (lowercase fixed point); `F` (uppercase fixed point); `g` (lowercase general format); `G` (uppercase general format); `n` (locale-sensitive general format that uses the appropriate characters for the decimal and thousands separators); and `%` (percentage).

The formatting specification elements must appear in the correct order or Python won't know what to do with them. If you specify the alignment before the fill character, Python displays an error message rather than performing the required formatting. The following steps help you see how the formatting specification works and demonstrate the order you need to follow in using the various formatting specification criteria.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
Formatted = "{:d}"
print(Formatted.format(7000))
Formatted = "{:,d}"
print(Formatted.format(7000))
Formatted = "{:^15,d}"
print(Formatted.format(7000))
Formatted = "{:*^15,d}"
print(Formatted.format(7000))
Formatted = "{:*^15.2f}"
print(Formatted.format(7000))
Formatted = "{:*>15X}"
print(Formatted.format(7000))
Formatted = "{:*<#15x}"
print(Formatted.format(7000))
Formatted = "A {0} {1} and a {0} {2}."
print(Formatted.format("blue", "car", "truck"))
```

   The example starts simply with a field formatted as a decimal value. It then adds a thousands separator to the output. The next step is to make the field wider than needed to hold the data and to center the data within the field. Finally, the field has an asterisk added to pad the output.

   Of course, the example contains other data types. The next step is to display the same data in fixed-point format. The example also shows the output in both uppercase and lowercase hexadecimal format. The uppercase output is right aligned and the lowercase output is left aligned.

   Finally, the example shows how you can use numbered fields to your advantage. In this case, it creates an interesting string output that repeats one of the input values.

2. **Click Run Cell.**

   Python outputs data in various forms, as shown in Figure 12-7.

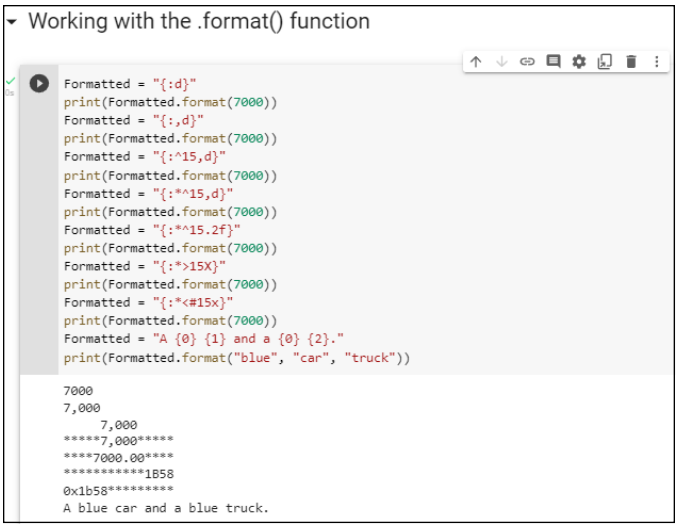


FIGURE 12-7: Use formatting to present data in precisely the form you want.

Simplifying things using f-string

Most developers today use the f-string because it emcompasses all the functionality of earlier Python string formatting and adds a few new tricks to it. An *f-string* is simply a string that is preceded by the lowercase letter *f*. In many respects, it looks like a less complicated version of the `format()` method demonstrated in the previous section. With this similarity in mind, it's time to look at an example.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
Value1 = 22
Value2 = 33
print(f"Value1 has a value of {Value1}.")
print(f"{Value1} + {Value2} = {Value1 + Value2}.")
print(f"This value: {str(Value1).center(8)}" \
      " is centered.")
print(f"Value1 as a hexadecimal value of " \
      f"{Value1:x}.")
```

This example starts with two values: `Value1` and `Value2`. The first `print()` call shows how you begin the string with the lowercase letter `f` and then add the content within single or double quotes. Notice how the use of a variable name within the curly braces (`{}`) makes reading the statement easy.

Unlike other forms of string formatting, an f-string can contain expressions, such as `{Value1 + Value2}`. In fact, you can make function calls and perform data conversion as part of an f-string like this: `{str(Value1).center(8)}`.

You still have the ability to use all the formatting functionality offered by the `format()` method, so f-strings can have formatting instructions like this: `{Value1:x}`, which produces hexadecimal output.

REMEMBER

Notice that if you have a line-contuation character but the next line doesn't contain a variable or expression, you don't need to preced it with an `f`. However, if the next line does contain replaceable content, you must include the `f` or Python won't know how to interpret the content.

2. **Click Run Cell.**

Python outputs data in various forms, as shown in Figure 12-8.

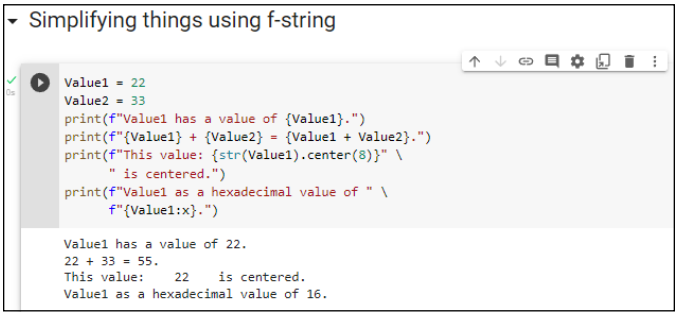


FIGURE 12-8: The f-string approach is simpler than other methods, yet quite flexible.

Creating and using string templates

String templates are for those times when you repeat the same information in different places in your application, but with different data. The template is simply a string, but it uses a $ in front of the replaceable values instead of the other methods shown in previous sections. The best way to see how a string template works is to see it in action, which you can do by following these steps:

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
from string import Template

X = 1
Y = 2

Sums = Template("$Value1 + $Value2 = $Value3")
print(Sums.substitute(Value1 = X, Value2 = Y,
                      Value3 = X + Y))
```

To use a string template, you must import `Template` from the `string` package using the `import` statement shown. (Chapter 11 tells you how packages work and how to use `import` statements.)

The code begins by creating a variable, `Sums`, as a template that contains three replacement values: `$Value1`, `$Value2`, and `$Value3`. You can replace these values with anything and in any order, as shown in the next line.

2. **Click Run Cell.**

   Python outputs the expected templated string, as shown in Figure 12-9.

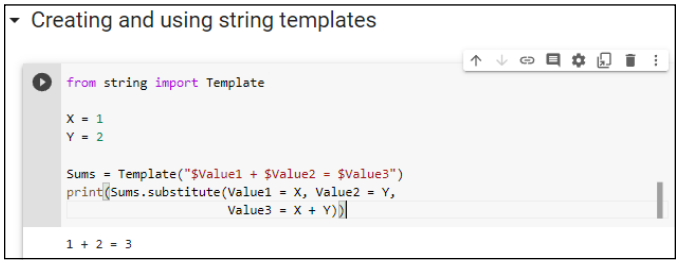


FIGURE 12-9: String templates enable you to create reusable content with ease.

REMEMBER

The replaceable values can literally be anything. For example, you could just as easily replace the preceding `print()` call with `print(Sums.substitute(Value1 = 'A', Value2 = "{:x}".format(90), Value3 = True))`. The output would be `A + 5a = True`. Of course, no one ever said that the output has to make sense.

IN THIS CHAPTER

- Defining why lists are important
- Generating lists
- Managing lists
- Using the Counter object to your advantage

Chapter 13
Managing Lists

A lot of people lose sight of the fact that most programming techniques are based on the real world. Part of the reason is that programmers often use terms that other people don't to describe these real-world objects. For example, most people would call a place to store something a box or a cupboard — but programmers insist on using the term *variable.* Lists are different. Everyone makes lists and uses them in various ways to perform an abundance of tasks. In fact, you're probably surrounded by lists of various sorts where you're sitting right now as you read this book. So, this chapter is about something you already use quite a lot. The only difference is that you need to think of lists in the same way Python does.

You may read that lists are hard to work with. The reason that some people find working with lists difficult is that they're not used to actually thinking about the lists they create. When you create a list, you simply write items down in whatever order makes sense to you. Sometimes you rewrite the list when you're done to put it in a specific order. In other cases, you use your finger as a guide when going down the list to make looking through it easier. The point is that everything you normally do with lists is also doable within Python. The difference is that you must now actually think about what you're doing in order to make Python understand what you want done.

Lists are incredibly important in Python. This chapter introduces you to the concepts used to create, manage, search, and print lists (among other tasks). When you complete the chapter, you can use lists to make your Python applications

more robust, faster, and more flexible. In fact, you'll wonder how you ever got along without using lists in the past. The important thing to keep in mind is that you have already used lists most of your life. There really isn't any difference now except that you must now think about the actions that you normally take for granted when managing your own lists.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 13; Managing Lists.ipynb` file of the downloadable source. See the Introduction for details on how to find these source files.

Organizing Information in an Application

People create lists to organize information and make it easier to access and change. You use lists in Python for the same reason. In many situations, you really do need some sort of organizational aid to hold data. For example, you might want to create a single place to look for days of the week or months of the year. The names of these items would appear in a list, much as they would if you needed to commit them to paper in the real world. The following sections describe lists and how they work in more detail.

Defining organization using lists

The Python specification defines a list as a kind of sequence. *Sequences* simply provide some means of allowing multiple data items to exist together in a single storage unit, but as separate entities. Think about one of those large mail holders you see in apartment buildings. A single mail holder contains a number of small mailboxes, each of which can contain mail. Python supports other kinds of sequences as well (Chapter 14 discusses a number of these sequences):

- Tuples
- Dictionaries
- Stacks
- Queues
- Deques

REMEMBER

Of all the sequences, lists are the easiest to understand and are the most directly related to a real-world object. Working with lists helps you become better able to work with other kinds of sequences that provide greater functionality and improved flexibility. The point is that the data is stored in a list much as you

would write it on a piece of paper — one item comes after another. The list has a beginning, a middle, and an end. (Even if you might not normally number them in real life, Python always numbers the items for you.)

Understanding how computers view lists

The computer doesn't view lists in the same way that you do. It doesn't have an internal notepad and use a pen to write on it. A computer has memory. The computer stores each item in a list in a separate memory location. The memory is contiguous, so as you add new items, they're added to the next location in memory. In many respects, the computer uses something like a mailbox to hold your list. The list as a whole is the mail holder. As you add items, the computer places them in the next mailbox within the mail holder.

REMEMBER

Just as the mailboxes are numbered in a mail holder, the memory slots used for a list are numbered. The numbers begin with 0, not with 1 as you might expect. (This is the same numbering system as that used with strings, as explained in the "Selecting Individual Characters" section of Chapter 12.) Each mailbox receives the next number in line. A mail holder with the months of the year would contain 12 mailboxes. The mailboxes would be numbered from 0 to 11 (not 1 to 12, as you might think). Getting the numbering scheme down as quickly as possible is essential because even experienced developers get into trouble by using 1 and not 0 as a starting point at times.

Depending on what sort of information you place in each mailbox, the mailboxes need not be of the same size. Python lets you store a string in one mailbox, an integer in another, and a floating-point value in another. The computer doesn't know what kind of information is stored in each mailbox and it doesn't care. All that the computer sees is slots that could hold anything. Python performs all the work required to treat the data elements according to the right type and to ensure that when you request item five, you actually get item five.

TIP

In general, it's good practice to create lists of like items to make the data easier to manage. When creating a list of all integers, for example, rather than of mixed data, you can make assumptions about the information and don't have to spend nearly as much time checking it. However, in some situations, you might need to mix data. Many other programming languages require that lists have just one type of data, but Python offers the flexibility of using mixed data sorts. Just remember that using mixed data in a list means that you must determine the data type when retrieving the information in order to work with the data correctly. Treating a string as an integer would cause problems in your application.

Creating Lists

As in real life, before you can do anything with a list, you must create it. As previously stated, Python lists can mix types. However, restricting a list to a single type when you can is always the best practice. The following steps demonstrate how to create Python lists.

1. **Open a new notebook.**

   You can also use the downloadable source file, `BPP4D3E; 13; Managing Lists.ipynb`.

2. **Type the following code into a new cell in the notebook — pressing Enter after each line.**

```
List1 = ["One", 1, "Two", True]
print(List1)
```

   Python creates a list named `List1` for you. This list contains two string values (`"One"` and `"Two"`), an integer value (`1`), and a Boolean value (`True`).

REMEMBER

   Notice that each data type that you type is a different color. The color of an entry is a cue that tells you whether you have typed the entry correctly, which helps reduce errors when creating a list.

3. **Click Run Cell.**

   You see the content of the list as a whole, as shown in Figure 13-1. Notice that the string entries appear in single quotes, even though you typed them using double quotes. Strings can appear in either single quotes or double quotes in Python.

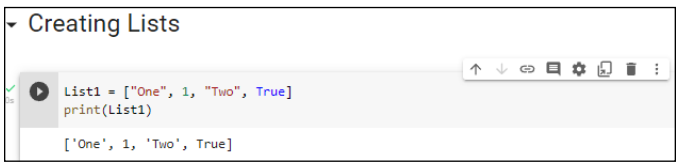


FIGURE 13-1: Python displays the content of `List1`.

4. **In a new cell in the notebook, type** dir(List1) **and click Run Cell.**

   Python displays a list of actions that you can perform using lists. Notice that the output is actually a list. So, you're using a list to determine what you can do with another list.

REMEMBER

As you start working with objects of greater complexity, you need to remember that the `dir()` command always shows what tasks you can perform using that object. The actions that appear without underscores are the main actions that you can perform using a list. These actions are the following:

- append
- clear
- copy
- count
- extend
- index
- insert
- pop
- remove
- reverse
- sort

Accessing Lists

After you create a list, you want to access the information it contains. An object isn't particularly useful if you can't at least access the information it contains. The previous section shows how to use the `print()` and `dir()` functions to interact with `List1`, but other ways exist to perform the task, as described in the following steps.

1. **In a new cell in the notebook, type** List1[1] **and click Run Cell.**

   You see the value `1` as output. The use of a number within a set of square brackets is called an *index.* Python always uses zero-based indexes, so asking for the element at index `1` means getting the second element in the list.

2. **In a new cell in the notebook, type** List1[1:3] **and click Run Cell.**

   You see a range of values that includes two elements. When typing a range, the end of the range is always one greater than the number of elements returned. In this case, that means that you get elements 1 and 2, not elements 1 through 3 as you might expect.

3. **In a new cell in the notebook, type** List1[1:] **and click Run Cell.**

   You see all the elements, starting from element `1` to the end of the list. A range can have a blank ending number, which simply means to print the rest of the list.

4. **In a new cell in the notebook, type** List1[:3] **and click Run Cell.**

   Python displays the elements from 0 through 2. Leaving the start of a range blank means that you want to start with element 0. Figure 13-2 shows the result of the various list-indexing tasks you've performed so far.

FIGURE 13-2: Knowing the correct indexing operation to perform is important.

TECHNICAL STUFF

Even though doing so is really confusing, you can use negative indexes with Python. Instead of working from the left, Python will work from the right and backward. For example, if you have `List1 = ["One", 1, "Two", True]` and type `List1[-2]`, you get `Two` as output. Likewise, typing `List1[-3]` results in an output of `1`. The rightmost element is element `-1` in this case.

Looping through Lists

To automate the processing of list elements, you need some way to loop through the list. The easiest way to perform this task is to rely on a `for` statement, as described in the following steps.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line.**

```
List1 = [0, 1, 2, 3, 4, 5]
for Item in List1:
   print(Item)
```

The example begins by creating a list consisting of numeric values. It then uses a `for` loop to obtain each element in turn and print it onscreen.

2. **Click Run Cell.**

   Python shows the individual values in the list, one on each line, as shown in Figure 13-3.

FIGURE 13-3: A loop makes it easy to obtain a copy of each item and process it as needed.

Modifying Lists

You can modify the content of a list as needed. Modifying a list means to change a particular entry, add a new entry, or remove an existing entry. To perform these tasks, you must sometimes read an entry. The concept of modification is found within the acronym CRUD, which stands for Create, Read, Update, and Delete. Here are the list functions associated with CRUD:

- `append()`: Adds a new entry to the end of the list.
- `clear()`: Removes all entries from the list.
- `copy()`: Creates a copy of the current list and places it in a new list.
- `extend()`: Adds items from an existing list and into the current list.
- `insert()`: Adds a new entry to the position specified in the list.
- `pop()`: Removes an entry from the end of the list.
- `remove()`: Removes an entry from the specified position in the list.

The following steps show how to perform modification tasks with lists. This is a hands-on exercise. As the book progresses, you see these same functions used

within application code. The purpose of this exercise is to help you gain a feel for how lists work.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line.**

```
List2 = []
len(List2)
```

REMEMBER

Python creates a list named `List2` for you. Notice that the square brackets are empty. `List2` doesn't contain any entries. You can create empty lists that you fill with information later. In fact, this is precisely how many lists start because you usually don't know what information they will contain until the user interacts with the list.

2. **Click Run Cell.**

The `len()` function outputs 0. When creating an application, you can check for an empty list by using the `len()` function. If a list is empty, you can't perform tasks such as removing elements from it because there is nothing to remove.

3. **Type the following code into a new cell in the notebook — pressing Enter after each line.**

```
List2.append(1)
len(List2)
```

4. **Click Run Cell.**

The `len()` function now reports a length of 1.

5. **In a new cell in the notebook, type** List2[0] **and click Run Cell.**

You see the value of `1` stored in element `0` of `List2`.

6. **Type the following code into a new cell in the notebook — pressing Enter after each line.**

```
List2.insert(0, 2)
List2
```

The `insert()` function requires two arguments. The first argument is the index of the insertion, which is element `0` in this case. The second argument is the object you want inserted at that point, which is `2` in this case.

7. **Click Run Cell.**

Python has added another element to List2. However, using the `insert()` function lets you add the new element before the first element, so the output is now `[2, 1]`.

8. **Type the following code into a new cell in the notebook — pressing Enter after each line.**

```
List3 = List2.copy()
List2.extend(List3)
List2
```

The new list, `List3`, is a precise copy of `List2`. Copying is often used to create a temporary version of an existing list so that a program can make temporary modifications to it rather than to the original list. When the user is done, the application can either delete the temporary list or copy it to the original list.

Python copies all the elements in `List3` to the end of `List2`. Extending is commonly used to consolidate two lists.

9. **Click Run Cell.**

You see that the copy and extend processes have worked. `List2` now contains the values `[2, 1, 2, 1]`.

10. **Type** List2.pop() **into a new cell of the notebook and click Run Cell.**

Python displays a value of 1. The 1 was stored at the end of the list, and pop( ) always removes values from the end.

USING OPERATORS WITH LISTS

Lists can also rely on operators to perform certain tasks. For example, if you want to create a list that contains four copies of the word *Hello*, you could use `MyList = ["Hello"] * 4` to fill it. A list allows repetition as needed. The multiplication operator (`*`) tells Python how many times to repeat a given item. You need to remember that every repeated element is separate, so what `MyList` contains is `['Hello', 'Hello', 'Hello', 'Hello']`.

You can also use concatenation to fill a list. For example, using `MyList = ["Hello"] + ["World"] + ["!"] * 4` creates six elements in `MyList`. The first element is `Hello`, followed by `World` and ending with four elements with one exclamation mark (`!`) in each element.

The membership operator (`in`) also works with lists. This chapter uses a straightforward and easy-to-understand method of searching lists (the recommended approach). However, you can use the membership operator to make things shorter and simpler by using `"Hello" in MyList`. Assuming that you have your list filled with `['Hello', 'World', '!', '!', '!', '!']`, the output of this statement is `True`.

11. **Type** List2.remove(1) **into a new cell of the notebook and click Run Cell.**

    This time, Python removes the item at element 1. Unlike the `pop()` function, the `remove()` function doesn't display the value of the item it removed.

12. **Type the following code into a new cell in the notebook — pressing Enter after each line.**

```
List2.clear()
len(List2)
```

    Using `clear()` means that the list shouldn't contain any elements now. The `len()` function will affirm that `List2` is definitely empty.

13. **Click Run Cell.**

    You see that the output is `0`. `List2` is definitely empty. At this point, you've tried all the modification methods that Python provides for lists as shown in Figure 13-4. Work with `List2` some more using these various functions until you feel comfortable making changes to the list.

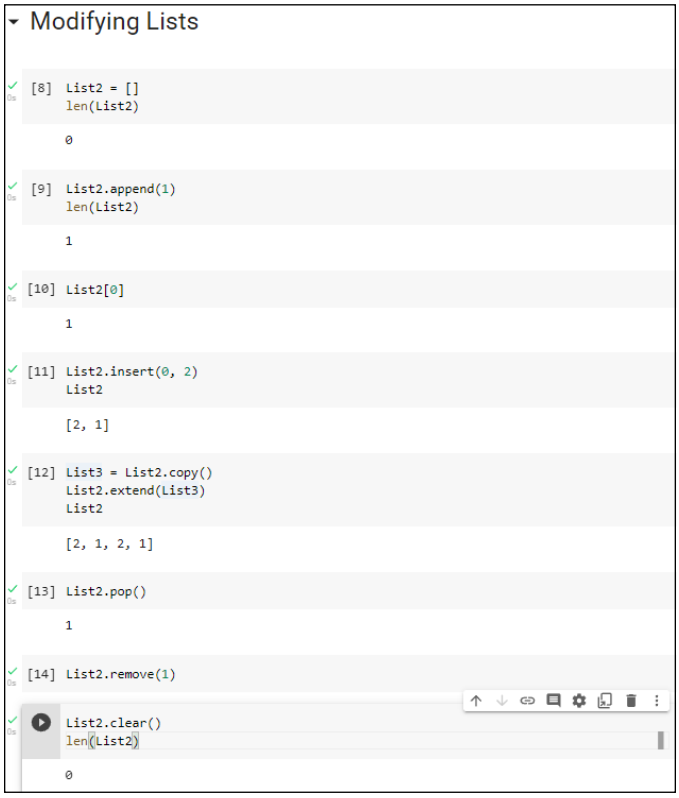


FIGURE 13-4: Python provides a wide range of actions you can perform using lists.

Searching Lists

Modifying a list isn't very easy when you don't know what the list contains. The ability to search a list is essential if you want to make maintenance tasks easier. The following steps help you create an application that demonstrates the ability to search a list for specific values.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
Colors = ["Red", "Orange", "Yellow", "Green", "Blue"]
ColorSelect = ""
while str.upper(ColorSelect) != "QUIT":
   ColorSelect = input("Please type a color name: ")
   if (Colors.count(ColorSelect) >= 1):
      print("The color exists in the list!")
   elif (str.upper(ColorSelect) != "QUIT"):
      print("The list doesn't contain the color.")
```

The example begins by creating a list named `Colors` that contains color names. It also creates a variable named `ColorSelect` to hold the name of the color that the user wants to find. The application then enters a loop where the user is asked for a color name that is placed in `ColorSelect`. As long as this variable doesn't contain the word `QUIT`, the application continues a loop that requests input.

Whenever the user inputs a color name, the application asks the list to count the number of occurrences of that color. When the value is equal to or greater than one, the list does contain the color and an appropriate message appears onscreen. On the other hand, when the list doesn't contain the requested color, an alternative message appears onscreen.

TIP

Notice how this example uses an `elif` clause to check whether `ColorSelect` contains the word `QUIT`. This technique of including an `elif` clause ensures that the application doesn't output a message when the user wants to quit the application. You need to use similar techniques when you create your applications to avoid potential user confusion or even data loss (when the application performs a task the user didn't actually request).

2. **Click Run Cell.**

Python asks you to type a color name.

3. **Type** Blue **and press Enter.**

You see a message telling you that the color does exist in the list.

4. **Type** Purple **and press Enter.**

   You see a message telling you that the color doesn't exist.

5. **Type** Quit **and press Enter.**

   The application ends. Notice that the application displays neither a success nor a failure message. Figure 13-5 shows the results of the various inputs.

FIGURE 13-5: Combining loops with conditional code allows multiple selection entries.

Sorting Lists

The computer can locate information in a list no matter what order it appears in. It's a fact, though, that longer lists are easier to search when you put them in sorted order. However, the main reason to put a list in sorted order is to make it easier for the human user to actually see the information the list contains. People work better with sorted information. This example begins with an unsorted list. It then sorts the list and outputs it to the display. The following steps demonstrate how to perform this task.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
Colors = ["Red", "Orange", "Yellow", "Green", "Blue"]
for Item in Colors:
   print(Item, end=" ")
print()
Colors.sort()
for Item in Colors:
   print(Item, end=" ")
print()
```

The example begins by creating an array of colors. The colors are currently in unsorted order. The example then prints the colors in the order in which they appear. Notice the use of the `end=" "` argument for the `print()` function to ensure that all color entries remain on one line (making them easier to compare).

REMEMBER

Sorting the list is as easy as calling the `sort()` function. After the example calls the `sort()` function, it prints the list again so that you can see the result.

2. **Click Run Cell.**

Python outputs both the unsorted and sorted lists, as shown in Figure 13-6.

FIGURE 13-6: Sorting a list is as easy as calling the `sort()` function.

TIP

You may need to sort items in reverse order at times. To accomplish this task, you use the `reverse()` function. The function must appear on a separate line. So the previous example would look like this if you wanted to sort the colors in reverse order:

```
Colors = ["Red", "Orange", "Yellow", "Green", "Blue"]
for Item in Colors:
   print(Item, end=" ")
print()
Colors.sort()
Colors.reverse()
for Item in Colors:
   print(Item, end=" ")
print()
```

Printing Lists

Python provides myriad ways to output information. In fact, the number of ways would amaze you. This chapter has shown just a few of the most basic methods for outputting lists so far, using the most basic methods. Real-world printing can

become more complex, so you need to know a few additional printing techniques to get you started. Using these techniques is actually a lot easier if you play with them as you go along.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
Colors = ["Red", "Orange", "Yellow", "Green", "Blue"]
print(*Colors, sep='\n')
```

This example begins by using the same list of colors in the previous section. In that section, you use a `for` loop to print the individual items. This example takes another approach. It uses the splat (`*`) operator, also called the positional expansion operator (and an assortment of other interesting terms), to unpack the list and send each element to the `print()` method one item at a time. The `sep` argument tells how to separate each of the printed outputs, relying on a newline character in this case.

2. **Click Run Cell.**

Python outputs the list one item at a time.

3. **Type the following code into a new cell in the notebook and click Run Cell.**

```
for Item in Colors: print(Item.rjust(8), sep='\n')
```

Code doesn't have to appear on multiple lines. This example takes two lines of code and places them on just a single line. However, it also demonstrates the use of the `rjust()` method, which right justifies the string. Numerous methods of this sort are described at `https://docs.python.org/3.8/library/string.html`.

4. **Type the following code into a new cell in the notebook and click Run Cell.**

```
print('\n'.join(Colors))
```

Python provides more than one way to perform any task. In this case, the code uses the `join()` method to join the newline character with each member of `Colors`. The output is the same as that shown in Step 3, even though the approach is different. The point is to use the approach that best suits a particular need.

5. **Type the following code into a new cell in the notebook and click Run Cell.**

```
print('First: {0}\nSecond: {1}'.format(*Colors))
```

In this case, the output is formatted in a specific way with accompanying text, and the result doesn't include every member of `Colors`. The `{0}` and `{1}` entries represent placeholders for the values supplied from `*Colors`. Figure 13-7 shows the output from all of the techniques demonstrated in these steps. You can read more about this approach (the topic is immense) at `https://docs.python.org/3/tutorial/inputoutput.html`.

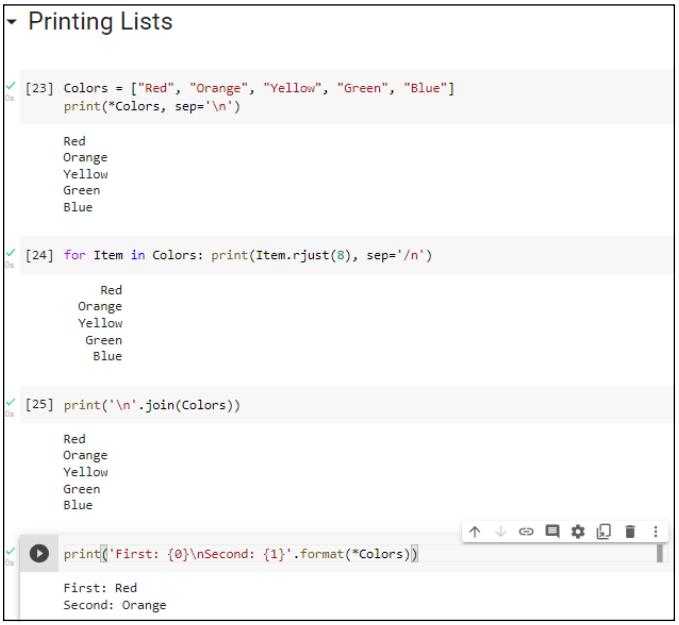


FIGURE 13-7: Python provides a lot of different ways to print lists.

REMEMBER

This section touches on only some of the common techniques used to format output in Python. There are lots more. You see many of these approaches demonstrated in the chapters that follow. Chapter 12 also demonstrates a number of ways to work with strings. These techniques are also useful when working with lists. The essential goal is to use a technique that's easy to read, works well with all anticipated inputs, and doesn't paint you into a corner when you're creating additional output later.

Working with the Counter Object

Sometimes you have a data source and you simply need to know how often things happen (such as the appearance of a certain item in the list). When you have a short list, you can simply count the items. However, when you have a really long

list, getting an accurate count is nearly impossible. For example, consider what it would take if you had a really long novel like *War and Peace* in a list and wanted to know the frequency of the words the novel used. The task would be nearly impossible without a computer. (Some people would insist on counting those words one by one, using 3-x-5 cards to record the result — you know who you are.)

REMEMBER

The `Counter` object lets you count items quickly. In addition, it's incredibly easy to use. This book shows the `Counter` object in use a number of times, but this chapter shows how to use it specifically with lists. The example in this section creates a list with repetitive elements and then counts how many times those elements actually appear.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
from collections import Counter

MyList = [1, 2, 3, 4, 1, 2, 3, 1, 2, 1, 5]
ListCount = Counter(MyList)
print(ListCount)
for ThisItem in ListCount.items():
   print("Item: ", ThisItem[0],
         " Appears: ", ThisItem[1])
print("The value 1 appears {0} times."
      .format(ListCount.get(1)))
```

To use the `Counter` object, you must import it from `collections`. Of course, if you work with other collection types in your application, you can import the entire `collections` package by typing **import collections** instead.

The example begins by creating a list, `MyList`, with repetitive numeric elements. You can easily see that some elements appear more than once. The example places the list into a new `Counter` object, `ListCount`. You can create `Counter` objects in all sorts of ways, but this is the most convenient method when working with a list.

WARNING

The `Counter` object and the list aren't actually connected in any way. When the list content changes, you must re-create the `Counter` object because it won't automatically see the change. An alternative to re-creating the counter is to call the `clear()` method first and then call the `update()` method to fill the `Counter` object with the new data.

The application prints `ListCount` in various ways. The first output is the `Counter` as it appears without any manipulation. The second output prints the individual unique elements in `MyList` along with the number of times each element appears. To obtain both the element and the number of times it

appears, you must use the `items()` function as shown. Finally, the example demonstrates how to obtain an individual count from the list by using the `get()` function.

2. **Click Run Cell.**

   Python outputs the results of using the `Counter` object, as shown in Figure 13-8.

   Notice that the information is actually stored in the `Counter` as a key and value pair. Chapter 14 discusses this topic in greater detail. All you really need to know for now is that the element found in `MyList` becomes a key in `ListCount` that identifies the unique element name. The value contains the number of times that that element appears within `MyList`.

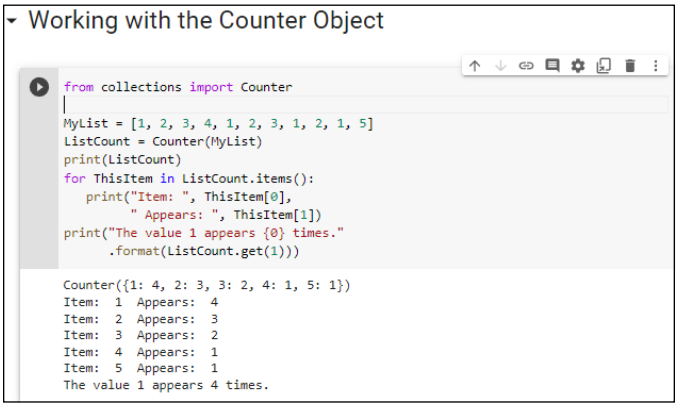


FIGURE 13-8: The `Counter` is helpful in obtaining statistics about longer lists.

IN THIS CHAPTER

- Defining a collection
- Using tuples and dictionaries
- Developing stacks using lists
- Using the queue and deque packages

Chapter 14
Collecting All Sorts of Data

People collect all sorts of things. The CDs stacked near your entertainment center, the plates that are part of a series, baseball cards, and even the pens from every restaurant you've ever visited are all collections. The collections you encounter when you write applications are the same as the collections in the real world. A *collection* is simply a grouping of like items in one place and usually organized into some easily understood form.

REMEMBER

This chapter is about collections of various sorts. The central idea behind every collection is to create an environment in which the collection is properly managed and lets you easily locate precisely what you want at any given time. A set of bookshelves works great for storing books, DVDs, and other sorts of flat items. However, you probably put your pen collection in a holder or even a display case. The difference in storage locations doesn't change the fact that both house collections. The same is true with computer collections. Yes, differences exist between a stack and a `queue`, but the main idea is to provide the means to manage data properly and make it easy to access when needed.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 14; Collection All Sorts of Data.ipynb` file of the downloadable source. See the Introduction for details on how to find these source files.

Understanding Collections

In Chapter 13, you're introduced to sequences. *Collections* are simply another kind of sequence, albeit a more complex sequence than you find in either a string or list. No matter which sequence you use, they all support two functions: `index()` and `count()`. The `index()` function always returns the position of a specified item in the sequence. For example, you can return the position of a character in a string or the position of an object in a list. The `count()` function returns the number of times a specific item appears in the list. Again, the kind of specific item depends upon the kind of sequence.

You can use collections to create database-like structures using Python. Each collection type has a different purpose, and you use the various types in specific ways. The important idea to remember is that collections are simply another kind of sequence. As with every other kind of sequence, collections always support the `index()` and `count()` functions as part of their base functionality.

Python is designed to be extensible. However, it does rely on a base set of collections that you can use to create most application types. This chapter describes the most common collections (some implemented as a programming convention, like tuple and dictionary, some implemented using another class, such as stack, and some implemented directly using classes, such as `queue` and `deque`):

- **Tuple:** A tuple is a collection used to create complex list-like sequences. An advantage of tuples is that you can nest the content of a tuple. This feature lets you create structures that can hold employee records or x-y coordinate pairs.
- **Dictionary:** As with the real dictionaries, you create key/value pairs when using the dictionary collection (think of a word and its associated definition). A dictionary provides incredibly fast search times and makes ordering data significantly easier. Python offers a number of classes to implement the dictionary data storage collection. See `https://docs.python.org/3/library/collections.html` for a typical list of `collections` classes.
- **Stack:** Most programming languages support stacks directly. However, Python doesn't support the stack, although there's a work-around for that. A stack is a last in/first out (LIFO) sequence. Think of a pile of pancakes: You can add new pancakes to the top and also take them off of the top. A stack is an important collection that you can simulate in Python by using a list, which is precisely what this chapter does.
- `queue`**:** A `queue` is a first in/first out (FIFO) collection. You use it to track items that need to be processed in some way. Think of a `queue` as a line at the bank. You go into the line, wait your turn, and are eventually called to talk with a teller.

» deque **(pronounced *deck*):** A double-ended queue (deque) is a queue-like structure that lets you add or remove items from either end, but not from the middle. You can use a deque as a queue or a stack or any other kind of collection to which you're adding and from which you're removing items in an orderly manner (in contrast to lists, tuples, and dictionaries, which allow randomized access and management).

Working with Tuples

As previously mentioned, a tuple is a collection used to create complex lists, in which you can embed one tuple within another. This embedding lets you create hierarchies with tuples. A hierarchy could be something as simple as the directory listing of your hard drive or an organizational chart for your company. The idea is that you can create complex data structures by using a tuple.

REMEMBER

Tuples are immutable, which means you can't change them. You can create a new tuple with the same name and modify it in some way, but you can't modify an existing tuple. Lists are mutable, which means that you can change them. So, a tuple can seem at first to be at a disadvantage, but immutability has all sorts of advantages, such as being more secure as well as faster. In addition, immutable objects are easier to use with multiple processors.

The biggest difference between a tuple and a list is that a tuple is immutable; making tuples used more often for data access, rather than data operations. Immutability also makes a tuple faster to use, consume less memory, and have fewer method calls associated with it. The following steps demonstrate how you can interact with a tuple in Python.

1. **Open a new notebook.**

   You can also use the downloadable source file, `BPP4D3E; 14; Collection All Sorts of Data.ipynb`.

2. **Type the following code into the notebook — pressing Enter after each line:**

```
MyTuple = ("Red", "Blue", "Green")
MyTuple
```

   Python creates a tuple containing three strings.

3. **Click Run Cell.**

   You see the content of `MyTuple`, which is three strings, as shown in Figure 14-1. Notice that the entries use single quotes, even though you used double quotes to create the tuple. In addition, notice that a tuple uses parentheses rather than square brackets, as lists do.

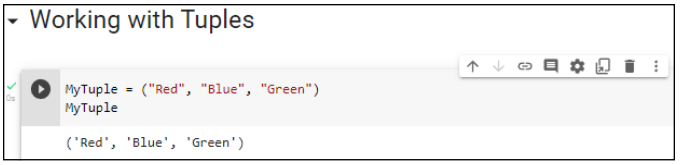


FIGURE 14-1: Tuples use parentheses, not square brackets.

4. **Type** print(dir(MyTuple)) **and click Run Cell.**

   Python presents a list of functions that you can use with tuples as shown here:

   ['__add__', '__class__', '__contains__',

   '__delattr__', '__dir__', '__doc__', '__eq__',

   '__format__', '__ge__', '__getattribute__',

   '__getitem__', '__getnewargs__', '__gt__',

   '__hash__', '__init__', '__init_subclass__',

   '__iter__', '__le__', '__len__', '__lt__', '__mul__',

   '__ne__', '__new__', '__reduce__', '__reduce_ex__',

   '__repr__', '__rmul__', '__setattr__', '__sizeof__',

   '__str__', '__subclasshook__', 'count', 'index']

   Notice that the list of functions appears significantly smaller than the list of functions provided with lists in Chapter 13. The `count()` and `index()` functions are present.

REMEMBER

However, appearances can be deceiving. For example, you can add new items by using the `__add__()` function. When working with Python objects, look at all the entries before you make a decision as to functionality.

5. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
MyTuple = MyTuple.__add__(("Purple",))
MyTuple
```

   This code adds a new tuple to `MyTuple` and places the result in a new copy of `MyTuple`. The old copy of `MyTuple` is destroyed after the call.

REMEMBER

The `__add__()` function accepts only tuples as input. This means that you must enclose the addition in parentheses. In addition, when creating a tuple with a single entry, you must add a comma after the entry, as shown in the example. This is an odd Python rule that you need to keep in mind or you'll see an error message similar to this one:

```
TypeError: can only concatenate tuple (not "str") to
tuple
```

6. **Click Run Cell.**

The addition to `MyTuple` appears at the end of the list. Notice that it appears at the same level as the other entries.

7. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
MyTuple = MyTuple.__add__(("Yellow",
                           ("Orange", "Black")))
MyTuple[4]
```

This step adds three entries: Yellow, Orange, and Black. However, Orange and Black are added as a tuple within the main tuple, which creates a hierarchy. These two entries are actually treated as a single entry within the main tuple.

TIP

You can replace the `__add__()` function with the concatenation operator. For example, if you wanted to add Magenta to the front of the tuple list, you could type `MyTuple = ("Magenta",) + MyTuple`.

8. **Click Run Cell.**

Python displays a single member of `MyTuple`, `'Yellow'`. Tuples use indexes to access individual members, just as lists do. You can also specify a range when needed. Anything you can do with a list index you can also do with a tuple index.

9. **Type** `MyTuple[5]` **and click Run Cell.**

You see a tuple that contains the tuple `('Orange', 'Black')`. Of course, you might not want to use both members in tuple form.

10. **Type** `type(MyTuple[5]) == tuple` **and click Run Cell.**

TIP

Tuples do contain hierarchies on a regular basis. You can detect when an index has returned another tuple, rather than a value, by testing for type. The output is `True` in this case.

11. **Type** MyTuple[5][0] **and click Run Cell.**

At this point, you see Orange as output. Figure 14-2 shows the results of the previous commands so that you can see the progression of index usage. The indexes always appear in order of their level in the hierarchy.

TIP

Using a combination of indexes and the `__add__()` function (or the concatenation operator, +), you can create flexible applications that rely on tuples. For example, you can remove an element from a tuple by making it equal to a range of values. If you wanted to remove the tuple containing Orange and Black, you type `MyTuple = MyTuple[0:5]`.

```
[1] MyTuple = ("Red", "Blue", "Green")
    MyTuple

    ('Red', 'Blue', 'Green')

[2] print(dir(MyTuple))

    ['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__e

[3] MyTuple = MyTuple.__add__(("Purple",))
    MyTuple

    ('Red', 'Blue', 'Green', 'Purple')

[4] MyTuple = MyTuple.__add__(("Yellow",
                               ("Orange", "Black")))
    MyTuple[4]

    'Yellow'

[5] MyTuple[5]

    ('Orange', 'Black')

[6] type(MyTuple[5]) == tuple

    True

    MyTuple[5][0]

    'Orange'
```

FIGURE 14-2: Use indexes to gain access to the individual tuple members.

Working with Dictionaries

A Python dictionary works just the same as its real-world counterpart — you create a key and value pair. It's just like the word and definition in a dictionary. As with lists, dictionaries are mutable, which means that you can change them as needed. The main reason to use a dictionary is to make information lookup faster. The key is normally short and is always unique so that the computer doesn't spend a lot of time looking for the information you need.

The following sections demonstrate how to create and use a dictionary in various ways. When you know how to work with dictionaries, you use that knowledge to make up for deficiencies in the Python language. Most languages include the concept of a switch statement, which is essentially a menu of choices from which one choice is selected. Python doesn't include this option, so you must normally rely on `if...elif` statements to perform the task, but you can also use dictionaries. (Such statements work, but they aren't as clear as they could be.)

Creating and using a dictionary

Creating and using a `dictionary` is much like working with a `list`, except that you must now define a key and value pair. Here are the special rules for creating a key:

- **The key must be unique.** When you enter a duplicate key, the information found in the second entry wins — the first entry is simply replaced with the second.
- **The key must be immutable.** This rule means that you can use strings, numbers, or tuples for the key. You can't, however, use a list for a key.

You have no restrictions on the values you provide. A value can be any Python object, so you can use a `dictionary` to access an employee record or other complex data. The following steps help you understand how to use dictionaries better.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
Colors = {"Sam": "Blue", "Amy": "Red",
          "Sarah": "Yellow"}
Colors
```

   Python creates a `dictionary` containing three entries with people's favorite colors. Notice how you create the key and value pair and that you use curly braces instead of square brackets or parentheses. The key comes first, followed by a colon and then the value. Each entry is separated by a comma.

2. **Click Run Cell.**

   You see the key and value pairs, as shown in Figure 14-3. The downside is that creating the `dictionary` takes longer than using something like a `list` because the computer needs to work with additional data.

FIGURE 14-3: A dictionary places entries in sorted order.

3. **Type** Colors["Sarah"] **and click Run Cell.**

   You see the color associated with `Sarah`, `'Yellow'`. Using a string as a key, rather than using a numeric index, makes the code easier to read and makes it self-documenting to an extent. By making your code more readable, dictionaries save you considerable time in the long run (which is why they're so popular). However, the access speed of a `dictionary` comes at the cost of additional creation time and a higher use of resources, so you have trade-offs to consider.

4. **Type** Colors.keys() **and click Run Cell.**

   The `dictionary` presents a list of the keys it contains: `dict_keys(['Sam', 'Amy', 'Sarah'])`. You can use these keys to automate access to the dictionary.

5. **Type the following code into a new cell in the notebook — pressing Enter after each line, and then click Run Cell.**

```
for Item in Colors.keys():
   print("{0} likes the color {1}."
      .format(Item, Colors[Item]))
```

REMEMBER

The example code outputs a listing of each of the usernames and the user's favorite color:

```
Sam likes the color Blue.
Amy likes the color Red.
Sarah likes the color Yellow.
```

Using dictionaries can make creating useful output a lot easier. The use of a meaningful key means that the key can easily be part of the output.

6. **Type the following code into a new cell in the notebook — pressing Enter after each line, and then click Run Cell.**

```
Colors["Sarah"] = "Purple"
Colors.update({"Harry": "Orange"})
```

```
for name, color in Colors.items():
    print("{0} likes the color {1}."
          .format(name, color))
```

The dictionary content is updated so that Sarah now likes Purple instead of Yellow. The code also adds new entry to the dictionary.

TIP

The loop used in this case also differs from the one used in Step 5. This version obtains each of the items one at a time and places the key in name and the value in color. The output will always work the same from the items() method. You need two variables, one for the key and another for the value, presented in the order shown. The reason to consider this second form is that it might be easier to read in some cases. There doesn't seem to be much of a speed difference between the two versions. Here is the updated output:

```
Sam likes the color Blue.
Amy likes the color Red.
Sarah likes the color Purple.
Harry likes the color Orange.
```

Notice that Harry is added to the dictionary. In addition, Sarah's entry is changed to the color Purple.

7. **Type the following code into a new cell in the notebook — pressing Enter after each line, and then click Run Cell.**

```
del Colors["Sam"]
for name, color in Colors.items():
    print("{0} likes the color {1}."
          .format(name, color))
```

Python removes Sam's entry from the dictionary. Here is the new output from the example:

```
Amy likes the color Red.
Sarah likes the color Purple.
Harry likes the color Orange.
```

8. **Type** len(Colors) **and click Run Cell.**

The output value of 3 verifies that the dictionary contains only three entries now, rather than four.

9. **Type the following code into a new cell in the notebook — pressing Enter after each line, and then click Run Cell.**

```
Colors.clear()
len(Colors)
```

Python reports that `Colors` has `0` entries, so the dictionary is now empty. Figure 14-4 shows the results of the previous commands so that you can see the progression of `dictionary` usage.

```
[9]  Colors["Sarah"]

     'Yellow'

[10] Colors.keys()

     dict_keys(['Sam', 'Amy', 'Sarah'])

[11] for Item in Colors.keys():
        print("{0} likes the color {1}."
           .format(Item, Colors[Item]))

     Sam likes the color Blue.
     Amy likes the color Red.
     Sarah likes the color Yellow.

[12] Colors["Sarah"] = "Purple"
     Colors.update({"Harry": "Orange"})
     for name, color in Colors.items():
         print("{0} likes the color {1}."
               .format(name, color))

     Sam likes the color Blue.
     Amy likes the color Red.
     Sarah likes the color Purple.
     Harry likes the color Orange.

[13] del Colors["Sam"]
     for name, color in Colors.items():
         print("{0} likes the color {1}."
               .format(name, color))

     Amy likes the color Red.
     Sarah likes the color Purple.
     Harry likes the color Orange.

[14] len(Colors)

     3

     Colors.clear()
     len(Colors)

     0
```

FIGURE 14-4: Dictionaries work much like lists do, but with a little extra functionality added.

Working with nested dictionaries

Unlike the dictionary on your screen (or possibly your shelf), a Python dictionary entry can contain another dictionary. It's like watching one of those Dr. Strange movies with the whole multiverse scenarios in action (see `https://www.imdb.com/title/tt9419884/` for details). Nesting dictionaries gives you all sorts of new Python powers that will amaze friends, family, and colleagues alike! Creating a nested dictionary is like creating a regular dictionary in that the keys still need to be unique, but the values are now other dictionaries. Here's how to create a nested dictionary:

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
MyCompany = {"Owners": "Me",
             "Managers": {151: "Marta",
                          152: "Jerry"},
             "Programmers": {251: "Amy",
                             252: "Albert"},
             "Grunts": {351: "Gus",
                        352: "Sidney"}
            }
MyCompany
```

Python creates a nested, two-level `dictionary` containing four entries at the first level: Owners, Managers, Programmers, and Grunts. The second level contains an employee number, followed by a name.

2. **Click Run Cell.**

You see the key and value pairs, as shown in Figure 14-5.

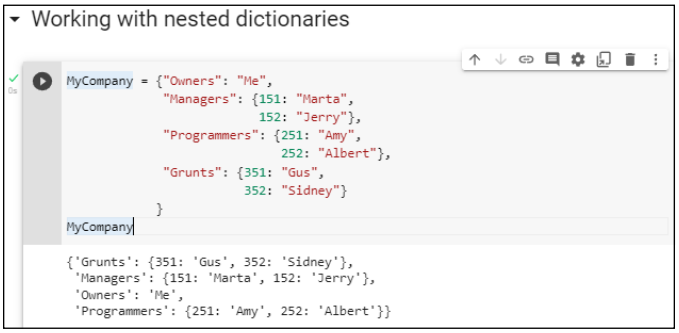


FIGURE 14-5: Nested dictionaries follow the same rules as regular dictionaries, just a little more complex.

REMEMBER

Nested dictionaries require a little extra processing if you want to fully display their content. This means using recursion or some other method to drill down into the dictionary. The next step demonstrates one technique for performing this task.

3. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
def DisplayDict(Dict):
    for key, value in Dict.items():
        if type(value) == dict:
            print(f"{key} department includes:")
            DisplayDict(value)
```

```
        else:
            print(f"{value} is {key}")

DisplayDict(MyCompany)
```

The code begins by calling `DisplayDict()` with `MyCompany`. This example uses a recursive technique in which the `value` extracted from `Dict` is first checked to determine whether it's another dictionary (type `dict`). If so, the `key` is actually the name of a department and is displayed as such. (Notice that this example uses an f-string, which is explained in the "Simplifying things using f-string" section of Chapter 12.) The code then recursively calls `DisplayDict()` with `value`.

After the code determines that `value` doesn't contain a dictionary, it proceeds to display the `value` with the `key`, the way it's done in the previous section of the chapter, except this version uses an f-string in place of the `.format()` function approach. Compare the two versions and you'll notice that the f-string approach definitely saves typing and is easier to read. Figure 14-6 shows the outputs of working with nested dictionaries.

FIGURE 14-6: Recursion makes working with nested dictionaries easier.

Replacing the switch statement with a dictionary

Most programming languages provide some sort of switch statement. A switch statement provides for elegant menu type selections. The user has a number of options but is allowed to choose only one of them. The program takes some course of action based on the user selection. Here is some representative code (it won't execute) of a `switch` statement you might find in another language:

```
switch(n)
{
```

```
    case 0:
       print("You selected blue.");
       break;
    case 1:
       print("You selected yellow.");
       break;
    case 2:
       print("You selected green.");
       break;
 }
```

The application normally presents a menu-type interface, obtains the number of the selection from the user, and then chooses the correct course of action from the `switch` statement. It's straightforward and much neater than using a series of `if` statements to accomplish the same task.

Unfortunately, Python doesn't come with a switch statement. The best you can hope to do is use an `if...elif` statement for the task. However, by using a `dictionary`, you can simulate the use of a switch statement. The following steps help you create an example that will demonstrate the required technique.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
def PrintBlue():
   print("You chose blue!\r\n")
def PrintRed():
   print("You chose red!\r\n")
def PrintOrange():
   print("You chose orange!\r\n")
def PrintYellow():
   print("You chose yellow!\r\n")
```

   Before the code can do anything for you, you must define the tasks. Each of these functions defines a task associated with selecting a color option onscreen. Only one of them gets called at any given time.

2. **Type the following code into the same cell in the notebook — pressing Enter after each line:**

```
ColorSelect = {
   0: PrintBlue,
   1: PrintRed,
   2: PrintOrange,
   3: PrintYellow
}
```

This code is the `dictionary`. Each key is like the case part of the switch statement. The values specify what to do. In other words, this is the switch structure. The functions that you created earlier are the action part of the switch — the part that goes between the case statement and the break clause.

3. **Type the following code into the same cell in the notebook — pressing Enter after each line:**

```
Selection = 0
while (Selection != 4):
   print("0. Blue")
   print("1. Red")
   print("2. Orange")
   print("3. Yellow")
   print("4. Quit")
   Selection = int(input("Select a color option: "))
   if (Selection >= 0) and (Selection < 4):
      ColorSelect[Selection]()
```

Finally, you see the user interface part of the example. The code begins by creating an input variable, `Selection`. It then goes into a loop until the user enters a value of `4`.

During each loop, the application displays a list of options and then waits for user input. When the user does provide input, the application performs a range check on it. Any value between `0` and `3` selects one of the functions defined earlier using the `dictionary` as the switching mechanism.

4. **Click Run Cell.**

Python displays a menu.

5. **Type** `0` **and press Enter.**

The application tells you that you selected blue and then displays the menu again.

6. **Type** `4` **and press Enter.**

The application ends. Figure 14-7 shows the results of using this technique.

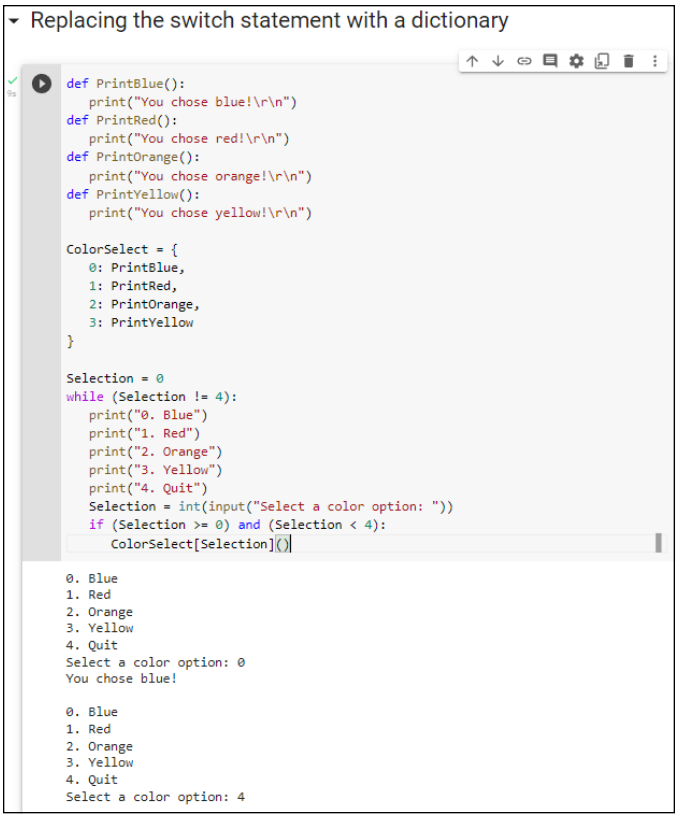


FIGURE 14-7: After displaying your selection, the application displays the menu again.

Creating Stacks Using Lists

A stack is a handy programming structure because you can use it to save an application execution environment (the state of variables and other attributes of the application environment at any given time) or as a means of determining an order of execution. Unfortunately, Python doesn't provide a stack as a collection. However, it does provide lists, and you can use a `list` as a perfectly acceptable stack. The following steps help you create an example of using a `list` as a stack.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
MyStack = []
StackSize = 3
def DisplayStack():
   print("Stack currently contains:")
   for Item in MyStack:
      print(Item)
```

```
def Push(Value):
   if len(MyStack) < StackSize:
      MyStack.append(Value)
      print(f"{Value} added")
   else:
      print("Stack is full!")
def Pop():
   if len(MyStack) > 0:
      Removed = MyStack.pop()
      print(f"{Removed} removed from stack")
   else:
      print("Stack is empty.")
Push(1)
Push(2)
Push(3)
DisplayStack()
input("Press any key when ready...")
Push(4)
DisplayStack()
input("Press any key when ready...")
Pop()
DisplayStack()
input("Press any key when ready...")
Pop()
Pop()
Pop()
DisplayStack()
```

In this example, the application creates a `list` and a variable to determine the maximum stack size. Stacks normally have a specific size range. This is admittedly a really small stack, but it serves well for the example's needs.

REMEMBER

Stacks work by pushing a value onto the top of the stack and popping values back off the top of the stack. The `Push()` and `Pop()` functions perform these two tasks. The code adds `DisplayStack()` to make it easier to see the stack content as needed.

The remaining code *exercises the stack* (demonstrates its functionality) by pushing values onto it and then removing them. Four main exercise sections test stack functionality.

2. **Click Run Cell.**

Python fills the stack with information and then displays it onscreen like this:

```
1 added
2 added
```

```
3 added
Stack currently contains:
1
2
3
```

In this case, 3 is at the top of the stack because it's the last value added. The `input()` function provides a good way to momentarily stop application execution so that you can see the current stack state. The `input()` function argument, `"Press any key when ready..."`, tells `input()` what to provide as a prompt.

3. **Press Enter.**

   The application attempts to push another value onto the stack. However, the stack is full, so the task fails with a `Stack is full!` message followed by the content of the stack, which is the same as before.

4. **Press Enter.**

   The application pops a value from the top of the stack. Remember that 3 is the top of the stack, so that's the value that is missing in the output:

```
3 removed from stack
Stack currently contains:
1
2
```

5. **Press Enter.**

   The application tries to pop more values from the stack than it contains, resulting in an error and then a display of the now empty stack:

```
2 removed from stack
1 removed from stack
Stack is empty.
Stack currently contains:
```

Any stack implementation that you create must be able to detect both overflows (too many entries) and underflows (too few entries).

Working with queues

A `queue` works differently from a stack. Think of any line you've ever stood in: You go to the back of the line, and when you reach the front of the line you get to do whatever you stood in the line to do. A `queue` is often used for task scheduling and

to maintain program flow — just as it is in the real world. The following steps help you create a queue-based application.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
import queue
MyQueue = queue.Queue(3)
print(MyQueue.empty())
input("Press any key when ready...")
MyQueue.put(1)
MyQueue.put(2)
print(MyQueue.empty())
print(MyQueue.full())
input("Press any key when ready...")
MyQueue.put(3)
print(MyQueue.full())
input("Press any key when ready...")
print(MyQueue.get())
print(MyQueue.empty())
print(MyQueue.full())
input("Press any key when ready...")
print(MyQueue.get())
print(MyQueue.get())
```

To create a queue, you must import the queue package. This package actually contains a number of queue types, but this example uses only the standard FIFO queue.

REMEMBER

When a queue is empty, the `empty()` function returns `True`. Likewise, when a queue is full, the `full()` function returns `True`. You can't iterate through a queue using a `for` loop as you have done with other collection types, so you must monitor `empty()` and `full()` instead.

The two functions used to work with data in a queue are `put()`, which adds new data, and `get()`, which removes data. A problem with queues is that if you try to put more items into the queue than it can hold, it simply waits until space is available to hold it.

2. **Click Run Cell.**

Python tests the state of the queue. In this case, you see an output of `True`, which means that the queue is empty.

3. **Press Enter.**

   The application adds two new values to the queue. In doing so, the queue is no longer empty, as shown by the call to `empty()`, but a test of `full()` shows that it's not full either.

4. **Press Enter.**

   The application adds another entry to the queue, which means that the queue is now full because it was set to a size of 3. This means that `full()` will return `True` because the queue is now full.

5. **Press Enter.**

   To free space in the queue, the application gets one of the entries. Whenever an application gets an entry, the `get()` function returns that entry. Given that `1` was the first value added to the queue, the `print()` function will display a value of `1`. In addition, both `empty()` and `full()` should now return `False`.

6. **Press Enter.**

   The application gets the remaining two entries. You see 2 and 3 (in turn) as output.

Working with deques

A `deque` is simply a `queue` where you can remove and add items from either end. In many languages, a `queue` or stack starts out as a `deque`. Specialized code serves to limit `deque` functionality to what is needed to perform a particular task.

When working with a `deque`, you need to think of the `deque` as a sort of horizontal line. Certain individual functions work with the left and right ends of the `deque` so that you can add and remove items from either side, but nothing changes the middle. The following steps help you create an example that demonstrates `deque` usage.

1. **Type the following code into a new cell in the Notebook — pressing Enter after each line.**

```
import collections
MyDeque = collections.deque("abcdef", 10)
print("Starting state:")
for Item in MyDeque:
   print(Item, end=" ")
print("\r\n\r\nAppending and extending right")
MyDeque.append("h")
```

```
MyDeque.extend("ij")
for Item in MyDeque:
   print(Item, end=" ")
print("\r\nMyDeque contains {0} items."
      .format(len(MyDeque)))
print("\r\nPopping right")
print("Popping {0}".format(MyDeque.pop()))
for Item in MyDeque:
   print(Item, end=" ")
print("\r\n\r\nAppending and extending left")
MyDeque.appendleft("a")
MyDeque.extendleft("bc")
for Item in MyDeque:
   print(Item, end=" ")
print("\r\nMyDeque contains {0} items."
      .format(len(MyDeque)))
print("\r\nPopping left")
print("Popping {0}".format(MyDeque.popleft()))
for Item in MyDeque:
   print(Item, end=" ")
print("\r\n\r\nRemoving")
MyDeque.remove("a")
for Item in MyDeque:
   print(Item, end=" ")
```

The implementation of `deque` is found in the `collections` package, so you need to import it into your code. When you create a `deque`, you can optionally specify a starting list of *iterable items* (items that can be accessed and processed as part of a loop structure) and a maximum size, as shown.

REMEMBER

A `deque` differentiates between adding one item and adding a group of items. You use `append()` or `appendleft()` when adding a single item. The `extend()` and `extendleft()` functions let you add multiple items. You use the `pop()` or `popleft()` functions to remove one item at a time. The act of popping values returns the value popped, so the example prints the value onscreen. The `remove()` function is unique in that it always works from the left side and always removes the first instance of the requested data.

Unlike some other collections, a `deque` is fully iterable. This means that you can obtain a list of items by using a `for` loop whenever necessary.

2. **Click Run Cell.**

Python outputs the following information:

```
Starting state:
a b c d e f

Appending and extending right
a b c d e f h i j
MyDeque contains 9 items.

Popping right
Popping j
a b c d e f h i

Appending and extending left
c b a a b c d e f h
MyDeque contains 10 items.

Popping left
Popping c
b a a b c d e f h

Removing
b a b c d e f h
```

WARNING

Following the output listing closely is important. Notice how the size of the `deque` changes over time. After the application pops the `j`, the `deque` still contains eight items. When the application appends and extends from the left, it adds three more items. However, the resulting `deque` contains only ten items. When you exceed the maximum size of a `deque`, the extra data simply falls off the other end.

IN THIS CHAPTER

- Defining the characteristics of a class
- Specifying the class components
- Creating and using your own class
- Working with subclasses

Chapter 15
Creating and Using Classes

Classes make working with Python code more convenient by helping to make your applications easy to read, understand, and use. You use classes to create containers for your code and data, so they stay together in one piece. Outsiders see your class as a black box — data goes in and results come out.

REMEMBER

At some point, you need to start constructing classes of your own if you want to avoid the dangers of the spaghetti code that is found in older applications. *Spaghetti code* is much as the name implies — various lines of procedures are interwoven and spread out in such a way that it's hard to figure out where one piece of spaghetti begins and another ends. Trying to maintain spaghetti code is nearly impossible, and some organizations have thrown out applications because no one could figure them out.

Besides helping you understand classes as a packaging method that avoids spaghetti code, this chapter helps you create and use your own classes for the first time. You gain insights into how Python classes work toward making your applications convenient to work with. This is an introductory sort of chapter, though, and you won't become so involved in classes that your head begins to spin around on its own. This chapter is about making class development simple and manageable.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 15; Creating and Using Classes.ipynb` and `BPP4D3E; 15; MyClass.ipynb` files of the downloadable source. See the Introduction for details on how to find these source files.

Considering the Parts of a Class

A *class* (the blueprint used to create an object) has a specific construction. Each part of a class performs a particular task that gives the class useful characteristics. Of course, the class begins with a container that is used to hold the entire class together, so that's the part that the first section that follows discusses. The remaining sections describe the other parts of a class and help you understand how they contribute to the class as a whole.

Creating the class definition

A class need not be particularly complex. In fact, you can create just the container and one class element and call it a class. Of course, the resulting class won't do much, but you can *instantiate it* (tell Python to build an object by using your class as a blueprint) and work with it as you would any other class. The following steps help you understand the basics behind a class by creating the simplest class possible.

1. **Open a new notebook.**

   You can also use the downloadable source file, `BPP4D3E; 15; Creating and Using Classes.ipynb`.

2. **Type the following code into the notebook — pressing Enter after each line:**

```
class MyClass:
   MyVar = 0

MyInstance = MyClass()

MyInstance.MyVar
```

   The first line defines the class container, which consists of the keyword `class` and the class name, which is `MyClass`. Every class you create must begin precisely this way. You must always include `class` followed by the class name.

The second line is the class suite. All the elements that comprise the class are called the *class suite.* In this case, you see a class variable named `MyVar`, which is set to a value of `0`. Every instance of the class will have the same variable and start at the same value.

The fourth line creates an instance of `MyClass` named `MyInstance`. Of course, you'll want to verify that you really have created such an instance.

The sixth line outputs the value of `MyVar` found in the instance of `MyInstance`.

3. **Click Run Cell.**

   The output of `0`, demonstrates that `MyInstance` does indeed have a class variable named `MyVar`.

4. **In a new cell, type `MyInstance.__class__` and click Run Cell.**

   Python displays the class used to create this instance, as shown in Figure 15-1. The output tells you that this class is part of the `__main__` package, which means that you typed it directly into the application code and not as part of another package.

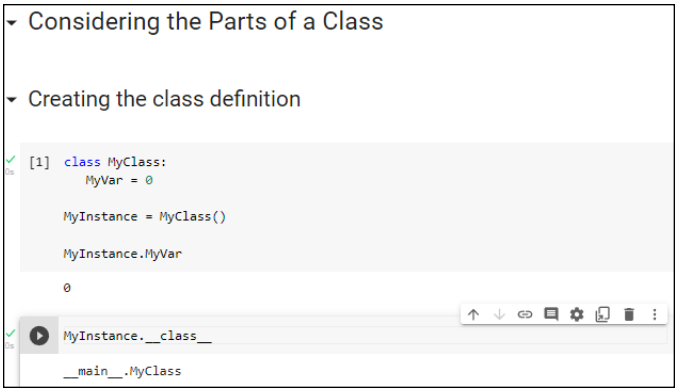


FIGURE 15-1: The class name is also correct, so you know that this instance is created by using `MyClass`.

Considering the built-in class attributes

When you create a class, you can easily think that all you get is the class. However, Python adds built-in functionality to your class. For example, in the preceding section, you type `__class__` and press Enter. The `__class__` attribute is built in; you didn't create it. It helps to know that Python provides this functionality so that you don't have to add it. The functionality is needed often enough that every

class should have it, so Python supplies it. The following steps help you work with the built-in class attributes. They assume that you followed the steps in the preceding section, "Creating the class definition."

1. **In a new cell, type** print(dir(MyInstance)) **and click Run Cell.**

   A list of attributes appears below. These attributes provide specific functionality for your class. They're also common to every other class you create, so you can count on always having this functionality in the classes you create.

'MyVar'	'__class__'	'__delattr__'
'__dict__'	'__dir__'	'__doc__'
'__eq__'	'__format__'	'__ge__'
'__getattribute__'	'__gt__'	'__hash__'
'__init__'	'__init_subclass__'	'__le__'
'__lt__'	'__module__'	'__ne__'
'__new__'	'__reduce__'	'__reduce_ex__'
'__repr__'	'__setattr__'	'__sizeof__'
'__str__'	'__subclasshook__'	'__weakref__'

   In a new cell, type help('__class__') **and press Enter.**

   Python displays information on the `__class__` attribute, as partially shown in Figure 15-2. You can use the same technique for learning more about any attribute that Python adds to your class.

Working with methods

Methods are simply another kind of function that resides in classes. You create and work with methods in precisely the same way that you do functions, except that methods are always associated with a class (you don't see freestanding methods as you do functions). You can create two kinds of methods: those associated with the class itself and those associated with an instance of a class. It's important to differentiate between the two. The following sections provide the details needed to work with both.

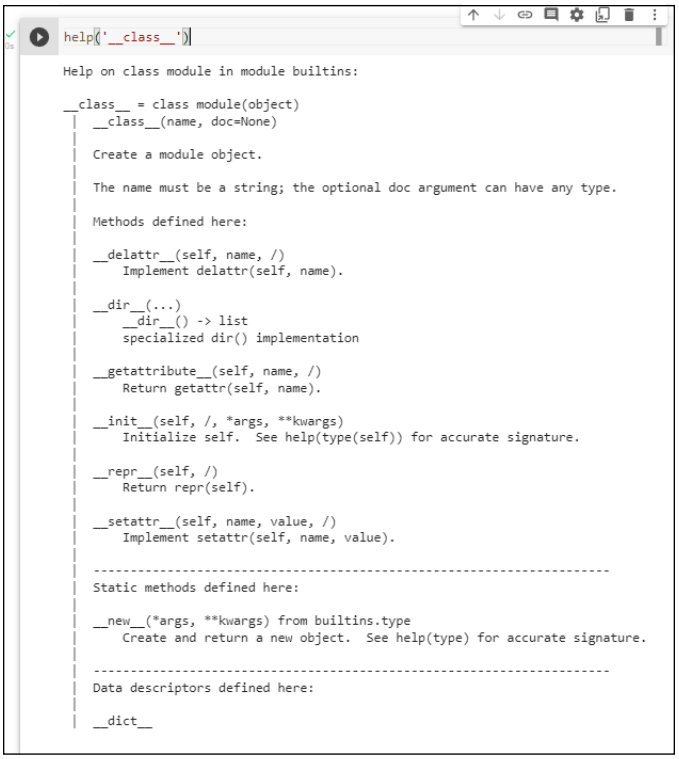


FIGURE 15-2: Python provides help for each of the attributes it adds to your class.

Creating class methods

A *class method* is one that you execute directly from the class without creating an instance of the class. Sometimes you need to create methods that execute from the class, such as the functions you used with the `str` class to modify strings. As an example, the multiple exception example in the "Nested exception handling" section of Chapter 10 uses the `str.upper()` function. The following steps demonstrate how to create and use a class method.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
class MyClass:
   def SayHello():
      print("Hello there!")

MyClass.SayHello()
```

The example class contains a single defined attribute, `SayHello()`. This method doesn't accept any arguments and doesn't return any values. It simply prints a message as output. However, the method works just fine for demonstration purposes.

2. **Click Run Cell.**

   The example outputs the expected string, as shown in Figure 15-3. Notice that you didn't need to create an instance of the class — the method is available immediately for use.

FIGURE 15-3: The class method outputs a simple message.

REMEMBER

A class method can work only with *class data* (the data associated with the class itself, rather than a particular instance of that class). You can pass the method data as an argument, and it can return information as needed, but it can't access the instance data. As a consequence, you need to exercise care when creating class methods to ensure that they're essentially self-contained.

Creating instance methods

An *instance method* is one that is part of the individual instances. You use instance methods to manipulate the data that the class manages. As a consequence, you can't use instance methods until you instantiate an object from the class.

REMEMBER

All instance methods accept a single argument as a minimum, `self`. The `self` argument points at the particular instance that the application is using to manipulate data. Without the `self` argument, the method wouldn't know which instance data to use. However, `self` is supplied by Python, and you can't change it as part of calling the method. The following steps demonstrate how to create and use instance methods in Python.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
class MyClass:
   def SayHello(self):
      print("Hello there!")

MyInstance = MyClass()
MyInstance.SayHello()
```

The example class contains a single defined attribute, `SayHello()`. This method doesn't accept any special arguments and doesn't return any values. It simply prints a message as output. Notice that this instance version of `SayHello()` does include the `self` argument, which is what differentiates it from a class method.

2. **Click Run Cell.**

   You see the same message shown in Figure 15-3.

Working with constructors

A *constructor* is a special kind of method that Python calls when it instantiates an object by using the definitions found in your class. Python relies on the constructor to perform tasks such as *initializing* (assigning values to) any instance variables that the object will need when it starts. Constructors can also verify that there are enough resources for the object and perform any other start-up task you can think of.

REMEMBER

The name of a constructor is always the same, `__init__()`. The constructor can accept arguments when necessary to create the object. When you create a class without a constructor, Python automatically creates a default constructor for you that doesn't do anything. Every class must have a constructor, even if it simply relies on the default constructor. The following steps demonstrate how to create a constructor:

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
class MyClass:
   Greeting = ""
   def __init__(self, Name="there"):
      self.Greeting = Name + "!"
   def SayHello(self):
      print("Hello {0}".format(self.Greeting))

MyInstance = MyClass()
MyInstance.SayHello()
```

   This example provides your first example of Python-style method overloading. Even though there is only one `__init__()` method declaration, there are actually two versions of `__init__()`. The first doesn't require any special input because it uses the default value for the `Name` of `"there"`. The second requires a name as an input. It sets `Greeting` to the value of this name, plus an exclamation mark. The `SayHello()` method is essentially the same as previous examples in this chapter.

Python doesn't support true function overloading. Many strict adherents to strict Object-Oriented Programming (OOP) principles consider default values to be something different from function overloading. However, the use of default values obtains the same result.

2. **Click Run Cell.**

   You see the message shown in Figure 15-3. Notice that this message provides the default, generic greeting.

3. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
MyInstance2 = MyClass("Amy")
MyInstance2.SayHello()
MyInstance.Greeting = "Harry!"
MyInstance.SayHello()
```

   In the first line, Python creates an instance of `MyClass` named `MyInstance2`. The instance `MyInstance` is completely different from the instance `MyInstance2`. In the second line, Python displays the message for `MyInstance2`, not the message for `MyInstance`.

   The third line changes the greeting for `MyInstance` without changing the greeting for `MyInstance2`. The fourth line displays the new `MyInstance` message.

4. **Click Run Cell.**

   You see the messages shown in Figure 15-4. Notice that this code provides a specific greeting for each of the instances. In addition, each instance is separate, and you were able to change the message for the first instance without affecting the second instance.

FIGURE 15-4: Supplying the constructor with a name provides a customized output.

Working with variables

As mentioned earlier in the book, variables are storage containers that hold data. When working with classes, you need to consider how the data is stored and managed. A class can include both class variables and instance variables. The class variables are defined as part of the class itself, while instance variables are defined as part of methods. The following sections show how to use both variable types.

Creating class variables

Class variables provide global access to data that your class manipulates in some way. The following steps demonstrate how class variables work.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
class MyClass:
   Greeting = ""
   def SayHello(self):
      print("Hello {0}".format(self.Greeting))

MyClass.Greeting = "Zelda"
MyClass.Greeting
```

   This is a version of the code found in the "Working with constructors" section, earlier in this chapter, but this version doesn't include the constructor. Normally you do include a constructor to ensure that the class variable is initialized properly. However, this series of steps shows how class variables can go wrong.

   The `MyClass.Greeting = "Zelda"` statement sets the value of `Greeting` to something other than the value that you used when you created the class. Of course, anyone could make this change. The big question is whether the change will take.

2. **Click Run Cell.**

   You see that the value of `Greeting` has changed, as shown in Figure 15-5.

3. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
MyInstance = MyClass()
MyInstance.SayHello()
```

   Python creates an instance of `MyClass` named `MyInstance`.

FIGURE 15-5: You can change the value of `Greeting`.

4. **Click Run Cell.**

   You see the message shown in Figure 15-6. The change that you made to `Greeting` has carried over to the instance of the class. It's true that the use of a class variable hasn't really caused a problem in this example, but you can imagine what would happen in a real application if someone wanted to cause problems.

FIGURE 15-6: The change to `Greeting` carries over to the instance of the class.

REMEMBER

This is just a simple example of how class variables can go wrong. The two concepts you should take away from this example are as follows:

- Avoid class variables when you can because they're inherently unsafe.
- Always initialize class variables to a known good value in the constructor code.

Creating instance variables

Instance variables are always defined as part of a method. The input arguments to a method are considered instance variables because they exist only when the method exists. Using instance variables is usually safer than using class variables because it's easier to maintain control over them and to ensure that the caller is providing the correct input. The following steps show an example of using instance variables.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
class MyClass:
   def DoAdd(self, Value1=0, Value2=0):
```

```
        Sum = Value1 + Value2
        print("The sum of {0} plus {1} is {2}."
              .format(Value1, Value2, Sum))

MyInstance = MyClass()
MyInstance.DoAdd(1, 4)
```

In this case, you have three instance variables. The input arguments, `Value1` and `Value2`, have default values of `0`, so `DoAdd()` can't fail simply because the user forgot to provide values. Of course, the user could always supply something other than numbers, so you should provide the appropriate checks as part of your code. The third instance variable is `Sum`, which is equal to `Value1 + Value2`. The code simply adds the two numbers together and displays the result.

After defining the class, the code creates an instance of `MyClass` named `MyInstance`. The last line executes `DoAdd()` with input values of `1` and `4`.

2. **Click Run Cell.**

   You see the message shown in Figure 15-7. In this case, you see the sum of adding 1 and 4.

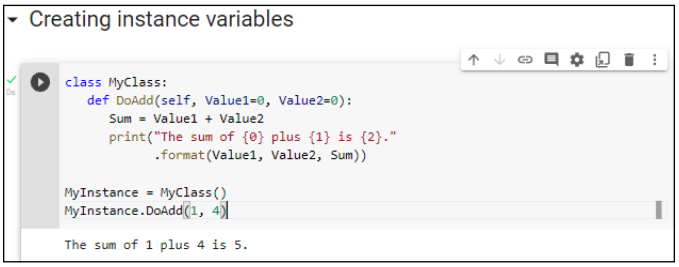


FIGURE 15-7: The output is simply the sum of two numbers.

Using methods with variable argument lists

Sometimes you create methods that can take a variable number of arguments. Handling this sort of situation is something Python does well. Here are the two kinds of variable arguments that you can create:

- `*args`: Provides a list of unnamed arguments.
- `**kwargs`: Provides a list of named arguments.

REMEMBER

The actual names of the arguments don't matter, but Python developers use `*args` and `**kwargs` as a convention so that other Python developers know that they're a variable list of arguments. Notice that the first variable argument has just one asterisk (*) associated with it, which means the arguments are unnamed. The second variable has two asterisks (**), which means that the arguments are named. The following steps demonstrate how to use both approaches to writing an application.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
class MyClass:
   def PrintList1(*args):
      for Count, Item in enumerate(args):
         print("{0}. {1}".format(Count, Item))
   def PrintList2(**kwargs):
      for Name, Value in kwargs.items():
         print("{0} likes {1}".format(Name, Value))
MyClass.PrintList1("Red", "Blue", "Green")
MyClass.PrintList2(George="Red", Sue="Blue",
                   Zarah="Green")
```

For the purposes of this example, you're seeing the arguments implemented as part of a class method. However, you can use them just as easily with an instance method.

TIP

Look carefully at `PrintList1()` and you see a new method of using a `for` loop to iterate through a list. In this case, the `enumerate()` function outputs both a count (the loop count) and the string that was passed to the function.

The `PrintList2()` function accepts a dictionary input. Just as with `PrintList1()`, this list can be any length. However, you must process the `items()` found in the dictionary to obtain the individual values.

2. **Click Run Cell.**

You see the output shown in Figure 15-8. The individual lists can be of any length. In fact, in this situation, playing with the code to see what you can do with it is a good idea. For example, try mixing numbers and strings with the first list to see what happens. Try adding Boolean values as well. The point is that using this technique makes your methods incredibly flexible if all you want is a list of values as input.

FIGURE 15-8: The code can process any number of entries in the list.

Overloading operators

In some situations, you want to be able to do something special as the result of using a standard operator such as add (+). In fact, sometimes Python doesn't provide a default behavior for operators because it has no default to implement. No matter what the reason might be, overloading operators makes it possible to assign new functionality to existing operators so that they do what you want, rather than what Python intended. The following steps demonstrate how to overload an operator and use it as part of an application.

1. **Type the following code into a new cell in the Notebook — pressing Enter after each line:**

```
class MyClass:
   def __init__(self, *args):
      self.Input = args
   def __add__(self, Other):
      Output = MyClass()
      Output.Input = self.Input + Other.Input
      return Output
   def __str__(self):
      Output = ""
      for Item in self.Input:
         Output += Item
         Output += " "
      return Output

Value1 = MyClass("Red", "Green", "Blue")
Value2 = MyClass("Yellow", "Purple", "Cyan")
Value3 = Value1 + Value2
print("{0} + {1} = {2}"
      .format(Value1, Value2, Value3))
```

The example demonstrates a few different techniques. The constructor, `__init__()`, demonstrates a method for creating an instance variable attached to the `self` object. You can use this approach to create as many variables as needed to support the instance.

REMEMBER

When you create your own classes, no + operator is defined until you define one, in most cases. The only exception is when you inherit from an existing class that already has the + operator defined (see the "Extending Classes to Make New Classes" section, later in this chapter, for details). To add two `MyClass` entries together, you must define the `__add__()` method, which equates to the + operator.

The code used for the `__add__()` method may look a little odd, too, but you need to think about it one line at a time. The code begins by creating a new object, `Output`, from `MyClass`. Nothing is added to `Output` at this point — it's a blank object. The two objects that you want to add, `self.Input` and `Other.Input`, are actually tuples. (See "Working with Tuples," in Chapter 14, for more details about tuples.) The code places the sum of these two objects into `Output.Input`. The `__add__()` method then returns the new combined object to the caller.

Of course, you may want to know why you can't simply add the two inputs together as you would a number. The answer is that you'd end up with a tuple as an output, rather than a `MyClass` as an output. The type of the output would be changed, and that would also change any use of the resulting object.

To print `MyClass` properly, you also need to define a `__str__()` method. This method converts a `MyClass` object into a string. In this case, the output is a *space-delimited string* (in which each of the items in the string is separated from the other items by a space) containing each of the values found in `self.Input`. Of course, the class that you create can output any string that fully represents the object.

The main procedure creates two test objects, `Value1` and `Value2`. It adds them together and places the result in `Value3`. The result is printed onscreen.

2. **Click Run Cell.**

Figure 15-9 shows the result of adding the two objects together, converting them to strings, and then printing the result. It's a lot of code for such a simple output statement, but the result definitely demonstrates that you can create classes that are self-contained and fully functional.

FIGURE 15-9: The result of adding two `MyClass` objects is a third object of the same type.

Creating and Using an External Class

Sometimes you want to reuse the classes you create, which means making them external to your main application. What you end up with is your own special library that is similar to the other Python libraries found in this book. In this case, you create a class that you place into an external package and eventually access within an application.

WARNING

Because of limitations in how Colab works, the example in the following sections will work well only with Jupyter Notebook because it's hard to export `.py` files correctly in Colab. These sections focus on Jupyter Notebook use for this reason.

Developing the external class

Before you can do anything else, you need to create the external class, which may eventually turn into a custom library filled with multiple classes. The following sections describe how to create and save this class.

Defining the MyClass class

Listing 15-1 shows the code that you need to create the class. You can also find this code in the `BPP4D3E; 15; MyClass.ipynb` file found in the downloadable source, as described in the Introduction.

LISTING 15-1: Creating an External Class

```
class MyClass:
   def __init__(self, Name="Sam", Age=32):
      self.Name = Name
      self.Age = Age
   def GetName(self):
      return self.Name
   def SetName(self, Name):
      self.Name = Name
   def GetAge(self):
      return self.Age
   def SetAge(self, Age):
      self.Age = Age
   def __str__(self):
      return "{0} is aged {1}.".format(self.Name,
                                       self.Age)
```

In this case, the class begins by creating an object with two instance variables: `Name` and `Age`. If the user fails to provide these values, they default to `Sam` and `32`.

REMEMBER

This example provides you with a new class feature. Most developers call this feature an *accessor.* Essentially, it provides access to an underlying value. There are two types of accessors: getters and setters. Both `GetName()` and `GetAge()` are *getters.* They provide read-only access to the underlying value. The `SetName()` and `SetAge()` methods are *setters,* which provide write-only access to the underlying value. Using a combination of methods like this allows you to check inputs for correct type and range, as well as verify that the caller has permission to view the information.

As with just about every other class you create, you need to define the `__str__()` method if you want the user to be able to print the object. In this case, the class provides formatted output that lists both of the instance variables.

Saving a class to disk

You could keep your class right in the same file as your test code, but that wouldn't reflect the real world very well. To use this class in a real-world way for the rest of the chapter, you must follow these steps:

1. **Create a new notebook called `BPP4D3E; 15; MyClass.ipynb` and add the code shown in Listing 15-1 to it.**

   You can also use the downloadable source file, `BPP4D3E; 15; MyClass.ipynb`.

2. **Click Run Cell.**

   Python will execute the code without error when you have typed the code correctly.

3. **Choose File ➪ Save and Checkpoint.**

   Notebook saves the file.

4. **Choose File ➪ Download As ➪ Python (.py).**

   Notebook outputs the code as a Python file.

REMEMBER

Note that you must name the file `BPP4D3E_15_MyClass.py` because Python can't import external libraries with spaces in their names.

Using MyClass in an application

Most of the time, you use external classes when working with Python. It isn't very often that a class exists within the confines of the application file because the application would become large and unmanageable. In addition, reusing the class code in another application would be difficult. The following steps help you use the `MyClass` class that you created in the previous section.

1. **Type the following code into a new cell in the `BPP4D3E; 15; Creating and Using Classes.ipynb` notebook — pressing Enter after each line:**

```
import BPP4D3E_15_MyClass

SamsRecord = BPP4D3E_15_MyClass.MyClass()
AmysRecord = BPP4D3E_15_MyClass.MyClass("Amy", 44)

print(SamsRecord.GetAge())
SamsRecord.SetAge(33)

print(AmysRecord.GetName())
AmysRecord.SetName("Aimee")

print(SamsRecord)
print(AmysRecord)
```

REMEMBER

The example code begins by importing the `BPP4D3E_15_MyClass` package. The package name is the name of the file used to store the external code, not the name of the class. A single package can contain multiple classes, so always think of the package as being the actual file that is used to hold one or more classes that you need to use with your application.

After the package is imported, the application creates two `MyClass` objects. Notice that you use the package name first, followed by the class name. The first object, `SamsRecord`, uses the default settings. The second object, `AmysRecord`, relies on custom settings.

Sam has become a year older. The application updates Sam's age.

Somehow, HR spelled Amy's name wrong. It turns out that *Amy* is an incorrect spelling. The application makes a correction to `AmysRecord`. The final step prints both records in their entirety.

2. **Click Run Cell.**

The application displays a series of messages as it puts `MyClass` through its paces, as shown in Figure 15-10. At this point, you know all the essentials of creating great classes.

Using MyClass in an application

```
In [14]: import BPP4D3E_15_MyClass

         SamsRecord = BPP4D3E_15_MyClass.MyClass()
         AmysRecord = BPP4D3E_15_MyClass.MyClass("Amy", 44)

         print(SamsRecord.GetAge())
         SamsRecord.SetAge(33)

         print(AmysRecord.GetName())
         AmysRecord.SetName("Aimee")

         print(SamsRecord)
         print(AmysRecord)

         32
         Amy
         Sam is aged 33.
         Aimee is aged 44.
```

FIGURE 15-10: The output shows that the class is fully functional.

Extending Classes to Make New Classes

As you might imagine, creating a fully functional, *production-grade class* (one that is used in a real-world application actually running on a system that is accessed by users) is time consuming because real classes perform a lot of tasks. Fortunately, Python supports a feature called *inheritance.* By using inheritance, you can obtain the features you want from a parent class when creating a child class. Overriding the features that you don't need and adding new features lets you create new classes relatively fast and with a lot less effort on your part. In addition, because the parent code is already tested, you don't have to put quite as much effort into ensuring that your new class works as expected. The following sections show how to build and use classes that inherit from each other.

Building the child class

Parent classes are normally supersets of something. In this case, you build a parent class named `Animal` and use it to define a child class named `Chicken`. Of course, you can easily add other child classes after you have `Animal` in place, such as a `Gorilla` class. However, for this example, you build just the one parent and one child class, as shown in Listing 15-2. Type this code into a new cell in the notebook.

LISTING 15-2: Building a Parent and Child Class

```
class Animal:
   def __init__(self, Name="", Age=0, Type=""):
      self.Name = Name
      self.Age = Age
      self.Type = Type
   def GetName(self):
      return self.Name
   def SetName(self, Name):
      self.Name = Name
   def GetAge(self):
      return self.Age
   def SetAge(self, Age):
      self.Age = Age
   def GetType(self):
      return self.Type
   def SetType(self, Type):
      self.Type = Type
   def __str__(self):
      return "{0} is a {1} aged {2}".format(self.Name,
                                            self.Type,
                                            self.Age)
class Chicken(Animal):
   def __init__(self, Name="", Age=0):
      self.Name = Name
      self.Age = Age
      self.Type = "Chicken"
   def SetType(self, Type):
      print("Sorry, {0} will always be a {1}"
            .format(self.Name, self.Type))
   def MakeSound(self):
      print("{0} says Cluck, Cluck, Cluck!".format(self.Name))
```

The Animal class tracks three characteristics: Name, Age, and Type. A production application would probably track more characteristics, but these characteristics do everything needed for this example. The code also includes the required accessors for each of the characteristics. The __str__() method completes the picture by printing a simple message stating the animal characteristics.

The Chicken class inherits from the Animal class. Notice the use of Animal in parentheses after the Chicken class name. This addition tells Python that Chicken is a kind of Animal, something that will inherit the characteristics of Animal.

Notice that the Chicken constructor accepts only Name and Age. The user doesn't have to supply a Type value because you already know that it's a chicken. This new constructor overrides the Animal constructor. The three attributes are still in place, but Type is supplied directly in the Chicken constructor.

Someone might try something funny, such as setting a chicken up as a gorilla. With this in mind, the Chicken class also overrides the SetType() setter. If someone tries to change the Chicken type, that user gets a message rather than the attempted change. Normally, you handle this sort of problem by using an exception, but the message works better for this example by making the coding technique clearer.

Finally, the Chicken class adds a new feature, MakeSound(). Whenever you want to see the sound a chicken makes, just call MakeSound() to see it printed on the screen.

Testing class inheritance in an application

Testing the Chicken class also tests the Animal class to some extent. Some functionality is different, but some classes aren't really meant to be used. The Animal class is simply a parent for specific kinds of animals, such as Chicken. The following steps demonstrate the Chicken class so that you can see how inheritance works.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
MyChicken = Chicken("Sally", 2)
print(MyChicken)

MyChicken.SetAge(MyChicken.GetAge() + 1)
print(MyChicken)
```

```
MyChicken.SetType("Gorilla")
print(MyChicken)

MyChicken.MakeSound()
```

The example creates a chicken, `MyChicken`, named `"Sally"`, who is age `2`. It then starts to work with `MyChicken` in various ways. For example, `Sally` has a birthday, so the code increases `Sally`'s age by `1`. Notice how the code combines the use of a setter, `SetAge()`, with a getter, `GetAge()`, to perform the task. After each change, the code displays the resulting object values for you.

2. **Click Run Cell.**

   You see each of the steps used to work with `MyChicken`, as shown in Figure 15-11.

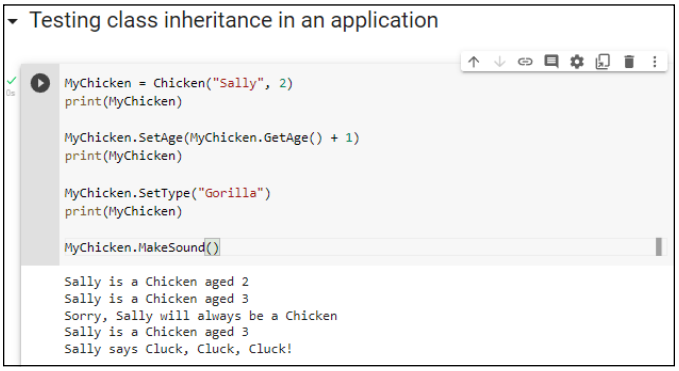


FIGURE 15-11: Sally has a birthday and then says a few words.

4 Performing Advanced Tasks

IN THIS PART . . .

Understanding local file storage

Performing CRUD operations on files

Working with email

IN THIS CHAPTER

- Considering how permanent storage works
- Using permanently stored content
- Creating, reading, updating, and deleting file data

Chapter 16
Storing Data in Files

Until now, application development might seem to be all about presenting information onscreen. Actually, applications revolve around a need to work with data in some way. Data is the focus of all applications because it's the data that users are interested in. Be prepared for a huge disappointment the first time you present a treasured application to a user base and find that the only thing users worry about is whether the application will help them leave work on time after creating a presentation. The fact is, the best applications are invisible, but they present data in the most appropriate manner possible for a user's needs.

If data is the focus of applications, then storing the data in a permanent manner is equally important. For most developers, data storage revolves around permanent media such as a hard drive, Solid State Drive (SSD), Universal Serial Bus (USB) flash drive, or some other methodology. (Cloud-based solutions work fine, but you won't see them used in this book because they require different programming techniques that are beyond the book's scope.) The data in memory is temporary because it lasts only as long as the machine is running. A permanent storage device holds onto the data long after the machine is turned off or the session has ended so that it can be retrieved during the next session.

REMEMBER

In addition to permanent storage, this chapter also helps you understand the four basic operations that you can perform on files: Create, Read, Update, and Delete (CRUD). You see the CRUD acronym used quite often in database circles, but it applies equally well to any application. No matter how your application stores the

data in a permanent location, it must be able to perform these four tasks in order to provide a complete solution to the user. Of course, CRUD operations must be performed in a secure, reliable, and controlled manner. This chapter also helps you set a few guidelines for how access must occur to ensure *data integrity* (a measure of how often data errors occur when performing CRUD operations).

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the `BPP4D3E; 16; Storing Data in Files.ipynb` file of the downloadable source. See the Introduction for details on how to find these source files.

Understanding How Permanent Storage Works

You don't need to understand absolutely every detail about how permanent storage works in order to use it. For example, just how the drive spins (assuming that it spins at all) is unimportant. However, most platforms adhere to a basic set of principles when it comes to permanent storage. These principles have developed over a period of time, starting with mainframe systems in the earliest days of computing. Data is generally stored in one of these ways:

- **Files:** The most common method that uses pure data representing application state information in separate containers called files, with each container having a specific filename. Files appear on local drives, on networks, and in the cloud.
- **Objects:** A method of storing serialized class instances that can appear locally with each object in a separate file or in the cloud with storage specifically designed to handle objects.
- **Blocks:** A method generally associated with the cloud, but also used on a network with Fiber Channel over Ethernet (FCoE) or Internet Small Computer Systems Interface (iSCSI), and that uses storage containers of the same size on disk within software-controlled volumes, with each of these volumes acting as a separate hard drive.

This chapter covers the use of files, not objects or blocks. In addition, it focuses on files used on a local drive, rather than networks or in the cloud. You probably know about files already because almost every useful application out there relies on them. For example, when you open a document in your word processor, you're actually opening a data file containing the words that you or someone else has typed.

Files typically have an *extension* associated with them that defines the file type. The extension is generally standardized for any given application and is separated from the filename by a period, such as `MyData.txt`. In this case, `.txt` is the file extension, and you probably have an application on your machine for opening such files. In fact, you can likely choose from a number of applications to perform the task because the `.txt` file extension is relatively common.

Internally, files structure the data in some specific manner to make it easy to write and read data to and from the file. Any application you write must know about the file structure in order to interact with the data the file contains. The examples in this chapter use a simple file structure to make it easy to write the code required to access them, but file structures can become quite complex.

Files would be nearly impossible to find if you placed them all in the same location on the hard drive. Consequently, files are organized into *directories.* Many computer systems also use the term *folder* for this organizational feature of permanent storage. No matter what you call it, permanent storage relies on directories to help organize the data and make individual files significantly easier to find. To find a particular file so that you can open it and interact with the data it contains, you must know which directory holds the file.

Directories are arranged in hierarchies that begin at the uppermost level of the hard drive. For example, when working with the downloadable source code for this book, you find the code for the entire book in the `BPP4D3E` directory within the user folder on your system when working with Jupyter Notebook or on GitHub (`https://github.com/JohnPaulMueller/BPP4D3E`) when working with Colab. On my Windows system, that directory hierarchy is `C:\Users\John\BPP4D3E`. However, my Mac and Linux systems have a different directory hierarchy to reach the same `BPP4D3E` directory, and the directory hierarchy on your system will be different as well.

REMEMBER

Notice that I use a backslash (\) to separate the directory levels. Some platforms use the forward slash (/); others use the backslash. You can read about this issue on my blog at `http://blog.johnmuellerbooks.com/2014/03/10/backslash-versus-forward-slash/`. The book uses backslashes when appropriate and assumes that you'll make any required changes for your platform.

A final consideration for Python developers (at least for this book) is that the hierarchy of directories is called a *path.* You see the term *path* in a few places in this book because Python must be able to find any resources you want to use based on the path you provide. For example, `C:\Users\John\BPP4D3E` is the complete path to the source code for this chapter on my Windows system. A path that traces the entire route that Python must search is called an *absolute path.* An incomplete path that traces the route to a resource using the current directory as a starting point is called a *relative path.*

To find a location using a relative path, you commonly use the current directory as the starting point. For example, `BPP4D3E\__pycache__` would be the relative path to the Python cache. Note that it has no drive letter or beginning backslash. However, sometimes you must add to the starting point in specific ways to define a relative path. Most platforms define these special relative path character sets:

- `\`: The root directory of the current drive. The drive is relative, but the path begins at the root, the uppermost part, of that drive.
- `.\`: The current directory. You use this shorthand for the current directory when the current directory name isn't known. For example, you could also define the location of the Python cache as `.\__pycache__`.
- `..\`: The parent directory. You use this shorthand when the parent directory name isn't known.
- `..\..\`: The parent of the parent directory. You can proceed up the hierarchy of directories as far as necessary to locate a particular starting point before you drill back down the hierarchy to a new location.

Creating Content for Permanent Storage

A file can contain structured or unstructured data. An example of *structured data* is a database in which each record has specific information in it. An employee database would include columns for name, address, employee ID, and so on. Each record would be an individual employee and each employee record would contain the name, address, and employee ID fields. An example of *unstructured data* is a word processing file whose text can contain any content in any order. There is no required order for the content of a paragraph, and sentences can contain any number of words. However, in both cases, the application must know how to perform CRUD operations with the file. This means that the content must be prepared in such a manner that the application can both write to and read from the file.

Even with word processing files, the text must follow a certain series of rules. Assume for a moment that the files are simple text. Even so, every paragraph must have some sort of delimiter telling the application to begin a new paragraph. The application reads the paragraph until it sees this delimiter, and then it begins a new paragraph. The more that the word processor offers in the way of features, the more structured the output becomes. For example, when the word processor offers a method of formatting the text, the formatting must appear as part of the output file.

REMEMBER

The cues that make content usable for permanent storage are often hidden from sight. All you see when you work with the file is the data itself. The formatting remains invisible for a number of reasons, such as these:

- The cue is a control character, such as a carriage return or linefeed, that is normally invisible by default at the platform level.
- The application relies on special character combinations, such as commas and double quotes, to delimit the data entries. These special character combinations are consumed by the application during reading.
- Part of the reading process converts the character to another form, such as when a word processing file reads in content that is formatted. The formatting appears onscreen in visual form (you see the effect, not the actual formatting), but in the background the file contains special characters to denote the formatting.
- The file is actually in an alternative format, such as eXtensible Markup Language (XML) (see `https://www.w3schools.com/xml/default.ASP` for information about XML). The alternative format is interpreted and presented onscreen in a manner the user can understand.

TECHNICAL STUFF

Other rules likely exist for formatting data. For example, Microsoft actually uses a `.zip` file to hold its latest word processing files (the `.docx`) file. The use of a compressed file catalog, such as `.zip`, makes storing a great deal of information in a small space possible. It's interesting to see how others store data because you can often find more efficient and secure means of data storage for your own applications.

Now that you have a better idea of what could happen as part of preparing content for disk storage, it's time to look at an example. In this case, the formatting strategy is quite simple. All this example does is accept input, format it for storage, and present the formatted version onscreen (rather than save it to disk just yet).

1. **Open a new notebook.**

   You can also use the downloadable source file, `BPP4D3E; 16; Storing Data in Files.ipynb`.

2. **Type the following code into the notebook — pressing Enter after each line:**

```
class FormatData:
   def __init__(self, Name="", Age=0, Married=False):
      self.Name = Name
      self.Age = Age
      self.Married = Married
```

```
    def __str__(self):
        OutString = "'{0}', {1}, {2}".format(
            self.Name,
            self.Age,
            self.Married)
        return OutString
```

This is a shortened class. Normally, you'd add accessors (getter and setter methods) and error-trapping code. (Remember that *getter methods* provide read-only access to class data and *setter methods* provide write-only access to class data.) However, the class works fine for the demonstration.

The main feature to look at is the `__str__()` function. Notice that it formats the output data in a specific way. The string value, `self.Name`, is enclosed in single quotes. Each of the values is also separated by a comma. This is actually a form of a standard output format, comma-separated value (CSV), that is used on a wide range of platforms because it's easy to translate and is in plain text, so nothing special is needed to work with it.

3. **Click Run Cell.**

   The IDE runs the code and creates the required class for you.

4. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
NewData = [FormatData("George", 65, True),
           FormatData("Sally", 47, False),
           FormatData("Doug", 52, True)]
for Entry in NewData:
    print(Entry)
```

   Most of the time, you work with multiple records when you save data to disk. You might have multiple paragraphs in a word processed document or multiple records, as in this case. The example creates a list of records and places them in `NewData`. In this case, `NewData` represents the entire document. The representation will likely take other forms in a production application, but the idea is the same.

   Any application that saves data goes through some sort of output loop. In this case, the loop simply prints the data onscreen. However, in the upcoming sections, you actually output the data to a file.

5. **Click Run Cell.**

   You see the output shown in Figure 16-1. This is a representation of how the data would appear in the file. In this case, each record is separated by a

carriage return and linefeed control character combination. That is, George, Sally, and Doug are all separate records in the file. Each *field* (data element) is separated by a comma. Text fields appear in quotes so that they aren't confused with other data types.

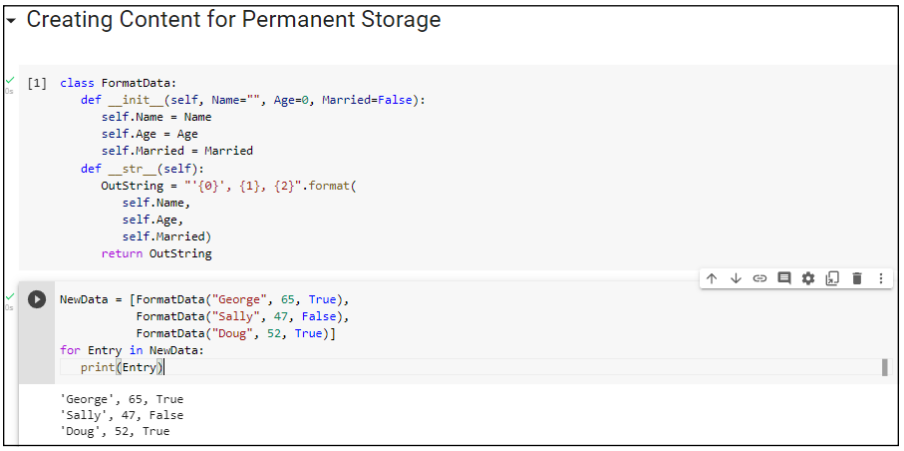


FIGURE 16-1: The example presents how the data might look in CSV format.

Creating a File

Any data that the user creates and wants to work with for more than one session must be put on some sort of permanent media. Creating a file and then placing the data into it is an essential part of working with Python. You can use the following steps to create code that will write data to the hard drive (albeit, only temporarily for Colab).

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
import csv

class FormatData2:
   def __init__(self, Name="", Age=0, Married=False):
      self.Name = Name
      self.Age = Age
      self.Married = Married

   def __str__(self):
      OutString = "'{0}', {1}, {2}".format(
         self.Name,
         self.Age,
```

```
            self.Married)
        return OutString

    def SaveData(Filename = "", DataList = []):
        with open(Filename,
                  "w", newline='\n') as csvfile:
            DataWriter = csv.writer(
                csvfile,
                delimiter='\n',
                quotechar=" ",
                quoting=csv.QUOTE_NONNUMERIC)
            DataWriter.writerow(DataList)
            csvfile.close()
            print("Data saved!")
```

The `csv` module contains everything needed to work with CSV files.

REMEMBER

Python actually supports a huge number of file types natively, and libraries that provide additional support are available. If you have a file type that you need to support using Python, you can usually find a third-party library to support it when Python doesn't support it natively. Unfortunately, no comprehensive list of supported files exists, so you need to search online to find how Python supports the file you need. The documentation divides the supported files by types and doesn't provide a comprehensive list. For example, you can find all the archive formats at `https://docs.python.org/3/library/archiving.html` and the miscellaneous file formats at `https://docs.python.org/3/library/fileformats.html`.

TIP

This example uses essentially the same text formatting code as you saw in the `FormatData` class, but now it adds the `SaveData()` method to put the formatted data on disk. Using a new class notifies everyone who views the code of the increased capability, so `FormatData2` is the same class, but with more features.

Notice that the `SaveData()` method accepts two arguments as input: a filename used to store the data and a list of items to store. This is a class method rather than an instance method. Later in this procedure, you see how using a class method is an advantage. The `DataList` argument defaults to an empty list so that if the caller doesn't pass anything at all, the method won't throw an exception. Instead, it produces an empty output file. Of course, you can also add code to detect an empty list as an error, if desired.

REMEMBER

The `with` statement tells Python to perform a series of tasks with a specific resource — an open `csvfile` named `Testfile.csv`. The `open()` function accepts a number of inputs depending in how you use it. For this example, you open it in write mode (signified by the `w`). The `newline` attribute tells Python to treat the `\n` control character (linefeed) as a newline character.

In order to write output, you need a writer object. The `DataWriter` object is configured to use `csvfile` as the output file, to use `\n` as the record character, to quote records using a space, and to provide quoting only on nonnumeric values. This setup will produce some interesting results later, but for now, just assume that this is what you need to make the output usable.

Actually writing the data takes less effort than you might think. A single call to `DataWriter.writerow()` with the `DataList` as input is all you need. Always close the file when you get done using it. This action *flushes the data* (makes sure that it gets written) to the hard drive. The code ends by telling you that the data has been saved.

2. **Click Run Cell.**

   The IDE runs the code and creates the required class for you.

3. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
NewData = [FormatData2("George", 65, True),
           FormatData2("Sally", 47, False),
           FormatData2("Doug", 52, True)]
FormatData2.SaveData("TestFile.csv", NewData)
```

   This example should look similar to the one you created in the "Creating Content for Permanent Storage" section, earlier in the chapter. You still create `NewData` as a list. However, instead of displaying the information onscreen, you send it to a file instead by calling `FormatData2.SaveData()`. This is one of those situations in which using an instance method would actually get in the way. To use an instance method, you would first need to create an instance of `FormatData` that wouldn't actually do anything for you.

4. **Click Run Cell.**

   The application runs, and you see a message that the data was saved as output. Of course, that doesn't tell you anything about the data. You use separate methods for viewing the data depending on whether you work with Colab or Jupyter Notebook:

   - **Colab:** Click the Files icon that appears on the left side of the browser window to display a list of files associated with this Colab session. Double-click `TestFile.csv` to see it displayed as shown in Figure 16-2. Notice that Colab assumes that every `.csv` file has a header row, so it displays the first data entry as a header.
   - **Jupyter Notebook:** Choose File ➪ Open to display the directory listing for the current project. Click the `TestFile.csv` entry near the bottom of the page. Jupyter Notebook displays the three entries as you might expect, with no header row, as shown in Figure 16-3.

FIGURE 16-2: Colab displays the file content using the first row as a header.

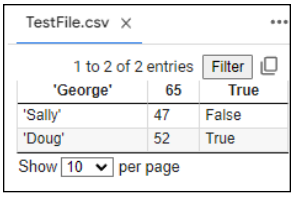
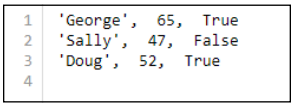


FIGURE 16-3: Jupyter Notebook makes no assumption about the header row.



LOCAL FILE STORAGE: COLAB VERSUS JUPYTER NOTEBOOK

The files you create in Colab aren't necessarily permanent. You may find that they simply disappear between sessions when Google stops the virtual machine hosting your code. On the other hand, the files you create in Jupyter Notebook reside on your local system and will remain in place for as long as you want to keep them. You can always check the content of the directories for either IDE to determine whether your file is still in place by using the following code:

```
import os

print(os.getcwd())

DirList = os.listdir()
print(DirList)
print("TestFile.csv" in DirList)
```

The first `print()` output tells you where your files are stored. The second `print()` output provides you with a list of files in that directory. The third `print()` output tells you specifically whether the file you need appears in that directory. It's important to know how to check for the files you need between sessions. The only way to ensure that your files are indeed permanent when working with Colab is to save them to cloud storage, such as Google Drive. The articles at `https://www.geeksforgeeks.org/uploading-files-on-google-drive-using-python/` and `https://www.projectpro.io/recipes/upload-files-to-google-drive-using-python` give you some ideas of how to accomplish this task.

Reading File Content

At this point, the data is on the hard drive. Of course, it's nice and safe there, but it really isn't useful because you can't see it. To see the data, you must read it into memory and then do something with it. The following steps show how to read data from the hard drive and into memory so that you can display it onscreen.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
import csv

class FormatData3:
   def __init__(self, Name="", Age=0, Married=False):
      self.Name = Name
      self.Age = Age
      self.Married = Married

   def __str__(self):
      OutString = "'{0}', {1}, {2}".format(
         self.Name,
         self.Age,
         self.Married)
      return OutString

   def SaveData(Filename = "", DataList = []):
       with open(Filename,
                 "w", newline='\n') as csvfile:
          DataWriter = csv.writer(
             csvfile,
             delimiter='\n',
             quotechar=" ",
             quoting=csv.QUOTE_NONNUMERIC)
          DataWriter.writerow(DataList)
          csvfile.close()
          print("Data saved!")

   def ReadData(Filename = ""):
       with open(Filename,
                 "r", newline='\n') as csvfile:
          DataReader = csv.reader(
             csvfile,
             delimiter="\n",
             quotechar=" ",
             quoting=csv.QUOTE_NONNUMERIC)
```

```
            Output = []
            for Item in DataReader:
               Output.append(Item[0])
            csvfile.close()
            print("Data read!")
            return Output
```

Opening a file for reading is much like opening it for writing. The big difference is that you need to specify `r` (for read) instead of `w` (for write) as part of the `open()` method call. Otherwise, the arguments are precisely the same and work the same.

REMEMBER

It's important to remember that you're starting with a text file when working with a `.csv` file. Yes, it has delimiters, but it's still text. When reading the text into memory, you must rebuild the Python structure. In this case, `Output` is an empty list when it starts.

The file currently contains three records that are separated by the `/n` control character. Python reads each record in using a `for` loop. Notice the odd use of `Item[0]`. When Python reads the record, it sees the nonterminating entries (those that aren't last in the file) as actually being two list entries. The first entry contains data; the second is blank. You want only the first entry. These entries are appended to `Output` so that you end up with a complete list of the records that appear in the file.

As before, make sure that you close the file when you get done with it. The method prints a data read message when it finishes. It then returns `Output` (a list of records) to the caller.

2. **Click Run Cell.**

   The IDE runs the code and creates the required class for you.

3. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
NewData = FormatData3.ReadData("TestFile.csv")
for Entry in NewData:
   print(Entry)
```

The first step is to create a `NewData` object, a list, by calling `FormatData.ReadData()`. Notice that the use of a class method is the right choice in this case as well because it makes the code shorter and simpler. In the next step, the application uses a `for` loop to display the `NewData` content.

4. **Click Run Cell.**

   You see the output shown in Figure 16-4. Notice that this output looks similar to the output in Figure 16-1, even though the data was written to disk and read back in. This is how applications that read and write data are supposed to work. The data should appear the same after you read it in as it did when you wrote it out to disk. Otherwise, the application is a failure because it has modified the data.

FIGURE 16-4: The application input after it has been processed.

Updating File Content

Some developers treat updating a file as something complex. It can be complex if you view it as a single task. However, updates actually consist of three activities:

1. Read the file content into memory.
2. Modify the in-memory presentation of the data.
3. Write the resulting content to permanent storage.

In most applications, you can further break down the second step of modifying the in-memory presentation of the data. An application can provide some or all of these features as part of the modification process:

- Provide an onscreen presentation of the data.
- Allow additions to the data list.
- Allow deletions from the data list.
- Make changes to existing data, which can actually be implemented by adding a new record with the changed data and deleting the old record.

So far in this chapter, you have performed all but one of the activities in these two lists. You've already read file content and written file content. In the modification list, you've added data to a list and presented the data onscreen. The only interesting activity that you haven't performed is deleting data from a list. The modification of data is often performed as a two-part process of creating a new record

that starts with the data from the old record and then deleting the old record after the new record is in place in the list.

REMEMBER

Don't get into a rut by thinking that you must perform every activity mentioned in this section for every application. A monitoring program wouldn't need to display the data onscreen. In fact, doing so might be harmful (or at least inconvenient). A data logger only creates new entries — it never deletes or modifies them. An email application usually allows the addition of new records and deletion of old records, but not modification of existing records. On the other hand, a word processor implements all the features mentioned. What you implement and how you implement it depends solely on the kind of application you create.

Separating the user interface from the activities that go on behind the user interface is important. To keep things simple, this example focuses on what needs to go on behind the user interface to make updates to the file you created in the "Creating a File" section, earlier in this chapter. The following steps demonstrate how to read, modify, and write a file in order to update it. The updates consist of an addition, a deletion, and a change. To allow you to run the application more than once, the updates are actually sent to another file.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
import os.path

if not os.path.isfile("TestFile.csv"):
   print("Please run the CreateFile.py example!")
   quit()

NewData = FormatData3.ReadData("TestFile.csv")
for Entry in NewData:
   print(Entry)

print("\r\nAdding a record for Harry.")
NewRecord = "'Harry', 23, False"
NewData.append(NewRecord)
for Entry in NewData:
   print(Entry)

print("\r\nRemoving Doug's record.")
Location = NewData.index("'Doug', 52, True")
Record = NewData[Location]
NewData.remove(Record)
for Entry in NewData:
   print(Entry)
```

```
print("\r\nModifying Sally's record.")
Location = NewData.index("'Sally', 47, False")
Record = NewData[Location]
Split = Record.split(",")
NewRecord = FormatData3(Split[0].replace("'", ""),
                        int(Split[1]),
                        bool(Split[2]))
NewRecord.Married = True
NewRecord.Age = 48
NewData.append(NewRecord.__str__())
NewData.remove(Record)
for Entry in NewData:
   print(Entry)

FormatData3.SaveData("ChangedFile.csv", NewData)
```

This example has quite a bit going on. First, it checks to ensure that the `TestFile.csv` file is actually present for processing. This is a check that you should always perform when you expect a file to be present. In this case, you aren't creating a new file, you're updating an existing file, so the file must be present. If the file isn't present, the application ends.

The next step is to read the data into `NewData`. This part of the process looks much like the data reading example earlier in the chapter.

REMEMBER

You have already seen code for using list functions in Chapter 13. This example uses those functions to perform practical work. The `append()` function adds a new record to `NewData`. However, notice that the data is added as a string, not as a `FormatData` object. The data is stored as strings on disk, so that's what you get when the data is read back in. You can either add the new data as a string or create a `FormatData` object and then use the `__str__()` method to output the data as a string.

The next step is to remove a record from `NewData`. To perform this task, you must first find the record. Of course, that's easy when working with just four records (remember that `NewData` now has a record for `Harry` in it). When working with a large number of records, you must first search for the record using the `index()` function. This act provides you with a number containing the location of the record, which you can then use to retrieve the actual record. After you have the actual record, you can remove it using the `remove()` function.

Modifying Sally's record looks daunting at first, but again, most of this code is part of dealing with the string storage on disk. When you obtain the record from `NewData`, what you receive is a single string with all three values in it. The `split()` function produces a list containing the three entries as strings, which still won't work for the application. In addition, Sally's name is enclosed in both double and single quotes.

The simplest way to manage the record is to create a `FormatData` object and to convert each of the strings into the proper form. This means removing the extra quotes from the name, converting the second value to an `int`, and converting the third value to a `bool`. The `FormatData` class doesn't provide accessors, so the application modifies both the `Married` and `Age` fields directly. Using accessors (getter methods that provide read-only access and setter methods that provide write-only access) is a better policy.

The application then appends the new record to and removes the existing record from `NewData`. Notice how the code uses `NewRecord.__str__()` to convert the new record from a `FormatData` object to the required string.

The final act is to save the changed record. Normally, you'd use the same file to save the data. However, the example saves the data to a different file in order to allow examination of both the old and new data.

2. **Click Run Cell.**

TIP

You see the following output:

```
Data read!
'George', 65, True
'Sally', 47, False
'Doug', 52, True

Adding a record for Harry.
'George', 65, True
'Sally', 47, False
'Doug', 52, True
'Harry', 23, False

Removing Doug's record.
'George', 65, True
'Sally', 47, False
'Harry', 23, False

Modifying Sally's record.
'George', 65, True
'Harry', 23, False
'Sally', 48, True
Data saved!
```

Notice that the application lists the records after each change so that you can see the status of `NewData`. This is actually a useful troubleshooting technique for your own applications. Of course, you want to remove the display code before you release the application to production.

3. **Open the `ChangedFile.csv` file using an appropriate technique.**

   You see output similar to the following:

```
'George',  65,  True
'Harry',  23,  False
'Sally',  48,  True
```

Deleting a File

The previous section of this chapter, "Updating File Content," explains how to add, delete, and update records in a file. However, at some point you may need to delete the file. The following steps describe how to delete files that you no longer need.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line:**

```
import os
os.remove("ChangedFile.csv")
print("File Removed!")
```

WARNING

The task looks simple in this case, and it is. All you need to do to remove a file is call `os.remove()` with the appropriate filename and path (as needed, Python defaults to the current directory, so you don't need to specify a path if the file you want to remove is in the default directory). The `os.remove()` call will throw an exception if you try to use it to delete a non-existent file. The ease with which you can perform this task is almost scary because it's too easy. Putting safeguards in place is always a good idea. You may want to remove other items, so here are other functions you should know about:

- `os.rmdir()`**:** Removes the specified directory. The directory must be empty or Python will display an exception message.
- `shutil.rmtree()`**:** Removes the specified directory, all subdirectories, and all files. This function is especially dangerous because it removes everything without checking (Python assumes that you know what you're doing). As a result, you can easily lose data using this function.

2. **Click Run Cell.**

   The application displays the `File Removed!` message. When you look in the directory that originally contained the `ChangedFile.csv` file, you see that the file is gone.

IN THIS CHAPTER

- Defining the series of events for sending an email
- Developing an email application
- Testing the email application

Chapter 17
Sending an Email

This chapter helps you understand the process of sending an email using Python. More important, this chapter is generally about helping you understand what happens when you communicate outside the local PC. Even though this chapter is specifically about email, it also contains principles you can use when performing other network tasks. For example, when working with an external service, you often need to create the same sort of packaging as you do for an email. So, the information you see in this chapter can help you understand all sorts of communication needs.

To make working with email as easy as possible, this chapter uses standard mail as a real-world equivalent of email. The comparison is apt. Email was actually modeled on real-world mail. Originally, the term email was used for any sort of electronic document transmission, and some forms of it required the sender and recipient to be online at the same time. As a result, you may find some confusing references online about the origins and development of email. This chapter views email as it exists today — as a storing and forwarding mechanism for exchanging documents of various types.

The examples in this chapter rely on the availability of a Simple Mail Transfer Protocol (SMTP) server. If that sounds like Greek to you, read the sidebar entitled "Considering the SMTP server" that appears later in the chapter.

WARNING

Note that the examples in this chapter won't work with Google Colab because of limitations with the IDE and security concerns. However, if you need to send an email using Colab, you can use the example at `https://colab.research.google.com/github/j-agrawal/coding/blob/master/send_mail.ipynb`, which isn't as

flexible as using the approach in this chapter but does work. The example at https://docs.python.org/3/library/smtplib.html#smtp-example is an even simpler method when you can send email without using the Secure Sockets Layer (SSL) encryption. If you need access to the newer STARTLS (see the differences at https://www.sparkpost.com/resources/email-explained/ssl-tls-starttls-encyption/) that relies on either SSL or Transport Layer Security (TLS), the discussion at https://stackoverflow.com/questions/33857698/sending-email-from-python-using-starttls will help you out.

REMEMBER

You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. You can find the source for this chapter in the BPP4D3E; 17; Sending an Email.ipynb file of the downloadable source. See the Introduction for details on how to find these source files.

Understanding What Happens When You Send Email

Email has become so reliable and so mundane that most people don't understand what a miracle it is that it works at all. Actually, the same can be said of the real mail service. When you think about it, the likelihood of one particular letter leaving one location and ending up precisely where it should at the other end seems impossible — mind-boggling, even. However, both email and its real-world equivalent have several aspects in common that improve the likelihood that they'll actually work as intended. The following sections examine what happens when you write an email, click Send, and the recipient receives it on the other end. You might be surprised at what you discover.

CONSIDERING THE SIMPLE MAIL TRANSFER PROTOCOL

When you work with email, you see a lot of references to Simple Mail Transfer Protocol (SMTP). Of course, the term looks really technical, and what happens under the covers truly *is* technical, but all you really need to know is that it works. On the other hand, understanding SMTP is little more than a "black box" that takes an email from the sender and spits it out at the other end to the recipient can be useful. Taking the term apart (in reverse order), you see these elements:

- **Protocol:** A standard set of rules. Email works by requiring rules that everyone agrees upon. Otherwise, email would become unreliable.

- **Mail transfer:** Documents are sent from one place to another, much the same as what the post office does with real mail. In email's case, the transfer process relies on short commands that your email application issues to the SMTP server. For example, the `MAIL FROM` command tells the SMTP server who is sending the email, while the `RCPT TO` command states where to send it.
- **Simple:** States that this activity goes on with the least amount of effort possible. The fewer parts to anything, the more reliable it becomes.

If you were to look at the rules for transferring the information, you would find they're anything but simple. For example, RFC1123 is a standard that specifies how Internet hosts are supposed to work (see `http://www.faqs.org/rfcs/rfc1123.html` for details). These rules are used by more than one Internet technology, which explains why most of them appear to work about the same (even though their resources and goals may be different).

Another, entirely different standard, RFC2821, describes how SMTP specifically implements the rules found in RFC1123 (see `http://www.faqs.org/rfcs/rfc2821.html` for details). The point is, a whole lot of rules are written in jargon that only a true geek could love (and even the geeks aren't sure). If you want a plain-English explanation of how email works, check out the article at `https://computer.howstuffworks.com/e-mail-messaging/email.htm`. Page 4 of this article (`https://computer.howstuffworks.com/e-mail-messaging/email3.htm`) describes the commands that SMTP uses to send information hither and thither across the Internet. In fact, if you want the shortest possible description of SMTP, page 4 is probably the right place to look.

Viewing email as you do a letter

The best way to view email is the same as how you view a letter. When you write a letter, you provide two pieces of paper as a minimum. The first contains the content of the letter, the second is an envelope. Assuming that the postal service is honest, the content is never examined by anyone other than the recipient. The same can be said of email. An email actually consists of these components:

- **Message:** The content of the email, which is actually composed of two subparts:
  - **Header:** The part of the email content that includes the subject, the list of recipients, and other features, such as the urgency of the email.
  - **Body:** The part of the email content that contains the actual message. The message can be in plain text, formatted as HTML (possibly Rich Text Format, RTF, as well), and consisting of one or more documents, or it can be a combination of all these elements.

» **Envelope:** A container for the message. The envelope provides sender and recipient information, just as the envelope for a physical piece of mail provides. However, an email doesn't include a stamp.

When working with email, you create a message using an email application. As part of the email application setup, you also define account information. When you click send:

1. The email application wraps up your message, with the header first, in an envelope that includes both your sender and the recipient's information.
2. The email application uses the account information to contact the SMTP server and send the message for you.
3. The SMTP server reads only the information found in the message envelope and redirects your email to the recipient.
4. The recipient email application logs on to the local server, picks up the email, and then displays only the message part for the user.

The process is a little more complex than this explanation, but this is essentially what happens. In fact, it's much the same as the process used when working with physical letters in that the essential steps are the same. With physical mail, the email application is replaced by you on one end and the recipient at the other. The SMTP server is replaced by the post office and the employees who work there (including the postal carriers). However, someone generates a message, the message is transferred to a recipient, and the recipient receives the message in both cases.

Defining the parts of the envelope

There is a difference in how the envelope for an email is configured and how it's actually handled. When you view the envelope for an email, it looks just like a letter in that it contains the address of the sender and the address of the recipient. It may not look physically like an envelope, but the same components are there. When you visualize a physical envelope, you see certain specifics, such as the sender's name, street address, city, state, and zip code. The same is true for the recipient. These elements define, in physical terms, where the postal carrier should deliver the letter or return the letter when it can't be delivered.

However, when the SMTP server processes the envelope for an email, it must look at the specifics of the address, which is where the analogy of a physical envelope used for mail starts to break down a little. An email address contains different information from a physical address. In summary, here is what the email address contains:

- **Host:** The host is akin to the city and state used by a physical mail envelope. A host address is the address used by the carrier or server that is physically connected to the Internet, and it handles all the traffic that the Internet consumes or provides for this particular machine. A PC can use Internet resources in a lot of ways, but the host address for all these uses is the same.
- **Port:** The port is akin to the street address used by a physical mail envelope. It specifies which specific part of the system should receive the message. For example, an SMTP server used for outgoing messages normally relies on port 25. However, the Post Office Protocol (POP3) server used for incoming email messages usually relies on port 110. Your browser typically uses port 80 to communicate with websites. However, secure websites (those that use https as a protocol, rather than http) rely on port 443 instead. You can see a list of typical ports at `https://en.wikipedia.org/wiki/List_of_TCP_and_UDP_port_numbers`.
- **Local hostname:** The local hostname is the human-readable form of the combination of the host and port. For example, the website `https://www.myplace.com` (not a real URL) might resolve to an address of 55.225.163.40:80 (where the first four numbers are the host address and the number after the colon is the port). Python takes care of these details behind the scenes for you, so normally you don't need to worry about them. However, it's nice to know that this information is available.

Now that you have a better idea of how the address is put together, it's time to look at it more carefully. The following sections describe the envelope of an email in more precise terms.

Host

A *host address* is the identifier for a connection to a server. Just as an address on an envelope isn't the actual location, neither is the host address the actual server. It merely specifies the location of the server.

REMEMBER

The connection used to access a combination of a host address and a port is called a *socket.* Just who came up with this odd name and why isn't important. What is important is that you can use the socket to find out all kinds of information that's useful in understanding how email works. The following steps help you see host-names and host addresses at work. More important, you begin to understand the whole idea of an email envelope and the addresses it contains.

1. **Open a new notebook.**

   You can also use the downloadable source file, `BPP4D3E; 17; Sending an Email.ipynb`, which contains the application code.

2. **Type the following code into the notebook — pressing Enter after each line — and then click Run Cell.**

```
import socket
print(socket.gethostbyname("localhost"))
print(socket.gethostbyaddr("127.0.0.1"))
```

Before you can work with sockets, you must import the socket library. This library contains all sorts of confusing attributes, so use it with caution. However, this library also contains some interesting functions that help you see how the Internet addresses work.

The next line displays a host address as output. In this case, you should see `127.0.0.1` as output because `localhost` is a standard hostname. The address, `127.0.0.1`, is associated with the host name, `localhost`.

Be prepared for a surprise. You get a tuple as output similar to this one:

```
('MAIN', [], ['127.0.0.1'])
```

However, instead of getting `localhost` as the name of the host, you get the name of your machine. You use `localhost` as a common name for the local machine, but when you specify the address, you get the machine name instead. In this case, `'MAIN'` is the name of my personal machine. The name you see on your screen will correspond to your machine.

3. **In a new cell, type** `print(socket.gethostbyname("www.johnmueller-books.com"))` **and click Run Cell.**

You see an address for my website, which is currently:

72.167.241.46

The point is that these addresses work wherever you are and whatever you're doing — just like those you place on a physical envelope. The physical mail uses addresses that are unique across the world, just as the Internet does.

Port

A *port* is a specific entryway for a server location. The host address specifies the location, but the port defines where to get in. Even if you don't specify a port every time you use a host address, the port is implied. Access is always granted using a combination of the host address and the port. The following steps help illustrate how ports work with the host address to provide server access:

1. **Type the following code into a new cell in the notebook — pressing Enter after each line — and then click Run Cell.**

```
import socket
socket.getaddrinfo("localhost", 110)
```

Remember that a socket provides both host address and port information. You use the socket to create a connection that includes both items. The first value is the name of a host you want to obtain information about. The second value is the port on that host. In this case, you obtain the information about localhost port 110. When you run the code, you see output similar to this:

```
[(<AddressFamily.AF_INET6: 23>, 0, 0, '',
 ('::1', 110, 0, 0)),
 (<AddressFamily.AF_INET: 2>, 0, 0, '',
 ('127.0.0.1', 110))]
```

The output consists of two tuples: one for the Internet Protocol version 6 (IPv6) output and one for the Internet Protocol version 4 (IPv4) address. The address is indicated by the `AddressFamily` entry, which is `AF_INET6` for IPv6 and `AF_INET` for IPv4. Each of these tuples contains five entries, four of which you really don't need to worry about because you'll likely never need them. However, the last entry, `('127.0.0.1', 110)`, shows the IPv4 address and port for localhost port 110.

If you really want to know more about address information content, check out the web pages at `https://www.ibm.com/docs/it/i/7.1?topic=families-using-af-inet-address-family` and `https://www.ibm.com/docs/it/i/7.1?topic=families-using-af-inet6-address-family`.

2. **In a new cell, type** `socket.getaddrinfo("johnmuellerbooks.com", 80)` **and click Run Cell.**

   You see this output when you run the code:

   ```
   [(<AddressFamily.AF_INET: 2>, 0, 0, '',
    ('72.167.241.46', 80))]
   ```

   Notice that this Internet location provides only an IPv4 address, not an IPv6, address, for port 80. The `socket.getaddrinfo()` method provides a useful method for determining how you can access a particular location. Using IPv6 provides significant benefits over IPv4 (see `https://www.mygreatlearning.com/blog/ipv4-vs-ipv6/` for details), but many Internet locations provide only IPv4 support now. (If you live in a larger city, you'll probably see both an IPv4 and an IPv6 address.)

3. **In a new cell, type** `socket.getservbyport(25)` **and click Run Cell.**

   You see the following output:

   ```
   'smtp'
   ```

The `socket.getservbyport()` method provides the means to determine how a particular port is used. Port 25 is always dedicated to SMTP support on any server. So, when you access 127.0.0.1:25, you're asking for the SMTP server on localhost. In short, a port provides a specific kind of access in many situations.

REMEMBER

Some people assume that the port information is always provided. However, this isn't always the case. Python will provide a default port when you don't supply one, but relying on the default port is a bad idea because you can't be certain which service will be accessed. In addition, some systems use nonstandard port assignments as a security feature. Always get into the habit of using the port number and ensuring that you have the right one for the task at hand.

Local hostname

A *hostname* is simply the human-readable form of the host address. Humans don't really understand 127.0.0.1 very well (and the IPv6 addresses make even less sense). However, humans do understand localhost just fine. There is a special server and setup to translate human-readable hostnames to host addresses, but you really don't need to worry about it for this book (or programming in general). When your application suddenly breaks for no apparent reason, it helps to know that one does exist, though.

The "Host" section, earlier in this chapter, introduces you to the hostname to a certain extent through the use of the `socket.gethostbyaddr()` method, whereby an address is translated into a hostname. You saw the process in reverse using the `socket.gethostbyname()` method. The following steps help you understand some nuances about working with the hostname:

1. **Type the following code into a new cell in the notebook — pressing Enter after each line — and then click Run Cell.**

```
import socket
socket.gethostname()
```

   You see the name of the local system, which is `'MAIN'` on my system, but will be different on yours.

2. **In a new cell, type `socket.gethostbyname(socket.gethostname())` and click Run Cell.**

   You see the IP address of the local system, which is `'192.168.0.103'` on my system. Again, your setup is likely different from mine, so the output you see will differ. This is a method you can use in your applications to determine the address of the sender when needed. Because it doesn't rely on any hard-coded value, the method works on any system.

Defining the parts of the letter

The "envelope" for an email address is what the SMTP server uses to route the email. However, the envelope doesn't include any content — that's the purpose of the letter. A lot of developers get the two elements confused because the letter contains sender and receiver information as well. This information appears in the letter just like the address information that appears in a business letter — it's for the benefit of the viewer. When you send a business letter, the postal delivery person doesn't open the envelope to see the address information inside. Only the information on the envelope matters.

It's because the information in the email letter is separate from its information in the envelope that nefarious individuals can spoof email addresses. The envelope potentially contains legitimate sender information, but the letter may not. (When you see the email in your email application, all that is present is the letter, not the envelope — the envelope has been stripped away by the email application.) For that matter, neither the sender nor the recipient information may be correct in the letter that you see onscreen in your email reader.

The letter part of an email is actually made of separate components, just as the envelope is. Here is a summary of the three components:

- **Sender:** The sender information tells you who sent the message. It contains just the email address of the sender.
- **Receiver:** The receiver information tells you who will receive the message. This is actually a list of recipient email addresses. Even if you want to send the message to only one person, you must supply the single email address in a list.
- **Message:** Contains the information that you want the recipient to see. This information can include the following:
  - **From:** The human-readable form of the sender.
  - **To:** The human-readable form of the recipients.
  - **CC:** Visible recipients who also received the message, even though they aren't the primary targets of the message.
  - **BCC:** Hidden recipients who also received the message, even though they aren't the primary targets of the message.
  - **Subject:** The purpose of the message.
  - **Documents:** One or more documents, including the text message that appears with the email.

Emails can actually become quite complex and lengthy. Depending on the kind of email that is sent, a message could include all sorts of additional information. However, most emails contain these simple components, and this is all the information you need to send an email from your application. The following sections describe the process used to generate a letter and its components in more detail.

Defining the message

Sending an empty envelope to someone will work, but it isn't very exciting. In order to make your email message worthwhile, you need to define a message. Python supports a number of methods of creating messages. However, the easiest and most reliable way to create a message is to use the Multipurpose Internet Mail Extensions (MIME) functionality that Python provides (and no, a MIME is not a silent person with white gloves who acts out in public).

As with many email features, MIME is standardized, so it works the same no matter which platform you use. There are also numerous forms of MIME that are all part of the `email.mime` module described at `https://docs.python.org/3/library/email.mime.html`. Here are the forms that you need to consider most often when working with email:

- **MIMEApplication:** Provides a method for sending and receiving application input and output
- **MIMEAudio:** Contains an audio file
- **MIMEImage:** Contains an image file
- **MIMEMultipart:** Allows a single message to contain multiple subparts, such as including both text and graphics in a single message
- **MIMEText:** Contains text data that can be in ASCII, HTML, or another standardized format

Although you can create any sort of an email message with Python, the easiest type to create is one that contains plain text. The lack of formatting in the content lets you focus on the technique used to create the message, rather than on the message content. The following steps help you understand how the message-creating process works, but you won't actually send the message anywhere.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line.**

```
from email.mime.text import MIMEText
msg = MIMEText("Hello There")
msg['Subject'] = "A Test Message"
```

```
msg['From']='John Mueller ' + \
    '<John@JohnMuellerBooks.com>'
msg['To'] = 'John Mueller ' + \
    '<John@JohnMuellerBooks.com>'

msg.as_string()
```

REMEMBER

This is a basic plain-text message. Before you can do anything, you must import the required class, which is `MIMEText`. If you were creating some other sort of message, you'd need to import other classes or import the `email.mime` module as a whole.

The `MIMEText()` constructor requires message text as input. This is the body of your message, so it might be quite long. In this case, the message is relatively short — just a greeting.

At this point, you assign values to standard attributes. The example shows the three common attributes that you always define: `Subject`, `From`, and `To`. The two address fields, `From` and `To`, contain both a human-readable name and the email address. All you have to include is the email address.

The call to `msg.as_string()` displays the actual message.

2. **Click Run Cell.**

You see the output shown in Figure 17-1. This is how the message actually looks. If you have ever looked under the covers at the messages produced by your email application, the text probably looks familiar.

Defining the parts of the letter

Defining the message

```
In [8]: from email.mime.text import MIMEText
        msg = MIMEText("Hello There")
        msg['Subject'] = "A Test Message"
        msg['From']='John Mueller ' + \
            '<John@JohnMuellerBooks.com>'
        msg['To'] = 'John Mueller ' + \
            '<John@JohnMuellerBooks.com>'

        msg.as_string()

Out[8]: 'Content-Type: text/plain; charset="us-ascii"\nMIME-Version: 1.0\nContent-Trans
        fer-Encoding: 7bit\nSubject: A Test Message\nFrom: John Mueller <John@JohnMuell
        erBooks.com>\nTo: John Mueller <John@JohnMuellerBooks.com>\n\nHello There'
```

FIGURE 17-1: Python adds some additional information required to make your message work.

The `Content-Type` reflects the kind of message you created, which is a plain-text message. The `charset` tells what kind of characters are used in the message so that the recipient knows how to handle them. The `MIME-Version` specifies the version of MIME used to create the message so that the recipient

knows whether it can handle the content. Finally, the `Context-Transfer-Encoding` determines how the message is converted into a bit stream before it is sent to the recipient.

Specifying the transmission

An earlier section ("Defining the parts of the envelope") describes how the envelope is used to transfer the message from one location to another. The process of sending the message entails defining a transmission method. Python actually creates the envelope for you and performs the transmission, but you must still define the particulars of the transmission. The following steps help you understand the simplest approach to transmitting a message using Python. These steps won't result in a successful transmission unless you modify them to match your setup. Read the "Considering the SMTP server" sidebar, later in this chapter, for additional information.

1. **Open an Anaconda Prompt, type** `python -m smtpd -n -c DebuggingServer localhost:1025`**, and press Enter.**

   Older platforms would allow you to use port 25 to transmit messages. Newer platforms display a `ConnectionRefusedError` instead. This is a security feature and it shows that certain platform features aren't installed by default. This command creates an SMTP server on port 1025 for you to use for this example. Don't worry when the command doesn't appear to have completed and you see just a blank line. The server is simply waiting for you to use it.

2. **Type the following code into a new cell in the notebook — pressing Enter after each line.**

```
import smtplib
s = smtplib.SMTP('localhost', 1025)

s.sendmail('SenderAddress',
           ['RecipientAddress'],
           msg.as_string())
```

   The `smtplib` module contains everything needed to create the message envelope and send it. The first step in this process is to create a connection to the SMTP server, which you name as a string in the constructor. If the SMTP server that you provide doesn't exist, the application will fail at this point, saying that the host actively refused the connection.

WARNING

   For the call to `sendmail()` to work, you must replace `SenderAddress` and `RecipientAddress` with real addresses. Don't include the human-readable form this time — the server requires only an address. If you don't include a real address, you'll definitely see an error message when you click Run Cell. You might also see an error if your email server is temporarily offline, there is a

glitch in the network connection, or any of a number of other odd things occur. If you're sure that you typed everything correctly, try sending the message a second time before you panic. See the sidebar "Considering the SMTP server," later in this chapter, for additional details.

3. **Click Run Cell.**

   This is the step that actually creates the envelope, packages the email message, and sends it off to the recipient. Notice that you specify the sender and recipient information separately from the message, which the SMTP server doesn't read. Figure 17-2 shows typical output (viewed at the Anaconda Prompt) from this part of the example.

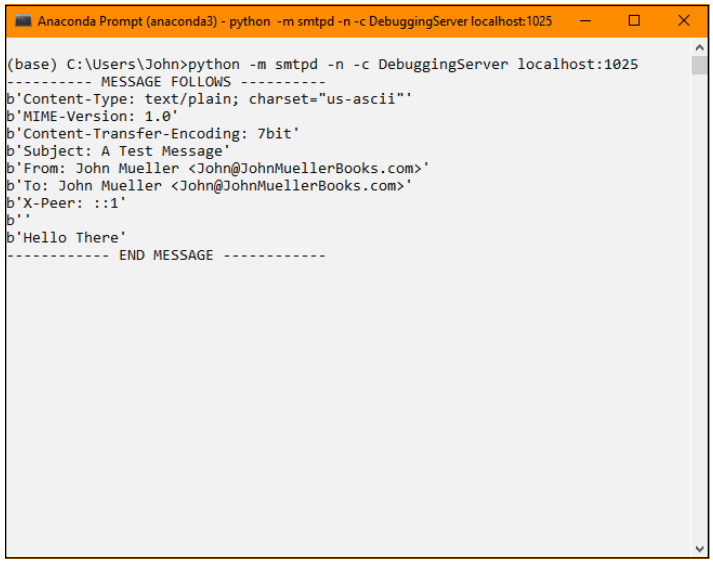


FIGURE 17-2: You see the message transmitted by the SMTP server at the Anaconda Prompt.

Considering the message subtypes

The "Defining the message" section, earlier in this chapter, describes the major email message types, such as application and text. However, if email had to rely on just those types, transmitting coherent messages to anyone would be difficult. The problem is that the type of information isn't explicit enough. If you send someone a text message, you need to know what sort of text it is before you can process it, and guessing just isn't a good idea. A text message could be formatted as plain text, or it might actually be an HTML page. You wouldn't know from just seeing the type, so messages require a subtype. The type is text and the subtype is html when you send an HTML page to someone. The type and subtype are separated by a forward slash, so you'd see text/html if you looked at the message.

REMEMBER

Theoretically, the number of subtypes is unlimited as long as the platform has a handler defined for that subtype. However, the reality is that everyone needs to agree on the subtypes or there won't be a handler (unless you're talking about a custom application for which the two parties have agreed to a custom subtype in advance). With this in mind, you can find a listing of standard types and subtypes at `https://www.freeformatter.com/mime-types-list.html`. The nice thing about the table on this site is that it provides you with a common file extension associated with the subtype and a reference to obtain additional information about it.

Putting everything together for text messages

Text messages represent the most efficient and least resource-intensive method of sending communication. However, text messages also convey the least amount of information. Yes, you can use emoticons to help get the point across, but the lack of formatting can become a problem in some situations. Here is what this chapter has covered, put into a single code block that you can test using the SMTP server created earlier at the Anaconda Prompt using the `python -m smtpd -n -c DebuggingServer localhost:1025` command:

```
from email.mime.text import MIMEText
import smtplib

msg = MIMEText("Hello There")
msg['Subject'] = "A Test Message"
msg['From']='SenderAddress'
msg['To'] = 'RecipientAddress'

s = smtplib.SMTP('localhost', 1025)

s.sendmail('SenderAddress',
           ['RecipientAddress'],
           msg.as_string())

print("Message Sent!")
```

If you run this code, you see the same output as that shown in Figure 17-2 at the Anaconda Prompt.

CONSIDERING THE SMTP SERVER

If you tried the example in this chapter without modifying it, you're probably scratching your head right now trying to figure out what went wrong. It's unlikely that your system has an SMTP server connected to localhost. The reason for the examples to use localhost is to provide a placeholder that you replace later with the information for your particular setup.

In order to see the example actually work, you need an SMTP server as well as a real-world email account. Of course, you could install all the software required to create such an environment on your own system, and some developers who work extensively with email applications do just that. Most platforms come with an email package that you can install, or you can use a freely available substitute such as Sendmail, an open source product available for download at `https://www.proofpoint.com/us/products/email-protection/open-source-email-solution`.

If you choose to use the SMTP server that your email application uses, you need to discover the SMTP server name and port. When working with a local application, such as Outlook, it's pretty easy to discover this information. For web-based email, locating it is usually a little harder, but some applications provide a special settings screen that contains the information you need. A problem occurs when your email application uses encryption to protect your messages. You need to log into the server and use the same encryption that the SMTP server does. The article at `https://zetcode.com/python/smtplib/` provides some insights into how to make this work, especially if SMTP server uses the STARTTLS encryption technique. If you're using Gmail, you can find a more specific coding example at `https://www.courier.com/blog/three-ways-to-send-emails-using-python-with-code-tutorials/`.

No matter what sort of SMTP server you eventually find, you need to have an account on that server in most cases to use the functionality it provides. Replace the information in the examples with the information for your SMTP server, such as smtp.myisp.com, along with your email address for both sender and receiver. Otherwise, the example won't work.

Working with an HTML message

An HTML message is basically a text message with special formatting. The following steps help you create an HTML email to send off.

1. **Type the following code into a new cell in the notebook — pressing Enter after each line.**

```
from email.mime.text import MIMEText
import smtplib
```

```
msg = MIMEText(
   "<h1>A Heading</h1><p>Hello There!</p>","html")
msg['Subject'] = 'A Test HTML Message'
msg['From']='SenderAddress'
msg['To'] = 'RecipientAddress'

s = smtplib.SMTP('localhost', 1025)
s.sendmail('SenderAddress',
           ['RecipientAddress'],
           msg.as_string())

print("Message Sent!")
```

The example follows the same flow as the text message example in the previous section. However, notice that the message now contains HTML tags. You create an HTML body, not an entire page. This message will have an H1 header and a paragraph.

REMEMBER

The most important part of this example is the text that comes after the message. The `"html"` argument changes the subtype from `text/plain` to `text/html`, so the recipient knows to treat the message as HTML content. If you don't make this change, the recipient won't see the HTML output.

2. **Click Run Cell.**

The application tells you that it has sent the message to the recipient. You can see the output at the Anaconda Prompt, and it should look similar to Figure 17-2, except for the HTML formatting.

5 The Part of Tens

IN THIS PART . . .

Finding the resources you need

Getting a job involving Python

Filling your programming toolbox

Locating just the right code library

IN THIS CHAPTER

- Getting better information about Python
- Creating online applications using Python
- Extending the Python programming environment
- Improving both application and developer performance

Chapter 18

Ten Amazing Programming Resources

This book is a great start to your Python programming experience, but you'll want additional resources at some point. This chapter provides you with ten amazing programming resources that you can use to make your development experience better. By using these resources, you save both time and energy in creating your next dazzling Python application.

REMEMBER

Of course, this chapter is only the beginning of your Python resource experience. Reams of Python documentation are out there, along with mountains of Python code. One might be able to write an entire book (or two) devoted solely to the Python libraries. This chapter is designed to provide you with ideas of where to look for additional information that's targeted toward meeting your specific needs. Don't let this be the end of your search — consider this chapter the start of your search instead.

Working with the Python Documentation Online

An essential part of working with Python is knowing what is available in the base language and how to extend it to perform other tasks. The Python documentation at `https://docs.python.org/3/` (created for the 3.*x* version of the product) contains a lot more than just the reference to the language that you receive as part of a download. (This is a multiversion site, so you can select the version of Python you want to read about in the top-left corner of the page.) You see these topics discussed as part of the documentation:

- New features in the current version of the language
- Access to a full-fledged tutorial
- Complete library reference
- Complete language reference
- How to install and configure Python
- How to perform specific tasks in Python
- Help with installing Python modules from other sources (as a means of extending Python)
- Help with distributing Python modules you create so that others can use them
- How to extend Python using C/C++ and then embed the new features you create
- Complete reference for C/C++ developers who want to extend their applications using Python
- Frequently Asked Questions (FAQ) pages

REMEMBER

All this information is provided in a form that is easy to access and use. In addition to the usual table-of-contents approach to finding information, you have access to a number of indexes. For example, if you aren't interested in anything but locating a particular module, class, or method, you can use the Global Module Index.

The `https://docs.python.org/3/` web page is also the place where you report problems with Python (the specific URL is `https://docs.python.org/3/bugs.html`). It's important to work through problems you're having with the product, but as with any other language, Python does have bugs in it. Locating and destroying the bugs will only make Python a better language.

Discovering Details Using a Tutorial

You can find various Python tutorials online, and it's usually a good idea to try more than one of them because each tutorial has something unique to offer. In addition, each tutorial has a unique method of presentation that may work better with your learning skills. Here are some tutorials you can use as starting points:

- **Learn Python:** `https://www.learnpython.org/`
- **Learn Python Programming:** `https://www.programiz.com/python-programming`
- **learnpython.org:** `https://www.learnpython.org/`
- **Real Python Tutorials:** `https://realpython.com/`
- **The Python Tutorial:** `https://docs.python.org/3/tutorial/`
- **Tutorials Point Python Tutorial:** `https://www.tutorialspoint.com/python/index.htm`
- **W3Schools Python Tutorial:** `https://www.w3schools.com/python/`

These tutorials present topics like:

- **Code introspection:** Provides the ability to examine classes, functions, and keywords to determine their purpose and capabilities.
- **Database access:** Most applications require some type of database interactivity to access large datasets.
- **Data science:** There are special techniques you can employ in Python to perform data science tasks, and some tutorials excel in this regard.
- **Decorator:** A method for making simple modifications to callable objects.
- **Exception handling:** An extension of the methods described in Chapter 10.
- **Generators:** Specialized functions that return iterators.
- **HR interview questions:** No one wants to go into an interview unprepared, so finding a tutorial to help you is a good idea.
- **List comprehensions:** A method to generate new lists based on existing lists.
- **Multiple function arguments:** An extension of the methods described in the "Using methods with variable argument lists" in Chapter 15.

- **Partial functions:** A technique for creating specialized versions of simple functions that derive from more complex functions. For example, if you have a `multiply()` function that requires two arguments, a partial function named `double()` may require only one argument that it always multiplies by 2.
- **Regular expressions:** Wildcard setups used to match patterns of characters, such as telephone numbers.
- **Serialization:** Shows how to use a data storage methodology called JavaScript Object Notation (JSON).
- **Sets:** Demonstrates a special kind of list that never contains duplicate entries.

Performing Web Programming by Using Python

Many developers specialize in creating online applications of various sorts using Python. The Web Programming in Python site at `https://wiki.python.org/moin/WebProgramming` helps you make the move from the desktop to online application development. It doesn't just cover one sort of online application — it covers almost all of them (an entire book free for the asking). The tutorials are divided into these three major (and many minor) areas:

- Server
  - Developing server-side frameworks for applications
  - Creating a Common Gateway Interface (CGI) script
  - Providing server applications
  - Developing Content Management Systems (CMS)
  - Designing data access methods through web services solutions
- Client
  - Interacting with browsers and browser-based technologies
  - Creating browser-based clients
  - Accessing data through various methodologies, including web services
- Related
  - Creating common solutions for Python-based online computing
  - Interacting with DataBase Management Systems (DBMSs)

- Designing application templates
- Building Intranet solutions

Locating Useful (versus Useless) Modules

If you're interested in useless modules, those that apparently do nothing at all (or, at least, nothing worthwhile), you're in the wrong spot. The UsefulModules site at `https://wiki.python.org/moin/UsefulModules` provides you with all sorts of useful modules for your Python installation, grouped by category. Of special note is that graphics are categorized in a detailed manner for the following uses:

- **Game development:** Creating game elements, including those exciting graphics everyone enjoys
- **GUI:** Designing the user interface elements for an application
- **ID3 handling:** Providing support for the ID3 multimedia packaging metadata normally associated with MP3, but used in other ways as well
- **Image manipulation:** Modifying the appearance of graphics in various ways, rather than provide a particular presentation
- **Plotting:** Displaying graphs and charts onscreen, rather than taking over the government (sorry)
- **Presentation:** Preparing graphical information for public display, such as the creation of slides
- **Resource Description Framework (RDF):** Defining the interconnections between data on the web, often in graphical format

Creating Applications Faster by Using an IDE

An Interactive Development Environment (IDE) helps you create applications in a specific language. The Integrated DeveLopment Environment (IDLE) editor (just type **idle** and press Enter at the Anaconda Prompt to see it) that comes with Python

works well for experimentation, but you may find it limited after a while. For example, IDLE doesn't provide the advanced debugging functionality that many developers favor. In addition, you may find that you want to create graphical applications, which is difficult using IDLE.

The problem is that you can talk to 50 developers and get little consensus as to the best tool for any job, especially when discussing IDEs. Every developer has a favorite product and isn't easily swayed to try another. Developers invest many hours learning a particular IDE and extending it to meet specific requirements (when the IDE allows such tampering).

REMEMBER

An inability (at times) to change IDEs later is why it's important to try a number of different IDEs before you settle on one. (The most common reason for not wanting to change an IDE after you select one is that the project types are incompatible, which would mean having to re-create your projects every time you change editors, but there are many other reasons that you can find listed online.) The PythonEditors wiki at `https://wiki.python.org/moin/PythonEditors` provides an extensive list of IDEs that you can try. The table provides you with particulars about each editor so that you can eliminate some of the choices immediately.

Checking Your Syntax with Greater Ease

Colab and Jupyter Notebook both provide syntax highlighting to help you see keywords, variables, and other elements. These IDEs also provide some advanced error checking to point out potential errors. However, for some developers, both these IDEs can come up short because you actually have to run the cell in order to see the error information. Some developers prefer interactive syntax checking, in which the IDE flags the error immediately, even before the developer leaves the errant line of code.

The python.vim utility (`https://www.vim.org/scripts/script.php?script_id=790`) provides enhanced syntax highlighting that makes finding errors in your Python script even easier. This utility runs as a script, which makes it fast and efficient to use on any platform. In addition, you can tweak the source code as needed to meet particular needs.

Using XML to Your Advantage

The eXtensible Markup Language (XML) is used for data storage of all types in most applications of any substance today. You probably have a number of XML files on your system and don't even know it because XML data appears under a number of file extensions. For example, many `.config` files, used to hold application settings, rely on XML. In short, it's not a matter of if you'll encounter XML when writing Python applications, but when.

XML has a number of advantages over other means of storing data. For example, it's platform independent. You can use XML on any system, and the same file is readable on any other system as long as that system knows the file format. The platform independence of XML is why it appears with so many other technologies, such as web services. In addition, XML is relatively easy to learn and because it's usually text, you can usually fix problems with it without too many problems.

REMEMBER

It's important to learn about XML itself, and you can do so using an easy tutorial such as the one found on the W3Schools site at `https://www.w3schools.com/xml/default.ASP`. Some developers rush ahead and later find that they can't understand the Python-specific materials that assume they already know how to write basic XML files. The W3Schools site is nice because it breaks up the learning process into chapters so that you can work with XML a little at a time, as follows:

- Taking a basic XML tutorial
- Validating your XML files
- Using XML with JavaScript (which may not seem important, but JavaScript is prominent in many online application scenarios)
- Gaining an overview of XML-related technologies
- Using advanced XML techniques
- Working with XML examples that make seeing XML in action easier

After you get the fundamentals down, you need a resource that shows how to use XML with Python. One of the better places to find this information is the Tutorials on XML Processing with Python site at `https://wiki.python.org/moin/Tutorials%20on%20XML%20processing%20with%20Python`. Between these two resources, you can quickly build a knowledge of XML that will have you building Python applications that use XML in no time.

Getting Past the Common Python Newbie Errors

Absolutely everyone makes coding mistakes — even that snobby fellow down the hall who has been programming for the last 30 years (he started in kindergarten). No one likes to make mistakes and some people don't like to own up to them, but everyone does make them. So you shouldn't feel too bad when you make a mistake. Simply fix it up and get on with your life.

REMEMBER

Of course, there is a difference between making a mistake and making an avoidable, common mistake. Yes, even the professionals sometimes make the avoidable common mistakes, but it's far less likely because they have seen the mistake in the past and have trained themselves to avoid it. You can gain an advantage over your competition by avoiding the newbie mistakes that everyone has to learn about sometime. To avoid these mistakes, check out the "Common beginner mistakes in Python" article at `https://deepsource.io/blog/python-common-mistakes/`.

Understanding Unicode

Although this book tries to sidestep the thorny topic of Unicode, you'll eventually encounter it when you start writing serious applications. Unfortunately, Unicode is one of those topics that had a committee deciding what Unicode would look like, so we ended up with more than one poorly explained definition of Unicode and a multitude of standards to define it. In short, there is no one definition for Unicode.

You'll encounter a wealth of Unicode standards when you start working with more advanced Python applications, especially when you start working with multiple human languages (each of which seems to favor its own flavor of Unicode). Keeping in mind the need to discover just what Unicode is, here are some resources you should check out:

- The Updated Guide to Unicode on Python (`https://lucumr.pocoo.org/2013/7/2/the-updated-guide-to-unicode/`)
- Python Encodings and Unicode (`https://eric.themoritzfamily.com/python-encodings-and-unicode.html`)
- Unicode Tutorials and Overviews (`http://www.unicode.org/standard/tutorial-info.html`)

- unicode_literals in Python (https://www.geeksforgeeks.org/unicode_literals-in-python/)
- Pragmatic Unicode (https://nedbatchelder.com/text/unipain.html)

Making Your Python Application Fast

Nothing turns off a user faster than an application that performs poorly. When an application performs poorly, you can count on users not using it at all. In fact, poor performance is a significant source of application failure in enterprise environments. An organization can spend a ton of money to build an impressive application that does everything, but no one uses it because it runs too slowly or has other serious performance problems.

Numerous resources are available to help you understand performance as it applies to Python applications. However, one of the best resources out there is "A guide to analyzing Python performance," at http://pythonic.zoomquiet.top/data/20170602154836/index.html. The author takes the time to explain why something is a performance bottleneck, rather than simply tell you that it is. After you read this article, make sure to check out the "PythonSpeed Performance Tips" at https://wiki.python.org/moin/PythonSpeed/PerformanceTips as well.

IN THIS CHAPTER

- Using Python for QA
- Creating opportunities within an organization
- Demonstrating programming techniques
- Performing specialized tasks

Chapter 19

Ten Ways to Make a Living with Python

You can literally write any application you want using any language you desire given enough time, patience, and effort. However, some undertakings would be so convoluted and time consuming as to make the effort a study in frustration. In short, most (possibly all) things are possible, but not everything is worth the effort. Using the right tool for the job is always a plus in a world that views time as something in short supply and not to be squandered.

Python excels at certain kinds of tasks, which means that it also lends itself to certain types of programming. Many people take this statement to mean that you'll be stuck in a data science job for the rest of your life (which is why data scientist isn't included in this chapter), but Python can do much more. For example, you may become a web developer, as described at `https://www.projectpro.io/article/python-for-data-science-vs-python-for-web-development/177`.

The kind of programming you can perform determines the job you get and the way in which you make your living. For example, Python probably isn't a very good choice for writing device drivers, as C/C++ are, so you probably won't find yourself working for a hardware company. Likewise, Python can work with databases, but not at the same depth that comes natively to other languages such as Structured Query Language (SQL), so you won't find yourself working on a huge corporate database project. However, you may find yourself using Python in academic

settings because Python does make a great learning language. (See my blog post on the topic at http://blog.johnmuellerbooks.com/2014/07/14/python-as-a-learning-tool/.)

The following sections describe some of the occupations that do use Python regularly so that you know what sorts of things you might do with your new-found knowledge. Of course, a single source can't list every kind of job. Consider this an overview of some of the more common uses for Python.

Working in QA

A lot of organizations have separate Quality Assurance (QA) departments that check applications to ensure that they work as advertised. Many different test script languages are on the market, but Python makes an excellent language in this regard because it's so incredibly flexible. In addition, you can use this single language in multiple environments — both on the client and on the server. The broad reach of Python means that you can learn a single language and use it for testing anywhere you need to test something, and in any environment.

REMEMBER

In this scenario, the developer usually knows another language, such as C++, and uses Python to test the applications written in C++. However, the QA person doesn't need to know another language in all cases. In some situations, blind testing may be used to confirm that an application behaves in a practical manner or as a means for checking the functionality of an external service provider. You need to check with the organization you want to work with as to the qualifications required for a job from a language perspective.

WHY YOU NEED TO KNOW MULTIPLE PROGRAMMING LANGUAGES

Most organizations see knowledge of multiple programming languages as a big plus (some see it as a requirement). Of course, when you're an employer, it's nice to get the best deal you can when hiring a new employee. Knowing a broader range of languages means that you can work in more positions and offer greater value to an organization. Rewriting applications in another language is time consuming, error prone, and expensive, so most companies look for people who can support an application in the existing language, rather than rebuild it from scratch.

From your perspective, knowing more languages means that you'll get more interesting jobs and will be less likely to get bored doing the same old thing every day. In addition, knowing multiple languages tends to reduce frustration. Most large applications today rely on components written in a number of computer languages. To understand the application and how it functions better, you need to know every language used to construct it.

Knowing multiple languages also makes it possible to learn new languages faster. After a while, you start to see patterns in how computer languages are put together, so you spend less time with the basics and can move right on to advanced topics. The faster you can learn new technologies, the greater your opportunities to work in exciting areas of computer science. In short, knowing more languages opens a lot of doors.

Becoming the IT Staff for a Smaller Organization

A smaller organization may have only one or two IT staff, which means that you have to perform a broad range of tasks quickly and efficiently. With Python, you can write utilities and in-house applications quite swiftly. Even though Python might not answer the needs of a large organization because it's interpreted (and potentially open to theft or fiddling by unskilled employees), using it in a smaller organization makes sense because you have greater access control and need to make changes fast. In addition, the ability to use Python in a significant number of environments reduces the need to use anything but Python to meet your needs. (There is actually a compiled form of Python called *Cython*; see `https://www.infoworld.com/article/3250299/what-is-cython-python-at-the-speed-of-c.html` for details).

TIP

Some developers are unaware that Python is available in some non-obvious products. For example, even though you can't use Python scripting with Internet Information Server (IIS) right out of the box, you can add Python scripting support to this product by using the steps found in the Microsoft Knowledge Base article at `https://docs.microsoft.com/visualstudio/python/configure-web-apps-for-iis-windows`. If you aren't sure whether a particular application can use Python for scripting, make sure that you check it out online.

You can also get Python support in some products that you might think couldn't possibly support it. For example, you can use Python with Visual Studio (see `https://visualstudio.microsoft.com/vs/features/python/`) to make use of Microsoft technologies with this language. The site at `https://code.visualstudio.com/docs/languages/python` provides some additional details about Python support.

Performing Specialty Scripting for Applications

A number of products can use Python for scripting purposes. For example, Maya (`https://www.autodesk.com/products/maya/overview`) relies on Python for scripting purposes. (The video series at `https://zurbrigg.com/tutorials/beginning-python-for-maya` provides you with additional details.) By knowing which high-end products support Python, you can find a job working with that application in any business that uses it. Here are some examples of products that rely on Python for scripting needs:

- 3ds Max
- Abaqus
- Blender
- Cinema 4D
- GIMP
- Google App Engine
- Houdini
- Inkscape
- Lightwave
- Modo
- MotionBuilder
- Nuke
- Paint Shop Pro
- Scribus
- Softimage

This is just the tip of the iceberg. You can also use Python with the GNU debugger to create more understandable output of complex structures, such as those found in C++ containers. Some video games also rely on Python as a scripting language. In short, you could build a career around creating application scripts using Python as the programming language.

Administering a Network

More than a few administrators use Python to perform tasks such as monitoring network health or creating utilities that automate tasks. Administrators are often short of time, so anything they can do to automate tasks is a plus. In fact, some network management software, such as Trigger (`https://trigger.readthedocs.io/en/latest/`), is actually written in Python. A lot of these tools are open source and free to download, so you can try them on your network. Also, some interesting articles discuss using Python for network administration, such as "Network Automation Using Python Programming," at `https://www.sevenmentor.com/network-automation-using-python-programming`. It's also possible to find courses on the topic, such as the one at `https://www.globalknowledge.com/us-en/course/182899/python-for-network-automation/`. The point is that knowing how to use Python on your network can ultimately decrease your workload and help you perform your tasks more easily. If you want to see some scripts that are written with network management in mind, check out 25 projects tagged "Network Management" at `http://freshmeat.sourceforge.net/tags/network-management`.

Teaching Programming Skills

Many teachers are looking for a faster, more consistent method of teaching computer technology. Raspberry Pi (`https://www.raspberrypi.org/`) is a single-board computer that makes obtaining the required equipment a lot less expensive for schools. The smallish device plugs into a television or computer monitor to provide full computing capabilities with an incredibly simple setup. Interestingly enough, Python plays a big role in making Raspberry Pi into a teaching platform for programming skills (`https://projects.raspberrypi.org/en/projects?software%5B%5D=python` and `https://realpython.com/python-raspberry-pi/`).

TIP

In reality, teachers often use Python to extend native Raspberry Pi capabilities so that it can perform all sorts of interesting tasks (`https://www.raspberrypi.org/blog/tag/python/`). The project entitled, Boris, the Twitter Dino-Bot (`https://www.raspberrypi.com/news/boris-the-twitter-dino-bot/`), is especially interesting. (There is a YouTube video about it at `https://www.youtube.com/watch?v=Q5Kc9Aqf_Zk`.) The point is that if you have a teaching goal in mind, combining Raspberry Pi with Python is a fantastic idea.

Helping People Decide on Location

A Geographic Information System (GIS) provides a means of viewing geographic information with business needs in mind. For example, you could use GIS to determine the best place to put a new business or to determine the optimum routes for shipping goods. However, GIS is used for more than simply deciding on locations — it also provides a means for communicating location information better than maps, reports, and other graphics, and a method of presenting physical locations to others. Also interesting is the fact that many GIS products use Python as their language of choice. In fact, a wealth of Python-specific information related to GIS is currently available, such as

- 15 Python Libraries for GIS and Mapping (`https://gisgeography.com/python-libraries-gis-mapping/`)
- Python and GIS Resources (`https://www.gislounge.com/python-and-gis-resources/`)
- GIS Programming and Software Development (`https://www.e-education.psu.edu/geog485/node/17`)

Many GIS-specific products, such as ArcGIS (`https://www.esri.com/arcgis/`), rely on Python to automate tasks. Entire communities develop around these software offerings, such as Python for ArcGIS (`https://resources.arcgis.com/en/communities/python/`). The point is that you can use your new programming skills in areas other than computing to earn an income.

Performing Data Mining

Everyone is collecting data about everyone and everything else. Trying to sift through the mountains of data collected is an impossible task without a lot of fine-tuned automation. The flexible nature of Python, combined with its terse language that makes changes extremely fast, makes it a favorite with people who perform data mining on a daily basis. In fact, you can find an online book on the topic, *A Programmer's Guide to Data Mining,* at `http://guidetodatamining.com/`. Python makes data mining tasks a lot easier. The purpose of data mining is to recognize trends, which means looking for patterns of various sorts. The use of artificial intelligence with Python makes such pattern recognition possible. A paper on the topic, "Practical Data Mining with Python" (`https://dzone.com/refcardz/data-mining-discovering-and`), helps you understand how such analysis is possible. You can use Python to create just the right tool to locate a pattern that could net sales missed by your competitor.

REMEMBER

Of course, data mining is used for more than generating sales. For example, people use data mining to perform tasks such as locating new planets around stars or other types of analysis that increase our knowledge of the universe. Python figures into this sort of data mining as well. You can likely find books and other resources dedicated to any kind of data mining that you want to perform, with many of them mentioning Python as the language of choice.

Interacting with Embedded Systems

An embedded system exists for nearly every purpose on the planet. For example, if you own a programmable thermostat for your house, you're interacting with an embedded system. Raspberry Pi (mentioned earlier in the chapter) is an example of a more complex embedded system. Many embedded systems rely on Python as their programming language. In fact, a special form of Python, Embedded Python (`https://wiki.python.org/moin/EmbeddedPython`), is sometimes used for these devices. You can even find a YouTube presentation on using Python to build an embedded system at `https://www.youtube.com/watch?v=WZoeqnsY9AY`.

TIP

Interestingly enough, you might already be interacting with a Python-driven embedded system. For example, Python is the language of choice for many car security systems (`https://www.pythoncarsecurity.com/`). The remote start feature that you might have relies on Python to get the job done. Your home automation and security system (`https://www.activestate.com/blog/how-to-automate-your-home-with-python/`) might also rely on Python.

Python is so popular for embedded systems because it doesn't require compilation. An embedded-system vendor can create an update for any embedded system and simply upload the Python file. The interpreter automatically uses this file without having to upload any new executables or jump through any of the types of hoops that other languages can require.

Carrying Out Scientific Tasks

Python seems to devote more time to scientific and numerical processing tasks than many of the computer languages out there. The number of Python's scientific and numeric processing packages is staggering (`https://wiki.python.org/moin/NumericAndScientific`). Scientists love Python because it's small, easy to learn, and yet quite precise in its treatment of data. You can produce results by

using just a few lines of code. Yes, you could produce the same result using another language, but the other language might not include the prebuilt packages to perform the task, and it would most definitely require more lines of code even if it did.

REMEMBER

The two sciences that have dedicated Python packages are space sciences and life sciences. For example, there is actually a package for performing tasks related to solar physics. You can also find a package for working in genomic biology. If you're in a scientific field, the chances are good that your Python knowledge will significantly impact your ability to produce results quickly while your colleagues are still trying to figure out how to analyze the data.

Performing Real-Time Analysis of Data

Making decisions requires timely, reliable, and accurate data. Often, this data must come from a wide variety of sources, which then require a certain amount of analysis before becoming useful. A number of the people who report using Python do so in a management capacity. They use Python to probe those disparate sources of information, perform the required analysis, and then present the big picture to the manager who has asked for the information. Given that this task occurs regularly, trying to do it manually every time would be time consuming. In fact, it would simply be a waste of time. By the time the manager performed the required work, the need to make a decision might already have passed. Python makes it possible to perform tasks quickly enough for a decision to have maximum impact.

Previous sections have pointed out Python's data-mining, number-crunching, and graphics capabilities. A manager can combine all these qualities while using a language that isn't nearly as complex to learn as C++. In addition, any changes are easy to make, and the manager doesn't need to worry about learning programming skills such as compiling the application. A few changes to a line of code in an interpreted package usually serve to complete the task.

REMEMBER

As with other sorts of occupational leads in this chapter, thinking outside the box is important when getting a job. A lot of people need real-time analysis. Launching a rocket into space, controlling product flow, ensuring that packages get delivered on time, and all sorts of other occupations rely on timely, reliable, and accurate data. You might be able to create your own new job simply by employing Python to perform real-time data analysis.

IN THIS CHAPTER

- Debugging, testing, and deploying applications
- Documenting and versioning your application
- Writing your application code
- Working within an interactive environment

Chapter 20

Ten Tools That Enhance Your Python Experience

Python, like most other programming languages, has strong third-party support in the form of various tools. A *tool* is any utility that enhances the natural capabilities of Python when building an application. So, a debugger is considered a tool because it's a utility, but a library isn't. Libraries are instead used to create better applications. (You can see some of them listed in Chapter 21.)

Even making the distinction between a tool and something that isn't a tool, such as a library, doesn't reduce the list by much. Python enjoys access to a wealth of general-purpose and special tools of all sorts. In fact, the site at `https://wiki.python.org/moin/DevelopmentTools` breaks these tools down into the following 13 categories:

- AutomatedRefactoringTools
- BugTracking
- ConfigurationAndBuildTools
- DistributionUtilities
- DocumentationTools
- IntegratedDevelopmentEnvironments

- PythonDebuggers
- PythonEditors
- PythonShells
- SkeletonBuilderTools
- TestSoftware
- UsefulModules
- VersionControl

Interestingly enough, the lists on the Python Development Tools site might not even be complete. You can find Python tools listed in quite a few places online. One of the most interesting new tools on the market now is the dashboard, a special kind of application that makes it possible to create an interactive version of your code for nonprogrammers, as described at `https://blog.esciencecenter.nl/forget-about-jupyter-notebooks-showcase-your-research-using-dashboards-5d13451ba374`.

Given that a single chapter can't possibly cover all the tools out there, this chapter discusses a few of the more interesting tools — those that merit a little extra attention on your part. After you whet your appetite with this chapter, seeing what other sorts of tools you can find online is a good idea. You may find that the tool you thought you might have to create is already available, and in several different forms.

WARNING

All the tools in this chapter do work with Jupyter Notebook because you have control of the environment on your local system. They may not work with Google Colab because you don't have control of Google's virtual servers. The only way to ensure that a product will work with Colab using your particular browser on your particular system is to test it.

Tracking Bugs with Roundup Issue Tracker

You can use a number of bug-tracking sites with Python, such as the following: GitHub (`https://github.com/`); Google Code (`https://code.google.com/`); BitBucket (`https://bitbucket.org/`); and Launchpad (`https://launchpad.net/`). However, these public sites are generally not as convenient to use as your own specific, localized bug-tracking software. You can use a number of tracking

systems on your local drive, but Roundup Issue Tracker (`https://roundup.sourceforge.io/`) is one of the better offerings. Roundup should work on any platform that supports Python, and it offers these basic features without any extra work:

- Bug tracking
- TODO list management

If you're willing to put a little more work into the installation, you can get additional features, and these additional features are what make the product special. However, to get them, you may need to install other products, such as a DataBase Management System (DBMS). The product instructions tell you what to install and which third-party products are compatible. After you make the additional installations, you get these upgraded features:

- Customer help-desk support with the following features:
  - Wizard for the phone answerers
  - Network links
  - System and development issue trackers
- Issue management for Internet Engineering Task Force (IETF) working groups
- Sales lead tracking
- Conference paper submission
- Double-blind referee management
- Blogging (extremely basic right now, but will become a stronger offering later)

Creating a Virtual Environment by Using VirtualEnv

Reasons abound to create virtual environments, but the main reason to do so with Python is to provide a safe and known testing environment. By using the same testing environment each time, you help ensure that the application has a stable environment until you have completed enough of it to test in a production-like environment. VirtualEnv (`https://pypi.python.org/pypi/virtualenv`) provides the means to create a virtual Python environment that you can use for the

early testing process or to diagnose issues that could occur because of the environment. It's important to remember that there are at least three standard levels of testing that you need to perform:

- **Bug:** Checking for errors in your application
- **Performance:** Validating that your application meets speed, reliability, and security requirements
- **Usability:** Verifying that your application meets user needs and will react to user input in the way the user expects

REMEMBER

Because of the manner in which most Python applications are used (see Chapter 19 for some ideas), you generally don't need to run them in a virtual environment after the application has gone to a production site. Most Python applications require access to the outside world, and the isolation of a virtual environment would prevent that access.

NEVER TEST ON A PRODUCTION SERVER

A mistake that some developers make is to test their unreleased application on the production server where the user can easily get to it. Of the many reasons not to test your application on a production server, data loss has to be the most important. If you allow users to gain access to an unreleased version of your application that contains bugs that might corrupt the database or other data sources, the data could be lost or damaged permanently.

You also need to realize that you get only one chance to make a first impression. Many software projects fail because users don't use the end result. The application is complete, but no one uses it because of the perception that the application is flawed in some way. Users have only one goal in mind: to complete their tasks and then go home. When users see that an application is costing them time, they tend not to use it.

Unreleased applications can also have security holes that nefarious individuals will use to gain access to your network. It doesn't matter how well your security software works if you leave the door open for anyone to come in. After they have come in, getting rid of them is nearly impossible, and even if you do get rid of them, the damage to your data is already done. Recovery from security breaches is notoriously difficult — and sometimes impossible. In short, never test on your production server because the costs of doing so are simply too high.

Installing Your Application by Using PyInstaller

Users don't want to spend a lot of time installing your application, no matter how much it might help them in the end. Even if you can get the user to attempt an installation, less skilled users are likely to fail. In short, you need a surefire method of getting an application from your system to the user's system. Installers, such as PyInstaller (`https://pyinstaller.org/en/stable/`), do just that. They make a nice package out of your application that the user can easily install.

Fortunately, PyInstaller works on all the platforms that Python supports, so you need just the one tool to meet every installation need you have. In addition, you can get platform-specific support when needed. For example, when working on a Windows platform, you can create code-signed executables. Mac developers will appreciate that PyInstaller provides support for bundles. In many cases, avoiding the platform-specific features is best unless you really do need them. When you use a platform-specific feature, the installation will succeed only on the target platform.

WARNING

A number of the installer tools that you find online are platform specific. For example, when you look at an installer that reportedly creates executables, you need to be careful that the executables aren't platform specific (or at least match the platform you want to use). It's important to get a product that will work everywhere it's needed so that you don't create an installation package that the user can't use. Having a language that works everywhere doesn't help when the installation package actually hinders installation.

AVOID THE ORPHANED PRODUCT

Some Python tools floating around the Internet are *orphaned,* which means that the developer is no longer actively supporting them. Developers still use the tool because they like the features it supports or how it works. However, doing so is always risky because you can't be sure that the tool will work with the latest version of Python. The best way to approach tools is to get tools that are fully supported by the vendor who created them.

If you absolutely must use an orphaned tool (such as when an orphaned tool is the only one available to perform the task), make sure that the tool still has good community support. The vendor may not be around any longer, but at least the community will provide a source of information when you need product support. Otherwise, you'll waste a lot of time trying to use an unsupported product that you might never get to work properly.

Building Developer Documentation by Using pdoc

Two kinds of documentation are associated with applications: user and developer. User documentation shows how to use the application, while developer documentation shows how the application works. A library requires only one sort of documentation, developer, while a desktop application may require only user documentation. A service might actually require both kinds of documentation depending on who uses it and how the service is put together. The majority of your documentation is likely to affect developers, and pdoc (`https://github.com/mitmproxy/pdoc`) is a simple solution for creating it.

The pdoc utility relies on the documentation that you place in your code in the form of docstrings and comments. The output is in the form of a text file or an HTML document. You can also have pdoc run in a way that provides output through a web server so that people can see the documentation directly in a browser.

WHAT IS A DOCSTRING?

This chapter talks about document strings (docstrings). A *docstring* is a special kind of comment that appears within a triple quote, like this:

```
"""This is a docstring."""
```

You associate a docstring with an object, such as packages, functions, classes, and methods. Any code object you can create in Python can have a docstring. The purpose of a docstring is to document the object. Consequently, you want to use descriptive sentences.

The easiest way to see a docstring is to follow the object's name with the special `__doc__` method. For example, typing **print(MyClass.__doc__)** would display the docstring for `MyClass`. You can also access a docstring by using help, such as `help(MyClass)`. Good docstrings tell what the object does, rather than how it does it.

Third-party utilities can also make use of docstrings. Given the right utility, you can write the documentation for an entire library without actually having to write anything. The utility uses the docstrings within your library to create the documentation. Consequently, even though docstrings and comments are used for different purposes, they're equally important in your Python code.

Developing Application Code by Using Komodo Edit

Several chapters in this book discuss the issue of Interactive Development Environments (IDEs), but none make a specific recommendation (except for the use of Google Colab and Jupyter Notebook throughout the book). The IDE you choose depends partly on your needs as a developer, your skill level, and the kinds of applications you want to create. Some IDEs are better than others when it comes to certain kinds of application development. One of the better general-purpose IDEs for novice developers is Komodo IDE (`https://www.activestate.com/products/komodo-ide/`). You can obtain this IDE for free, and it includes a wealth of features that will make your coding experience much better than what you'll get from IDLE (included with Python). Here are some of those features:

- Support for multiple programming languages
- Automatic completion of keywords
- Indentation checking
- Project support so that applications are partially coded before you even begin
- Superior support

Debugging Your Application by Using pydbgr

A high-end IDE, such as Komodo IDE, comes with a complete debugger. However, if you're using something smaller, less expensive, and less capable than a high-end IDE, you might not have a debugger at all. A *debugger* helps you locate errors in your application and fix them. The better your debugger, the less effort required to locate and fix the error. When your editor doesn't include a debugger, you need an external debugger such as pydbgr (`https://github.com/rocky/python2-trepan` and `https://pypi.org/project/trepan3k/`), which is currently being updated.

REMEMBER

A reasonably good debugger includes a number of standard features, such as code colorization (the use of color to indicate things like keywords). However, pydbgr also includes a number of nonstandard features that set it apart. Here are some of the standard and nonstandard features that make pydbgr a good choice when your editor doesn't come with a debugger:

- **Smarteval:** The `eval` command helps you see what will happen when you execute a line of code, before you actually execute it in the application. It helps you perform "what if" analysis to see what is going wrong with the application.
- **Out-of-process debugging:** Normally you have to debug applications that reside on the same machine. In fact, the debugger is part of the application's process, which means that the debugger can actually interfere with the debugging process. Using out-of-process debugging means that the debugger doesn't affect the application and you don't even have to run the application on the same machine as the debugger.
- **Thorough byte-code inspection:** Viewing how the code you write is turned into *byte code* (the code that the Python interpreter actually understands) can sometimes help you solve tough problems.
- **Event filtering and tracing:** As your application runs in the debugger, it generates events that help the debugger understand what is going on. For example, moving to the next line of code generates an event, returning from a function call generates another event, and so on. This feature makes it possible to control just how the debugger traces through an application and which events it reacts to.

Entering an Interactive Environment by Using IPython

The Python shell works fine for many interactive tasks. However, if you have used this product, you may have already noted that the default shell has certain deficiencies. Of course, the biggest deficiency is that the Python shell is a pure text environment in which you must type commands to perform any given task. A more advanced shell, such as IPython (`http://ipython.org/`), can make the interactive environment friendlier by providing GUI features so that you don't have to remember the syntax for odd commands.

REMEMBER

IPython is actually more than just a simple shell. It provides an environment in which you can interact with Python in new ways, such as by displaying graphics that show the result of formulas you create using Python. In addition, IPython is designed as a kind of front end that can accommodate other languages. The

IPython application actually sends commands to the real shell in the background, so you can use shells from other languages such as Julia and Haskell. (Don't worry if you've never heard of these languages.)

One of the more exciting features of IPython is the ability to work in parallel computing environments. Normally a shell is single threaded, which means that you can't perform any sort of parallel computing. In fact, you can't even create a multithreaded environment. This feature alone makes IPython worthy of a trial.

Testing Python Applications by Using PyUnit

At some point, you need to test your applications to ensure that they work as instructed. You can test them by entering in one command at a time and verifying the result, or you can automate the process. Obviously, the automated approach is better because you really do want to get home for dinner someday and manual testing is really, really slow (especially when you make mistakes, which are guaranteed to happen). Products such as PyUnit (`https://wiki.python.org/moin/PyUnit`) make unit testing (the testing of individual features) significantly easier.

The nice part of this product is that you actually create Python code to perform the testing. Your script is simply another, specialized, application that tests the main application for problems.

REMEMBER

You may be thinking that the scripts, rather than your professionally written application, could be bug ridden. The testing script is designed to be extremely simple, which will keep scripting errors small and quite noticeable. Of course, errors can (and sometimes do) happen, so yes, when you can't find a problem with your application, you do need to check the script.

Tidying Your Code by Using Isort

It may seem like an incredibly small thing, but code can get messy, especially if you don't place all your `import` statements at the top of the file in alphabetical order. In some situations, it becomes difficult, if not impossible, to figure out what's going on with your code when it isn't kept neat. The Isort utility (`https://pycqa.github.io/isort/`) performs the seemingly small task of sorting your

`import` statements and ensuring that they all appear at the top of the source code file. This small step can have a significant effect on your ability to understand and modify the source code.

Just knowing which modules a particular module needs can be a help in locating potential problems. For example, if you somehow get an older version of a needed module on your system, knowing which modules the application needs can make the process of finding that module easier.

In addition, knowing which modules an application needs is important when it comes time to distribute your application to users. Knowing that the user has the correct modules available helps ensure that the application will run as anticipated.

Providing Version Control by Using Mercurial

The applications you created while working through this book aren't very complex. In fact, after you finish this book and move on to more advanced training applications, you're unlikely to need version control. However, after you start working in an organizational development environment in which you create real applications that users need to have available at all times, version control becomes essential. *Version control* is simply the act of keeping track of the changes that occur in an application between application releases to the production environment. When you say you're using MyApp 1.2, you're referring to version 1.2 of the MyApp application. Versioning lets everyone know which application release is being used when bug fixes and other kinds of support take place.

Numerous version control products are available for Python. One of the more interesting offerings is Mercurial (`https://www.mercurial-scm.org/`). You can get a version of Mercurial for almost any platform that Python will run on, so you don't have to worry about changing products when you change platforms. (If your platform doesn't offer a binary, executable release, you can always build one from the source code provided on the download site.)

Unlike a lot of the other offerings out there, Mercurial is free. Even if you find that you need a more advanced product later, you can gain useful experience by working with Mercurial on a project or two.

REMEMBER

The act of storing each version of an application in a separate place so that changes can be undone or redone as needed is called *source code management* or SCM. For many people, source code management seems like a hard task. Because the Mercurial environment is quite forgiving, you can learn about SCM in a friendly environment. Being able to interact with any version of the source code for a particular application is essential when you need to go back and fix problems created by a new release.

The best part about Mercurial is that it provides a great online tutorial at `https://www.mercurial-scm.org/wiki/Tutorial`. Following along on your own machine is the best way to learn about SCM, but even just reading the material is helpful. Of course, the first tutorial is all about getting a good installation of Mercurial. The tutorials then lead you through the process of creating a repository (a place where application versions are stored) and using the repository as you create your application code. By the time you finish the tutorials, you should have a great idea of how source control should work and why versioning is an important part of application development.

IN THIS CHAPTER

- Securing your data by using cryptography
- Managing and displaying data
- Working with graphics
- Finding the information you need
- Allowing access to Java code

Chapter 21

Ten (Plus) Libraries You Need to Know About

Python provides you with considerable power when it comes to creating average applications. However, most applications aren't average and require some sort of special processing to make them work. That's where libraries come into play. A good library will extend Python functionality so that it supports the special programming needs that you have. For example, you might need to plot statistics or interact with a scientific device. These sorts of tasks require the use of one or more libraries.

TIP

One of the best places to find a library listing online is the UsefulModules site at `https://wiki.python.org/moin/UsefulModules`. Of course, you have many other places to look for libraries as well. For example, the article entitled "Top 10 Python Libraries You Must Know In 2022" (`https://www.edureka.co/blog/python-libraries/`) provides you with a relatively complete description of the ten libraries its title refers to. If you're working on a specific platform, such as Windows, you can find platform-specific sites, such as "Unofficial Windows Binaries for Python Extension Packages" (`https://www.lfd.uci.edu/~gohlke/pythonlibs/`). The point is that you can find lists of libraries everywhere.

The purpose of this chapter isn't to add to your already overflowing list of potential library candidates. Instead, it provides you with a list of ten libraries that work on every platform and provide basic services that just about everyone will need. Think of this chapter as a source for a core group of libraries to use for your next coding adventure.

Developing a Secure Environment by Using CryptLib

Data security is an essential part of any programming effort. The reason that applications are so valued is that they make it easy to manipulate and use data of all sorts. However, the application must protect the data or the efforts to work with it are lost. It's the data that is ultimately the valuable part of a business — the application is simply a tool. Part of protecting the data is to ensure that no one can steal it or use it in a manner that the originator didn't intend, which is where cryptographic libraries such as CryptLib (`https://www.cs.auckland.ac.nz/~pgut001/cryptlib/`) come into play. One of the best parts of this particular library is that it includes bindings for:

- C/C++
- C# and other .NET languages
- Delphi
- Java
- Python
- Visual Basic (VB)

REMEMBER

The main purpose of this library is to turn your data into something that others can't read while it sits in permanent storage. The purposeful modification of data in this manner is called *encryption.* However, when you read the data into memory, a *decryption* routine takes the mangled data and turns it back into its original form so that the application can manage it. At the center of all this is the *key,* which is used to encrypt and decrypt the data.

Interacting with Databases by Using SQLAlchemy

A *database* is essentially an organized manner of storing repetitive or structured data on disk. For example, customer *records* (individual entries in the database) are repetitive because each customer has the same sort of information requirements, such as name, address, and telephone number. The precise organization of the data (such as tabular or hierarchical) determines the sort of database you need. Some database products specialize in text organization, others in tabular information, and still others in random bits of data (such as readings taken from a scientific instrument). Databases can use a tree-like structure or a flat-file configuration to store data. You'll hear all sorts of odd terms when you start looking into DataBase Management System (DBMS) technology — most of which mean something only to a DataBase Administrator (DBA) and won't matter to you.

REMEMBER

The most common type of database is called a Relational DataBase Management System (RDBMS), which uses tables that are organized into records and fields (just like a table you might draw on a sheet of paper). Each *field* is part of a column of the same kind of information, such as the customer's name. Tables are related to each other in various ways, so creating complex relationships is possible. For example, each customer may have one or more entries in a purchase order table, and the customer table and the purchase order table are therefore related to each other.

An RDBMS relies on a special language called the Structured Query Language (SQL) to access the individual records inside. Of course, you need some means of interacting with both the RDBMS and SQL, which is where SQLAlchemy (`https://www.sqlalchemy.org/`) comes into play. This product reduces the amount of work needed to ask the database to perform tasks such as returning a specific customer record, creating a new customer record, updating an existing customer record, and deleting an old customer record.

Seeing the World by Using Google Maps

Geocoding (the finding of geographic coordinates, such as longitude and latitude from geographic data, such as address) has lots of uses in the world today. People use the information to do everything from finding a good restaurant to locating a lost hiker in the mountains. Getting from one place to another often revolves around geocoding today as well. Google Maps (`https://pypi.org/project/googlemaps/`) lets you add directional data to your applications.

In addition to getting from one point to another or finding a lost soul in the desert, Google Maps can also help in Geographic Information System (GIS) applications. The "Helping People Decide on Location" section of Chapter 19 describes this particular technology in more detail, but essentially, GIS is all about deciding on a location for something or determining why one location works better than another location for a particular task. In short, Google Maps presents your application with a look at the outside world that it can use to help your user make decisions.

Adding a Graphical User Interface by Using TkInter

Users respond to the Graphical User Interface (GUI) because it's friendlier and requires less thought than using a command-line interface. Many products out there can give your Python application a GUI. However, the most commonly used product is TkInter (`https://wiki.python.org/moin/TkInter`). Developers like it so much because TkInter keeps things simple. It's actually an interface for the Tool Command Language (Tcl)/Toolkit (Tk) found at `http://www.tcl.tk/`. A number of languages use Tcl/Tk as the basis for creating a GUI.

TIP

You might not relish the idea of adding a GUI to your application. Doing so tends to be time consuming and doesn't make the application any more functional (it also slows the application down in many cases). The point is that users like GUIs, and if you want your application to see strong use, you need to meet user requirements.

WORKING WITH DASHBOARDS

Developers are always coming up with new ways to present information. Although you can use your Notebooks for any presentation and allow traditional data manipulation using a GUI like TkInter, the resulting Notebooks don't provide an interactive experience for nonprogrammers who simply want to work with outputs. This is where a dashboard can come in handy. You export your Notebook as a `.py` file after adding dashboard functionality to it using standard Python programming methods. To run the dashboard, you call on the dashboard server from the command prompt or use something like a batch file and provide the `.py` file as input. The user will see any plots included in your Notebook with the addition of controls, such as sliders, that allow them to interact with

the plot and play *what-if* scenarios that allow the user to see what happens when the data to the plot changes. You can find a number of dashboards online, including these:

- Dash (`https://plotly.com/dash/`)
- Panel (`https://panel.holoviz.org/`)
- Streamlit (`https://streamlit.io/`)
- voila (`https://voila.readthedocs.io/en/stable/`)

Providing a Nice Tabular Data Presentation by Using PrettyTable

Displaying tabular data in a manner the user can understand is important. From the examples you've seen throughout the book, you know that Python stores this type of data in a form that works best for programming needs. However, users need something that is organized in a manner that humans understand and that is visually appealing. The PrettyTable library (`https://pypi.org/project/prettytable/`) makes it easy to add an appealing tabular presentation to your command-line application.

Enhancing Your Application with Sound by Using PyAudio

Sound is a useful way to convey certain types of information to the user. Of course, you have to be careful in using sound because special-needs users might not be able to hear it, and for those who can, using too much sound can interfere with normal business operations. However, sometimes audio is an important means of communicating supplementary information to users who can interact with it (or of simply adding a bit of pizzazz to make your application more interesting).

One of the better platform-independent libraries to make sound work with your Python application is PyAudio (`http://people.csail.mit.edu/hubert/pyaudio/`). This library makes it possible to record and play back sounds as needed (such as a user recording an audio note of tasks to perform later and then playing back the list of items as needed).

CLASSIFYING PYTHON SOUND TECHNOLOGIES

Realize that sound comes in many forms in computers. The basic multimedia services provided by Python (see the documentation at `https://docs.python.org/3/library/mm.html`) provide essential playback functionality. You can also write certain types of audio files, but the selection of file formats is limited. In addition, some packages, such as winsound (`https://docs.python.org/3/library/winsound.html`), are platform dependent, so you can't use them in an application designed to work everywhere. The standard Python offerings are designed to provide basic multimedia support for playing back system sounds.

The middle ground, augmented audio functionality designed to improve application usability, is covered by libraries such as PyAudio. You can see a list of these libraries at `https://wiki.python.org/moin/Audio`. However, these libraries usually focus on business needs, such as recording notes and playing them back later. Hi-fidelity output isn't part of the plan for these libraries.

Gamers need special audio support to ensure that they can hear special effects, such as a monster walking behind them. These needs are addressed by libraries such as PyGame (`http://www.pygame.org/news.html`). When using these libraries, you need higher-end equipment and have to plan to spend considerable time working on just the audio features of your application. You can see a list of these libraries at `https://wiki.python.org/moin/PythonGameLibraries`.

TIP

Working with sound on a computer always involves trade-offs. For example, a platform-independent library can't take advantage of special features that a particular platform might possess. In addition, it might not support all the file formats that a particular platform uses. The reason to use a platform-independent library is to ensure that your application provides basic sound support on all systems that it might interact with.

Manipulating Images by Using PyQtGraph

Humans are visually oriented. If you show someone a table of information and then show the same information as a graph, the graph is always the winner when it comes to conveying information. Graphs help people see trends and understand why the data has taken the course that it has. However, getting those pixels that

represent the tabular information onscreen is difficult, which is why you need a library such as PyQtGraph (`https://www.pyqtgraph.org/`) to make things simpler.

Even though the library is designed around engineering, mathematical, and scientific requirements, you have no reason to avoid using it for other purposes. PyQtGraph supports both 2D and 3D displays, and you can use it to generate new graphics based on numeric input. The output is completely interactive, so a user can select image areas for enhancement or other sorts of manipulation. In addition, the library comes with a wealth of useful widgets (controls, such as buttons, that you can display onscreen) to make the coding process even easier. Make sure you review the Requirements section of the PyQtGraph website for add-ins needed to use this product.

Locating Your Information by Using Whoosh

Finding your information can be difficult when the information grows to a certain size. Consider your hard drive as a large, free-form, tree-based database that lacks a useful index. Any time such a structure becomes large enough, data simply gets lost. (Just try to find those pictures you took last summer and you'll get the idea.) As a result, having some type of search capability built into your application is important so that users can find that lost file or other information. Note that a search library isn't a search engine; it provides you with the means for building your own search engine.

WARNING

A number of search libraries are available for Python. The problem with most of them is that they are hard to install or don't provide consistent platform support. In fact, some of them work on only one or two platforms. However, Whoosh (`https://whoosh.readthedocs.io/en/latest/intro.html`) is written in pure Python, which ensures that it works on every platform. If you find that Whoosh doesn't meet your needs, make sure the product you do get will provide the required search functionality on all the platforms you select and that the installation requirements are within reason.

Creating an Interoperable Java Environment by Using JPype

Python does provide access to a huge array of libraries, and you're really unlikely to use them all. However, you might be in a situation in which you find a Java library that is a perfect fit but can't use it from your Python application unless you're willing to jump through a whole bunch of hoops. The JPype library (`http://jpype.sourceforge.net/`) makes it possible to access most (but not all) of the Java libraries out there directly from Python. The library works by creating a bridge between the two languages at the byte-code level. Consequently, you don't have to do anything weird to get your Python application to work with Java.

WARNING

CONVERTING YOUR PYTHON APPLICATION TO JAVA

Many different ways exist to achieve interoperability between two languages. Creating a bridge between them, as JPype does, is one way. Another alternative is to convert the code created for one language into code for the other language. This is the approach used by Jython (`https://wiki.python.org/jython/`). This utility converts your Python code into Java code so that you can make full use of Java functionality in your application while maintaining the features that you like about Python.

You'll encounter trade-offs in language interoperability no matter which solution you use. In the case of JPype, you won't have access to some Java libraries. In addition, there is a speed penalty in using this approach because the JPype bridge is constantly converting calls and data. The problem with Jython is that you lose the ability to modify your code after conversion. Any changes that you make will create an incompatibility between the original Python code and its Java counterpart. In short, no perfect solutions exist for the problem of getting the best features of two languages into one application.

Accessing Local Network Resources by Using Twisted Matrix

Depending on your network setup, you may need access to files and other resources that you can't reach using the platform's native capabilities. In this case, you need a library that makes such access possible, such as Twisted Matrix (`https://twisted.org/`). The basic idea behind this library is to provide you with the calls needed to establish a connection, no matter what sort of protocol is in use.

The feature that makes this library so useful is its event-driven nature. This means that your application need not get hung up while waiting for the network to respond. In addition, the use of an event-driven setup makes asynchronous communication (in which a request is sent by one routine and then handled by a completely separate routine) easy to implement.

Accessing Internet Resources by Using Libraries

Although products such as Twisted Matrix can handle online communication, getting a dedicated HTTP protocol library is often a better option when working with the Internet because a dedicated library is both faster and more feature complete. When you specifically need HTTP or HTTPS support, using a library such as `httplib2` (`https://github.com/httplib2`) is a good idea. This library is written in pure Python and makes handling HTTP-specific needs, such as setting a Keep-Alive value, relatively easy. (A *Keep-Alive* is a value that determines how long a port stays open waiting for a response so that the application doesn't have to continuously re-create the connection, wasting resources and time as a result.)

You can use `httplib2` for any Internet-specific methodology — it provides full support for both the `GET` and `POST` request methods. This library also includes routines for standard Internet compression methods, such as `deflate` and `gzip`. It also supports a level of automation. For example, `httplib2` adds ETags back into `PUT` requests when resources are already cached.

Index

(hashtag), 61
% (percent sign), 80
%% (double percent sign), 80
(#) number sign, 70
/ (forward slash), 305
: (colon), 129
[] (square brackets), 243
\ (backslash), 219, 305
{} (curly brackets), 232–233
= (assignment operator), 97

A

Abaqus, 352
about this book, 1–2
absolute path, 305
accented (special character category), 219
__add__( ) method, 292
Add New Form Field dialog box, 65
administrators, 353
%alias_magic line magic command, 86
%alias line magic command, 86
align formatting specification, 234
Amazon, Echo, 9
American Standard Code for Information Interchange (ASCII), 216–217
Anaconda Prompt, 205
Android tablet, 307
append( ) function, 245, 276, 317
appendleft( ) function, 276
Apple, Siri, 9
application (app)
  adding comments, 70–73
  adding documentation cells, 61–62
  browser-based, 17
  converting to Java, 376
  creating, 60–66
  CRUD acronym to describe what it does, 41
  defined, 11, 13–15
  developing code, 60–61
  as a form of communication, 11–13
  interacting with form fields, 64–66
  making applications fast, 347
  managing data within, 108
  organizing information in, 240–241
  other cell content, 62–63
  as procedure, 13
  questions to ask when creating, 15
  running, 66–68
  scratch cells in, 63–64
  seeing result of, 66
  testing class inheritance in, 298–299, 365
  using if statement in, 129–133
  using indentation in, 68–70
  using the if...elif statement in, 135–138
  using the if...else statement in, 134
  using while statement in, 152–154
  viewing executed code history, 68
  world's worst, 15
  writing your first, 57–77
ArcGIS, 354
arguments
  creating functions with variable number of, 122–123
  defined, 120
  giving function arguments a default value, 121–122
  positional, 121
  sending by name, 121
  sending required arguments, 120–121
ArithmeticError exception, 176–177
arithmetic operators, 112
assignment operator (=), 97
assignment operators, 114–115
assumptions, in this book, 2–3
attributes
  automatically generated by Python, 210
  built-in class, 285–286
  defined, 191
audio library, 373–374
%autocall line magic command, 86
%automagic line magic command, 86
automatic indentation, 69
%autosave line magic command, 86

B

backslash (\), 219, 305
Base 2, 98
Base 8, 98
Base 10, 98
Base 16 (hex), 98
base classes, 164
bin( ) function, 99
BitBucket, 358
bitwise operators, 112, 114
black-swan events, 159
Blender, 352
blocks, data stored as, 304
body (email), 323
bold type, 74
%bookmark line magic command, 86
Boole, George, 102
Boolean algebra, 102
Boolean values, 102
bool type, 102
Boris, the Twitter Dino-Bot, 353
branches, 31
break clause, 146
breaking change, 161
break statement, 146–147
bugs, 40, 59, 158
bug-tracking sites, 358–359
built-in class attributes, 281–282
built-in exceptions, 164–165
built-in packages, 194–195
__builtins__ attribute, 210
bulleted list, adding, 75
byte code, 364

C

C# programming language, 17, 20
__cached__ attribute, 210
caller, 117, 181
capitalize( ) function, 223
car security systems, 355
catching exceptions, 157, 163–180
C/C++, 17
%cd line magic command, 86
cell magic commands, 83, 90–91
cells
  editing, 38
  moving, 39
  running all, 40
  running current, 39
  running other, 39–40
center( ) function, 226
center(width, fillchar=" ") function, 223
characters
  accented, 219
  creating strings with, 217–218
  drawing, 219
  numeric values, 216–217
  selecting, 219
  special, 218–221
  typographical, 219
chat rooms, 9
Cheat sheet, 4
checkpoints, 77
child class, 297–298
Cinema 4D, 352
Clarke, Arthur C. (science writer)
classes
  built-in class attributes, 281–282
  child class, 297–298
  creating and using, 279–299
  creating class definition, 280–281
  creating class methods, 283–284
  defined, 280
  extending classes to make new classes, 296–299
  external, 293–296
  overloading operators, 291–293
  parent class, 296–297
  production-grade class, 296–297
  saving to disk, 294–295
  testing class inheritance in application, 298–299
  tests of in application, 298–299
  working with constructors, 285–286
  working with methods, 282–284
  working with variables, 287–289
class suite, 281
class variables, 287–288
clause, 133. *See also specific clauses*
clear( ) function, 245, 248
%clear line magic command, 86

cloud, 304
%cls line magic command, 86
code
  creating better code, 58–59
  dead, 149
  developing, 60–61
  formatting text as, 74
  groupings, 190–196
  introspection, 341
  reusability, 117–118
  sorting, 365–366
  spaghetti code, 279
  using comments to keep from executing, 73
  viewing executed code history, 68
code block, 130–131
code cells
  adding comments to, 36
  adding forms to, 37
  clearing output, 37
  copying, 37
  creating, 35–37
  deleting, 36
  mirroring in tabs, 36
  moving up/down, 36
  selecting, 37
  viewing output in full-screen mode, 37
code groupings, creating, 190–196
code snippets, searching for, 40
coding mistakes, 346
Colaboratory (Colab)
  about, 23–24
  comparing files, 28–29
  configuring settings, 26–27
  creating code cells, 35–37
  creating text cells, 37–38
  customizing keyboard shortcuts, 28
  defined, 24
  executing the code, 39–40
  getting help, 40
  local file storage, 312
  locating commands, 25
  performing common tasks, 35–39
  understanding what it does, 24–25
  working with features, 25–29
Colab Pro, 27
collections, 257–259
colon (:), 129
%colors line magic command, 86
command line, 58, 82, 205
commands. *See also specific commands*
  defined, 43
  seeing results of, 44
  telling computer what to do, 43
  telling computer when you're done, 43
  typing, 42–44
comments
  adding, 70–73
  commenting out, 73
  vs. headings, 70
  multiline, 71–72
  single-line, 71
  understanding, 70
communication, application as a form of, 11
companion files, 4
comparisons, 107–108
compile time error, 160
complex numbers, 101–102
computer
  communicating with your, 10–11
  helping humans speak to, 14–15
  making computers do funny things, 13
  talking to, 9
  telling it when you're done, 43
  tell it what to do, 43
  understanding how computers make comparisons, 109
  understanding how computers read instructions, 13
  understanding how computers view lists, 241
  as using special language, 14
concatenation, 69–70, 222
concrete exceptions, 164
Conda, 58
conda clean command, 196
conda config command, 196
conda create command, 196
conda help command, 196
conda info command, 196, 207
conda install command, 196
%conda line magic command, 87

conda list command, 196
conda packages
  installing, 206
  overview, 195
  removing, 208
  updating, 207–208
  viewing, 206
  working with, 205–208
conda remove command, 196
conda search command, 196
conda update command, 196
%config line magic command, 86
%connect_info line magic command, 86
continue clause, 148–149
control, 218–219
conventions, used in this book, 2
copy( ) function, 245
copyright( ) command, 46
count( ) function, 228, 258
count(str, beg= 0, end=len(string)) function, 227
credits( ) command, 46
CRUD acronym, 41, 93, 245, 303–304
CryptLib, 370
curly brackets ({}), 232–233
current directory, 202, 306
custom exceptions, 182–183
custom packages, 195
Cython, 351

D

Dash, 373
dashboards, 372–373
data
  controlling how Python views data, 108–109
  flushing, 311
  a.ka. information, 107
  integrity, 304
  processing by using for statement, 144–151
  processing by using while statement, 152–154
  real-time data analysis, 356
  storing, 96–97
  storing as objects, 304
  storing of in files, 303–319
  structured, 306
  unstructured, 306
DataBase Management System (DBMS), 359, 371
databases
  interacting with, 18
  interacting with by using SQLAlchemy, 371
  tutorials, 341
data mining, 354–355
data science, 341
data types
  Boolean values, 102
  complex numbers, 101–102
  defined, 96–97
  floating-point values, 99–101
  integers, 98–99
  numeric types, 97–102
  strings, 103–104
  variables, 97
DataWriter object, 310
dates, working with, 104–105
dead code, 149
%%debug function, 83–84
debugger, 83, 158, 363–364
debugging, 59, 364
%debug line magic command, 87
%%debug line magic command, 87
decision making
  about, 127
  nested decision statements, 138–141
  using the if statement in, 127–133
decorators, 341
decryption, 370
default directories, 202
deflate, 377
denominator( ) function, 56
deques, 258, 275–277
development tool, 19
%dhist line magic command, 87
dictionaries
  as collections, 258
  creating and using, 263–266
  nested, 266–268
  replacing switch statement with, 268–271
  working with, 262–271
dir( ) command, 243
directories
  current, 202
  current directory, 306

default, 202
files as organized into, 305
interacting with current Python directory, 193–194
parent directory, 306
parent of the parent directory, 306
root directory, 306
dir( ) function, 52–53, 210
%dirs line magic command, 87
__doc__ attribute, 210
docstring, 71, 362
%doctest_mode line magic command, 87
documentation, 340
documentation cells, adding, 61–62
double-ended queue, 259
double percent sign (%%), 80
double underlines (__), 53
drawing characters, 219

E

Echo (Amazon), 9
Editor (Colab), 27
else clause, 133, 150–151
elif clausem, 135, 249
email
components of, 323–324
defining message, 329–332
defining parts of envelope, 324–326
defining parts of letter, 329–334
host, 325–326
local hostname, 325, 328
message, 328–329
message subtypes, 333–334
port, 325–328
receiver, 328
sender, 328
sending, 321–336
Simple Mail Transfer Protocol, 322–323
specifying transmission, 331–333
text messages, 334
viewing email as a letter, 323–324
working with HTML messages, 335–336
Embedded Python, 355
embedded systems, 355
emoji, inserting, 75
empty( ) function, 274–275
encryption, 370
endswith(suffix, beg=0, end=len(string)) function, 227
envelope, 324
environment variables, 202
%env line magic command, 87
error handling, 157
errors
classifying when errors occur, 160–162
considering sources of, 159–163
dealing, 157–186
distinguishing error types, 162–164
logical error, 163–164
of omission, 162
passing error information to caller, 181–182
runtime error, 161–162
semantic error, 163
syntactical error, 162–163
escape sequences, 219–220
ETags, 377
eval command, 364
event filtering and tracing, 364
except clause
handling more specific to less specific exceptions, 176–178
handling multiple exceptions with multiple except clauses, 175–176
using except clause without an exception, 167–170
exception arguments, 169–173
exception handling
basic, 165
defined, 157
more specific to less specific exceptions, 176–177
multiple, 173–176
nested, 178–180
single, 165–167
tutorials, 341
exceptions
base classes, 164
built-in, 164–165
catching, 157, 163–180
concrete, 164
custom, 182–183
defined, 157
handling more specific to less specific exceptions, 176–178

exceptions *(continued)*
handling multiple exceptions with multiple except clauses, 175–176
handling multiple exceptions with single except clause, 173–174
handling single exception, 165–167
nested exception handling, 177–180
raising or throwing, 157, 180–182
using except clause without an exception, 167–170
using finally clause, 184–186
using the if...else statement in, 132
working with exception arguments, 170–171
execution
controlling with break statement, 146–147
controlling with continue statement, 148–149
expandtabs(tabsize=8) function, 223
exponent, 100
expressions, returning from functions, 124
extend( ) function, 245, 276
extendleft( ) function, 276
eXtensible Markup Language (XML), 18, 307, 345
extensions, 305
external class
creating and using, 293–296
developing, 293–294
saving class to disk, 294–295

F

feedback, sending, 40
Fiber Channel over Ethernet (FCoE), 304
fields, 309, 371
__file__ attribute, 210
%file line magic command, 87
files
creating, 309–312
deleting, 319
extensions on, 305
reading file content, 313–314
storing, 304–306
storing data in, 303–319
updating file content, 315–318
fill formatting specification, 233
finally clause, 184–186
find(str, beg=0, end=len(string)) function, 227
Firefox browser, 26
float( ) command, 126
floating-point type, 234
floating-point values, 99–101
folders, 305
for loop, creating, 145–146
FormatData class, 310
format flags, 230
format( ) function, 232–236
format method, 236–237
form fields
choosing types of, 65
defined, 64
defining variable type and name, 65–66
using, 66
forms, adding, 37
for statement, processing data using, 144–151
forward slash (/), 305
frequently asked questions (FAQs), 40
from...import statement, 197–200
f-string, 236–237
full( ) function, 274–275
functions. *See also specific functions*
accessing, 119
code reusability, 117–118
comparing output, 124–125
creating and using, 117–124
defining, 118–119
returning information from, 123–124
returning results from other, 124
sending information to, 119–123
with variable number of agreements, 122–123
viewing as code packages, 117

G

geocoding, 371
Geographic Information System (GIS), 354, 372
get( ) function, 274–275
getter methods, 308
GIMP, 352
gists
defined, 33
downloading, 34
locating, 34
public, 34
publicizing, 34

secret, 34
sharing, 34
GitHub, 25
bug-tracking using, 358
gists, 33–34
saving notebooks, 33
using for existing notebooks, 31
GitHub settings tab (Colab), 27
Google App Engine, 352
Google Code, 358
Google Colab, 2
Google Drive
saving notebooks, 32
using for existing notebooks, 31
working with, 34–35
Google Maps, 371–372
graphical user interface (GUI), 18
graphics processing unit (GPU), 39
gzip, 377

H

hardware acceleration, 39
hashtag (#), 61
Haskell, 22
header (email), 323
heading levels, 74
headings
vs. comments, 70
for special cells, 38
help( ) command, 46
Help menu (Colab), 40
help mode, 46–47
hexadecimal, 98
hex( ) function, 99
%hist line magic command, 87
%history line magic command, 87
home automation and security systems, 355
horizontal rule, adding, 75
host, 325–326
host address, 325
Houdini, 352
HTML messages, 335–336
httplib2, 377
HTTP protocol library, 377
humor, in programming, 13

I

icons, used in this book, 3
identity operators, 114–115
if...elif statement, 135–138
if...else statement
understanding, 133
using in application, 133–135
if statement
understanding, 128
using in application, 129–133
images, inserting, 75
images, manipulating, 374–375
import...as statement, 200–201
importing, 104, 189, 196–201
import statement, 197–198, 366
indent, 75
indentation, 68–70
index, 243
index( ) function, 258, 317
index(str, beg=0, end=len(string)) function, 227
information
a.k.a. data, 107
managing, 107–126
organizing in application, 240–241
returning from information from functions, 123–124
sending to functions, 119–123
storing, 96–97
inheritance, 296
__init__( ) constructor, 292
__init__( ) function, 183
__init__( ) method declaration, 285
initializing, 285
__initializing__ attribute, 210
Inkscape, 352
input, validating with else statement, 150–151
input( ) function, 64, 125
insert( ) function, 245–246
insertion pointer, 218–219
inspect module
defined, 55
importing, 55
using, 55–56
%install_ext line magic command, 87
installers, 361
instance methods, 284–285

instance variable, 288–289
instantiation, 280
integers, 98–99
integer type, 234
Integrated Development Environment (IDE), 23, 57–59
Integrated DeveLopment Environment (IDLE), 58, 343–344
Interactive Development Environment (IDE), 343–344, 363
Internet compression methods, 377
Internet Information Server (IIS), 351
Internet Small Computer Systems Interface (iSCSI), 304
introspection, 56
.ipnyb files, 24
IPython, 82, 364–365
IPython Notebook. *See* Jupyter Notebook
isalnum( ) function, 223
isalpha( ) function, 223
isdecimal( ) function, 223
isdigit( ) function, 224
islower( ) function, 224
isnumeric( ) function, 224
Isort utility, 365–366
isspace( ) function, 224
istitle( ) function, 224
isupper( ) function, 224
italics, 74
iterable items, 276
IT staff, 351

J

Java
  converting Python application to, 376
  library, 376
  vs. Python, 17, 20–21
Javascript, 17
jobs. *See* occupations
join(seq) function, 224
JPype, 376
Jupyter Notebook. *See also* notebooks
  about, 23–25
  downloading, 57
  local file storage, 312
  running applications, 67
Jython, 376

K

Keep- Alive value, 377
kernel
  changing, 77
  closing and halting, 77
  defined, 77
  reconnecting to, 77
  reverting to checkpoint, 77
  saving, 77
  shutting down, 77
key, 370
KeyboardInterrupt exception, 179
keyboard shortcuts, 28
keywords, 48
keyword search, 212–213
Knuth, Donald (computer scientist), 59
Komodo Edit, 58, 363

L

%ldir_ext line magic command, 87
len( ) function, 246
length checking, 148
len(str) function, 224
libraries
  accessing Internet resources by using, 377
  CryptLib, 370
  defined, 160, 189
  effects of updates on codes, 160–161
  Google Maps, 371–372
  httplib2, 377
  JPype, 376
  PrettyTable, 373
  PyAudio, 373–374
  PyQtGraph, 374–375
  SQLAlchemy, 371
  TkInter, 372
  Twisted Matrix, 377
  Useful Modules, 369
  Whoosh, 375
license( ) command, 46
life sciences, 356
Lightwave, 352
line magic commands, 82, 85–89. *See also specific line magic command*s
links, inserting, 74

LINQ (Language INtegrated Query), 18
lists
  accessing, 243–244
  comprehension, 341
  creating, 242–243
  creating stacks using, 271–273
  defining organization by using, 240–241
  looping through, 244–245
  managing, 239–255
  modifying, 245–248
  organizing information in applications, 240–241
  printing, 251–253
  searching, 249–250
  sorting, 250–251
  understanding how computers view, 241
  using methods with variable argument lists, 289–291
  using operators with, 247
  working with Counter object, 254–255
literate programming, 59–60
ljust(width, fillchar=" ") function, 224
%load_ext line magic command, 88
__loader__ attribute, 210
%load line magic command, 88
%loadpy line magic command, 88
local hostname, 325, 328
local network resources, accessing, 377
logical error, 163–164
logical operators, 113, 131–133
%logoff line magic command, 88
%logon line magic command, 88
%logstart line magic command, 88
%logstate line magic command, 88
%logstop line magic command, 88
loop statements, nesting, 154–156
lower( ) function, 224
%lsmagic line magic command, 88
lstrip( ) function, 224

M

magic commands
  adding magic to, 81
  cell, 83, 90–91
  defined, 80
  line, 82, 85–89
  overview, 79
  watching for environmental differences, 84
%magic line magic command, 88
mantissa, 100
Mathematica, 60
MATLAB, 60
MATLAB For Dummies (Mueller/Sizemore), 60
%matplotlib line magic command, 88
max(str) function, 224
Maya, 352
membership operators, 114–115
MemoryError exception, 164
Mercurial, 366–367
message, 323
methods
  creating class methods, 283–284
  creating instance methods, 284–285
  defined, 282
  using methods with variable argument lists, 289–291
  working with, 282–284
Microsoft Knowledge Base, 351
MIMEApplication, 330
MIMEAudio, 330
MIMEImage, 330
MIMEMultipart, 330
MIMEText, 330
min(str) function, 224
%mkdir line magic command, 88
Modo, 352
modules, 48, 190–191, 343
modulo (%) operator approach, 229–232
Motionbuilder, 352
Mueller, John Paul (author), 60
multiline comments, creating, 71–72
multiple comparisons, 131–133
multiple function arguments, 341
multiple programming languages, IT staff, 351
multiple tasks, performing, 130–131
Multipurpose Internet Mail Extensions (MIME), 330
MyClass class, 293–296
myNum class, 54

N

nested decision statements, 138–141
nested dictionaries, 266–268
nested exception handling, 177–180

nesting, 178
network administrators, 353
newbie errors, 346
newline attribute, 310
non-conda packages, 195
%notebook line magic command, 88
notebooks. *See also* Jupyter Notebook
  benefits of using, 59–60
  closing and halting, 77
  creating, 29–30
  opening, 30
  renaming, 76
  saving using GitHub, 33
  section headers, 76
  table of contents, 76
  text cell options for, 74–75
  vs. text editors, 59
  using GitHub for, 30
  using Google Drive for, 30
  using local storage for, 31–32
now( ) function, 104
Nuke, 352
numbered list, adding, 75
number sign (#), 70
numeric bases, 98
numeric data types, 97–102
NumPy, 17

O

object-oriented programming (OOP), 53
objects, data stored as, 304
occupations
  data mining, 354–355
  embedded systems, 355
  Geographic Information System, 354
  network administration, 353
  overview, 349–350
  quality assurance, 350
  real-time data analysis, 356
  scientific tasks, 355–356
  scripting, 352
  teaching programming skills, 353
oct( ) function, 99
open( ) function, 310
open( ) method, 314
operands, 110
operators. *See also specific operators*
  arithmetic, 112
  assignment, 114–115
  bitwise, 112–114
  categories, 110
  defined, 109
  identity, 114–115
  logical, 113, 131–133
  membership, 114–115
  overloading, 291–293
  precedence, 116
  relational, 112, 129–130
  ternary, 110–111
  unary, 111
  using lists with, 247
orphaned products, 361
os.chdir( ) method, 194
os.rmdir( ) function, 319
out-of-process debugging, 364

P

__package__ attribute, 210
packages
  built-in, 194–195
  cache, 195–196
  conda, 195, 205–208
  creating, 191–193
  creating code groupings, 190–196
  custom, 195
  defined, 160, 189
  downloading from other sources, 204–210
  effects of updates on codes, 160–161
  finding, 201–204
  importing, 104, 189, 196–201
  installing by using pip, 209
  installing using %pip magics, 209–210
  interacting with, 189–213
  locating on disk, 202–203
  locating online, 204
  vs. modules, 190–191
  non-conda, 195
  types, 194–195
  viewing content of, 210–211
  viewing documentation, 211–213

Paint Shop Pro, 352
Panel, 373
parent class, 296–297
parent directory, 306
parent of the parent directory, 306
partial functions, 342
pass clause, 149–150
%paste line magic command, 88
path, 305
%pdef line magic command, 88
%pdoc line magic command, 88
pdoc utility, 362
percent sign (%), 80
Perl (Practical Extraction and Report Language), 21
%pinfo2 line magic command, 88
%pinfo line magic command, 88
pip (Pip Installs Packages) utilities, 205, 209
%pip line magic command, 88
%popd line magic command, 87
pop( ) function, 245, 276
popleft( ) function, 276
port, 325–328
positional arguments, 121
%pprint line magic command, 88
.precision formatting specification, 234
%precision line magic command, 88
PrettyTable, 373
print( ) command, 49
print( ) method, 60
procedures
  defined, 11
  seeing application as, 13
  using, 11–12
  writing down, 12
production-grade class, 296–297
production servers, 360
Programmer's Guide to Data Mining, 354
programming languages, 14
programming resources
  eXtensible Markup Language, 345
  Interactive Development Environment, 343–344
  making applications fast, 347
  modules, 343
  newbie errors, 346
  online Python documentation, 340
  overview, 339
  syntax checking, 344
  tutorials, 340–341
  Unicode, 346–347
  web programming in Python, 342–343
protocol, 322
prototype, 17
%prun line magic command, 88
%psearch line magic command, 88
public gists, 34
%pushd line magic command, 87
%pwd line magic command, 87
PyAudio, 373–374
pydbgr, 363–364
PyDoc, 211–213
!pydoc, 211–213
PyGame, 374
PyInstaller, 361
%pylab line magic command, 88
PyQtGraph, 374–375
Python
  about, 1
  common uses of, 17–18
  comparing with other languages, 20–22
  controlling how data is viewed, 108–109
  data types, 97–104
  deciding how to personally benefit from, 17–18
  finding useful applications written in, 19
  as the first-ranked language on Statistics Times, 17
  as the first-ranked language on TIOBE, 1, 17
  help mode, 46–50
  interacting with, 41–56
  knowing why it doesn't understand you, 158–159
  obtaining help directly, 50–51
  online documentation, 340
  organizations using, 18–19
  reasons for using, 16–17
  topics help, 48–50
  typing commands, 42–44
  why it's so cool, 16–22
Python Development Tools, 357–358
Python for ArcGIS, 354
Python Package Index (PyPI), 205
python.vim utility, 344
Python with Visual Studio, 351
PyUnit, 365

Q

quality assurance (QA), 350
queues, 258, 273–275
%quickref function, 84–85
%quickref line magic command, 88

R

R (programming language), 21
raise call, 181
range checking, 131
range( ) function, 155
Raspberry Pi, 353, 355
README file, 42
real-time data analysis, 356
receiver (email), 328
recursion, 144
reflection, 56
regular expressions, 342
Relational DataBase Management System (RDBMS), 371
relational operators, 112–113, 129–130
relative path, 305
release notes, 40, 42
%reload_ext line magic command, 88
remove( ) function, 245, 276, 317
repetition, 222
repetitive tasks, performing, 142–156
replace(old, new [, max]) function, 227
repositories, 31
%reset_selective line magic command, 88
%reset line magic command, 88
revisions, 32
RFC1123 standard, 323
rfind(str, beg=0, end=len(string)) function, 227
rindex(str, beg=0, end=len(string)) function, 227
rjust(width, fillchar=“ ”) function, 224
%rmdir line magic command, 88
%rm line magic command, 88
root directory, 306
Roundup Issue Tracker, 358–359
rstrip( ) function, 225
Run Cell icon, 46–47
%run line magic command, 88
runtime error, 161–162

S

SaveData( ) method, 310
scientific tasks, 355–356
SciPy, 17–18
scratch cells, 63–64
Scribus, 352
scripting, 352
search libraries, 375
search term, 212–213
secret gists, 34
section headers, adding, 76
Secure Sockets Layer (SSL) encryption, 322
selection tree, 138
semantic error, 163
sender (email), 328
sequences, 240
serialization, 342
%set_env line magic command, 87
sets, 342
setter methods, 308
shutil.rmtree( ) function, 319
sign formatting specification, 234
significand, 100
Simple Mail Transfer Protocol (SMTP), 321–323, 335
single-line comments, 71
Siri (Apple), 9
Sizemore, Jim (author), 60
smartphone, coding on, 1–2
smtplib module, 332
socket, 325
socket.gethostbyaddr( ) method, 328
socket.gethostbyname( ) method, 328
Softimage, 352
sound library, 373–374
source code management (SCM), 367
%source line magic command, 88
space-delimited string, 292
spaces (chat rooms), 9
space sciences, 356
spaghetti code, 279
special cells, creating, 38
special characters, 218–221
split( ) function, 317
splitlines(num=string.count(‘\n’)) function, 225

split(str=" ", num=string.count(str)) function, 225
SQLAlchemy, 371
square brackets ([]), 222
Stack Overflow, 40
stacks
  as collections, 258
  creating, 271–273
startswith(prefix, beg=0, end=len(string)) function, 227
Statistics Times, 17
storage
  creating content for permanent storage, 306–309
  of data in files, 303–319
  local file storage, 312
  understanding how permanent storage works, 304–306
%store line magic command, 88
__str__( ) function, 308
Streamlit, 373
strerror attribute, 183
string interpolation, 229
  % (modulo) operator approach, 229–232
  working with format( ) function, 232–236
strings
  creating of with special character, 218–221
  defining character by using numbers, 216–217
  described, 216
  locating value in, 227–229
  selecting individual characters, 221–223
  slicing and dicing, 223–226
  space-delimited, 292
  understanding, 103
  using characters to create, 217–218
  working with, 212–213
string templates, creating and using, 237–238
string type, 234
strip( ) function, 225–226
structured data, 306
Structured Query Language (SQL), 18, 371
str.upper( ) function, 179
swapcase( ) function, 225
switch statement, 268–271
symbols, 48
syntactical error, 162–163
syntax checking, 344
sys.exit( ) call, 185
sys.path attribute, 202

T

table of contents, 38, 76
teachers, 353
tensor processing unit (TPU), 39
ternary operator, 110–111
text cells, 37–38, 74–75
text messages, 334
3ds Max, 352
time( ) function, 105
%timeit line magic command, 88
%time line magic command, 88
times, working with, 104–105
TIOBE, 1, 17
title( ) function, 225
TkInter, 372
Tool Command Language (Tcl)/Toolkit (Tk), 372
tools
  categories, 357–358
  defined, 357
  IPython, 364–365
  Isort utility, 365–366
  Komodo Edit, 363
  Mercurial, 366–367
  pdoc utility, 362
  pydbgr, 363–364
  PyInstaller, 361
  PyUnit, 365
  Roundup Issue Tracker, 358–359
  VirtualEnv, 359–360
topics, 48
Transport Layer Security (TLS), 322
Trigger, 353
tuples, 258–262
tutorials, 340–341
Tutorials on XML Processing with Python, 345
Twisted Matrix, 377
type formatting format, 234
typographical characters, 219

U

%unalias line magic command, 88
unary operators, 111
Unicode, 346–347
%unload_ext line magic command, 89

Unofficial Windows Binaries for Python Extension Packages, 369
unstructured data, 306
updates, 4
upper( ) function, 225
Useful Modules, 343, 369
user input, getting, 125–126
user interfaces, 18

V

ValueError exception, 182–184
values, returning from functions, 124
Van Rossum, Guido, 15, 53
variables
  class variables, 287–288
  defining type and name, 65–66
  determining type of, 102
  environment, 202
  instance variables, 288–289
  putting information into, 97
  returning from functions, 124
  as storage boxes, 96
  using methods with variable argument lists, 289–291
  working with, 287–289
version control, 366–367
VirtualEnv, 359–360
virtual environments, 359–360
Visual Basic, 17
Visual Studio, 351
voila, 373

W

W3Schools, 345
Web programming in Python, 342–343
while statement, 152–154
%who_ls line magic command, 89
%who line magic command, 89
Whoosh, 375
%whos line magic command, 89
width formatting specification, 234
winsound, 374

X

%xdel line magic command, 89
XML (eXtensible Markup Language), 18, 307, 345
%xmode line magic command, 89

Z

ZeroDivisionError exception, 176
zfill(width) function, 225
.zip file, 307

About the Author

John Mueller is a freelance author and technical editor. He has writing in his blood, having produced 122 books and more than 600 articles to date. The topics range from networking to artificial intelligence and from database management to heads-down programming. Some of his current books include discussions of data science, machine learning, and algorithms. He also writes about computer languages such as C++, C#, and Python. His technical editing skills have helped more than 70 authors refine the content of their manuscripts. John has provided technical editing services to a variety of magazines, performed various kinds of consulting, and he writes certification exams. Be sure to read John's blog at `http://blog.johnmuellerbooks.com/`. You can reach John on the Internet at `John@JohnMuellerBooks.com`. John also has a website at `http://www.johnmuellerbooks.com/`.

Dedication

This book is dedicated to Paige and Parker Groom, my inspiration for many great things and the joys of my life.

Acknowledgments

Thanks to my wife, Rebecca. Even though she is gone now, her spirit is in every book I write and in every word that appears on the page. She believed in me when no one else would.

Rod Stephens deserves thanks for his technical edit of this book. He greatly added to the accuracy and depth of the material you see here. Rod's critical thinking skills force me to truly think through all the book elements. He's also the sanity check for my work.

Matt Wagner, my agent, deserves credit for helping me get the contract in the first place and taking care of all the details that most authors don't really consider. I always appreciate his assistance. It's good to know that someone wants to help.

A number of people read all or part of this book to help me refine the approach, test the coding examples, and generally provide input that all readers wish they could have. These unpaid volunteers helped in ways too numerous to mention here. I especially appreciate the efforts of Eva Beattie, who provided general input, read the entire book, and selflessly devoted herself to this project.

Finally, I would like to thank Kelsey Baird, Susan Christophersen, and the rest of the editorial and production staff.

Publisher's Acknowledgments

Acquisitions Editor: Kelsey Baird

Project Manager and Copy Editor: Susan Christophersen

Technical Editor: Rod Stephens

Production Editor: Mohammed Zafar Ali

Cover Image: © antoniodiaz/Shutterstock

Take dummies with you everywhere you go!

Whether you are excited about e-books, want more from the web, must have your mobile apps, or are swept up in social media, dummies makes everything easier.

Find us online!

dummies.com

dummies
A Wiley Brand

Leverage the power

Dummies is the global leader in the reference category and one of the most trusted and highly regarded brands in the world. No longer just focused on books, customers now have access to the dummies content they need in the format they want. Together we'll craft a solution that engages your customers, stands out from the competition, and helps you meet your goals.

Advertising & Sponsorships

Connect with an engaged audience on a powerful multimedia site, and position your message alongside expert how-to content. Dummies.com is a one-stop shop for free, online information and know-how curated by a team of experts.

- Targeted ads
- Video
- Email Marketing
- Microsites
- Sweepstakes sponsorship

20 MILLION PAGE VIEWS **EVERY SINGLE MONTH**

15 MILLION **UNIQUE** **VISITORS PER MONTH**

43% OF ALL VISITORS ACCESS THE SITE **VIA THEIR MOBILE DEVICES**

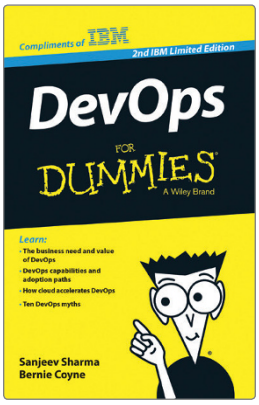
700,000 NEWSLETTER **SUBSCRIPTIONS** **TO THE INBOXES OF** *300,000* UNIQUE **INDIVIDUALS EVERY WEEK**

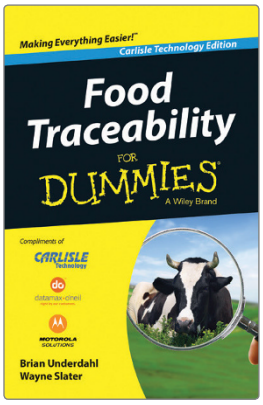
# of dummies

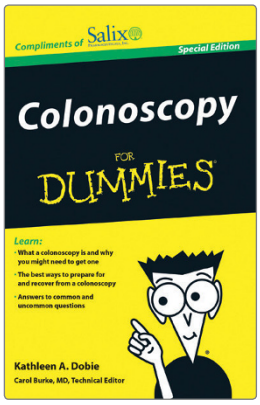
Custom Publishing

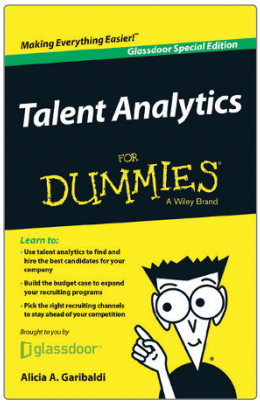
Reach a global audience in any language by creating a solution that will differentiate you from competitors, amplify your message, and encourage customers to make a buying decision.

- Apps
- Books
- eBooks
- Video
- Audio
- Webinars









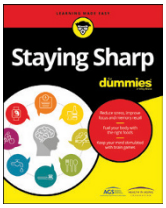
Brand Licensing & Content

Leverage the strength of the world's most popular reference brand to reach new audiences and channels of distribution.

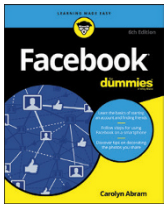
For more information, visit dummies.com/biz

dummies
A Wiley Brand

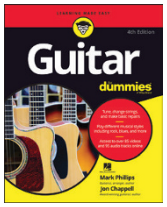
PERSONAL ENRICHMENT



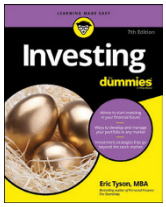
9781119187790
USA $26.00
CAN $31.99
UK £19.99



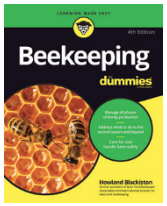
9781119179030
USA $21.99
CAN $25.99
UK £16.99



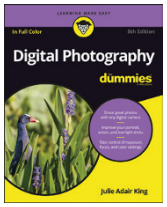
9781119293354
USA $24.99
CAN $29.99
UK £17.99



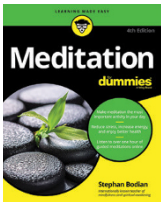
9781119293347
USA $22.99
CAN $27.99
UK £16.99



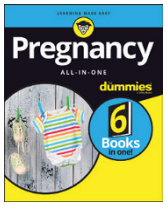
9781119310068
USA $22.99
CAN $27.99
UK £16.99



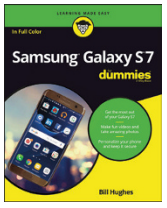
9781119235606
USA $24.99
CAN $29.99
UK £17.99



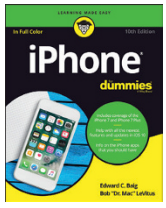
9781119251163
USA $24.99
CAN $29.99
UK £17.99



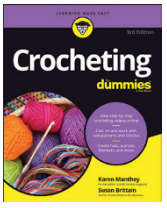
9781119235491
USA $26.99
CAN $31.99
UK £19.99



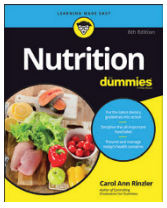
9781119279952
USA $24.99
CAN $29.99
UK £17.99



9781119283133
USA $24.99
CAN $29.99
UK £17.99

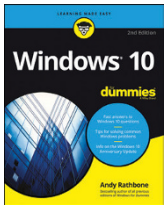


9781119287117
USA $24.99
CAN $29.99
UK £16.99

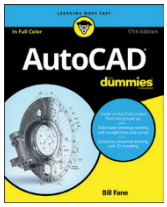


9781119130246
USA $22.99
CAN $27.99
UK £16.99

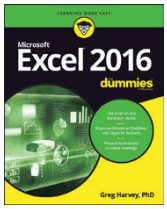
PROFESSIONAL DEVELOPMENT



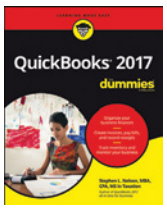
9781119311041
USA $24.99
CAN $29.99
UK £17.99



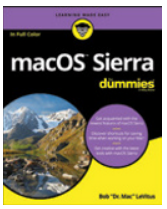
9781119255796
USA $39.99
CAN $47.99
UK £27.99



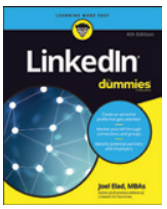
9781119293439
USA $26.99
CAN $31.99
UK £19.99



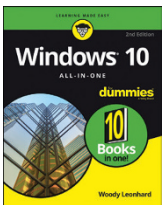
9781119281467
USA $26.99
CAN $31.99
UK £19.99



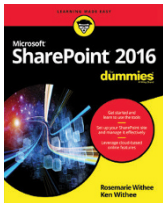
9781119280651
USA $29.99
CAN $35.99
UK £21.99



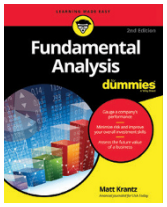
9781119251132
USA $24.99
CAN $29.99
UK £17.99



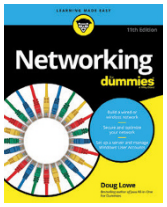
9781119310563
USA $34.00
CAN $41.99
UK £24.99



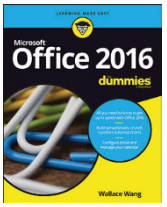
9781119181705
USA $29.99
CAN $35.99
UK £21.99



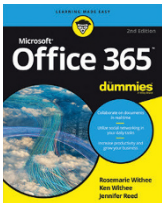
9781119263593
USA $26.99
CAN $31.99
UK £19.99



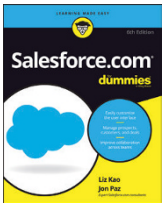
9781119257769
USA $29.99
CAN $35.99
UK £21.99



9781119293477
USA $26.99
CAN $31.99
UK £19.99



9781119265313
USA $24.99
CAN $29.99
UK £17.99

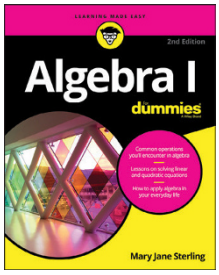


9781119239314
USA $29.99
CAN $35.99
UK £21.99

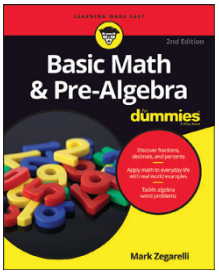
9781119293323
USA $29.99
CAN $35.99
UK £21.99

Learning Made Easy

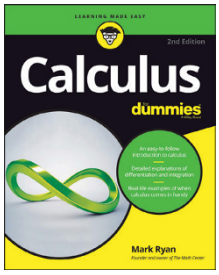
ACADEMIC



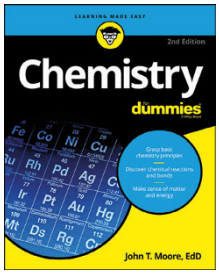
9781119293576
USA $19.99
CAN $23.99
UK £15.99



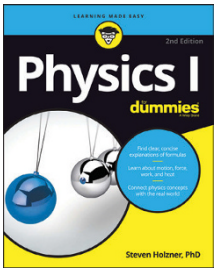
9781119293637
USA $19.99
CAN $23.99
UK £15.99



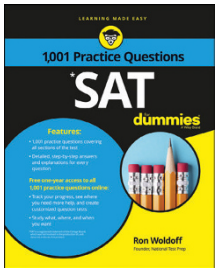
9781119293491
USA $19.99
CAN $23.99
UK £15.99



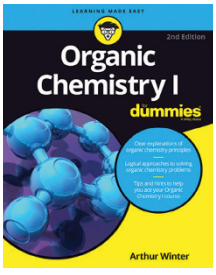
9781119293460
USA $19.99
CAN $23.99
UK £15.99



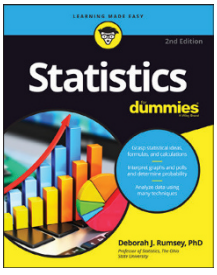
9781119293590
USA $19.99
CAN $23.99
UK £15.99



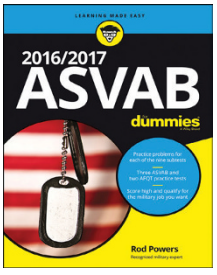
9781119215844
USA $26.99
CAN $31.99
UK £19.99



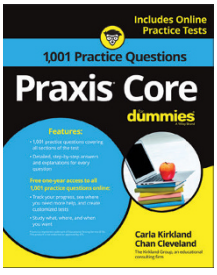
9781119293378
USA $22.99
CAN $27.99
UK £16.99



9781119293521
USA $19.99
CAN $23.99
UK £15.99



9781119239178
USA $18.99
CAN $22.99
UK £14.99

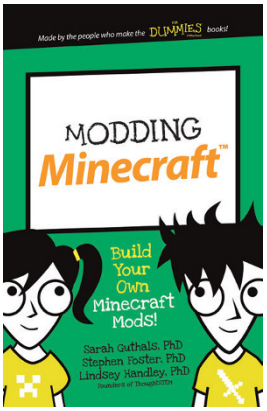


9781119263883
USA $26.99
CAN $31.99
UK £19.99

Available Everywhere Books Are Sold

Small books for big imaginations

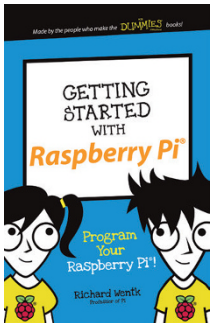
9781119177173
USA $9.99
CAN $9.99
UK £8.99



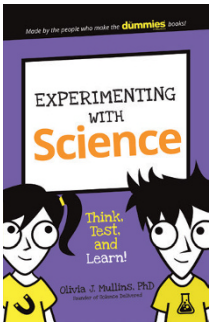
9781119177272
USA $9.99
CAN $9.99
UK £8.99

9781119177241
USA $9.99
CAN $9.99
UK £8.99

9781119177210
USA $9.99
CAN $9.99
UK £8.99

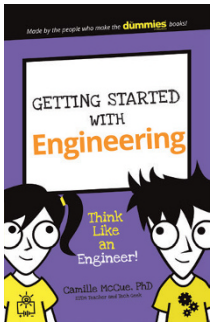


9781119262657
USA $9.99
CAN $9.99
UK £6.99

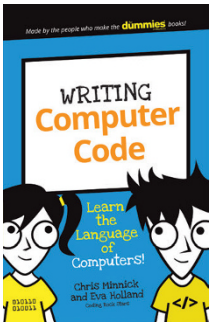


9781119291336
USA $9.99
CAN $9.99
UK £6.99

9781119233527
USA $9.99
CAN $9.99
UK £6.99



9781119291220
USA $9.99
CAN $9.99
UK £6.99



9781119177302
USA $9.99
CAN $9.99
UK £8.99

Unleash Their Creativity

dummies.com

dummies®
A Wiley Brand

WILEY END USER LICENSE AGREEMENT

Go to www.wiley.com/go/eula to access Wiley’s ebook EULA.

Full Stack

Audience Note

If you are already a full-stack developer you should visit these roadmaps instead.

Frontend | Backend | DevOps

Target audience for this roadmap is absolute beginners wanting to get into full stack development.

Find the detailed version of this checklist With details on how to implement these

https://roadmap.sh

- Project Ideas Inside
- Key topics to learn
- Change of Domain

HTML

CSS

JavaScript

Checkpoint — Static Webpages

Checkpoint — Interactivity

npm

Checkpoint — Collaborative Work

Checkpoint — External Packages

React

Tailwind

GitHub

Git

Feel free to skip and revisit after learning some backend.

Checkpoint — Frontend Apps

You can pick any backend programming language. My recommendation is Node.js because you are already familiar with JavaScript and it's easier to pick.

Start Backend Development

Node.js

Backend Starts Here

Checkpoint — CLI Apps

Learn Frontend Frameworks If you skipped earlier

PostgreSQL

Checkpoint — Simple CRUD

Redis

JWT Auth

RESTful APIs

Checkpoint — Complete App

Use the checkpoints and do not forget to practice what you learn. There are project ideas at each checkpoint that you can build to solidify your knowledge.

Route53

SES

Get Some DevOps Knowledge

EC2

VPC

S3

Linux Basics

Basic AWS Services

Checkpoint — Deployment

Monit

Checkpoint — Automation

Checkpoint — CI / CD

Checkpoint — Monitoring

Ansible

GitHub Actions

Terraform

Checkpoint — Infrastructure

Continue Learning with following relevant tracks

Frontend Roadmap | Backend Roadmap | DevOps Roadmap

JAVA PROGRAMMING

Lecture Notes

B.TECH
(III YEAR – I SEM)
(2022-23)

Prepared by:

Mrs. G.VAIDEHI, Assistant Professor

Department of Electronics and Communication Engineering

MALLA REDDY COLLEGE OF ENGINEERING & TECHNOLOGY
(Autonomous Institution – UGC, Govt. of India)

Recognized under 2(f) and 12 (B) of UGC ACT 1956
(AffiliatedtoJNTUH,Hyderabad,ApprovedbyAICTE-AccreditedbyNBA&NAAC–'A'Grade-ISO9001:2015Certified)
Maisammaguda,Dhulapally(PostVia.Kompally),Secunderabad–500100,TelanganaState,India

MALLA REDDY COLLEGE OF ENGINEERING &TECHNOLOGY

DEPARTMENT OF ECE

IIIYearB.Tech.ECE-I Sem **L/T/P/C**
3/-/-/3

(R20A0552) JAVA PROGRAMMING

COURSE OBJECTIVES:

1) To create Java programs that leverage the object-oriented features of the Java language, such as encapsulation, inheritance and polymorphism;
2) Use data types, arrays and strings;
3) Implement error-handling techniques using exception handling,
4) To know about Applets and Event Handling
5) Create and event-driven GUI using AWT components.

UNIT I:

Java Programming-OOP Concepts, History of Java, Java buzzwords, Data types, Variables, Constants, Scope and Life time of variables, Operators, Type conversion and casting, Control Flow Statements, simple java programs, concepts of classes, objects, arrays, strings, constructors, methods, access control, this keyword, overloading methods and constructors, garbage collection, recursion.

UNIT II:

Inheritance – Types of Inheritance, super keyword, and preventing inheritance: final classes and methods.

Polymorphism – Dynamic binding, method overriding, abstract classes and methods.

Interfaces-Interfaces Vs Abstract classes, defining an interface, implement interfaces, extending interface.

Packages- Defining, creating and accessing a package, importing packages.

UNIT III:

Exception handling - Benefits of exception handling, exception hierarchy, Classification of exceptions - checked exceptions and unchecked exceptions, usage of try, catch, throw, throws and finally, built in exceptions.

Multi-threading- Differences between multi-threading and multitasking, thread life cycle, creating threads, synchronizing threads.

UNIT IV

Applets – Concepts of Applets, differences between applets and applications, life cycle of an applet, types of applets, creating applets, passing parameters to applets.

Event Handling: Events, Handling mouse and keyboard events.

Files- Streams, Byte streams, Character streams, Text input/output

UNIT V

GUI Programming with Java – AWT class hierarchy, AWT controls - Labels, button, text field, check box, and graphics. Layout Manager – Layout manager types: border, grid and flow. Swing – Introduction, limitations of AWT, Swing vs AWT.

TEXT BOOKS:

1) Java- The Complete Reference, 7th edition, Herbert schildt,TMH.
2) Understanding OOP with Java, updated edition, T. Budd, Pearsoneducation.
3) Core Java an integrated approach, dreamtech publication, Dr. R.NageswaraRao.

REFERENCE BOOKS:

1) Java for Programmers, P.J.Deitel and H.M.Deitel, PEA (or) Java: How to Program,P.J.Deitel and H.M.Deitel, PHI
2) Object Oriented Programming through Java, P. Radha Krishna, Universities Press.

COURSE OUTCOMES:

1) An understanding of the principles and practice of object-oriented programming anddesign in the construction of robust, maintainable programs which satisfy theirrequirements;
2) A competence to design, write, compile, test and execute straightforward programsusinga high-level language;
3) An awareness of the need for a professional approach to design and the importance of good documentation to the finished programs.
4) Be able to make use of members of classes found in the Java API.
5) Demonstrate the ability to employ various types of constructs and a hierarchy of Javaclasses to provide solution to a given set of requirements.

INDEX

S. No	Unit	Topic	Page no
1	I	**OOP Concepts:-** Data abstraction, encapsulation inheritance	1
2	I	Benefits of Inheritance	2
3	I	Polymorphism, classes and objects	2
4	I	Procedural and object oriented programming paradigms	3
5	I	**Java Programming-** History of Java	4
6	I	Comments, Data types, Variables, Constants	5-9
7	I	Scope and Lifetime of variables	10
8	I	Operators, Operator Hierarchy, Expressions	11-12
9	I	Type conversion and casting	12-13
10	I	Control flow- block scope, conditional statements, loops, break and continue statements	13-14
11	I	Simple java stand alone programs, arrays	14-18
12	I	Console input and output, formatting output	18-19
13	I	Constructors, methods, parameter passing	19-20
14	I	Static fields and methods, access control, this reference,	21-30
15	I	Overloading methods and constructors, recursion, garbage collection,	30-34
16	I	Building strings, exploring string class.	34-36

S. No	Unit	Topic	Page no
17	II	**Inheritance –** Inheritance hierarchies super and sub classes, Member access rules	37-40
18	II	super keyword, preventing inheritance: final classes and methods, the Object class and its methods.	40-41
19	II	**Polymorphism – dynamic** binding, method overriding,	41-42
20	II	abstract classes and methods**.**	43
21	II	**Interfaces-** Interfaces Vs Abstract classes, defining an interface, implement interfaces	43-44
22	II	Accessing implementations through interface references, extendinginterface.	45
23	II	**Inner classes-** Uses of inner classes, local inner classes	45-46
24	II	Anonymous inner classes, static inner classes, examples.	46
25	II	**Packages-** Defining, creating and accessing a package,	46-47
26	II	Understanding CLASSPATH, importing packages.	47
27	III	**Exception handling-** Dealing with errors, benefits of exception handling	48
28	III	The classification of exceptions- exception hierarchy, checked exceptions and unchecked exceptions	48-50
29	III	Usage of try, catch, throw, throws and finally,	50-54
30	III	Built in exceptions, creating own exception sub classes.	54
31	III	**Multithreading –** Differences between multiple processes and multiple threads, thread states	55-56
32	III	Creating threads, interrupting threads, thread priorities, synchronizing threads	56-59

S. No	Unit	Topic	Page no
33	IV	**Applets –** Inheritance hierarchy for applets	61
34	IV	Differences between applets and applications, Life cycle of an applet	62-64
35	IV	Passing parameters to applets, applet security issues.	65-66
36	IV	**Event Handling-** Events, Event sources, Event classes, Event Listeners,	66-67
37	IV	Relationship between Event sources and Listeners, Delegation event model,	68-70
38	IV	Handling a button click, Handling Mouse events.	71-74
39	IV	**Files-** Streams- Byte streams, Character streams, Text input/output, Binary input/output	75-79
40	V	**GUI Programming with Java-** The AWT class hierarchy, AWT classes	80-81
41	V	Layout management – Layout manager types – border, grid and flow	82-87
42	V	Overview of some AWT Controls – Button, Label, TextField, TextArea, Checkbox and graphics	88-103
43	V	Swing – Introduction, limitations of AWT, Swing vs AWT.	104-108

Unit-1

OOPs Concepts

Object Oriented Programming is a paradigm that provides many concepts such as **inheritance**, **data binding**, **polymorphism** etc.

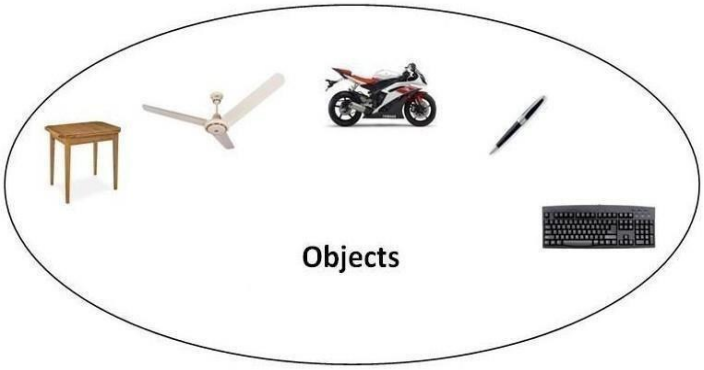
Simula is considered as the first object-oriented programming language. The programming paradigm where everything is represented as an object is known as truly object-oriented programming language.

Smalltalk is considered as the first truly object-oriented programming language.

OOPs (Object Oriented Programming System)

Object means a real word entity such as pen, chair, table etc. **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects. It simplifies the software development and maintenance by providing someconcepts:

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation



Object

Any entity that has state and behavior is known as an object. For example: chair, pen, table, keyboard, bike etc. It can be physical and logical.

Class

Collection of objects is called class. It is a logical entity.

Inheritance

When one object acquires all the properties and behaviours of parent object i.e. known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.

Polymorphism

When **one task is performed by different ways** i.e. known as polymorphism. For example: to convince the customer differently, to draw something e.g. shape or rectangle etc.

In java, we use method overloading and method overriding to achieve polymorphism. Another example can be to speak something e.g. cat speaks meaw, dog barks woof etc.

Abstraction

Hiding internal details and showing functionality is known as abstraction. For example: phone call, we don't know the internal processing.
In java, we use abstract class and interface to achieve abstraction.

Encapsulation

Binding (or wrapping) code and data together into a single unit is known as encapsulation. For example: capsule, it is wrapped with different medicines.

A java class is the example of encapsulation. Java bean is the fully encapsulated class because allthe data members are private here.

Benefits of Inheritance

- One of the key benefits of inheritance is to minimize the amount of duplicate code in an application by sharing common code amongst several subclasses. Where equivalent code exists in two related classes, the hierarchy can usually be refactored to move the common code up to a mutual superclass. This also tends to result in a better organization of code and smaller, simpler compilationunits.
  - Inheritance can also make application code more flexible to change because classesthat inherit from a common superclass can be used interchangeably. If the return type of a method issuperclass
  - **Reusability** - facility to use public methods of base class without rewriting thesame.
  - **Extensibility** - extending the base class logic as per business logic of the derivedclass.

- **Data hiding** - base class can decide to keep some data private so that it cannot be altered by the derived class

Procedural and object oriented programming paradigms

Features	Procedural Oriented Programming (POP)	Object Oriented Programming (OOPS)
Divided into	In POP, program is divided into smaller parts called as functions.	in OOPs , the program is divided into parts known as **objects**.
Importance	In POP, importance is not given to **data** but to functions as well as **sequence** of actions to be done.	In OOPs, Importance is given to the data rather than procedures or functions because it works as a **real world**.
Approach	POP follows **Top Down approach**.	OOPs follows **Bottom Up approach**.
Access Specifiers	POP does not have any access specifier.	OOPs has access specifiers named Public, Private, Protected, etc.
Data Moving	In POP, Data can move freely from function to function in the system.	In OOPs, objects can move and communicate with each other through member functions.
Data Access	In POP, Most function uses Global data for sharing that can be accessed freely from function to function in the system.	In OOPs, data can not move easily from function to function,it can be kept public or private so we can control the access of data.
Data Hiding	POP does not have any proper way for hiding data so it is **less secure**.	OOPs provides Data Hiding so provides **more security**.
Overloading	In POP, Overloading is not possible.	In OOPs, overloading is possible in the form of Function Overloading and Operator Overloading.
Examples	C, VB, FORTRAN, Pascal.	C++, JAVA, VB.NET, C#.NET.

Java Programming- History of Java

The history of java starts from Green Team. Java team members (also known as **Green Team**), initiated a revolutionary task to develop a language for digital devices such as set-top boxes, televisionsetc.

For the green team members, it was an advance concept at that time. But, it was suited for internet programming. Later, Java technology as incorporated by Netscape.

Currently, Java is used in internet programming, mobile devices, games, e-business solutions etc. There are given the major points that describes the history of java.

1) **James Gosling**, **Mike Sheridan**, and **Patrick Naughton** initiated the Java language project in June 1991. The small team of sun engineers called **Green Team**.

2) Originally designed for small, embedded systems in electronic appliances like set-topboxes.

3) Firstly, it was called **"Greentalk"** by James Gosling and file extension was.gt.

4) After that, it was called Oak and was developed as a part of the Green project.

Java Version History

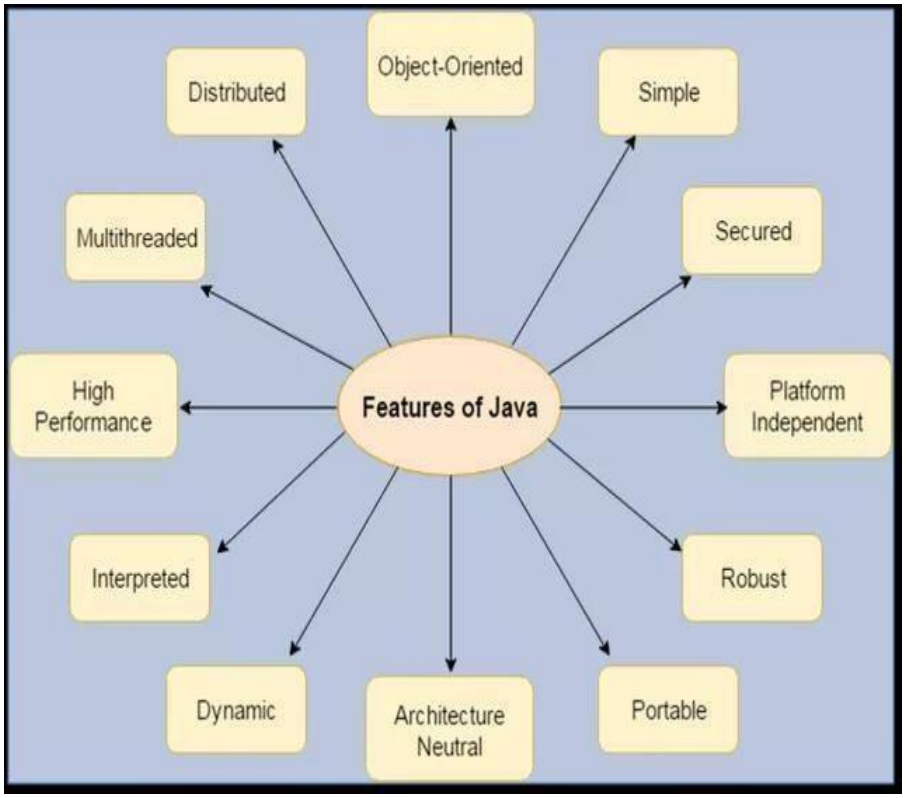
There are many java versions that has been released. Current stable release of Java is Java SE 8.

1. JDK Alpha and Beta (1995)
2. JDK 1.0 (23rd Jan, 1996)
3. JDK 1.1 (19th Feb, 1997)
4. J2SE 1.2 (8th Dec, 1998)
5. J2SE 1.3 (8th May, 2000)
6. J2SE 1.4 (6th Feb, 2002)
7. J2SE 5.0 (30th Sep,2004)
8. Java SE 6 (11th Dec,2006)
9. Java SE 7 (28th July, 2011)
10. Java SE 8 (18th March,2014)

Features of Java

There is given many features of java. They are also known as java buzzwords. The Java Featuresgiven below are simple and easy to understand.

1. Simple
2. Object-Oriented
3. Portable
4. Platformindependent
5. Secured
6. Robust
7. Architectureneutral
8. Dynamic
9. Interpreted
10. HighPerformance
11. Multithreaded
12. Distributed



Java Comments

The java comments are statements that are not executed by the compiler and interpreter. The comments can be used to provide information or explanation about the variable, method, class or any statement. It can also be used to hide program code for specific time.

Types of Java Comments

There are 3 types of comments in java.

1. Single LineComment
2. Multi LineComment
3. DocumentationComment

Java Single Line Comment

The single line comment is used to comment only one line.

Syntax:

```
1. //This is single line comment
```

Example:

```
public class CommentExample1 {
public static void main(String[] args) {
   int i=10;//Here, i is a variable
   System.out.println(i);
}
}
```

Output:

```
10
```

Java Multi Line Comment

The multi line comment is used to comment multiple lines of code.

Syntax:

```
/* Thisis
multi linecomment
*/
```

Example:

```
public class CommentExample2 {
public static void main(String[] args) {
/* Let's declare and print variable in java.*/
inti=10; System.out.println(i);
} }
```

Output:

```
10
```

Java Documentation Comment

The documentation comment is used to create documentation API. To create documentation API, you need to use **javadoc tool**.

Syntax:

```
/**
This
is
documentation
comment
*/
```

Example:

```
/** The Calculator class provides methods to get addition and subtraction of given 2 numbers.*/
public class Calculator {
/** The add() method returns addition of given numbers.*/
public static int add(int a, int b){return a+b;}
/** The sub() method returns subtraction of given numbers.*/
public static int sub(int a, int b){return a-b;}
}
```

Compile it by javac tool:

```
javac Calculator.java
```

Create Documentation API by javadoc tool:

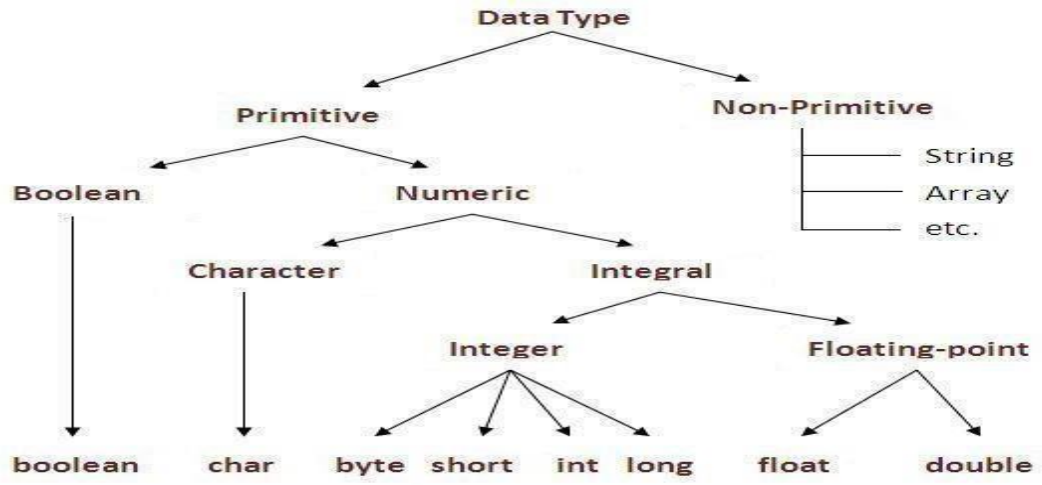
```
javadoc Calculator.java
```

Now, there will be HTML files created for your Calculator class in the current directory. Open the HTML files and see the explanation of Calculator class provided through documentation comment.

Data Types

Data types represent the different values to be stored in the variable. In java, there are two types of data types:

- Primitive datatypes
- Non-primitive datatypes



DataType	DefaultValue	Default size
boolean	False	1 bit
char	'\u0000'	2 byte
byte	0	1 byte
short	0	2 byte
int	0	4 byte
long	0L	8 byte
float	0.0f	4 byte
double	0.0d	8 byte

Java Variable Example: Add Two Numbers

```
class Simple{
public static void main(String[] args){
int a=10;
int b=10;
int c=a+b;
System.out.println(c);
}}
```

Output:20

Variables and Data Types in Java

Variable is a name of memory location. There are three types of variables in java: local, instanceand static.

There are two types of data types in java: primitive and non-primitive.

Types of Variable

There are three types of variables in java:

- localvariable
- instancevariable
- staticvariable

1) LocalVariable

A variable which is declared inside the method is called local variable.

2) Instance Variable

A variable which is declared inside the class but outside the method, is called instance variable. Itis not declared as static.

3) Staticvariable

A variable that is declared as static is called static variable. It cannot be local.We will have detailed learning of these variables in next chapters.

Example to understand the types of variables in java

```
class A{
int data=50;//instance variable
static int m=100;//static variable
void method(){
int n=90;//local variable
}
}//end of class
```

Constants in Java

A constant is a variable which cannot have its value changed after declaration. It uses the **'final' keyword.**

Syntax

```
modifierfinal dataType variableName = value; //global constant

modifierstatic final dataType variableName = value; //constant within a c
```

Scope and Life Time of Variables

The scope of a variable defines the section of the code in which the variable is visible. As a general rule, variables that are defined within a block are not accessible outside that block. The lifetime of a variable refers to how long the variable exists before it isdestroyed. Destroying variables refers to deallocating the memory that was allotted to the variables when declaring it. We have written a few classes till now. You might have observed that not all variables are the same. The ones declared in the body of a method were different from those that were declared in the class itself. There are three types of variables: instance variables, formal parameters or local variables and localvariables.

Instance variables

Instance variables are those that are defined within a class itself and not in any method or constructor of the class. They are known as instance variables because every instance of the class (object) contains a copy of these variables. The scope of instance variables is determined by the access specifier that is applied to these variables. We have already seen about it earlier. The lifetime of these variables is the same as the lifetime of the object to which it belongs. Object once created do not exist for ever. They are destroyed by the garbage collector of Java when there are no more reference to that object. We shall see about Java's automatic garbage collector later on.

Argument variables

These are the variables that are defined in the header oaf constructor or a method. The scope of these variables is the method or constructor in which they are defined. The lifetime is limited to the time for which the method keeps executing. Once the method finishes execution, these variables aredestroyed.

Local variables

A local variable is the one that is declared within a method or a constructor (not in the header). The scope and lifetime are limited to the methoditself.

One important distinction between these three types of variables is that access specifiers can be applied to instance variables only and not to argument or local variables.

In addition to the local variables defined in a method, we also have variables that are defined in bocks life an if block and an else block. The scope and is the same as that of the block itself.

Operators in java

Operator in java is a symbol that is used to perform operations. For example: +, -, *, / etc.There are many types of operators in java which are given below:

- UnaryOperator,
- ArithmeticOperator,
- shiftOperator,
- RelationalOperator,
- BitwiseOperator,
- LogicalOperator,
- Ternary Operatorand
- AssignmentOperator.

Operators Hierarchy

Operator Precedence

Operators	Precedence
postfix	`expr++ expr--`
unary	`++expr --expr +expr -expr ~ !`
multiplicative	`* / %`
additive	`+ -`
shift	`<< >> >>>`
relational	`< > <= >= instanceof`
equality	`== !=`
bitwise AND	`&`
bitwise exclusive OR	`^`
bitwise inclusive OR	`|`
logical AND	`&&`
logical OR	`||`
ternary	`? :`
assignment	`= += -= *= /= %= &= ^= |= <<= >>= >>>=`

Expressions

Expressions are essential building blocks of any Java program, usually created to produce a newvalue, although sometimes an expression simply assigns a value to a variable. Expressions are built using values, variables, operators and method calls.

Types of Expressions

While an expression frequently produces a result, it doesn't always. There are three types ofexpressions in Java:

- Those that produce a value, i.e. the result of (1 + 1)
- Those that assign a variable, for example (v =10)
- Those that have no result but might have a "side effect" because an expression can include a wide range of elements such as method invocations or increment operators that modify the state (i.e. memory) of aprogram.

Java Type casting and Type conversion

Widening or Automatic Type Conversion

Widening conversion takes place when two data types are automatically converted. This happenswhen:

- The two data types arecompatible.
- When we assign value of a smaller data type to a bigger datatype.

For Example, in java the numeric data types are compatible with each other but no automatic conversion is supported from numeric type to char or boolean. Also, char and boolean are not compatible with each other.

Byte –> Short –> Int –> Long – > Float –> Double

Widening or Automatic Conversion

Narrowing or Explicit Conversion

If we want to assign a value of larger data type to a smaller data type we perform explicit type casting or narrowing.

- This is useful for incompatible data types where automatic conversion cannot bedone.
- Here, target-type specifies the desired type to convert the specified valueto.

Double –> Float –> Long –> Int –> Short –> Byte

- Narrowing or Explicit Conversion

Control Flow Statements

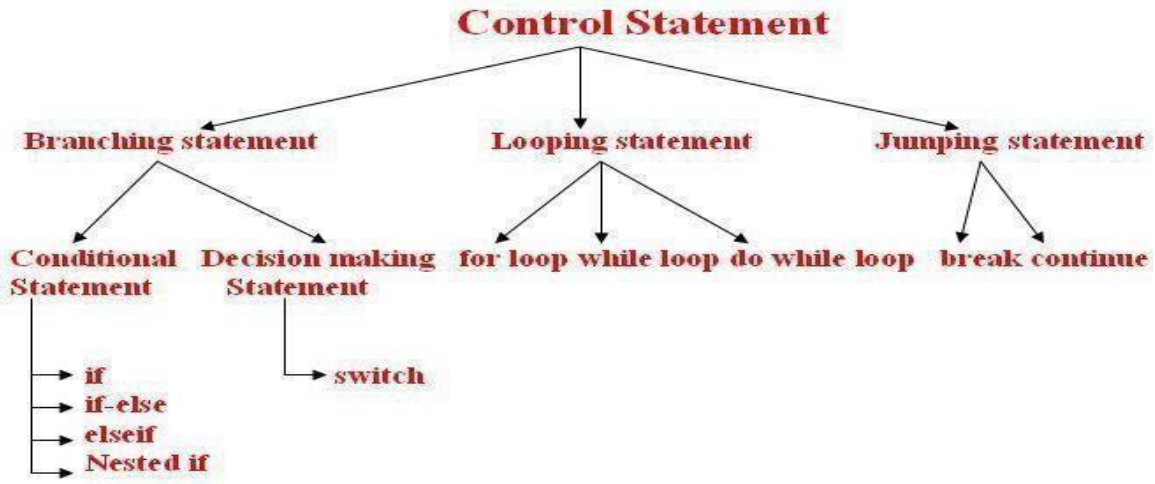
The control flow statements in Java allow you to run or skip blocks of code when specialconditions are met.

The "if" Statement

The "if" statement in Java works exactly like in most programming languages. With the help of"if" you can choose to execute a specific block of code when a predefined condition is met. Thestructure of the "if" statement in Java looks like this:

```
if(condition) {
// execute this code
}
```

The condition is Boolean. Boolean means it may be true or false. For example you may put amathematical equation as condition. Look at this full example:



Creating a Stand-Alone Java Application

1. Write a main method that runs your program. You can write this method anywhere. In this example, I'll write my main method in a class called Main that has no other methods. **For example:**
2. public class Main
3. {

```
4. public static void main(String[] args)5. {
6. Game.play();
7. } }
```

8. Make sure your code is compiled, and that you have tested it thoroughly.
9. If you're using Windows, you will need to set your path to include Java, if you haven't done so already. This is a delicate operation. Open Explorer, and look inside C:\ProgramFiles\Java, and you should see some version of the JDK. Open this folder, and then open the bin folder. Select the complete path from the top of the Explorer window, and press Ctrl-C to copyit.

Next, find the "My Computer" icon (on your Start menu or desktop), right-click it, and selectproperties. Click on the Advanced tab, and then click on the Environment variables button. Look at the variables listed for all users, and click on the Path variable. Do not delete the contents of this variable! Instead, edit the contents by moving the cursor to the right end, entering a semicolon (;), and pressing Ctrl-V to paste the path you copied earlier. Then go ahead and save your changes. (If you have any Cmd windows open, you will need to close them.)

10. If you're using Windows, go to the Start menu and type "cmd" to run a program that brings up a command prompt window. If you're using a Mac or Linux machine, run the Terminal program to bring up a commandprompt.
11. In Windows, type dir at the command prompt to list the contents of the current directory. On a Mac or Linux machine, type ls to dothis.

12. Now we want to change to the directory/folder that contains your compiled code. Look at the listing of sub-directories within this directory, and identify which one contains your code. Type cd followed by the name of that directory, to change to that directory. For example, to change to a directory called Desktop, you would type:

cd Desktop

To change to the parent directory, type:

cd ..

Every time you change to a new directory, list the contents of that directory to see where to gonext. Continue listing and changing directories until you reach the directory that contains your .class files.

13. If you compiled your program using Java 1.6, but plan to run it on a Mac, you'll needto recompile your code from the command line, bytyping:

javac -target 1.5 *.java

14. Now we'll create a single JAR file containing all of the files needed to run yourprogram.

Arrays

Java provides a data structure, the **array**, which stores a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type.

Instead of declaring individual variables, such as number0, number1, ..., and number99, you declare one array variable such as numbers and use numbers[0], numbers[1], and ..., numbers[99] to represent individual variables.

This tutorial introduces how to declare array variables, create arrays, and process arrays using indexed variables.

Declaring Array Variables:

To use an array in a program, you must declare a variable to reference the array, and you must specify the type of array the variable can reference. Here is the syntax for declaring an array variable:

```
dataType[] arrayRefVar; // preferred way.or
dataType arrayRefVar[]; // works but not preferred way.
```

Note: The style**dataType[] arrayRefVar** is preferred. The style **dataType arrayRefVar[]** comes from the C/C++ language and was adopted in Java to accommodate C/C++programmers.

Example:

The following code snippets are examples of this syntax:

```
double[] List;                  // preferred way.
or
double List[];                  // works but not preferred way.
```

Creating Arrays:

You can create an array by using the new operator with the following syntax:

```
arrayRefVar= new dataType[arraySize];
```

The above statement does two things:

- It creates an array using newdataType[arraySize];
- It assigns the reference of the newly created array to the variablearrayRefVar.

Declaring an array variable, creating an array, and assigning the reference of the array to thevariable can be combined in one statement, as shown below:

```
dataType[] arrayRefVar = new dataType[arraySize];
```

Alternatively you can create arrays as follows:

```
dataType[] arrayRefVar = {value0, value1, ..., valuek};
```

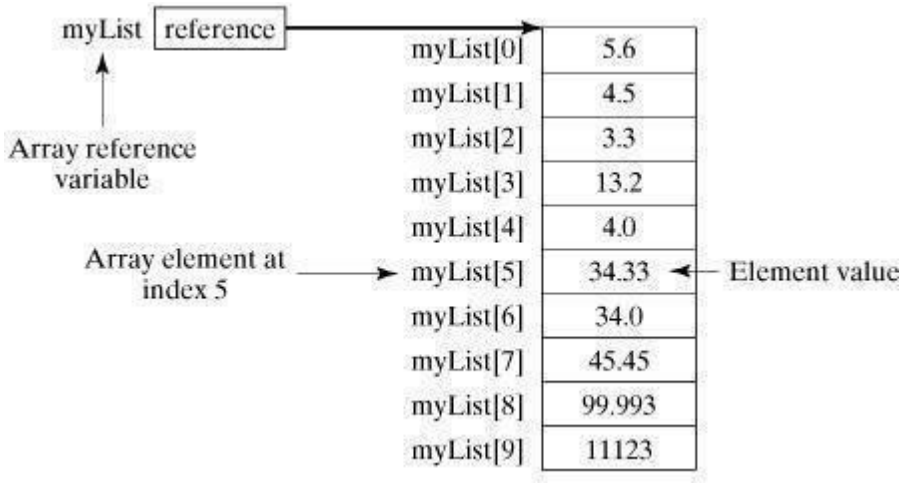
The array elements are accessed through the **index**. Array indices are 0-based; that is, they startfrom 0 to **arrayRefVar.length-1**.

Example:

Following statement declares an array variable, myList, creates an array of 10 elements ofdouble type and assigns its reference tomyList:

```
double[] myList = new double[10];
```

Following picture represents array myList. Here, myList holds ten double values and the indicesare from 0 to 9.



Processing Arrays:

When processing array elements, we often use either for loop or for each loop because all of theelements in an array are of the same type and the size of the array is known.

Example:

Here is a complete example of showing how to create, initialize and process arrays:

```
public class TestArray
{
public static void main(String[] args) { double[] myList = {1.9, 2.9, 3.4, 3.5};
// Print all the array elements for (int i = 0; i <myList.length; i++){
System.out.println(myList[i] + " ");
}
// Summing all elementsdouble total = 0;
for(int i = 0; i <myList.length; i++) {total += myList[i];
}
System.out.println("Total is " + total);
// Finding the largest elementdouble max = myList[0];
for(int i = 1; i <myList.length; i++) { if(myList[i] >max) max = myList[i];
}
System.out.println("Max is " + max);
}
}
```

This would produce the following result:

```
1.9
2.9
3.4
3.5
Total is 11.7
Max is 3.5
```

```
public class TestArray {
public static void main(String[] args) { double[] myList = {1.9, 2.9, 3.4, 3.5};
// Print all the array elements for(double element: myList) {
System.out.println(element);
}}}
```

Constructors

Constructor in java is a *special type of method* that is used to initialize the object.

Java constructor is *invoked at the time of object creation*. It constructs the values i.e. provides data for the object that is why it is known as constructor.

There are basically two rules defined for the constructor.

1. Constructor name must be same as its classname
2. Constructor must have no explicit returntype

Types of java constructors

There are two types of constructors:

1. Default constructor (no-argconstructor)
2. Parameterizedconstructor

Java Default Constructor

A constructor that have no parameter is known as default constructor.

Syntax of default constructor:

1. <class_name>(){}

Example of default constructor

In this example, we are creating the no-arg constructor in the Bike class. It will be invoked atthe time of object creation.

```
classBike1{
Bike1(){System.out.println("Bike is created");}
public static void main(String args[]){
Bike1 b=new Bike1();
}}
```

Output: Bike is created

Example of parameterized constructor
In this example, we have created the constructor of Student class that have two parameters. Wecan have any number of parameters in the constructor.

```
classStudent4{
 int id;
 Stringname;

Student4(int i,String n){
id = i;
name = n;
}
void display(){System.out.println(id+" "+name);}

public static void main(String args[]){
Student4 s1 = new Student4(111,"Karan");
Student4 s2 = new Student4(222,"Aryan");s1.display();
s2.display();
} }
```

Output:

```
111Karan222Aryan
```

Constructor Overloading in Java

Constructor overloading is a technique in Java in which a class can have any number of constructors that differ in parameter lists.The compiler differentiates these constructors bytaking into account the number of parameters in the list and their type.

Example of Constructor Overloading

```
  classStudent5{
  intid; String
  name;
  intage;
Student5(int i,String n){id = i;
name = n;
}
  Student5(int i,String n,int a){
  id = i;
name = n;age=a;
}
voiddisplay(){System.out.println(id+" "+name+" "+age);}

public static void main(String args[]){
Student5 s1 = new Student5(111,"Karan");
Student5 s2 = new Student5(222,"Aryan",25);
s1.display();
```

```
s2.display();
} }
```

Output:

```
111 Karan 0
222 Aryan 25
```

Java CopyConstructor

There is no copy constructor in java. But, we can copy the values of one object to another likecopy constructor inC++.

There are many ways to copy the values of one object into another in java. They are:

- By constructor
- By assigning the values of one object into another
- By clone() method of Object class

In this example, we are going to copy the values of one object into another using javaconstructor.

```
classStudent6{intid; Stringname;
Student6(int i,String n){id = i;
name = n;
}

Student6(Student6 s){id = s.id;
name =s.name;
}
voiddisplay(){System.out.println(id+" "+name);}

   public static void main(String args[]){
   Student6 s1 = new Student6(111,"Karan");
   Student6 s2 = new Student6(s1);
   s1.display();
s2.display();
} }
```

Output:

```
111Karan 111Karan
```

Java -Methods

A Java method is a collection of statements that are grouped together to perform an operation. When you call the System.out.**println()** method, for example, the system actually executes several statements in order to display a message on the console.

Now you will learn how to create your own methods with or without return values, invoke amethod with or without parameters, and apply method abstraction in the program design.

Creating Method

Considering the following example to explain the syntax of a method −

Syntax

```
public static int methodName(int a, int b) {
// body
}
```

Here,

- **public static** −modifier
- **int** − returntype
- **methodName** − name of the method
- **a, b** − formalparameters
- **int a, int b** − list ofparameters

Method definition consists of a method header and a method body. The same is shown in thefollowing syntax −

Syntax

```
modifier returnType nameOfMethod (Parameter List) {
// method body
}
```

The syntax shown above includes −

- **modifier**− It defines the access type of the method and it is optional touse.
- **returnType**− Method may return avalue.
- **nameOfMethod**− This is the method name. The method signature consists of themethod name and the parameter list.
- **Parameter List** − The list of parameters, it is the type, order, and number of parameters of a method. These are optional, method may contain zeroparameters.
- **method body** − The method body defines what the method does with thestatements.

Static Fields and Methods

The **static keyword** in java is used for memory management mainly. We can apply java static keyword with variables, methods, blocks and nested class. The static keyword belongs to the classthan instance of the class.

The static can be:

1. variable (also known as classvariable)
2. method (also known as classmethod)
3. block
4. nestedclass

Java static variable

If you declare any variable as static, it is known static variable.

- The static variable can be used to refer the common property of all objects (that is not unique for each object) e.g. company name of employees,college name of studentsetc.
- The static variable gets memory only once in class area at the time of classloading.

Advantage of static variable

It makes your program **memory efficient** (i.e it saves memory).

Understanding problem without static variable

```
1. classStudent{
2.     introllno;
3.     Stringname;
4.     String college="ITS";
5.}
```

Example of static variable

```
//Program of static variable
classStudent8{
  introllno;
```

```
  String name;
  staticString college ="ITS";
  Student8(int r,String n){
  rollno =r;
  name =n;
  }voiddisplay (){System.out.println(rollno+" "+name+" "+college);}
public static void main(String args[]){
Student8 s1 = new Student8(111,"Karan");
Student8 s2 = new Student8(222,"Aryan");

s1.display();
s2.display();
} }
```

```
Output:111 KaranITS
       222 AryanITS
```

Java static method

If you apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than object of aclass.
- A static method can be invoked without the need for creating an instance of aclass.
- static method can access static data member and can change the value ofit.

Example of static method

```
//Program of changing the common property of all objects(static field).

classStudent9{
   int rollno;
   String name;
   staticString college = "ITS";
   static void change(){
   college = "BBDIT";
}
Student9(int r, String n){rollno =r;
name =n;
```

```
}
 voiddisplay (){System.out.println(rollno+" "+name+" "+college);}
   public static void main(String args[]){
   Student9.change();
   Student9 s1 = new Student9 (111,"Karan");
   Student9 s2 = new Student9 (222,"Aryan");
   Student9 s3 = new Student9 (333,"Sonoo");
   s1.display();
 s2.display();
 s3.display();
 } }
```

```
Output:111 Karan BBDIT222 Aryan BBDIT
333 Sonoo BBDIT
```

Java static block

- Is used to initialize the static datamember.
- It is executed before main method at the time of classloading.

Example of static block

```
class A2{
static{
System.out.println("static block is invoked");
}
public static void main(String args[]){
System.out.println("Hello main");
} }
```

Output: static block is invokedHello main

Access Control

Access Modifiers in java

There are two types of modifiers in java: **access modifiers** and **non-access modifiers**.

The access modifiers in java specifies accessibility (scope) of a data member, method, constructoror class. There are 4 types of java access modifiers:

1. private
2. default
3. protected
4. public

private access modifier

The private access modifier is accessible only within class.

Simple example of private access modifier

In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is compile time error.

```
class A{
private int data=40;
private void msg(){System.out.println("Hello java");}}
public class Simple{
public static void main(String args[]){A obj=new A();
System.out.println(obj.data);//Compile Time Errorobj.msg();//Compile Time Error
} }
```

2) default accessmodifier

If you don't use any modifier, it is treated as **default** bydefault. The default modifier isaccessible only withinpackage.

Example of default accessmodifier

In this example, we have created two packages pack and mypack. We are accessing the A class from outside its package, since A class is not public, so it cannot be accessed from outside thepackage.

```
//save by A.java
package pack;
class A{
voidmsg(){System.out.println("Hello");}
}

//save by B.java
package mypack;
import pack.*;
```

```
class B{
  public static void main(String args[]){
   A obj = new A();//Compile Time Error
   obj.msg();//Compile Time Error }}
```

In the above example, the scope of class A and its method msg() is default so it cannot beaccessed from outside thepackage.

3) protected accessmodifier

The **protected access modifier** is accessible within package and outside the package but through inheritance only.

The protected access modifier can be applied on the data member, method and constructor. It can'tbe applied on the class.
Example of protected access modifier

In this example, we have created the two packages pack and mypack. The A class of pack package is public, so can be accessed from outside the package. But msg method of this package is declared as protected, so it can be accessed from outside the class only throughinheritance.

```
//save by A.java
package pack;
public class A{
protected void msg(){System.out.println("Hello");} }
//save by B.java
package mypack;
import pack.*;
class B extends A{
  public static void main(String args[]){
   B obj = new B();
obj.msg();
} }
```

Output:Hello

4) public accessmodifier

The **public access modifier** is accessible everywhere. It has the widest scope among all other modifiers.

Example of public access modifier

```
//save by A.java
package pack;
public class A{
public void msg(){System.out.println("Hello");}}
//save by B.java
package mypack;
import pack.*;
class B{
  public static void main(String args[]){
   A obj = new A();
obj.msg();
} }
```

Output:Hello

Understanding all java access modifiers

Let's understand the access modifiers by a simple table.

Access Modifier	within class	within package	outsidepackageby subclassonly	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

this keyword in java

Usage of java this keyword

Here is given the 6 usage of java this keyword.

1. this can be used to refer current class instancevariable.
2. this can be used to invoke current class method(implicitly)
3. this() can be used to invoke current classconstructor.

4. this can be passed as an argument in the method call.
5. this can be passed as argument in the constructorcall.
6. this can be used to return the current class instance from themethod.

```
class Student{
int rollno;
String name;
float fee;
Student(int rollno,String name,float fee){
this.rollno=rollno;
this.name=name;
this.fee=fee;
}
void display(){System.out.println(rollno+" "+name+" "+fee);}
}
class TestThis2{
public static void main(String args[]){
Student s1=new Student(111,"ankit",5000f);
Student s2=newStudent(112,"sumit",6000f);
s1.display();
s2.display();
}}
```

Output:
111 ankit 5000
112 sumit 6000

Difference between constructor and method in java

There are many differences between constructors and methods. They are given below:

Java Constructor	Java Method
Constructor is used to initialize the state of an object.	Method is used to expose behaviourof anobject.
Constructor must not have return type.	Method must have return type.
Constructor is invoked implicitly.	Method is invoked explicitly.
The java compiler provides a default constructor if youdon't have any constructor.	Method is not provided by compiler inany case.
Constructor name must be same as the class name.	Method name may or may not be

	same as class name.

Constructor Overloading in Java

Constructor overloading is a technique in Java in which a class can have any number of constructors that differ in parameter lists.The compiler differentiates these constructors by taking into account the number of parameters in the list and their type.

Example of Constructor Overloading

```
class Student5{
int id; String
name;
intage;
Student5(int i,String n){
id = i;
name = n;
}
Student5(int i,String n,int a){
id = i;
name = n;
age=a;
}
void display(){System.out.println(id+" "+name+" "+age);}

public static void main(String args[]){
Student5 s1 = new Student5(111,"Karan");
Student5 s2 = newStudent5(222,"Aryan",25);
s1.display();
s2.display();
}
}
```

Output:

```
111 Karan 0
222 Aryan 25
```

Method Overloading in java

If a class has multiple methods having same name but different in parameters, it is knownas **MethodOverloading**.

If we have to perform only one operation, having same name of the methods increases thereadability of the program.

Method Overloading: changing no. of arguments

In this example, we have created two methods, first add() method performs addition of twonumbers and second add method performs addition of three numbers.

In this example, we are creating static methods so that we don't need to create instance for callingmethods.

```
classAdder{
static int add(int a,int b){return a+b;}
static int add(int a,int b,int c){return a+b+c;}
}
classTestOverloading1{
public static void main(String[] args){ System.out.println(Adder.add(11,11));
System.out.println(Adder.add(11,11,11));
}}
```

Output:

```
22
33
```

Method Overloading: changing data type of arguments

In this example, we have created two methods that differs in data type. The first add methodreceives two integer arguments and second add method receives two double arguments.

Recursion in Java

Recursion in java is a process in which a method calls itself continuously. A method in java thatcalls itself is called recursive method.

Java Recursion Example 1: Factorial Number

```
public class RecursionExample3 {
  static int factorial(int n){
if(n == 1)
    return 1;
    else
   return(n * factorial(n-1));
} }
public static void main(String[] args) {
System.out.println("Factorial of 5 is: "+factorial(5));
} }
```

Output:

```
Factorial of 5 is: 120
```

Java Garbage Collection

In java, garbage means unreferenced objects.

Garbage Collection is process of reclaiming the runtime unused memory automatically. In otherwords, it is a way to destroy the unused objects.

To do so, we were using free() function in C language and delete() in C++. But, in java it isperformed automatically. So, java provides better memory management.

Advantage of Garbage Collection

- It makes java **memory efficient** because garbage collector removes the unreferenced objects from heapmemory.
- It is **automatically done** by the garbage collector(a part of JVM) so we don't need to make extraefforts.

gc() method

The gc() method is used to invoke the garbage collector to perform cleanup processing. Thegc() is found in System and Runtime classes.

```
public static void gc(){}
```

Simple Example of garbage collection in java

```
public class TestGarbage1{
public void finalize(){
System.out.println("object is garbage collected");
}
public static void main(String args[]){

TestGarbage1 s1=new TestGarbage1();

TestGarbage1 s2=new TestGarbage1();

s1=null;

s2=null; System.gc();
} }
```

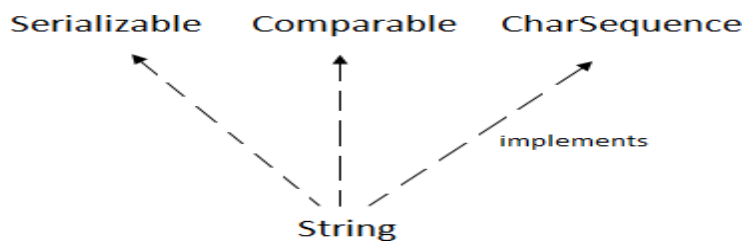
```
object is garbage collectedobject is garbage collected
```

Java String

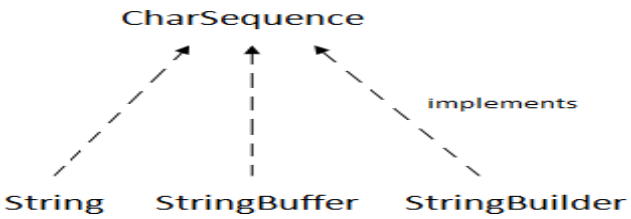
string is basically an object that represents sequence of char values. An array of characters workssame as java string. For example:

1. **char**[] ch={'j','a','v','a','t','p','o','i','n','t'};
2. String s=**new** String(ch);ssame as:
1. Strings="javatpoint";
2. **Java String** class provides a lot of methods to perform operations on string such as compare(), concat(), equals(), split(), length(), replace(), compareTo(), intern(), substring()etc.
3. The java.lang.String class implements *Serializable*, *Comparable* and *CharSequence*interfaces.

CharSequence Interface



The CharSequence interface is used to represent sequence of characters. It is implemented by String, StringBuffer and StringBuilder classes. It means, we can create string in java by using these 3 classes.



The java String is immutable i.e. it cannot be changed. Whenever we change any string, a new instance is created. For mutable string, you can use StringBuffer and StringBuilderclasses.
There are two ways to create String object:

1. By stringliteral
2. By newkeyword

String Literal

Java String literal is created by using double quotes. For Example:

```
1. String s="welcome";
```

Each time you create a string literal, the JVM checks the string constant pool first. If the string already exists in the pool, a reference to the pooled instance is returned. If string doesn't exist in the pool, a new string instance is created and placed in the pool. For example:

```
1. Strings1="Welcome";
2. String s2="Welcome";//will not create newinstance
```

By new keyword

```
1. String s=new String("Welcome");//creates two objects and one reference variable
```

In such case, JVM will create a new string object in normal (non pool) heap memory and the literal "Welcome" will be placed in the string constant pool. The variable s will refer to the object in heap (non pool).

Java String Example

```
public class StringExample{
public static void main(String args[]){
String s1="java";//creating string by java string literal
charch[]={'s','t','r','i','n','g','s'};
String s2=new String(ch);//converting char array to string
String s3=new String("example");//creating java string by new keywordSystem.out.println(s1);
System.out.println(s2);System.out.println(s3);
}}
```

OUTPUT:

java strings example

Immutable String in Java

In java, **string objects are immutable**. Immutable simply means unmodifiable or unchangeable.Once string object is created its data or state can't be changed but a new string object is created. Let's try to understand the immutability concept by the example given below:

```
class Testimmutablestring{
public static void main(String args[]){String s="Sachin";
s.concat(" Tendulkar");//concat() method appends the string at the end System.out.println(s);//will print Sachin because strings are immutable objects
}}
```

Output:Sachin

```
class Testimmutablestring1{
public static void main(String args[]){String s="Sachin";
s=s.concat(" Tendulkar");System.out.println(s);
} }
```

Output:Sachin Tendulkar

Unit-2

Inheritance in Java

Inheritance in java is a mechanism in which one object acquires all the properties and behaviors of parent object. Inheritance represents the **IS-A relationship**, also known as *parent-child*relationship.

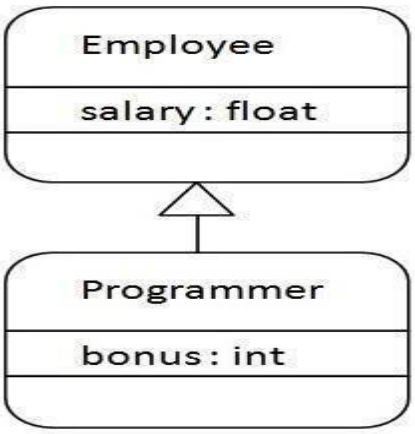
Why use inheritance in java

- For Method Overriding (so runtime polymorphism can beachieved).
- For CodeReusability.

Syntax of Java Inheritance

```
1. classSubclass-name extends Superclass-name
2. {
3.         //methods andfields
4. }
```

The **extends keyword** indicates that you are making a new class that derives from an existingclass. The meaning of "extends" is to increase the functionality.

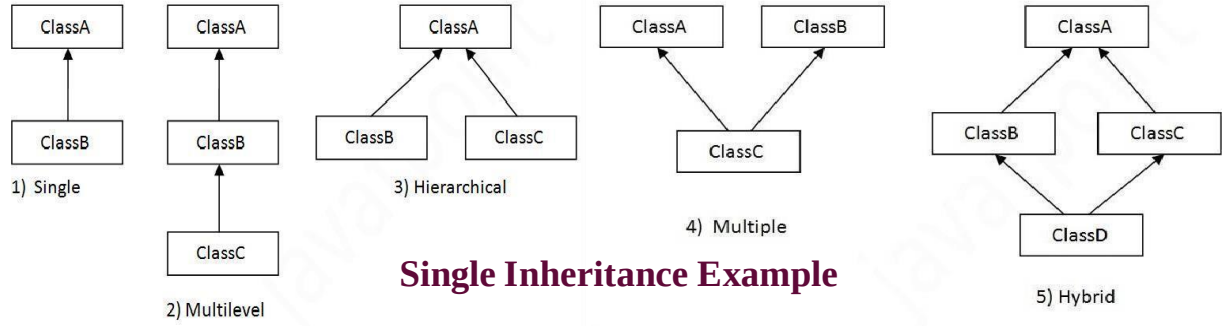



```
classEmployee{
float salary=40000;
 }
 classProgrammer extends Employee{
 int bonus=10000;
public static void main(String args[]){
Programmer p=new Programmer();
 System.out.println("Programmer salary is:"+p.salary);
 System.out.println("Bonus of Programmer is:"+p.bonus);
 } }
```

```
Programmer salary is:40000.0
```

Bonus of programmeris:10000

Types of inheritance in java



Single Inheritance Example

File: TestInheritance.java

```
classAnimal{ voideat(){System.out.println("eating...");}

classDog extends Animal{
voidbark(){System.out.println("barking...");}
}
classTestInheritance{
public static void main(String args[]){
Dog d=new Dog();
d.bark();
d.eat();
}}
```

Output: barking...eating...

Multilevel Inheritance Example

File: TestInheritance2.java

```
classAnimal{
voideat(){
System.out.println("eating...");
}
}
classDog extends Animal{
voidbark(){
System.out.println("barking...");
}
}
classBabyDog extends Dog{
voidweep(){
System.out.println("weeping...");
}
}
classTestInheritance2{
```

```
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}}
```

Output:

```
weeping...barking... eating...
```

Hierarchical Inheritance Example

File: TestInheritance3.java

```
classAnimal{
 voideat(){
System.out.println("eating...");
}
}
 classDog extends Animal{
 voidbark(){
 System.out.println("barking...");
 }
 }
 classCat extends Animal{
 voidmeow(){
 System.out.println("meowing...");
 }

 classTestInheritance3{
        public static void main(String args[]){
        Cat c=new Cat();
c.meow();
c.eat();
//c.bark();//C.T.Error
}}
```

Output:

```
meowing...eating...
```

Member access and Inheritance

A subclass includes all of the members of its super class but it cannot access those members ofthe super class that have been declared as private. Attempt to access a private variable would cause compilation error as it causes access violation. The variables declared as private, is onlyaccessible by other members of its own class. Subclass have no access to it.

super keyword in java

The **super** keyword in java is a reference variable which is used to refer immediate parent classobject.

Whenever you create the instance of subclass, an instance of parent class is created implicitlywhich is referred by super reference variable.

Usage of java super Keyword

1. super can be used to refer immediate parent class instancevariable.
2. super can be used to invoke immediate parent classmethod.
3. super() can be used to invoke immediate parent classconstructor.

super is used to refer immediate parent class instance variable.

```
classAnimal{ Stringcolor="white";
}
classDog extends Animal{Stringcolor="black";
voidprintColor(){ System.out.println(color);//printscolor of Dogclass
System.out.println(super.color);//prints color of Animal class
}
}
classTestSuper1{
public static void main(String args[]){Dog d=new Dog();
```

```
d.printColor();
}}
```

Output:

```
black
white
```

Final Keyword in Java

The **final keyword** in java is used to restrict the user. The java final keyword can be used in many context. Final can be:

1. variable
2. method
3. class

The final keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable. It can be initialized in the constructor only. The blank final variable can be static also which will be initialized in the static block only.

Object class in Java

The **Object class** is the parent class of all the classes in java by default. In other words, it is thetopmost class of java.

The Object class is beneficial if you want to refer any object whose type you don't know. Noticethat parent class reference variable can refer the child class object, know as upcasting.

Let's take an example, there is getObject() method that returns an object but it can be of any typelike Employee,Student etc, we can use Object class reference to refer that object. For example:

```
1. Object obj=getObject();//we don't know what object will be returned from thismethod
```

The Object class provides some common behaviors to all the objects such as object can becompared, object can be cloned, object can be notified etc.

Method Overriding in Java

If subclass (child class) has the same method as declared in the parent class, it is knownas **method overriding in java**.

Usage of Java Method Overriding

- Method overriding is used to provide specific implementation of a method that is already provided by its superclass.
- Method overriding is used for runtimepolymorphism

Rules for Java Method Overriding

1. method must have same name as in the parentclass
2. method must have same parameter as in the parentclass.
3. must be IS-A relationship(inheritance).

Example of method overriding

```
Class Vehicle{
voidrun(){System.out.println("Vehicle is running");}
}
classBike2 extends Vehicle{
voidrun(){System.out.println("Bike is running safely");}
public static void main(String args[]){
Bike2 obj = new Bike2();obj.run();
}
```

Output:Bike is running safely

```
1. classBank{
intgetRateOfInterest(){return 0;}

classSBI extends Bank{
intgetRateOfInterest(){return 8;}

 classICICI extends Bank{
intgetRateOfInterest(){return 7;}

classAXIS extends Bank{
intgetRateOfInterest(){return 9;}

classTest2{
public static void main(String args[]){
SBI s=new SBI();
ICICI i=new ICICI();
AXIS a=new AXIS();
System.out.println("SBI Rate of Interest: "+s.getRateOfInterest());
System.out.println("ICICI Rate of Interest: "+i.getRateOfInterest());
System.out.println("AXIS Rate of Interest: "+a.getRateOfInterest());
} }
```

```
Output:
SBI Rate of Interest: 8
```

ICICI Rate of Interest: 7AXIS Rate of Interest: 9

Abstract class in Java

A class that is declared with abstract keyword is known as abstract class in java. It can have abstract and non-abstract methods (method with body). It needs to be extended and its method implemented. It cannot be instantiated.

Example abstract class

```
1. abstract classA{}
```

abstractmethod

```
1. abstract void printStatus();//no body and abstract
```

Example of abstract class that has abstract method

```
abstract class Bike{
  abstract void run();
}

classHonda4 extends Bike{
voidrun(){System.out.println("running safely..");}
public static void main(String args[]){
Bike obj = new Honda4();obj.run();
}
1. }
```

```
runningsafely..
```

Interface in Java

An **interface in java** is a blueprint of a class. It has static constants and abstract methods.

The interface in java is **a mechanism to achieve abstraction**. There can be only abstract methodsin the java interface not method body. It is used to achieve abstraction and multiple inheritance in Java.

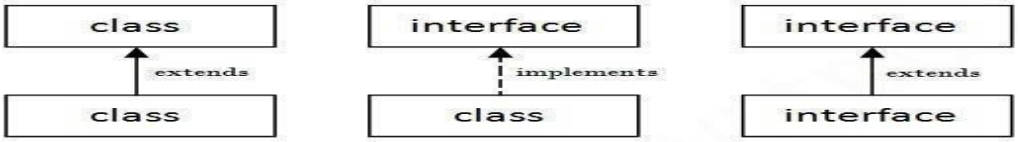
Java Interface also **represents IS-A relationship**.It cannot be instantiated just like abstract class.

There are mainly three reasons to use interface. They are given below.

- It is used to achieveabstraction.
- By interface, we can support the functionality of multipleinheritance.
- It can be used to achieve loosecoupling.

Internal addition by compiler

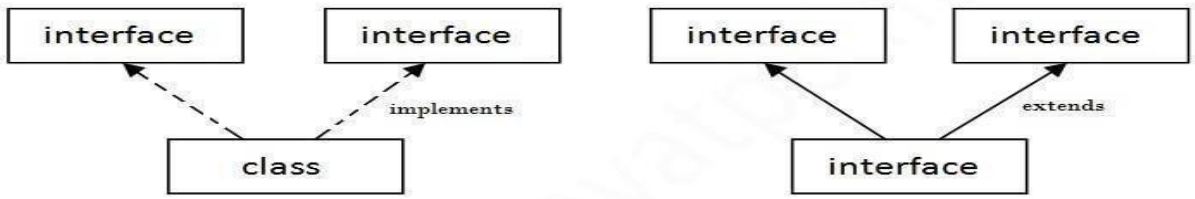
Understanding relationship between classes and interfaces




```
//Interface declaration: by first user
interfaceDrawable{
voiddraw();
}
//Implementation: by second user
classRectangle implements Drawable{
public void draw(){System.out.println("drawing rectangle");}
}
classCircle implements Drawable{
public void draw(){System.out.println("drawing circle");}
}
//Using interface: by third user
classTestInterface1{
public static void main(String args[]){
Drawable d=new Circle();//In real scenario, object is provided by method e.g. getDrawable()d.draw();
}}
```

Output:drawing circle

Multiple inheritance in Java by interface



Multiple Inheritance in Java

```
interfacePrintable{
```

```
voidprint();

interfaceShowable{
voidshow();

classA7 implements Printable,Showable{
public void print(){System.out.println("Hello");}
public void show(){System.out.println("Welcome");}
public static void main(String args[]){
A7 obj = new A7();obj.print();
obj.show();
} }
```

Output:Hello
Welcome

Abstract class	Interface
1) Abstract class can **have abstract and non-abstract**methods.	Interface can have **only abstract** methods. Since Java 8, it can have **default and static methods**also.
2) Abstract class **doesn't support multipleinheritance**.	Interface **supports multiple inheritance**.
3) Abstract class **can have final, non-final, static and non-static variables**.	Interface has **only static and final variables**.
4) Abstract class **can provide the implementation ofinterface**.	Interface **can't provide the implementation of abstract class**.
5) The **abstract keyword** is used to declare abstract class.	The**interface keyword** is used to declare interface.

6) **Example:**
public abstract class Shape{
public abstract void draw();
}
	Example:
	public interface Drawable{
	void draw();
	}

Java Inner Classes

Java inner class or nested class is a class which is declared inside the class or interface.

We use inner classes to logically group classes and interfaces in one place so that it can be morereadable and maintainable.

Syntax of Inner class

```
1. classJava_Outer_class{
2.  //code
3.  classJava_Inner_class{
4.   //code
5.  } }
```

Advantage of java inner classes

There are basically three advantages of inner classes in java. They are as follows:

1) Nested classes represent a special type of relationship that is **it can access all the members (data members and methods) of outer class** includingprivate.

2) Nested classes are used **to develop more readable and maintainable code** because it logically group classes and interfaces in one placeonly.

3) **Code Optimization**: It requires less code to write.

Difference between nested class and inner class in Java

Inner class is a part of nested class. Non-static nested classes are known as inner classes.

Types of Nested classes

There are two types of nested classes non-static and static nested classes.The non-static nestedclasses are also known as inner classes.

- Non-static nested class (innerclass)
  1. Member inner class
  2. Anonymous innerclass
  3. Local inner class
- Static nestedclass

Java Package

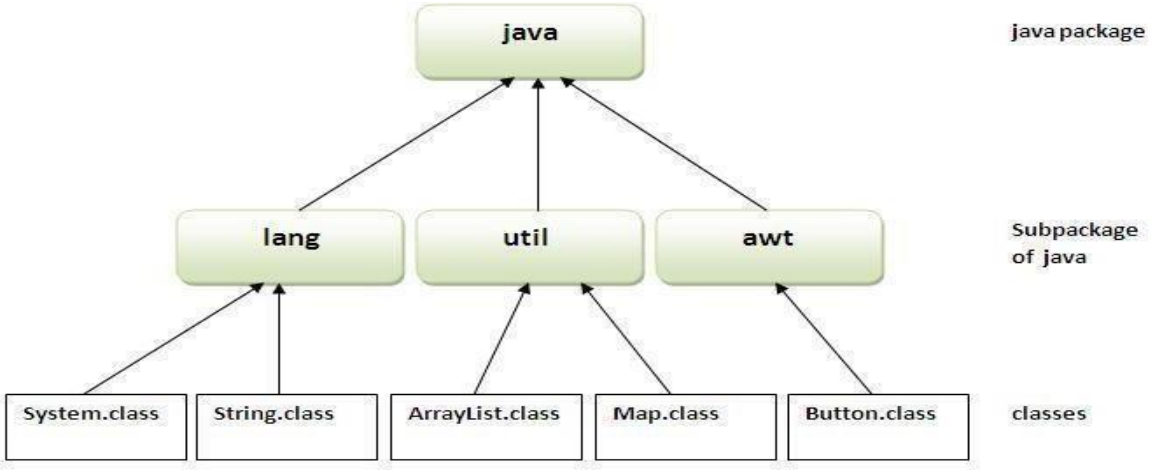
A **java package** is a group of similar types of classes, interfaces and sub-packages. Package in java can be categorized in two form, built-in package and user-defined package.There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.**Advantage of JavaPackage**

1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.

2) Java package provides accessprotection.

3) Java package removes namingcollision.

```
package mypack;
public class Simple{
 public static void main(String args[]){
   System.out.println("Welcome to package");
}}
```



How to compile java package
If you are not using any IDE, you need to follow the **syntax** given below:javac -d directory javafilename

How to run java package program

To Compile: javac -d . Simple.java

To Run: java mypack.Simple

Using fully qualified name

Example of package by import fully qualified name

```
//save by A.javapackage pack; public class A{
public void msg(){System.out.println("Hello");} }
//save by B.java package mypack;class B{
public static void main(String args[]){
pack.A obj = new pack.A();//using fully qualified nameobj.msg();
}
}
```

Output:Hello

UNIT-3

Exception Handling

The **exception handling in java** is one of the powerful *mechanism to handle the runtime errors* so that normal flow of the application can bemaintained.

What is exception

In java, exception is an event that disrupts the normal flow of the program. It is an object which isthrown at runtime.

Advantage of Exception Handling

The core advantage of exception handling is **to maintain the normal flow of the application**. Exception normally disrupts the normal flow of the application that is why we use exception handling.

Types of Exception

There are mainly two types of exceptions: checked and unchecked where error is considered asunchecked exception. The sun microsystem says there are three types of exceptions:

1. CheckedException
2. UncheckedException
3. Error

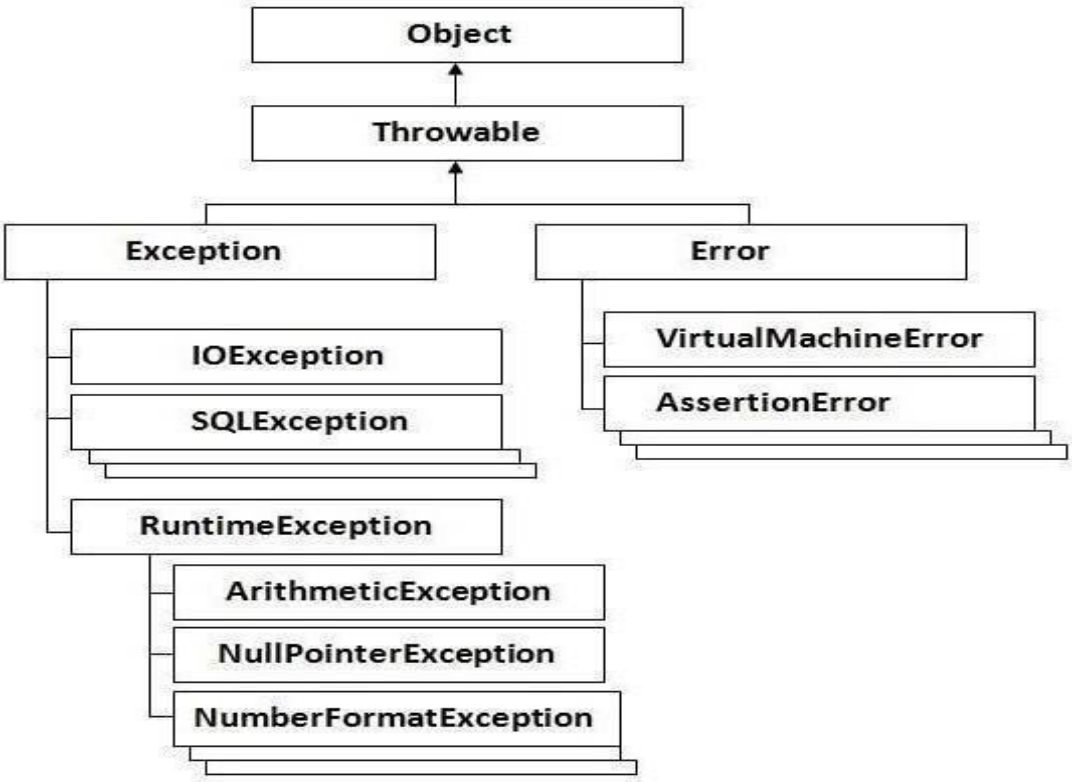
Difference between checked and unchecked exceptions

1) **Checked Exception:** The classes that extend Throwable class except RuntimeException and Error are known as checked exceptions e.g.IOException, SQLException etc. Checked exceptions are checked atcompile-time.

2) **Unchecked Exception:** The classes that extend RuntimeException are known as unchecked exceptions e.g. ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException etc. Unchecked exceptions are not checked at compile-time rather they are checkedatruntime.

3) **Error:** Error is irrecoverable e.g. OutOfMemoryError, VirtualMachineError, AssertionErroretc.

Hierarchy of Java Exception classes



Checked and UnChecked Exceptions

Checked Exceptions	Unchecked Exceptions
• Exception which are checked at Compile time called Checked Exception • If a method throws a checked exception, then the method must either handle the exception or it must specify the exception using throws keyword	• Exceptions whose handling is NOT verified during Compile time. • These exceptions are handled at run-time i.e., by JVM after they occurred by using the try and catch block
• Examples: ○ IOException ○ SQLException ○ DataAccessException ○ ClassNotFoundException ○ InvocationTargetException ○ MalformedURLException	• Examples ○ NullPointerException ○ ArrayIndexOutOfBound ○ IllegalArgumentException ○ IllegalStateException

Java try block

Java try block is used to enclose the code that might throw an exception. It must be used within the method.

Java try block must be followed by either catch or finally block.

Syntax of java try-catch

```
1. try{
2. //code that may throwexception
3. }catch(Exception_class_Nameref){}Syntax of try-finallyblock
1. try{
2. //code that may throwexception
3. }finally{}
```

Java catch block

Java catch block is used to handle the Exception. It must be used after the try block only. You can use multiple catch block with a single try.

Problem without exception handling

Let's try to understand the problem if we don't use try-catch block.

```
public class Testtrycatch1{
public static void main(String args[]){ int data=50/0;//may throw exception System.out.println("rest of thecode...");
} }
```

Output:

```
Exception in thread main java.lang.ArithmeticException:/ byzero
```

As displayed in the above example, rest of the code is not executed (in such case, rest of the code... statement is not printed).

There can be 100 lines of code after exception. So all the code after exception will not beexecuted.

Solution by exception handling

Let's see the solution of above problem by java try-catch block.

```
public class Testtrycatch2{
```

```
public static void main(String args[]){
try{
intdata=50/0;
}catch(ArithmeticException e){System.out.println(e);}System.out.println("rest of the code...");
} }
```

1. Output:

```
Exception in thread main java.lang.ArithmeticException:/ by zerorest of the code...
```

Now, as displayed in the above example, rest of the code is executed i.e. rest of the code... statement is printed.

Java Multi catch block

If you have to perform different tasks at the occurrence of different Exceptions, use java multicatch block.

Let's see a simple example of java multi-catch block.

```
public classTestMultipleCatchBlock{
public static void main(Stringargs[]){
try{
inta[]=new int[5];
a[5]=30/0;
}
catch(ArithmeticException e){System.out.println("task1 iscompleted");}
catch(ArrayIndexOutOfBoundsException e){System.out.println("task 2 completed"); }
catch(Exception e){System.out.println("common taskcompleted");
 }
System.out.println("rest of the code...");13. } }
```

```
Output:task1 completedrest of the code...
```

Java nested try example

Let's see a simple example of java nested try block.

```
class Excep6{
public static void main(String args[]){
try{try{
System.out.println("going to divide");
intb =39/0;
}catch(ArithmeticException e){System.out.println(e);}
```

```
try{
   int a[]=new
   int[5];
   a[5]=4;
}catch(ArrayIndexOutOfBoundsException e){
System.out.println(e);
}
System.out.println("other statement);
}catch(Exception e){
System.out.println("handeled");
}
System.out.println("normal flow..");
} }
```

Java finally block

Java finally block is a block that is used *to execute important code* such as closing connection, stream etc.

Java finally block is always executed whether exception is handled or not.Java finally block follows try or catch block.

Usage of Java finally

Case 1

Let's see the java finally example where **exception doesn't occur**.

```
class TestFinallyBlock{
public static void main(String args[]){
try{ intdata=25/5;
System.out.println(data);
}
catch(NullPointerException e){System.out.println(e);} finally{System.out.println("finally block is always executed");}System.out.println("rest of the code...");
}
}
```

```
Output:5
finally block is always executedrest of the code...
```

Java throw keyword

The Java throw keyword is used to explicitly throw an exception.

We can throw either checked or uncheked exception in java by throw keyword. The throw keyword is mainly used to throw custom exception. We will see custom exceptions later.

The syntax of java throw keyword is given below.

1. **throw**exception;

Java throw keyword example

In this example, we have created the validate method that takes integer value as a parameter. Ifthe age is less than 18, we are throwing the ArithmeticException otherwise print a message welcome to vote.

```
 public class TestThrow1{
static void validate(int age){
if(age<18)
throw new ArithmeticException("not valid");
else
System.out.println("welcome to vote");
}
public static void main(String args[]){validate(13);
System.out.println("rest of the code...");
}}
```

Output:

```
Exception in thread main java.lang.ArithmeticException:notvalid
```

Java throws keyword

The **Java throws keyword** is used to declare an exception. It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.

Exception Handling is mainly used to handle the checked exceptions. If there occurs any unchecked exception such as NullPointerException, it is programmers fault that he is not performing check up before the code being used.

Syntax of java throws

```
1. return_type method_name() throwsexception_class_name{
   2.  //methodcode3. }
   4.
```

Java throws example

Let's see the example of java throws clause which describes that checked exceptions can be propagated by throws keyword.

```
import java.io.IOException;
class Testthrows1{
void m() throwsIOException{
throw new IOException("device error");//checked exception
```

```
}
        Void n() throws
          IOException{m();
}
        voidp(){
         try{
         n();
}catch(Exception e){System.out.println("exception handled");}
}
        public static void main(String args[]){
         Testthrows1 obj=new Testthrows1();
         obj.p();
System.out.println("normal flow..."); } }Output:
```

```
exception handlednormal flow...
```

Java Custom Exception

If you are creating your own Exception that is known as custom exception or user-defined exception. Java custom exceptions are used to customize the exception according to user need.

By the help of custom exception, you can have your own exception and message.Let's see a simple example of java custom exception.

```
class InvalidAgeException extends Exception{InvalidAgeException(String s){super(s);
} }
class TestCustomException1{
static void validate(int age)throws InvalidAgeException{
if(age<18)
throw new InvalidAgeException("not valid");
else
System.out.println("welcome to vote");
}
public static void main(String args[]){
try{ validate(13);
}catch(Exception m){System.out.println("Exception occured: "+m);}

System.out.println("rest of the code...");
}}
```

Output:Exception occured: InvalidAgeException:not valid rest of the code...

Multithreading

Multithreading in java is a process of executing multiple threads simultaneously.

Thread is basically a lightweight sub-process, a smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.

But we use multithreading than multiprocessing because threads share a common memory area. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.

Java Multithreading is mostly used in games, animation etc.

Advantages of Java Multithreading

1) It **doesn't block the user** because threads are independent and you can performmultiple operations at sametime.

2) You **can perform many operations together so it savestime**.

3) Threads are **independent** so it doesn't affect other threads if exception occur in a singlethread.

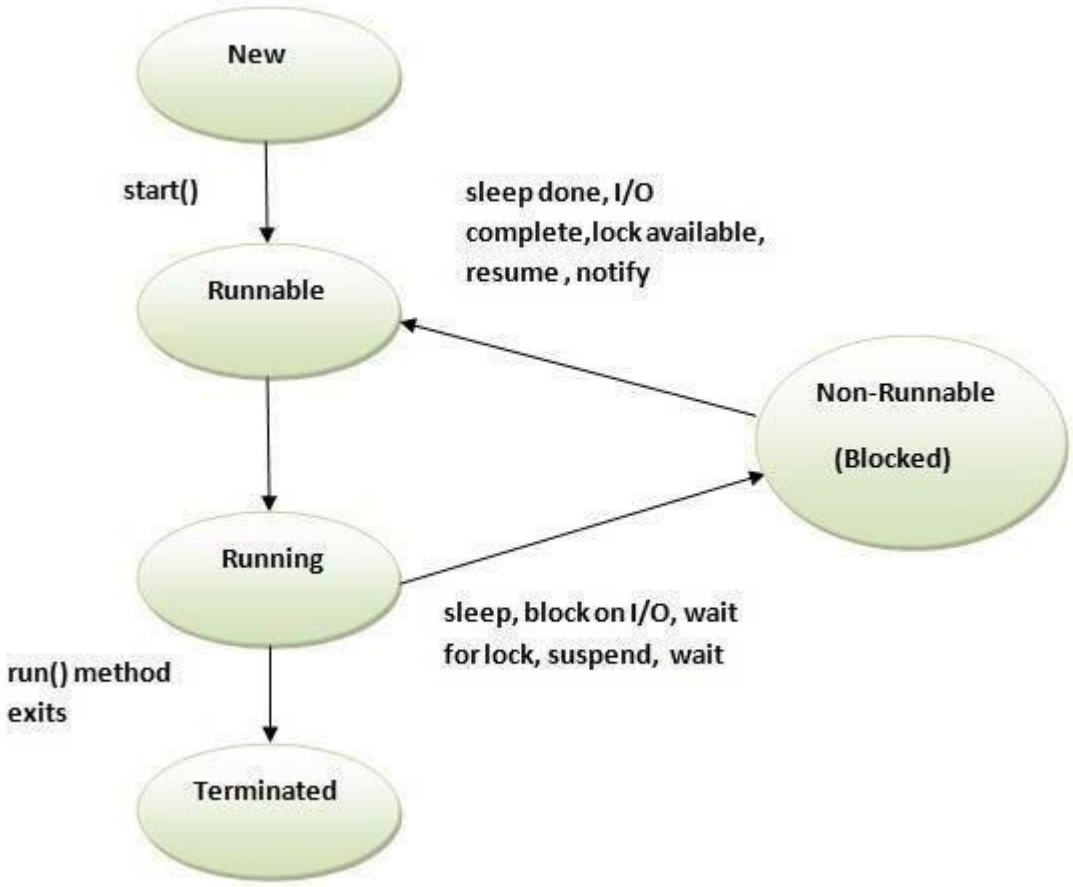
Life cycle of a Thread (Thread States)

A thread can be in one of the five states. According to sun, there is only 4 states in **thread lifecycle in java** new, runnable, non-runnable and terminated. There is no running state.

But for better understanding the threads, we are explaining it in the 5 states.

The life cycle of the thread in java is controlled by JVM. The java thread states are as follows:

1. New
2. Runnable
3. Running
4. Non-Runnable(Blocked)
5. Terminated



How to create thread

There are two ways to create a thread:

1. By extending Threadclass
2. By implementing Runnableinterface.

Thread class:

Thread class provide constructors and methods to create and perform operations on a thread.
Thread class extends Object class and implements Runnable interface.

Commonly used Constructors of Thread class:

○Thread() ○Thread(String name)○Thread(Runnable r)
○Thread(Runnable r,String name)

Commonly used methods of Thread class:

1. **public void run():** is used to perform action for athread.
2. **public void start():** starts the execution of the thread.JVM calls the run() method onthethread.
3. **public void sleep(long miliseconds):** Causes the currently executing thread to sleep (temporarily cease execution) for the specified number ofmilliseconds.
4. **public void join():** waits for a thread todie.
5. **public void join(long miliseconds):** waits for a thread to die for the specifiedmiliseconds.
6. **public int getPriority():** returns the priority of thethread.
7. **public int setPriority(int priority):** changes the priority of thethread.
8. **public String getName():** returns the name of thethread.
9. **public void setName(String name):** changes the name of the thread.
10. **public Thread currentThread():** returns the reference of currently executingthread.
11. **public int getId():** returns the id of thethread.
12. **public Thread.State getState():** returns the state of thethread.
13. **public boolean isAlive():** tests if the thread isalive.
14. **public void yield():** causes the currently executing thread object to temporarily pause and allow other threads toexecute.
15. **public void suspend():** is used to suspend thethread(depricated).
16. **public void resume():** is used to resume the suspendedthread(depricated).
17. **public void stop():** is used to stop thethread(depricated).
18. **public boolean isDaemon():** tests if the thread is a daemonthread.
19. **public void setDaemon(boolean b):** marks the thread as daemon or userthread.
20. **public void interrupt():** interrupts thethread.
21. **public boolean isInterrupted():** tests if the thread has beeninterrupted.
22. **public static boolean interrupted():** tests if the current thread has beeninterrupted.

Runnable interface:

The Runnable interface should be implemented by any class whose instances are intended to beexecuted by a thread. Runnable interface have only one method named run().

1. **public void run():** is used to perform action for athread.

Starting a thread:

start() method of Thread class is used to start a newly created thread. It performs followingtasks:

- A new thread starts(with new callstack).
- The thread moves from New state to the Runnable state.
- When the thread gets a chance to execute, its target run() method will run.

Java Thread Example by extending Thread class

```
class Multi extends Thread{
public void run(){ System.out.println("thread is running...");
}
public static void main(String args[]){
Multi t1=new Multi();
t1.start();
} }
```

Output:thread isrunning...

Java Thread Example by implementing Runnable interface

```
class Multi3 implements Runnable{
public void run(){
 System.out.println("thread is running...");
}
public static void main(String args[]){
Multi3 m1=new Multi3();
Thread t1 =new Thread(m1);t1.start();
} }
```

Output:thread isrunning...

Priority of a Thread (Thread Priority):

Each thread have a priority. Priorities are represented by a number between 1 and 10. In mostcases, thread schedular schedules the threads according to their priority (known as preemptivescheduling). But it is not guaranteed because it depends on JVM specification that which scheduling it chooses.

3 constants defined in Thread class:

1. public static intMIN_PRIORITY
2. public static intNORM_PRIORITY
3. public static intMAX_PRIORITY

Default priority of a thread is 5 (NORM_PRIORITY). The value of MIN_PRIORITY is 1 andthe value of MAX_PRIORITY is 10.

Example of priority of a Thread:

```
class TestMultiPriority1 extends
Thread{public void run(){
System.out.println("running thread name is:"+Thread.currentThread().getName());
 System.out.println("running thread priority is:"+Thread.currentThread().getPriority());

 public static void main(String args[]){
```

```
TestMultiPriority1 m1=new TestMultiPriority1();
TestMultiPriority1 m2=new TestMultiPriority1();
m1.setPriority(Thread.MIN_PRIORITY);
m2.setPriority(Thread.MAX_PRIORITY);
 m1.start();
m2.start();
} }
```

Output:running thread name is:Thread-0running thread priority is:10
running thread name is:Thread-1running thread priority is:1

Java synchronized method

If you declare any method as synchronized, it is known as synchronized method.Synchronized method is used to lock an object for any shared resource.
When a thread invokes a synchronized method, it automatically acquires the lock for that objectand releases it when the thread completes its task.

UNIT 4

APPLETS:

An applet is a program that comes from server into a client and gets executed at client side anddisplaysthe result.

An applet represents byte code embedded in a html page. (Applet = bytecode + html) and runwith thehelp of Java enabled browsers such as Internet Explorer.

An applet is a Java program that runs in a browser. Unlike Java applications applets do not havea main() method.

To create applet we can use java.applet.Applet or javax.swing.JApplet class. All applets inherit the super class 'Applet'. An Applet class contains several methods that help to control the execution of an applet.

Advantages:

1. Applets provide dynamic nature for a webpage.
2. Applets are used in developing games and animations.
3. Writing and displaying (browser) graphics and animations is easier thanapplications.
4. In GUI development, constructor, size of frame, window closing code etc. are notrequired

Restrictions of Applets of Applets Vs Applications

1. Applets are required separate compilation before opening in a browser.
2. In realtime environment, the bytecode of applet is to be downloaded from the server to theclient machine.
3. Applets are treated as **untrusted** (as they were developed by unknown people and placed onunknown servers whose trustworthiness is not guaranteed).
4. Extra Code is required to communicate between applets using **AppletContext**.

DIFFERENCES BETWEEN APPLETS AND APPLICATIONS

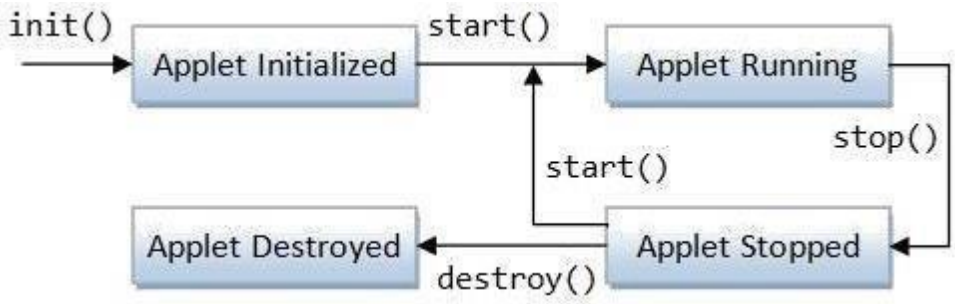
FEATURE	APPLICATION	APPLET
main() method	main() method Present	main() method Not present
Execution	Can be executed on standalone computer system. (JDK & JRE)	Used to run a program on client Browser like Chrome.
Nature	Called as stand-alone application as application can be executed from command prompt	Requires some third party tool help like a browser to execute
Restrictions	Can access any data or software available on the system	Cannot access anything on the system except browser"s services
Security	Does not require any security	Requires highest security for the system as they are untrusted
Programming	larger programs	small programs
Platform	platform independent	platform independent
Accessibility	The java applications are designed to work with the	Applets are designed just for handling the client site

	client as well as server.	problems.
Working	Applications are created by writing public static void main(String[] s) method	Applets are created by extending the java.applet.Applet class
Client side / Server side	The applications don't have such type of criteria	Applets are designed for the client site programming purpose
Methods	Application has a single start point which is main method	Applet application has 5 methods which will be automatically invoked.

Example	public class MyClass
	{ public static void main(String args[]){}
	}
		import java.awt.*;
		import java.applet.*;
		public class Myclass extends Applet{
		public void init(){}
		public void start(){}
		public void stop(){}
		public void destroy() {}
		public void paint(Graphics g) {}
		}

LIFE CYCLE OF AN APPLET

Let the Applet class extends Applet or JApplet class.



Initialization:

public void init(): This method is used for initializing variables, parameters to create components. This method is executed only once at the time of applet loaded into memory.

```
public void init(){
//initialization
}
```

Runnning:

public void start (): After init() method is executed, the start method is executed automatically. Start method is executed as long as applet gains focus. In this method code related to opening files and connecting to database and retrieving the data and processingthe data is written.

Idle / Runnable:

public void stop (): This method is executed when the applet loses focus. Code related to closing the files and database, stopping threads and performing clean up operations are written in this stop method.

Dead/Destroyed:

public void destroy (): This method is executed only once when the applet is terminatedfrom the memory.

Executing above methods in that sequence is called applet life cycle. We can also use **public void paint (Graphics g)** in applets.

//An Applet skeleton.

```
import java.awt.*;
import java.applet.*;
/*
<applet code="AppletSkel" width=300 height=100>
</applet>
*/
public class AppletSkel extends Applet {
public voidinit() {
      //initialization
}
/* Called second, after init(). Also called whenever the applet is restarted. */
public void start() {
   //start or resume execution
}
/* Called when the applet is stopped. */
public void stop() {
   //suspends execution
}
/* Called when applet is terminated. This is the last method executed. */
public void destroy() {
      //perform shutdown activities
}
//Called when an applet's window must berestored.
public void paint(Graphics g) {
      //redisplay contents of window
}
}
```

After writing an applet, an applet is compiled in the same way as Java application but running ofan applet is different.

There are two ways to run an applet.

- Executing an applet within a Java compatible web browser.

Executing an applet using 'appletviewer'. This executes the applet in a window.

To execute an applet using web browser, we must write a small HTML file which contains the appropriate „APPLET" tag. <APPLET> tag is useful to embed an applet into an HTML page. It has the following form:

–

<APPLET CODE=”name of the applet class file” HEIGHT = maximum height of applet in pixels WIDTH = maximum width of applet in pixels ALIGN = alignment (LEFT, RIGHT, MIDDLE, TOP, BOTTOM)>
<PARAM NAME = parameter name VALUE = its value> </APPLET>

Execution: ***appletviewer programname.java or appletviewer programname.html***

The <PARAM> tag useful to define a variable (parameter) and its value inside the HTML page which can be passed to the applet. The applet can access the parameter value using **getParameter () method**, as: String value = getParameter (“pname”);

Example Program:

Following is a simple applet named **HelloWorldApplet.java** –

```
import java.applet.*;
import java.awt.*;
public class HelloWorldApplet extends Applet {
public void paint (Graphics g) {
g.drawString ("Hello World", 25, 50);
} }
```

Invoking an Applet - HelloWorldApplet.html

```
<html>
<title>The Hello, World Applet</title>
<applet code = "HelloWorldApplet.class" width = "320" height = "120">
</applet>
</html>
```

OUTPUT: javac HelloWorldApplet. java appletviewer HelloWorldApplet.html

```
//First.java
import java.applet.Applet;
import java.awt.Graphics;
/*
<applet code="First.class" width="300" height="300">
</applet>
*/
public class First extends Applet
{
public void paint(Graphics g){ g.drawString("welcome to applet",150,150);
}
}
```

OUTPUT: javac First.java
appletviewer First.java

TYPES OF APPLETS

Applets are of two types:
// Local Applets
// Remote Applets

Local Applets: An applet developed locally and stored in a local system is called local applets. So, local system does not require internet. We can write our own applets and embed them into the web pages.

Remote Applets: The applet that is downloaded from a remote computer system and embed applet into a web page. The internet should be present in the system to download the applet and run it. To download the applet we must know the applet address on web known as Uniform Resource Locator(URL) and must be specified in the applets HTML document as the value of CODEBASE.

PASSING PARAMETERS TO AN APPLE

Java applet has the feature of retrieving the parameter values passed from the html page. So, youto the applet embedded in your page. The **param** can pass the parameters from your html page.

tag(**<parma name="" value=""></param**>) is used to pass the parameters to an applet. Theapplet has to call the getParameter() method supplied by the java.applet.Applet parent class.

Ex1: Write a program to pass employ name and id number to an applet.

```
import java.applet.*;
import java.awt.*;
/* <applet code="MyApplet2.class" width = 600 height= 450>
<param name = "t1" value="Hari Prasad"> <param name =
"t2" value ="101">
</applet> */

public class MyApplet2 extends Applet
{
String n;
String id;
public void init()
{
  n = getParameter("t1");
  id = getParameter("t2");
}
public void paint(Graphics g)
{
                    rawString("Name is : "
                    + n, 100,100);
                    g.drawString("Id is :
                    "+ id, 100,150);
}
}
```

Ex2: Write a program to pass two numbers and pass result to an applet.

```
import java.awt.*;
import java.applet.*;
  /*<APPLET code="Pp" width="300"
height="250"> <PARAM name="a" value="5">
<PARAM name="b" value="5">
</APPLET>*/
public class Pp extends Applet
{
```

```
      String  str;
      int a,b,result;
public void init()
 {
           str=getParameter("a");
              a=Integer.parseInt(str);
          str=getParameter("b");

             b=Integer.parseInt(str);
          result=a+b;
          str=String.valueOf(result);
       }
  public void paint(Graphics g)
   {
        g.drawString(" Result of Addition is : "+str,0,15);
    }
}
```

Ex3: Hai.java

```
import java.applet.*;
import java.awt.*;
/*<Applet code="hai" height="250" width="250">
   <PARAM name="Message" value="Hai friend how are you ..?"></APPLET>
*/
 class hai extends Applet
  {
    private String defaultMessage = &amp;quot;Hello!&amp;quot;;
   public void paint(Graphics g) {

    String inputFromPage = this.getParameter(&amp;quot;Message&amp;quot;); if
    (inputFromPage == null) inputFromPage = defaultMessage;
    g.drawString(inputFromPage, 50, 55);
  }
}
```

Output:

EVENT HANDLING

Event handling is at the core of successful applet programming. Most events to which the applet will respond are generated by the user. The most commonly handled events are those generated by the mouse, the keyboard, and various controls, such as a push button.

Events are supported by the **java.awt.event** package.

The Delegation Event Model

The modern approach to handling events is based on the delegation event model, whichdefines standard and consistent mechanisms to generate and process events.

Its concept is quite simple: a source generates an event and sends it to one or morelisteners. In this scheme, the listener simply waits until it receives an event. Once received, the listener processes the event and then returns.

The advantage of this design is that the application logic that processes events is cleanly separated from the user interface logic that generates those events. A user interface element is able to "delegate" the processing of an event to a separate piece of code.

In the delegation event model, listeners must register with a source in order to receive an event notification. This provides an important benefit: notifications are sent only to listeners that want to receive them.

EVENTS

In the delegation model, an event is an object that describes a state change in a source. It can be generated as a consequence of a person interacting with the elements in a graphical user interface. Some of the activities that cause events to be generated are pressing a button, enteringa character via the keyboard, selecting an item in a list, and clicking the mouse.

Events may also occur that are not directly caused by interactions with a user interface.

For example, an event may be generated when a timer expires, a counter exceeds a value, software or hardware failure occurs, or an operation is completed.

EVENT SOURCES

A source is an object that generates an event. This occurs when the internal state of that object changes in some way. Sources may generate more than one type of event. A source must register listeners in order for the listeners to receive notifications about a specific type of event. Each type of event has its own registration method.

Here is the general form:

public void add Type Listener(Type Listener el)

EVENT LISTENERS

A listener is an object that is notified when an event occurs. It has two major requirements. First,it must have been registered with one or more sources to receive notifications about specific types of events. Second, it must implement methods to receive and process these notifications. The methods that receive and process events are defined in a set of interfaces found in **java.awt.event**.

For example, the MouseMotionListener interface defines two methods to receive notifications when the mouse is dragged or moved.

EVENT CLASSES

The classes that represent events are at the core of Java's event handling mechanism. At the root of the Java event class hierarchy is EventObject, which is in **java.util**. It is the superclass for all events.

It's one constructor is shown here:

EventObject(Object src)

EventObject contains two methods: getSource( ) and toString( ) .

The **getSource( ) method** returns the source of the event. **Ex:** Object getSource( )

toString( ) returns the string equivalent of the event.

The package **java.awt.event** defines several types of events that are generated by various userinterface elements.

Event Class	Description
ActionEvent	Generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
AdjustmentEvent	Generated when a scroll bar is manipulated.
ComponentEvent	Generated when a component is hidden, moved, resized or becomes visible.
ContainerEvent	Generated when a component is added to or removed from a container.
FocusEvent	Generated when a component gains or loses keyboard focus.
InputEvent	Abstract super class for all component input event classes.
ItemEvent	Generated when a check box or list item is clicked; so occurs when a choice selection is made or a checkable menu item is selected or deselected.
KeyEvent	Generated when input is received from the keyboard.
MouseEvent	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
MouseWheelEvent	Generated when the mouse wheel is moved. (Added by Java 2, version 1.4)
TextEvent	Generated when the value of a text area or text field is changed.
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

The ActionEvent Class

An ActionEvent is generated when a button is pressed, a list item is double-clicked, or a menuitem is selected. The ActionEvent class defines four integer constants that can be used to identify any modifiers associated with an action event: ALT_MASK , CTRL_MASK , META_MASK , and SHIFT_MASK .

ActionEvent has these three constructors: ActionEvent(Object src , int type , String cmd)

ActionEvent(Object src , int type , String cmd , int modifiers)

ActionEvent(Object src , int type, String cmd, long when , int modifiers)

The ComponentEvent Class

// ComponentEvent is generated when the size, position, or visibility of a component is changed. There are four types of component events. The constants and their meanings areshown here:
COMPONENT_HIDDEN The component was hidden. COMPONENT_MOVED The component was moved. COMPONENT_RESIZED The component was resized. COMPONENT_SHOWN The component became visible.

The ContainerEvent Class

//ContainerEvent is generated when a component is added to or removed from a container.
There are two types of container events.
→
COMPONENT_ADDED and
COMPONENT_REMOVED **The KeyEvent Class**
//KeyEvent is generated when keyboard input occurs. There are three types of key events, which are identified by these integer constants: KEY_PRESSED, KEY_RELEASED, and KEY_TYPED .
The first two events are generated when any key is pressed or released. The last event occurs only when a character is generated.

The MouseEvent Class

There are **eight types of mouse events**. The MouseEvent class defines the following integer constants that can be used to identify them:

MOUSE_CLICKED	The user clicked the mouse.
MOUSE_DRAGGED	The user dragged the mouse.
MOUSE_ENTERED	The mouse entered a component.
MOUSE_EXITED	The mouse exited from a component.
MOUSE_MOVED	The mouse moved.
MOUSE_PRESSED	The mouse was pressed.
MOUSE_RELEASED	The mouse was released.
MOUSE_WHEEL	The mouse wheel was moved (Java 2, v1.4).

The WindowEvent Class

There are **ten types of window events**. The WindowEvent class defines integer constants thatcan be used to identify them. The constants and their meanings are shown here:

WINDOW_ACTIVATED	The window was activated.
WINDOW_CLOSED	The window has been closed.
WINDOW_CLOSING	The user requested that the window be closed.
WINDOW_DEACTIVATED	The window was deactivated.
WINDOW_DEICONIFIED	The window was deiconified.
WINDOW_GAINED_FOCUS	The window gained input focus.
WINDOW_ICONIFIED	The window was iconified.
WINDOW_LOST_FOCUS	The window lost input focus.
WINDOW_OPENED	The window was opened.
WINDOW_STATE_CHANGED	The state of the window changed.

EVENT LISTENER INTERFACES

When an event occurs, the event source invokes the appropriate method defined by the listenerand provides an event object as its argument

Interface	Description
ActionListener	Defines one method to receive action events.
AdjustmentListener	Defines one method to receive adjustment events.
ComponentListener	Defines **four methods** to recognize when a component is hidden, moved, resized, or shown.
ContainerListener	Defines **two methods** to recognize when a component is added to or removed from a container.
FocusListener	Defines **two methods** to recognize when a component gains or losses keyboard focus.
ItemListener	Defines one method to recognize when the state of an item changes.
KeyListener	Defines **three methods** to recognize when a key is pressed, released, or typed.
MouseListener	Defines **five methods** to recognize when the mouse is clicked, enters a component, exits a component, is pressed, or is released.
MouseMotionListener	Defines **two methods** to recognize when the mouse is dragged or moved.
MouseWheelListener	Defines one method to recognize when the mouse wheel is moved.
TextListener	Defines one method to recognize when a text value changes.
WindowListener	Defines **seven methods** to recognize when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

The delegation event model has two parts: sources and listeners. Listeners are created by implementing one or more of the interfaces defined by the **java.awt.event** package.

The ActionListener Interface

This interface defines the actionPerformed( ) method that is invoked when an action eventoccurs.

Its general form is shown here: void actionPerformed(ActionEvent ae)

The ItemListener Interface

This interface defines the itemStateChanged( ) method that is invoked when the state of an itemchanges.

Its general form is shown here: void itemStateChanged(ItemEvent ie)

The KeyListener Interface

This interface defines three methods. The keyPressed( ) and keyReleased( ) methods are invokedwhen a key is pressed and released, respectively. The keyTyped( ) method is invoked when a character has been entered. For example, if a user presses and releases the key, three events are generated in A sequence: key pressed, typed, and released.

The general forms of these methods are shown here:

void keyPressed(KeyEvent ke)
void keyReleased(KeyEvent ke)
void keyTyped(KeyEvent ke)

The MouseListener Interface

This interface defines five methods. If the mouse is pressed and released at the same point, mouseClicked( ) is invoked. When the mouse enters a component, the mouseEntered( ) method is called. When it leaves, mouseExited( ) is called. The mousePressed( ) and mouseReleased( ) methods are invoked when the mouse is pressed and released, respectively.

The general forms of these methods are shown here:

void mouseClicked(MouseEvent me)
void mouseEntered(MouseEvent me)
void mouseExited(MouseEvent me)
void mousePressed(MouseEvent me)
void mouseReleased(MouseEvent me)

The MouseMotionListener Interface

This interface defines two methods.
The mouseDragged( ) method is called multiple times as themouse is dragged.
The mouseMoved( ) method is called multiple times as the mouse is moved.

Their general forms are shown here:

void mouseDragged(MouseEvent me)
void mouseMoved(MouseEvent me)

The TextListener Interface

This interface defines the textChanged( ) method that is invoked when a change occurs in a textarea or text field.

Its general form is shown here: void textChanged(TextEvent te)

Handling Mouse Events

To handle mouse events, we must implement the MouseListener and the MouseMotion Listenerinterfaces.

EX: // Demonstrate the mouse event handlers.

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="MouseEvents" width=300 height=100>
</applet>
*/
public class MouseEvents extends Applet implements MouseListener, MouseMotionListener
{
 String msg = "";
int mouseX = 0, mouseY = 0; // coordinates of mouse
public void init() {
addMouseListener(this);
 addMouseMotionListener(this);
}
```

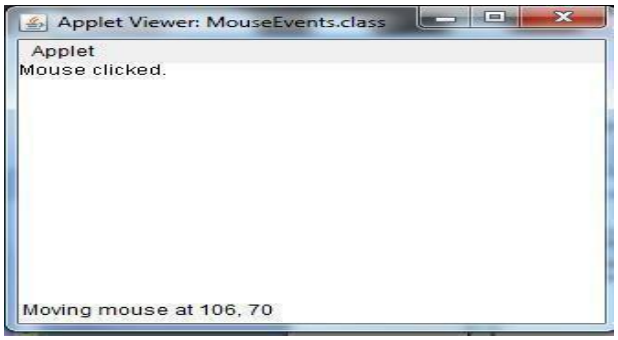
```
// Handle mouse clicked.
public void mouseClicked(MouseEvent me) {
mouseX = 0;      // save coordinates
mouseY = 10;
msg = "Mouse clicked.";
repaint();
}

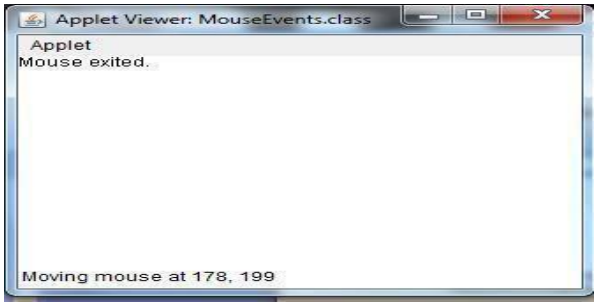
// Handle mouse entered.
public void mouseEntered(MouseEvent me) {
   // save
      coordinates
      mouseX = 0;
      mouseY = 10;
msg = "Mouse entered.";
repaint();
}

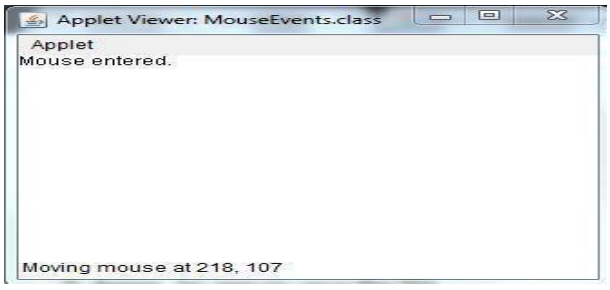
   // Handle mouse exited.
public void mouseExited(MouseEvent me) {
       // save
          coordinate
          s mouseX
          = 0;
          mouseY =
          10;
msg = "Mouse exited.";
repaint();
}

       // Handle button pressed.
public void mousePressed(MouseEvent me) {
// save coordinates
   mouseX =
   me.getX();
   mouseY =
   me.getY(); msg =
   "Down"; repaint();
}

// Handle button released.
public void mouseReleased(MouseEvent me) {
   1. save
      coordinates
      mouseX =
      me.getX();
      mouseY =
      me.getY(); msg
      = "Up";
      repaint();
}
```







```
  Handle mouse dragged.
public void mouseDragged(MouseEvent me) {
//save coordinates
mouseX = me.getX();
mouseY = me.getY();msg = "*";
showStatus("Dragging mouse at " + mouseX + ", " + mouseY);repaint();
}
// Handle mouse moved.
public void mouseMoved(MouseEvent me) {
// show status
showStatus("Moving mouse at " + me.getX() + ", " + me.getY());
}
// Display msg in applet window at current X,Y location.public void paint(Graphics g) {
g.drawString(msg, mouseX, mouseY);
}
}
```

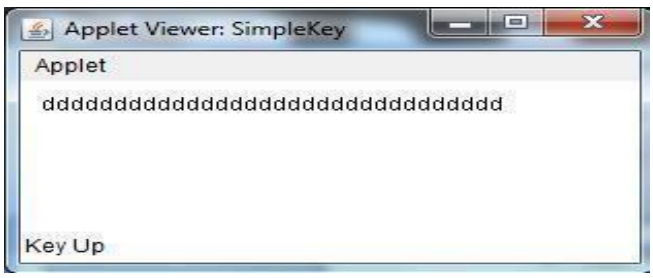
Handling Keyboard Events

When a key is pressed, a **KEY_PRESSED** event is generated. This results in a call to the keyPressed( ) event handler. When the key is released, a **KEY_RELEASED** event is generated and the keyReleased( ) handler is executed. If a character is generated by the keystroke, then a **KEY_TYPED** event is sent and the keyTyped( ) handler is invoked.

Thus, each time the user presses a key, at least two and often three events are generated. If all you care about are actual characters, then you can ignore the information passed by the key pressand release events.

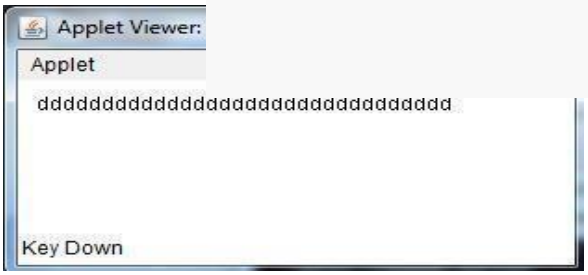
EX: // Demonstrate the key event handlers.

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/* <applet code="SimpleKey" width=300height=100> </applet> */
public class SimpleKey extends Applet implements KeyListener
{
String msg = "";
int X = 10, Y = 20; // output coordinates
public void init() {
 addKeyListener(this);
requestFocus(); // request input focus
}
public void keyPressed(KeyEvent ke)
```



```
{
 showStatus("Key Down");
 }
public void keyReleased(KeyEvent ke) {showStatus("Key Up");

public void keyTyped(KeyEvent ke) {msg += ke.getKeyChar();
repaint();
}
// Display keystrokes.
public void paint(Graphics g)
{
 g.drawString(msg, X, Y);
 }
}
```



FILES AND STREAMS

Stream

A stream can be defined as a sequence of data. There are two kinds of Streams −

- **InPutStream** − The InputStream is used to read data from a source.
- **OutPutStream** − The OutputStream is used for writing data to a destination.

Java provides strong but flexible support for I/O related to files and networks.

Byte Streams

Java byte streams are used to perform **input and output of 8-bit bytes**. Though there are many classes related to byte streams but the most frequently used classes are, **FileInputStream** and **FileOutputStream**.

Example

```
import  java.io.*;
public class CopyFile {
public static void main(String args[]) throws IOException {
FileInputStream in = null;
FileOutputStream out = null;
try {
in = new FileInputStream("input.txt");
out = new FileOutputStream("output.txt");
int c;
while ((c = in.read()) != -1) {
out.write(c);
}
}finally {
if (in != null) {
in.close();
}
if (out != null) {
out.close();
}
}
} }
```

Now let's have a file input.txt with the following content:

```
This is test for copy file.
$javac CopyFile.java
$java CopyFile
```

Character Streams

Java **Byte** streams are used to perform input and output of 8-bit bytes, whereas Java **Character** streams are used to perform **input and output for 16-bit unicode**. Though there are many classes related to character streams but the most frequently used classes are, **FileReader** and **FileWriter**.

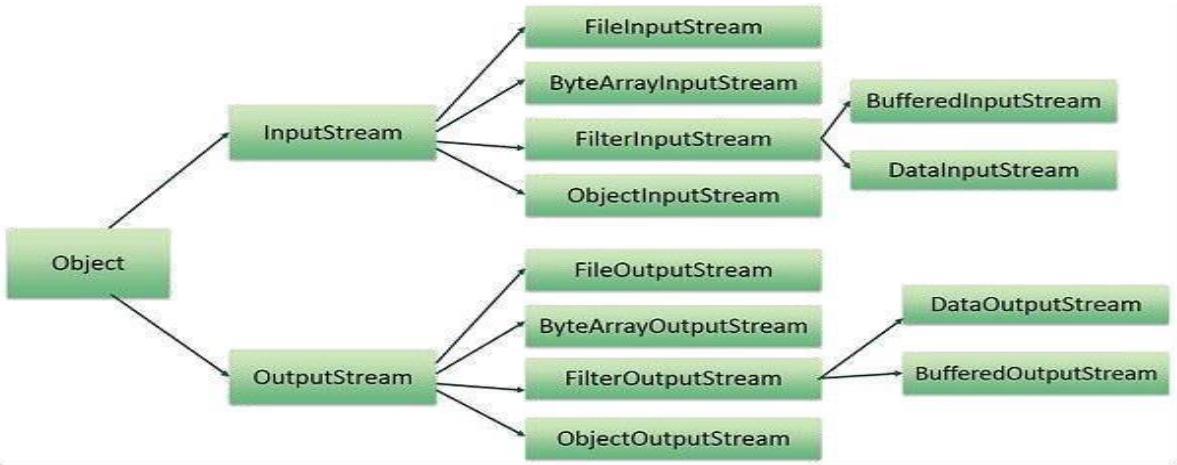
Though internally FileReader uses FileInputStream and FileWriter uses FileOutputStream but here the major difference is that FileReader reads two bytes at a time and FileWriter writes two bytes at a time.

Example

```
import  java.io.*;
public class CopyFile {
public static void main(String args[]) throws IOException {
FileReader in = null;
FileWriter out = null;
try {
in = new FileReader("input.txt");
out = new FileWriter("output.txt");
int c;
while ((c = in.read()) != -1) {
out.write(c);
}
}finally {
if (in != null) {
in.close();
}
if (out != null) {
out.close();
}
}
}
}
```

Reading and Writing Files Text input/output,

As described earlier, a stream can be defined as a sequence of data. The **InputStream** is used toread data from a source and the **OutputStream** is used for writing data to a destination. Here is a hierarchy of classes to deal with Input and Output streams.



The two important streams are **FileInputStream** and **FileOutputStream** (File Handling)
In Java, FileInputStream and FileOutputStream classes are used to read and write data in file. Inanother words, they are used for file handling in java.

Java FileOutputStream class

Java FileOutputStream is an output stream for writing data to a file. It is a class belongs to **bytestreams.** It can be used to create text files.

First we should read data from the keyword. It uses **DataInputStream** class for reading data from the keyboard is as:

DataInputStream dis=new DataInputStream(System.in);

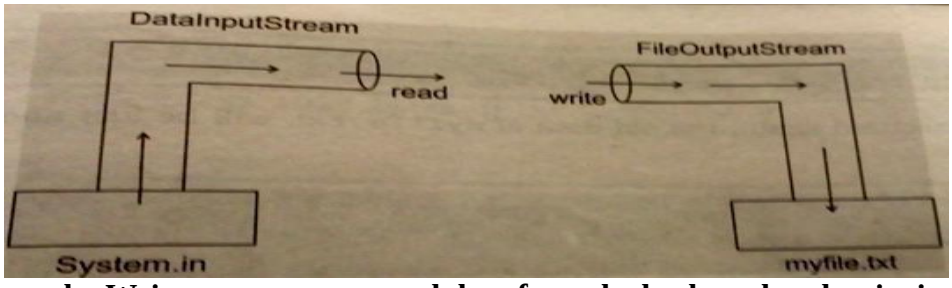
FileOutputStream **used to send data to the file and attaching the file to FileOutputStream.** i.e., **FileOutputStream fout=new FileOutputStream("File_name");**

The next step is to read data from DataInputStream and write it into FileOutputStream. It means read data from **dis object** and write it into **fout object. i.e.,**

```
ch=(char)dis.read(); //read one character into ch
fout.write(ch); //write ch into file.
```

→ **Finally** closing the file using: **fout.close();**

Creating a Text file:



Example: Write a program to read data from the keyboard and write it to myfile.txt file.

```
import java.io.*;
class Test{
public static void main(String args[])
{
DataInputStream dis=new DataInputStream(System.in);
FileOutputstream fout=new FileOutputStream("myfile.txt");
System.out.println("Entertext @ at the end:");
char ch;
while((ch=(char)dis.read())!="@")
fout.write(ch);fout.close();
}
}
```

Output: javac Test.java
Java Test

```
Enter text @ at the end:
This is my file line one
This is my file line two@
```

Java FileInputStream class

It is useful to read data from a file in the form of sequence of bytes. It is possible to read datafrom a text file using FileInputStream. i.e.,

FileInputStream fin= new FileInputStream("myfile.txt");

To read data fromthe file is,

ch=fin.read();

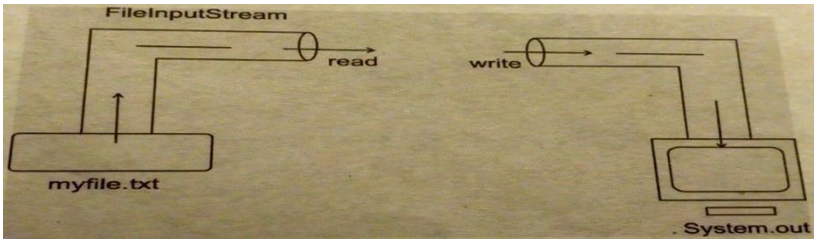
When there is no more data available to read then it returns **-1**.

The Output Stream is used to send data to the **monitor**. i.e., **PrintStream,** for displaying the data we can use **System.out.**

System.out.print(ch);

Reading data from a text file using FileInputStream:

Java FileInputStream class obtains input bytes from a file. It is used for reading streams of raw bytes such as image data. It should be used to read byte-oriented data for example to read image, audio, video etc.



Example: Write a program to read data from myfile.txt using FileInputStream and display it on monitor.

```
import java.io.*;
class ReadFile
{
public static void main(String args[])
{
FileInputStream fin=new FileInputStream("myfile.txt");
System.out.println("File Contents:");
int ch;
while((ch=fin.read())!=-1)
{
System.out.println((char)ch);
}
fin.close();
}
}
```

Output: javac ReadFile.java
java ReadFile

File Contents:
This is my file line one
This is my file line two

UNIT V:

GUI Programming with Java – AWT class hierarchy, component, container,panel,window, frame, graphics.

AWT controls: Labels, button, text field, check box, check box groups, choices, lists,scrollbars, and graphics.

Layout Manager – Layout manager types: border, grid and flow.

Swing – Introduction, limitations of AWT, Swing vs AWT.

GUI PROGRAMMING WITH JAVA

ABSTRACT WINDOW TOOLKIT (AWT)

Java AWT (Abstract Window Toolkit) is *an **API to develop GUI** or window-based application in java*. Java AWT components are **platform-dependent** i.e. components are displayed according to the view of operating system. AWT is heavyweight i.e. its components uses the resources of system. The Abstract Window Toolkit(AWT) support for applets. The AWT contains numerous classes and methods that allow you to create and manage windows.

The **java.awt package** provides classes for AWT api such as **TextField, Label, TextArea, RadioButton, CheckBox, Choice, List etc.**

AWT Classes

The AWT classes are contained in the **java.awt package**. It is one of Java's largest packages.

Class	Description
AWTEvent	Encapsulates AWT events.
AWTEventMulticaster	Dispatches events to multiple listeners.
BorderLayout	Border layouts use five components: North, South, East, West, and Center.
CardLayout	Card layouts emulate index cards.Only the one on top is showing.
Checkbox	Creates a check box control.
CheckboxGroup	Creates a group of check box controls.
CheckboxMenuItem	Creates an on/off menu item.
Choice	Creates a pop-up list.
Color	Manages colors in a portable, platform-independent fashion.
Component	An abstract superclass for various AWT components.
Container	A subclass of Component that can hold other components.
Cursor	Encapsulates a bitmapped cursor.
Dialog	Creates a dialog window.
Dimension	Specifies the dimensions of an object. The width is stored in width , and the height is stored in height .
Event	Encapsulates events.
FlowLayout	The flow layout manager. Flow layout positions components left to right, top to bottom.
Frame	Creates a standard window that has a title bar, resize corners, anda menu bar.
Graphics	Encapsulates the graphics context.

The AWT supports the following types of controls:

3. Labels
4. Push buttons
5. Check boxes
6. Choice lists
7. Lists
8. Scroll bars
9. Text editing

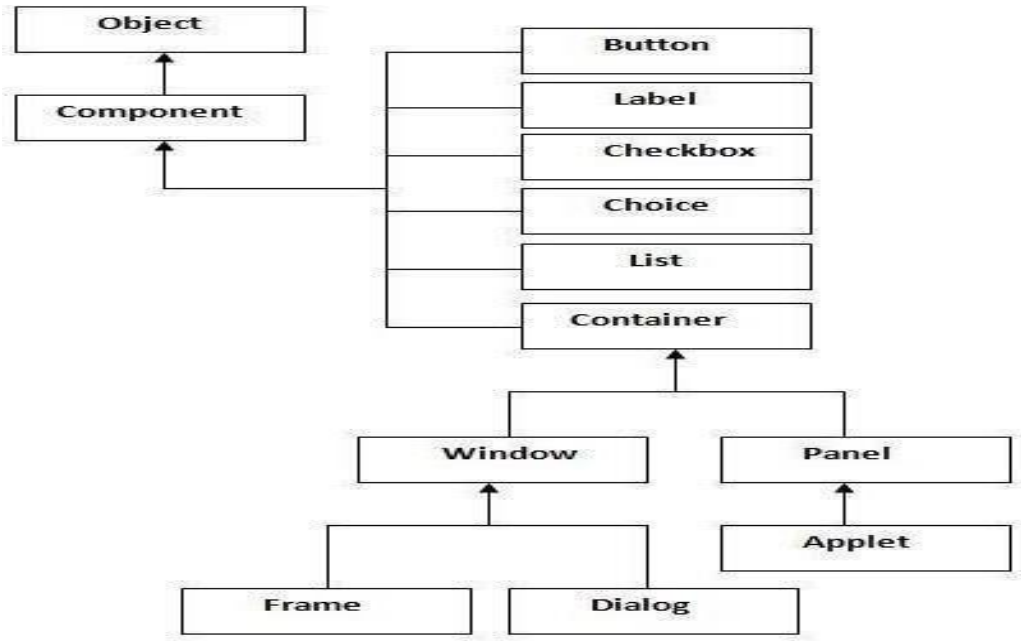
User interaction with the program is of two types:

CUI (Character User Interface): In CUI user interacts with the application by typing characters or commands. In CUI user should remember the commands. It isnot user friendly.

2. **GUI (Graphical User Interface):** In GUI user interacts with the application through graphics. GUI is user friendly. GUI makes application attractive. It is possible to simulate real object in GUI programs. In java to write GUI programs we can use awt (Abstract Window Toolkit) package.

Java AWT Class Hierarchy

The hierarchy of Java AWT classes is given below.



Container

The Container is a component in AWT that can contain other components like buttons, textfields,labels etc. The classes that extend Container class are known as container such as **Frame, Dialogand Panel**.

Window

The window is the container that has no borders and menu bars. You must use frame, dialog or another window for creating a window.

Panel

The Panel is the container that doesn't contain title bar and menu bars. It can have other components like button, textfield etc.

Frame

The Frame is the container that contain title bar and can have menu bars. It can have other components like button, textfield etc.

Useful Methods of Component class

Method	Description
public void add(Component c)	inserts a component on this component.
public void setSize(int width,int height)	sets the size(width and height) of the component.
public void setLayout(LayoutManager m)	defines the layout manager for the component.
public void setVisible(boolean status)	changes the visibility of the component, by default false.

Listeners and Listener Methods:

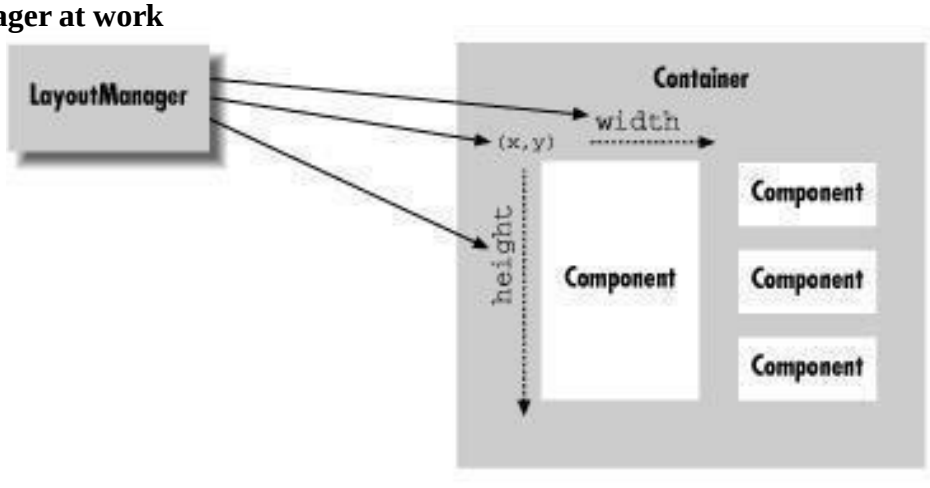
Listeners are available for components. A Listener is an interface that listens to an event from a component. Listeners are available in **java.awt.event package**. The methods in the listener interface are to be implemented, when using that listener.

Component	Listener	Listener methods
Button	ActionListener	public void actionPerformed (ActionEvent e)
Checkbox	ItemListener	public void itemStateChanged (ItemEvent e)
CheckboxGroup	ItemListener	public void itemStateChanged (ItemEvent e)
TextField	ActionListener FocusListener	public void actionPerformed (ActionEvent e) public void focusGained (FocusEvent e) public void focusLost (FocusEvent e)
TextArea	ActionListener FocusListener	public void actionPerformed (ActionEvent e) public void focusGained (FocusEvent e) public void focusLost (FocusEvent e)
Choice	ActionListener ItemListener	public void actionPerformed (ActionEvent e) public void itemStateChanged (ItemEvent e)
List	ActionListener ItemListener	public void actionPerformed (ActionEvent e) public void itemStateChanged (ItemEvent e)
Scrollbar	AdjustmentListener MouseMotionListener	public void adjustmentValueChanged (AdjustmentEvent e) public void mouseDragged (MouseEvent e) public void mouseMoved (MouseEvent e)
Label	No listener is needed	

Layout Managers

A layout manager arranges the child components of a container. It positions and sets the size of components within the container's display area according to a particular layout scheme.
The layout manager's job is to fit the components into the available area, while maintaining the proper spatial relationships between the components. AWT comes with a few standard layout managers that will collectively handle most situations; you can make your own layout managers if you have special requirements.

LayoutManager at work



Every container has a **default layout manager**; therefore, when you make a new container, it comes with a LayoutManager object of the appropriate type. You can install a new layout manager at any time with the setLayout() method. Below, we set the layout manager of a container to a BorderLayout:

setLayout (new BorderLayout( ));
Every component determines three important pieces of information used by the layout manager in placing and sizing it: a minimum size, a maximum size, and a preferred size.
These are reported by the getMinimumSize(), getMaximumSize(), and getPreferredSize(), methods of Component, respectively.
When a layout manager is called to arrange its components, it is working within a fixed area. It usually begins by looking at its container's dimensions, and the preferred or minimum sizes of the child components.

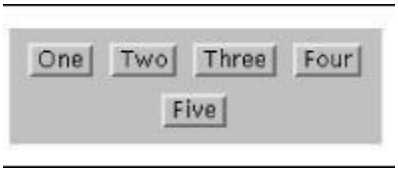
Layout manager types

Flow Layout

`FlowLayout` is a simple layout manager that tries to arrange components with their preferred sizes, from left to right and top to bottom in the display. A `FlowLayout` can have a specified justification of `LEFT`, `CENTER`, or `RIGHT`, and a fixed horizontal and vertical padding.
By default, a flow layout uses **`CENTER`** justification, meaning that all components are centered within the area allotted to them. `FlowLayout` is the default for `Panel` components like `Applet`.

The following applet adds five buttons to the default `FlowLayout`.

```
import java.awt.*;
/*
<applet code="Flow" width="500" height="500">
</applet>
*/
public class Flow extends java.applet.Applet
{
public void init()
{
        //Default for Applet is FlowLayout
            add( new Button("One") );
        add( new Button("Two") );
        add( new Button("Three") );
        add( new Button("Four") );
        add( new Button("Five") );
}
}
```



If the applet is small enough, some of the buttons spill over to a second or third row.

Grid Layout

`GridLayout` arranges components into regularly spaced rows and columns. The components are arbitrarily resized to fit in the resulting areas; their minimum and preferred sizes are consequently ignored.

`GridLayout` is most useful for arranging very regular, identically sized objects and for allocating space for Panels to hold other layouts in each region of the container.

`GridLayout` takes the number of rows and columns in its constructor. If you subsequently give it too many objects to manage, it adds extra columns to make the objects fit. You can also set the number of rows or columns to zero, which means that you don't care how many elements the layout manager packs in that dimension.

For example, `GridLayout(2,0)` requests a layout with two rows and an unlimited number of columns; if you put ten components into this layout, you'll get two rows of five columns each. The following applet sets a `GridLayout` with three rows and two columns as its layout manager;

```
import java.awt.*;
/*
<applet code="Grid" width="500"height="500"> </applet>
*/
public class Grid extends java.applet.Applet
{
public void init()
{
setLayout( new GridLayout( 3, 2 ));
```

```
add( new Button("One") );
add( new Button("Two") ); add( new Button("Three") );add( new Button("Four") );
add( new Button("Five") );
}
}
```

The five buttons are laid out, in order, from left to right, top to bottom, with one empty spot.

Border Layout

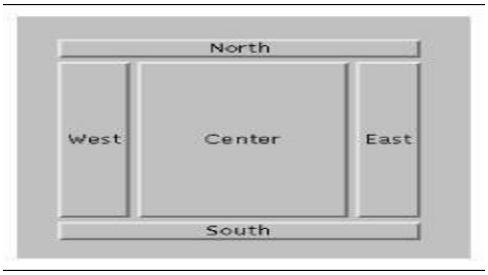
`BorderLayout` is a little more interesting. It tries to arrange objects in one of five geographical locations: "North," "South," "East," "West," and "Center," possibly with some padding between.

`BorderLayout` is the default layout for `Window` and `Frame` objects. Because each component is associated with a direction, `BorderLayout` can manage at most five components; it squashes or stretches those components to fit its constraints.

When we add a component to a border layout, we need to specify both the component and the position at which to add it. To do so, we use an overloaded version of the **`add()`** method that takes an additional argument as a constraint.

The following applet sets a `BorderLayout` layout and adds our five buttons again, named for their locations;

```
import java.awt.*;
/*
<applet code="Border" width="500" height="500">
</applet>
*/
public class Border extends java.applet.Applet
{
public void init()
{
        setLayout( new java.awt.BorderLayout() );
        add( new Button("North"), "North" ); add(
        new Button("East"), "East" );
add( new Button("South"), "South" );
add( new Button("West"), "West" ); add( new Button("Center"), "Center" );
}
}
```



Compile: javac Border.java
Run : appletviewer Border.java

Java AWT Example

To create simple awt example, you need a frame. There are two ways to create a frame in AWT.

1. By extending Frame class (inheritance)
2. By creating the object of Frame class (association)

Simple example of AWT by inheritance

```
import java.awt.*;
class First extends Frame{
First(){
   Button b=new Button("click me");
   b.setBounds(30,100,80,30);     // setting button position
   add(b);  //adding button into frame
   setSize(300,300);  //frame size 300 width and 300 height
   setLayout(null);  //no layout manager
    setVisible(true);  //now frame will be visible
   }
public static void main(String args[]){
First f=new First();
}
}
```

Simple example of AWT by association

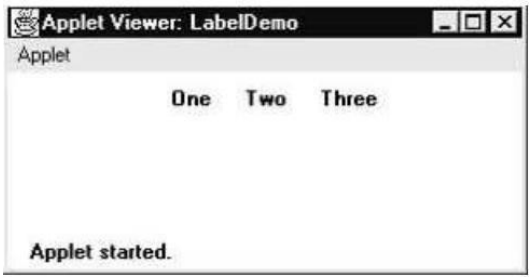
```
import java.awt.*;
class First2{
First2(){
Frame f=new Frame();
Button b=new Button("click me");
b.setBounds(30,50,80,30);
 f.add(b);
f.setSize(300,300);
f.setLayout(null);
f.setVisible(true);
}
public static void main(String args[]){
First2 f=new First2();
}
}
```

AWT controls

Labels:

The easiest control to use is a label. A label is an object of type Label, and it contains a string,which it displays. Labels are passive controls that do not support any interaction with the user.

```
// Demonstrate Labels import java.awt.*; importjava.applet.*; /*
<applet code="LabelDemo" width=300 height=200>
</applet> */
public class LabelDemo extends Applet
{
public void init()
{
Label one = new Label("One");
Label two = new Label("Two");
Label three = new Label("Three");
add labels to applet windowadd(one);
add(two); add(three);
}
}
```



Buttons:

The most widely used control is the push button. A push button is a component that contains a label and that generates an event when it is pressed. Push buttons are objects of type Button. Button class is useful to create push buttons. A push button triggers a series of events.

To create push button: Button b1 =new Button("label");

To get the label of the button: String l = b1.getLabel();

To set the label of the button: b1.setLabel("label");

To get the label of the button clicked: String str = ae.getActionCommand();

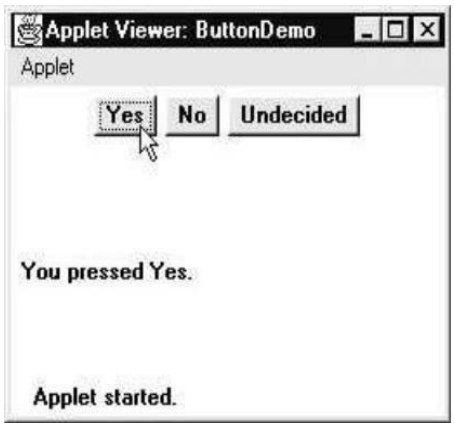
Demonstrate Buttons

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/* <applet code="ButtonDemo" width=250 height=150>
</applet> */

public class ButtonDemo extends Applet implements ActionListener
{
String msg = "";
Button yes, no, maybe;
```

```
public void init()
{
yes = new Button("Yes");no = new Button("No");
maybe = new Button("Undecided");add(yes);
add(no); add(maybe);
yes.addActionListener(this); no.addActionListener(this); maybe.addActionListener(this);
}

public void actionPerformed(ActionEvent ae)
{
                String str =
                  ae.getActionCommand();
                  if(str.equals("Yes"))
{
msg = "You pressed Yes.";
}
else if(str.equals("No"))
{
msg = "You pressed No.";
}
else
{
msg = "You pressed Undecided.";
}
repaint();
}
public void paint(Graphics g)
{
g.drawString(msg, 6, 100);
}
}
```



Check Boxes:

A check box is a control that is used to turn an option on or off. It consists of a small box that can either contain a check mark or not. There is a label associated with each check box that describes what option the box represents. You change the state of a check box by clicking on it. Check boxes can be used individually or as part of a group.

Demonstrate check boxes

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
```

```
/*
<applet code="CheckboxDemo" width=250 height=200>
</applet>
*/
public class CheckboxDemo extends Applet implements ItemListener
{
String msg = "";
checkbox Win98, winNT, solaris, mac;

public void init()
{
win98 = new Checkbox("Windows 98/XP", null, true);
winNT = new Checkbox("Windows NT/2000");
solaris = new Checkbox("Solaris");
mac = new Checkbox("MacOS");
add(Win98);
add(winNT);
add(solaris);
add(mac);
Win98.addItemListener(this);
winNT.addItemListener(this);solaris.addItemListener(this);
mac.addItemListener(this);
}
public void itemStateChanged(ItemEvent ie)
{
repaint();
}
// Display current state of the check boxes.

public void paint(Graphics g)
{
msg = "Current state: "; g.drawString(msg, 6, 80);
msg = " Windows 98/XP: " + Win98.getState();g.drawString(msg, 6, 100);
msg = " Windows NT/2000: " + winNT.getState();g.drawString(msg, 6, 120);
msg = " Solaris: " + solaris.getState();g.drawString(msg, 6, 140);
msg = " MacOS: " + mac.getState();g.drawString(msg, 6, 160);
```

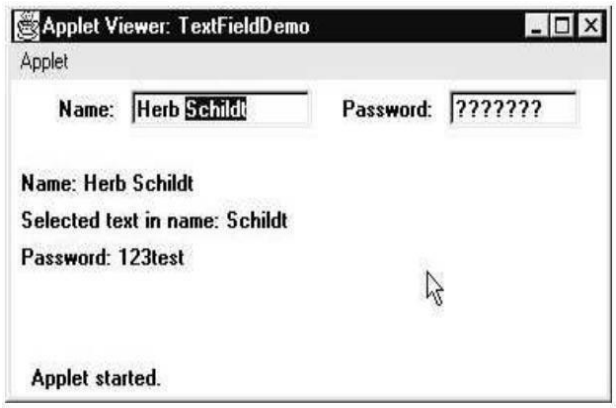
TextField:

The TextField class implements a single-line text-entry area, usually called an edit control. Text fields allow the user to enter strings and to edit the text using the arrow keys, cut and paste keys, and mouse selections.

```
// Demonstrate text field.import java.awt.*; import java.awt.event.*; import java.applet.*;
/*
<applet code="TextFieldDemo" width=380 height=150>
</applet>
*/
public class TextFieldDemo extends Applet implements ActionListener
{
TextField name, pass;

public void init()
{
Label namep = new Label("Name: ", Label.RIGHT);
Label passp = new Label("Password: ", Label.RIGHT);
name = new TextField(12);
pass = new TextField(8);
pass.setEchoChar('?');
add(namep);
add(name);
add(passp);
add(pass);
// register to receive action events
name.addActionListener(this);
pass.addActionListener(this);
}
// User pressed Enter.
public void actionPerformed(ActionEvent ae)
{
repaint();
}
public void paint(Graphics g)
{
g.drawString("Name: " + name.getText(), 6, 60);
g.drawString("Selected text in name: " + name.getSelectedText(), 6, 80);
g.drawString("Password: " + pass.getText(), 6, 100);

}
}
```

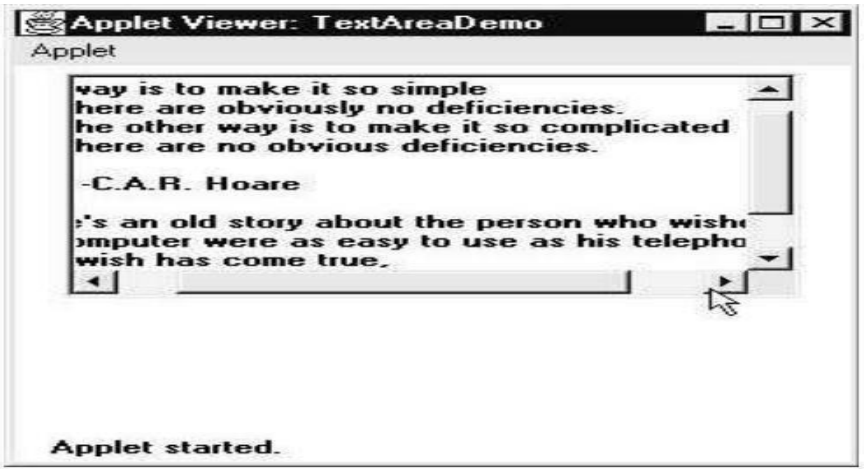


TextArea:

Sometimes a single line of text input is not enough for a given task. To handle thesesituations, the AWT includes a simple multiline editor called TextArea .

DemonstrateT extArea

```
import java.awt.*;
importjava.applet.*;
/*
<applet code="TextAreaDemo" width=300height=250> </applet>
*/
public class TextAreaDemo extends Applet
{
public void init()
{
String val = "There are two ways of constructing " + "a software design.\n" + "One way is to make it so simple\n" + "that there are obviously no deficiencies.\n" + "And the other way is to make it so complicated\n" + "that there are no obvious deficiencies.\n\n" + " -C.A.R. Hoare\n\n"
+ "There's an old story about the person who wished\n" + "his computer were as easy to use as his telephone.\n" + "That wish has come true,\n" + "since I no longer know how to use my telephone.\n\n" + " -Bjarne Stroustrup, AT&T, (inventor of C++)";
TextArea text = new TextArea(val, 10, 30);
add(text);
}
}
```



CheckboxGroup

It is possible to create a set of mutually exclusive check boxes in which one and only one check box in the group can be checked at any one time. These check boxes are often called radio buttons. A Radio button represents a round shaped button such that only one can be selected from a panel. Radio button can be created using CheckboxGroup class and Checkbox classes.

· To create a radio button:

CheckboxGroup cbg = new CheckboxGroup ();
Checkbox cb = new Checkbox ("label", cbg, true);

· To know the selected checkbox:
Checkbox cb = cbg.getSelectedCheckbox ();
· To know the selected checkbox label:
· String label = cbg.getSelectedCheckbox().getLabel ();

Demonstrate check box group.

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="CBGroup" width=250 height=200>
</applet>
*/
public class CBGroup extends Applet implements ItemListener
{
String msg = "";
Checkbox Win98, winNT, solaris, mac;
CheckboxGroup cbg;
public void init() {
cbg = new CheckboxGroup();
Win98 = new Checkbox("Windows 98/XP", cbg, true);
 winNT = new Checkbox("Windows NT/2000", cbg, false);
solaris = new Checkbox("Solaris", cbg, false);
mac = new Checkbox("MacOS", cbg, false);
add(Win98);
add(winNT);
add(solaris);
add(mac);
Win98.addItemListener(this);
winNT.addItemListener(this);
solaris.addItemListener(this);
mac.addItemListener(this);
}
public void itemStateChanged(ItemEvent ie) {
repaint();
}
// Display current  state of the check boxes.
public void paint(Graphics g) {
msg = "Current selection: ";
msg += cbg.getSelectedCheckbox().getLabel();
g.drawString(msg, 6, 100);
}
}
```

Choice Controls

The Choice class is used to create a pop-up list of items from which the user may choose.Thus, a Choice control is a form of menu. Choice menu is a popdown list of items. Only one item can be selected.

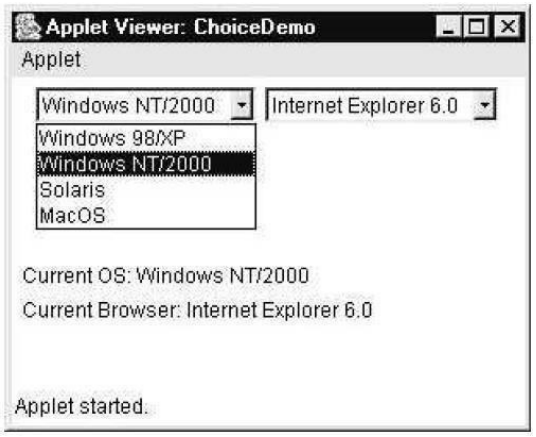
- To create a choice menu:

  Choice ch = new Choice();
- To add items to the choice menu:
  ch.add ("text");
- To know the name of the item selected from the choice menu:
  String s = ch.getSelectedItem ();
- To know the index of the currently selected item:
  int i = ch.getSelectedIndex();

This method returns -1, if nothing is selected.

//Demonstrate Choice lists.

```
import java.awt.*;
import java.awt.event.*;
 import java.applet.*;
/*
<applet code="ChoiceDemo" width=300height=180> </applet>
*/
public class ChoiceDemo extends Applet implements ItemListener
{
Choice os, browser;
String msg = "";
public void init() {
 os = new Choice();
browser = newChoice();
//add items to os list
os.add("Windows 98/XP");
os.add("Windows NT/2000");
os.add("Solaris");
os.add("MacOS");
// add items to browser list
browser.add("Netscape 3.x");
browser.add("Netscape 4.x");
browser.add("Netscape 5.x");
browser.add("Netscape 6.x");
browser.add("Internet Explorer4.0");
browser.add("InternetExplorer 5.0");
browser.add("Internet Explorer6.0");
browser.add("Lynx 2.4");
browser.select("Netscape 4.x");
// add choice lists towindow add(os);
add(browser);
// register to receive item events
 os.addItemListener(this);
browser.addItemListener(this);
}
```



```
public void itemStateChanged(ItemEvent ie) {
repaint();
}
// Display current selections.
public void paint(Graphics g)
{
 msg = "Current OS: ";
msg += os.getSelectedItem();
g.drawString(msg, 6, 120);
msg = "Current Browser: ";
msg += browser.getSelectedItem();
g.drawString(msg, 6, 140);
}
}
```

Lists

The List class provides a compact, multiple-choice, scrolling selection list. Unlike the Choiceobject, which shows only the single selected item in the menu, a List object can be constructed to show any number of choices in the visible window. It can also be created to allow multiple selections.

List provides these constructors:

```
List( )
List(int numRows )
List(int numRows , boolean multipleSelect )
```

A List box is similar to a choice box, it allows the user to select multiple items.

- To create a list box:

```
List lst = new List();
```

(or)

```
List lst = new List (3, true);
```

This list box initially displays 3 items. The next parameter true represents that the user can select more than one item from the available items. If it is false, then the user can select onlyone item.

= To add items to the list box: lst.add("text");
= To get the selected items: String x[] = lst.getSelectedItems();
= To get the selected indexes: int x[] = lst.getSelectedIndexes ();

//**Demonstrate Lists.**

```
 import java.awt.*;
importjava.awt.event.*;
import java.applet.*; /*
<applet code="ListDemo" width=300 height=180>
</applet>
*/
public class ListDemo extends Applet implements ActionListener
{
List os, browser; String msg = "";
 public void init() {
 os = new List(4, true);
```

```
browser = new List(4, false);
      //add items to os list
          os.add("Windows 98/XP");
          os.add("Windows
          NT/2000");
          os.add("Solaris");
          os.add("MacOS");
      // add items to browser list
          browser.add("Netscape 3.x");
          browser.add("Netscape 4.x");
          browser.add("Netscape 5.x");
          browser.add("Netscape 6.x");
browser.add("Internet Explorer4.0");
browser.add("InternetExplorer 5.0");
browser.add("Internet Explorer6.0");
browser.add("Lynx 2.4");
browser.select(1);
// add lists to window
add(os);
add(browser);
        //register to receive action events
os.addActionListener(this);
browser.addActionListener(this);
}
public void actionPerformed(ActionEvent ae)
{
 repaint();
}
//Display current selections
public void paint(Graphics g)
{
int idx[];
msg = "Current OS: ";
idx = os.getSelectedIndexes();
for(int i=0; i<idx.length; i++)
msg += os.getItem(idx[i]) + " ";
g.drawString(msg, 6,  120);
msg = "Current Browser: ";
msg += browser.getSelectedItem();
g.drawString(msg, 6, 140);
}
}
```

Scroll Bars

Scroll bars are used to select continuous values between a specified minimum and maximum.Scroll bars may be oriented horizontally or vertically. Scrollbar class is useful to create scrollbars that can be attached to a frame or text area. Scrollbars can be arranged vertically or horizontally.

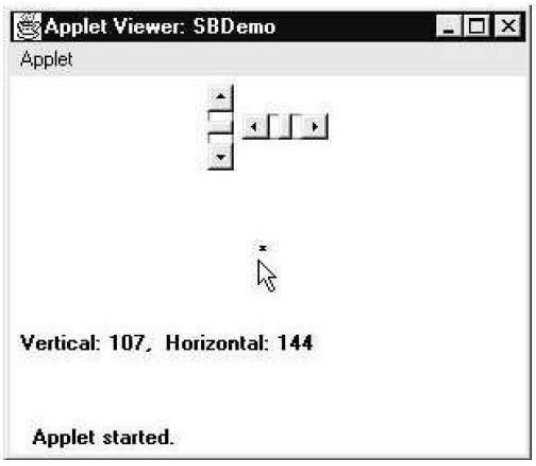
To create a scrollbar :
Scrollbar sb = new Scrollbar (alignment, start, step, min, max);
alignment: Scrollbar.VERTICAL, Scrollbar.HORIZONTAL
start: starting value (e.g. 0)
step: step value (e.g. 30) // represents scrollbar lengthmin: minimum value (e.g. 0)
max: maximum value (e.g. 300)

- To know the location of a scrollbar: int n = sb.getValue ();
- To update scrollbar position to a new position: sb.setValue (int position);
- To get the maximum value of the scrollbar: int x = sb.getMaximum ();
- To get the minimum value of the scrollbar: int x = sb.getMinimum ();
- To get the alignment of the scrollbar: int x = getOrientation ();

This method return 0 if the scrollbar is aligned HORIZONTAL, 1 if aligned VERTICAL.

```
// Demonstrate scroll bars.
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="SBDemo" width=300 height=200>
</applet>
*/
public class SBDemo extends Applet implements AdjustmentListener, MouseMotionListener
{
 String msg = ""; Scrollbar vertSB, horzSB;
public void init() {
int width = Integer.parseInt(getParameter("width"));
intheight = Integer.parseInt(getParameter("height"));
vertSB = new Scrollbar(Scrollbar.VERTICAL, 0, 1, 0, height);
horzSB = new Scrollbar(Scrollbar.HORIZONTAL, 0, 1, 0, width);
add(vertSB);
add(horzSB);
      // register to receive adjustment events
          vertSB.addAdjustmentListener(this);
          horzSB.addAdjustmentListener(this);
          addMouseMotionListener(this);
}
public void adjustmentValueChanged(AdjustmentEvent ae) {
repaint();
}
// Update scroll bars to reflect mouse dragging.
public void mouseDragged(MouseEvent me) {
          int x = me.getX();
int y = me.getY();
vertSB.setValue(y);
horzSB.setValue(x);
```

```
repaint();
}
//Necessary for MouseMotionListener
public void mouseMoved(MouseEvent me) {
}
//Display current value of scroll bars.
public void paint(Graphics g) {
msg = "Vertical: " + vertSB.getValue();
 msg += ", Horizontal: " + horzSB.getValue();
g.drawString(msg, 6, 160);
// show current mouse drag position
g.drawString("*", horzSB.getValue(),vertSB.getValue());
}
}
```



Graphics

The AWT supports a rich assortment of graphics methods. All graphics are drawn relative toa window. Graphics class and is obtained in two ways:

It is passed to an applet when one of its various methods, such as paint( ) or update( ),is called.
It is returned by the getGraphics( ) method of Component.

Drawing Lines

Lines are drawn by means of the drawLine( ) method, shown here:
void drawLine(int startX, int startY, int endX, int endY)
drawLine( ) displays a line in the current drawing color that begins at startX,startY and endsat endX,endY.
The following applet draws several lines:

```
//Draw lines
importjava.awt.*;
 import java.applet.*;
 /*
<applet code="Lines" width=300height=200> </applet>
*/
public class Lines extends Applet
{ public void paint(Graphics g) {g.drawLine(0, 0, 100, 100);
g.drawLine(0, 100, 100, 0);
g.drawLine(40, 25, 250, 180);
g.drawLine(75, 90, 400, 400);
g.drawLine(20, 150, 400, 40);
g.drawLine(5, 290, 80, 19);

 }
 }
```

Drawing Rectangles

The drawRect( ) and fillRect( ) methods display an outlined and filled rectangle, respectively.They are shown here:

void drawRect(int top, int left, int width, int height)

void fillRect(int top, int left, int width, int height)

The upper-left corner of the rectangle is at top, left. The dimensions of the rectangleare specified by width and height.

To draw a rounded rectangle, use drawRoundRect( ) or fillRoundRect( ), both shown here:

void drawRoundRect(int top, int left, int width, int height,int xDiam, int yDiam)

void fillRoundRect(int top, int left, int width, int height, int xDiam, int yDiam)

// Draw rectangles

```
import java.awt.*;
import java.applet.*;
/*
<applet code="Rectangles" width=300height=200> </applet>
*/
public class Rectangles extends Applet {
 public void paint(Graphics g) {
g.drawRect(10, 10, 60, 50);
g.fillRect(100,10, 60, 50);
g.drawRoundRect(190, 10,60, 50, 15, 15);
 g.fillRoundRect(70, 90,140, 100, 30, 40);
}}
```

Drawing Ellipses and Circles

To draw an ellipse, use drawOval( ). To fill an ellipse, use fillOval( ).

These method sare shown here:

void drawOval(int top, int left, int width, int height)

void fillOval(int top, int left, int width, int height)

// **Draw Ellipses**

```
import java.awt.*;
 import java.applet.*;
/*
<applet code="Ellipses" width=300height=200> </applet>
*/
public class Ellipses extends Applet
{
public void paint(Graphics g) {
g.drawOval(10, 10, 50, 50);
g.fillOval(100, 10, 75, 50);
g.drawOval(190, 10, 90, 30);
g.fillOval(70, 90, 140, 100);
}
}
```

Drawing Arcs

Arcs can be drawn with drawArc( ) and fillArc( ), shown here:

void drawArc(int top, int left, int width, int height, int startAngle,int sweepAngle)

void fillArc(int top, int left, int width, int height, int startAngle,int sweepAngle)

- The arc is bounded by the rectangle whose upper-left corner is specified by top, left and whosewidth and height are specified by width and height.
- The arc is drawn from startAngle through the angular distance specified by sweepAngle. Angles are specified in degrees.
- The arc is drawn counterclockwise if sweepAngle is positive, and clockwise if sweepAngle is negative. Therefore, to draw an arc from twelve o' clock to six o' clock, the start angle would be 90 and the sweep angle 180.

The following applet draws several arcs:

```
//Draw Arcs
import java.awt.*;
 import java.applet.*;
/*
<applet code="Arcs" width=300 height=200>
</applet>
*/
public class Arcs extends Applet {
public void paint(Graphics g) {
g.drawArc(10, 40, 70, 70, 0, 75);
g.fillArc(100, 40, 70, 70, 0, 75);
g.drawArc(10, 100, 70, 80, 0, 175);
g.fillArc(100, 100, 70, 90, 0, 270);
g.drawArc(200, 80, 80, 80, 0, 180);
}}
```

Drawing Polygons

It is possible to draw arbitrarily shaped figures using drawPolygon( ) and fillPolygon(), shown here:

void drawPolygon(int x[ ], int y[ ], int numPoints)

void fillPolygon(int x[ ], int y[ ], int numPoints)

- The polygons endpoints are specified by the coordinate pairs contained within the x and y arrays.
- The number of points defined by x and y is specified by numPoints.
- There are alternative forms of these methods in which the polygon is specified by a Polygon object.

The following applet draws an hour glass shape:

```
// Draw Polygon
import java.awt.*;
 import java.applet.*;
/*
<applet code="HourGlass"width=230 height=210>
</applet>
*/
public class HourGlass extends Applet {
public void paint(Graphics g) {
int xpoints[] = {30, 200, 30, 200, 30};
int ypoints[] = {30, 30, 200, 200, 30};
int num = 5;
```

g.drawPolygon(xpoints, ypoints, num);

}}

SWINGS

- Swing is a set of classes that provides more powerful and flexible components than are possible with the AWT. **Swing** is a GUI widget toolkit for **Java**. It is part of Oracle's **Java** Foundation Classes (JFC) that is *used to create window-based applications*. It is built on the top of AWT (Abstract Windowing Toolkit) API and entirely written in java.

- In addition to the familiar components, such as buttons, check boxes, and labels, Swing supplies several exciting additions, including tabbed panes, scroll panes, trees, and tables. Even familiar components such as buttons have more capabilities in Swing. For example, a button may have both an image and a text string associated with it. Also, the image can be changed as the state of the button changes.

- Unlike AWT components, Swing components are not implemented by platform specific code. Instead, they are written entirely in Java and, therefore, are platform-independent. The term *lightweight* is used to describe such elements.
- The javax.swing package provides classes for java swing API such as JButton, JTextField, JTextArea, JRadioButton, JCheckbox, JMenu, JColorChooser etc.

Differences between AWT and Swing

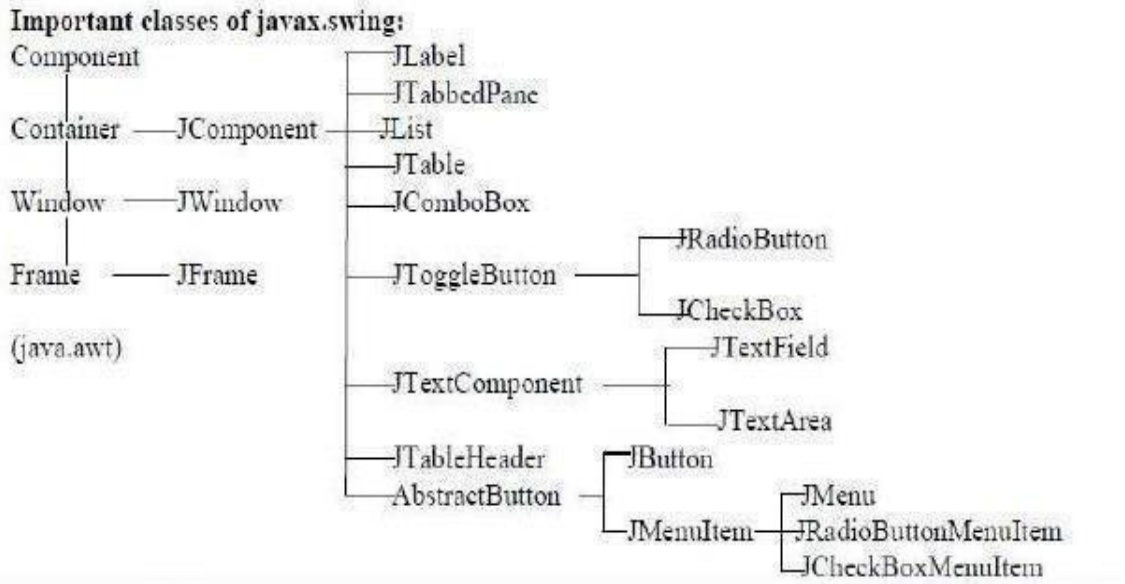
AWT	Swing
AWT components are called Heavyweight component.	Swings are called light weight component because swing components sits on the top of AWT components and do the work.
AWT components are platform dependent.	Swing components are made in purely java and they are platform independent.
AWT components require java.awt package.	Swing components require javax.swing package.
AWT is a thin layer of code on top of the OS.	Swing is much larger. Swing also has very much richer functionality.
AWT stands for Abstract windows toolkit.	Swing is also called as JFC's (Java Foundation classes).
This feature is not supported in AWT.	We can have different look and feel in Swing.
Using AWT, you have to implement a lot of things yourself.	Swing has them built in.

This feature is not available in AWT.	Swing has many advanced features like JTabel, Jtabbed pane which is not available in AWT. Also.Swing components are called "lightweight" because they do not require a native OS object to implement their functionality. JDialog and JFrame are heavyweight, because they do have a peer. So components like JButton, JTextArea, etc., are lightweight because they do not have an OS peer.

The Swing component classes are:

Class	Description
AbstractButton	Abstract superclass for Swing buttons.
ButtonGroup	Encapsulates a mutually exclusive set ofbuttons.
ImageIcon	Encapsulates an icon.
JApplet	The Swing version of Applet.
JButton	The Swing push button class.
JCheckBox	The Swing check box class.
JComboBox	Encapsulates a combo box (combination of a drop-down list & text field).
JLabel	The Swing version of a label.
JRadioButton	The Swing version of a radio button.
JScrollPane	Encapsulates a scrollable window.
JTabbedPane	Encapsulates a tabbed window.
JTable	Encapsulates a table-based control.
JTextField	The Swing version of a text field.
JTree	Encapsulates a tree-based control.

Hierarchy for Swing components:



BUTTONS

Swing buttons provide features that are not found in the Button class defined by the AWT.

For example, you can associate an icon with a Swing button. Swing buttons are subclasses of the AbstractButton class, which extends JComponent. AbstractButton contains many methods that allow you to control the behavior of buttons, check boxes, and radio buttons.

The JButton Class

The JButton class provides the functionality of a push button. JButton allows an icon, a string, orboth to be associated with the push button. Some of its constructors are shown here:

- To create a JButton with text: JButton b = new JButton (“OK”);
- To create a JButton with image: JButton b = new JButton (ImageIcon ii);
- To create a JButton with text & image: JButton b = new JButton (“OK”, ImageIcon ii);

It is possible to create components in swing with images on it. The image is specified byImageIcon class object.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
/*
<applet code="JButtonDemo" width=250 height=300>
</applet>
*/
public class JButtonDemo extends JApplet implements ActionListener {
JTextField jtf;
public void init() {
// Get content pane
Container contentPane = getContentPane();
contentPane.setLayout(new FlowLayout());
// Add buttons to content pane

ImageIcon france = new ImageIcon("france.gif");
JButton jb = new JButton(france);
jb.setActionCommand("France");
 jb.addActionListener(this);
contentPane.add(jb);

ImageIcon germany = new ImageIcon("germany.gif")
;jb = new JButton(germany);
jb.setActionCommand("Germany");
 jb.addActionListener(this);
contentPane.add(jb);
```

```
ImageIcon italy = new ImageIcon("italy.gif");
jb = new JButton(italy);
 jb.setActionCommand("Italy");
jb.addActionListener(this);
contentPane.add(jb);
ImageIcon japan = new ImageIcon("japan.gif");
jb = new JButton(japan);
jb.setActionCommand("Japan");
jb.addActionListener(this);
contentPane.add(jb);
//  Add text field to content pane
jtf = newJTextField(15);
contentPane.add(jtf);
}
public void actionPerformed(ActionEvent ae)
{
 jtf.setText(ae.getActionCommand());
}
}
```

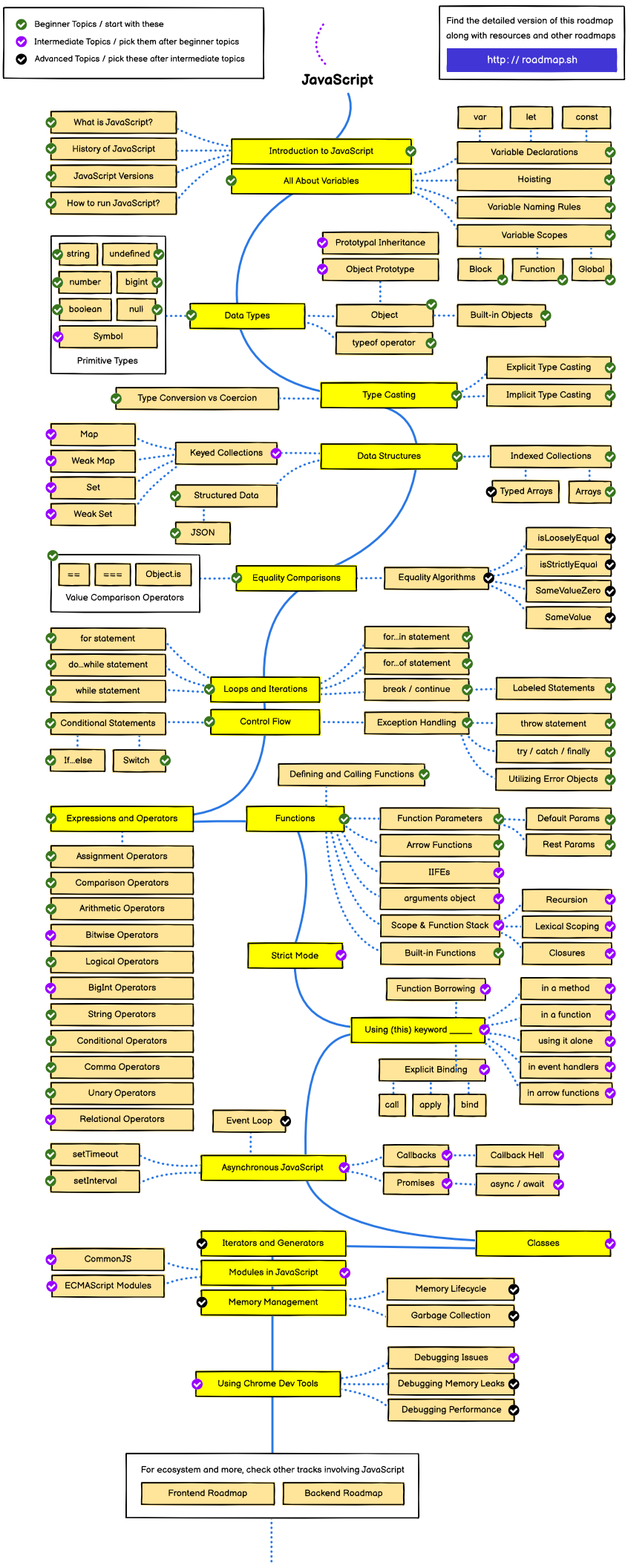
Limitations of AWT:

- The AWT defines a basic set of controls, windows, and dialog boxes that support a usable, but limited graphical interface.
- One reason for the limited nature of the AWT is that it translates its various visual components into their corresponding, platform-specific equivalents or peers. This means that the look and feel of a component is defined by the platform, not by java.
- Because the AWT components use native code resources, they are referred to as heavy weight.

The use of native peers led to several problems.

- **First**, because of variations between operating systems, a component might look, or even act, differently on different platforms. This variability threatened java's philosophy: write once, run anywhere.
- **Second,** the look and feel of each component was fixed and could not be changed.
- **Third**, the use of heavyweight components caused some frustrating restrictions.

Due to these limitations Swing came and was integrated to java. Swing is built on the AWT. Two key Swing features are: Swing components are light weight, Swing supports a pluggable look and feel.


Beginner Topics / start with these
Intermediate Topics / pick them after beginner topics
Advanced Topics / pick these after intermediate topics
Find the detailed version of this roadmap along with resources and other roadmaps
http: // roadmap.sh
JavaScript
What is JavaScript?
History of JavaScript
JavaScript Versions
How to run JavaScript?
Introduction to JavaScript
All About Variables
var
let
const
Variable Declarations
Hoisting
Variable Naming Rules
Variable Scopes
Block
Function
Global
string
undefined
number
bigint
boolean
null
Symbol
Primitive Types
Data Types
Prototypal Inheritance
Object Prototype
Object
Built-in Objects
typeof operator
Type Conversion vs Coercion
Type Casting
Explicit Type Casting
Implicit Type Casting
Map
Weak Map
Set
Weak Set
Keyed Collections
Structured Data
JSON
Data Structures
Indexed Collections
Typed Arrays
Arrays
==
===
Object.is
Value Comparison Operators
Equality Comparisons
Equality Algorithms
isLooselyEqual
isStrictlyEqual
SameValueZero
SameValue
for statement
do...while statement
while statement
Loops and Iterations
for...in statement
for...of statement
break / continue
Labeled Statements
Conditional Statements
If...else
Switch
Control Flow
Exception Handling
throw statement
try / catch / finally
Utilizing Error Objects
Defining and Calling Functions
Expressions and Operators
Assignment Operators
Comparison Operators
Arithmetic Operators
Bitwise Operators
Logical Operators
BigInt Operators
String Operators
Conditional Operators
Comma Operators
Unary Operators
Relational Operators
Functions
Function Parameters
Default Params
Rest Params
Arrow Functions
IIFEs
arguments object
Recursion
Scope & Function Stack
Lexical Scoping
Built-in Functions
Closures
Strict Mode
Function Borrowing
Using (this) keyword ____
in a method
in a function
using it alone
in event handlers
in arrow functions
Explicit Binding
call
apply
bind
Event Loop
setTimeout
setInterval
Asynchronous JavaScript
Callbacks
Callback Hell
Promises
async / await
Iterators and Generators
Classes
CommonJS
ECMAScript Modules
Modules in JavaScript
Memory Management
Memory Lifecycle
Garbage Collection
Using Chrome Dev Tools
Debugging Issues
Debugging Memory Leaks
Debugging Performance
For ecosystem and more, check other tracks involving JavaScript
Frontend Roadmap
Backend Roadmap

Top 100 JavaScript Interview Questions And Answers

January 6, 2022

JavaScript is, without a doubt, the most famous and widely used programming language in the world. It is also abbreviated as JS but notes that Java and JavaScript are totally different programming languages.

JavaScript is a high-level programming language and easy to learn as well; however, you need a clear understanding of fundamental concepts in order to proceed to become a better programmer.

When you apply for a job, you need to go through an interview where mixed types of questions are asked from basic to advance. Therefore, we have bundled together with the top 100 JavaScript questions that are frequently asked.

Below is the list which contains basic JavaScript interview questions to JavaScript advanced interview questions which will help you clear an interview round. Below JavaScript, interview coding questions are helpful for freshers as well as experienced programmers because we have divided the questions into multiple parts.

Table of Contents

JavaScript Developer Interview Questions

Following are the most often asked JavaScript basics interview questions which will help the newcomers to clear the interviews. The great part about this bundle is, we have also added the HTML, CSS, JavaScript interview questions, JavaScript algorithm interview questions, and so many other particular topic-related questions. Here we go!

Basic JavaScript Interview Questions

Here are some vanilla JavaScript interview questions which will help freshers who don't have experience in any JS frameworks. Vanilla JavaScript is often called a core JavaScript, don't be confused!

These JavaScript fundamentals interview questions should be known to every fresher before attending an interview.

Q1. What Is Javascript? What Is The Use Of It?

Ans- This is one of the most basic JavaScript programming interview questions. JavaScript is a client side scripting or programming language which is used to implement complex operations in web pages. JavaScript is present everywhere, presenting timely content updates, interactive maps, dynamic 2D/3D visuals, scrolling video jukeboxes, and so on, you name it.

JavaScript is also used for DOM manipulation, which includes dynamically adding, updating, and deleting HTML elements and CSS properties. Apart from this, JavaScript is also used for desktop application development, and nowadays, we can use JavaScript as a server-side programming language using nodeJS, which is a JavaScript runtime.

Q2. Tell The Actual Name Of Javascript.

Ans- JavaScript was started as Mocha, and then it became a LiveScript. Then it became JavaScript. It is among the most famous and widely used programming languages; however, it has nothing to do with Java programming language. It is said that, because of the popularity of Java, LiveScript became JavaScript to go with Java, with the compiled language.

Q3. Whether Javascript Is Case Sensitive Or Not?

Ans- Yes, JavaScript is case sensitive programming language. This means the keywords, function names, class names, variables, and other identifiers should be written by considering the case sensitivity. For e.g., JavaScript and JavaScript are two different variables, and they will be interpreted differently by JavaScript. Utmost care should be taken while declaring variables and functions.

Q4. Whether Javascript Is Compiled Or Interpreted Language?

Ans- JavaScript is a programming language that is interpreted instead of compiled. Programming languages like C++ or Java are compiled first before running. The original code is run through a compiler, which converts it to bytecode that the machine can understand and execute.

Whereas JavaScript does not require any compilation. Instead, a browser interpreter reads the JavaScript code, interprets each line, and then executes it.

Modern browsers like Google Chrome, Mozilla Firefox, apple safari use just-in-time compilation technology to compile the JavaScript code and convert it to executable bytecode.

Q5. State The Features Of Javascript.

Ans- JavaScript is a programming language that can achieve absolutely anything; website, mobile app, desktop app, server side programs, you guess it. It has some special kinds of features which make complex tasks easier than you think, and that is what makes it a most popular programing language. Here are some amazing features of JavaScript:

- Input validation- Not surprising, but this is among the most useful features of JavaScript. JavaScript is very useful for form validations like validating emails or phone numbers.
- Platform independent- You don’t need to install a separate compiler for JavaScript. Almost all browsers support JavaScript, and code runs in a web browser on any platform like Windows, Mac, or Linux.
- DOM manipulation- Using JavaScript, you can have control over the HTML document. You can add, update and delete the HTML elements and CSS properties.
- Object-oriented language- you can have an object-oriented structure rather than functional or monolithic programing.

Q.6 Which Data Types Does Javascript Support?

Ans- In JavaScript, there are six basic data types that can be classified into three groups: primitive, composite, and special data types. The primitive data types are String, Number, and Boolean. Composite data types include Object, Array, and Function (which are all sorts of objects). On the other hand, Null and Undefined are special types of data.

Composite data types can handle collections of values and more complicated entities, whereas primitive data types can only retain one item at a time.

Q7. What Is A Javascript Object?

Ans- In JavaScript, everything is an object. The Object is an entity with properties and types. Or, we can say the Object is a collection of properties, and again, the properties are a set of key-value data. JavaScript object is similar to objects in other programming languages.

We can take an example of a laptop as an object which has multiple properties. A laptop has color, size, battery capacity, storage type, etc. Similarly, JavaScript has an object with properties.

Q8. Difference Between Java And Javascript.

Ans- Very first, Java and JavaScript are totally different programming languages. We will see the difference between these two points by point.

- **Type**– Java is a strongly typed language which means you must declare the type of variables to use throughout the program. Whereas JavaScript is a weakly typed language that means the interpreter automatically detects the type of data as soon as the value is assigned to it.
- **Language type**– Java is an object-oriented programming language, whereas JavaScript is an object-based scripting language.
- **Language scope**– Java is used for both client side and server-side programming and runs in JVM or browser. Whereas JavaScript is a client side scripting language that runs in the browser; however, nowadays, we can

create server side applications also using nodejs.

- **File type**– java code is saved by .java extension, and the class file is generated after code gets successfully compiled. On the other hand, the JavaScript file is saved by the .js extension, and no class file or bytecode is generated for it.

Q9. How Can You Define A Variable In Javascript?

Ans- JavaScript is a weakly typed language, so you don't need to declare the data type of variable while writing it. But, you need to declare the variables using the "var" keyword for e.g., var ab, var arg, etc.

Q.10 What Is Implicit Type Conversion In Javascript?

Ans- In programing language, type conversion is converting data of one type to another. for example, converting a float to int. Implicit type conversion means JavaScript automatically converts the data type of data to another without your acknowledgment. However, implicit type conversion takes place for converting lower data types to higher data types.

Q11. Tell The Procedure To Create An Array In Javascript.

Ans- This is one of the fundamental array interview questions in JavaScript. In JavaScript, you can create an array in two ways. Either traditional method or using "array" keyword. Check syntax below:

- const cars = ["BMW","Audi","Maruti"];
- const cars= new Array("BMW","Audi","Maruti");

Q12. What Is Pass By Reference And Pass By Value?

Ans- The pass-by-value and pass-by-reference points are related to function. It is available in almost every programming language. Pass by reference is passing the address of data/ value to the function where both actual

parameter and formal parameters refer to the same memory location. Although, you may use an alias for the passed parameter.

Whereas pass-by-value is the method in which values of actual parameters are passed to the function, but the formal parameters save these values in a separate memory location. The scope of these values is limited to that function only, and any changes made to this value will not reflect the actual parameters.

Q13. Explain Recursion In A Programming Language.

Ans- Recursion is one of the most important parts of any programming language to solve complex problems. Recursion simply means calling the same function again and again until it meets some condition to terminate itself.

Q14. State The Difference Between “==” And “===”.

Ans- “==” and “===” both are comparison operators; however, “==” operator compares two variables irrespective to data type, whereas “===” operator strictly compares two variables and return true only and only if those two variables hold the same value and have the same data type.

Q15. Explain NaN In Javascript With Its Use.

Ans- In JavaScript, NaN stands for Not a Number which simply means the given value is not a valid number. It helps to check whether or not a given value or entered value through the form is a valid number.

See also **Top 100 Ansible Interview Questions And Answers**

Q16. What Is The Use Of The “This” Keyword?

Ans- “this” keyword is present in almost every object-oriented programming language. “this” keyword refers to the current Object or the Object that it belongs to.

In DOM, the “this” keyword is often used to get the properties of the current element as well.

Q17. Explain “BOM” In Javascript. Why Is It Used?

Ans- BOM in JavaScript stands for Browser Object Model. It is different from DOM because instead of interacting with an HTML document, it interacts with the browser. There are a bunch of objects and properties belonging to BOM like a window, screen, history, navigator, location, etc.

Q18. Explain “DOM” In Javascript. What Is It Used For?

Ans- DOM in JavaScript stands for Document Object Model. It is a way to interact with HTML document elements via JavaScript. When the web page is loaded, the browser generates its Document Object Model, and it is constructed as

a tree of objects. DOM helps to dynamically change the HTML elements and CSS properties.

Q19. Explain The Difference Between “Let” And “Var” Keywords.

Ans- Let and Var both are used to declare a variable in JavaScript; however, both declare variables in a different way. When you declare a variable using “var”, it will be available throughout the scope. Whereas a variable declared using “let” will be available only for a block. Simply saying, “var” has the scope of function, whereas “let” is a block-scoped.

Q20. Explain The “Typeof” Operator Along With One Example.

Ans- “typeof” operator in JavaScript is exactly the same as “gettype” in PHP. It helps to find the datatype of a JavaScript variable. For example –

```
typeof "JavaScript" //string

typeof 2022 //number

typeof false //boolean

typeof [2,0,2,2] //object
```

Q21. Tell The Difference Between “Undefined” And “Null”.

Ans- Many new learners don’t understand the actual difference between “undefined” and “null” keywords. When you use a variable that is declared but has no value assigned then it is supposed to be an “undefined” whereas “null” is an intentionally assigned value to a variable.

Q22. Explain “IsNaN” In Javascript.

Ans- isNaN function is similar to NaN property. It is a function that returns true if the given value is NaN, i.e., not a number. However, the isNaN function first converts the given value to the number and then tests it.

Q23. Can You Tell The Difference Between Window And Document?

Ans- This is one of the tricky JavaScript objects interview questions that you must be prepared with. When you open a browser, it is a Window that gets loaded first. The Window has some properties like height, width, length, screen, etc.

Talking about the document, it actually gets loaded inside the Window. This document may consist of your scripting code like HTML, PHP, ASPX, etc. To access the Window properties, you use a “window.property” whereas to access the document properties; you may go with something like “window.document.property” which in turn, is available in short as “document.property”.

Q24. Explain JSON And Why Is It Used?

Ans- JSON stands for JavaScript Object Notation which is a lightweight format for data interchange. It is used to send the data between two or more nodes/compters/applications. It consists of human-readable text and is stored as attribute-value pairs and arrays.

JSON is usually used to send and receive the data to and from web applications (data exchange in client and server in both directions). Check detailed information about JSON on Mozilla official JavaScript documentation.

For eg. `{"name" : "JavaScript" , "version" : "ES6"}`

Q25. How Can You Enable Fullscreen Mode Using Javascript?

Ans- JavaScript offers a variety of functions to achieve small to huge operations and tasks. To enable the fullscreen in JavaScript, you have a method “element.requestFullScreen()”. Note that the method name is case-sensitive, and the “element” is an HTML element.

For example, I want to display a video on a full page and it has the id “myvideo”.

Code-

```
var ele= document.getElementById(""myvideo");
ele.requestFullScreen();
```

JavaScript Interview Questions For Experienced Programmers

Q26. Explain What Is Dynamically-Typed Language.

Ans- The typed language is a programming language in which the type of data is known at either compile-time or run time. The language is dynamically typed if the type of variable is known at runtime instead of compile-time. Sometimes, dynamically-typed languages are referred to as loosely-typed languages.

For example, Python, JavaScript is dynamically-typed languages because you don't need to specify the type of variable every time. You are not required to specify that string is a string or a parameter is of type float or something like that.

Q27. What Do You Mean By Asynchronous Programming?

Ans- Asynchronous refers to concurrency. When two or multiple tasks are executing parallel to each other, it is known as asynchronous execution. Asynchronous programming means some task runs independently from the main application and acknowledges the calling thread upon completion, failure, or progress.

It helps to increase the overall performance of the application because no other process depends on the previous process to be completed first. In JavaScript, we can achieve asynchronous programing using concepts like async, await, promises, and callbacks.

Q28. Explain "Immediately Invoked Function" In Javascript.

Ans- In JavaScript, you can define a method or function in multiple ways. You can define a function as an anonymous function, named function, and the functions that are called and executed as soon as they are mounted which are known as "immediately invoked functions".

The name suggests everything to us. We don't need an explicit call to invoke the function. The syntax for creating "immediately invoked function" is as follows:

```
(function(){ //statements})();
```

Q29. What Are “Higher-Order Functions” In Javascript?

Ans- in JavaScript, when a function receives another function as a parameter or returns a new function or both is known as a “higher-order function”. Check the example below:

Code-

```
const msg= function(name){
return function(m){
console.log("hi"+ name);
}
}
const greet= msg("XYZ");
greet("welcome");
```

Q30. Explain Call(), Apply() And Bind() In Javascript.

Ans- call() is a predefined method in JavaScript that is used to invoke a method with the owner object as a parameter. With the call() method, an object can access the method which belongs to another object.

The apply() method in JavaScript is similar to the call() method. Both methods work in a similar way; the only difference between those is that the call() method takes parameters separately, whereas apply() method takes parameters as an array.

The bind() method, unlike the call() and apply() methods, does not instantly run the function. It simply returns a new version of the function with this set to this argument.

Q31. Explain Currying In Javascript.

Ans-Curring is not only limited to JavaScript but it is also used in various other programming languages as well. Currying is a function conversion that converts a function from being callable as f(a, b, c) to being callable as f(a)(b)(c). In simple terms, currying doesn't call a function, it just transforms or converts it. Check the example below.

```
function cur(f) { // curry(f) does the currying transform
return function(x) {
return function(y) {
return f(x,y);
};
};
}
function sum(a,b){ return a+b;}
let cursum= cur(sum);
console.log(cursum(5)(6));
```

Q32. What Is The Scope And Scope Chain In Javascript?

Ans- Scope in JavaScript is related to accessing the variables, functions, and objects in your program. The scope functionality is provided in all the programming to provide the extra layer of security. Basically, there are two types of scope- local scope and global scope.

The local scope of a variable is limited to a block of code, for e.g. a function or a loop. Whereas the global scope of a variable is not limited to the block of code, and once the variable is declared as global, it can be used anywhere in the program.

The scope chain, as the name suggests, is a chain of scopes. Let me explain. When we define a function inside another function, each of those has its own scope. The innermost function has a local scope that is linked to the outer function; this link is called a scope chain.

Q33. What Are Object Prototypes?

Ans- In JavaScript, the technique through which JavaScript objects inherit features from one another is called prototype. JavaScript is also called a prototype-based language that provides inheritance.

It’s possible that an object’s prototype object has a prototype object from which it inherits methods and attributes, and so on. This is known as a prototype chain, and it explains why distinct objects have access to attributes and methods defined on other objects.

Q34. What Is Memoization?

Ans- The terms “memorization” and “memorization” are two totally different concepts in JavaScript. It is very similar to fetching the results from cache. Memoization helps to increase the overall performance of the application by storing the results of expensive functions in the cache and returning them whenever the same inputs are supplied.

Here, expensive function refers to the functions which take a lot of time to execute and consume lots of memory during the execution.

Q35. State Use Of Constructor Function In Javascript.

Ans- Constructors are available in almost every programing language. Constructor is a special method and is used to create and initialize the object of a class. In most programming languages, the constructor has the same name as the class name; however, it totally depends on the programing language.

In JavaScript, the constructor function is used to create and initialize the Object. Like any other programing language, JavaScript also uses the “new” keyword to create an object using the constructor.

Check an example below:

```
function names(){ // constructor function
this.name="john",
this.sirname="wick"
}
const person= new names(); //creating an object
```

Q36. What Is The Arrow Function In Javascript?

Ans- The array function in the JavaScript was introduced in the ES6 version and it is supposed as one of the most popular features in ES6. It allows you to write a shorthand version of a function definition.

The previous method to create a function-

```
fun= function(a,b){
console.log("hello")
}
Using arrow function-
fun=()=>{
console.log("hello")
}
```

This syntax further can get shorter. If the function body contains only one statement that returns something, then you can write a function something like-

```
fun=()=> "hello"
```

You can use the parameters to the function as well.

```
fun=(val)=>"hello"+val;
```

Q37. Explain The Use Of The Array Slice Method.

Ans- It is one of the most important JavaScript array interview questions. There are a few methods in JavaScript with almost identical names, and students get confused between them, but you shouldn't!

The array slice method is used to return the set of elements from the array as specified in the parameters. It doesn't change the original array and returns up to (last-1) elements.

Note- The array index starts from 0.

Check the below example.

```
var names=["john", "naruto", "saitama", "gojo"];
console.log(names.slice(1,3);
//output-naruto, saitama
```

Here, we have specified parameters 1 and 3, which means we want to get elements starting from index 1 to (3-1), i.e., 2. That means we should get the second and third elements in the array.

Q38. Difference Between Object And Map.

Ans- In JavaScript, both map and object store values in key-value pairs. But there are slight differences between these two. The map is a data structure that contains the unique key to which the value is mapped. The keys are always unique in the map data structure so it avoids data duplication.

In Object, keys are generally specified as property and are associated with a single value.

The main difference between objects and maps is, keys in objects are of type strings or int, whereas, in the map, keys can be of any type.

The way to create objects and maps is also different. You don't need to use any built-in function or method to create an object, whereas to create a map, you must use a "map" constructor.

Eg. create an object-

```
const obj={
name:"john",
"age": 50
}
create a map-
const map= new map([
["name","john"],
["age",50]
]);
```

Q39. What Is The First-Class Function?

Ans- In JavaScript, we can use function as a variable and store the value of the function in a variable. The JavaScript treats function like any other variable or an object that means you can store them in a variable, pass them as arguments, return from the function, and so on.

Q40. How To Redeclare A Variable In The Switch Block Without Any Error?

Ans- If you initialize the variable in the switch case, it gets the scope of the whole block without even knowing yourself. To avoid this, you just need to put curly braces around the particular case block. Check the code below.

switch (a)

```
{
case 0:
  int b = 5;
  cout << "I declared variable b." << end;
  break;
case 1:
  cout << "variable b has scope here too!" << end;
  break;
default:
  break;
}
```

The above code will generate a compile-time error. To solve this, you can declare a variable inside of curly braces, so it will be declared as a blocked scope variable. Check the right code below.

switch (a)

```
{
case 0:
  {
    int b = 5;
    cout << "I declared variable b." << end;
  }
  break;
case 1:
  cout << "I don't have access to b." << end;
  break;
```

```
  default:
    break;
  }
```

Q41. What Is A Module? What Is The Benefit Of Using It?

Ans- Module is similar to the library, which consists of reusable code that is intended to be used as the building block in the JavaScript application. Modules are small independent units of code that let the developers define private and public data members and member functions.

In JavaScript, you can have your own created module or a third-party module as well, that you can install and use in your application via package managers like NPM. The main benefit of creating and using modules is the reusability and the app's performance.

Q42. How To Manipulate Dom Using A Service Worker?

Ans- Before seeing how to manipulate DOM using a service worker, let's see what is the service worker. A service worker is a script that runs in the background of your browser, independent of a web page, allowing you to access services that don't require a web page or user involvement.

The service worker is a JavaScript worker, so it cannot directly access and update the DOM. So instead of directly manipulating the DOM, a service worker communicates with the pages that it controls by giving a response to a message that is sent via the postMessage interface, and then those pages can manipulate the DOM.

Q43. What Is A Cookie? Why Do We Need It?

Ans- You can think of a cookie as data that is stored on your local machine as a text file. A cookie helps the website to remember the information about the user, which then helps to visit the site again. In general, it stores the information of a visited user and uses that information to make the site more useful to that user.

See also **Top 100 Ansible Interview Questions And Answers**

For example, you can store the id or username in the cookie and set the time limit to destroy itself so that the next time user opens the site within the specified time limit; the user will be logged in automatically without entering id and passwords.

Another use case of a cookie is, if your website is more like a web application like a todo app or a note app, you can store the last visited URL in a cookie so that whenever the user closes the browser and opens it again after some time, the web application will be redirected to that specific last visited URL which reminds the user what was he performing during the last session.

Q44. Difference Between A Cookie, LocalStorage, And SessionStorage.

Ans- Cookie, localStorage, and sessionStorage, are all used to store some information. We will see each of those, one by one, and you will understand the difference between them easily.

A cookie is a small text file that stores and remembers the user's data on the local machine. It holds the data in key-value pairs. You can set the expiry date/time to the cookies; if not specified, cookies will be deleted on browser close.

The localStorage and sessionStorage are relatively new features of JavaScript and not necessarily be supported by legacy browsers. The localStorage will store the data on your local machine and has no expiration time. The localStorage can store the data up to 5 MB using JavaScript and store the data as plain text.

The sessionStorage is quite similar to the localStorage, but as the name suggests, it only lasts for the session, meaning that all the data stored in sessionStorage will be wiped out on browser close.

Check out this **StackOverflow thread** to know more in-depth.

Q45. What Is The Primary Difference Between Local Storage And Session Storage?

Ans- The localStorage and sessionStorage are both used to store some information on the client-side. However, there is a slight difference between them. The localStorage stores the data in key-value pairs which have no expiration date.

This means, even if you close the browser and open it again on next day, the data/information in localStorage remains as it is. However, the user can delete all the localStorage data from the browser.

On the other hand, sessionStorage stores the data in key-value pair same as localStorage but have life until the browser close. As soon as the user closes the browser, sessionStorage information is cleared.

Q46. How To Check The Web Worker's Browser Support In Javascript?

Ans- There are two ways you can check if the browser supports web workers using JavaScript.

code-

```
tests[webWorkers]= function(){
return !!window.Worker;
}
By another way,
code- if(typeof(Worker)!=="undefined"){
//broser supports web workers
}else{
//sorry, browser doesn't support web workers
}
```

Q47. Explain Strict Mode In Javascript.

Ans- Strict mode feature is introduced in the ECMAScript version 5. The strict mode in JavaScript is achieved by the "use strict" literal expression. The strict mode won't allow you to use undeclared variables. The main purpose of introducing strict mode in JavaScript is to enhance security.

In an earlier version of JS, if the programmer mistypes a variable name, it creates a new global-scope variable. Using strict mode, there is no possibility of accidentally creating new variables.

The syntax of strict mode is=

code – `"use strict";`

Q49. What Is The Use Of Double Exclamation?

Ans- In JavaScript, !! Converts object to the boolean. If you closely look at it, you will realize it is two times “NOT”. Let me explain to you. The first “!” is NOT, and again “!” is the second NOT, which means you are first converting the value to boolean, inverting it, and again inverting it.

See, ! is the “NOT”; so, !true is false and !false is true. In terms on 0 and 1, !0 is 1 (true) and !1 is 0(false).

If the value is falsey(0, null, undefined), it will be false else true.

For example-

console.log(!!0) // returns false because 0 is falsey value

console.log(!!1) // returns true because its truthy value

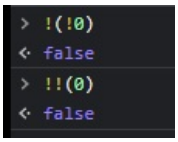
Explaination-

simplify the expression first-

ex 1- !!0 is equivalent to !(!0)

!(!0) = !(1) = (0)

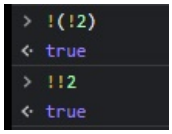
so !!0 is 0 = false.



ex 2- !!1 equivalent to !(!1)

!(!1)= !(0) = (1)

so !!1 is 1 = true



Q50. Explain The Use Of The Delete Operator.

Ans- The JavaScript delete operator is used to remove the object property.

Syntax- delete expression;

eg. code=

```
var names= {
first:"john",
last:"wick"
}
console.log(names); //show first and last names
delete names.first;
console.log(names); // only show last name
```

Q51. How Can You Access History In Javascript?

Ans- In JavaScript, history is a part of BOM, or you can say it is an object of BOM. Whenever you navigate to the different pages or open a new web page, a record is created in history. To manipulate the browser history, you get a “history” object.

Syntax of history object-

window.history;

To go backward-

window.history.back() or history.back()

To go forward-

window.history.forward() or history.forward();

To navigate using steps-

window.go(2); //to go forward

window.go(-2); //to go backward

Q52. How To Detect Caps Lock Is On Or Off In Javascript?

Ans- Capslock is a key on the keyboard so we have to use a keyboard event listener to detect if the caps lock is active. JavaScript provides us getModifierState() method of a keyboardEvent object to return the status of the modifier. If the modifier is active, it returns true else false. Check the example below.

code-

```
if(e.getModifierState('capslock')){
//show message on label capslock is on
}else{
//capslock is disabled
}
```

Where e is the event object and you can get it from the keyup or keypressed event. check code below:

code-

```
var ip= document.getElementById("inputField");
ip.addEventListener("keyup", function(event){
if(event.getModifierState('capslock')){
alert("caps lock is on");
}else{
}
});
```

Q53. What Is Event Bubbling In Javascript?

Ans-Event bubbling is a way of event propagation in the HTML DOM API. When an event is in an element inside another element and both elements have registered a handle to that event, event bubbling happens.

It's a procedure that starts with the element that caused the event and then moves up the hierarchy to the contained elements. The event is caught and processed by the innermost element first, and then transmitted to the outermost elements through event bubbling.

Q54. How To Find Os Details Using Javascript?

Ans- In JavaScript, you can find the operating system details using two ways; using either navigator.appVersion property or navigator.userAgent property. Here is the code that displays the name of the operating system.

Save the following code by .HTML extension and open it in the browser to see result.

code-

```
<!DOCTYPE HTML>
<html>
<head>
  <title> JavaScript Detecting the Operating System of User. </title>
</head>
<body style="text-align:center;" id="body">
  <h1 style="color:green;">   JavaScript <3  </h1>
  <p id="UP" style="font-size: 19px; font-weight: bold;"> </p>
  <button onclick="Fun()">  click here </button>
  <p id="DOWN"
  style="color: green; font-size: 24px; font-weight: bold;"> </p>
  <script>
    HTMLDocument.prototype.e = document.getElementById;
    var el_up = document.e("UP");
    var el_down = document.e("DOWN");
    el_up.innerHTML = "Click on the button to get the OS of User's System.";
    var Name = "Not known";
    if (navigator.appVersion.indexOf("Win") != -1) Name =
    "Windows OS";
    if (navigator.appVersion.indexOf("Mac") != -1) Name =
    "MacOS";
    if (navigator.appVersion.indexOf("X11") != -1) Name =
    "UNIX OS";
    if (navigator.appVersion.indexOf("Linux") != -1) Name =
    "Linux OS";

    function Fun() {
      el_down.innerHTML = Name;
```

```
    }
  </script>
</body>
</html>
```

Q55. Difference Between Native Objects And Host Objects.

Ans- The native objects and host objects specification is available in the ECMAScript 2015. Host Items are objects that come from a certain context. They aren't always the same since each environment is unique and has host objects that enable ECMAScript execution. The browser environment, for example, provides things such as windows. Objects like NodeList are available in a node.js/server context.

On the other hand, Native Objects, also known as Built-in Objects, are JavaScript's standard built-in objects. Native objects are also known as 'Global Things' because they are objects that JavaScript has made available for use natively. There are a number of native objects available, which you can check on **Mozilla's official website**.

Q56. How Can You Debug The Javascript Code?

Ans- There are multiple ways to debug the JavaScript code. You can use the browsers' debugging environment which is common for most developers out there. All the major browsers like Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge, etc. provide debugging tools.

Apart from this, you can have the following ways to debug the JavaScript code.

- console.log() method to check whether a particular statement throws the error.
- You can set breakpoints in the browser's debugger tool.
- Using the "debugger" keyword in JavaScript, you can easily debug the code.

Q57. What Is A Callback In Javascript?

Ans- As the name suggests, call the function after the particular work completes. In simple terms, the callback in JavaScript is a function that is passed to another function as a parameter to be executed later. The callback function executes when another function completes its execution. For example, when you have to perform operations on the numbers after the addition is done. Check the example below.

Code-

```
function display(sum){
console.log(sum/10); //displays the division
}
function addition(a,b,callback){
let sum= a+b;
callback(sum); //calls the display function
}
addition(10,10,display); //function call with the display function as a parameter
```

Q58. What Do You Mean By Escape Characters In Javascript?

Ans-The symbol used to start an escape command in order to perform some activity is known as an escape character. Escape characters are present in almost every programming language and all of them are nearly identical.

In JavaScript, there are a bunch of escape characters present that you can use throughout the program. Following are the available escape characters in JavaScript-

- \n = new line
- \b = backspace
- \\ = backslash
- \” = double quote
- \’ = single quote
- \t = horizontal tabulator
- \v = vertical tabulator

Q59. How Can You Use An External Javascript File?

Ans- External JavaScript files are useful for code reusability, which simply means you can write the piece of code in a separate file that you need multiple times in your application and include it in the main application file.

In the HTML file, you can include the js file in following way:

```
<script type="text/JavaScript" src="jsfile.js"> </script>
```

Q60. Tell The Purpose Of The Window Object.

Ans- In JavaScript, the Window is a tab that is opened in front of you. The window object has document property which consists of the DOM document and the nodes and methods which we can use for DOM manipulation.

When you create any variable without the let, const, or var keywords, it automatically becomes a part of a window object. In a web browser, the window object refers to the opened tab and there can be any number of window objects depending on the number of tabs opened.

Q61. What's The Difference Between Event.Preventdefault() And Event.Stoppropagation() Methods In Javascript?

Ans- The two methods, event.preventDefault() and event.stopPropagation() are used to prevent something, but what? The preventDefault() method prevents the browsers' default action like clicking on the link redirecting you to a new page.

On the other hand, the stopPropagation() method prevents further propagation in the capturing and bubbling phase of the current event.

Q62. How To Convert String To Integer In Javascript?

Ans- Converting a value from one type to another type is tricky and many programming languages offer prebuilt functions to do so. JavaScript offers you an easy way to convert a string to an integer using the method called "parseInt".

The syntax to convert a string to an integer is:

parseInt(string);

eg.

- `parseInt("10") //returns 10`
- `parseInt(" 10 ") // returns 10`
- `parseInt("10 ok") // returns 10`
- `parseInt("ok 10"); // returns NaN`

Key points to remember before using parseInt method in JavaScript-

- It ignores trailing and leading spaces
- Only the First integer in the string found is returned
- The first character is parsed first to the int and converted; returns NaN if fails

Q63. Explain Import And Export Keywords In Javascript.

Ans- As the names suggest, these keywords are used to import and export the data members and member functions from and to different js files.

The import and export keywords were introduced in the ES6 version. Using the ES6 standards, we can create modules in the JavaScript, which can be used in other js files for reusability. These modules contain classes, variables, and functions.

The export keyword is used to export a variable_name from the current file.

The import keyword is used to import a variable_name from another file that exports that variable.

The variable_name is anything like a function, class, or whole js file.

Q64. Difference Between View State And Session State.

Ans- In JavaScript, ViewState is used for client-side state management, whereas sessionState is used for server-side state management.

In ViewState, changes made on one page don't reflect on other pages. Because it is client-side state management, data gathered using ViewState is stored for the client and cannot be sent across.

On the other hand, sessionState stores and maintains the data on the server-side. The information that is gathered by sessionState remains as long as the user is active and gets destroyed when the session is completed, or the user closes the browser.

Q65. Is Javascript Faster Than Asp? Justify.

Ans- JavaScript is faster than ASP because it is a client-side language, whereas ASP is a server-side language. JavaScript doesn't need to communicate with the server as it is made for client-side scripting purposes, but ASP needs to communicate with the server in order to exchange information.

However, the NodeJS, which is a run time of a JavaScript that is made for server-side scripting, is comparatively slower than ASP.

Q66. Explain The Use Of The Debugger Keyword.

Ans- debugger keyword in JavaScript is the same as a breakpoint used in the various debugger tools. The debugger keyword stops the execution before the next instruction executes in the browser.

Copy and paste the following code into an HTML file and run it in the browser to check the working of the debugger keyword.

```
<!DOCTYPE HTML>
<html>
<head>
  <title>JavaScript debugger</title>
</head>
<body style="text-align:center;" id="body">
  <h4> demonstrations of Debugger keyword</h4>
  The solution of 10 * 20 is:
  <p id="test"></p>
  <p>code stopsexecution at the start of the debugger If the debugger is turned on</p>
  <script>
    debugger;
    var x = 20;
    document.getElementById("test").innerHTML = x;
    debugger;
    var y = 5;
    document.getElementById("test").innerHTML = y;
    var z = x * y;
    debugger;
    document.getElementById("test").innerHTML = z;
  </script>
</body>
</html>
```

Javascript Closure Interview Questions

JavaScript closure, hoisting, callbacks, and promises are some of the important topics for advance level development. There is no doubt you will face half of the questions based on these topics if you are an experienced programmer.

See also **Top 100 Ansible Interview Questions And Answers**

Q67. What Is Closure In Javascript? What Is It Used For?

Ans- Closure is one of the important parts of the modern JavaScript paradigm. A closure is a function containing references to its surroundings that have been bundled together (enclosed) (the lexical environment).

Simply put, a closure enables an inner function to get access to the scope of an external function. When a function is generated in JavaScript, closures are formed throughout the function creation process.

Q68. Tell Me The Advantages Of Closure.

Ans- The closure allows to bind a variable to an execution context in the JavaScript. In simple terms, the main purpose of the closure is data encapsulation.

Because most APIs that require callback functions (for example, an “onclick” function) do not provide other means to

transmit parameters to those callback functions, closures are required in JavaScript (or to explicitly set the “this” pointer). To allow the callback to access variables in the “parent” function, you must use closures.

Q69. State The Difference Between Closure And Scope.

Ans- The closure and scope can be explained in simple terms. The variable is scoped to the particular function if it is declared inside the function and can’t be used outside it.

On the other hand, when you define a function inside of another function then the inner function is called a closure.

Q70. What Is Promise In Javascript?

Ans- As the name suggests, promise in JavaScript assures you a result. In any code, there is two following- a producing code and a consuming code. A producing code is something that performs something and takes time. On the other hand, a consuming code is something that takes results from the producing code.

A promise in JavaScript is an object that connects producing code and consuming code. As soon as producing code gets the result, it must call the consuming code; either the output is successful or generates an error.

Javascript Promises Interview Questions

Q71. What Are The Three States Of Promise?

Ans- In JavaScript, a promise can be pending, fulfilled, or rejected. There are two properties of a promise; state and result. When the promise is pending, the result is undefined. When the promise is fulfilled, you get value as a result. When the promise is rejected, you get an error object as a result.

Q72. State Benefits And Drawbacks Of Promises Over Callbacks.

Ans- There are advantages and disadvantages as well of using promises in JavaScript over callbacks.

The advantages are:

- Good control flow of asynchronous logic.
- Code maintainability
- Reduced coupling

The disadvantages are:

- Kills the intention of asynchronous programming
- One Object is returned only
- multiple arguments cannot be returned

Q73. How To Write Sleep Function Using Promise?

Ans- Following code is used to write sleep function using promise in JavaScript.

code-

```
function sl(timeMS) {
```

```
  return new Promise(result => {
    setTimeout(result, timeMs);
  });
}
```

Javascript Hoisting Interview Questions

Q74. What Is Hoisting? What Is The Use Of It?

Ans-Hoisting is the default behavior in JavaScript, which moves all declarations to the top of the scope before code execution. Essentially, it allows us to benefit from the fact that no matter where functions and variables are declared, they are moved to the top of their scope, whether global or local.

Q75. How Many Ways Can You Create Function Hoisting In Javascript?

Ans- In JavaScript, you can create a function in two ways, through function declaration and function expression. The function declaration hoists the fiction definition in JavaScript. On the other hand, function expressions are not hoisted in JavaScript. Hence, there is no way to use function expressions before you define them.

HTML CSS JavaScript Interview Questions

Since JavaScript is a client-side programing language, you may be asked some frontend questions as well. Hence, we have found a bunch of JavaScript front end interview questions here if you are giving an interview for frontend developer role.

These are some entry level JavaScript interview questions which include basic HTML and CSS questions as well.

Q76. What Is The Role Of Javascript In Client-Side Programming?

Ans- JavaScript is a client-side programming/scripting language and is the backbone of the webpage to make it interactive. In short, it is responsible for making a webpage alive.

Using JavaScript for the web, you can perform multiple things like DOM manipulation, event handling, and so on. JavaScript is not limited to the web; you can use it for mobile application development as well.

Q77. In How Many Ways Can You Validate Email In Javascript?

Ans- There are two ways you can validate an email in JavaScript; using either a prebuilt feature or a regex (regular expression). The HTML allows you to set the type of input field on the form so that it will automatically understand if the entered value is valid or not. Some of the prebuilt types in HTML forms are email, number, text, radio, checkbox, submit, etc.

However, the email type of input field is not a strict validator for an email. If you want to check an email as per the standards, you can use the following regular expression.

regular expression for email validation- /^(([^<>()[\]\\.,;:\s@”]+(\.[^<>()[\]\\.,;:\s@”]+)*)|(“.+”))@((\[[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\])|(([a-zA-Z\-0-9]+\.)+[a-zA-Z]{2,}))$/

Here is the JavaScript code to perform email validation-

```
const validate= (email) => {

 return String(email).toLowerCase().match(

/^(([^<>()[\]\\.,;:\s@"]+(\.[^<>()[\]\\.,;:\s@"]+)*)|(".+"))@((\[[0-9]{1,3}\.[0-9]{1,3}\.[0-9]
{1,3}\.[0-9]{1,3}\])|(([a-zA-Z\-0-9]+\.)+[a-zA-Z]{2,}))$/

  );};
```

Q78. How Can You Change The Html Document Background Color Using Javascript?

Ans- To change the HTML document background color, you can use the DOM manipulation property of the JavaScript. Following is the code to change the background color of a body of an HTML page when the user clicks on a button.

```
<!DOCTYPE HTML>

<html>

<head>

  <title>JavaScript change background color</title>

</head>

<body style="text-align:center;" id="body">

  <button id="btn">change body background color</button>

  <script>

    document.getElementById("btn").addEventListener("click",function(){

      document.getElementById("body").style.backgroundColor="red";

    })

  </script>

</body>

</html>
```

Q79. How Can You Submit A Form Using Javascript?

Ans- You can submit a form using JavaScript by clicking on a button. You can add an event listener to the button and perform a submit action on it. Check the example below:

```
<!DOCTYPE HTML>
<html>
<head>
  <title>JavaScript submit form using JavaScript</title>
</head>
<body style="text-align:center;" id="body">
  <form action="" id="form">
    <label for="">enter name</label>
    <input type="text" name="name">
    <br>
    <input type="button" onclick="submitForm()" value="submit">
  </form>
  <script>
    function submitForm(){
    document.getElementById("form").submit();
    }
  </script>
</body>
</html>
```

Q80. Explain How To Submit A Form On Link Click In Javascript.

Ans- You can submit a form by clicking on a link as well in the JavaScript. Check the code below-

```
<!DOCTYPE HTML>
<html>
<head>
  <title>JavaScript submit form on link click</title>
</head>
<body style="text-align:center;" id="body">
  <form action="" id="form">
    <label for="">enter name</label>
    <input type="text" name="name">
    <br>
    <a href="#" onclick="submitForm()">submit Form</a>
  </form>
  <script>
    function submitForm(){
    document.getElementById("form").submit();
    }
  </script>
</body>
</html>
```

Q81. State Difference Between Innerhtml And Innertext.

Ans- The innerHTML returns everything that a block of HTML element consists including other HTML tags. On the other hand, innerText only returns the plain text. Check the below example to understand it better.

code-

```
<!DOCTYPE html>
<html>
 <head>
  <title>innerText vs innerHTML</title>
 </head>
```

```
<body style="text-align: center" id="body">
 <div id="div">
  <h3>h3 tag</h3>
  <p>another p tag</p>
 </div>
 <button id="btn1" onclick="innnerHTML()">innerHTML</button>
 <button id="btn2" onclick="innnerText()">innerText</button>
 <script>
  function innnerHTML() {
   alert(document.getElementById("div").innerHTML);
  }
  function innnerText() {
   alert(document.getElementById("div").innerText);
  }
 </script>
</body>
</html>
```

innerHTML-

innerText-

Q82. Name A Few Javascript Frontend Libraries And Frameworks.

Ans- There are thousands of JavaScript libraries and frameworks available out there. However, here are among the famous in the JavaScript community as per the usage, simplicity, flexibility, and features. These libraries and frameworks are created to serve different purposes like charts, images, DOM manipulation, games, forms, animation, math functions, UI, etc.

- jQuery- used for Easy DOM manipulation, JSON parsing, AJAX, etc.
- ReactJS- Frontend library for creating interactive single-page website/applications (SPA).
- AngularJS- A JS framework to create modern and dynamic single-page applications.
- AnimeJS- Popular JS library for animations.
- AOS- Library to add effects to element on scroll.
- ChartJS- Most popular library to create charts in your website/web application.

Q83. What Is The Difference Between Client-Side Javascript And Server Side Javascript?

Ans- The client-side JavaScript is used to perform actions on the user side and doesn't exchange data with the server. JavaScript is basically developed for client-side programming.

However, NodeJS, a JavaScript runtime, is developed to use JavaScript as a server-side programming language. Using server-side JavaScript, you can interact with the database, exchange data from and to the client's machine and server.

Q84. In How Many Ways, You Can Access An Html Element Using Javascript?

Ans- Using DOM manipulation in JavaScript, you can access an HTML element in multiple ways. Here are the ways to access an HTML element with the syntax-

- get element by ID- document.getElementById(id);
- get element by class- document.getElementsByClassName(class);
- get element by name- document.getElementsByName(name);
- get element by tag name – document.getElementsByTagName(name);

Q85. Explain Load And Domcontentloaded In Javascript.

Ans- The load and DOMContentLoaded are two events in JavaScript that are used to check if the webpage is loaded completely. However, both work in a slightly different way.

The DOMContentLoaded gets executed as soon as the basic HTML structure of the page gets loaded, which only consists of HTML tags and excludes CSS, js files, and images.

On the other hand, the load event gets executed as soon as the whole HTML page including the CSS, js files, and images are loaded.

Q86. In How Many Ways, Javascript Code Can Be Involved In Html Files?

Ans-You can include the JavaScript code in two ways in your HTML document, either by writing a code inside a script tag or adding an external js file.

For e.g.,

Method 1-(code)-

```
<body>
<h1>heading</h1>
<script>
//js code
</script>
</body>
```

method 2- (code)

```
<body>
<h1>heading</h1>
<script type="text/JavaScript" src="externalJSFile.js"></script>
</body>
```

JavaScript Algorithms Interview Questions

Q87. Check Number Is Prime Or Not.

Ans-

```
function checkPrime(n) {
  var dvsr = 2; //start divisor from 2
  while (n > dvsr) {
    if (n % dvsr == 0) {
      return false;
    }
    else
      dvsr++; //increment division
  }
  return true;
}
console.log(checkPrime(19));
```

Q88. Find The Nth Fibonacci Term.

Ans-

```
function fib(n) {
  var fibe = [0, 1];
  if (n <= 2) {
    return 1;
  }
  for (var i = 2; i <= n; i++) {
    fibe[i] = fibe[i - 2] + fibe[i - 1];
  }
```

```
  return fibe[n];
}
console.log(fib(4));
```

Q89. Swap Two Numbers.

Ans-

```
function swap(a, b) {
  console.log(`before swap=a=${a} , b= ${b}`);
  var t;
  t = a;
  a = b;
  b = t;
  console.log(`after swap=a=${a} , b= ${b}`);
}
swap(10,20);
```

Q90. Swap Two Numbers Without A Third Variable.

Ans –

```
console.log(`before swap=a=${a} , b= ${b}`);
  b = b - a;
  a = a + b;
  b = a - b;
  console.log(`after swap=a=${a} , b= ${b}`);
}
swap(10, 20);
```

Q91. Reverse The String

Ans-

```
function reverseString(string){
  var reverse = '';
  for(var i = string.length-1; i>=0;i--){
   reverse +=string[i];
  }
  console.log(reverse);
 }
```

```
reverseString("reverse string");
```

Q91. Check Whether The Word Is Palindrome Or Not.

Ans-

```
function checkPalindrome(string) {
  var i, length = string.length;
  for (i = 0; i < length / 2; i++) {
    if (string[i] !== string[length - 1 - i])
      return false;
  }
  return true;
}
console.log(checkPalindrome("tut")); //true
console.log(checkPalindrome("tuts")); // false
```

JavaScript Es6 Interview Questions

(https://www.javatpoint.com/es6-interview-questions)

If you want to make a career as a JavaScript developer then you must be familiar with ECMAScript or ES. It is a superset of JavaScript and releases new features and upgrades. The major release of ECMAScript is ES6 which was released in 2015; therefore, it is often called ECMAScript 2015 as well.

Q92. Explain Ecmascript. How Is It Related To Javascript?

Ans- You can think of an ECMAScript as a parent that creates a set of rules for the scripting languages such as JavaScript, script, etc. JavaScript is based on the rules of ECMAScript standards.

Q93. State New Features Introduced In Es6.

Ans- The ES6 was the major release for JavaScript that includes a bunch of new features which helped programmers to overcome various difficulties. Following are the features that were introduced in ES6.

- arrows
- classes
- object literals
- template strings
- destructuring
- let and const keywords
- Unicode
- modules

- map, set, weakmap, weakset
- proxies
- symbol
- math, number, string, array, and Object APIs
- tail calls
- reflect API

Q94. Explain Let And Const Keywords.

Ans- In JavaScript, you can decide the scope of the variable by the way how you declare it. There are three ways you can declare a variable in JavaScript; using var, let, and const.

Let and const keywords are used to create a block-scope variable. The let and const keywords are very similar to each other. The only difference between them is that you cannot reassign the value to the variable which is declared as const.

Code-

```
let a=10;
a=20;
console.log(a); //returns 20
const b=20;
b=30;
console.log(b); //returns error
```

Q95. What Are Template Literals?

Ans- Template literals, also known as template strings, is one of the best features introduced in version ES6. Alternatively, you can call it a string template or a back-tics syntax.

Using template literals, you can create a string that starts and ends with the backticks (`) which is capable of consisting of expressions. It helps you to write double-quoted strings inside it or display variables and concatenate two strings easily as well.

for example-

```
var a=10, b=20, str="string";
console.log(`values of a=${a}, b= ${b} , str= ${str}`);
//output- values of a=10, b= 20 , str= string
```

Output-

Q96. Explain Destructuring Assignment.

Ans- As the name suggests, restructuring allows you to break down the arrays, objects into separate values. It just unpacks the value from an array or an object and does not destruct or modify the existing array/object.

Example-

```
class cl {
constructor(){
console.log("constructor");
}
  fun() {
    console.log("JavaScript method inside class");
  }
};
var c = new cl(); //creating object c of class cl
c.fun(); //calling a method
//output-
constructor
JavaScript method inside class
```

Q98. What Is The Generator Function?

Ans- A generator function is defined as functionName* that returns a Generator object in JavaScript. A generator function is a function that can be exited and later entered again. The Generators in JavaScript are more powerful when used with promises in asynchronous programming.

Q99. Explain Default Parameters.

Ans- As the name suggests, default parameters are the default values when no parameter is passed. Check the example below –

```
function mul(a, b = 10) {
  console.log(a * b);
}
mul(10, 20); //output -200
mul(10); //output - 100
```

Q100.Define Babel.

Ans- Writing a JavaScript code that runs in the older version of the JavaScript engine is a kind of headache. That is where Babel comes into action. Babel is a transcompiler that converts ES6+ code into a backward-compatible version of JavaScript which can run in older JS engines.

Conclusion

As stated earlier, JavaScript is one of the most popular programming languages and in high demand. To get a high-paying job, these JavaScript interview questions will help you to clear the coding rounds as well as interview rounds.

It is recommended to try these questions and coding answers yourself before attending any interview so that you will have a rough experience about how the particular function or method works and you won't get confused when you are asked any question. Happy coding!

Interview Questions
‹ 15 Best Free Photo Editor Software For Windows PC
› Top 30+ Best Internet Of Things Companies In The World

© 2022 Software Test Tips

Top 55

JavaScript

Interview Questions

& Answers

WSCUBE TECH®
System For Satisfaction

JavaScript Interview Questions for Freshers & Beginners

1. What is JavaScript?

JavaScript, also known as JS, is a popular dynamic programming language used for front-end development. The primary purpose of using JS on web pages is to show things in a dynamic manner.

In front-end development, HTML and CSS are used to show static content on a web page. In scenarios where you need to show dynamic content, animation, interactive maps, etc., the role of JavaScript comes into play. Developers call it the third layer, followed by HTML and CSS.

You probably have seen the use of JS if you have gone through websites where additional content gets added without reloading, changing colours on the pages, etc. The use of JavaScript programming is common in website development, and web application development, as well as in servers, browsers, games, etc.

Over the years, it has become an in-demand language, and an increasing number of people are learning it with reliable web development course online or offline.

2. What is JavaScript Promise?

A Promise in JavaScript is an object or proxy that sets up the connection between producing code and consuming code. It shows whether the asynchronous operation is pending, fulfilled, or rejected. And if it is fulfilled, it will also show the result.

3. What is JavaScript closure?

When you need access to the features of a function that is located outside the inner function, the role of JS closure comes into play. It is such an option that helps even when the function located outside is closed.

4. What is JavaScript array?

In JS, an array is a variable that can store multiple values. In simple terms, you can say that an array has the ability to store a set of more than one item with the name of a single variable.

5. What are the characteristics of an array in JavaScript?

Following are the main characteristics of JavaScrip array:

- Resizable
- Zero-indexed
- Ability to build shallow copies
- Feature to store collection of different items or data types

6. What is JavaScript function?

A function in JavaScript is a set of code which is used to do a certain task. It can also be used for calculation. Before using the function, ensure that it is defined somewhere.

The *function* keyword in JS is used to execute the functions.

7. What is JavaScript object?

An object in JS is like a thing in real life, having some properties, methods, behaviour, etc. For example, a bike is an object with properties like color, brand name, model, etc. The methods of this bike can be a brake, stop, start, drive, etc.

8. What is isNan() function in JavaScript?

The full form of NaN is Not a Number. So, in simple terms, the isNaN() is a function in JavaScript that is used to check whether a value is a number or not once it is converted to a number.

If the value is Not a Number, it will show true. It should be noted that this function first changes the values to numbers and then checks whether it is NaN or not.

9. What is the difference between client-side and server-side JS?

Such JavaScript interview questions can be asked to both freshers and experienced developers. So, let's know the primary differences between client-side scripting and server-side scripting in JavaScript.

Client-Side	**Server-Side**
Users can see the source code	Users can't see the source code
It serves the requests of the users and shows the content accordingly.	It is used to manage the database and other backend aspects.
Dependent on browser	No dependency on browsers
Runs on the web browsers	Runs on the web servers
Less data security	Strong data security

10. Is JavaScript case-sensitive?

Yes. It is a case-sensitive language.

11. What is JavaScript DOM?

DOM stands for Document Object Model. It is an interface in coding that is used to represent the data and content of a web document. It helps in enabling programs to modify the style, content, and structure of a web document.

For instance, you can call the web page a document. If you want to edit this page in JavaScript, then the role of DOM comes into play.

12. What are operators in JavaScript?

An operator in JS is simply a symbol that is utilized to do specific operations, like addition, subtraction, and more. The operations are done on values and variables, or you can call them operands.

Let's understand it with an example. If you want to add two values, then the "+" operator will be used.

10 + 45: // 55

Here, + is an arithmetic operator, which has added the given values. 10 and 45 are the operands here.

13. What are the different operators in JavaScript?

Below is a list of JavaScript operators with their details:

- Assignment Operators

Used when you need to assign values to variables.

Operator	Name
=	Assignment operator
+=	Addition assignment
-=	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Remainder assignment
**=	Exponentiation assignment

- Comparison Operators

Role of these operators is when you want to do a comparison of two values and then have a result. The result will be a boolean value, showing true or false.

Operator	Name
==	Equal to

!=	Not equal to
===	Strict equal to
!==	Strict not equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to

- Arithmetic Operators

Role of these operators is to do arithmetic calculations.

Operator	Name
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder
++	Increment
- -	Decrement
**	Exponential

WsCUBE TECH® System For Satisfaction

- Logical Operators

Used to do logical operators. After the operation, it shows a boolean value (true/false) as the result.

Operator	Name
&&	Logical AND
| |	Logical OR
!	Logical NOT

- String Operators

Used for concatenation of strings.

Operator	Meaning
typeof	To know the type of the variable
instanceof	To know whether an object is an instance of an object type

- Bitwise Operators

Used when you need to do operations binary numbers.

Operator	Name
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
>>>	Zero-fill right shift
>>	Right shift
<<	Left Shift

WSCUBE TECH® System For Satisfaction

14. What is JavaScript event?

Every action that occurs in code or programming is an event. For instance, the loading of a page is an event. Clicks by the end users on a button are an event. The role of events in JavaScript is to make the program understand that some action has been taken and to react accordingly to that action.

15. What is JavaScript callback?

The callback is a function in JavaScript. Its role is to work as an argument by getting passed to other functions. When it is passed to the other functions, those functions will also have the argument executed.

Since it is passed in functions to call back the arguments in more functions as well, its name is justified. It must be noted that you can use callback in other functions only if they are already executed.

16. How to write hello world in JS?

This is one of the basic JavaScript interview questions for freshers. Writing a hello world program in JS has three different ways, as discussed here:

- console.log()

```
// write hello world program in JS
console.log('Hello World');
```

- alert()

```
// write hello world program in JS
alert("Hello, World!");
```

- document.write()

```
// writing hello world program in JS
document.write('Hello, World!');
```

17. What is BOM in JavaScript?

BOM stands for Browser Object Model. This is one of the most crucial parts of JavaScript in terms of using it on the web part. It helps you to know the functionalities of a browser using objects. As a result, the JS-based projects can interact with the browser.

18. What are comments in JavaScript?

JS comments are simply used to represent a comment or message related to the code. For example, we can use comments to mention details of the code, recommendations, warnings, etc. It should be noted that the JS engine will neglect the comments.

19. What is runtime environment in JavaScript?

The runtime environment is the place to run or execute a program. This environment can access the objects, libraries, etc. and hence runs the code accordingly.

Generally, there are two types of JS environments used by a developer. One is the browser-based runtime environment, and another is the NodeJS-based runtime environment.

The browser-based runtime environment makes use of a JS engine, web APIs, an event loop, and the callback queue.

20. What are the advantages of JavaScript?

Following are the primary advantages or benefits of using JavaScript:

- Fast Performance

Since JS is an interpreted language, you don’t have to compile it every single time it runs. As a result, it speeds up the development process. Furthermore, it runs on the client side,

where it doesn't depend on the server. This again optimizes the performance of the website or web app by avoiding the need for requesting resources from the web server.

- Easy to Understand and Simple

Even if you are a beginner with no knowledge of coding, you will find this language easy to learn. Opting for an offline or online web development course further makes things effortless for you so that you can become well-versed with the complete website and web application development skills.

- Works Well With Other Languages and Systems

JavaScript is an interoperable language which means it runs well with other languages and web technologies.

- Cross-platform Development

There are several JS frameworks that help you to use JavaScript for cross-platform development. For example, you can use React Native framework to use JS for mobile app development. You can use Electron to implement JS for desktop app development.

- Frameworks and Libraries

The wide range of JavaScript libraries and frameworks make it an ideal choice for programmers and developers. The tasks can be performed with minimum efforts, accelerating the development process. In addition, these tools enable plenty of other benefits, like cross-platform development.

21. What is JavaScript used for?

Here are the primary uses and applications of JavaScript:

- Creating Interactive Web Pages

Using JS, developers can make web pages more interactive and appealing. For example, new features can be added that bring some functionalities to the users and also enhance the overall UX.

Here are some examples of JavaScript in use on web pages.

- Hover colours on buttons
- Carousel slider
- Images zoom in/out
- Countdown timer
- Animations
- Hamburger menu

WSCUBE TECH® System For Satisfaction

- Web-based Game Development

You can build games using JavaScript that runs on browsers. For example, you can build puzzles, racing games, role-playing games, etc. A few of the popular games running on JavaScript include Angry Birds, CrossCode, Polycraft, HexGL, etc.

- Web App Development

In addition to robust and highly-interactive websites, you can also develop web applications using JavaScript. For instance, Google Maps is one such web app where JS is used.

- Mobile App Development

Using the wide range of JavaScript frameworks available today, you can also develop mobile games. For example, React Native is one popular JS framework for app development.

- Web Servers

Numerous server-side activities can be managed using JavaScript and relevant frameworks.

- User input validation when a user is about to submit a form
- Show pop-ups and dialog boxes
- Change the look of HTML docs
- Setting up dynamic forms
- Show date and time

22. Who developed JavaScript?

Brendon Eich developed JavaScript.

23. What are the disadvantages of JavaScript?

While preparing for the JavaScript interview questions and answers in 2023, don't forget to know about the main drawbacks or disadvantages of JS.

- Non-restrictive

Compared to other languages, JS is a bit more non-restrictive. Developers need to be attentive so that they can implement everything as required. That's why good web development courses cover these aspects of JavaScript so that the developers know the best practices.

- Tricky Debugging

Since JS runs on the client side in the browser, the debugging options and features are limited.

- Security

Again, the client-side running of the language makes it a bit less secure. Developers need to be aware of these aspects of JavaScript so that they don't leave any space available for attackers.

24. What is the difference between JavaScript and Java?

There are several differences between Java and JavaScript. Whether you are a fresher or an experienced professional, this is among the top JavaScript interview questions for you.

Below, we have curated a tabular comparison of Java vs JavaScript so that it becomes easier for you to understand the main differences.

Java	**JavaScript**
Object-oriented programming language	Object-based scripting language
Can be used for complicated tasks and processes	Can't be used for complicated tasks
Needs code compilation	Text-based code
Independent language	Needs to be used with HTML
Strongly typed programming language. Need to declare variables before using them in the program.	Loosely typed language. No issues whether data types are declared or not
It's statically-type	It's dynamically-typed
High memory consumption	Low memory consumption
Saved as byte code	Saved as source code
For concurrency, it uses threads	For concurrency, it uses events
.java extension used to save programs	.js extension used to save programs
Supports multithreading	Doesn't support multithreading
Objects are based on class	Objects are based on prototype
Need JDK or Java Development Kit to run the code	Need text editor to run the code
Primarily used for backend development	Can use for both front-end and back-end

25. What was the first name of JavaScript?

During the interview, you can be asked what was JavaScript called when it was developed, or what was the former name of JS. You must be prepared for such JavaScript interview questions and answers.

The original name of JavaScript was Mocha when it was developed. It was later changed to LiveScript and then finally became JavaScript.

26. When was JavaScript invented?

It was developed in 1995.

27. What is JavaScript ES6?

ES6 stands for ECMAScript 6. It is a version of JavaScript which was released in 2015. Hence, it is also called ES 2015.

There are plenty of new features in JavaScript ES6 compared to the prior versions. These are intended to write better and clean code, while helping developers to get more done with less code.

28. What is a named function in JS?

As the term suggests, a named function is simply a function in JS that has been given a name. This name is given with the use of syntax. The function keyword in JavaScript is used to assign a name to the function or make it a named function. This concept should be in the list of top JavaScript interview questions and answers for freshers.

29. What is an anonymous function?

Unlike named functions, anonymous functions are the ones that do not have any names. Such functions are declared without the use of identifiers.

There are certain rules for using an anonymous function in JavaScript. First, you can't access it right after creating it. For this, you need to use a variable where the anonymous function is saved.

30. What is the difference between JavaScript and JScript?

You need to know the differences between the two as these sound similar. This can be one of the top JavaScript interview questions for freshers.

JavaScript	JScript
A trademark of Oracle Corp.	Microsoft owns JScript
No support for active content creation	Supports active content creation
Can manage multi-browser compatibility	Supports Internet Explorer only
Code runs in browser	Code runs only in Internet Explorer
Can't access objects of the browser	Can access objects of Internet Explorer
Widely used	Not popular as it is supported by Internet Explorer only

JavaScript Interview Questions for Experienced

31. Which are the different states of a JS Promise?

JavaScript Promise can have any of the three states:

- **rejected**: when the operation fails
- **pending**: when it's neither rejected nor fulfilled
- **fulfilled**: when the operation is successful

32. What is the difference between event.preventDefault() and event.stopPropagation() methods in JavaScript?

Among the list of JavaScript interview questions for 3 years experience or above, keep such concepts and learn about them.

- preventDefault() method

It is used when you want to *prevent* the *default* action of elements in the browser. You must know that not all actions can be cancelled, and is applicable to only cancellable events.

Syntax:
event.preventDefault();

- stopPropagation() method

It is used to stop the propagation of the parent element so that it can not find access to specified events.

Syntax:
event.stopPropagation();

33. How many ways are there to create an object in JavaScript?

There are three ways to create a JavaScript object:

Ways	Syntax
Object literal	object={property1:value1,property2:value2.. ...propertyN:valueN}
Using new keyword	var objectname=new Object();
Using object constructor	var objectname=new Object();

34. Which method can we use in order to find out the character from a particular index?

For this, we can use the string charAt() method.

For example:

```
var str="WsCube Tech";
document.writeln(str.charAt(3));
```

35. How can we use an external JavaScript file?

For doing so, we need to add the below-mentioned script tag to the header:

```
<script type="text/javascript" src="wscubetech.js"></script>
```

Here, wscubetech.js is the name of the external JS file we are looking to use.

36. What is JavaScript prototype?

A prototype in JS is one of the most useful objects using which developers can inherit one object's features to another. You can say that a prototype is linked to all the objects and functions in JS. That's what it makes capable of inheriting the features of an object.

37. What is the role of JavaScript callback function?

Since numerous actions in JS are asynchronous, these actions are not capable of holding a program from running before completion. Using the callback function, we can define when to run the code.

WSCUBE TECH
System For Satisfaction

38. What is the use of window object in JS?

In JavaScript, the window object shows that a window is open in the browser. This object supports all the popular web browsers and therefore the objects, functions, as well as variables are members of the window object.

Moreover, you can say that the global variables of JS are considered properties, whereas the global functions are considered methods of the window.

39. What is the use of history object in JavaScript?

The use of the JS history object is to find the web URLs that a user has visited in the browser. In addition, it also helps in browsing the last and next pages, as well as specific pages that a user has visited.

As an important property of the window object, the history object is accessible by:

- window.history
- history

40. How to use JavaScript in HTML?

The primary purpose behind using JavaScript on HTML pages is to make them interactive and dynamic.

For adding JS code to HTML code, the <script> tag is used. The JS code that you want to add to the HTML is used within the <script> tag, whether you want to place it in the <head> or <body> section.

This method is recommended for small codes. However, for JS scripts that are heavy, it would be great to opt for a dedicated JavaScript file. It helps you to reduce the overall maintenance of the site, differentiates the JS code and HTML code, and optimizes the loading speed.

41. What are the benefits of using JavaScript comments?

If you have been a developer using JS for quite some time now, then this can be among the top JavaScript interview questions for experienced professionals. You must know that there are a couple of good reasons to use comments in JavaScript.

- Adding information about the program or specific lines of code, helping the users to get the scope or use of that code.
- There are several instances when you need to write a few lines of code on a temporary basis. On such occasions, you can add a comment to the code like “to be removed later on”. And when you want to remove it, the comment helps you easily find that code, instead of searching or reading every line of code.

WsCube Tech®
System For Satisfaction

42. What are the different types of JS comments?

Whether you are a beginner or someone with experience, knowing the different types of comments in JavaScript is important. It’s because this can be among the top interview questions on JavaScript for freshers and experienced both.

To answer, start by saying that there are two types of comments.

- Single-line comments

These comments are written with two forward slashes (//) before your message of the comment. You can use single-line comments before and after the statement.

Before statement

```
<html>
<body>
<script>
// This is your single-line comment
document.write("Welcome to JavaScript Interview Questions");
</script>
</body>
```

</html>

After statement
<html>
<body>
<script>
var x=15;
var y=30;
var z=x+y;
//adding the value of variables x and y
document.write(c);//add 15 and 30
</script>
</body>
</html>

- Multi-line comments

Usually the developers prefer multi-line comments more because these can be used to represent single lines and multiple lines of comments both.

For using it, you need to use a forward slash with an asterisk (at the opening of the comment), and an asterisk with a forward slash (at the closing of the comment).

Example:

<html>
<body>
<script>
/* This is your multi-line comment.
Browser won't display it */
document.write("Learn Top JavaScript Interview Questions");
</script>
</body>
</html>

VSCUBE TECH®
System For Satisfaction

43. What is null and undefined in JavaScript?

null in JS is an object which is used to represent that a variable doesn't have any value. This is not available by default. The developer needs to do it intentionally.

Whereas, *undefined* is a global object in JS which means that the declared variable has not been assigned any value.

44. What is void in JavaScript?

As you might know, the meaning of void is empty. In the case of JS, the void is actually an operator with its role when a function doesn't return any result. The result it will return will be undefined.

45. Why do we use strict mode in JS?

The purpose of using strict mode in JavaScript is to create silent errors. It helps in building an expression named ‘use strict’ so that we can easily enable the strict mode.

Using it, we can modify the normal JS semantics, such as converting the silent errors to throw errors. As a result, silent errors can be avoided. In addition, the performance of the code can be improved in several cases when we use the strict mode.

46. What is JavaScript hoisting?

It is a technique in JS that helps in using a variable, function, or class before the code execution or declaring these things.

What hoisting in JavaScript does is prioritize the variables, classes, and functions by shifting them to the top level. As a result, you can find their values without code execution. It will not show any errors.

In simple words, the role of JS hoisting is to have the feature to call functions, variables, or classes even if you have not defined them in your code.

47. Which are the best JavaScript frameworks?

The top 10 popular frameworks of JavaScript are:

- ReactJS
- AngularJS
- VueJS
- NodeJS
- BackboneJS
- EmberJS
- Meteor
- Polymer
- Mithril
- Aurelia

WSCUBE TECH®
System For Satisfaction

48. What is an argument object?

An argument object in JavaScript is a set of multiple arguments stored in the form of arrays. This is used at the time of invoking the functions.

It is an in-built JS object that can be accessed only inside a function. Also, this function needs to have the values of the arguments being passed.

49. How can you use JS to find the operating system of the client machine?

We can make use of the navigator.appVersion string to find the client OS version.

50. How can you use JS to modify the background color of an HTML document?

We can use the following code to do so:

```
<script type="text/javascript">
document.body.bgColor="blue";
</script>
```

In place of “blue”, we can use any colour that we intend to use.

51. What is the best way for exception handling in JavaScript?

JS exception handling is performed using try, catch, finally, and throw keywords. Developers can use try or catch blocks to handle exceptions easily.

52. Explain the role of the debugger keyword in JS.

The debugger keyword in JavaScript is used to halt the program’s execution at the point you implement it. It helps you to control the code execution and its flow in a manual manner.

53. What is the syntax of the JavaScript anonymous function?

Here is the syntax for an anonymous function in JavaScript:

```
function() {
   // Function Body
}
```

WsCube Tech® System For Satisfaction

54. How can we create a JavaScript function?

The below syntax can be used for creating a function in JS:

```
function function_name(){
//function body
}
```

55. Is it possible to assign an anonymous function to a variable?

Yes. We can do it.

Learn JavaScript

With WsCube Tech's Online Course

WSCUBE TECH
System For Satisfaction

This comprehensive course Includes:

- 10+ Hours of Video Sessions
- Professional Certification
- Training by Experts
- Top-rated MCQs and Quizzes
- Interview Preparation
- Accessible on mobile and desktop both

~~₹3999~~ *87% OFF*

₹499

View Details Full Curriculum

A Brief About WsCube Tech

(India's Most Trusted IT Training Institute)

WsCube Tech is a leading IT training institute and software development company based in Jodhpur. Since 2011, WsCube Tech has trained 1,50,000+ students and offered internship opportunities to 3,500+ candidates.

In addition, WsCube Tech is one of the fastest-growing tech-based YouTube channels in India, with a community of 1.8+ Million subscribers and 20+ Million monthly views.

With an expert team of trainers in various technological fields, the institute has helped thousands of students across India and other Asian countries to acquire new skills and explore high-paying career opportunities.

Our learners are working at top brands, enterprises, and unicorns across India and globally.

Live Online Certification Courses Offered by WsCube Tech

Rigorous regular training with interactive live classes

WSCUBE TECH®
System For Satisfaction

- ☐ Ethical Hacking Certification
- ☐ Penetration Testing Certification
- ☐ Web Development Certification
- ☐ Content Writing Certification
- ☐ SEO Certification
- ☐ Digital Marketing Certification
- ☐ Data Science Certification
- ☐ Python Certification
- ☐ WordPress Certification
- ☐ Flutter Certification
- ☐ Android App Development Certification

Students We've Trained Work at Renowned Companies, Startups, and Unicorns

WSCUBE TECH®
System For Satisfaction

Let's connect on social media and around the web:

- Website: https://www.wscubetech.com/
- YouTube: https://www.youtube.com/c/wscubetechjodhpur
- Facebook: https://www.facebook.com/wscubetech.india
- LinkedIn: https://www.linkedin.com/company/wscubetech
- Instagram: https://www.instagram.com/wscubetechindia/
- Twitter: https://twitter.com/wscubetechindia

JAVASCRIPT

JavaScript is the scripting language of the Web.

JavaScript is used in millions of Web pages to add functionality, validate forms, detect browsers, and much more.

Introduction to JavaScript

JavaScript is used in millions of Web pages to improve the design, validate forms, detect browsers, create cookies, and much more.

JavaScript is the most popular scripting language on the Internet, and works in all major browsers, such as Internet Explorer, Mozilla Firefox, and Opera.

What is JavaScript?

- JavaScript was designed to add interactivity to HTML pages
- JavaScript is a scripting language
- A scripting language is a lightweight programming language
- JavaScript is usually embedded directly into HTML pages
- JavaScript is an interpreted language (means that scripts execute without preliminary compilation)
- Everyone can use JavaScript without purchasing a license

Java and JavaScript are two completely different languages in both concept and design!

Java (developed by Sun Microsystems) is a powerful and much more complex programming language - in the same category as C and C++.

What can a JavaScript Do ?

- **JavaScript gives HTML designers a programming tool -** HTML authors are normally not programmers, but JavaScript is a scripting language with a very simple syntax! Almost anyone can put small "snippets" of code into their HTML pages
- **JavaScript can put dynamic text into an HTML page -** A JavaScript statement like this: document.write("<h1>" + name + "</h1>") can write a variable text into an HTML page
- **JavaScript can react to events -** A JavaScript can be set to execute when something happens, like when a page has finished loading or when a user clicks on an HTML element
- **JavaScript can read and write HTML elements -** A JavaScript can read and change the content of an HTML element
- **JavaScript can be used to validate data -** A JavaScript can be used to validate form data before it is submitted to a server. This saves the server from extra processing
- **JavaScript can be used to detect the visitor's browser -** A JavaScript can be used to detect the visitor's browser, and - depending on the browser - load another page specifically designed for that browser

- **JavaScript can be used to create cookies** - A JavaScript can be used to store and retrieve information on the visitor's computer.

JavaScript Variables

Variables are "containers" for storing information.

JavaScript variables are used to hold values or expressions.

A variable can have a short name, like x, or a more descriptive name, like carname.

Rules for JavaScript variable names:

- Variable names are case sensitive (y and Y are two different variables)
- Variable names must begin with a letter or the underscore character

Note: Because JavaScript is case-sensitive, variable names are case-sensitive.

Example

A variable's value can change during the execution of a script. You can refer to a variable by its name to display or change its value.

```
<html>
<body>
<script type="text/javascript">
var firstname;
firstname="Welcome";
document.write(firstname);
document.write("<br />");
firstname="XYZ";
document.write(firstname);
</script>

<p>The script above declares a variable,
assigns a value to it, displays the value, change the value,
and displays the value again.</p>

</body>
</html>
```

Output :
Welcome
XYZ

The script above declares a variable, assigns a value to it, displays the value, change the value, and displays the value again.

Declaring (Creating) JavaScript Variables

Creating variables in JavaScript is most often referred to as "declaring" variables.

You can declare JavaScript variables with the **var statement**:

```
var x;
var carname;
```

After the declaration shown above, the variables are empty (they have no values yet).

However, you can also assign values to the variables when you declare them:

```
var x=5;
var carname="Scorpio";
```

After the execution of the statements above, the variable **x** will hold the value **5**, and **carname** will hold the value **Scorpio**.

Note: When you assign a text value to a variable, use quotes around the value.

Assigning Values to Undeclared JavaScript Variables

If you assign values to variables that have not yet been declared, the variables will automatically be declared.

These statements:

```
x=5;
carname="Scorpio";
```

have the same effect as:

```
var x=5;
var carname="Scorpio";
```

Redeclaring JavaScript Variables

If you redeclare a JavaScript variable, it will not lose its original value.

```
var x=5;
var x;
```

After the execution of the statements above, the variable x will still have the value of 5. The value of x is not reset (or cleared) when you redeclare it.

DataTypes

- Numbers - are values that can be processed and calculated. You don't enclose them in quotation marks. The numbers can be either positive or negative.
- Strings - are a series of letters and numbers enclosed in quotation marks. JavaScript uses the string literally; it doesn't process it. You'll use strings for text you want displayed or values you want passed along.
- Boolean (**true**/**false**) - lets you evaluate whether a condition meets or does not meet specified criteria.
- Null - is an empty value. **null** is not the same as 0 -- 0 is a real, calculable number, whereas **null** is the **absence of any value**.

Data Types

TYPE	EXAMPLE
Numbers	Any number, such as 17, 21, or 54e7
Strings	"Greetings!" or "Fun"
Boolean	Either true or false
Null	A special keyword for exactly that – the null value (that is, nothing)

JavaScript Arithmetic

As with algebra, you can do arithmetic operations with JavaScript variables:

```
y=x-5;
z=y+5;
```

JavaScript Operators

The operator = is used to assign values.

The operator + is used to add values.

The assignment operator = is used to assign values to JavaScript variables.

The arithmetic operator + is used to add values together.

```
y=5;
z=2;
x=y+z;
```

The value of x, after the execution of the statements above is 7.

JavaScript Arithmetic Operators

Arithmetic operators are used to perform arithmetic between variables and/or values.

Given that **y=5**, the table below explains the arithmetic operators:

Operator	Description	Example	Result
+	Addition	x=y+2	x=7
-	Subtraction	x=y-2	x=3
*	Multiplication	x=y*2	x=10
/	Division	x=y/2	x=2.5
%	Modulus (division remainder)	x=y%2	x=1
++	Increment	x=++y	x=6
--	Decrement	x=--y	x=4

JavaScript Assignment Operators

Assignment operators are used to assign values to JavaScript variables.

Given that **x=10** and **y=5**, the table below explains the assignment operators:

Operator	Example	Same As	Result
=	x=y		x=5
+=	x+=y	x=x+y	x=15
-=	x-=y	x=x-y	x=5
*=	x*=y	x=x*y	x=50
/=	x/=y	x=x/y	x=2
%=	x%=y	x=x%y	x=0

The + Operator Used on Strings

The + operator can also be used to add string variables or text values together.

To add two or more string variables together, use the + operator.

```
txt1="What a very";
txt2="nice day";
txt3=txt1+txt2;
```

After the execution of the statements above, the variable txt3 contains "What a verynice day".

To add a space between the two strings, insert a space into one of the strings:

```
txt1="What a very ";
txt2="nice day";
txt3=txt1+txt2;
```

or insert a space into the expression:

```
txt1="What a very";
txt2="nice day";
txt3=txt1+" "+txt2;
```

After the execution of the statements above, the variable txt3 contains:

"What a very nice day"

Adding Strings and Numbers

Look at these examples:

```
x=5+5;
document.write(x);

x="5"+"5";
document.write(x);

x=5+"5";
document.write(x);

x="5"+5;
document.write(x);
```

The rule is:

If you add a number and a string, the result will be a string.

JavaScript Comparison and Logical Operators

Comparison and Logical operators are used to test for true or false.

Comparison Operators

Comparison operators are used in logical statements to determine equality or difference between variables or values.

Given that **x=5**, the table below explains the comparison operators:

Operator	Description	Example
==	is equal to	x==8 is false
===	is exactly equal to (value and type)	x===5 is true x==="5" is false
!=	is not equal	x!=8 is true
>	is greater than	x>8 is false
<	is less than	x<8 is true
>=	is greater than or equal to	x>=8 is false
<=	is less than or equal to	x<=8 is true

How Can it be Used

Comparison operators can be used in conditional statements to compare values and take action depending on the result:

```
if (age<18) document.write("Too young");
```

You will learn more about the use of conditional statements in the next chapter of this tutorial.

Logical Operators

Logical operators are used to determine the logic between variables or values.

Given that **x=6 and y=3**, the table below explains the logical operators:

Operator	Description	Example
&&	and	(x < 10 && y > 1) is true
||	or	(x==5 || y==5) is false
!	not	!(x==y) is true

Conditional Operator

JavaScript also contains a conditional operator that assigns a value to a variable based on some condition.

Syntax

```
variablename=(condition)?value1:value2
```

Example

```
greeting=(visitor=="PRES")?"Dear President ":"Dear ";
```

If the variable **visitor** has the value of "PRES", then the variable **greeting** will be assigned the value "Dear President " else it will be assigned "Dear".

Conditional Statements

Very often when you write code, you want to perform different actions for different decisions. You can use conditional statements in your code to do this.

In JavaScript we have the following conditional statements:

- **if statement** - use this statement if you want to execute some code only if a specified condition is true
- **if...else statement** - use this statement if you want to execute some code if the condition is true and another code if the condition is false
- **if...else if....else statement** - use this statement if you want to select one of many blocks of code to be executed
- **switch statement** - use this statement if you want to select one of many blocks of code to be executed

If Statement

You should use the if statement if you want to execute some code only if a specified condition is true.

Syntax

```
if (condition)
{
code to be executed if condition is true
}
```

Note that if is written in lowercase letters. Using uppercase letters (IF) will generate a JavaScript error!

Example 1

```
<script type="text/javascript">
//Write a "Good morning" greeting if
//the time is less than 10
var d=new Date();
var time=d.getHours();
```

```
if (time<10)
{
document.write("<b>Good morning</b>");
}
</script>
```

Example 2

```
<script type="text/javascript">
//Write "Lunch-time!" if the time is 11
var d=new Date();
var time=d.getHours();

if (time==11)
{
document.write("<b>Lunch-time!</b>");
}
</script>
```

Note: When **comparing** variables you must always use two equals signs next to each other (==)!

Notice that there is no ..else.. in this syntax. You just tell the code to execute some code **only if the specified condition is true**.

If...else Statement

If you want to execute some code if a condition is true and another code if the condition is not true, use the if....else statement.

Syntax

```
if (condition)
{
code to be executed if condition is true
}
else
{
code to be executed if condition is not true
}
```

Example

```
<script type="text/javascript">
//If the time is less than 10,
//you will get a "Good morning" greeting.
//Otherwise you will get a "Good day" greeting.
var d = new Date();
```

```
var time = d.getHours();

if (time < 10)
{
document.write("Good morning!");
}
else
{
document.write("Good day!");
}
</script>
```

If...else if...else Statement

You should use the if....else if...else statement if you want to select one of many sets of lines to execute.

Syntax

```
if (condition1)
{
code to be executed if condition1 is true
}
else if (condition2)
{
code to be executed if condition2 is true
}
else
{
code to be executed if condition1 and
condition2 are not true
}
```

Example

```
<script type="text/javascript">
var d = new Date()
var time = d.getHours()
if (time<10)
{
document.write("<b>Good morning</b>");
}
else if (time>10 && time<16)
{
document.write("<b>Good day</b>");
}
else
```

```
{
document.write("<b>Hello World!</b>");
}
</script>
```

The JavaScript Switch Statement

You should use the switch statement if you want to select one of many blocks of code to be executed.

Syntax

```
switch(n)
{
case 1:
  execute code block 1
  break;
case 2:
  execute code block 2
  break;
default:
  code to be executed if n is
  different from case 1 and 2
}
```

This is how it works: First we have a single expression *n* (most often a variable), that is evaluated once. The value of the expression is then compared with the values for each case in the structure. If there is a match, the block of code associated with that case is executed. Use **break** to prevent the code from running into the next case automatically.

Example

```
<script type="text/javascript">
//You will receive a different greeting based
//on what day it is. Note that Sunday=0,
//Monday=1, Tuesday=2, etc.
var d=new Date();
theDay=d.getDay();
switch (theDay)
{
case 5:
  document.write("Finally Friday");
  break;
case 6:
  document.write("Super Saturday");
  break;
case 0:
  document.write("Sleepy Sunday");
```

```
  break;
default:
  document.write("I'm looking forward to this weekend!");
}
</script>
```

JavaScript Controlling(Looping) Statements

Loops in JavaScript are used to execute the same block of code a specified number of times or while a specified condition is true.

JavaScript Loops

Very often when you write code, you want the same block of code to run over and over again in a row. Instead of adding several almost equal lines in a script we can use loops to perform a task like this.

In JavaScript there are two different kind of loops:

- **for** - loops through a block of code a specified number of times
- **while** - loops through a block of code while a specified condition is true

The for Loop

The for loop is used when you know in advance how many times the script should run.

Syntax

```
for (var=startvalue;var<=endvalue;var=var+increment)
{
   code to be executed
}
```

Example

Explanation: The example below defines a loop that starts with i=0. The loop will continue to run as long as **i** is less than, or equal to 10. **i** will increase by 1 each time the loop runs.

Note: The increment parameter could also be negative, and the <= could be any comparing statement.

```
<html>
<body>
<script type="text/javascript">
var i=0;
for (i=0;i<=10;i++)
{
```

```
document.write("The number is " + i);
document.write("<br />");
}
</script>
</body>
</html>
```

Result

```
The number is 0
The number is 1
The number is 2
The number is 3
The number is 4
The number is 5
The number is 6
The number is 7
The number is 8
The number is 9
The number is 10
```

JavaScript While Loop

Loops in JavaScript are used to execute the same block of code a specified number of times or while a specified condition is true.

The while loop

The while loop is used when you want the loop to execute and continue executing while the specified condition is true.

```
while (var<=endvalue)
{
   code to be executed
}
```

Note: The <= could be any comparing statement.

Example

Explanation: The example below defines a loop that starts with i=0. The loop will continue to run as long as **i** is less than, or equal to 10. **i** will increase by 1 each time the loop runs.

```
<html>
```

```
<body>
<script type="text/javascript">
var i=0;
while (i<=10)
{
document.write("The number is " + i);
document.write("<br />");
i=i+1;
}
</script>
</body>
</html>
```

Result

```
The number is 0
The number is 1
The number is 2
The number is 3
The number is 4
The number is 5
The number is 6
The number is 7
The number is 8
The number is 9
The number is 10
```

The do...while Loop

The do...while loop is a variant of the while loop. This loop will always execute a block of code ONCE, and then it will repeat the loop as long as the specified condition is true. This loop will always be executed at least once, even if the condition is false, because the code is executed before the condition is tested.

```
do
{
   code to be executed
}
while (var<=endvalue);
```

Example

```
<html>
<body>
<script type="text/javascript">
```

```
var i=0;
do
{
document.write("The number is " + i);
document.write("<br />");
i=i+1;
}
while (i<0);
</script>
</body>
</html>
```

Result

```
The number is 0
```

JavaScript Break and Continue

There are two special statements that can be used inside loops: break and continue.

JavaScript break and continue Statements

There are two special statements that can be used inside loops: break and continue.

Break

The break command will break the loop and continue executing the code that follows after the loop (if any).

Example

```
<html>
<body>
<script type="text/javascript">
var i=0;
for (i=0;i<=10;i++)
{
if (i==3)
{
break;
}
document.write("The number is " + i);
document.write("<br />");
}
</script>
</body>
```

```
</html>
```

Result

```
The number is 0
The number is 1
The number is 2
```

Continue

The continue command will break the current loop and continue with the next value.

Example

```
<html>
<body>
<script type="text/javascript">
var i=0
for (i=0;i<=10;i++)
{
if (i==3)
{
continue;
}
document.write("The number is " + i);
document.write("<br />");
}
</script>
</body>
</html>
```

Result

```
The number is 0
The number is 1
The number is 2
The number is 4
The number is 5
The number is 6
The number is 7
The number is 8
The number is 9
The number is 10
```

JavaScript Functions

A function (also known as a *method*) is a self-contained piece of code that performs a particular "function". You can recognise a function by its format - it's a piece of descriptive text, followed by open and close brackets.A function is a reusable code-block that will be executed by an event, or when the function is called.

To keep the browser from executing a script when the page loads, you can put your script into a function.

A function contains code that will be executed by an event or by a call to that function.

You may call a function from anywhere within the page (or even from other pages if the function is embedded in an external .js file).

Functions can be defined both in the <head> and in the <body> section of a document. However, to assure that the function is read/loaded by the browser before it is called, it could be wise to put it in the <head> section.

Example

```
<html>
<head>
<script type="text/javascript">
function displaymessage()
{
alert("Hello World!");
}
</script>
</head>
<body>
<form>
<input type="button" value="Click me!"
onclick="displaymessage()" >
</form>
</body>
</html>
```

If the line: alert("Hello world!!") in the example above had not been put within a function, it would have been executed as soon as the line was loaded. Now, the script is not executed before the user hits the button. We have added an onClick event to the button that will execute the function displaymessage() when the button is clicked.

You will learn more about JavaScript events in the JS Events chapter.

How to Define a Function

The syntax for creating a function is:

```
function functionname(var1,var2,...,varX)
{
some code
}
```

var1, var2, etc are variables or values passed into the function. The { and the } defines the start and end of the function.

Note: A function with no parameters must include the parentheses () after the function name:

```
function functionname()
{
some code
}
```

Note: Do not forget about the importance of capitals in JavaScript! The word function must be written in lowercase letters, otherwise a JavaScript error occurs! Also note that you must call a function with the exact same capitals as in the function name.

The return Statement

The return statement is used to specify the value that is returned from the function.

So, functions that are going to return a value must use the return statement.

Example

The function below should return the product of two numbers (a and b):

```
function prod(a,b)
{
x=a*b;
return x;
}
```

When you call the function above, you must pass along two parameters:

```
product=prod(2,3);
```

The returned value from the prod() function is 6, and it will be stored in the variable called product.

The Lifetime of JavaScript Variables

When you declare a variable within a function, the variable can only be accessed within that function. When you exit the function, the variable is destroyed. These variables are called local variables. You can have local variables with the same name in different functions, because each is recognized only by the function in which it is declared.

If you declare a variable outside a function, all the functions on your page can access it. The lifetime of these variables starts when they are declared, and ends when the page is closed.

What is an Event?

Event Handlers

Event Handlers are JavaScript methods, i.e. functions of objects, that allow us as JavaScript programmers to control what happens when events occur.

Directly or indirectly, an **Event** is always the result of something a user does. For example, we've already seen Event Handlers like **onClick** and **onMouseOver** that respond to mouse actions. Another type of Event, an internal change-of-state to the page (**completion** of loading or **leaving** the page). An **onLoad** Event can be considered an indirect result of a user action.

Although we often refer to Events and Event Handlers interchangeably, it's important to keep in mind the distinction between them. An **Event** is merely something that happens - something that it is initiated by an **Event Handler** (**onClick**, **onMouseOver**, etc...).

The elements on a page which can trigger events are known as "targets" or "target elements," and we can easily understand how a button which triggers a *Click* event is a target element for this event. Typically, events are defined through the use of Event Handlers, which are bits of script that tell the browser what to do when a particular event occurs at a particular target. These Event Handlers are commonly written as attributes of the target element's HTML tag.

The Event Handler for a *Click* event at a form field button element is quite simple to understand:

```
<INPUT TYPE="button" NAME="click1" VALUE="Click me for fun!"
 onClick="event_handler_code">
```

The event_handler_code portion of this example is any valid JavaScript and it will be executed when the specified event is triggered at this target element. This particular topic will be continued in Incorporating JavaScripts into your HTML pages.

There are "three different ways" that Event Handlers can be used to trigger Events or Functions.

Method 1 (Link Events):

Places an Event Handler as an attribute within an <A HREF= > tag, like this:

```
<A HREF="foo.html" onMouseOver="doSomething()">   ...   </A>
```

You can use an Event Handler located within an <A HREF= > tag to make either an image or a text link respond to a mouseover Event. Just enclose the image or text string between the <A HREF= > and the </A> tags.

Whenever a user clicks on a link, or moves her cursor over one, JavaScript is sent a **Link Event**. One Link Event is called **onClick**, and it gets sent whenever someone clicks on a link. Another link event is called **onMouseOver**. This one gets sent when someone moves the cursor over the link.

You can use these events to affect what the user sees on a page. Here's an example of how to use link events. Try it out, View Source, and we'll go over it.

```
<A HREF="javascript:void(')"
  onClick="open('index.htm', 'links', 'height=200,width=200');">How to Use Link Events
</A>
```

The first interesting thing is that there are no <SCRIPT> tags. That's because anything that appears in the quotes of an **onClick** or an **onMouseOver** is automatically interpreted as JavaScript. In fact, because semicolons mark the end of statements allowing you to write entire JavaScripts in one line, you can fit an entire JavaScript program between the quotes of an **onClick**. It'd be ugly, but you could do it.

Here are the three lines of interest:

1. <A HREF="#" **onClick**="**alert**('Ooo, do it again!');">Click on me!</A>
2. <A HREF="javascript:**void**(')" **onClick**="**alert**('Ooo, do it again!');">
   Click on me!
   </A>
3. <A HREF="javascript:**alert**('Ooo, do it again!')" >Click on me!</A>

In the first example we have a normal <A> tag, but it has the magic **onClick**="" element, which says, "When someone clicks on this link, run the little bit of JavaScript between my quotes." Notice, there's even a terminating semicolon at the end of the alert. **Question**: is this required? NO.

Let's go over each line:

1. HREF="#" tells the browser to look for the anchor #, but there is no anchor "#", so the browser reloads the page and goes to top of the page since it couldn't find the anchor.
2. <A HREF="javascript:**void**(')" tells the browser not to go anywhere - it "deadens" the link when you click on it. HREF="javascript: is the way to call a function when a link (hyperlink or an HREFed image) is clicked.
3. HREF="javascript:**alert**('Ooo, do it again!')" here we kill two birds with one stone. The default behavior of a hyperlink is to click on it. By clicking on the link we call the window Method **alert**() and also at the same time "deaden" the link.

The next line is

```
<A HREF="javascript:void(")" onMouseOver="alert('Hee hee!');">
Mouse over me!
</A>
```

This is just like the first line, but it uses an **onMouseOver** instead of an **onClick**.

Method 2 (Actions within FORMs):

The second technique we've seen for triggering a Function in response to a mouse action is to place an **onClick** Event Handler inside a button type form element, like this:

```
<FORM>
    <INPUT TYPE="button" onClick="doSomething()">
</FORM>
```

While any JavaScript statement, methods, or functions can appear inside the quotation marks of an Event Handler, typically, the JavaScript script that makes up the Event Handler is actually a call to a function defined in the header of the document or a single JavaScript command. Essentially, though, anything that appears inside a command block (inside curly braces {}) can appear between the quotation marks.

For instance, if you have a form with a text field and want to call the function **checkField**() whenever the value of the text field changes, you can define your text field as follows:

```
<INPUT TYPE="text" onChange="checkField(this)">
```

Nonetheless, the entire code for the function could appear in quotation marks rather than a function call:

```
<INPUT TYPE="text" onChange="if (this.value <= 5) {
    alert("Please enter a number greater than 5");
}">
```

To separate multiple commands in an Event Handler, use semicolons

```
<INPUT TYPE="text" onChange="alert('Thanks for the entry.');
  confirm('Do you want to continue?');">
```

The advantage of using functions as Event Handlers, however, is that you can use the same Event Handler code for multiple items in your document and, functions make your code easier to read and understand.

Method 3 (BODY onLoad & onUnLoad):

The third technique is to us an Event Handler to ensure that all required objects are defined involve the **onLoad** and **onUnLoad**. These Event Handlers are defined in the <BODY> or <FRAMESET> tag of an HTML file and are invoked when the document or frameset are fully loaded or unloaded. If you set a flag

within the **onLoad** Event Handler, other Event Handlers can test this flags to see if they can safely run, with the knowledge that the document is fully loaded and all objects are defined. For example:

```
<SCRIPT>

var loaded = false;

function doit() {
    // alert("Everything is \"loaded\" and loaded = " + loaded);
    alert('Everything is "loaded" and loaded = ' + loaded);
}

</SCRIPT>

<BODY onLoad="loaded = true;">
-- OR --
<BODY onLoad="window.loaded = true;">

<FORM>
    <INPUT TYPE="button" VALUE="Press Me"
      onClick="if (loaded == true) doit();">
-- OR --
    <INPUT TYPE="button" VALUE="Press Me"
      onClick="if (window.loaded == true) doit();">
-- OR --
    <INPUT TYPE="button" VALUE="Press Me"
      onClick="if (loaded) doit();">
</FORM>

</BODY>
```

The **onLoad** Event Handler is executed when the document or frameset is fully loaded, which means that all images have been downloaded and displayed, all subframes have loaded, any Java Applets and Plugins (Navigator) have started running, and so on. The **onUnLoad** Event Handler is executed just before the page is unloaded, which occurs when the browser is about to move on to a new page. Be aware that when you are working with multiple frames, there is no guarantee of the order in which the **onLoad** Event Handler is invoked for the various frames, except that the Event Handlers for the parent frame is invoked after the Event Handlers of all its children frames -- This will be discussed in detail in **Week 8**.

Setting the bgColor Property

The first example allows the user to change the color by clicking buttons, while the second example allows you to change colors by using drop down boxes.

Event Handlers

EVENT	DESCRIPTION
onAbort	the user cancels loading of an image
onBlur	input focus is removed from a form element (when the user clicks outside the field) or focus is removed from a window
onClick	the user clicks on a link or form element
onChange	the value of a form field is changed by the user
onError	an error happens during loading of a document or image
onFocus	input focus is given to a form element or a window
onLoad	once a page is loaded, NOT while loading
onMouseOut	the user moves the pointer off of a link or clickable area of an image map
onMouseOver	the user moves the pointer over a hypertext link
onReset	the user clears a form using the Reset button
onSelect	the user selects a form element's field
onSubmit	a form is submitted (ie, when the users clicks on a submit button)
onUnload	the user leaves a page

Note: *Input focus* refers to the act of clicking on or in a form element or field. This can be done by clicking in a text field or by tabbing between text fields.

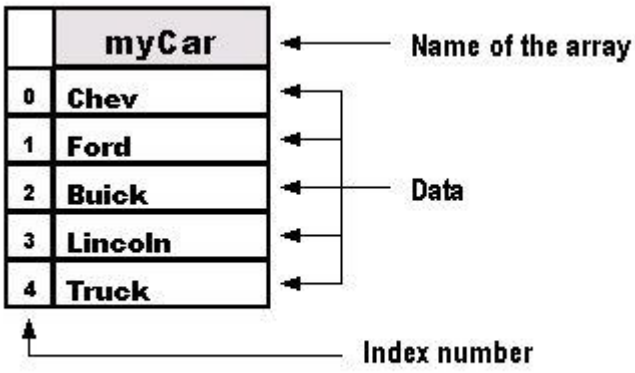
Which Event Handlers Can Be Used

OBJECT	EVENT HANDLERS AVAILABLE
Button element	**onClick, onMouseOver**
Checkbox	**onClick**
Clickable ImageMap area	**onClick, onMouseOver, onMouseOut**
Document	**onLoad, onUnload, onError**
Form	**onSubmit, onReset**
Framesets	**onBlur, onFocus**
Hypertext link	**onClick, onMouseOver, onMouseOut**
Image	**onLoad, onError, onAbort**

Radio button	**onClick**
Reset button	**onClick**
Selection list	**onBlur, onChange, onFocus**
Submit button	**onClick**
TextArea element	**onBlur, onChange, onFocus, onSelect**
Text element	**onBlur, onChange, onFocus, onSelect**
Window	**onLoad, onUnload, onBlur, onFocus**

JavaScript Arrays

An array object is used to create a database-like structure within a script. Grouping data points (*array elements*) together makes it easier to access and use the data in a script. There are methods of accessing actual databases (which are beyond the scope of this series) but here we're talking about small amounts of data.



Comparison of an array to a column of data

An array can be viewed like a column of data in a spreadsheet. The name of the array would be the same as the name of the column. Each piece of data (*element*) in the array is referred to by a number (*index*), just like a row number in a column.

An array is an *object*. Earlier, I said that an object is a thing, a collection of properties (*array elements*, in this case) grouped together.

You can name an array using the same format as a variable, a function or an object. Remember our basic rules: The first character cannot be a number, you cannot use a reserved word, and you cannot use spaces. Also, be sure to remember that the name of the array object is capitalized, e.g. **A**rray.

The JavaScript interpreter uses numbers to access the collection of elements (i.e. the data) in an array. Each index number (as it is the number of the data in the array's index) refers to a specific piece of data in the array, similar to an ID number. It's important to remember that the index numbering of the data starts at "0." So, if you have 8 elements, the first element will be numbered "0" and the last one will be "7."

Elements can be of any type: character string, integer, Boolean, or even another array. An array can even have different types of elements within the same array. Each element in the

array is accessed by placing its index number in brackets, i.e. myCar[4]. This would mean that we are looking for data located in the array myCar which has an index of "4." Since the numbering of an index starts at "0," this would actually be the fifth index. For instance, in the following array,

```
var myCar = new Array("Chev","Ford","Buick","Lincoln","Truck");
alert(myCar[4])
```

the data point with an index of "4" would be Truck. In this example, the indexes are numbered as follows: 0=Chev, 1=Ford, 2=Buick, 3=Lincoln, and 4=Truck. When creating loops, it's much easier to refer to a number than to the actual data itself.

The Size of the Array

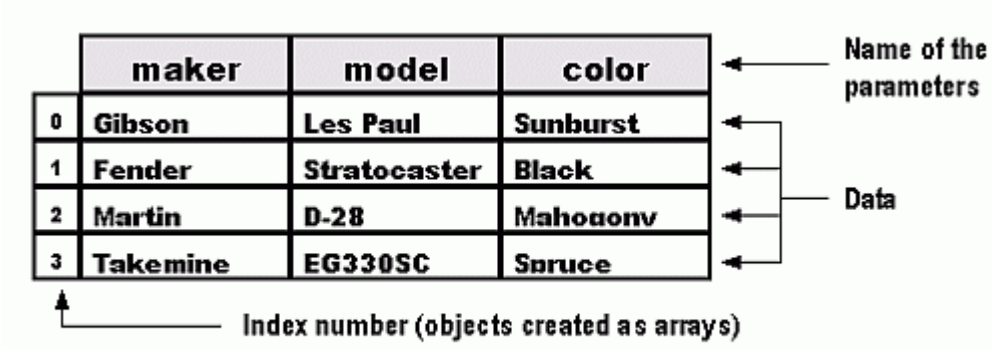
The size of an array is determined by either the actual number of elements it contains or by actually specifying a given size. You don't need to specify the size of the array. Sometimes, though, you may want to pre-set the size, e.g.:

```
var myCar = new Array(20);
```

That would pre-size the array with 20 elements. You might pre-size the array in order to set aside the space in memory.

Multidimensional Arrays

This type of an array is similar to parallel arrays. In a multidimensional array, instead of creating two or more arrays in tandem as we did with the parallel array, we create an array with several levels or "dimensions." Remember our example of a spreadsheet with rows and columns? This time, however, we have a couple more columns.



Comparison of a multidimensional array to a column of data

Multidimensional arrays can be created in different ways. Let's look at one of these method. First, we create the main array, which is similar to what we did with previous arrays.

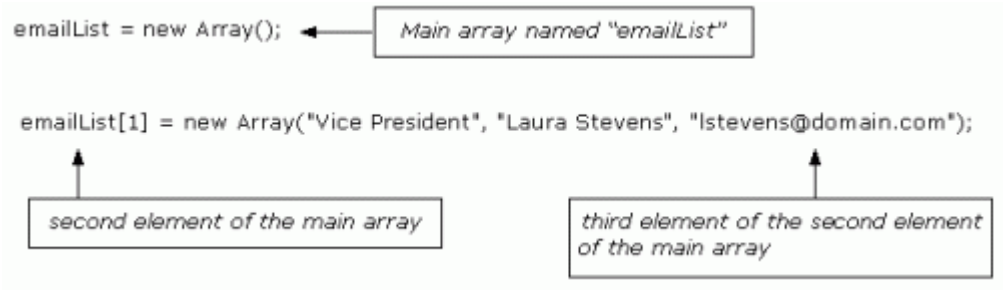
```
var emailList = new Array();
```

Next, we create arrays for elements of the main array:

```
emailList[0] = new Array("President", "Paul Smith", "psmith@domain.com");
emailList[1] = new Array("Vice President", "Laura Stevens", "lstevens@domain.com");
emailList[2] = new Array("General Manager", "Mary Larsen", "mlarsen@domain.com");
emailList[3] = new Array("Sales Manager", "Bob Lark", "blark@domain.com");
```

In this script we created "sub arrays" or arrays from another level or "dimension." We used the name of the main array and gave it an index number (e.g., emailList[0]). Then we created a new instance of an array and gave it a value with three elements.

In order to access a single element, we need to use a double reference. For example, to get the e-mail address for the Vice President in our example above, access the third element "[2]" of the second element "[1]" of the array named emailList.



It would be written like this:

```
var vpEmail = emailList[1][2]
alert("The address is: "+ vpEmail)
```

1. We declared a variable, named it emailList, and initialized it with a value of a new instance of an array.
2. Next, we created an array for each of the elements within the original array. Each of the new arrays contained three elements.
3. Then we declared a variable named vpEmail and initialized it with the value of the third element (lstevens@domain.com) of the second element "[1]" of the array named emailList.

You could also retrieve the information using something like:

```
var title = emailList[1][0]
var email = emailList[1][2]
alert("The e-mail address for the " + title +" is: " + email)
```

Array Properties

length

The length property returns the number of elements in an array. The format is arrayName.length. The length property is particularly useful when using a loop to cycle through an array. One example would be an array used to cycle banners:

```
var bannerImg = new Array();
  bannerImg[0]="image-1.gif";
  bannerImg[1]="image-2.gif";
  bannerImg[2]="image-3.gif";

var newBanner = 0
var totalBan = bannerImg.length

function cycleBan() {
  newBanner++
  if (newBanner == totalBan) {
    newBanner = 0
  }
  document.banner.src=bannerImg[newBanner]
  setTimeout("cycleBan()", 3*1000)
}
window.onload=cycleBan;
```

This portion is then placed in the body where the banner is to be displayed:

```
&lt;img src="image-1.gif" name="banner">
```

Let's take a look and see what happened here:

1. On the first line, we created a new instance of the array bannerImg, and gave it three data elements. (Remember, we are only making a copy of the Array object here.)
2. Next, we created two variables: newBanner, which has a beginning value of zero; and totalBan, which returns the length of the array (the total number of elements contained in the array).
3. Then we created a function named cycleBan. This function will be used to create a loop to cycle the images.
    a. We set the newBanner variable to be increased each time the function cycles. (*Review: By placing the increment operator [" ++ "] after the variable [the "operand"], the variable is incremented only after it returns its current value to the script. For example, its beginning value is "0", so in the first cycle it will return a value of "0" to the script and then its value will be increased by "1".*)
    b. When the value of the newBanner variable is equal to the variable totalBan (which is the length of the array), it is then reset to "0". This allows the images to start the cycle again, from the beginning.
    c. The next statement uses the Document Object Method (DOM - we'll be taking a look at that soon) to display the images on the Web page. Remember, we use the *dot operator* to access the properties of an object. We also read the statement backwards, i.e., "take the element from the array bannerImg, that is specified by the current value of the variable newBanner, and place it in the src attribute located in the element with the name attribute of banner, which is located in the document object."
    d. We then used the setTimeout function to tell the script how long to display each image. This is always measured in milliseconds so, in this case, the function cycleBan is called every 3,000 milliseconds (i.e., every 3 seconds).
4. Finally, we used the window.onload statement to execute the function cycleBan as soon as the document is loaded.

There are a total of five properties for the Array object. In addition to the length property listed above, the others are:

1. *constructor*: Specifies the function that creates an object's prototype.
2. *index*: Only applies to JavaScript arrays created by a regular expression match.
3. *input*: Only applies to JavaScript arrays created by a regular expression match.
4. *prototype*: Used to add properties or methods.

The other properties listed here are either more advanced or seldom used. For now, we'll stick to the basics.

Javascript Object Hierarchy

Hierarchy Objects			
Object	**Properties**	**Methods**	**Event Handlers**
Window	defaultStatus frames opener parent scroll self status top window	alert blur close confirm focus open prompt clearTimeout setTimeout	onLoad onUnload onBlur onFocus
History	length forward go	back	none
Navigator	appCodeName appName appVersion mimeTypes plugins userAgent	javaEnabled	none
document	alinkColor anchors applets area bgColor cookie fgColor forms images lastModified linkColor links location referrer title	clear close open write writeln	none (the onLoad and onUnload event handlers belong to the Window object.

	vlinkColor		
image	border complete height hspace lowsrc name src vspace width	none	none
form	action elements encoding FileUpload method name target	submit reset	onSubmit onReset
text	defaultValue name type value	focus blur select	onBlur onCharge onFocus onSelect

Built-in Objects			
Array	length	join reverse sort xx	none
Date	none	getDate getDay getHours getMinutes getMonth getSeconds getTime getTimeZoneoffset getYear parse prototype setDate setHours setMinutes setMonth setSeconds setTime	none

		setYear toGMTString toLocaleString UTC	
String	length prototype	anchor big blink bold charAt fixed fontColor fontSize indexOf italics lastIndexOf link small split strike sub substring sup toLowerCase toUpperCase	Window

JavaScript Array Object

The Array object is used to store multiple values in a single variable.

Create an Array

The following code creates an Array object called myCars:

```
var myCars=new Array();
```

There are two ways of adding values to an array (you can add as many values as you need to define as many variables you require).

1:

```
var myCars=new Array();
myCars[0]="Saab";
myCars[1]="Volvo";
myCars[2]="BMW";
```

You could also pass an integer argument to control the array's size:

```
var myCars=new Array(3);
myCars[0]="Saab";
myCars[1]="Volvo";
myCars[2]="BMW";
```

2:

```
var myCars=new Array("Saab","Volvo","BMW");
```

Note: If you specify numbers or true/false values inside the array then the type of variables will be numeric or Boolean instead of string.

Access an Array

You can refer to a particular element in an array by referring to the name of the array and the index number. The index number starts at 0.

The following code line:

```
document.write(myCars[0]);
```

will result in the following output:

```
Saab
```

Modify Values in an Array

To modify a value in an existing array, just add a new value to the array with a specified index number:

```
myCars[0]="Opel";
```

Now, the following code line:

```
document.write(myCars[0]);
```

will result in the following output:

```
Opel
```

JavaScript Date Object

Create a Date Object

The Date object is used to work with dates and times.

The following code create a Date object called myDate:

```
var myDate=new Date()
```

Note: The Date object will automatically hold the current date and time as its initial value!

Set Dates

We can easily manipulate the date by using the methods available for the Date object.

In the example below we set a Date object to a specific date (14th January 2010):

```
var myDate=new Date();
myDate.setFullYear(2010,0,14);
```

And in the following example we set a Date object to be 5 days into the future:

```
var myDate=new Date();
myDate.setDate(myDate.getDate()+5);
```

Note: If adding five days to a date shifts the month or year, the changes are handled automatically by the Date object itself!

Compare Two Dates

The Date object is also used to compare two dates.

The following example compares today's date with the 14th January 2010:

```
var myDate=new Date();
myDate.setFullYear(2010,0,14);
var today = new Date();
if (myDate>today)
{
alert("Today is before 14th January 2010");
}
else
{
alert("Today is after 14th January 2010");
}
```

JavaScript Math Object

Math Object

The Math object allows you to perform mathematical tasks.

The Math object includes several mathematical constants and methods.

Syntax for using properties/methods of Math:

```
var pi_value=Math.PI;
var sqrt_value=Math.sqrt(16);
```

Note: Math is not a constructor. All properties and methods of Math can be called by using Math as an object without creating it.

Mathematical Constants

JavaScript provides eight mathematical constants that can be accessed from the Math object. These are: E, PI, square root of 2, square root of 1/2, natural log of 2, natural log of 10, base-2 log of E, and base-10 log of E.

You may reference these constants from your JavaScript like this:

```
Math.E
Math.PI
Math.SQRT2
Math.SQRT1_2
Math.LN2
Math.LN10
Math.LOG2E
Math.LOG10E
```

Mathematical Methods

In addition to the mathematical constants that can be accessed from the Math object there are also several methods available.

The following example uses the round() method of the Math object to round a number to the nearest integer:

```
document.write(Math.round(4.7));
```

The code above will result in the following output:

```
5
```

The following example uses the random() method of the Math object to return a random number between 0 and 1:

```
document.write(Math.random());
```

The code above can result in the following output:

```
0.4218824567728053
```

The following example uses the floor() and random() methods of the Math object to return a random number between 0 and 10:

```
document.write(Math.floor(Math.random()*11));
```

The code above can result in the following output:

```
4
```

JavaScript String Object

String object

The String object is used to manipulate a stored piece of text.

Examples of use:

The following example uses the length property of the String object to find the length of a string:

```
var txt="Hello world!";
document.write(txt.length);
```

The code above will result in the following output:

```
12
```

The following example uses the toUpperCase() method of the String object to convert a string to uppercase letters:

```
var txt="Hello world!";
document.write(txt.toUpperCase());
```

The code above will result in the following output:

```
HELLO WORLD!
```

Window Object

The Window object is the top level object in the JavaScript hierarchy.

The Window object represents a browser window.

A Window object is created automatically with every instance of a <body> or <frameset> tag.

IE: Internet Explorer, **F:** Firefox, **O:** Opera.

Window Object Collections

Collection	Description	IE	F	O
frames[]	Returns all named frames in the window	4	1	9

Window Object Properties

Property	Description	IE	F	O
closed	Returns whether or not a window has been closed	4	1	9
defaultStatus	Sets or returns the default text in the statusbar of the window	4	No	9
document	See Document object	4	1	9
history	See History object	4	1	9
length	Sets or returns the number of frames in the window	4	1	9
location	See Location object	4	1	9
name	Sets or returns the name of the window	4	1	9
opener	Returns a reference to the window that created the window	4	1	9
outerHeight	Sets or returns the outer height of a window	No	1	No
outerWidth	Sets or returns the outer width of a window	No	1	No
pageXOffset	Sets or returns the X position of the current page in relation to the upper left corner of a window's display area	No	No	No
pageYOffset	Sets or returns the Y position of the current page in relation to the upper left corner of a window's display area	No	No	No
parent	Returns the parent window	4	1	9
personalbar	Sets whether or not the browser's personal bar (or directories bar) should be visible			
scrollbars	Sets whether or not the scrollbars should be visible			
self	Returns a reference to the current window	4	1	9
status	Sets the text in the statusbar of a window	4	No	9
statusbar	Sets whether or not the browser's statusbar should be visible			
toolbar	Sets whether or not the browser's tool bar is visible or not (can only be set before the window is opened and you must have UniversalBrowserWrite privilege)			
top	Returns the topmost ancestor window	4	1	9

Window Object Methods

Method	Description	IE	F	O
alert()	Displays an alert box with a message and an OK button	4	1	9
blur()	Removes focus from the current window	4	1	9
clearInterval()	Cancels a timeout set with setInterval()	4	1	9
clearTimeout()	Cancels a timeout set with setTimeout()	4	1	9
close()	Closes the current window	4	1	9
confirm()	Displays a dialog box with a message and an OK and a Cancel button	4	1	9
createPopup()	Creates a pop-up window	4	No	No
focus()	Sets focus to the current window	4	1	9
moveBy()	Moves a window relative to its current position	4	1	9
moveTo()	Moves a window to the specified position	4	1	9
open()	Opens a new browser window	4	1	9
print()	Prints the contents of the current window	5	1	9
prompt()	Displays a dialog box that prompts the user for input	4	1	9
resizeBy()	Resizes a window by the specified pixels	4	1	9
resizeTo()	Resizes a window to the specified width and height	4	1.5	9
scrollBy()	Scrolls the content by the specified number of pixels	4	1	9
scrollTo()	Scrolls the content to the specified coordinates	4	1	9
setInterval()	Evaluates an expression at specified intervals	4	1	9
setTimeout()	Evaluates an expression after a specified number of milliseconds	4	1	9

Document Object

The Document object represents the entire HTML document and can be used to access all elements in a page.

The Document object is part of the Window object and is accessed through the window.document property.

IE: Internet Explorer, **F:** Firefox, **O:** Opera, **W3C:** World Wide Web Consortium (Internet Standard).

Document Object Collections

Collection	Description	IE	F	O	W3C
anchors[]	Returns a reference to all Anchor objects in the document	4	1	9	Yes
forms[]	Returns a reference to all Form objects in the document	4	1	9	Yes
images[]	Returns a reference to all Image objects in the document	4	1	9	Yes
links[]	Returns a reference to all Area and Link objects in	4	1	9	Yes

	the document				

Document Object Properties

Property	Description	IE	F	O	W3C
body	Gives direct access to the <body> element				
cookie	Sets or returns all cookies associated with the current document	4	1	9	Yes
domain	Returns the domain name for the current document	4	1	9	Yes
lastModified	Returns the date and time a document was last modified	4	1	No	No
referrer	Returns the URL of the document that loaded the current document	4	1	9	Yes
title	Returns the title of the current document	4	1	9	Yes
URL	Returns the URL of the current document	4	1	9	Yes

Document Object Methods

Method	Description	IE	F	O	W3C
close()	Closes an output stream opened with the document.open() method, and displays the collected data	4	1	9	Yes
getElementById()	Returns a reference to the first object with the specified id	5	1	9	Yes
getElementsByName()	Returns a collection of objects with the specified name	5	1	9	Yes
getElementsByTagName()	Returns a collection of objects with the specified tagname	5	1	9	Yes
open()	Opens a stream to collect the output from any document.write() or document.writeln() methods	4	1	9	Yes
write()	Writes HTML expressions or JavaScript code to a document	4	1	9	Yes
writeln()	Identical to the write() method, with the addition of writing a new line character after each expression	4	1	9	Yes

History Object

The History object is actually a JavaScript object, not an HTML DOM object.

The History object is automatically created by the JavaScript runtime engine and consists of an array of URLs. These URLs are the URLs the user has visited within a browser window.

The History object is part of the Window object and is accessed through the window.history property.

IE: Internet Explorer, **F:** Firefox, **O:** Opera.

History Object Properties

Property	Description	IE	F	O
length	Returns the number of elements in the history list	4	1	9

History Object Methods

Method	Description	IE	F	O
back()	Loads the previous URL in the history list	4	1	9
forward()	Loads the next URL in the history list	4	1	9
go()	Loads a specific page in the history list	4	1	9

Form Object

The Form object represents an HTML form.

For each instance of a <form> tag in an HTML document, a Form object is created.

IE: Internet Explorer, **F:** Firefox, **O:** Opera, **W3C:** World Wide Web Consortium (Internet Standard).

Form Object Collections

Collection	Description	IE	F	O	W3C
elements[]	Returns an array containing each element in the form	5	1	9	Yes

Form Object Properties

Property	Description	IE	F	O	W3C
acceptCharset	Sets or returns a list of possible character-sets for the form data	No	No	No	Yes
action	Sets or returns the action attribute of a form	5	1	9	Yes
enctype	Sets or returns the MIME type used to encode the content of a form	6	1	9	Yes
id	Sets or returns the id of a form	5	1	9	Yes
length	Returns the number of elements in a form	5	1	9	Yes
method	Sets or returns the HTTP method for sending data to the server	5	1	9	Yes
name	Sets or returns the name of a form	5	1	9	Yes
target	Sets or returns where to open the action-URL in a form	5	1	9	Yes

Standard Properties

Property	Description	IE	F	O	W3C
className	Sets or returns the class attribute of an element	5	1	9	Yes
dir	Sets or returns the direction of text	5	1	9	Yes
lang	Sets or returns the language code for an element	5	1	9	Yes
title	Sets or returns an element's advisory title	5	1	9	Yes

Form Object Methods

Method	Description	IE	F	O	W3C
reset()	Resets the values of all elements in a form	5	1	9	Yes
submit()	Submits a form	5	1	9	Yes

Image Object

The Image object represents an embedded image.

For each instance of an <img> tag in an HTML document, an Image object is created.

IE: Internet Explorer, **F:** Firefox, **O:** Opera, **W3C:** World Wide Web Consortium (Internet Standard).

Image Object Properties

Property	Description	IE	F	O	W3C
align	Sets or returns how to align an image according to the surrounding text	5	1	9	Yes
alt	Sets or returns an alternate text to be displayed, if a browser cannot show an image	5	1	9	Yes
border	Sets or returns the border around an image	4	1	9	Yes
complete	Returns whether or not the browser has finished loading the image	4	1	9	No
height	Sets or returns the height of an image	4	1	9	Yes
hspace	Sets or returns the white space on the left and right side of the image	4	1	9	Yes
id	Sets or returns the id of the image	4	1	9	Yes
isMap	Returns whether or not an image is a server-side image map	5	1	9	Yes
longDesc	Sets or returns a URL to a document containing a description of the image	6	1	9	Yes
lowsrc	Sets or returns a URL to a low-resolution version of an image	4	1	9	No
name	Sets or returns the name of an image	4	1	9	Yes
src	Sets or returns the URL of an image	4	1	9	Yes
useMap	Sets or returns the value of the usemap attribute of an client-side image map	5	1	9	Yes
vspace	Sets or returns the white space on the top and bottom of the image	4	1	9	Yes
width	Sets or returns the width of an image	4	1	9	Yes

Standard Properties

Property	Description	IE	F	O	W3C
className	Sets or returns the class attribute of an element	5	1	9	Yes
title	Sets or returns an element's advisory title	5	1	9	Yes

Area Object

The Area object represents an area of an image-map (An image-map is an image with clickable regions).

For each instance of an <area> tag in an HTML document, an Area object is created.

IE: Internet Explorer, **F:** Firefox, **O:** Opera, **W3C:** World Wide Web Consortium (Internet Standard).

Area Object Properties

Property	Description	IE	F	O	W3C
accessKey	Sets or returns the keyboard key to access an area	5	1	No	Yes
alt	Sets or returns an alternate text to be displayed, if a browser cannot show an area	5	1	9	Yes
coords	Sets or returns the coordinates of a clickable area in an image-map	5	1	9	Yes
hash	Sets or returns the anchor part of the URL in an area	4	1	No	No
host	Sets or returns the hostname and port of the URL in an area	4	1	No	No
href	Sets or returns the URL of a link in an image-map	4	1	9	Yes
id	Sets or returns the id of an area	4	1	9	Yes
noHref	Sets or returns whether an area should be active or inactive	5	1	9	Yes
pathname	Sets or returns the pathname of the URL in an area	4	1	9	No
protocol	Sets or returns the protocol of the URL in an area	4	1	9	No
search	Sets or returns the query string part of the URL in an area	4	1	9	No
shape	Sets or returns the shape of an area in an image-map	5	1	9	Yes
tabIndex	Sets or returns the tab order for an area	5	1	9	Yes
target	Sets or returns where to open the link-URL in an area	4	1	9	Yes

Standard Properties

Property	Description	IE	F	O	W3C
className	Sets or returns the class attribute of an element	5	1	9	Yes
dir	Sets or returns the direction of text	5	1	9	Yes
lang	Sets or returns the language code for an element	5	1	9	Yes
title	Sets or returns an element's advisory title	5	1	9	Yes

Navigator Object

The Navigator object is actually a JavaScript object, not an HTML DOM object.

The Navigator object is automatically created by the JavaScript runtime engine and contains information about the client browser.

IE: Internet Explorer, **F:** Firefox, **O:** Opera.

Navigator Object Collections

Collection	Description	IE	F	O
plugins[]	Returns a reference to all embedded objects in the document	4	1	9

Navigator Object Properties

Property	Description	IE	F	O
appCodeName	Returns the code name of the browser	4	1	9
appMinorVersion	Returns the minor version of the browser	4	No	No
appName	Returns the name of the browser	4	1	9
appVersion	Returns the platform and version of the browser	4	1	9
browserLanguage	Returns the current browser language	4	No	9
cookieEnabled	Returns a Boolean value that specifies whether cookies are enabled in the browser	4	1	9
cpuClass	Returns the CPU class of the browser's system	4	No	No
onLine	Returns a Boolean value that specifies whether the system is in offline mode	4	No	No
platform	Returns the operating system platform	4	1	9
systemLanguage	Returns the default language used by the OS	4	No	No
userAgent	Returns the value of the user-agent header sent by the client to the server	4	1	9
userLanguage	Returns the OS' natural language setting	4	No	9

Navigator Object Methods

Method	Description	IE	F	O
javaEnabled()	Specifies whether or not the browser has Java enabled	4	1	9
taintEnabled()	Specifies whether or not the browser has data tainting enabled	4	1	9

ZIP CODE VALIDATION

<!-- TWO STEPS TO INSTALL ZIP CODE VALIDATION:

1. Copy the coding into the HEAD of your HTML document

2. Add the last code into the BODY of your HTML document -->

<!-- STEP ONE: Paste this code into the HEAD of your HTML document -->

<HEAD>

<SCRIPT LANGUAGE="JavaScript">

<!-- Original: Brian Swalwell -->

```
<!-- This script and many more are available free online at -->

<!-- The JavaScript Source!! http://javascript.internet.com -->

<!-- Begin

function validateZIP(field) {

var valid = "0123456789-";

var hyphencount = 0;

if (field.length!=5 && field.length!=10) {

alert("Please enter your 5 digit or 5 digit+4 zip code.");

return false;

}

for (var i=0; i < field.length; i++) {

temp = "" + field.substring(i, i+1);

if (temp == "-") hyphencount++;

if (valid.indexOf(temp) == "-1") {

alert("Invalid characters in your zip code.  Please try again.");

return false;

}

if ((hyphencount > 1) || ((field.length==10) && ""+field.charAt(5)!="-")) {

alert("The hyphen character should be used with a properly formatted 5 digit+four zip code, like '12345-
6789'.   Please try again.");

return false;

   }

}

return true;
```

```
}

//  End -->

</script>

</HEAD>


<!-- STEP TWO: Copy this code into the BODY of your HTML document  -->


<BODY>

<center>

<form name=zip onSubmit="return validateZIP(this.zip.value)">

Zip: <input type=text size=30 name=zip>

<input type=submit value="Submit">

</form>

</center>

<p><center>

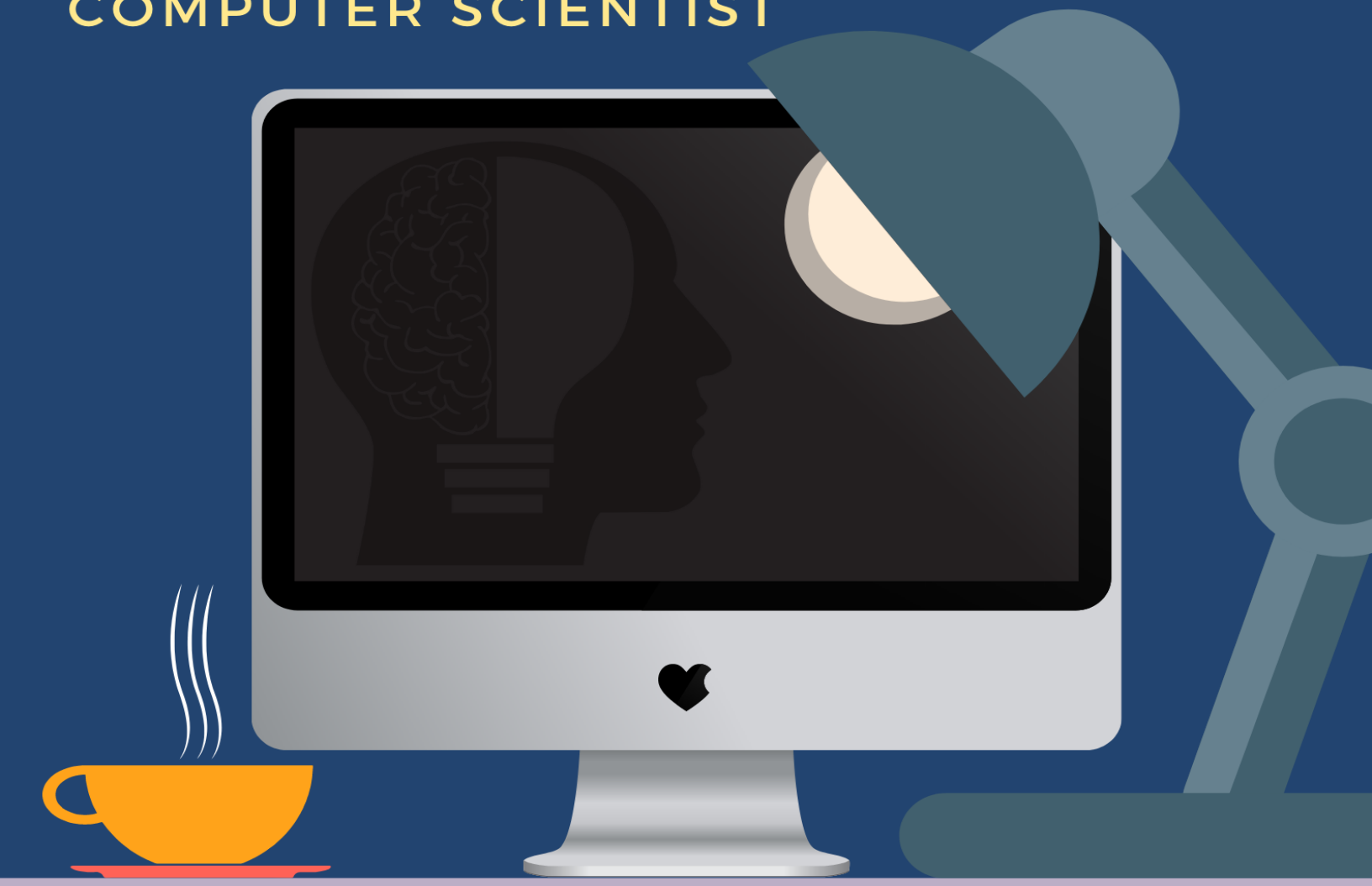
<font face="arial, helvetica" size="-2">Free JavaScripts provided<br>

by <a href="http://javascriptsource.com">The JavaScript Source</a></font>

</center><p>
```

LEARN PYTHON THE RIGHT WAY

HOW TO THINK LIKE A COMPUTER SCIENTIST



Ritza
Marketing as Education

Learn Python the right way

How to think like a computer scientist

Ritza

© 2021 - 2022 Ritza

Contents

Copyright Notice **1**

Foreword **2**

Preface **4**
How and why I came to use Python 4
Finding a textbook 5
Introducing programming with Python 6
Building a community 7

Contributor List **8**
Second Edition 8
First Edition 9

Chapter 1: The way of the program **11**
1.1. The Python programming language 11
1.2. What is a program? 13
1.3. What is debugging? 13
1.4. Syntax errors 14
1.5. Runtime errors 14
1.6. Semantic errors 14
1.7. Experimental debugging 14
1.8. Formal and natural languages 15
1.9. The first program 17
1.10. Comments 18
1.11. Glossary 18
1.12. Exercises 20

Chapter 2: Variables, expressions and statements **23**
2.1. Values and data types 23
2.2. Variables 25
2.3. Variable names and keywords 27
2.4. Statements 28
2.5. Evaluating expressions 28
2.6. Operators and operands 29

2.7. Type converter functions . . . 30
2.8. Order of operations . . . 31
2.9. Operations on strings . . . 32
2.10. Input . . . 33
2.11. Composition . . . 34
2.12. The modulus operator . . . 35
2.13. Glossary . . . 35
2.14. Exercises . . . 38

Chapter 3: Hello, little turtles! . . . **40**
3.1. Our first turtle program . . . 40
3.2. Instances — a herd of turtles . . . 43
3.3. The for loop . . . 45
3.4. Flow of Execution of the for loop . . . 46
3.5. The loop simplifies our turtle program . . . 47
3.6. A few more turtle methods and tricks . . . 49
3.7. Glossary . . . 52
3.8. Exercises . . . 53

Chapter 4: Functions . . . **56**
4.1. Functions . . . 56
4.2. Functions can call other functions . . . 59
4.3. Flow of execution . . . 60
4.4. Functions that require arguments . . . 63
4.5. Functions that return values . . . 63
4.6. Variables and parameters are local . . . 65
4.7. Turtles Revisited . . . 66
4.8. Glossary . . . 67
4.9. Exercises . . . 69

Chapter 5: Conditionals . . . **73**
5.1. Boolean values and expressions . . . 73
5.2. Logical operators . . . 74
5.3. Truth Tables . . . 75
5.4. Simplifying Boolean Expressions . . . 75
5.5. Conditional execution . . . 76
5.6. Omitting the else clause . . . 78
5.7. Chained conditionals . . . 79
5.8. Nested conditionals . . . 81
5.9. The return statement . . . 82
5.10. Logical opposites . . . 82
5.11. Type conversion . . . 84
5.12. A Turtle Bar Chart . . . 85
5.13. Glossary . . . 89

5.14. Exercises 90

Chapter 6: Fruitful functions 93
6.1. Return values 93
6.2. Program development 95
6.3. Debugging with print 98
6.4. Composition 99
6.5. Boolean functions 100
6.6. Programming with style 101
6.7. Unit testing 102
6.8. Glossary 104
6.9. Exercises 105

Chapter 7: Iteration 110
7.1. Assignment 110
7.2. Updating variables 111
7.3. The `for` loop revisited 112
7.4. The `while` statement 112
7.5. The Collatz 3n + 1 sequence 114
7.6. Tracing a program 116
7.7. Counting digits 117
7.8. Abbreviated assignment 118
7.9. Help and meta-notation 119
7.10. Tables 121
7.11. Two-dimensional tables 122
7.12. Encapsulation and generalization 123
7.13. More encapsulation 124
7.14. Local variables 124
7.15. The break statement 125
7.16. Other flavours of loops 126
7.17. An example 128
7.18. The continue statement 129
7.19. More generalization 130
7.20. Functions 131
7.21. Paired Data 132
7.22. Nested Loops for Nested Data 133
7.23. Newton's method for finding square roots 134
7.24. Algorithms 135
7.25. Glossary 136
7.26. Exercises 138

Chapter 8: Strings 143
8.1. A compound data type 143
8.2. Working with strings as single things 143

8.3. Working with the parts of a string . . . 145
8.4. Length . . . 146
8.5. Traversal and the `for` loop . . . 146
8.6. Slices . . . 147
8.7. String comparison . . . 148
8.8. Strings are immutable . . . 149
8.9. The `in` and `not in` operators . . . 149
8.10. A `find` function . . . 150
8.11. Looping and counting . . . 151
8.12. Optional parameters . . . 151
8.13. The built-in `find` method . . . 153
8.14. The `split` method . . . 153
8.15. Cleaning up your strings . . . 154
8.16. The string format method . . . 155
8.17. Summary . . . 158
8.18. Glossary . . . 159
8.19. Exercises . . . 160

Chapter 9: Tuples . . . 165
9.1. Tuples are used for grouping data . . . 165
9.2. Tuple assignment . . . 166
9.3. Tuples as return values . . . 167
9.4. Composability of Data Structures . . . 168
9.5. Glossary . . . 168
9.6. Exercises . . . 169

Chapter 10: Event handling . . . 170
10.1. Event-driven programming . . . 170
10.2. Keypress events . . . 170
10.3. Mouse events . . . 171
10.4. Automatic events from a timer . . . 173
10.5. An example: state machines . . . 174
10.6. Glossary . . . 177
10.7. Exercises . . . 177

Chapter 11: Lists . . . 180
11.1. List values . . . 180
11.2. Accessing elements . . . 180
11.3. List length . . . 181
11.4. List membership . . . 182
11.5. List operations . . . 183
11.6. List slices . . . 183
11.7. Lists are mutable . . . 183
11.8. List deletion . . . 185

11.9. Objects and references . . . 185
11.10. Aliasing . . . 186
11.11. Cloning lists . . . 187
11.12. Lists and `for` loops . . . 188
11.13. List parameters . . . 189
11.14. List methods . . . 190
11.15. Pure functions and modifiers . . . 191
11.16. Functions that produce lists . . . 192
11.17. Strings and lists . . . 193
11.18. `list` and `range` . . . 194
11.19. Nested lists . . . 195
11.20. Matrices . . . 196
11.21. Glossary . . . 196
11.22. Exercises . . . 198

Chapter 12: Modules . . . 202
12.1. Random numbers . . . 202
12.2. The `time` module . . . 205
12.3. The `math` module . . . 206
12.4. Creating your own modules . . . 207
12.5. Namespaces . . . 208
12.6. Scope and lookup rules . . . 210
12.7. Attributes and the dot operator . . . 211
12.8. Three `import` statement variants . . . 212
12.9. Turn your unit tester into a module . . . 213
12.10. Glossary . . . 213
12.11. Exercises . . . 214

Chapter 13: Files . . . 219
13.1. About files . . . 219
13.2. Writing our first file . . . 219
13.3. Reading a file line-at-a-time . . . 220
13.4. Turning a file into a list of lines . . . 221
13.5. Reading the whole file at once . . . 222
13.6. Working with binary files . . . 222
13.7. An example . . . 223
13.8. Directories . . . 224
13.9. What about fetching something from the web? . . . 225
13.10. Glossary . . . 226
13.11. Exercises . . . 227

Chapter 14: List Algorithms . . . 228
14.1. Test-driven development . . . 228
14.2. The linear search algorithm . . . 228

14.3. A more realistic problem . . . 230
14.4. Binary Search . . . 234
14.5. Removing adjacent duplicates from a list . . . 238
14.6. Merging sorted lists . . . 239
14.7. Alice in Wonderland, again! . . . 241
14.8. Eight Queens puzzle, part 1 . . . 243
14.9. Eight Queens puzzle, part 2 . . . 247
14.10. Glossary . . . 249
14.11. Exercises . . . 250

Chapter 15: Classes and Objects — the Basics . . . 254
15.1. Object-oriented programming . . . 254
15.2. User-defined compound data types . . . 254
15.3. Attributes . . . 256
15.4. Improving our initializer . . . 257
15.5. Adding other methods to our class . . . 258
15.6. Instances as arguments and parameters . . . 260
15.7. Converting an instance to a string . . . 260
15.8. Instances as return values . . . 261
15.9. A change of perspective . . . 263
15.10. Objects can have state . . . 263
15.11. Glossary . . . 264
15.12. Exercises . . . 265

Chapter 16: Classes and Objects — Digging a little deeper . . . 267
16.1. Rectangles . . . 267
16.2. Objects are mutable . . . 268
16.3. Sameness . . . 269
16.4. Copying . . . 271
16.5. Glossary . . . 272
16.6. Exercises . . . 273

Chapter 17: PyGame . . . 275
17.1. The game loop . . . 275
17.2. Displaying images and text . . . 279
17.3. Drawing a board for the N queens puzzle . . . 282
17.4. Sprites . . . 288
17.5. Events . . . 292
17.6. A wave of animation . . . 295
17.7. Aliens - a case study . . . 300
17.8. Reflections . . . 301
17.9. Glossary . . . 301
17.10. Exercises . . . 302

Chapter 18: Recursion **303**
18.1. Drawing Fractals 303
18.2. Recursive data structures 306
18.3. Processing recursive number lists 307
18.4. Case study: Fibonacci numbers 309
18.5. Example with recursive directories and files 310
18.6. An animated fractal, using PyGame 311
18.7. Glossary 314
18.8. Exercises 315

Chapter 19: Exceptions **319**
19.1. Catching exceptions 319
19.2. Raising our own exceptions 321
19.3. Revisiting an earlier example 322
19.4. The `finally` clause of the `try` statement 322
19.5. Glossary 323
19.6. Exercises 324

Chapter 20: Dictionaries **325**
20.1. Dictionary operations 326
20.2. Dictionary methods 327
20.3. Aliasing and copying 329
20.4. Sparse matrices 330
20.5. Memoization 331
20.6. Counting letters 332
20.7. Glossary 333
20.8. Exercises 334

Chapter 21: A Case Study: Indexing your files **337**
21.1. The Crawler 337
21.2. Saving the dictionary to disk 340
21.3. The Query Program 340
21.4. Compressing the serialized dictionary 342
21.5. Glossary 343

Chapter 22: Even more OOP **344**
22.1. MyTime 344
22.2. Pure functions 344
22.3. Modifiers 346
22.4. Converting `increment` to a method 347
22.5. An “Aha!” insight 347
22.6. Generalization 349
22.7. Another example 350
22.8. Operator overloading 351

22.9. Polymorphism . . . 353
22.10. Glossary . . . 354
22.11. Exercises . . . 355

Chapter 23: Collections of objects . . . 357
23.1. Composition . . . 357
23.2. `Card` objects . . . 357
23.3. Class attributes and the `__str__` method . . . 358
23.4. Comparing cards . . . 360
23.5. Decks . . . 362
23.6. Printing the deck . . . 362
23.7. Shuffling the deck . . . 364
23.8. Removing and dealing cards . . . 365
23.9. Glossary . . . 366
23.10. Exercises . . . 366

Chapter 24: Inheritance . . . 367
24.1. Inheritance . . . 367
24.2. A hand of cards . . . 367
24.3. Dealing cards . . . 368
24.4. Printing a Hand . . . 369
24.5. The `CardGame` class . . . 370
24.6. `OldMaidHand` class . . . 371
24.7. `OldMaidGame` class . . . 372
24.8. Glossary . . . 376
24.9. Exercises . . . 377

Chapter 25: Linked lists . . . 378
25.1. Embedded references . . . 378
25.2. The `Node` class . . . 378
25.3. Lists as collections . . . 379
25.4. Lists and recursion . . . 380
25.5. Infinite lists . . . 381
25.6. The fundamental ambiguity theorem . . . 382
25.7. Modifying lists . . . 383
25.8. Wrappers and helpers . . . 384
25.9. The `LinkedList` class . . . 384
25.10. Invariants . . . 386
25.11. Glossary . . . 386
25.12. Exercises . . . 387

Chapter 26: Stacks . . . 388
26.1. Abstract data types . . . 388
26.2. The Stack ADT . . . 388

26.3. Implementing stacks with Python lists . . . 389
26.4. Pushing and popping . . . 390
26.5. Using a stack to evaluate postfix . . . 390
26.6. Parsing . . . 391
26.7. Evaluating postfix . . . 391
26.8. Clients and providers . . . 392
26.9. Glossary . . . 393
26.10. Exercises . . . 394

Chapter 27: Queues . . . 395
27.1. The Queue ADT . . . 395
27.2. Linked Queue . . . 395
27.3. Performance characteristics . . . 397
27.4. Improved Linked Queue . . . 397
27.5. Priority queue . . . 398
27.6. The `Golfer` class . . . 400
27.7. Glossary . . . 401
27.8. Exercises . . . 402

Chapter 28: Trees . . . 403
28.1. Building trees . . . 404
28.2. Traversing trees . . . 404
28.3. Expression trees . . . 405
28.4. Tree traversal . . . 406
28.5. Building an expression tree . . . 407
28.6. Handling errors . . . 411
28.7. The animal tree . . . 412
28.8. Glossary . . . 414
28.9. Exercises . . . 415

Appendix A: Debugging . . . 416
A.1. Syntax errors . . . 416
A.2. I can't get my program to run no matter what I do. . . . 417
A.3. Runtime errors . . . 417
A.4. My program does absolutely nothing. . . . 418
A.5. My program hangs. . . . 418
A.6. Infinite Loop . . . 418
A.7. Infinite Recursion . . . 419
A.8. Flow of Execution . . . 419
A.9. When I run the program I get an exception. . . . 420
A.10. I added so many `print` statements I get inundated with output. . . . 421
A.11. Semantic errors . . . 421
A.12. My program doesn't work. . . . 422
A.13. I've got a big hairy expression and it doesn't do what I expect. . . . 423

A.14. I've got a function or method that doesn't return what I expect. . . . 423
A.15. I'm really, really stuck and I need help. . . . 424
A.16. No, I really need help. . . . 424

Appendix B: An odds-and-ends Workbook . . . 426
B.1. The Five Strands of Proficiency . . . 426
B.2. Sending Email . . . 427
B.3. Write your own Web Server . . . 428
B.4. Using a Database . . . 430

Appendix C: Configuring Ubuntu for Python Development . . . 433
C.1. Vim . . . 433
C.2. $HOME environment . . . 434
C.3. Making a Python script executable and runnable from anywhere . . . 435

Appendix D: Customizing and Contributing to the Book . . . 436
D.1. Getting the Source . . . 436
D.2. Making the HTML Version . . . 437

Appendix E: Some Tips, Tricks, and Common Errors . . . 438
E.1. Functions . . . 438
E.2. Problems with logic and flow of control . . . 439
E.3. Local variables . . . 441
E.4. Event handler functions . . . 442
E.5. String handling . . . 442
E.6. Looping and lists . . . 444

Copyright Notice

Copyright (C) Peter Wentworth, Jeffrey Elkner, Allen B. Downey and Chris Meyers. Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with Invariant Sections being Foreword, Preface, and Contributor List, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled "GNU Free Documentation License".

Foreword

By David Beazley

As an educator, researcher, and book author, I am delighted to see the completion of this book. Python is a fun and extremely easy-to-use programming language that has steadily gained in popularity over the last few years. Developed over ten years ago by Guido van Rossum, Python's simple syntax and overall feel is largely derived from ABC, a teaching language that was developed in the 1980's. However, Python was also created to solve real problems and it borrows a wide variety of features from programming languages such as C++, Java, Modula-3, and Scheme. Because of this, one of Python's most remarkable features is its broad appeal to professional software developers, scientists, researchers, artists, and educators.

Despite Python's appeal to many different communities, you may still wonder why Python? or why teach programming with Python? Answering these questions is no simple task—especially when popular opinion is on the side of more masochistic alternatives such as C++ and Java. However, I think the most direct answer is that programming in Python is simply a lot of fun and more productive.

When I teach computer science courses, I want to cover important concepts in addition to making the material interesting and engaging to students. Unfortunately, there is a tendency for introductory programming courses to focus far too much attention on mathematical abstraction and for students to become frustrated with annoying problems related to low-level details of syntax, compilation, and the enforcement of seemingly arcane rules. Although such abstraction and formalism is important to professional software engineers and students who plan to continue their study of computer science, taking such an approach in an introductory course mostly succeeds in making computer science boring. When I teach a course, I don't want to have a room of uninspired students. I would much rather see them trying to solve interesting problems by exploring different ideas, taking unconventional approaches, breaking the rules, and learning from their mistakes. In doing so, I don't want to waste half of the semester trying to sort out obscure syntax problems, unintelligible compiler error messages, or the several hundred ways that a program might generate a general protection fault.

One of the reasons why I like Python is that it provides a really nice balance between the practical and the conceptual. Since Python is interpreted, beginners can pick up the language and start doing neat things almost immediately without getting lost in the problems of compilation and linking. Furthermore, Python comes with a large library of modules that can be used to do all sorts of tasks ranging from web-programming to graphics. Having such a practical focus is a great way to engage students and it allows them to complete significant projects. However, Python can also serve as an excellent foundation for introducing important computer science concepts. Since Python fully supports procedures and classes, students can be gradually introduced to topics such as procedural abstraction, data structures, and object-oriented programming — all of which are applicable to later

courses on Java or C++. Python even borrows a number of features from functional programming languages and can be used to introduce concepts that would be covered in more detail in courses on Scheme and Lisp.

In reading Jeffrey's preface, I am struck by his comments that Python allowed him to see a higher level of success and a lower level of frustration and that he was able to move faster with better results. Although these comments refer to his introductory course, I sometimes use Python for these exact same reasons in advanced graduate level computer science courses at the University of Chicago. In these courses, I am constantly faced with the daunting task of covering a lot of difficult course material in a blistering nine week quarter. Although it is certainly possible for me to inflict a lot of pain and suffering by using a language like C++, I have often found this approach to be counterproductive—especially when the course is about a topic unrelated to just programming. I find that using Python allows me to better focus on the actual topic at hand while allowing students to complete substantial class projects.

Although Python is still a young and evolving language, I believe that it has a bright future in education. This book is an important step in that direction. David Beazley University of Chicago Author of the Python Essential Reference

Preface

By Jeffrey Elkner

This book owes its existence to the collaboration made possible by the Internet and the free software movement. Its three authors—a college professor, a high school teacher, and a professional programmer—never met face to face to work on it, but we have been able to collaborate closely, aided by many other folks who have taken the time and energy to send us their feedback.

We think this book is a testament to the benefits and future possibilities of this kind of collaboration, the framework for which has been put in place by Richard Stallman and the Free Software Foundation.

How and why I came to use Python

In 1999, the College Board's Advanced Placement (AP) Computer Science exam was given in C++ for the first time. As in many high schools throughout the country, the decision to change languages had a direct impact on the computer science curriculum at Yorktown High School in Arlington, Virginia, where I teach. Up to this point, Pascal was the language of instruction in both our first-year and AP courses. In keeping with past practice of giving students two years of exposure to the same language, we made the decision to switch to C++ in the first year course for the 1997-98 school year so that we would be in step with the College Board's change for the AP course the following year.

Two years later, I was convinced that C++ was a poor choice to use for introducing students to computer science. While it is certainly a very powerful programming language, it is also an extremely difficult language to learn and teach. I found myself constantly fighting with C++'s difficult syntax and multiple ways of doing things, and I was losing many students unnecessarily as a result. Convinced there had to be a better language choice for our first-year class, I went looking for an alternative to C++.

I needed a language that would run on the machines in our GNU/Linux lab as well as on the Windows and Macintosh platforms most students have at home. I wanted it to be free software, so that students could use it at home regardless of their income. I wanted a language that was used by professional programmers, and one that had an active developer community around it. It had to support both procedural and object-oriented programming. And most importantly, it had to be easy to learn and teach. When I investigated the choices with these goals in mind, Python stood out as the best candidate for the job.

I asked one of Yorktown's talented students, Matt Ahrens, to give Python a try. In two months he not only learned the language but wrote an application called pyTicket that enabled our staff to report technology problems via the Web. I knew that Matt could not have finished an application of that

scale in so short a time in C++, and this accomplishment, combined with Matt's positive assessment of Python, suggested that Python was the solution I was looking for.

Finding a textbook

Having decided to use Python in both of my introductory computer science classes the following year, the most pressing problem was the lack of an available textbook.

Free documents came to the rescue. Earlier in the year, Richard Stallman had introduced me to Allen Downey. Both of us had written to Richard expressing an interest in developing free educational materials. Allen had already written a first-year computer science textbook, How to Think Like a Computer Scientist. When I read this book, I knew immediately that I wanted to use it in my class. It was the clearest and most helpful computer science text I had seen. It emphasized the processes of thought involved in programming rather than the features of a particular language. Reading it immediately made me a better teacher.

How to Think Like a Computer Scientist was not just an excellent book, but it had been released under the GNU public license, which meant it could be used freely and modified to meet the needs of its user. Once I decided to use Python, it occurred to me that I could translate Allen's original Java version of the book into the new language. While I would not have been able to write a textbook on my own, having Allen's book to work from made it possible for me to do so, at the same time demonstrating that the cooperative development model used so well in software could also work for educational materials.

Working on this book for the last two years has been rewarding for both my students and me, and my students played a big part in the process. Since I could make instant changes whenever someone found a spelling error or difficult passage, I encouraged them to look for mistakes in the book by giving them a bonus point each time they made a suggestion that resulted in a change in the text. This had the double benefit of encouraging them to read the text more carefully and of getting the text thoroughly reviewed by its most important critics, students using it to learn computer science.

For the second half of the book on object-oriented programming, I knew that someone with more real programming experience than I had would be needed to do it right. The book sat in an unfinished state for the better part of a year until the open source community once again provided the needed means for its completion.

I received an email from Chris Meyers expressing interest in the book. Chris is a professional programmer who started teaching a programming course last year using Python at Lane Community College in Eugene, Oregon. The prospect of teaching the course had led Chris to the book, and he started helping out with it immediately. By the end of the school year he had created a companion project on our Website at http://openbookproject.net called *Python for Fun* and was working with some of my most advanced students as a master teacher, guiding them beyond where I could take them.

Introducing programming with Python

The process of translating and using How to Think Like a Computer Scientist for the past two years has confirmed Python's suitability for teaching beginning students. Python greatly simplifies programming examples and makes important programming ideas easier to teach.

The first example from the text illustrates this point. It is the traditional `hello, world` program, which in the Java version of the book looks like this:

```
class Hello {

  public static void main (String[] args) {
      System.out.println ("Hello, world.");
  }
}
```

in the Python version it becomes:

```
print("Hello, World!")
```

Even though this is a trivial example, the advantages of Python stand out. Yorktown's Computer Science I course has no prerequisites, so many of the students seeing this example are looking at their first program. Some of them are undoubtedly a little nervous, having heard that computer programming is difficult to learn. The Java version has always forced me to choose between two unsatisfying options: either to explain the class Hello, public static void main, String[] args„ statements and risk confusing or intimidating some of the students right at the start, or to tell them, Just don't worry about all of that stuff now; we will talk about it later, and risk the same thing. The educational objectives at this point in the course are to introduce students to the idea of a programming statement and to get them to write their first program, thereby introducing them to the programming environment. The Python program has exactly what is needed to do these things, and nothing more.

Comparing the explanatory text of the program in each version of the book further illustrates what this means to the beginning student. There are seven paragraphs of explanation of Hello, world! in the Java version; in the Python version, there are only a few sentences. More importantly, the missing six paragraphs do not deal with the big ideas in computer programming but with the minutia of Java syntax. I found this same thing happening throughout the book. Whole paragraphs simply disappear from the Python version of the text because Python's much clearer syntax renders them unnecessary.

Using a very high-level language like Python allows a teacher to postpone talking about low-level details of the machine until students have the background that they need to better make sense of the details. It thus creates the ability to put first things first pedagogically. One of the best examples of this is the way in which Python handles variables. In Java a variable is a name for a place that

holds a value if it is a built-in type, and a reference to an object if it is not. Explaining this distinction requires a discussion of how the computer stores data. Thus, the idea of a variable is bound up with the hardware of the machine. The powerful and fundamental concept of a variable is already difficult enough for beginning students (in both computer science and algebra). Bytes and addresses do not help the matter. In Python a variable is a name that refers to a thing. This is a far more intuitive concept for beginning students and is much closer to the meaning of variable that they learned in their math courses. I had much less difficulty teaching variables this year than I did in the past, and I spent less time helping students with problems using them.

Another example of how Python aids in the teaching and learning of programming is in its syntax for functions. My students have always had a great deal of difficulty understanding functions. The main problem centers around the difference between a function definition and a function call, and the related distinction between a parameter and an argument. Python comes to the rescue with syntax that is nothing short of beautiful. Function definitions begin with the keyword `def`, so I simply tell my students, When you define a function, begin with `def`, followed by the name of the function that you are defining; when you call a function, simply call (type) out its name. Parameters go with definitions; arguments go with calls. There are no return types, parameter types, or reference and value parameters to get in the way, so I am now able to teach functions in less than half the time that it previously took me, with better comprehension.

Using Python improved the effectiveness of our computer science program for all students. I saw a higher general level of success and a lower level of frustration than I experienced teaching with either C++ or Java. I moved faster with better results. More students left the course with the ability to create meaningful programs and with the positive attitude toward the experience of programming that this engenders.

Building a community

I have received emails from all over the globe from people using this book to learn or to teach programming. A user community has begun to emerge, and many people have been contributing to the project by sending in materials for the companion Website at http://openbookproject.net/pybiblio.

With the continued growth of Python, I expect the growth in the user community to continue and accelerate. The emergence of this user community and the possibility it suggests for similar collaboration among educators have been the most exciting parts of working on this project for me. By working together, we can increase the quality of materials available for our use and save valuable time. I invite you to join our community and look forward to hearing from you. Please write to me at jeff@elkner.net.

Jeffrey Elkner
Governor's Career and Technical Academy in Arlington
Arlington, Virginia

Contributor List

To paraphrase the philosophy of the Free Software Foundation, this book is free like free speech, but not necessarily free like free pizza. It came about because of a collaboration that would not have been possible without the GNU Free Documentation License. So we would like to thank the Free Software Foundation for developing this license and, of course, making it available to us.

We would also like to thank the more than 100 sharp-eyed and thoughtful readers who have sent us suggestions and corrections over the past few years. In the spirit of free software, we decided to express our gratitude in the form of a contributor list. Unfortunately, this list is not complete, but we are doing our best to keep it up to date. It was also getting too large to include everyone who sends in a typo or two. You have our gratitude, and you have the personal satisfaction of making a book you found useful better for you and everyone else who uses it. New additions to the list for the 2nd edition will be those who have made on-going contributions.

If you have a chance to look through the list, you should realize that each person here has spared you and all subsequent readers from the confusion of a technical error or a less-than-transparent explanation, just by sending us a note.

Impossible as it may seem after so many corrections, there may still be errors in this book. If you should stumble across one, we hope you will take a minute to contact us. The email address (for the Python 3 version of the book) is p.wentworth@ru.ac.za . Substantial changes made due to your suggestions will add you to the next version of the contributor list (unless you ask to be omitted). Thank you!

Second Edition

- An email from Mike MacHenry set me straight on tail recursion. He not only pointed out an error in the presentation, but suggested how to correct it.
- It wasn't until 5th Grade student Owen Davies came to me in a Saturday morning Python enrichment class and said he wanted to write the card game, Gin Rummy, in Python that I finally knew what I wanted to use as the case study for the object oriented programming chapters.
- A special thanks to pioneering students in Jeff's Python Programming class at GCTAA during the 2009-2010 school year: Safath Ahmed, Howard Batiste, Louis Elkner-Alfaro, and Rachel Hancock. Your continual and thoughtfull feedback led to changes in most of the chapters of the book. You set the standard for the active and engaged learners that will help make the new Governor's Academy what it is to become. Thanks to you this is truly a student tested text.
- Thanks in a similar vein to the students in Jeff's Computer Science class at the HB-Woodlawn program during the 2007-2008 school year: James Crowley, Joshua Eddy, Eric Larson, Brian McGrail, and Iliana Vazuka.

- Ammar Nabulsi sent in numerous corrections from Chapters 1 and 2.
- Aldric Giacomoni pointed out an error in our definition of the Fibonacci sequence in Chapter 5.
- Roger Sperberg sent in several spelling corrections and pointed out a twisted piece of logic in Chapter 3.
- Adele Goldberg sat down with Jeff at PyCon 2007 and gave him a list of suggestions and corrections from throughout the book.
- Ben Bruno sent in corrections for chapters 4, 5, 6, and 7.
- Carl LaCombe pointed out that we incorrectly used the term commutative in chapter 6 where symmetric was the correct term.
- Alessandro Montanile sent in corrections for errors in the code examples and text in chapters 3, 12, 15, 17, 18, 19, and 20.
- Emanuele Rusconi found errors in chapters 4, 8, and 15.
- Michael Vogt reported an indentation error in an example in chapter 6, and sent in a suggestion for improving the clarity of the shell vs. script section in chapter 1.

First Edition

- Lloyd Hugh Allen sent in a correction to Section 8.4.
- Yvon Boulianne sent in a correction of a semantic error in Chapter 5.
- Fred Bremmer submitted a correction in Section 2.1.
- Jonah Cohen wrote the Perl scripts to convert the LaTeX source for this book into beautiful HTML.
- Michael Conlon sent in a grammar correction in Chapter 2 and an improvement in style in Chapter 1, and he initiated discussion on the technical aspects of interpreters.
- Benoit Girard sent in a correction to a humorous mistake in Section 5.6.
- Courtney Gleason and Katherine Smith wrote horsebet.py, which was used as a case study in an earlier version of the book. Their program can now be found on the website.
- Lee Harr submitted more corrections than we have room to list here, and indeed he should be listed as one of the principal editors of the text.
- James Kaylin is a student using the text. He has submitted numerous corrections.
- David Kershaw fixed the broken catTwice function in Section 3.10.
- Eddie Lam has sent in numerous corrections to Chapters 1, 2, and 3. He also fixed the Makefile so that it creates an index the first time it is run and helped us set up a versioning scheme.
- Man-Yong Lee sent in a correction to the example code in Section 2.4.
- David Mayo pointed out that the word unconsciously in Chapter 1 needed to be changed to subconsciously .
- Chris McAloon sent in several corrections to Sections 3.9 and 3.10.
- Matthew J. Moelter has been a long-time contributor who sent in numerous corrections and suggestions to the book.
- Simon Dicon Montford reported a missing function definition and several typos in Chapter 3. He also found errors in the increment function in Chapter 13.

- John Ouzts corrected the definition of return value in Chapter 3.
- Kevin Parks sent in valuable comments and suggestions as to how to improve the distribution of the book.
- David Pool sent in a typo in the glossary of Chapter 1, as well as kind words of encouragement.
- Michael Schmitt sent in a correction to the chapter on files and exceptions.
- Robin Shaw pointed out an error in Section 13.1, where the printTime function was used in an example without being defined.
- Paul Sleigh found an error in Chapter 7 and a bug in Jonah Cohen's Perl script that generates HTML from LaTeX.
- Craig T. Snydal is testing the text in a course at Drew University. He has contributed several valuable suggestions and corrections.
- Ian Thomas and his students are using the text in a programming course. They are the first ones to test the chapters in the latter half of the book, and they have make numerous corrections and suggestions.
- Keith Verheyden sent in a correction in Chapter 3.
- Peter Winstanley let us know about a longstanding error in our Latin in Chapter 3.
- Chris Wrobel made corrections to the code in the chapter on file I/O and exceptions.
- Moshe Zadka has made invaluable contributions to this project. In addition to writing the first draft of the chapter on Dictionaries, he provided continual guidance in the early stages of the book.
- Christoph Zwerschke sent several corrections and pedagogic suggestions, and explained the difference between gleich and selbe.
- James Mayer sent us a whole slew of spelling and typographical errors, including two in the contributor list.
- Hayden McAfee caught a potentially confusing inconsistency between two examples.
- Angel Arnal is part of an international team of translators working on the Spanish version of the text. He has also found several errors in the English version.
- Tauhidul Hoque and Lex Berezhny created the illustrations in Chapter 1 and improved many of the other illustrations.
- Dr. Michele Alzetta caught an error in Chapter 8 and sent some interesting pedagogic comments and suggestions about Fibonacci and Old Maid.
- Andy Mitchell caught a typo in Chapter 1 and a broken example in Chapter 2.
- Kalin Harvey suggested a clarification in Chapter 7 and caught some typos.
- Christopher P. Smith caught several typos and is helping us prepare to update the book for Python 2.2.
- David Hutchins caught a typo in the Foreword.
- Gregor Lingl is teaching Python at a high school in Vienna, Austria. He is working on a German translation of the book, and he caught a couple of bad errors in Chapter 5.
- Julie Peters caught a typo in the Preface.

Chapter 1: The way of the program

(Watch a video based on this chapter here on YouTube[1].)

The goal of this book is to teach you to think like a computer scientist. This way of thinking combines some of the best features of mathematics, engineering, and natural science. Like mathematicians, computer scientists use formal languages to denote ideas (specifically computations). Like engineers, they design things, assembling components into systems and evaluating tradeoffs among alternatives. Like scientists, they observe the behavior of complex systems, form hypotheses, and test predictions.

The single most important skill for a computer scientist is **problem solving**. Problem solving means the ability to formulate problems, think creatively about solutions, and express a solution clearly and accurately. As it turns out, the process of learning to program is an excellent opportunity to practice problem-solving skills. That's why this chapter is called, The way of the program.

On one level, you will be learning to program, a useful skill by itself. On another level, you will use programming as a means to an end. As we go along, that end will become clearer.

1.1. The Python programming language

The programming language you will be learning is Python. Python is an example of a high-level language; other **high-level languages** you might have heard of are C++, PHP, Pascal, C#, and Java.

As you might infer from the name high-level language, there are also **low-level languages**, sometimes referred to as machine languages or assembly languages. Loosely speaking, computers can only execute programs written in low-level languages. Thus, programs written in a high-level language have to be translated into something more suitable before they can run.

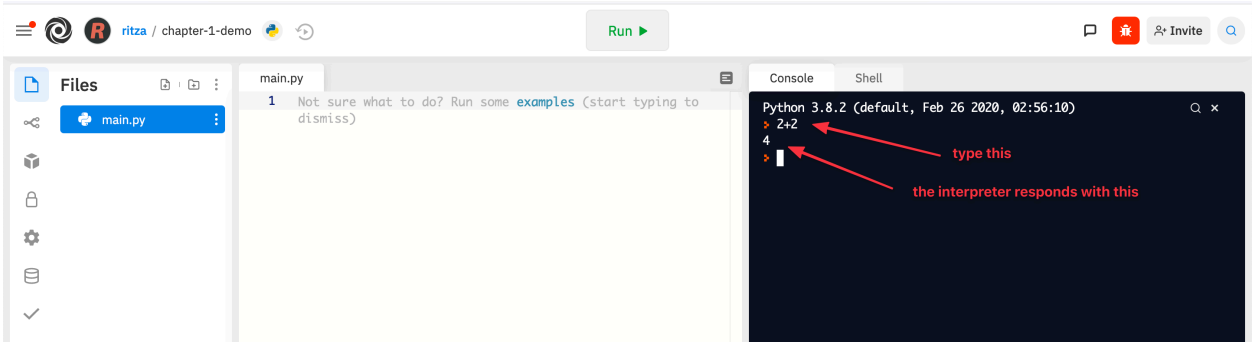
Almost all programs are written in high-level languages because of their advantages. It is much easier to program in a high-level language so programs take less time to write, they are shorter and easier to read, and they are more likely to be correct. Second, high-level languages are **portable**, meaning that they can run on different kinds of computers with few or no modifications.

In this edition of the textbook, we use an online programming environment called **Replit**. To follow along with the examples and complete the exercises, all you need is a free account - just navigate to https://replit.com and complete the sign up process.

Once you have an account, create a new repl and choose Python as the language from the dropdown. You'll see it automatically creates a file called `main.py`. By convention, files that contain Python programs have names that end with `.py`.

[1] https://youtu.be/lhtUREG6vAg

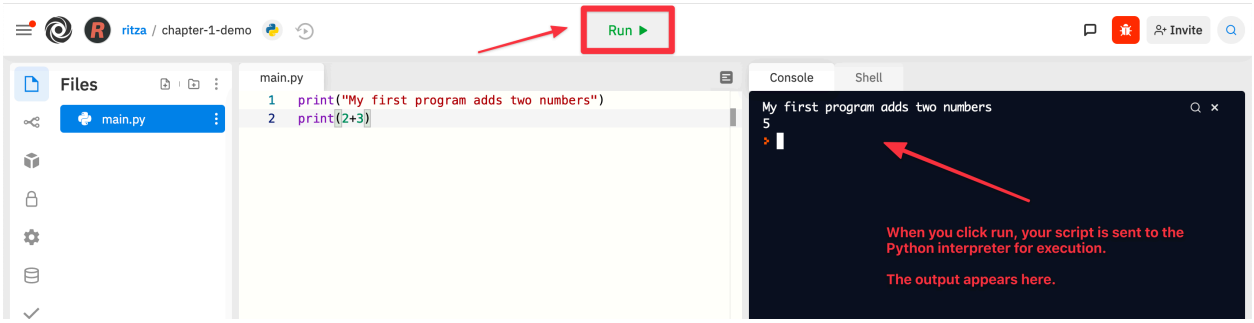
The engine that translates and runs Python is called the **Python Interpreter**: There are two ways to use it: *immediate mode* and *script mode*. In immediate mode, you type Python expressions into the Python Interpreter window, and the interpreter immediately shows the result:



Running code in the interpreter (immediate mode)

The ››› or › is called the **Python prompt**. The interpreter uses the prompt to indicate that it is ready for instructions. We typed `2 + 2`, and the interpreter evaluated our expression, and replied `4`, and on the next line it gave a new prompt, indicating that it is ready for more input.

Working directly in the interpreter is convenient for testing short bits of code because you get immediate feedback. Think of it as scratch paper used to help you work out problems. Anything longer than a few lines should be put into a script. Scripts have the advantage that they can be saved to disk, printed, and so on. To create a script, you can enter the code into the middle pane, as shown below



Running code from a file (script mode)

```
1 print("My first program adds two numbers")
2 print(2+3)
```

To execute the program, click the **Run** button in Replit. You're now a computer programmer! Let's take a look at some more theory before we start writing more advanced programs.

1.2. What is a program?

A **program** is a sequence of instructions that specifies how to perform a computation. The computation might be something mathematical, such as solving a system of equations or finding the roots of a polynomial, but it can also be a symbolic computation, such as searching and replacing text in a document or (strangely enough) compiling a program.

The details look different in different languages, but a few basic instructions appear in just about every language:

input

- Get data from the keyboard, a file, or some other device.

output

- Display data on the screen or send data to a file or other device.

math

- Perform basic mathematical operations like addition and multiplication.

conditional execution

- Check for certain conditions and execute the appropriate sequence of statements.

repetition

- Perform some action repeatedly, usually with some variation.

Believe it or not, that's pretty much all there is to it. Every program you've ever used, no matter how complicated, is made up of instructions that look more or less like these. Thus, we can describe programming as the process of breaking a large, complex task into smaller and smaller subtasks until the subtasks are simple enough to be performed with sequences of these basic instructions.

That may be a little vague, but we will come back to this topic later when we talk about **algorithms**.

1.3. What is debugging?

Programming is a complex process, and because it is done by human beings, it often leads to errors. Programming errors are called **bugs** and the process of tracking them down and correcting them is called **debugging**. Use of the term bug to describe small engineering difficulties dates back to at least 1889, when Thomas Edison had a bug with his phonograph.

Three kinds of errors can occur in a program: syntax errors[2], runtime errors[3], and semantic errors[4].

[2] https://en.wikipedia.org/wiki/Syntax_error
[3] https://en.wikipedia.org/wiki/Runtime_(program_lifecycle_phase)
[4] https://en.wikipedia.org/wiki/Logic_error

It is useful to distinguish between them in order to track them down more quickly.

1.4. Syntax errors

Python can only execute a program if the program is syntactically correct; otherwise, the process fails and returns an error message. **Syntax** refers to the structure of a program and the rules about that structure. For example, in English, a sentence must begin with a capital letter and end with a period. this sentence contains a **syntax error**. So does this one

For most readers, a few syntax errors are not a significant problem, which is why we can read the poetry of E. E. Cummings without problems. Python is not so forgiving. If there is a single syntax error anywhere in your program, Python will display an error message and quit, and you will not be able to run your program. During the first few weeks of your programming career, you will probably spend a lot of time tracking down syntax errors. As you gain experience, though, you will make fewer errors and find them faster.

1.5. Runtime errors

The second type of error is a runtime error, so called because the error does not appear until you run the program. These errors are also called **exceptions** because they usually indicate that something exceptional (and bad) has happened.

Runtime errors are rare in the simple programs you will see in the first few chapters, so it might be a while before you encounter one.

1.6. Semantic errors

The third type of error is the **semantic error**. If there is a semantic error in your program, it will run successfully, in the sense that the computer will not generate any error messages, but it will not do the right thing. It will do something else. Specifically, it will do what you told it to do.

The problem is that the program you wrote is not the program you wanted to write. The meaning of the program (its semantics) is wrong. Identifying semantic errors can be tricky because it requires you to work backward by looking at the output of the program and trying to figure out what it is doing.

1.7. Experimental debugging

One of the most important skills you will acquire is debugging. Although it can be frustrating, debugging is one of the most intellectually rich, challenging, and interesting parts of programming.

In some ways, debugging is like detective work. You are confronted with clues, and you have to infer the processes and events that led to the results you see.

Debugging is also like an experimental science. Once you have an idea what is going wrong, you modify your program and try again. If your hypothesis was correct, then you can predict the result of the modification, and you take a step closer to a working program. If your hypothesis was wrong, you have to come up with a new one. As Sherlock Holmes pointed out, When you have eliminated the impossible, whatever remains, however improbable, must be the truth. (A. Conan Doyle, *The Sign of Four*)

For some people, programming and debugging are the same thing. That is, programming is the process of gradually debugging a program until it does what you want. The idea is that you should start with a program that does something and make small modifications, debugging them as you go, so that you always have a working program.

For example, Linux is an operating system kernel that contains millions of lines of code, but it started out as a simple program Linus Torvalds used to explore the Intel 80386 chip. According to Larry Greenfield, one of Linus's earlier projects was a program that would switch between displaying AAAA and BBBB. This later evolved to Linux (*The Linux Users' Guide* Beta Version 1).

Later chapters will make more suggestions about debugging and other programming practices.

1.8. Formal and natural languages

Natural languages are the languages that people speak, such as English, Spanish, and French. They were not designed by people (although people try to impose some order on them); they evolved naturally.

Formal languages are languages that are designed by people for specific applications. For example, the notation that mathematicians use is a formal language that is particularly good at denoting relationships among numbers and symbols. Chemists use a formal language to represent the chemical structure of molecules. And most importantly:

Programming languages are formal languages that have been designed to express computations.

Formal languages tend to have strict rules about syntax. For example, `3+3=6` is a syntactically correct mathematical statement, but `3=+6$` is not. `H2O` is a syntactically correct chemical name, but `2Zz` is not.

Syntax rules come in two flavors, pertaining to **tokens** and structure. Tokens are the basic elements of the language, such as words, numbers, parentheses, commas, and so on. In Python, a statement like `print("Happy New Year for ",2013)` has 6 tokens: a function name, an open parenthesis (round bracket), a string, a comma, a number, and a close parenthesis.

It is possible to make errors in the way one constructs tokens. One of the problems with `3=+6$` is that `$` is not a legal token in mathematics (at least as far as we know). Similarly, `2Zz` is not a legal token in chemistry notation because there is no element with the abbreviation `Zz`.

The second type of syntax rule pertains to the **structure** of a statement— that is, the way the tokens are arranged. The statement `3=+6$` is structurally illegal because you can't place a plus sign immediately after an equal sign. Similarly, molecular formulas have to have subscripts after the element name, not before. And in our Python example, if we omitted the comma, or if we changed the two parentheses around to say `print)"Happy New Year for ",2013(` our statement would still have six legal and valid tokens, but the structure is illegal.

When you read a sentence in English or a statement in a formal language, you have to figure out what the structure of the sentence is (although in a natural language you do this subconsciously). This process is called **parsing**.

For example, when you hear the sentence, "The other shoe fell", you understand that the other shoe is the subject and fell is the verb. Once you have parsed a sentence, you can figure out what it means, or the **semantics** of the sentence. Assuming that you know what a shoe is and what it means to fall, you will understand the general implication of this sentence.

Although formal and natural languages have many features in common — tokens, structure, syntax, and semantics — there are many differences:

ambiguity

- Natural languages are full of ambiguity, which people deal with by using contextual clues and other information. Formal languages are designed to be nearly or completely unambiguous, which means that any statement has exactly one meaning, regardless of context.

redundancy

- In order to make up for ambiguity and reduce misunderstandings, natural languages employ lots of redundancy. As a result, they are often verbose. Formal languages are less redundant and more concise.

literalness

- Formal languages mean exactly what they say. On the other hand, natural languages are full of idiom and metaphor. If someone says, "The other shoe fell", there is probably no shoe and nothing falling. You'll need to find the original joke to understand the idiomatic meaning of the other shoe falling. Yahoo! Answers thinks it knows!

People who grow up speaking a natural language—everyone—often have a hard time adjusting to formal languages. In some ways, the difference between formal and natural language is like the difference between poetry and prose, but more so:

poetry

- Words are used for their sounds as well as for their meaning, and the whole poem together creates an effect or emotional response. Ambiguity is not only common but often deliberate.

prose

- The literal meaning of words is more important, and the structure contributes more meaning. Prose is more amenable to analysis than poetry but still often ambiguous.

program

- The meaning of a computer program is unambiguous and literal, and can be understood entirely by analysis of the tokens and structure.

Here are some suggestions for reading programs (and other formal languages). First, remember that formal languages are much more dense than natural languages, so it takes longer to read them. Also, the structure is very important, so it is usually not a good idea to read from top to bottom, left to right. Instead, learn to parse the program in your head, identifying the tokens and interpreting the structure. Finally, the details matter. Little things like spelling errors and bad punctuation, which you can get away with in natural languages, can make a big difference in a formal language.

1.9. The first program

Traditionally, the first program written in a new language is called Hello, World! because all it does is display the words, Hello, World! In Python, the script looks like this: (For scripts, we'll show line numbers to the left of the Python statements.)

```
1 print("Hello, World!")
```

This is an example of using the **print function**, which doesn't actually print anything on paper. It displays a value on the screen. In this case, the result shown is

```
1 Hello, World!
```

The quotation marks in the program mark the beginning and end of the value; they don't appear in the result.

Some people judge the quality of a programming language by the simplicity of the Hello, World! program. By this standard, Python does about as well as possible.

1.10. Comments

As programs get bigger and more complicated, they get more difficult to read. Formal languages are dense, and it is often difficult to look at a piece of code and figure out what it is doing, or why.

For this reason, it is a good idea to add notes to your programs to explain in natural language what the program is doing.

A **comment** in a computer program is text that is intended only for the human reader — it is completely ignored by the interpreter.

In Python, the `#` token starts a comment. The rest of the line is ignored. Here is a new version of Hello, World!.

```
#-----------------------------------------------
# This demo program shows off how elegant Python is!
# Written by Joe Soap, December 2010.
# Anyone may freely copy or modify this program.
#-----------------------------------------------

print("Hello, World!")     # Isn't this easy!
```

You'll also notice that we've left a blank line in the program. Blank lines are also ignored by the interpreter, but comments and blank lines can make your programs much easier for humans to parse. Use them liberally!

1.11. Glossary

algorithm

A set of specific steps for solving a category of problems.

bug

An error in a program.

comment

Information in a program that is meant for other programmers (or anyone reading the source code) and has no effect on the execution of the program.

debugging

The process of finding and removing any of the three kinds of programming errors.

exception

Another name for a runtime error.

formal language

Any one of the languages that people have designed for specific purposes, such as representing mathematical ideas or computer programs; all programming languages are formal languages.

high-level language

A programming language like Python that is designed to be easy for humans to read and write.

immediate mode

A style of using Python where we type expressions at the command prompt, and the results are shown immediately. Contrast with script, and see the entry under Python shell.

interpreter

The engine that executes your Python scripts or expressions.

low-level language

A programming language that is designed to be easy for a computer to execute; also called machine language or assembly language.

natural language

Any one of the languages that people speak that evolved naturally.

object code

The output of the compiler after it translates the program.

parse

To examine a program and analyze the syntactic structure.

portability

A property of a program that can run on more than one kind of computer.

print function

A function used in a program or script that causes the Python interpreter to display a value on its output device.

problem solving

The process of formulating a problem, finding a solution, and expressing the solution.

program

a sequence of instructions that specifies to a computer actions and computations to be performed.

Python shell

An interactive user interface to the Python interpreter. The user of a Python shell types commands at the prompt (>>>), and presses the return key to send these commands immediately to the interpreter

for processing. The word shell comes from Unix. In the PyScripter used in this RLE version of the book, the Interpreter Window is where we'd do the immediate mode interaction.

runtime error

An error that does not occur until the program has started to execute but that prevents the program from continuing.

script

A program stored in a file (usually one that will be interpreted).

semantic error

An error in a program that makes it do something other than what the programmer intended.

semantics

The meaning of a program.

source code

A program in a high-level language before being compiled.

syntax

The structure of a program.

syntax error

An error in a program that makes it impossible to parse — and therefore impossible to interpret.

token

One of the basic elements of the syntactic structure of a program, analogous to a word in a natural language.

1.12. Exercises

1. Write an English sentence with understandable semantics but incorrect syntax. Write another English sentence which has correct syntax but has semantic errors.
2. Using the Python interpreter, type `1 + 2` and then hit return. Python evaluates this expression, displays the result, and then shows another prompt. `*` is the multiplication operator, and `**` is the exponentiation operator. Experiment by entering different expressions and recording what is displayed by the Python interpreter.
3. Type `1 2` and then hit return. Python tries to evaluate the expression, but it can't because the expression is not syntactically legal. Instead, it shows the error message:

```
  File "<interactive input>", line 1
    1 2
      ^
SyntaxError: invalid syntax
```

In many cases, Python indicates where the syntax error occurred, but it is not always right, and it doesn't give you much information about what is wrong.

So, for the most part, the burden is on you to learn the syntax rules.

In this case, Python is complaining because there is no operator between the numbers.

See if you can find a few more examples of things that will produce error messages when you enter them at the Python prompt. Write down what you enter at the prompt and the last line of the error message that Python reports back to you.

4. Type `print("hello")`. Python executes this, which has the effect of printing the letters h-e-l-l-o. Notice that the quotation marks that you used to enclose the string are not part of the output. Now type `"hello"` and describe your result. Make notes of when you see the quotation marks and when you don't.
5. Type cheese without the quotation marks. The output will look something like this:

```
Traceback (most recent call last):
  File "<interactive input>", line 1, in ?
NameError: name 'cheese' is not defined
```

This is a run-time error; specifically, it is a NameError, and even more specifically, it is an error because the name cheese is not defined. If you don't know what that means yet, you will soon.

6. Type `6 + 4 * 9` at the Python prompt and hit enter. Record what happens.

Now create a Python script with the following contents:

```
6 + 4 * 9
```

What happens when you run this script? Now change the script contents to:

```
print(6 + 4 * 9)
```

and run it again.

What happened this time?

Whenever an expression is typed at the Python prompt, it is evaluated and the result is automatically shown on the line below. (Like on your calculator, if you type this expression you'll get the result 42.)

A script is different, however. Evaluations of expressions are not automatically displayed, so it is necessary to use the **print** function to make the answer show up.

It is hardly ever necessary to use the print function in immediate mode at the command prompt.

Chapter 2: Variables, expressions and statements

(Watch a video based on this chapter here on YouTube[5].)

2.1. Values and data types

A **value** is one of the fundamental things — like a letter or a number — that a program manipulates. The values we have seen so far are `4` (the result when we added `2 + 2`), and `"Hello, World!"`.

These values are classified into different **classes**, or **data types**: `4` is an *integer*, and `"Hello, World!"` is a *string*, so-called because it contains a string of letters. You (and the interpreter) can identify strings because they are enclosed in quotation marks.

If you are not sure what class a value falls into, Python has a function called **type** which can tell you.

```
>>> type("Hello, World!")
<class 'str'>
>>> type(17)
<class 'int'>
```

Not surprisingly, strings belong to the class **str** and integers belong to the class **int**. Less obviously, numbers with a decimal point belong to a class called **float**, because these numbers are represented in a format called *floating-point*. At this stage, you can treat the words *class* and *type* interchangeably. We'll come back to a deeper understanding of what a class is in later chapters.

```
>>> type(3.2)
<class 'float'>
```

What about values like `"17"` and `"3.2"`? They look like numbers, but they are in quotation marks like strings.

[5] https://youtu.be/gIvstR16coI

```
>>> type("17")
<class 'str'>
>>> type("3.2")
<class 'str'>
```

They're strings!

Strings in Python can be enclosed in either single quotes (') or double quotes ("), or three of each (''' or """)

```
>>> type('This is a string.')
<class 'str'>
>>> type("And so is this.")
<class 'str'>
>>> type("""and this.""")
<class 'str'>
>>> type('''and even this...''')
<class 'str'>
```

Double quoted strings can contain single quotes inside them, as in `"Bruce's beard"`, and single quoted strings can have double quotes inside them, as in `'The knights who say "Ni!"'`.

Strings enclosed with three occurrences of either quote symbol are called triple quoted strings. They can contain either single or double quotes:

```
>>> print('''"Oh no", she exclaimed, "Ben's bike is broken!"''')
"Oh no", she exclaimed, "Ben's bike is broken!"
>>>
```

Triple quoted strings can even span multiple lines:

```
>>> message = """This message will
... span several
... lines."""
>>> print(message)
This message will
span several
lines.
>>>
```

Python doesn't care whether you use single or double quotes or the three-of-a-kind quotes to surround your strings: once it has parsed the text of your program or command, the way it stores the value is identical in all cases, and the surrounding quotes are not part of the value. But when the interpreter wants to display a string, it has to decide which quotes to use to make it look like a string.

```
>>> 'This is a string.'
'This is a string.'
>>> """And so is this."""
'And so is this.'
```

So the Python language designers usually chose to surround their strings by single quotes. What do you think would happen if the string already contained single quotes?

When you type a large integer, you might be tempted to use commas between groups of three digits, as in `42,000`. This is not a legal integer in Python, but it does mean something else, which is legal:

```
>>> 42000
42000
>>> 42,000
(42, 0)
```

Well, that's not what we expected at all! Because of the comma, Python chose to treat this as a pair of values. We'll come back to learn about pairs later. But, for the moment, remember not to put commas or spaces in your integers, no matter how big they are. Also revisit what we said in the previous chapter: formal languages are strict, the notation is concise, and even the smallest change might mean something quite different from what you intended.

2.2. Variables

One of the most powerful features of a programming language is the ability to manipulate **variables**. A variable is a name that refers to a value.

The **assignment statement** gives a value to a variable:

```
>>> message = "What's up, Doc?"
>>> n = 17
>>> pi = 3.14159
```

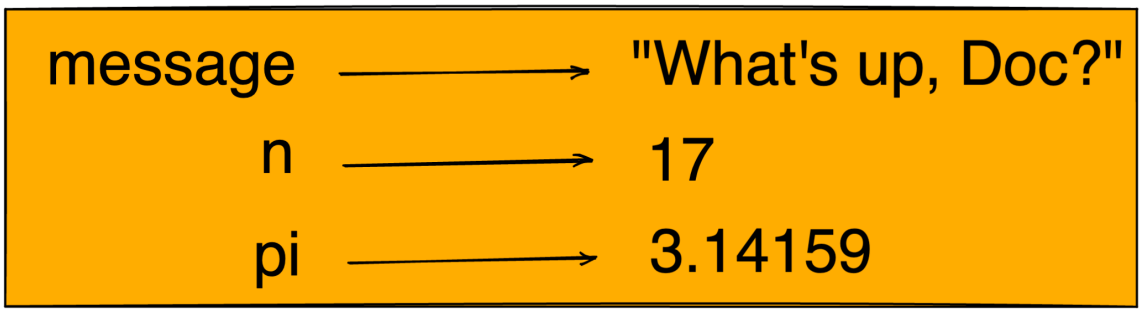
This example makes three assignments. The first assigns the string value `"What's up, Doc?"` to a variable named message. The second gives the integer `17` to `n`, and the third assigns the floating-point number `3.14159` to a variable called `pi`.

The assignment token, `=`, should not be confused with *equals*, which uses the token `==`. The assignment statement binds a *name*, on the left-hand side of the operator, to a *value*, on the right-hand side. This is why you will get an error if you enter:

```
>>> 17 = n
File "<interactive input>", line 1
SyntaxError: can't assign to literal
```

Tip:
When reading or writing code, say to yourself "n is assigned 17" or "n gets the value 17". Don't say "n equals 17".

A common way to represent variables on paper is to write the name with an arrow pointing to the variable's value. This kind of figure is called a **state snapshot** because it shows what state each of the variables is in at a particular instant in time. (Think of it as the variable's state of mind). This diagram shows the result of executing the assignment statements:



State Snapshot

If you ask the interpreter to evaluate a variable, it will produce the value that is currently linked to the variable:

```
>>> message
"What's up, Doc?"
>>> n
17
>>> pi
3.14159
```

We use variables in a program to "remember" things, perhaps the current score at the football game. But variables are *variable*. This means they can change over time, just like the scoreboard at a football game. You can assign a value to a variable, and later assign a different value to the same variable. (*This is different from maths. In maths, if you give* `x` *the value* `3`*, it cannot change to link to a different value half-way through your calculations!*)

```
>>> day = "Thursday"
>>> day
'Thursday'
>>> day = "Friday"
>>> day
'Friday'
>>> day = 21
>>> day
21
```

You'll notice we changed the value of `day` three times, and on the third assignment we even made it refer to a value that was of a different type.

A great deal of programming is about having the computer remember things, e.g. *The number of missed calls on your phone*, and then arranging to update or change the variable when you miss another call.

2.3. Variable names and keywords

Variable names can be arbitrarily long. They can contain both letters and digits, but they have to begin with a letter or an underscore. Although it is legal to use uppercase letters, by convention we don't. If you do, remember that case matters. `Bruce` and `bruce` are different variables.

The underscore character (_) can appear in a name. It is often used in names with multiple words, such as `my_name` or `price_of_tea_in_china`.

There are some situations in which names beginning with an underscore have special meaning, so a safe rule for beginners is to start all names with a letter.

If you give a variable an illegal name, you get a syntax error:

```
>>> 76trombones = "big parade"
SyntaxError: invalid syntax
>>> more$ = 1000000
SyntaxError: invalid syntax
>>> class = "Computer Science 101"
SyntaxError: invalid syntax
```

`76trombones` is illegal because it does not begin with a letter. `more$` is illegal because it contains an illegal character, the dollar sign. But what's wrong with `class`?

It turns out that `class` is one of the Python **keywords**. Keywords define the language's syntax rules and structure, and they cannot be used as variable names.

Python has thirty-something keywords (and every now and again improvements to Python introduce or eliminate one or two):

and	as	assert	break	class	continue
def	del	elif	else	except	exec
finally	for	from	global	if	import
in	is	lambda	nonlocal	not	or
pass	raise	return	try	while	with
yield	True	False	None		

You might want to keep this list handy. If the interpreter complains about one of your variable names and you don't know why, see if it is on this list.

Programmers generally choose names for their variables that are meaningful to the human readers of the program — they help the programmer document, or remember, what the variable is used for.

Caution

Beginners sometimes confuse "meaningful to the human readers" with "meaningful to the computer". So they'll wrongly think that because they've called some variable `average` *or* `pi`*, it will somehow magically calculate an average, or magically know that the variable* `pi` *should have a value like* `3.14159`*. No! The computer doesn't understand what you intend the variable to mean.*

So you'll find some instructors who deliberately don't choose meaningful names when they teach beginners — not because we don't think it is a good habit, but because we're trying to reinforce the message that you — the programmer — must write the program code to calculate the average, and you must write an assignment statement to give the variable `pi` *the value you want it to have.*

2.4. Statements

A **statement** is an instruction that the Python interpreter can execute. We have only seen the assignment statement so far. Some other kinds of statements that we'll see shortly are `while` statements, `for` statements, `if` statements, and `import` statements. (There are other kinds too!)

When you type a statement on the command line, Python executes it. Statements don't produce any result.

2.5. Evaluating expressions

An **expression** is a combination of values, variables, operators, and calls to functions. If you type an expression at the Python prompt, the interpreter **evaluates** it and displays the result:

```
>>> 1 + 1
2
>>> len("hello")
5
```

In this example `len` is a built-in Python function that returns the number of characters in a `string`. We've previously seen the `print` and the `type` functions, so this is our third example of a function!

The *evaluation* of an *expression* produces a value, which is why expressions can appear on the right hand side of assignment statements. A value all by itself is a simple expression, and so is a variable.

```
>>> 17
17
>>> y = 3.14
>>> x = len("hello")
>>> x
5
>>> y
3.14
```

2.6. Operators and operands

Operators are special tokens that represent computations like addition, multiplication and division. The values the operator uses are called **operands**.

The following are all legal Python expressions whose meaning is more or less clear:

```
20+32   hour-1   hour*60+minute   minute/60   5**2   (5+9)*(15-7)
```

The tokens `+`, `-`, and `*`, and the use of parenthesis for grouping, mean in Python what they mean in mathematics. The asterisk (`*`) is the token for multiplication, and `**` is the token for exponentiation.

```
>>> 2 ** 3
8
>>> 3 ** 2
9
```

When a variable name appears in the place of an operand, it is replaced with its value before the operation is performed.

Addition, subtraction, multiplication, and exponentiation all do what you expect.

Example: so let us convert 645 minutes into hours:

```
>>> minutes = 645
>>> hours = minutes / 60
>>> hours
10.75
```

Oops! In Python 3, the division operator `/` always yields a floating point result. What we might have wanted to know was how many whole hours there are, and how many minutes remain. Python gives us two different flavors of the division operator. The second, called **floor division** uses the token `//`. Its result is always a whole number — and if it has to adjust the number it always moves it to the left on the number line. So `6 // 4` yields `1`, but `-6 // 4` might surprise you!

```
>>> 7 / 4
1.75
>>> 7 // 4
1
>>> minutes = 645
>>> hours = minutes // 60
>>> hours
10
```

Take care that you choose the correct flavor of the division operator. If you're working with expressions where you need floating point values, use the division operator that does the division accurately.

2.7. Type converter functions

Here we'll look at three more Python functions, `int`, `float` and `str`, which will (attempt to) convert their arguments into types `int`, `float` and `str` respectively. We call these **type converter** functions.

The int function can take a floating point number or a string, and turn it into an int. For floating point numbers, it discards the decimal portion of the number — a process we call truncation towards zero on the number line. Let us see this in action:

```
>>> int(3.14)
3
>>> int(3.9999)            # This doesn't round to the closest int!
3
>>> int(3.0)
3
>>> int(-3.999)            # Note that the result is closer to zero
-3
>>> int(minutes / 60)
10
>>> int("2345")            # Parse a string to produce an int
2345
>>> int(17)                # It even works if arg is already an int
17
>>> int("23 bottles")
```

This last case doesn't look like a number — what do we expect?

```
Traceback (most recent call last):
File "<interactive input>", line 1, in <module>
ValueError: invalid literal for int() with base 10: '23 bottles'
```

The type converter `float` can turn an integer, a float, or a syntactically legal string into a float:

```
>>> float(17)
17.0
>>> float("123.45")
123.45
```

The type converter `str` turns its argument into a string:

```
>>> str(17)
'17'
>>> str(123.45)
'123.45'
```

2.8. Order of operations

When more than one operator appears in an expression, the order of evaluation depends on the **rules of precedence**. Python follows the same precedence rules for its mathematical operators that mathematics does. The acronym PEMDAS is a useful way to remember the order of operations:

1. **P**arentheses have the highest precedence and can be used to force an expression to evaluate in the order you want. Since expressions in parentheses are evaluated first, `2 * (3-1)` is `4`, and `(1+1)**(5-2)` is `8`. You can also use parentheses to make an expression easier to read, as in `(minute * 100) / 60`, even though it doesn't change the result.
2. **E**xponentiation has the next highest precedence, so `2**1+1` is `3` and not `4`, and `3*1**3` is `3` and not `27`.
3. **M**ultiplication and both **D**ivision operators have the same precedence, which is higher than **A**ddition and **S**ubtraction, which also have the same precedence. So `2*3-1` yields `5` rather than `4`, and `5-2*2` is `1`, not `6`.

Operators with the same precedence are evaluated from left-to-right. In algebra we say they are left-associative. So in the expression `6-3+2`, the subtraction happens first, yielding `3`. We then add `2` to get the result `5`. If the operations had been evaluated from right to left, the result would have been `6-(3+2)`, which is `1`. (The acronym PEDMAS could mislead you to thinking that division has higher precedence than multiplication, and addition is done ahead of subtraction - don't be misled. Subtraction and addition are at the same precedence, and the left-to-right rule applies.)

Due to some historical quirk, an exception to the left-to-right left-associative rule is the exponentiation operator `**`, so a useful hint is to always use parentheses to force exactly the order you want when exponentiation is involved:

```
>>> 2 ** 3 ** 2     # The right-most ** operator gets done first!
512
>>> (2 ** 3) ** 2   # Use parentheses to force the order you want!
64
```

The immediate mode command prompt of Python is great for exploring and experimenting with expressions like this.

2.9. Operations on strings

In general, you cannot perform mathematical operations on strings, even if the strings look like numbers. The following are illegal (assuming that message has type string):

```
>>> message - 1        # Error
>>> "Hello" / 123      # Error
>>> message * "Hello"  # Error
>>> "15" + 2           # Error
```

Interestingly, the `+` operator does work with strings, but for strings, the `+` operator represents **concatenation**, not addition. Concatenation means joining the two operands by linking them end-to-end. For example:

```
fruit = "banana"
baked_good = " nut bread"
print(fruit + baked_good)
```

The output of this program is banana nut bread. The space before the word nut is part of the string, and is necessary to produce the space between the concatenated strings.

The * operator also works on strings; it performs repetition. For example, `'Fun'*3` is `'FunFunFun'`. One of the operands has to be a string; the other has to be an integer.

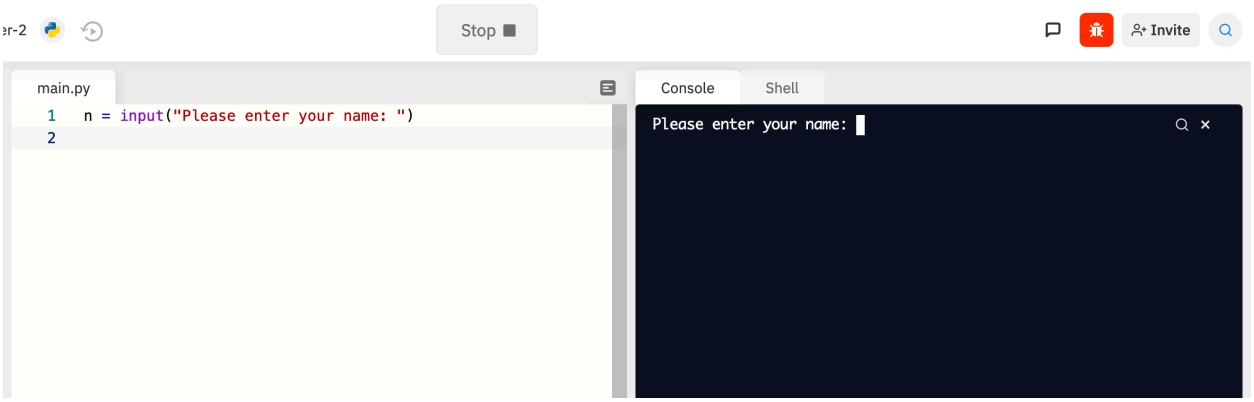
On one hand, this interpretation of + and * makes sense by analogy with addition and multiplication. Just as `4*3` is equivalent to `4+4+4`, we expect `"Fun"*3` to be the same as `"Fun"+"Fun"+"Fun"`, and it is. On the other hand, there is a significant way in which string concatenation and repetition are different from integer addition and multiplication. Can you think of a property that addition and multiplication have that string concatenation and repetition do not?

2.10. Input

There is a built-in function in Python for getting input from the user:

```
n = input("Please enter your name: ")
```

A sample run of this script in Replit would populate your input question in the console to the left like this:



Input Prompt

The user of the program can enter the name and press enter, and when this happens the text that has been entered is returned from the input function, and in this case assigned to the variable n.

Even if you asked the user to enter their age, you would get back a string like `"17"`. It would be your job, as the programmer, to convert that string into a int or a float, using the `int` or `float` converter functions we saw earlier.

2.11. Composition

So far, we have looked at the elements of a program — variables, expressions, statements, and function calls — in isolation, without talking about how to combine them.

One of the most useful features of programming languages is their ability to take small building blocks and **compose** them into larger chunks.

For example, we know how to get the user to enter some input, we know how to convert the string we get into a float, we know how to write a complex expression, and we know how to print values. Let's put these together in a small four-step program that asks the user to input a value for the radius of a circle, and then computes the area of the circle from the formula

$$A = \pi r^2$$

Area of a circle

Firstly, we'll do the four steps one at a time:

```
response = input("What is your radius? ")
r = float(response)
area = 3.14159 * r**2
print("The area is ", area)
```

Now let's compose the first two lines into a single line of code, and compose the second two lines into another line of code.

```
r = float( input("What is your radius? ") )
print("The area is ", 3.14159 * r**2)
```

If we really wanted to be tricky, we could write it all in one statement:

```
print("The area is ", 3.14159*float(input("What is your radius?"))**2)
```

Such compact code may not be most understandable for humans, but it does illustrate how we can compose bigger chunks from our building blocks.

If you're ever in doubt about whether to compose code or fragment it into smaller steps, try to make it as simple as you can for the human to follow. My choice would be the first case above, with four separate steps.

2.12. The modulus operator

The modulus operator works on integers (and integer expressions) and gives the remainder when the first number is divided by the second. In Python, the modulus operator is a percent sign (`%`). The syntax is the same as for other operators. It has the same precedence as the multiplication operator.

```
>>> q = 7 // 3     # This is integer division operator
>>> print(q)
2
>>> r  = 7 % 3
>>> print(r)
1
```

So `7` divided by `3` is `2` with a remainder of `1`.

The modulus operator turns out to be surprisingly useful. For example, you can check whether one number is divisible by another—if `x % y` is zero, then `x` is divisible by `y`.

Also, you can extract the right-most digit or digits from a number. For example, `x % 10` yields the right-most digit of `x` (in base `10`). Similarly `x % 100` yields the last two digits.

It is also extremely useful for doing conversions, say from seconds, to hours, minutes and seconds. So let's write a program to ask the user to enter some seconds, and we'll convert them into hours, minutes, and remaining seconds.

```
total_secs = int(input("How many seconds, in total?"))
hours = total_secs // 3600
secs_still_remaining = total_secs % 3600
minutes =  secs_still_remaining // 60
secs_finally_remaining = secs_still_remaining  % 60

print("Hrs=", hours, "  mins=", minutes,
                        "secs=", secs_finally_remaining)
```

2.13. Glossary

assignment statement

A statement that assigns a value to a name (variable). To the left of the assignment operator, =, is a name. To the right of the assignment token is an expression which is evaluated by the Python interpreter and then assigned to the name. The difference between the left and right hand sides of the assignment statement is often confusing to new programmers. In the following assignment:

```
1 n = n + 1
```

`n` plays a very different role on each side of the =. On the right it is a value and makes up part of the expression which will be evaluated by the Python interpreter before assigning it to the name on the left.

assignment token

= is Python's assignment token. Do not confuse it with *equals*, which is an operator for comparing values.

composition

The ability to combine simple expressions and statements into compound statements and expressions in order to represent complex computations concisely.

concatenate

To join two strings end-to-end.

data type

A set of values. The type of a value determines how it can be used in expressions. So far, the types you have seen are integers (`int`), floating-point numbers (`float`), and strings (`str`).

evaluate

To simplify an expression by performing the operations in order to yield a single value.

expression

A combination of variables, operators, and values that represents a single result value.

float

A Python data type which stores *floating-point* numbers. Floating-point numbers are stored internally in two parts: a *base* and an *exponent*. When printed in the standard format, they look like decimal numbers. Beware of rounding errors when you use `floats`, and remember that they are only approximate values.

floor division

An operator (denoted by the token `//`) that divides one number by another and yields an integer, or, if the result is not already an integer, it yields the next smallest integer.

int

A Python data type that holds positive and negative whole numbers.

keyword

A reserved word that is used by the compiler to parse programs; you cannot use keywords like `if`, `def`, and `while` as variable names.

modulus operator

An operator, denoted with a percent sign (`%`), that works on integers and yields the remainder when one number is divided by another.

operand

One of the values on which an operator operates.

operator

A special symbol that represents a simple computation like addition, multiplication, or string concatenation.

rules of precedence

The set of rules governing the order in which expressions involving multiple operators and operands are evaluated.

state snapshot

A graphical representation of a set of variables and the values to which they refer, taken at a particular instant during the program's execution.

statement

An instruction that the Python interpreter can execute. So far we have only seen the assignment statement, but we will soon meet the `import` statement and the `for` statement.

str

A Python data type that holds a string of characters.

value

A number or string (or other things to be named later) that can be stored in a variable or computed in an expression.

variable

A name that refers to a value.

variable name

A name given to a variable. Variable names in Python consist of a sequence of letters (`a..z`, `A..Z`, and _) and digits (0..9) that begins with a letter. In best programming practice, variable names should be chosen so that they describe their use in the program, making the program *self documenting*.

2.14. Exercises

1. Take the sentence: All work and no play makes Jack a dull boy. Store each word in a separate variable, then print out the sentence on one line using print.
2. Add parenthesis to the expression `6 * 1 - 2` to change its value from `4` to `-6`.
3. Place a comment before a line of code that previously worked, and record what happens when you rerun the program.
4. Start the Python interpreter and enter `bruce + 4` at the prompt. This will give you an error:

```
NameError: name 'bruce' is not defined
```

   Assign a value to bruce so that `bruce + 4` evaluates to `10`.
5. The formula for computing the final amount if one is earning compound interest is given on Wikipedia as

   Compounded Interest Formula

$$P' = P\left(1 + \frac{r}{n}\right)^{nt}$$

where:

P is the original principal sum

P′ is the new principal sum

r is the nominal annual interest rate

n is the compounding frequency

t is the overall length of time the interest is applied (expressed using the same time units as *r*, usually years).

Compounded Interest Formula

   Write a Python program that assigns the principal amount of $10000 to variable `P`, assign to n the value `12`, and assign to `r` the interest rate of `8%`. Then have the program prompt the user for the number of years `t` that the money will be compounded for. Calculate and print the final amount after `t` years.
6. Evaluate the following numerical expressions in your head, then use the Python interpreter to check your results:

```
>>> 5 % 2
>>> 9 % 5
>>> 15 % 12
>>> 12 % 15
>>> 6 % 6
>>> 0 % 7
>>> 7 % 0
```

   What happened with the last example? Why? If you were able to correctly anticipate the computer's response in all but the last one, it is time to move on. If not, take time now to

make up examples of your own. Explore the modulus operator until you are confident you understand how it works.

7. You look at the clock and it is exactly 2pm. You set an alarm to go off in 51 hours. At what time does the alarm go off? (*Hint: you could count on your fingers, but this is not what we're after. If you are tempted to count on your fingers, change the 51 to 5100.*)
8. Write a Python program to solve the general version of the above problem. Ask the user for the time now (in hours), and ask for the number of hours to wait. Your program should output what the time will be on the clock when the alarm goes off.

Chapter 3: Hello, little turtles!

There are many *modules* in Python that provide very powerful features that we can use in our own programs. Some of these can send email, or fetch web pages. The one we'll look at in this chapter allows us to create turtles and get them to draw shapes and patterns.

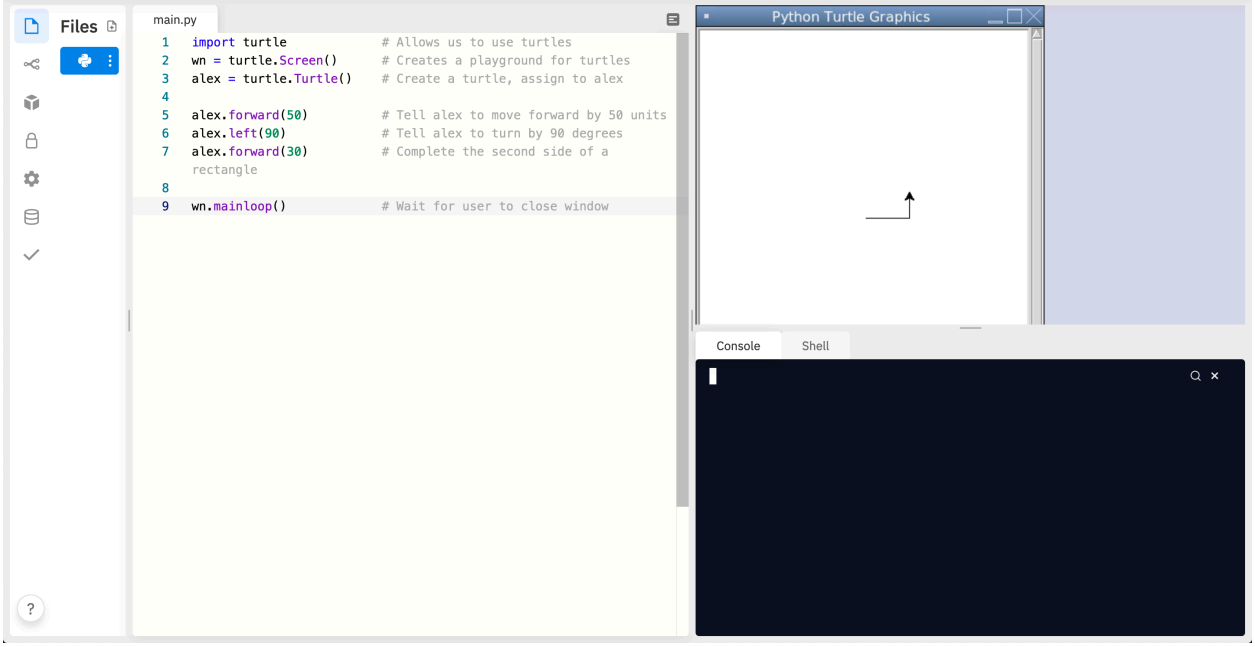
The turtles are fun, but the real purpose of the chapter is to teach ourselves a little more Python, and to develop our theme of *computational thinking*, or *thinking like a computer scientist*. Most of the Python covered here will be explored in more depth later.

3.1. Our first turtle program

Let's write a couple of lines of Python program to create a new turtle and start drawing a rectangle. (We'll call the variable that refers to our first turtle `alex`, but we can choose another name if we follow the naming rules from the previous chapter).

```
1  import turtle              # Allows us to use turtles
2  wn = turtle.Screen()       # Creates a playground for turtles
3  alex = turtle.Turtle()     # Create a turtle, assign to alex
4
5  alex.forward(50)           # Tell alex to move forward by 50 units
6  alex.left(90)              # Tell alex to turn by 90 degrees
7  alex.forward(30)           # Complete the second side of a rectangle
8
9  wn.mainloop()              # Wait for user to close window
```

When we run this program, a new window pops up:



Turtle Window

Here are a couple of things we'll need to understand about this program.

The first line tells Python to load a module named `turtle`. That module brings us two new types that we can use: the `Turtle` type, and the `Screen` type. The dot notation `turtle.Turtle` means "*The Turtle type that is defined within the turtle module*". (Remember that Python is case sensitive, so the module name, with a lowercase `t`, is different from the type `Turtle`.)

We then create and open what it calls a screen (we would prefer to call it a window), which we assign to variable `wn`. Every window contains a **canvas**, which is the area inside the window on which we can draw.

In line 3 we create a turtle. The variable `alex` is made to refer to this turtle.

So these first three lines have set things up, we're ready to get our turtle to draw on our canvas.

In lines 5-7, we instruct the **object** `alex` to move, and to turn. We do this by **invoking**, or activating, `alex's` **methods** — these are the instructions that all turtles know how to respond to.

The last line plays a part too: the `wn` variable refers to the window shown above. When we invoke its `mainloop` method, it enters a state where it waits for events (like keypresses, or mouse movement and clicks). The program will terminate when the user closes the window.

An object can have various methods — things it can do — and it can also have **attributes** — (sometimes called properties). For example, each turtle has a *color* attribute. The method invocation `alex.color("red")` will make `alex` red, and drawing will be red too. (Note the word *color* is spelled the American way!)

The color of the turtle, the width of its pen, the position of the turtle within the window, which way it is facing, and so on are all part of its current **state**. Similarly, the window object has a background

color, and some text in the title bar, and a size and position on the screen. These are all part of the state of the window object.

Quite a number of methods exist that allow us to modify the turtle and the window objects. We'll just show a couple. In this program we've only commented those lines that are different from the previous example (and we've used a different variable name for this turtle):

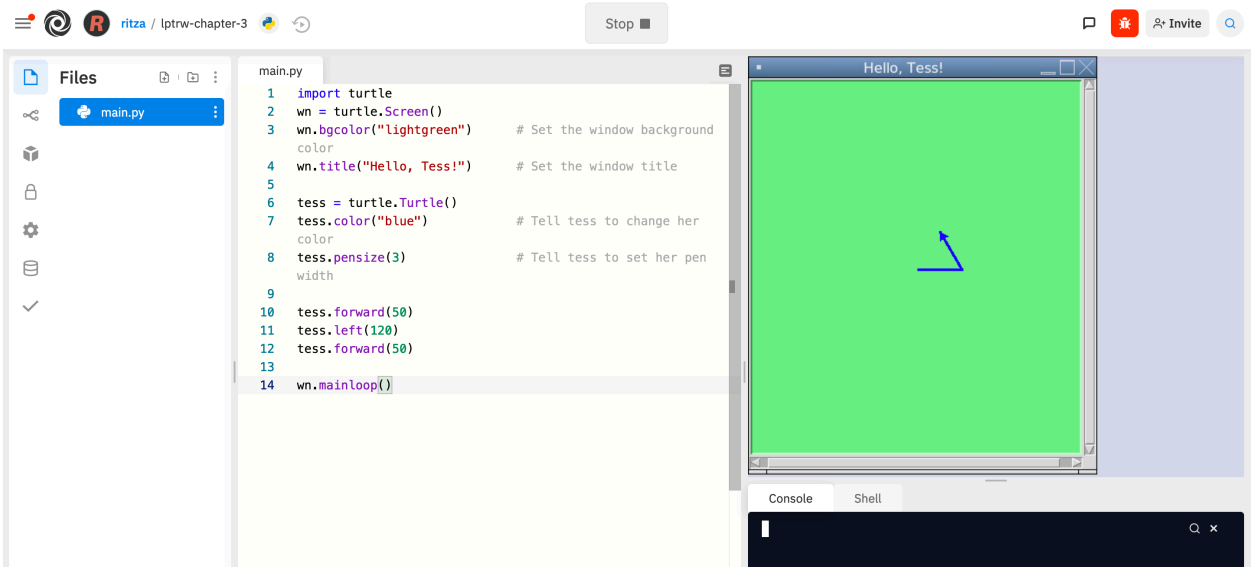
```
import turtle
wn = turtle.Screen()
wn.bgcolor("lightgreen")      # Set the window background color
wn.title("Hello, Tess!")      # Set the window title

tess = turtle.Turtle()
tess.color("blue")            # Tell tess to change her color
tess.pensize(3)               # Tell tess to set her pen width

tess.forward(50)
tess.left(120)
tess.forward(50)

wn.mainloop()
```

When we run this program, this new window pops up, and will remain on the screen until we close it.



tess mainloop

Extend this program ...

1. Modify this program so that before it creates the window, it prompts the user to enter the desired background color. It should store the user's responses in a variable, and modify the color of the window according to the user's wishes. (*Hint: you can find a list of permitted color names at http://www.tcl.tk/man/tcl8.4/TkCmd/colors.htm. It includes some quite unusual ones, like "peach puff" and "HotPink".*)
2. Do similar changes to allow the user, at runtime, to set tess' color.
3. Do the same for the width of `tess'` pen. *Hint: your dialog with the user will return a string, but tess'* `pensize` *method expects its argument to be an int. So you'll need to convert the string to an int before you pass it to* `pensize`.

3.2. Instances — a herd of turtles

Just like we can have many different integers in a program, we can have many turtles. Each of them is called an **instance**. Each instance has its own attributes and methods — so alex might draw with a thin black pen and be at some position, while `tess` might be going in her own direction with a fat pink pen.

```
import turtle
wn = turtle.Screen()         # Set up the window and its attributes
wn.bgcolor("lightgreen")
wn.title("Tess & Alex")

tess = turtle.Turtle()       # Create tess and set some attributes
tess.color("hotpink")
tess.pensize(5)

alex = turtle.Turtle()       # Create alex

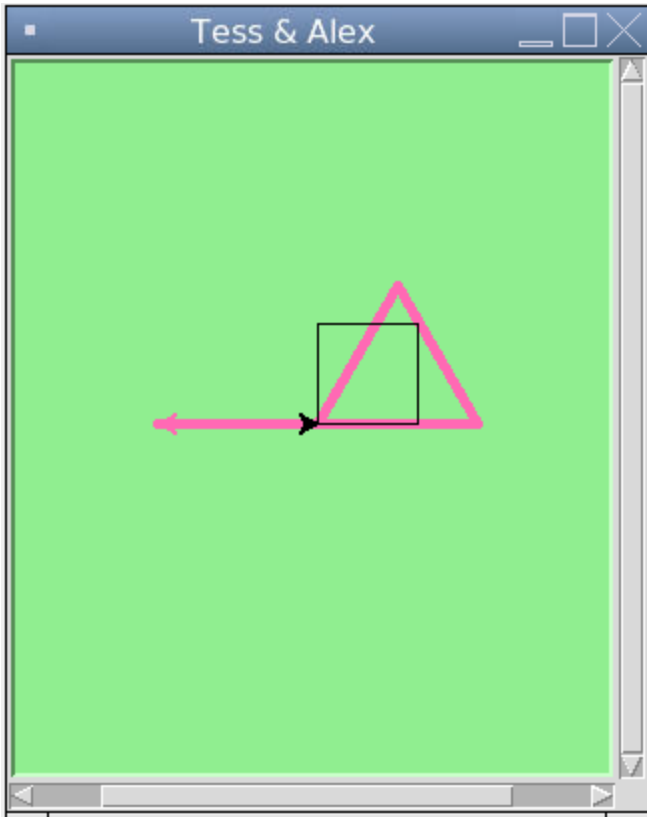
tess.forward(80)             # Make tess draw equilateral triangle
tess.left(120)
tess.forward(80)
tess.left(120)
tess.forward(80)
tess.left(120)               # Complete the triangle

tess.right(180)              # Turn tess around
tess.forward(80)             # Move her away from the origin

alex.forward(50)             # Make alex draw a square
alex.left(90)
alex.forward(50)
alex.left(90)
```

```
26 alex.forward(50)
27 alex.left(90)
28 alex.forward(50)
29 alex.left(90)
30
31 wn.mainloop()
```

Here is what happens when `alex` completes his rectangle, and `tess` completes her triangle:



Alex and Tess

Here are some *How to think like a computer scientist* observations:

- There are 360 degrees in a full circle. If we add up all the turns that a turtle makes, no matter what steps occurred between the turns, we can easily figure out if they add up to some multiple of 360. This should convince us that alex is facing in exactly the same direction as he was when he was first created. (Geometry conventions have 0 degrees facing East, and that is the case here too!)
- We could have left out the last turn for alex, but that would not have been as satisfying. If we're asked to draw a closed shape like a square or a rectangle, it is a good idea to complete all the turns and to leave the turtle back where it started, facing the same direction as it started in. This makes reasoning about the program and composing chunks of code into bigger programs easier for us humans!

- We did the same with tess: she drew her triangle, and turned through a full 360 degrees. Then we turned her around and moved her aside. Even the blank line 18 is a hint about how the programmer's mental chunking is working: in big terms, tess' movements were chunked as "draw the triangle" (lines 12-17) and then "move away from the origin" (lines 19 and 20).
- One of the key uses for comments is to record our mental chunking, and big ideas. They're not always explicit in the code.
- And, uh-huh, two turtles may not be enough for a herd. But the important idea is that the turtle module gives us a kind of factory that lets us create as many turtles as we need. Each instance has its own state and behaviour.

3.3. The for loop

When we drew the square, it was quite tedious. We had to explicitly repeat the steps of moving and turning four times. If we were drawing a hexagon, or an octagon, or a polygon with 42 sides, it would have been worse.

So a basic building block of all programs is to be able to repeat some code, over and over again.

Python's **`for`** loop solves this for us. Let's say we have some friends, and we'd like to send them each an email inviting them to our party. We don't quite know how to send email yet, so for the moment we'll just print a message for each friend:

```
1 for f in ["Joe","Zoe","Brad","Angelina","Zuki","Thandi","Paris"]:
2     invite = "Hi " + f + ".  Please come to my party on Saturday!"
3     print(invite)
4 # more code can follow here ...
```

When we run this, the output looks like this:

```
1 Hi Joe.  Please come to my party on Saturday!
2 Hi Zoe.  Please come to my party on Saturday!
3 Hi Brad.  Please come to my party on Saturday!
4 Hi Angelina.  Please come to my party on Saturday!
5 Hi Zuki.  Please come to my party on Saturday!
6 Hi Thandi.  Please come to my party on Saturday!
7 Hi Paris.  Please come to my party on Saturday!
```

- The variable `f` in the for statement at line `1` is called the **loop variable**. We could have chosen any other variable name instead.

Lines `2` and `3` are the **loop body**. The loop body is always indented. The indentation determines exactly what statements are "in the body of the loop".

- On each *iteration* or *pass* of the loop, first a check is done to see if there are still more items to be processed. If there are none left (this is called the **terminating condition** of the loop), the loop has finished. Program execution continues at the next statement after the loop body, (e.g. in this case the next statement below the comment in line `4`).
- If there are items still to be processed, the loop variable is updated to refer to the next item in the list. This means, in this case, that the loop body is executed here `7` times, and each time `f` will refer to a different friend.
- At the end of each execution of the body of the loop, Python returns to the for statement, to see if there are more items to be handled, and to assign the next one to `f`.

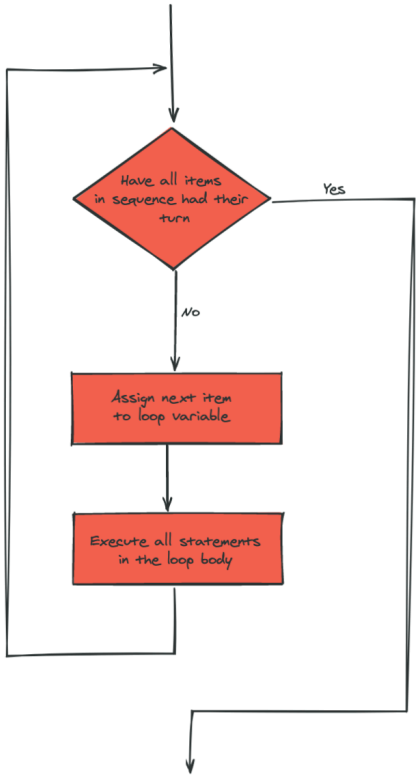
3.4. Flow of Execution of the for loop

As a program executes, the interpreter always keeps track of which statement is about to be executed. We call this the **control flow**, of the **flow of execution** of the program. When humans execute programs, they often use their finger to point to each statement in turn. So we could think of control flow as "Python's moving finger".

Control flow until now has been strictly top to bottom, one statement at a time. The for loop changes this.

Flowchart of a for loop

Control flow is often easy to visualize and understand if we draw a flowchart. This shows the exact steps and logic of how the for statement executes.



For loop flowchart

3.5. The loop simplifies our turtle program

To draw a square we'd like to do the same thing four times — move the turtle, and turn. We previously used 8 lines to have alex draw the four sides of a square. This does exactly the same, but using just three lines:

```
for i in [0,1,2,3]:
    alex.forward(50)
    alex.left(90)
```

Some observations:

- While "saving some lines of code" might be convenient, it is not the big deal here. What is much more important is that we've found a "repeating pattern" of statements, and reorganized our program to repeat the pattern. Finding the chunks and somehow getting our programs arranged around those chunks is a vital skill in computational thinking.

- The values `[0,1,2,3]` were provided to make the loop body execute `4` times. We could have used any four values, but these are the conventional ones to use. In fact, they are so popular that Python gives us special built-in range objects:

```
for i in range(4):
    # Executes the body with i = 0, then 1, then 2, then 3
for x in range(10):
    # Sets x to each of ... [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- Computer scientists like to count from `0`!
- `range` can deliver a sequence of values to the loop variable in the `for` loop. They start at `0`, and in these cases do not include the `4` or the `10`.
- Our little trick earlier to make sure that alex did the final turn to complete 360 degrees has paid off: if we had not done that, then we would not have been able to use a loop for the fourth side of the square. It would have become a "special case", different from the other sides. When possible, we'd much prefer to make our code fit a general pattern, rather than have to create a special case.

So to repeat something four times, a good Python programmer would do this:

```
for i in range(4):
    alex.forward(50)
    alex.left(90)
```

By now you should be able to see how to change our previous program so that tess can also use a `for` loop to draw her equilateral triangle.

But now, what would happen if we made this change?

```
for c in ["yellow", "red", "purple", "blue"]:
    alex.color(c)
    alex.forward(50)
    alex.left(90)
```

A variable can also be assigned a value that is a list. So lists can also be used in more general situations, not only in the `for` loop. The code above could be rewritten like this:

```
# Assign a list to a variable
clrs = ["yellow", "red", "purple", "blue"]
for c in clrs:
    alex.color(c)
    alex.forward(50)
    alex.left(90)
```

3.6. A few more turtle methods and tricks

Turtle methods can use negative angles or distances. So `tess.forward(-100)` will move tess backwards, and `tess.left(-30)` turns her to the right. Additionally, because there are 360 degrees in a circle, turning 30 to the left will get tess facing in the same direction as turning 330 to the right! (The on-screen animation will differ, though — you will be able to tell if tess is turning clockwise or counter-clockwise!)

This suggests that we don't need both a left and a right turn method — we could be minimalists, and just have one method. There is also a *backward* method. (If you are very nerdy, you might enjoy saying `alex.backward(-100)` to move `alex` forward!)

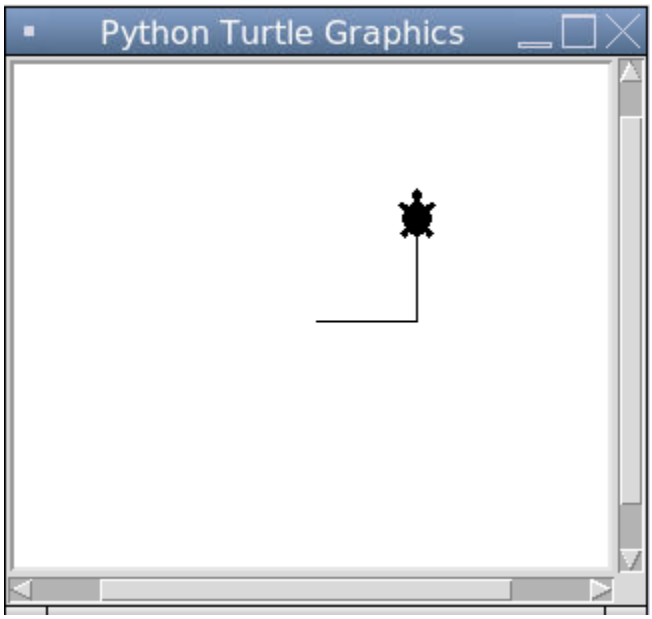
Part of thinking like a scientist is to understand more of the structure and rich relationships in our field. So revising a few basic facts about geometry and number lines, and spotting the relationships between left, right, backward, forward, negative and positive distances or angles values is a good start if we're going to play with turtles.

A turtle's pen can be picked up or put down. This allows us to move a turtle to a different place without drawing a line. The methods are

```
alex.penup()
alex.forward(100)     # This moves alex, but no line is drawn
alex.pendown()
```

Every turtle can have its own shape. The ones available "out of the box" are `arrow`, `blank`, `circle`, `classic`, `square`, `triangle`, `turtle`.

```
alex.shape("turtle")
```



Turtle Shape

We can speed up or slow down the turtle's animation speed. (Animation controls how quickly the turtle turns and moves forward). Speed settings can be set between `1` (slowest) to `10` (fastest). But if we set the speed to `0`, it has a special meaning — turn off animation and go as fast as possible.

```
alex.speed(10)
```

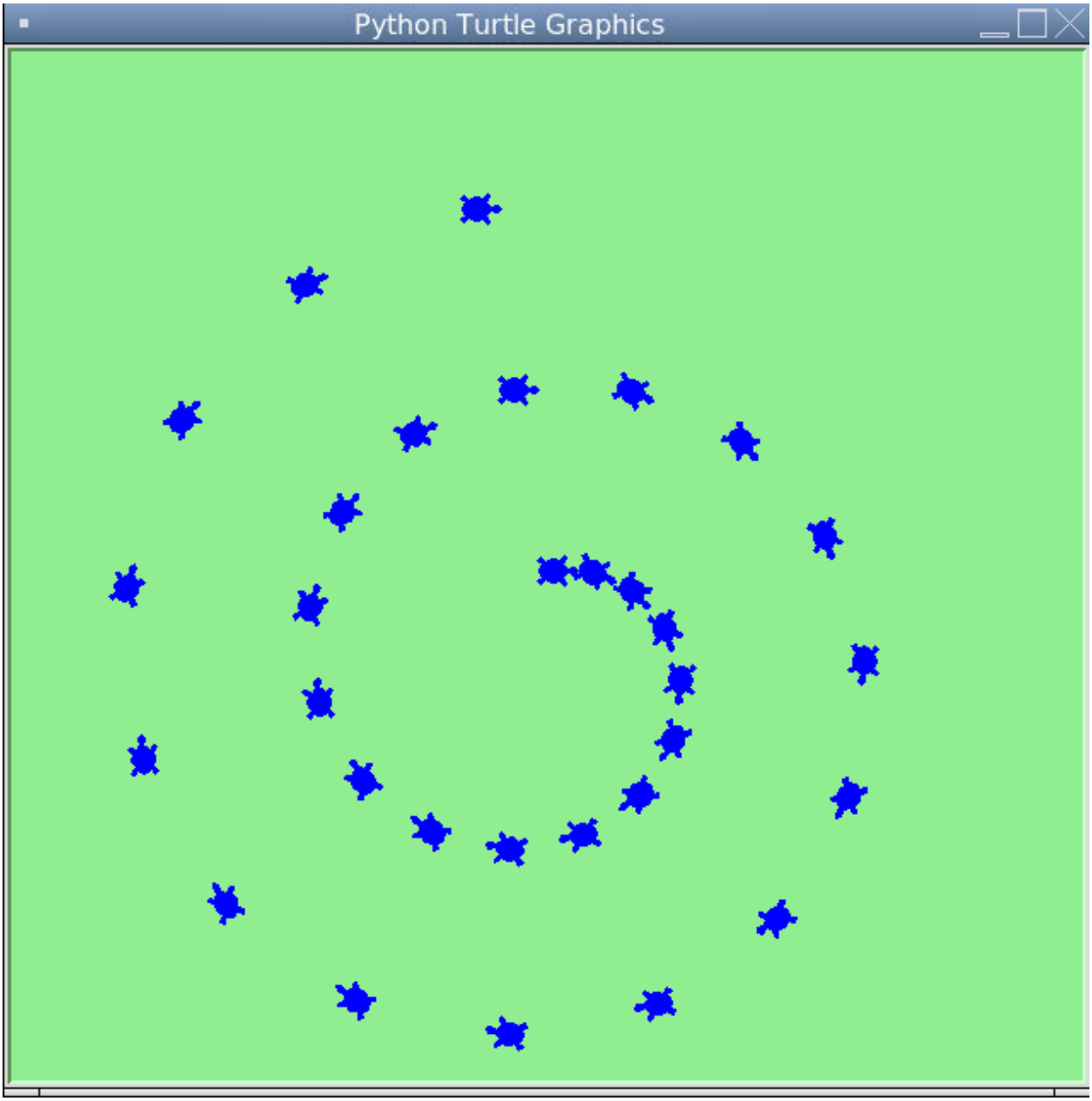
A turtle can "stamp" its footprint onto the canvas, and this will remain after the turtle has moved somewhere else. Stamping works, even when the pen is up.

Let's do an example that shows off some of these new features:

```
import turtle
wn = turtle.Screen()
wn.bgcolor("lightgreen")
tess = turtle.Turtle()
tess.shape("turtle")
tess.color("blue")

tess.penup()                # This is new
size = 20
for i in range(30):
    tess.stamp()            # Leave an impression on the canvas
    size = size + 3         # Increase the size on every iteration
    tess.forward(size)      # Move tess along
    tess.right(24)          #  ...  and turn her
```

```
15
16 wn.mainloop()
```



Turtle Spiral

Be careful now! How many times was the body of the loop executed? How many turtle images do we see on the screen? All except one of the shapes we see on the screen here are footprints created by stamp. But the program still only has one turtle instance — can you figure out which one here is the real tess? (*Hint: if you're not sure, write a new line of code after the for loop to change tess' color, or to put her pen down and draw a line, or to change her shape, etc.*)

3.7. Glossary

attribute

Some state or value that belongs to a particular object. For example, tess has a color.

canvas

A surface within a window where drawing takes place.

control flow

See flow of execution in the next chapter.

for loop

A statement in Python for convenient repetition of statements in the body of the loop.

loop body

Any number of statements nested inside a loop. The nesting is indicated by the fact that the statements are indented under the `for` loop statement.

loop variable

A variable used as part of a `for` loop. It is assigned a different value on each iteration of the loop.

instance

An object of a certain type, or class. tess and alex are different instances of the class Turtle.

method

A function that is attached to an object. Invoking or activating the method causes the object to respond in some way, e.g. forward is the method when we say `tess.forward(100)`.

invoke

An object has methods. We use the verb invoke to mean activate the method. Invoking a method is done by putting parentheses after the method name, with some possible arguments. So `tess.forward()` is an invocation of the forward method.

module

A file containing Python definitions and statements intended for use in other Python programs. The contents of a module are made available to the other program by using the import statement.

object

A "thing" to which a variable can refer. This could be a screen window, or one of the turtles we have created.

range

A built-in function in Python for generating sequences of integers. It is especially useful when we need to write a for loop that executes a fixed number of times.

terminating condition

A condition that occurs which causes a loop to stop repeating its body. In the for loops we saw in this chapter, the terminating condition has been when there are no more elements to assign to the loop variable.

3.8. Exercises

1. Write a program that prints `We like Python's turtles!` 1000 times.
2. Give three attributes of your cellphone object. Give three methods of your cellphone.
3. Write a program that uses a `for` loop to print

```
One of the months of the year is January
One of the months of the year is February
...
```

4. Suppose our turtle tess is at heading `0` — facing east. We execute the statement `tess.left(3645)`. What does tess do, and what is her final heading?
5. Assume you have the assignment `xs = [12, 10, 32, 3, 66, 17, 42, 99, 20]`

   a. Write a loop that prints each of the numbers on a new line.

   b. Write a loop that prints each number and its square on a new line.

   c. Write a loop that adds all the numbers from the list into a variable called `total`. You should set the `total` variable to have the value `0` before you start adding them up, and print the value in total after the loop has completed.

   d. Print the product of all the numbers in the list. (product means all multiplied together)
6. Use `for` loops to make a turtle draw these regular polygons (regular means all sides the same lengths, all angles the same):

   - An equilateral triangle
   - A square
   - A hexagon (six sides)
   - An octagon (eight sides)
7. A drunk pirate makes a random turn and then takes `100` steps forward, makes another random turn, takes another `100` steps, turns another random amount, etc. A social science student records the angle of each turn before the next `100` steps are taken. Her experimental data is `[160, -43, 270, -97, -43, 200, -940, 17, -86]`. (Positive angles are counter-clockwise.) Use a turtle to draw the path taken by our drunk friend.
8. Enhance your program above to also tell us what the drunk pirate's heading is after he has finished stumbling around. (Assume he begins at heading `0`).
9. If you were going to draw a regular polygon with 18 sides, what angle would you need to turn the turtle at each corner?
10. At the interactive prompt, anticipate what each of the following lines will do, and then record what happens. Score yourself, giving yourself one point for each one you anticipate correctly:

```
>>> import turtle
>>> wn = turtle.Screen()
>>> tess = turtle.Turtle()
>>> tess.right(90)
>>> tess.left(3600)
>>> tess.right(-90)
>>> tess.speed(10)
>>> tess.left(3600)
>>> tess.speed(0)
>>> tess.left(3645)
>>> tess.forward(-100)
```

11. Write a program to draw a shape like this:

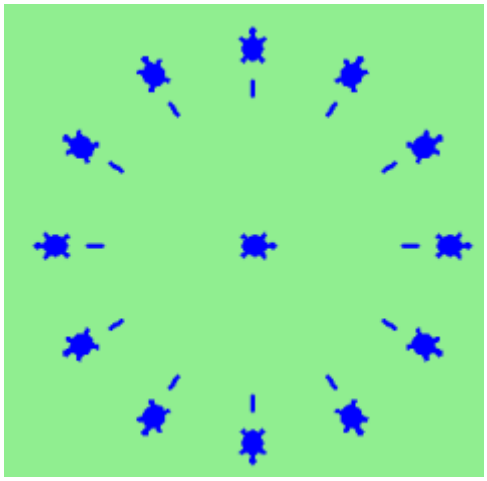
Star

Hints:

- Try this on a piece of paper, moving and turning your cellphone as if it was a turtle. Watch how many complete rotations your cellphone makes before you complete the star. Since each full rotation is 360 degrees, you can figure out the total number of degrees that your phone was rotated through. If you divide that by 5, because there are five points to the star, you'll know how many degrees to turn the turtle at each point.

- You can hide a turtle behind its invisibility cloak if you don't want it shown. It will still draw its lines if its pen is down. The method is invoked as `tess.hideturtle()`. To make the turtle visible again, use `tess.showturtle()`.

12. Write a program to draw a face of a clock that looks something like this:



Clock face

13. Create a turtle, and assign it to a variable. When you ask for its type, what do you get?
14. What is the collective noun for turtles? (Hint: they don't come in *herds*.)
15. What the collective noun for pythons? Is a python a viper? Is a python venomous?

Chapter 4: Functions

4.1. Functions

In Python, a **function** is a named sequence of statements that belong together. Their primary purpose is to help us organize programs into chunks that match how we think about the problem.

The syntax for a **function definition** is:

```
def NAME( PARAMETERS ):
    STATEMENTS
```

We can make up any names we want for the functions we create, except that we can't use a name that is a Python keyword, and the names must follow the rules for legal identifiers.

There can be any number of statements inside the function, but they have to be indented from the `def`. In the examples in this book, we will use the standard indentation of four spaces. Function definitions are the second of several **compound statements** we will see, all of which have the same pattern:

1. A header line which begins with a keyword and ends with a colon.
2. A **body** consisting of one or more Python statements, each indented the same amount — the *Python style guide recommends 4 spaces* — from the header line.

We've already seen the `for` loop which follows this pattern.

So looking again at the function definition, the keyword in the header is `def`, which is followed by the name of the function and some *parameters* enclosed in parentheses. The parameter list may be empty, or it may contain any number of parameters separated from one another by commas. In either case, the parentheses are required. The parameters specifies what information, if any, we have to provide in order to use the new function.

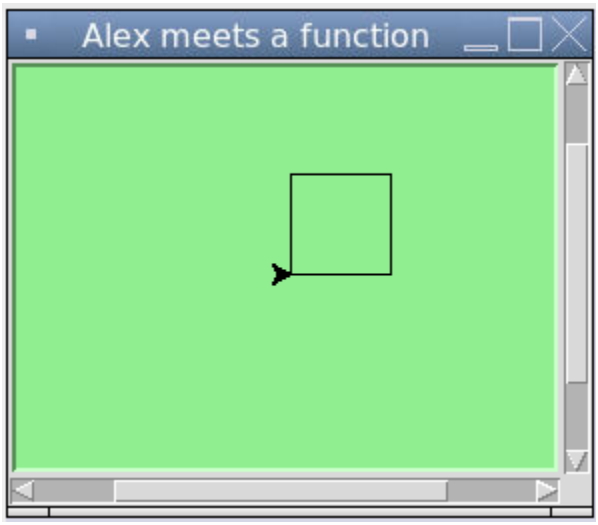
Suppose we're working with turtles, and a common operation we need is to draw squares. "Draw a square" is an abstraction, or a mental chunk, of a number of smaller steps. So let's write a function to capture the pattern of this "building block":

```
import turtle

def draw_square(t, sz):
    """Make turtle t draw a square of sz."""
    for i in range(4):
        t.forward(sz)
        t.left(90)


wn = turtle.Screen()        # Set up the window and its attributes
wn.bgcolor("lightgreen")
wn.title("Alex meets a function")

alex = turtle.Turtle()      # Create alex
draw_square(alex, 50)       # Call the function to draw the square
wn.mainloop()
```



alex function

This function is named `draw_square`. It has two parameters: one to tell the function which turtle to move around, and the other to tell it the size of the square we want drawn. Make sure you know where the body of the function ends — it depends on the indentation, and the blank lines don't count for this purpose!

Docstrings for documentation

If the first thing after the function header is a string, it is treated as a **docstring** and gets special treatment in Python and in some programming tools. For example, when we type a built-in function name with an unclosed parenthesis in Repl.it, a tooltip pops up, telling us what arguments the function takes, and it shows us any other text contained in the docstring.

Docstrings are the key way to document our functions in Python and the documentation part is important. Because whoever calls our function shouldn't have to need to know what is going on in the function or how it works; they just need to know what arguments our function takes, what it does, and what the expected result is. Enough to be able to use the function without having to look underneath. This goes back to the concept of abstraction of which we'll talk more about.

Docstrings are usually formed using triple-quoted strings as they allow us to easily expand the docstring later on should we want to write more than a one-liner.

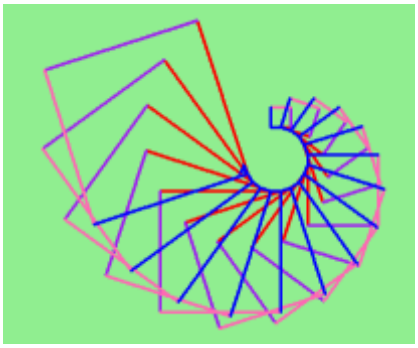
Just to differentiate from comments, a string at the start of a function (a docstring) is retrievable by Python tools at runtime. By contrast, comments are completely eliminated when the program is parsed.

Defining a new function does not make the function run. To do that we need a **function call**. We've already seen how to call some built-in functions like **`print`**, **`range`** and **`int`**. Function calls contain the name of the function being executed followed by a list of values, called *arguments*, which are assigned to the parameters in the function definition. So in the second last line of the program, we call the function, and pass alex as the turtle to be manipulated, and `50` as the size of the square we want. While the function is executing, then, the variable `sz` refers to the value `50`, and the variable t refers to the same turtle instance that the variable `alex` refers to.

Once we've defined a function, we can call it as often as we like, and its statements will be executed each time we call it. And we could use it to get any of our turtles to draw a square. In the next example, we've changed the draw_square function a little, and we get tess to draw 15 squares, with some variations.

```
1  import turtle
2
3  def draw_multicolor_square(t, sz):
4      """Make turtle t draw a multi-color square of sz."""
5      for i in ["red", "purple", "hotpink", "blue"]:
6          t.color(i)
7          t.forward(sz)
8          t.left(90)
9
10 wn = turtle.Screen()        # Set up the window and its attributes
11 wn.bgcolor("lightgreen")
12
13 tess = turtle.Turtle()      # Create tess and set some attributes
14 tess.pensize(3)
15
16 size = 20                   # Size of the smallest square
17 for i in range(15):
18     draw_multicolor_square(tess, size)
19     size = size + 10        # Increase the size for next time
```

```
20      tess.forward(10)        # Move tess along a little
21      tess.right(18)          #    and give her some turn
22
23  wn.mainloop()
```



Draw multicolor square

4.2. Functions can call other functions

Let's assume now we want a function to draw a rectangle. We need to be able to call the function with different arguments for width and height. And, unlike the case of the square, we cannot repeat the same thing 4 times, because the four sides are not equal.

So we eventually come up with this rather nice code that can draw a rectangle.

```
1  def draw_rectangle(t, w, h):
2      """Get turtle t to draw a rectangle of width w and height h."""
3      for i in range(2):
4          t.forward(w)
5          t.left(90)
6          t.forward(h)
7          t.left(90)
```

The parameter names are deliberately chosen as single letters to ensure they're not misunderstood. In real programs, once we've had more experience, we will insist on better variable names than this. But the point is that the program doesn't "understand" that we're drawing a rectangle, or that the parameters represent the width and the height. Concepts like rectangle, width, and height are the meaning we humans have, not concepts that the program or the computer understands.

Thinking like a scientist involves looking for patterns and relationships. In the code above, we've done that to some extent. We did not just draw four sides. Instead, we spotted that we could draw the rectangle as two halves, and used a loop to repeat that pattern twice.

But now we might spot that a square is a special kind of rectangle. We already have a function that draws a rectangle, so we can use that to draw our square.

```
def draw_square(tx, sz):        # A new version of draw_square
    draw_rectangle(tx, sz, sz)
```

There are some points worth noting here:

- Functions can call other functions.
- Rewriting `draw_square` like this captures the relationship that we've spotted between squares and rectangles.
- A caller of this function might say `draw_square(tess, 50)`. The parameters of this function, `tx` and `sz`, are assigned the values of the tess object, and the `int 50` respectively.
- In the body of the function they are just like any other variable.
- When the call is made to `draw_rectangle`, the values in variables `tx` and `sz` are fetched first, then the call happens. So as we enter the top of function `draw_rectangle`, its variable `t` is assigned the `tess` object, and `w` and `h` in that function are both given the value `50`.

So far, it may not be clear why it is worth the trouble to create all of these new functions. Actually, there are a lot of reasons, but this example demonstrates two:

1. Creating a new function gives us an opportunity to name a group of statements. Functions can simplify a program by hiding a complex computation behind a single command. The function (including its name) can capture our mental chunking, or *abstraction*, of the problem.
2. Creating a new function can make a program smaller by eliminating repetitive code.

As we might expect, we have to create a function before we can execute it. In other words, the function definition has to be executed before the function is called.

4.3. Flow of execution

In order to ensure that a function is defined before its first use, we have to know the order in which statements are executed, which is called the **flow of execution**. We've already talked about this a little in the previous chapter.

Execution always begins at the first statement of the program. Statements are executed one at a time, in order from top to bottom.

Function definitions do not alter the flow of execution of the program, but remember that statements inside the function are not executed until the function is called. Although it is not common, we can define one function inside another. In this case, the inner definition isn't executed until the outer function is called.

Function calls are like a detour in the flow of execution. Instead of going to the next statement, the flow jumps to the first line of the called function, executes all the statements there, and then comes back to pick up where it left off.

That sounds simple enough, until we remember that one function can call another. While in the middle of one function, the program might have to execute the statements in another function. But while executing that new function, the program might have to execute yet another function!

Fortunately, Python is adept at keeping track of where it is, so each time a function completes, the program picks up where it left off in the function that called it. When it gets to the end of the program, it terminates.

What's the moral of this sordid tale? When we read a program, don't read from top to bottom. Instead, follow the flow of execution.

Watch the flow of execution in action

Repl.it does not have "single-stepping" functionality. For this we would recommend a different IDE like PyScripter[6].

In PyScripter, we can watch the flow of execution by "single-stepping" through any program. PyScripter will highlight each line of code just before it is about to be executed.

PyScripter also lets us hover the mouse over any variable in the program, and it will pop up the current value of that variable. So this makes it easy to inspect the "state snapshot" of the program — the current values that are assigned to the program's variables.

This is a powerful mechanism for building a deep and thorough understanding of what is happening at each step of the way. Learn to use the single-stepping feature well, and be mentally proactive: as you work through the code, challenge yourself before each step: *"What changes will this line make to any variables in the program?"* and "Where will flow of execution go next?"

Let us go back and see how this works with the program above that draws 15 multicolor squares. First, we're going to add one line of magic below the import statement — not strictly necessary, but it will make our lives much simpler, because it prevents stepping into the module containing the turtle code.

```
1 import turtle
2 __import__("turtle").__traceable__ = False
```

Now we're ready to begin. Put the mouse cursor on the line of the program where we create the turtle screen, and press the F4 key. This will run the Python program up to, but not including, the line where we have the cursor. Our program will "break" now, and provide a highlight on the next line to be executed, something like this:

[6] https://sourceforge.net/projects/pyscripter/

```
 1 import turtle
 2 __import__("turtle").__traceable__ = False
 3
 4 def draw_multicolor_square(t, sz):
 5     """Make turtle t draw a multi-color square of sz."""
 6     for i in ["red", "purple", "hotpink", "blue"]:
 7         t.color(i)
 8         t.forward(sz)
 9         t.left(90)
10
11 wn = turtle.Screen()        # Set up the window and its attributes
12 wn.bgcolor("lightgreen")
13
14 tess = turtle.Turtle()      # Create tess and set some attributes
15 tess.pensize(3)
16
17 size = 20                   # Size of the smallest square
18 for i in range(15):
19     draw_multicolor_square(tess, size)
20     size = size + 10        # Increase the size for next time
21     tess.forward(10)        # Move tess along a little
22     tess.right(18)          #  ...  and give her some extra turn
23
24 wn.mainloop()
25
```

PyScripter Breakpoint

At this point we can press the F7 key (*step into*) repeatedly to single step through the code. Observe as we execute lines 10, 11, 12, … how the turtle window gets created, how its canvas color is changed, how the title gets changed, how the turtle is created on the canvas, and then how the flow of execution gets into the loop, and from there into the function, and into the function's loop, and then repeatedly through the body of that loop.

While we do this, we can also hover our mouse over some of the variables in the program, and confirm that their values match our conceptual model of what is happening.

After a few loops, when we're about to execute line 20 and we're starting to get bored, we can use the key F8 to "step over" the function we are calling. This executes all the statements in the function, but without having to step through each one. We always have the choice to either "go for the detail", or to "take the high-level view" and execute the function as a single chunk.

There are some other options, including one that allow us to resume execution without further stepping. Find them under the *Run* menu of PyScripter.

4.4. Functions that require arguments

Most functions require arguments: the arguments provide for generalization. For example, if we want to find the absolute value of a number, we have to indicate what the number is. Python has a built-in function for computing the absolute value:

```
>>> abs(5)
5
>>> abs(-5)
5
```

In this example, the arguments to the `abs` function are `5` and `-5`.

Some functions take more than one argument. For example the built-in function `pow` takes two arguments, the base and the exponent. Inside the function, the values that are passed get assigned to variables called **parameters**.

```
>>> pow(2, 3)
8
>>> pow(7, 4)
2401
```

Another built-in function that takes more than one argument is `max`.

```
>>> max(7, 11)
11
>>> max(4, 1, 17, 2, 12)
17
>>> max(3 * 11, 5**3, 512 - 9, 1024**0)
503
```

`max` can be passed any number of arguments, separated by commas, and will return the largest value passed. The arguments can be either simple values or expressions. In the last example, `503` is returned, since it is larger than `33`, `125`, and `1`.

4.5. Functions that return values

All the functions in the previous section return values. Furthermore, functions like `range`, `int`, `abs` all return values that can be used to build more complex expressions.

So an important difference between these functions and one like `draw_square` is that `draw_square` was not executed because we wanted it to compute a value — on the contrary, we wrote `draw_square` because we wanted it to execute a sequence of steps that caused the turtle to draw.

A function that returns a value is called a **fruitful function** in this book. The opposite of a fruitful function is **void function** — one that is not executed for its resulting value, but is executed because it does something useful. (Languages like Java, C#, C and C++ use the term "void function", other languages like Pascal call it a **procedure**.) Even though void functions are not executed for their resulting value, Python always wants to return something. So if the programmer doesn't arrange to return a value, Python will automatically return the value `None`.

How do we write our own fruitful function? In the exercises at the end of chapter 2 we saw the standard formula for compound interest, which we'll now write as a fruitful function:

$$P' = P\left(1 + \frac{r}{n}\right)^{nt}$$

where:

P is the original principal sum

P′ is the new principal sum

r is the nominal annual interest rate

n is the compounding frequency

t is the overall length of time the interest is applied (expressed using the same time units as *r*, usually years).

Compound interest

```
def final_amt(p, r, n, t):
    """
      Apply the compound interest formula to p
       to produce the final amount.
    """

    a = p * (1 + r/n) ** (n*t)
    return a          # This is new, and makes the function fruitful.

# now that we have the function above, let us call it.
toInvest = float(input("How much do you want to invest?"))
fnl = final_amt(toInvest, 0.08, 12, 5)
print("At the end of the period you'll have", fnl)
```

- The return statement is followed by an expression (`a` in this case). This expression will be evaluated and returned to the caller as the "fruit" of calling this function.
- We prompted the user for the principal amount. The type of `toInvest` is a string, but we need a number before we can work with it. Because it is money, and could have decimal places, we've used the `float` type converter function to parse the string and return a float.
- Notice how we entered the arguments for 8% interest, compounded 12 times per year, for 5 years.
- When we run this, we get the output

```
1    At the end of the period you'll have 14898.457083
```

This is a bit messy with all these decimal places, but remember that Python doesn't understand that we're working with money: it just does the calculation to the best of its ability, without rounding. Later we'll see how to format the string that is printed in such a way that it does get nicely rounded to two decimal places before printing.

- The line `toInvest = float(input("How much do you want to invest?"))` also shows yet another example of *composition* — we can call a function like `float`, and its arguments can be the results of other function calls (like input) that we've called along the way.

Notice something else very important here. The name of the variable we pass as an argument — `toInvest` — has nothing to do with the name of the parameter — `p`. It is as if `p = toInvest` is executed when `final_amt` is called. It doesn't matter what the value was named in the caller, in `final_amt` its name is `p`.

These short variable names are getting quite tricky, so perhaps we'd prefer one of these versions instead:

```
1  def final_amt_v2(principalAmount, nominalPercentageRate,
2                                    numTimesPerYear, years):
3      a = principalAmount * (1 + nominalPercentageRate /
4                           numTimesPerYear) ** (numTimesPerYear*years)
5      return a
6
7  def final_amt_v3(amt, rate, compounded, years):
8      a = amt * (1 + rate/compounded) ** (compounded*years)
9      return a
```

They all do the same thing. Use your judgement to write code that can be best understood by other humans! Short variable names are more economical and sometimes make code easier to read: `E = mc2` would not be nearly so memorable if Einstein had used longer variable names! If you do prefer short names, make sure you also have some comments to enlighten the reader about what the variables are used for.

4.6. Variables and parameters are local

When we create a **local variable** inside a function, it only exists inside the function, and we cannot use it outside. For example, consider again this function:

```
def final_amt(p, r, n, t):
    a = p * (1 + r/n) ** (n*t)
    return a
```

If we try to use `a`, outside the function, we'll get an error:

```
>>> a
NameError: name 'a' is not defined
```

The variable `a` is local to `final_amt`, and is not visible outside the function.

Additionally, `a` only exists while the function is being executed — we call this its **lifetime**. When the execution of the function terminates, the local variables are destroyed.

Parameters are also local, and act like local variables. For example, the lifetimes of `p`, `r`, `n`, `t` begin when `final_amt` is called, and the lifetime ends when the function completes its execution.

So it is not possible for a function to set some local variable to a value, complete its execution, and then when it is called again next time, recover the local variable. Each call of the function creates new local variables, and their lifetimes expire when the function returns to the caller.

4.7. Turtles Revisited

Now that we have fruitful functions, we can focus our attention on reorganizing our code so that it fits more nicely into our mental chunks. This process of rearrangement is called **refactoring** the code.

Two things we're always going to want to do when working with turtles is to create the window for the turtle, and to create one or more turtles. We could write some functions to make these tasks easier in future:

```
def make_window(colr, ttle):
    """
      Set up the window with the given background color and title.
      Returns the new window.
    """
    w = turtle.Screen()
    w.bgcolor(colr)
    w.title(ttle)
    return w


def make_turtle(colr, sz):
```

```
    """
      Set up a turtle with the given color and pensize.
      Returns the new turtle.
    """
    t = turtle.Turtle()
    t.color(colr)
    t.pensize(sz)
    return t


wn = make_window("lightgreen", "Tess and Alex dancing")
tess = make_turtle("hotpink", 5)
alex = make_turtle("black", 1)
dave = make_turtle("yellow", 2)
```

The trick about refactoring code is to anticipate which things we are likely to want to change each time we call the function: these should become the parameters, or changeable parts, of the functions we write.

4.8. Glossary

argument

A value provided to a function when the function is called. This value is assigned to the corresponding parameter in the function. The argument can be the result of an expression which may involve operators, operands and calls to other fruitful functions.

body

The second part of a compound statement. The body consists of a sequence of statements all indented the same amount from the beginning of the header. The standard amount of indentation used within the Python community is 4 spaces.

compound statement

A statement that consists of two parts:

1. header - which begins with a keyword determining the statement type, and ends with a colon.
2. body - containing one or more statements indented the same amount from the header.

The syntax of a compound statement looks like this:

```
keyword ... :
    statement
    statement ...
```

docstring

A special string that is attached to a function as its `__doc__` attribute. Tools like Repl.it can use docstrings to provide documentation or hints for the programmer. When we get to modules, classes, and methods, we'll see that docstrings can also be used there.

flow of execution

The order in which statements are executed during a program run.

frame

A box in a stack diagram that represents a function call. It contains the local variables and parameters of the function.

function

A named sequence of statements that performs some useful operation. Functions may or may not take parameters and may or may not produce a result.

function call

A statement that executes a function. It consists of the name of the function followed by a list of arguments enclosed in parentheses.

function composition

Using the output from one function call as the input to another.

function definition

A statement that creates a new function, specifying its name, parameters, and the statements it executes.

fruitful function

A function that returns a value when it is called.

header line

The first part of a compound statement. A header line begins with a keyword and ends with a colon (:)

import statement

A statement which permits functions and variables defined in another Python module to be brought into the environment of another script. To use the features of the turtle, we need to first import the turtle module.

lifetime

Variables and objects have lifetimes — they are created at some point during program execution, and will be destroyed at some time.

local variable

A variable defined inside a function. A local variable can only be used inside its function. Parameters of a function are also a special kind of local variable.

parameter

A name used inside a function to refer to the value which was passed to it as an argument.

refactor

A fancy word to describe reorganizing our program code, usually to make it more understandable. Typically, we have a program that is already working, then we go back to "tidy it up". It often involves choosing better variable names, or spotting repeated patterns and moving that code into a function.

stack diagram

A graphical representation of a stack of functions, their variables, and the values to which they refer.

traceback

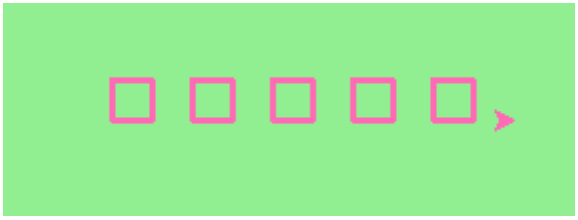
A list of the functions that are executing, printed when a runtime error occurs. A traceback is also commonly referred to as a *stack trace*, since it lists the functions in the order in which they are stored in the runtime stack.[7]

void function

The opposite of a fruitful function: one that does not return a value. It is executed for the work it does, rather than for the value it returns.

4.9. Exercises

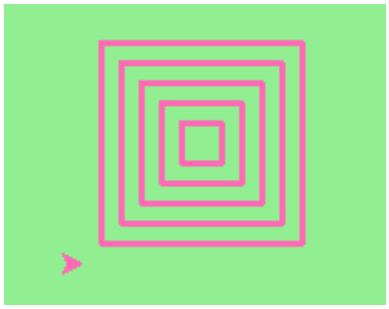
1. Write a void (non-fruitful) function to draw a square. Use it in a program to draw the image shown below. Assume each side is 20 units. (*Hint: notice that the turtle has already moved away from the ending point of the last square when the program ends.*)



Five Squares

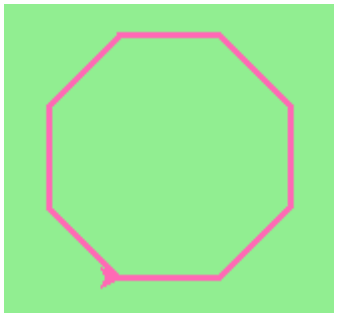
[7] http://en.wikipedia.org/wiki/Runtime_stack

2. Write a program to draw this. Assume the innermost square is 20 units per side, and each successive square is 20 units bigger, per side, than the one inside it.



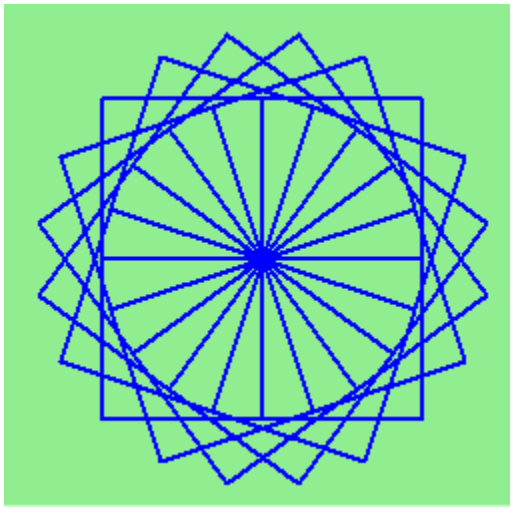
Nested Squares

3. Write a void function `draw_poly(t, n, sz)` which makes a turtle draw a regular polygon. When called with `draw_poly(tess, 8, 50)`, it will draw a shape like this:



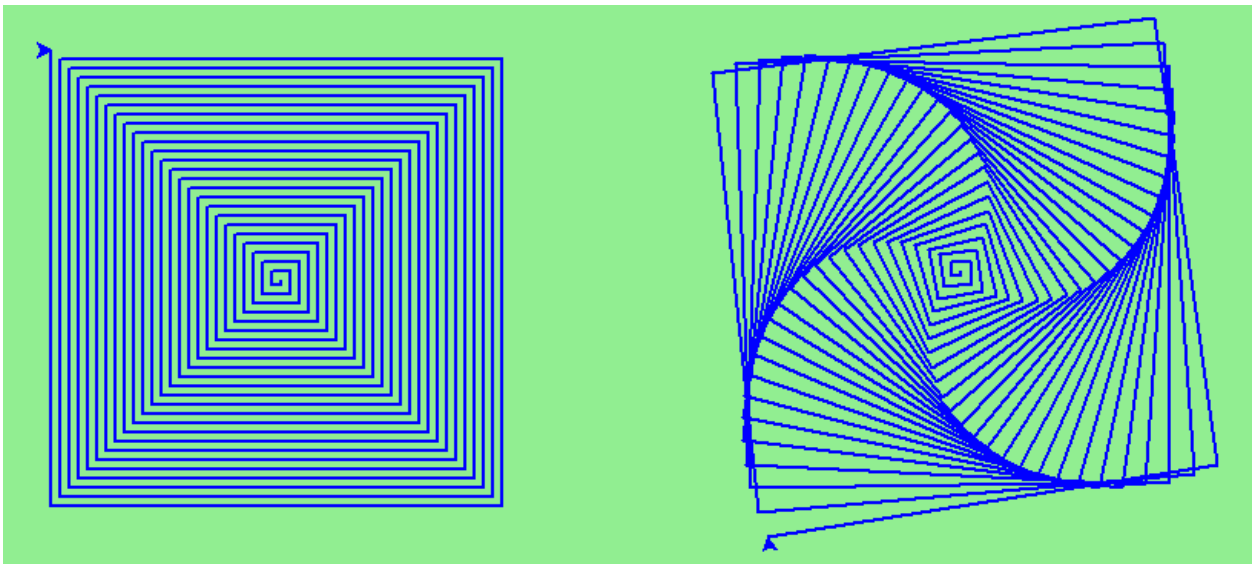
Regular polygon

4. Draw this pretty pattern.



Regular Polygon

5. The two spirals in this picture differ only by the turn angle. Draw both.

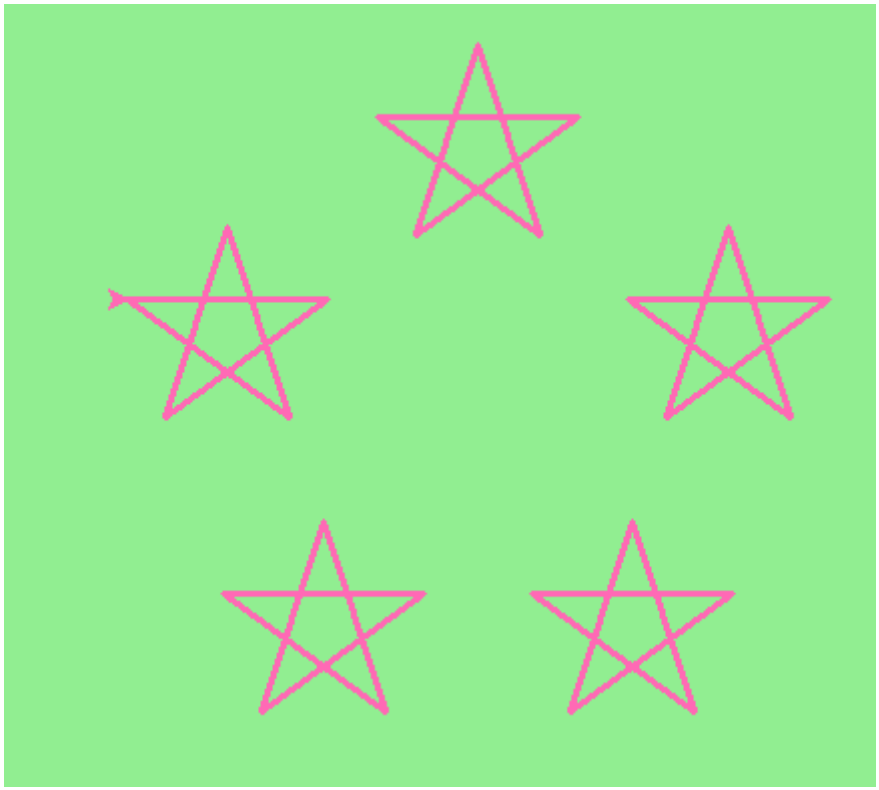


Spirals

6. Write a void function `draw_equitriangle(t, sz)` which calls `draw_poly` from the previous question to have its turtle draw a equilateral triangle.
7. Write a fruitful function `sum_to(n)` that returns the sum of all integer numbers up to and including `n`. So `sum_to(10)` would be `1+2+3...+10` which would return the value `55`.
8. Write a function `area_of_circle(r)` which returns the area of a circle of radius `r`.
9. Write a void function to draw a star, where the length of each side is 100 units. (*Hint: You should turn the turtle by 144 degrees at each point.*)

Star

10. Extend your program above. Draw five stars, but between each, pick up the pen, move forward by 350 units, turn right by 144, put the pen down, and draw the next star. You'll get something like this:



Five Stars

What would it look like if you didn't pick up the pen?

Chapter 5: Conditionals

Programs get really interesting when we can test conditions and change the program behaviour depending on the outcome of the tests. That's what this chapter is about.

5.1. Boolean values and expressions

A Boolean value is either true or false. It is named after the British mathematician, George Boole, who first formulated Boolean algebra — some rules for reasoning about and combining these values. This is the basis of all modern computer logic.

In Python, the two Boolean values are `True` and `False` (the capitalization must be exactly as shown), and the Python type is **`bool`**.

```
>>> type(True)
<class 'bool'>
>>> type(true)
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
NameError: name 'true' is not defined
```

A **Boolean expression** is an expression that evaluates to produce a result which is a Boolean value. For example, the operator == tests if two values are equal. It produces (or *yields*) a Boolean value:

```
>>> 5 == (3 + 2)   # Is 5 equal to the result of 3 + 2?
True
>>> 5 == 6
False
>>> j = "hel"
>>> j + "lo" == "hello"
True
```

In the first statement, the two operands evaluate to equal values, so the expression evaluates to `True`; in the second statement, `5` is not equal to `6`, so we get `False`.

The == operator is one of six common **comparison operators** which all produce a bool result; here are all six:

```
x == y               # Produce True if ... x is equal to y
x != y               # ... x is not equal to y
x > y                # ... x is greater than y
x < y                # ... x is less than y
x >= y               # ... x is greater than or equal to y
x <= y               # ... x is less than or equal to y
```

Although these operations are probably familiar, the Python symbols are different from the mathematical symbols. A common error is to use a single equal sign (=) instead of a double equal sign (==). Remember that= is an assignment operator and == is a comparison operator. Also, there is no such thing as =< or =>.

Like any other types we've seen so far, Boolean values can be assigned to variables, printed, etc.

```
>>> age = 18
>>> old_enough_to_get_driving_licence = age >= 17
>>> print(old_enough_to_get_driving_licence)
True
>>> type(old_enough_to_get_driving_licence)
<class 'bool'>
```

5.2. Logical operators

There are three **logical operators**, `and`, `or`, and `not`, that allow us to build more complex Boolean expressions from simpler Boolean expressions. The semantics (meaning) of these operators is similar to their meaning in English. For example, `x > 0 and x < 10` produces `True` only if `x` is greater than `0` and at the same time, `x` is less than `10`.

`n % 2 == 0 or n % 3 == 0` is `True` if *either* of the conditions is `True`, that is, if the number `n` is divisible by `2` *or* it is divisible by `3`. (What do you think happens if `n` is divisible by both `2` and by `3` at the same time? Will the expression yield `True` or `False`? Try it in your Python interpreter.)

Finally, the `not` operator negates a Boolean value, so `not (x > y)` is `True` if `(x > y)` is `False`, that is, if `x` is less than or equal to `y`.

The expression on the left of the or operator is evaluated first: if the result is `True`, Python does not (and need not) evaluate the expression on the right — this is called *short-circuit evaluation*. Similarly, for the `and` operator, if the expression on the left yields `False`, Python does not evaluate the expression on the right.

So there are no unnecessary evaluations.

5.3. Truth Tables

A truth table is a small table that allows us to list all the possible inputs, and to give the results for the logical operators. Because the `and` and `or` operators each have two operands, there are only four rows in a truth table that describes the semantics of `and`.

a	b	a and b
False	False	False
False	True	False
True	False	False
True	True	True

In a Truth Table, we sometimes use `T` and `F` as shorthand for the two Boolean values: here is the truth table describing or:

a	b	a or b
F	F	F
F	T	T
T	F	T
T	T	T

The third logical operator, `not`, only takes a single operand, so its truth table only has two rows:

a	not a
F	T
T	F

5.4. Simplifying Boolean Expressions

A set of rules for simplifying and rearranging expressions is called an algebra. For example, we are all familiar with school *algebra* rules, such as:

```
1  n * 0 == 0
```

Here we see a different algebra — the Boolean algebra — which provides rules for working with Boolean values.

First, the `and` operator:

```
x and False == False
False and x == False
y and x == x and y
x and True == x
True and x == x
x and x == x
```

Here are some corresponding rules for the `or` operator:

```
x or False == x
False or x == x
y or x == x or y
x or True == True
True or x == True
x or x == x
```

Two not operators cancel each other:

```
not (not x) == x
```

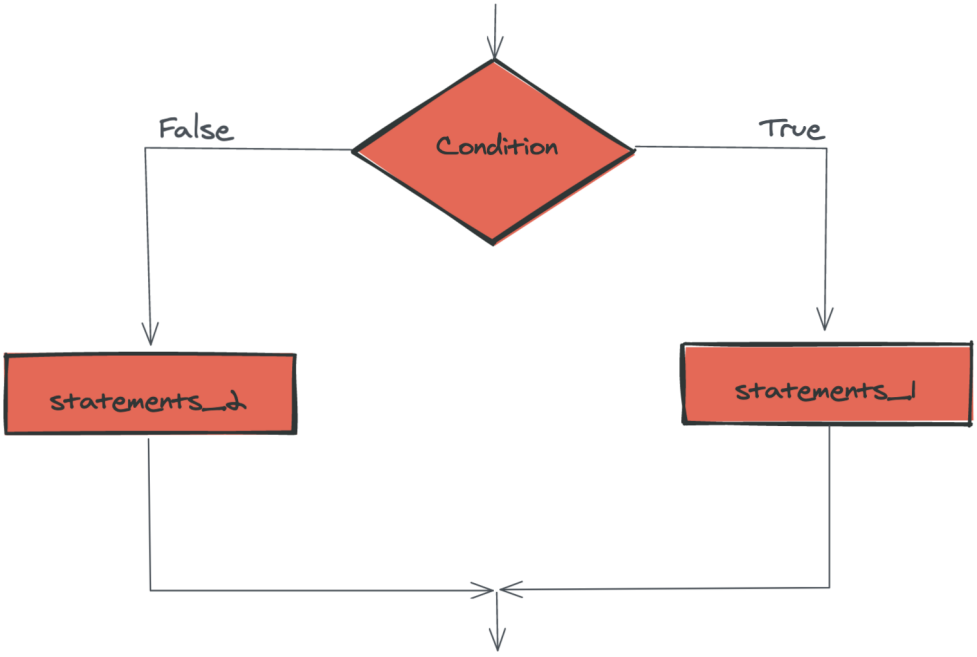
5.5. Conditional execution

In order to write useful programs, we almost always need the ability to check conditions and change the behavior of the program accordingly. **Conditional statements** give us this ability. The simplest form is the **`if`** statement:

```
if x % 2 == 0:
    print(x, " is even.")
    print("Did you know that 2 is the only even number that is prime?")
else:
    print(x, " is odd.")
    print("Did you know that multiplying two odd numbers " +
                                        "always gives an odd result?")
```

The Boolean expression after the `if` statement is called the **condition**. If it is true, then all the indented statements get executed. If not, then all the statements indented under the `else` clause get executed.

Flowchart of an `if` statement with an `else` clause



Flowchart - if else

The syntax for an if statement looks like this:

```
if BOOLEAN EXPRESSION:
    STATEMENTS_1        # Executed if condition evaluates to True
else:
    STATEMENTS_2        # Executed if condition evaluates to False
```

As with the function definition from the last chapter and other compound statements like `for`, the `if` statement consists of a header line and a body. The header line begins with the keyword `if` followed by a *Boolean expression* and ends with a colon (`:`).

The indented statements that follow are called a **block**. The first unindented statement marks the end of the block.

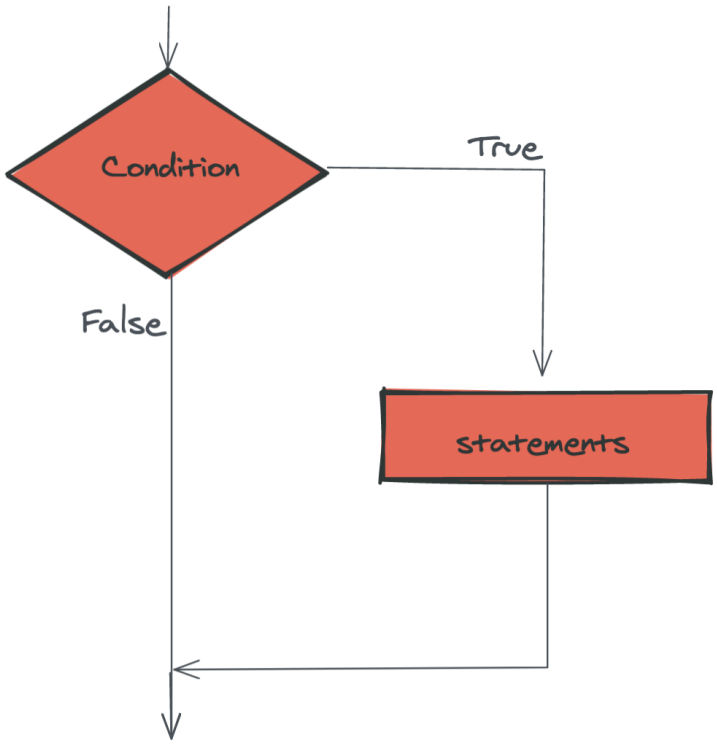
Each of the statements inside the first block of statements are executed in order if the Boolean expression evaluates to `True`. The entire first block of statements is skipped if the Boolean expression evaluates to `False`, and instead all the statements indented under the `else` clause are executed.

There is no limit on the number of statements that can appear under the two clauses of an `if` statement, but there has to be at least one statement in each block. Occasionally, it is useful to have a section with no statements (usually as a place keeper, or scaffolding, for code we haven't written yet). In that case, we can use the `pass` statement, which does nothing except act as a placeholder.

```
if True:          # This is always True,
    pass          #   so this is always executed, but it does nothing
else:
    pass
```

5.6. Omitting the else clause

Flowchart of an `if` statement with no `else` clause



Flowchart - if only

Another form of the `if` statement is one in which the `else` clause is omitted entirely. In this case, when the condition evaluates to `True`, the statements are executed, otherwise the flow of execution continues to the statement after the `if`.

```
if x < 0:
    print("The negative number ",  x, " is not valid here.")
    x = 42
    print("I've decided to use the number 42 instead.")

print("The square root of ", x, "is", math.sqrt(x))
```

In this case, the `print` function that outputs the square root is the one after the `if` — not because we left a blank line, but because of the way the code is indented. Note too that the function call `math.sqrt(x)` will give an error unless we have an `import math` statement, usually placed near the top of our script.

Python terminology

Python documentation sometimes uses the term **suite** of statements to mean what we have called a *block* here. They mean the same thing, and since most other languages and computer scientists use the word block, we'll stick with that.

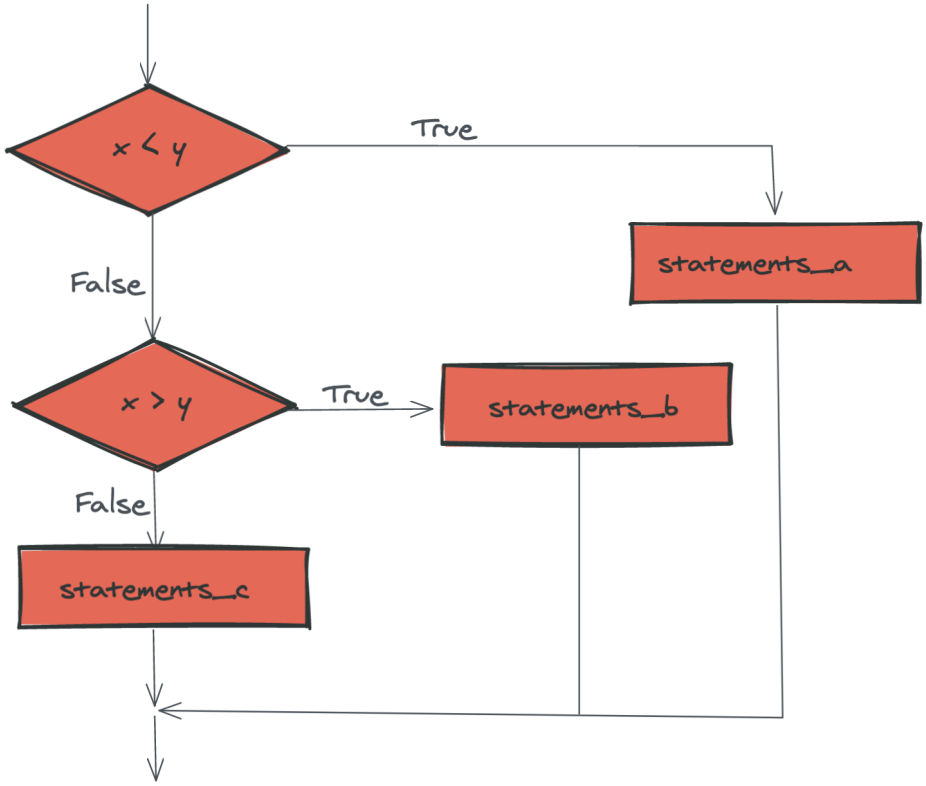
Notice too that `else` is not a statement. The `if` statement has two clauses, one of which is the (optional) `else` clause.

5.7. Chained conditionals

Sometimes there are more than two possibilities and we need more than two branches. One way to express a computation like that is a **chained conditional**:

```
if x < y:
    STATEMENTS_A
elif x > y:
    STATEMENTS_B
else:
    STATEMENTS_C
```

Flowchart of this chained conditional



Flowchart - chained conditional

`elif` is an abbreviation of else if. Again, exactly one branch will be executed. There is no limit of the number of `elif` statements but only a single (and optional) final `else` statement is allowed and it must be the last branch in the statement:

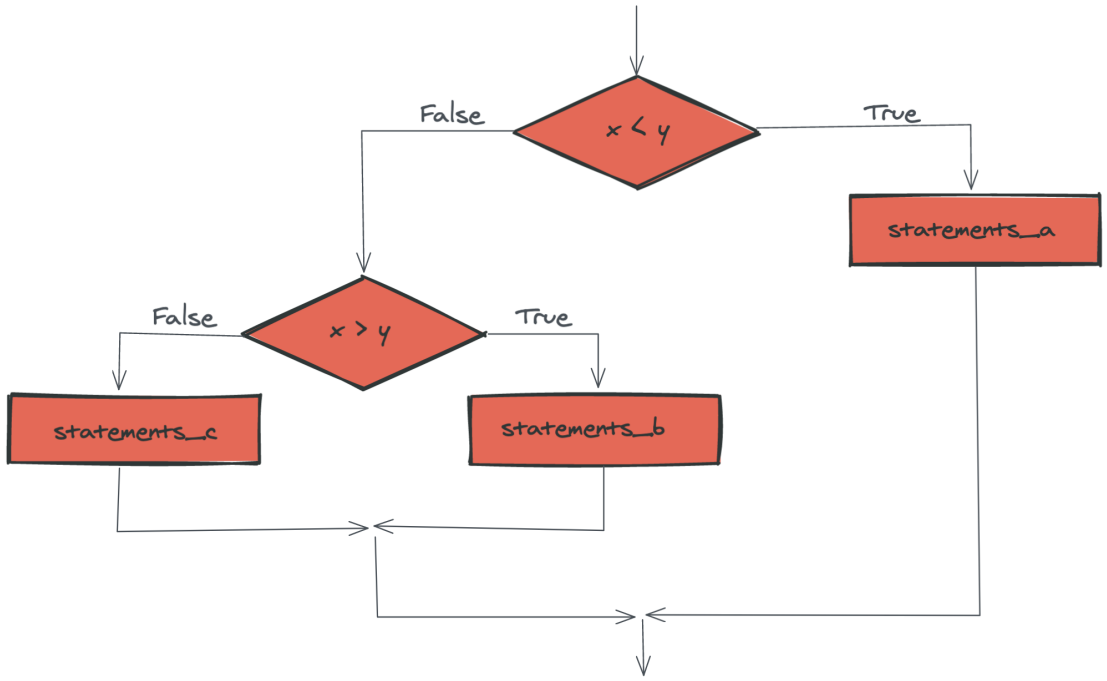
```
if choice == "a":
    function_one()
elif choice == "b":
    function_two()
elif choice == "c":
    function_three()
else:
    print("Invalid choice.")
```

Each condition is checked in order. If the first is false, the next is checked, and so on. If one of them is true, the corresponding branch executes, and the statement ends. Even if more than one condition is true, only the first true branch executes.

5.8. Nested conditionals

One conditional can also be **nested** within another. (It is the same theme of composability, again!) We could have written the previous example as follows:

Flowchart of this nested conditional



Flowchart - nested conditional

```
if x < y:
    STATEMENTS_A
else:
    if x > y:
        STATEMENTS_B
    else:
        STATEMENTS_C
```

The outer conditional contains two branches. The second branch contains another `if` statement, which has two branches of its own. Those two branches could contain conditional statements as well.

Although the indentation of the statements makes the structure apparent, nested conditionals very quickly become difficult to read. In general, it is a good idea to avoid them when we can.

Logical operators often provide a way to simplify nested conditional statements. For example, we can rewrite the following code using a single conditional:

```
if 0 < x:            # Assume x is an int here
    if x < 10:
        print("x is a positive single digit.")
```

The `print` function is called only if we make it past both the conditionals, so instead of the above which uses two `if` statements each with a simple condition, we could make a more complex condition using the `and` operator. Now we only need a single `if` statement:

```
if 0 < x and x < 10:
    print("x is a positive single digit.")
```

5.9. The return statement

The `return` statement, with or without a value, depending on whether the function is fruitful or void, allows us to terminate the execution of a function before (or when) we reach the end. One reason to use an *early return* is if we detect an error condition:

```
def print_square_root(x):
    if x <= 0:
        print("Positive numbers only, please.")
        return

    result = x**0.5
    print("The square root of", x, "is", result)
```

The function `print_square_root` has a parameter named `x`. The first thing it does is check whether `x` is less than or equal to `0`, in which case it displays an error message and then uses `return` to exit the function. The flow of execution immediately returns to the caller, and the remaining lines of the function are not executed.

5.10. Logical opposites

Each of the six relational operators has a logical opposite: for example, suppose we can get a driving licence when our age is greater or equal to 17, we can not get the driving licence when we are less than 17.

Notice that the opposite of `>=` is `<`.

operator	logical opposite
==	!=
!=	==
<	>=
<=	>
>	<=
>=	<

Understanding these logical opposites allows us to sometimes get rid of `not` operators. `not` operators are often quite difficult to read in computer code, and our intentions will usually be clearer if we can eliminate them.

For example, if we wrote this Python:

```
if not (age >= 17):
    print("Hey, you're too young to get a driving licence!")
```

it would probably be clearer to use the simplification laws, and to write instead:

```
if age < 17:
    print("Hey, you're too young to get a driving licence!")
```

Two powerful simplification laws (called de Morgan's laws) that are often helpful when dealing with complicated Boolean expressions are:

```
not (x and y)  ==  (not x) or (not y)
not (x or y)   ==  (not x) and (not y)
```

For example, suppose we can slay the dragon only if our magic lightsabre sword is charged to 90% or higher, and we have 100 or more energy units in our protective shield. We find this fragment of Python code in the game:

```
if not ((sword_charge >= 0.90) and (shield_energy >= 100)):
    print("Your attack has no effect, the dragon fries you to a crisp!")
else:
    print("The dragon crumples in a heap. You rescue the gorgeous princess!")
```

de Morgan's laws together with the logical opposites would let us rework the condition in a (perhaps) easier to understand way like this:

```
if (sword_charge < 0.90) or (shield_energy < 100):
    print("Your attack has no effect, the dragon fries you to a crisp!")
else:
    print("The dragon crumples in a heap. You rescue the gorgeous princess!")
```

We could also get rid of the `not` by swapping around the then and else parts of the conditional. So here is a third version, also equivalent:

```
if (sword_charge >= 0.90) and (shield_energy >= 100):
    print("The dragon crumples in a heap. You rescue the gorgeous princess!")
else:
    print("Your attack has no effect, the dragon fries you to a crisp!")
```

This version is probably the best of the three, because it very closely matches the initial English statement. Clarity of our code (for other humans), and making it easy to see that the code does what was expected should always be a high priority.

As our programming skills develop we'll find we have more than one way to solve any problem. So good programs are *designed*. We make choices that favour clarity, simplicity, and elegance. The job title *software architect* says a lot about what we do — we are *architects* who engineer our products to balance beauty, functionality, simplicity and clarity in our creations.

Tip

Once our program works, we should play around a bit trying to polish it up. Write good comments. Think about whether the code would be clearer with different variable names. Could we have done it more elegantly? Should we rather use a function? Can we simplify the conditionals?

We think of our code as our creation, our work of art! We make it great.

5.11. Type conversion

We've had a first look at this in an earlier chapter. Seeing it again won't hurt!

Many Python types come with a built-in function that attempts to convert values of another type into its own type. The `int` function, for example, takes any value and converts it to an integer, if possible, or complains otherwise:

```
>>> int("32")
32
>>> int("Hello")
ValueError: invalid literal for int() with base 10: 'Hello'
```

`int` can also convert floating-point values to integers, but remember that it truncates the fractional part:

```
>>> int(-2.3)
-2
>>> int(3.99999)
3
>>> int("42")
42
>>> int(1.0)
1
```

The float function converts integers and strings to floating-point numbers:

```
>>> float(32)
32.0
>>> float("3.14159")
3.14159
>>> float(1)
1.0
```

It may seem odd that Python distinguishes the integer value `1` from the floating-point value `1.0`. They may represent the same number, but they belong to different types. The reason is that they are represented differently inside the computer.

The `str` function converts any argument given to it to type string:

```
>>> str(32)
'32'
>>> str(3.14149)
'3.14149'
>>> str(True)
'True'
>>> str(true)
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
NameError: name 'true' is not defined
```

`str` will work with any value and convert it into a string. As mentioned earlier, `True` is a Boolean value; true is just an ordinary variable name, and is not defined here, so we get an error.

5.12. A Turtle Bar Chart

The turtle has a lot more power than we've seen so far. The full documentation can be found at http://docs.python.org/py3k/library/turtle.html.

Here are a couple of new tricks for our turtles:

- We can get a turtle to display text on the canvas at the turtle's current position. The method to do that is `alex.write("Hello")`.
- We can fill a shape (circle, semicircle, triangle, etc.) with a color. It is a two-step process. First we call the method `alex.begin_fill()`, then we draw the shape, then we call `alex.end_fill()`.
- We've previously set the color of our turtle — we can now also set its fill color, which need not be the same as the turtle and the pen color. We use `alex.color("blue","red")` to set the turtle to draw in blue, and fill in red.

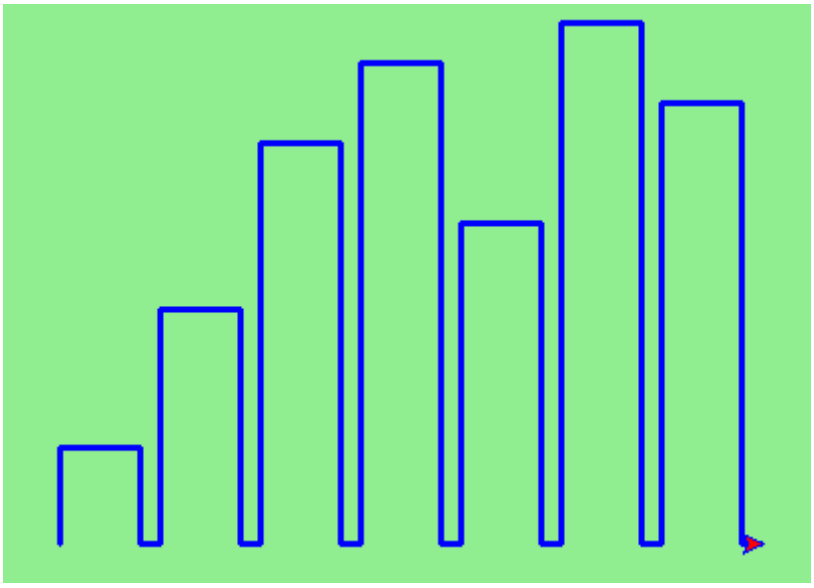
Ok, so can we get tess to draw a bar chart? Let us start with some data to be charted,

```
xs = [48, 117, 200, 240, 160, 260, 220]
```

Corresponding to each data measurement, we'll draw a simple rectangle of that height, with a fixed width.

```
def draw_bar(t, height):
    """ Get turtle t to draw one bar, of height. """
    t.left(90)
    t.forward(height)     # Draw up the left side
    t.right(90)
    t.forward(40)         # Width of bar, along the top
    t.right(90)
    t.forward(height)     # And down again!
    t.left(90)            # Put the turtle facing the way we found it.
    t.forward(10)         # Leave small gap after each bar

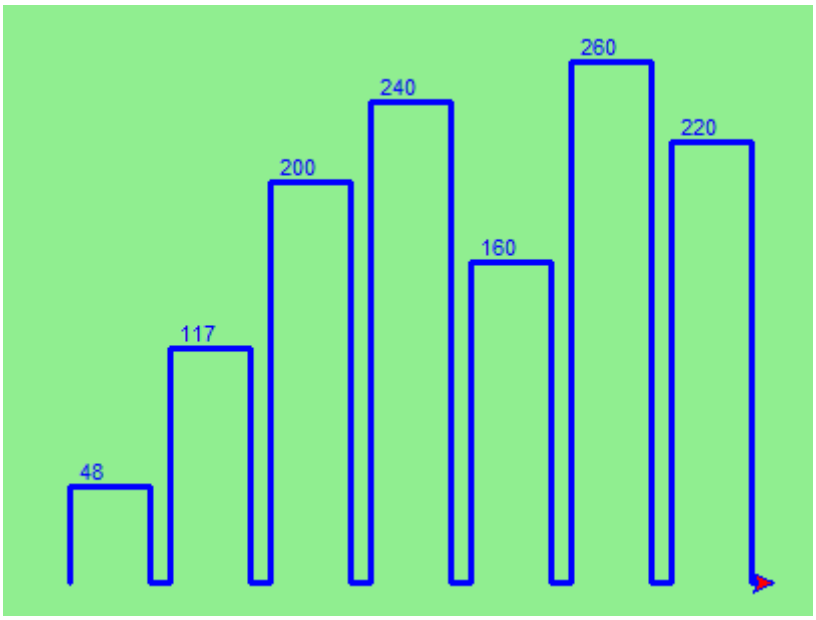
...
for v in xs:              # Assume xs and tess are ready
    draw_bar(tess, v)
```



Simple bar chart

Ok, not fantastically impressive, but it is a nice start! The important thing here was the mental chunking, or how we broke the problem into smaller pieces. Our chunk is to draw one bar, and we wrote a function to do that. Then, for the whole chart, we repeatedly called our function.

Next, at the top of each bar, we'll print the value of the data. We'll do this in the body of `draw_bar`, by adding `t.write(' ' + str(height))` as the new third line of the body. We've put a little space in front of the number, and turned the number into a string. Without this extra space we tend to cramp our text awkwardly against the bar to the left. The result looks a lot better now:



Numbered bar chart

And now we'll add two lines to fill each bar. Our final program now looks like this:

```
import turtle

def draw_bar(t, height):
    """ Get turtle t to draw one bar, of height. """
    t.begin_fill()           # Added this line
    t.left(90)
    t.forward(height)
    t.write("  "+ str(height))
    t.right(90)
    t.forward(40)
    t.right(90)
    t.forward(height)
    t.left(90)
    t.end_fill()             # Added this line
    t.forward(10)

wn = turtle.Screen()         # Set up the window and its attributes
wn.bgcolor("lightgreen")

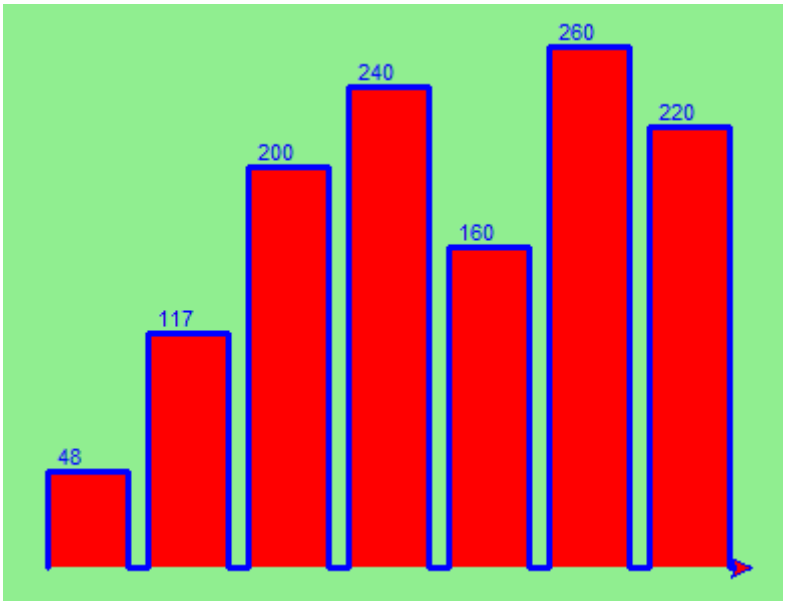
tess = turtle.Turtle()       # Create tess and set some attributes
tess.color("blue", "red")
tess.pensize(3)

xs = [48,117,200,240,160,260,220]

for a in xs:
    draw_bar(tess, a)

wn.mainloop()
```

It produces the following, which is more satisfying:



Filled bar chart

Mmm. Perhaps the bars should not be joined to each other at the bottom. We'll need to pick up the pen while making the gap between the bars. We'll leave that (and a few more tweaks) as exercises for you!

5.13. Glossary

block

A group of consecutive statements with the same indentation.

body

The block of statements in a compound statement that follows the header.

Boolean algebra

Some rules for rearranging and reasoning about Boolean expressions.

Boolean expression

An expression that is either true or false.

Boolean value

There are exactly two Boolean values: `True` and `False`. Boolean values result when a Boolean expression is evaluated by the Python interpreter. They have type `bool`.

branch

One of the possible paths of the flow of execution determined by conditional execution.

chained conditional

A conditional branch with more than two possible flows of execution. In Python chained conditionals are written with `if` ... `elif` ... `else` statements.

comparison operator

One of the six operators that compares two values: `==`, `!=`, `>`, `<`, `>=`, and `<=`.

condition

The Boolean expression in a conditional statement that determines which branch is executed.

conditional statement

A statement that controls the flow of execution depending on some condition. In Python the keywords `if`, `elif`, and `else` are used for conditional statements.

logical operator

One of the operators that combines Boolean expressions: `and`, `or`, and `not`.

nesting

One program structure within another, such as a conditional statement inside a branch of another conditional statement.

prompt

A visual cue that tells the user that the system is ready to accept input data.

truth table

A concise table of Boolean values that can describe the semantics of an operator.

type conversion

An explicit function call that takes a value of one type and computes a corresponding value of another type.

wrapping code in a function

The process of adding a function header and parameters to a sequence of program statements is often referred to as "wrapping the code in a function". This process is very useful whenever the program statements in question are going to be used multiple times. It is even more useful when it allows the programmer to express their mental chunking, and how they've broken a complex problem into pieces.

5.14. Exercises

1. Assume the days of the week are numbered `0,1,2,3,4,5,6` from Sunday to Saturday. Write a function which is given the day number, and it returns the day name (a string).

2. You go on a wonderful holiday (perhaps to jail, if you don't like happy exercises) leaving on day number 3 (a Wednesday). You return home after 137 sleeps. Write a general version of the program which asks for the starting day number, and the length of your stay, and it will tell you the name of day of the week you will return on.
3. Give the logical opposites of these conditions

```
a > b
a >= b
a >= 18  and  day == 3
a >= 18  and  day != 3
```

4. What do these expressions evaluate to?

```
3 == 3
3 != 3
3 >= 4
not (3 < 4)
```

5. Complete this truth table:

p	q	r	(not (p and q)) or r
F	F	F	?
F	F	T	?
F	T	F	?
F	T	T	?
T	F	F	?
T	F	T	?
T	T	F	?
T	T	T	?

6. Write a function which is given an exam mark, and it returns a string — the grade for that mark — according to this scheme:

Mark	Grade
>= 75	First
[70-75)	Upper Second
[60-70)	Second
[50-60)	Third
[45-50)	F1 Supp
[40-45)	F2
< 40	F3

The square and round brackets denote closed and open intervals. A closed interval includes the number, and open interval excludes it. So `39.99999` gets grade `F3`, but `40` gets grade `F2`. Assume

```
xs = [83, 75, 74.9, 70, 69.9, 65, 60, 59.9, 55, 50,
                    49.9, 45, 44.9, 40, 39.9, 2, 0]
```

Test your function by printing the mark and the grade for all the elements in this list.

7. Modify the turtle bar chart program so that the pen is up for the small gaps between each bar.
8. Modify the turtle bar chart program so that the bar for any value of 200 or more is filled with red, values between [100 and 200) are filled with yellow, and bars representing values less than 100 are filled with green.
9. In the turtle bar chart program, what do you expect to happen if one or more of the data values in the list is negative? Try it out. Change the program so that when it prints the text value for the negative bars, it puts the text below the bottom of the bar.
10. Write a function `find_hypot` which, given the length of two sides of a right-angled triangle, returns the length of the hypotenuse. (*Hint: `x ** 0.5` will return the square root.*)
11. Write a function `is_rightangled` which, given the length of three sides of a triangle, will determine whether the triangle is right-angled. Assume that the third argument to the function is always the longest side. It will return `True` if the triangle is right-angled, or `False` otherwise.

    Hint: Floating point arithmetic is not always exactly accurate, so it is not safe to test floating point numbers for equality. If a good programmer wants to know whether `x` is equal or close enough to `y`, they would probably code it up as:

```
if  abs(x-y) < 0.000001:    # If x is approximately equal to y
    ...
```

12. Extend the above program so that the sides can be given to the function in any order.
13. If you're intrigued by why floating point arithmetic is sometimes inaccurate, on a piece of paper, divide 10 by 3 and write down the decimal result. You'll find it does not terminate, so you'll need an infinitely long sheet of paper. The representation of numbers in computer memory or on your calculator has similar problems: memory is finite, and some digits may have to be discarded. So small inaccuracies creep in. Try this script:

```
import math
a = math.sqrt(2.0)
print(a, a*a)
print(a*a == 2.0)
```

Chapter 6: Fruitful functions

6.1. Return values

The built-in functions we have used, such as `abs`, `pow`, `int`, `max`, and `range`, have produced results. Calling each of these functions generates a value, which we usually assign to a variable or use as part of an expression.

```
biggest = max(3, 7, 2, 5)
x = abs(3 - 11) + 10
```

We also wrote our own function to return the final amount for a compound interest calculation.

In this chapter, we are going to write more functions that return values, which we will call *fruitful functions*, for want of a better name. The first example is area, which returns the area of a circle with the given radius:

```
def area(radius):
    b = 3.14159 * radius**2
    return b
```

We have seen the return statement before, but in a fruitful function the `return` statement includes a **return value**. This statement means: evaluate the return expression, and then return it immediately as the result (the fruit) of this function. The expression provided can be arbitrarily complicated, so we could have written this function like this:

```
def area(radius):
    return 3.14159 * radius * radius
```

On the other hand, **temporary variables** like `b` above often make debugging easier.

Sometimes it is useful to have multiple `return` statements, one in each branch of a conditional. We have already seen the built-in `abs`, now we see how to write our own:

```
def absolute_value(x):
    if x < 0:
        return -x
    else:
        return x
```

Another way to write the above function is to leave out the `else` and just follow the `if` condition by the second `return` statement.

```
def absolute_value(x):
    if x < 0:
        return -x
    return x
```

Think about this version and convince yourself it works the same as the first one.

Code that appears after a return statement, or any other place the flow of execution can never reach, is called **dead code**, or **unreachable code**.

In a fruitful function, it is a good idea to ensure that every possible path through the program hits a `return` statement. The following version of `absolute_value` fails to do this:

```
def bad_absolute_value(x):
    if x < 0:
        return -x
    elif x > 0:
        return x
```

This version is not correct because if `x` happens to be `0`, neither condition is true, and the function ends without hitting a `return` statement. In this case, the `return` value is a special value called **`None`**:

```
>>> print(bad_absolute_value(0))
None
```

All Python functions return `None` whenever they do not return another value.

It is also possible to use a `return` statement in the middle of a `for` loop, in which case control immediately returns from the function. Let us assume that we want a function which looks through a list of words. It should return the first 2-letter word. If there is not one, it should return the empty string:

```
def find_first_2_letter_word(xs):
    for wd in xs:
        if len(wd) == 2:
           return wd
    return ""
```

```
>>> find_first_2_letter_word(["This",  "is", "a", "dead", "parrot"])
'is'
>>> find_first_2_letter_word(["I",  "like",  "cheese"])
''
```

Single-step through this code and convince yourself that in the first test case that we've provided, the function returns while processing the second element in the list: it does not have to traverse the whole list.

6.2. Program development

At this point, you should be able to look at complete functions and tell what they do. Also, if you have been doing the exercises, you have written some small functions. As you write larger functions, you might start to have more difficulty, especially with runtime and semantic errors.

To deal with increasingly complex programs, we are going to suggest a technique called **incremental development**. The goal of incremental development is to avoid long debugging sessions by adding and testing only a small amount of code at a time.

As an example, suppose we want to find the distance between two points, given by the coordinates `(x1, y1)` and `(x2, y2)`. By the Pythagorean theorem, the distance is:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Distance formula

The first step is to consider what a `distance` function should look like in Python. In other words, what are the inputs (parameters) and what is the output (return value)?

In this case, the two points are the inputs, which we can represent using four parameters. The return value is the distance, which is a floating-point value.

Already we can write an outline of the function that captures our thinking so far:

```
def distance(x1, y1, x2, y2):
    return 0.0
```

Obviously, this version of the function doesn't compute distances; it always returns zero. But it is syntactically correct, and it will run, which means that we can test it before we make it more complicated.

To test the new function, we call it with sample values:

```
>>> distance(1, 2, 4, 6)
0.0
```

We chose these values so that the horizontal distance equals `3` and the vertical distance equals `4`; that way, the result is `5` (the hypotenuse of a `3`-`4`-`5` triangle). When testing a function, it is useful to know the right answer.

At this point we have confirmed that the function is syntactically correct, and we can start adding lines of code. After each incremental change, we test the function again. If an error occurs at any point, we know where it must be — in the last line we added.

A logical first step in the computation is to find the differences `x2- x1` and `y2- y1`. We will refer to those values using temporary variables named `dx` and `dy`.

```
def distance(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    return 0.0
```

If we call the function with the arguments shown above, when the flow of execution gets to the return statement, `dx` should be `3` and `dy` should be `4`. We can check that this is the case in **PyScripter** by putting the cursor on the return statement, and running the program to break execution when it gets to the cursor (using the `F4` key). Then we inspect the variables `dx` and `dy` by hovering the mouse above them, to confirm that the function is getting the right parameters and performing the first computation correctly. If not, there are only a few lines to check.

Next we compute the sum of squares of `dx` and `dy`:

```
def distance(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    dsquared = dx*dx + dy*dy
    return 0.0
```

Again, we could run the program at this stage and check the value of `dsquared` (which should be 25).

Finally, using the fractional exponent `0.5` to find the square root, we compute and return the result:

```
def distance(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    dsquared = dx*dx + dy*dy
    result = dsquared**0.5
    return result
```

If that works correctly, you are done. Otherwise, you might want to inspect the value of `result` before the return statement.

When you start out, you might add only a line or two of code at a time. As you gain more experience, you might find yourself writing and debugging bigger conceptual chunks. Either way, stepping

through your code one line at a time and verifying that each step matches your expectations can save you a lot of debugging time. As you improve your programming skills you should find yourself managing bigger and bigger chunks: this is very similar to the way we learned to read letters, syllables, words, phrases, sentences, paragraphs, etc., or the way we learn to chunk music — from individual notes to chords, bars, phrases, and so on.

The key aspects of the process are:

1. Start with a working skeleton program and make small incremental changes. At any point, if there is an error, you will know exactly where it is.
2. Use temporary variables to refer to intermediate values so that you can easily inspect and check them.
3. Once the program is working, relax, sit back, and play around with your options. (There is interesting research that links "playfulness" to better understanding, better learning, more enjoyment, and a more positive mindset about what you can achieve — so spend some time fiddling around!) You might want to consolidate multiple statements into one bigger compound expression, or rename the variables you've used, or see if you can make the function shorter. A good guideline is to aim for making code as easy as possible for others to read.

Here is another version of the function. It makes use of a square root function that is in the `math` module (we'll learn about modules shortly). Which do you prefer? Which looks "closer" to the Pythagorean formula we started out with?

```
import math

def distance(x1, y1, x2, y2):
    return math.sqrt( (x2-x1)**2 + (y2-y1)**2 )
```

```
>>> distance(1, 2, 4, 6)
5.0
```

6.3. Debugging with print

Another powerful technique for debugging (an alternative to single-stepping and inspection of program variables), is to insert extra print functions in carefully selected places in your code. Then, by inspecting the output of the program, you can check whether the algorithm is doing what you expect it to. Be clear about the following, however:

- You must have a clear solution to the problem, and must know what should happen before you can debug a program. Work on *solving* the problem on a piece of paper (perhaps using a flowchart to record the steps you take) *before* you concern yourself with writing code. Writing

a program doesn't solve the problem — it simply *automates* the manual steps you would take. So first make sure you have a pen-and-paper manual solution that works. Programming then is about making those manual steps happen automatically.
- Do not write **chatterbox** functions. A chatterbox is a fruitful function that, in addition to its primary task, also asks the user for input, or prints output, when it would be more useful if it simply shut up and did its work quietly.

For example, we've seen built-in functions like `range`, `max` and `abs`. None of these would be useful building blocks for other programs if they prompted the user for input, or printed their results while they performed their tasks.

So a good tip is to avoid calling `print` and `input` functions inside fruitful functions, *unless the primary purpose of your function is to perform input and output.* The one exception to this rule might be to temporarily sprinkle some calls to `print` into your code to help debug and understand what is happening when the code runs, but these will then be removed once you get things working.

6.4. Composition

As you should expect by now, you can call one function from within another. This ability is called **composition**.

As an example, we'll write a function that takes two points, the center of the circle and a point on the perimeter, and computes the area of the circle.

Assume that the center point is stored in the variables `xc` and `yc`, and the perimeter point is in `xp` and `yp`. The first step is to find the radius of the circle, which is the distance between the two points. Fortunately, we've just written a function, `distance`, that does just that, so now all we have to do is use it:

```
radius = distance(xc, yc, xp, yp)
```

The second step is to find the area of a circle with that radius and return it. Again we will use one of our earlier functions:

```
result = area(radius)
return result
```

Wrapping that up in a function, we get:

```
def area2(xc, yc, xp, yp):
    radius = distance(xc, yc, xp, yp)
    result = area(radius)
    return result
```

We called this function `area2` to distinguish it from the `area` function defined earlier.

The temporary variables `radius` and `result` are useful for development, debugging, and single-stepping through the code to inspect what is happening, but once the program is working, we can make it more concise by composing the function calls:

```
def area2(xc, yc, xp, yp):
    return area(distance(xc, yc, xp, yp))
```

6.5. Boolean functions

Functions can return Boolean values, which is often convenient for hiding complicated tests inside functions. For example:

```
def is_divisible(x, y):
    """ Test if x is exactly divisible by y """
    if x % y == 0:
        return True
    else:
        return False
```

It is common to give **Boolean functions** names that sound like yes/no questions. `is_divisible` returns either `True` or `False` to indicate whether the `x` is or is not divisible by `y`.

We can make the function more concise by taking advantage of the fact that the condition of the `if` statement is itself a Boolean expression. We can return it directly, avoiding the `if` statement altogether:

```
def is_divisible(x, y):
    return x % y == 0
```

This session shows the new function in action:

```
>>> is_divisible(6, 4)
False
>>> is_divisible(6, 3)
True
```

Boolean functions are often used in conditional statements:

```
if is_divisible(x, y):
    ... # Do something ...
else:
    ... # Do something else ...
```

It might be tempting to write something like:

```
if is_divisible(x, y) == True:
```

but the extra comparison is unnecessary.

6.6. Programming with style

Readability is very important to programmers, since in practice programs are read and modified far more often then they are written. But, like most rules, we occasionally break them. Most of the code examples in this book will be consistent with the *Python Enhancement Proposal 8* (PEP 8[8]), a style guide developed by the Python community.

We'll have more to say about style as our programs become more complex, but a few pointers will be helpful already:

- use 4 spaces (instead of tabs) for indentation
- limit line length to 78 characters
- when naming identifiers, use CamelCase for classes (we'll get to those) and lowercase_with_-underscores for functons and variables
- place imports at the top of the file
- keep function definitions together
- use docstrings to document functions
- use two blank lines to separate function definitions from each other
- keep top level statements, including function calls, together at the bottom of the program

[8] http://www.python.org/dev/peps/pep-0008/

6.7. Unit testing

It is a common best practice in software development to include automatic **unit testing** of source code. Unit testing provides a way to automatically verify that individual pieces of code, such as functions, are working properly. This makes it possible to change the implementation of a function at a later time and quickly test that it still does what it was intended to do.

Some years back organizations had the view that their valuable asset was the program code and documentation. Organizations will now spend a large portion of their software budgets on crafting (and preserving) their tests.

Unit testing also forces the programmer to think about the different cases that the function needs to handle. You also only have to type the tests once into the script, rather than having to keep entering the same test data over and over as you develop your code.

Extra code in your program which is there because it makes debugging or testing easier is called **scaffolding**.

A collection of tests for some code is called its **test suite**.

There are a few different ways to do unit testing in Python — but at this stage we're going to ignore what the Python community usually does, and we're going to start with two functions that we'll write ourselves. We'll use these for writing our unit tests.

Let's start with the `absolute_value` function that we wrote earlier in this chapter. Recall that we wrote a few different versions, the last of which was incorrect, and had a bug. Would tests have caught this bug?

First we plan our tests. We'd like to know if the function returns the correct value when its argument is negative, or when its argument is positive, or when its argument is zero. When planning your tests, you'll always want to think carefully about the "edge" cases — here, an argument of `0` to `absolute_value` is on the edge of where the function behaviour changes, and as we saw at the beginning of the chapter, it is an easy spot for the programmer to make a mistake! So it is a good case to include in our test suite.

We're going to write a helper function for checking the results of one test. It takes a boolean argument and will either print a message telling us that the test passed, or it will print a message to inform us that the test failed. The first line of the body (after the function's docstring) magically determines the line number in the script where the call was made from. This allows us to print the line number of the test, which will help when we want to identify which tests have passed or failed.

```
import sys

def test(did_pass):
    """  Print the result of a test.  """
    linenum = sys._getframe(1).f_lineno   # Get the caller's line number.
    if did_pass:
        msg = "Test at line {0} ok.".format(linenum)
    else:
        msg = "Test at line {0} FAILED.".format(linenum)
    print(msg)
```

There is also some slightly tricky string formatting using the format method which we will gloss over for the moment, and cover in detail in a future chapter. But with this function written, we can proceed to construct our test suite:

```
def test_suite():
    """ Run the suite of tests for code in this module (this file).
    """
    test(absolute_value(17) == 17)
    test(absolute_value(-17) == 17)
    test(absolute_value(0) == 0)
    test(absolute_value(3.14) == 3.14)
    test(absolute_value(-3.14) == 3.14)

test_suite()        # Here is the call to run the tests
```

Here you'll see that we've constructed five tests in our test suite. We could run this against the first or second versions (the correct versions) of `absolute_value`, and we'd get output similar to the following:

```
Test at line 25 ok.
Test at line 26 ok.
Test at line 27 ok.
Test at line 28 ok.
Test at line 29 ok.
```

But let's say you change the function to an incorrect version like this:

```
def absolute_value(n):   # Buggy version
    """ Compute the absolute value of n """
    if n < 0:
        return 1
    elif n > 0:
        return n
```

Can you find at least two mistakes in this code? Our test suite can! We get:

```
Test at line 25 ok.
Test at line 26 FAILED.
Test at line 27 FAILED.
Test at line 28 ok.
Test at line 29 FAILED.
```

These are three examples of *failing tests*.

There is a built-in Python statement called **assert** that does almost the same as our **`test`** function (except the program stops when the first assertion fails). You may want to read about it, and use it instead of our `test` function.

6.8. Glossary

Boolean function

A function that returns a Boolean value. The only possible values of the bool type are `False` and `True`.

chatterbox function

A function which interacts with the user (using `input` or `print`) when it should not. Silent functions that just convert their input arguments into their output results are usually the most useful ones.

composition (of functions)

Calling one function from within the body of another, or using the `return` value of one function as an argument to the call of another.

dead code

Part of a program that can never be executed, often because it appears after a `return` statement.

fruitful function

A function that yields a `return` value instead of `None`.

incremental development

A program development plan intended to simplify debugging by adding and testing only a small amount of code at a time.

None

A special Python value. One use in Python is that it is returned by functions that do not execute a `return` statement with a `return` argument.

return value

The value provided as the result of a function call.

scaffolding

Code that is used during program development to assist with development and debugging. The unit test code that we added in this chapter are examples of scaffolding.

temporary variable

A variable used to store an intermediate value in a complex calculation.

test suite

A collection of tests for some code you have written.

unit testing

An automatic procedure used to validate that individual units of code are working properly. Having a test suite is extremely useful when somebody modifies or extends the code: it provides a safety net against going backwards by putting new bugs into previously working code. The term *regression* testing is often used to capture this idea that we don't want to go backwards!

6.9. Exercises

All of the exercises below should be added to a single file. In that file, you should also add the `test` and `test_suite` scaffolding functions shown above, and then, as you work through the exercises, add the new tests to your test suite. (If you open the online version of the textbook, you can easily copy and paste the tests and the fragments of code into your Python editor.)

After completing each exercise, confirm that all the tests pass.

1. The four compass points can be abbreviated by single-letter strings as "`N`", "`E`", "`S`", and "`W`". Write a function `turn_clockwise` that takes one of these four compass points as its parameter, and returns the next compass point in the clockwise direction. Here are some tests that should pass:

```
test(turn_clockwise("N") == "E")
test(turn_clockwise("W") == "N")
```

   You might ask "What if the argument to the function is some other value?" For all other cases, the function should return the value `None`:

```
test(turn_clockwise(42) == None)
test(turn_clockwise("rubbish") == None)
```

2. Write a function `day_name` that converts an integer number `0` to `6` into the name of a day. Assume day `0` is "`Sunday`". Once again, return `None` if the arguments to the function are not valid. Here are some tests that should pass:

```
test(day_name(3) == "Wednesday")
test(day_name(6) == "Saturday")
test(day_name(42) == None)
```

3. Write the inverse function `day_num` which is given a day name, and returns its number:

```
test(day_num("Friday") == 5)
test(day_num("Sunday") == 0)
test(day_num(day_name(3)) == 3)
test(day_name(day_num("Thursday")) == "Thursday")
```

   Once again, if this function is given an invalid argument, it should return `None`:

```
test(day_num("Halloween") == None)
```

4. Write a function that helps answer questions like '"Today is Wednesday. I leave on holiday in 19 days time. What day will that be?"' So the function must take a day name and a delta argument — the number of days to add — and should return the resulting day name:

```
test(day_add("Monday", 4) ==  "Friday")
test(day_add("Tuesday", 0) == "Tuesday")
test(day_add("Tuesday", 14) == "Tuesday")
test(day_add("Sunday", 100) == "Tuesday")
```

   Hint: use the first two functions written above to help you write this one.
5. Can your `day_add` function already work with negative deltas? For example, `-1` would be yesterday, or `-7` would be a week ago:

```
test(day_add("Sunday", -1) == "Saturday")
test(day_add("Sunday", -7) == "Sunday")
test(day_add("Tuesday", -100) == "Sunday")
```

   If your function already works, explain why. If it does not work, make it work.

   Hint: Play with some cases of using the modulus function `%` (introduced at the beginning of the previous chapter). Specifically, explore what happens to `x % 7` when `x` is negative.
6. Write a function `days_in_month` which takes the name of a month, and returns the number of days in the month. Ignore leap years:

```
test(days_in_month("February") == 28)
test(days_in_month("December") == 31)
```

If the function is given invalid arguments, it should return `None`.

7. Write a function `to_secs` that converts hours, minutes and seconds to a total number of seconds. Here are some tests that should pass:

```
test(to_secs(2, 30, 10) == 9010)
test(to_secs(2, 0, 0) == 7200)
test(to_secs(0, 2, 0) == 120)
test(to_secs(0, 0, 42) == 42)
test(to_secs(0, -10, 10) == -590)
```

8. Extend `to_secs` so that it can cope with real values as inputs. It should always return an integer number of seconds (truncated towards zero):

```
test(to_secs(2.5, 0, 10.71) == 9010)
test(to_secs(2.433,0,0) == 8758)
```

9. Write three functions that are the "inverses" of `to_secs`:

   1. `hours_in` returns the whole integer number of hours represented by a total number of seconds.

   2. `minutes_in` returns the whole integer number of left over minutes in a total number of seconds, once the hours have been taken out.

   3. `seconds_in` returns the left over seconds represented by a total number of seconds.

   You may assume that the total number of seconds passed to these functions is an integer. Here are some test cases:

```
test(hours_in(9010) == 2)
test(minutes_in(9010) == 30)
test(seconds_in(9010) == 10)
```

It won't always be obvious what is wanted ...

In the third case above, the requirement seems quite ambiguous and fuzzy. But the test clarifies what we actually need to do.

Unit tests often have this secondary benefit of clarifying the specifications. If you write your own test suites, consider it part of the problem-solving process as you ask questions about what you really expect to happen, and whether you've considered all the possible cases.

Given our emphasis on *thinking like a computer scientist*, you might enjoy reading at least one reference about thinking, and about fun ideas like *fluid intelligence*, a key ingredient in problem solving. See, for example, http://psychology.about.com/od/cognitivepsychology/a/fluid-crystal.htm. Learning Computer Science requires a good mix of both fluid and crystallized kinds of intelligence.

10. Which of these tests fail? Explain why.

```
test(3 % 4 == 0)
test(3 % 4 == 3)
test(3 / 4 == 0)
test(3 // 4 == 0)
test(3+4  *  2 == 14)
test(4-2+2 == 0)
test(len("hello, world!") == 13)
```

11. Write a compare function that returns `1` if `a > b`, `0` if `a == b`, and `-1` if `a < b`

```
test(compare(5, 4) == 1)
test(compare(7, 7) == 0)
test(compare(2, 3) == -1)
test(compare(42, 1) == 1)
```

12. Write a function called hypotenuse that returns the length of the hypotenuse of a right triangle given the lengths of the two legs as parameters:

```
test(hypotenuse(3, 4) == 5.0)
test(hypotenuse(12, 5) == 13.0)
test(hypotenuse(24, 7) == 25.0)
test(hypotenuse(9, 12) == 15.0)
```

13. Write a function `slope(x1, y1, x2, y2)` that returns the slope of the line through the points `(x1, y1)` and `(x2, y2)`. Be sure your implementation of slope can pass the following tests:

```
test(slope(5, 3, 4, 2) == 1.0)
test(slope(1, 2, 3, 2) == 0.0)
test(slope(1, 2, 3, 3) == 0.5)
test(slope(2, 4, 1, 2) == 2.0)
```

    Then use a call to `slope` in a new function named `intercept(x1, y1, x2, y2)` that returns the y-intercept of the line through the points `(x1, y1)` and `(x2, y2)`

```
test(intercept(1, 6, 3, 12) == 3.0)
test(intercept(6, 1, 1, 6) == 7.0)
test(intercept(4, 6, 12, 8) == 5.0)
```

14. Write a function called `is_even(n)` that takes an integer as an argument and returns `True` if the argument is an **even number** and `False` if it is **odd**.

    Add your own tests to the test suite.
15. Now write the function `is_odd(n)` that returns `True` when `n` is odd and `False` otherwise. Include unit tests for this function too.

    Finally, modify it so that it uses a call to `is_even` to determine if its argument is an odd integer, and ensure that its test still pass.
16. Write a function `is_factor(f, n)` that passes these tests:

```
test(is_factor(3, 12))
test(not is_factor(5, 12))
test(is_factor(7, 14))
test(not is_factor(7, 15))
test(is_factor(1, 15))
test(is_factor(15, 15))
test(not is_factor(25, 15))
```

An important role of unit tests is that they can also act as unambiguous "specifications" of what is expected. These test cases answer the question "Do we treat 1 and 15 as factors of 15"?

17. Write `is_multiple` to satisfy these unit tests:

```
test(is_multiple(12, 3))
test(is_multiple(12, 4))
test(not is_multiple(12, 5))
test(is_multiple(12, 6))
test(not is_multiple(12, 7))
```

Can you find a way to use `is_factor` in your definition of `is_multiple`?

18. Write the function `f2c(t)` designed to return the integer value of the nearest degree Celsius for given temperature in Fahrenheit. (*hint: you may want to make use of the built-in function, `round`. Try printing `round.__doc__` in a Python shell or looking up help for the round function, and experimenting with it until you are comfortable with how it works.*)

```
test(f2c(212) == 100)     # Boiling point of water
test(f2c(32) == 0)        # Freezing point of water
test(f2c(-40) == -40)     # Wow, what an interesting case!
test(f2c(36) == 2)
test(f2c(37) == 3)
test(f2c(38) == 3)
test(f2c(39) == 4)
```

19. Now do the opposite: write the function `c2f` which converts Celsius to Fahrenheit:

```
test(c2f(0) == 32)
test(c2f(100) == 212)
test(c2f(-40) == -40)
test(c2f(12) == 54)
test(c2f(18) == 64)
test(c2f(-48) == -54)
```

Chapter 7: Iteration

Computers are often used to automate repetitive tasks. Repeating identical or similar tasks without making errors is something that computers do well and people do poorly.

Repeated execution of a set of statements is called **iteration**. Because iteration is so common, Python provides several language features to make it easier. We've already seen the `for` statement in chapter 3. This the the form of iteration you'll likely be using most often. But in this chapter we've going to look at the `while` statement — another way to have your program do iteration, useful in slightly different circumstances.

Before we do that, let's just review a few ideas…

7.1. Assignment

As we have mentioned previously, it is legal to make more than one assignment to the same variable. A new assignment makes an existing variable refer to a new value (and stop referring to the old value).

```
airtime_remaining = 15
print(airtime_remaining)
airtime_remaining = 7
print(airtime_remaining)
```

The output of this program is:

```
15
7
```

because the first time `airtime_remaining` is printed, its value is `15`, and the second time, its value is `7`.

It is especially important to distinguish between an assignment statement and a Boolean expression that tests for equality. Because Python uses the equal token (=) for assignment, it is tempting to interpret a statement like `a = b` as a Boolean test. Unlike mathematics, it is not! Remember that the Python token for the equality operator is ==.

Note too that an equality test is symmetric, but assignment is not. For example, if `a == 7` then `7 == a`. But in Python, the statement `a = 7` is legal and `7 = a` is not.

In Python, an assignment statement can make two variables equal, but because further assignments can change either of them, they don't have to stay that way:

```
a = 5
b = a    # After executing this line, a and b are now equal
a = 3    # After executing this line, a and b are no longer equal
```

The third line changes the value of `a` but does not change the value of `b`, so they are no longer equal. (In some programming languages, a different symbol is used for assignment, such as `<-` or `:=`, to avoid confusion. Some people also think that variable was an unfortunate word to choose, and instead we should have called them assignables. Python chooses to follow common terminology and token usage, also found in languages like C, C++, Java, and C#, so we use the tokens `=` for assignment, `==` for equality, and we talk of variables.

7.2. Updating variables

When an assignment statement is executed, the right-hand side expression (i.e. the expression that comes after the assignment token) is evaluated first. This produces a value. Then the assignment is made, so that the variable on the left-hand side now refers to the new value.

One of the most common forms of assignment is an update, where the new value of the variable depends on its old value. Deduct 40 cents from my airtime balance, or add one run to the scoreboard.

```
n = 5
n = 3 * n + 1
```

Line 2 means *get the current value of `n`, multiply it by three and add one, and assign the answer to `n`, thus making `n` refer to the value.* So after executing the two lines above, `n` will point/refer to the integer `16`.

If you try to get the value of a variable that has never been assigned to, you'll get an error:

```
>>> w = x + 1
Traceback (most recent call last):
  File "<interactive input>", line 1, in
NameError: name 'x' is not defined
```

Before you can update a variable, you have to **initialize** it to some starting value, usually with a simple assignment:

```
runs_scored = 0
...
runs_scored = runs_scored + 1
```

Line 3 — updating a variable by adding `1` to it — is very common. It is called an **increment** of the variable; subtracting `1` is called a **decrement**. Sometimes programmers also talk about bumping a variable, which means the same as incrementing it by `1`.

7.3. The `for` loop revisited

Recall that the for loop processes each item in a list. Each item in turn is (re-)assigned to the loop variable, and the body of the loop is executed. We saw this example in an earlier chapter:

```
for f in ["Joe", "Zoe", "Brad", "Angelina", "Zuki", "Thandi", "Paris"]:
    invitation = "Hi " + f + ".  Please come to my party on Saturday!"
    print(invitation)
```

Running through all the items in a list is called **traversing** the list, or **traversal**.

Let us write a function now to sum up all the elements in a list of numbers. Do this by hand first, and try to isolate exactly what steps you take. You'll find you need to keep some "running total" of the sum so far, either on a piece of paper, in your head, or in your calculator. Remembering things from one step to the next is precisely why we have variables in a program: so we'll need some variable to remember the "running total". It should be initialized with a value of zero, and then we need to traverse the items in the list. For each item, we'll want to update the running total by adding the next number to it.

```
def mysum(xs):
    """ Sum all the numbers in the list xs, and return the total. """
    running_total = 0
    for x in xs:
        running_total = running_total + x
    return running_total

# Add tests like these to your test suite ...
test(mysum([1, 2, 3, 4]) == 10)
test(mysum([1.25, 2.5, 1.75]) == 5.5)
test(mysum([1, -2, 3]) == 2)
test(mysum([ ]) == 0)
test(mysum(range(11)) == 55)  # 11 is not included in the list.
```

7.4. The `while` statement

Here is a fragment of code that demonstrates the use of the while statement:

```
1  def sum_to(n):
2      """ Return the sum of 1+2+3 ... n """
3      ss  = 0
4      v = 1
5      while v <= n:
6          ss = ss + v
7          v = v + 1
8      return ss
9
10 # For your test suite
11 test(sum_to(4) == 10)
12 test(sum_to(1000) == 500500)
```

You can almost read the `while` statement as if it were English. It means, while `v` is less than or equal to `n`, continue executing the body of the loop. Within the body, each time, increment `v`. When `v` passes n, return your accumulated sum.

More formally, here is precise flow of execution for a `while` statement:

- Evaluate the condition at line 5, yielding a value which is either False or True.
- If the value is False, exit the while statement and continue execution at the next statement (line 8 in this case).
- If the value is True, execute each of the statements in the body (lines 6 and 7) and then go back to the `while` statement at line 5.

The body consists of all of the statements indented below the while keyword.

Notice that if the loop condition is False the first time we get loop, the statements in the body of the loop are never executed.

The body of the loop should change the value of one or more variables so that eventually the condition becomes false and the loop terminates. Otherwise the loop will repeat forever, which is called an **infinite loop**. An endless source of amusement for computer scientists is the observation that the directions on shampoo, "lather, rinse, repeat", are an infinite loop.

In the case here, we can prove that the loop terminates because we know that the value of `n` is finite, and we can see that the value of `v` increments each time through the loop, so eventually it will have to exceed `n`. In other cases, it is not so easy, even impossible in some cases, to tell if the loop will ever terminate.

What you will notice here is that the `while` loop is more work for you — the programmer — than the equivalent `for` loop. When using a `while` loop one has to manage the loop variable yourself: give it an initial value, test for completion, and then make sure you change something in the body so that the loop terminates. By comparison, here is an equivalent function that uses `for` instead:

```
def sum_to(n):
    """ Return the sum of 1+2+3 ... n """
    ss  = 0
    for v in range(n+1):
        ss = ss + v
    return ss
```

Notice the slightly tricky call to the `range` function — we had to add one onto `n`, because `range` generates its list up to but excluding the value you give it. It would be easy to make a programming mistake and overlook this, but because we've made the investment of writing some unit tests, our test suite would have caught our error.

So why have two kinds of loop if `for` looks easier? This next example shows a case where we need the extra power that we get from the `while` loop.

7.5. The Collatz 3n + 1 sequence

Let's look at a simple sequence that has fascinated and foxed mathematicians for many years. They still cannot answer even quite simple questions about this.

The "computational rule" for creating the sequence is to start from some given `n`, and to generate the next term of the sequence from `n`, either by halving `n`, (whenever `n` is even), or else by multiplying it by three and adding 1. The sequence terminates when `n` reaches `1`.

This Python function captures that algorithm:

```
1  def seq3np1(n):
2      """ Print the 3n+1 sequence from n,
3          terminating when it reaches 1.
4      """
5      while n != 1:
6          print(n, end=", ")
7          if n % 2 == 0:        # n is even
8              n = n // 2
9          else:                 # n is odd
10             n = n * 3 + 1
11     print(n, end=".\n")
```

Notice first that the `print` function on line 6 has an extra argument `end=", "`. This tells the `print` function to follow the printed string with whatever the programmer chooses (in this case, a comma followed by a space), instead of ending the line. So each time something is printed in the loop, it is printed on the same output line, with the numbers separated by commas. The call to `print(n, end=".\n")` at line 11 after the loop terminates will then print the final value of `n` followed by a period and a newline character. (You'll cover the `\n` (newline character) in the next chapter).

The condition for continuing with this loop is `n != 1`, so the loop will continue running until it reaches its termination condition, (i.e. `n == 1`).

Each time through the loop, the program outputs the value of `n` and then checks whether it is even or odd. If it is even, the value of `n` is divided by `2` using integer division. If it is odd, the value is replaced by `n * 3 + 1`. Here are some examples:

```
>>> seq3np1(3)
3, 10, 5, 16, 8, 4, 2, 1.
>>> seq3np1(19)
19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13,
                    40, 20, 10, 5, 16, 8, 4, 2, 1.
>>> seq3np1(21)
21, 64, 32, 16, 8, 4, 2, 1.
>>> seq3np1(16)
16, 8, 4, 2, 1.
>>>
```

Since `n` sometimes increases and sometimes decreases, there is no obvious proof that `n` will ever reach `1`, or that the program terminates. For some particular values of `n`, we can prove termination. For example, if the starting value is a power of two, then the value of `n` will be even each time through the loop until it reaches `1`. The previous example ends with such a sequence, starting with `16`.

See if you can find a small starting number that needs more than a hundred steps before it terminates.

Particular values aside, the interesting question was first posed by a German mathematician called Lothar Collatz: the *Collatz conjecture* (also known as the *3n + 1 conjecture*), is that this sequence terminates for all positive values of `n`. So far, no one has been able to prove it or disprove it! (A conjecture is a statement that might be true, but nobody knows for sure.)

Think carefully about what would be needed for a proof or disproof of the conjecture "All positive integers will eventually converge to `1` using the Collatz rules". With fast computers we have been able to test every integer up to very large values, and so far, they have all eventually ended up at `1`. But who knows? Perhaps there is some as-yet untested number which does not reduce to `1`.

You'll notice that if you don't stop when you reach `1`, the sequence gets into its own cyclic loop: `1`, `4`, `2`, `1`, `4`, `2`, `1`, `4` ... So one possibility is that there might be other cycles that we just haven't found yet.

Wikipedia has an informative article about the Collatz conjecture. The sequence also goes under other names (Hailstone sequence, Wonderous numbers, etc.), and you'll find out just how many integers have already been tested by computer, and found to converge!

Choosing between for and while

Use a `for` loop if you know, before you start looping, the maximum number of times that you'll need to execute the body. For example, if you're traversing a list of elements, you know that the

maximum number of loop iterations you can possibly need is "all the elements in the list". Or if you need to print the 12 times table, we know right away how many times the loop will need to run.

So any problem like "iterate this weather model for 1000 cycles", or "search this list of words", "find all prime numbers up to 10000" suggest that a `for` loop is best.

By contrast, if you are required to repeat some computation until some condition is met, and you cannot calculate in advance when (or if) this will happen, as we did in this `3n + 1` problem, you'll need a `while` loop.

We call the first case **definite iteration** — we know ahead of time some definite bounds for what is needed. The latter case is called **indefinite iteration** — we're not sure how many iterations we'll need — we cannot even establish an upper bound!

7.6. Tracing a program

To write effective computer programs, and to build a good conceptual model of program execution, a programmer needs to develop the ability to **trace** the execution of a computer program. Tracing involves becoming the computer and following the flow of execution through a sample program run, recording the state of all variables and any output the program generates after each instruction is executed.

To understand this process, let's trace the call to `seq3np1(3)` from the previous section. At the start of the trace, we have a variable, `n` (the parameter), with an initial value of `3`. Since `3` is not equal to `1`, the `while` loop body is executed. `3` is printed and `3 % 2 == 0` is evaluated. Since it evaluates to `False`, the `else` branch is executed and `3 * 3 + 1` is evaluated and assigned to `n`.

To keep track of all this as you hand trace a program, make a column heading on a piece of paper for each variable created as the program runs and another one for output. Our trace so far would look something like this:

```
n               output printed so far
--              ----------------------
3               3,
10
```

Since `10 != 1` evaluates to `True`, the loop body is again executed, and `10` is printed. `10 % 2 == 0` is `True`, so the `if` branch is executed and `n` becomes `5`. By the end of the trace we have:

```
n               output printed so far
--              ---------------------
3               3,
10              3, 10,
5               3, 10, 5,
16              3, 10, 5, 16,
8               3, 10, 5, 16, 8,
4               3, 10, 5, 16, 8, 4,
2               3, 10, 5, 16, 8, 4, 2,
1               3, 10, 5, 16, 8, 4, 2, 1.
```

Tracing can be a bit tedious and error prone (that's why we get computers to do this stuff in the first place!), but it is an essential skill for a programmer to have. From this trace we can learn a lot about the way our code works. We can observe that as soon as `n` becomes a power of `2`, for example, the program will require `log2(n)` executions of the loop body to complete. We can also see that the final `1` will not be printed as output within the body of the loop, which is why we put the special `print` function at the end.

Tracing a program is, of course, related to single-stepping through your code and being able to inspect the variables. Using the computer to **single-step** for you is less error prone and more convenient. Also, as your programs get more complex, they might execute many millions of steps before they get to the code that you're really interested in, so manual tracing becomes impossible. Being able to set a **breakpoint** where you need one is far more powerful. So we strongly encourage you to invest time in learning using to use your programming environment to full effect.

There are also some great visualization tools becoming available to help you trace and understand small fragments of Python code. The one we recommend is at http://pythontutor.com/

We've cautioned against chatterbox functions, but used them here. As we learn a bit more Python, we'll be able to show you how to generate a list of values to hold the sequence, rather than having the function print them. Doing this would remove the need to have all these pesky print functions in the middle of our logic, and will make the function more useful.

7.7. Counting digits

The following function counts the number of decimal digits in a positive integer:

```
def num_digits(n):
    count = 0
    while n != 0:
        count = count + 1
        n = n // 10
    return count
```

A call to `print(num_digits(710))` will print `3`. Trace the execution of this function call (perhaps using the single step function in PyScripter, or the Python visualizer, or on some paper) to convince yourself that it works.

This function demonstrates an important pattern of computation called a **counter**. The variable `count` is initialized to `0` and then incremented each time the loop body is executed. When the loop exits, `count` contains the result — the total number of times the loop body was executed, which is the same as the number of digits.

If we wanted to only count digits that are either `0` or `5`, adding a conditional before incrementing the counter will do the trick:

```
def num_zero_and_five_digits(n):
    count = 0
    while n > 0:
        digit = n % 10
        if digit == 0 or digit == 5:
            count = count + 1
        n = n // 10
    return count
```

Confirm that `test(num_zero_and_five_digits(1055030250) == 7)` passes.

Notice, however, that `test(num_digits(0) == 1)` fails. Explain why. Do you think this is a bug in the code, or a bug in the specifications, or our expectations, or the tests?

7.8. Abbreviated assignment

Incrementing a variable is so common that Python provides an abbreviated syntax for it:

```
>>> count = 0
>>> count += 1
>>> count
1
>>> count += 1
>>> count
2
```

`count += 1` is an abbreviation for `count = count + 1`. We pronounce the operator as "plus-equals". The increment value does not have to be `1`:

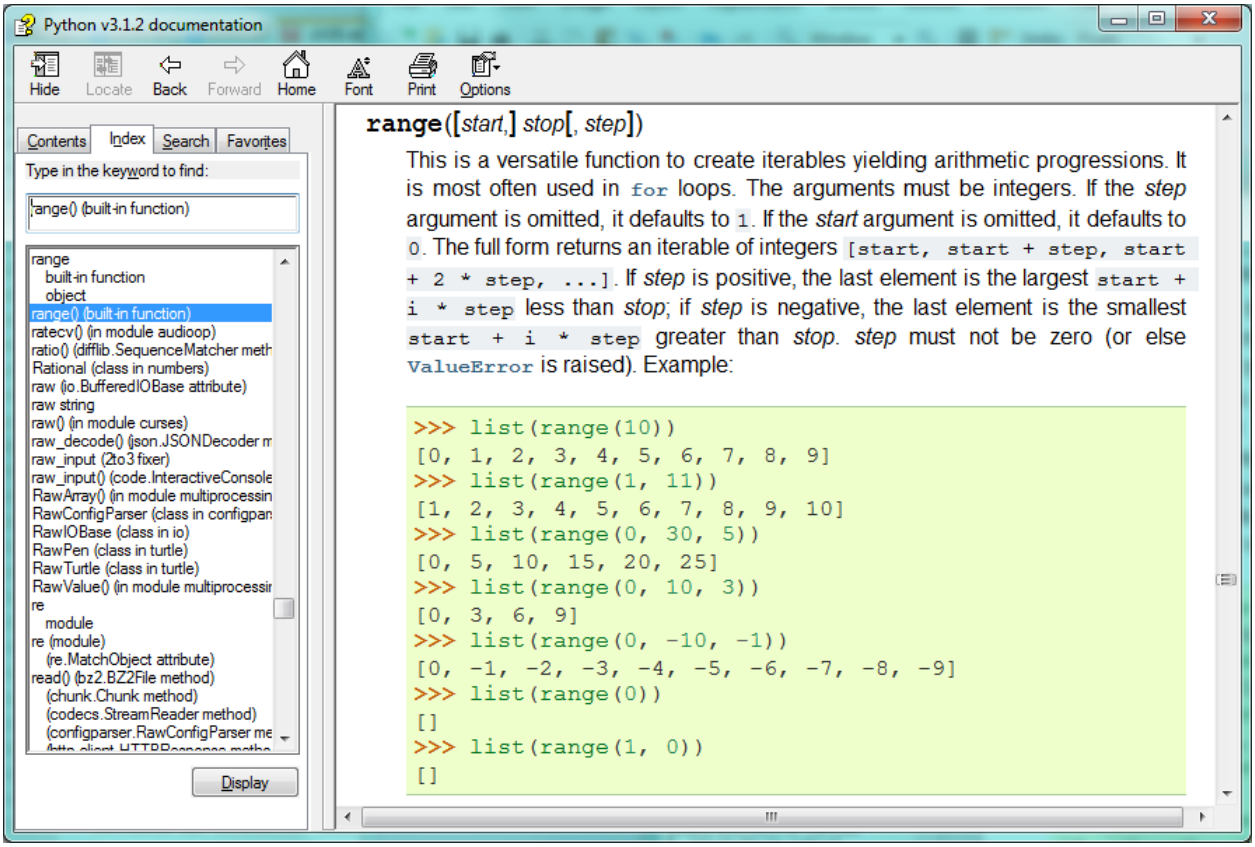
```
>>> n = 2
>>> n += 5
>>> n
7
```

There are similar abbreviations for `-=`, `*=`, `/=`, `//=` and `%=`:

```
>>> n = 2
>>> n *= 5
>>> n
10
>>> n -= 4
>>> n
6
>>> n //= 2
>>> n
3
>>> n %= 2
>>> n
1
```

7.9. Help and meta-notation

Python comes with extensive documentation for all its built-in functions, and its libraries. Different systems have different ways of accessing this help. In PyScripter, click on the Help menu item, and select Python Manuals. Then search for help on the built-in function `range`. You'll get something like this:



Notice the square brackets in the description of the arguments. These are examples of **meta-notation** — notation that describes Python syntax, but is not part of it. The square brackets in this documentation mean that the argument is optional — the programmer can omit it. So what this first line of help tells us is that `range` must always have a `stop` argument, but it may have an optional `start` argument (which must be followed by a comma if it is present), and it can also have an optional `step` argument, preceded by a comma if it is present.

The examples from help show that `range` can have either 1, 2 or 3 arguments. The list can start at any starting value, and go up or down in increments other than 1. The documentation here also says that the arguments must be integers.

Other meta-notation you'll frequently encounter is the use of bold and italics. The bold means that these are tokens — keywords or symbols — typed into your Python code exactly as they are, whereas the italic terms stand for "something of this type". So the syntax description

for *variable* **in** *list*:

means you can substitute any legal variable and any legal list when you write your Python code.

This (simplified) description of the `print` function, shows another example of meta-notation in which the ellipses (...) mean that you can have as many objects as you like (even zero), separated by commas:

print([*object, ...***])**

Meta-notation gives us a concise and powerful way to describe the pattern of some syntax or feature.

7.10. Tables

One of the things loops are good for is generating tables. Before computers were readily available, people had to calculate logarithms, sines and cosines, and other mathematical functions by hand. To make that easier, mathematics books contained long tables listing the values of these functions. Creating the tables was slow and boring, and they tended to be full of errors.

When computers appeared on the scene, one of the initial reactions was, "*This is great! We can use the computers to generate the tables, so there will be no errors.*" That turned out to be true (mostly) but shortsighted. Soon thereafter, computers and calculators were so pervasive that the tables became obsolete.

Well, almost. For some operations, computers use tables of values to get an approximate answer and then perform computations to improve the approximation. In some cases, there have been errors in the underlying tables, most famously in the table the Intel Pentium processor chip used to perform floating-point division.

Although a log table is not as useful as it once was, it still makes a good example of iteration. The following program outputs a sequence of values in the left column and 2 raised to the power of that value in the right column:

```
for x in range(13):   # Generate numbers 0 to 12
    print(x, "\t", 2**x)
```

The string `"\t"` represents a **tab character**. The backslash character in `"\t"` indicates the beginning of an **escape sequence**. Escape sequences are used to represent invisible characters like tabs and newlines. The sequence `\n` represents a **newline**.

An escape sequence can appear anywhere in a string; in this example, the tab escape sequence is the only thing in the string. How do you think you represent a backslash in a string?

As characters and strings are displayed on the screen, an invisible marker called the cursor keeps track of where the next character will go. After a `print` function, the cursor normally goes to the beginning of the next line.

The tab character shifts the cursor to the right until it reaches one of the tab stops. Tabs are useful for making columns of text line up, as in the output of the previous program:

```
0       1
1       2
2       4
3       8
4       16
5       32
6       64
7       128
8       256
9       512
10      1024
11      2048
12      4096
```

Because of the tab characters between the columns, the position of the second column does not depend on the number of digits in the first column.

7.11. Two-dimensional tables

A two-dimensional table is a table where you read the value at the intersection of a row and a column. A multiplication table is a good example. Let's say you want to print a multiplication table for the values from `1` to `6`.

A good way to start is to write a loop that prints the multiples of `2`, all on one line:

```
for i in range(1, 7):
    print(2 * i, end="   ")
print()
```

Here we've used the `range` function, but made it start its sequence at `1`. As the loop executes, the value of `i` changes from `1` to `6`. When all the elements of the range have been assigned to `i`, the loop terminates. Each time through the loop, it displays the value of `2 * i`, followed by three spaces.

Again, the extra `end="  "` argument in the `print` function suppresses the newline, and uses three spaces instead. After the loop completes, the call to `print` at line `3` finishes the current line, and starts a new line.

The output of the program is:

```
2     4     6     8     10    12
```

So far, so good. The next step is to **encapsulate** and **generalize**.

7.12. Encapsulation and generalization

Encapsulation is the process of wrapping a piece of code in a function, allowing you to take advantage of all the things functions are good for. You have already seen some examples of encapsulation, including `is_divisible` in a previous chapter.

Generalization means taking something specific, such as printing the multiples of `2`, and making it more general, such as printing the multiples of any integer.

This function encapsulates the previous loop and generalizes it to print multiples of `n`:

```
def print_multiples(n):
    for i in range(1, 7):
        print(n * i, end="   ")
    print()
```

To encapsulate, all we had to do was add the first line, which declares the name of the function and the parameter list. To generalize, all we had to do was replace the value `2` with the parameter `n`.

If we call this function with the argument `2`, we get the same output as before. With the argument `3`, the output is:

```
3     6     9     12    15    18
```

With the argument `4`, the output is:

```
4     8     12    16    20    24
```

By now you can probably guess how to print a multiplication table — by calling `print_multiples` repeatedly with different arguments. In fact, we can use another loop:

```
for i in range(1, 7):
    print_multiples(i)
```

Notice how similar this loop is to the one inside `print_multiples`. All we did was replace the print function with a function call.

The output of this program is a multiplication table:

```
1  1      2      3      4      5      6
2  2      4      6      8      10     12
3  3      6      9      12     15     18
4  4      8      12     16     20     24
5  5      10     15     20     25     30
6  6      12     18     24     30     36
```

7.13. More encapsulation

To demonstrate encapsulation again, let's take the code from the last section and wrap it up in a function:

```
def print_mult_table():
    for i in range(1, 7):
        print_multiples(i)
```

This process is a common **development plan**. We develop code by writing lines of code outside any function, or typing them in to the interpreter. When we get the code working, we extract it and wrap it up in a function.

This development plan is particularly useful if you don't know how to divide the program into functions when you start writing. This approach lets you design as you go along.

7.14. Local variables

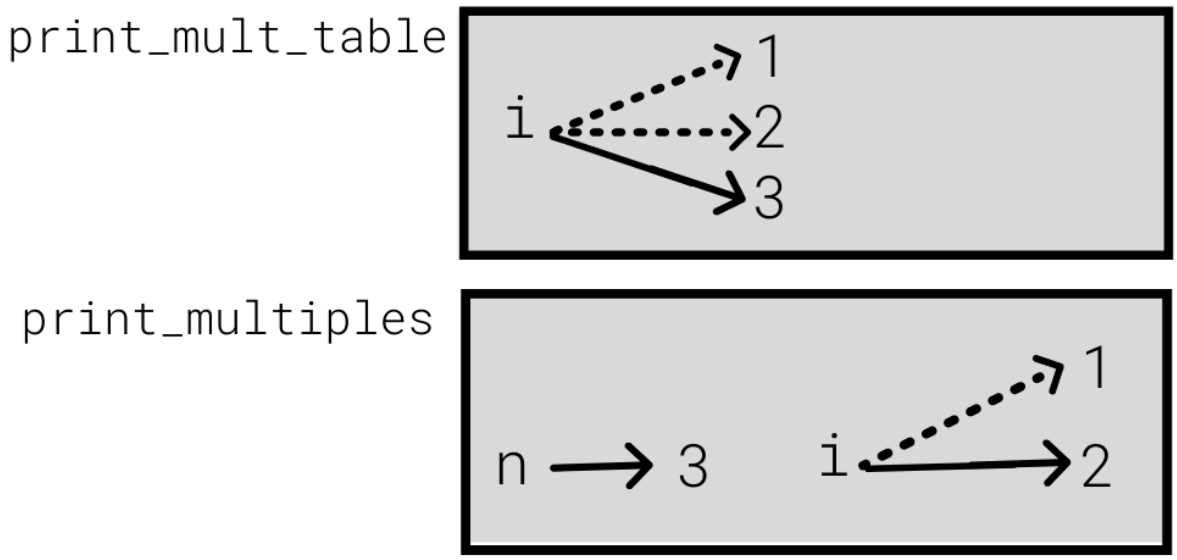
You might be wondering how we can use the same variable, `i`, in both `print_multiples` and `print_mult_table`. Doesn't it cause problems when one of the functions changes the value of the variable?

The answer is no, because the `i` in `print_multiples` and the `i` in `print_mult_table` are not the same variable.

Variables created inside a function definition are local; you can't access a local variable from outside its home function. That means you are free to have multiple variables with the same name as long as they are not in the same function.

Python examines all the statements in a function — if any of them assign a value to a variable, that is the clue that Python uses to make the variable a local variable.

The stack diagram for this program shows that the two variables named `i` are not the same variable. They can refer to different values, and changing one does not affect the other.



Stack 2 diagram

The value of `i` in `print_mult_table` goes from `1` to `6`. In the diagram it happens to be `3`. The next time through the loop it will be `4`. Each time through the loop, `print_mult_table` calls `print_multiples` with the current value of `i` as an argument. That value gets assigned to the parameter `n`.

Inside `print_multiples`, the value of `i` goes from `1` to `6`. In the diagram, it happens to be `2`. Changing this variable has no effect on the value of `i` in `print_mult_table`.

It is common and perfectly legal to have different local variables with the same name. In particular, names like `i` and `j` are used frequently as loop variables. If you avoid using them in one function just because you used them somewhere else, you will probably make the program harder to read.

The visualizer at http://pythontutor.com/ shows very clearly how the two variables `i` are distinct variables, and how they have independent values.

7.15. The break statement

The *break* statement is used to immediately leave the body of its loop. The next statement to be executed is the first one after the body:

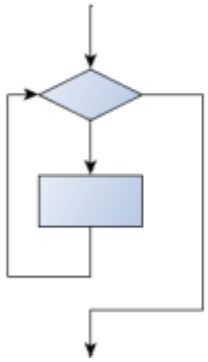
```
for i in [12, 16, 17, 24, 29]:
    if i % 2 == 1:  # If the number is odd
        break       #  ... immediately exit the loop
    print(i)
print("done")
```

This prints:

```
12
16
done
```

The pre-test loop — standard loop behaviour

`for` and `while` loops do their tests at the start, before executing any part of the body. They're called **pre-test** loops, because the test happens before (pre) the body. `break` and `return` are our tools for adapting this standard behaviour.

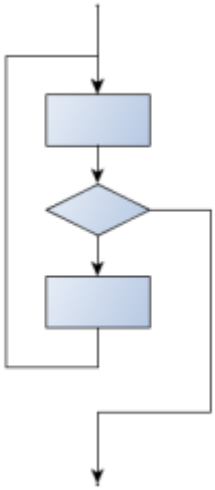


7.16. Other flavours of loops

Sometimes we'd like to have the **middle-test** loop with the exit test in the middle of the body, rather than at the beginning or at the end. Or a **post-test** loop that puts its exit test as the last thing in the body. Other languages have different syntax and keywords for these different flavours, but Python just uses a combination of `while` and `if condition: break` to get the job done.

A typical example is a problem where the user has to input numbers to be summed. To indicate that there are no more inputs, the user enters a special value, often the value `-1`, or the empty string. This needs a middle-exit loop pattern: input the next number, then test whether to exit, or else process the number:

The middle-test loop flowchart



```
1 total = 0
2 while True:
3     response = input("Enter the next number. (Leave blank to end)")
4     if response == "":
5         break
6     total += int(response)
7 print("The total of the numbers you entered is ", total)
```

Convince yourself that this fits the middle-exit loop flowchart: line 3 does some useful work, lines 4 and 5 can exit the loop, and if they don't line 6 does more useful work before the next iteration starts.

The `while bool-expr:` uses the Boolean expression to determine whether to iterate again. `True` is a trivial Boolean expression, so `while True:` means always do the loop body again. This is a language idiom — a convention that most programmers will recognize immediately. Since the expression on line 2 will never terminate the loop, (it is a dummy test) the programmer must arrange to break (or return) out of the loop body elsewhere, in some other way (i.e. in lines 4 and 5 in this sample). A clever compiler or interpreter will understand that line 2 is a fake test that must always succeed, so it won't even generate a test, and our flowchart never even put the diamond-shape dummy test box at the top of the loop!

Similarly, by just moving the `if condition: break` to the end of the loop body we create a pattern for a post-test loop. Post-test loops are used when you want to be sure that the loop body always executes at least once (because the first test only happens at the end of the execution of the first loop body). This is useful, for example, if we want to play an interactive game against the user — we always want to play at least one game:

```
while True:
    play_the_game_once()
    response = input("Play again? (yes or no)")
    if response != "yes":
        break
print("Goodbye!")
```

Hint: Think about where you want the exit test to happen.

Once you've recognized that you need a loop to repeat something, think about its terminating condition — when will I want to stop iterating? Then figure out whether you need to do the test before starting the first (and every other) iteration, or at the end of the first (and every other) iteration, or perhaps in the middle of each iteration. Interactive programs that require input from the user or read from files often need to exit their loops in the middle or at the end of an iteration, when it becomes clear that there is no more data to process, or the user doesn't want to play our game anymore.

7.17. An example

The following program implements a simple guessing game:

```
import random                   # We cover random numbers in the
rng = random.Random()           #  modules chapter, so peek ahead.
number = rng.randrange(1, 1000) # Get random number between [1 and 1000).

guesses = 0
msg = ""

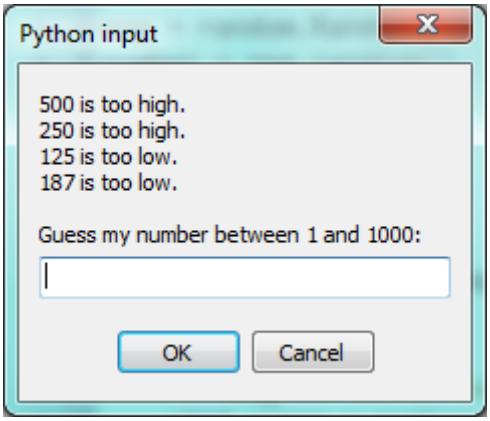
while True:
    guess = int(input(msg + "\nGuess my number between 1 and 1000: "))
    guesses += 1
    if guess > number:
        msg += str(guess) + " is too high.\n"
    elif guess < number:
        msg += str(guess) + " is too low.\n"
    else:
        break

input("\n\nGreat, you got it in {0} guesses!\n\n".format(guesses))
```

This program makes use of the mathematical law of **trichotomy** (given real numbers `a` and `b`, exactly one of these three must be true: `a > b`, `a < b`, or `a == b`).

At line 18 there is a call to the `input` function, but we don't do anything with the result, not even assign it to a variable. This is legal in Python. Here it has the effect of popping up the input dialog window and waiting for the user to respond before the program terminates. Programmers often use the trick of doing some extra input at the end of a script, just to keep the window open.

Also notice the use of the `msg` variable, initially an empty string, on lines 6, 12 and 14. Each time through the loop we extend the message being displayed: this allows us to display the program's feedback right at the same place as we're asking for the next guess.



7.18. The continue statement

This is a control flow statement that causes the program to immediately skip the processing of the rest of the body of the loop, for the current iteration. But the loop still carries on running for its remaining iterations:

```
1 for i in [12, 16, 17, 24, 29, 30]:
2     if i % 2 == 1:      # If the number is odd
3         continue        # Don't process it
4     print(i)
5 print("done")
```

This prints:

```
1 12
2 16
3 24
4 30
5 done
```

7.19. More generalization

As another example of generalization, imagine you wanted a program that would print a multiplication table of any size, not just the six-by-six table. You could add a parameter to `print_mult_table`:

```
def print_mult_table(high):
    for i in range(1, high+1):
        print_multiples(i)
```

We replaced the value 7 with the expression high+1. If we call `print_mult_table` with the argument 7, it displays:

```
1      2      3      4      5      6
2      4      6      8      10     12
3      6      9      12     15     18
4      8      12     16     20     24
5      10     15     20     25     30
6      12     18     24     30     36
7      14     21     28     35     42
```

This is fine, except that we probably want the table to be square — with the same number of rows and columns. To do that, we add another parameter to `print_multiples` to specify how many columns the table should have.

Just to be annoying, we call this parameter high, demonstrating that different functions can have parameters with the same name (just like local variables). Here's the whole program:

```
def print_multiples(n, high):
    for i in range(1, high+1):
        print(n * i, end="   ")
    print()

def print_mult_table(high):
    for i in range(1, high+1):
        print_multiples(i, high)
```

Notice that when we added a new parameter, we had to change the first line of the function (the function heading), and we also had to change the place where the function is called in `print_mult_table`.

Now, when we call `print_mult_table(7)`:

```
1  1      2      3      4      5      6      7
2  2      4      6      8      10     12     14
3  3      6      9      12     15     18     21
4  4      8      12     16     20     24     28
5  5      10     15     20     25     30     35
6  6      12     18     24     30     36     42
7  7      14     21     28     35     42     49
```

When you generalize a function appropriately, you often get a program with capabilities you didn't plan. For example, you might notice that, because `ab = ba`, all the entries in the table appear twice. You could save ink by printing only half the table. To do that, you only have to change one line of `print_mult_table`. Change

```
1  print_multiples(i, high+1)
```

to

```
1  print_multiples(i, i+1)
```

and you get:

```
1  1
2  2      4
3  3      6      9
4  4      8      12     16
5  5      10     15     20     25
6  6      12     18     24     30     36
7  7      14     21     28     35     42     49
```

7.20. Functions

A few times now, we have mentioned all the things functions are good for. By now, you might be wondering what exactly those things are. Here are some of them:

1. Capturing your mental chunking. Breaking your complex tasks into sub-tasks, and giving the sub-tasks a meaningful name is a powerful mental technique. Look back at the example that illustrated the post-test loop: we assumed that we had a function called `play_the_game_once`. This chunking allowed us to put aside the details of the particular game — is it a card game, or noughts and crosses, or a role playing game — and simply focus on one isolated part of our program logic — letting the player choose whether they want to play again.

2. Dividing a long program into functions allows you to separate parts of the program, debug them in isolation, and then compose them into a whole.
3. Functions facilitate the use of iteration.
4. Well-designed functions are often useful for many programs. Once you write and debug one, you can reuse it.

7.21. Paired Data

We've already seen lists of names and lists of numbers in Python. We're going to peek ahead in the textbook a little, and show a more advanced way of representing our data. Making a pair of things in Python is as simple as putting them into parentheses, like this:

```
year_born = ("Paris Hilton", 1981)
```

We can put many pairs into a list of pairs:

```
celebs = [("Brad Pitt", 1963), ("Jack Nicholson", 1937),
                                ("Justin Bieber", 1994)]
```

Here is a quick sample of things we can do with structured data like this. First, print all the celebs:

```
print(celebs)
print(len(celebs))
```

```
[("Brad Pitt", 1963), ("Jack Nicholson", 1937), ("Justin Bieber", 1994)]
3
```

Notice that the celebs list has just 3 elements, each of them pairs.

Now we print the names of those celebrities born before 1980:

```
for (nm, yr) in celebs:
    if yr < 1980:
        print(nm)
```

```
Brad Pitt
Jack Nicholson
```

This demonstrates something we have not seen yet in the `for` loop: instead of using a single loop control variable, we've used a pair of variable names, `(nm, yr)`, instead. The loop is executed three times — once for each pair in the list, and on each iteration both the variables are assigned values from the pair of data that is being handled.

7.22. Nested Loops for Nested Data

Now we'll come up with an even more adventurous list of structured data. In this case, we have a list of students. Each student has a name which is paired up with another list of subjects that they are enrolled for:

```
students = [
    ("John", ["CompSci", "Physics"]),
    ("Vusi", ["Maths", "CompSci", "Stats"]),
    ("Jess", ["CompSci", "Accounting", "Economics", "Management"]),
    ("Sarah", ["InfSys", "Accounting", "Economics", "CommLaw"]),
    ("Zuki", ["Sociology", "Economics", "Law", "Stats", "Music"])]
```

Here we've assigned a list of five elements to the variable `students`. Let's print out each student name, and the number of subjects they are enrolled for:

```
# Print all students with a count of their courses.
for (name, subjects) in students:
    print(name, "takes", len(subjects), "courses")
```

Python agreeably responds with the following output:

```
John takes 2 courses
Vusi takes 3 courses
Jess takes 4 courses
Sarah takes 4 courses
Zuki takes 5 courses
```

Now we'd like to ask how many students are taking CompSci. This needs a counter, and for each student we need a second loop that tests each of the subjects in turn:

```
# Count how many students are taking CompSci
counter = 0
for (name, subjects) in students:
    for s in subjects:                # A nested loop!
        if s == "CompSci":
            counter += 1

print("The number of students taking CompSci is", counter)
```

```
The number of students taking CompSci is 3
```

You should set up a list of your own data that interests you — perhaps a list of your CDs, each containing a list of song titles on the CD, or a list of movie titles, each with a list of movie stars who acted in the movie. You could then ask questions like "Which movies starred Angelina Jolie?"

7.23. Newton's method for finding square roots

Loops are often used in programs that compute numerical results by starting with an approximate answer and iteratively improving it.

For example, before we had calculators or computers, people needed to calculate square roots manually. Newton used a particularly good method (there is some evidence that this method was known many years before). Suppose that you want to know the square root of `n`. If you start with almost any approximation, you can compute a better approximation (closer to the actual answer) with the following formula:

```
better = (approx + n/approx)/2
```

Repeat this calculation a few times using your calculator. Can you see why each iteration brings your estimate a little closer? One of the amazing properties of this particular algorithm is how quickly it converges to an accurate answer — a great advantage for doing it manually.

By using a loop and repeating this formula until the better approximation gets close enough to the previous one, we can write a function for computing the square root. (In fact, this is how your calculator finds square roots — it may have a slightly different formula and method, but it is also based on repeatedly improving its guesses.)

This is an example of an indefinite iteration problem: we cannot predict in advance how many times we'll want to improve our guess — we just want to keep getting closer and closer. Our stopping condition for the loop will be when our old guess and our improved guess are "close enough" to each other.

Ideally, we'd like the old and new guess to be exactly equal to each other when we stop. But exact equality is a tricky notion in computer arithmetic when real numbers are involved. Because real numbers are not represented absolutely accurately (after all, a number like `pi` or the square root of two has an infinite number of decimal places because it is irrational), we need to formulate the stopping test for the loop by asking "is `a` close enough to `b`"? This stopping condition can be coded like this:

```
if abs(a-b) < 0.001:  # Make this smaller for better accuracy
      break
```

Notice that we take the absolute value of the difference between `a` and `b`!

This problem is also a good example of when a middle-exit loop is appropriate:

```
def sqrt(n):
    approx = n/2.0     # Start with some or other guess at the answer
    while True:
        better = (approx + n/approx)/2.0
        if abs(approx - better) < 0.001:
            return better
        approx = better

# Test cases
print(sqrt(25.0))
print(sqrt(49.0))
print(sqrt(81.0))
```

The output is:

```
5.00000000002
7.0
9.0
```

See if you can improve the approximations by changing the stopping condition. Also, step through the algorithm (perhaps by hand, using your calculator) to see how many iterations were needed before it achieved this level of accuracy for sqrt(25).

7.24. Algorithms

Newton's method is an example of an algorithm: it is a mechanical process for solving a category of problems (in this case, computing square roots).

Some kinds of knowledge are not algorithmic. For example, learning dates from history or your multiplication tables involves memorization of specific solutions.

But the techniques you learned for addition with carrying, subtraction with borrowing, and long division are all algorithms. Or if you are an avid Sudoku puzzle solver, you might have some specific set of steps that you always follow.

One of the characteristics of algorithms is that they do not require any intelligence to carry out. They are mechanical processes in which each step follows from the last according to a simple set of rules. And they're designed to solve a general class or category of problems, not just a single problem.

Understanding that hard problems can be solved by step-by-step algorithmic processes (and having technology to execute these algorithms for us) is one of the major breakthroughs that has had enormous benefits. So while the execution of the algorithm may be boring and may require no intelligence, algorithmic or computational thinking — i.e. using algorithms and automation as the basis for approaching problems — is rapidly transforming our society. Some claim that this shift

towards algorithmic thinking and processes is going to have even more impact on our society than the invention of the printing press. And the process of designing algorithms is interesting, intellectually challenging, and a central part of what we call programming.

Some of the things that people do naturally, without difficulty or conscious thought, are the hardest to express algorithmically. Understanding natural language is a good example. We all do it, but so far no one has been able to explain how we do it, at least not in the form of a step-by-step mechanical algorithm.

7.25. Glossary

algorithm

A step-by-step process for solving a category of problems.

body

The statements inside a loop.

breakpoint

A place in your program code where program execution will pause (or break), allowing you to inspect the state of the program's variables, or single-step through individual statements, executing them one at a time.

bump

Programmer slang. Synonym for increment.

continue statement

A statement that causes the remainder of the current iteration of a loop to be skipped. The flow of execution goes back to the top of the loop, evaluates the condition, and if this is true the next iteration of the loop will begin.

counter

A variable used to count something, usually initialized to zero and incremented in the body of a loop.

cursor

An invisible marker that keeps track of where the next character will be printed.

decrement

Decrease by 1.

definite iteration

A loop where we have an upper bound on the number of times the body will be executed. Definite iteration is usually best coded as a `for` loop.

development plan

A process for developing a program. In this chapter, we demonstrated a style of development based on developing code to do simple, specific things and then encapsulating and generalizing.

encapsulate

To divide a large complex program into components (like functions) and isolate the components from each other (by using local variables, for example).

escape sequence

An escape character, `\`, followed by one or more printable characters used to designate a non printable character.

generalize

To replace something unnecessarily specific (like a constant value) with something appropriately general (like a variable or parameter). Generalization makes code more versatile, more likely to be reused, and sometimes even easier to write.

increment

Both as a noun and as a verb, increment means to increase by `1`.

infinite loop

A loop in which the terminating condition is never satisfied.

indefinite iteration

A loop where we just need to keep going until some condition is met. A `while` statement is used for this case.

initialization (of a variable)

To initialize a variable is to give it an initial value. Since in Python, variables don't exist until they are assigned values, they are initialized when they are created. In other programming languages this is not the case, and variables can be created without being initialized, in which case they have either default or garbage values.

iteration

Repeated execution of a set of programming statements.

loop

The construct that allows allows us to repeatedly execute a statement or a group of statements until a terminating condition is satisfied.

loop variable

A variable used as part of the terminating condition of a loop.

meta-notation

Extra symbols or notation that helps describe other notation. Here we introduced square brackets, ellipses, italics, and bold as meta-notation to help describe optional, repeatable, substitutable and fixed parts of the Python syntax.

middle-test loop

A loop that executes some of the body, then tests for the exit condition, and then may execute some more of the body. We don't have a special Python construct for this case, but can use `while` and `break` together.

nested loop

A loop inside the body of another loop.

newline

A special character that causes the cursor to move to the beginning of the next line.

post-test loop

A loop that executes the body, then tests for the exit condition. We don't have a special Python construct for this, but can use `while` and `break` together.

pre-test loop

A loop that tests before deciding whether the execute its body. `for` and `while` are both pre-test loops.

single-step

A mode of interpreter execution where you are able to execute your program one step at a time, and inspect the consequences of that step. Useful for debugging and building your internal mental model of what is going on.

tab

A special character that causes the cursor to move to the next tab stop on the current line.

trichotomy

Given any real numbers `a` and `b`, exactly one of the following relations holds: `a < b`, `a > b`, or `a == b`. Thus when you can establish that two of the relations are false, you can assume the remaining one is true.

trace

To follow the flow of execution of a program by hand, recording the change of state of the variables and any output produced.

7.26. Exercises

This chapter showed us how to sum a list of items, and how to count items. The counting example also had an `if` statement that let us only count some selected items. In the previous chapter we also

showed a function `find_first_2_letter_word` that allowed us an "early exit" from inside a loop by using `return` when some condition occurred. We now also have `break` to exit a loop but not the enclosing function, and `continue` to abandon the current iteration of the loop without ending the loop.

Composition of list traversal, summing, counting, testing conditions and early exit is a rich collection of building blocks that can be combined in powerful ways to create many functions that are all slightly different.

The first six questions are typical functions you should be able to write using only these building blocks.

1. Write a function to count how many odd numbers are in a list.
2. Sum up all the even numbers in a list.
3. Sum up all the negative numbers in a list.
4. Count how many words in a list have length 5.
5. Sum all the elements in a list up to but not including the first even number. (Write your unit tests. What if there is no even number?)
6. Count how many words occur in a list up to and including the first occurrence of the word "sam". (Write your unit tests for this case too. What if "sam" does not occur?)
7. Add a print function to Newton's sqrt function that prints out better each time it is calculated. Call your modified function with 25 as an argument and record the results.
8. Trace the execution of the last version of print_mult_table and figure out how it works.
9. Write a function print_triangular_numbers(n) that prints out the first n triangular numbers. A call to print_triangular_numbers(5) would produce the following output:

```
1  1       1
2  2       3
3  3       6
4  4       10
5  5       15
```

   (hint: use a web search to find out what a triangular number is.)
10. Write a function, is_prime, which takes a single integer argument and returns True when the argument is a prime number and False otherwise. Add tests for cases like this:

```
test(is_prime(11))
test(not is_prime(35))
test(is_prime(19911121))
```

The last case could represent your birth date. Were you born on a prime day? In a class of 100 students, how many do you think would have prime birth dates?

11. Revisit the drunk pirate problem from the exercises in chapter 3. This time, the drunk pirate makes a turn, and then takes some steps forward, and repeats this. Our social science student now records pairs of data: the angle of each turn, and the number of steps taken after the turn. Her experimental data is [(160, 20), (-43, 10), (270, 8), (-43, 12)]. Use a turtle to draw the path taken by our drunk friend.
12. Many interesting shapes can be drawn by the turtle by giving a list of pairs like we did above, where the first item of the pair is the angle to turn, and the second item is the distance to move forward. Set up a list of pairs so that the turtle draws a house with a cross through the centre, as show here. This should be done without going over any of the lines / edges more than once, and without lifting your pen.

13. Not all shapes like the one above can be drawn without lifting your pen, or going over an edge more than once. Which of these can be drawn?

Now read Wikipedia's article (http://en.wikipedia.org/wiki/Eulerian_path) about Eulerian paths. Learn how to tell immediately by inspection whether it is possible to find a solution or not. If the path is possible, you'll also know where to put your pen to start drawing, and where you should end up!

14. What will `num_digits(0)` return? Modify it to return 1 for this case. Why does a call to `num_digits(-24)` result in an infinite loop? (hint: -1//10 evaluates to -1) Modify `num_digits` so that it works correctly with any integer value. Add these tests:

```
test(num_digits(0) == 1)
test(num_digits(-12345) == 5)
```

15. Write a function `num_even_digits(n)` that counts the number of even digits in n. These tests should pass:

```
test(num_even_digits(123456) == 3)
test(num_even_digits(2468) == 4)
test(num_even_digits(1357) == 0)
test(num_even_digits(0) == 1)
```

16. Write a function `sum_of_squares(xs)` that computes the sum of the squares of the numbers in the list xs. For example, `sum_of_squares([2, 3, 4])` should return 4+9+16 which is 29:

```
test(sum_of_squares([2, 3, 4]) == 29)
test(sum_of_squares([ ]) == 0)
test(sum_of_squares([2, -3, 4]) == 29)
```

17. You and your friend are in a team to write a two-player game, human against computer, such as Tic-Tac-Toe / Noughts and Crosses. Your friend will write the logic to play one round of the game, while you will write the logic to allow many rounds of play, keep score, decide who plays, first, etc. The two of you negotiate on how the two parts of the program will fit together, and you come up with this simple scaffolding (which your friend will improve later):

```
# Your friend will complete this function
def play_once(human_plays_first):
    """
    Must play one round of the game. If the parameter
    is True, the human gets to play first, else the
    computer gets to play first.  When the round ends,
    the return value of the function is one of
    -1 (human wins),  0 (game drawn),   1 (computer wins).
    """
    # This is all dummy scaffolding code right at the moment...
    import random                   # See Modules chapter ...
    rng = random.Random()
    # Pick a random result between -1 and 1.
    result = rng.randrange(-1,2)
    print("Human plays first={0},  winner={1} "
                    .format(human_plays_first, result))
    return result
```

    a. Write the main program which repeatedly calls this function to play the game, and after each round it announces the outcome as "I win!", "You win!", or "Game drawn!". It then asks the player "Do you want to play again?" and either plays again, or says "Goodbye", and terminates.
    b. Keep score of how many wins each player has had, and how many draws there have been. After each round of play, also announce the scores.
    c. Add logic so that the players take turns to play first.
    d. Compute the percentage of wins for the human, out of all games played. Also announce this at the end of each round.

e. Draw a flowchart of your logic.

Chapter 8: Strings

8.1. A compound data type

So far we have seen built-in types like `int`, `float`, `bool`, `str` and we've seen lists and pairs. Strings, lists, and pairs are qualitatively different from the others because
they are made up of smaller pieces. In the case of strings, they're made up of smaller strings each containing one **character**

Types that comprise smaller pieces are called **compound data types**. Depending on what we are doing, we may want to treat a compound data type as a single thing, or we may want to
access its parts. This ambiguity is useful.

8.2. Working with strings as single things

We previously saw that each turtle instance has its own attributes and a number of methods that can be applied to the instance. For example, we could set the turtle's color, and we wrote `tess.turn(90)`.

Just like a turtle, a string is also an object. So each string instance has its own attributes and methods.

For example:

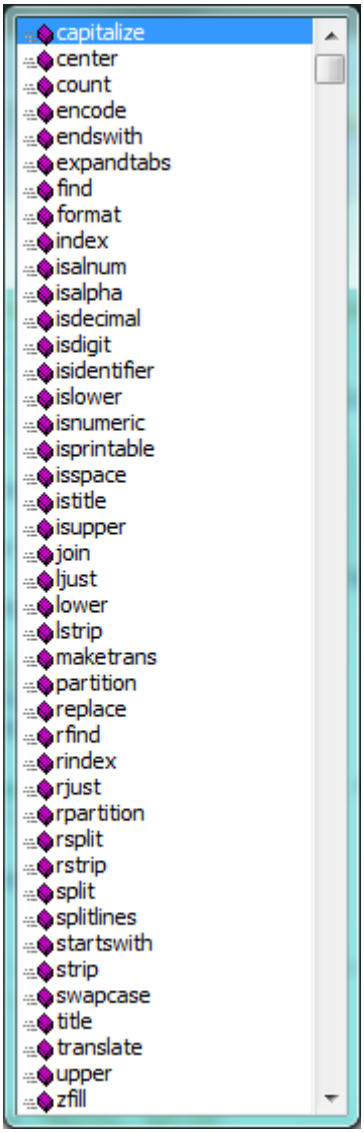
```
>>> ss = "Hello, World!"
>>> tt = ss.upper()
>>> tt
'HELLO, WORLD!'
```

`upper` is a method that can be invoked on any string object to create a new string, in which all the characters are in uppercase. (The original string `ss` remains unchanged.)

There are also methods such as `lower`, `capitalize`, and `swapcase` that do other interesting stuff.

To learn what methods are available, you can consult the Help documentation, look for string methods, and read the documentation. Or, if you're a bit lazier, simply type the following into a PyScripter script:

When you type the period to select one of the methods of `ss`, PyScripter will pop up a selection window showing all the methods (there are around 70 of them — thank goodness we'll only use a few of those!) that could be used on your string.



When you type the name of the method, some further help about its parameter and return type, and its docstring, will be displayed. This is a good example of a tool — PyScripter — using the meta-information — the docstrings — provided by the module programmers.

```
greet = "Hello, World"
xx= greet.swapcase()
print(xx)
```

** No/Unknown parameters **
S.swapcase() -> str

Return a copy of S with uppercase characters converted to lowercase and vice versa.

8.3. Working with the parts of a string

The **indexing operator** (Python uses square brackets to enclose the index) selects a single character substring from a string:

```
>>> fruit = "banana"
>>> m = fruit[1]
>>> print(m)
```

The expression `fruit[1]` selects character number 1 from `fruit`, and creates a new string containing just this one character. The variable `m` refers to the result. When we display `m`, we could get a surprise:

```
a
```

Computer scientists always start counting from zero! The letter at subscript position zero of `"banana"` is `b`. So at position `[1]` we have the letter `a`.

If we want to access the zero-eth letter of a string, we just place `0`, or any expression that evaluates to `0`, inbetween the brackets:

```
>>> m = fruit[0]
>>> print(m)
b
```

The expression in brackets is called an **index**. An index specifies a member of an ordered collection, in this case the collection of characters in the string. The index *indicates* which one you want, hence the name. It can be any integer expression.

We can use `enumerate` to visualize the indices:

```
>>> fruit = "banana"
>>> list(enumerate(fruit))
[(0, 'b'), (1, 'a'), (2, 'n'), (3, 'a'), (4, 'n'), (5, 'a')]
```

Do not worry about `enumerate` at this point, we will see more of it in the chapter on lists.

Note that indexing returns a *string* — Python has no special type for a single character. It is just a string of length 1.

We've also seen lists previously. The same indexing notation works to extract elements from a list:

```
>>> prime_nums = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]
>>> prime_nums[4]
11
>>> friends = ["Joe", "Zoe", "Brad", "Angelina", "Zuki", "Thandi", "Paris"]
>>> friends[3]
'Angelina'
```

8.4. Length

The `len` function, when applied to a string, returns the number of characters in a string:

```
>>> fruit = "banana"
>>> len(fruit)
6
```

To get the last letter of a string, you might be tempted to try something like this:

That won't work. It causes the runtime error `IndexError: string index out of range`. The reason is that there is no character at index position 6 in `"banana"`. Because we start counting at zero, the six indexes are numbered 0 to 5. To get the last character, we have to subtract 1 from the length of `fruit`:

Alternatively, we can use negative indices, which count backward from the end of the string. The expression `fruit[-1]` yields the last letter, `fruit[-2]` yields the second to last, and so on.

As you might have guessed, indexing with a negative index also works like this for lists.

We won't use negative indexes in the rest of these notes — not many computer languages use this idiom, and you'll probably be better off avoiding it. But there is plenty of Python code out on the Internet that will use this trick, so it is best to know that it exists.

8.5. Traversal and the `for` loop

A lot of computations involve processing a string one character at a time. Often they start at the beginning, select each character in turn, do something to it, and continue until the end. This pattern of processing is called a **traversal**. One way to encode a traversal is with a `while` statement:

This loop traverses the string and displays each letter on a line by itself. The loop condition is `ix < len(fruit)`, so when `ix` is equal to the length of the string, the condition is false, and the body of the loop is not executed. The last character accessed is the one with the index `len(fruit)-1`, which is the last character in the string.

But we've previously seen how the `for` loop can easily iterate over the elements in a list and it can do so for strings as well:

Each time through the loop, the next character in the string is assigned to the variable `c`. The loop continues until no characters are left. Here we can see the expressive power the `for` loop gives us compared to the `while` loop when traversing a string.

The following example shows how to use concatenation and a `for` loop to generate an abecedarian series. Abecedarian refers to a series or list in which the elements appear in alphabetical order. For example, in Robert McCloskey's book Make Way for Ducklings, the names of the ducklings are Jack, Kack, Lack, Mack, Nack, Ouack, Pack, and Quack. This loop outputs these names in order:

The output of this program is:

```
Jack
Kack
Lack
Mack
Nack
Oack
Pack
Qack
```

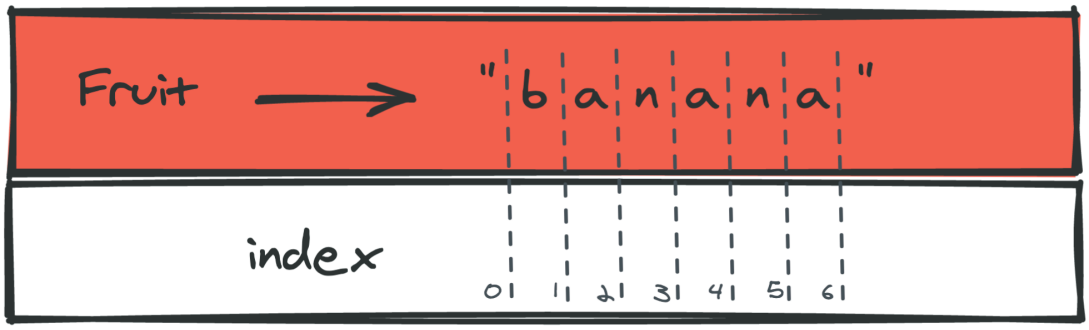
Of course, that's not quite right because Ouack and Quack are misspelled. You'll fix this as an exercise below.

8.6. Slices

A *substring* of a string is obtained by taking a **slice**. Similarly, we can slice a list to refer to some sublist of the items in the list:

```
>>> s = "Pirates of the Caribbean"
>>> print(s[0:7])
Pirates
>>> print(s[11:14])
the
>>> print(s[15:24])
Caribbean
>>> friends = ["Joe", "Zoe", "Brad", "Angelina", "Zuki", "Thandi", "Paris"]
>>> print(friends[2:4])
['Brad', 'Angelina']
```

The operator `[n:m]` returns the part of the string from the n'th character to the m'th character, including the first but excluding the last. This behavior makes sense if you imagine the indices pointing *between* the characters, as in the following diagram:



If you imagine this as a piece of paper, the slice operator `[n:m]` copies out the part of the paper between the `n` and `m` positions. Provided `m` and `n` are both within the bounds of the string, your result will be of length (m-n).

Three tricks are added to this: if you omit the first index (before the colon), the slice starts at the beginning of the string (or list). If you omit the second index, the slice extends to the end of the string (or list). Similarly, if you provide value for `n` that is bigger than the length of the string (or list), the slice will take all the values up to the end. (It won't give an "out of range" error like the normal indexing operation does.) Thus:

```
>>> fruit = "banana"
>>> fruit[:3]
'ban'
>>> fruit[3:]
'ana'
>>> fruit[3:999]
'ana'
```

What do you think `s[:]` means? What about `friends[4:]`?

8.7. String comparison

The comparison operators work on strings. To see if two strings are equal:

Other comparison operations are useful for putting words in lexicographical order:

This is similar to the alphabetical order you would use with a dictionary, except that all the uppercase letters come before all the lowercase letters. As a result:

```
Your word, Zebra, comes before banana.
```

A common way to address this problem is to convert strings to a standard format, such as all lowercase, before performing the comparison. A more difficult problem is making the program realize that zebras are not fruit.

8.8. Strings are immutable

It is tempting to use the `[]` operator on the left side of an assignment, with the intention of changing a character in a string. For example:

Instead of producing the output `Jello, world!`, this code produces the runtime error `TypeError: 'str' object does not support item assignment`.

Strings are **immutable**, which means you can't change an existing string. The best you can do is create a new string that is a variation on the original:

The solution here is to concatenate a new first letter onto a slice of `greeting`. This operation has no effect on the original string.

8.9. The `in` and `not in` operators

The `in` operator tests for membership. When both of the arguments to `in` are strings, `in` checks whether the left argument is a substring of the right argument.

```
>>> "p" in "apple"
True
>>> "i" in "apple"
False
>>> "ap" in "apple"
True
>>> "pa" in "apple"
False
```

Note that a string is a substring of itself, and the empty string is a substring of any other string. (Also note that computer scientists like to think about these edge cases quite carefully!)

```
>>> "a" in "a"
True
>>> "apple" in "apple"
True
>>> "" in "a"
True
>>> "" in "apple"
True
```

The `not in` operator returns the logical opposite results of `in`:

```
>>> "x" not in "apple"
True
```

Combining the `in` operator with string concatenation using `+`, we can write a function that removes all the vowels from a string:

```
def remove_vowels(s):
    vowels = "aeiouAEIOU"
    s_sans_vowels = ""
    for x in s:
        if x not in vowels:
            s_sans_vowels += x
    return s_sans_vowels

test(remove_vowels("compsci") == "cmpsc")
test(remove_vowels("aAbEefIijOopUus") == "bfjps")
```

8.10. A `find` function

What does the following function do?

```
def find(strng, ch):
    """
      Find and return the index of ch in strng.
      Return -1 if ch does not occur in strng.
    """
    ix = 0
    while ix < len(strng):
        if strng[ix] == ch:
            return ix
```

```
        ix += 1
    return -1

test(find("Compsci", "p") == 3)
test(find("Compsci", "C") == 0)
test(find("Compsci", "i") == 6)
test(find("Compsci", "x") == -1)
```

In a sense, `find` is the opposite of the indexing operator. Instead of taking an index and extracting the corresponding character, it takes a character and finds the index where that character appears. If the character is not found, the function returns `-1`.

This is another example where we see a `return` statement inside a loop. If `strng[ix] == ch`, the function returns immediately, breaking out of the loop prematurely.

If the character doesn't appear in the string, then the program exits the loop normally and returns `-1`.

This pattern of computation is sometimes called a **eureka traversal** or **short-circuit evaluation**, because as soon as we find what we are looking for, we can cry "Eureka!", take the short-circuit, and stop looking.

8.11. Looping and counting

The following program counts the number of times the letter `a` appears in a string, and is another example of the counter pattern introduced in counting.

```
def count_a(text):
    count = 0
    for c in text:
        if c == "a":
            count += 1
    return(count)

test(count_a("banana") == 3)
```

8.12. Optional parameters

To find the locations of the second or third occurrence of a character in a string, we can modify the `find` function, adding a third parameter for the starting position in the search string:

```
def find2(strng, ch, start):
    ix = start
    while ix < len(strng):
        if strng[ix] == ch:
            return ix
        ix += 1
    return -1

test(find2("banana", "a", 2) == 3)
```

The call `find2("banana", "a", 2)` now returns 3, the index of the first occurrence of "a" in "banana" starting the search at index 2. What does `find2("banana", "n", 3)` return? If you said, 4, there is a good chance you understand how `find2` works.

Better still, we can combine `find` and `find2` using an **optional parameter**:

```
def find(strng, ch, start=0):
    ix = start
    while ix < len(strng):
        if strng[ix] == ch:
            return ix
        ix += 1
    return -1
```

When a function has an optional parameter, the caller may provide a matching argument. If the third argument is provided to `find`, it gets assigned to `start`. But if the caller leaves the argument out, then start is given a default value indicated by the assignment `start=0` in the function definition.

So the call `find("banana", "a", 2)` to this version of `find` behaves just like `find2`, while in the call `find("banana", "a")`, `start` will be set to the **default value** of `0`.

Adding another optional parameter to `find` makes it search from a starting position, up to but not including the end position:

```
def find(strng, ch, start=0, end=None):
    ix = start
    if end is None:
       end = len(strng)
    while ix < end:
        if strng[ix] == ch:
            return ix
        ix += 1
    return -1
```

The optional value for `end` is interesting: we give it a default value `None` if the caller does not supply any argument. In the body of the function we test what `end` is, and if the caller did not supply any argument, we reassign `end` to be the length of the string. If the caller has supplied an argument for `end`, however, the caller's value will be used in the loop.

The semantics of `start` and `end` in this function are precisely the same as they are in the `range` function.

Here are some test cases that should pass:

```
ss = "Python strings have some interesting methods."
test(find(ss, "s") == 7)
test(find(ss, "s", 7) == 7)
test(find(ss, "s", 8) == 13)
test(find(ss, "s", 8, 13) == -1)
test(find(ss, ".") == len(ss)-1)
```

8.13. The built-in `find` method

Now that we've done all this work to write a powerful `find` function, we can reveal that strings already have their own built-in `find` method. It can do everything that our code can do, and more!

The built-in `find` method is more general than our version. It can find substrings, not just single characters:

```
>>> "banana".find("nan")

>>> "banana".find("na", 3)
4
```

Usually we'd prefer to use the methods that Python provides rather than reinvent our own equivalents. But many of the built-in functions and methods make good teaching exercises, and the underlying techniques you learn are your building blocks to becoming a proficient programmer.

8.14. The `split` method

One of the most useful methods on strings is the `split` method: it splits a single multi-word string into a list of individual words, removing all the whitespace between them. (Whitespace means any tabs, newlines, or spaces.) This allows us to read input as a single string, and split it into words.

```
>>> ss = "Well I never did said Alice"
>>> wds = ss.split()
>>> wds
['Well', 'I', 'never', 'did', 'said', 'Alice']
```

8.15. Cleaning up your strings

We'll often work with strings that contain punctuation, or tab and newline characters, especially, as we'll see in a future chapter, when we read our text from files or from the Internet. But if we're writing a program, say, to count word frequencies or check the spelling of each word, we'd prefer to strip off these unwanted characters.

We'll show just one example of how to strip punctuation from a string. Remember that strings are immutable, so we cannot change the string with the punctuation — we need to traverse the original string and create a new string, omitting any punctuation:

```
punctuation = "!\"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~"

def remove_punctuation(s):
    s_sans_punct = ""
    for letter in s:
        if letter not in punctuation:
            s_sans_punct += letter
    return s_sans_punct
```

Setting up that first assignment is messy and error-prone. Fortunately, the Python `string` module already does it for us. So we will make a slight improvement to this program — we'll import the `string` module and use its definition:

```
import string

def remove_punctuation(s):
    s_without_punct = ""
    for letter in s:
        if letter not in string.punctuation:
            s_without_punct += letter
    return s_without_punct

test(remove_punctuation('"Well, I never did!", said Alice.') ==
                            "Well I never did said Alice")
test(remove_punctuation("Are you very, very, sure?") ==
                             "Are you very very sure")
```

Composing together this function and the `split` method from the previous section makes a useful combination — we'll clean out the punctuation, and `split` will clean out the newlines and tabs while turning the string into a list of words:

```
my_story = """
Pythons are constrictors, which means that they will 'squeeze' the life
out of their prey. They coil themselves around their prey and with
each breath the creature takes the snake will squeeze a little tighter
until they stop breathing completely. Once the heart stops the prey
is swallowed whole. The entire animal is digested in the snake's
stomach except for fur or feathers. What do you think happens to the fur,
feathers, beaks, and eggshells? The 'extra stuff' gets passed out as ---
you guessed it --- snake POOP! """

wds = remove_punctuation(my_story).split()
print(wds)
```

The output:

```
['Pythons', 'are', 'constrictors', ... , 'it', 'snake', 'POOP']
```

There are other useful string methods, but this book isn't intended to be a reference manual. On the other hand, the *Python Library Reference* is. Along with a wealth of other documentation, it is available at the Python website[9].

8.16. The string format method

The easiest and most powerful way to format a string in Python 3 is to use the `format` method. To see how this works, let's start with a few examples:

```
s1 = "His name is {0}!".format("Arthur")
print(s1)

name = "Alice"
age = 10
s2 = "I am {1} and I am {0} years old.".format(age, name)
print(s2)

n1 = 4
n2 = 5
s3 = "2**10 = {0} and {1} * {2} = {3:f}".format(2**10, n1, n2, n1 * n2)
print(s3)
```

[9] https://www.python.org/

Running the script produces:

```
His name is Arthur!
I am Alice and I am 10 years old.
2**10 = 1024 and 4 * 5 = 20.000000
```

The template string contains place holders, `... {0} ... {1} ... {2} ...` etc. The `format` method substitutes its arguments into the place holders. The numbers in the place holders are indexes that determine which argument gets substituted — make sure you understand line 6 above!

But there's more! Each of the replacement fields can also contain a **format specification** — it is always introduced by the `:` symbol (Line 11 above uses one.) This modifies how the substitutions are made into the template, and can control things like:

- whether the field is aligned to the left `<`, center `^`, or right `>`
- the width allocated to the field within the result string (a number like `10`)
- the type of conversion (we'll initially only force conversion to float, `f`, as we did in line 11 of the code above, or perhaps we'll ask integer numbers to be converted to hexadecimal using `x`)
- if the type conversion is a float, you can also specify how many decimal places are wanted (typically, `.2f` is useful for working with currencies to two decimal places.)

Let's do a few simple and common examples that should be enough for most needs. If you need to do anything more esoteric, use help and read all the powerful, gory details.

```
n1 = "Paris"
n2 = "Whitney"
n3 = "Hilton"

print("Pi to three decimal places is {0:.3f}".format(3.1415926))
print("123456789 123456789 123456789 123456789 123456789 123456789")
print("|||{0:<15}|||{1:^15}|||{2:>15}|||Born in {3}|||"
        .format(n1,n2,n3,1981))
print("The decimal value {0} converts to hex value {0:x}"
        .format(123456))
```

This script produces the output:

```
Pi to three decimal places is 3.142
123456789 123456789 123456789 123456789 123456789 123456789
|||Paris          |||    Whitney    |||         Hilton|||Born in 1981|||
The decimal value 123456 converts to hex value 1e240
```

You can have multiple placeholders indexing the same argument, or perhaps even have extra arguments that are not referenced at all:

```
letter = """
Dear {0} {2}.
 {0}, I have an interesting money-making proposition for you!
 If you deposit $10 million into my bank account, I can
 double your money ...
"""

print(letter.format("Paris", "Whitney", "Hilton"))
print(letter.format("Bill", "Henry", "Gates"))
```

This produces the following:

```
Dear Paris Hilton.
 Paris, I have an interesting money-making proposition for you!
 If you deposit $10 million into my bank account, I can
 double your money ...


Dear Bill Gates.
 Bill, I have an interesting money-making proposition for you!
 If you deposit $10 million into my bank account I can
 double your money ...
```

As you might expect, you'll get an index error if your placeholders refer to arguments that you do not provide:

```
>>> "hello {3}".format("Dave")
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
IndexError: tuple index out of range
```

The following example illustrates the real utility of string formatting. First, we'll try to print a table without using string formatting:

```
print("i\ti**2\ti**3\ti**5\ti**10\ti**20")
for i in range(1, 11):
    print(i, "\t", i**2, "\t", i**3, "\t", i**5, "\t",
                                           i**10, "\t", i**20)
```

This program prints out a table of various powers of the numbers from 1 to 10. (This assumes that the tab width is 8. You might see something even worse than this if you tab width is set to 4.) In its current form it relies on the tab character (`\t`) to align the columns of values, but this breaks down when the values in the table get larger than the tab width:

```
i       i**2    i**3    i**5    i**10   i**20
1       1       1       1       1       1
2       4       8       32      1024    1048576
3       9       27      243     59049   3486784401
4       16      64      1024    1048576         1099511627776
5       25      125     3125    9765625         95367431640625
6       36      216     7776    60466176        3656158440062976
7       49      343     16807   282475249       79792266297612001
8       64      512     32768   1073741824      1152921504606846976
9       81      729     59049   3486784401      12157665459056928801
10      100     1000    100000  10000000000     100000000000000000000
```

One possible solution would be to change the tab width, but the first column already has more space than it needs. The best solution would be to set the width of each column independently. As you may have guessed by now, string formatting provides a much nicer solution. We can also right-justify each field:

```
layout = "{0:>4}{1:>6}{2:>6}{3:>8}{4:>13}{5:>24}"

print(layout.format("i", "i**2", "i**3", "i**5", "i**10", "i**20"))
for i in range(1, 11):
    print(layout.format(i, i**2, i**3, i**5, i**10, i**20))
```

Running this version produces the following (much more satisfying) output:

```
   i  i**2  i**3    i**5        i**10                  i**20
   1     1     1       1           1                      1
   2     4     8      32        1024                1048576
   3     9    27     243       59049             3486784401
   4    16    64    1024     1048576          1099511627776
   5    25   125    3125     9765625         95367431640625
   6    36   216    7776    60466176       3656158440062976
   7    49   343   16807   282475249      79792266297612001
   8    64   512   32768  1073741824    1152921504606846976
   9    81   729   59049  3486784401   12157665459056928801
  10   100  1000  100000 10000000000  100000000000000000000
```

8.17. Summary

This chapter introduced a lot of new ideas. The following summary may prove helpful in remembering what you learned.

indexing (`[]`)
Access a single character in a string using its position (starting from 0). Example: `"This"[2]` evaluates to `"i"`.

length function (`len`)
Returns the number of characters in a string. Example: `len("happy")` evaluates to `5`.

for loop traversal (`for`)
Traversing a string means accessing each character in the string, one at a time. For example, the following for loop:

```
for ch in "Example":
    ...
```

executes the body of the loop `7` times with different values of `ch` each time.

slicing (`[:]`)
A *slice* is a substring of a string. Example: `'bananas and cream'[3:6]` evaluates to `ana` (so does `'bananas and cream'[1:4]`).

string comparison (`>, <, >=, <=, ==, !=`)
The six common comparison operators work with strings, evaluating according to `lexicographical` order. Examples: `"apple" < "banana"` evaluates to `True`. `"Zeta" < "Appricot"` evaluates to `False`. `"Zebra" <= "aardvark"` evaluates to `True` because all upper case letters precede lower case letters.

in and not in operator (`in`, `not in`)
The `in` operator tests for membership. In the case of strings, it tests whether one string is contained inside another string. Examples: `"heck" in "I'll be checking for you."` evaluates to `True`. `"cheese" in "I'll be checking for you."` evaluates to `False`.

8.18. Glossary

compound data type
A data type in which the values are made up of components, or elements, that are themselves values.

default value
The value given to an optional parameter if no argument for it is provided in the function call.

docstring
A string constant on the first line of a function or module definition (and as we will see later, in class and method definitions as well).

Docstrings provide a convenient way to associate documentation with code. Docstrings are also used by programming tools to provide interactive help.

dot notation
Use of the **dot operator**, `.`, to access methods and attributes of an object.

immutable data value
A data value which cannot be modified. Assignments to elements or slices (sub-parts) of immutable values cause a runtime error.

index
A variable or value used to select a member of an ordered collection, such as a character from a string, or an element from a list.

mutable data value
A data value which can be modified. The types of all mutable values are compound types. Lists and dictionaries are mutable; strings and tuples are not.

optional parameter
A parameter written in a function header with an assignment to a default value which it will receive if no corresponding argument is given for it in the function call.

short-circuit evaluation
A style of programming that shortcuts extra work as soon as the outcome is know with certainty. In this chapter our `find` function returned as soon as it found what it was looking for; it didn't traverse all the rest of the items in the string.

slice
A part of a string (substring) specified by a range of indices. More generally, a subsequence of any sequence type in Python can be created using the slice operator (`sequence[start:stop]`).

traverse
To iterate through the elements of a collection, performing a similar operation on each.

whitespace
Any of the characters that move the cursor without printing visible characters. The constant `string.whitespace` contains all the white-space characters.

8.19. Exercises

We suggest you create a single file containing the test scaffolding from our previous chapters, and put all functions that require tests into that file.

1. What is the result of each of the following:

```
>>> "Python"[1]
>>> "Strings are sequences of characters."[5]
>>> len("wonderful")
>>> "Mystery"[:4]
>>> "p" in "Pineapple"
>>> "apple" in "Pineapple"
>>> "pear" not in "Pineapple"
>>> "apple" > "pineapple"
>>> "pineapple" < "Peach"
```

2. Modify:

```
prefixes = "JKLMNOPQ"
suffix = "ack"

for letter in prefixes:
    print(letter + suffix)
```

so that `Ouack` and `Quack` are spelled correctly.

3. Encapsulate

```
fruit = "banana"
count = 0
for char in fruit:
    if char == "a":
        count += 1
print(count)
```

in a function named `count_letters`, and generalize it so that it accepts the string and the letter as arguments. Make the function return the number of characters, rather than print the answer. The caller should do the printing.

4. Now rewrite the count_letters function so that instead of traversing the string, it repeatedly calls the find method, with the optional third parameter to locate new occurrences of the letter being counted.
5. Assign to a variable in your program a triple-quoted string that contains your favourite paragraph of text — perhaps a poem, a speech, instructions to bake a cake, some inspirational verses, etc.

   Write a function which removes all punctuation from the string, breaks the string into a list of words, and counts the number of words in your text that contain the letter "e". Your program should print an analysis of the text like this:

```
Your text contains 243 words, of which 109 (44.8%) contain an "e".
```

6. Print a neat looking multiplication table like this:

```
      1   2   3   4   5   6   7   8   9  10  11  12
   :-----------------------------------------------
 1:    1   2   3   4   5   6   7   8   9  10  11  12
 2:    2   4   6   8  10  12  14  16  18  20  22  24
 3:    3   6   9  12  15  18  21  24  27  30  33  36
 4:    4   8  12  16  20  24  28  32  36  40  44  48
 5:    5  10  15  20  25  30  35  40  45  50  55  60
 6:    6  12  18  24  30  36  42  48  54  60  66  72
 7:    7  14  21  28  35  42  49  56  63  70  77  84
 8:    8  16  24  32  40  48  56  64  72  80  88  96
 9:    9  18  27  36  45  54  63  72  81  90  99 108
10:   10  20  30  40  50  60  70  80  90 100 110 120
11:   11  22  33  44  55  66  77  88  99 110 121 132
12:   12  24  36  48  60  72  84  96 108 120 132 144
```

7. Write a function that reverses its string argument, and satisfies these tests:

```
test(reverse("happy") == "yppah")
test(reverse("Python") == "nohtyP")
test(reverse("") == "")
test(reverse("a") == "a")
```

8. Write a function that mirrors its argument:

```
test(mirror("good") == "gooddoog")
test(mirror("Python") == "PythonnohtyP")
test(mirror("") == "")
test(mirror("a") == "aa")
```

9. Write a function that removes all occurrences of a given letter from a string:

```
test(remove_letter("a", "apple") == "pple")
test(remove_letter("a", "banana") == "bnn")
test(remove_letter("z", "banana") == "banana")
test(remove_letter("i", "Mississippi") == "Mssssppp")
test(remove_letter("b", "") = "")
test(remove_letter("b", "c") = "c")
```

10. Write a function that recognizes palindromes. (Hint: use your `reverse` function to make this easy!):

```
test(is_palindrome("abba"))
test(not is_palindrome("abab"))
test(is_palindrome("tenet"))
test(not is_palindrome("banana"))
test(is_palindrome("straw warts"))
test(is_palindrome("a"))
# test(is_palindrome(""))    # Is an empty string a palindrome?
```

11. Write a function that counts how many times a substring occurs in a string:

```
test(count("is", "Mississippi") == 2)
test(count("an", "banana") == 2)
test(count("ana", "banana") == 2)
test(count("nana", "banana") == 1)
test(count("nanan", "banana") == 0)
test(count("aaa", "aaaaaa") == 4)
```

12. Write a function that removes the first occurrence of a string from another string:

```
test(remove("an", "banana") == "bana")
test(remove("cyc", "bicycle") == "bile")
test(remove("iss", "Mississippi") == "Missippi")
test(remove("eggs", "bicycle") == "bicycle")
```

13. Write a function that removes all occurrences of a string from another string:

```
test(remove_all("an", "banana") == "ba")
test(remove_all("cyc", "bicycle") == "bile")
test(remove_all("iss", "Mississippi") == "Mippi")
test(remove_all("eggs", "bicycle") == "bicycle")
```

Chapter 9: Tuples

9.1. Tuples are used for grouping data

We saw earlier that we could group together pairs of values by surrounding with parentheses. Recall this example:

```
>>> year_born = ("Paris Hilton", 1981)
```

This is an example of a **data structure** - a mechanism for grouping and organizing data to make it easier to use.

The pair is an example of a **tuple**. Generalizing this, a tuple can be used to group any number of items into a single compound value. Syntactically, a tuple is a comma-separated sequence of values. Although it is not necessary, it is conventional to enclose tuples in parentheses:

```
>>> julia = ("Julia", "Roberts", 1967, "Duplicity", 2009, "Actress", "Atlanta, Georg\
ia")
```

Tuples are useful for representing what other languages often call *records* -some related information that belongs together, like your student record. There is no description of what each of these fields means, but we can guess. A tuple lets us ”chunk” together related information and use it as a single thing.

Tuples support the same sequence operations as strings. The index operator selects an element from a tuple.

```
>>> julia[2]
1967
```

But if we try to use item assignment to modify one of the elements of the tuple, we get an error:

```
>>> julia[0] = "X"
TypeError: 'tuple' object does not support item assignment
```

So like strings, tuples are immutable. Once Python has created a tuple in memory, it cannot be changed.

Of course, even if we can’t modify the elements of a tuple, we can always make the `julia` variable reference a new tuple holding different information. To construct the new tuple, it is convenient that we can slice parts of the old tuple and join up the bits to make the new tuple. So if `julia` has a new recent film, we could change her variable to
reference a new tuple that used some information from the old one:

```
>>> julia = julia[:3] + ("Eat Pray Love", 2010) + julia[5:]
>>> julia
("Julia", "Roberts", 1967, "Eat Pray Love", 2010, "Actress", "Atlanta, Georgia")
```

To create a tuple with a single element (but you're probably not likely to do that too often), we have to include the final comma, because without the final comma, Python treats the `(5)` below as an integer in parentheses:

```
>>> tup = (5,)
>>> type(tup)
<class 'tuple'>
>>> x = (5)
>>> type(x)
<class 'int'>
```

9.2. Tuple assignment

Python has a very powerful **tuple assignment** feature that allows a tuple of variables on the left of an assignment to be assigned values from a tuple on the right of the assignment. (We already saw this used for pairs, but it generalizes.)

```
(name, surname, b_year, movie, m_year, profession, b_place) = julia
```

This does the equivalent of seven assignment statements, all on one easy line. One requirement is that the number of variables on the left must match the number of elements in the tuple.

One way to think of tuple assignment is as tuple packing/unpacking.

In tuple packing, the values on the left are 'packed' together in a tuple:

```
>>> b = ("Bob", 19, "CS")    # tuple packing
```

In tuple unpacking, the values in a tuple on the right are 'unpacked' into the variables/names on the left:

```
>>> b = ("Bob", 19, "CS")
>>> (name, age, studies) = b    # tuple unpacking
>>> name
'Bob'
>>> age
19
>>> studies
'CS'
```

Once in a while, it is useful to swap the values of two variables. With conventional assignment statements, we have to use a temporary variable. For example, to swap `a` and `b`:

```
temp = a
a = b
b = temp
```

Tuple assignment solves this problem neatly:

```
(a, b) = (b, a)
```

The left side is a tuple of variables; the right side is a tuple of values. Each value is assigned to its respective variable. All the expressions on the right side are evaluated before any of the assignments. This feature makes tuple assignment quite versatile.

Naturally, the number of variables on the left and the number of values on the right have to be the same:

```
>>> (a, b, c, d) = (1, 2, 3)
ValueError: need more than 3 values to unpack
```

9.3. Tuples as return values

Functions can always only return a single value, but by making that value a tuple, we can effectively group together as many values as we like, and return them together. This is very useful - we often want to know some batsman's highest and lowest score, or we want to find the mean and the standard deviation, or we want to know the year, the month,
and the day, or if we're doing some some ecological modelling we may want to know the number of rabbits and the number of wolves on an island at a given time.

For example, we could write a function that returns both the area and the circumference of a circle of radius r:

```
def f(r):
    """ Return (circumference, area) of a circle of radius r """
    c = 2 * math.pi * r
    a = math.pi * r * r
    return (c, a)
```

9.4. Composability of Data Structures

We saw in an earlier chapter that we could make a list of pairs, and we had an example where one of the items in the tuple was itself a list:

```
 students = [
     ("John", ["CompSci", "Physics"]),
     ("Vusi", ["Maths", "CompSci", "Stats"]),
     ("Jess", ["CompSci", "Accounting", "Economics", "Management"]),
     ("Sarah", ["InfSys", "Accounting", "Economics", "CommLaw"]),
     ("Zuki", ["Sociology", "Economics", "Law", "Stats", "Music"])]
```

Tuples items can themselves be other tuples. For example, we could improve the information about our movie stars to hold the full date of birth rather than just the year, and we could have a list of some of her movies and dates that they were made, and so on:

```
 julia_more_info = ( ("Julia", "Roberts"), (8, "October", 1967),
                     "Actress", ("Atlanta", "Georgia"),
                     [ ("Duplicity", 2009),
                       ("Notting Hill", 1999),
                       ("Pretty Woman", 1990),
                       ("Erin Brockovich", 2000),
                       ("Eat Pray Love", 2010),
                       ("Mona Lisa Smile", 2003),
                       ("Oceans Twelve", 2004) ])
```

Notice in this case that the tuple has just five elements - but each of those in turn can be another tuple, a list, a string, or any other kind of Python value. This property is known as being **heterogeneous**, meaning that it can be composed of elements of different types.

9.5. Glossary

data structure
An organization of data for the purpose of making it easier to use.

immutable data value
A data value which cannot be modified. Assignments to elements or slices (sub-parts) of immutable values cause a runtime error.

mutable data value
A data value which can be modified. The types of all mutable values are compound types. Lists and dictionaries are mutable; strings and tuples are not.

tuple
An immutable data value that contains related elements. Tuples are used to group together related data, such as a person's name, their age, and their gender.

tuple assignment
An assignment to all of the elements in a tuple using a single assignment statement. Tuple assignment occurs *simultaneously* rather than in sequence, making it useful for swapping values.

9.6. Exercises

1. We've said nothing in this chapter about whether you can pass tuples as arguments to a function. Construct a small Python example to test whether this is possible, and write up your findings.
2. Is a pair a generalization of a tuple, or is a tuple a generalization of a pair?
3. Is a pair a kind of tuple, or is a tuple a kind of pair?

Chapter 10: Event handling

10.1. Event-driven programming

Most programs and devices like a cellphone respond to *events* – things that happen. For example, you might move your mouse, and the computer responds. Or you click a button, and the program does something interesting. In this chapter we'll touch very briefly on how event-driven programming works.

10.2. Keypress events

Here's a program with some new features. Copy it into your workspace, run it. When the turtle window opens, press the arrow keys and make tess move about!

```
import turtle

turtle.setup(400,500)                # Determine the window size
wn = turtle.Screen()                 # Get a reference to the window
wn.title("Handling keypresses!")     # Change the window title
wn.bgcolor("lightgreen")             # Set the background color
tess = turtle.Turtle()               # Create our favorite turtle

# The next four functions are our "event handlers".
def h1():
   tess.forward(30)

def h2():
   tess.left(45)

def h3():
   tess.right(45)

def h4():
    wn.bye()                         # Close down the turtle window

# These lines "wire up" keypresses to the handlers we've defined.
wn.onkey(h1, "Up")
```

```
24 wn.onkey(h2, "Left")
25 wn.onkey(h3, "Right")
26 wn.onkey(h4, "q")
27
28 # Now we need to tell the window to start listening for events,
29 # If any of the keys that we're monitoring is pressed, its
30 # handler will be called.
31 wn.listen()
32 wn.mainloop()
```

Here are some points to note:

- We need the call to the window's `listen` method at line 31, otherwise it won't notice our keypresses.
- We named our handler functions `h1`, `h2` and so on, but we can choose better names. The handlers can be arbitrarily complex functions that call other functions, etc.
- Pressing the `q` key on the keyboard calls function `h4` (because we bound the `q` key to `h4` on line 26). While executing `h4`, the window's `bye` method (line 20) closes the turtle window, which causes the window's mainloop call (line 31) to end its execution. Since we did not write any more statements after line 32, this means that our program has completed everything, so it too will terminate.
- We can refer to keys on the keyboard by their character code (as we did in line 26), or by their symbolic names. Some of the symbolic names to try are Cancel (the Break key), BackSpace, Tab, Return(the Enter key), Shift_L (any Shift key), Control_L (any Control key), Alt_L (any Alt key), Pause, Caps_Lock, Escape, Prior (Page Up), Next (Page Down), End, Home, Left, Up, Right, Down, Print, Insert, Delete, F1, F2, F3, F4, F5, F6, F7, F8, F9, F10, F11, F12, Num_Lock, and Scroll_Lock.

10.3. Mouse events

A mouse event is a bit different from a keypress event because its handler needs two parameters to receive x,y coordinate information telling us where the mouse was when the event occurred.

```
1  import turtle
2
3  turtle.setup(400,500)
4  wn = turtle.Screen()
5  wn.title("How to handle mouse clicks on the window!")
6  wn.bgcolor("lightgreen")
7
8  tess = turtle.Turtle()
9  tess.color("purple")
10 tess.pensize(3)
11 tess.shape("circle")
12
```

```
13 def h1(x, y):
14     tess.goto(x, y)
15
16 wn.onclick(h1)  # Wire up a click on the window.
17 wn.mainloop()
```

There is a new turtle method used at line 14 – this allows us to move the turtle to an *absolute* coordinate position. (Most of the examples that we've seen so far move the turtle *relative* to where it currently is). So what this program does is move the turtle (and draw a line) to wherever the mouse is clicked. Try it out!

If we add this line before line 14, we'll learn a useful debugging trick too:

```
1 wn.title("Got click at coords {0}, {1}".format(x, y))
```

Because we can easily change the text in the window's title bar, it is a useful place to display occasional debugging or status information. (Of course, this is not the real purpose of the window title!)

But there is more!

Not only can the window receive mouse events: individual turtles can also have their own handlers for mouse clicks. The turtle that "receives" the click event will be the one under the mouse. So we'll create two turtles. Each will bind a handler to its own `onclick` event. And the two handlers can do different things for their turtles.

```
1  import turtle
2
3  turtle.setup(400,500)              # Determine the window size
4  wn = turtle.Screen()               # Get a reference to the window
5  wn.title("Handling mouse clicks!") # Change the window title
6  wn.bgcolor("lightgreen")           # Set the background color
7  tess = turtle.Turtle()             # Create two turtles
8  tess.color("purple")
9  alex = turtle.Turtle()             # Move them apart
10 alex.color("blue")
11 alex.forward(100)
12
13 def handler_for_tess(x, y):
14     wn.title("Tess clicked at {0}, {1}".format(x, y))
15     tess.left(42)
16     tess.forward(30)
17
18 def handler_for_alex(x, y):
```

```
19      wn.title("Alex clicked at {0}, {1}".format(x, y))
20      alex.right(84)
21      alex.forward(50)
22
23  tess.onclick(handler_for_tess)
24  alex.onclick(handler_for_alex)
25
26  wn.mainloop()
```

Run this, click on the turtles, see what happens!

10.4. Automatic events from a timer

Alarm clocks, kitchen timers, and thermonuclear bombs in James Bond movies are set to create an "automatic" event after a certain interval. The turtle module in Python has a timer that can cause an event when its time is up.

```
1   import turtle
2
3   turtle.setup(400,500)
4   wn = turtle.Screen()
5   wn.title("Using a timer")
6   wn.bgcolor("lightgreen")
7
8   tess = turtle.Turtle()
9   tess.color("purple")
10  tess.pensize(3)
11
12  def h1():
13      tess.forward(100)
14      tess.left(56)
15
16  wn.ontimer(h1, 2000)
17  wn.mainloop()
```

On line 16 the timer is started and set to explode in 2000 milliseconds (2 seconds). When the event does occur, the handler is called, and tess springs into action.

Unfortunately, when one sets a timer, it only goes off once. So a common idiom, or style, is to restart the timer inside the handler. In this way the timer will keep on giving new events. Try this program:

```
import turtle

turtle.setup(400,500)
wn = turtle.Screen()
wn.title("Using a timer to get events!")
wn.bgcolor("lightgreen")

tess = turtle.Turtle()
tess.color("purple")

def h1():
    tess.forward(100)
    tess.left(56)
    wn.ontimer(h1, 60)

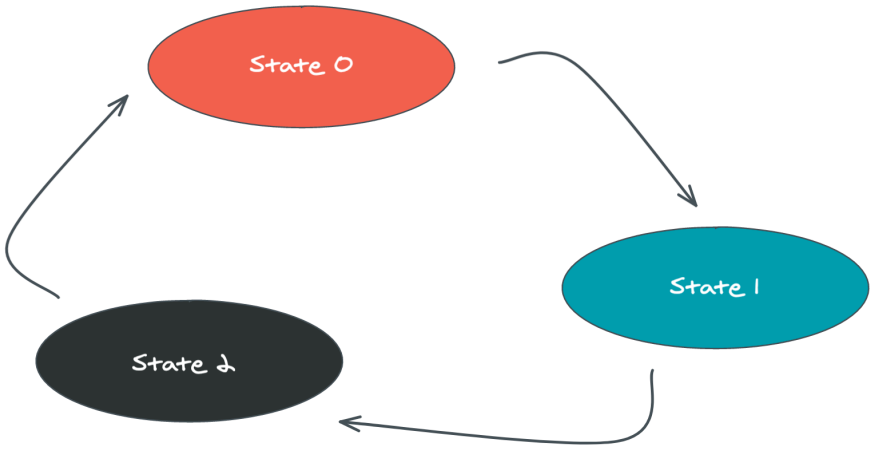
h1()
wn.mainloop()
```

10.5. An example: state machines

A state machine is a system that can be in one of a few different states. We draw a state diagram to represent the machine, where each state is drawn as a circle or an ellipse. Certain events occur which cause the system to leave one state and transition into a different state. These state transitions are usually drawn as an arrow on the diagram.

This idea is not new: when first turning on a cellphone, it goes into a state which we could call "Awaiting PIN". When the correct PIN is entered, it transitions into a different state – say "Ready". Then we could lock the phone, and it would enter a "Locked" state, and so on.

A simple state machine that we encounter often is a traffic light. Here is a state diagram which shows that the machine continually cycles through three different states, which we've numbered 0, 1 and 2.



We're going to build a program that uses a turtle to simulate the traffic lights. There are three lessons here. The first shows off some different ways to use our turtles. The second demonstrates how we would program a state machine in Python, by using a variable to keep track of the current state, and a number of different `if` statements to inspect
the current state, and take the actions as we change to a different state. The third lesson is to use events from the keyboard to trigger the state changes.

Copy and run this program. Make sure you understand what each line does, consulting the documentation as you need to.

```
import turtle           # Tess becomes a traffic light.

turtle.setup(400,500)
wn = turtle.Screen()
wn.title("Tess becomes a traffic light!")
wn.bgcolor("lightgreen")
tess = turtle.Turtle()


def draw_housing():
    """ Draw a nice housing to hold the traffic lights """
    tess.pensize(3)
    tess.color("black", "darkgrey")
    tess.begin_fill()
```

```
    tess.forward(80)
    tess.left(90)
    tess.forward(200)
    tess.circle(40, 180)
    tess.forward(200)
    tess.left(90)
    tess.end_fill()


draw_housing()

tess.penup()
# Position tess onto the place where the green light should be
tess.forward(40)
tess.left(90)
tess.forward(50)
# Turn tess into a big green circle
tess.shape("circle")
tess.shapesize(3)
tess.fillcolor("green")

# A traffic light is a kind of state machine with three states,
# Green, Orange, Red.  We number these states  0, 1, 2
# When the machine changes state, we change tess' position and
# her fillcolor.

# This variable holds the current state of the machine
state_num = 0


def advance_state_machine():
    global state_num
    if state_num == 0:       # Transition from state 0 to state 1
        tess.forward(70)
        tess.fillcolor("orange")
        state_num = 1
    elif state_num == 1:     # Transition from state 1 to state 2
        tess.forward(70)
        tess.fillcolor("red")
        state_num = 2
    else:                    # Transition from state 2 to state 0
        tess.back(140)
        tess.fillcolor("green")
```

```
58          state_num = 0
59
60  # Bind the event handler to the space key.
61  wn.onkey(advance_state_machine, "space")
62
63  wn.listen()                      # Listen for events
64  wn.mainloop()
```

The new Python statement is at line 46. The `global` keyword tells Python not to create a new local variable for `state_num` (in spite of the fact that the function assigns to this variable at lines 50, 54, and 58). Instead, in this function, `state_num` always refers to the variable that was created at line 42.

What the code in `advance_state_machine` does is advance from whatever the current state is, to the next state. On the state change we move tess to her new position, change her color, and, of course, we assign to `state_num` the number of the new state we've just entered.

Each time the space bar is pressed, the event handler causes the traffic light machine to move to its new state.

10.6. Glossary

bind
We bind a function (or associate it) with an event, meaning that when the event occurs, the function is called to handle it.

event
Something that happens "outside" the normal control flow of our program, usually from some user action. Typical events are mouse operations and keypresses. We've also seen that a timer can be primed to create an event.

handler
A function that is called in response to an event.

10.7. Exercises

1. Add some new key bindings to the first sample program:

- Pressing keys R, G or B should change tess's color to Red, Green or Blue.
- Pressing keys + or - should increase or decrease the width of tess's pen. Ensure that the pen size stays between 1 and 20 (inclusive).
- Handle some other keys to change some attributes of tess, or attributes of the window, or to give her new behaviour that can be controlled from the keyboard.

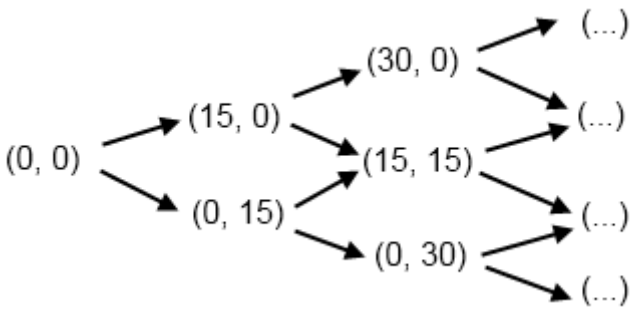
2. Change the traffic light program so that changes occur automatically, driven by a timer.

3. In an earlier chapter we saw two turtle methods, `hideturtle` and `showturtle` that can hide or show a turtle. This suggests that we could take a different approach to the traffic lights program. Add to your program above as follows: draw a second housing for another set of traffic lights. Create three separate turtles to represent each of the green, orange and red lights, and position them appropriately within your new housing. As your state changes occur, just make one of the three turtles visible at any time. Once you've made the changes, sit back and ponder some deep thoughts: you've now got two different ways to use turtles to simulate the traffic lights, and both seem to work. Is one approach somehow preferable to the other? Which one more closely resembles reality – i.e. the traffic lights in your town?

4. Now that you've got a traffic light program with different turtles for each light, perhaps the visibility / invisibility trick wasn't such a great idea. If we watch traffic lights, they turn on and off – but when they're off they are still there, perhaps just a darker color. Modify the program now so that the lights don't
disappear: they are either on, or off. But when they're off, they're still visible.

5. Your traffic light controller program has been patented, and you're about to become seriously rich. But your new client needs a change. They want four states in their state machine: Green, then Green and Orange together, then Orange only, and then Red. Additionally, they want different times spent in each state. The machine should spend 3
seconds in the Green state, followed by one second in the Green+Orange state, then one second in the Orange state, and then 2 seconds in the Red state. Change the logic in the state machine.

6. If you don't know how tennis scoring works, ask a friend or consult Wikipedia. A single game in tennis between player A and player B always has a score. We want to think about the "state of the score" as a state machine. The game starts in state (0, 0), meaning neither player has any score yet. We'll assume the first element in this pair is the score for player A. If player A wins the first point, the score becomes (15, 0). If B wins the first point, the state becomes (0, 15). Below are the first few states and transitions for a state diagram. In this diagram, each state has two possible outcomes (A wins the next point, or B does), and the uppermost arrow is always the transition that happens when A wins the point. Complete the diagram, showing all transitions and all states. (Hint: there are twenty states, if you include the duece state, the advantage states, and the "A wins" and "B wins" states in your diagram.)


(0, 0)
(15, 0)
(0, 15)
(30, 0)
(15, 15)
(0, 30)
(...)
(...)
(...)
(...)

Chapter 11: Lists

A **list** is an ordered collection of values. The values that make up a list are called its **elements**, or its **items**. We will use the term element or item to mean the same thing. Lists are similar to strings, which are ordered collections of characters, except that the elements of a list can be of any type. Lists and strings — and other collections that maintain the order of their items — are called **sequences**.

11.1. List values

There are several ways to create a new list; the simplest is to enclose the elements in square brackets (`[` and `]`):

```
ps = [10, 20, 30, 40]
qs = ["spam", "bungee", "swallow"]
```

The first example is a list of four integers. The second is a list of three strings. The elements of a list don't have to be the same type. The following list contains a string, a float, an integer, and (amazingly) another list:

```
zs = ["hello", 2.0, 5, [10, 20]]
```

A list within another list is said to be **nested**.

Finally, a list with no elements is called an empty list, and is denoted `[]`.

We have already seen that we can assign list values to variables or pass lists as parameters to functions:

```
>>> vocabulary = ["apple", "cheese", "dog"]
>>> numbers = [17, 123]
>>> an_empty_list = []
>>> print(vocabulary, numbers, an_empty_list)
["apple", "cheese", "dog"] [17, 123] []
```

11.2. Accessing elements

The syntax for accessing the elements of a list is the same as the syntax for accessing the characters of a string — the index operator: `[]` (not to be confused with an empty list). The expression inside the
brackets specifies the index. Remember that the indices start at 0:

```
>>> numbers[0]
17
```

Any expression evaluating to an integer can be used as an index:

```
>>> numbers[9-8]
5
>>> numbers[1.0]
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
TypeError: list indices must be integers, not float
```

If you try to access or assign to an element that does not exist, you get a runtime error:

```
>>> numbers[2]
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
IndexError: list index out of range
```

It is common to use a loop variable as a list index.

```
horsemen = ["war", "famine", "pestilence", "death"]

for i in [0, 1, 2, 3]:
    print(horsemen[i])
```

Each time through the loop, the variable `i` is used as an index into the list, printing the `i`'th element. This pattern of computation is called a **list traversal**.

The above sample doesn't need or use the index `i` for anything besides getting the items from the list, so this more direct version — where the `for` loop gets the items — might be preferred:

```
horsemen = ["war", "famine", "pestilence", "death"]

for h in horsemen:
    print(h)
```

11.3. List length

The function `len` returns the length of a list, which is equal to the number of its elements. If you are going to use an integer index to access the list, it is a good idea to use this value as the upper bound of a loop instead of a constant. That way, if the size of the list changes, you won't have to go through the program changing all the loops; they will work correctly for any size list:

```
horsemen = ["war", "famine", "pestilence", "death"]

for i in range(len(horsemen)):
    print(horsemen[i])
```

The last time the body of the loop is executed, `i` is `len(horsemen) - 1`, which is the index of the last element. (But the version without the index looks even better now!)

Although a list can contain another list, the nested list still counts as a single element in its parent list. The length of this list is 4:

```
>>> len(["car makers", 1, ["Ford", "Toyota", "BMW"], [1, 2, 3]])
4
```

11.4. List membership

`in` and `not in` are Boolean operators that test membership in a sequence. We used them previously with strings, but they also work with lists and other sequences:

```
>>> horsemen = ["war", "famine", "pestilence", "death"]
>>> "pestilence" in horsemen
True
>>> "debauchery" in horsemen
False
>>> "debauchery" not in horsemen
True
```

Using this produces a more elegant version of the nested loop program we previously used to count the number of students doing Computer Science in the section `nested_data`:

```
students = [
    ("John", ["CompSci", "Physics"]),
    ("Vusi", ["Maths", "CompSci", "Stats"]),
    ("Jess", ["CompSci", "Accounting", "Economics", "Management"]),
    ("Sarah", ["InfSys", "Accounting", "Economics", "CommLaw"]),
    ("Zuki", ["Sociology", "Economics", "Law", "Stats", "Music"])]

# Count how many students are taking CompSci
counter = 0
for (name, subjects) in students:
    if "CompSci" in subjects:
            counter += 1

print("The number of students taking CompSci is", counter)
```

11.5. List operations

The + operator concatenates lists:

```
>>> a = [1, 2, 3]
>>> b = [4, 5, 6]
>>> c = a + b
>>> c
[1, 2, 3, 4, 5, 6]
```

Similarly, the * operator repeats a list a given number of times:

```
>>> [0] * 4
[0, 0, 0, 0]
>>> [1, 2, 3] * 3
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

The first example repeats `[0]` four times. The second example repeats the list `[1, 2, 3]` three times.

11.6. List slices

The slice operations we saw previously with strings let us work with sublists:

```
>>> a_list = ["a", "b", "c", "d", "e", "f"]
>>> a_list[1:3]
['b', 'c']
>>> a_list[:4]
['a', 'b', 'c', 'd']
>>> a_list[3:]
['d', 'e', 'f']
>>> a_list[:]
['a', 'b', 'c', 'd', 'e', 'f']
```

11.7. Lists are mutable

Unlike strings, lists are **mutable**, which means we can change their elements. Using the index operator on the left side of an assignment, we can update one of the elements:

```
>>> fruit = ["banana", "apple", "quince"]
>>> fruit[0] = "pear"
>>> fruit[2] = "orange"
>>> fruit
['pear', 'apple', 'orange']
```

The bracket operator applied to a list can appear anywhere in an expression. When it appears on the left side of an assignment, it changes one of the elements in the list, so the first element of `fruit` has been changed from `"banana"` to `"pear"`, and the last from `"quince"` to `"orange"`. An assignment to an element of a list is called **item assignment**. Item assignment does not work for strings:

```
>>> my_string = "TEST"
>>> my_string[2] = "X"
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

but it does for lists:

```
>>> my_list = ["T", "E", "S", "T"]
>>> my_list[2] = "X"
>>> my_list
['T', 'E', 'X', 'T']
```

With the slice operator we can update a whole sublist at once:

```
>>> a_list = ["a", "b", "c", "d", "e", "f"]
>>> a_list[1:3] = ["x", "y"]
>>> a_list
['a', 'x', 'y', 'd', 'e', 'f']
```

We can also remove elements from a list by assigning an empty list to them:

```
>>> a_list = ["a", "b", "c", "d", "e", "f"]
>>> a_list[1:3] = []
>>> a_list
['a', 'd', 'e', 'f']
```

And we can add elements to a list by squeezing them into an empty slice at the desired location:

```
>>> a_list = ["a", "d", "f"]
>>> a_list[1:1] = ["b", "c"]
>>> a_list
['a', 'b', 'c', 'd', 'f']
>>> a_list[4:4] = ["e"]
>>> a_list
['a', 'b', 'c', 'd', 'e', 'f']
```

11.8. List deletion

Using slices to delete list elements can be error-prone. Python provides an alternative that is more readable. The `del` statement removes an element from a list:

```
>>> a = ["one", "two", "three"]
>>> del a[1]
>>> a
['one', 'three']
```

As you might expect, `del` causes a runtime error if the index is out of range.

You can also use `del` with a slice to delete a sublist:

```
>>> a_list = ["a", "b", "c", "d", "e", "f"]
>>> del a_list[1:5]
>>> a_list
['a', 'f']
```

As usual, the sublist selected by slice contains all the elements up to, but not including, the second index.

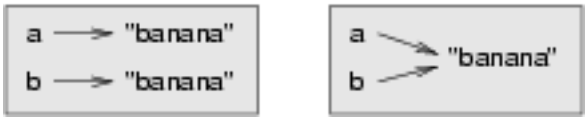
11.9. Objects and references

After we execute these assignment statements

```
a = "banana"
b = "banana"
```

we know that `a` and `b` will refer to a string object with the letters `"banana"`. But we don't know yet whether they point to the *same* string object.

There are two possible ways the Python interpreter could arrange its memory:



In one case, `a` and `b` refer to two different objects that have the same value. In the second case, they refer to the same object.

We can test whether two names refer to the same object using the `is` operator:

```
>>> a is b
True
```

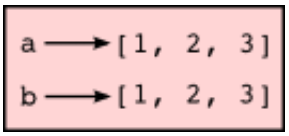
This tells us that both `a` and `b` refer to the same object, and that it is the second of the two state snapshots that accurately describes the relationship.

Since strings are *immutable*, Python optimizes resources by making two names that refer to the same string value refer to the same object.

This is not the case with lists:

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> a == b
True
>>> a is b
False
```

The state snapshot here looks like this:



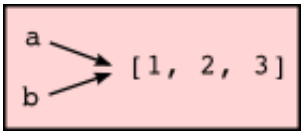
`a` and `b` have the same value but do not refer to the same object.

11.10. Aliasing

Since variables refer to objects, if we assign one variable to another, both variables refer to the same object:

```
>>> a = [1, 2, 3]
>>> b = a
>>> a is b
True
```

In this case, the state snapshot looks like this:



Because the same list has two different names, `a` and `b`, we say that it is **aliased**. Changes made with one alias affect the other:

```
>>> b[0] = 5
>>> a
[5, 2, 3]
```

Although this behavior can be useful, it is sometimes unexpected or undesirable. In general, it is safer to avoid aliasing when you are working with mutable objects (i.e. lists at this point in our textbook, but we'll meet more mutable objects as we cover classes and objects, dictionaries and sets). Of course, for immutable objects (i.e. strings, tuples), there's no problem — it is just not possible to change something and get a surprise when you access an alias name. That's why Python is free to alias strings (and any other immutable kinds of data) when it sees an opportunity to economize.

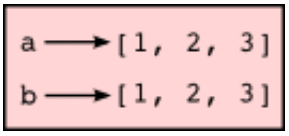
11.11. Cloning lists

If we want to modify a list and also keep a copy of the original, we need to be able to make a copy of the list itself, not just the reference. This process is sometimes called **cloning**, to avoid the ambiguity of the word copy.

The easiest way to clone a list is to use the slice operator:

```
>>> a = [1, 2, 3]
>>> b = a[:]
>>> b
[1, 2, 3]
```

Taking any slice of `a` creates a new list. In this case the slice happens to consist of the whole list. So now the relationship is like this:



Now we are free to make changes to `b` without worrying that we'll inadvertently be changing `a`:

```
>>> b[0] = 5
>>> a
[1, 2, 3]
```

11.12. Lists and `for` loops

The `for` loop also works with lists, as we've already seen. The generalized syntax of a `for` loop is:

```
for VARIABLE in LIST:
    BODY
```

So, as we've seen

```
friends = ["Joe", "Zoe", "Brad", "Angelina", "Zuki", "Thandi", "Paris"]
for friend in friends:
    print(friend)
```

It almost reads like English: For (every) friend in (the list of) friends, print (the name of the) friend.

Any list expression can be used in a `for` loop:

```
for number in range(20):
    if number % 3 == 0:
        print(number)

for fruit in ["banana", "apple", "quince"]:
    print("I like to eat " + fruit + "s!")
```

The first example prints all the multiples of 3 between 0 and 19. The second example expresses enthusiasm for various fruits.

Since lists are mutable, we often want to traverse a list, changing each of its elements. The following squares all the numbers in the list `xs`:

```
xs = [1, 2, 3, 4, 5]

for i in range(len(xs)):
    xs[i] = xs[i]**2
```

Take a moment to think about `range(len(xs))` until you understand how it works.

In this example we are interested in both the *value* of an item, (we want to square that value), and its *index* (so that we can assign the new value to that position). This pattern is common enough that Python
provides a nicer way to implement it:

```
xs = [1, 2, 3, 4, 5]

for (i, val) in enumerate(xs):
    xs[i] = val**2
```

`enumerate` generates pairs of both (index, value) during the list traversal. Try this next example to see more clearly how `enumerate` works:

```
for (i, v) in enumerate(["banana", "apple", "pear", "lemon"]):
     print(i, v)
```

```
0 banana
1 apple
2 pear
3 lemon
```

11.13. List parameters

Passing a list as an argument actually passes a reference to the list, not a copy or clone of the list. So parameter passing creates an alias for you: the caller has one variable referencing the list, and the called function has an alias, but there is only one underlying list object. For example, the function below takes a list as an argument and multiplies each element in the list by 2:

```
def double_stuff(a_list):
    """ Overwrite each element in a_list with double its value. """
    for (idx, val) in enumerate(a_list):
        a_list[idx] = 2 * val
```

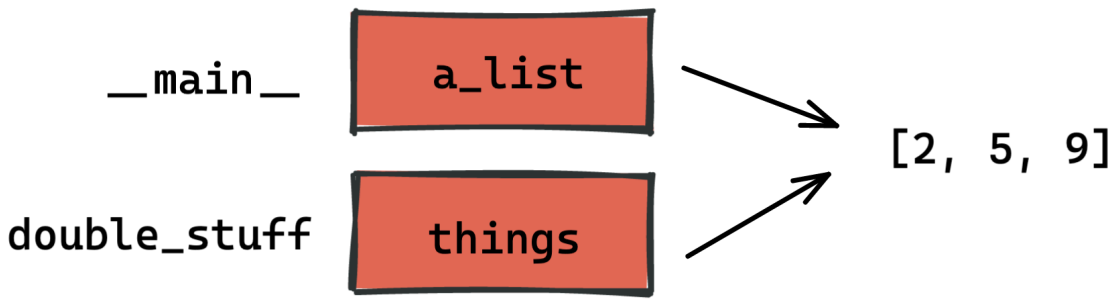
If we add the following onto our script:

```
things = [2, 5, 9]
double_stuff(things)
print(things)
```

When we run it we'll get:

```
[4, 10, 18]
```

In the function above, the parameter `a_list` and the variable `things` are aliases for the same object. So before any changes to the elements in the list, the state snapshot looks like this:



Since the list object is shared by two frames, we drew it between them.

If a function modifies the items of a list parameter, the caller sees the change.

Use the Python visualizer!

We've already mentioned the Python visualizer at http://pythontutor.com. It is a very useful tool for building a good understanding of references, aliases, assignments, and passing arguments to functions. Pay special attention to cases where you clone a list or have two separate lists, and cases where there is only one underlying list, but more than one variable is aliased to reference the list.

11.14. List methods

The dot operator can also be used to access built-in methods of list objects. We'll start with the most useful method for adding something onto the end of an existing list:

```
>>> mylist = []
>>> mylist.append(5)
>>> mylist.append(27)
>>> mylist.append(3)
>>> mylist.append(12)
>>> mylist
[5, 27, 3, 12]
```

append is a list method which adds the argument passed to it to the end of the list. We'll use it heavily when we're creating new lists. Continuing with this example, we show several other list methods:

```
>>> mylist.insert(1, 12)  # Insert 12 at pos 1, shift other items up
>>> mylist
[5, 12, 27, 3, 12]
>>> mylist.count(12)       # How many times is 12 in mylist?
2
>>> mylist.extend([5, 9, 5, 11])   # Put whole list onto end of mylist
>>> mylist
[5, 12, 27, 3, 12, 5, 9, 5, 11])
>>> mylist.index(9)                # Find index of first 9 in mylist
6
>>> mylist.reverse()
>>> mylist
[11, 5, 9, 5, 12, 3, 27, 12, 5]
>>> mylist.sort()
>>> mylist
[3, 5, 5, 5, 9, 11, 12, 12, 27]
>>> mylist.remove(12)             # Remove the first 12 in the list
>>> mylist
[3, 5, 5, 5, 9, 11, 12, 27]
```

Experiment and play with the list methods shown here, and read their documentation until you feel confident that you understand how they work.

11.15. Pure functions and modifiers

Functions which take lists as arguments and change them during execution are called **modifiers** and the changes they make are called **side effects**.

A **pure function** does not produce side effects. It communicates with the calling program only through parameters, which it does not modify, and a return value. Here is `double_stuff` written as a pure function:

```
def double_stuff(a_list):
    """ Return a new list which contains
        doubles of the elements in a_list.
    """
    new_list = []
    for value in a_list:
        new_elem = 2 * value
        new_list.append(new_elem)

    return new_list
```

This version of `double_stuff` does not change its arguments:

```
>>> things = [2, 5, 9]
>>> xs = double_stuff(things)
>>> things
[2, 5, 9]
>>> xs
[4, 10, 18]
```

An early rule we saw for assignment said "first evaluate the right hand side, then assign the resulting value to the variable". So it is quite safe to assign the function result to the same variable that was passed to the function:

```
>>> things = [2, 5, 9]
>>> things = double_stuff(things)
>>> things
[4, 10, 18]
```

Which style is better?

Anything that can be done with modifiers can also be done with pure functions. In fact, some programming languages only allow pure functions. There is some evidence that programs that use pure functions are faster to develop and less error-prone than programs that use modifiers. Nevertheless, modifiers are convenient at times, and in some cases, functional programs are less efficient.

In general, we recommend that you write pure functions whenever it is reasonable to do so and resort to modifiers only if there is a compelling advantage. This approach might be called a *functional programming style*.

11.16. Functions that produce lists

The pure version of `double_stuff` above made use of an important **pattern** for your toolbox. Whenever you need to write a function that creates and returns a list, the pattern is usually:

```
initialize a result variable to be an empty list
loop
   create a new element
   append it to result
return the result
```

Let us show another use of this pattern. Assume you already have a function `is_prime(x)` that can test if x is prime. Write a function to return a list of all prime numbers less than n:

```
def primes_lessthan(n):
    """ Return a list of all prime numbers less than n. """
    result = []
    for i in range(2, n):
        if is_prime(i):
            result.append(i)
    return result
```

11.17. Strings and lists

Two of the most useful methods on strings involve conversion to and from lists of substrings. The `split` method (which we've already seen) breaks a string into a list of words. By default, any number of whitespace characters is considered a word boundary:

```
>>> song = "The rain in Spain..."
>>> wds = song.split()
>>> wds
['The', 'rain', 'in', 'Spain...']
```

An optional argument called a **delimiter** can be used to specify which string to use as the boundary marker between substrings. The following example uses the string `ai` as the delimiter:

```
>>> song.split("ai")
['The r', 'n in Sp', 'n...']
```

Notice that the delimiter doesn't appear in the result.

The inverse of the `split` method is `join`. You choose a desired **separator** string, (often called the *glue*) and join the list with the glue between each of the elements:

```
>>> glue = ";"
>>> s = glue.join(wds)
>>> s
'The;rain;in;Spain...'
```

The list that you glue together (`wds` in this example) is not modified. Also, as these next examples show, you can use empty glue or multi-character strings as glue:

```
>>> " --- ".join(wds)
'The --- rain --- in --- Spain...'
>>> "".join(wds)
'TherainінSpain...'
```

11.18. `list` and `range`

Python has a built-in type conversion function called `list` that tries to turn whatever you give it into a list.

```
>>> xs = list("Crunchy Frog")
>>> xs
["C", "r", "u", "n", "c", "h", "y", " ", "F", "r", "o", "g"]
>>> "".join(xs)
'Crunchy Frog'
```

One particular feature of `range` is that it doesn't instantly compute all its values: it "puts off" the computation, and does it on demand, or "lazily". We'll say that it gives a **promise** to produce the values
when they are needed. This is very convenient if your computation short-circuits a search and returns early, as in this case:

```
def f(n):
    """ Find the first positive integer between 101 and less
        than n that is divisible by 21
    """
    for i in range(101, n):
       if (i % 21 == 0):
           return i


test(f(110) == 105)
test(f(1000000000) == 105)
```

In the second test, if range were to eagerly go about building a list with all those elements, you would soon exhaust your computer's available memory and crash the program. But it is cleverer than that! This computation works just fine, because the `range` object is just a promise to produce the elements if and when they are needed. Once the condition in the `if` becomes true, no further elements are generated, and the function returns. (Note: Before Python 3, `range` was not lazy. If you use an earlier versions of Python, YMMV!)

YMMV: Your Mileage May Vary

The acronym YMMV stands for *your mileage may vary*. American car advertisements often quoted fuel consumption figures for cars, e.g. that they would get 28 miles per gallon. But this always had to be accompanied by legal small-print warning the reader that they might not get the same. The term YMMV is now used idiomatically to mean "your results may differ", e.g. *The battery life on this phone is 3 days, but YMMV.*

You'll sometimes find the lazy `range` wrapped in a call to `list`. This forces Python to turn the lazy promise into an actual list:

```
>>> range(10)           # Create a lazy promise
range(0, 10)
>>> list(range(10))     # Call in the promise, to produce a list.
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

11.19. Nested lists

A nested list is a list that appears as an element in another list. In this list, the element with index 3 is a nested list:

```
>>> nested = ["hello", 2.0, 5, [10, 20]]
```

If we output the element at index 3, we get:

```
>>> print(nested[3])
[10, 20]
```

To extract an element from the nested list, we can proceed in two steps:

```
>>> elem = nested[3]
>>> elem[0]
10
```

Or we can combine them:

```
>>> nested[3][1]
20
```

Bracket operators evaluate from left to right, so this expression gets the 3'th element of `nested` and extracts the 1'th element from it.

11.20. Matrices

Nested lists are often used to represent matrices. For example, the matrix:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

might be represented as:

```
>>> mx = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

`mx` is a list with three elements, where each element is a row of the matrix. We can select an entire row from the matrix in the usual way:

```
>>> mx[1]
[4, 5, 6]
```

Or we can extract a single element from the matrix using the double-index form:

```
>>> mx[1][2]
6
```

The first index selects the row, and the second index selects the column. Although this way of representing matrices is common, it is not the only possibility. A small variation is to use a list of columns instead of a list of rows. Later we will see a more radical alternative using a dictionary.

11.21. Glossary

aliases
Multiple variables that contain references to the same object.

clone
To create a new object that has the same value as an existing object.

Copying a reference to an object creates an alias but doesn't clone the object.

delimiter
A character or string used to indicate where a string should be split.

element
One of the values in a list (or other sequence). The bracket operator selects elements of a list. Also called *item*.

immutable data value
A data value which cannot be modified. Assignments to elements or slices (sub-parts) of immutable values cause a runtime error.

index
An integer value that indicates the position of an item in a list. Indexes start from `0`.

item
See *element*.

list
A collection of values, each in a fixed position within the list. Like other types `str`, `int`, `float`, etc. there is also a `list` type-converter function that tries to turn whatever argument you give it into a list.

list traversal
The sequential accessing of each element in a list.

modifier
A function which changes its arguments inside the function body. Only mutable types can be changed by modifiers.

mutable data value
A data value which can be modified. The types of all mutable values are compound types. Lists and dictionaries are mutable; strings and tuples are not.

nested list
A list that is an element of another list.

object
A thing to which a variable can refer.

pattern
A sequence of statements, or a style of coding something that has general applicability in a number of different situations. Part of becoming a mature Computer Scientist is to learn and establish the

patterns and algorithms that form your toolkit. Patterns often correspond to your "mental chunking".

promise
An object that promises to do some work or deliver some values if they're eventually needed, but it lazily puts off doing the work immediately. Calling `range` produces a promise.

pure function
A function which has no side effects. Pure functions only make changes to the calling program through their return values.

sequence
Any of the data types that consist of an ordered collection of elements, with each element identified by an index.

side effect
A change in the state of a program made by calling a function. Side effects can only be produced by modifiers.

step size
The interval between successive elements of a linear sequence. The third (and optional argument) to the `range` function is called the step size. If not specified, it defaults to 1.

11.22. Exercises

1. What is the Python interpreter's response to the following?

```
>>> list(range(10, 0, -2))
```

The three arguments to the *range* function are *start*, *stop*, and *step*, respectively. In this example, `start` is greater than `stop`. What happens if `start < stop` and `step < 0`? Write a rule for the relationships among `start`, `stop`, and `step`.

2. Consider this fragment of code:

```
import turtle

tess = turtle.Turtle()
alex = tess
alex.color("hotpink")
```

Does this fragment create one or two turtle instances? Does setting the color of `alex` also change the color of `tess`? Explain in detail.

3. Draw a state snapshot for `a` and `b` before and after the third line of the following Python code is executed:

```
a = [1, 2, 3]
b = a[:]
b[0] = 5
```

4. What will be the output of the following program?

```
this = ["I", "am", "not", "a", "crook"]
that = ["I", "am", "not", "a", "crook"]
print("Test 1: {0}".format(this is that))
that = this
print("Test 2: {0}".format(this is that))
```

Provide a *detailed* explanation of the results.

5. Lists can be used to represent mathematical *vectors*. In this exercise and several that follow you will

write functions to perform standard operations on vectors. Create a script named `vectors.py` and write Python code to pass the tests in each case.

Write a function `add_vectors(u, v)` that takes two lists of numbers of the same length, and returns a new list containing the sums of the corresponding elements of each:

```
test(add_vectors([1, 1], [1, 1]) == [2, 2])
test(add_vectors([1, 2], [1, 4]) == [2, 6])
test(add_vectors([1, 2, 1], [1, 4, 3]) == [2, 6, 4])
```

6. Write a function `scalar_mult(s, v)` that takes a number, `s`, and a list, `v` and returns the scalar multiple[10] of `v` by `s`:

[10] http://en.wikipedia.org/wiki/Scalar_multiple

```
test(scalar_mult(5, [1, 2]) == [5, 10])
test(scalar_mult(3, [1, 0, -1]) == [3, 0, -3])
test(scalar_mult(7, [3, 0, 5, 11, 2]) == [21, 0, 35, 77, 14])
```

7. Write a function `dot_product(u, v)` that takes two lists of numbers of the same length, and returns the

sum of the products of the corresponding elements of each (the dot_product[11]).

```
test(dot_product([1, 1], [1, 1]) ==  2)
test(dot_product([1, 2], [1, 4]) ==  9)
test(dot_product([1, 2, 1], [1, 4, 3]) == 12)
```

8. **Extra challenge for the mathematically inclined**: Write a function `cross_product(u, v)` that takes two

lists of numbers of length 3 and returns their cross product[12]. You should write your own tests.

9. Describe the relationship between `" ".join(song.split())` and `song` in the fragment of code below. Are

they the same for all strings assigned to `song`? When would they be different?

```
song = "The rain in Spain..."
```

10. Write a function `replace(s, old, new)` that replaces all occurrences of `old` with `new` in a string `s`:

```
test(replace("Mississippi", "i", "I") == "MIssIssIppI")

s = "I love spom! Spom is my favorite food. Spom, spom, yum!"
test(replace(s, "om", "am") ==
    "I love spam! Spam is my favorite food. Spam, spam, yum!")

test(replace(s, "o", "a") ==
    "I lave spam! Spam is my favarite faad. Spam, spam, yum!")
```

[11] http://en.wikipedia.org/wiki/Dot_product
[12] http://en.wikipedia.org/wiki/Cross_product

Hint: use the `split` and `join` methods.

11. Suppose you want to swap around the values in two variables. You decide to factor this out into a

reusable function, and write this code:

```
def swap(x, y):      # Incorrect version
    print("before swap statement: x:", x, "y:", y)
    (x, y) = (y, x)
    print("after swap statement: x:", x, "y:", y)

a = ["This", "is", "fun"]
b = [2,3,4]
print("before swap function call: a:", a, "b:", b)
swap(a, b)
print("after swap function call: a:", a, "b:", b)
```

Run this program and describe the results. Oops! So it didn't do what you intended! Explain why not. Using a Python visualizer like the one at http://pythontutor.com may help you build a good conceptual model of what is going on. What will be the values of `a` and `b` after the call to `swap`?

Chapter 12: Modules

A **module** is a file containing Python definitions and statements intended for use in other Python programs. There are many Python modules that come with Python as part of the **standard library**. We have seen at least two of these already, the `turtle` module and the `string` module.

We have also shown you how to access help. The help system contains a listing of all the standard modules that are available with Python. Play with help!

12.1. Random numbers

We often want to use random numbers in programs, here are a few typical uses:

- To play a game of chance where the computer needs to throw some dice, pick a number, or flip a coin
- To shuffle a deck of playing cards randomly,
- To allow/make an enemy spaceship appear at a random location and start shooting at the player
- To simulate possible rainfall when we make a computerized model for estimating the environmental impact

of building a dam,
- For encrypting banking sessions on the Internet.

Python provides a module `random` that helps with tasks like this. You can look it up using help, but here are the key things we'll do with it:

```
1  import random
2
3  # Create a black box object that generates random numbers
4  rng = random.Random()
5
6  dice_throw = rng.randrange(1,7)   # Return an int, one of 1,2,3,4,5,6
7  delay_in_seconds = rng.random() * 5.0
```

The `randrange` method call generates an integer between its lower and upper argument, using the same semantics as `range` — so the lower bound is included, but the upper bound is excluded. All the values have
an equal probability of occurring (i.e. the results are *uniformly* distributed). Like `range`, `randrange` can also take an optional step argument. So let's assume we needed a random odd number less than 100, we could say:

```
r_odd = rng.randrange(1, 100, 2)
```

Other methods can also generate other distributions e.g. a bell-shaped, or "normal" distribution might be more appropriate for estimating seasonal rainfall, or the concentration of a compound in the body after taking a dose of medicine.

The `random` method returns a floating point number in the interval [0.0, 1.0) — the square bracket means "closed interval on the left" and the round parenthesis means "open interval on the right". In other words, 0.0 is possible, but all returned numbers will be strictly less than 1.0. It is usual to *scale* the results after calling this method, to get them into an interval suitable for your application. In the case shown here, we've converted the result of the method call to a number in the interval [0.0, 5.0). Once more, these are uniformly distributed numbers — numbers close to 0 are just as likely to occur as numbers close to 0.5, or numbers close to 1.0.

This example shows how to shuffle a list. (`shuffle` cannot work directly with a lazy promise, so notice that we had to convert the range object using the `list` type converter first.)

```
cards = list(range(52))  # Generate ints [0 .. 51]
                         # representing a pack of cards.
rng.shuffle(cards)       # Shuffle the pack
```

12.1.1. Repeatability and Testing

Random number generators are based on a **deterministic** algorithm — repeatable and predictable. So they're called **pseudo-random** generators — they are not genuinely random. They start with a *seed* value. Each time you ask for another random number, you'll get one based on the current seed attribute, and the state of the seed (which is one of the attributes of the generator) will be updated.

For debugging and for writing unit tests, it is convenient to have repeatability — programs that do the same thing every time they are run. We can arrange this by forcing the random number generator to be initialized with a known seed every time. (Often this is only wanted during testing — playing a game of cards where the shuffled deck was always in the same order as last time you played would get boring very
rapidly!)

```
drng = random.Random(123)  # Create generator with known starting state
```

This alternative way of creating a random number generator gives an explicit seed value to the object. Without this argument, the system probably uses something based on the time. So grabbing some random numbers from `drng` today will give you precisely the same random sequence as it will tomorrow!

12.1.2. Picking balls from bags, throwing dice, shuffling a pack of cards

Here is an example to generate a list containing n random ints between a lower and an upper bound:

```
import random

def make_random_ints(num, lower_bound, upper_bound):
    """
      Generate a list containing num random ints between lower_bound
      and upper_bound. upper_bound is an open bound.
    """
    rng = random.Random()  # Create a random number generator
    result = []
    for i in range(num):
        result.append(rng.randrange(lower_bound, upper_bound))
    return result
```

```
>>> make_random_ints(5, 1, 13)  # Pick 5 random month numbers
[8, 1, 8, 5, 6]
```

Notice that we got a duplicate in the result. Often this is wanted, e.g. if we throw a die five times, we would expect some duplicates.

But what if you don't want duplicates? If you wanted 5 distinct months, then this algorithm is wrong. In this case a good algorithm is to generate the list of possibilities, shuffle it, and slice off the number of elements you want:

```
xs = list(range(1,13))  # Make list 1..12  (there are no duplicates)
rng = random.Random()   # Make a random number generator
rng.shuffle(xs)         # Shuffle the list
result = xs[:5]         # Take the first five elements
```

In statistics courses, the first case — allowing duplicates — is usually described as pulling balls out of a bag *with replacement* — you put the drawn ball back in each time, so it can occur again. The latter case, with no duplicates, is usually described as pulling balls out of the bag *without replacement*. Once the ball is drawn, it doesn't go back to be drawn again. TV lotto games work like this.

The second "shuffle and slice" algorithm would not be so great if you only wanted a few elements, but from a very large domain. Suppose I wanted five numbers between one and ten million, without duplicates. Generating a list of ten million items, shuffling it, and then slicing off the first five would be a performance disaster! So let us have another try:

```
import random

def make_random_ints_no_dups(num, lower_bound, upper_bound):
    """
      Generate a list containing num random ints between
      lower_bound and upper_bound. upper_bound is an open bound.
      The result list cannot contain duplicates.
    """
    result = []
    rng = random.Random()
    for i in range(num):
         while True:
             candidate = rng.randrange(lower_bound, upper_bound)
             if candidate not in result:
                 break
         result.append(candidate)
    return result

xs = make_random_ints_no_dups(5, 1, 10000000)
print(xs)
```

This agreeably produces 5 random numbers, without duplicates:

```
[3344629, 1735163, 9433892, 1081511, 4923270]
```

Even this function has its pitfalls. Can you spot what is going to happen in this case?

```
xs = make_random_ints_no_dups(10, 1, 6)
```

12.2. The `time` module

As we start to work with more sophisticated algorithms and bigger programs, a natural concern is *"is our code efficient?"* One way to experiment is to time how long various operations take. The `time` module has a function called `process_time` that is recommended for this purpose. Whenever `process_time` is called, it returns a floating point number representing how many seconds have elapsed since your program started running.

The way to use it is to call `process_time` assign the result to a variable, say `t0`, just before you start executing the code you want to measure. Then after execution, call `process_time` again, (this time we'll save
the result in variable `t1`). The difference `t1-t0` is the time elapsed, and is a measure of how fast your program is running.

Let's try a small example. Python has a built-in `sum` function that can sum the elements in a list. We can also write our own. How do we think they would compare for speed? We'll try to do the summation of a list
[0, 1, 2 ...] in both cases, and compare the results:

```
import time

def do_my_sum(xs):
    sum = 0
    for v in xs:
        sum += v
    return sum

sz = 10000000        # Lets have 10 million elements in the list
testdata = range(sz)

t0 = time.process_time()
my_result = do_my_sum(testdata)
t1 = time.process_time()
print("my_result    = {0} (time taken = {1:.4f} seconds)"
        .format(my_result, t1-t0))

t2 = time.process_time()
their_result = sum(testdata)
t3 = time.process_time()
print("their_result = {0} (time taken = {1:.4f} seconds)"
        .format(their_result, t3-t2))
```

On a reasonably modest laptop, we get these results:

```
my_sum    = 49999995000000 (time taken = 1.5567 seconds)
their_sum = 49999995000000 (time taken = 0.9897 seconds)
```

So our function runs about 57% slower than the built-in one. Generating and summing up ten million elements in under a second is not too shabby!

12.3. The `math` module

The `math` module contains the kinds of mathematical functions you'd typically find on your calculator (`sin`, `cos`, `sqrt`, `asin`, `log`, `log10`) and some mathematical constants like `pi` and `e`:

```
>>> import math
>>> math.pi                 # Constant pi
3.141592653589793
>>> math.e                  # Constant natural log base
2.718281828459045
>>> math.sqrt(2.0)          # Square root function
1.4142135623730951
>>> math.radians(90)        # Convert 90 degrees to radians
1.5707963267948966
>>> math.sin(math.radians(90))  # Find sin of 90 degrees
1.0
>>> math.asin(1.0) * 2      # Double the arcsin of 1.0 to get pi
3.141592653589793
```

Like almost all other programming languages, angles are expressed in *radians* rather than degrees. There are two functions `radians` and `degrees` to convert between these two popular ways of measuring angles.

Notice another difference between this module and our use of `random` and `turtle`: in `random` and `turtle` we create objects and we call methods on the object. This is because objects have *state* — a turtle has a color, a position, a heading, etc., and every random number generator has a seed value that determines its next result.

Mathematical functions are "pure" and don't have any state — calculating the square root of 2.0 doesn't depend on any kind of state or history about what happened in the past. So the functions are not methods of an object — they are simply functions that are grouped together in a module called `math`.

12.4. Creating your own modules

All we need to do to create our own modules is to save our script as a file with a `.py` extension. Suppose, for example, this script is saved as a file named `seqtools.py`:

```
def remove_at(pos, seq):
    return seq[:pos] + seq[pos+1:]
```

We can now use our module, both in scripts we write, or in the interactive Python interpreter. To do so, we must first `import` the module.

```
>>> import seqtools
>>> s = "A string!"
>>> seqtools.remove_at(4, s)
'A sting!'
```

We do not include the `.py` file extension when importing. Python expects the file names of Python modules to end in `.py`, so the file extension is not included in the **import statement**.

The use of modules makes it possible to break up very large programs into manageable sized parts, and to keep related parts together.

12.5. Namespaces

A **namespace** is a collection of identifiers that belong to a module, or to a function, (and as we will see soon, in classes too). Generally, we like a namespace to hold "related" things, e.g. all the math functions, or all the typical things we'd do with random numbers.

Each module has its own namespace, so we can use the same identifier name in multiple modules without causing an identification problem.

```
# Module1.py

question = "What is the meaning of Life, the Universe, and Everything?"
answer = 42
```

```
# Module2.py

question = "What is your quest?"
answer = "To seek the holy grail."
```

We can now import both modules and access `question` and `answer` in each:

```
import module1
import module2

print(module1.question)
print(module2.question)
print(module1.answer)
print(module2.answer)
```

will output the following:

```
What is the meaning of Life, the Universe, and Everything?
What is your quest?
42
To seek the holy grail.
```

Functions also have their own namespaces:

```
def f():
    n = 7
    print("printing n inside of f:", n)

def g():
    n = 42
    print("printing n inside of g:", n)

n = 11
print("printing n before calling f:", n)
f()
print("printing n after calling f:", n)
g()
print("printing n after calling g:", n)
```

Running this program produces the following output:

```
printing n before calling f: 11
printing n inside of f: 7
printing n after calling f: 11
printing n inside of g: 42
printing n after calling g: 11
```

The three `n`'s here do not collide since they are each in a different namespace — they are three names for three different variables, just like there might be three different instances of people, all called "Bruce".

Namespaces permit several programmers to work on the same project without having naming collisions.

How are namespaces, files and modules related?

Python has a convenient and simplifying one-to-one mapping, one module per file, giving rise to one namespace. Also, Python takes the module name from the file name, and this becomes the name of the namespace.
`math.py` is a filename, the module is called `math`, and its namespace is `math`. So in Python the concepts are more or less interchangeable.

But you will encounter other languages (e.g. C#), that allow one module to span multiple files, or one file to have multiple namespaces, or many files to all share the same namespace. So the name of the file doesn't need to be the same as the namespace.

So a good idea is to try to keep the concepts distinct in your mind.

Files and directories organize *where* things are stored in our computer. On the other hand, namespaces and modules are a programming concept: they help us organize how we want to group related functions and attributes. They are not about "where" to store things, and should not have to coincide with the file and directory structures.

So in Python, if you rename the file `math.py`, its module name also changes, your `import` statements would need to change, and your code that refers to functions or attributes inside that namespace would also need to change.

In other languages this is not necessarily the case. So don't blur the concepts, just because Python blurs them!

12.6. Scope and lookup rules

The **scope** of an identifier is the region of program code in which the identifier can be accessed, or used.

There are three important scopes in Python:

- **Local scope** refers to identifiers declared within a function. These identifiers are kept in the namespace that belongs to the function, and each function has its own namespace.
- **Global scope** refers to all the identifiers declared within the current module, or file.
- **Built-in scope** refers to all the identifiers built into Python — those like `range` and `min` that can be used without having to import anything, and are (almost) always available.

Python (like most other computer languages) uses precedence rules: the same name could occur in more than one of these scopes, but the innermost, or local scope, will always take precedence over the global scope, and the global scope always gets used in preference to the built-in scope. Let's start with a simple example:

```
1 def range(n):
2     return 123*n
3
4 print(range(10))
```

What gets printed? We've defined our own function called `range`, so there is now a potential ambiguity. When we use `range`, do we mean our own one, or the built-in one? Using the scope

lookup rules determines this: our own `range` function, not the built-in one, is called, because our function `range` is in the global namespace, which takes precedence over the built-in names.

So although names likes `range` and `min` are built-in, they can be "hidden" from your use if you choose to define your own variables or functions that reuse those names. (It is a confusing practice to redefine built-in names — so to be a good programmer you need to understand the scope rules and understand that you can do nasty things that will cause confusion, and then you avoid doing them!)

Now, a slightly more complex example:

```
1  n = 10
2  m = 3
3  def f(n):
4      m = 7
5      return 2*n+m
6
7  print(f(5), n, m)
```

This prints 17 10 3. The reason is that the two variables `m` and `n` in lines 1 and 2 are outside the function in the global namespace. Inside the function, new variables called `n` and `m` are created *just for the duration of the execution of f*. These are created in the local namespace of function `f`. Within the body of `f`, the scope lookup rules determine that we use the local variables `m` and `n`. By contrast, after we've returned from `f`, the `n` and `m` arguments to the `print` function refer to the original variables on lines 1 and 2, and these have not been changed in any way by executing function `f`.

Notice too that the `def` puts name `f` into the global namespace here. So it can be called on line 7.

What is the scope of the variable `n` on line 1? Its scope — the region in which it is visible — is lines 1, 2, 6, 7. It is hidden from view in lines 3, 4, 5 because of the local variable `n`.

12.7. Attributes and the dot operator

Variables defined inside a module are called **attributes** of the module. We've seen that objects have attributes too: for example, most objects have a `__doc__` attribute, some functions have a `__annotations__` attribute. Attributes are accessed using the **dot operator** (`.`). The `question` attribute of `module1` and `module2` is accessed using `module1.question` and `module2.question`.

Modules contain functions as well as attributes, and the dot operator is used to access them in the same way. `seqtools.remove_at` refers to the `remove_at` function in the `seqtools` module.

When we use a dotted name, we often refer to it as a **fully qualified name**, because we're saying exactly which `question` attribute we mean.

12.8. Three `import` statement variants

Here are three different ways to import names into the current namespace, and to use them:

```
import math
x = math.sqrt(10)
```

Here just the single identifier `math` is added to the current namespace. If you want to access one of the functions in the module, you need to use the dot notation to get to it.

Here is a different arrangement:

```
from math import cos, sin, sqrt
x = sqrt(10)
```

The names are added directly to the current namespace, and can be used without qualification. The name `math` is not itself imported, so trying to use the qualified form `math.sqrt` would give an error.

Then we have a convenient shorthand:

```
from math import *   # Import all the identifiers from math,
                     #   adding them to the current namespace.
x = sqrt(10)         # Use them without qualification.
```

Of these three, the first method is generally preferred, even though it means a little more typing each time. Although, we can make things shorter by importing a module under a different name:

```
>>> import math as m
>>> m.pi
3.141592653589793
```

But hey, with nice editors that do auto-completion, and fast fingers, that's a small price!

Finally, observe this case:

```
def area(radius):
    import math
    return math.pi * radius * radius

x = math.sqrt(10)      # This gives an error
```

Here we imported `math`, but we imported it into the local namespace of `area`. So the name is usable within the function body, but not in the enclosing script, because it is not in the global namespace.

12.9. Turn your unit tester into a module

Near the end of Chapter 6 (Fruitful functions) we introduced unit testing, and our own `test` function, and you've had to copy this into each module for which you wrote tests. Now we can put that definition into a module of its own, say `unit_tester.py`, and simply use one line in each new script instead:

```
from unit_tester import test
```

12.10. Glossary

attribute
A variable defined inside a module (or class or instance – as we will see later). Module attributes are accessed by using the **dot operator** (`.`).

dot operator
The dot operator (`.`) permits access to attributes and functions of a module (or attributes and methods of a class or instance – as we have seen elsewhere).

fully qualified name
A name that is prefixed by some namespace identifier and the dot operator, or by an instance object, e.g. `math.sqrt` or `tess.forward(10)`.

import statement
A statement which makes the objects contained in a module available for use within another module. There are two forms for the import statement. Using hypothetical modules named `mymod1` and `mymod2` each containing functions `f1` and `f2`, and variables `v1` and `v2`, examples of these two forms include:

```
import mymod1
from mymod2 import f1, f2, v1, v2
```

The second form brings the imported objects into the namespace of the importing module, while the first form preserves a separate namespace for the imported module, requiring `mymod1.v1` to access the `v1` variable from that module.

method
Function-like attribute of an object. Methods are *invoked* (called) on an object using the dot operator. For example:

```
>>> s = "this is a string."
>>> s.upper()
'THIS IS A STRING.'
>>>
```

We say that the method, `upper` is invoked on the string, `s`. `s` is implicitely the first argument to `upper`.

module
A file containing Python definitions and statements intended for use in other Python programs. The contents of a module are made available to the other program by using the `import` statement.

namespace
A syntactic container providing a context for names so that the same name can reside in different namespaces without ambiguity. In Python, modules, classes, functions and methods all form namespaces.

naming collision
A situation in which two or more names in a given namespace cannot be unambiguously resolved. Using

```
import string
```

instead of

```
from string import *
```

prevents naming collisions.

standard library
A library is a collection of software used as tools in the development of other software. The standard library of a programming language is the set of such tools that are distributed with the core programming language. Python comes with an extensive standard library.

12.11. Exercises

1. Open help for the `calendar` module.

1. Try the following:

```
import calendar
cal = calendar.TextCalendar()        # Create an instance
cal.pryear(2012)                     # What happens here?
```

2. Observe that the week starts on Monday. An adventurous CompSci student believes that it is better
mental chunking to have his week start on Thursday, because then there are only two working days to the weekend, and every week has a break in the middle. Read the documentation for TextCalendar, and see how you can help him print a calendar that suits his needs.
3. Find a function to print just the month in which your birthday occurs this year.
4. Try this:

```
d = calendar.LocaleTextCalendar(6, "SPANISH")
d.pryear(2012)

Try a few other languages, including one that doesn't work, and see what happens.
```

5. Experiment with `calendar.isleap`. What does it expect as an argument? What does it return as a result? What kind of a function is this?

Make detailed notes about what you learned from these exercises.

2. Open help for the `math` module.

1. How many functions are in the `math` module?
2. What does `math.ceil` do? What about `math.floor`? (*hint:* both `floor` and `ceil` expect floating point arguments.)
3. Describe how we have been computing the same value as `math.sqrt` without using the `math` module.
4. What are the two data constants in the `math` module?

Record detailed notes of your investigation in this exercise.

3. Investigate the `copy` module. What does `deepcopy` do? In which exercises from last chapter would `deepcopy` have come in handy?

4. Create a module named `mymodule1.py`. Add attributes `myage` set to your current age, and `year` set to

the current year. Create another module named `mymodule2.py`. Add attributes `myage` set to `0`, and `year` set to the year you were born. Now create a file named `namespace_test.py`. Import both of

the modules above and write the following statement:

```
    print( (mymodule2.myage - mymodule1.myage) ==
           (mymodule2.year - mymodule1.year)  )
```

When you will run `namespace_test.py` you will see either `True` or `False` as output depending on whether or not you've already had your birthday this year.

What this example illustrates is that out different modules can both have attributes named `myage` and `year`. Because they're in different namespaces, they don't clash with one another. When we write `namespace_test.py`, we fully qualify exactly which variable `year` or `myage` we are referring to.

5. Add the following statement to `mymodule1.py`, `mymodule2.py`, and `namespace_test.py` from the previous

exercise:

```
    print("My name is", __name__)
```

Run `namespace_test.py`. What happens? Why? Now add the following to the bottom of `mymodule1.py`:

```
    if __name__ == "__main__":
        print("This won't run if I'm  imported.")
```

Run `mymodule1.py` and `namespace_test.py` again. In which case do you see the new print statement?

6. In a Python shell / interactive interpreter, try the following:

```
    >>> import this
```

What does Tim Peters have to say about namespaces?

7. Give the Python interpreter's response to each of the following from a continuous interpreter session:

```
>>> s = "If we took the bones out, it wouldn't be crunchy, would it?"
>>> s.split()
>>> type(s.split())
>>> s.split("o")
>>> s.split("i")
>>> "0".join(s.split("o"))
```

Be sure you understand why you get each result. Then apply what you have learned to fill in the body of the function below using the `split` and `join` methods of `str` objects:

```
def myreplace(old, new, s):
    """ Replace all occurrences of old with new in s. """
    ...


test(myreplace(",", ";", "this, that, and some other thing") ==
                         "this; that; and some other thing")
test(myreplace(" ", "**",
                "Words will now      be  separated by stars.") ==
                "Words**will**now**be**separated**by**stars.")
```

Your solution should pass the tests.

8. Create a module named `wordtools.py` with our test scaffolding in place.

Now add functions to these tests pass:

```
test(cleanword("what?") == "what")
test(cleanword("'now!'") == "now")
test(cleanword("?+='w-o-r-d!,@$()'") ==  "word")

test(has_dashdash("distance--but"))
test(not has_dashdash("several"))
test(has_dashdash("spoke--"))
test(has_dashdash("distance--but"))
test(not has_dashdash("-yo-yo-"))

test(extract_words("Now is the time!  'Now', is the time? Yes, now.") == ['now','is'\
,'the','time','now','is','the','time','yes','now'])
test(extract_words("she tried to curtsey as she spoke--fancy") == ['she','tried','to\
','curtsey','as','she','spoke','fancy'])
```

```
15
16 test(wordcount("now", ["now","is","time","is","now","is","is"]) == 2)
17 test(wordcount("is", ["now","is","time","is","now","the","is"]) == 3)
18 test(wordcount("time", ["now","is","time","is","now","is","is"]) == 1)
19 test(wordcount("frog", ["now","is","time","is","now","is","is"]) == 0)
20
21 test(wordset(["now", "is", "time", "is", "now", "is", "is"]) == ["is", "now", "time"\
22 ])
23 test(wordset(["I", "a", "a", "is", "a", "is", "I", "am"]) == ["I", "a", "am", "is"])
24 test(wordset(["or", "a", "am", "is", "are", "be", "but", "am"]) == ["a", "am", "are"\
25 , "be", "but", "is", "or"])
26
27 test(longestword(["a", "apple", "pear", "grape"]) == 5)
28 test(longestword(["a", "am", "I", "be"]) == 2)
29 test(longestword(["this","supercalifragilisticexpialidocious"]) == 34)
30 test(longestword([ ]) == 0)
```

Save this module so you can use the tools it contains in future programs.

Chapter 13: Files

13.1. About files

While a program is running, its data is stored in *random access memory* (RAM). RAM is fast and inexpensive, but it is also **volatile**, which means that when the program ends, or the computer shuts down, data in
RAM disappears. To make data available the next time the computer is turned on and the program is started, it has to be written to a **non-volatile** storage medium, such a hard drive, usb drive, or CD-RW.

Data on non-volatile storage media is stored in named locations on the media called **files**. By reading and writing files, programs can save information between program runs.

Working with files is a lot like working with a notebook. To use a notebook, it has to be opened. When done, it has to be closed. While the notebook is open, it can either be read from or written to. In either case, the notebook holder knows where they are. They can read the whole notebook in its natural order or they can skip around.

All of this applies to files as well. To open a file, we specify its name and indicate whether we want to read or write.

13.2. Writing our first file

Let's begin with a simple program that writes three lines of text into a file:

```
1 myfile = open("test.txt", "w")
2 myfile.write("My first file written from Python\n")
3 myfile.write("---------------------------------\n")
4 myfile.write("Hello, world!\n")
5 myfile.close()
```

Opening a file creates what we call a file **handle**. In this example, the variable `myfile` refers to the new handle object. Our program calls methods on the handle, and this makes changes to the actual file which
is usually located on our disk.

On line 1, the open function takes two arguments. The first is the name of the file, and the second is the **mode**. Mode `"w"` means that we are opening the file for writing.

With mode `"w"`, if there is no file named `test.txt` on the disk, it will be created. If there already is one, it will be replaced by the file we are writing.

To put data in the file we invoke the `write` method on the handle, shown in lines 2, 3 and 4 above. In bigger programs, lines 2–4 will usually be replaced by a loop that writes many more lines into the file.

Closing the file handle (line 5) tells the system that we are done writing and makes the disk file available for reading by other programs (or by our own program).

A handle is somewhat like a TV remote control

We're all familiar with a remote control for a TV. We perform operations on the remote control — switch channels, change the volume, etc. But the real action happens on the TV. So, by simple analogy, we'd call the remote control our handle to the underlying TV.

Sometimes we want to emphasize the difference — the file handle is not the same as the file, and the remote control is not the same as the TV. But at other times we prefer to treat them as a single mental chunk, or abstraction, and we'll just say "close the file", or "flip the TV channel".

13.3. Reading a file line-at-a-time

Now that the file exists on our disk, we can open it, this time for reading, and read all the lines in the file, one at a time. This time, the mode argument is `"r"` for reading:

```
1  mynewhandle = open("test.txt", "r")
2  while True:                          # Keep reading forever
3      theline = mynewhandle.readline()   # Try to read next line
4      if len(theline) == 0:              # If there are no more lines
5          break                          #     leave the loop
6
7      # Now process the line we've just read
8      print(theline, end="")
9
10 mynewhandle.close()
```

This is a handy pattern for our toolbox. In bigger programs, we'd squeeze more extensive logic into the body of the loop at line 8 —for example, if each line of the file contained the name and email address of one of our friends, perhaps we'd split the line into some pieces and call a function to send the friend a party invitation.

On line 8 we suppress the newline character that `print` usually appends to our strings. Why? This is because the string already has its own newline: the `readline` method in line 3 returns everything up to *and including* the newline character. This also explains the end-of-file detection logic: when

there are no more lines to be read from the file, `readline` returns an empty string — one that does not even have a newline at the end, hence its length is 0.

Fail first ...

In our sample case here, we have three lines in the file, yet we enter the loop *four* times. In Python, you only learn that the file has no more lines by failure to read another line. In some other programming languages (e.g. Pascal), things are different: there you read three lines, but you have what is called *look ahead* — after reading the third line you already know that there are no more lines in the file. You're not even allowed to try to read the fourth line.

So the templates for working line-at-a-time in Pascal and Python are subtly different!

When you transfer your Python skills to your next computer language, be sure to ask how you'll know when the file has ended: is the style in the language "try, and after you fail you'll know", or is it "look ahead"?

If we try to open a file that doesn't exist, we get an error:

```
>>> mynewhandle = open("wharrah.txt", "r")
IOError: [Errno 2] No such file or directory: "wharrah.txt"
```

13.4. Turning a file into a list of lines

It is often useful to fetch data from a disk file and turn it into a list of lines. Suppose we have a file containing our friends and their email addresses, one per line in the file. But we'd like the lines sorted into alphabetical order. A good plan is to read everything into a list of lines, then sort the list, and then write the sorted list back to another file:

```
1  f = open("friends.txt", "r")
2  xs = f.readlines()
3  f.close()
4
5  xs.sort()
6
7  g = open("sortedfriends.txt", "w")
8  for v in xs:
9      g.write(v)
10 g.close()
```

The `readlines` method in line 2 reads all the lines and returns a list of the strings.

We could have used the template from the previous section to read each line one-at-a-time, and to build up the list ourselves, but it is a lot easier to use the method that the Python implementors gave us!

13.5. Reading the whole file at once

Another way of working with text files is to read the complete contents of the file into a string, and then to use our string-processing skills to work with the contents.

We'd normally use this method of processing files if we were not interested in the line structure of the file. For example, we've seen the `split` method on strings which can break a string into words. So here is how we might count the number of words in a file:

```
1 f = open("somefile.txt")
2 content = f.read()
3 f.close()
4
5 words = content.split()
6 print("There are {0} words in the file.".format(len(words)))
```

Notice here that we left out the `"r"` mode in line 1. By default, if we don't supply the mode, Python opens the file for reading.

Your file paths may need to be explicitly named.

In the above example, we're assuming that the file `somefile.txt` is in the same directory as your Python source code. If this is not the case, you may need to provide a full or a relative path to the file. On Windows, a full path could look like `"C:\\temp\\somefile.txt"`, while on a Unix system the full path could be `"/home/jimmy/somefile.txt"`.

We'll return to this later in this chapter.

13.6. Working with binary files

Files that hold photographs, videos, zip files, executable programs, etc. are called **binary** files: they're not organized into lines, and cannot be opened with a normal text editor. Python works just as easily with binary files, but when we read from the file we're going to get bytes back rather than a string. Here we'll copy one binary file to another:

```
1  f = open("somefile.zip", "rb")
2  g = open("thecopy.zip", "wb")
3
4  while True:
5      buf = f.read(1024)
6      if len(buf) == 0:
7          break
8      g.write(buf)
9
10 f.close()
11 g.close()
```

There are a few new things here. In lines 1 and 2 we added a `"b"` to the mode to tell Python that the files are binary rather than text files. In line 5, we see `read` can take an argument which tells it how many bytes to attempt to read from the file. Here we chose to read and write up to 1024 bytes on each iteration of the loop. When we get back an empty buffer from our attempt to read, we know we can break out of the loop and close both the files.

If we set a breakpoint at line 6, (or print `type(buf)` there) we'll see that the type of `buf` is `bytes`. We don't do any detailed work with `bytes` objects in this textbook.

13.7. An example

Many useful line-processing programs will read a text file line-at-a-time and do some minor processing as they write the lines to an output file. They might number the lines in the output file, or insert extra blank lines after every 60 lines to make it convenient for printing on sheets of paper, or extract some specific columns only from each line in the source file, or only print lines that contain a specific substring. We call this kind of program a **filter**.

Here is a filter that copies one file to another, omitting any lines that begin with `#`:

```
1  def filter(oldfile, newfile):
2      infile = open(oldfile, "r")
3      outfile = open(newfile, "w")
4      while True:
5          text = infile.readline()
6          if len(text) == 0:
7              break
8          if text[0] == "#":
9              continue
10
11         # Put any more processing logic here
12         outfile.write(text)
```

```
13
14      infile.close()
15      outfile.close()
```

The `continue` statement at line 9 skips over the remaining lines in the current iteration of the loop, but the loop will still iterate. This style looks a bit contrived here, but it is often useful to say *"get the lines we're not concerned with out of the way early, so that we have cleaner more focused logic in the meaty part of the loop that might be written around line 11."*

Thus, if `text` is the empty string, the loop exits. If the first character of `text` is a hash mark, the flow of execution goes to the top of the loop, ready to start processing the next line. Only if both conditions fail do we fall through to do the processing at line 11, in this example, writing the line into the new file.

Let's consider one more case: suppose our original file contained empty lines. At line 6 above, would this program find the first empty line in the file, and terminate immediately? No! Recall that `readline` always includes the newline character in the string it returns. It is only when we try to read *beyond* the end of the file that we get back the empty string of length 0.

13.8. Directories

Files on non-volatile storage media are organized by a set of rules known as a **file system**. File systems are made up of files and **directories**, which are containers for both files and other directories.

When we create a new file by opening it and writing, the new file goes in the current directory (wherever we were when we ran the program). Similarly, when we open a file for reading, Python looks for it in the current directory.

If we want to open a file somewhere else, we have to specify the **path** to the file, which is the name of the directory (or folder) where the file is located:

```
1  >>> wordsfile = open("/usr/share/dict/words", "r")
2  >>> wordlist = wordsfile.readlines()
3  >>> print(wordlist[:6])
4  ['\n', 'A\n', "A's\n", 'AOL\n', "AOL's\n", 'Aachen\n']
```

This (Unix) example opens a file named `words` that resides in a directory named `dict`, which resides in `share`, which resides in `usr`, which resides in the top-level directory of the system, called `/`. It then reads in each line into a list using `readlines`, and prints out the first 5 elements from that list.

A Windows path might be `"c:/temp/words.txt"` or `"c:\\temp\\words.txt"`. Because backslashes are used to escape things like newlines and tabs, we need to write two backslashes in a literal string to get one! So the length of these two strings is the same!

We cannot use `/` or `\` as part of a filename; they are reserved as a **delimiter** between directory and filenames.

The file `/usr/share/dict/words` should exist on Unix-based systems, and contains a list of words in alphabetical order.

13.9. What about fetching something from the web?

The Python libraries are pretty messy in places. But here is a very simple example that copies the contents at some web URL to a local file.

```
import urllib.request

url = "https://www.ietf.org/rfc/rfc793.txt"
destination_filename = "rfc793.txt"

urllib.request.urlretrieve(url, destination_filename)
```

The `urlretrieve` function — just one call — could be used to download any kind of content from the Internet.

We'll need to get a few things right before this works:
- The resource we're trying to fetch must exist! Check this using a browser.
- We'll need permission to write to the destination filename, and the file will be created in the "current directory" - i.e. the same folder that the Python program is saved in.
- If we are behind a proxy server that requires authentication, (as some students are), this may require some more special handling to work around our proxy. Use a local resource for the purpose of this demonstration!

Here is a slightly different example. Rather than save the web resource to our local disk, we read it directly into a string, and return it:

```
import urllib.request

def retrieve_page(url):
    """ Retrieve the contents of a web page.
        The contents is converted to a string before returning it.
    """
    my_socket = urllib.request.urlopen(url)
    dta = str(my_socket.read())
    my_socket.close()
```

```
    return dta

the_text = retrieve_page("https://www.ietf.org/rfc/rfc793.txt")
print(the_text)
```

Opening the remote url returns what we call a **socket**. This is a handle to our end of the connection between our program and the remote web server. We can call read, write, and close methods on the socket
object in much the same way as we can work with a file handle.

13.10. Glossary

delimiter
A sequence of one or more characters used to specify the boundary between separate parts of text.

directory
A named collection of files, also called a folder. Directories can contain files and other directories, which are referred to as *subdirectories* of the directory that contains them.

file
A named entity, usually stored on a hard drive, floppy disk, or CD-ROM, that contains a stream of characters.

file system
A method for naming, accessing, and organizing files and the data they contain.

handle
An object in our program that is connected to an underlying resource (e.g. a file). The file handle lets our program manipulate/read/write/close the actual file that is on our disk.

mode
A distinct method of operation within a computer program. Files in Python can be opened in one of four modes: read (`"r"`), write (`"w"`), append (`"a"`), and read and write (`"+"`).

non-volatile memory
Memory that can maintain its state without power. Hard drives, flash drives, and rewritable compact disks (CD-RW) are each examples of non-volatile memory.

path
A sequence of directory names that specifies the exact location of a file.

text file
A file that contains printable characters organized into lines separated by newline characters.

socket
One end of a connection allowing one to read and write information to or from another computer.

volatile memory
Memory which requires an electrical current to maintain state. The *main memory* or RAM of a computer is volatile. Information stored in RAM is lost when the computer is turned off.

13.11. Exercises

1. Write a program that reads a file and writes out a new file with the lines in reversed order (i.e. the first line in the old file becomes the last one in the new file.)
2. Write a program that reads a file and prints only those lines that contain the substring `snake`.
3. Write a program that reads a text file and produces an output file which is a copy of the file, except the first five columns of each line contain a four digit line number, followed by a space. Start numbering the first line in the output file at 1. Ensure that every line number is formatted to the same width in the output file. Use one of your Python programs as test data for this exercise: your output should be a printed and numbered listing of the Python program.
4. Write a program that undoes the numbering of the previous exercise: it should read a file with numbered lines and produce another file without line numbers.

Chapter 14: List Algorithms

This chapter is a bit different from what we've done so far: rather than introduce more new Python syntax and features, we're going to focus on the program development process, and some algorithms that work with lists.

As in all parts of this book, our expectation is that you, the reader, will copy our code into your Python environment, play and experiment, and work along with us.

Part of this chapter works with the book *Alice in Wonderland* and a *vocabulary file*. Download these two files to your local machine at the following links. https://learnpythontherightway.com/_downloads/alice_in_wonderland.txt[13] and https://learnpythontherightway.com/_downloads/vocab.txt[14].

14.1. Test-driven development

Early in our Fruitful functions chapter we introduced the idea of *incremental development*, where we added
small fragments of code to slowly build up the whole, so that we could easily find problems early. Later in that same chapter we introduced unit testing and gave code for our testing framework so that we could capture, in code, appropriate tests for the functions we were writing.

Test-driven development (TDD) is a software development practice which takes these practices one step further. The key idea is that automated tests should be written *first*. This technique is called
test-driven because — if we are to believe the extremists — non-testing code should only be written when there is a failing test to make pass.

We can still retain our mode of working in small incremental steps, but now we'll define and express those steps in terms of a sequence of increasingly sophisticated unit tests that demand more from our code at
each stage.

We'll turn our attention to some standard algorithms that process lists now, but as we proceed through this chapter we'll attempt to do so in the spirit envisaged by TDD.

14.2. The linear search algorithm

We'd like to know the index where a specific item occurs within in a list of items. Specifically, we'll return the index of the item if it is found, or we'll return -1 if the item doesn't occur in the list. Let

[13] https://learnpythontherightway.com/_downloads/alice_in_wonderland.txt
[14] https://learnpythontherightway.com/_downloads/vocab.txt

us
start with some tests:

```
friends = ["Joe", "Zoe", "Brad", "Angelina", "Zuki", "Thandi", "Paris"]
test(search_linear(friends, "Zoe") == 1)
test(search_linear(friends, "Joe") == 0)
test(search_linear(friends, "Paris") == 6)
test(search_linear(friends, "Bill") == -1)
```

Motivated by the fact that our tests don't even run, let alone pass, we now write the function:

```
def search_linear(xs, target):
    """ Find and return the index of target in sequence xs """
    for (i, v) in enumerate(xs):
       if v == target:
           return i
    return -1
```

There are a some points to learn here: We've seen a similar algorithm in section 8.10 when we searched for a character in a string. There we used a `while` loop, here we've used a `for` loop, coupled with `enumerate` to extract the `(i, v)` pair on each iteration. There are other variants — for example, we could have used `range` and made the loop run only over the indexes, or we could have used the idiom of returning `None` when the item was not found in the list. But the essential similarity in all these variations is that we test every item in the list in turn, from first to last, using the pattern of the short-circuit *eureka*
traversal that we introduced earlier —that we return from the function as soon as we find the target that we're looking for.

Searching all items of a sequence from first to last is called a **linear search**. Each time we check whether `v == target` we'll call it a **probe**. We like to count probes as a measure of how efficient our algorithm is, and this will be a good enough indication of how long our algorithm will take to execute.

Linear searching is characterized by the fact that the number of probes needed to find some target depends directly on the length of the list. So if the list becomes ten times bigger, we can expect to wait ten times longer when searching for things. Notice too, that if we're searching for a target that is not present in the list, we'll have to go all the way to the end before we can return the negative value. So this case needs N probes, where N is the length of the list. However, if we're searching for a target that does exist in the list, we could be lucky and find it immediately in position 0, or we might have to look further, perhaps even all the way to the last item. On average, when the target is present, we're going to need to go about halfway through the list, or N/2 probes.

We say that this search has **linear performance** (linear meaning straight line) because, if we were to measure the average search times for different sizes of lists (N), and then plot a graph of time-to-search against N, we'd get a more-or-less straight line graph.

Analysis like this is pretty meaningless for small lists — the computer is quick enough not to bother if the list only has a handful of items. So generally, we're interested in the **scalability** of our algorithms — how do they perform if we throw bigger problems at them. Would this search be a sensible one to use if we had a million or ten million items (perhaps the catalog of books in your local library) in our list? What happens for really large datasets, e.g. how does Google search so brilliantly well?

14.3. A more realistic problem

As children learn to read, there are expectations that their vocabulary will grow. So a child of age 14 is expected to know more words than a child of age 8. When prescribing reading books for a grade, an important question might be *"which words in this book are not in the expected vocabulary at this level?"*

Let us assume we can read a vocabulary of words into our program, and read the text of a book, and split it into words. Let us write some tests for what we need to do next. Test data can usually be very small, even if we intend to finally use our program for larger cases:

```
vocab = ["apple", "boy", "dog", "down",
                          "fell", "girl", "grass", "the", "tree"]
book_words = "the apple fell from the tree to the grass".split()
test(find_unknown_words(vocab, book_words) == ["from", "to"])
test(find_unknown_words([], book_words) == book_words)
test(find_unknown_words(vocab, ["the", "boy", "fell"]) == [])
```

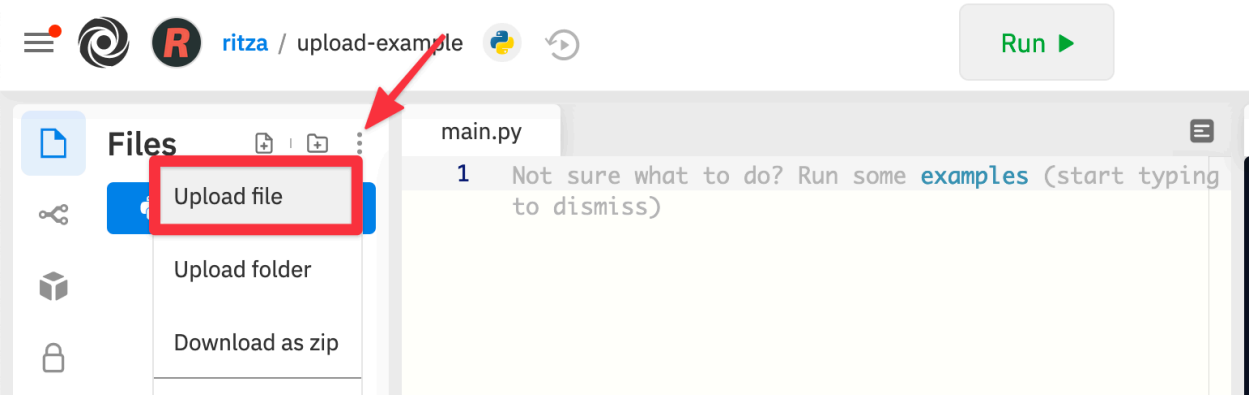
Notice we were a bit lazy, and used `split` to create our list of words —it is easier than typing out the list, and very convenient if you want to input a sentence into the program and turn it into a list of words.

We now need to implement the function for which we've written tests, and we'll make use of our linear search. The basic strategy is to run through each of the words in the book, look it up in the vocabulary, and if it is not in the vocabulary, save it into a new resulting list which we return from the function:

```
def find_unknown_words(vocab, wds):
    """ Return a list of words in wds that do not occur in vocab """
    result = []
    for w in wds:
        if (search_linear(vocab, w) < 0):
            result.append(w)
    return result
```

We can happily report now that the tests all pass.

Now let us look at the scalability. We have more realistic vocabulary in the text file that could be downloaded at the beginning of this chapter. Upload the `vocab.txt` file to a new repl so that you can access it from your code.



Now let us read in the file (as a single string) and split it into a list of words. For convenience, we'll create a function to do this for us, and test it on the vocab file.

```
def load_words_from_file(filename):
    """ Read words from filename, return list of words. """
    f = open(filename, "r")
    file_content = f.read()
    f.close()
    wds = file_content.split()
    return wds

bigger_vocab = load_words_from_file("vocab.txt")
print("There are {0} words in the vocab, starting with\n {1} "
              .format(len(bigger_vocab), bigger_vocab[:6]))
```

Python responds with:

```
There are 19469 words in the vocab, starting with
['a', 'aback', 'abacus', 'abandon', 'abandoned', 'abandonment']
```

So we've got a more sensible size vocabulary. Now let us load up a book, once again we'll use the one we downloaded at the beginning of this chapter. Loading a book is much like loading words from a file, but
we're going to do a little extra black magic. Books are full of punctuation, and have mixtures of lowercase and uppercase letters. We need to clean up the contents of the book. This will involve removing punctuation, and converting everything to the same case (lowercase, because our vocabulary is all in lowercase). So we'll want a more sophisticated way of converting text to words.

```
test(text_to_words("My name is Earl!") == ["my", "name", "is", "earl"])
test(text_to_words('"Well, I never!", said Alice.') ==
                            ["well", "i", "never", "said", "alice"])
```

There is a powerful `translate` method available for strings. The idea is that one sets up desired substitutions — for every character, we can give a corresponding replacement character. The `translate` method will apply these replacements throughout the whole string. So here we go:

```
def text_to_words(the_text):
    """ return a list of words with all punctuation removed,
        and all in lowercase.
    """

    my_substitutions = the_text.maketrans(
      # If you find any of these
      "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!\"#$%&()*+,-./:;<=>?@[]^_`{|}~'\\",
      # Replace them by these
      "abcdefghijklmnopqrstuvwxyz                                    ")

    # Translate the text now.
    cleaned_text = the_text.translate(my_substitutions)
    wds = cleaned_text.split()
    return wds
```

The translation turns all uppercase characters into lowercase, and all punctuation characters and digits into spaces. Then, of course, `split` will get rid of the spaces as it breaks the text into a list of words.
The tests pass.

Now we're ready to read in our book:

```
def get_words_in_book(filename):
    """ Read a book from filename, and return a list of its words. """
    f = open(filename, "r")
    content = f.read()
    f.close()
    wds = text_to_words(content)
    return wds

book_words = get_words_in_book("alice_in_wonderland.txt")
print("There are {0} words in the book, the first 100 are\n{1}".
           format(len(book_words), book_words[:100]))
```

Python prints the following (all on one line, we've cheated a bit for the textbook):

```
There are 27336 words in the book, the first 100 are
['alice', 's', 'adventures', 'in', 'wonderland', 'lewis', 'carroll',
    'chapter', 'i', 'down', 'the', 'rabbit', 'hole', 'alice', 'was',
    'beginning', 'to', 'get', 'very', 'tired', 'of', 'sitting', 'by',
    'her', 'sister', 'on', 'the', 'bank', 'and', 'of', 'having',
    'nothing', 'to', 'do', 'once', 'or', 'twice', 'she', 'had',
    'peeped', 'into', 'the', 'book', 'her', 'sister', 'was', 'reading',
    'but', 'it', 'had', 'no', 'pictures', 'or', 'conversations', 'in',
    'it', 'and', 'what', 'is', 'the', 'use', 'of', 'a', 'book',
    'thought', 'alice', 'without', 'pictures', 'or', 'conversation',
    'so', 'she', 'was', 'considering', 'in', 'her', 'own', 'mind',
    'as', 'well', 'as', 'she', 'could', 'for', 'the', 'hot', 'day',
    'made', 'her', 'feel', 'very', 'sleepy', 'and', 'stupid',
    'whether', 'the', 'pleasure', 'of', 'making', 'a']
```

Well now we have all the pieces ready. Let us see what words in this book are not in the vocabulary:

```
>>> missing_words = find_unknown_words(bigger_vocab, book_words)
```

We wait a considerable time now, something like a minute, before Python finally works its way through this, and prints a list of 3398 words in the book that are not in the vocabulary. Mmm... This is not particularly
scaleable. For a vocabulary that is twenty times larger (you'll often find school dictionaries with 300 000 words, for example), and longer books, this is going to be slow. So let us make some timing measurements
while we think about how we can improve this in the next section.

```
import time

t0 = time.process_time()
missing_words = find_unknown_words(bigger_vocab, book_words)
t1 = time.process_time()
print("There are {0} unknown words.".format(len(missing_words)))
print("That took {0:.4f} seconds.".format(t1-t0))
```

We get the results and some timing that we can refer back to later:

```
There are 3398 unknown words.
That took 49.8014 seconds.
```

14.4. Binary Search

If you think about what we've just done, it is not how we work in real life. If you were given a vocabulary and asked to tell if some word was present, you'd probably start in the middle. You can do this because the
vocabulary is ordered — so you can probe some word in the middle, and immediately realize that your target was before (or perhaps after) the one you had probed. Applying this principle repeatedly leads us to a
very much better algorithm for searching in a list of items that are already ordered. (Note that if the items are not ordered, you have little choice other than to look through all of them. But, if we know the items are in order, we can improve our searching technique).

Lets start with some tests. Remember, the list needs to be sorted:

```
1  xs = [2,3,5,7,11,13,17,23,29,31,37,43,47,53]
2  test(search_binary(xs, 20) == -1)
3  test(search_binary(xs, 99) == -1)
4  test(search_binary(xs, 1) == -1)
5  for (i, v) in enumerate(xs):
6      test(search_binary(xs, v) == i)
```

Even our test cases are interesting this time: notice that we start with items not in the list and look at boundary conditions — in the middle of the list, less than all items in the list, bigger than the biggest.
Then we use a loop to use every list item as a target, and to confirm that our binary search returns the corresponding index of that item in the list.

It is useful to think about having a *region-of-interest* (ROI) within the list being searched. This ROI will be the portion of the list in which it is still possible that our target might be found. Our algorithm will start with the ROI set to all the items in the list. On the first probe in the middle of the ROI, there are three possible outcomes: either we find the target, or we learn that we can discard the top half
of the ROI, or we learn that we can discard the bottom half of the ROI. And we keep doing this repeatedly, until we find our target, or until we end up with no more items in our region of interest. We can code this as follows:

```
def search_binary(xs, target):
    """ Find and return the index of key in sequence xs """
    lb = 0
    ub = len(xs)
    while True:
        if lb == ub:   # If region of interest (ROI) becomes empty
           return -1

        # Next probe should be in the middle of the ROI
        mid_index = (lb + ub) // 2

        # Fetch the item at that position
        item_at_mid = xs[mid_index]

        # print("ROI[{0}:{1}](size={2}), probed='{3}', target='{4}'"
        #       .format(lb, ub, ub-lb, item_at_mid, target))

        # How does the probed item compare to the target?
        if item_at_mid == target:
            return mid_index      # Found it!
        if item_at_mid < target:
            lb = mid_index + 1    # Use upper half of ROI next time
        else:
            ub = mid_index        # Use lower half of ROI next time
```

The region of interest is represented by two variables, a lower bound `lb` and an upper bound `ub`. It is important to be precise about what values these indexes have. We'll make `lb` hold the index of the first
item in the ROI, and make `ub` hold the index just *beyond* the last item of interest. So these semantics are similar to a Python slice semantics: the region of interest is exactly the slice `xs[lb:ub]`. (The algorithm never actually takes any array slices!)

With this code in place, our tests pass. Great. Now if we substitute a call to this search algorithm instead of calling the `search_linear` in `find_unknown_words`, can we improve our performance? Let's do that, and
again run this test:

```
t0 = time.process_time()
missing_words = find_unknown_words(bigger_vocab, book_words)
t1 = time.process_time()
print("There are {0} unknown words.".format(len(missing_words)))
print("That took {0:.4f} seconds.".format(t1-t0))
```

What a spectacular difference! More than 200 times faster!

```
There are 3398 unknown words.
That took 0.2262 seconds.
```

Why is this binary search so much faster than the linear search? If we uncomment the print statement on lines 15 and 16, we'll get a trace of the probes done during a search. Let's go ahead, and try that:

```
>>> search_binary(bigger_vocab, "magic")
ROI[0:19469](size=19469), probed='known', target='magic'
ROI[9735:19469](size=9734), probed='retailer', target='magic'
ROI[9735:14602](size=4867), probed='overthrow', target='magic'
ROI[9735:12168](size=2433), probed='mission', target='magic'
ROI[9735:10951](size=1216), probed='magnificent', target='magic'
ROI[9735:10343](size=608), probed='liken', target='magic'
ROI[10040:10343](size=303), probed='looks', target='magic'
ROI[10192:10343](size=151), probed='lump', target='magic'
ROI[10268:10343](size=75), probed='machete', target='magic'
ROI[10306:10343](size=37), probed='mafia', target='magic'
ROI[10325:10343](size=18), probed='magnanimous', target='magic'
ROI[10325:10334](size=9), probed='magical', target='magic'
ROI[10325:10329](size=4), probed= maggot', target='magic'
ROI[10328:10329](size=1), probed='magic', target='magic'
10328
```

Here we see that finding the target word "magic" needed just 14 probes before it was found at index 10328. The important thing is that each probe halves (with some truncation) the remaining region of interest. By
contrast, the linear search would have needed 10329 probes to find the same target word.

The word *binary* means *two*. Binary search gets its name from the fact that each probe splits the list into two pieces and discards the one half from the region of interest.

The beauty of the algorithm is that we could double the size of the vocabulary, and it would only need one more probe! And after another doubling, just another one probe. So as the vocabulary gets bigger, this
algorithm's performance becomes even more impressive.

Can we put a formula to this? If our list size is N, what is the biggest number of probes k we could need? The maths is a bit easier if we turn the question around: how big a list N could we deal with, given that we
were only allowed to make k probes?

With 1 probe, we can only search a list of size 1. With two probes we could cope with lists up to size 3 - (test the middle item with the first probe, then test either the left or right sublist with the second probe). With one more probe, we could cope with 7 items (the middle item, and two sublists of size 3). With four probes, we can search 15 items, and 5 probes lets us search up to 31 items. So the general relationship is given by the formula

```
N = 2 ** k - 1
```

where k is the number of probes we're allowed to make, and N is the maximum size of the list that can be searched in that many probes. This function is *exponential* in k (because k occurs in the exponent part).
If we wanted to turn the formula around and solve for k in terms of N, we need to move the constant 1 to the other side, and take a log (base 2) on each side. (The log is the inverse of an exponent.) So the formula
for k in terms of N is now:

$$k = \lceil log_2(N + 1) \rceil$$

The square-only-on-top brackets are called *ceiling brackets*: this means that you must round the number up to the next whole integer.

Let us try this on a calculator, or in Python, which is the mother of all calculators: suppose I have 1000 elements to be searched, what is the maximum number of probes I'll need? (There is a pesky +1 in the formula, so let us not forget to add it on…):

```
>>> from math import log
>>> log(1000 + 1, 2)
9.967226258835993
```

Telling us that we'll need 9.96 probes maximum, to search 1000 items is not quite what we want. We forgot to take the ceiling. The `ceil` function in the `math` module does exactly this. So more accurately, now:

```
>>> from math import log, ceil
>>> ceil(log(1000 + 1, 2))
10
>>> ceil(log(1000000 + 1, 2))
20
>>> ceil(log(1000000000 + 1, 2))
30
```

This tells us that searching 1000 items needs 10 probes. (Well technically, with 10 probes we can search exactly 1023 items, but the easy and useful stuff to remember here is that "1000 items needs 10 probes, a million needs 20 probes, and a billion items only needs 30 probes").

You will rarely encounter algorithms that scale to large datasets as beautifully as binary search does!

14.5. Removing adjacent duplicates from a list

We often want to get the unique elements in a list, i.e. produce a new list in which each different element occurs just once. Consider our case of looking for words in Alice in Wonderland that are not in our vocabulary. We had a report that there are 3398 such words, but there are duplicates in that list. In fact, the word "alice" occurs 398 times in the book, and it is not in our vocabulary! How should we remove these
duplicates?

A good approach is to sort the list, then remove all adjacent duplicates. Let us start with removing adjacent duplicates

```
test(remove_adjacent_dups([1,2,3,3,3,3,5,6,9,9]) == [1,2,3,5,6,9])
test(remove_adjacent_dups([]) == [])
test(remove_adjacent_dups(["a", "big", "big", "bite", "dog"]) ==
                                  ["a", "big", "bite", "dog"])
```

The algorithm is easy and efficient. We simply have to remember the most recent item that was inserted into the result, and avoid inserting it again:

```
def remove_adjacent_dups(xs):
    """ Return a new list in which all adjacent
        duplicates from xs have been removed.
    """
    result = []
    most_recent_elem = None
    for e in xs:
        if e != most_recent_elem:
            result.append(e)
            most_recent_elem = e

    return result
```

The amount of work done in this algorithm is linear — each item in `xs` causes the loop to execute exactly once, and there are no nested loops. So doubling the number of elements in `xs` should cause this function to run twice as long: the relationship between the size of the list and the time to run will be graphed as a straight (linear) line.

Let us go back now to our analysis of Alice in Wonderland. Before checking the words in the book against the
vocabulary, we'll sort those words into order, and eliminate duplicates. So our new code looks like this:

```
all_words = get_words_in_book("alice_in_wonderland.txt")
all_words.sort()
book_words = remove_adjacent_dups(all_words)
print("There are {0} words in the book. Only {1} are unique.".
                      format(len(all_words), len(book_words)))
print("The first 100 words are\n{0}".
           format(book_words[:100]))
```

Almost magically, we get the following output:

```
There are 27336 words in the book. Only 2570 are unique.
The first 100 words are
['_i_', 'a', 'abide', 'able', 'about', 'above', 'absence', 'absurd',
 'acceptance', 'accident', 'accidentally', 'account', 'accounting',
 'accounts', 'accusation', 'accustomed', 'ache', 'across', 'act',
 'actually', 'ada', 'added', 'adding', 'addressed', 'addressing',
 'adjourn', 'adoption', 'advance', 'advantage', 'adventures',
 'advice', 'advisable', 'advise', 'affair', 'affectionately',
 'afford', 'afore', 'afraid', 'after', 'afterwards', 'again',
 'against', 'age', 'ago', 'agony', 'agree', 'ah', 'ahem', 'air',
 'airs', 'alarm', 'alarmed', 'alas', 'alice', 'alive', 'all',
 'allow', 'almost', 'alone', 'along', 'aloud', 'already', 'also',
 'altered', 'alternately', 'altogether', 'always', 'am', 'ambition',
 'among', 'an', 'ancient', 'and', 'anger', 'angrily', 'angry',
 'animal', 'animals', 'ann', 'annoy', 'annoyed', 'another',
 'answer', 'answered', 'answers', 'antipathies', 'anxious',
 'anxiously', 'any', 'anything', 'anywhere', 'appealed', 'appear',
 'appearance', 'appeared', 'appearing', 'applause', 'apple',
 'apples', 'arch']
```

Lewis Carroll was able to write a classic piece of literature using only 2570 different words!

14.6. Merging sorted lists

Suppose we have two sorted lists. Devise an algorithm to merge them together into a single sorted list.

A simple but inefficient algorithm could be to simply append the two lists together, and sort the result:

```
newlist = (xs + ys)
newlist.sort()
```

But this doesn't take advantage of the fact that the two lists are already sorted, and is going to have poor scalability and performance for very large lists.

Lets get some tests together first:

```
xs = [1,3,5,7,9,11,13,15,17,19]
ys = [4,8,12,16,20,24]
zs = xs+ys
zs.sort()
test(merge(xs, []) == xs)
test(merge([], ys) == ys)
test(merge([], []) == [])
test(merge(xs, ys) == zs)
test(merge([1,2,3], [3,4,5]) == [1,2,3,3,4,5])
test(merge(["a", "big", "cat"], ["big", "bite", "dog"]) ==
               ["a", "big", "big", "bite", "cat", "dog"])
```

Here is our merge algorithm:

```
def merge(xs, ys):
    """ merge sorted lists xs and ys. Return a sorted result """
    result = []
    xi = 0
    yi = 0

    while True:
        if xi >= len(xs):          # If xs list is finished,
            result.extend(ys[yi:]) # Add remaining items from ys
            return result          # And we're done.

        if yi >= len(ys):          # Same again, but swap roles
            result.extend(xs[xi:])
            return result

        # Both lists still have items, copy smaller item to result.
        if xs[xi] <= ys[yi]:
            result.append(xs[xi])
            xi += 1
        else:
            result.append(ys[yi])
            yi += 1
```

The algorithm works as follows: we create a result list, and keep two indexes, one into each list (lines 3-5). On each iteration of the loop, whichever list item is smaller is copied to the result list, and that list's index is advanced. As soon as either index reaches the end of its list, we copy all the remaining items from the other list into the result, which we return.

14.7. Alice in Wonderland, again!

Underlying the algorithm for merging sorted lists is a deep pattern of computation that is widely reusable. The pattern essence is *"Run through the lists always processing the smallest remaining items from each, with these cases to consider:"*

- What should we do when either list has no more items?
- What should we do if the smallest items from each list are equal to each other?
- What should we do if the smallest item in the first list is smaller than the smallest one the second list?
- What should we do in the remaining case?

Lets assume we have two sorted lists. Exercise your algorithmic skills by adapting the merging algorithm pattern for each of these cases:

- Return only those items that are present in both lists.
- Return only those items that are present in the first list, but not in the second.
- Return only those items that are present in the second list, but not in the first.
- Return items that are present in either the first or the second list.
- Return items from the first list that are not eliminated by a matching element in the second list. In this case, an item in the second list "knocks out" just one matching item in the first list. This operation is sometimes called *bagdiff*. For example `bagdiff([5,7,11,11,11,12,13], [7,8,11])` would return `[5,11,11,12,13]`

In the previous section we sorted the words from the book, and eliminated duplicates. Our vocabulary is also sorted. So third case above — find all items in the second list that are not in the first list, would be another way to implement `find_unknown_words`. Instead of searching for every word in the dictionary (either by linear or binary search), why not use a variant of the merge to return the words that occur in the book, but not in the vocabulary.

```
def find_unknowns_merge_pattern(vocab, wds):
    """ Both the vocab and wds must be sorted.  Return a new
        list of words from wds that do not occur in vocab.
    """

    result = []
    xi = 0
    yi = 0

    while True:
        if xi >= len(vocab):
            result.extend(wds[yi:])
            return result

        if yi >= len(wds):
            return result

        if vocab[xi] == wds[yi]:  # Good, word exists in vocab
            yi += 1

        elif vocab[xi] < wds[yi]: # Move past this vocab word,
            xi += 1

        else:                     # Got word that is not in vocab
            result.append(wds[yi])
            yi += 1
```

Now we put it all together:

Even more stunning performance here:

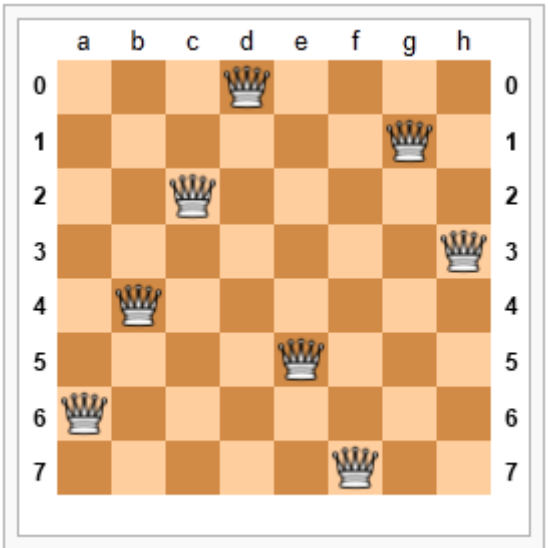
```
There are 828 unknown words.
That took 0.0410 seconds.
```

Let's review what we've done. We started with a word-by-word linear lookup in the vocabulary that ran in about 50 seconds. We implemented a clever binary search, and got that down to 0.22 seconds, more than 200
times faster. But then we did something even better: we sorted the words from the book, eliminated duplicates, and used a merging pattern to find words from the book that were not in the dictionary. This was about five times faster than even the binary lookup algorithm. At the end of the chapter our algorithm is more than a 1000 times faster than our first attempt!

That is what we can call a good day at the office!

14.8. Eight Queens puzzle, part 1

As told by Wikipedia, *"The eight queens puzzle is the problem of placing eight chess queens on an 8x8 chessboard so that no two queens attack each other. Thus, a solution requires that no two queens share
the same row, column, or diagonal."*



One solution to the eight queens puzzle

Please try this yourself, and find a few more solutions by hand.

We'd like to write a program to find solutions to this puzzle. In fact, the puzzle generalizes to placing N queens on an NxN board, so we're going to think about the general case, not just the 8x8 case. Perhaps we
can find solutions for 12 queens on a 12x12 board, or 20 queens on a 20x20 board.

How do we approach a complex problem like this? A good starting point is to think about our *data structures* — how exactly do we plan to represent the state of the chessboard and its queens in our program? Once we have some handle on what our puzzle is going to look like in memory, we can begin to think about the functions and logic we'll need to solve the puzzle, i.e. how do we put another queen onto the board
somewhere, and to check whether it clashes with any of the queens already on the board.

The steps of finding a good representation, and then finding a good algorithm to operate on the data cannot always be done independently of each other. As you think about the operations you require, you may want
to change or reorganize the data somewhat to make it easier to do the operations you need.

This relationship between algorithms and data was elegantly expressed in the title of a book *Algorithms + Data Structures = Programs*, written by one of the pioneers in Computer Science,

Niklaus Wirth, the inventor
of Pascal.

Let's brainstorm some ideas about how a chessboard and queens could be represented in memory.

- A two dimensional matrix (a list of 8 lists, each containing 8 squares) is one possibility. At each square of the board would like to know whether it contains a queen or not — just two possible states for each square — so perhaps each element in the lists could be True or False, or, more simply, 0 or 1.

Our state for the solution above could then have this data representation:

```
bd1 = [[0,0,0,1,0,0,0,0],
       [0,0,0,0,0,0,1,0],
       [0,0,1,0,0,0,0,0],
       [0,0,0,0,0,0,0,1],
       [0,1,0,0,0,0,0,0],
       [0,0,0,0,1,0,0,0],
       [1,0,0,0,0,0,0,0],
       [0,0,0,0,0,1,0,0]]
```

You should also be able to see how the empty board would be represented, and you should start to imagine what operations or changes you'd need to make to the data to place another queen somewhere on the board.

- Another idea might be to keep a list of coordinates of where the queens are. Using the notation in the illustration, for example, we could represent the state of that solution as:

```
bd2 = [ "a6", "b4", "c2", "d0", "e5", "f7", "g1", "h3" ]
```

- We could make other tweaks to this — perhaps each element in this list should rather be a tuple, with integer coordinates for both axes. And being good computer scientists, we'd probably start numbering each axis from 0 instead of at 1. Now our representation could be:

```
bd3 = [(0,6), (1,4), (2,2), (3,0), (4,5), (5,7), (6,1), (7,3)]
```

- Looking at this representation, we can't help but notice that the

first coordinates are `0,1,2,3,4,5,6,7` and they correspond exactly to the index position of the pairs in the list. So we could discard them, and come up with this really compact alternative representation of the solution:

```
bd4 = [6, 4, 2, 0, 5, 7, 1, 3]
```

This will be what we'll use, let's see where that takes us.

Let us now take some grand insight into the problem. Do you think it is a coincidence that there are no repeated numbers in the solution? The solution `[6,4,2,0,5,7,1,3]` contains the numbers `0,1,2,3,4,5,6,7`, but none are duplicated! Could other solutions contain duplicate numbers, or not?

A little thinking should convince you that there can never be duplicate numbers in a solution: the numbers represent the row on which the queen is placed, and because we are never permitted to put two queens in the
same row, no solution will ever have duplicate row numbers in it.

Our key insight

In our representation, any solution to the N queens problem musttherefore be a permutation of the numbers [0 .. N-1].

Note that not all permutations are solutions. For example, `[0,1,2,3,4,5,6,7]` has all queens on the same diagonal.

Wow, we seem to be making progress on this problem merely by thinking, rather than coding!

Our algorithm should start taking shape now. We can start with the list [0..N-1], generate various permutations of that list, and check each permutation to see if it has any clashes (queens that are on the same diagonal). If it has no clashes, it is a solution, and we can print it.

Let us be precise and clear on this issue: if we only use permutations of the rows, and we're using our compact representation, no queens can clash on either rows or columns, and we don't even have to concern
ourselves with those cases. So the only clashes we need to test for are clashes on the diagonals.

It sounds like a useful function will be one that can test if two queens share a diagonal. Each queen is on some (x,y) position. So does the queen at (5,2) share a diagonal with the one at (2,0)? Does (5,2) clash
with (3,0)?

```
test(not share_diagonal(5,2,2,0))
test(share_diagonal(5,2,3,0))
test(share_diagonal(5,2,4,3))
test(share_diagonal(5,2,4,1))
```

A little geometry will help us here. A diagonal has a slope of either 1 or -1. The question we really want to ask is *is their distance between them the same in the x and the y direction?* If it is, they share a
diagonal. Because diagonals can be to the left or right, it will make sense for this program to use the absolute distance in each direction:

```
def share_diagonal(x0, y0, x1, y1):
    """ Is (x0, y0) on a shared diagonal with (x1, y1)? """
    dy = abs(y1 - y0)        # Calc the absolute y distance
    dx = abs(x1 - x0)        # CXalc the absolute x distance
    return dx == dy          # They clash if dx == dy
```

If you copy the code and run it, you'll be happy to learn that the tests pass!

Now let's consider how we construct a solution by hand. We'll put a queen somewhere in the first column, then place one in the second column, only if it does not clash with the one already on the board. And
then we'll put a third one on, checking it against the two queens already to its left. When we consider the queen on column 6, we'll need to check for clashes against those in all the columns to its left, i.e. in columns 0,1,2,3,4,5.

So the next building block is a function that, given a partially completed puzzle, can check whether the queen at column `c` clashes with any of the queens to its left, at columns 0,1,2,..c-1:

```
# Solutions cases that should not have any clashes
test(not col_clashes([6,4,2,0,5], 4))
test(not col_clashes([6,4,2,0,5,7,1,3], 7))

# More test cases that should mostly clash
test(col_clashes([0,1], 1))
test(col_clashes([5,6], 1))
test(col_clashes([6,5], 1))
test(col_clashes([0,6,4,3], 3))
test(col_clashes([5,0,7], 2))
test(not col_clashes([2,0,1,3], 1))
test(col_clashes([2,0,1,3], 2))
```

Here is our function that makes them all pass:

```
def col_clashes(bs, c):
    """ Return True if the queen at column c clashes
         with any queen to its left.
    """
    for i in range(c):     # Look at all columns to the left of c
          if share_diagonal(i, bs[i], c, bs[c]):
              return True

    return False           # No clashes - col c has a safe placement.
```

Finally, we're going to give our program one of our permutations — i.e. all queens placed somewhere, one on each row, one on each column. But does the permutation have any diagonal clashes?

```
test(not has_clashes([6,4,2,0,5,7,1,3])) # Solution from above
test(has_clashes([4,6,2,0,5,7,1,3]))     # Swap rows of first two
test(has_clashes([0,1,2,3]))             # Try small 4x4 board
test(not has_clashes([2,0,3,1]))         # Solution to 4x4 case
```

And the code to make the tests pass:

```
def has_clashes(the_board):
    """ Determine whether we have any queens clashing on the diagonals.
        We're assuming here that the_board is a permutation of column
        numbers, so we're not explicitly checking row or column clashes.
    """
    for col in range(1,len(the_board)):
        if col_clashes(the_board, col):
            return True
    return False
```

Summary of what we've done so far: we now have a powerful function called `has_clashes` that can tell if a configuration is a solution to the queens puzzle. Let's get on now with generating lots of permutations
and finding solutions!

14.9. Eight Queens puzzle, part 2

This is the fun, easy part. We could try to find all permutations of `[0,1,2,3,4,5,6,7]` — that might be algorithmically challenging, and would be a *brute force* way of tackling the problem. We just try everything, and find all possible solutions.

Of course we know there are N! permutations of N things, so we can get an early idea of how long it would take to search all of them for all solutions. Not too long at all, actually -8! is only 40320 different
cases to check out. This is vastly better than starting with 64 places to put eight queens. If you do the sums for how many ways can you choose 8 of the 64 squares for your queens, the formula (called *N choose k*
where you're choosing k=8 squares of the available N=64) yields a whopping 4426165368, obtained from (64! / (8! x 56!)).

So our earlier key insight — that we only need to consider permutations — has reduced what we call the *problem space* from about 4.4 billion cases to just 40320!

We're not even going to explore all those, however. When we introduced the random number module, we learned that it had a `shuffle` method that randomly permuted a list of items. So we're going to write a "random"
algorithm to find solutions to the N queens problem. We'll begin with the permutation [0,1,2,3,4,5,6,7] and we'll repeatedly shuffle the list, and test each to see if it works! Along the way we'll count how many attempts we need before we find each solution, and we'll find 10 solutions (we could hit the same solution more than once, because shuffle is random!):

```
def main():
    import random
    rng = random.Random()   # Instantiate a generator

    bd = list(range(8))     # Generate the initial permutation
    num_found = 0
    tries = 0
    while num_found < 10:
       rng.shuffle(bd)
       tries += 1
       if not has_clashes(bd):
           print("Found solution {0} in {1} tries.".format(bd, tries))
           tries = 0
           num_found += 1

main()
```

Almost magically, and at great speed, we get this:

```
Found solution [3, 6, 2, 7, 1, 4, 0, 5] in 693 tries.
Found solution [5, 7, 1, 3, 0, 6, 4, 2] in 82 tries.
Found solution [3, 0, 4, 7, 1, 6, 2, 5] in 747 tries.
Found solution [1, 6, 4, 7, 0, 3, 5, 2] in 428 tries.
Found solution [6, 1, 3, 0, 7, 4, 2, 5] in 376 tries.
Found solution [3, 0, 4, 7, 5, 2, 6, 1] in 204 tries.
Found solution [4, 1, 7, 0, 3, 6, 2, 5] in 98 tries.
Found solution [3, 5, 0, 4, 1, 7, 2, 6] in 64 tries.
Found solution [5, 1, 6, 0, 3, 7, 4, 2] in 177 tries.
Found solution [1, 6, 2, 5, 7, 4, 0, 3] in 478 tries.
```

Here is an interesting fact. On an 8x8 board, there are known to be 92 different solutions to this puzzle. We are randomly picking one of 40320 possible permutations of our representation. So our chances of picking a
solution on each try are 92/40320. Put another way, on average we'll need 40320/92 tries — about 438.26 — before we stumble across a solution. The number of tries we printed looks like our experimental data agrees quite nicely with our theory!

Save this code for later.

In the chapter on PyGame we plan to write a module to draw the board with its queens, and integrate that module with this code.

14.10. Glossary

binary search
A famous algorithm that searches for a target in a sorted list. Each probe in the list allows us to discard half the remaining items, so the algorithm is very efficient.

linear
Relating to a straight line. Here, we talk about graphing how the time taken by an algorithm depends on the size of the data it is processing. Linear algorithms have straight-line graphs that can describe this relationship.

linear search
A search that probes each item in a list or sequence, from first, until it finds what it is looking for. It is used for searching for a target in unordered lists of items.

Merge algorithm
An efficient algorithm that merges two already sorted lists, to produce a sorted list result. The merge algorithm is really a pattern of

computation that can be adapted and reused for various other scenarios, such as finding words that are in a book, but not in a vocabulary.

probe
Each time we take a look when searching for an item is called a probe. In our chapter on Iteration we also played a guessing game where the computer tried to guess the user's secret number. Each of those tries would also be called a probe.

test-driven development (TDD)
A software development practice which arrives at a desired feature through a series of small, iterative steps motivated by automated tests which are *written first* that express increasing refinements of the desired feature. (see the Wikipedia article on Test-driven development[15] for more information.)

14.11. Exercises

1. The section in this chapter called Alice in Wonderland, again started with the observation that the merge

algorithm uses a pattern that can be reused in other situations. Adapt the merge algorithm to write each of these functions, as was suggested there:

1. Return only those items that are present in both lists.
2. Return only those items that are present in the first list, but not in the second.
3. Return only those items that are present in the second list, but not in the first.
4. Return items that are present in either the first or the second list.
5. Return items from the first list that are not eliminated by a matching element in the second list. In this case, an item in the second list "knocks out" just one matching item in the first list. This operation is sometimes called *bagdiff*. For example `bagdiff([5,7,11,11,11,12,13], [7,8,11])` would return `[5,11,11,12,13]`

2. Modify the queens program to solve some boards of size 4, 12, and 16. What is the maximum size puzzle you

[15] http://en.wikipedia.org/wiki/Test_driven_development

can usually solve in under a minute?

3. Adapt the queens program so that we keep a list of solutions that have already printed, so that we don't

print the same solution more than once.

4. Chess boards are symmetric: if we have a solution to the queens problem, its mirror solution — either

flipping the board on the X or in the Y axis, is also a solution. And giving the board a 90 degree, 180 degree, or 270 degree rotation is also a solution. In some sense, solutions that are just mirror images or rotations of other solutions — in the same family —are less interesting than the unique "core cases". Of the 92 solutions for the 8 queens problem, there are only 12 unique families if you take rotations and mirror images into account. Wikipedia has some fascinating stuff about this.

1. Write a function to mirror a solution in the Y axis,

2. Write a function to mirror a solution in the X axis,

3. Write a function to rotate a solution by 90 degrees anti-clockwise, and use this to provide 180 and 270 degree rotations too.

4. Write a function which is given a solution, and it generates the family of symmetries for that solution. For example, the symmetries of `[0,4,7,5,2,6,1,3]` are :

[[0,4,7,5,2,6,1,3],[7,1,3,0,6,4,2,5],
[4,6,1,5,2,0,3,7],[2,5,3,1,7,4,6,0],
[3,1,6,2,5,7,4,0],[0,6,4,7,1,3,5,2],
[7,3,0,2,5,1,6,4],[5,2,4,6,0,3,1,7]]

5. Now adapt the queens program so it won't list solutions that are in the same family. It only prints solutions from unique families.

5. Every week a computer scientist buys four lotto tickets. She always chooses the same prime numbers, with

the hope that if she ever hits the jackpot, she will be able to go onto TV and Facebook and tell everyone her secret. This will suddenly create widespread public interest in prime numbers, and will be the trigger event that ushers in a new age of enlightenment. She represents her weekly tickets in
Python as a list of lists:

```
my_tickets = [ [ 7, 17, 37, 19, 23, 43],
[ 7, 2, 13, 41, 31, 43],
[ 2, 5, 7, 11, 13, 17],
[13, 17, 37, 19, 23, 43] ]
```

Complete these exercises.

1. Each lotto draw takes six random balls, numbered from 1 to 49. Write a function to return a lotto draw.

2. Write a function that compares a single ticket and a draw, and returns the number of correct picks on that ticket:

```
test(lotto_match([42,4,7,11,1,13], [2,5,7,11,13,17]) == 3)
```

3. Write a function that takes a list of tickets and a draw, and returns a list telling how many picks were correct on each ticket:

```
test(lotto_matches([42,4,7,11,1,13], my_tickets) == [1,2,3,1])
```

4. Write a function that takes a list of integers, and returns the number of primes in the list:

```
test(primes_in([42, 4, 7, 11, 1, 13]) == 3)
```

5. Write a function to discover whether the computer scientist has missed any prime numbers in her selection of the four tickets. Return a list of all primes that she has missed:

```
test(prime_misses(my_tickets) == [3, 29, 47])
```

6. Write a function that repeatedly makes a new draw, and compares the draw to the four tickets.

1. Count how many draws are needed until one of the computer scientist's tickets has at least 3 correct picks. Try the experiment twenty times, and average out the number of draws needed.
2. How many draws are needed, on average, before she gets at least 4 picks correct?
3. How many draws are needed, on average, before she gets at least 5 correct? (Hint: this might take a while. It would be nice if you could print some dots, like a progress bar, to show when each of the 20 experiments has completed.)

Notice that we have difficulty constructing test cases here, because our random numbers are not deterministic. Automated testing only really works if you already know what the answer should be!

6. Read *Alice in Wonderland*. You can read the plain text version we have with this textbook, or if you

have e-book reader software on your PC, or a Kindle, iPhone, Android, etc. you'll be able to find a suitable version for your device at http://www.gutenberg.org. They also have html and pdf versions, with pictures, and thousands of other classic books!

Chapter 15: Classes and Objects — the Basics

15.1. Object-oriented programming

Python is an **object-oriented programming language**, which means that it provides features that support object-oriented programming[16] (**OOP**).

Object-oriented programming has its roots in the 1960s, but it wasn't until the mid 1980s that it became the main programming paradigm[17] used in the creation of new software. It was developed as a way to handle the rapidly increasing size and complexity of software systems, and to make it easier to modify these large and complex systems over time.

Up to now, most of the programs we have been writing use a procedural programming[18] paradigm. In procedural programming the focus is on writing functions or
procedures which operate on data. In object-oriented programming the focus is on the creation of **objects** which contain both data and functionality together. (We have seen turtle objects, string objects,
and random number generators, to name a few places where we've already worked with objects.)

Usually, each object definition corresponds to some object or concept in the real world, and the functions that operate on that object correspond to the ways real-world objects interact.

15.2. User-defined compound data types

We've already seen classes like `str`, `int`, `float` and `Turtle`. We are now ready to create our own user-defined class: the `Point`.

Consider the concept of a mathematical point. In two dimensions, a point is two numbers (coordinates) that are treated collectively as a single object. Points are often written in parentheses with a comma separating
the coordinates. For example, `(0, 0)` represents the origin, and `(x, y)` represents the point `x` units to the right and `y` units up from the origin.

Some of the typical operations that one associates with points might be calculating the distance of a point from the origin, or from another point, or finding a midpoint of two points, or asking if a

[16] http://en.wikipedia.org/wiki/Object-oriented_programming
[17] http://en.wikipedia.org/wiki/Programming_paradigm
[18] http://en.wikipedia.org/wiki/Procedural_programming

point falls
within a given rectangle or circle. We'll shortly see how we can organize these together with the data.

A natural way to represent a point in Python is with two numeric values. The question, then, is how to group these two values into a compound object. The quick and dirty solution is to use a tuple, and for some
applications that might be a good choice.

An alternative is to define a new **class**. This approach involves a bit more effort, but it has advantages that will be apparent soon. We'll want our points to each have an `x` and a `y` attribute, so our first class definition looks like this:

```
class Point:
    """ Point class represents and manipulates x,y coords. """

    def __init__(self):
        """ Create a new point at the origin """
        self.x = 0
        self.y = 0
```

Class definitions can appear anywhere in a program, but they are usually near the beginning (after the `import` statements). Some programmers and languages prefer to put every class in a module of its own — we won't do that here. The syntax rules for a class definition are the same as for other compound statements. There is a header which begins with the keyword, `class`, followed by the name of the class, and ending with a
colon. Indentation levels tell us where the class ends.

If the first line after the class header is a string, it becomes the docstring of the class, and will be recognized by various tools. (This is also the way docstrings work in functions.)

Every class should have a method with the special name `__init__`. This **initializer method** is automatically called whenever a new instance of `Point` is created. It gives the programmer the opportunity to set up the attributes required within the new instance by giving them their initial state/values. The `self` parameter (we could choose any other name, but `self` is the convention) is automatically set to reference the newly created object that needs to be initialized.

So let's use our new `Point` class now:

```
p = Point()         # Instantiate an object of type Point
q = Point()         # Make a second point

print(p.x, p.y, q.x, q.y)  # Each point object has its own x and y
```

This program prints:

```
0 0 0 0
```

because during the initialization of the objects, we created two attributes called `x` and `y` for each, and gave them both the value 0.

This should look familiar — we've used classes before to create more than one object:

```
from turtle import Turtle

tess = Turtle()     # Instantiate objects of type Turtle
alex = Turtle()
```

The variables `p` and `q` are assigned references to two new `Point` objects. A function like `Turtle` or `Point` that creates a new object instance is called a **constructor**, and every class automatically provides a constructor function which is named the same as the class.

It may be helpful to think of a class as a *factory* for making objects. The class itself isn't an instance of a point, but it contains the machinery to make point instances. Every time we call the constructor, we're asking the factory to make us a new object. As the object comes off the production line, its initialization method is executed to get the object properly set up with its factory default settings.

The combined process of "make me a new object" and "get its settings initialized to the factory default settings" is called **instantiation**.

15.3. Attributes

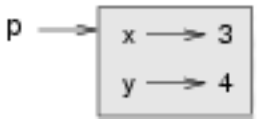
Like real world objects, object instances have both attributes and methods.

We can modify the attributes in an instance using dot notation:

```
>>> p.x = 3
>>> p.y = 4
```

Both modules and instances create their own namespaces, and the syntax for accessing names contained in each, called **attributes**, is the same. In this case the attribute we are selecting is a data item from an
instance.

The following state diagram shows the result of these assignments:



The variable `p` refers to a `Point` object, which contains two attributes. Each attribute refers to a number.

We can access the value of an attribute using the same syntax:

```
>>> print(p.y)
4
>>> x = p.x
>>> print(x)
3
```

The expression `p.x` means, "Go to the object `p` refers to and get the value of `x`". In this case, we assign that value to a variable named `x`. There is no conflict between the variable `x` (in the global namespace here) and the attribute `x` (in the namespace belonging to the instance). The purpose of dot notation is to fully qualify which variable we are referring to unambiguously.

We can use dot notation as part of any expression, so the following statements are legal:

```
print("(x={0}, y={1})".format(p.x, p.y))
distance_squared_from_origin = p.x * p.x + p.y * p.y
```

The first line outputs `(x=3, y=4)`. The second line calculates the value 25.

15.4. Improving our initializer

To create a point at position (7, 6) currently needs three lines of code:

```
p = Point()
p.x = 7
p.y = 6
```

We can make our class constructor more general by placing extra parameters into the `__init__`-method, as shown in this example:

```
class Point:
    """ Point class represents and manipulates x,y coords. """

    def __init__(self, x=0, y=0):
        """ Create a new point at x, y """
        self.x = x
        self.y = y

# Other statements outside the class continue below here.
```

The `x` and `y` parameters here are both optional. If the caller does not supply arguments, they'll get the default values of 0. Here is our improved class in action:

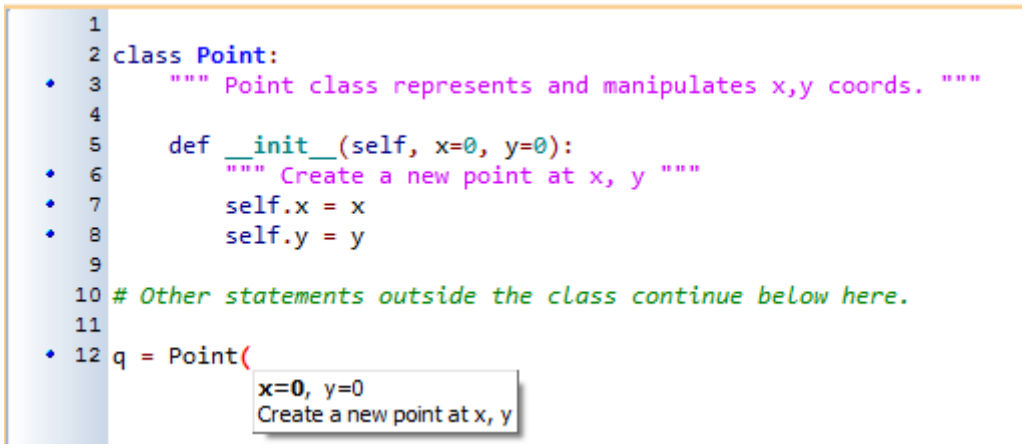
```
>>> p = Point(4, 2)
>>> q = Point(6, 3)
>>> r = Point()       # r represents the origin (0, 0)
>>> print(p.x, q.y, r.x)
4 3 0
```

Technically speaking ...

If we are really fussy, we would argue that the `__init__` method's docstring is inaccurate. `__init__` doesn't *create* the object (i.e. set aside memory for it), — it just initializes the object to its factory-default settings after its creation.

But tools like PyScripter understand that instantiation — creation and initialization — happen together, and they choose to display the *initializer's* docstring as the tooltip to guide the programmer that calls the class constructor.

So we're writing the docstring so that it makes the most sense when it pops up to help the programmer who is using our `Point` class:



15.5. Adding other methods to our class

The key advantage of using a class like `Point` rather than a simple tuple `(6, 7)` now becomes apparent. We can add methods to the `Point` class that are sensible operations for points, but which may not be
appropriate for other tuples like `(25, 12)` which might represent, say, a day and a month, e.g. Christmas day. So being able to calculate the distance from the origin is sensible for points, but not for (day,
month) data. For (day, month) data, we'd like different operations, perhaps to find what day of the week it will fall on in 2020.

Creating a class like `Point` brings an exceptional amount of "organizational power" to our programs,

and to our thinking. We can group together the sensible operations, and the kinds of data they apply to, and each instance of the class can have its own state.

A **method** behaves like a function but it is invoked on a specific instance, e.g. `tess.right(90)`. Like a data attribute, methods are accessed using dot notation.

Let's add another method, `distance_from_origin`, to see better how methods work:

```
class Point:
    """ Create a new Point, at coordinates x, y """

    def __init__(self, x=0, y=0):
        """ Create a new point at x, y """
        self.x = x
        self.y = y

    def distance_from_origin(self):
        """ Compute my distance from the origin """
        return ((self.x ** 2) + (self.y ** 2)) ** 0.5
```

Let's create a few point instances, look at their attributes, and call our new method on them: (We must run our program first, to make our `Point` class available to the interpreter.)

```
>>> p = Point(3, 4)
>>> p.x
3
>>> p.y
4
>>> p.distance_from_origin()
5.0
>>> q = Point(5, 12)
>>> q.x
5
>>> q.y
12
>>> q.distance_from_origin()
13.0
>>> r = Point()
>>> r.x
0
>>> r.y
0
>>> r.distance_from_origin()
0.0
```

When defining a method, the first parameter refers to the instance being manipulated. As already noted, it is customary to name this parameter `self`.

Notice that the caller of `distance_from_origin` does not explicitly supply an argument to match the `self` parameter — this is done for us, behind our back.

15.6. Instances as arguments and parameters

We can pass an object as an argument in the usual way. We've already seen this in some of the turtle examples, where we passed the turtle to some function like `draw_bar` in the chapter titled Conditionals, so that the function could control and use whatever turtle instance we passed to it.

Be aware that our variable only holds a reference to an object, so passing `tess` into a function creates an alias: both the caller and the called function now have a reference, but there is only one turtle!

Here is a simple function involving our new `Point` objects:

```
def print_point(pt):
    print("({0}, {1})".format(pt.x, pt.y))
```

`print_point` takes a point as an argument and formats the output in whichever way we choose. If we call `print_point(p)` with point `p` as defined previously, the output is `(3, 4)`.

15.7. Converting an instance to a string

Most object-oriented programmers probably would not do what we've just done in `print_point`. When we're working with classes and objects, a preferred alternative is to add a new method to the class. And we don't
like chatterbox methods that call `print`. A better approach is to have a method so that every instance can produce a string representation of itself. Let's initially call it `to_string`:

```
class Point:
    # ...

    def to_string(self):
        return "({0}, {1})".format(self.x, self.y)
```

Now we can say:

```
>>> p = Point(3, 4)
>>> print(p.to_string())
(3, 4)
```

But don't we already have a `str` type converter that can turn our object into a string? Yes! And doesn't `print` automatically use this when printing things? Yes again! But these automatic mechanisms do not
yet do exactly what we want:

```
>>> str(p)
'<__main__.Point object at 0x01F9AA10>'
>>> print(p)
'<__main__.Point object at 0x01F9AA10>'
```

Python has a clever trick up its sleeve to fix this. If we call our new method `__str__` instead of `to_string`, the Python interpreter will use our code whenever it needs to convert a `Point` to a string. Let's re-do this again, now:

```
class Point:
    # ...

    def __str__(self):    # All we have done is renamed the method
        return "({0}, {1})".format(self.x, self.y)
```

and now things are looking great!

```
>>> str(p)     # Python now uses the __str__ method that we wrote.
(3, 4)
>>> print(p)
(3, 4)
```

15.8. Instances as return values

Functions and methods can return instances. For example, given two `Point` objects, find their midpoint. First we'll write this as a regular function:

```
def midpoint(p1, p2):
    """ Return the midpoint of points p1 and p2 """
    mx = (p1.x + p2.x)/2
    my = (p1.y + p2.y)/2
    return Point(mx, my)
```

The function creates and returns a new `Point` object:

```
>>> p = Point(3, 4)
>>> q = Point(5, 12)
>>> r = midpoint(p, q)
>>> r
(4.0, 8.0)
```

Now let us do this as a method instead. Suppose we have a point object, and wish to find the midpoint halfway between it and some other target point:

```
class Point:
   # ...

   def halfway(self, target):
        """ Return the halfway point between myself and the target """
        mx = (self.x + target.x)/2
        my = (self.y + target.y)/2
        return Point(mx, my)
```

This method is identical to the function, aside from some renaming. It's usage might be like this:

```
>>> p = Point(3, 4)
>>> q = Point(5, 12)
>>> r = p.halfway(q)
>>> r
(4.0, 8.0)
```

While this example assigns each point to a variable, this need not be done. Just as function calls are composable, method calls and object instantiation are also composable, leading to this alternative that uses
no variables:

```
>>> print(Point(3, 4).halfway(Point(5, 12)))
(4.0, 8.0)
```

15.9. A change of perspective

The original syntax for a function call, `print_time(current_time)`, suggests that the function is the active agent. It says something like, *"Hey, print_time! Here's an object for you to print."*

In object-oriented programming, the objects are considered the active agents. An invocation like `current_time.print_time()` says *"Hey current_time! Please print yourself!"*

In our early introduction to turtles, we used an object-oriented style, so that we said `tess.forward(100)`, which asks the turtle to move itself forward by the given number of steps.

This change in perspective might be more polite, but it may not initially be obvious that it is useful. But sometimes shifting responsibility from the functions onto the objects makes it possible to write more versatile functions, and makes it easier to maintain and reuse code.

The most important advantage of the object-oriented style is that it fits our mental chunking and real-life experience more accurately. In real life our `cook` method is part of our microwave oven — we don't
have a `cook` function sitting in the corner of the kitchen, into which we pass the microwave! Similarly, we use the cellphone's own methods to send an sms, or to change its state to silent. The functionality of
real-world objects tends to be tightly bound up inside the objects themselves. OOP allows us to accurately mirror this when we organize our programs.

15.10. Objects can have state

Objects are most useful when we also need to keep some state that is updated from time to time. Consider a turtle object. Its state consists of things like its position, its heading, its color, and its shape. A
method like `left(90)` updates the turtle's heading, `forward` changes its position, and so on.

For a bank account object, a main component of the state would be the current balance, and perhaps a log of all transactions. The methods would allow us to query the current balance, deposit new funds, or make
a payment. Making a payment would include an amount, and a description, so that this could be added to the transaction log. We'd also want a method to show the transaction log.

15.11. Glossary

attribute
One of the named data items that makes up an instance.

class
A user-defined compound type. A class can also be thought of as a template for the objects that are instances of it. (The iPhone is a class. By December 2010, estimates are that 50 million instances had been sold!)

constructor
Every class has a "factory", called by the same name as the class, for making new instances. If the class has an *initializer method*, this method is used to get the attributes (i.e. the state) of the new object properly set up.

initializer method
A special method in Python (called `__init__`) that is invoked automatically to set a newly created object's attributes to their initial (factory-default) state.

instance
An object whose type is of some class. Instance and object are used interchangeably.

instantiate
To create an instance of a class, and to run its initializer.

method
A function that is defined inside a class definition and is invoked on instances of that class.

object
A compound data type that is often used to model a thing or concept in the real world. It bundles together the data and the operations that are relevant for that kind of data. Instance and object are used interchangeably.

object-oriented programming
A powerful style of programming in which data and the operations that manipulate it are organized into objects.

object-oriented language
A language that provides features, such as user-defined classes and inheritance, that facilitate object-oriented programming.

15.12. Exercises

1. Rewrite the `distance` function from the chapter titled *Fruitful functions* so that it takes two `Points` as parameters instead of four numbers.

2. Add a method `reflect_x` to `Point` which returns a new `Point`, one which is the reflection of the point about the x-axis. For example, `Point(3, 5).reflect_x()` is (3, -5)

3. Add a method `slope_from_origin` which returns the slope of the line joining the origin to the point. For example:

```
>>> Point(4, 10).slope_from_origin()
2.5
```

What cases will cause this method to fail?

4. The equation of a straight line is "y = ax + b", (or perhaps "y = mx + c"). The coefficients a and b completely describe the line. Write a method in the `Point` class so that if a point instance is given another point, it will compute the equation of the straight ine joining the two points. It must return the two coefficients as a tuple of two values. For example, :

```
>>> print(Point(4, 11).get_line_to(Point(6, 15)))
>>> (2, 3)
```

This tells us that the equation of the line joining the two points is "y = 2x + 3". When will this method fail?

5. Given four points that fall on the circumference of a circle, find the midpoint of the circle. When will this function fail?

Hint: You *must* know how to solve the geometry problem *before* you think of going anywhere near programming. You cannot program a solution to a problem if you don't understand what you want the computer to do!

6. Create a new class, SMS_store. The class will instantiate SMS_store objects, similar to an inbox or

outbox on a cellphone:

```
my_inbox = SMS_store()
```

This store can hold multiple SMS messages (i.e. its internal state will just be a list of messages). Each message will be represented as a tuple:

```
(has_been_viewed, from_number, time_arrived, text_of_SMS)
```

The inbox object should provide these methods:

```
my_inbox.add_new_arrival(from_number, time_arrived, text_of_SMS)
  # Makes new SMS tuple, inserts it after other messages
  # in the store. When creating this message, its
  # has_been_viewed status is set False.

my_inbox.message_count()
  # Returns the number of sms messages in my_inbox

my_inbox.get_unread_indexes()
  # Returns list of indexes of all not-yet-viewed SMS messages

my_inbox.get_message(i)
  # Return (from_number, time_arrived, text_of_sms) for message[i]
  # Also change its state to "has been viewed".
  # If there is no message at position i, return None

my_inbox.delete(i)     # Delete the message at index i
my_inbox.clear()       # Delete all messages from inbox
```

Write the class, create a message store object, write tests for these methods, and implement the methods.

Chapter 16: Classes and Objects — Digging a little deeper

16.1. Rectangles

Let's say that we want a class to represent a rectangle which is located somewhere in the XY plane. The question is, what information do we have to provide in order to specify such a rectangle? To keep things simple, assume that the rectangle is oriented either vertically or horizontally, never at an angle.

There are a few possibilities: we could specify the center of the rectangle (two coordinates) and its size (width and height); or we could specify one of the corners and the size; or we could specify two opposing corners. A conventional choice is to specify the upper-left corner of the rectangle, and the size.

Again, we'll define a new class, and provide it with an initializer and a string converter method:

```
class Rectangle:
    """ A class to manufacture rectangle objects """

    def __init__(self, posn, w, h):
        """ Initialize rectangle at posn, with width w, height h """
        self.corner = posn
        self.width = w
        self.height = h

    def __str__(self):
        return  "({0}, {1}, {2})"
                  .format(self.corner, self.width, self.height)

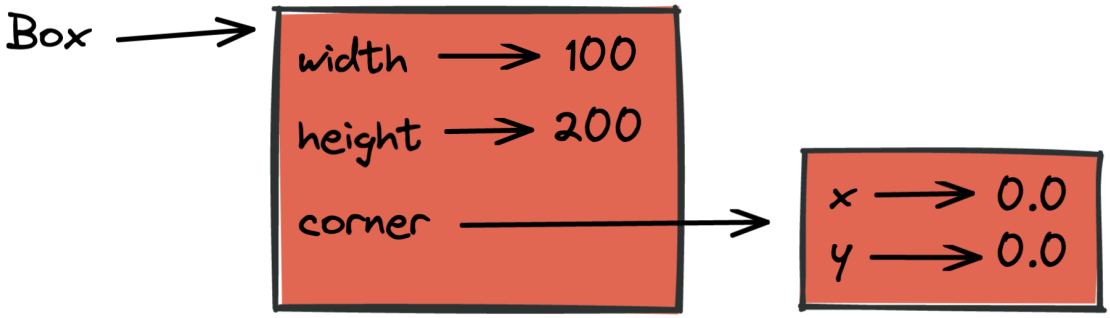
box = Rectangle(Point(0, 0), 100, 200)
bomb = Rectangle(Point(100, 80), 5, 10)    # In my video game
print("box: ", box)
print("bomb: ", bomb)
```

To specify the upper-left corner, we have embedded a `Point` object (as we used it in the previous chapter) within our new `Rectangle` object! We create two new `Rectangle` objects, and then print them producing:

```
box: ((0, 0), 100, 200)
bomb: ((100, 80), 5, 10)
```

The dot operator composes. The expression `box.corner.x` means, "Go to the object that `box` refers to and select its attribute named `corner`, then go to that object and select its attribute named `x`".

The figure shows the state of this object:



16.2. Objects are mutable

We can change the state of an object by making an assignment to one of its attributes. For example, to grow the size of a rectangle without changing its position, we could modify the values of `width` and `height`:

```
box.width += 50
box.height += 100
```

Of course, we'd probably like to provide a method to encapsulate this inside the class. We will also provide another method to move the position of the rectangle elsewhere:

```
class Rectangle:
    # ...

    def grow(self, delta_width, delta_height):
        """ Grow (or shrink) this object by the deltas """
        self.width += delta_width
        self.height += delta_height

    def move(self, dx, dy):
```

```
        """ Move this object by the deltas """
        self.corner.x += dx
        self.corner.y += dy
```

Let us try this:

```
>>> r = Rectangle(Point(10,5), 100, 50)
>>> print(r)
((10, 5), 100, 50)
>>> r.grow(25, -10)
>>> print(r)
((10, 5), 125, 40)
>>> r.move(-10, 10)
print(r)
((0, 15), 125, 40)
```

16.3. Sameness

The meaning of the word "same" seems perfectly clear until we give it some thought, and then we realize there is more to it than we initially expected.

For example, if we say, "Alice and Bob have the same car", we mean that her car and his are the same make and model, but that they are two different cars. If we say, "Alice and Bob have the same mother", we mean
that her mother and his are the same person.

When we talk about objects, there is a similar ambiguity. For example, if two `Points` are the same, does that mean they contain the same data (coordinates) or that they are actually the same object?

We've already seen the `is` operator in the chapter on lists, where we talked about aliases: it allows us to find out if two references refer to the same object:

```
>>> p1 = Point(3, 4)
>>> p2 = Point(3, 4)
>>> p1 is p2
False
```

Even though `p1` and `p2` contain the same coordinates, they are not the same object. If we assign `p1` to `p3`, then the two variables are aliases of the same object:

```
>>> p3 = p1
>>> p1 is p3
True
```

This type of equality is called **shallow equality** because it compares only the references, not the contents of the objects.

To compare the contents of the objects — **deep equality** —we can write a function called `same_coordinates`:

```
def same_coordinates(p1, p2):
    return (p1.x == p2.x) and (p1.y == p2.y)
```

Now if we create two different objects that contain the same data, we can use `same_point` to find out if they represent points with the same coordinates.

```
>>> p1 = Point(3, 4)
>>> p2 = Point(3, 4)
>>> same_coordinates(p1, p2)
True
```

Of course, if the two variables refer to the same object, they have both shallow and deep equality.

Beware of ==

"When I use a word," Humpty Dumpty said, in a rather scornful tone, "it means just what I choose it to mean — neither more nor less." *Alice in Wonderland*

Python has a powerful feature that allows a designer of a class to decide what an operation like == or < should mean. (We've just shown how we can control how our own objects are converted to strings, so we've already made a start!) We'll cover more detail later. But sometimes the implementors will attach shallow equality semantics, and sometimes deep equality, as shown in this little experiment:

```
p = Point(4, 2)
s = Point(4, 2)
print("== on Points returns", p == s)
# By default, == on Point objects does a shallow equality test

a = [2,3]
b = [2,3]
print("== on lists returns",  a == b)
# But by default, == does a deep equality test on lists
```

This outputs:

```
== on Points returns False
== on lists returns True
```

So we conclude that even though the two lists (or tuples, etc.) are distinct objects with different memory addresses, for lists the == operator tests for deep equality, while in the case of points it makes a
shallow test.

16.4. Copying

Aliasing can make a program difficult to read because changes made in one place might have unexpected effects in another place. It is hard to keep track of all the variables that might refer to a given object.

Copying an object is often an alternative to aliasing. The `copy` module contains a function called `copy` that can duplicate any object:

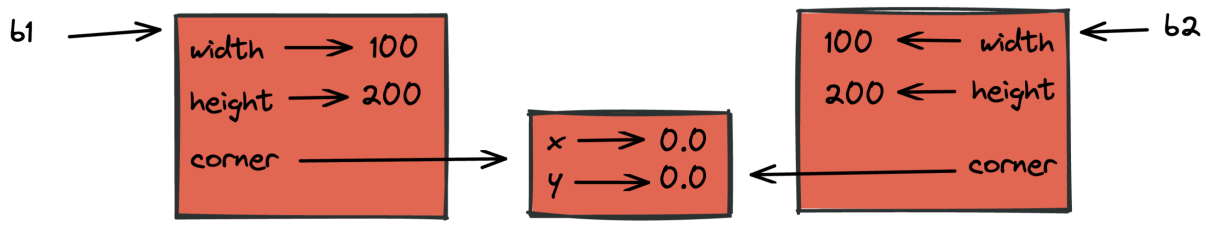
```
>>> import copy
>>> p1 = Point(3, 4)
>>> p2 = copy.copy(p1)
>>> p1 is p2
False
>>> same_coordinates(p1, p2)
True
```

Once we import the `copy` module, we can use the `copy` function to make a new `Point`. `p1` and `p2` are not the same point, but they contain the same data.

To copy a simple object like a `Point`, which doesn't contain any embedded objects, `copy` is sufficient. This is called **shallow copying**.

For something like a `Rectangle`, which contains a reference to a `Point`, `copy` doesn't do quite the right thing. It copies the reference to the `Point` object, so both the old `Rectangle` and the new one refer to a single `Point`.

If we create a box, `b1`, in the usual way and then make a copy, `b2`, using `copy`, the resulting state diagram looks like this:



This is almost certainly not what we want. In this case, invoking `grow` on one of the `Rectangle` objects would not affect the other, but invoking `move` on either would affect both! This behavior is confusing
and error-prone. The shallow copy has created an alias to the `Point` that represents the corner.

Fortunately, the `copy` module contains a function named `deepcopy` that copies not only the object but also any embedded objects. It won't be surprising to learn that this operation is called a **deep copy**.

```
>>> b2 = copy.deepcopy(b1)
```

Now `b1` and `b2` are completely separate objects.

16.5. Glossary

deep copy

To copy the contents of an object as well as any embedded objects, and any objects embedded in them, and so on; implemented by the `deepcopy` function in the `copy` module.

deep equality

Equality of values, or two references that point to objects that have the same value.

shallow copy

To copy the contents of an object, including any references to embedded objects; implemented by the `copy` function in the `copy` module.

shallow equality

Equality of references, or two references that point to the same object.

16.6. Exercises

1. Add a method `area` to the `Rectangle` class that returns the area of any instance:

```
r = Rectangle(Point(0, 0), 10, 5)
test(r.area() == 50)
```

2. Write a `perimeter` method in the `Rectangle` class so that we can find the perimeter of any rectangle instance:

```
r = Rectangle(Point(0, 0), 10, 5)
test(r.perimeter() == 30)
```

3. Write a `flip` method in the `Rectangle` class that swaps the width and the height of any rectangle instance:

```
r = Rectangle(Point(100, 50), 10, 5)
test(r.width == 10 and r.height == 5)
r.flip()
test(r.width == 5 and r.height == 10)
```

4. Write a new method in the `Rectangle` class to test if a `Point` falls within the rectangle. For this exercise, assume that a rectangle at (0,0) with width 10 and height 5 has *open* upper bounds on the width and height, i.e. it stretches in the x direction from [0 to 10), where 0 is included but 10 is excluded, and from [0 to 5) in the y direction. So it does not contain the point (10,2). These tests should pass:

```
r = Rectangle(Point(0, 0), 10, 5)
test(r.contains(Point(0, 0)))
test(r.contains(Point(3, 3)))
test(not r.contains(Point(3, 7)))
test(not r.contains(Point(3, 5)))
test(r.contains(Point(3, 4.99999)))
test(not r.contains(Point(-3, -3)))
```

5. In games, we often put a rectangular "bounding box" around our sprites. (A sprite is an object that can

move about in the game, as we will see shortly.) We can then do *collision detection* between, say, bombs and spaceships, by comparing whether their rectangles overlap anywhere.

Write a function to determine whether two rectangles collide. *Hint: this might be quite a tough exercise! Think carefully about all the cases before you code.*

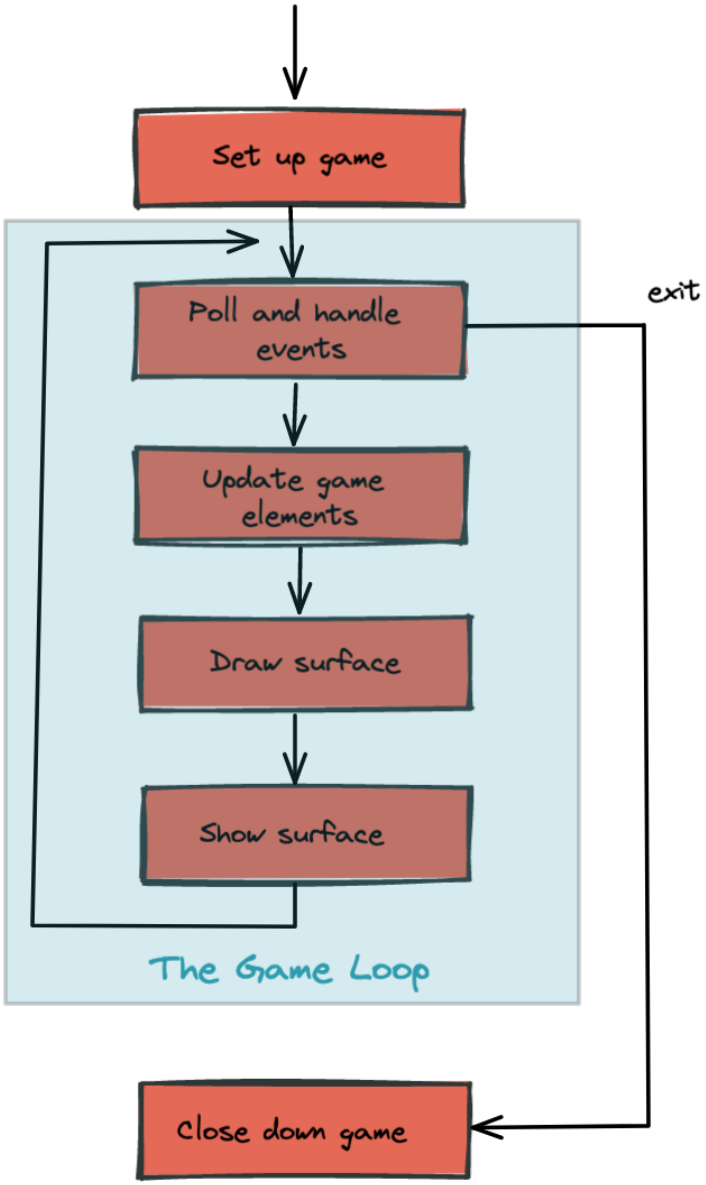
Chapter 17: PyGame

PyGame is a package that is not part of the standard Python distribution, so if you do not already have it installed (i.e. `import pygame` fails), download and install a suitable version from http://pygame.org/download.shtml. These notes are based on PyGame 1.9.1, the most recent version at the time of writing.

PyGame comes with a substantial set of tutorials, examples, and help, so there is ample opportunity to stretch yourself on the code. You may need to look around a bit to find these resources, though: if you've
installed PyGame on a Windows machine, for example, they'll end up in a folder like C:\Python31\Lib\site-packages\pygame\ where you will find directories for *docs* and *examples*.

17.1. The game loop

The structure of the games we'll consider always follows this fixed pattern:



In every game, in the *setup* section we'll create a window, load and prepare some content, and then enter the **game loop**. The game loop continuously does four main things:

- it **polls** for events — i.e. asks the system whether events have occurred — and responds appropriately,
- it updates whatever internal data structures or objects need changing,
- it draws the current state of the game into a (non-visible) surface,
- it puts the just-drawn surface on display.

```
import pygame

def main():
    """ Set up the game and run the main game loop """
    pygame.init()      # Prepare the pygame module for use
    surface_sz = 480   # Desired physical surface size, in pixels.

    # Create surface of (width, height), and its window.
    main_surface = pygame.display.set_mode((surface_sz, surface_sz))

    # Set up some data to describe a small rectangle and its color
    small_rect = (300, 200, 150, 90)
    some_color = (255, 0, 0)        # A color is a mix of (Red, Green, Blue)

    while True:
        ev = pygame.event.poll()    # Look for any event
        if ev.type == pygame.QUIT:  # Window close button clicked?
            break                   #   ... leave game loop

        # Update your game objects and data structures here...

        # We draw everything from scratch on each frame.
        # So first fill everything with the background color
        main_surface.fill((0, 200, 255))

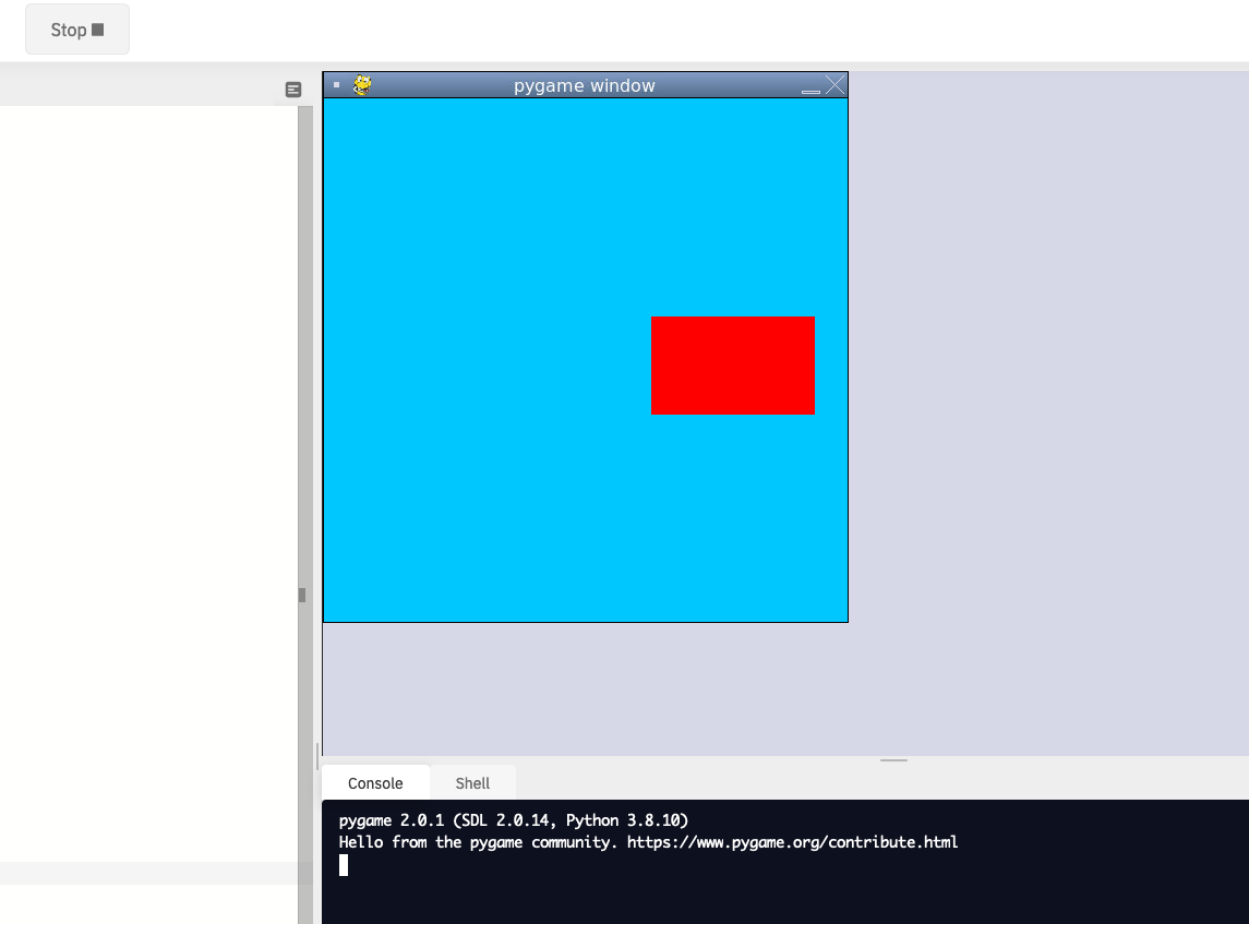
        # Overpaint a smaller rectangle on the main surface
        main_surface.fill(some_color, small_rect)

        # Now the surface is ready, tell pygame to display it!
        pygame.display.flip()

    pygame.quit()     # Once we leave the loop, close the window.

main()
```

This program pops up a window which stays there until we close it:



PyGame does all its drawing onto rectangular *surfaces*. After initializing PyGame at line 5, we create a window holding our main surface. The main loop of the game extends from line 15 to 30, with the following key bits of logic:

- First (line 16) we poll to fetch the next event that might be ready for us. This step will always be followed by some conditional statements that will determine whether any event that we're interested in has happened. Polling for the event consumes it, as far as PyGame is concerned, so we only get one chance to fetch and use each event. On line 17 we test whether the type of the event is the predefined constant called pygame.QUIT. This is the event that we'll see when the user clicks the close button on the PyGame window. In response to this event, we leave the loop.
- Once we've left the loop, the code at line 32 closes window, and we'll return from function `main`. Your program could go on to do other things, or reinitialize pygame and create another window, but it will usually just end too.
- There are different kinds of events — key presses, mouse motion,

mouse clicks, joystick movement, and so on. It is usual that we test and handle all these cases with new code squeezed in before line 19. The general idea is "handle events first, then worry about the other stuff".
- At line 20 we'd update objects or data — for example, if we wanted to vary the color, position, or size of the rectangle we're about to draw, we'd re-assign `some_color`, and `small_rect` here.
- A modern way to write games (now that we have fast computers and fast graphics cards) is to redraw everything from scratch on every iteration of the game loop. So the first thing we do at line 24 is fill the entire surface with a background color. The `fill` method of a surface takes two arguments — the color to use for filling, and the rectangle to be filled. But the second argument is optional, and if it is left out the entire surface is filled.
- In line 27 we fill a second rectangle, this time using `some_color`. The placement and size of the rectangle are given by the tuple `small_rect`, a 4-element tuple `(x, y, width, height)`.
- It is important to understand that the origin of the PyGame's surface is at the top left corner (unlike the turtle module that puts its origin in the middle of the screen). So, if you wanted the rectangle closer to the top of the window, you need to make its y coordinate smaller.
- If your graphics display hardware tries to read from memory at the same time as the program is writing to that memory, they will interfere with each other, causing video noise and flicker. To get around this, PyGame keeps two buffers in the main surface — the *back buffer* that the program draws to, while the *front buffer* is being shown to the user. Each time the program has fully prepared its back buffer, it flips the back/front role of the two buffers. So the drawing on lines 24 and 27 does does not change what is seen on the screen until we `flip` the buffers, on line 30.

17.2. Displaying images and text

To draw an image on the main surface, we load the image, say a beach ball, into its own new surface. The main surface has a `blit` method that copies pixels from the beach ball surface into its own surface. When we call `blit`, we can specify where the beach ball should be placed on the main surface. The term **blit** is widely used in computer graphics, and means *to make a fast copy of pixels from one area of memory to another.*

So in the setup section, before we enter the game loop, we'd load the image, like this:

```
ball = pygame.image.load("ball.png")
```

and after line 28 in the program above, we'd add this code to display our image at position (100,120):

```
main_surface.blit(ball, (50, 70))
```

To display text, we need do do three things. Before we enter the game loop, we instantiate a `font` object:

```
# Instantiate 16 point Courier font to draw text.
my_font = pygame.font.SysFont("Courier", 16)
```

and after line 28, again, we use the font's `render` method to create a new surface containing the pixels of the drawn text, and then, as in the case for images, we blit our new surface onto the main surface. Notice
that `render` takes two extra parameters — the second tells it whether to carefully smooth edges of the text while drawing (this process is called *anti-aliasing*), and the second is the color that we want the
text text be. Here we've used `(0,0,0)` which is black:

```
the_text = my_font.render("Hello, world!", True, (0,0,0))
main_surface.blit(the_text, (10, 10))
```

We'll demonstrate these two new features by counting the frames — the iterations of the game loop — and keeping some timing information. On each frame, we'll display the frame count, and the frame rate. We will
only update the frame rate after every 500 frames, when we'll look at the timing interval and can do the calculations.

First download an image of a beach ball. You can find one at https://learnpythontherightway.com/_-downloads/ball.png. Upload it to your repl by using the "Upload file" menu. Now you can run the following code.

```
import pygame
import time

def main():

    pygame.init()    # Prepare the PyGame module for use
    main_surface = pygame.display.set_mode((480, 240))

    # Load an image to draw. Substitute your own.
    # PyGame handles gif, jpg, png, etc. image types.
    ball = pygame.image.load("ball.png")

    # Create a font for rendering text
    my_font = pygame.font.SysFont("Courier", 16)

    frame_count = 0
    frame_rate = 0
    t0 = time.process_time()

    while True:

        # Look for an event from keyboard, mouse, joystick, etc.
        ev = pygame.event.poll()
        if ev.type == pygame.QUIT:   # Window close button clicked?
            break                    # Leave game loop

        # Do other bits of logic for the game here
        frame_count += 1
        if frame_count % 500 == 0:
            t1 = time.process_time()
            frame_rate = 500 / (t1-t0)
            t0 = t1

        # Completely redraw the surface, starting with background
        main_surface.fill((0, 200, 255))

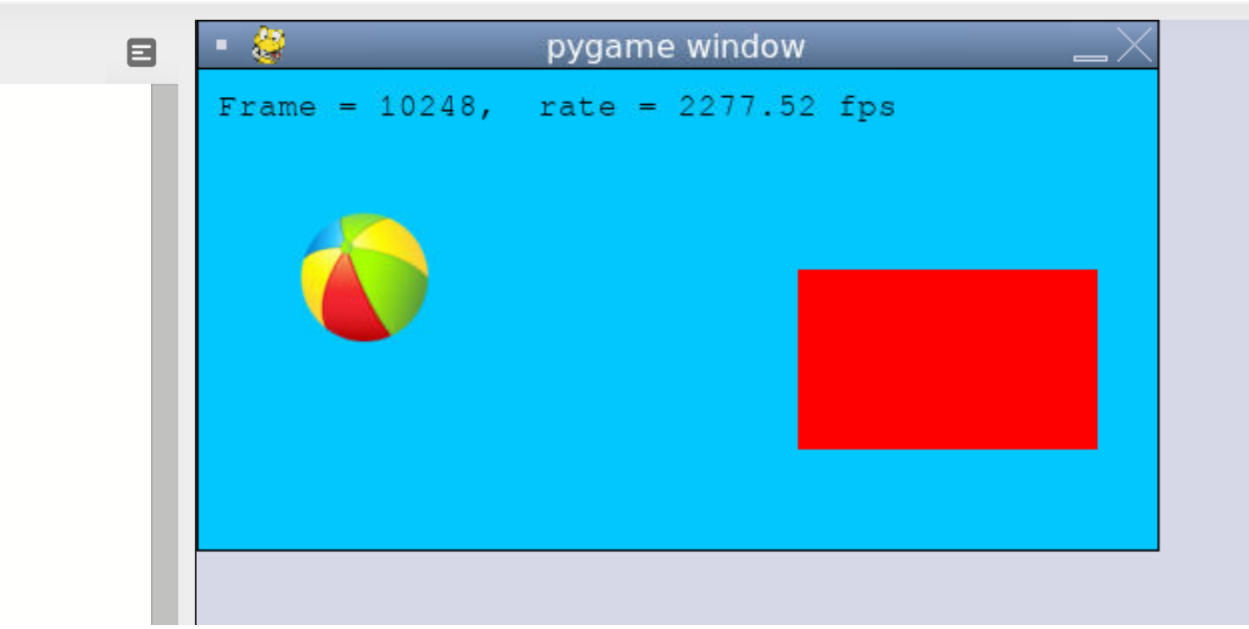
        # Put a red rectangle somewhere on the surface
        main_surface.fill((255,0,0), (300, 100, 150, 90))

        # Copy our image to the surface, at this (x,y) posn
        main_surface.blit(ball, (50, 70))

        # Make a new surface with an image of the text
```

```
44          the_text = my_font.render("Frame = {0},  rate = {1:.2f} fps"
45                    .format(frame_count, frame_rate), True, (0,0,0))
46          # Copy the text surface to the main surface
47          main_surface.blit(the_text, (10, 10))
48
49          # Now that everything is drawn, put it on display!
50          pygame.display.flip()
51
52      pygame.quit()
53
54
55  main()
```

The frame rate is close to ridiculous — a lot faster than one's eye can process frames. (Commercial video games usually plan their action for 60 frames per second (fps).) Of course, our rate will drop once we
start doing something a little more strenuous inside our game loop.



17.3. Drawing a board for the N queens puzzle

We previously solved our N queens puzzle. For the 8x8 board, one of the solutions was the list `[6,4,2,0,5,7,1,3]`. Let's use that solution as testdata, and now use PyGame to draw that chessboard with its queens.

We'll create a new module for the drawing code, called `draw_queens.py`. When we have our test case(s) working, we can go back to our solver, import this new module, and add a call to our new function to draw a
board each time a solution is discovered.

We begin with a background of black and red squares for the board. Perhaps we could create an image that we could load and draw, but that approach would need different background images for different size boards. Just drawing our own red and black rectangles of the appropriate size sounds like much more fun!

```
def draw_board(the_board):
    """ Draw a chess board with queens, from the_board. """

    pygame.init()
    colors = [(255,0,0), (0,0,0)]    # Set up colors [red, black]

    n = len(the_board)          # This is an NxN chess board.
    surface_sz = 480            # Proposed physical surface size.                  \

    sq_sz = surface_sz // n     # sq_sz is length of a square.
    surface_sz = n * sq_sz      # Adjust to exactly fit n squares.

    # Create the surface of (width, height), and its window.
    surface = pygame.display.set_mode((surface_sz, surface_sz))
```

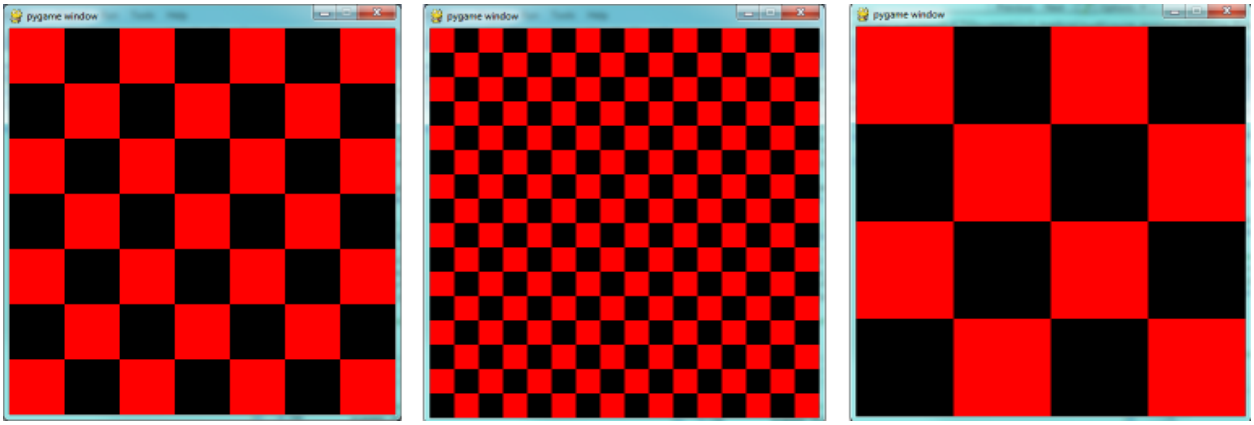
Here we precompute `sq_sz`, the integer size that each square will be, so that we can fit the squares nicely into the available window. So if we'd like the board to be 480x480, and we're drawing an 8x8 chessboard,
then each square will need to have a size of 60 units. But we notice that a 7x7 board cannot fit nicely into 480 — we're going to get some ugly border that our squares don't fill exactly. So we recompute the
surface size to exactly fit our squares before we create the window.

Now let's draw the squares, in the game loop. We'll need a nested loop: the outer loop will run over the rows of the chessboard, the inner loop over the columns:

```
# Draw a fresh background (a blank chess board)
for row in range(n):            # Draw each row of the board.
    c_indx = row % 2            # Change starting color on each row
    for col in range(n):        # Run through cols drawing squares
        the_square = (col*sq_sz, row*sq_sz, sq_sz, sq_sz)
        surface.fill(colors[c_indx], the_square)
        # now flip the color index for the next square
        c_indx = (c_indx + 1) % 2
```

There are two important ideas in this code: firstly, we compute the rectangle to be filled from the `row` and `col` loop variables, multiplying them by the size of the square to get their position. And, of course, each square is a fixed width and height. So `the_square` represents the rectangle to be filled on the current iteration of the loop. The second idea is that we have to alternate colors on every square. In the earlier setup code we created a list containing two colors, here we manipulate `c_indx` (which will always either have the value 0 or 1) to start each row on a color that is different from the previous row's starting color, and to switch colors each time a square is filled.

This (together with the other fragments not shown to flip the surface onto the display) leads to the pleasing backgrounds like this, for different size boards:

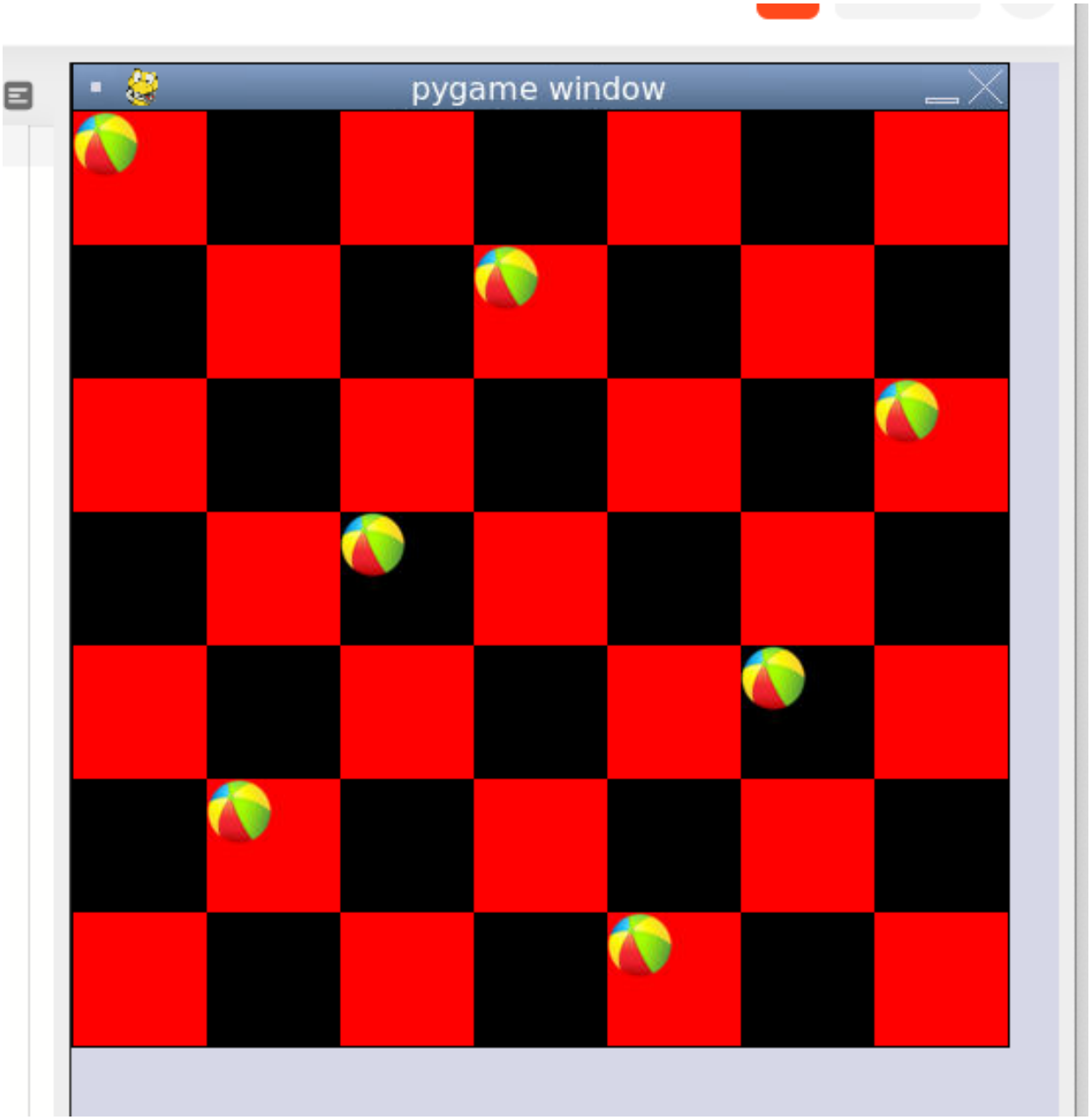


Now, on to drawing the queens! Recall that our solution `[6,4,2,0,5,7,1,3]` means that in column 0 of the board we want a queen at row 6, at column 1 we want a queen at row 4, and so on. So we need a loop running over each queen:

```
for (col, row) in enumerate(the_board):
    # draw a queen at col, row...
```

In this chapter we already have a beach ball image, so we'll use that for our queens. In the setup code before our game loop, we load the ball image (as we did before), and in the body of the loop, we add the line:

```
surface.blit(ball, (col * sq_sz, row * sq_sz))
```



We're getting there, but those queens need to be centred in their squares! Our problem arises from the fact that both the ball and the rectangle have their upper left corner as their reference points. If we're going to centre this ball in the square, we need to give it an extra offset in both the x and y direction. (Since the ball is round and the square is square, the offset in the two directions will be the same, so we'll just compute a single offset value, and use it in both directions.)

The offset we need is half the (size of the square less the size of the ball). So we'll precompute this

in the game's setup section, after we've loaded the ball and determined the square size:

```
ball_offset = (sq_sz - ball.get_width()) // 2
```

Now we touch up the drawing code for the ball and we're done:

```
surface.blit(ball, (col * sq_sz + ball_offset, row * q_sz + ball_offset))
```

We might just want to think about what would happen if the ball was bigger than the square. In that case, `ball_offset` would become negative. So it would still be centered in the square - it would just spill over the boundaries, or perhaps obscure the square entirely!

Here is the complete program:

```
import pygame

def draw_board(the_board):
    """ Draw a chess board with queens, as determined by the the_board. """

    pygame.init()
    colors = [(255,0,0), (0,0,0)]    # Set up colors [red, black]

    n = len(the_board)          # This is an NxN chess board.
    surface_sz = 480            # Proposed physical surface size.                   \

    sq_sz = surface_sz // n     # sq_sz is length of a square.
    surface_sz = n * sq_sz      # Adjust to exactly fit n squares.

    # Create the surface of (width, height), and its window.
    surface = pygame.display.set_mode((surface_sz, surface_sz))

    ball = pygame.image.load("ball.png")

    # Use an extra offset to centre the ball in its square.
    # If the square is too small, offset becomes negative,
    #   but it will still be centered :-)
    ball_offset = (sq_sz-ball.get_width()) // 2

    while True:

        # Look for an event from keyboard, mouse, etc.
        ev = pygame.event.poll()
        if ev.type == pygame.QUIT:
```

```
30              break;
31
32          # Draw a fresh background (a blank chess board)
33          for row in range(n):           # Draw each row of the board.
34              c_indx = row % 2           # Alternate starting color
35              for col in range(n):       # Run through cols drawing squares
36                  the_square = (col*sq_sz, row*sq_sz, sq_sz, sq_sz)
37                  surface.fill(colors[c_indx], the_square)
38                  # Now flip the color index for the next square
39                  c_indx = (c_indx + 1) % 2
40
41          # Now that squares are drawn, draw the queens.
42          for (col, row) in enumerate(the_board):
43            surface.blit(ball,
44                     (col*sq_sz+ball_offset,row*sq_sz+ball_offset))
45
46          pygame.display.flip()
47
48
49      pygame.quit()
50
51  if __name__ == "__main__":
52      draw_board([0, 5, 3, 1, 6, 4, 2])    # 7 x 7 to test window size
53      draw_board([6, 4, 2, 0, 5, 7, 1, 3])
54      draw_board([9, 6, 0, 3, 10, 7, 2, 4, 12, 8, 11, 5, 1])  # 13 x 13
55      draw_board([11, 4, 8, 12, 2, 7, 3, 15, 0, 14, 10, 6, 13, 1, 5, 9])
```

There is one more thing worth reviewing here. The conditional statement on line 50 tests whether the name of the currently executing program is `__main__`. This allows us to distinguish whether this module is being
run as a main program, or whether it has been imported elsewhere, and used as a module. If we run this module in Python, the test cases in lines 51-54 will be executed. However, if we import this module into another program (i.e. our N queens solver from earlier) the condition at line 50 will be false, and the statements on lines 51-54 won't run.

In Chapter 14, (14.9. Eight Queens puzzle, part 2) our main program looked like this:

```
def main():

    bd = list(range(8))     # Generate the initial permutation
    num_found = 0
    tries = 0
    while num_found < 10:
       random.shuffle(bd)
       tries += 1
       if not has_clashes(bd):
           print("Found solution {0} in {1} tries.".format(bd, tries))
           tries = 0
           num_found += 1

main()
```

Now we just need two changes. At the top of that program, we import the module that we've been working on here (assume we called it `draw_queens`). (You'll have to ensure that the two modules are saved in the same folder.) Then after line 10 here we add a call to draw the solution that we've just discovered:

```
draw_queens.draw_board(bd)
```

And that gives a very satisfying combination of program that can search for solutions to the N queens problem, and when it finds each, it pops up the board showing the solution.

17.4. Sprites

A sprite is an object that can move about in a game, and has internal behaviour and state of its own. For example, a spaceship would be a sprite, the player would be a sprite, and bullets and bombs would all be
sprites.

Object oriented programming (OOP) is ideally suited to a situation like this: each object can have its own attributes and internal state, and a couple of methods. Let's have some fun with our N queens board. Instead
of placing the queen in her final position, we'd like to drop her in from the top of the board, and let her fall into position, perhaps bouncing along the way.

The first encapsulation we need is to turn each of our queens into an object. We'll keep a list of all the active sprites (i.e. a list of queen objects), and arrange two new things in our game loop:

- After handling events, but before drawing, call an `update` method

on every sprite. This will give each sprite a chance to modify its internal state in some way — perhaps change its image, or change its position, or rotate itself, or make itself grow a bit bigger or a bit smaller.
- Once all the sprites have updated themselves, the game loop can begin drawing - first the background, and then call a `draw` method on each sprite in turn, and delegate (hand off) the task of drawing to the object itself. This is in line with the OOP idea that we don't say "Hey, *draw*, show this queen!", but we prefer to say "Hey, *queen*, draw yourself!".

We start with a simple object, no movement or animation yet, just scaffolding, to see how to fit all the pieces together:

```
class QueenSprite:

    def __init__(self, img, target_posn):
        """ Create and initialize a queen for this
            target position on the board
        """
        self.image = img
        self.target_posn = target_posn
        self.posn = target_posn

    def update(self):
        return                # Do nothing for the moment.

    def draw(self, target_surface):
        target_surface.blit(self.image, self.posn)
```

We've given the sprite three attributes: an image to be drawn, a target position, and a current position. If we're going to move the spite about, the current position may need to be different from the target, which is where we want the queen finally to end up. In this code at this time we've done nothing in the `update` method, and our `draw` method (which can probably remain this simple in future) simply draws itself at
its current position on the surface that is provided by the caller.

With its class definition in place, we now instantiate our N queens, put them into a list of sprites, and arrange for the game loop to call the `update` and `draw` methods on each frame. The new bits of code, and the
revised game loop look like this:

```
all_sprites = []      # Keep a list of all sprites in the game

# Create a sprite object for each queen, and populate our list.
for (col, row) in enumerate(the_board):
    a_queen = QueenSprite(ball,
               (col*sq_sz+ball_offset, row*sq_sz+ball_offset))
    all_sprites.append(a_queen)

while True:
    # Look for an event from keyboard, mouse, etc.
    ev = pygame.event.poll()
    if ev.type == pygame.QUIT:
        break;

    # Ask every sprite to update itself.
    for sprite in all_sprites:
        sprite.update()

    # Draw a fresh background (a blank chess board)
    # ... same as before ...

    # Ask every sprite to draw itself.
    for sprite in all_sprites:
        sprite.draw(surface)

    pygame.display.flip()
```

This works just like it did before, but our extra work in making objects for the queens has prepared the way for some more ambitious extensions.

Let us begin with a falling queen object. At any instant, it will have a velocity i.e. a speed, in a certain direction. (We are only working with movement in the y direction, but use your imagination!) So in the
object's `update` method, we want to change its current position by its velocity. If our N queens board is floating in space, velocity would stay constant, but hey, here on Earth we have gravity! Gravity changes
the velocity on each time interval, so we'll want a ball that speeds up as it falls further. Gravity will be constant for all queens, so we won't keep it in the instances — we'll just make it a variable in our module. We'll make one other change too: we will start every queen at the top of the board, so that it can fall towards its target position. With these changes, we now get the following:

```
gravity = 0.0001

class QueenSprite:

    def __init__(self, img, target_posn):
        self.image = img
        self.target_posn = target_posn
        (x, y) = target_posn
        self.posn = (x, 0)     # Start ball at top of its column
        self.y_velocity = 0    #    with zero initial velocity

    def update(self):
        self.y_velocity += gravity       # Gravity changes velocity
        (x, y) = self.posn
        new_y_pos = y + self.y_velocity  # Velocity moves the ball
        self.posn = (x, new_y_pos)       #   to this new position.

    def draw(self, target_surface):      # Same as before.
        target_surface.blit(self.image, self.posn)
```

Making these changes gives us a new chessboard in which each queen starts at the top of its column, and speeds up, until it drops off the bottom of the board and disappears forever. A good start — we have
movement!

The next step is to get the ball to bounce when it reaches its own target position. It is pretty easy to bounce something — you just change the sign of its velocity, and it will move at the same speed in the opposite direction. Of course, if it is travelling up towards the top of the board it will be slowed down by gravity. (Gravity always sucks down!) And you'll find it bounces all the way up to where it began
from, reaches zero velocity, and starts falling all over again. So we'll have bouncing balls that never settle.

A realistic way to settle the object is to lose some energy (probably to friction) each time it bounces — so instead of simply reversing the sign of the velocity, we multiply it by some fractional factor — say
-0.65. This means the ball only retains 65% of its energy on each bounce, so it will, as in real life, stop bouncing after a short while, and settle on its "ground".

The only changes are in the `update` method, which now looks like this:

```
def update(self):
    self.y_velocity += gravity
    (x, y) = self.posn
    new_y_pos = y + self.y_velocity
    (target_x, target_y) = self.target_posn   # Unpack the position
    dist_to_go = target_y - new_y_pos         # How far to our floor?

    if dist_to_go < 0:                        # Are we under floor?
        self.y_velocity = -0.65 * self.y_velocity     # Bounce
        new_y_pos = target_y + dist_to_go     # Move back above floor

    self.posn = (x, new_y_pos)                # Set our new position.
```

Heh, heh, heh! We're not going to show animated screenshots, so copy the code into your Python environment and see for yourself.

17.5. Events

The only kind of event we're handled so far has been the QUIT event. But we can also detect keydown and keyup events, mouse motion, and mousebutton down or up events. Consult the PyGame documentation and follow the link to Event.

When your program polls for and receives an event object from PyGame, its event type will determine what secondary information is available. Each event object carries a *dictionary* (which you may only cover in
due course in these notes). The dictionary holds certain *keys* and *values* that make sense for the type of event.

For example, if the type of event is MOUSEMOTION, we'll be able to find the mouse position and information about the state of the mouse buttons in the dictionary attached to the event. Similarly, if the event is
KEYDOWN, we can learn from the dictionary which key went down, and whether any modifier keys (shift, control, alt, etc.) are also down. You also get events when the game window becomes active (i.e. gets focus) or
loses focus.

The event object with type NOEVENT is returned if there are no events waiting. Events can be printed, allowing you to experiment and play around. So dropping these lines of code into the game loop directly
after polling for any event is quite informative:

```
if ev.type != pygame.NOEVENT:   # Only print if it is interesting!
    print(ev)
```

With this is place, hit the space bar and the escape key, and watch the events you get. Click your three mouse buttons. Move your mouse over the window. (This causes a vast cascade of events, so you may also need to
filter those out of the printing.) You'll get output that looks something like this:

```
<Event(17-VideoExpose {})>
<Event(1-ActiveEvent {'state': 1, 'gain': 0})>
<Event(2-KeyDown {'scancode': 57, 'key': 32, 'unicode': ' ', 'mod': 0})>
<Event(3-KeyUp {'scancode': 57, 'key': 32, 'mod': 0})>
<Event(2-KeyDown {'scancode': 1, 'key': 27, 'unicode': '\x1b', 'mod': 0})>
<Event(3-KeyUp {'scancode': 1, 'key': 27, 'mod': 0})>
...
<Event(4-MouseMotion {'buttons': (0, 0, 0), 'pos': (323, 194), 'rel': (-3, -1)})>
<Event(4-MouseMotion {'buttons': (0, 0, 0), 'pos': (322, 193), 'rel': (-1, -1)})>
<Event(4-MouseMotion {'buttons': (0, 0, 0), 'pos': (321, 192), 'rel': (-1, -1)})>
<Event(4-MouseMotion {'buttons': (0, 0, 0), 'pos': (319, 192), 'rel': (-2, 0)})>
<Event(5-MouseButtonDown {'button': 1, 'pos': (319, 192)})>
<Event(6-MouseButtonUp {'button': 1, 'pos': (319, 192)})>
<Event(4-MouseMotion {'buttons': (0, 0, 0), 'pos': (319, 191), 'rel': (0, -1)})>
<Event(5-MouseButtonDown {'button': 2, 'pos': (319, 191)})>
<Event(5-MouseButtonDown {'button': 5, 'pos': (319, 191)})>
<Event(6-MouseButtonUp {'button': 5, 'pos': (319, 191)})>
<Event(6-MouseButtonUp {'button': 2, 'pos': (319, 191)})>
<Event(5-MouseButtonDown {'button': 3, 'pos': (319, 191)})>
<Event(6-MouseButtonUp {'button': 3, 'pos': (319, 191)})>
 ...
<Event(1-ActiveEvent {'state': 1, 'gain': 0})>
<Event(12-Quit {})>
```

So let us now make these changes to the code near the top of our game loop:

```
1  while True:
2
3      # Look for an event from keyboard, mouse, etc.
4      ev = pygame.event.poll()
5      if ev.type == pygame.QUIT:
6          break;
7      if ev.type == pygame.KEYDOWN:
8          key = ev.dict["key"]
9          if key == 27:                   # On Escape key ...
10             break                       #   leave the game loop.
11         if key == ord("r"):
12             colors[0] = (255, 0, 0)     # Change to red + black.
13         elif key == ord("g"):
14             colors[0] = (0, 255, 0)     # Change to green + black.
15         elif key == ord("b"):
16             colors[0] = (0, 0, 255)     # Change to blue + black.
17
18     if ev.type == pygame.MOUSEBUTTONDOWN: # Mouse gone down?
19         posn_of_click = ev.dict["pos"]    # Get the coordinates.
20         print(posn_of_click)              # Just print them.
```

Lines 7-16 show typical processing for a KEYDOWN event — if a key has gone down, we test which key it is, and take some action. With this in place, we have another way to quit our queens program —by hitting the
escape key. Also, we can use keys to change the color of the board that is drawn.

Finally, at line 20, we respond (pretty lamely) to the mouse button going down.

As a final exercise in this section, we'll write a better response handler to mouse clicks. What we will do is figure out if the user has clicked the mouse on one of our sprites. If there is a sprite under the
mouse when the click occurs, we'll send the click to the sprite and let it respond in some sensible way.

We'll begin with some code that finds out which sprite is under the clicked position, perhaps none! We add a method to the class, `contains_point`, which returns True if the point is within the rectangle of the sprite:

```
def contains_point(self, pt):
    """ Return True if my sprite rectangle contains point pt """
    (my_x, my_y) = self.posn
    my_width = self.image.get_width()
    my_height = self.image.get_height()
    (x, y) = pt
    return ( x >= my_x and x < my_x + my_width and
             y >= my_y and y < my_y + my_height)
```

Now in the game loop, once we've seen the mouse event, we determine which queen, if any, should be told to respond to the event:

```
if ev.type == pygame.MOUSEBUTTONDOWN:
    posn_of_click = ev.dict["pos"]
    for sprite in all_sprites:
        if sprite.contains_point(posn_of_click):
            sprite.handle_click()
            break
```

And the final thing is to write a new method called `handle_click` in the `QueenSprite` class. When a sprite is clicked, we'll just add some velocity in the up direction, i.e. kick it back into the air.

```
def handle_click(self):
    self.y_velocity += -0.3   # Kick it up
```

With these changes we have a playable game! See if you can keep all the balls on the move, not allowing any one to settle!

17.6. A wave of animation

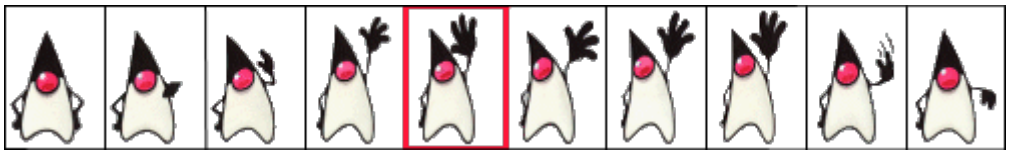
Many games have sprites that are animated: they crouch, jump and shoot. How do they do that?

Consider this sequence of 10 images: if we display them in quick succession, Duke will wave at us. (Duke is a friendly visitor from the kingdom of Javaland.)

A compound image containing smaller *patches* which are intended for animation is called a **sprite sheet**. Download this sprite sheet by right-clicking in your browser and saving it in your working

directory
with the name `duke_spritesheet.png`.

The sprite sheet has been quite carefully prepared: each of the 10 patches are spaced exactly 50 pixels apart. So, assuming we want to draw patch number 4 (numbering from 0), we want to draw only the rectangle
that starts at x position 200, and is 50 pixels wide, within the sprite sheet. Here we've shown the patches and highlighted the patch we want to draw.



The `blit` method we've been using — for copying pixels from one surface to another —can copy a sub-rectangle of the source surface. So the grand idea here is that each time we draw Duke, we won't blit the
whole sprite sheet. Instead we'll provide an extra rectangle argument that determines which portion of the sprite sheet will be blitted.

We're going to add new code in this section to our existing N queens drawing game. What we want is to put some instances of Duke on the chessboard somewhere. If the user clicks on one of them, we'll get him
to respond by waving back, for one cycle of his animation.

But before we do that, we need another change. Up until now, our game loop has been running at really fast frame rates that are unpredictable. So we've chosen some *magic numbers* for gravity and for bouncing and
kicking the ball on the basis of trial-and-error. If we're going to start animating more sprites, we need to tame our game loop to operate at a fixed, known frame rate. This will allow us to plan our animation better.

PyGame gives us the tools to do this in just two lines of code. In the setup section of the game, we instantiate a new `Clock` object:

```
1  my_clock = pygame.time.Clock()
```

and right at the bottom of the game loop, we call a method on this object that limits the frame rate to whatever we specify. So let's plan our game and animation for 60 frames per second, by adding this line at
the bottom of our game loop:

```
1  my_clock.tick(60)  # Waste time so that frame rate becomes 60 fps
```

You'll find that you have to go back and adjust the numbers for gravity and kicking the ball now, to match this much slower frame rate. When we plan an animation so that it only works sensibly at a

fixed frame rate,
we say that we've *baked* the animation. In this case we're baking our animations for 60 frames per second.

To fit into the existing framework that we already have for our queens board, we want to create a `DukeSprite` class that has all the same methods as the `QueenSprite` class. Then we can add one or more Duke
instances onto our list of `all_sprites`, and our existing game loop will then call methods of the Duke instance. Let us start with skeleton scaffolding for the new class:

```
class DukeSprite:

    def __init__(self, img, target_posn):
        self.image = img
        self.posn = target_posn

    def update(self):
        return

    def draw(self, target_surface):
        return

    def handle_click(self):
        return

    def contains_point(self, pt):
        # Use code from QueenSprite here
        return
```

The only changes we'll need to the existing game are all in the setup section. We load up the new sprite sheet and instantiate a couple of instances of Duke, at the positions we want on the chessboard. So before
entering the game loop, we add this code:

```
# Load the sprite sheet
duke_sprite_sheet = pygame.image.load("duke_spritesheet.png")

# Instantiate two duke instances, put them on the chessboard
duke1 = DukeSprite(duke_sprite_sheet,(sq_sz*2, 0))
duke2 = DukeSprite(duke_sprite_sheet,(sq_sz*5, sq_sz))

# Add them to the list of sprites which our game loop manages
all_sprites.append(duke1)
all_sprites.append(duke2)
```

Now the game loop will test if each instance has been clicked, will call the click handler for that instance. It will also call update and draw for all sprites. All the remaining changes we need to make will be made
in the methods of the `DukeSprite` class.

Let's begin with drawing one of the patches. We'll introduce a new attribute `curr_patch_num` into the class. It holds a value between 0 and 9, and determines which patch to draw. So the job of the `draw` method is to compute the sub-rectangle of the patch to be drawn, and to blit only that portion of the spritesheet:

```
def draw(self, target_surface):
    patch_rect = (self.curr_patch_num * 50, 0,
                  50, self.image.get_height())
    target_surface.blit(self.image, self.posn, patch_rect)
```

Now on to getting the animation to work. We need to arrange logic in `update` so that if we're busy animating, we change the `curr_patch_num` every so often, and we also decide when to bring Duke back to his rest position, and stop the animation. An important issue is that the game loop frame rate —in our case 60 fps — is not the same as the *animation rate* — the rate at which we want to change Duke's animation patches. So we'll plan Duke wave's animation cycle for a duration of 1 second. In other words, we want to play out Duke's 10 animation patches over 60 calls to `update`. (This is how the baking of the animation takes place!) So we'll keep another animation frame counter in the class, which will be zero when we're not animating, and each call to `update` will increment the counter up to 59, and then back to 0. We can then divide that animation counter by 6, to set the `curr_patch_num` variable to select the patch we want to show.

```
def update(self):
    if self.anim_frame_count > 0:
       self.anim_frame_count = (self.anim_frame_count + 1 ) % 60
       self.curr_patch_num = self.anim_frame_count // 6
```

Notice that if `anim_frame_count` is zero, i.e. Duke is at rest, nothing happens here. But if we start the counter running, it will count up to 59 before settling back to zero. Notice also, that because `anim_frame_count` can only be a value between 0 and 59, the `curr_patch_num` will always stay between 0 and 9. Just what we require!

Now how do we trigger the animation, and start it running? On the mouse click.

```
def handle_click(self):
     if self.anim_frame_count == 0:
        self.anim_frame_count = 5
```

Two things of interest here. We only start the animation if Duke is at rest. Clicks on Duke while he is already waving get ignored. And when we do start the animation, we set the counter to 5 — this

means that on
the very next call to `update` the counter becomes 6, and the image changes. If we had set the counter to 1, we would have needed to wait for 5 more calls to `update` before anything happened — a slight lag, but enough to make things feel sluggish.

The final touch-up is to initialize our two new attributes when we instantiate the class. Here is the code for the whole class now:

```
class DukeSprite:

    def __init__(self, img, target_posn):
        self.image = img
        self.posn = target_posn
        self.anim_frame_count = 0
        self.curr_patch_num = 0

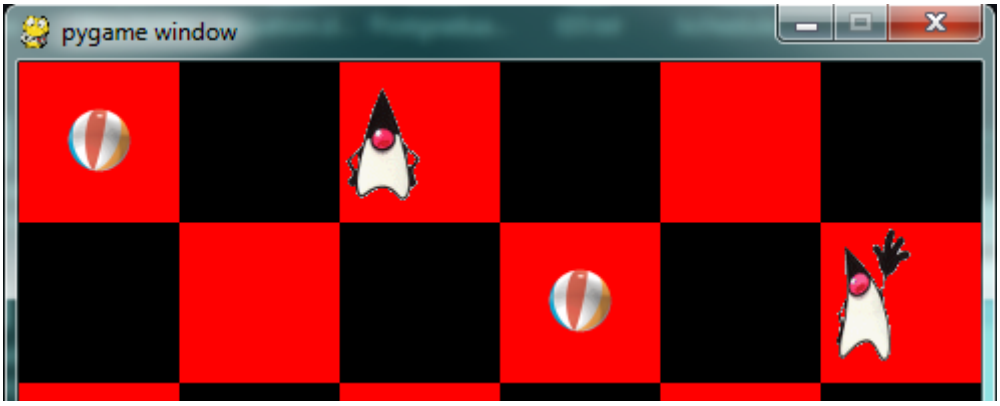
    def update(self):
        if self.anim_frame_count > 0:
           self.anim_frame_count = (self.anim_frame_count + 1 ) % 60
           self.curr_patch_num = self.anim_frame_count // 6

    def draw(self, target_surface):
        patch_rect = (self.curr_patch_num * 50, 0,
                      50, self.image.get_height())
        target_surface.blit(self.image, self.posn, patch_rect)

    def contains_point(self, pt):
         """ Return True if my sprite rectangle contains  pt """
         (my_x, my_y) = self.posn
         my_width = self.image.get_width()
         my_height = self.image.get_height()
         (x, y) = pt
         return ( x >= my_x and x < my_x + my_width and
                  y >= my_y and y < my_y + my_height)

    def handle_click(self):
         if self.anim_frame_count == 0:
            self.anim_frame_count = 5
```

Now we have two extra Duke instances on our chessboard, and clicking on either causes that instance to wave.



17.7. Aliens - a case study

Find the example games with the PyGame package, (On a windows system, something like C:\Python3\Lib\site-packages\pygame\examples) and play the Aliens game. Then read the code, in an editor or Python environment that shows line numbers.

It does a number of much more advanced things than we do, and relies on the PyGame framework for more of its logic. Here are some of the points to notice:

- The frame rate is deliberately constrained near the bottom of the game loop at line 311. If we change that number we can make the game very slow or unplayably fast!
- There are different kinds of sprites: Explosions, Shots, Bombs, Aliens and a Player. Some of these have more than one image — by swapping the images, we get animation of the sprites, i.e. the Alien spacecraft lights change, and this is done at line 112.
- Different kinds of objects are referenced in different groups of sprites, and PyGame helps maintain these. This lets the program check for collisions between, say, the list of shots fired by the player, and the list of spaceships that are attacking. PyGame does a lot of the hard work for us.
- Unlike our game, objects in the Aliens game have a limited lifetime, and have to get killed. For example, if we shoot, a Shot object is created — if it reaches the top of the screen without expoding against anything, it has to be removed from the game. Lines 141-142 do this. Similarly, when a falling bomb gets close to the ground (line 156), it instantiates a new Explosion sprite, and the bomb kills itself.
- There are random timings that add to the fun — when to spawn the

next Alien, when an Alien drops the next bomb, etc.
- The game plays sounds too: a less-than-relaxing loop sound, plus sounds for the shots and explosions.

17.8. Reflections

Object oriented programming is a good organizational tool for software. In the examples in this chapter, we've started to use (and hopefully appreciate) these benefits. Here we had N queens each with its own state, falling to its own floor level, bouncing, getting kicked, etc. We might have managed without the organizational power of objects — perhaps we could have kept lists of velocities for each queen, and lists of target positions, and so on — our code would likely have been much more complicated, ugly, and a lot poorer!

17.9. Glossary

animation rate
The rate at which we play back successive patches to create the illusion of movement. In the sample we considered in this chapter, we played Duke's 10 patches over the duration of one second. Not the same as the frame rate.

baked animation
An animation that is designed to look good at a predetermined fixed frame rate. This reduces the amount of computation that needs to be done when the game is running. High-end commercial games usually bake their animations.

blit
A verb used in computer graphics, meaning to make a fast copy of an image or pixels from a sub-rectangle of one image or surface to another surface or image.

frame rate
The rate at which the game loop executes and updates the display.

game loop
A loop that drives the logic of a game. It will usually poll for events, then update each of the objects in the game, then get everything drawn, and then put the newly drawn frame on display.

pixel
A single picture element, or dot, from which images are made.

poll
To ask whether something like a keypress or mouse movement has happened. Game loops usually poll to discover what events have occurred. This is different from event-driven programs like the ones seen in the chapter titled “Events”. In those cases, the button click or keypress event triggers the call of a handler function in your program, but this happens behind your back.

sprite
An active agent or element in a game, with its own state, position and behaviour.

surface
This is PyGame’s term for what the Turtle module calls a *canvas*. A surface is a rectangle of pixels used for displaying shapes and images.

17.10. Exercises

1. Have fun with Python, and with PyGame.
2. We deliberately left a bug in the code for animating Duke. If you click on one of the chessboard squares

to the right of Duke, he waves anyway. Why? Find a one-line fix for the bug.
3. Use your preferred search engine to search their image library for “sprite sheet playing cards”. Create a

list [0..51] to represent an encoding of the 52 cards in a deck. Shuffle the cards, slice off the top five as your hand in a poker deal. Display the hand you have been dealt.
4. So the Aliens game is in outer space, without gravity. Shots fly away forever, and bombs don’t speed up

when they fall. Add some gravity to the game. Decide if you’re going to allow your own shots to fall back on your head and kill you.
5. Those pesky Aliens seem to pass right through each other! Change the game so that they collide, and

destroy each other in a mighty explosion.

Chapter 18: Recursion

Recursion means "defining something in terms of itself" usually at some smaller scale, perhaps multiple times, to achieve your objective. For example, we might say "A human being is someone whose mother is a human being", or "a directory is a structure that holds files and (smaller) directories", or "a family tree starts with a couple who have children, each with their own family sub-trees".

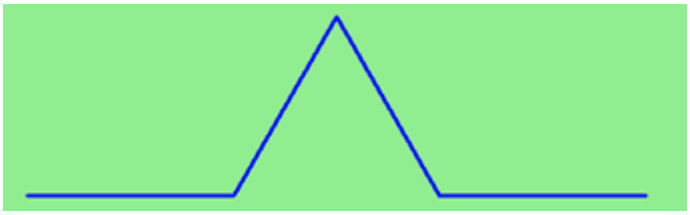
Programming languages generally support **recursion**, which means that, in order to solve a problem,functions can *call themselves* to solve smaller subproblems.

18.1. Drawing Fractals

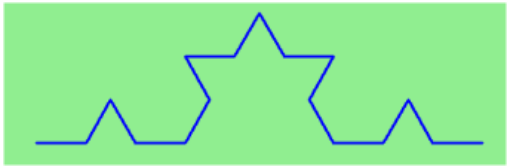
For our purposes, a **fractal** is a drawing which also has *self-similar* structure, where it can be defined in terms of itself.

Let us start by looking at the famous Koch fractal. An order 0 Koch fractal is simply a straight line of a given size.

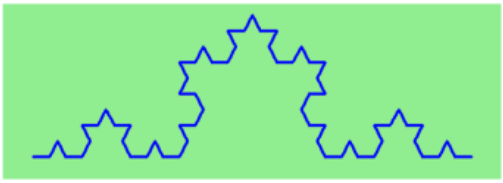
An order 1 Koch fractal is obtained like this: instead of drawing just one line, draw instead four smaller segments, in the pattern shown here:



Now what would happen if we repeated this Koch pattern again on each of the order 1 segments? We'd get this order 2 Koch fractal:



Repeating our pattern again gets us an order 3 Koch fractal:



Now let us think about it the other way around. To draw a Koch fractal of order 3, we can simply draw four order 2 Koch fractals. But each of these in turn needs four order 1 Koch fractals, and each of those in turn needs four order 0 fractals. Ultimately, the only drawing that will take place is at order 0. This is very simple to code up in Python:

```
1  def koch(t, order, size):
2      """
3         Make turtle t draw a Koch fractal of 'order' and 'size'.
4         Leave the turtle facing the same direction.
5      """
6
7      if order == 0:          # The base case is just a straight line
8          t.forward(size)
9      else:
10         koch(t, order-1, size/3)   # Go 1/3 of the way
11         t.left(60)
12         koch(t, order-1, size/3)
13         t.right(120)
14         koch(t, order-1, size/3)
15         t.left(60)
16         koch(t, order-1, size/3)
```

The key thing that is new here is that if order is not zero, `koch` calls itself recursively to get its job done.

Let's make a simple observation and tighten up this code. Remember that turning right by 120 is the same as turning left by -120. So with a bit of clever rearrangement, we can use a loop instead of lines 10-16:

```
def koch(t, order, size):
    if order == 0:
        t.forward(size)
    else:
        for angle in [60, -120, 60, 0]:
           koch(t, order-1, size/3)
           t.left(angle)
```

The final turn is 0 degrees — so it has no effect. But it has allowed us to find a pattern and reduce seven lines of code to three, which will make things easier for our next observations.

Recursion, the high-level view

One way to think about this is to convince yourself that the function works correctly when you call it for an order 0 fractal. Then do a mental *leap of faith*, saying *"the fairy godmother* (or Python, if you can think of Python as your fairy godmother) *knows how to handle the recursive level 0 calls for me on lines 11, 13, 15, and 17, so I don't need to think about that detail!"* All I need to focus on is how to draw an order 1 fractal *if I can assume the order 0 one is already working.*

You're practicing *mental abstraction* — ignoring the subproblem while you solve the big problem.

If this mode of thinking works (and you should practice it!), then take it to the next level. Aha! now can I see that it will work when called for order 2 *under the assumption that it is already working for level 1.*

And, in general, if I can assume the order n-1 case works, can I just solve the level n problem?

Students of mathematics who have played with proofs of induction should see some very strong similarities here.

Recursion, the low-level operational view

Another way of trying to understand recursion is to get rid of it! If we had separate functions to draw a level 3 fractal, a level 2 fractal, a level 1 fractal and a level 0 fractal, we could simplify the above code,
quite mechanically, to a situation where there was no longer any recursion, like this:

```
def koch_0(t, size):
    t.forward(size)

def koch_1(t, size):
    for angle in [60, -120, 60, 0]:
       koch_0(t, size/3)
       t.left(angle)

def koch_2(t, size):
    for angle in [60, -120, 60, 0]:
```

```
        koch_1(t, size/3)
        t.left(angle)

def koch_3(t, size):
    for angle in [60, -120, 60, 0]:
        koch_2(t, size/3)
        t.left(angle)
```

This trick of "unrolling" the recursion gives us an operational view of what happens. You can trace the program into `koch_3`, and from there, into `koch_2`, and then into `koch_1`, etc., all the way down the different layers of the recursion.

This might be a useful hint to build your understanding. The mental goal is, however, to be able to do the abstraction!

18.2. Recursive data structures

All of the Python data types we have seen can be grouped inside lists and tuples in a variety of ways. Lists and tuples can also be nested, providing many possibilities for organizing data. The organization of data for the purpose of making it easier to use is called a **data structure**.

It's election time and we are helping to compute the votes as they come in. Votes arriving from individual wards, precincts, municipalities, counties, and states are sometimes reported as a sum total of votes and sometimes as a list of subtotals of votes. After considering how best to store the tallies, we decide to use a *nested number list*, which we define as follows:

A *nested number list* is a list whose elements are either:

1. numbers
2. nested number lists

Notice that the term, *nested number list* is used in its own definition. **Recursive definitions** like this are quite common in mathematics and computer science. They provide a concise and powerful way to describe **recursive data structures** that are partially composed of smaller and simpler instances of themselves. The definition is not circular, since at some point we will reach a list that does not have any lists as elements.

Now suppose our job is to write a function that will sum all of the values in a nested number list. Python has a built-in function which finds the sum of a sequence of numbers:

```
>>> sum([1, 2, 8])
11
```

For our *nested number list*, however, `sum` will not work:

```
>>> sum([1, 2, [11, 13], 8])
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'list'
>>>
```

The problem is that the third element of this list, `[11, 13]`, is itself a list, so it cannot just be added to `1`, `2`, and `8`.

18.3. Processing recursive number lists

To sum all the numbers in our recursive nested number list we need to traverse the list, visiting each of the elements within its nested structure, adding any numeric elements to our sum, and *recursively repeating the summing process* with any elements which are themselves sub-lists.

Thanks to recursion, the Python code needed to sum the values of a nested number list is surprisingly short:

```
1  def r_sum(nested_num_list):
2      tot = 0
3      for element in nested_num_list:
4          if type(element) == type([]):
5              tot += r_sum(element)
6          else:
7              tot += element
8      return tot
```

The body of `r_sum` consists mainly of a `for` loop that traverses `nested_num_list`. If `element` is a numerical value (the `else` branch), it is simply added to `tot`. If `element` is a list, then `r_sum` is called again, with the element as an argument. The statement inside the function definition in which the function calls itself is known as the **recursive call**.

The example above has a **base case** (on line `7`) which does not lead to a recursive call: the case where the element is not a (sub-) list. Without a base case, you'll have **infinite recursion**, and your program will not work.

Recursion is truly one of the most beautiful and elegant tools in computer science.

A slightly more complicated problem is finding the largest value in our nested number list:

```
1  def r_max(nxs):
2      """
3        Find the maximum in a recursive structure of lists
4        within other lists.
5        Precondition: No lists or sublists are empty.
6      """
7      largest = None
8      first_time = True
9      for e in nxs:
10         if type(e) == type([]):
11             val = r_max(e)
12         else:
13             val = e
14
15         if first_time or val > largest:
16             largest = val
17             first_time = False
18
19     return largest
20
21 test(r_max([2, 9, [1, 13], 8, 6]) == 13)
22 test(r_max([2, [[100, 7], 90], [1, 13], 8, 6]) == 100)
23 test(r_max([[[13, 7], 90], 2, [1, 100], 8, 6]) == 100)
24 test(r_max(["joe", ["sam", "ben"]]) == "sam")
```

Tests are included to provide examples of `r_max` at work.

The added twist to this problem is finding a value for initializing `largest`. We can't just use `nxs[0]`, since that could be either a element or a list. To solve this problem (at every recursive call) we initialize a Boolean flag (at line 8). When we've found the value of interest, (at line 15) we check to see whether this is the initializing (first) value for `largest`, or a value that could potentially change `largest`.

Again here we have a base case at line 13. If we don't supply a base case, Python stops after reaching a maximum recursion depth and returns a runtime error. See how this happens, by running this little script which we will call 'infinite_recursion.py':

```
1  def recursion_depth(number):
2      print("{0}, ".format(number), end="")
3      recursion_depth(number + 1)
4
5  recursion_depth(0)
```

After watching the messages flash by, you will be presented with the end of a long traceback that ends with a message like the following:

```
RuntimeError: maximum recursion depth exceeded ...
```

We would certainly never want something like this to happen to a user of one of our programs, so in the next chapter we'll see how errors, any kinds of errors, are handled in Python.

18.4. Case study: Fibonacci numbers

The famous **Fibonacci sequence** 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 134, ... was devised by Fibonacci (1170-1250), who used this to model the breeding of (pairs) of rabbits. If, in generation 7 you had 21 pairs in total, of which 13 were adults, then next generation the adults will all have bred new children, and the previous children will have grown up to become adults. So in generation 8 you'll have 13+21=34, of which 21 are adults.

This *model* to explain rabbit breeding made the simplifying assumption that rabbits never died. Scientists often make (unrealistic) simplifying assumptions and restrictions to make some headway with the problem.

If we number the terms of the sequence from 0, we can describe each term recursively as the sum of the previous two terms:

```
fib(0) = 0
fib(1) = 1
fib(n) = fib(n-1) + fib(n-2)  for n >= 2
```

This translates very directly into some Python:

```
def fib(n):
    if n <= 1:
        return n
    t = fib(n-1) + fib(n-2)
    return t
```

This is a particularly inefficient algorithm, and we'll show one way of fixing it when we learn about dictionaries:

```
import time
t0 = time.clock()
n = 35
result = fib(n)
t1 = time.clock()

print("fib({0}) = {1}, ({2:.2f} secs)".format(n, result, t1-t0))
```

We get the correct result, but an exploding amount of work! :

```
fib(35) = 9227465, (10.54 secs)
```

18.5. Example with recursive directories and files

The following program lists the contents of a directory and all its subdirectories.

```
import os

def get_dirlist(path):
    """
      Return a sorted list of all entries in path.
      This returns just the names, not the full path to the names.
    """
    dirlist = os.listdir(path)
    dirlist.sort()
    return dirlist

def print_files(path, prefix = ""):
    """ Print recursive listing of contents of path """
    if prefix == "":  # Detect outermost call, print a heading
        print("Folder listing for", path)
        prefix = "| "

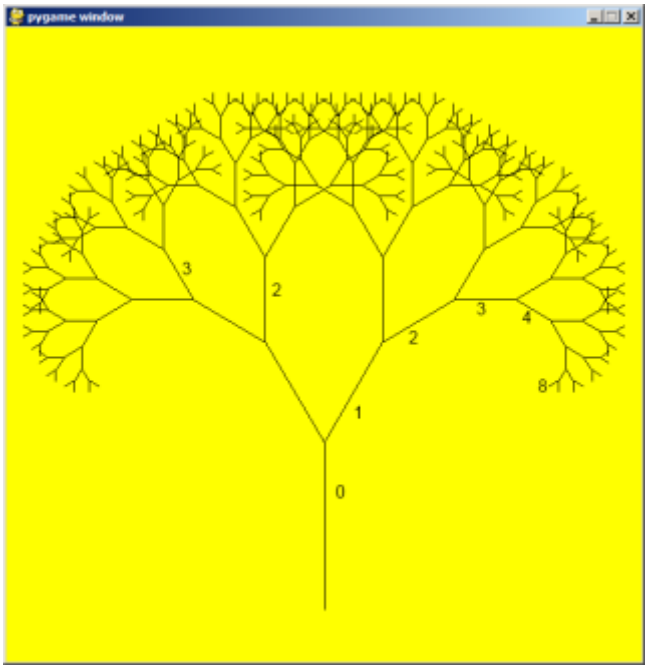
    dirlist = get_dirlist(path)
    for f in dirlist:
        print(prefix+f)                     # Print the line
        fullname = os.path.join(path, f)    # Turn name into full pathname
        if os.path.isdir(fullname):         # If a directory, recurse.
            print_files(fullname, prefix + "| ")
```

Calling the function `print_files` with some folder name will produce output similar to this:

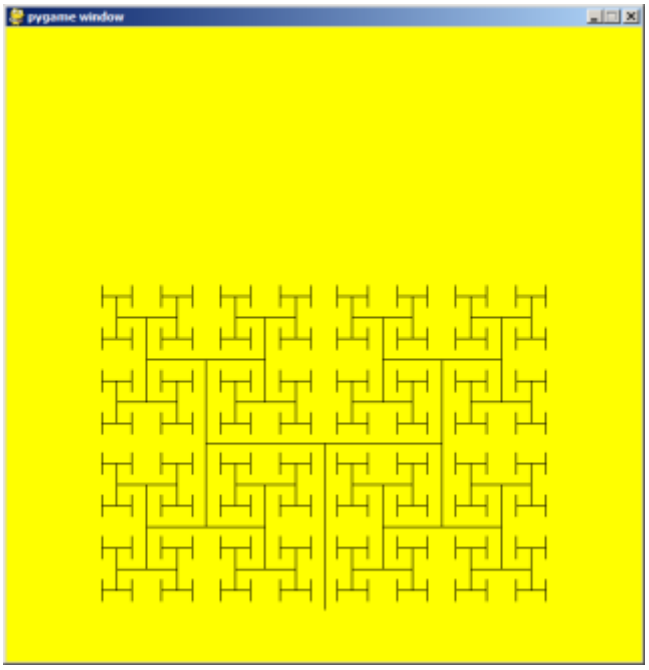
```
Folder listing for c:\python31\Lib\site-packages\pygame\examples
| __init__.py
| aacircle.py
| aliens.py
| arraydemo.py
| blend_fill.py
| blit_blends.py
| camera.py
| chimp.py
| cursors.py
| data
| | alien1.png
| | alien2.png
| | alien3.png
...
```

18.6. An animated fractal, using PyGame

Here we have a tree fractal pattern of order 8. We've labelled some of the edges, showing the depth of the recursion at which each edge was drawn.



In the tree above, the angle of deviation from the trunk is 30 degrees. Varying that angle gives other interesting shapes, for example, with the angle at 90 degrees we get this:



An interesting animation occurs if we generate and draw trees very rapidly, each time varying the angle a little. Although the Turtle module can draw trees like this quite elegantly, we could struggle for good frame rates. So we'll use PyGame instead, with a few embellishments and observations. (Once again, we suggest you cut and paste this code into your Python environment.)

```
import pygame, math
pygame.init()           # prepare the pygame module for use

# Create a new surface and window.
surface_size = 1024
main_surface = pygame.display.set_mode((surface_size,surface_size))
my_clock = pygame.time.Clock()


def draw_tree(order, theta, sz, posn, heading, color=(0,0,0), depth=0):

    trunk_ratio = 0.29       # How big is the trunk relative to whole tree?
    trunk = sz * trunk_ratio # length of trunk
    delta_x = trunk * math.cos(heading)
    delta_y = trunk * math.sin(heading)
    (u, v) = posn
    newpos = (u + delta_x, v + delta_y)
    pygame.draw.line(main_surface, color, posn, newpos)

    if order > 0:   # Draw another layer of subtrees
```

```
21
22        # These next six lines are a simple hack to make the two major halves
23        # of the recursion different colors. Fiddle here to change colors
24        # at other depths, or when depth is even, or odd, etc.
25        if depth == 0:
26            color1 = (255, 0, 0)
27            color2 = (0, 0, 255)
28        else:
29            color1 = color
30            color2 = color
31
32        # make the recursive calls to draw the two subtrees
33        newsz = sz*(1 - trunk_ratio)
34        draw_tree(order-1, theta, newsz, newpos, heading-theta, color1, depth+1)
35        draw_tree(order-1, theta, newsz, newpos, heading+theta, color2, depth+1)
36
37
38  def gameloop():
39
40      theta = 0
41      while True:
42
43          # Handle evente from keyboard, mouse, etc.
44          ev = pygame.event.poll()
45          if ev.type == pygame.QUIT:
46              break;
47
48          # Updates - change the angle
49          theta += 0.01
50
51          # Draw everything
52          main_surface.fill((255, 255, 0))
53          draw_tree(9, theta, surface_size*0.9, (surface_size//2, surface_size-50), -m\
54  ath.pi/2)
55
56          pygame.display.flip()
57          my_clock.tick(120)
58
59
60  gameloop()
61  pygame.quit()
```

- The `math` library works with angles in radians rather than degrees.

- Lines 14 and 15 uses some high school trigonmetry. From the length of the desired line (`trunk`), and its desired angle, `cos` and `sin` help us to calculate the `x` and `y` distances we need to move.

- Lines 22-30 are unnecessary, except if we want a colorful tree.

- In the main game loop at line 49 we change the angle on every frame, and redraw the new tree.

- Line 18 shows that PyGame can also draw lines, and plenty more. Check out the documentation. For example, drawing a small circle at each branch point of the tree can be accomplished by adding this line directly below line 18:

```
1     pygame.draw.circle(main_surface, color, (int(posn[0]), int(posn[1])), 3)
```

Another interesting effect — instructive too, if you wish to reinforce the idea of different instances of the function being called at different depths of recursion —is to create a list of colors, and let each recursive depth use a different color for drawing. (Use the depth of the recursion to index the list of colors.)

18.7. Glossary

base case

A branch of the conditional statement in a recursive function that does not give rise to further recursive calls.

infinite recursion

A function that calls itself recursively without ever reaching any base case. Eventually, infinite recursion causes a runtime error.

recursion

The process of calling a function that is already executing.

recursive call

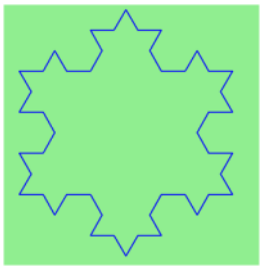
The statement that calls an already executing function. Recursion can also be indirect — function f can call g which calls h, and h could make a call back to f.

recursive definition

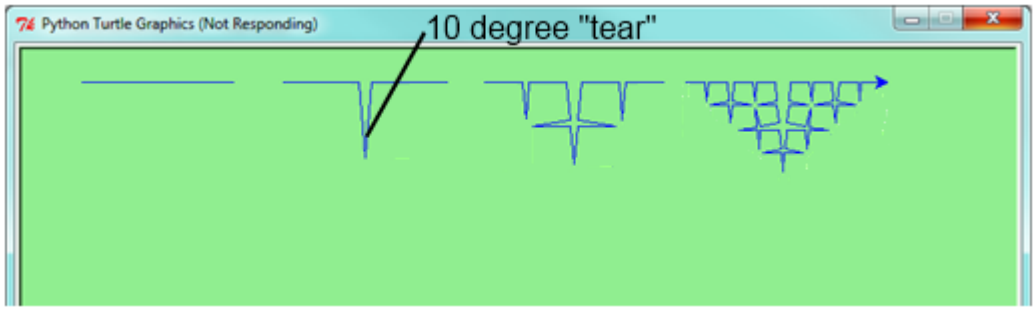
A definition which defines something in terms of itself. To be useful it must include *base cases* which are not recursive. In this way it differs from a *circular definition*. Recursive definitions often provide an elegant way to express complex data structures, like a directory that can contain other directories, or a menu that can contain other menus.

18.8. Exercises

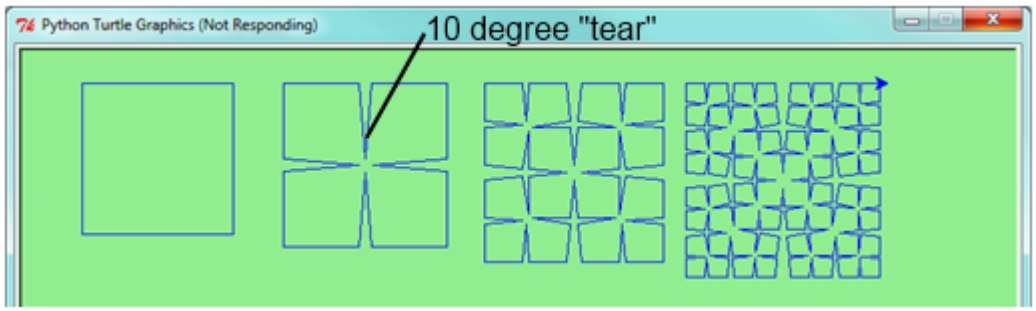
1. Modify the Koch fractal program so that it draws a Koch snowflake, like this:



2. 1. Draw a Cesaro torn line fractal, of the order given by the user. We show four different lines of orders 0,1,2,3. In this example, the angle of the tear is 10 degrees.



2. Four lines make a square. Use the code in part a) to draw cesaro squares. Varying the angle gives interesting effects — experiment a bit, or perhaps let the user input the angle of the tear.



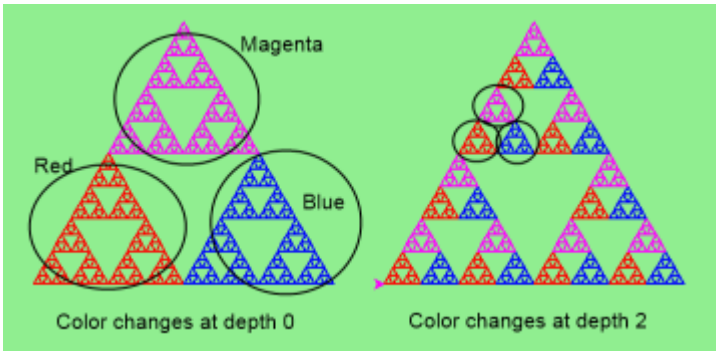
3. (For the mathematically inclined). In the squares shown here, the higher-order drawings become a little larger. (Look at the bottom lines of each square - they're not aligned.) This is because we just halved the drawn part of the line for each recursive subproblem. So we've "grown" the overall square by the width of the tear(s). Can you solve the geometry problem so that the total size of the subproblem case (including the tear) remains exactly the same size as the original?

3. A Sierpinski triangle of order 0 is an equilateral triangle. An order 1 triangle can be drawn by drawing

3 smaller triangles (shown slightly disconnected here, just to help our understanding). Higher order 2 and 3 triangles are also shown. Draw Sierpinski triangles of any order input by the user.

4. Adapt the above program to change the color of its three sub-triangles at some depth of recursion. The

illustration below shows two cases: on the left, the color is changed at depth 0 (the outmost level of recursion), on the right, at depth 2. If the user supplies a negative depth, the color never changes. (Hint: add a new optional parameter `colorChangeDepth` (which defaults to -1), and make this one smaller on each recursive subcall. Then, in the section of code before you recurse, test whether the parameter is
zero, and change color.)



5. Write a function, `recursive_min`, that returns the smallest value in a nested number list. Assume there

are no empty lists or sublists:

```
test(recursive_min([2, 9, [1, 13], 8, 6]) == 1)
test(recursive_min([2, [[100, 1], 90], [10, 13], 8, 6]) == 1)
test(recursive_min([2, [[13, -7], 90], [1, 100], 8, 6]) == -7)
test(recursive_min([[[-13, 7], 90], 2, [1, 100], 8, 6]) == -13)
```

6. Write a function `count` that returns the number of occurrences of `target` in a nested list:

```
test(count(2, []), 0)
test(count(2, [2, 9, [2, 1, 13, 2], 8, [2, 6]]) == 4)
test(count(7, [[9, [7, 1, 13, 2], 8], [7, 6]]) == 2)
test(count(15, [[9, [7, 1, 13, 2], 8], [2, 6]]) == 0)
test(count(5, [[5, [5, [1, 5], 5], 5], [5, 6]]) == 6)
test(count("a",
     [["this",["a",["thing","a"],"a"],"is"], ["a","easy"]]) == 4)
```

7. Write a function `flatten` that returns a simple list containing all the values in a nested list:

```
test(flatten([2,9,[2,1,13,2],8,[2,6]]) == [2,9,2,1,13,2,8,2,6])
test(flatten([[9,[7,1,13,2],8],[7,6]]) == [9,7,1,13,2,8,7,6])
test(flatten([[9,[7,1,13,2],8],[2,6]]) == [9,7,1,13,2,8,2,6])
test(flatten([["this",["a",["thing"],"a"],"is"],["a","easy"]]) ==
              ["this","a","thing","a","is","a","easy"])
test(flatten([]) == [])
```

8. Rewrite the fibonacci algorithm without using recursion. Can you find bigger terms of the sequence? Can

you find `fib(200)`?

9. Use help to find out what `sys.getrecursionlimit()` and `sys.setrecursionlimit(n)` do. Create several

experiments similar to what was done in infinite_recursion.py to test your understanding of how these module functions work.

10. Write a program that walks a directory structure (as in the last section of this chapter), but instead of
printing filenames, it returns a list of all the full paths of files in the directory or the subdirectories. (Don't include directories in this list — just files.) For example, the output list might have elements like this:

    ["C:Python31Lib\site-packages\pygame\docs\ref\mask.html",
    "C:Python31Lib\site-packages\pygame\docs\ref\midi.html",
    …
    "C:Python31Lib\site-packages\pygame\examples\aliens.py",
    …
    "C:Python31Lib\site-packages\pygame\examples\data\boom.wav",
    …]

11. Write a program named `litter.py` that creates an empty file named `trash.txt` in each subdirectory of a
directory tree given the root of the tree as an argument (or the current directory as a default). Now write a program named `cleanup.py` that removes all these files.

Hint #1: Use the program from the example in the last section of this chapter as a basis for these two recursive programs. Because you're going to destroy files on your disks, you better get this right, or you risk losing files you care about. So excellent advice is that initially you should fake the deletion of the files — just print the full path names of each file that you intend to delete. Once you're happy that your logic is correct, and you can see that you're not deleting the wrong things, you can replace the print statement with the real thing.

Hint #2: Look in the `os` module for a function that removes files.

Chapter 19: Exceptions

19.1. Catching exceptions

Whenever a runtime error occurs, it creates an **exception** object. The program stops running at this point and Python prints out the traceback, which ends with a message describing the exception that occurred.

For example, dividing by zero creates an exception:

```
>>> print(55/0)
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
ZeroDivisionError: integer division or modulo by zero
```

So does accessing a non-existent list item:

```
>>> a = []
>>> print(a[5])
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
IndexError: list index out of range
```

Or trying to make an item assignment on a tuple:

```
>>> tup = ("a", "b", "d", "d")
>>> tup[2] = "c"
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

In each case, the error message on the last line has two parts: the type of error before the colon, and specifics about the error after the colon.

Sometimes we want to execute an operation that might cause an exception, but we don't want the program to stop. We can **handle the exception** using the `try` statement to "wrap" a region of code.

For example, we might prompt the user for the name of a file and then try to open it. If the file doesn't exist, we don't want the program to crash; we want to handle the exception:

```
filename = input("Enter a file name: ")
try:
    f = open(filename, "r")
except:
    print("There is no file named", filename)
```

The `try` statement has three separate clauses, or parts, introduced by the keywords `try` ... `except` ... `finally`. Either the `except` or the `finally` clauses can be omitted, so the above code considers the most common version of the `try` statement first.

The `try` statement executes and monitors the statements in the first block. If no exceptions occur, it skips the block under the `except` clause. If any exception occurs, it executes the statements in the `except` clause and then continues.

We could encapsulate this capability in a function: `exists` which takes a filename and returns true if the file exists, false if it doesn't:

```
def exists(filename):
    try:
        f = open(filename)
        f.close()
        return True
    except:
        return False
```

.. The try statement in this function was already introduced previously (the same code), so I thought it would be appropriate to add an else clause here.

pw: HI Victor - I looked at the else: and went against it! It is just a horrible language feature in my view. Not only do we complicate things by overload the keyword else (here, in the if, and in the for loop), but it adds no new expressive power over just doing the "didn't get an exception" inline. And the allowable combinations are hard to explain. You can omit the except clause if you have a finally clause. But you cannot have the else if you omit except, ... and so on. Too much risk for too little return, in my view.

A template to test if a file exists, without using exceptions

The function we've just shown is not one we'd recommend. It opens and closes the file, which is semantically different from asking "does it exist?". How? Firstly, it might update some timestamps on the file. Secondly, it might tell us that there is no such file if some other program already happens to have the file open, or if our permission settings don't allow us to open the file.

Python provides a module called `os.path` within the `os` module. It provides a number of useful functions to work with paths, files and directories, so you should check out the help.

```
import os

# This is the preferred way to check if a file exists.
if os.path.isfile("c:/temp/testdata.txt"):
```

We can use multiple `except` clauses to handle different kinds of exceptions (see the Errors and Exceptions[19] lesson from Python creator Guido van Rossum's Python Tutorial[20] for a more complete discussion of exceptions). So the program could do one thing if the file does not exist, but do something else if the file was in use by another program.

19.2. Raising our own exceptions

Can our program deliberately cause its own exceptions? If our program detects an error condition, we can **raise** an exception. Here is an example that gets input from the user and checks that the number is non-negative:

Line 5 creates an exception object, in this case, a `ValueError` object, which encapsulates specific information about the error. Assume that in this case function `A` called `B` which called `C` which called `D` which called `get_age`. The `raise` statement on line 6 carries this object out as a kind of "return value", and immediately exits from `get_age()` to its caller `D`. Then `D` again exits to its caller `C`, and `C` exits to `B` and so on, each returning the exception object to their caller, until it encounters a `try ... except` that can handle the exception. We call this "unwinding the call stack".

`ValueError` is one of the built-in exception types which most closely matches the kind of error we want to raise. The complete listing of built-in exceptions can be found at the Built-inExceptions[21] section of the Python Library Reference[22] , again by Python's creator, Guido van Rossum.

If the function that called `get_age` (or its caller, or their caller, ...) handles the error, then the program can carry on running; otherwise, Python prints the traceback and exits:

```
>>> get_age()
Please enter your age: 42
42
>>> get_age()
Please enter your age: -2
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
  File "learn_exceptions.py", line 4, in get_age
    raise ValueError("{0} is not a valid age".format(age))
ValueError: -2 is not a valid age
```

[19] http://docs.python.org/py3k/tutorial/errors.html
[20] http://docs.python.org/py3k/tutorial/index.html
[21] http://docs.python.org/py3k/library/exceptions.html
[22] http://docs.python.org/py3k/library/index.html

The error message includes the exception type and the additional information that was provided when the exception object was first created.

It is often the case that lines 5 and 6 (creating the exception object, then raising the exception) are combined into a single statement, but there are really two different and independent things happening, so
perhaps it makes sense to keep the two steps separate when we first learn to work with exceptions. Here we show it all in a single statement:

```
raise ValueError("{0} is not a valid age".format(age))
```

19.3. Revisiting an earlier example

Using exception handling, we can now modify our `recursion_depth` example from the previous chapter so that it stops when it reaches the maximum recursion depth allowed:

```
def recursion_depth(number):
    print("Recursion depth number", number)
    try:
        recursion_depth(number + 1)
    except:
        print("I cannot go any deeper into this wormhole.")

recursion_depth(0)
```

Run this version and observe the results.

19.4. The `finally` clause of the `try` statement

A common programming pattern is to grab a resource of some kind — e.g. we create a window for turtles to draw on, or we dial up a connection to our internet service provider, or we may open a file for writing. Then we perform some computation which may raise an exception, or may work without any problems.

Whatever happens, we want to "clean up" the resources we grabbed — e.g. close the window, disconnect our dial-up connection, or close the file. The `finally` clause of the `try` statement is the way to do just this. Consider this (somewhat contrived) example:

```
1  import turtle
2  import time
3
4  def show_poly():
5      try:
6          win = turtle.Screen()   # Grab/create a resource, e.g. a window
7          tess = turtle.Turtle()
8
9          # This dialog could be cancelled,
10         #   or the conversion to int might fail, or n might be zero.
11         n = int(input("How many sides do you want in your polygon?"))
12         angle = 360 / n
13         for i in range(n):      # Draw the polygon
14             tess.forward(10)
15             tess.left(angle)
16         time.sleep(3)           # Make program wait a few seconds
17     finally:
18         win.bye()               # Close the turtle's window
19
20 show_poly()
21 show_poly()
22 show_poly()
```

In lines 20–22, `show_poly` is called three times. Each one creates a new window for its turtle, and draws a polygon with the number of sides input by the user. But what if the user enters a string that cannot be converted to an `int`? What if they close the dialog? We'll get an exception, *but even though we've had an exception, we still want to close the turtle's window.* Lines 17–18 does this for us. Whether we complete the statements in the `try` clause successfully or not, the `finally` block will always be executed.

Notice that the exception is still unhandled — only an `except` clause can handle an exception, so our program will still crash. But at least its turtle window will be closed before it crashes!

19.5. Glossary

```
exception
    An error that occurs at runtime.

handle an exception
    To prevent an exception from causing our program to crash, by wrapping
    the block of code in a `try` ... `except` construct.

raise
    To create a deliberate exception by using the `raise` statement.
```

19.6. Exercises

1. Write a function named `readposint` that uses the `input` dialog to prompt the user for a positive integer and then checks the input to confirm that it meets the requirements. It should be able to handle
inputs that cannot be converted to `int`, as well as negative `int`s, and edge cases (e.g. when the user closes the dialog, or does not enter anything at all.)

Chapter 20: Dictionaries

All of the compound data types we have studied in detail so far — strings, lists, and tuples — are sequence types, which use integers as indices to access the values they contain within them.

Dictionaries are yet another kind of compound type. They are Python's built-in **mapping type**. They map **keys**, which can be any immutable type, to values, which can be any type (heterogeneous), just like the elements of a list or tuple. In other languages, they are called associative arrays since they associate a key with a value.

As an example, we will create a dictionary to translate English words into Spanish. For this dictionary, the keys are strings.

One way to create a dictionary is to start with the empty dictionary and add **key:value pairs**. The empty dictionary is denoted `{}`:

```
>>> eng2sp = {}
>>> eng2sp["one"] = "uno"
>>> eng2sp["two"] = "dos"
```

The first assignment creates a dictionary named `eng2sp`; the other assignments add new key:value pairs to the dictionary. We can print the current value of the dictionary in the usual way:

```
>>> print(eng2sp)
{"two": "dos", "one": "uno"}
```

The key:value pairs of the dictionary are separated by commas. Each pair contains a key and a value separated by a colon.

Hashing

The order of the pairs may not be what was expected. Python uses complex algorithms, designed for very fast access, to determine where the key:value pairs are stored in a dictionary. For our purposes we can think of this ordering as unpredictable.

You also might wonder why we use dictionaries at all when the same concept of mapping a key to a value could be implemented using a list of tuples:

```
>>> {"apples": 430, "bananas": 312, "oranges": 525, "pears": 217}
{'pears': 217, 'apples': 430, 'oranges': 525, 'bananas': 312}
>>> [('apples', 430), ('bananas', 312), ('oranges', 525), ('pears', 217)]
[('apples', 430), ('bananas', 312), ('oranges', 525), ('pears', 217)]
```

The reason is dictionaries are very fast, implemented using a technique called hashing, which allows us to access a value very quickly. By contrast, the list of tuples implementation is slow. If we wanted to find a value associated with a key, we would have to iterate over every tuple, checking the 0th element. What if the key wasn't even in the list? We would have to get to the end of it to find out.

```
Another way to create a dictionary is to provide a list of key:value pairs using the\
 same syntax as the previous output:

```python
>>> eng2sp = {"one": "uno", "two": "dos", "three": "tres"}
```

It doesn't matter what order we write the pairs. The values in a dictionary are accessed with keys, not with indices, so there is no need to care about ordering.

Here is how we use a key to look up the corresponding value:

```
>>> print(eng2sp["two"])
'dos'
```

The key `"two"` yields the value `"dos"`.

Lists, tuples, and strings have been called *sequences*, because their items occur in order. The dictionary is the first compound type that we've seen that is not a sequence, so we can't index or slice a dictionary.

20.1. Dictionary operations

The `del` statement removes a key:value pair from a dictionary. For example, the following dictionary contains the names of various fruits and the number of each fruit in stock:

```
>>> inventory = {"apples": 430, "bananas": 312, "oranges": 525, "pears": 217}
>>> print(inventory)
{'pears': 217, 'apples': 430, 'oranges': 525, 'bananas': 312}
```

If someone buys all of the pears, we can remove the entry from the dictionary:
```

```
>>> del inventory["pears"]
>>> print(inventory)
{'apples': 430, 'oranges': 525, 'bananas': 312}
```

Or if we're expecting more pears soon, we might just change the value associated with pears:

```
>>> inventory["pears"] = 0
>>> print(inventory)
{'pears': 0, 'apples': 430, 'oranges': 525, 'bananas': 312}
```

A new shipment of bananas arriving could be handled like this:

```
>>> inventory["bananas"] += 200
>>> print(inventory)
{'pears': 0, 'apples': 430, 'oranges': 525, 'bananas': 512}
```

The `len` function also works on dictionaries; it returns the number of key:value pairs:

```
>>> len(inventory)
4
```

20.2. Dictionary methods

Dictionaries have a number of useful built-in methods.

The `keys` method returns what Python 3 calls a **view** of its underlying keys. A view object has some similarities to the `range` object we saw earlier —it is a lazy promise, to deliver its elements when they're needed by the rest of the program. We can iterate over the view, or turn the view into a list like this:

```
for k in eng2sp.keys():   # The order of the k's is not defined
   print("Got key", k, "which maps to value", eng2sp[k])

ks = list(eng2sp.keys())
print(ks)
```

This produces this output:

```
Got key three which maps to value tres
Got key two which maps to value dos
Got key one which maps to value uno
['three', 'two', 'one']
```

It is so common to iterate over the keys in a dictionary that we can omit the `keys` method call in the `for` loop — iterating over a dictionary implicitly iterates over its keys:

```
for k in eng2sp:
   print("Got key", k)
```

The `values` method is similar; it returns a view object which can be turned into a list:

```
>>> list(eng2sp.values())
['tres', 'dos', 'uno']
```

The `items` method also returns a view, which promises a list of tuples — one tuple for each key:value pair:

```
>>> list(eng2sp.items())
[('three', 'tres'), ('two', 'dos'), ('one', 'uno')]
```

Tuples are often useful for getting both the key and the value at the same time while we are looping:

```
for (k,v) in eng2sp.items():
    print("Got",k,"that maps to",v)
```

This produces:

```
Got three that maps to tres
Got two that maps to dos
Got one that maps to uno
```

The `in` and `not in` operators can test if a key is in the dictionary:

```
>>> "one" in eng2sp
True
>>> "six" in eng2sp
False
>>> "tres" in eng2sp    # Note that 'in' tests keys, not values.
False
```

This method can be very useful, since looking up a non-existent key in a dictionary causes a runtime error:

```
>>> eng2esp["dog"]
Traceback (most recent call last):
  ...
KeyError: 'dog'
```

20.3. Aliasing and copying

As in the case of lists, because dictionaries are mutable, we need to be aware of aliasing. Whenever two variables refer to the same object, changes to one affect the other.

If we want to modify a dictionary and keep a copy of the original, use the `copy` method. For example, `opposites` is a dictionary that contains pairs of opposites:

```
>>> opposites = {"up": "down", "right": "wrong", "yes": "no"}
>>> alias = opposites
>>> copy = opposites.copy()  # Shallow copy
```

`alias` and `opposites` refer to the same object; `copy` refers to a fresh copy of the same dictionary. If we modify `alias`, `opposites` is also changed:

```
>>> alias["right"] = "left"
>>> opposites["right"]
'left'
```

If we modify `copy`, `opposites` is unchanged:

```
>>> copy["right"] = "privilege"
>>> opposites["right"]
'left'
```

20.4. Sparse matrices

We previously used a list of lists to represent a matrix. That is a good choice for a matrix with mostly nonzero values, but consider a sparse matrix[23] like this one:

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{bmatrix}$$

The list representation contains a lot of zeroes:

```
matrix = [[0, 0, 0, 1, 0],
          [0, 0, 0, 0, 0],
          [0, 2, 0, 0, 0],
          [0, 0, 0, 0, 0],
          [0, 0, 0, 3, 0]]
```

An alternative is to use a dictionary. For the keys, we can use tuples that contain the row and column numbers. Here is the dictionary representation of the same matrix:

```
>>> matrix = {(0, 3): 1, (2, 1): 2, (4, 3): 3}
```

We only need three key:value pairs, one for each nonzero element of the matrix. Each key is a tuple, and each value is an integer.

To access an element of the matrix, we could use the `[]` operator:

```
>>> matrix[(0, 3)]
1
```

Notice that the syntax for the dictionary representation is not the same as the syntax for the nested list representation. Instead of two integer indices, we use one index, which is a tuple of integers.

There is one problem. If we specify an element that is zero, we get an error, because there is no entry in the dictionary with that key:

```
>>> matrix[(1, 3)]
KeyError: (1, 3)
```

The `get` method solves this problem:

[23] http://en.wikipedia.org/wiki/Sparse_matrix

```
>>> matrix.get((0, 3), 0)
1
```

The first argument is the key; the second argument is the value `get` should return if the key is not in the dictionary:

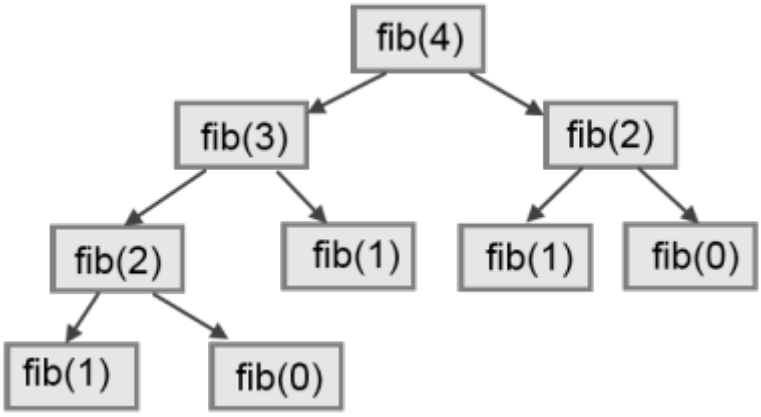
```
>>> matrix.get((1, 3), 0)
0
```

`get` definitely improves the semantics of accessing a sparse matrix. Shame about the syntax.

20.5. Memoization

If you played around with the `fibo` function from the chapter on recursion, you might have noticed that the bigger the argument you provide, the longer the function takes to run. Furthermore, the run time increases very quickly. On one of our machines, `fib(20)` finishes instantly, `fib(30)` takes about a second, and `fib(40)` takes roughly forever.

To understand why, consider this **call graph** for `fib` with `n = 4`:



A call graph shows some function frames (instances when the function has been invoked), with lines connecting each frame to the frames of the functions it calls. At the top of the graph, `fib` with `n = 4` calls `fib` with `n = 3` and `n = 2`. In turn, `fib` with `n = 3` calls `fib` with `n = 2` and `n = 1`. And so on.

Count how many times `fib(0)` and `fib(1)` are called. This is an inefficient solution to the problem, and it gets far worse as the argument gets bigger.

A good solution is to keep track of values that have already been computed by storing them in a dictionary. A previously computed value that is stored for later use is called a **memo**. Here is an implementation of `fib` using memos:

```
alreadyknown = {0: 0, 1: 1}

def fib(n):
    if n not in alreadyknown:
        new_value = fib(n-1) + fib(n-2)
        alreadyknown[n] = new_value
    return alreadyknown[n]
```

The dictionary named `alreadyknown` keeps track of the Fibonacci numbers we already know. We start with only two pairs: 0 maps to 1; and 1 maps to 1.

Whenever `fib` is called, it checks the dictionary to determine if it contains the result. If it's there, the function can return immediately without making any more recursive calls. If not, it has to compute the new value. The new value is added to the dictionary before the function returns.

Using this version of `fib`, our machines can compute `fib(100)` in an eyeblink.

```
>>> fib(100)
354224848179261915075
```

20.6. Counting letters

In the exercises in Chapter 8 (Strings) we wrote a function that counted the number of occurrences of a letter in a string. A more general version of this problem is to form a frequency table of the letters in the string, that is, how many times each letter appears.

Such a frequency table might be useful for compressing a text file. Because different letters appear with different frequencies, we can compress a file by using shorter codes for common letters and longer codes for letters that appear less frequently.

Dictionaries provide an elegant way to generate a frequency table:

```
>>> letter_counts = {}
>>> for letter in "Mississippi":
...     letter_counts[letter] = letter_counts.get(letter, 0) + 1
...
>>> letter_counts
{'M': 1, 's': 4, 'p': 2, 'i': 4}
```

We start with an empty dictionary. For each letter in the string, we find the current count (possibly zero) and increment it. At the end, the dictionary contains pairs of letters and their frequencies.

It might be more appealing to display the frequency table in alphabetical order. We can do that with the `items` and `sort` methods:

```
>>> letter_items = list(letter_counts.items())
>>> letter_items.sort()
>>> print(letter_items)
[('M', 1), ('i', 4), ('p', 2), ('s', 4)]
```

Notice in the first line we had to call the type conversion function `list`. That turns the promise we get from `items` into a list, a step that is needed before we can use the list's `sort` method.

20.7. Glossary

```
call graph
    A graph consisting of nodes which represent function frames (or
    invocations), and directed edges (lines with arrows) showing which
    frames gave rise to other frames.

dictionary
    A collection of key:value pairs that maps from keys to values. The keys
    can be any immutable value, and the associated value can be of any type.

immutable data value
    A data value which cannot be modified. Assignments to elements or slices
    (sub-parts) of immutable values cause a runtime error.

key
    A data item that is *mapped to* a value in a dictionary. Keys are used
    to look up values in a dictionary. Each key must be unique across the
    dictionary.

key:value pair
    One of the pairs of items in a dictionary. Values are looked up in a
    dictionary by key.

mapping type
    A mapping type is a data type comprised of a collection of keys and
    associated values. Python's only built-in mapping type is the
    dictionary. Dictionaries implement the [associative array](http://en.wikipedia.o\
rg/wiki/Associative_array) abstract data type.

memo
    Temporary storage of precomputed values to avoid duplicating the same
    computation.

```

```
mutable data value
    A data value which can be modified. The types of all mutable values are
    compound types. Lists and dictionaries are mutable; strings and tuples
    are not.
```

20.8. Exercises

1. Write a program that reads a string and returns a table of the letters of the alphabet in alphabetical order which occur in the string together with the number of times each letter occurs. Case should be ignored. A sample output of the program when the user enters the data "ThiS is String with Upper and lower case Letters", would look this this:

a 2
c 1
d 1
e 5
g 1
h 2
i 4
l 2
n 2
o 1
p 2
r 4
s 5
t 5
u 1
w 2

2. Give the Python interpreter's response to each of the following from a continuous interpreter session:

```
>>> d = {"apples": 15, "bananas": 35, "grapes": 12}
>>> d["bananas"]
```

```
>>> d["oranges"] = 20
>>> len(d)
```

```
>>> "grapes" in d
```

```
>>> d["pears"]
```

```
>>> d.get("pears", 0)
```

```
>>> fruits = list(d.keys())
>>> fruits.sort()
>>> print(fruits)
```

```
>>> del d["apples"]
>>> "apples" in d
```

Be sure you understand why you get each result. Then apply what you have learned to fill in the body of the function below:

```
def add_fruit(inventory, fruit, quantity=0):
    return

# Make these tests work...
new_inventory = {}
add_fruit(new_inventory, "strawberries", 10)
test("strawberries" in new_inventory)
test(new_inventory["strawberries"] == 10)
add_fruit(new_inventory, "strawberries", 25)
test(new_inventory["strawberries"] == 35)
```

3. Write a program called `alice_words.py` that creates a text file named `alice_words.txt` containing an

alphabetical listing of all the words, and the number of times each occurs, in the text version of Alice's Adventures in Wonderland. (You can obtain a free plain text version of the book, along with many others, from http://www.gutenberg.org.) The first 10 lines of your output file should look something like this:

Word Count
==========
a 631

a-piece 1
abide 1
able 1
about 94
above 3
absence 1
absurd 2

How many times does the word `alice` occur in the book?

4. What is the longest word in Alice in Wonderland? How many characters does it have?

Chapter 21: A Case Study: Indexing your files

We present a small case study that ties together modules, recursion, files, dictionaries and introduces simple serialization and deserialization.

In this chapter we're going to use a dictionary to help us find a file rapidly.

The case study has two components:

- A crawler program that scans the disk (or folder) and constructs and saves the dictionary to your disk.
- A query program that loads the dictionary and can rapidly answer user queries about where a file is

located.

21.1. The Crawler

Near the end of the chapter on recursion we showed an example of how to recursively list all files under a given path of our filesystem.

We'll borrow (and change) that code somewhat to provide the skeleton of our crawler. It's function is to recursively traverse every file in a given path. (We'll figure out what to do with the file soon: for the moment we'll just print it's short name, and its full path.)

```
1  # Crawler crawls the filesystem and builds a dictionary
2  import os
3
4  def crawl_files(path):
5      """ Recursively visit all files in path """
6
7      # Fetch all the entries in the current folder.
8      dirlist = os.listdir(path)
9      for f in dirlist:
10         # Turn each name into full pathname
11         fullname = os.path.join(path, f)
12
13         # If it is a directory, recurse.
```

```
14          if os.path.isdir(fullname):
15              crawl_files(fullname)
16          else:  # Do something useful with the file
17              print("{0:30} {1}".format(f, fullname))
18
19  crawl_files("C:\\Python32")
```

We get output similar to this:

```
1   CherryPy-wininst.log           C:\Python32\CherryPy-wininst.log
2   bz2.pyd                        C:\Python32\DLLs\bz2.pyd
3   py.ico                         C:\Python32\DLLs\py.ico
4   pyc.ico                        C:\Python32\DLLs\pyc.ico
5   pyexpat.pyd                    C:\Python32\DLLs\pyexpat.pyd
6   python3.dll                    C:\Python32\DLLs\python3.dll
7   select.pyd                     C:\Python32\DLLs\select.pyd
8   sqlite3.dll                    C:\Python32\DLLs\sqlite3.dll
9   tcl85.dll                      C:\Python32\DLLs\tcl85.dll
10  tclpip85.dll                   C:\Python32\DLLs\tclpip85.dll
11  tk85.dll                       C:\Python32\DLLs\tk85.dll
12  ...
```

We'll adapt this now to store the short name and the full path of the file as a key:value pair in a dictionary. But first, two observations:

- We can have many instances of files with the same name (in different paths). For example, the name index.

html is quite common. But dictionary keys must be unique. Our solution is to map each key in our dictionary to a *list* of paths.

- File names are not case sensitive. (Well, not for Windows users!) So a good technique is to *normalize*

the keys before storing them. Here we'll just ensure that all keys are converted to lowercase. Of course we'll do the same later when we write the query program.

We'll change the code above by setting up a global dictionary, initially empty: The statement `thedict = {}` inserted at line 3 will do this. Then instead of printing the information at line 17, we'll add the filename and path to the dictionary. The code will need to check whether the key already exists:

```
key = f.lower()  # Normalize the filename
if key in thedict:
   thedict[key].append(fullname)
else:   # insert the key and a list of one pathname
   thedict[key] = [fullname]
```

After running for a while the program terminates. We can interactively confirm that the dictionary seems to have been built correctly:

```
>>> len(thedict)
14861
>>> thedict["python.exe"]
['C:\\Python32\\python.exe']
>>> thedict["logo.png"]
['C:\\Python32\\Lib\\site-packages\\PyQt4\\doc\\html\\_static\\logo.png',
 'C:\\Python32\\Lib\\site-packages\\PyQt4\\doc\\sphinx\\static\\logo.png',
 'C:\\Python32\\Lib\\site-packages\\PyQt4\\examples\\demos\\textedit\\images\\logo.p\
ng',
 'C:\\Python32\\Lib\\site-packages\\sphinx-1.1.3-py3.2.egg\\sphinx\\themes\\scrolls\\
\static\\logo.png']
>>>
```

It would be nice to add a progress bar while the crawler is running: a typical technique is to print dots to show progress. We'll introduce a count of how many files have been indexed (this can be a global variable), and after we've handled the current file, we'll add this code:

```
filecount += 1
if filecount % 100 == 0:
    print(".", end="")
    if filecount % 5000 == 0:
        print()
```

As we complete each 100 files we print a dot. After every 50 dots we start a new line. You'll need to also create the global variable, initialize it to zero, and remember to declare the variable as global in the crawler.

The main calling code can now print some statistics for us. It becomes

```
crawl_files("C:\\Python32")
print()  # End the last line of dots ...
print("Indexed {0} files, {1} entries in the dictionary.".
                    format(filecount, len(thedict)))
```

We'll now get something like

```
>>>
.................................................
.................................................
.................................................
.................................
Indexed 18635 files, 14861 entries in the dictionary.
>>>
```

It is reassuring to look at the properties of the folder in our operating system, and note that it counts exactly the same number of files as our program does!

21.2. Saving the dictionary to disk

The dictionary we've built is an object. To save it we're going to turn it into a string, and write the string to a file on our disk. The string must be in a format that allows another program to unambiguously reconstruct another dictionary with the same key-value entries. The process of turning an object into a string representation is called **serialization**, and the inverse operation — reconstructing a new object from a string —is called **deserialization**.

There are a few ways to do this: some use binary formats, some use text formats, and the way different types of data are encoded differs. A popular, lightweight technique used extensively in web servers and web pages is to use JSON (JavaScript Object Notation) encoding.

Amazingly, we need just four new lines of code to save our dictionary to our disk:

```
import json

f = open("C:\\temp\\mydict.txt", "w")
json.dump(thedict, f)
f.close()
```

You can find the file on your disk and open it with a text editor to see what the JSON encoding looks like.

21.3. The Query Program

This needs to reconstruct the dictionary from the disk file, and then provide a lookup function:

```
import json

f = open("C:\\temp\\mydict.txt", "r")
dict = json.load(f)
f.close()
print("Loaded {0} filenames for querying.".format(len(dict)))

def query(filename):
    f = filename.lower()
    if f not in dict:
        print("No hits for {0}".format(filename))
    else:
        print("{0} is at ".format(filename))
        for p in dict[f]:
            print("...", p)
```

And here is a sample run:

```
>>>
Loaded 14861 filenames for querying.
>>> query('python.exe')
python.exe is at
... C:\Python32\python.exe
>>> query('java.exe')
No hits for java.exe
>>> query('INDEX.HtMl')
INDEX.HtMl is at
... C:\Python32\Lib\site-packages\cherrypy\test\static\index.html
... C:\Python32\Lib\site-packages\eric5\Documentation\Source\index.html
... C:\Python32\Lib\site-packages\IPython\frontend\html\notebook\static\codemirror\m\
ode\css\index.html
... C:\Python32\Lib\site-packages\IPython\frontend\html\notebook\static\codemirror\m\
ode\htmlmixed\index.html
... C:\Python32\Lib\site-packages\IPython\frontend\html\notebook\static\codemirror\m\
ode\javascript\index.html
... C:\Python32\Lib\site-packages\IPython\frontend\html\notebook\static\codemirror\m\
ode\markdown\index.html
... C:\Python32\Lib\site-packages\IPython\frontend\html\notebook\static\codemirror\m\
ode\python\index.html
... C:\Python32\Lib\site-packages\IPython\frontend\html\notebook\static\codemirror\m\
ode\rst\index.html
... C:\Python32\Lib\site-packages\IPython\frontend\html\notebook\static\codemirror\m\
ode\xml\index.html
```

```
... C:\Python32\Lib\site-packages\pygame\docs\index.html
... C:\Python32\Lib\site-packages\pygame\docs\ref\index.html
... C:\Python32\Lib\site-packages\PyQt4\doc\html\index.html
>>>
```

21.4. Compressing the serialized dictionary

The JSON file might get quite big. Gzip compression is available in Python, so let's take advantage of it...

When we saved the dictionary to disk we opened a text file for writing. We simply have to change that one line of the program (and import the correct modules), to create a gzip file instead of a normal text file. The replacement code is

```
import json, gzip, io

## f = open("C:\\temp\\mydict.txt", "w")
f = io.TextIOWrapper(gzip.open("C:\\temp\\mydict.gz", mode="wb"))
json.dump(thedict, f)
f.close()
```

Magically, we now get a zipped file that is about 7 times smaller than the text version. (Compressiion/decompression like this is often done by web servers and browsers for significantly faster downloads.)

Now, of course, our query program needs to uncompress the data:

```
import json, gzip, io

## f = open("C:\\temp\\mydict.txt", "r")
f = io.TextIOWrapper(gzip.open("C:\\temp\\mydict.gz", mode="r"))
dict = json.load(f)
f.close()
print("Loaded {0} filenames for querying.".format(len(dict)))
```

Composability is the key...

In the earliest chapters of the book we talked about composability: the ability to join together or *compose* different fragments of code and functionality to build more powerful constructs.

This case study has shown an excellent example of this. Our JSON serializer and deserializer can link with our file mechanisms. The gzip compressor / decompressor can also present itself to our program as as if it was just a specialized stream of data, as one might get from reading a file. The end result is a very elegant composition of powerful tools. Instead of requiring separate steps for serializing the dictionary to a string, compressing the string, writing the resulting bytes to a file, etc., the composability has let us do it all very easily!

21.5. Glossary

deserialization
Reconstruction an in-memory object from some external text representation

gzip
A lossless compression technique that reduces the storage size of data. (Lossless means you can recover the original data exactly.)

JSON
JavaScript Object Notation is a format for serializing and transporting objects, often used between web servers and web browsers that run JavasScript. Python contains a `json` module to provide this capability.

serialization
Turning an object into a string (or bytes) so that it can be sent over the internet, or saved in a file. The recipient can reconstruct a new object from the data.

Chapter 22: Even more OOP

22.1. MyTime

As another example of a user-defined type, we'll define a class called `MyTime` that records the time of day. We'll provide an `__init__` method to ensure that every instance is created with appropriate attributes and initialization. The class definition looks like this:

```
class MyTime:

    def __init__(self, hrs=0, mins=0, secs=0):
        """ Create a MyTime object initialized to hrs, mins, secs """
        self.hours = hrs
        self.minutes = mins
        self.seconds = secs
```

We can instantiate a new `MyTime` object:

```
tim1 = MyTime(11, 59, 30)
```

The state diagram for the object looks like this:

We'll leave it as an exercise for the readers to add a `__str__` method so that MyTime objects can print themselves decently.

22.2. Pure functions

In the next few sections, we'll write two versions of a function called `add_time`, which calculates the sum of two `MyTime` objects. They will demonstrate two kinds of functions: pure functions and modifiers.

The following is a rough version of `add_time`:

```
def add_time(t1, t2):
    h = t1.hours + t2.hours
    m = t1.minutes + t2.minutes
    s = t1.seconds + t2.seconds
    sum_t = MyTime(h, m, s)
    return sum_t
```

The function creates a new `MyTime` object and returns a reference to the new object. This is called a **pure function** because it does not modify any of the objects passed to it as parameters and it has no side effects, such as updating global variables, displaying a value, or getting user input.

Here is an example of how to use this function. We'll create two `MyTime` objects: `current_time`, which contains the current time; and `bread_time`, which contains the amount of time it takes for a breadmaker to make bread. Then we'll use `add_time` to figure out when the bread will be done.

```
>>> current_time = MyTime(9, 14, 30)
>>> bread_time = MyTime(3, 35, 0)
>>> done_time = add_time(current_time, bread_time)
>>> print(done_time)
12:49:30
```

The output of this program is `12:49:30`, which is correct. On the other hand, there are cases where the result is not correct. Can you think of one?

The problem is that this function does not deal with cases where the number of seconds or minutes adds up to more than sixty. When that happens, we have to carry the extra seconds into the minutes column or the extra minutes into the hours column.

Here's a better version of the function:

```
def add_time(t1, t2):

    h = t1.hours + t2.hours
    m = t1.minutes + t2.minutes
    s = t1.seconds + t2.seconds

    if s >= 60:
        s -= 60
        m += 1

    if m >= 60:
        m -= 60
        h += 1

```

```
    sum_t = MyTime(h, m, s)
    return sum_t
```

This function is starting to get bigger, and still doesn't work for all possible cases. Later we will suggest an alternative approach that yields better code.

22.3. Modifiers

There are times when it is useful for a function to modify one or more of the objects it gets as parameters. Usually, the caller keeps a reference to the objects it passes, so any changes the function makes are visible to the caller. Functions that work this way are called **modifiers**.

`increment`, which adds a given number of seconds to a `MyTime` object, would be written most naturally as a modifier. A rough draft of the function looks like this:

```
def increment(t, secs):
    t.seconds += secs

    if t.seconds >= 60:
        t.seconds -= 60
        t.minutes += 1

    if t.minutes >= 60:
        t.minutes -= 60
        t.hours += 1
```

The first line performs the basic operation; the remainder deals with the special cases we saw before.

Is this function correct? What happens if the parameter `seconds` is much greater than sixty? In that case, it is not enough to carry once; we have to keep doing it until `seconds` is less than sixty. One solution is to replace the `if` statements with `while` statements:

```
def increment(t, seconds):
    t.seconds += seconds

    while t.seconds >= 60:
        t.seconds -= 60
        t.minutes += 1

    while t.minutes >= 60:
        t.minutes -= 60
        t.hours += 1
```

This function is now correct when seconds is not negative, and when hours does not exceed 23, but it is not a particularly good solution.

22.4. Converting `increment` to a method

Once again, OOP programmers would prefer to put functions that work with `MyTime` objects into the `MyTime` class, so let's convert `increment` to a method. To save space, we will leave out previously defined methods, but you should keep them in your version:

```
class MyTime:
    # Previous method definitions here...

    def increment(self, seconds):
        self.seconds += seconds

        while self.seconds >= 60:
            self.seconds -= 60
            self.minutes += 1

        while self.minutes >= 60:
            self.minutes -= 60
            self.hours += 1
```

The transformation is purely mechanical — we move the definition into the class definition and (optionally) change the name of the first parameter to `self`, to fit with Python style conventions.

Now we can invoke `increment` using the syntax for invoking a method.

```
current_time.increment(500)
```

Again, the object on which the method is invoked gets assigned to the first parameter, `self`. The second parameter, `seconds` gets the value `500`.

22.5. An "Aha!" insight

Often a high-level insight into the problem can make the programming much easier.

In this case, the insight is that a `MyTime` object is really a three-digit number in base 60! The `second` component is the ones column, the `minute` component is the sixties column, and the `hour` component is the thirty-six hundreds column.

When we wrote `add_time` and `increment`, we were effectively doing addition in base 60, which is why we had to carry from one column to the next.

This observation suggests another approach to the whole problem — we can convert a `MyTime` object into a single number and take advantage of the fact that the computer knows how to do arithmetic with numbers. The following method is added to the `MyTime` class to convert any instance into a corresponding number of seconds:

```
class MyTime:
    # ...

    def to_seconds(self):
        """ Return the number of seconds represented
            by this instance
        """
        return self.hours * 3600 + self.minutes * 60 + self.seconds
```

Now, all we need is a way to convert from an integer back to a `MyTime` object. Supposing we have `tsecs` seconds, some integer division and mod operators can do this for us:

```
hrs = tsecs // 3600
leftoversecs = tsecs % 3600
mins = leftoversecs // 60
secs = leftoversecs % 60
```

You might have to think a bit to convince yourself that this technique to convert from one base to another is correct.

In OOP we're really trying to wrap together the data and the operations that apply to it. So we'd like to have this logic inside the `MyTime` class. A good solution is to rewrite the class initializer so that it can cope with initial values of seconds or minutes that are outside the **normalized** values. (A normalized time would be something like 3 hours 12 minutes and 20 seconds. The same time, but unnormalized could be 2 hours 70 minutes and 140 seconds.)

Let's rewrite a more powerful initializer for `MyTime`:

```
class MyTime:
    # ...

    def __init__(self, hrs=0, mins=0, secs=0):
        """ Create a new MyTime object initialized to hrs, mins, secs.
            The values of mins and secs may be outside the range 0-59,
            but the resulting MyTime object will be normalized.
        """

        # Calculate total seconds to represent
        totalsecs = hrs*3600 + mins*60 + secs
        self.hours = totalsecs // 3600        # Split in h, m, s
        leftoversecs = totalsecs % 3600
        self.minutes = leftoversecs // 60
        self.seconds = leftoversecs % 60
```

Now we can rewrite `add_time` like this:

```
def add_time(t1, t2):
    secs = t1.to_seconds() + t2.to_seconds()
    return MyTime(0, 0, secs)
```

This version is much shorter than the original, and it is much easier to demonstrate or reason that it is correct.

22.6. Generalization

In some ways, converting from base 60 to base 10 and back is harder than just dealing with times. Base conversion is more abstract; our intuition for dealing with times is better.

But if we have the insight to treat times as base 60 numbers and make the investment of writing the conversions, we get a program that is shorter, easier to read and debug, and more reliable.

It is also easier to add features later. For example, imagine subtracting two `MyTime` objects to find the duration between them. The naive approach would be to implement subtraction with borrowing. Using the conversion functions would be easier and more likely to be correct.

Ironically, sometimes making a problem harder (or more general) makes the programming easier, because there are fewer special cases and fewer opportunities for error.

Specialization versus Generalization

Computer Scientists are generally fond of specializing their types, while mathematicians often take the opposite approach, and generalize everything.

What do we mean by this?

If we ask a mathematician to solve a problem involving weekdays, days of the century, playing cards, time, or dominoes, their most likely response is to observe that all these objects can be represented by integers. Playing cards, for example, can be numbered from 0 to 51. Days within the century can be numbered. Mathematicians will say *"These things are enumerable — the elements can be uniquely numbered (and we can reverse this numbering to get back to the original concept). So let's number them, and confine our thinking to integers. Luckily, we have powerful techniques and a good understanding of integers, and so our abstractions — the way we tackle and simplify these problems — is to try to reduce them to problems about integers."*

Computer Scientists tend to do the opposite. We will argue that there are many integer operations that are simply not meaningful for dominoes, or for days of the century. So we'll often define new specialized types, like `MyTime`, because we can restrict, control, and specialize the operations that are possible. Object-oriented programming is particularly popular because it gives us a good way to bundle methods and specialized data into a new type.

Both approaches are powerful problem-solving techniques. Often it may help to try to think about the problem from both points of view — *"What would happen if I tried to reduce everything to very few primitive types?"*, versus *"What would happen if this thing had its own specialized type?"*

22.7. Another example

The `after` function should compare two times, and tell us whether the first time is strictly after the second, e.g.

```
>>> t1 = MyTime(10, 55, 12)
>>> t2 = MyTime(10, 48, 22)
>>> after(t1, t2)             # Is t1 after t2?
True
```

This is slightly more complicated because it operates on two `MyTime` objects, not just one. But we'd prefer to write it as a method anyway — in this case, a method on the first argument:

```
class MyTime:
    # Previous method definitions here...

    def after(self, time2):
        """ Return True if I am strictly greater than time2 """
        if self.hours > time2.hours:
            return True
        if self.hours < time2.hours:
            return False

        if self.minutes > time2.minutes:
            return True
        if self.minutes < time2.minutes:
            return False
        if self.seconds > time2.seconds:
            return True

        return False
```

We invoke this method on one object and pass the other as an argument:

```
if current_time.after(done_time):
    print("The bread will be done before it starts!")
```

We can almost read the invocation like English: If the current time is after the done time, then...

The logic of the `if` statements deserve special attention here. Lines 11-18 will only be reached if the two hour fields are the same. Similarly, the test at line 16 is only executed if both times have the same hours and the same minutes.

Could we make this easier by using our "Aha!" insight and extra work from earlier, and reducing both times to integers? Yes, with spectacular results!

```
class MyTime:
    # Previous method definitions here...

    def after(self, time2):
        """ Return True if I am strictly greater than time2 """
        return self.to_seconds() > time2.to_seconds()
```

This is a great way to code this: if we want to tell if the first time is after the second time, turn them both into integers and compare the integers.

22.8. Operator overloading

Some languages, including Python, make it possible to have different meanings for the same operator when applied to different types. For example, + in Python means quite different things for integers and for strings. This feature is called **operator overloading**.

It is especially useful when programmers can also overload the operators for their own user-defined types.

For example, to override the addition operator +, we can provide a method named `__add__`:

```
class MyTime:
    # Previously defined methods here...

    def __add__(self, other):
        return MyTime(0, 0, self.to_seconds() + other.to_seconds())
```

As usual, the first parameter is the object on which the method is invoked. The second parameter is conveniently named `other` to distinguish it from `self`. To add two `MyTime` objects, we create and return a new `MyTime` object that contains their sum.

Now, when we apply the + operator to `MyTime` objects, Python invokes the `__add__` method that we have written:

```
>>> t1 = MyTime(1, 15, 42)
>>> t2 = MyTime(3, 50, 30)
>>> t3 = t1 + t2
>>> print(t3)
05:06:12
```

The expression `t1 + t2` is equivalent to `t1.__add__(t2)`, but obviously more elegant. As an exercise, add a method `__sub__(self, other)` that overloads the subtraction operator, and try it out.

For the next couple of exercises we'll go back to the `Point` class defined in our first chapter about objects, and overload some of its operators. Firstly, adding two points adds their respective (x, y) coordinates:

```
class Point:
    # Previously defined methods here...

    def __add__(self, other):
        return Point(self.x + other.x,  self.y + other.y)
```

There are several ways to override the behavior of the multiplication operator: by defining a method named `__mul__`, or `__rmul__`, or both.

If the left operand of `*` is a `Point`, Python invokes `__mul__`, which assumes that the other operand is also a `Point`. It computes the **dot product** of the two Points, defined according to the rules of linear algebra:

```
def __mul__(self, other):
    return self.x * other.x + self.y * other.y
```

If the left operand of `*` is a primitive type and the right operand is a `Point`, Python invokes `__rmul__`, which performs **scalar multiplication**:

```
def __rmul__(self, other):
    return Point(other * self.x,  other * self.y)
```

The result is a new `Point` whose coordinates are a multiple of the original coordinates. If `other` is a type that cannot be multiplied by a floating-point number, then `__rmul__` will yield an error.

This example demonstrates both kinds of multiplication:

```
>>> p1 = Point(3, 4)
>>> p2 = Point(5, 7)
>>> print(p1 * p2)
43
>>> print(2 * p2)
(10, 14)
```

What happens if we try to evaluate `p2 * 2`? Since the first parameter is a `Point`, Python invokes `__mul__` with `2` as the second argument. Inside `__mul__`, the program tries to access the `x` coordinate of `other`, which fails because an integer has no attributes:

```
>>> print(p2 * 2)
AttributeError: 'int' object has no attribute 'x'
```

Unfortunately, the error message is a bit opaque. This example demonstrates some of the difficulties of object-oriented programming. Sometimes it is hard enough just to figure out what code is running.

22.9. Polymorphism

Most of the methods we have written only work for a specific type. When we create a new object, we write methods that operate on that type.

But there are certain operations that we will want to apply to many types, such as the arithmetic operations in the previous sections. If many types support the same set of operations, we can write functions that work on any of those types.

For example, the `multadd` operation (which is common in linear algebra) takes three parameters; it multiplies the first two and then adds the third. We can write it in Python like this:

```
def multadd (x, y, z):
    return x * y + z
```

This function will work for any values of `x` and `y` that can be multiplied and for any value of `z` that can be added to the product.

We can invoke it with numeric values:

```
>>> multadd (3, 2, 1)
7
```

Or with `Points`:

```
>>> p1 = Point(3, 4)
>>> p2 = Point(5, 7)
>>> print(multadd (2, p1, p2))
(11, 15)
>>> print(multadd (p1, p2, 1))
44
```

In the first case, the `Point` is multiplied by a scalar and then added to another `Point`. In the second case, the dot product yields a numeric value, so the third parameter also has to be a numeric value.

A function like this that can take arguments with different types is called **polymorphic**.

As another example, consider the function `front_and_back`, which prints a list twice, forward and backward:

```
def front_and_back(front):
    import copy
    back = copy.copy(front)
    back.reverse()
    print(str(front) + str(back))
```

Because the `reverse` method is a modifier, we make a copy of the list before reversing it. That way, this function doesn't modify the list it gets as a parameter.

Here's an example that applies `front_and_back` to a list:

```
>>> my_list = [1, 2, 3, 4]
>>> front_and_back(my_list)
[1, 2, 3, 4][4, 3, 2, 1]
```

Of course, we intended to apply this function to lists, so it is not surprising that it works. What would be surprising is if we could apply it to a `Point`.

To determine whether a function can be applied to a new type, we apply Python's fundamental rule of polymorphism, called the **duck typing rule**: *If all of the operations inside the function can be applied to the type, the function can be applied to the type.* The operations in the `front_and_back` function include `copy`, `reverse`, and `print`.

Not all programming languages define polymorphism in this way. Look up *duck typing*, and see if you can figure out why it has this name.

`copy` works on any object, and we have already written a `__str__` method for `Point` objects, so all we need is a `reverse` method in the `Point` class:

```
def reverse(self):
    (self.x , self.y) = (self.y, self.x)
```

Then we can pass `Points` to `front_and_back`:

```
>>> p = Point(3, 4)
>>> front_and_back(p)
(3, 4)(4, 3)
```

The most interesting polymorphism is the unintentional kind, where we discover that a function we have already written can be applied to a type for which we never planned.

22.10. Glossary

```
dot product
    An operation defined in linear algebra that multiplies two `Point`s and
    yields a numeric value.

functional programming style
    A style of program design in which the majority of functions are pure.

modifier
    A function or method that changes one or more of the objects it receives
    as parameters. Most modifier functions are void (do not return a value).

normalized
    Data is said to be normalized if it fits into some reduced range or set
    of rules. We usually normalize our angles to values in the range
    \[0..360). We normalize minutes and seconds to be values in the range
    \[0..60). And we'd be surprised if the local store advertised its cold
    drinks at "One dollar, two hundred and fifty cents".

operator overloading
    Extending built-in operators ( `+`, `-`, `*`, `>`, `<`, etc.) so that
    they do different things for different types of arguments. We've seen
    early in the book how `+` is overloaded for numbers and strings, and
    here we've shown how to further overload it for user-defined types.

polymorphic
    A function that can operate on more than one type. Notice the subtle
    distinction: overloading has different functions (all with the same
    name) for different types, whereas a polymorphic function is a single
    function that can work for a range of types.

pure function
    A function that does not modify any of the objects it receives as
    parameters. Most pure functions are fruitful rather than void.

scalar multiplication
    An operation defined in linear algebra that multiplies each of the
    coordinates of a `Point` by a numeric value.
```

22.11. Exercises

1. Write a Boolean function `between` that takes two `MyTime` objects, `t1` and `t2`, as arguments, and

returns `True` if the invoking object falls between the two times. Assume `t1 <= t2`, and make the test closed at the lower bound and open at the upper bound, i.e. return True if `t1 <= obj < t2`.

2. Turn the above function into a method in the `MyTime` class.

3. Overload the necessary operator(s) so that instead of having to write :

if t1.after(t2): ...

we can use the more convenient :

if t1 > t2: ...

4. Rewrite `increment` as a method that uses our "Aha" insight.

5. Create some test cases for the `increment` method. Consider specifically the case where the number of

seconds to add to the time is negative. Fix up `increment` so that it handles this case if it does not do so already. (You may assume that you will never subtract more seconds than are in the time object.)

6. Can physical time be negative, or must time always move in the forward direction? Some serious physicists

think this is not such a dumb question. See what you can find on the Internet about this.

Chapter 23: Collections of objects

23.1. Composition

By now, we have seen several examples of composition. One of the first examples was using a method invocation as part of an expression. Another example is the nested structure of statements; we can put an `if` statement within a `while` loop, within another `if` statement, and so on.

Having seen this pattern, and having learned about lists and objects, we should not be surprised to learn that we can create lists of objects. We can also create objects that contain lists (as attributes); we can create lists that contain lists; we can create objects that contain objects; and so on.

In this chapter and the next, we will look at some examples of these combinations, using `Card` objects as an example.

23.2. `Card` objects

If you are not familiar with common playing cards, now would be a good time to get a deck, or else this chapter might not make much sense. There are fifty-two cards in a deck, each of which belongs to one of four suits and one of thirteen ranks. The suits are Spades, Hearts, Diamonds, and Clubs (in descending order in bridge). The ranks are Ace, 2, 3, 4, 5, 6, 7, 8, 9, 10, Jack, Queen, and King. Depending on the game that we are playing, the rank of Ace may be higher than King or lower than 2. The rank is sometimes called the face-value of the card.

If we want to define a new object to represent a playing card, it is obvious what the attributes should be: `rank` and `suit`. It is not as obvious what type the attributes should be. One possibility is to use strings containing words like `"Spade"` for suits and `"Queen"` for ranks. One problem with this implementation is that it would not be easy to compare cards to see which had a higher rank or suit.

An alternative is to use integers to **encode** the ranks and suits. By encode, we do not mean what some people think, which is to encrypt or translate into a secret code. What a computer scientist means by encode
is to define a mapping between a sequence of numbers and the items I want to represent. For example:

```
Spades   -->  3
Hearts   -->  2
Diamonds -->  1
Clubs    -->  0
```

An obvious feature of this mapping is that the suits map to integers in order, so we can compare suits by comparing integers. The mapping for ranks is fairly obvious; each of the numerical ranks maps to the corresponding integer, and for face cards:

```
Jack   -->  11
Queen  -->  12
King   -->  13
```

The reason we are using mathematical notation for these mappings is that they are not part of the Python program. They are part of the program design, but they never appear explicitly in the code. The class definition for the `Card` type looks like this:

```
class Card:
    def __init__(self, suit=0, rank=0):
        self.suit = suit
        self.rank = rank
```

As usual, we provide an initialization method that takes an optional parameter for each attribute.

To create some objects, representing say the 3 of Clubs and the Jack of Diamonds, use these commands:

```
three_of_clubs = Card(0, 3)
card1 = Card(1, 11)
```

In the first case above, for example, the first argument, `0`, represents the suit Clubs.

Save this code for later use ...

In the next chapter we assume that we have save the `Cards` class, and the upcoming `Deck` class in a file called `Cards.py`.

23.3. Class attributes and the `__str__` method

In order to print `Card` objects in a way that people can easily read, we want to map the integer codes onto words. A natural way to do that is with lists of strings. We assign these lists to **class attributes** at the top of the class definition:

```
class Card:
    suits = ["Clubs", "Diamonds", "Hearts", "Spades"]
    ranks = ["narf", "Ace", "2", "3", "4", "5", "6", "7",
             "8", "9", "10", "Jack", "Queen", "King"]

    def __init__(self, suit=0, rank=0):
        self.suit = suit
        self.rank = rank

    def __str__(self):
        return (self.ranks[self.rank] + " of " + self.suits[self.suit])
```

A class attribute is defined outside of any method, and it can be accessed from any of the methods in the class.

Inside `__str__`, we can use `suits` and `ranks` to map the numerical values of `suit` and `rank` to strings. For example, the expression `self.suits[self.suit]` means use the attribute `suit` from the object `self` as an index into the class attribute named `suits`, and select the appropriate string.

The reason for the `"narf"` in the first element in `ranks` is to act as a place keeper for the zero-eth element of the list, which will never be used. The only valid ranks are 1 to 13. This wasted item is not entirely necessary. We could have started at 0, as usual, but it is less confusing to encode the rank 2 as integer 2, 3 as 3, and so on.

With the methods we have so far, we can create and print cards:

```
>>> card1 = Card(1, 11)
>>> print(card1)
Jack of Diamonds
```

Class attributes like `suits` are shared by all `Card` objects. The advantage of this is that we can use any `Card` object to access the class attributes:

```
>>> card2 = Card(1, 3)
>>> print(card2)
3 of Diamonds
>>> print(card2.suits[1])
Diamonds
```

Because every `Card` instance references the same class attribute, we have an aliasing situation. The disadvantage is that if we modify a class attribute, it affects every instance of the class. For example, if we decide that Jack of Diamonds should really be called Jack of Swirly Whales, we could do this:

```
>>> card1.suits[1] = "Swirly Whales"
>>> print(card1)
Jack of Swirly Whales
```

The problem is that *all* of the Diamonds just became Swirly Whales:

```
>>> print(card2)
3 of Swirly Whales
```

It is usually not a good idea to modify class attributes.

23.4. Comparing cards

For primitive types, there are six relational operators (`<`, `>`, `==`, etc.) that compare values and determine when one is greater than, less than, or equal to another. If we want our own types to be comparable using the syntax of these relational operators, we need to define six corresponding special methods in our class.

We'd like to start with a single method named `cmp` that houses the logic of ordering. By convention, a comparison method takes two parameters, `self` and `other`, and returns `1` if the first object is greater, `-1` if the second object is greater, and `0` if they are equal to each other.

Some types are completely ordered, which means that we can compare any two elements and tell which is bigger. For example, the integers and the floating-point numbers are completely ordered. Some types are unordered, which means that there is no meaningful way to say that one element is bigger than another. For example, the fruits are unordered, which is why we cannot compare apples and oranges, and we cannot meaningfully order a
collection of images, or a collection of cellphones.

Playing cards are partially ordered, which means that sometimes we can compare cards and sometimes not. For example, we know that the `3` of Clubs is higher than the `2` of Clubs, and the `3` of Diamonds is higher than the `3` of Clubs. But which is better, the `3` of Clubs or the `2` of Diamonds? One has a higher rank, but the other has a higher suit.

In order to make cards comparable, we have to decide which is more important, rank or suit. To be honest, the choice is arbitrary. For the sake of choosing, we will say that suit is more important, because a new deck of cards comes sorted with all the Clubs together, followed by all the Diamonds, and so on.

With that decided, we can write `cmp`:

```
def cmp(self, other):
    # Check the suits
    if self.suit > other.suit: return 1
    if self.suit < other.suit: return -1
    # Suits are the same... check ranks
    if self.rank > other.rank: return 1
    if self.rank < other.rank: return -1
    # Ranks are the same... it's a tie
    return 0
```

In this ordering, Aces appear lower than Deuces (2s).

Now, we can define the six special methods that do the overloading of each of the relational operators for us:

```
def __eq__(self, other):
    return self.cmp(other) == 0

def __le__(self, other):
    return self.cmp(other) <= 0

def __ge__(self, other):
    return self.cmp(other) >= 0

def __gt__(self, other):
    return self.cmp(other) > 0

def __lt__(self, other):
    return self.cmp(other) < 0

def __ne__(self, other):
    return self.cmp(other) != 0
```

With this machinery in place, the relational operators now work as we'd like them to:

```
>>> card1 = Card(1, 11)
>>> card2 = Card(1, 3)
>>> card3 = Card(1, 11)
>>> card1 < card2
False
>>> card1 == card3
True
```

23.5. Decks

Now that we have objects to represent `Card`s, the next logical step is to define a class to represent a `Deck`. Of course, a deck is made up of cards, so each `Deck` object will contain a list of cards as an attribute. Many card games will need at least two different decks — a red deck and a blue deck.

The following is a class definition for the `Deck` class. The initialization method creates the attribute `cards` and generates the standard pack of fifty-two cards:

```
class Deck:
    def __init__(self):
        self.cards = []
        for suit in range(4):
            for rank in range(1, 14):
                self.cards.append(Card(suit, rank))
```

The easiest way to populate the deck is with a nested loop. The outer loop enumerates the suits from 0 to 3. The inner loop enumerates the ranks from 1 to 13. Since the outer loop iterates four times, and the inner loop iterates thirteen times, the total number of times the body is executed is fifty-two (thirteen times four). Each iteration creates a new instance of `Card` with the current suit and rank, and appends that card to the `cards` list.

With this in place, we can instantiate some decks:

```
red_deck = Deck()
blue_deck = Deck()
```

23.6. Printing the deck

As usual, when we define a new type we want a method that prints the contents of an instance. To print a `Deck`, we traverse the list and print each `Card`:

```
class Deck:
    ...
    def print_deck(self):
        for card in self.cards:
            print(card)
```

Here, and from now on, the ellipsis (. . .) indicates that we have omitted the other methods in the class.

As an alternative to `print_deck`, we could write a `__str__` method for the `Deck` class. The advantage of `__str__` is that it is more flexible. Rather than just printing the contents of the object, it generates

a string representation that other parts of the program can manipulate before printing, or store for later use.

Here is a version of `__str__` that returns a string representation of a `Deck`. To add a bit of pizzazz, it arranges the cards in a cascade where each card is indented one space more than the previous card:

```
class Deck:
    ...
    def __str__(self):
        s = ""
        for i in range(len(self.cards)):
            s = s + " " * i + str(self.cards[i]) + "\n"
        return s
```

This example demonstrates several features. First, instead of traversing `self.cards` and assigning each card to a variable, we are using `i` as a loop variable and an index into the list of cards.

Second, we are using the string multiplication operator to indent each card by one more space than the last. The expression `" " * i` yields a number of spaces equal to the current value of `i`.

Third, instead of using the `print` command to print the cards, we use the `str` function. Passing an object as an argument to `str` is equivalent to invoking the `__str__` method on the object.

Finally, we are using the variable `s` as an **accumulator**. Initially, `s` is the empty string. Each time through the loop, a new string is generated and concatenated with the old value of `s` to get the new value. When the loop ends, `s` contains the complete string representation of the `Deck`, which looks like this:

```
>>> red_deck = Deck()
>>> print(red_deck)
Ace of Clubs
 2 of Clubs
  3 of Clubs
   4 of Clubs
    5 of Clubs
     6 of Clubs
      7 of Clubs
       8 of Clubs
        9 of Clubs
         10 of Clubs
          Jack of Clubs
           Queen of Clubs
            King of Clubs
             Ace of Diamonds
              2 of Diamonds
                ...
```

And so on. Even though the result appears on 52 lines, it is one long string that contains newlines.

23.7. Shuffling the deck

If a deck is perfectly shuffled, then any card is equally likely to appear anywhere in the deck, and any location in the deck is equally likely to contain any card.

To shuffle the deck, we will use the `randrange` function from the `random` module. With two integer arguments, `a` and `b`, `randrange` chooses a random integer in the range `a <= x < b`. Since the upper bound is strictly less than `b`, we can use the length of a list as the second parameter, and we are guaranteed to get a legal index. For example, if `rng` has already been instantiated as a random number source, this expression chooses the index of a random card in a deck:

```
rng.randrange(0, len(self.cards))
```

An easy way to shuffle the deck is by traversing the cards and swapping each card with a randomly chosen one. It is possible that the card will be swapped with itself, but that is fine. In fact, if we precluded that possibility, the order of the cards would be less than entirely random:

```
class Deck:
    ...
    def shuffle(self):
        import random
        rng = random.Random()        # Create a random generator
        num_cards = len(self.cards)
        for i in range(num_cards):
            j = rng.randrange(i, num_cards)
            (self.cards[i], self.cards[j]) = (self.cards[j], self.cards[i])
```

Rather than assume that there are fifty-two cards in the deck, we get the actual length of the list and store it in `num_cards`.

For each card in the deck, we choose a random card from among the cards that haven't been shuffled yet. Then we swap the current card (`i`) with the selected card (`j`). To swap the cards we use a tuple assignment:

```
(self.cards[i], self.cards[j]) = (self.cards[j], self.cards[i])
```

While this is a good shuffling method, a random number generator object also has a `shuffle` method that can shuffle elements in a list, in place. So we could rewrite this function to use the one provided for us:

```
class Deck:
    ...
    def shuffle(self):
        import random
        rng = random.Random()        # Create a random generator
        rng.shuffle(self.cards)      # uUse its shuffle method
```

23.8. Removing and dealing cards

Another method that would be useful for the `Deck` class is `remove`, which takes a card as a parameter, removes it, and returns `True` if the card was in the deck and `False` otherwise:

```
class Deck:
    ...
    def remove(self, card):
        if card in self.cards:
            self.cards.remove(card)
            return True
        else:
            return False
```

The `in` operator returns `True` if the first operand is in the second. If the first operand is an object, Python uses the object's `__eq__` method to determine equality with items in the list. Since the `__eq__` we provided in the `Card` class checks for deep equality, the `remove` method checks for deep equality.

To deal cards, we want to remove and return the top card. The list method `pop` provides a convenient way to do that:

```
class Deck:
    ...
    def pop(self):
        return self.cards.pop()
```

Actually, `pop` removes the *last* card in the list, so we are in effect dealing from the bottom of the deck.

One more operation that we are likely to want is the Boolean function `is_empty`, which returns `True` if the deck contains no cards:

```
class Deck:
    ...
    def is_empty(self):
        return self.cards == []
```

23.9. Glossary

encode
: To represent one type of value using another type of value by constructing a mapping between them.

class attribute
: A variable that is defined inside a class definition but outside any method. Class attributes are accessible from any method in the class and are shared by all instances of the class.

accumulator
: A variable used in a loop to accumulate a series of values, such as by concatenating them onto a string or adding them to a running sum.

23.10. Exercises

1. Modify `cmp` so that Aces are ranked higher than Kings.

Chapter 24: Inheritance

24.1. Inheritance

The language feature most often associated with object-oriented programming is **inheritance**. Inheritance is the ability to define a new class that is a modified version of an existing class.

The primary advantage of this feature is that you can add new methods to a class without modifying the existing class. It is called inheritance because the new class inherits all of the methods of the existing class. Extending this metaphor, the existing class is sometimes called the **parent** class. The new class may be called the **child** class or sometimes subclass.

Inheritance is a powerful feature. Some programs that would be complicated without inheritance can be written concisely and simply with it. Also, inheritance can facilitate code reuse, since you can customize the behavior of parent classes without having to modify them. In some cases, the inheritance structure reflects the natural structure of the problem, which makes the program easier to understand.

On the other hand, inheritance can make programs difficult to read. When a method is invoked, it is sometimes not clear where to find its definition. The relevant code may be scattered among several modules. Also, many of the things that can be done using inheritance can be done as elegantly (or more so) without it. If the natural structure of the problem does not lend itself to inheritance, this style of programming can do more harm than good.

In this chapter we will demonstrate the use of inheritance as part of a program that plays the card game Old Maid. One of our goals is to write code that could be reused to implement other card games.

24.2. A hand of cards

For almost any card game, we need to represent a hand of cards. A hand is similar to a deck, of course. Both are made up of a set of cards, and both require operations like adding and removing cards. Also, we might like the ability to shuffle both decks and hands.

A hand is also different from a deck. Depending on the game being played, we might want to perform some operations on hands that don't make sense for a deck. For example, in poker we might classify a hand(straight, flush, etc.) or compare it with another hand. In bridge, we might want to compute a score for a hand in order to make a bid.

This situation suggests the use of inheritance. If `Hand` is a subclass of `Deck`, it will have all the methods of `Deck`, and new methods can be added.

We add the code in this chapter to our `Cards.py` file from the previous chapter. In the class definition, the name of the parent class appears in parentheses:

```
class Hand(Deck):
    pass
```

This statement indicates that the new `Hand` class inherits from the existing `Deck` class.

The `Hand` constructor initializes the attributes for the hand, which are `name` and `cards`. The string `name` identifies this hand, probably by the name of the player that holds it. The name is an optional parameter with the empty string as a default value. `cards` is the list of cards in the hand, initialized to the empty list:

```
class Hand(Deck):
    def __init__(self, name=""):
        self.cards = []
        self.name = name
```

For just about any card game, it is necessary to add and remove cards from the deck. Removing cards is already taken care of, since `Hand` inherits `remove` from `Deck`. But we have to write `add`:

```
class Hand(Deck):
    ...
    def add(self, card):
        self.cards.append(card)
```

Again, the ellipsis indicates that we have omitted other methods. The list `append` method adds the new card to the end of the list of cards.

24.3. Dealing cards

Now that we have a `Hand` class, we want to deal cards from the `Deck` into hands. It is not immediately obvious whether this method should go in the `Hand` class or in the `Deck` class, but since it operates on a
single deck and (possibly) several hands, it is more natural to put it in `Deck`.

`deal` should be fairly general, since different games will have different requirements. We may want to deal out the entire deck at once or add one card to each hand.

`deal` takes two parameters, a list (or tuple) of hands and the total number of cards to deal. If there are not enough cards in the deck, the method deals out all of the cards and stops:

```
class Deck:
    ...
    def deal(self, hands, num_cards=999):
        num_hands = len(hands)
        for i in range(num_cards):
            if self.is_empty():
                break                   # Break if out of cards
            card = self.pop()           # Take the top card
            hand = hands[i % num_hands] # Whose turn is next?
            hand.add(card)              # Add the card to the hand
```

The second parameter, `num_cards`, is optional; the default is a large number, which effectively means that all of the cards in the deck will get dealt.

The loop variable `i` goes from 0 to `num_cards-1`. Each time through the loop, a card is removed from the deck using the list method `pop`, which removes and returns the last item in the list.

The modulus operator (`%`) allows us to deal cards in a round robin (one card at a time to each hand). When `i` is equal to the number of hands in the list, the expression `i % num_hands` wraps around to the beginning of the list (index 0).

24.4. Printing a Hand

To print the contents of a hand, we can take advantage of the `__str__` method inherited from `Deck`. For example:

```
>>> deck = Deck()
>>> deck.shuffle()
>>> hand = Hand("frank")
>>> deck.deal([hand], 5)
>>> print(hand)
Hand frank contains
2 of Spades
 3 of Spades
  4 of Spades
   Ace of Hearts
    9 of Clubs
```

It's not a great hand, but it has the makings of a straight flush.

Although it is convenient to inherit the existing methods, there is additional information in a `Hand` object we might want to include when we print one. To do that, we can provide a `__str__` method in the `Hand` class that overrides the one in the `Deck` class:

```
class Hand(Deck)
    def __str__(self):
        s = "Hand " + self.name
        if self.is_empty():
            s += " is empty\n"
        else:
            s += " contains\n"
        return s + Deck.__str__(self)
```

Initially, `s` is a string that identifies the hand. If the hand is empty, the program appends the words `is empty` and returns `s`.

Otherwise, the program appends the word `contains` and the string representation of the `Deck`, computed by invoking the `__str__` method in the `Deck` class on `self`.

It may seem odd to send `self`, which refers to the current `Hand`, to a `Deck` method, until you remember that a `Hand` is a kind of `Deck`. `Hand` objects can do everything `Deck` objects can, so it is legal to send a `Hand` to a `Deck` method.

In general, it is always legal to use an instance of a subclass in place of an instance of a parent class.

24.5. The `CardGame` class

The `CardGame` class takes care of some basic chores common to all games, such as creating the deck and shuffling it:

```
class CardGame:
    def __init__(self):
        self.deck = Deck()
        self.deck.shuffle()
```

This is the first case we have seen where the initialization method performs a significant computation, beyond initializing attributes.

To implement specific games, we can inherit from `CardGame` and add features for the new game. As an example, we'll write a simulation of Old Maid.

The object of Old Maid is to get rid of cards in your hand. You do this by matching cards by rank and color. For example, the 4 of Clubs matches the 4 of Spades since both suits are black. The Jack of Hearts matches the Jack of Diamonds since both are red.

To begin the game, the Queen of Clubs is removed from the deck so that the Queen of Spades has no match. The fifty-one remaining cards are dealt to the players in a round robin. After the deal, all players match and discard as many cards as possible.

When no more matches can be made, play begins. In turn, each player picks a card (without looking) from the closest neighbor to the left who still has cards. If the chosen card matches a card in the player's hand, the pair is removed. Otherwise, the card is added to the player's hand. Eventually all possible matches are made, leaving only the Queen of Spades in the loser's hand.

In our computer simulation of the game, the computer plays all hands. Unfortunately, some nuances of the real game are lost. In a real game, the player with the Old Maid goes to some effort to get their neighbor to pick that card, by displaying it a little more prominently, or perhaps failing to display it more prominently, or even failing to fail to display that card more prominently. The computer simply picks a neighbor's card at random.

24.6. `OldMaidHand` class

A hand for playing Old Maid requires some abilities beyond the general abilities of a `Hand`. We will define a new class, `OldMaidHand`, that inherits from `Hand` and provides an additional method called `remove_matches`:

```
class OldMaidHand(Hand):
    def remove_matches(self):
        count = 0
        original_cards = self.cards[:]
        for card in original_cards:
            match = Card(3 - card.suit, card.rank)
            if match in self.cards:
                self.cards.remove(card)
                self.cards.remove(match)
                print("Hand {0}: {1} matches {2}"
                        .format(self.name, card, match))
                count += 1
        return count
```

We start by making a copy of the list of cards, so that we can traverse the copy while removing cards from the original. Since `self.cards` is modified in the loop, we don't want to use it to control the traversal. Python can get quite confused if it is traversing a list that is changing!

For each card in the hand, we figure out what the matching card is and go looking for it. The match card has the same rank and the other suit of the same color. The expression `3 - card.suit` turns a Club (suit 0) into a Spade (suit 3) and a Diamond (suit 1) into a Heart (suit 2). You should satisfy yourself that the opposite operations also work. If the match card is also in the hand, both cards are removed.

The following example demonstrates how to use `remove_matches`:

```
>>> game = CardGame()
>>> hand = OldMaidHand("frank")
>>> game.deck.deal([hand], 13)
>>> print(hand)
Hand frank contains
Ace of Spades
 2 of Diamonds
  7 of Spades
   8 of Clubs
    6 of Hearts
     8 of Spades
      7 of Clubs
       Queen of Clubs
        7 of Diamonds
         5 of Clubs
          Jack of Diamonds
           10 of Diamonds
            10 of Hearts
>>> hand.remove_matches()
Hand frank: 7 of Spades matches 7 of Clubs
Hand frank: 8 of Spades matches 8 of Clubs
Hand frank: 10 of Diamonds matches 10 of Hearts
>>> print(hand)
Hand frank contains
Ace of Spades
 2 of Diamonds
  6 of Hearts
   Queen of Clubs
    7 of Diamonds
     5 of Clubs
      Jack of Diamonds
```

Notice that there is no `__init__` method for the `OldMaidHand` class. We inherit it from `Hand`.

24.7. `OldMaidGame` class

Now we can turn our attention to the game itself. `OldMaidGame` is a subclass of `CardGame` with a new method called `play` that takes a list of players as a parameter.

Since `__init__` is inherited from `CardGame`, a new `OldMaidGame` object contains a new shuffled deck:

```
class OldMaidGame(CardGame):
    def play(self, names):
        # Remove Queen of Clubs
        self.deck.remove(Card(0,12))

        # Make a hand for each player
        self.hands = []
        for name in names:
            self.hands.append(OldMaidHand(name))

        # Deal the cards
        self.deck.deal(self.hands)
        print("---------- Cards have been dealt")
        self.print_hands()

        # Remove initial matches
        matches = self.remove_all_matches()
        print("---------- Matches discarded, play begins")
        self.print_hands()

        # Play until all 50 cards are matched
        turn = 0
        num_hands = len(self.hands)
        while matches < 25:
            matches += self.play_one_turn(turn)
            turn = (turn + 1) % num_hands

        print("---------- Game is Over")
        self.print_hands()
```

The writing of `print_hands` has been left as an exercise.

Some of the steps of the game have been separated into methods. `remove_all_matches` traverses the list of hands and invokes `remove_matches` on each:

```
class OldMaidGame(CardGame):
    ...
    def remove_all_matches(self):
        count = 0
        for hand in self.hands:
            count += hand.remove_matches()
        return count
```

count is an accumulator that adds up the number of matches in each hand. When we've gone through every hand, the total is returned (count).

When the total number of matches reaches twenty-five, fifty cards have been removed from the hands, which means that only one card is left and the game is over.

The variable turn keeps track of which player's turn it is. It starts at 0 and increases by one each time; when it reaches num_hands, the modulus operator wraps it back around to 0.

The method play_one_turn takes a parameter that indicates whose turn it is. The return value is the number of matches made during this turn:

```
class OldMaidGame(CardGame):
    ...
    def play_one_turn(self, i):
        if self.hands[i].is_empty():
            return 0
        neighbor = self.find_neighbor(i)
        picked_card = self.hands[neighbor].pop()
        self.hands[i].add(picked_card)
        print("Hand", self.hands[i].name, "picked", picked_card)
        count = self.hands[i].remove_matches()
        self.hands[i].shuffle()
        return count
```

If a player's hand is empty, that player is out of the game, so he or she does nothing and returns 0.

Otherwise, a turn consists of finding the first player on the left that has cards, taking one card from the neighbor, and checking for matches. Before returning, the cards in the hand are shuffled so that the next player's choice is random.

The method find_neighbor starts with the player to the immediate left and continues around the circle until it finds a player that still has cards:

```
class OldMaidGame(CardGame):
    ...
    def find_neighbor(self, i):
        num_hands = len(self.hands)
        for next in range(1,num_hands):
            neighbor = (i + next) % num_hands
            if not self.hands[neighbor].is_empty():
                return neighbor
```

If find_neighbor ever went all the way around the circle without finding cards, it would return None and cause an error elsewhere in the program. Fortunately, we can prove that that will never happen (as long as the end of the game is detected correctly).

We have omitted the `print_hands` method. You can write that one yourself.

The following output is from a truncated form of the game where only the top fifteen cards (tens and higher) were dealt to three players. With this small deck, play stops after seven matches instead of twenty-five.

```
>>> import cards
>>> game = cards.OldMaidGame()
>>> game.play(["Allen","Jeff","Chris"])
---------- Cards have been dealt
Hand Allen contains
King of Hearts
 Jack of Clubs
  Queen of Spades
   King of Spades
    10 of Diamonds

Hand Jeff contains
Queen of Hearts
 Jack of Spades
  Jack of Hearts
   King of Diamonds
    Queen of Diamonds

Hand Chris contains
Jack of Diamonds
 King of Clubs
  10 of Spades
   10 of Hearts
    10 of Clubs

Hand Jeff: Queen of Hearts matches Queen of Diamonds
Hand Chris: 10 of Spades matches 10 of Clubs
---------- Matches discarded, play begins
Hand Allen contains
King of Hearts
 Jack of Clubs
  Queen of Spades
   King of Spades
    10 of Diamonds

Hand Jeff contains
Jack of Spades
```

```
 Jack of Hearts
  King of Diamonds

Hand Chris contains
Jack of Diamonds
 King of Clubs
  10 of Hearts

Hand Allen picked King of Diamonds
Hand Allen: King of Hearts matches King of Diamonds
Hand Jeff picked 10 of Hearts
Hand Chris picked Jack of Clubs
Hand Allen picked Jack of Hearts
Hand Jeff picked Jack of Diamonds
Hand Chris picked Queen of Spades
Hand Allen picked Jack of Diamonds
Hand Allen: Jack of Hearts matches Jack of Diamonds
Hand Jeff picked King of Clubs
Hand Chris picked King of Spades
Hand Allen picked 10 of Hearts
Hand Allen: 10 of Diamonds matches 10 of Hearts
Hand Jeff picked Queen of Spades
Hand Chris picked Jack of Spades
Hand Chris: Jack of Clubs matches Jack of Spades
Hand Jeff picked King of Spades
Hand Jeff: King of Clubs matches King of Spades
---------- Game is Over
Hand Allen is empty

Hand Jeff contains
Queen of Spades

Hand Chris is empty
```

So Jeff loses.

24.8. Glossary

```
inheritance
    The ability to define a new class that is a modified version of a
    previously defined class.

parent class
    The class from which a child class inherits.

child class
    A new class created by inheriting from an existing class; also called a
    subclass.
```

24.9. Exercises

1. Add a method, `print_hands`, to the `OldMaidGame` class which traverses `self.hands` and prints each hand.
2. Define a new kind of Turtle, `TurtleGTX`, that comes with some extra features: it can jump forward a

given distance, and it has an odometer that keeps track of how far the turtle has travelled since it came off the production line. (The parent class has a number of synonyms like `fd`, `forward`, `back`, `backward`, and `bk`: for this exercise, just focus on putting this functionality into the `forward` method.) Think carefully about how to count the distance if the turtle is asked to move forward by a negative amount. (We would not want to buy a second-hand turtle whose odometer reading was faked because its previous owner drove it backwards around the block too often. Try this in a car near you, and see if the car's odometer counts up or down when you reverse.)
3. After travelling some random distance, your turtle should break down with a flat tyre. After this

happens, raise an exception whenever `forward` is called. Also provide a `change_tyre` method that can fix the flat.

Chapter 25: Linked lists

25.1. Embedded references

We have seen examples of attributes that refer to other objects, which we called **embedded references**. A common data structure, the **linked list**, takes advantage of this feature.

Linked lists are made up of **nodes**, where each node contains a reference to the next node in the list. In addition, each node contains a unit of data called the **cargo**.

A linked list is considered a **recursive data structure** because it has a recursive definition.

A linked list is either:

1. the empty list, represented by `None`, or
2. a node that contains a cargo object and a reference to a linked list.

Recursive data structures lend themselves to recursive methods.

25.2. The `Node` class

As usual when writing a new class, we'll start with the initialization and `__str__` methods so that we can test the basic mechanism of creating and displaying the new type:

```
class Node:
    def __init__(self, cargo=None, next=None):
        self.cargo = cargo
        self.next  = next

    def __str__(self):
        return str(self.cargo)
```

As usual, the parameters for the initialization method are optional. By default, both the cargo and the link, `next`, are set to `None`.

The string representation of a node is just the string representation of the cargo. Since any value can be passed to the `str` function, we can store any value in a list.

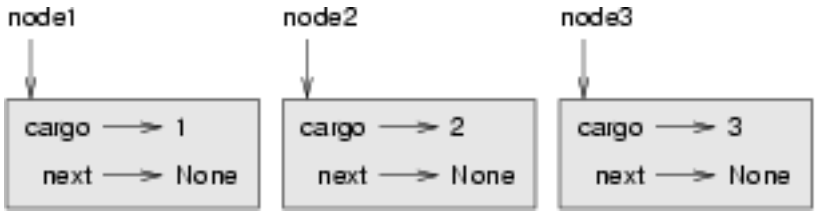
To test the implementation so far, we can create a `Node` and print it:

```
>>> node = Node("test")
>>> print(node)
test
```

To make it interesting, we need a list with more than one node:

```
>>> node1 = Node(1)
>>> node2 = Node(2)
>>> node3 = Node(3)
```

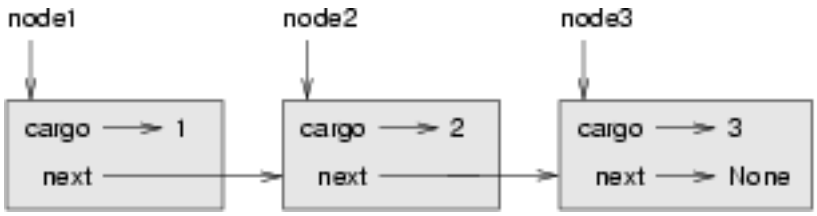
This code creates three nodes, but we don't have a list yet because the nodes are not **linked**. The state diagram looks like this:



To link the nodes, we have to make the first node refer to the second and the second node refer to the third:

```
>>> node1.next = node2
>>> node2.next = node3
```

The reference of the third node is `None`, which indicates that it is the end of the list. Now the state diagram looks like this:



Now you know how to create nodes and link them into lists. What might be less clear at this point is why.

25.3. Lists as collections

Lists are useful because they provide a way to assemble multiple objects into a single entity, sometimes called a **collection**. In the example, the first node of the list serves as a reference to the entire list.

To pass the list as a parameter, we only have to pass a reference to the first node. For example, the function `print_list` takes a single node as an argument. Starting with the head of the list, it prints each node until it gets to the end:

```
def print_list(node):
    while node is not None:
        print(node, end=" ")
        node = node.next
    print()
```

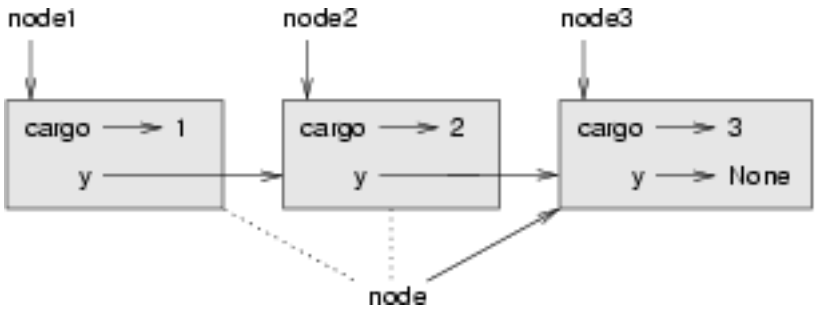
To invoke this method, we pass a reference to the first node:

```
>>> print_list(node1)
1 2 3
```

Inside `print_list` we have a reference to the first node of the list, but there is no variable that refers to the other nodes. We have to use the `next` value from each node to get to the next node.

To traverse a linked list, it is common to use a loop variable like `node` to refer to each of the nodes in succession.

This diagram shows the value of `list` and the values that `node` takes on:



25.4. Lists and recursion

It is natural to express many list operations using recursive methods. For example, the following is a recursive algorithm for printing a list backwards:

1. Separate the list into two pieces: the first node (called the head); and the rest (called the tail).
2. Print the tail backward.
3. Print the head.

Of course, Step 2, the recursive call, assumes that we have a way of printing a list backward. But if we assume that the recursive call works — the leap of faith — then we can convince ourselves that this algorithm works.

All we need are a base case and a way of proving that for any list, we will eventually get to the base case. Given the recursive definition of a list, a natural base case is the empty list, represented by None:

```
def print_backward(list):
    if list is None: return
    head = list
    tail = list.next
    print_backward(tail)
    print(head, end=" ")
```

The first line handles the base case by doing nothing. The next two lines split the list into head and tail. The last two lines print the list. The end argument of the print statement keeps Python from printing a newline after each node.

We invoke this method as we invoked print_list:

```
>>> print_backward(node1)
3 2 1
```

The result is a backward list.

You might wonder why print_list and print_backward are functions and not methods in the Node class. The reason is that we want to use None to represent the empty list and it is not legal to invoke a method on None. This limitation makes it awkward to write list – manipulating code in a clean object-oriented style.

Can we prove that print_backward will always terminate? In other words, will it always reach the base case? In fact, the answer is no. Some lists will make this method crash.

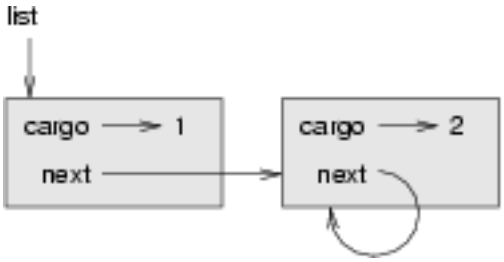
Revisit the Recursion chapter

In our earlier chapter on recursion we distinguished between the high-level view that requires a leap of faith, and the low-level operational view. In terms of mental chunking, we want to encourage the more abstract high-level view.

But if you'd like to see the detail you should use your single-stepping debugging tools to step into the recursive levels and to examine the execution stack frames at every call to print_backward.

25.5. Infinite lists

There is nothing to prevent a node from referring back to an earlier node in the list, including itself. For example, this figure shows a list with two nodes, one of which refers to itself:



If we invoke `print_list` on this list, it will loop forever. If we invoke `print_backward`, it will recurse infinitely. This sort of behavior makes infinite lists difficult to work with.

Nevertheless, they are occasionally useful. For example, we might represent a number as a list of digits and use an infinite list to represent a repeating fraction.

Regardless, it is problematic that we cannot prove that `print_list` and `print_backward` terminate. The best we can do is the hypothetical statement, "If the list contains no loops, then these methods will terminate." This sort of claim is called a **precondition**. It imposes a constraint on one of the parameters and describes the behavior of the method if the constraint is satisfied. You will see more examples soon.

25.6. The fundamental ambiguity theorem

One part of `print_backward` might have raised an eyebrow:

```
1  head = list
2  tail = list.next
```

After the first assignment, `head` and `list` have the same type and the same value. So why did we create a new variable?

The reason is that the two variables play different roles. We think of `head` as a reference to a single node, and we think of `list` as a reference to the first node of a list. These roles are not part of the program; they are in the mind of the programmer.

In general we can't tell by looking at a program what role a variable plays. This ambiguity can be useful, but it can also make programs difficult to read. We often use variable names like `node` and `list` to document how we intend to use a variable and sometimes create additional variables to disambiguate.

We could have written `print_backward` without `head` and `tail`, which makes it more concise but possibly less clear:

```
def print_backward(list):
    if list is None: return
    print_backward(list.next)
    print(list, end=" ")
```

Looking at the two function calls, we have to remember that `print_backward` treats its argument as a collection and `print` treats its argument as a single object.

The **fundamental ambiguity theorem** describes the ambiguity that is inherent in a reference to a node: *A variable that refers to a node might treat the node as a single object or as the first in a list of nodes.*

25.7. Modifying lists

There are two ways to modify a linked list. Obviously, we can change the cargo of one of the nodes, but the more interesting operations are the ones that add, remove, or reorder the nodes.

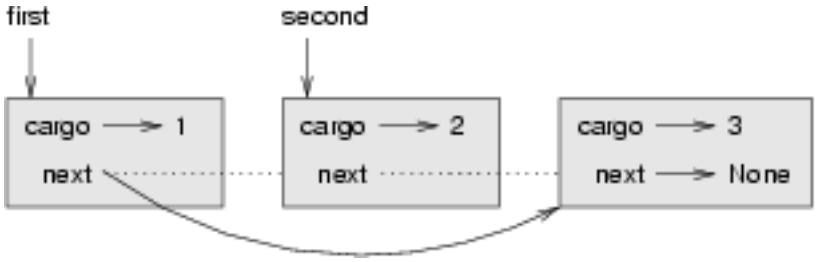
As an example, let's write a method that removes the second node in the list and returns a reference to the removed node:

```
def remove_second(list):
    if list is None: return
    first = list
    second = list.next
    # Make the first node refer to the third
    first.next = second.next
    # Separate the second node from the rest of the list
    second.next = None
    return second
```

Again, we are using temporary variables to make the code more readable. Here is how to use this method:

```
>>> print_list(node1)
1 2 3
>>> removed = remove_second(node1)
>>> print_list(removed)
2
>>> print_list(node1)
1 3
```

This state diagram shows the effect of the operation:



What happens if you invoke this method and pass a list with only oneelement (a **singleton**)? What happens if you pass the empty list as an argument? Is there a precondition for this method? If so, fix the method to handle a violation of the precondition in a reasonable way.

25.8. Wrappers and helpers

It is often useful to divide a list operation into two methods. For example, to print a list backward in the conventional list format `[3, 2, 1]` we can use the `print_backward` method to print `3, 2,` but we need a separate method to print the brackets and the first node. Let's call it `print_backward_nicely`:

```
def print_backward_nicely(list):
    print("[", end=" ")
    print_backward(list)
    print("]")
```

Again, it is a good idea to check methods like this to see if they work with special cases like an empty list or a singleton.

When we use this method elsewhere in the program, we invoke `print_backward_nicely` directly, and it invokes `print_backward` on our behalf. In that sense, `print_backward_nicely` acts as a **wrapper**, and it uses `print_backward` as a **helper**.

25.9. The `LinkedList` class

There are some subtle problems with the way we have been implementing lists. In a reversal of cause and effect, we'll propose an alternative implementation first and then explain what problems it solves.

First, we'll create a new class called `LinkedList`. Its attributes are an integer that contains the length of the list and a reference to the first node. `LinkedList` objects serve as handles for manipulating lists of `Node` objects:

```
class LinkedList:
    def __init__(self):
        self.length = 0
        self.head = None
```

One nice thing about the `LinkedList` class is that it provides a natural place to put wrapper functions like `print_backward_nicely`, which we can make a method of the `LinkedList` class:

```
class LinkedList:
    ...
    def print_backward(self):
        print("[", end=" ")
        if self.head is not None:
            self.head.print_backward()

class Node:
    ...
    def print_backward(self):
        if self.next is not None:
            tail = self.next
            tail.print_backward()
        print(self.cargo, end=" ")
```

Just to make things confusing, we renamed `print_backward_nicely`. Now there are two methods named `print_backward`: one in the `Node` class (the helper); and one in the `LinkedList` class (the wrapper). When the wrapper invokes `self.head.print_backward`, it is invoking the helper, because `self.head` is a `Node` object.

Another benefit of the `LinkedList` class is that it makes it easier to add or remove the first element of a list. For example, `add_first` is a method for `LinkedLists`; it takes an item of cargo as an argument and puts it at the beginning of the list:

```
class LinkedList:
    ...
    def add_first(self, cargo):
        node = Node(cargo)
        node.next = self.head
        self.head = node
        self.length += 1
```

As usual, you should check code like this to see if it handles the special cases. For example, what happens if the list is initially empty?

25.10. Invariants

Some lists are well formed; others are not. For example, if a list contains a loop, it will cause many of our methods to crash, so we might want to require that lists contain no loops. Another requirement is that the `length` value in the `LinkedList` object should be equal to the actual number of nodes in the list.

Requirements like these are called **invariants** because, ideally, they should be true of every object all the time. Specifying invariants for objects is a useful programming practice because it makes it easier to prove the correctness of code, check the integrity of data structures, and detect errors.

One thing that is sometimes confusing about invariants is that there are times when they are violated. For example, in the middle of `add_first`, after we have added the node but before we have incremented `length`,the invariant is violated. This kind of violation is acceptable; in fact, it is often impossible to modify an object without violating an invariant for at least a little while. Normally, we require that every method that violates an invariant must restore the invariant.

If there is any significant stretch of code in which the invariant is violated, it is important for the comments to make that clear, so that no operations are performed that depend on the invariant.

25.11. Glossary

```
embedded reference
    A reference stored in an attribute of an object.

linked list
    A data structure that implements a collection using a sequence of linked
    nodes.

node
    An element of a list, usually implemented as an object that contains a
    reference to another object of the same type.

cargo
    An item of data contained in a node.

link
    An embedded reference used to link one object to another.

precondition
    An assertion that must be true in order for a method to work correctly.

fundamental ambiguity theorem
```

```
    A reference to a list node can be treated as a single object or as the
    first in a list of nodes.

singleton
    A linked list with a single node.

wrapper
    A method that acts as a middleman between a caller and a helper method,
    often making the method easier or less error-prone to invoke.

helper
    A method that is not invoked directly by a caller but is used by another
    method to perform part of an operation.

invariant
    An assertion that should be true of an object at all times (except
    perhaps while the object is being modified).
```

25.12. Exercises

1. By convention, lists are often printed in brackets with commas between the elements, as in `[1, 2, 3]`.

Modify `print_list` so that it generates output in this format.

Chapter 26: Stacks

26.1. Abstract data types

The data types you have seen so far are all concrete, in the sense that we have completely specified how they are implemented. For example, the `Card` class represents a card using two integers. As we discussed at the time, that is not the only way to represent a card; there are many alternative implementations.

An **abstract data type**, or ADT, specifies a set of operations (or methods) and the semantics of the operations (what they do), but it does not not specify the implementation of the operations. That's what makes it abstract.

Why is that useful?

1. It simplifies the task of specifying an algorithm if you can denote the operations you need without having to think at the same time about how the operations are performed.
2. Since there are usually many ways to implement an ADT, it might be useful to write an algorithm that can be used with any of the possible implementations.
3. Well-known ADTs, such as the Stack ADT in this chapter, are often implemented in standard libraries so they can be written once and used by many programmers.
4. The operations on ADTs provide a common high-level language for specifying and talking about algorithms.

When we talk about ADTs, we often distinguish between the code that uses the ADT, called the **client** code, from the code that implements the ADT, called the **provider/implementor** code.

26.2. The Stack ADT

In this chapter, we will look at one common ADT, the **stack**. A stack is a collection, meaning that it is a data structure that contains multiple elements. Other collections we have seen include dictionaries and lists.

An ADT is defined by the operations that can be performed on it, which is called an **interface**. The interface for a stack consists of these operations:

`__init__`
Initialize a new empty stack.

`push`
Add a new item to the stack.

`pop`
Remove and return an item from the stack. The item that is returned is always the last one that was added.

`is_empty`
Check whether the stack is empty.

A stack is sometimes called a "Last in, First out" or **LIFO** data structure, because the last item added is the first to be removed.

26.3. Implementing stacks with Python lists

The list operations that Python provides are similar to the operations that define a stack. The interface isn't exactly what it is supposed to be, but we can write code to translate from the Stack ADT to the built-in operations.

This code is called an **implementation** of the Stack ADT. In general, an implementation is a set of methods that satisfy the syntactic and semantic requirements of an interface.

Here is an implementation of the Stack ADT that uses a Python list:

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def is_empty(self):
        return (self.items == [])
```

A `Stack` object contains an attribute named `items` that is a list of items in the stack. The initialization method sets `items` to the empty list.

To push a new item onto the stack, `push` appends it onto `items`. To pop an item off the stack, `pop` uses the homonymous (*same-named*) list method to remove and return the last item on the list.

Finally, to check if the stack is empty, `is_empty` compares `items` to the empty list.

An implementation like this, in which the methods consist of simple invocations of existing methods, is called a **veneer**. In real life, veneer is a thin coating of good quality wood used in furniture-making

to hide lower quality wood underneath. Computer scientists use this metaphor to describe a small piece of code that hides the details of an implementation and provides a simpler, or more standard, interface.

26.4. Pushing and popping

A stack is a **generic data structure**, which means that we can add any type of item to it. The following example pushes two integers and a string onto the stack:

```
>>> s = Stack()
>>> s.push(54)
>>> s.push(45)
>>> s.push("+")
```

We can use `is_empty` and `pop` to remove and print all of the items on the stack:

```
while not s.is_empty():
    print(s.pop(), end=" ")
```

The output is `+ 45 54`. In other words, we just used a stack to print the items backward! Granted, it's not the standard format for printing a list, but by using a stack, it was remarkably easy to do.

You should compare this bit of code to the implementation of `print_backward` in the last chapter. There is a natural parallel between the recursive version of `print_backward` and the stack algorithm here. The difference is that `print_backward` uses the runtime stack to keep track of the nodes while it traverses the list, and then prints them on the way back from the recursion. The stack algorithm does the same thing, except that it uses a `Stack` object instead of the runtime stack.

26.5. Using a stack to evaluate postfix

In most programming languages, mathematical expressions are written with the operator between the two operands, as in `1 + 2`. This format is called **infix**. An alternative used by some calculators is called **postfix**. In postfix, the operator follows the operands, as in `1 2 +`.

The reason postfix is sometimes useful is that there is a natural way to evaluate a postfix expression using a stack:

1. Starting at the beginning of the expression, get one term (operator or operand) at a time.

- If the term is an operand, push it on the stack.
- If the term is an operator, pop two operands off the stack, perform the operation on them, and push the result back on the stack.

2. When you get to the end of the expression, there should be exactly one operand left on the stack. That

operand is the result.

26.6. Parsing

To implement the previous algorithm, we need to be able to traverse a string and break it into operands and operators. This process is an example of **parsing**, and the results — the individual chunks of the string — are called **tokens**. You might remember these words from Chapter 1.

Python provides a `split` method in both string objects and the `re` (regular expression) module. A string's `split` method splits it into a list using a single character as a **delimiter**. For example:

```
>>> "Now is the time".split(" ")
['Now', 'is', 'the', 'time']
```

In this case, the delimiter is the space character, so the string is split at each space.

The function `re.split` is more powerful, allowing us to provide a regular expression instead of a delimiter. A regular expression is a way of specifying a set of strings. For example, `[A-z]` is the set of all letters and `[0-9]` is the set of all numbers. The `^` operator negates a set, so `[^0-9]` is the set of everything that is not a number, which is exactly the set we want to use to split up postfix expressions:

```
>>> import re
>>> re.split("([^0-9])", "123+456*/")
['123', '+', '456', '*', '', '/', '']
```

The resulting list includes the operands `123` and `456` and the operators `*` and `/`. It also includes two empty strings that are inserted after the operands.

26.7. Evaluating postfix

To evaluate a postfix expression, we will use the parser from the previous section and the algorithm from the section before that. To keep things simple, we'll start with an evaluator that only implements the operators `+` and `*`:

```
def eval_postfix(expr):
    import re
    token_list = re.split("([^0-9])", expr)
    stack = Stack()
    for token in token_list:
        if token == "" or token == " ":
            continue
        if token == "+":
            sum = stack.pop() + stack.pop()
            stack.push(sum)
        elif token == "*":
            product = stack.pop() * stack.pop()
            stack.push(product)
        else:
            stack.push(int(token))
    return stack.pop()
```

The first condition takes care of spaces and empty strings. The next two conditions handle operators. We assume, for now, that anything else must be an operand. Of course, it would be better to check for erroneous input and report an error message, but we'll get to that later.

Let's test it by evaluating the postfix form of `(56 + 47) * 2`:

```
>> eval_postfix("56 47 + 2 *")
206
```

That's close enough.

26.8. Clients and providers

One of the fundamental goals of an ADT is to separate the interests of the provider, who writes the code that implements the ADT, and the client, who uses the ADT. The provider only has to worry about whether the implementation is correct — in accord with the specification of the ADT — and not how it will be used.

Conversely, the client *assumes* that the implementation of the ADT is correct and doesn't worry about the details. When you are using one of Python's built-in types, you have the luxury of thinking exclusively as a client.

Of course, when you implement an ADT, you also have to write client code to test it. In that case, you play both roles, which can be confusing. You should make some effort to keep track of which role you are playing
at any moment.

26.9. Glossary

abstract data type (ADT)
: A data type (usually a collection of objects) that is defined by a set of operations but that can be implemented in a variety of ways.

client
: A program (or the person who wrote it) that uses an ADT.

delimiter
: A character that is used to separate tokens, such as punctuation in a natural language.

generic data structure
: A kind of data structure that can contain data of any type.

implementation
: Code that satisfies the syntactic and semantic requirements of an interface.

interface
: The set of operations that define an ADT.

infix
: A way of writing mathematical expressions with the operators between the operands.

parse
: To read a string of characters or tokens and analyze its grammatical structure.

postfix
: A way of writing mathematical expressions with the operators after the operands.

provider
: The code (or the person who wrote it) that implements an ADT.

token
: A set of characters that are treated as a unit for purposes of parsing, such as the words in a natural language.

veneer

```
41      A class definition that implements an ADT with method definitions that
42      are invocations of other methods, sometimes with simple transformations.
43      The veneer does no significant work, but it improves or standardizes the
44      interface seen by the client.
```

26.10. Exercises

1. Apply the postfix algorithm to the expression `1 2 + 3 *`.This example demonstrates one of the advantages

of postfix—there is no need to use parentheses to control the order of operations. To get the same result in infix, we would have to write `(1 + 2) * 3`.
2. Write a postfix expression that is equivalent to `1 + 2 * 3`.

Chapter 27: Queues

This chapter presents two ADTs: the Queue and the Priority Queue. In real life, a **queue** is a line of people waiting for something. In most cases, the first person in line is the next one to be served. There are exceptions, though. At airports, peoples whose flights are leaving soon are sometimes taken from the middle of the queue. At supermarkets, a polite person might let someone with only a few items go in front of them.

The rule that determines who goes next is called the **queueing policy**. The simplest queueing policy is called "First in, First out", **FIFO** for short. The most general queueing policy is **priority queueing**, in which each person is assigned a priority and the person with the highest priority goes first, regardless of the order of arrival. We say this is the most general policy because the priority can be based on anything: what time a flight leaves; how many groceries the person has; or how important the person is. Of course, not all queueing policies are fair, but fairness is in the eye of the beholder.

The Queue ADT and the Priority Queue ADT have the same set of operations. The difference is in the semantics of the operations: a queue uses the FIFO policy; and a priority queue (as the name suggests) uses the priority queueing policy.

27.1. The Queue ADT

The Queue ADT is defined by the following operations:

`__init__`
Initialize a new empty queue.

`insert`
Add a new item to the queue.

`remove`
Remove and return an item from the queue. The item that is returned is the first one that was added.

`is_empty`
Check whether the queue is empty.

27.2. Linked Queue

The first implementation of the Queue ADT we will look at is called a **linked queue** because it is made up of linked `Node` objects. Here is the class definition:

```
class Queue:
    def __init__(self):
        self.length = 0
        self.head = None

    def is_empty(self):
        return self.length == 0

    def insert(self, cargo):
        node = Node(cargo)
        if self.head is None:
            # If list is empty the new node goes first
            self.head = node
        else:
            # Find the last node in the list
            last = self.head
            while last.next:
                last = last.next
            # Append the new node
            last.next = node
        self.length += 1

    def remove(self):
        cargo = self.head.cargo
        self.head = self.head.next
        self.length -= 1
        return cargo
```

The methods `is_empty` and `remove` are identical to the `LinkedList` methods `is_empty` and `remove_first`. The `insert` method is new and a bit more complicated.

We want to insert new items at the end of the list. If the queue is empty, we just set `head` to refer to the new node.

Otherwise, we traverse the list to the last node and tack the new node on the end. We can identify the last node because its `next` attribute is `None`.

There are two invariants for a properly formed `Queue` object. The value of `length` should be the number of nodes in the queue, and the last node should have `next` equal to `None`. Convince yourself that this method preserves both invariants.

27.3. Performance characteristics

Normally when we invoke a method, we are not concerned with the details of its implementation. But there is one detail we might want to know: the performance characteristics of the method. How long does it take, and how does the run time change as the number of items in the collection increases?

First look at `remove`. There are no loops or function calls here, suggesting that the runtime of this method is the same every time. Such a method is called a **constant time** operation. In reality, the method might be slightly faster when the list is empty since it skips the body of the conditional, but that difference is not significant.

The performance of `insert` is very different. In the general case, we have to traverse the list to find the last element.

This traversal takes time proportional to the length of the list. Since the runtime is a linear function of the length, this method is called **linear time**. Compared to constant time, that's very bad.

27.4. Improved Linked Queue

We would like an implementation of the Queue ADT that can perform all operations in constant time. One way to do that is to modify the `Queue` class so that it maintains a reference to both the first and the last node, as shown in the figure:

The `ImprovedQueue` implementation looks like this:

```
class ImprovedQueue:
    def __init__(self):
        self.length = 0
        self.head = None
        self.last = None

    def is_empty(self):
        return self.length == 0
```

So far, the only change is the attribute `last`. It is used in `insert` and `remove` methods:

```
class ImprovedQueue:
    ...
    def insert(self, cargo):
        node = Node(cargo)
        if self.length == 0:
            # If list is empty, the new node is head and last
            self.head = self.last = node
        else:
            # Find the last node
            last = self.last
            # Append the new node
            last.next = node
            self.last = node
        self.length += 1
```

Since `last` keeps track of the last node, we don't have to search for it. As a result, this method is constant time.

There is a price to pay for that speed. We have to add a special case to `remove` to set `last` to `None` when the last node is removed:

```
class ImprovedQueue:
    ...
    def remove(self):
        cargo = self.head.cargo
        self.head = self.head.next
        self.length -= 1
        if self.length == 0:
            self.last = None
        return cargo
```

This implementation is more complicated than the Linked Queue implementation, and it is more difficult to demonstrate that it is correct. The advantage is that we have achieved the goal — both `insert` and `remove` are constant time operations.

27.5. Priority queue

The Priority Queue ADT has the same interface as the Queue ADT, but different semantics. Again, the interface is:

`__init__`
Initialize a new empty queue.

`insert`
Add a new item to the queue.

`remove`
Remove and return an item from the queue. The item that is returned is the one with the highest priority.

`is_empty`
Check whether the queue is empty.

The semantic difference is that the item that is removed from the queue is not necessarily the first one that was added. Rather, it is the item in the queue that has the highest priority. What the priorities are and how they compare to each other are not specified by the Priority Queue implementation. It depends on which items are in the queue.

For example, if the items in the queue have names, we might choose them in alphabetical order. If they are bowling scores, we might go from highest to lowest, but if they are golf scores, we would go from lowest to highest. As long as we can compare the items in the queue, we can find and remove the one with the highest priority.

This implementation of Priority Queue has as an attribute a Python list that contains the items in the queue.

```
class PriorityQueue:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return not self.items

    def insert(self, item):
        self.items.append(item)
```

The initialization method, `is_empty`, and `insert` are all veneers on list operations. The only interesting method is `remove`:

```
class PriorityQueue:
    ...
    def remove(self):
        maxi = 0
        for i in range(1, len(self.items)):
            if self.items[i] > self.items[maxi]:
                maxi = i
        item = self.items[maxi]
        del self.items[maxi]
        return item
```

At the beginning of each iteration, `maxi` holds the index of the biggest item (highest priority) we have seen *so far*. Each time through the loop, the program compares the `i`'th item to the champion. If the new item is bigger, the value of `maxi` is set to `i`.

When the `for` statement completes, `maxi` is the index of the biggest item. This item is removed from the list and returned.

Let's test the implementation:

```
>>> q = PriorityQueue()
>>> for num in [11, 12, 14, 13]:
...     q.insert(num)
...
>>> while not q.is_empty():
...     print(q.remove())
...
14
13
12
11
```

If the queue contains simple numbers or strings, they are removed in numerical or alphabetical order, from highest to lowest. Python can find the biggest integer or string because it can compare them using the built-in comparison operators.

If the queue contains an object type, it has to provide a `__gt__` method. When `remove` uses the `>` operator to compare items, it invokes the `__gt__` method for one of the items and passes the other as a parameter. As long as the `__gt__` method works correctly, the Priority Queue will work.

27.6. The `Golfer` class

As an example of an object with an unusual definition of priority, let's implement a class called `Golfer` that keeps track of the names and scores of golfers. As usual, we start by defining `__init__` and `__str__`:

```
class Golfer:
    def __init__(self, name, score):
        self.name = name
        self.score= score

    def __str__(self):
        return "{0:16}: {1}".format(self.name, self.score)
```

__str__ uses the format method to put the names and scores in neat columns.

Next we define a version of __gt__ where the lowest score gets highest priority. As always, __gt__ returns `True` if `self` is greater than `other`, and `False` otherwise.

```
class Golfer:
    ...
    def __gt__(self, other):
        return self.score < other.score    # Less is more
```

Now we are ready to test the priority queue with the `Golfer` class:

```
>>> tiger = Golfer("Tiger Woods", 61)
>>> phil = Golfer("Phil Mickelson", 72)
>>> hal = Golfer("Hal Sutton", 69)
>>>
>>> pq = PriorityQueue()
>>> for g in [tiger, phil, hal]:
...     pq.insert(g)
...
>>> while not pq.is_empty():
...     print(pq.remove())
...
Tiger Woods     : 61
Hal Sutton      : 69
Phil Mickelson  : 72
```

27.7. Glossary

```
constant time
    An operation whose runtime does not depend on the size of the data
    structure.

FIFO (First In, First Out)
    a queueing policy in which the first member to arrive is the first to be
    removed.

linear time
    An operation whose runtime is a linear function of the size of the data
    structure.

linked queue
```

```
    An implementation of a queue using a linked list.

priority queue
    A queueing policy in which each member has a priority determined by
    external factors. The member with the highest priority is the first to
    be removed.

Priority Queue
    An ADT that defines the operations one might perform on a priority
    queue.

queue
    An ordered set of objects waiting for a service of some kind.

Queue
    An ADT that performs the operations one might perform on a queue.

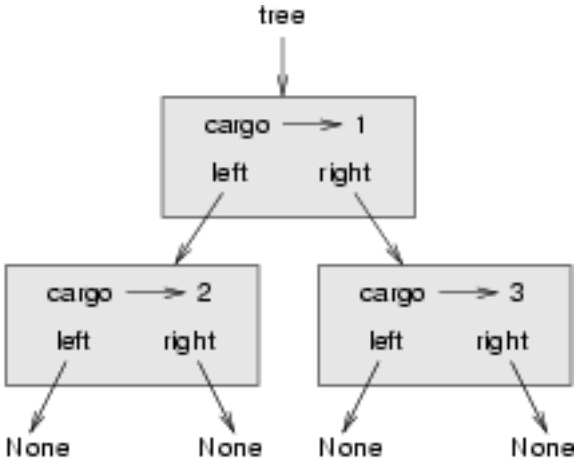
queueing policy
    The rules that determine which member of a queue is removed next.
```

27.8. Exercises

1. Write an implementation of the Queue ADT using a Python list. Compare the performance of this implementation to the `ImprovedQueue` for a range of queue lengths.
2. Write an implementation of the Priority Queue ADT using a linked list. You should keep the list sorted so that removal is a constant time operation. Compare the performance of this implementation with the Python list implementation.

Chapter 28: Trees

Like linked lists, trees are made up of nodes. A common kind of tree is a **binary tree**, in which each node contains a reference to two other nodes (possibly `None`). These references are referred to as the left and right subtrees. Like list nodes, tree nodes also contain cargo. A state diagram for a tree looks like this:



To avoid cluttering up the picture, we often omit the `None`s.

The top of the tree (the node `tree` refers to) is called the **root**. In keeping with the tree metaphor, the other nodes are called branches and the nodes at the tips with null references are called **leaves**. It may seem odd that we draw the picture with the root at the top and the leaves at the bottom, but that is not the strangest thing.

To make things worse, computer scientists mix in another metaphor: the family tree. The top node is sometimes called a **parent** and the nodes it refers to are its **children**. Nodes with the same parent are called **siblings**.

Finally, there is a geometric vocabulary for talking about trees. We already mentioned left and right, but there is also up (toward the parent/root) and down (toward the children/leaves). Also, all of the nodes that are the same distance from the root comprise a **level** of the tree.

We probably don't need three metaphors for talking about trees, but there they are.

Like linked lists, trees are recursive data structures because they are defined recursively. A tree is either:

1. the empty tree, represented by `None`, or
2. a node that contains an object reference (cargo) and two tree references.

28.1. Building trees

The process of assembling a tree is similar to the process of assembling a linked list. Each constructor invocation builds a single node.

```
class Tree:
    def __init__(self, cargo, left=None, right=None):
        self.cargo = cargo
        self.left = left
        self.right = right

    def __str__(self):
        return str(self.cargo)
```

The `cargo` can be any type, but the `left` and `right` parameters should be tree nodes. `left` and `right` are optional; the default value is `None`.

To print a node, we just print the cargo.

One way to build a tree is from the bottom up. Allocate the child nodes first:

```
left = Tree(2)
right = Tree(3)
```

Then create the parent node and link it to the children:

```
tree = Tree(1, left, right)
```

We can write this code more concisely by nesting constructor invocations:

```
>>> tree = Tree(1, Tree(2), Tree(3))
```

Either way, the result is the tree at the beginning of the chapter.

28.2. Traversing trees

Any time you see a new data structure, your first question should be, “How do I traverse it?” The most natural way to traverse a tree is recursively. For example, if the tree contains integers as cargo, this function returns their sum:

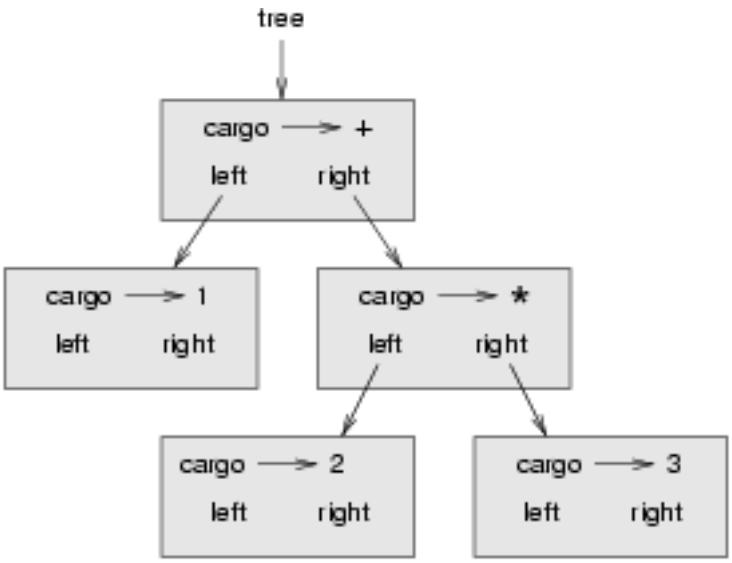
```
def total(tree):
    if tree is None: return 0
    return total(tree.left) + total(tree.right) + tree.cargo
```

The base case is the empty tree, which contains no cargo, so the sum is 0. The recursive step makes two recursive calls to find the sum of the child trees. When the recursive calls complete, we add the cargo of the parent and return the total.

28.3. Expression trees

A tree is a natural way to represent the structure of an expression. Unlike other notations, it can represent the computation unambiguously. For example, the infix expression `1 + 2 * 3` is ambiguous unless we know that the multiplication happens before the addition.

This expression tree represents the same computation:



The nodes of an expression tree can be operands like `1` and `2` or operators like `+` and `*`. Operands are leaf nodes; operator nodes contain references to their operands. (All of these operators are **binary**, meaning they have exactly two operands.)

We can build this tree like this:

```
>>> tree = Tree("+", Tree(1), Tree("*", Tree(2), Tree(3)))
```

Looking at the figure, there is no question what the order of operations is; the multiplication happens first in order to compute the second operand of the addition.

Expression trees have many uses. The example in this chapter uses trees to translate expressions to postfix, prefix, and infix. Similar trees are used inside compilers to parse, optimize, and translate programs.

28.4. Tree traversal

We can traverse an expression tree and print the contents like this:

```
def print_tree(tree):
    if tree is None: return
    print(tree.cargo, end=" ")
    print_tree(tree.left)
    print_tree(tree.right)
```

In other words, to print a tree, first print the contents of the root, then print the entire left subtree, and then print the entire right subtree. This way of traversing a tree is called a **preorder**, because the contents of the root appear *before* the contents of the children. For the previous example, the output is:

```
>>> tree = Tree("+", Tree(1), Tree("*", Tree(2), Tree(3)))
>>> print_tree(tree)
+ 1 * 2 3
```

This format is different from both postfix and infix; it is another notation called **prefix**, in which the operators appear before their operands.

You might suspect that if you traverse the tree in a different order, you will get the expression in a different notation. For example, if you print the subtrees first and then the root node, you get:

```
def print_tree_postorder(tree):
    if tree is None: return
    print_tree_postorder(tree.left)
    print_tree_postorder(tree.right)
    print(tree.cargo, end=" ")
```

The result, `1 2 3 * +`, is in postfix! This order of traversal is called **postorder**.

Finally, to traverse a tree **inorder**, you print the left tree, then the root, and then the right tree:

```
def print_tree_inorder(tree):
    if tree is None: return
    print_tree_inorder(tree.left)
    print(tree.cargo, end=" ")
    print_tree_inorder(tree.right)
```

The result is `1 + 2 * 3`, which is the expression in infix.

To be fair, we should point out that we have omitted an important complication. Sometimes when we write an expression in infix, we have to use parentheses to preserve the order of operations. So an inorder traversal is not quite sufficient to generate an infix expression.

Nevertheless, with a few improvements, the expression tree and the three recursive traversals provide a general way to translate expressions from one format to another.

If we do an inorder traversal and keep track of what level in the tree we are on, we can generate a graphical representation of a tree:

```
def print_tree_indented(tree, level=0):
    if tree is None: return
    print_tree_indented(tree.right, level+1)
    print("  " * level + str(tree.cargo))
    print_tree_indented(tree.left, level+1)
```

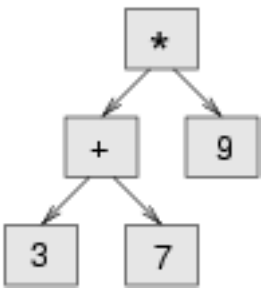
The parameter `level` keeps track of where we are in the tree. By default, it is initially `0`. Each time we make a recursive call, we pass `level+1` because the child's level is always one greater than the parent's. Each item is indented by two spaces per level. The result for the example tree is:

```
>>> print_tree_indented(tree)
    3
  *
    2
+
  1
```

If you look at the output sideways, you see a simplified version of the original figure.

28.5. Building an expression tree

In this section, we parse infix expressions and build the corresponding expression trees. For example, the expression `(3 + 7) * 9` yields the following tree:



Notice that we have simplified the diagram by leaving out the names of the attributes.

The parser we will write handles expressions that include numbers, parentheses, and the operators `+` and `*`. We assume that the input string has already been tokenized into a Python list (producing this list is left as an exercise). The token list for `(3 + 7) * 9` is:

```
["(", 3, "+", 7, ")", "*", 9, "end"]
```

The `end` token is useful for preventing the parser from reading past the end of the list.

The first function we'll write is `get_token`, which takes a token list and an expected token as parameters. It compares the expected token to the first token on the list: if they match, it removes the token from the list and returns `True`; otherwise, it returns `False`:

```
def get_token(token_list, expected):
    if token_list[0] == expected:
        del token_list[0]
        return True
    return False
```

Since `token_list` refers to a mutable object, the changes made here are visible to any other variable that refers to the same object.

The next function, `get_number`, handles operands. If the next token in `token_list` is a number, `get_number` removes it and returns a leaf node containing the number; otherwise, it returns `None`.

```
def get_number(token_list):
    x = token_list[0]
    if type(x) != type(0): return None
    del token_list[0]
    return Tree(x, None, None)
```

Before continuing, we should test `get_number` in isolation. We assign a list of numbers to `token_list`, extract the first, print the result, and print what remains of the token list:

```
>>> token_list = [9, 11, "end"]
>>> x = get_number(token_list)
>>> print_tree_postorder(x)
9
>>> print(token_list)
[11, "end"]
```

The next method we need is `get_product`, which builds an expression tree for products. A simple product has two numbers as operands, like `3 * 7`.

Here is a version of `get_product` that handles simple products.

```
def get_product(token_list):
    a = get_number(token_list)
    if get_token(token_list, "*"):
        b = get_number(token_list)
        return Tree("*", a, b)
    return a
```

Assuming that `get_number` succeeds and returns a singleton tree, we assign the first operand to `a`. If the next character is `*`, we get the second number and build an expression tree with `a`, `b`, and the operator.

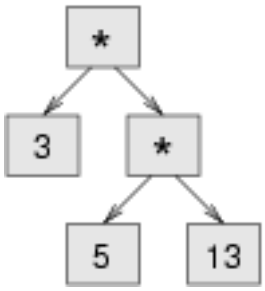
If the next character is anything else, then we just return the leaf node with `a`. Here are two examples:

```
>>> token_list = [9, "*", 11, "end"]
>>> tree = get_product(token_list)
>>> print_tree_postorder(tree)
9 11 *
```

```
>>> token_list = [9, "+", 11, "end"]
>>> tree = get_product(token_list)
>>> print_tree_postorder(tree)
9
```

The second example implies that we consider a single operand to be a kind of product. This definition of product is counter-intuitive, but it turns out to be useful.

Now we have to deal with compound products, like like `3 * 5 * 13`. We treat this expression as a product of products, namely `3 * (5 * 13)`. The resulting tree is:



With a small change in `get_product`, we can handle an arbitrarily long product:

```
def get_product(token_list):
    a = get_number(token_list)
    if get_token(token_list, "*"):
        b = get_product(token_list)       # This line changed
        return Tree("*", a, b)
    return a
```

In other words, a product can be either a singleton or a tree with * at the root, a number on the left, and a product on the right. This kind of recursive definition should be starting to feel familiar.

Let's test the new version with a compound product:

```
>>> token_list = [2, "*", 3, "*", 5 , "*", 7, "end"]
>>> tree = get_product(token_list)
>>> print_tree_postorder(tree)
2 3 5 7 * * *
```

Next we will add the ability to parse sums. Again, we use a slightly counter-intuitive definition of sum. For us, a sum can be a tree with + at the root, a product on the left, and a sum on the right. Or, a sum can be just a product.

If you are willing to play along with this definition, it has a nice property: we can represent any expression (without parentheses) as a sum of products. This property is the basis of our parsing algorithm.

`get_sum` tries to build a tree with a product on the left and a sum on the right. But if it doesn't find a +, it just builds a product.

```
def get_sum(token_list):
    a = get_product(token_list)
    if get_token(token_list, "+"):
        b = get_sum(token_list)
        return Tree("+", a, b)
    return a
```

Let's test it with `9 * 11 + 5 * 7`:

```
>>> token_list = [9, "*", 11, "+", 5, "*", 7, "end"]
>>> tree = get_sum(token_list)
>>> print_tree_postorder(tree)
9 11 * 5 7 * +
```

We are almost done, but we still have to handle parentheses. Anywhere in an expression where there can be a number, there can also be an entire sum enclosed in parentheses. We just need to modify `get_number` to handle **subexpressions**:

```
def get_number(token_list):
    if get_token(token_list, "("):
        x = get_sum(token_list)         # Get the subexpression
        get_token(token_list, ")")      # Remove the closing parenthesis
        return x
    else:
        x = token_list[0]
        if type(x) != type(0): return None
        del token_list[0]
        return Tree(x, None, None)
```

Let's test this code with `9 * (11 + 5) * 7`:

```
>>> token_list = [9, "*", "(", 11, "+", 5, ")", "*", 7, "end"]
>>> tree = get_sum(token_list)
>>> print_tree_postorder(tree)
9 11 5 + 7 * *
```

The parser handled the parentheses correctly; the addition happens before the multiplication.

In the final version of the program, it would be a good idea to give `get_number` a name more descriptive of its new role.

28.6. Handling errors

Throughout the parser, we've been assuming that expressions are well-formed. For example, when we reach the end of a subexpression, we assume that the next character is a close parenthesis. If there is an error and the next character is something else, we should deal with it.

```
def get_number(token_list):
    if get_token(token_list, "("):
        x = get_sum(token_list)
        if not get_token(token_list, ")"):
            raise ValueError('Missing close parenthesis")
        return x
    else:
        # The rest of the function omitted
```

The `raise` statement throws the exception object which we create. In this case we simply used the most appropriate type of built-in exception that we could find, but you should be aware that you can create your own more specific user-defined exceptions if you need to. If the function that called `get_number`, or one of the other functions in the traceback, handles the exception, then the program can continue. Otherwise, Python will print an error message and quit.

28.7. The animal tree

In this section, we develop a small program that uses a tree to represent a knowledge base.

The program interacts with the user to create a tree of questions and animal names. Here is a sample run:

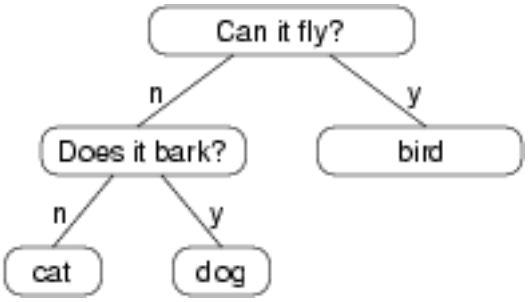
```
Are you thinking of an animal? y
Is it a bird? n
What is the animals name? dog
What question would distinguish a dog from a bird? Can it fly
If the animal were dog the answer would be? n

Are you thinking of an animal? y
Can it fly? n
Is it a dog? n
What is the animals name? cat
What question would distinguish a cat from a dog? Does it bark
If the animal were cat the answer would be? n

Are you thinking of an animal? y
Can it fly? n
Does it bark? y
Is it a dog? y
I rule!

Are you thinking of an animal? n
```

Here is the tree this dialog builds:



At the beginning of each round, the program starts at the top of the tree and asks the first question. Depending on the answer, it moves to the left or right child and continues until it gets to a leaf node. At that point, it makes a guess. If the guess is not correct, it asks the user for the name of the new animal and a question that distinguishes the (bad) guess from the new animal. Then it adds a node to the tree with the new question and the new animal.

Here is the code:

```
def yes(ques):
    ans = input(ques).lower()
    return ans[0] == "y"

def animal():
    # Start with a singleton
    root = Tree("bird")

    # Loop until the user quits
    while True:
        print()
        if not yes("Are you thinking of an animal? "): break

        # Walk the tree
        tree = root
        while tree.left is not None:
            prompt = tree.cargo + "? "
            if yes(prompt):
                tree = tree.right
            else:
                tree = tree.left

        # Make a guess
        guess = tree.cargo
        prompt = "Is it a " + guess + "? "
        if yes(prompt):
            print("I rule!")
            continue

        # Get new information
        prompt  = "What is the animal's name? "
        animal  = input(prompt)
        prompt  = "What question would distinguish a {0} from a {1}? "
        question = input(prompt.format(animal, guess))

        # Add new information to the tree
        tree.cargo = question
        prompt = "If the animal were {0} the answer would be? "
        if yes(prompt.format(animal)):
            tree.left = Tree(guess)
            tree.right = Tree(animal)
```

```
        else:
            tree.left = Tree(animal)
            tree.right = Tree(guess)
```

The function `yes` is a helper; it prints a prompt and then takes input from the user. If the response begins with *y* or *Y*, the function returns `True`.

The condition of the outer loop of `animal` is `True`, which means it will continue until the `break` statement executes, if the user is not thinking of an animal.

The inner `while` loop walks the tree from top to bottom, guided by the user's responses.

When a new node is added to the tree, the new question replaces the cargo, and the two children are the new animal and the original cargo.

One shortcoming of the program is that when it exits, it forgets everything you carefully taught it! Fixing this problem is left as an exercise.

28.8. Glossary

```
binary operator
    An operator that takes two operands.

binary tree
    A tree in which each node refers to zero, one, or two dependent nodes.

child
    One of the nodes referred to by a node.

leaf
    A bottom-most node in a tree, with no children.

level
    The set of nodes equidistant from the root.

parent
    The node that refers to a given node.

postorder
    A way to traverse a tree, visiting the children of each node before the
    node itself.

prefix notation
    A way of writing a mathematical expression with each operator appearing
```

```
    before its operands.

preorder
    A way to traverse a tree, visiting each node before its children.

root
    The topmost node in a tree, with no parent.

siblings
    Nodes that share a common parent.

subexpression
    An expression in parentheses that acts as a single operand in a larger
    expression.
```

28.9. Exercises

1. Modify `print_tree_inorder` so that it puts parentheses around every operator and pair of operands. Is

the output correct and unambiguous? Are the parentheses always necessary?
2. Write a function that takes an expression string and returns a token list.
3. Find other places in the expression tree functions where errors can occur and add appropriate `raise`

statements. Test your code with improperly formed expressions.
4. Think of various ways you might save the animal knowledge tree in a file. Implement the one you think is

easiest.

Appendix A: Debugging

Different kinds of errors can occur in a program, and it is useful to distinguish among them in order to track them down more quickly:

1. Syntax errors are produced by Python when it is translating the source code into byte code. They usually indicate that there is something wrong with the syntax of the program. Example: Omitting the colon at the end of a `def` statement yields the somewhat redundant message `SyntaxError: invalid syntax`.
2. Runtime errors are produced by the runtime system if something goes wrong while the program is running. Most runtime error messages include information about where the error occurred and what functions were executing. Example: An infinite recursion eventually causes a runtime error of maximum recursion depth exceeded.
3. Semantic errors are problems with a program that compiles and runs but doesn't do the right thing. Example: An expression may not be evaluated in the order you expect, yielding an unexpected result.

The first step in debugging is to figure out which kind of error you are dealing with. Although the following sections are organized by error type, some techniques are applicable in more than one situation.

A.1. Syntax errors

Syntax errors are usually easy to fix once you figure out what they are. Unfortunately, the error messages are often not helpful. The most common messages are `SyntaxError: invalid syntax` and `SyntaxError: invalid token`, neither of which is very informative.

On the other hand, the message does tell you where in the program the problem occurred. Actually, it tells you where Python noticed a problem, which is not necessarily where the error is. Sometimes the error is prior to the location of the error message, often on the preceding line.

If you are building the program incrementally, you should have a good idea about where the error is. It will be in the last line you added.

If you are copying code from a book, start by comparing your code to the book's code very carefully. Check every character. At the same time, remember that the book might be wrong, so if you see something that looks like a syntax error, it might be.

Here are some ways to avoid the most common syntax errors:

1. Make sure you are not using a Python keyword for a variable name.
2. Check that you have a colon at the end of the header of every compound statement, including `for`, `while`, `if`, and `def` statements.
3. Check that indentation is consistent. You may indent with either spaces or tabs but it's best not to mix them. Each level should be nested the same amount.
4. Make sure that any strings in the code have matching quotation marks.
5. If you have multiline strings with triple quotes (single or double), make sure you have terminated the string properly. An unterminated string may cause an `invalid token` error at the end of your program, or it may treat the following part of the program as a string until it comes to the next string. In the second case, it might not produce an error message at all!
6. An unclosed bracket — (, {, or [— makes Python continue with the next line as part of the current statement. Generally, an error occurs almost immediately in the next line.
7. Check for the classic = instead of == inside a conditional.

If nothing works, move on to the next section…

A.2. I can't get my program to run no matter what I do.

If the compiler says there is an error and you don't see it, that might be because you and the compiler are not looking at the same code. Check your programming environment to make sure that the program you are editing is the one Python is trying to run. If you are not sure, try putting an obvious and deliberate syntax error at the beginning of the program. Now run (or import) it again. If the compiler doesn't find the new error, there is probably something wrong with the way your environment is set up.

If this happens, one approach is to start again with a new program like Hello, World!, and make sure you can get a known program to run. Then gradually add the pieces of the new program to the working one.

A.3. Runtime errors

Once your program is syntactically correct, Python can import it and at least start running it. What could possibly go wrong?

A.4. My program does absolutely nothing.

This problem is most common when your file consists of functions and classes but does not actually invoke anything to start execution. This may be intentional if you only plan to import this module to supply classes and functions.

If it is not intentional, make sure that you are invoking a function to start execution, or execute one from the interactive prompt. Also see the Flow of Execution section below.

A.5. My program hangs.

If a program stops and seems to be doing nothing, we say it is hanging. Often that means that it is caught in an infinite loop or an infinite recursion.

1. If there is a particular loop that you suspect is the problem, add a `print` statement immediately before the loop that says entering the loop and another immediately after that says exiting the loop.
2. Run the program. If you get the first message and not the second, you've got an infinite loop. Go to the Infinite Loop section below.
3. Most of the time, an infinite recursion will cause the program to run for a while and then produce a RuntimeError: Maximum recursion depth exceeded error. If that happens, go to the Infinite Recursion section below.
4. If you are not getting this error but you suspect there is a problem with a recursive method or function, you can still use the techniques in the Infinite Recursion section.
5. If neither of those steps works, start testing other loops and other recursive functions and methods.
6. If that doesn't work, then it is possible that you don't understand the flow of execution in your program. Go to the Flow of Execution section below.

A.6. Infinite Loop

If you think you have an infinite loop and you think you know what loop is causing the problem, add a `print` statement at the end of the loop that prints the values of the variables in the condition and the value of the condition.

For example:

```
while x > 0 and y < 0:
    # Do something to x
    # Do something to y

    print("x: ", x)
    print("y: ", y)
    print("condition: ", (x > 0 and y < 0))
```

Now when you run the program, you will see three lines of output for each time through the loop. The last time through the loop, the condition should be `False`. If the loop keeps going, you will be able to see the values of `x` and `y`, and you might figure out why they are not being updated correctly.

In a development environment like PyScripter, one can also set a breakpoint at the start of the loop, and single-step through the loop. While you do this, inspect the values of `x` and `y` by hovering your cursor over them.

Of course, all programming and debugging require that you have a good mental model of what the algorithm ought to be doing: if you don't understand what ought to happen to `x` and `y`, printing or inspecting its value is of little use. Probably the best place to debug the code is away from your computer, working on your understanding of what should be happening.

A.7. Infinite Recursion

Most of the time, an infinite recursion will cause the program to run for a while and then produce a `Maximum recursion depth exceeded` error.

If you suspect that a function or method is causing an infinite recursion, start by checking to make sure that there is a base case. In other words, there should be some condition that will cause the function or method to return without making a recursive invocation. If not, then you need to rethink the algorithm and identify a base case.

If there is a base case but the program doesn't seem to be reaching it, add a `print` statement at the beginning of the function or method that prints the parameters. Now when you run the program, you will see a few lines of output every time the function or method is invoked, and you will see the parameters. If the parameters are not moving toward the base case, you will get some ideas about why not.

Once again, if you have an environment that supports easy single-stepping, breakpoints, and inspection, learn to use them well. It is our opinion that walking through code step-by-step builds the best and most accurate mental model of how computation happens. Use it if you have it!

A.8. Flow of Execution

If you are not sure how the flow of execution is moving through your program, add `print` statements to the beginning of each function with a message like entering function `foo`, where `foo` is the name

of the function.

Now when you run the program, it will print a trace of each function as it is invoked.

If you're not sure, step through the program with your debugger.

A.9. When I run the program I get an exception.

If something goes wrong during runtime, Python prints a message that includes the name of the exception, the line of the program where the problem occurred, and a traceback.

Put a breakpoint on the line causing the exception, and look around!

The traceback identifies the function that is currently running, and then the function that invoked it, and then the function that invoked *that*, and so on. In other words, it traces the path of function invocations that got you to where you are. It also includes the line number in your file where each of these calls occurs.

The first step is to examine the place in the program where the error occurred and see if you can figure out what happened. These are some of the most common runtime errors:

NameError
You are trying to use a variable that doesn't exist in the current environment. Remember that local variables are local. You cannot refer to them from outside the function where they are defined.

TypeError
There are several possible causes:

1. You are trying to use a value improperly. Example: indexing a string, list, or tuple with something other than an integer.
2. There is a mismatch between the items in a format string and the items passed for conversion. This can happen if either the number of items does not match or an invalid conversion is called for.
3. You are passing the wrong number of arguments to a function or method. For methods, look at the method definition and check that the first parameter is `self`. Then look at the method invocation; make sure you are invoking the method on an object with the right type and providing the other arguments correctly.

KeyError
You are trying to access an element of a dictionary using a key value that the dictionary does not contain.

AttributeError
You are trying to access an attribute or method that does not exist.

IndexError
The index you are using to access a list, string, or tuple is greater than its length minus one. Immediately before the site of the error, add a `print` statement to display the value of the index and the length of the sequence. Is the sequence the right size? Is the index the right value?

A.10. I added so many `print` statements I get inundated with output.

One of the problems with using `print` statements for debugging is that you can end up buried in output. There are two ways to proceed: simplify the output or simplify the program.

To simplify the output, you can remove or comment out `print` statements that aren't helping, or combine them, or format the output so it is easier to understand.

To simplify the program, there are several things you can do. First, scale down the problem the program is working on. For example, if you are sorting a sequence, sort a *small* sequence. If the program takes input from the user, give it the simplest input that causes the problem.

Second, clean up the program. Remove dead code and reorganize the program to make it as easy to read as possible. For example, if you suspect that the problem is in a deeply nested part of the program, try rewriting that part with simpler structure. If you suspect a large function, try splitting it into smaller functions and testing them separately.

Often the process of finding the minimal test case leads you to the bug. If you find that a program works in one situation but not in another, that gives you a clue about what is going on.

Similarly, rewriting a piece of code can help you find subtle bugs. If you make a change that you think doesn't affect the program, and it does, that can tip you off.

You can also wrap your debugging print statements in some condition, so that you suppress much of the output. For example, if you are trying to find an element using a binary search, and it is not working, you might code up a debugging print statement inside a conditional: if the range of candidate elements is less that 6, then print debugging information, otherwise don't print.

Similarly, breakpoints can be made conditional: you can set a breakpoint on a statement, then edit the breakpoint to say "only break if this expression becomes true".

A.11. Semantic errors

In some ways, semantic errors are the hardest to debug, because the compiler and the runtime system provide no information about what is wrong. Only you know what the program is supposed to do, and only you know that it isn't doing it.

The first step is to make a connection between the program text and the behavior you are seeing. You need a hypothesis about what the program is actually doing. One of the things that makes that hard is that computers run so fast.

You will often wish that you could slow the program down to human speed, and with some debuggers you can. But the time it takes to insert a few well-placed `print` statements is often short compared to setting up the debugger, inserting and removing breakpoints, and walking the program to where the error is occurring.

A.12. My program doesn't work.

You should ask yourself these questions:

1. Is there something the program was supposed to do but which doesn't seem to be happening? Find the section of the code that performs that function and make sure it is executing when you think it should.
2. Is something happening that shouldn't? Find code in your program that performs that function and see if it is executing when it shouldn't.
3. Is a section of code producing an effect that is not what you expected? Make sure that you understand the code in question, especially if it involves invocations to functions or methods in other Python modules. Read the documentation for the functions you invoke. Try them out by writing simple test cases and checking the results.

In order to program, you need to have a mental model of how programs work. If you write a program that doesn't do what you expect, very often the problem is not in the program; it's in your mental model.

The best way to correct your mental model is to break the program into its components (usually the functions and methods) and test each component independently. Once you find the discrepancy between your model and reality, you can solve the problem.

Of course, you should be building and testing components as you develop the program. If you encounter a problem, there should be only a small amount of new code that is not known to be correct.

A.13. I've got a big hairy expression and it doesn't do what I expect.

Writing complex expressions is fine as long as they are readable, but they can be hard to debug. It is often a good idea to break a complex expression into a series of assignments to temporary variables.

For example:

```
self.hands[i].add_card(self.hands[self.find_neighbor(i)].pop_card())
```

This can be rewritten as:

```
neighbor = self.find_neighbor (i)
picked_card = self.hands[neighbor].pop_card()
self.hands[i].add_card(picked_card)
```

The explicit version is easier to read because the variable names provide additional documentation, and it is easier to debug because you can check the types of the intermediate variables and display or inspect their values.

Another problem that can occur with big expressions is that the order of evaluation may not be what you expect. For example, if you are translating the expression `x/2pi` into Python, you might write:

```
y = x / 2 * math.pi
```

That is not correct because multiplication and division have the same precedence and are evaluated from left to right. So this expression computes `(x/2)pi`.

A good way to debug expressions is to add parentheses to make the order of evaluation explicit:

```
y = x / (2 * math.pi)
```

Whenever you are not sure of the order of evaluation, use parentheses. Not only will the program be correct (in the sense of doing what you intended), it will also be more readable for other people who haven't memorized the rules of precedence.

A.14. I've got a function or method that doesn't return what I expect.

If you have a `return` statement with a complex expression, you don't have a chance to print the `return` value before returning. Again, you can use a temporary variable. For example, instead of:

```
return self.hands[i].remove_matches()
```

you could write:

```
count = self.hands[i].remove_matches()
return count
```

Now you have the opportunity to display or inspect the value of `count` before returning.

A.15. I'm really, really stuck and I need help.

First, try getting away from the computer for a few minutes. Computers emit waves that affect the brain, causing these effects:

1. Frustration and/or rage.
2. Superstitious beliefs (the computer hates me) and magical thinking

(the program only works when I wear my hat backward).
3. Random-walk programming (the attempt to program by writing every

possible program and choosing the one that does the right thing).

If you find yourself suffering from any of these symptoms, get up and go for a walk. When you are calm, think about the program. What is it doing? What are some possible causes of that behavior? When was the last time you had a working program, and what did you do next?

Sometimes it just takes time to find a bug. We often find bugs when we are away from the computer and let our minds wander. Some of the best places to find bugs are trains, showers, and in bed, just before you fall asleep.

A.16. No, I really need help.

It happens. Even the best programmers occasionally get stuck. Sometimes you work on a program so long that you can't see the error. A fresh pair of eyes is just the thing.

Before you bring someone else in, make sure you have exhausted the techniques described here. Your program should be as simple as possible, and you should be working on the smallest input that causes the error. You should have `print` statements in the appropriate places (and the output they produce should be comprehensible). You should understand the problem well enough to describe it concisely.

When you bring someone in to help, be sure to give them the information they need:

1. If there is an error message, what is it and what part of the

program does it indicate?
2. What was the last thing you did before this error occurred? What

were the last lines of code that you wrote, or what is the new test
case that fails?
3. What have you tried so far, and what have you learned?

Good instructors and helpers will also do something that should not offend you: they won't believe when you tell them *"I'm sure all the input routines are working just fine, and that I've set up the data correctly!"*. They will want to validate and check things for themselves. After all, your program has a bug. Your understanding and inspection of the code have not found it yet. So you should expect to have your assumptions challenged. And as you gain skills and help others, you'll need to do the same for them.

When you find the bug, take a second to think about what you could have done to find it faster. Next time you see something similar, you will be able to find the bug more quickly.

Remember, the goal is not just to make the program work. The goal is to learn how to make the program work.

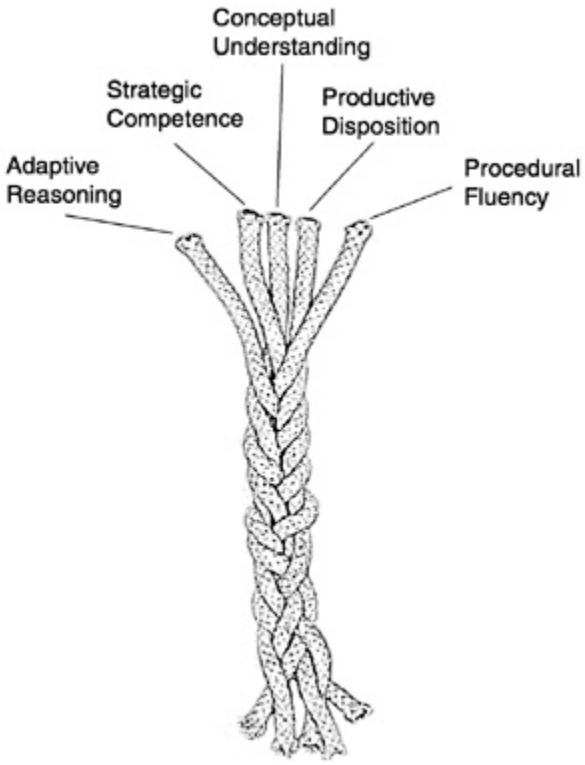
Appendix B: An odds-and-ends Workbook

This workbook / cookbook of recipes is still very much under construction.

B.1. The Five Strands of Proficiency

This was an important study commissioned by the President in the USA. It looked at what was needed for students to become proficient in maths.

But it is also an amazingly accurate fit for what we need for proficiency in Computer Science, or even for proficiency in playing Jazz!



1. **Procedural Fluency**: Learn the syntax. Learn to type. Learn your way around your tools. Learn and practice your scales. Learn to rearrange formulae.

2. **Conceptual Understanding**: Understand why the bits fit together like they do.

3. **Strategic Competence**: Can you see what to do next? Can you formulate this word problem into your notation? Can you take the music where you want it to go?

4. **Adaptive Reasoning**: Can you see how to change what you've learned for this new problem?

5. A **Productive Disposition**: We need that *Can Do!* attitude!

1. You habitually think it is worthwhile studying this stuff.
2. You are diligent and disciplined enough to grind through the tough stuff, and to put in your practice hours.
3. You develop a sense of *efficacy* — that you can make things happen!

Check out http://mason.gmu.edu/~jsuh4/teaching/strands.htm, or Kilpatrick's book at http://www.nap.edu/openbook.php?isbn=0309069955

B.2. Sending Email

Sometimes it is fun to do powerful things with Python — remember that part of the "productive disposition" we saw under the five threads of proficiency included *efficacy* — the sense of being able to accomplish something useful. Here is a Python example of how you can send email to someone.

```
import smtplib, email.mime.text

me = "joe@my.org.com"                # Put your own email here
fred = "fred@his.org.com"            # And fred's email address here
your_mail_server = "mail.my.org.com" # Ask your system administrator

# Create a text message containing the body of the email.
# You could read this from a file, of course.
msg = email.mime.text.MIMEText("""Hey Fred,

I'm having a party, please come at 8pm.
Bring a plate of snacks and your own drinks.
```

```
13
14  Joe""" )
15
16  msg["From"] = me                # Add headers to the message object
17  msg["To"] = fred
18  msg["Subject"] = "Party on Saturday 23rd"
19
20  # Create a connection to your mail server
21  svr = smtplib.SMTP(your_mail_server)
22  response = svr.sendmail(me, fred, msg.as_string()) # Send message
23  if response != {}:
24      print("Sending failed for ", response)
25  else:
26      print("Message sent.")
27
28  svr.quit()                                    # Close the connection
```

In the context of the course, notice how we use the two objects in this program: we create a message object on line 9, and set some attributes at lines 16-18. We then create a connection object at line 21, and ask it to send our message.

B.3. Write your own Web Server

Python is gaining in popularity as a tool for writing web applications. Although one will probably use Python to process requests behind a web server like Apache, there are powerful libraries which allow you to write your own stand-alone web server in a couple of lines. This simpler approach means that you can have a test web server running on your own desktop machine in a couple of minutes, without having to install any extra software.

In this cookbook example we use the `wsgi` ("wizz-gee") protocol: a modern way of connecting web servers to code that runs to provide the services. See http://en.wikipedia.org/wiki/Web_Server_-Gateway_Interface for more on `wsgi`.

```
1  from codecs import latin_1_encode
2  from wsgiref.simple_server import make_server
3
4  def my_handler(environ, start_response):
5      path_info = environ.get("PATH_INFO", None)
6      query_string = environ.get("QUERY_STRING", None)
7      response_body = "You asked for {0} with query {1}".format(
8                                              path_info, query_string)
9      response_headers = [("Content-Type", "text/plain"),
```

```
                ("Content-Length", str(len(response_body)))]
    start_response("200 OK", response_headers)
    response = latin_1_encode(response_body)[0]
    return [response]

httpd = make_server("127.0.0.1",  8000,  my_handler)
httpd.serve_forever()   # Start the server listening for requests
```

When you run this, your machine will listen on port 8000 for requests. (You may have to tell your firewall software to be kind to your new application!)

In a web browser, navigate to http://127.0.0.1:8000/catalogue?category=guitars. Your browser should get the response

```
You asked for /catalogue with query category=guitars
```

Your web server will keep running until you interrupt it (**Ctrl+F2** if you are using PyScripter).

The important lines 15 and 16 create a web server on the local machine, listening at port 8000. Each incoming html request causes the server to call `my_handler` which processes the request and returns the appropriate response.

We modify the above example below: `my_handler` now interrogates the `path_info`, and calls specialist functions to deal with each different kind of incoming request. (We say that `my_handler` *dispatches* the request to the appropriate function.) We can easily add other more request cases:

```
import time

def my_handler(environ, start_response):
    path_info = environ.get("PATH_INFO", None)
    if path_info == "/gettime":
        response_body = gettime(environ, start_response)
    elif path_info == "/classlist":
        response_body = classlist(environ, start_response)
    else:
        response_body = ""
        start_response("404 Not Found", [("Content-Type", "text/plain")])

    response = latin_1_encode(response_body)[0]
    return [response]

def gettime(env, resp):
    html_template = """<html>
    <body bgcolor='lightblue'>
```

```
      <h2>The time on the server is {0}</h2>
    <body>
    </html>
    """
    response_body = html_template.format(time.ctime())
    response_headers = [("Content-Type", "text/html"),
            ("Content-Length", str(len(response_body)))]
    resp("200 OK", response_headers)
    return response_body

def classlist(env, resp):
    return  # Will be written in the next section!
```

Notice how `gettime` returns an (admittedly simple) html document which is built on the fly by using `format` to substitute content into a predefined template.

B.4. Using a Database

Python has a library for using the popular and lightweight **sqlite** database. Learn more about this self-contained, embeddable, zero-configuration SQL database engine at http://www.sqlite.org.

Firstly, we have a script that creates a new database, creates a table, and stores some rows of test data into the table: (Copy and paste this code into your Python system.)

We get this output:

```
Database table StudentSubjects has been created.
StudentSubjects table now has 18 rows of data.
```

Our next recipe adds to our web browser from the previous section. We'll allow a query like `classlist?subject=CompSci&year=2012` and show how our server can extract the arguments from the query string, query the database, and send the rows back to the browser as a formatted table within an html page. We'll start with two new imports to get access to `sqlite3` and `cgi`, a library which helps us parse forms and query strings that are sent to the server:

```
import sqlite3
import cgi
```

Now we replace the stub function `classlist` with a handler that can do what we need:

```
classlistTemplate = """<html>
<body bgcolor='lightgreen'>
  <h2>Students taking {0} during {1}:</h2>
  <table border=3 cellspacing=2 cellpadding=2>
{2}
  </table>
<body>
</html>
"""

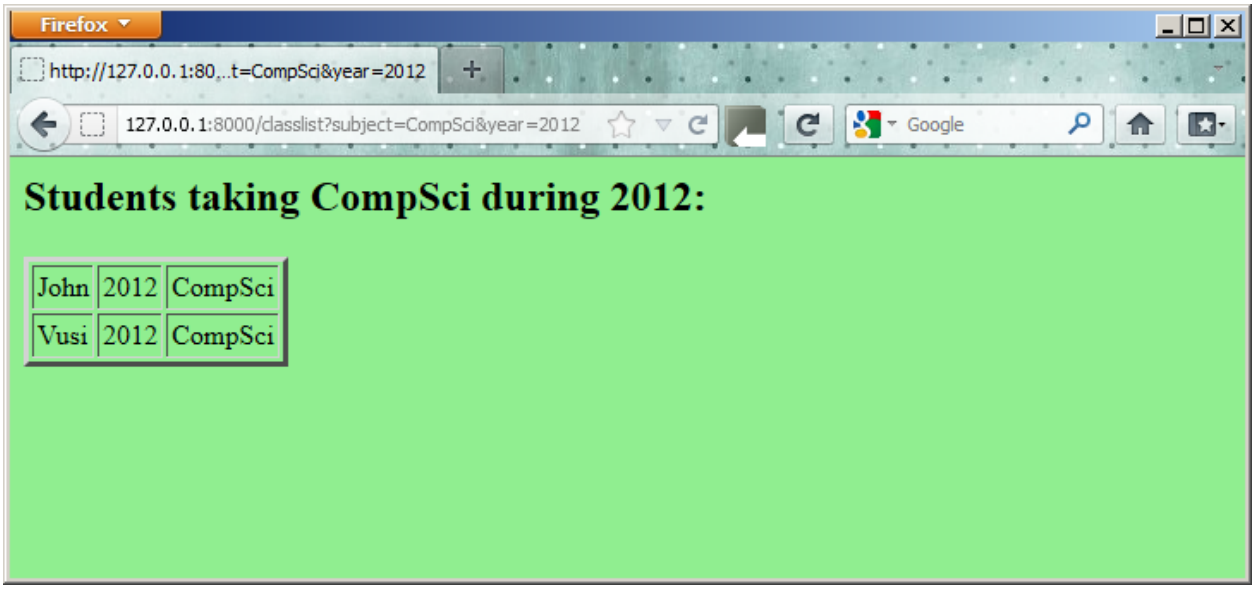
def classlist(env, resp):

    # Parse the field value from the query string (or from a submitted form)
    # In a real server you'd want to check thay they were present!
    the_fields = cgi.FieldStorage(environ = env)
    subj = the_fields["subject"].value
    year = the_fields["year"].value

    # Attach to the database, build a query, fetch the rows.
    connection = sqlite3.connect("c:\studentRecords.db")
    cursor = connection.cursor()
    cursor.execute("SELECT * FROM StudentSubjects WHERE subject=? AND year=?",
                                                                (subj, year))
    result = cursor.fetchall()
    # Build the html rows for the table
    table_rows = ""
    for (sn, yr, subj) in result:
        table_rows += "    <tr><td>{0}<td>{1}<td>{2}\n". format(sn, yr, subj)

    # Now plug the headings and data into the template, and complete the response
    response_body = classlistTemplate.format(subj, year, table_rows)
    response_headers = [("Content-Type", "text/html"),
            ("Content-Length", str(len(response_body)))]
    resp("200 OK", response_headers)
    return response_body
```

When we run this and navigate to http://127.0.0.1:8000/classlist?subject=CompSci&year=2012 with a browser, we'll get output like this:



It is unlikely that we would write our own web server from scratch. But the beauty of this approach is that it creates a great test environment for working with server-side applications that use the `wsgi` protocols. Once our code is ready, we can deploy it behind a web server like Apache which can interact with our handlers using `wsgi`.

Appendix C: Configuring Ubuntu for Python Development

Note: the following instructions assume that you are connected to the Internet and that you have both the `main` and `universe` package repositories enabled. All unix shell commands are assumed to be running from your home directory ($HOME). Finally, any command that begins with `sudo` assumes that you have administrative rights on your machine. If you do not — please ask your system administrator about installing the software you need.

What follows are instructions for setting up an Ubuntu 9.10 (Karmic) home environment for use with this book. I use Ubuntu GNU/Linux for both development and testing of the book, so it is the only system about which I can personally answer setup and configuration questions.

In the spirit of software freedom and open collaboration, please contact me if you would like to maintain a similar appendix for your own favorite system. I'd be more than happy to link to it or put it on the Open Book Project site, provided you agree to answer user feedback concerning it.

Thanks!

Jeffrey Elkner[24]
Governor's Career and Technical Academy in Arlington
Arlington, Virginia

C.1. Vim

Vim[25] can be used very effectively for Python development, but Ubuntu only comes with the vim-tiny package installed by default, so it doesn't support color syntax highlighting or auto-indenting.

To use Vim, do the following:

1. From the unix command prompt, run:

```
$ sudo apt-get install vim-gnome
```

2. Create a file in your home directory named .vimrc that contains the following:

```
syntax enable
filetype indent on
set et
```

[24] mailto:jeff@elkner.net
[25] http://www.vim.org

```
set sw=4
set smarttab
map <f2> :w|!python %
```

When you edit a file with a .py extension, you should now have color syntax highlighting and auto indenting. Pressing the key should run your program, and bring you back to the editor when the program completes.

To learn to use vim, run the following command at a unix command prompt:

```
$ vimtutor
```

C.2. $HOME environment

The following creates a useful environment in your home directory for adding your own Python libraries and executable scripts:

1. From the command prompt in your home directory, create bin and lib/python subdirectories by running the

following commands:

```
$ mkdir bin lib
$ mkdir lib/python
```

2. Add the following lines to the bottom of your .bashrc in your home directory:

```
PYTHONPATH=$HOME/lib/python
EDITOR=vim

export PYTHONPATH EDITOR
```

This will set your prefered editor to Vim, add your own lib/python subdirectory for your Python libraries to your Python path, and add your own bin directory as a place to put executable scripts. You need to logout and log back in before your local bin directory will be in your search path[26].

[26] http://en.wikipedia.org/wiki/Path_(variable)

C.3. Making a Python script executable and runnable from anywhere

On unix systems, Python scripts can be made *executable* using the following process:

1. Add this line as the first line in the script:

```
1    #!/usr/bin/env python3
```

2. At the unix command prompt, type the following to make myscript.py executable:

$ chmod +x myscript.py

3. Move myscript.py into your bin directory, and it will be runnable from anywhere.

Appendix D: Customizing and Contributing to the Book

Note: the following instructions assume that you are connected to the Internet and that you have both the `main` and `universe` package repositories enabled. All unix shell commands are assumed to be running from your home directory ($HOME). Finally, any command that begins with `sudo` assumes that you have administrative rights on your machine. If you do not — please ask your system administrator about installing the software you need.

This book is free as in freedom, which means you have the right to modify it to suite your needs, and to redistribute your modifications so that our whole community can benefit.

That freedom lacks meaning, however, if you the tools needed to make a custom version or to contribute corrections and additions are not within your reach. This appendix attempts to put those tools in your hands.

Thanks!

Jeffrey Elkner[27]
Governor's Career and Technical Academy in Arlington
Arlington, Virginia

D.1. Getting the Source

This book is marked up[28] in ReStructuredText[29] using a document generation system called Sphinx[30].

The source code is located at https://code.launchpad.net/~thinkcspy-rle-team/thinkcspy/thinkcspy3-rle.

The easiest way to get the source code on an Ubuntu computer is:

1. run `sudo apt-get install bzr` on your system to install

bzr[31].
2. run `bzr branch lp:thinkcspy`.

The last command above will download the book source from Launchpad into a directory named `thinkcspy` which contains the Sphinx source and configuration information needed to build the book.

[27] mailto:jeff@elkner.net
[28] http://en.wikipedia.org/wiki/Markup_language
[29] http://en.wikipedia.org/wiki/ReStructuredText
[30] http://en.wikipedia.org/wiki/Sphinx_%28documentation_generator%29
[31] http://en.wikipedia.org/wiki/Bazaar_%28software%29

D.2. Making the HTML Version

To generate the html version of the book:

1. run `sudo apt-get install python-sphinx` to install the Sphinx

documentation system.
2. `cd thinkcspy` - change into the `thinkcspy` directory containing

the book source.
3. `make html`.

The last command will run sphinx and create a directory named `build` containing the html version of the text.

Note: Sphinx supports building other output types as well, such as PDF[32]. This requires that LaTeX[33] be present on your system. Since I only personally use the html version, I will not attempt to document that process here.

[32] http://en.wikipedia.org/wiki/PDF
[33] http://en.wikipedia.org/wiki/LaTeX

Appendix E: Some Tips, Tricks, and Common Errors

These are small summaries of ideas, tips, and commonly seen errors that might be helpful to those beginning Python.

E.1. Functions

Functions help us with our mental chunking: they allow us to group together statements for a high-level purpose, e.g. a function to sort a list of items, a function to make the turtle draw a spiral, or a function to compute the mean and standard deviation of some measurements.

There are two kinds of functions: fruitful, or value-returning functions, which *calculate and return a value*, and we use them because we're primarily interested in the value they'll return. Void (non-fruitful) functions are used because they *perform actions* that we want done — e.g. make a turtle draw a rectangle, or print the first thousand prime numbers. They always return `None` — a special dummy value.

Tip: `None` is not a string

Values like `None`, `True` and `False` are not strings: they are special values in Python, and are in the list of keywords we gave in chapter 2 (Variables, expressions, and statements). Keywords are special in the language: they are part of the syntax. So we cannot create our own variable or function with a name `True` — we'll get a syntax error. (Built-in functions are not privileged like keywords: we can define our own variable or function called `len`, but we'd be silly to do so!)

Along with the fruitful/void families of functions, there are two flavors of the `return` statement in Python: one that returns a useful value, and the other that returns nothing, or `None`. And if we get to the end of any function and we have not explicitly executed any `return` statement, Python automatically returns the value `None`.

Tip: Understand what the function needs to return

Perhaps nothing — some functions exists purely to perform actions rather than to calculate and return a result. But if the function should return a value, make sure all execution paths do return the value.

To make functions more useful, they are given *parameters*. So a function to make a turtle draw a square might have two parameters — one for the turtle that needs to do the drawing, and another for the size of the square. See the first example in Chapter 4 (Functions) — that function can be used

with any turtle, and for any size square. So it is much more general than a function that always uses a specific turtle, say `tess` to draw a square of a specific size, say 30.

Tip: Use parameters to generalize functions

Understand which parts of the function will be hard-coded and unchangeable, and which parts should become parameters so that they can be customized by the caller of the function.

Tip: Try to relate Python functions to ideas we already know

In math, we're familiar with functions like `f(x) = 3x + 5`. We already understand that when we call the function `f(3)` we make some association between the parameter `x` and the argument 3. Try to draw parallels to argument passing in Python.

Quiz: Is the function `f(z) = 3z + 5` the same as function `f` above?

E.2. Problems with logic and flow of control

We often want to know if some condition holds for any item in a list, e.g. "does the list have any odd numbers?" This is a common mistake:

```
1  def any_odd(xs):  # Buggy version
2      """ Return True if there is an odd number in xs, a list of integers. """
3      for v in xs:
4          if v % 2 == 1:
5              return True
6          else:
7              return False
```

Can we spot two problems here? As soon as we execute a `return`, we'll leave the function. So the logic of saying "If I find an odd number I can return `True`" is fine. However, we cannot return `False` after only looking at one item — we can only return `False` if we've been through all the items, and none of them are odd. So line 6 should not be there, and line 7 has to be outside the loop. To find the second problem above, consider what happens if you call this function with an argument that is an empty list. Here is a corrected version:

```
1  def any_odd(xs):
2      """ Return True if there is an odd number in xs, a list of integers. """
3      for v in xs:
4          if v % 2 == 1:
5              return True
6      return False
```

This "eureka", or "short-circuit" style of returning from a function as soon as we are certain what the outcome will be was first seen in Section 8.10, in the chapter on strings.

It is preferred over this one, which also works correctly:

```
1  def any_odd(xs):
2      """ Return True if there is an odd number in xs, a list of integers. """
3      count = 0
4      for v in xs:
5          if v % 2 == 1:
6              count += 1    # Count the odd numbers
7      if count > 0:
8          return True
9      else:
10         return False
```

The performance disadvantage of this one is that it traverses the whole list, even if it knows the outcome very early on.

Tip: Think about the return conditions of the function

Do I need to look at all elements in all cases? Can I shortcut and take an early exit? Under what conditions? When will I have to examine all the items in the list?

The code in lines 7-10 can also be tightened up. The expression `count > 0` evaluates to a Boolean value, either `True` or `False`. The value can be used directly in the `return` statement. So we could cut out that code and simply have the following:

```
1  def any_odd(xs):
2      """ Return True if there is an odd number in xs, a list of integers. """
3      count = 0
4      for v in xs:
5          if v % 2 == 1:
6              count += 1    # Count the odd numbers
7      return count > 0    # Aha! a programmer who understands that Boolean
8                          #   expressions are not just used in if statements!
```

Although this code is tighter, it is not as nice as the one that did the short-circuit return as soon as the first odd number was found.

Tip: Generalize your use of Booleans

Mature programmers won't write `if is_prime(n) == True:` when they could say instead `if is_prime(n):` Think more generally about Boolean values, not just in the context of `if` or `while` statements. Like arithmetic expressions, they have their own set of operators (`and`, `or`, `not`) and values (`True`, `False`) and can be assigned to variables, put into lists, etc. A good resource for improving your use of Booleans is http://en.wikibooks.org/wiki/Non-Programmer%27s_Tutorial_for_Python_-3/Boolean_Expressions

Exercise time:

- How would we adapt this to make another function which returns

`True` if *all* the numbers are odd? Can you still use a short-circuit style?
- How would we adapt it to return `True` if at least three of the numbers are odd? Short-circuit the traversal when the third odd number is found — don't traverse the whole list unless we have to.

E.3. Local variables

Functions are called, or activated, and while they're busy they create their own stack frame which holds local variables. A local variable is one that belongs to the current activation. As soon as the function returns (whether from an explicit return statement or because Python reached the last statement), the stack frame and its local variables are all destroyed. The important consequence of this is that a function cannot use its own variables to remember any kind of state between different activations. It cannot count how many times it has been called, or remember to switch colors between red and blue UNLESS it makes use of variables that are global. Global variables will survive even after our function has exited, so they are the correct way to maintain information between calls.

```
1  sz = 2
2  def h2():
3      """ Draw the next step of a spiral on each call. """
4      global sz
5      tess.turn(42)
6      tess.forward(sz)
7      sz += 1
```

This fragment assumes our turtle is `tess`. Each time we call `h2()` it turns, draws, and increases the global variable `sz`. Python always assumes that an assignment to a variable (as in line 7) means that we want a new local variable, unless we've provided a `global` declaration (on line 4). So leaving out the global declaration means this does not work.

Tip: Local variables do not survive when you exit the function

Use a Python visualizer like the one at http://pythontutor.com/ to build a strong understanding of function calls, stack frames, local variables, and function returns.

Tip: Assignment in a function creates a local variable

Any assignment to a variable within a function means Python will make a local variable, unless we override with `global`.

E.4. Event handler functions

Our chapter on event handling showed three different kinds of events that we could handle. They each have their own subtle points that can trip us up.

- Event handlers are void functions — they don't return any values.
- They're automatically called by the Python interpreter in response to an event, so we don't get to see the code that calls them.
- A mouse-click event passes two coordinate arguments to its handler, so when we write this handler we have to provide for two parameters (usually named `x` and `y`). This is how the handler knows where the mouse click occurred.
- A keypress event handler has to be bound to the key it responds to. There is a messy extra step when using keypresses: we have to remember to issue a `wn.listen()` before our program will receive any keypresses. But if the user presses the key 10 times, the handler will be called ten times.
- Using a timer to create a future-dated event only causes one call to the handler. If we want repeated periodic handler activations, then from within the handler we call `wn.ontimer(....)` to set up the next event.

E.5. String handling

There are only four *really* important operations on strings, and we'll be able to do just about anything. There are many more nice-to-have methods (we'll call them sugar coating) that can make life easier, but if we can work with the basic four operations smoothly, we'll have a great grounding.

- len(str) finds the length of a string.
- str[i] the subscript operation extracts the i'th character of the string, as a new string.
- str[i:j] the slice operation extracts a substring out of a string.
- str.find(target) returns the index where target occurs within the string, or -1 if it is not found.

So if we need to know if "snake" occurs as a substring within `s`, we could write

```
if s.find("snake") >= 0:  ...
if "snake" in s: ...           # Also works, nice-to-know sugar coating!
```

It would be wrong to split the string into words unless we were asked whether the *word* "snake" occurred in the string.

Suppose we're asked to read some lines of data and find function definitions, e.g.: `def some_function_name(x, y):`, and we are further asked to isolate and work with the name of the function. (Let's say, print it.)

```
s = "..."                       # Get the next line from somewhere
def_pos = s.find("def ")        # Look for "def " in the line
if def_pos == 0:                # If it occurs at the left margin
  op_index = s.find("(")        # Find the index of the open parenthesis
  fnname = s[4:op_index]        # Slice out the function name
  print(fnname)                 # ... and work with it.
```

One can extend these ideas:

- What if the function def was indented, and didn't start at column 0?

The code would need a bit of adjustment, and we'd probably want to be sure that all the characters in front of the `def_pos` position were spaces. We would not want to do the wrong thing on data like this: `# I def initely like Python!`

- We've assumed on line 3 that we will find an open parenthesis. It

may need to be checked that we did!

- We have also assumed that there was exactly one space between the

keyword `def` and the start of the function name. It will not work nicely for `def f(x)`

As we've already mentioned, there are many more "sugar-coated" methods that let us work more easily with strings. There is an `rfind` method, like `find`, that searches from the end of the string backwards. It is useful if we want to find the last occurrence of something. The `lower` and `upper` methods can do case conversion. And the `split` method is great for breaking a string into a list of words, or into a list of lines. We've also made extensive use in this book of the `format` method. In fact, if we want to practice reading the Python documentation and learning some new methods on our own, the string methods are an excellent resource.

Exercises:

- Suppose any line of text can contain at most one url that starts

with "http://" and ends at the next space in the line. Write a fragment of code to extract and print the full url if it is present.

(Hint: read the documentation for `find`. It takes some extra arguments, so you can set a starting point from which it will search.)
- Suppose a string contains at most one substring "< ... >".

Write a fragment of code to extract and print the portion of the string between the angle brackets.

E.6. Looping and lists

Computers are useful because they can repeat computation, accurately and fast. So loops are going to be a central feature of almost all programs you encounter.

Tip: Don't create unnecessary lists

Lists are useful if you need to keep data for later computation. But if you don't need lists, it is probably better not to generate them.

Here are two functions that both generate ten million random numbers, and return the sum of the numbers. They both work.

```
import random
joe = random.Random()

def sum1():
    """ Build a list of random numbers, then sum them """
    xs = []
    for i in range(10000000):
        num = joe.randrange(1000)  # Generate one random number
        xs.append(num)             # Save it in our list

    tot = sum(xs)
    return tot

def sum2():
    """ Sum the random numbers as we generate them """
    tot = 0
    for i in range(10000000):
        num = joe.randrange(1000)
        tot += num
    return tot

print(sum1())
print(sum2())
```

What reasons are there for preferring the second version here? (Hint: open a tool like the Performance Monitor on your computer, and watch the memory usage. How big can you make the list before you get a fatal memory error in `sum1`?)

In a similar way, when working with files, we often have an option to read the whole file contents into a single string, or we can read one line at a time and process each line as we read it. Line-at-a-time is the more traditional and perhaps safer way to do things — you'll be able to work comfortably no matter how large the file is. (And, of course, this mode of processing the files was essential in the old days when computer memories were much smaller.) But you may find whole-file-at-once is sometimes more convenient!

Unix/Linux Command Reference

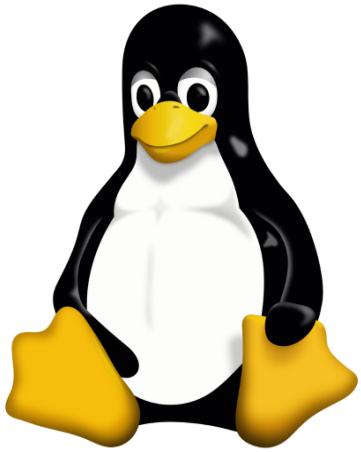
File Commands		
1.	ls	Directory listing
2.	ls -al	Formatted listing with hidden files
3.	ls -lt	Sorting the Formatted listing by time modification
4.	cd dir	Change directory to dir
5.	cd	Change to home directory
6.	pwd	Show current working directory
7.	mkdir dir	Creating a directory dir
8.	cat >file	Places the standard input into the file
9.	more file	Output the contents of the file
10.	head file	Output the first 10 lines of the file
11.	tail file	Output the last 10 lines of the file
12.	tail -f file	Output the contents of file as it grows,starting with the last 10 lines
13.	touch file	Create or update file
14.	rm file	Deleting the file
15.	rm -r dir	Deleting the directory
16.	rm -f file	Force to remove the file
17.	rm -rf dir	Force to remove the directory dir
18.	cp file1 file2	Copy the contents of file1 to file2
19.	cp -r dir1 dir2	Copy dir1 to dir2;create dir2 if not present
20.	mv file1 file2	Rename or move file1 to file2,if file2 is an existing directory
21.	ln -s file link	Create symbolic link link to file
Process management		
1.	ps	To display the currently working processes
2.	top	Display all running process

3.	kill pid	Kill the process with given pid
4.	killall proc	Kill all the process named proc
5.	pkill pattern	Will kill all processes matching the pattern
6.	bg	List stopped or background jobs,resume a stopped job in the background
7.	fg	Brings the most recent job to foreground
8.	fg n	Brings job n to the foreground
File permission		
1.	chmod octal file	Change the permission of file to octal,which can be found separately for user,group,world by adding, • 4-read(r) • 2-write(w) • 1-execute(x)
Searching		
1.	grep pattern file	Search for pattern in file
2.	grep -r pattern dir	Search recursively for pattern in dir
3.	command | grep pattern	Search pattern in the output of a command
4.	locate file	Find all instances of file
5.	find . -name filename	Searches in the current directory (represented by a period) and below it, for files and directories with names starting with filename
6.	pgrep pattern	Searches for all the named processes , that matches with the pattern and, by default, returns their ID
System Info		
1.	date	Show the current date and time
2.	cal	Show this month's calender
3.	uptime	Show current uptime
4.	w	Display who is on line
5.	whoami	Who you are logged in as

6.	finger user	Display information about user
7.	uname -a	Show kernel information
8.	cat /proc/cpuinfo	Cpu information
9.	cat proc/meminfo	Memory information
10.	man command	Show the manual for command
11.	df	Show the disk usage
12.	du	Show directory space usage
13.	free	Show memory and swap usage
14.	whereis app	Show possible locations of app
15.	which app	Show which applications will be run by default
Compression		
1.	tar cf file.tar file	Create tar named file.tar containing file
2.	tar xf file.tar	Extract the files from file.tar
3.	tar czf file.tar.gz files	Create a tar with Gzip compression
4.	tar xzf file.tar.gz	Extract a tar using Gzip
5.	tar cjf file.tar.bz2	Create tar with Bzip2 compression
6.	tar xjf file.tar.bz2	Extract a tar using Bzip2
7.	gzip file	Compresses file and renames it to file.gz
8.	gzip -d file.gz	Decompresses file.gz back to file
Network		
1.	ping host	Ping host and output results
2.	whois domain	Get whois information for domains
3.	dig domain	Get DNS information for domain
4.	dig -x host	Reverse lookup host
5.	wget file	Download file
6.	wget -c file	Continue a stopped download

Shortcuts		
1.	ctrl+c	Halts the current command
2.	ctrl+z	Stops the current command, resume with fg in the foreground or bg in the background
3.	ctrl+d	Logout the current session, similar to exit
4.	ctrl+w	Erases one word in the current line
5.	ctrl+u	Erases the whole line
6.	ctrl+r	Type to bring up a recent command
7.	!!	Repeats the last command
8.	exit	Logout the current session

THE LINUX



COMMANDS HANDBOOK

Flavio Copes

Table of Contents

Preface

Introduction to Linux and shells

man

ls

cd

pwd

mkdir

rmdir

mv

cp

open

touch

find

ln

gzip

gunzip

tar

alias

cat

less

tail

wc

grep

sort

uniq

diff

echo

chown

chmod

umask

du

df

basename

dirname

ps

top

kill

killall

jobs

bg

fg

type

which

nohup

xargs

vim

emacs

nano

whoami

who

su

sudo

passwd
ping
traceroute
clear
history
export
crontab
uname
env
printenv
Conclusion

Preface

The Linux Commands Handbook follows the 80/20 rule: learn in 20% of the time the 80% of a topic.

I find this approach gives a well-rounded overview.

This book does not try to cover everything under the sun related to Linux and its commands. It focuses on the small core commands that you will use the 80% or 90% of the time, trying to simplify the usage of the more complex ones.

All those commands work on Linux, macOS, WSL, and anywhere you have a UNIX environment.

I hope the contents of this book will help you achieve what you want: **get comfortable with Linux**.

This book is written by Flavio. I **publish programming tutorials** every day on my website flaviocopes.com.

You can reach me on Twitter @flaviocopes.

Enjoy!

Introduction to Linux and shells

Linux is an operating system, like macOS or Windows.

It is also the most popular Open Source and free, as in freedom, operating system.

It powers the vast majority of the servers that compose the Internet. It's the base upon which everything is built upon. But not just that. Android is based on (a modified version of) Linux.

The Linux "core" (called *kernel*) was born in 1991 in Finland, and it went a really long way from its humble beginnings. It went on to be the kernel of the GNU Operating System, creating the duo GNU/Linux.

There's one thing about Linux that corporations like Microsoft and Apple, or Google, will never be able to offer: the freedom to do whatever you want with your computer.

They're actually going in the opposite direction, building walled gardens, especially on the mobile side.

Linux is the ultimate freedom.

It is developed by volunteers, some paid by companies that rely on it, some independently, but there's no single commercial company that can dictate what goes into Linux, or the project priorities.

Linux can also be used as your day to day computer. I use macOS because I really enjoy the applications, the design and I also used to be an iOS and Mac apps developer, but before using it I used Linux as my main computer Operating System.

No one can dictate which apps you can run, or "call home" with apps that track you, your position, and more.

Linux is also special because there's not just "one Linux", like it happens on Windows or macOS. Instead, we have **distributions**.

A "distro" is made by a company or organization and packages the Linux core with additional programs and tooling.

For example you have Debian, Red Hat, and Ubuntu, probably the most popular.

Many, many more exist. You can create your own distribution, too. But most likely you'll use a popular one, one that has lots of users and a community of people around it, so you can do what you need to do without losing too much time reinventing the wheel and figuring out answers to common problems.

Some desktop computers and laptops ship with Linux preinstalled. Or you can install it on your Windows-based computer, or on a Mac.

But you don't need to disrupt your existing computer just to get an idea of how Linux works.

I don't have a Linux computer.

If you use a Mac you need to know that under the hood macOS is a UNIX Operating System, and it shares a lot of the same ideas and software that a GNU/Linux system uses, because GNU/Linux is a free alternative to UNIX.

> UNIX is an umbrella term that groups many operating systems used in big corporations and institutions, starting from the 70's

The macOS terminal gives you access to the same exact commands I'll describe in the rest of this handbook.

Microsoft has an official Windows Subsystem for Linux which you can (and should!) install on Windows. This will give you the ability to run Linux in a very easy way on your PC.

But the vast majority of the time you will run a Linux computer in the cloud via a VPS (Virtual Private Server) like DigitalOcean.

A shell is a command interpreter that exposes to the user an interface to work with the underlying operating system.

It allows you to execute operations using text and commands, and it provides users advanced features like being able to create scripts.

This is important: shells let you perform things in a more optimized way than a GUI (Graphical User Interface) could ever possibly let you do. Command line tools can offer many different configuration options without being too complex to use.

There are many different kind of shells. This post focuses on Unix shells, the ones that you will find commonly on Linux and macOS computers.

Many different kind of shells were created for those systems over time, and a few of them dominate the space: Bash, Csh, Zsh, Fish and many more!

All shells originate from the Bourne Shell, called `sh`. "Bourne" because its creator was Steve Bourne.

Bash means *Bourne-again shell*. `sh` was proprietary and not open source, and Bash was created in 1989 to create a free alternative for the GNU project and the Free Software Foundation. Since projects had to pay to use the Bourne shell, Bash became very popular.

If you use a Mac, try opening your Mac terminal. That by default is running ZSH. (or, pre-Catalina, Bash)

You can set up your system to run any kind of shell, for example I use the Fish shell.

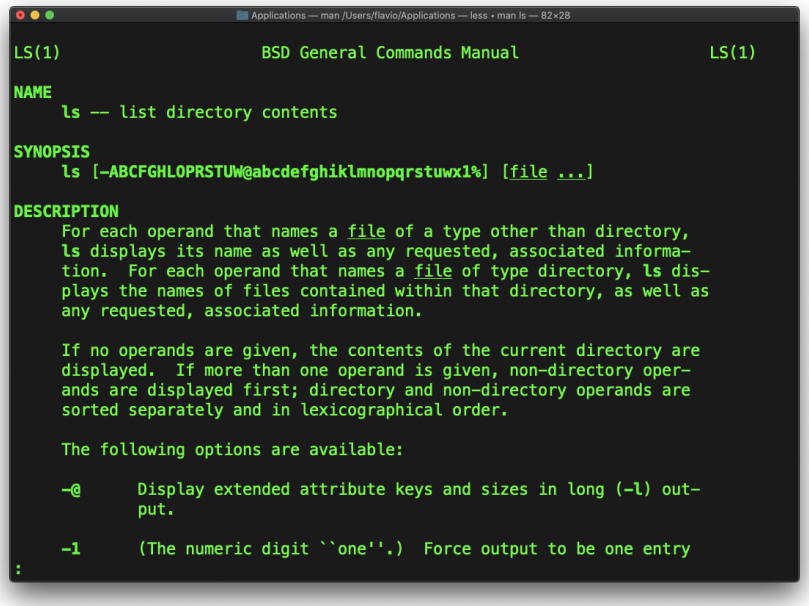
Each single shell has its own unique features and advanced usage, but they all share a common functionality: they can let you execute programs, and they can be programmed.

In the rest of this handbook we'll see in detail the most common commands you will use.

man

The first command I want to introduce is a command that will help you understand all the other commands.

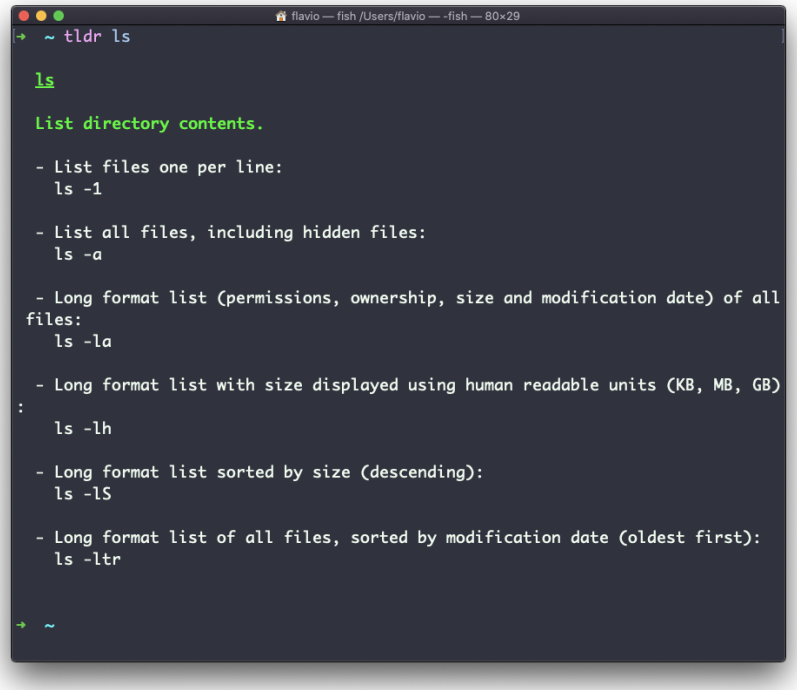
Every time I don't know how to use a command, I type `man <command>` to get the manual:



This is a man (from *manual*) page. Man pages are an essential tool to learn, as a developer. They contain so much information that sometimes it's almost too much.

The above screenshot is just 1 of 14 screens of explanation for the `ls` command.

Most of the times when I'm in need to learn a command quickly I use this site called **tldr pages**: https://tldr.sh/. It's a command you can install, then you run it like this: `tldr <command>`, which gives you a very quick overview of a command, with some handy examples of common usage scenarios:



This is not a substitute for `man`, but a handy tool to avoid losing yourself in the huge amount of information present in a man page. Then you can use the man page to explore all the different options and parameters you can use on a command.

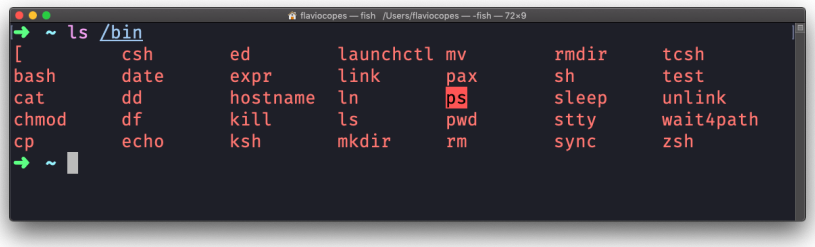
ls

Inside a folder you can list all the files that the folder contains using the `ls` command:

```
ls
```

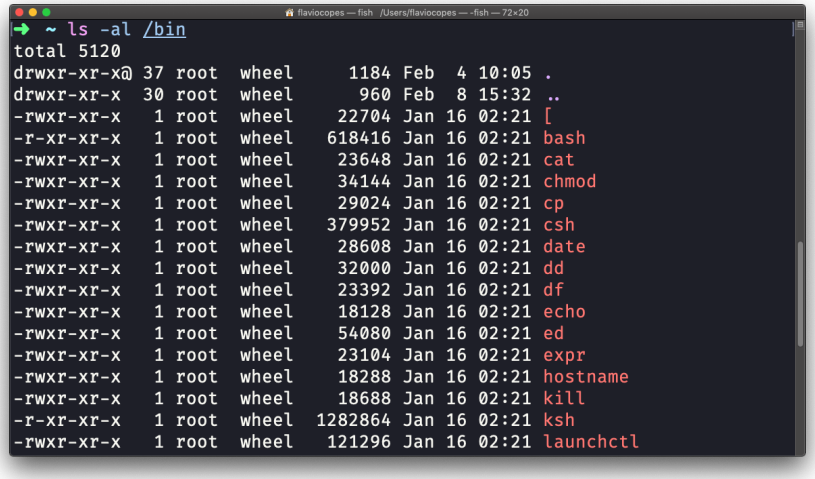
If you add a folder name or path, it will print that folder contents:

```
ls /bin
```



`ls` accepts a lot of options. One of my favorite options combinations is `-al`. Try it:

```
ls -al /bin
```



compared to the plain `ls`, this returns much more information.

You have, from left to right:

- the file permissions (and if your system supports ACLs, you get an ACL flag as well)
- the number of links to that file
- the owner of the file
- the group of the file
- the file size in bytes
- the file modified datetime
- the file name

This set of data is generated by the `l` option. The `a` option instead also shows the hidden files.

Hidden files are files that start with a dot (`.`).

cd

Once you have a folder, you can move into it using the `cd` command. `cd` means **c**hange **d**irectory. You invoke it specifying a folder to move into. You can specify a folder name, or an entire path.

Example:

```
mkdir fruits
cd fruits
```

Now you are into the `fruits` folder.

You can use the `..` special path to indicate the parent folder:

```
cd .. #back to the home folder
```

The # character indicates the start of the comment, which lasts for the entire line after it's found.

You can use it to form a path:

```
mkdir fruits
mkdir cars
cd fruits
cd ../cars
```

There is another special path indicator which is `.`, and indicates the **current** folder.

You can also use absolute paths, which start from the root folder `/`:

```
cd /etc
```

> This command works on Linux, macOS, WSL, and anywhere you have a UNIX environment

pwd

Whenever you feel lost in the filesystem, call the `pwd` command to know where you are:

```
pwd
```

It will print the current folder path.

mkdir

You create folders using the `mkdir` command:

```
mkdir fruits
```

You can create multiple folders with one command:

```
mkdir dogs cars
```

You can also create multiple nested folders by adding the `-p` option:

```
mkdir -p fruits/apples
```

Options in UNIX commands commonly take this form. You add them right after the command name, and they change how the command behaves. You can often combine multiple options, too.

You can find which options a command supports by typing `man <commandname>`. Try now with `man mkdir` for example (press the `q` key to esc the man page). Man pages are the amazing built-in help for UNIX.

rmdir

Just as you can create a folder using `mkdir`, you can delete a folder using `rmdir`:

```
mkdir fruits
rmdir fruits
```

You can also delete multiple folders at once:

```
mkdir fruits cars
rmdir fruits cars
```

The folder you delete must be empty.

To delete folders with files in them, we'll use the more generic `rm` command which deletes files and folders, using the `-rf` options:

```
rm -rf fruits cars
```

Be careful as this command does not ask for confirmation and it will immediately remove anything you ask it to remove.

There is no **bin** when removing files from the command line, and recovering lost files can be hard.

mv

Once you have a file, you can move it around using the `mv` command. You specify the file current path, and its new path:

```
touch test
mv pear new_pear
```

The `pear` file is now moved to `new_pear`. This is how you **rename** files and folders.

If the last parameter is a folder, the file located at the first parameter path is going to be moved into that folder. In this case, you can specify a list of files and they will all be moved in the folder path identified by the last parameter:

```
touch pear
touch apple
mkdir fruits
mv pear apple fruits #pear and apple moved to the fr
```

cp

You can copy a file using the `cp` command:

```
touch test
cp apple another_apple
```

To copy folders you need to add the `-r` option to recursively copy the whole folder contents:

```
mkdir fruits
cp -r fruits cars
```

open

The `open` command lets you open a file using this syntax:

```
open <filename>
```

You can also open a directory, which on macOS opens the Finder app with the current directory open:

```
open <directory name>
```

I use it all the time to open the current directory:

```
open .
```

> The special `.` symbol points to the current directory, as `..` points to the parent directory

The same command can also be be used to run an application:

```
open <application name>
```

touch

You can create an empty file using the `touch` command:

```
touch apple
```

If the file already exists, it opens the file in write mode, and the timestamp of the file is updated.

find

The `find` command can be used to find files or folders matching a particular search pattern. It searches recursively.

Let's learn it by example.

Find all the files under the current tree that have the `.js` extension and print the relative path of each file matching:

```
find . -name '*.js'
```

It's important to use quotes around special characters like `*` to avoid the shell interpreting them.

Find directories under the current tree matching the name "src":

```
find . -type d -name src
```

Use `-type f` to search only files, or `-type l` to only search symbolic links.

`-name` is case sensitive. use `-iname` to perform a case-insensitive search.

You can search under multiple root trees:

```
find folder1 folder2 -name filename.txt
```

Find directories under the current tree matching the name "node_modules" or 'public':

```
find . -type d -name node_modules -or -name public
```

You can also exclude a path, using `-not -path`:

```
find . -type d -name '*.md' -not -path 'node_modules
```

You can search files that have more than 100 characters (bytes) in them:

```
find . -type f -size +100c
```

Search files bigger than 100KB but smaller than 1MB:

```
find . -type f -size +100k -size -1M
```

Search files edited more than 3 days ago

```
find . -type f -mtime +3
```

Search files edited in the last 24 hours

```
find . -type f -mtime -1
```

You can delete all the files matching a search by adding the `-delete` option. This deletes all the files edited in the last 24 hours:

```
find . -type f -mtime -1 -delete
```

You can execute a command on each result of the search. In this example we run `cat` to print the file content:

```
find . -type f -exec cat {} \;
```

notice the terminating `\;` . `{}` is filled with the file name at execution time.

ln

The `ln` command is part of the Linux file system commands.

It's used to create links. What is a link? It's like a pointer to another file. A file that points to another file. You might be familiar with Windows shortcuts. They're similar.

We have 2 types of links: **hard links** and **soft links**.

Hard links

Hard links are rarely used. They have a few limitations: you can't link to directories, and you can't link to external filesystems (disks).

A hard link is created using

```
ln <original> <link>
```

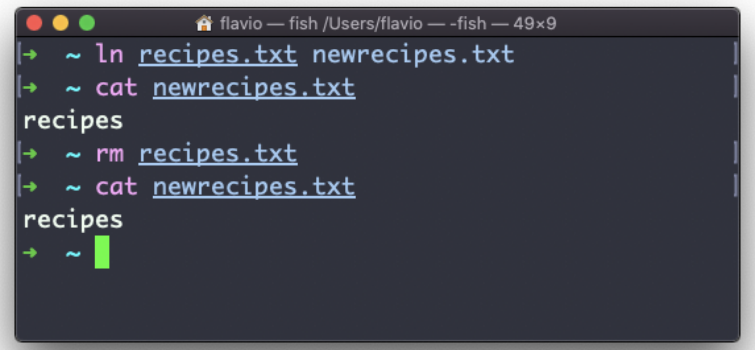
For example, say you have a file called recipes.txt. You can create a hard link to it using:

```
ln recipes.txt newrecipes.txt
```

The new hard link you created is indistinguishable from a regular file:

Now any time you edit any of those files, the content will be updated for both.

If you delete the original file, the link will still contain the original file content, as that's not removed until there is one hard link pointing to it.



Soft links

Soft links are different. They are more powerful as you can link to other filesystems and to directories, but when the original is removed, the link will be broken.

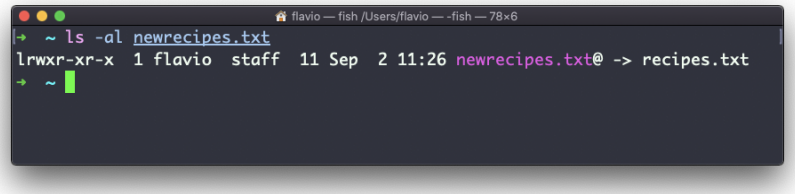
You create soft links using the `-s` option of `ln`:

```
ln -s <original> <link>
```

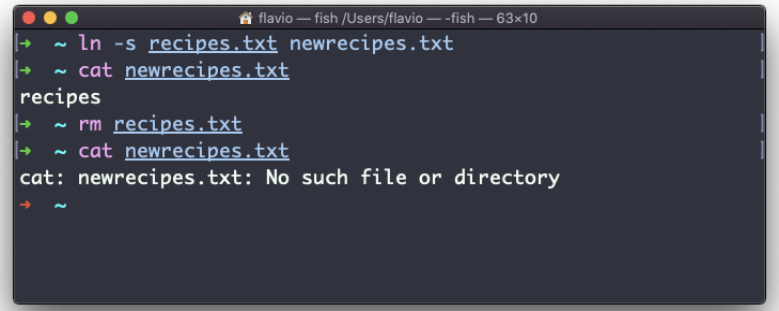
For example, say you have a file called recipes.txt. You can create a soft link to it using:

```
ln -s recipes.txt newrecipes.txt
```

In this case you can see there's a special `l` flag when you list the file using `ls -al`, and the file name has a `@` at the end, and it's colored differently if you have colors enabled:



Now if you delete the original file, the links will be broken, and the shell will tell you "No such file or directory" if you try to access it:



gzip

You can compress a file using the gzip compression protocol named LZ77 using the `gzip` command.

Here's the simplest usage:

```
gzip filename
```

This will compress the file, and append a `.gz` extension to it. The original file is deleted. To prevent this, you can use the `-c` option and use output redirection to write the output to the `filename.gz` file:

```
gzip -c filename > filename.gz
```

> The `-c` option specifies that output will go to the standard output stream, leaving the original file intact

Or you can use the `-k` option:

```
gzip -k filename
```

There are various levels of compression. The more the compression, the longer it will take to compress (and decompress). Levels range from 1 (fastest, worst compression) to 9 (slowest, better compression), and the default is 6.

You can choose a specific level with the `-<NUMBER>` option:

```
gzip -1 filename
```

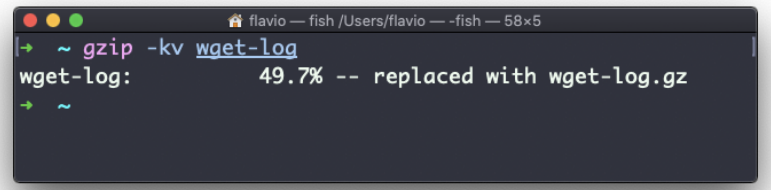
You can compress multiple files by listing them:

```
gzip filename1 filename2
```

You can compress all the files in a directory, recursively, using the `-r` option:

```
gzip -r a_folder
```

The `-v` option prints the compression percentage information. Here's an example of it being used along with the `-k` (keep) option:



`gzip` can also be used to decompress a file, using the `-d` option:

```
gzip -d filename.gz
```

gunzip

The `gunzip` command is basically equivalent to the `gzip` command, except the `-d` option is always enabled by default.

The command can be invoked in this way:

```
gunzip filename.gz
```

This will gunzip and will remove the `.gz` extension, putting the result in the `filename` file. If that file exists, it will overwrite that.

You can extract to a different filename using output redirection using the `-c` option:

```
gunzip -c filename.gz > anotherfilename
```

tar

The `tar` command is used to create an archive, grouping multiple files in a single file.

Its name comes from the past and means *tape archive*. Back when archives were stored on tapes.

This command creates an archive named `archive.tar` with the content of `file1` and `file2`:

```
tar -cf archive.tar file1 file2
```

> The `c` option stands for *create*. The `f` option is used to write to file the archive.

To extract files from an archive in the current folder, use:

```
tar -xf archive.tar
```

> the `x` option stands for *extract*

and to extract them to a specific directory, use:

```
tar -xf archive.tar -C directory
```

You can also just list the files contained in an archive:

`tar` is often used to create a **compressed archive**, gzipping the archive.

This is done using the `z` option:

```
tar -czf archive.tar.gz file1 file2
```

This is just like creating a tar archive, and then running `gzip` on it.

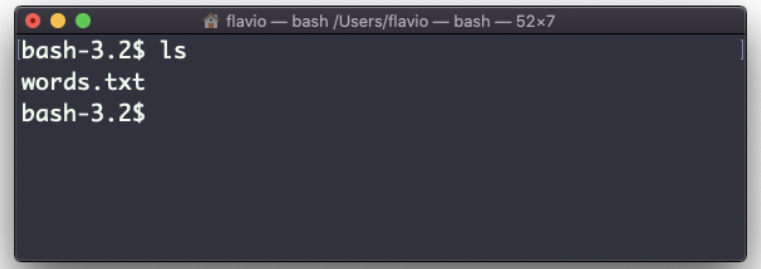
To unarchive a gzipped archive, you can use `gunzip`, or `gzip -d`, and then unarchive it, but `tar -xf` will recognize it's a gzipped archive, and do it for you:

```
tar -xf archive.tar.gz
```

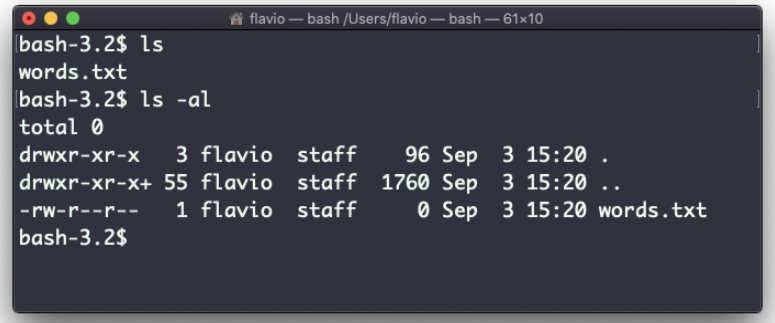
alias

It's common to always run a program with a set of options you like using.

For example, take the `ls` command. By default it prints very little information:



while using the `-al` option it will print something more useful, including the file modification date, the size, the owner, and the permissions, also listing hidden files (files starting with a `.`:

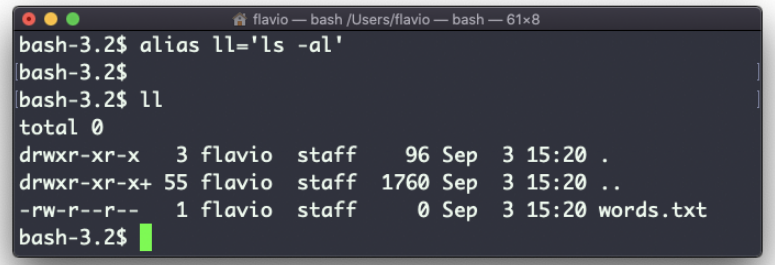


You can create a new command, for example I like to call it `ll`, that is an alias to `ls -al`.

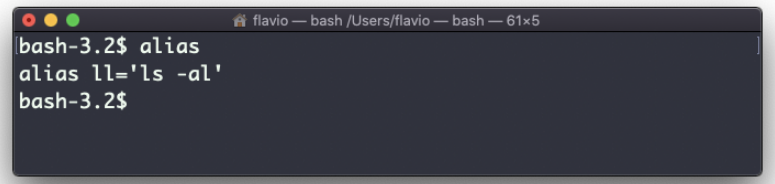
You do it in this way:

```
alias ll='ls -al'
```

Once you do, you can call `ll` just like it was a regular UNIX command:



Now calling `alias` without any option will list the aliases defined:



The alias will work until the terminal session is closed.

To make it permanent, you need to add it to the shell configuration, which could be `~/.bashrc` or `~/.profile` or `~/.bash_profile` if you use the Bash shell, depending on the use case.

Be careful with quotes if you have variables in the command: using double quotes the variable is resolved at definition time, using single quotes it's resolved at invocation time. Those 2 are different:

```
alias lsthis="ls $PWD"
alias lscurrent='ls $PWD'
```

$PWD refers to the current folder the shell is into. If you now navigate away to a new folder, `lscurrent` lists the files in the new folder, `lsthis` still lists the files in the folder you were when you defined the alias.

cat

Similar to `tail` in some way, we have `cat`. Except `cat` can also add content to a file, and this makes it super powerful.

In its simplest usage, `cat` prints a file's content to the standard output:

```
cat file
```

You can print the content of multiple files:

```
cat file1 file2
```

and using the output redirection operator `>` you can concatenate the content of multiple files into a new file:

```
cat file1 file2 > file3
```

Using `>>` you can append the content of multiple files into a new file, creating it if it does not exist:

```
cat file1 file2 >> file3
```

When watching source code files it's great to see the line numbers, and you can have `cat` print them using the `-n` option:

```
cat -n file1
```

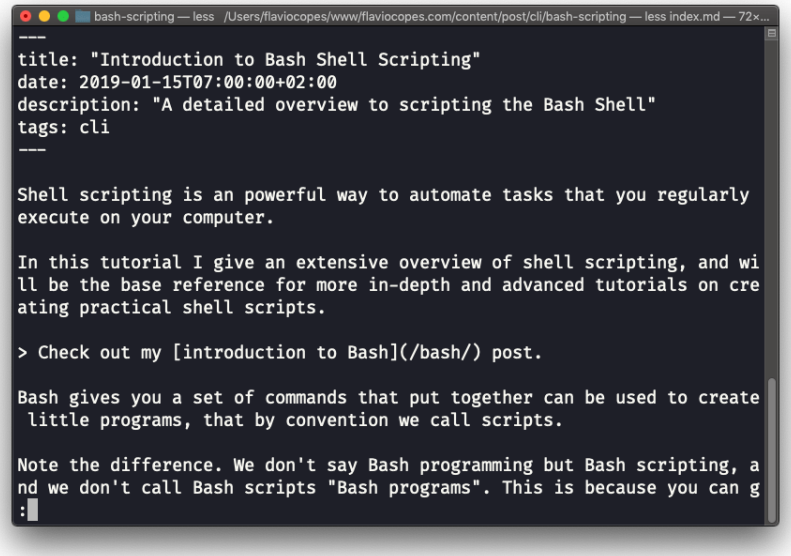
You can only add a number to non-blank lines using `-b`, or you can also remove all the multiple empty lines using `-s`.

`cat` is often used in combination with the pipe operator `|` to feed a file content as input to another command: `cat file1 | anothercommand`.

less

The `less` command is one I use a lot. It shows you the content stored inside a file, in a nice and interactive UI.

Usage: `less <filename>`.



Once you are inside a `less` session, you can quit by pressing `q`.

You can navigate the file contents using the `up` and `down` keys, or using the `space bar` and `b` to navigate page by page. You can also jump to the end of the file pressing `G` and jump back to the start pressing `g`.

You can search contents inside the file by pressing `/` and typing a word to search. This searches *forward*. You can search backwards using the `?` symbol and typing a word.

This command just visualises the file's content. You can directly open an editor by pressing `v`. It will use the system editor, which in most cases is `vim`.

Pressing the `F` key enters *follow mode*, or *watch mode*. When the file is changed by someone else, like from another program, you get to see the changes *live*. By default this is not happening, and you only see the file version at the time you opened it. You need to press `ctrl-C` to quit this mode. In this case the behaviour is similar to running the `tail -f <filename>` command.

You can open multiple files, and navigate through them using `:n` (to go to the next file) and `:p` (to go to the previous).

tail

The best use case of tail in my opinion is when called with the `-f` option. It opens the file at the end, and watches for file changes. Any time there is new content in the file, it is printed in the window. This is great for watching log files, for example:

```
tail -f /var/log/system.log
```

To exit, press `ctrl-C`.

You can print the last 10 lines in a file:

```
tail -n 10 <filename>
```

You can print the whole file content starting from a specific line using `+` before the line number:

```
tail -n +10 <filename>
```

`tail` can do much more and as always my advice is to check `man tail`.

wc

The `wc` command gives us useful information about a file or input it receives via pipes.

```
echo test >> test.txt
wc test.txt
1       1       5 test.txt
```

Example via pipes, we can count the output of running the `ls -al` command:

```
ls -al | wc
6      47     284
```

The first column returned is the number of lines. The second is the number of words. The third is the number of bytes.

We can tell it to just count the lines:

```
wc -l test.txt
```

or just the words:

```
wc -w test.txt
```

or just the bytes:

```
wc -c test.txt
```

Bytes in ASCII charsets equate to characters, but with non-ASCII charsets, the number of characters might differ because some characters might take multiple bytes, for example this happens in Unicode.

In this case the `-m` flag will help getting the correct value:

```
wc -m test.txt
```

grep

The `grep` command is a very useful tool, that when you master will help you tremendously in your day to day.

> If you're wondering, `grep` stands for *global regular expression print*

You can use `grep` to search in files, or combine it with pipes to filter the output of another command.

For example here's how we can find the occurences of the `document.getElementById` line in the `index.md` file:

```
grep document.getElementById index.md
```

Using the `-n` option it will show the line numbers:

```
grep -n document.getElementById index.md
```

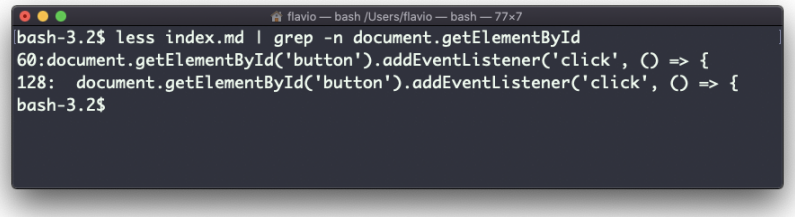
One very useful thing is to tell grep to print 2 lines before, and 2 lines after the matched line, to give us more context. That's done using the `-C` option, which accepts a number of lines:

```
grep -nC 2 document.getElementById index.md
```

Search is case sensitive by default. Use the `-i` flag to make it insensitive.

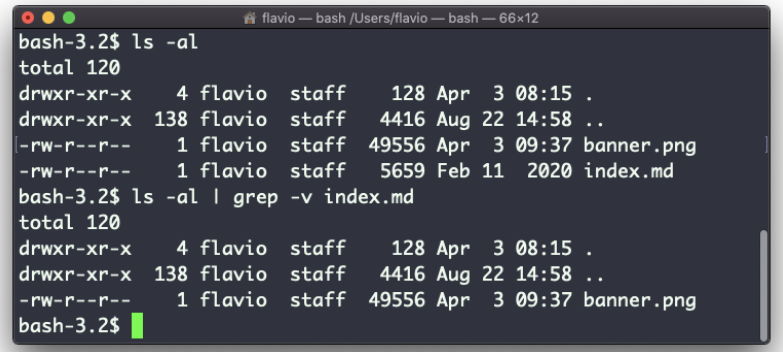
As mentioned, you can use grep to filter the output of another command. We can replicate the same functionality as above using:

```
less index.md | grep -n document.getElementById
```



The search string can be a regular expression, and this makes `grep` very powerful.

Another thing you might find very useful is to invert the result, excluding the lines that match a particular string, using the `-v` option:

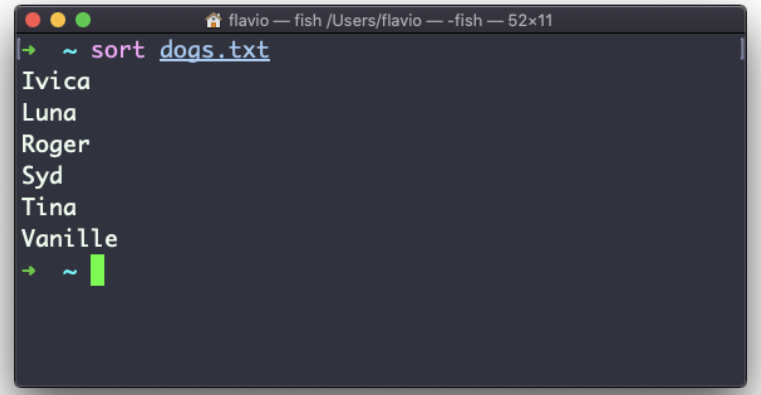


sort

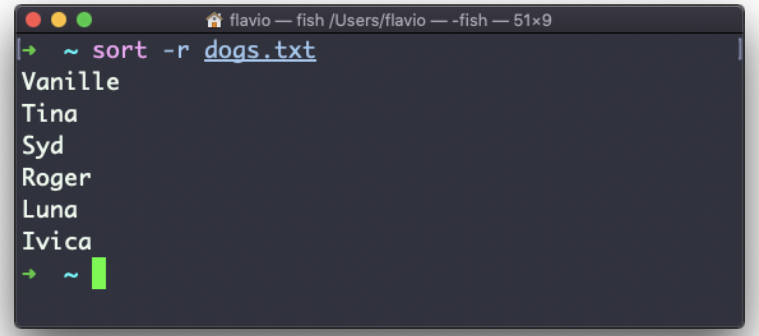
Suppose you have a text file which contains the names of dogs:

This list is unordered.

The `sort` command helps us sorting them by name:

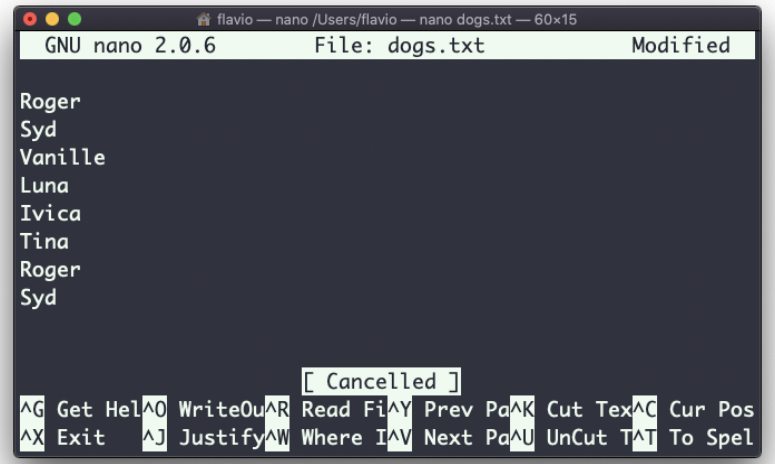


Use the `r` option to reverse the order:

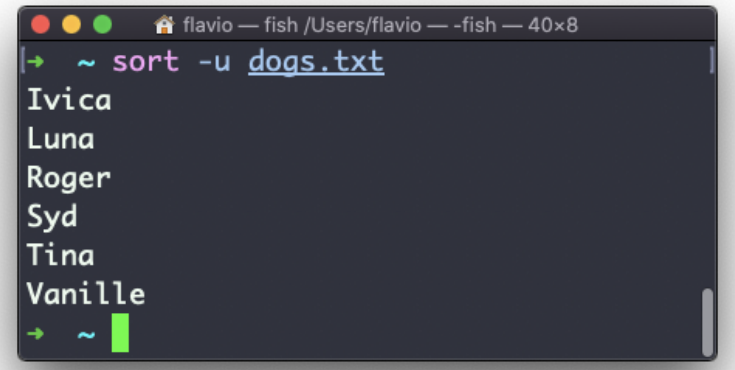


Sorting by default is case sensitive, and alphabetic. Use the `--ignore-case` option to sort case insensitive, and the `-n` option to sort using a numeric order.

If the file contains duplicate lines:



You can use the `-u` option to remove them:



`sort` does not just works on files, as many UNIX commands it also works with pipes, so you can use on the output of another command, for example you can order the files returned by `ls` with:

```
ls | sort
```

`sort` is very powerful and has lots more options, which you can explore calling `man sort`.

flavio — man /Users/flavio — less • man sort — 115×55
SORT(1) BSD General Commands Manual SORT(1)
NAME
sort -- sort or merge records (lines) of text and binary files
SYNOPSIS
sort [-bcCdfghiRMmnrsuVz] [-k field1[,field2]] [-S memsize] [-T dir] [-t char] [-o output]
[file ...]
sort --help
sort --version
DESCRIPTION
The sort utility sorts text and binary files by lines. A line is a record separated from the subsequent record by a newline (default) or NUL '\0' character (-z option). A record can contain any printable or unprintable characters. Comparisons are based on one or more sort keys extracted from each line of input, and are performed lexicographically, according to the current locale's collating rules and the specified command-line options that can tune the actual sorting behavior. By default, if keys are not given, sort uses entire lines for comparison.
The command line options are as follows:
-c, --check, -C, --check=silent|quiet
Check that the single input file is sorted. If the file is not sorted, sort produces the appropriate error messages and exits with code 1, otherwise returns 0. If -C or --check=silent is specified, sort produces no output. This is a "silent" version of -c.
-m, --merge
Merge only. The input files are assumed to be pre-sorted. If they are not sorted the output order is undefined.
-o output, --output=output
Print the output to the output file instead of the standard output.
-S size, --buffer-size=size
Use size for the maximum size of the memory buffer. Size modifiers %,b,K,M,G,T,P,E,Z,Y can be used. If a memory limit is not explicitly specified, sort takes up to about 90% of available memory. If the file size is too big to fit into the memory buffer, the temporary disk files are used to perform the sorting.
-T dir, --temporary-directory=dir
Store temporary files in the directory dir. The default path is the value of the environment variable TMPDIR or /var/tmp if TMPDIR is not defined.
-u, --unique
Unique keys. Suppress all lines that have a key that is equal to an already processed one. This option, similarly to -s, implies a stable sort. If used with -c or -C, sort also checks that there are no lines with duplicate keys.
-s
Stable sort. This option maintains the original record order of records that have an equal key. This is a non-standard feature, but it is widely accepted and used.
--version
Print the version and silently exits.
:

uniq

`uniq` is a command useful to sort lines of text.

You can get those lines from a file, or using pipes from the output of another command:

```
uniq dogs.txt

ls | uniq
```

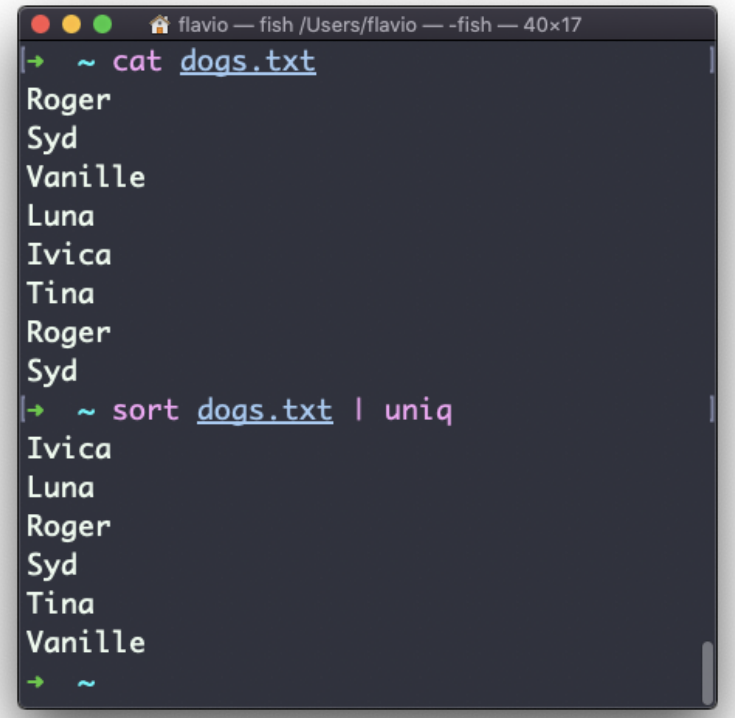
You need to consider this key thing: `uniq` will only detect adjacent duplicate lines.

This implies that you will most likely use it along with `sort`:

```
sort dogs.txt | uniq
```

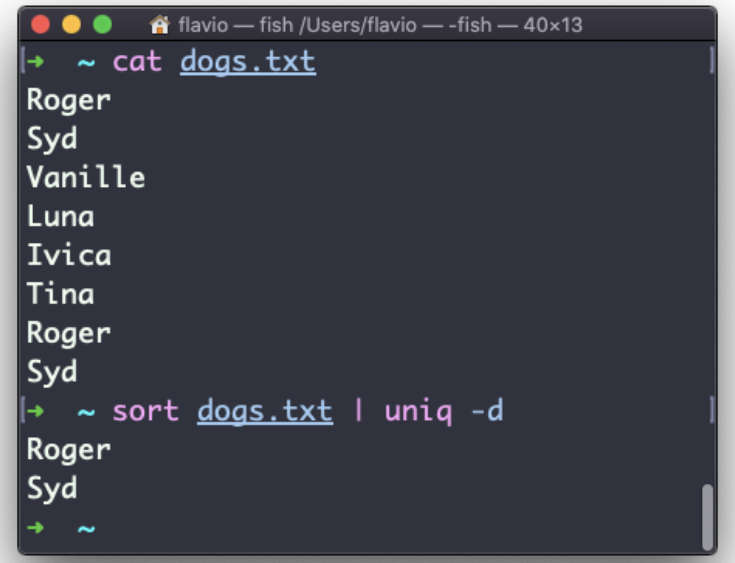
The `sort` command has its own way to remove duplicates with the `-u` (*unique*) option. But `uniq` has more power.

By default it removes duplicate lines:

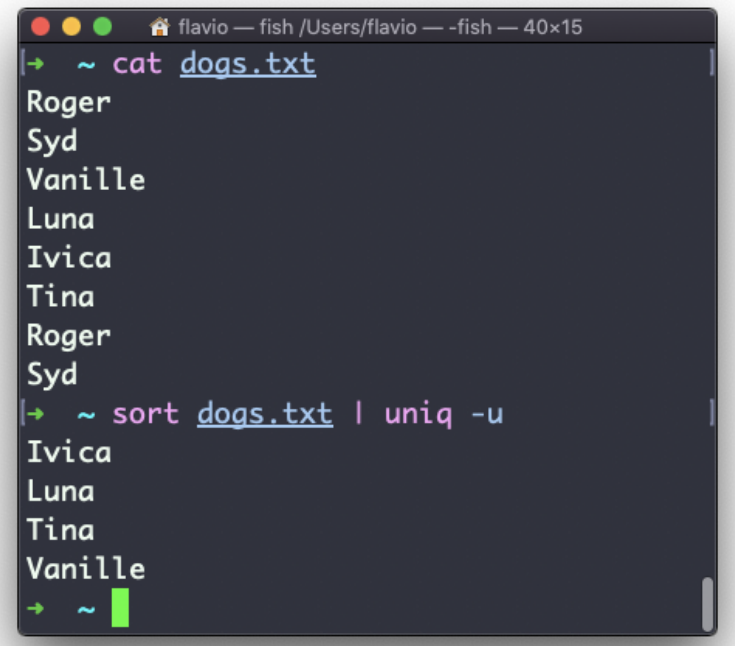


You can tell it to only display duplicate lines, for example, with the `-d` option:

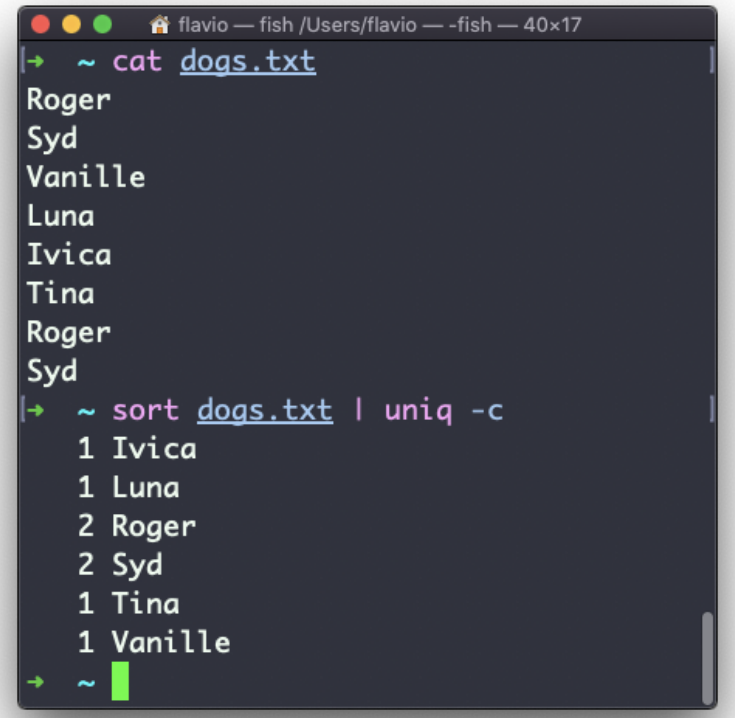
```
sort dogs.txt | uniq -d
```



You can use the `-u` option to only display non-duplicate lines:



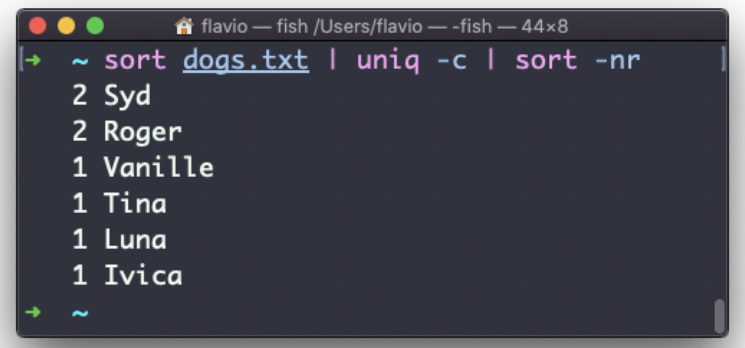
You can count the occurrences of each line with the `-c` option:



Use the special combination:

```
sort dogs.txt | uniq -c | sort -nr
```

to then sort those lines by most frequent:

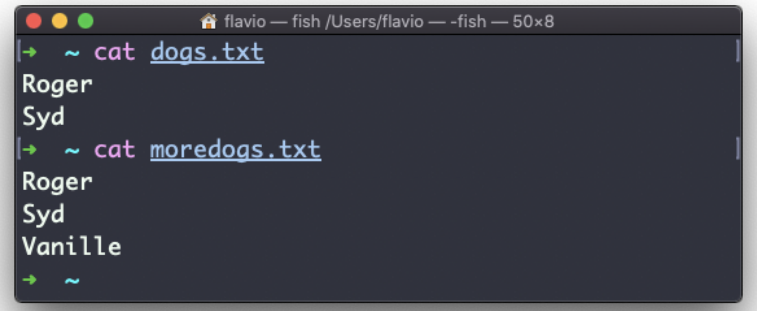


diff

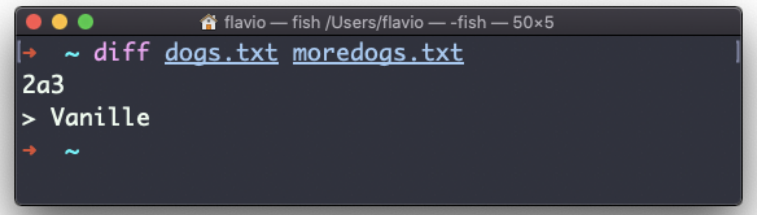
`diff` is a handy command. Suppose you have 2 files, which contain almost the same information, but you can't find the difference between the two.

`diff` will process the files and will tell you what's the difference.

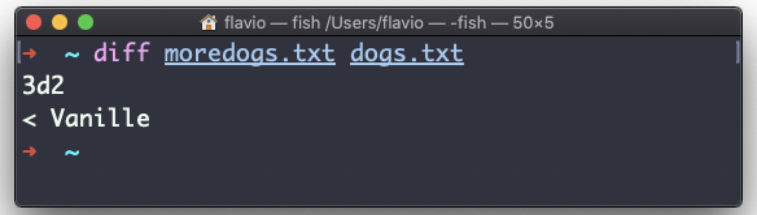
Suppose you have 2 files: `dogs.txt` and `moredogs.txt`. The difference is that `moredogs.txt` contains one more dog name:



`diff dogs.txt moredogs.txt` will tell you the second file has one more line, line 3 with the line `Vanille`:

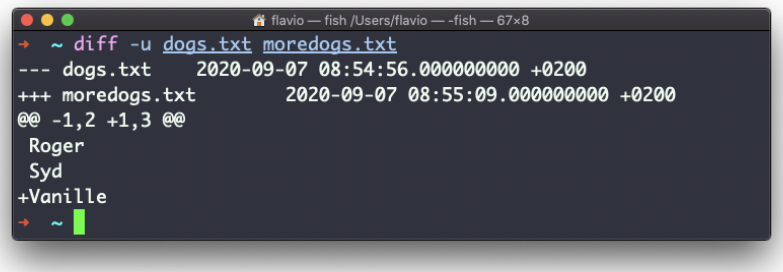


If you invert the order of the files, it will tell you that the second file is missing line 3, whose content is `Vanille`:

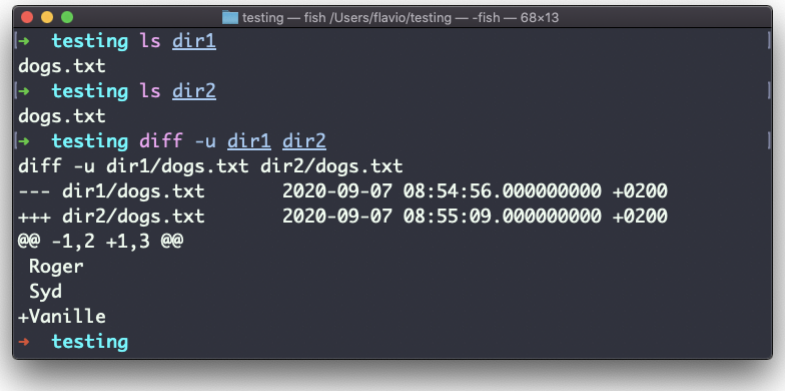


Using the `-y` option will compare the 2 files line by line:

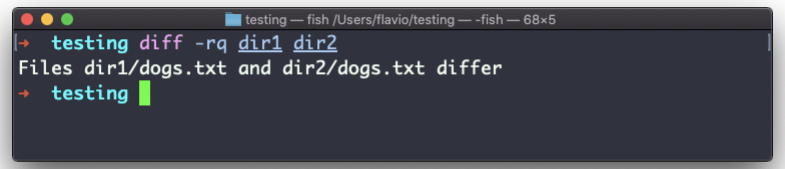
The `-u` option however will be more familiar to you, because that's the same used by the Git version control system to display differences between versions:



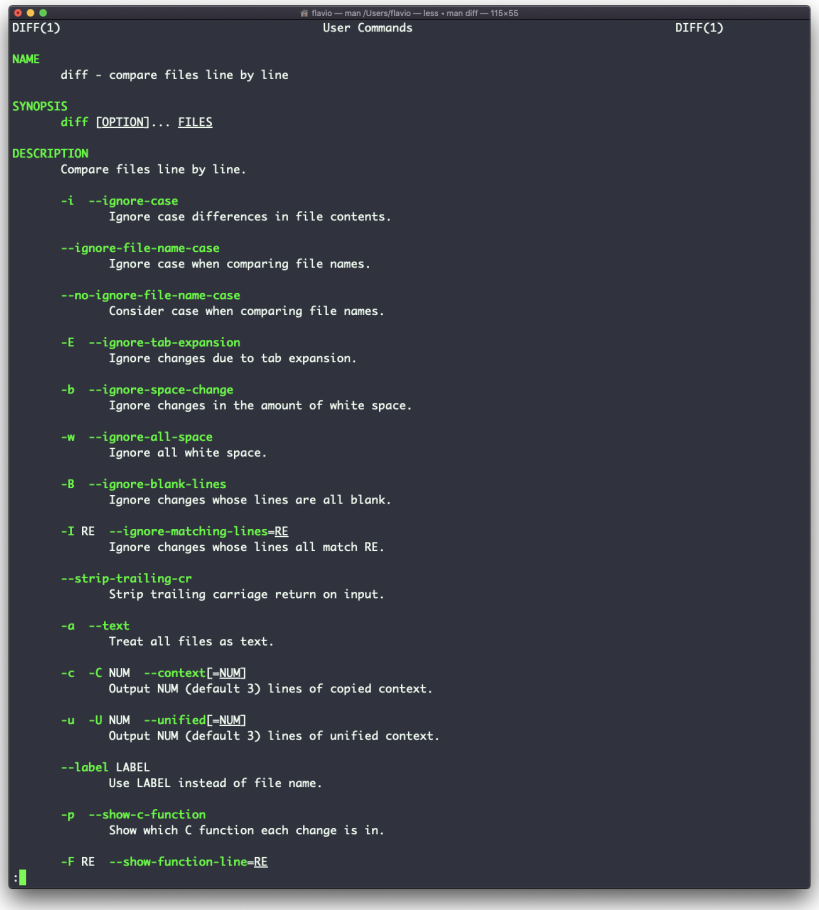
Comparing directories works in the same way. You must use the `-r` option to compare recursively (going into subdirectories):



In case you're interested in which files differ, rather than the content, use the `r` and `q` options:



There are many more options you can explore in the man page running `man diff`:


flavio — man /Users/flavio — less « man diff — 115×55
DIFF(1) User Commands DIFF(1)
NAME
diff - compare files line by line
SYNOPSIS
diff [OPTION]... FILES
DESCRIPTION
Compare files line by line.
-i --ignore-case
Ignore case differences in file contents.
--ignore-file-name-case
Ignore case when comparing file names.
--no-ignore-file-name-case
Consider case when comparing file names.
-E --ignore-tab-expansion
Ignore changes due to tab expansion.
-b --ignore-space-change
Ignore changes in the amount of white space.
-w --ignore-all-space
Ignore all white space.
-B --ignore-blank-lines
Ignore changes whose lines are all blank.
-I RE --ignore-matching-lines=RE
Ignore changes whose lines all match RE.
--strip-trailing-cr
Strip trailing carriage return on input.
-a --text
Treat all files as text.
-c -C NUM --context[=NUM]
Output NUM (default 3) lines of copied context.
-u -U NUM --unified[=NUM]
Output NUM (default 3) lines of unified context.
--label LABEL
Use LABEL instead of file name.
-p --show-c-function
Show which C function each change is in.
-F RE --show-function-line=RE
:

echo

The `echo` command does one simple job: it prints to the output the argument passed to it.

This example:

```
echo "hello"
```

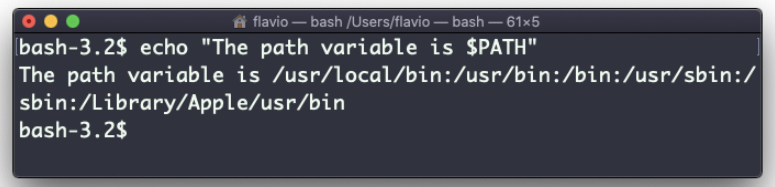
will print `hello` to the terminal.

We can append the output to a file:

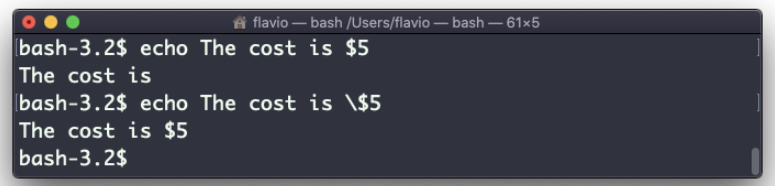
```
echo "hello" >> output.txt
```

We can interpolate environment variables:

```
echo "The path variable is $PATH"
```



Beware that special characters need to be escaped with a backslash `\` . `$` for example:



This is just the start. We can do some nice things when it comes to interacting with the shell features.

We can echo the files in the current folder:

```
echo *
```

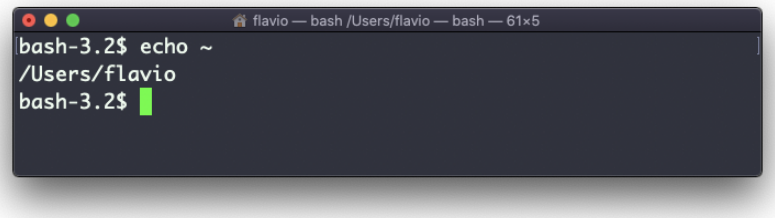
We can echo the files in the current folder that start with the letter `o`:

```
echo o*
```

Any valid Bash (or any shell you are using) command and feature can be used here.

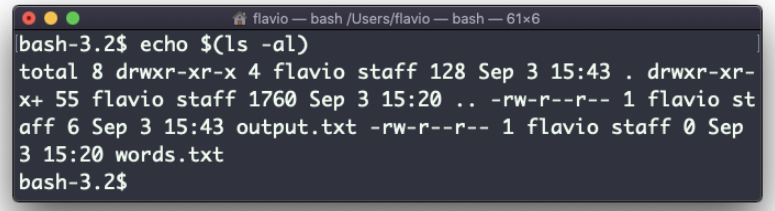
You can print your home folder path:

```
echo ~
```

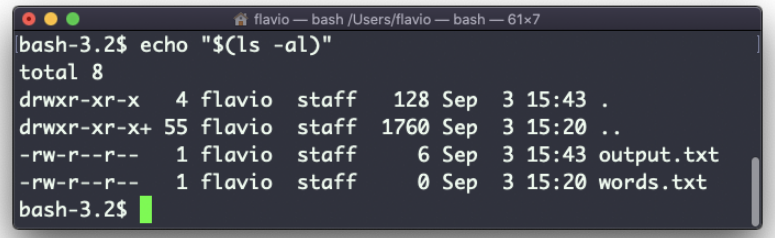


You can also execute commands, and print the result to the standard output (or to file, as you saw):

```
echo $(ls -al)
```

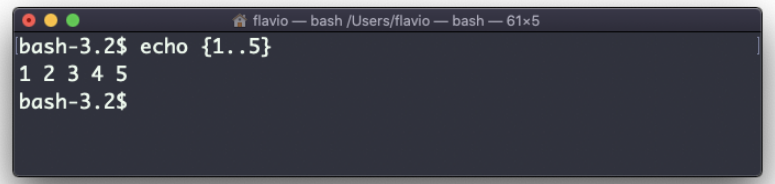


Note that whitespace is not preserved by default. You need to wrap the command in double quotes to do so:



You can generate a list of strings, for example ranges:

```
echo {1..5}
```



chown

Every file/directory in an Operating System like Linux or macOS (and every UNIX systems in general) has an **owner**.

The owner of a file can do everything with it. It can decide the fate of that file.

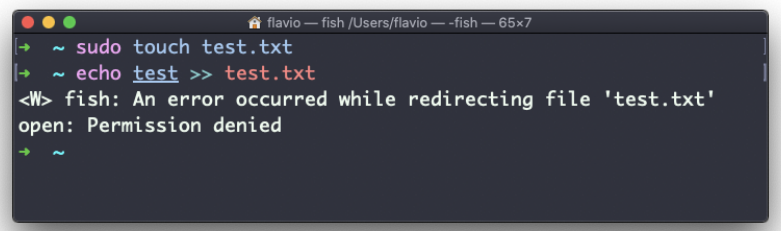
The owner (and the `root` user) can change the owner to another user, too, using the `chown` command:

```
chown <owner> <file>
```

Like this:

```
chown flavio test.txt
```

For example if you have a file that's owned by `root`, you can't write to it as another user:



You can use `chown` to transfer the ownership to you:

It's rather common to have the need to change the ownership of a directory, and recursively all the files contained, plus all the subdirectories and the files contained in them, too.

You can do so using the `-R` flag:

```
chown -R <owner> <file>
```

Files/directories don't just have an owner, they also have a **group**. Through this command you can change that simultaneously while you change the owner:

```
chown <owner>:<group> <file>
```

Example:

```
chown flavio:users test.txt
```

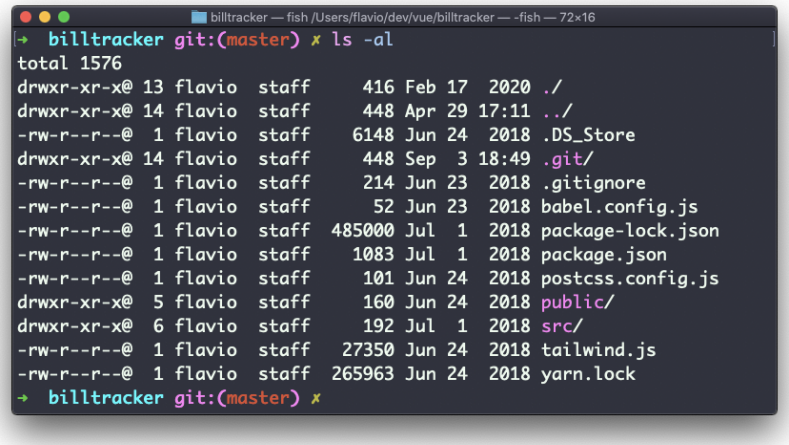
You can also just change the group of a file using the `chgrp` command:

```
chgrp <group> <filename>
```

chmod

Every file in the Linux / macOS Operating Systems (and UNIX systems in general) has 3 permissions: Read, write, execute.

Go into a folder, and run the `ls -al` command.



The weird strings you see on each file line, like `drwxr-xr-x` , define the permissions of the file or folder.

Let's dissect it.

The first letter indicates the type of file:

- `-` means it's a normal file
- `d` means it's a directory
- `l` means it's a link

Then you have 3 sets of values:

- The first set represents the permissions of the **owner** of the file
- The second set represents the permissions of the members of the **group** the file is associated to

- The third set represents the permissions of the **everyone else**

Those sets are composed by 3 values. `rwx` means that specific *persona* has read, write and execution access. Anything that is removed is swapped with a `-`, which lets you form various combinations of values and relative permissions: `rw-`, `r--`, `r-x`, and so on.

You can change the permissions given to a file using the `chmod` command.

`chmod` can be used in 2 ways. The first is using symbolic arguments, the second is using numeric arguments. Let's start with symbols first, which is more intuitive.

You type `chmod` followed by a space, and a letter:

- `a` stands for *all*
- `u` stands for *user*
- `g` stands for *group*
- `o` stands for *others*

Then you type either `+` or `-` to add a permission, or to remove it. Then you enter one or more permissions symbols (`r`, `w`, `x`).

All followed by the file or folder name.

Here are some examples:

```
chmod a+r filename #everyone can now read
chmod a+rw filename #everyone can now read and write
chmod o-rwx filename #others (not the owner, not in
```

You can apply the same permissions to multiple personas by adding multiple letters before the `+` / `-`:

```
chmod og-r filename #other and group can't read any
```

In case you are editing a folder, you can apply the permissions to every file contained in that folder using the `-r` (recursive) flag.

Numeric arguments are faster but I find them hard to remember when you are not using them day to day. You use a digit that represents the permissions of the persona. This number value can be a maximum of 7, and it's calculated in this way:

- `1` if has execution permission
- `2` if has write permission
- `4` if has read permission

This gives us 4 combinations:

- `0` no permissions
- `1` can execute
- `2` can write
- `3` can write, execute
- `4` can read
- `5` can read, execute
- `6` can read, write
- `7` can read, write and execute

We use them in pairs of 3, to set the permissions of all the 3 groups altogether:

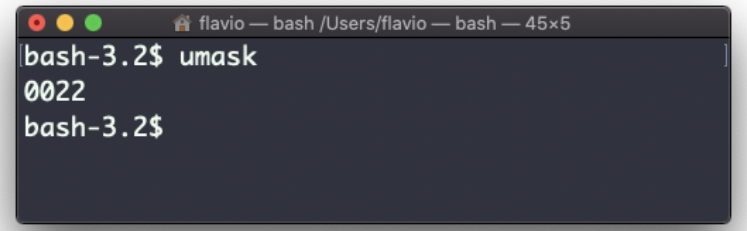
```
chmod 777 filename
chmod 755 filename
chmod 644 filename
```

umask

When you create a file, you don't have to decide permissions up front. Permissions have defaults.

Those defaults can be controlled and modified using the `umask` command.

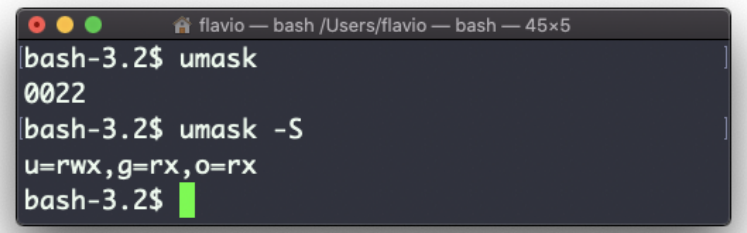
Typing `umask` with no arguments will show you the current umask, in this case `0022`:



What does `0022` mean? That's an octal value that represent the permissions.

Another common value is `0002`.

Use `umask -S` to see a human-readable notation:



In this case, the user (`u`), owner of the file, has read, write and execution permissions on files.

Other users belonging to the same group (`g`) have read and execution permission, same as all the other users (`o`).

In the numeric notation, we typically change the last 3 digits.

Here's a list that gives a meaning to the number:

- `0` read, write, execute
- `1` read and write
- `2` read and execute
- `3` read only
- `4` write and execute
- `5` write only
- `6` execute only
- `7` no permissions

Note that this numeric notation differs from the one we use in `chmod` .

We can set a new value for the mask setting the value in numeric format:

```
umask 002
```

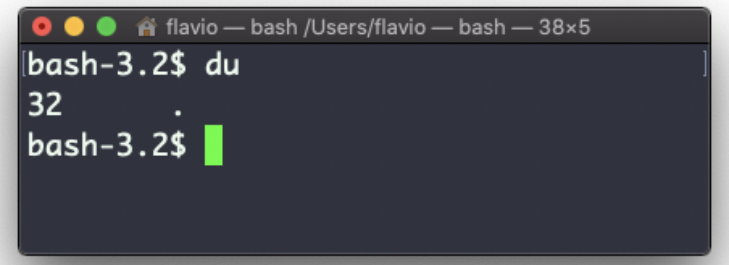
or you can change a specific role's permission:

```
umask g+r
```

du

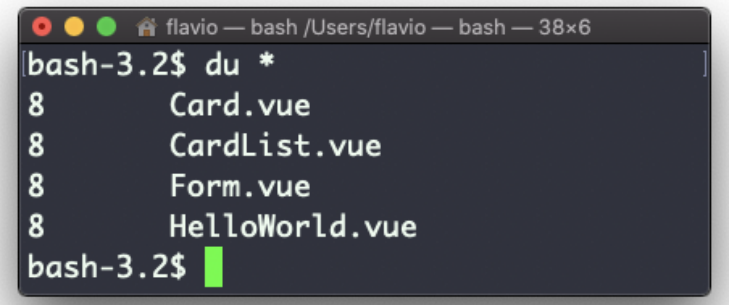
The `du` command will calculate the size of a directory as a whole:

```
du
```



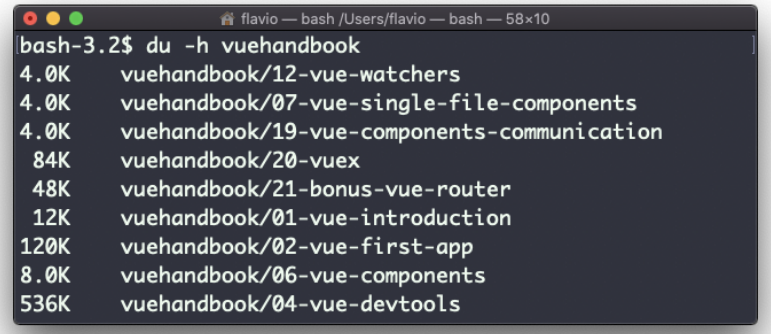
The `32` number here is a value expressed in bytes.

Running `du *` will calculate the size of each file individually:

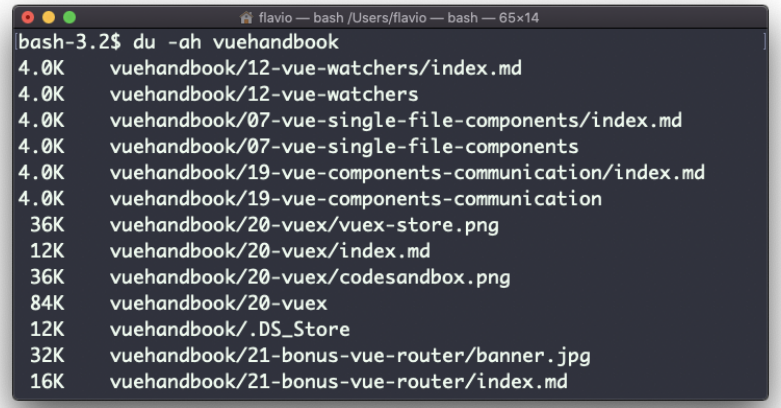


You can set `du` to display values in MegaBytes using `du -m`, and GigaBytes using `du -g`.

The `-h` option will show a human-readable notation for sizes, adapting to the size:



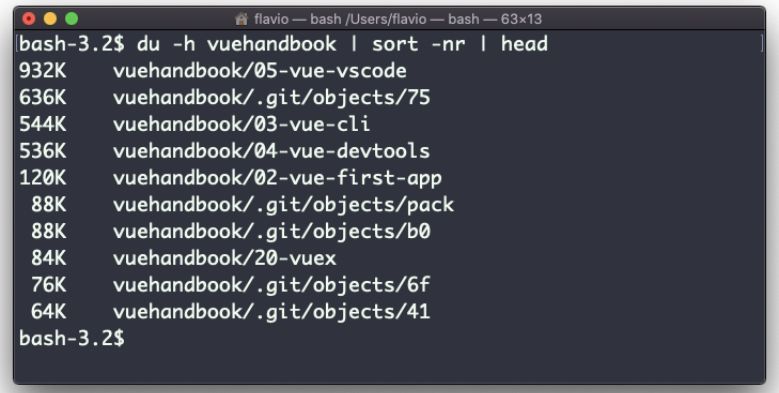
Adding the `-a` option will print the size of each file in the directories, too:



A handy thing is to sort the directories by size:

```
du -h <directory> | sort -nr
```

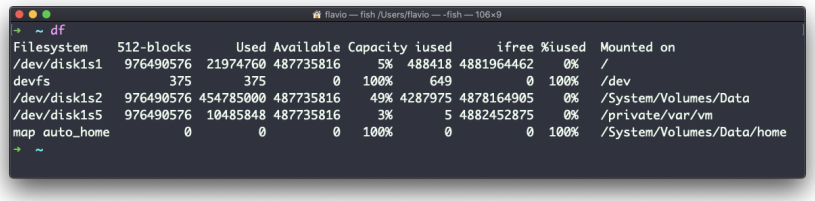
and then piping to `head` to only get the first 10 results:


flavio — bash /Users/flavio — bash — 63×13
bash-3.2$ du -h vuehandbook | sort -nr | head
932K	vuehandbook/05-vue-vscode
636K	vuehandbook/.git/objects/75
544K	vuehandbook/03-vue-cli
536K	vuehandbook/04-vue-devtools
120K	vuehandbook/02-vue-first-app
 88K	vuehandbook/.git/objects/pack
 88K	vuehandbook/.git/objects/b0
 84K	vuehandbook/20-vuex
 76K	vuehandbook/.git/objects/6f
 64K	vuehandbook/.git/objects/41
bash-3.2$

df

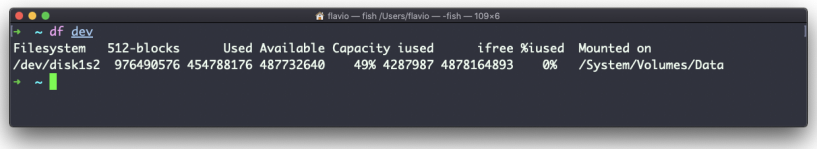
The `df` command is used to get disk usage information.

Its basic form will print information about the volumes mounted:



Using the `-h` option (`df -h`) will show those values in a human-readable format:

You can also specify a file or directory name to get information about the specific volume it lives on:



basename

Suppose you have a path to a file, for example `/Users/flavio/test.txt`.

Running

```
basename /Users/flavio/test.txt
```

will return the `test.txt` string:

If you run `basename` on a path string that points to a directory, you will get the last segment of the path. In this example, `/Users/flavio` is a directory:

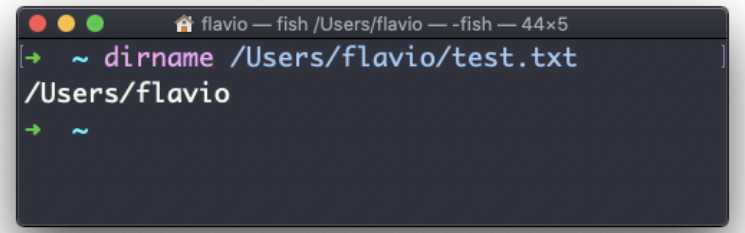
dirname

Suppose you have a path to a file, for example `/Users/flavio/test.txt`.

Running

```
dirname /Users/flavio/test.txt
```

will return the `/Users/flavio` string:



ps

Your computer is running, at all times, tons of different processes.

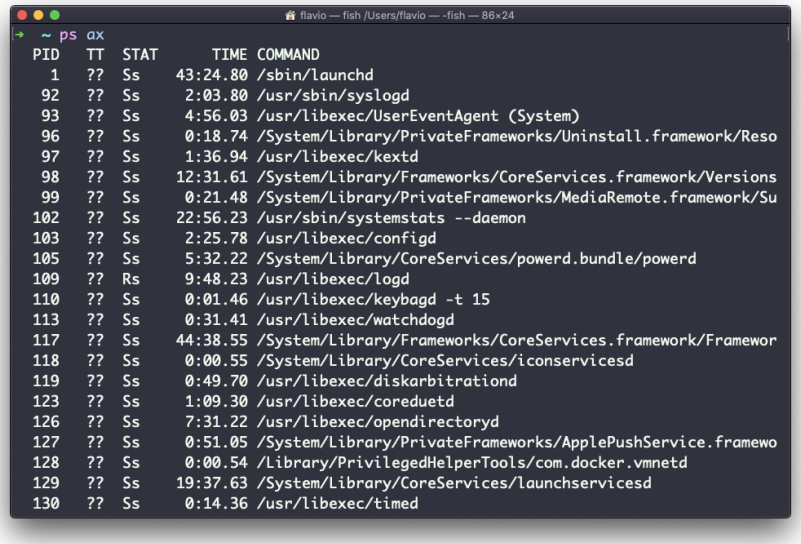
You can inspect them all using the `ps` command:

This is the list of user-initiated processes currently running in the current session.

Here I have a few `fish` shell instances, mostly opened by VS Code inside the editor, and an instances of Hugo running the development preview of a site.

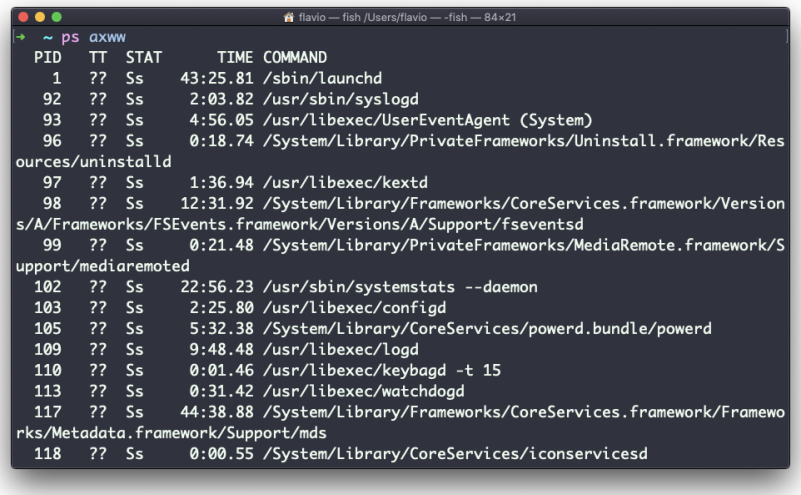
Those are just the commands assigned to the current user. To list **all** processes we need to pass some options to `ps`.

The most common I use is `ps ax`:



> The `a` option is used to also list other users processes, not just our own. `x` shows processes not linked to any terminal (not initiated by users through a terminal).

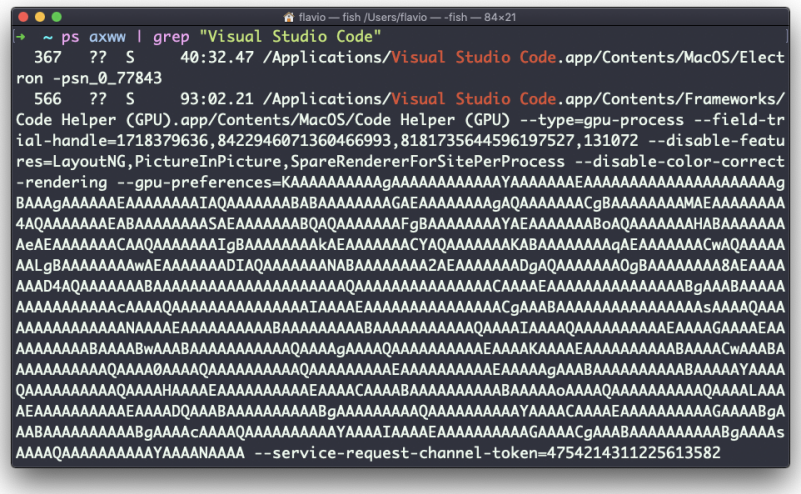
As you can see, the longer commands are cut. Use the command `ps axww` to continue the command listing on a new line instead of cutting it:



> We need to specify `w` 2 times to apply this setting, it's not a typo.

You can search for a specific process combining `grep` with a pipe, like this:

```
ps axww | grep "Visual Studio Code"
```



The columns returned by `ps` represent some key information.

The first information is `PID`, the process ID. This is key when you want to reference this process in another command, for example to kill it.

Then we have `TT` that tells us the terminal id used.

Then `STAT` tells us the state of the process:

`I` a process that is idle (sleeping for longer than about 20 seconds) `R` a runnable process `S` a process that is sleeping for less than about 20 seconds `T` a stopped process `U` a process in uninterruptible wait `Z` a dead process (a *zombie*)

If you have more than one letter, the second represents further information, which can be very technical.

It's common to have `+` which indicates the process is in the foreground in its terminal. `s` means the process is a session leader.

`TIME` tells us how long the process has been running.

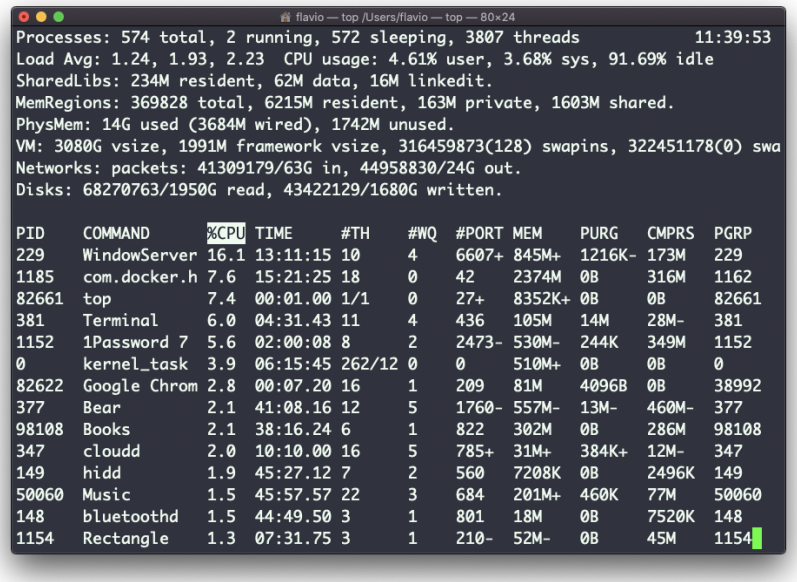
top

A quick guide to the `top` command, used to list the processes running in real time

The `top` command is used to display dynamic real-time information about running processes in the system.

It's really handy to understand what is going on.

Its usage is simple, you just type `top`, and the terminal will be fully immersed in this new view:



The process is long-running. To quit, you can type the `q` letter or `ctrl-C`.

There's a lot of information being given to us: the number of processes, how many are running or sleeping, the system load, the CPU usage, and a lot more.

Below, the list of processes taking the most memory and CPU is constantly updated.

By default, as you can see from the `%CPU` column highlighted, they are sorted by the CPU used.

You can add a flag to sort processes by memory utilized:

```
top -o mem
```

kill

Linux processes can receive **signals** and react to them.

That's one way we can interact with running programs.

The `kill` program can send a variety of signals to a program.

It's not just used to terminate a program, like the name would suggest, but that's its main job.

We use it in this way:

```
kill <PID>
```

By default, this sends the `TERM` signal to the process id specified.

We can use flags to send other signals, including:

```
kill -HUP <PID>
kill -INT <PID>
kill -KILL <PID>
kill -TERM <PID>
kill -CONT <PID>
kill -STOP <PID>
```

`HUP` means **hang up**. It's sent automatically when a terminal window that started a process is closed before terminating the process.

`INT` means **interrupt**, and it sends the same signal used when we press `ctrl-C` in the terminal, which usually terminates the process.

`KILL` is not sent to the process, but to the operating system kernel, which immediately stops and terminates the process.

`TERM` means **terminate**. The process will receive it and terminate itself. It's the default signal sent by `kill`.

`CONT` means **continue**. It can be used to resume a stopped process.

`STOP` is not sent to the process, but to the operating system kernel, which immediately stops (but does not terminate) the process.

You might see numbers used instead, like `kill -1 <PID>`. In this case,

`1` corresponds to `HUP`. `2` corresponds to `INT`. `9` corresponds to `KILL`. `15` corresponds to `TERM`. `18` corresponds to `CONT`. `15` corresponds to `STOP`.

killall

Similar to the `kill` command, `killall` instead of sending a signal to a specific process id will send the signal to multiple processes at once.

This is the syntax:

```
killall <name>
```

where `name` is the name of a program. For example you can have multiple instances of the `top` program running, and `killall top` will terminate them all.

You can specify the signal, like with `kill` (and check the `kill` tutorial to read more about the specific kinds of signals we can send), for example:

```
killall -HUP top
```

jobs

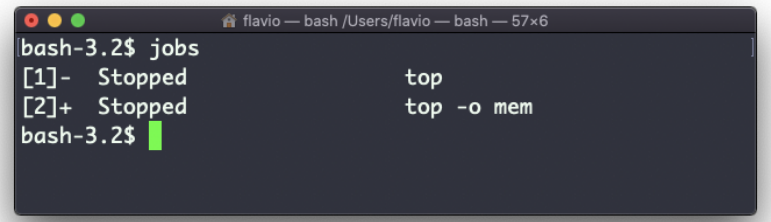
When we run a command in Linux / macOS, we can set it to run in the background using the `&` symbol after the command. For example we can run `top` in the background:

```
top &
```

This is very handy for long-running programs.

We can get back to that program using the `fg` command. This works fine if we just have one job in the background, otherwise we need to use the job number: `fg 1` , `fg 2` and so on. To get the job number, we use the `jobs` command.

Say we run `top &` and then `top -o mem &` , so we have 2 top instances running. `jobs` will tell us this:



Now we can switch back to one of those using `fg <jobid>` . To stop the program again we can hit `cmd-Z` .

Running `jobs -l` will also print the process id of each job.

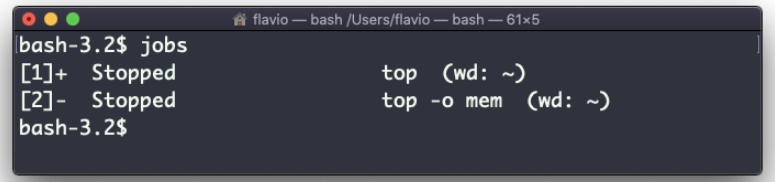
bg

When a command is running you can suspend it using `ctrl-Z`.

The command will immediately stop, and you get back to the shell terminal.

You can resume the execution of the command in the background, so it will keep running but it will not prevent you from doing other work in the terminal.

In this example I have 2 commands stopped:



I can run `bg 1` to resume in the background the execution of the job #1.

I could have also said `bg` without any option, as the default is to pick the job #1 in the list.

fg

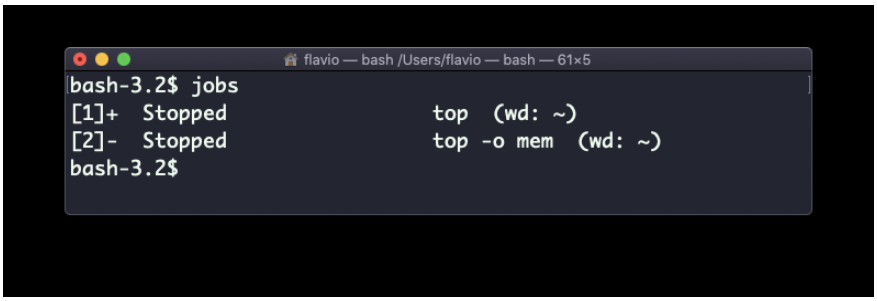
When a command is running in the background, because you started it with `&` at the end (example: `top &` or because you put it in the background with the `bg` command, you can put it to the foreground using `fg`.

Running

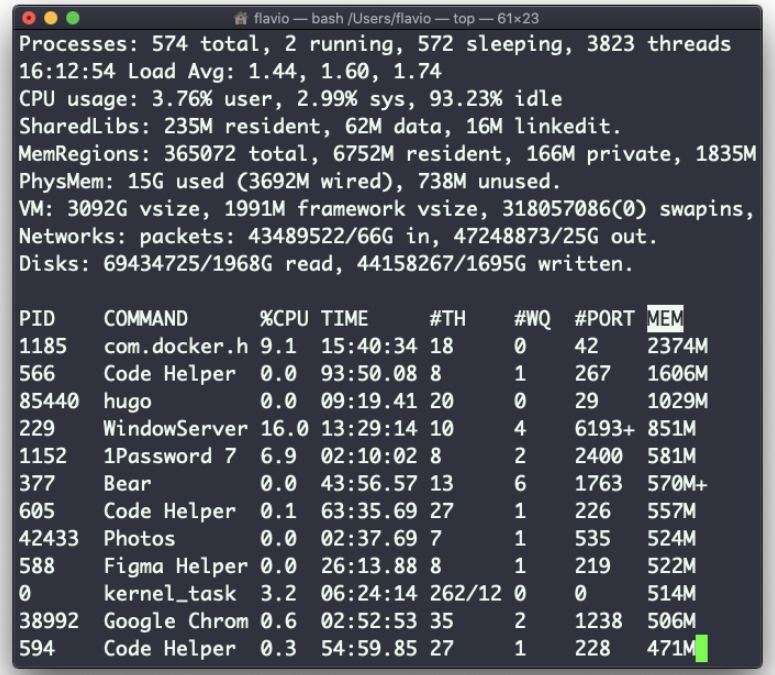
```
fg
```

will resume to the foreground the last job that was suspended.

You can also specify which job you want to resume to the foreground passing the job number, which you can get using the `jobs` command.



Running `fg 2` will resume job #2:


flavio — bash /Users/flavio — top — 61×23
Processes: 574 total, 2 running, 572 sleeping, 3823 threads
16:12:54 Load Avg: 1.44, 1.60, 1.74
CPU usage: 3.76% user, 2.99% sys, 93.23% idle
SharedLibs: 235M resident, 62M data, 16M linkedit.
MemRegions: 365072 total, 6752M resident, 166M private, 1835M
PhysMem: 15G used (3692M wired), 738M unused.
VM: 3092G vsize, 1991M framework vsize, 318057086(0) swapins,
Networks: packets: 43489522/66G in, 47248873/25G out.
Disks: 69434725/1968G read, 44158267/1695G written.
PID COMMAND %CPU TIME #TH #WQ #PORT MEM
1185 com.docker.h 9.1 15:40:34 18 0 42 2374M
566 Code Helper 0.0 93:50.08 8 1 267 1606M
85440 hugo 0.0 09:19.41 20 0 29 1029M
229 WindowServer 16.0 13:29:14 10 4 6193+ 851M
1152 1Password 7 6.9 02:10:02 8 2 2400 581M
377 Bear 0.0 43:56.57 13 6 1763 570M+
605 Code Helper 0.1 63:35.69 27 1 226 557M
42433 Photos 0.0 02:37.69 7 1 535 524M
588 Figma Helper 0.0 26:13.88 8 1 219 522M
0 kernel_task 3.2 06:24:14 262/12 0 0 514M
38992 Google Chrom 0.6 02:52:53 35 2 1238 506M
594 Code Helper 0.3 54:59.85 27 1 228 471M

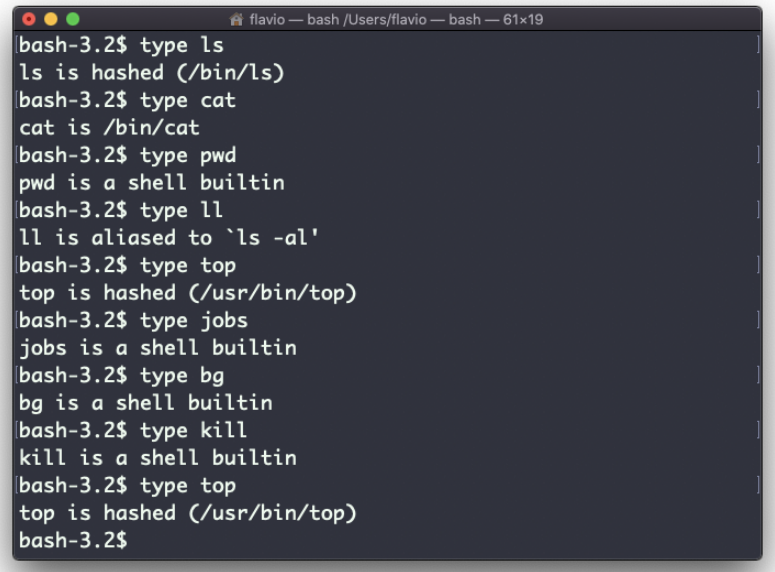
type

A command can be one of those 4 types:

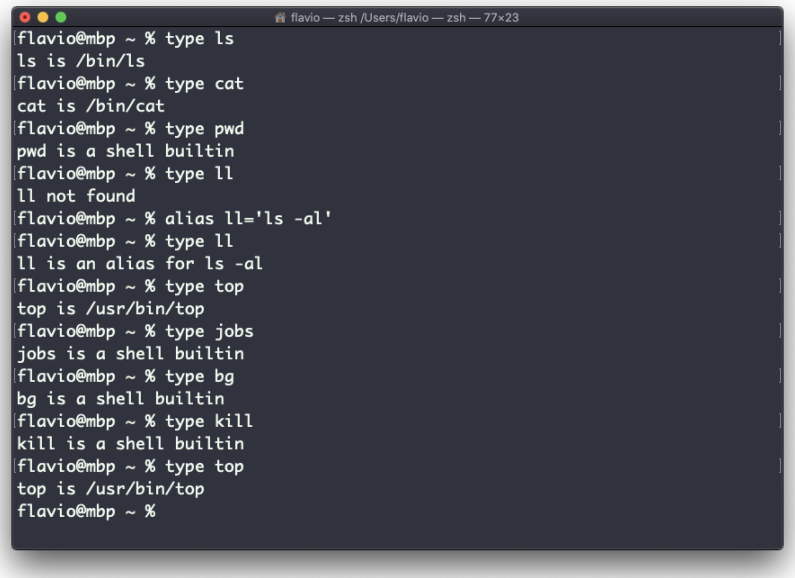
- an executable
- a shell built-in program
- a shell function
- an alias

The `type` command can help figure out this, in case we want to know or we're just curious. It will tell you how the command will be interpreted.

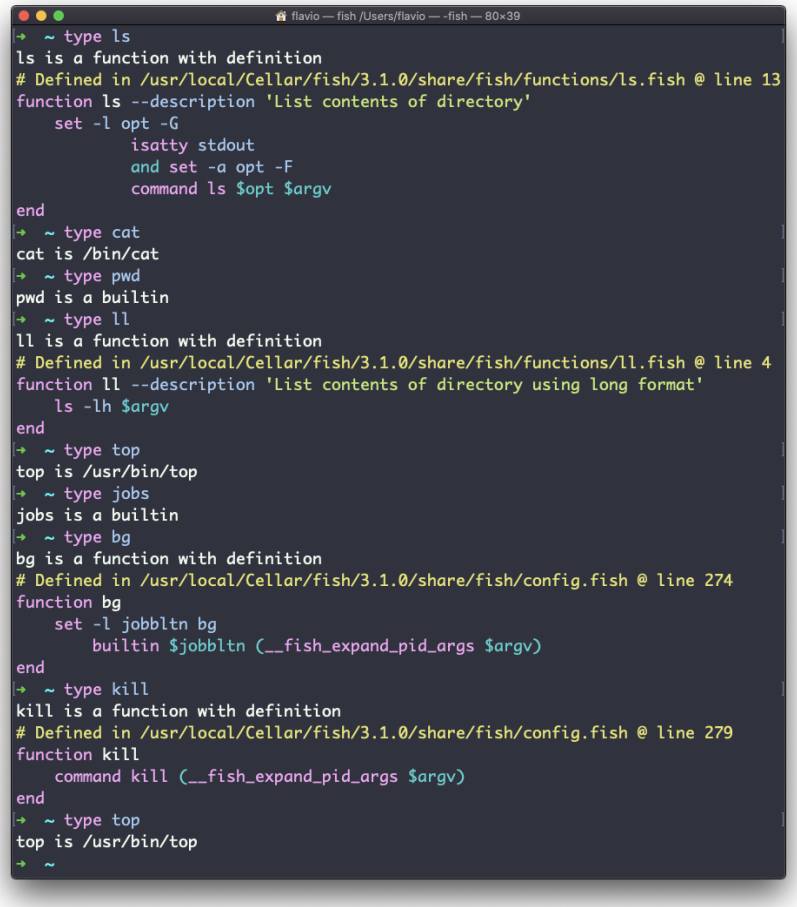
The output will depend on the shell used. This is Bash:



This is Zsh:


flavio — zsh /Users/flavio — zsh — 77×23
flavio@mbp ~ % type ls
ls is /bin/ls
flavio@mbp ~ % type cat
cat is /bin/cat
flavio@mbp ~ % type pwd
pwd is a shell builtin
flavio@mbp ~ % type ll
ll not found
flavio@mbp ~ % alias ll='ls -al'
flavio@mbp ~ % type ll
ll is an alias for ls -al
flavio@mbp ~ % type top
top is /usr/bin/top
flavio@mbp ~ % type jobs
jobs is a shell builtin
flavio@mbp ~ % type bg
bg is a shell builtin
flavio@mbp ~ % type kill
kill is a shell builtin
flavio@mbp ~ % type top
top is /usr/bin/top
flavio@mbp ~ %

This is Fish:

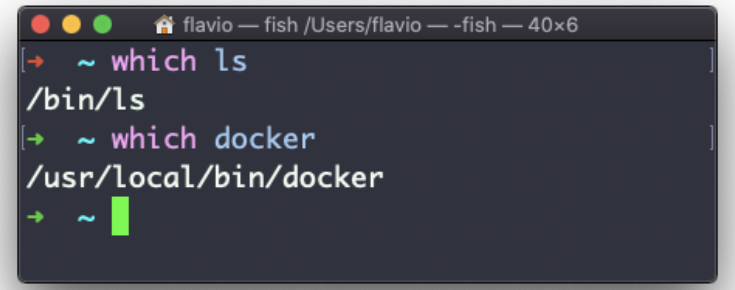

flavio — fish /Users/flavio — -fish — 80×39
➜ ~ type ls
ls is a function with definition
Defined in /usr/local/Cellar/fish/3.1.0/share/fish/functions/ls.fish @ line 13
function ls --description 'List contents of directory'
set -l opt -G
isatty stdout
and set -a opt -F
command ls $opt $argv
end
➜ ~ type cat
cat is /bin/cat
➜ ~ type pwd
pwd is a builtin
➜ ~ type ll
ll is a function with definition
Defined in /usr/local/Cellar/fish/3.1.0/share/fish/functions/ll.fish @ line 4
function ll --description 'List contents of directory using long format'
ls -lh $argv
end
➜ ~ type top
top is /usr/bin/top
➜ ~ type jobs
jobs is a builtin
➜ ~ type bg
bg is a function with definition
Defined in /usr/local/Cellar/fish/3.1.0/share/fish/config.fish @ line 274
function bg
set -l jobbltn bg
builtin $jobbltn (__fish_expand_pid_args $argv)
end
➜ ~ type kill
kill is a function with definition
Defined in /usr/local/Cellar/fish/3.1.0/share/fish/config.fish @ line 279
function kill
command kill (__fish_expand_pid_args $argv)
end
➜ ~ type top
top is /usr/bin/top
➜ ~

One of the most interesting things here is that for aliases it will tell you what is aliasing to. You can see the `ll` alias, in the case of Bash and Zsh, but Fish provides it by default, so it will tell you it's a built-in shell function.

which

Suppose you have a command you can execute, because it's in the shell path, but you want to know where it is located.

You can do so using `which`. The command will return the path to the command specified:



`which` will only work for executables stored on disk, not aliases or built-in shell functions.

nohup

Sometimes you have to run a long-lived process on a remote machine, and then you need to disconnect.

Or you simply want to prevent the command to be halted if there's any network issue between you and the server.

The way to make a command run even after you log out or close the session to a server is to use the `nohup` command.

Use `nohup <command>` to let the process continue working even after you log out.

xargs

The `xargs` command is used in a UNIX shell to convert input from standard input into arguments to a command.

In other words, through the use of `xargs` the output of a command is used as the input of another command.

Here's the syntax you will use:

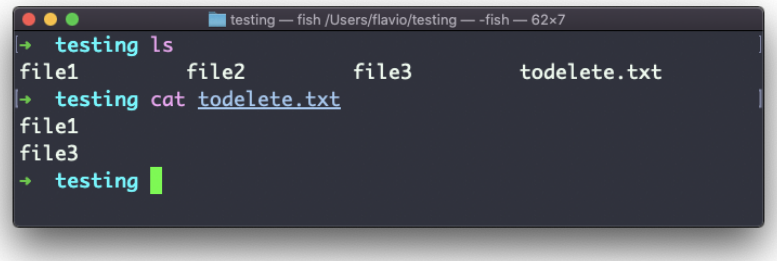
```
command1 | xargs command2
```

We use a pipe (`|`) to pass the output to `xargs` . That will take care of running the `command2` command, using the output of `command1` as its argument(s).

Let's do a simple example. You want to remove some specific files from a directory. Those files are listed inside a text file.

We have 3 files: `file1` , `file2` , `file3` .

In `todelete.txt` we have a list of files we want to delete, in this example `file1` and `file3` :



We will channel the output of `cat todelete.txt` to the `rm` command, through `xargs`.

In this way:

```
cat todelete.txt | xargs rm
```

That's the result, the files we listed are now deleted:

The way it works is that `xargs` will run `rm` 2 times, one for each line returned by `cat`.

This is the simplest usage of `xargs`. There are several options we can use.

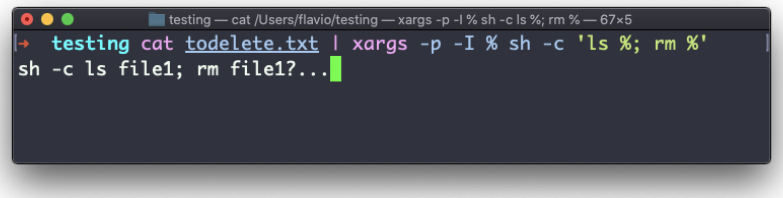
One of the most useful in my opinion, especially when starting to learn `xargs`, is `-p`. Using this option will make `xargs` print a confirmation prompt with the action it's going to take:

The `-n` option lets you tell `xargs` to perform one iteration at a time, so you can individually confirm them with `-p`. Here we tell `xargs` to perform one iteration at a time with `-n1`:

The `-I` option is another widely used one. It allows you to get the output into a placeholder, and then you can do various things.

One of them is to run multiple commands:

```
command1 | xargs -I % /bin/bash -c 'command2 %; comm
```



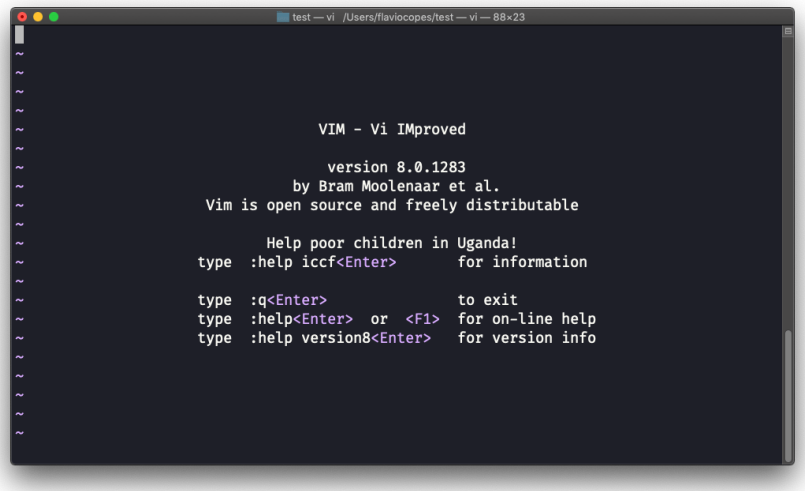
> You can swap the `%` symbol I used above with anything else, it's a variable

vim

`vim` is a **very** popular file editor, especially among programmers. It's actively developed and frequently updated, and there's a very big community around it. There's even a Vim conference!

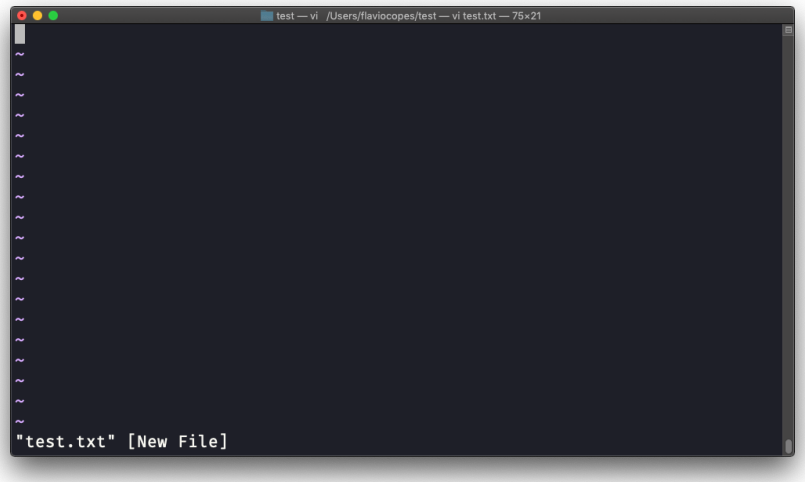
`vi` in modern systems is just an alias to `vim`, which means `vi` i `m` proved.

You start it by running `vi` on the command line.



You can specify a filename at invocation time to edit that specific file:

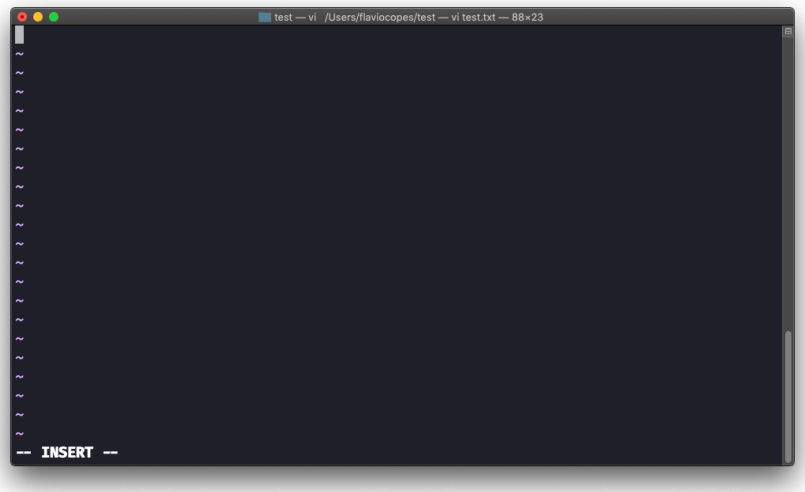
```
vi test.txt
```



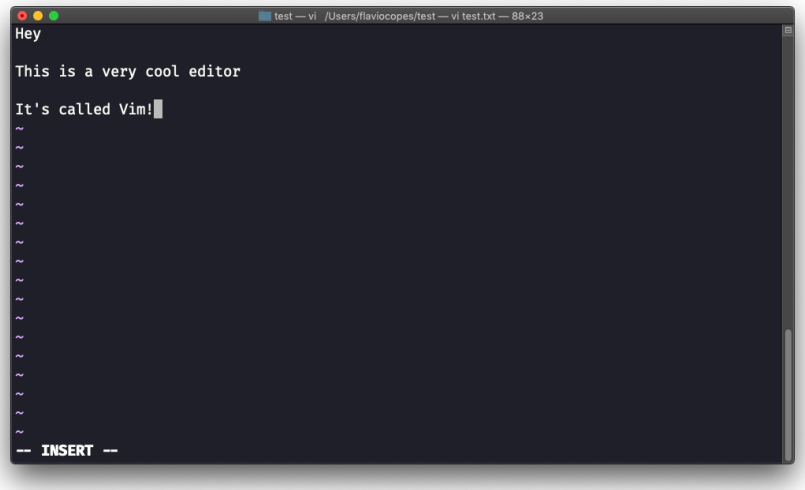
You have to know that Vim has 2 main modes:

- *command* (or *normal*) mode
- *insert* mode

When you start the editor, you are in command mode. You can't enter text like you expect from a GUI-based editor. You have to enter **insert mode**. You can do this by pressing the `i` key. Once you do so, the `-- INSERT --` word appear at the bottom of the editor:

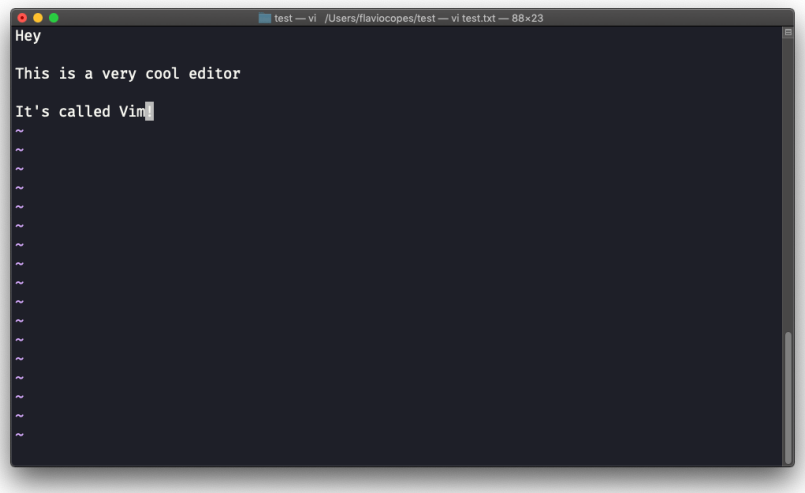


Now you can start typing and filling the screen with the file contents:



You can move around the file with the arrow keys, or using the `h` - `j` - `k` - `l` keys. `h-l` for left-right, `j-k` for down-up.

Once you are done editing you can press the `esc` key to exit insert mode, and go back to **command mode**.



At this point you can navigate the file, but you can't add content to it (and be careful which keys you press as they might be commands).

One thing you might want to do now is **saving the file**. You can do so by pressing `:` (colon), then `w`.

You can **save and quit** pressing `:` then `w` and `q`: `:wq`

You can **quit without saving**, pressing `:` then `q` and `!`: `:q!`

You can **undo** and edit by going to command mode and pressing `u`. You can **redo** (cancel an undo) by pressing `ctrl-r`.

Those are the basics of working with Vim. From here starts a rabbit hole we can't go into in this little introduction.

I will only mention those commands that will get you started editing with Vim:

- pressing the `x` key deletes the character currently highlighted
- pressing `A` goes at the end of the currently selected line
- press `0` to go to the start of the line
- go to the first character of a word and press `d` followed by `w` to delete that word. If you follow it with `e` instead of `w`, the white space before the next word is preserved
- use a number between `d` and `w` to delete more than 1 word, for example use `d3w` to delete 3 words forward
- press `d` followed by `d` to delete a whole entire line. Press `d` followed by `$` to delete the entire line from where the cursor is, until the end

To find out more about Vim I can recommend the Vim FAQ and especially running the `vimtutor` command, which should already be installed in your system and

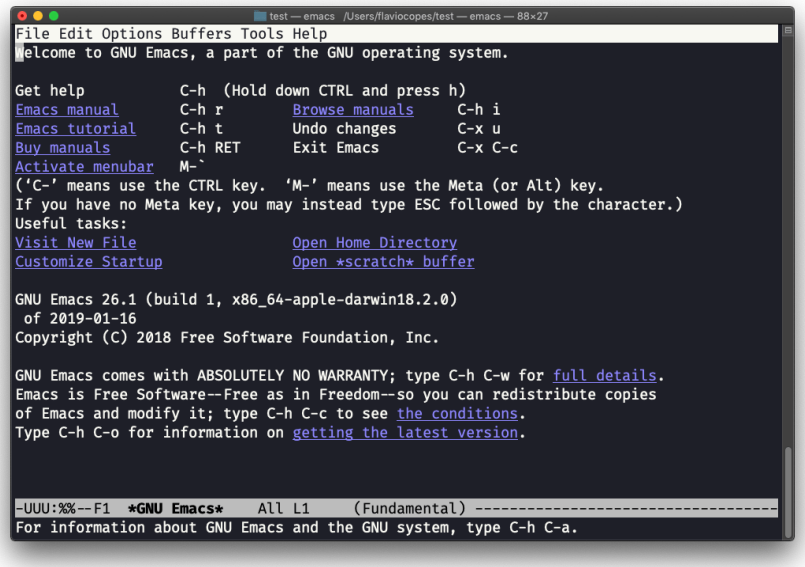
will greatly help you start your `vim` explorations.

emacs

`emacs` is an awesome editor and it's historically regarded as *the* editor for UNIX systems. Famously `vi` vs `emacs` flame wars and heated discussions caused many unproductive hours for developers around the world.

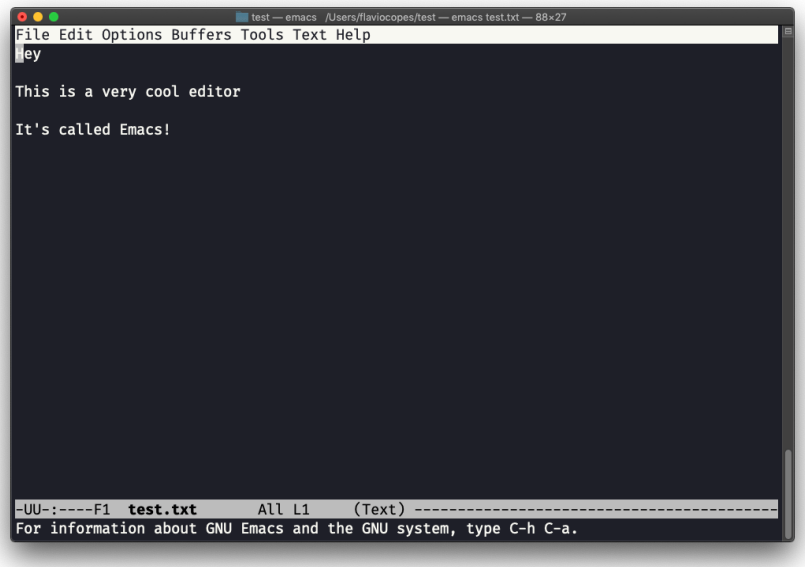
`emacs` is very powerful. Some people use it all day long as a kind of operating system (https://news.ycombinator.com/item?id=19127258). We'll just talk about the basics here.

You can open a new emacs session simply by invoking `emacs`:

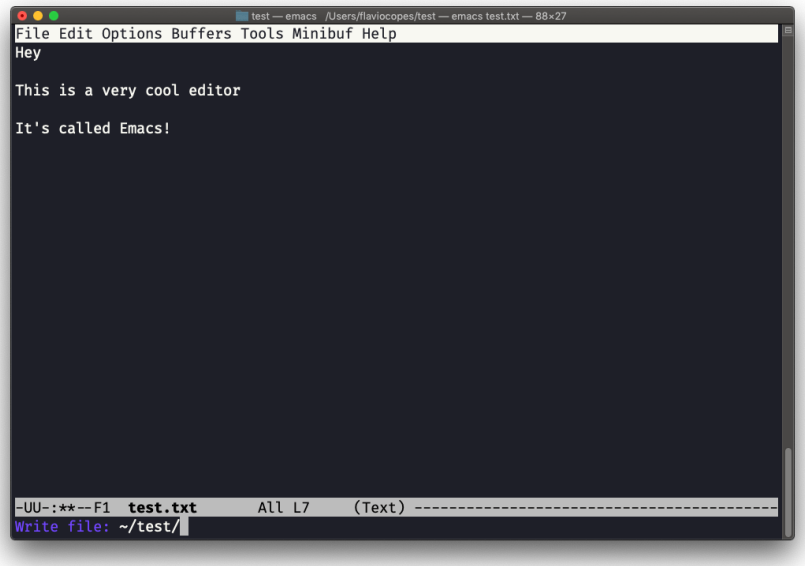


> macOS users, stop a second now. If you are on Linux there are no problems, but macOS does not ship applications using GPLv3, and every built-in UNIX command that has been updated to GPLv3 has not been updated. While there is a little problem with the commands I listed up to now, in this case using an emacs version from 2007 is not exactly the same as using a version with 12 years of improvements and change. This is not a problem with Vim, which is up to date. To fix this, run `brew install emacs` and running `emacs` will use the new version from Homebrew (make sure you have Homebrew installed)

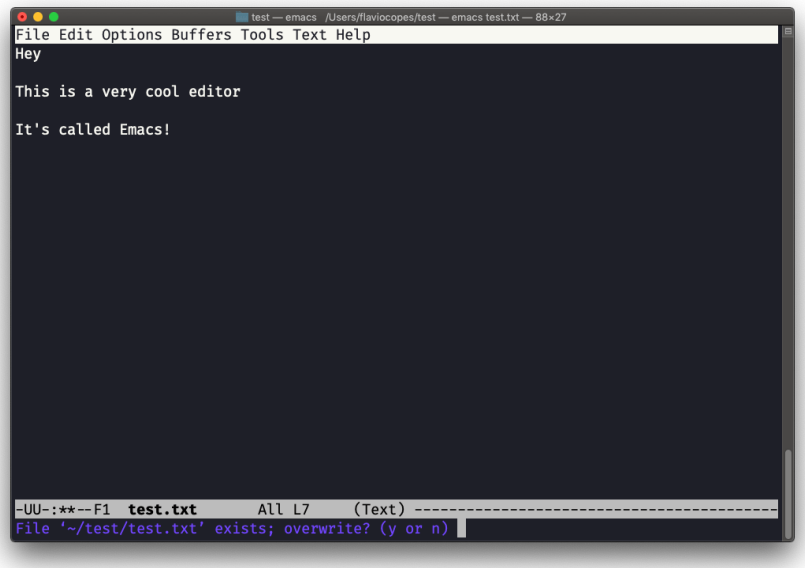
You can also edit an existing file calling `emacs <filename>`:



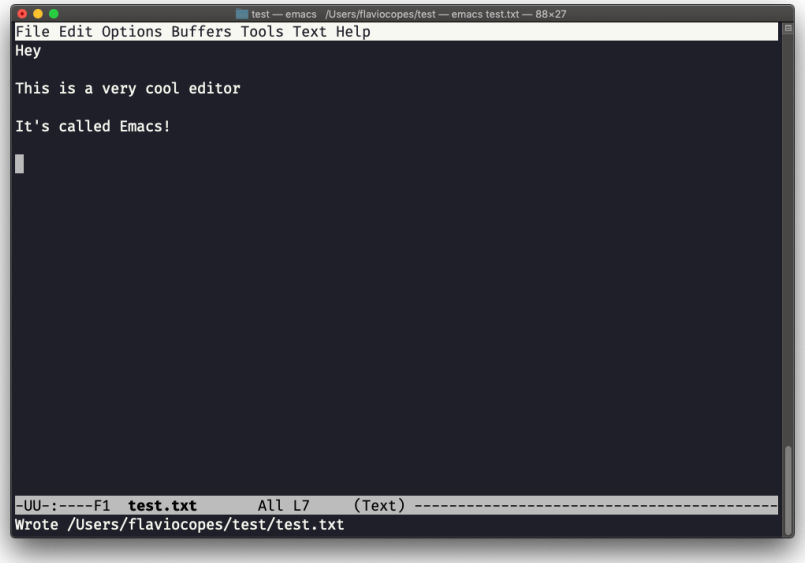
You can start editing and once you are done, press `ctrl-x` followed by `ctrl-w`. You confirm the folder:



and Emacs tell you the file exists, asking you if it should overwrite it:



Answer `y`, and you get a confirmation of success:



You can exit Emacs pressing `ctrl-x` followed by `ctrl-c`. Or `ctrl-x` followed by `c` (keep `ctrl` pressed).

There is a lot to know about Emacs. More than I am able to write in this little introduction. I encourage you to open Emacs and press `ctrl-h` `r` to open the built-in manual and `ctrl-h` `t` to open the official tutorial.

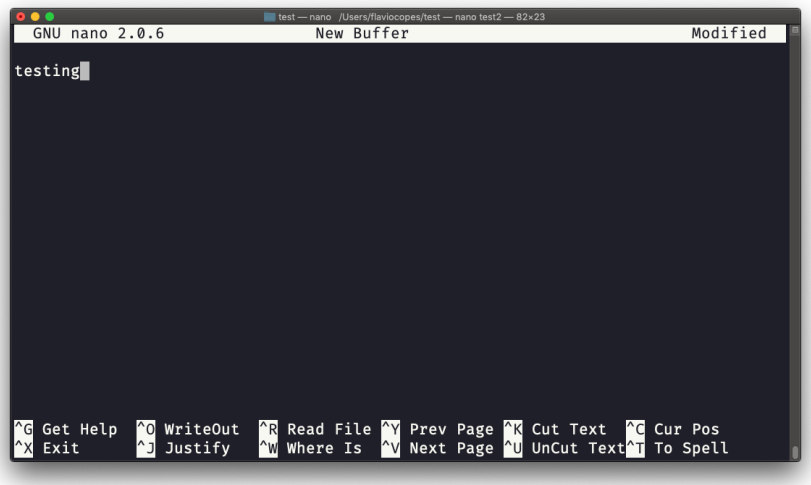
nano

`nano` is a beginner friendly editor.

Run it using `nano <filename>`.

You can directly type characters into the file without worrying about modes.

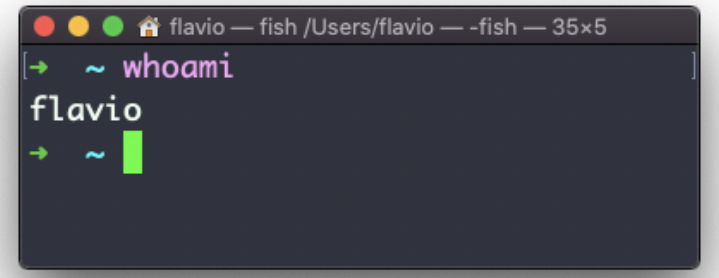
You can quit without editing using `ctrl-X`. If you edited the file buffer, the editor will ask you for confirmation and you can save the edits, or discard them. The help at the bottom shows you the keyboard commands that let you work with the file:



`pico` is more or less the same, although `nano` is the GNU version of `pico` which at some point in history was not open source and the `nano` clone was made to satisfy the GNU operating system license requirements.

whoami

Type `whoami` to print the user name currently logged in to the terminal session:

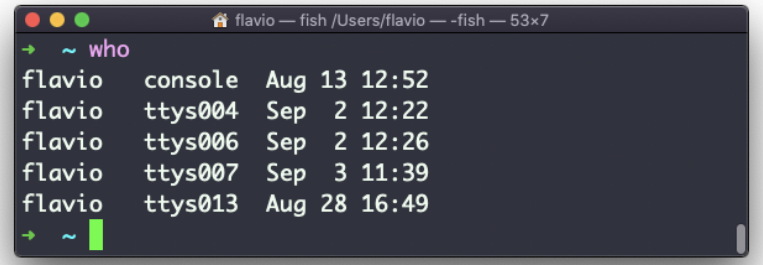


> Note: this is different from the `who am i` command, which prints more information

who

The `who` command displays the users logged in to the system.

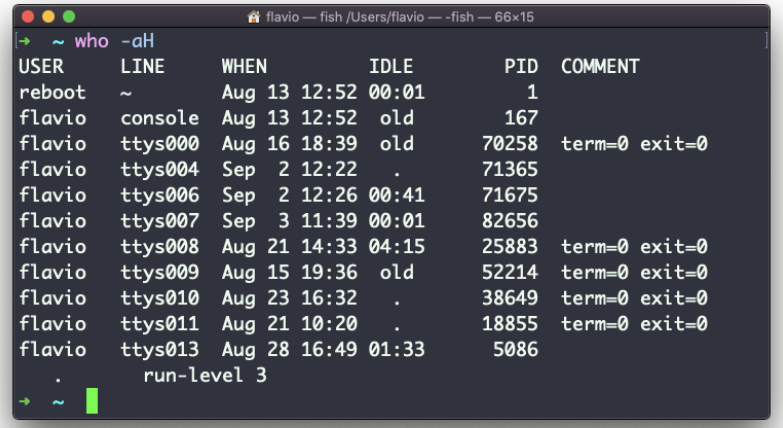
Unless you're using a server multiple people have access to, chances are you will be the only user logged in, multiple times:



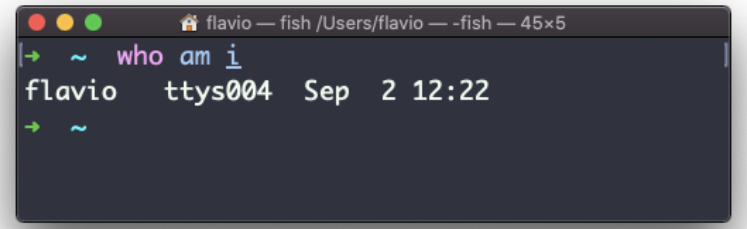
Why multiple times? Because each shell opened will count as an access.

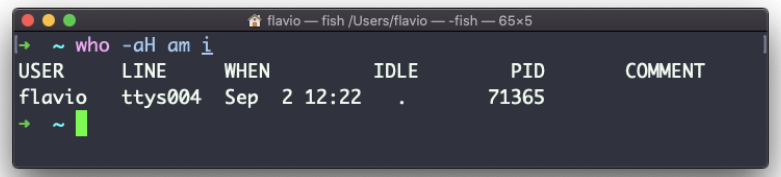
You can see the name of the terminal used, and the time/day the session was started.

The `-aH` flags will tell `who` to display more information, including the idle time and the process ID of the terminal:



The special `who am i` command will list the current terminal session details:





su

While you're logged in to the terminal shell with one user, you might have the need to switch to another user.

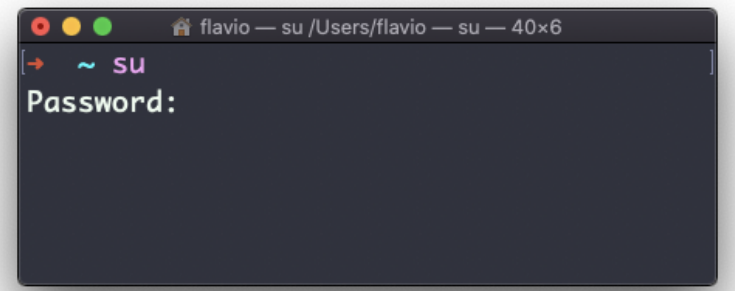
For example you're logged in as root to perform some maintenance, but then you want to switch to a user account.

You can do so with the `su` command:

```
su <username>
```

For example: `su flavio`.

If you're logged in as a user, running `su` without anything else will prompt to enter the `root` user password, as that's the default behavior.



`su` will start a new shell as another user.

When you're done, typing `exit` in the shell will close that shell, and will return back to the current user's shell.

sudo

`sudo` is commonly used to run a command as root.

You must be enabled to use `sudo`, and once you do, you can run commands as root by entering your user's password (*not* the root user password).

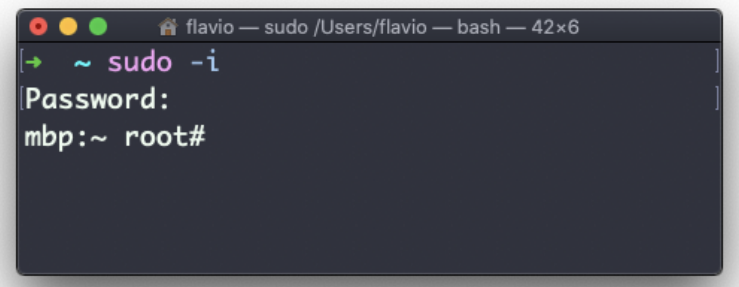
The permissions are highly configurable, which is great especially in a multi-user server environment, and some users can be granted access to running specific commands through `sudo`.

For example you can edit a system configuration file:

```
sudo nano /etc/hosts
```

which would otherwise fail to save since you don't have the permissions for it.

You can run `sudo -i` to start a shell as root:



You can use `sudo` to run commands as any user. `root` is the default, but use the `-u` option to specify another user:

```
sudo -u flavio ls /Users/flavio
```

passwd

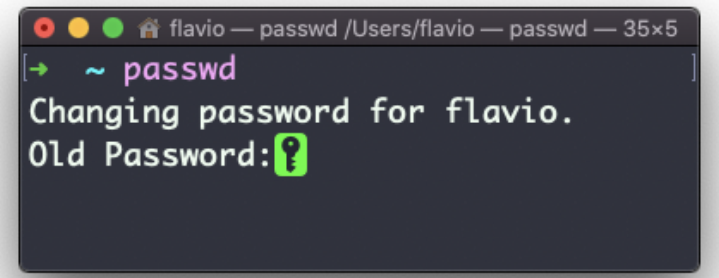
Users in Linux have a password assigned. You can change the password using the `passwd` command.

There are two situations here.

The first is when you want to change your password. In this case you type:

```
passwd
```

and an interactive prompt will ask you for the old password, then it will ask you for the new one:



When you're `root` (or have superuser privileges) you can set the username of which you want to change the password:

```
passwd <username> <new password>
```

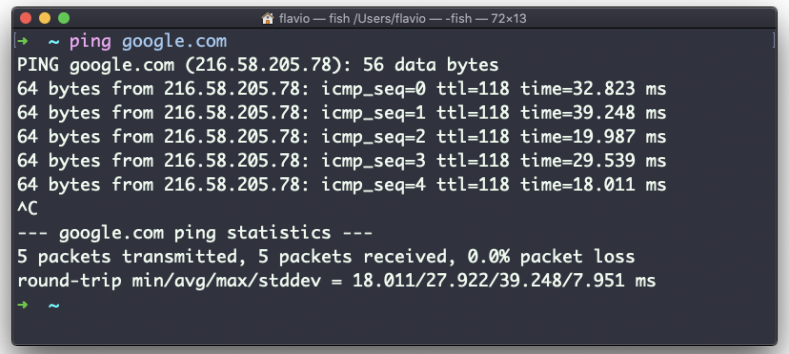
In this case you don't need to enter the old one.

ping

The `ping` command pings a specific network host, on the local network or on the Internet.

You use it with the syntax `ping <host>` where `<host>` could be a domain name, or an IP address.

Here's an example pinging `google.com`:



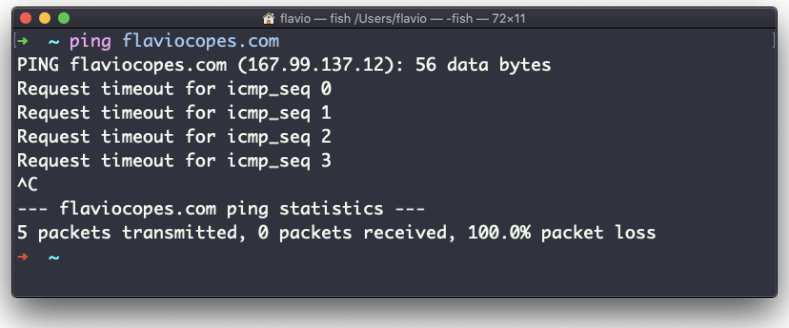
The commands sends a request to the server, and the server returns a response.

`ping` keep sending the request every second, by default, and will keep running until you stop it with `ctrl-C`, unless you pass the number of times you want to try with the `-c` option: `ping -c 2 google.com`.

Once `ping` is stopped, it will print some statistics about the results: the percentage of packages lost, and statistics about the network performance.

As you can see the screen prints the host IP address, and the time that it took to get the response back.

Not all servers support pinging, in case the requests times out:



Sometimes this is done on purpose, to "hide" the server, or just to reduce the load. The ping packets can also be filtered by firewalls.

`ping` works using the **ICMP protocol** (*Internet Control Message Protocol*), a network layer protocol just like TCP or UDP.

The request sends a packet to the server with the `ECHO_REQUEST` message, and the server returns a `ECHO_REPLY` message. I won't go into details, but this is the basic concept.

Pinging a host is useful to know if the host is reachable (supposing it implements ping), and how distant it is in terms of how long it takes to get back to you. Usually the nearest the server is geographically, the less time it will take to return back to you, for simple physical laws that cause a longer distance to introduce more delay in the cables.

traceroute

When you try to reach a host on the Internet, you go through your home router, then you reach your ISP network, which in turn goes through its own upstream network router, and so on, until you finally reach the host.

Have you ever wanted to know what are the steps that your packets go through to do that?

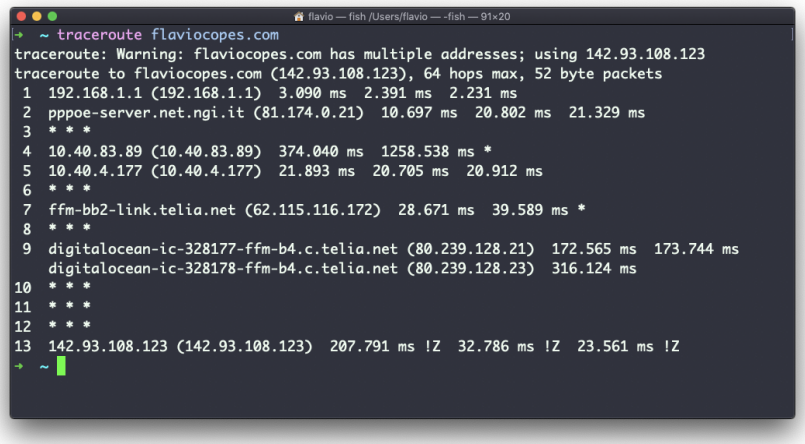
The `traceroute` command is made for this.

You invoke

```
traceroute <host>
```

and it will (slowly) gather all the information while the packet travels.

In this example I tried reaching for my blog with `traceroute flaviocopes.com`:

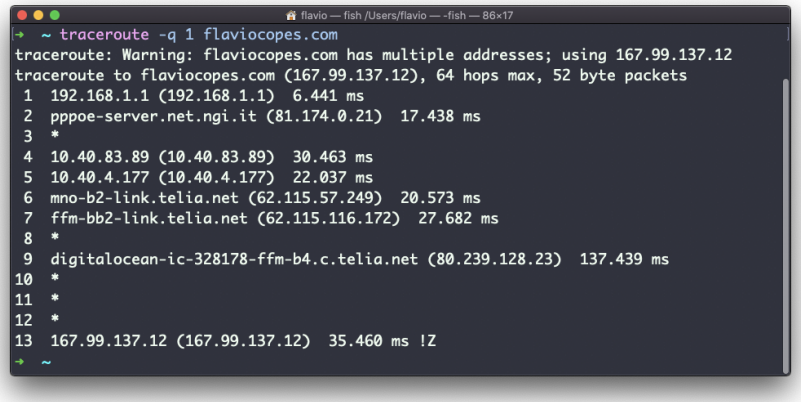


Not every router travelled returns us information. In this case, `traceroute` prints `* * *`. Otherwise, we can see the hostname, the IP address, and some performance indicator.

For every router we can see 3 samples, which means traceroute tries by default 3 times to get you a good indication of the time needed to reach it. This is why it takes this long to execute `traceroute` compared to simply doing a `ping` to that host.

You can customize this number with the `-q` option:

```
traceroute -q 1 flaviocopes.com
```



clear

Type `clear` to clear all the previous commands that were ran in the current terminal.

The screen will clear and you will just see the prompt at the top:

> Note: this command has a handy shortcut: `ctrl-L`

Once you do that, you will lose access to scrolling to see the output of the previous commands entered.

So you might want to use `clear -x` instead, which still clears the screen, but lets you go back to see the previous work by scrolling up.

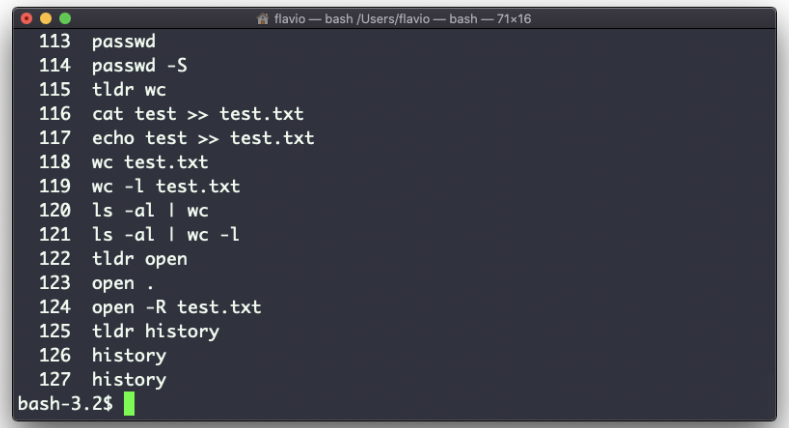
history

Every time we run a command, that's memorized in the history.

You can display all the history using:

```
history
```

This shows the history with numbers:

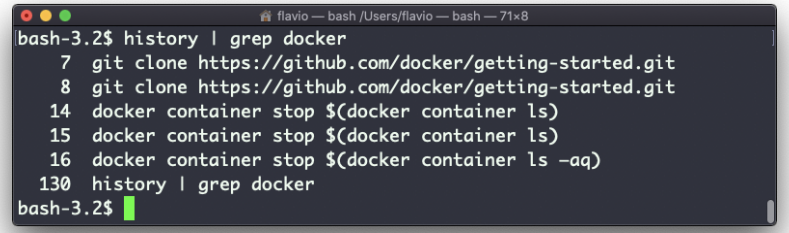


You can use the syntax `!<command number>` to repeat a command stored in the history, in the above example typing `!121` will repeat the `ls -al | wc -l` command.

Typically the last 500 commands are stored in the history.

You can combine this with `grep` to find a command you ran:

```
history | grep docker
```



To clear the history, run `history -c`

export

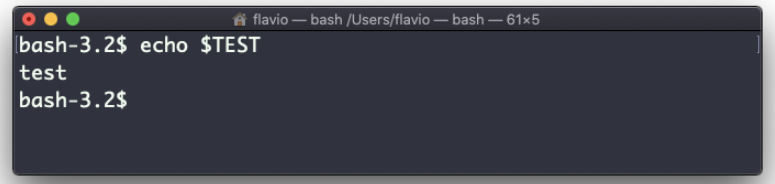
The `export` command is used to export variables to child processes.

What does this mean?

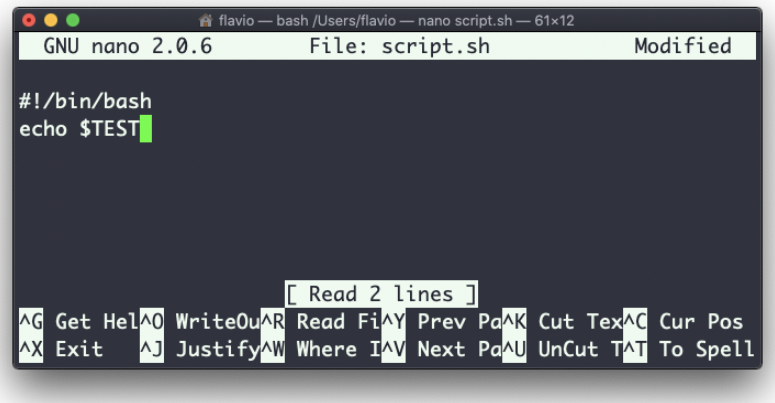
Suppose you have a variable TEST defined in this way:

```
TEST="test"
```

You can print its value using `echo $TEST`:



But if you try defining a Bash script in a file `script.sh` with the above command:



Then you set `chmod u+x script.sh` and you execute this script with `./script.sh`, the `echo $TEST` line will print nothing!

This is because in Bash the `TEST` variable was defined local to the shell. When executing a shell script or another command, a subshell is launched to execute it, which does not contain the current shell local variables.

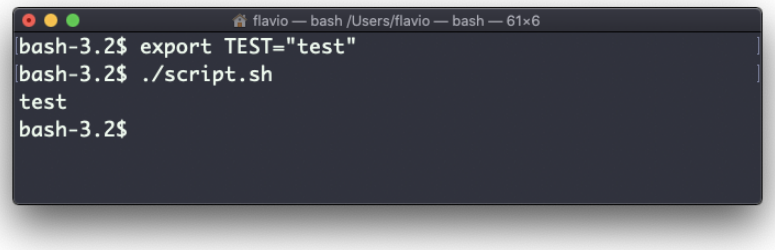
To make the variable available there we need to define `TEST` not in this way:

```
TEST="test"
```

but in this way:

```
export TEST="test"
```

Try that, and running `./script.sh` now should print "test":



Sometimes you need to append something to a variable. It's often done with the `PATH` variable. You use this syntax:

```
export PATH=$PATH:/new/path
```

It's common to use `export` when you create new variables in this way, but also when you create variables in the `.bash_profile` or `.bashrc` configuration files with Bash, or in `.zshenv` with Zsh.

To remove a variable, use the `-n` option:

```
export -n TEST
```

Calling `export` without any option will list all the exported variables.

crontab

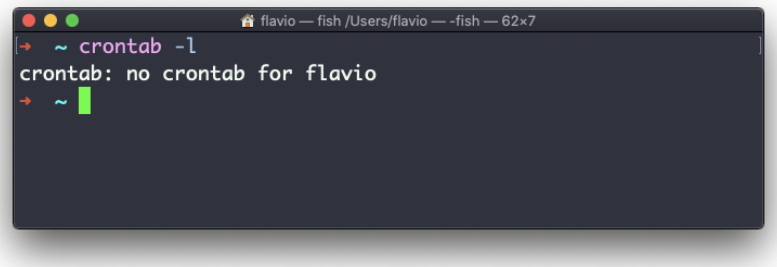
Cron jobs are jobs that are scheduled to run at specific intervals. You might have a command perform something every hour, or every day, or every 2 weeks. Or on weekends. They are very powerful, especially on servers to perform maintenance and automations.

The `crontab` command is the entry point to work with cron jobs.

The first thing you can do is to explore which cron jobs are defined by you:

```
crontab -l
```

You might have none, like me:



Run

```
crontab -e
```

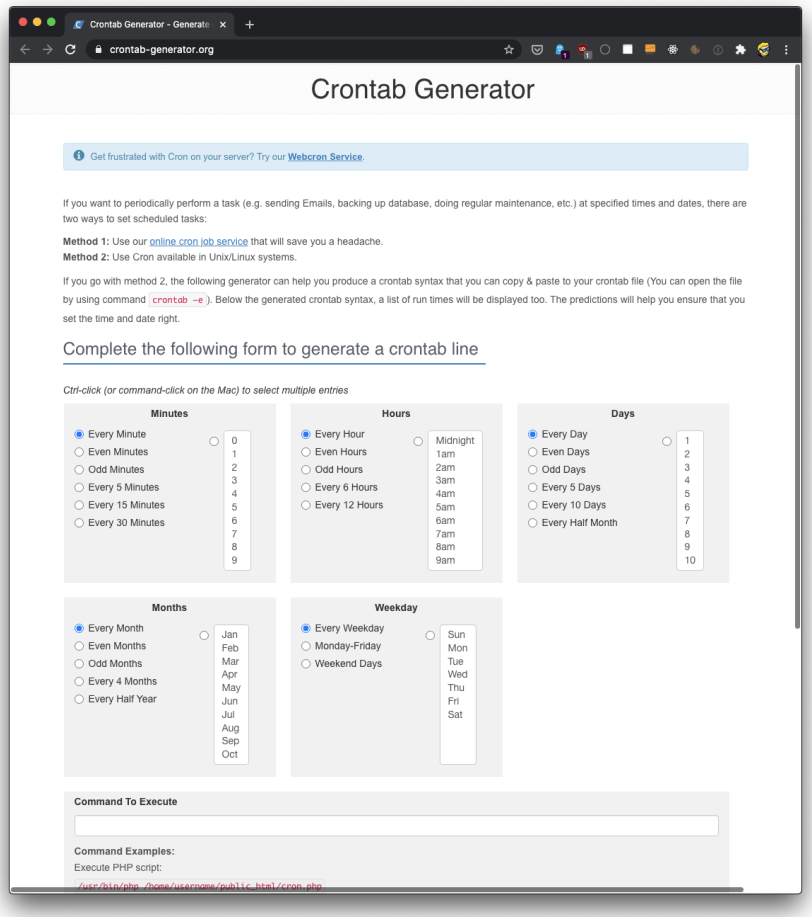
to edit the cron jobs, and add new ones.

By default this opens with the default editor, which is usually `vim`. I like `nano` more, you can use this line to use a different editor:

```
EDITOR=nano crontab -e
```

Now you can add one line for each cron job.

The syntax to define cron jobs is kind of scary. This is why I usually use a website to help me generate it without errors: https://crontab-generator.org/



You pick a time interval for the cron job, and you type the command to execute.

I chose to run a script located in `/Users/flavio/test.sh` every 12 hours. This is the crontab line I need to run:

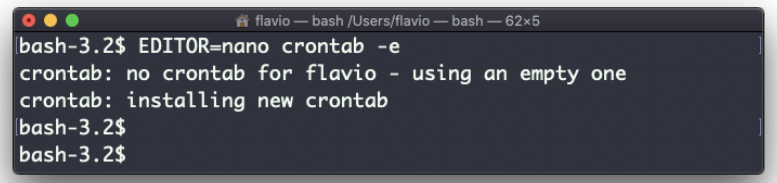
```
* */12 * * * /Users/flavio/test.sh >/dev/null 2>&1
```

I run `crontab -e`:

```
EDITOR=nano crontab -e
```

and I add that line, then I press `ctrl-X` and press `y` to save.

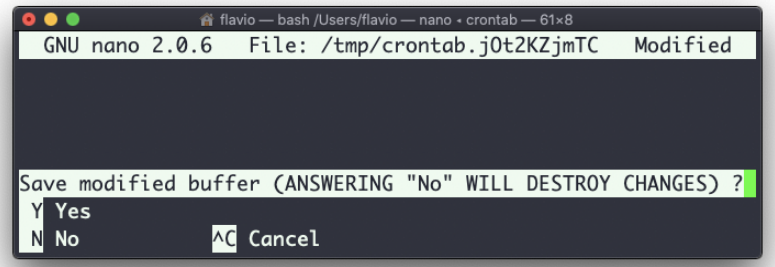
If all goes well, the cron job is set up:

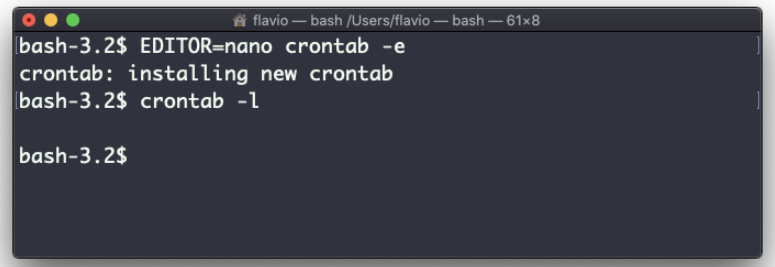


Once this is done, you can see the list of active cron jobs by running:

```
crontab -l
```

You can remove a cron job running `crontab -e` again, removing the line and exiting the editor:



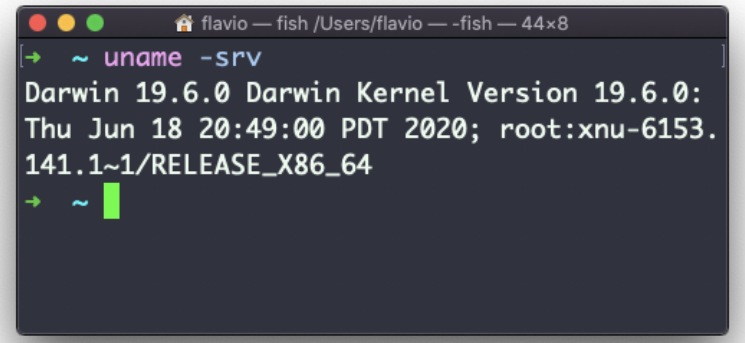


uname

Calling `uname` without any options will return the Operating System codename:

The `m` option shows the hardware name (`x86_64` in this example) and the `p` option prints the processor architecture name (`i386` in this example):

The `s` option prints the Operating System name. `r` prints the release, `v` prints the version:

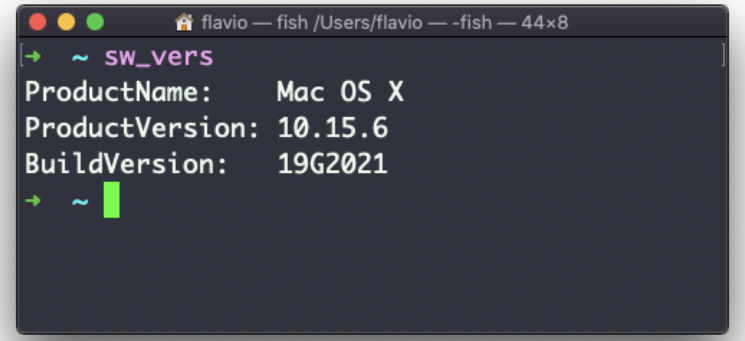


The `n` option prints the node network name:

The `a` option prints all the information available:

On macOS you can also use the `sw_vers` command to print more information about the macOS Operating System. Note that this differs from the Darwin (the Kernel) version, which above is `19.6.0`.

> Darwin is the name of the kernel of macOS. The kernel is the "core" of the Operating System, while the Operating System as a whole is called macOS. In Linux, Linux is the kernel, GNU/Linux would be the Operating System name, although we all refer to it as "Linux"



env

The `env` command can be used to pass environment variables without setting them on the outer environment (the current shell).

Suppose you want to run a Node.js app and set the `USER` variable to it.

You can run

```
env USER=flavio node app.js
```

and the `USER` environment variable will be accessible from the Node.js app via the Node `process.env` interface.

You can also run the command clearing all the environment variables already set, using the `-i` option:

```
env -i node app.js
```

In this case you will get an error saying `env: node: No such file or directory` because the `node` command is not reachable, as the `PATH` variable used by the shell to look up commands in the common paths is unset.

So you need to pass the full path to the `node` program:

```
env -i /usr/local/bin/node app.js
```

Try with a simple `app.js` file with this content:

```
console.log(process.env.NAME)
console.log(process.env.PATH)
```

You will see the output being

```
undefined
undefined
```

You can pass an env variable:

```
env -i NAME=flavio node app.js
```

and the output will be

```
flavio
undefined
```

Removing the `-i` option will make `PATH` available again inside the program:

The `env` command can also be used to print out all the environment variables, if ran with no options:

```
env
```

it will return a list of the environment variables set, for example:

```
HOME=/Users/flavio
LOGNAME=flavio
PATH=/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin:/l
PWD=/Users/flavio
SHELL=/usr/local/bin/fish
```

You can also make a variable inaccessible inside the program you run, using the `-u` option, for example this code removes the `HOME` variable from the command environment:

```
env -u HOME node app.js
```

printenv

A quick guide to the `printenv` command, used to print the values of environment variables

In any shell there are a good number of environment variables, set either by the system, or by your own shell scripts and configuration.

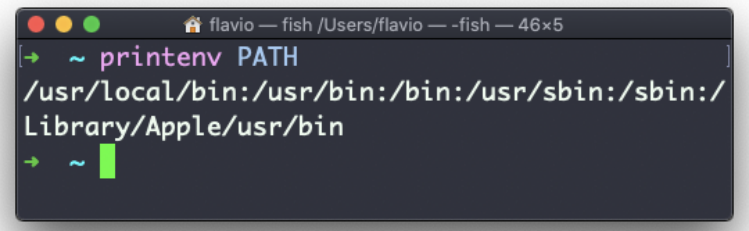
You can print them all to the terminal using the `printenv` command. The output will be something like this:

```
HOME=/Users/flavio
LOGNAME=flavio
PATH=/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin:/l
PWD=/Users/flavio
SHELL=/usr/local/bin/fish
```

with a few more lines, usually.

You can append a variable name as a parameter, to only show that variable value:

```
printenv PATH
```



Conclusion

Thanks a lot for reading this book.

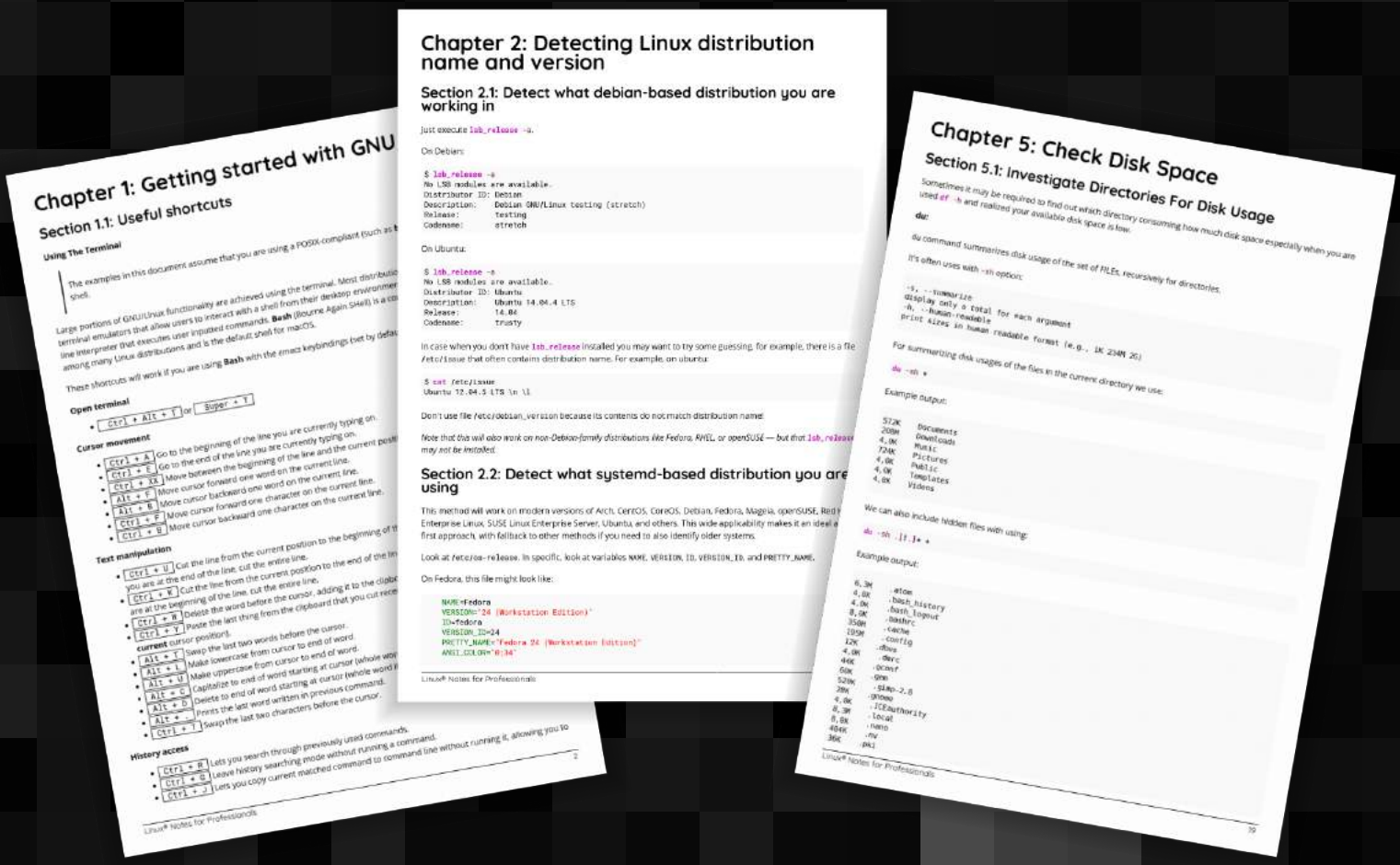
I hope it will inspire you to know more about Linux and its capabilities.

To learn more, check out my blog flaviocopes.com.

Send any feedback, errata or opinions at hey@flaviocopes.com

Linux®

Notes for Professionals



50+ pages

of professional hints and tricks

GoalKicker.com
Free Programming Books

Disclaimer
This is an unofficial free book created for educational purposes and is not affiliated with official Linux® group(s) or company(s). All trademarks and registered trademarks are the property of their respective owners

Contents

About 1

Chapter 1: Getting started with GNU/Linux 2

Section 1.1: Useful shortcuts 2

Section 1.2: File Management Commands 3

Section 1.3: Hello World 5

Section 1.4: Basic Linux Utilities 5

Section 1.5: Searching for files by patterns in name/contents 6

Section 1.6: File Manipulation 7

Section 1.7: File/Directory details 8

Chapter 2: Detecting Linux distribution name and version 11

Section 2.1: Detect what debian-based distribution you are working in 11

Section 2.2: Detect what systemd-based distribution you are using 11

Section 2.3: Detect what RHEL / CentOS / Fedora distribution you are working in 12

Section 2.4: Uname - Print information about the current system 13

Section 2.5: Detect basic information about your distro 13

Section 2.6: Using GNU coreutils 13

Section 2.7: Find your linux os (both debian & rpm) name and release number 14

Chapter 3: Getting information on a running Linux kernel 15

Section 3.1: Getting details of Linux kernel 15

Chapter 4: Shell 16

Section 4.1: Changing default shell 16

Section 4.2: Basic Shell Utilities 17

Section 4.3: Create Your Own Command Alias 18

Section 4.4: Locate a file on your system 18

Chapter 5: Check Disk Space 19

Section 5.1: Investigate Directories For Disk Usage 19

Section 5.2: Checking Disk Space 21

Chapter 6: Getting System Information 23

Section 6.1: Statistics about CPU, Memory, Network and Disk (I/O operations) 23

Section 6.2: Using tools like lscpu and lshw 23

Section 6.3: List Hardware 24

Section 6.4: Find CPU model/speed information 25

Section 6.5: Process monitoring and information gathering 26

Chapter 7: ls command 28

Section 7.1: Options for ls command 28

Section 7.2: ls command with most used options 28

Chapter 8: File Compression with 'tar' command 30

Section 8.1: Compress a folder 30

Section 8.2: Extract a folder from an archive 30

Section 8.3: List contents of an archive 30

Section 8.4: List archive content 31

Section 8.5: Compress and exclude one or multiple folder 31

Section 8.6: Strip leading components 31

Chapter 9: Services 32

Section 9.1: List running service on Ubuntu 32

Section 9.2: Systemd service management 32

Chapter 10: Managing Services ... 33
Section 10.1: Diagnosing a problem with a service ... 33
Section 10.2: Starting and Stopping Services ... 33
Section 10.3: Getting the status of a service ... 34
Chapter 11: Modifying Users ... 35
Section 11.1: Setting your own password ... 35
Section 11.2: Setting another user's password ... 35
Section 11.3: Adding a user ... 35
Section 11.4: Removing a user ... 35
Section 11.5: Removing a user and its home folder ... 35
Section 11.6: Listing groups the current user is in ... 35
Section 11.7: Listing groups a user is in ... 35
Chapter 12: LAMP Stack ... 36
Section 12.1: Installing LAMP on Arch Linux ... 36
Section 12.2: Installing LAMP on Ubuntu ... 37
Section 12.3: Installing LAMP stack on CentoOS ... 38
Chapter 13: tee command ... 40
Section 13.1: Write output to stdout, and also to a file ... 40
Section 13.2: Write output from the middle of a pipe chain to a file and pass it back to the pipe ... 40
Section 13.3: write the output to multiple files ... 40
Section 13.4: Instruct tee command to append to the file ... 40
Chapter 14: Secure Shell (SSH) ... 42
Section 14.1: Connecting to a remote server ... 42
Section 14.2: Installing OpenSSH suite ... 42
Section 14.3: Configuring an SSH server to accept connections ... 43
Section 14.4: Passwordless connection (using a key pair) ... 43
Section 14.5: Generate public and private key ... 43
Section 14.6: Disable ssh service ... 43
Chapter 15: SCP ... 45
Section 15.1: Secure Copy ... 45
Section 15.2: Basic Usage ... 45
Chapter 16: GnuPG (GPG) ... 46
Section 16.1: Exporting your public key ... 46
Section 16.2: Create and use a GnuPG key quickly ... 46
Chapter 17: Network Configuration ... 47
Section 17.1: Local DNS resolution ... 47
Section 17.2: Configure DNS servers for domain name resolution ... 47
Section 17.3: See and manipulate routes ... 47
Section 17.4: Configure a hostname for some other system on your network ... 48
Section 17.5: Interface details ... 49
Section 17.6: Adding IP to an interface ... 50
Chapter 18: Midnight Commander ... 52
Section 18.1: Midnight Commander function keys in browsing mode ... 52
Section 18.2: Midnight Commander function keys in file editing mode ... 52
Chapter 19: Change root (chroot) ... 54
Section 19.1: Requirements ... 54
Section 19.2: Manually changing root in a directory ... 54
Section 19.3: Reasons to use chroot ... 55
Chapter 20: Package Managers ... 56

Section 20.1: How to update packages with the apt package manager 56
Section 20.2: How to install a package with the pacman package manager 56
Section 20.3: How to update packages with the pacman package manager 56
Section 20.4: How to update packages with yum 57
Chapter 21: Compiling the Linux kernel 58
Section 21.1: Compilation of Linux Kernel on Ubuntu 58
Credits 59
You may also like 61

About

Please feel free to share this PDF with anyone for free,
latest version of this book can be downloaded from:
https://goalkicker.com/LinuxBook

This *Linux® Notes for Professionals* book is compiled from Stack Overflow Documentation, the content is written by the beautiful people at Stack Overflow. Text content is released under Creative Commons BY-SA, see credits at the end of this book whom contributed to the various chapters. Images may be copyright of their respective owners unless otherwise specified

This is an unofficial free book created for educational purposes and is not affiliated with official Linux® group(s) or company(s) nor Stack Overflow. All trademarks and registered trademarks are the property of their respective company owners

The information presented in this book is not guaranteed to be correct nor accurate, use at your own risk

Please send feedback and corrections to web@petercv.com

Chapter 1: Getting started with GNU/Linux

Section 1.1: Useful shortcuts

Using The Terminal

> The examples in this document assume that you are using a POSIX-compliant (such as **bash**, **sh**, **zsh**, **ksh**) shell.

Large portions of GNU/Linux functionality are achieved using the terminal. Most distributions of Linux include terminal emulators that allow users to interact with a shell from their desktop environment. A shell is a command-line interpreter that executes user inputted commands. **Bash** (Bourne Again SHell) is a common default shell among many Linux distributions and is the default shell for macOS.

These shortcuts will work if you are using **Bash** with the *emacs* keybindings (set by default):

Open terminal

- `Ctrl + Alt + T` or `Super + T`

Cursor movement

- `Ctrl + A` Go to the beginning of the line you are currently typing on.
- `Ctrl + E` Go to the end of the line you are currently typing on.
- `Ctrl + XX` Move between the beginning of the line and the current position of the cursor.
- `Alt + F` Move cursor forward one word on the current line.
- `Alt + B` Move cursor backward one word on the current line.
- `Ctrl + F` Move cursor forward one character on the current line.
- `Ctrl + B` Move cursor backward one character on the current line.

Text manipulation

- `Ctrl + U` Cut the line from the current position to the beginning of the line, adding it to the clipboard. If you are at the end of the line, cut the entire line.
- `Ctrl + K` Cut the line from the current position to the end of the line, adding it to the clipboard. If you are at the beginning of the line, cut the entire line.
- `Ctrl + W` Delete the word before the cursor, adding it to the clipboard.
- `Ctrl + Y` Paste the last thing from the clipboard that you cut recently (undo the last delete at the **current** cursor position).
- `Alt + T` Swap the last two words before the cursor.
- `Alt + L` Make lowercase from cursor to end of word.
- `Alt + U` Make uppercase from cursor to end of word.
- `Alt + C` Capitalize to end of word starting at cursor (whole word if cursor is at the beginning of word).
- `Alt + D` Delete to end of word starting at cursor (whole word if cursor is at the beginning of word).
- `Alt + .` Prints the last word written in previous command.
- `Ctrl + T` Swap the last two characters before the cursor.

History access

- `Ctrl + R` Lets you search through previously used commands.
- `Ctrl + G` Leave history searching mode without running a command.
- `Ctrl + J` Lets you copy current matched command to command line without running it, allowing you to

make modifications before running the command.
- `Alt + R` Revert any changes to a command you've pulled from your history, if you've edited it.
- `Ctrl + P` Shows last executed command, i.e. walk back through the command history (Similar to up arrow).
- `Ctrl + N` Shows next executed command, i.e. walk forward through the command history (Similar to down arrow).

Terminal control

- `Ctrl + L` Clears the screen, similar to the clear command.
- `Ctrl + S` Stop all output to the screen. This is useful when running commands with lots of long output. But this doesn't stop the running command.
- `Ctrl + Q` Resume output to the screen after stopping it with Ctrl+S.
- `Ctrl + C` End currently running process and return the prompt.
- `Ctrl + D` Log out of the current shell session, similar to the exit or logout command. In some commands, acts as End of File signal to indicate that a file end has been reached.
- `Ctrl + Z` Suspends (pause) currently running foreground process, which returns shell prompt. You can then use `bg` command allowing that process to run in the background. To again bring that process to foreground, use `fg` command. To view all background processes, use `jobs` command.
- `Tab` Auto-complete files and directory names.
- `Tab Tab` Shows all possibilities, when typed characters doesn't uniquely match to a file or directory name.

Special characters

- `Ctrl + H` Same as Backspace.
- `Ctrl + J` Same as Return (historically Line Feed).
- `Ctrl + M` Same as Return (historically Carriage Return).
- `Ctrl + I` Same as Tab.
- `Ctrl + G` Bell Character.
- `Ctrl + @` Null Character.
- `Esc` Deadkey equivalent to the `Alt` modifier.

Close Terminal

- `Ctrl + Shift + W` To close terminal tab.
- `Ctrl + Shift + Q` To close entire terminal.

Alternatively, you can switch to the *vi* keybindings in **bash** using `set -o vi`. Use `set -o emacs` to switch back to the *emacs* keybindings.

Section 1.2: File Management Commands

Linux uses some conventions for present and parent directories. This can be a little confusing for beginners.

Whenever you are in a terminal in Linux, you will be in what is called the *current working directory*. Often your command prompt will display either the full working directory, or just the last part of that directory. Your prompt could look like one of the following:

```
user@host ~/somedir $
user@host somedir $
user@host /home/user/somedir $
```

which says that your current working directory is `/home/user/somedir`.

In Linux `..` represents the parent directory and `.` represents the current directory.

Therefore, if the current directory is `/home/user/somedir`, then `cd ../somedir` will not change the working directory.

The table below lists some of the most used file management commands

Directory navigation

Command	Utility
`pwd`	Get the full path of the current working directory.
`cd -`	Navigate to the last directory you were working in.
`cd ~` or just `cd`	Navigate to the current user's home directory.
`cd ..`	Go to the parent directory of current directory (mind the space between `cd` and `..`)

Listing files inside a directory

Command	Utility
`ls -l`	List the files and directories in the current directory in long (table) format (It is recommended to use -l with ls for better readability).
`ls -ld dir-name`	List information about the directory `dir-name` instead of its contents.
`ls -a`	List all the files including the hidden ones (File names starting with a `.` are hidden files in Linux).
`ls -F`	Appends a symbol at the end of a file name to indicate its type (`*` means executable, `/` means directory, `@` means symbolic link, `=` means socket, `|` means named pipe, `>` means door).
`ls -lt`	List the files sorted by last modified time with most recently modified files showing at the top (remember -l option provides the long format which has better readability).
`ls -lh`	List the file sizes in human readable format.
`ls -lR`	Shows all subdirectories recursively.
`tree`	Will generate a tree representation of the file system starting from the current directory.

File/directory create, copy and remove

Command	Utility
`cp -p source destination`	Will copy the file from `source` to *destination*. -p stands for preservation. It preserves the original attributes of file while copying like file owner, timestamp, group, permissions etc.
`cp -R source_dir destination_dir`	Will copy source directory to specified destination recursively.
`mv file1 file2`	In Linux there is no **rename** command as such. Hence `mv` moves/renames the file1 to file2.
`rm -i filename`	Asks you before every file removal for confirmation. **IF YOU ARE A NEW USER TO LINUX COMMAND LINE, YOU SHOULD ALWAYS USE `rm -i`.** You can specify multiple files.
`rm -R dir-name`	Will remove the directory `dir-name` recursively.
`rm -rf dir-name`	Will remove the directory `dir` recursively, ignoring non-existent files and will never prompt for anything. **BE CAREFUL USING THIS COMMAND!** You can specify multiple directories.
`rmdir dir-name`	Will remove the directory `dir-name`, if it's empty. This command can only remove empty directories.
`mkdir dir-name`	Create a directory `dir-name`.
`mkdir -p dir-name/dir-name`	Create a directory hierarchy. Create parent directories as needed, if they don't exist. You can specify multiple directories.
`touch filename`	Create a file `filename`, if it doesn't exist, otherwise change the timestamp of the file to current time.

File/directory permissions and groups

Command	Utility

`chmod <specification> filename`	Change the file permissions. Specifications = `u` user, `g` group, `o` other, `+` add permission, `-` remove, `r` read, `w` write,`x` execute.
`chmod -R <specification> dir-name`	Change the permissions of a directory recursively. To change permission of a directory and everything within that directory, use this command.
`chmod go=+r myfile`	Add read permission for the owner and the group.
`chmod a +rwx myfile`	Allow all users to read, write or execute `myfile`.
`chmod go -r myfile`	Remove read permission from the group and others.
`chown owner1 filename`	Change ownership of a file to user `owner1`.
`chgrp grp_owner filename`	Change primary group ownership of file `filename` to group `grp_owner`.
`chgrp -R grp_owner dir-name`	Change primary group ownership of directory `dir-name` to group `grp_owner` recursively. To change group ownership of a directory and everything within that directory, use this command.

Section 1.3: Hello World

Type the following code into your terminal, then press `Enter`:

```
echo "Hello World"
```

This will produce the following output:

```
Hello World
```

Section 1.4: Basic Linux Utilities

Linux has a command for almost any tasks and most of them are intuitive and easily interpreted.

Getting Help in Linux

Command	Usability
`man <name>`	Read the manual page of <name>.
`man <section> <name>`	Read the manual page of <name>, related to the given section.
`man -k <editor>`	Output all the software whose man pages contain <editor> keyword.
`man -K <keyword>`	Outputs all man pages containing <keyword> within them.
`apropos <editor>`	Output all the applications whose one line description matches the word *editor*. When **not able to recall** the name of the application, use this command.
`help`	In Bash shell, this will display the list of all available bash commands.
`help <name>`	In Bash shell, this will display the info about the <name> bash command.
`info <name>`	View all the information about <name>.
`dpkg -l`	Output a list of all installed packages on a Debian-based system.
`dpkg -L packageName`	Will list out the files installed and path details for a given package on Debian.
`dpkg -l | grep -i <edit>`	Return all .deb installed packages with <edit> irrespective of cases.
`less /var/lib/dpkg/available`	Return descriptions of all available packages.
`whatis vim`	List a one-line description of vim.
`<command-name> --help`	Display usage information about the <tool-name>. Sometimes `command -h` also works, but not for all commands.

User identification and who is who in Linux world

Command	Usability
`hostname`	Display hostname of the system.

`hostname -f`	Displays Fully Qualified Domain Name (FQDN) of the system.
`passwd`	Change password of current user.
`whoami`	Username of the users logged in at the terminal.
`who`	List of all the users currently logged in as a user.
`w`	Display current system status, time, duration, list of users currently logged in on system and other user information.
`last`	Who recently used the system.
`last root`	When was the last time **root** logged in as user.
`lastb`	Shows all bad login attempts into the system.
`chmod`	Changing permissions - read,write,execute of a file or directory.

Process related information

Command	Usability
`top`	List all processes sorted by their current system resource usage. Displays a continually updated display of processes (By default 3 seconds). Use `q` key to exit top.
`ps`	List processes currently running on current shell session
`ps -u root`	List all of the processes and commands root is running
`ps aux`	List all the processes by all users on the current system

Section 1.5: Searching for files by patterns in name/contents

A common and task of someone using the Linux Command Line (shell) is to search for files/directories with a certain name or containing certain text. There are 2 commands you should familiarise yourself with in order to accomplish this:

Find files by name

```
find /var/www -name '*.css'
```

This will print out the full path/filename to all files under `/var/www` that end in `.css`. Example output:

```
/var/www/html/text-cursor.css
/var/www/html/style.css
```

For more info:

```
man find
```

Find files containing text

```
grep font /var/www/html/style.css
```

This will print all lines containing the pattern `font` in the specified file. Example output:

```
font-weight: bold;
font-family: monospace;
```

Another example:

```
grep font /var/www/html/
```

This doesn't work as you'd hoped. You get:

```
grep: /var/www/html/: Is a directory
```

You need to `grep` recursively to make it work, using the `-R` option:

```
grep -R font /var/www/html/
```

Hey nice! Check out the output of this one:

```
/var/www/html/admin/index.php:  echo '<font color=red><b>Error: no dice</b></font><br/>';
/var/www/html/admin/index.php:  echo '<font color=red><b>Error: try again</b></font><br/>';
/var/www/html/style.css:  font-weight: bold;
/var/www/html/style.css:  font-family: monospace;
```

Notice that when `grep` is matching multiple files, it prefixes the matched lines with the filenames. You can use the `-h` option to get rid of that, if you want.

For more info:

```
man grep
```

Section 1.6: File Manipulation

Files and directories (another name for folders) are at the heart of Linux, so being able to create, view, move, and delete them from the command line is very important and quite powerful. These file manipulation commands allow you to perform the same tasks that a graphical file explorer would perform.

Create an empty text file called `myFile`:

```
touch myFile
```

Rename `myFile` to `myFirstFile`:

```
mv myFile myFirstFile
```

View the contents of a file:

```
cat myFirstFile
```

View the content of a file with pager (one screenful at a time):

```
less myFirstFile
```

View the first several lines of a file:

```
head myFirstFile
```

View the last several lines of a file:

```
tail myFirstFile
```

Edit a file:

```
vi myFirstFile
```

See what files are in your current working directory:

```
ls
```

Create an empty directory called `myFirstDirectory`:

```
mkdir myFirstDirectory
```

Create multi path directory: (creates two directories, src and myFirstDirectory)

```
mkdir -p src/myFirstDirectory
```

Move the file into the directory:

```
mv myFirstFile myFirstDirectory/
```

You can also rename the file:

```
user@linux-computer:~$ mv myFirstFile secondFileName
```

Change the current working directory to `myFirstDirectory`:

```
cd myFirstDirectory
```

Delete a file:

```
rm myFirstFile
```

Move into the parent directory (which is represented as `..`):

```
cd ..
```

Delete an empty directory:

```
rmdir myFirstDirectory
```

Delete a non-empty directory (i.e. contains files and/or other directories):

```
rm -rf myFirstDirectory
```

Make note that when deleting directories, that you delete `./` not `/` that will wipe your whole filesystem.

Section 1.7: File/Directory details

The `ls` command has several options that can be used together to show more information.

Details/Rights

The `l` option shows the file permissions, size, and last modified date. So if the root directory contained a dir called `test` and a file `someFile` the command:

```
user@linux-computer:~$ ls -l
```

Would output something like

```
-rw-r--r-- 1 user users   70 Jul 22 13:36 someFile.txt
drwxrwxrwx 2 user users 4096 Jul 21 07:18 test
```

The permissions are in format of `drwxrwxrwx`. The first character represents the file type `d` if it's a directory `-` otherwise. The next three `rwx` are the permissions the user has over the file, the next three are the permissions the group has over the file, and the last three are the permissions everyone else has over the file.

The `r` of `rwx` stands for if a file can be read, the `w` represents if the file can be modified, and the `x` stands for if the file can be executed. If any permission isn't granted a `-` will be in place of `r`, `w`, or `x`.

So from above `user` can read and modify `someFile.txt` but the group has only read-only rights.

To change rights you can use the `chmod ### fileName` command if you have sudo rights. `r` is represented by a value of 4, `w` is represented by 2, and `x` is represented by a 1. So if only you want to be able to modify the contents to the `test` directory

```
Owner rwx = 4+2+1 = 7
Group r-x = 4+0+1 = 5
Other r-x = 4+0+1 = 5
```

So the whole command is

```
chmod 755 test
```

Now doing a `ls -l` would show something like

```
drwxr-xr-x 2 user users 4096 Jul 21 07:20 test
```

Readable Size

Used in conjunction with the `l` option the `h` option shows file sizes that are human readable. Running

```
user@linux-computer:~$ ls -lh
```

Would output:

```
total 4166
-rw-r--r-- 1 user users   70 Jul 22 13:36 someFile.txt
drwxrwxrwx 2 user users 4.0K Jul 21 07:18 test
```

Hidden

To view hidden files use the `a` option. For example

```
user@linux-computer:~$ ls -a
```

Might list

```
.profile
someFile.txt
test
```

Total Directory Size

To view the size of the current directory use the `s` option (the `h` option can also be used to make the size more readable).

```
user@linux-computer:~$ ls -s
```

Outputs

```
total 4166
someFile.txt      test
```

Recursive View

Lets say `test` directory had a file `anotherFile` and you wanted to see it from the root folder, you could use the `R` option which would list the recursive tree.

```
user@linux-computer:~$ ls -R
```

Outputs

```
.:
someFile.txt    test

./test:
anotherFile
```

Chapter 2: Detecting Linux distribution name and version

Section 2.1: Detect what debian-based distribution you are working in

Just execute `lsb_release -a`.

On Debian:

```
$ lsb_release -a
No LSB modules are available.
Distributor ID: Debian
Description:    Debian GNU/Linux testing (stretch)
Release:        testing
Codename:       stretch
```

On Ubuntu:

```
$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 14.04.4 LTS
Release:        14.04
Codename:       trusty
```

In case when you don't have `lsb_release` installed you may want to try some guessing, for example, there is a file `/etc/issue` that often contains distribution name. For example, on ubuntu:

```
$ cat /etc/issue
Ubuntu 12.04.5 LTS \n \l
```

Don't use file `/etc/debian_version` because its contents do not match distribution name!

Note that this will also work on non-Debian-family distributions like Fedora, RHEL, or openSUSE — but that `lsb_release` may not be installed.

Section 2.2: Detect what systemd-based distribution you are using

This method will work on modern versions of Arch, CentOS, CoreOS, Debian, Fedora, Mageia, openSUSE, Red Hat Enterprise Linux, SUSE Linux Enterprise Server, Ubuntu, and others. This wide applicability makes it an ideal as a first approach, with fallback to other methods if you need to also identify older systems.

Look at `/etc/os-release`. In specific, look at variables `NAME`, `VERSION`, `ID`, `VERSION_ID`, and `PRETTY_NAME`.

On Fedora, this file might look like:

```
NAME=Fedora
VERSION="24 (Workstation Edition)"
ID=fedora
VERSION_ID=24
PRETTY_NAME="Fedora 24 (Workstation Edition)"
ANSI_COLOR="0;34"
```

```
CPE_NAME="cpe:/o:fedoraproject:fedora:24"
HOME_URL="https://fedoraproject.org/"
BUG_REPORT_URL="https://bugzilla.redhat.com/"
REDHAT_BUGZILLA_PRODUCT="Fedora"
REDHAT_BUGZILLA_PRODUCT_VERSION=24
REDHAT_SUPPORT_PRODUCT="Fedora"
REDHAT_SUPPORT_PRODUCT_VERSION=24
PRIVACY_POLICY_URL=https://fedoraproject.org/wiki/Legal:PrivacyPolicy
VARIANT="Workstation Edition"
VARIANT_ID=workstation
```

On CentOS, this file might look like this:

```
NAME="CentOS Linux"
VERSION="7 (Core)"
ID="centos"
ID_LIKE="rhel fedora"
VERSION_ID="7"
PRETTY_NAME="CentOS Linux 7 (Core)"
ANSI_COLOR="0;31"
CPE_NAME="cpe:/o:centos:centos:7"
HOME_URL="https://www.centos.org/"
BUG_REPORT_URL="https://bugs.centos.org/"

CENTOS_MANTISBT_PROJECT="CentOS-7"
CENTOS_MANTISBT_PROJECT_VERSION="7"
REDHAT_SUPPORT_PRODUCT="centos"
REDHAT_SUPPORT_PRODUCT_VERSION="7"
```

This file is documented on the freedesktop web site; in principle, it is not systemd specific — but it will exist on all systemd-based distributions.

From the bash shell, one can source the `/etc/os-release` file and then use the various variables directly, like this:

```
$ ( source /etc/os-release && echo "$PRETTY_NAME" )
Fedora 24 (Workstation Edition)
```

Section 2.3: Detect what RHEL / CentOS / Fedora distribution you are working in

Look at the contents of `/etc/redhat-release`

```
cat /etc/redhat-release
```

Here is the output from a Fedora 24 machine: `Fedora release 24 (Twenty Four)`

As mentioned in the debian-based response, you can also use the `lsb_release -a` command, which outputs this from a Fedora 24 machine:

```
LSB Version:    :core-4.1-amd64:core-4.1-noarch:cxx-4.1-amd64:cxx-4.1-noarch:desktop-4.1-
amd64:desktop-4.1-noarch:languages-4.1-amd64:languages-4.1-noarch:printing-4.1-amd64:printing-4.1-
noarch
Distributor ID:    Fedora
Description:    Fedora release 24 (Twenty Four)
Release:    24
Codename:    TwentyFour
```

Section 2.4: Uname - Print information about the current system

Uname is the short name for **u**nix **name**. Just type `uname` in console to get information about your operating system.

```
uname [OPTION]
```

If no *OPTION* is specified, `uname` assumes the `-s` option.

`-a` or `--all` - Prints all information, omitting `-p` and `-i` if the information is unknown.

Example:

```
> uname -a

SunOS hope 5.7 Generic_106541-08 sun4m sparc SUNW,SPARCstation-10
```

All the options:

-s, **--kernel-name**	Print the kernel name.
-n, **--nodename**	Print the network node hostname.
-r, **--kernel-release**	Print the kernel release.
-v, **--kernel-version**	Print the kernel version.
-m, **--machine**	Print the machine hardware name.
-p, **--processor**	Print the processor type, or "**unknown**".
-i, **--hardware-platform**	Print the hardware platform, or "**unknown**".
-o, **--operating-system**	Print the operating system.
--help	Display a help message, and exit.
--version	Display version information, and exit.

Section 2.5: Detect basic information about your distro

just execute `uname -a`.

On Arch:

```
$ uname -a
Linux nokia 4.6.4-1-ARCH #1 SMP PREEMPT Mon Jul 11 19:12:32 CEST 2016 x86_64       GNU/Linuxenter
code here
```

Section 2.6: Using GNU coreutils

So the GNU coreutils should be avaialable on all linux based systems (please correct me if I am wrong here).

If you do not know what system you are using you may not be able to directly jump to one of the examples above, hence this may be your first port of call.

```
$ uname -a
```

On my system this gives me the following...

```
Linux Scibearspace 3.16.0-4-amd64 #1 SMP Debian 3.16.7-ckt25-2+deb8u3 (2016-07-02) x86_64
```

```
GNU/Linux
```

Here you can see the following :

Scibearspace : the name of my pc

- Scibearspace : the name of my pc
- 3.16.0-4-amd64 : the kernel and architecture
- SMP Debian 3.16.7-CKT25-2+deb8u3 : tells me I am running debian with the 3.16 kernel
- Finaly the last part I am running debian 8 (update 3).

I would welcome any others to add in results for RHEL, and SuSe systems.

Section 2.7: Find your linux os (both debian & rpm) name and release number

Most of linux distros stores its version info in the /etc/lsb-release (debian) or /etc/redhat-release (RPM based) file. Using below generic command should get you past most of the Debian and RPM derivatives as Linux Mint and Cent-Os.

Example on Ubuntu Machine:

```
cat /etc/*release
```

```
DISTRIB_ID=Ubuntu
DISTRIB_RELEASE=14.04
DISTRIB_CODENAME=trusty
DISTRIB_DESCRIPTION="Ubuntu 14.04 LTS"
```

Chapter 3: Getting information on a running Linux kernel

Section 3.1: Getting details of Linux kernel

We can use command uname with various options to get complete details of running kernel.

```
uname -a
```

```
Linux df1-ws-5084 4.4.0-64-generic #85-Ubuntu SMP Mon Feb 20 11:50:30 UTC 2017 x86_64
x86_64 x86_64 GNU/Linux
```

As per man page here few more options

Usage: uname [OPTION]...
Print certain system information. With no OPTION, same as -s.

```
-a, --all                print all information, in the following order,
except omit -p and -i if unknown:
-s, --kernel-name        print the kernel name
-n, --nodename           print the network node hostname
-r, --kernel-release     print the kernel release
-v, --kernel-version     print the kernel version
-m, --machine            print the machine hardware name
-p, --processor          print the processor type (non-portable)
-i, --hardware-platform  print the hardware platform (non-portable)
-o, --operating-system   print the operating system
--help     display this help and exit
--version  output version information and exit
```

Chapter 4: Shell

The shell executes a program in response to its prompt. When you give a command, the shell searches for the program, and then executes it. For example, when you give the command ls, the shell searches for the utility/program named ls, and then runs it in the shell. The arguments and the options that you provide with the utilities can impact the result that you get. The shell is also known as a CLI, or command line interface.

Section 4.1: Changing default shell

Most modern distributions will come with BASH (**B**ourne **A**gain **SH**ell) pre-installed and configured as a default shell.

The command (actually an executable binary, an ELF) that is responsible for changing shells in Linux is `chsh` (**ch**ange **sh**ell).

We can first check which shells are already installed and configured on our machine by using the `chsh -l` command, which will output a result similar to this:

```
[user@localhost ~]$ chsh -l
/bin/sh
/bin/bash
/sbin/nologin
/usr/bin/sh
/usr/bin/bash
/usr/sbin/nologin
/usr/bin/fish
```

In some Linux distributions, `chsh -l` is invalid. In this case, the list of all available shells can be found at /etc/shells file. You can show the file contents with `cat`:

```
[user@localhost ~]$ cat /etc/shells
# /etc/shells: valid login shells
/bin/sh
/bin/bash
/sbin/nologin
/usr/bin/sh
/usr/bin/bash
/usr/sbin/nologin
/usr/bin/fish
```

Now we can choose our new default shell, e.g. `fish`, and configure it by using `chsh -s`,

```
[user@localhost ~]$ chsh -s /usr/bin/fish
Changing shell for user.
Password:
Shell changed.
```

Now all that is left to do is preform a logoff-logon cycle, and enjoy our new default shell.

If you wish to change the default shell for a different user, and you have administrative privileges on the machine, you'll be able to accomplish this by using `chsh` as `root`. So assuming we want to change `user_2`'s default shell to fish, we will use the same command as before, but with the addition of the other user's username, `chsh -s /usr/bin/fish user_2`.

In order to check what the current default shell is, we can view the `$SHELL` environment variable, which points to the path to our default shell, so after our change, we would expect to get a result similar to this,

```
 ~ ↱ echo $SHELL
/usr/bin/fish
```

`chsh` options:

`-s shell`

Sets shell as the login shell.

`-l`, `--list-shells`

Print the list of shells listed in /etc/shells and exit.

`-h`, `--help`

Print a usage message and exit.

`-v`, `--version`

Print version information and exit.

Section 4.2: Basic Shell Utilities

Customizing the Shell prompt

Default command prompt can be changed to look different and short. In case the current directory is long default command prompt becomes too large. Using `PS1` becomes useful in these cases. A short and customized command pretty and elegant. In the table below `PS1` has been used with a number of arguments to show different forms of shell prompts. Default command prompt looks something like this: `user@host ~ $` in my case it looks like this: `bruce@gotham ~ $`. It can changed as per the table below:

Command	Utility
PS1='\w $ '	`~ $` shell prompt as directory name. In this case root directory is Root.
PS1='\h $ '	`gotham $` shell prompt as hostname
PS1='\u $ '	`bruce $` shell prompt as username
PS1='\t $ '	`22:37:31 $` shell prompt in 24 hour format
PS1='@ $ '	`10:37 PM` shell prompt in 12 hour time format
PS1='! $ '	`732` will show the history number of command in place of shell prompt
PS1='dude $ '	`dude $` will show the shell prompt the way you like

Some basic shell commands

Command	Utility
`Ctrl-k`	cut/kill
`Ctrl-y`	yank/paste
`Ctrl-a`	will take cursor to the start of the line
`Ctrl-e`	will take cursor to the end of the line
`Ctrl-d`	will delete the character after/at the cursor
`Ctrl-l`	will clear the screen/terminal
`Ctrl-u`	will clear everything between prompt and the cursor
`Ctrl-_`	will undo the last thing typed on the command line
`Ctrl-c`	will interrupt/stop the job/process running in the foreground

`Ctrl-r`	reverse search in history
`~/.bash_history`	stores last 500 commands/events used on the shell
`history`	will show the command history
`history | grep <key-word>`	will show all the commands in history having keyword <key-word> (useful in cases when you remember part of the command used in the past)

Section 4.3: Create Your Own Command Alias

If you are tired of using long commands in bash you can create your own command alias.

The best way to do this is to modify (or create if it does not exist) a file called .bash_aliases in your home folder. The general syntax is:

```
alias command_alias='actual_command'
```

where `actual_command` is the command you are renaming and `command_alias` is the new name you have given it. For example

```
alias install='sudo apt-get -y install'
```

maps the new command alias `install` to the actual command `sudo apt-get -y install`. This means that when you use **install** in a terminal this is interpreted by bash as **sudo apt-get -y install**.

Section 4.4: Locate a file on your system

Using bash you can easily locate a file with the `locate` command. For example say you are looking for the file mykey.pem:

```
locate mykey.pem
```

Sometimes files have strange names for example you might have a file like `random7897_mykey_0fidw.pem`. Let's say you're looking for this file but you only remember the mykey and pem parts. You could combine the `locate` command with `grep` using a pipe like this:

```
locate pem | grep mykey
```

Which would bring up all results which contain both of these pieces.

Note that not all systems have the `locate` utility installed, and many that do have not enabled it. `locate` is fast and efficient because it periodically scans your system and caches the names and locations for every file on it, but if that data collection is not enabled then it cannot tell you anything. You can use `updatedb` to manually initiate the filesystem scan in order to update the cached info about files on your filesystem.

Should you not have a working `locate`, you can fall back on the `find` utility:

```
find / -name mykey.pem -print
```

is roughly equivalent to `locate mykey.pem` but has to scan your filesystem(s) each time you run it for the file in question, rather than using cached data. This is obviously slower and less efficient, but more real-time. The `find` utility can do much more than find files, but a full description of its capabilities is beyond the scope of this example.

Chapter 5: Check Disk Space

Section 5.1: Investigate Directories For Disk Usage

Sometimes it may be required to find out which directory consuming how much disk space especially when you are used `df -h` and realized your available disk space is low.

du:

`du` command summarizes disk usage of the set of FILEs, recursively for directories.

It's often uses with `-sh` option:

```
-s, --summarize
display only a total for each argument
-h, --human-readable
print sizes in human readable format (e.g., 1K 234M 2G)
```

For summarizing disk usages of the files in the current directory we use:

```
du -sh *
```

Example output:

```
572K    Documents
208M    Downloads
4,0K    Music
724K    Pictures
4,0K    Public
4,0K    Templates
4,0K    Videos
```

We can also include hidden files with using:

```
du -sh .[!.]* *
```

Example output:

```
6,3M    .atom
4,0K    .bash_history
4,0K    .bash_logout
8,0K    .bashrc
350M    .cache
195M    .config
12K    .dbus
4,0K    .dmrc
44K    .gconf
60K    .gem
520K    .gimp-2.8
28K    .gnome
4,0K    .ICEauthority
8,3M    .local
8,0K    .nano
404K    .nv
36K    .pki
```

```
4,0K    .profile
8,0K    .ssh
0    .sudo_as_admin_successful
4,0K    .Xauthority
4,0K    .xsession-errors
4,0K    .xsession-errors.old
572K    Documents
208M    Downloads
4,0K    Music
724K    Pictures
4,0K    Public
4,0K    Templates
4,0K    Videos
```

Thirdly, you can add total to the output by adding ,-c, option:

```
du -sch .[!.]* *
```

Result:

```
.
.
.
4,0K    Templates
4,0K    Videos
769M    total
```

Most importantly using `du` command properly on the root directory is a life saving action to find out what application/service or user is consuming your disk space wildly. For example, in case of a ridiculously low level of disk space availability for a web and mail server, the reason could be a spam attack to your mail service and you can diagnose it just by using `du` command.

Investigate root directory for disk usage:

```
sudo du -sch /.[!.]* /*
```

Example output:

```
16K    /.VolumeIcon.icns
24K    /.VolumeIcon.png
13M    /bin
57M    /boot
4,0K    /cdrom
620K    /dev
13M    /etc
779M    /home
0    /initrd.img
406M    /lib
3,9M    /lib32
4,0K    /lib64
16K    /lost+found
4,0K    /media
4,0K    /mnt
367M    /opt
du: cannot access '/proc/18221/task/18221/fd/4': No such file or directory
du: cannot access '/proc/18221/task/18221/fdinfo/4': No such file or directory
du: cannot access '/proc/18221/fd/4': No such file or directory
```

```
du: cannot access '/proc/18221/fdinfo/4': No such file or directory
0    /proc
20K    /root
du: cannot access '/run/user/1000/gvfs': Permission denied
9,4M    /run
13M    /sbin
4,0K    /srv
0    /sys
72K    /tmp
3,5G    /usr
639M    /var
0    /vmlinuz
5,8G    total
```

Lastly, the best method forms when you add a threshold size value for directories to ignore small ones. This command will only show folders with more than 1GB in size which located under root directory up to the farthermost branch of the whole directory tree in your file system:

```
sudo du --threshold=1G -ch /.[!.]* /*
```

Example output:

```
1,4G    /usr/lib
1,8G    /usr/share
3,5G    /usr
5,8G    total
```

Section 5.2: Checking Disk Space

It's quite common to want to check the status of the various partitions/drives on your server/computer to see how full they are. The following command is the one you'll want to run:

```
df -h
```

This will produce output similar to the following:

```
[root@mail ~]# df -h
Filesystem            Size  Used Avail Use% Mounted on
/dev/mapper/VolGroup-lv_root
                       19G  1.6G   16G   9% /
tmpfs                 245M     0  245M   0% /dev/shm
/dev/sda1             485M   47M  413M  11% /boot
```

In this basic example, we can see that the `/` partition only has `9%` used.

For a more complex example that also covers using `df` to see various mountpoints, see below:

```
[root@mail ~]# df -h
Filesystem            Size  Used Avail Use% Mounted on
/dev/mapper/VG-root   1.9T  1.7T   89G  95% /
/dev/mapper/VG-var    431G  145G  264G  36% /var
devtmpfs              7.8G  204K  7.8G   1% /dev
tmpfs                 7.8G  4.0K  7.8G   1% /dev/shm
/dev/md1              495M  126M  344M  27% /boot
ku.example.com:9421   2.5T  487G  2.0T  20% /mnt/test
tmpfs                 500M   86M  415M  18% /var/ngx_pagespeed_cache
```

In this example, we have a `/` partition that's `95%` full along with an additional `/var` partition that's only `36%` full.

It's got an external network mount of `2T` that's mounted on `/mnt/test` and a ramdisk/tmpfs mount of 500M mounted on `/var/ngx_pagespeed_cache`.

Chapter 6: Getting System Information

Collection of commands to fetch system related information.

Section 6.1: Statistics about CPU, Memory, Network and Disk (I/O operations)

To get general statistics about main components of Linux family of **stat** commands are extremely useful

CPU

To get processors related statistics you can use `mpstat` command but with some options it will provide better visibility:

```
$ mpstat 2 10
```

Memory

We all know command **free** to show amount of (remaining) RAM but to see all statistic including I/O operations:

```
$ vmstat 2 10
```

Disk

To get general information about your disk operations in real time you can utilise `iostat`.

```
$ iostat -kx 2
```

Network

To be able to see what is happening with your network services you can use **netstat**

```
$ netstat -ntlp # open TCP sockets
$ netstat -nulp # open UDP sockets
$ netstat -nxlp # open Unix sockets
```

But you can find useful monitoring to see network traffic in real time:

```
$ sudo iftop
```

Optional

To generate statistics in real time related to I/O operations across all components you can use `dstat`. That tool that is a versatile replacement for **vmstat**, `iostat` and `ifstat`

Section 6.2: Using tools like lscpu and lshw

By using tools like lscpu as lscpu is an easy way to get CPU information.

```
$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
```

```
CPU(s):                4
On-line CPU(s) list:   0-3
Thread(s) per core:    1
Core(s) per socket:    4
Socket(s):             1
NUMA node(s):          1
Vendor ID:             GenuineIntel
CPU family:            6
Model:                 23
Stepping:              10
CPU MHz:               1998.000
BogoMIPS:              5303.14
Virtualization:        VT-x
L1d cache:             32K
L1i cache:             32K
L2 cache:              2048K
NUMA node0 CPU(s):     0-3
```

By using tool lshw

```
$ lshw | grep cpu

df1-ws-5084
    description: Computer
    width: 64 bits
    capabilities: vsyscall32
  *-core
       description: Motherboard
       physical id: 0
     *-memory
          description: System memory
          physical id: 0
          size: 5881MiB
     *-cpu
          product: Intel(R) Pentium(R) CPU G3220 @ 3.00GHz
          vendor: Intel Corp.
          physical id: 1
          bus info: cpu@0
          size: 3GHz
          capacity: 3GHz
          width: 64 bits
```

Section 6.3: List Hardware

Ubuntu:

lshw is a small tool to extract detailed information on the hardware configuration of the machine. It can report exact memory configuration, firmware version, mainboard configuration, CPU version and speed, cache configuration, bus speed, etc.

```
$ sudo lshw | less (or more)
$ sudo lshw -html > myhardware.html
$ sudo lshw -xml > myhardware.xml
```

To show PCI info

```
$ lspci -tv
```

To see USB info

```
$ lsusb -tv
```

To display BIOS information

```
$ dmidecode -q | less
```

To see specific information about disk (disk sda in example) you can use:

```
$ hdparm -i /dev/sda
```

Few additional utilities/commands will help gather some extra information:

```
$ smartctl -A /dev/sda | grep Power_On_Hours # How long has this disk (system) been powered on in total
$ hdparm -tT /dev/sda # Do a read speed test on disk sda
$ badblocks -s /dev/sda # Test for unreadable blocks on disk sda
```

Section 6.4: Find CPU model/speed information

Ubuntu:

```
$ cat /proc/cpuinfo
```

Sample Output:

```
processor    : 0
vendor_id    : GenuineIntel
cpu family    : 6
model        : 15
model name    : Intel(R) Core(TM)2 Quad CPU    Q6600  @ 2.40GHz
stepping    : 11
cpu MHz        : 1596.000
cache size    : 4096 KB
physical id    : 0
siblings    : 4
core id        : 0
cpu cores    : 4
apicid        : 0
initial apicid    : 0
fpu        : yes
fpu_exception    : yes
cpuid level    : 10
wp        : yes
flags        : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts
acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx lm constant_tsc arch_perfmon pebs bts rep_good pni
dtes64 monitor ds_cpl vmx est tm2 ssse3 cx16 xtpr pdcm lahf_lm tpr_shadow vnmi flexpriority
bogomips    : 4800.18
clflush size    : 64
cache_alignment    : 64
address sizes    : 36 bits physical, 48 bits virtual
power management:
....
..
processor    : 3
vendor_id    : GenuineIntel
cpu family    : 6
```

```
model        : 15
model name    : Intel(R) Core(TM)2 Quad CPU    Q6600  @ 2.40GHz
stepping    : 11
cpu MHz        : 1596.000
cache size    : 4096 KB
physical id    : 0
siblings    : 4
core id        : 3
cpu cores    : 4
apicid        : 3
initial apicid    : 3
fpu        : yes
fpu_exception    : yes
cpuid level    : 10
wp        : yes
flags        : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts
acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx lm constant_tsc arch_perfmon pebs bts rep_good pni
dtes64 monitor ds_cpl vmx est tm2 ssse3 cx16 xtpr pdcm lahf_lm tpr_shadow vnmi flexpriority
bogomips    : 4800.30
clflush size    : 64
cache_alignment    : 64
address sizes    : 36 bits physical, 48 bits virtual
power management:
```

count processor (including cores):

```
$ grep -c processor /proc/cpuinfo
```

Section 6.5: Process monitoring and information gathering

Overall you have two ways to monitor processes at linux host

Static monitoring

Most widely used command is `ps` (i.e., process status) command is used to provide information about the currently running processes, including their process identification numbers (PIDs).

Here few useful options to gather specific information.

List processes in a hierarchy

```
$ ps -e -o pid,args --forest
```

List processes sorted by % cpu usage

```
$ ps -e -o pcpu,cpu,nice,state,cputime,args --sort pcpu | sed '/^ 0.0 /d'
```

List processes sorted by mem (KB) usage.

```
$ ps -e -orss=,args= | sort -b -k1,1n | pr -TW$COLUMNS
```

List all threads for a particular process ("firefox-bin" process in example)

```
$ ps -C firefox-bin -L -o pid,tid,pcpu,state
```

After finding specific process you can gather information related to it using **`lsof`** to list paths that process id has open

```
$ lsof -p $$
```

Or based on path find out list processes that have specified path open

```
$ lsof ~
```

Interactive monitoring

Most commonly known tool for dynamic monitoring is:

```
$ top
```

That mostly default command that have huge amount options to filter and represent information in real time (in comparison to `ps` command.

Still there are more advance options that can be considered and installed as `top` replacement

```
$ htop -d 5
```

or

```
$ atop
```

Which has ability to log all the activities into log file (default atop will log all the activity on every 600 seconds) To this list there are few specialised commands as `iotop` or `iftop`

```
$ sudo iotop
```

Chapter 7: ls command

Section 7.1: Options for ls command

Full list of options:

`ls -a` list all files including hidden file starting with '.'

`ls --color` colored list [=always/never/auto]

`ls -d` list directories - with ' */'

`ls -F` add one char of */=>@| to enteries

`ls -i` list file's inode index number

`ls -l` list with long format - show permissions

`ls -la` list long format including hidden files

`ls -lh` list long format with readable file size

`ls -ls` list with long format with file size

`ls -r` list in reverse order

`ls -R` list recursively directory tree

`ls -s` list file size

`ls -S` sort by file size

`ls -t` sort by time & date

`ls -X` sort by extension name

Section 7.2: ls command with most used options

ls shows files and directories in present working directory. (if no arguments are passed.) (It doesn't show hidden files which starts with . by default.)

```
user@ubuntu14:/usr$ ls
bin  games  include  lib  lib32  local  sbin  share  src
```

To see all files (hidden files/folders also). Use `ls -a` OR `ls -all`

```
user@ubuntu14:/usr$ ls -a
.  ..  bin  games  include  lib  lib32  local  sbin  share  src
```

To differentiate between files and folders and symbolic links and other, use `ls -F` OR `ls --classify`

```
user@ubuntu14:~$ ls -F
bash_profile_course  chat_apps/      Desktop/    Downloads/    foxitsoftware/
Public/     test/    bin/    ClionProjects/  Documents/    IDE/    Music/
Pictures/  Templates/  Videos/
```

Here, ending characters are used to distinguish files and folders.

“/” suggest directory.

“*”suggest executables.

“@” suggest symbolic links.

To get more details about the files and directories, use `ls -l`

```
user@ubuntu14:~/example$ ls -l
total 6464

-rw-r--r-- 1 dave dave      41 Dec 24 12:19 Z.txt
drwxr-xr-x 2 user group    4096 Dec 24 12:00 a_directory
-rw-r--r-- 1 user group       6 Dec 24 12:01 a_file
lrwxrwxrwx 1 user group       6 Dec 24 12:04 a_link -> a_file
-rw-r--r-- 1 user group       6 Dec 24 12:03 a_newer_file
-rw-r----- 1 user group 6586816 Dec 24 12:07 big.zip
```

In this example, the total size of the contents is 6460KB.

Then there is an entry for each file/directory in alphabetical order with upper case before lower case.

The first character is the type (e.g. d - directory, l - link).

The next 9 characters show the permissions for the user, group and other.

This is followed by the number of hard links, then the owner's name and group.

The next field is the size in bytes. This can be displayed in a human friendly form by adding the `-h` option e.g. 6586816 is displayed as 6.3M

There then follows a timestamp (usually the modification time).

The final field is the name. Note: links also show the target of the link.

Chapter 8: File Compression with 'tar' command

Common Options	-
-c --create	Create a new archive.
-x --extract	Extract files from an archive.
-t --list	List the contents of an archive.
-f --file=*ARCHIVE*	Use archive file or dir *ARCHIVE*.
-v --verbose	Verbosely list files processed.
Compression Options	-
-a --auto-compress	Use archive suffix to determine the compression program.
-j --bzip2	Filter the archive through bzip2.
-J --xz --lzma	Filter the archive through xz.
-z --gzip	Filter the archive through gzip.

Section 8.1: Compress a folder

This creates a simple archive of a folder :

```
tar -cf ./my-archive.tar ./my-folder/
```

Verbose output shows which files and directories are added to the archive, use the -v option:

```
tar -cvf ./my-archive.tar ./my-folder/
```

For archiving a folder compressed 'gzip', you have to use the -z option :

```
tar -czf ./my-archive.tar.gz ./my-folder/
```

You can instead compress the archive with 'bzip2', by using the -j option:

```
tar -cjf  ./my-archive.tar.bz2 ./my-folder/
```

Or compress with 'xz', by using the -J option:

```
tar -cJf  ./my-archive.tar.xz ./my-folder/
```

Section 8.2: Extract a folder from an archive

There is an example for extract a folder from an archive in the current location :

```
tar -xf archive-name.tar
```

If you want to extract a folder from an archive to a specfic destination :

```
tar -xf archive-name.tar -C ./directory/destination
```

Section 8.3: List contents of an archive

List the contents of an archive file without extracting it:

```
tar -tf archive.tar.gz
Folder-In-Archive/
Folder-In-Archive/file1
Folder-In-Archive/Another-Folder/
Folder-In-Archive/Another-Folder/file2
```

Section 8.4: List archive content

There is an example of listing content :

```
tar -tvf archive.tar
```

The option `-t` is used for the listing. For listing the content of a tar.gz archive, you have to use the `-z` option anymore :

```
tar -tzvf archive.tar.gz
```

Section 8.5: Compress and exclude one or multiple folder

If you want to extract a folder, but you want to exclude one or several folders during the extraction, you can use the `--exclude` option.

```
tar -cf archive.tar ./my-folder/ --exclude="my-folder/sub1" --exclude="my-folder/sub3"
```

With this folder tree :

```
my-folder/
   sub1/
   sub2/
   sub3/
```

The result will be :

```
./archive.tar
   my-folder/
      sub2/
```

Section 8.6: Strip leading components

To strip any number of leading components, use the --strip-components option:

```
 --strip-components=NUMBER
   strip NUMBER leading components from file names on extraction
```

For example to strip the leading folder, use:

```
tar -xf --strip-components=1 archive-name.tar
```

Chapter 9: Services

Section 9.1: List running service on Ubuntu

To get a list of the service on your system, you may run:

```
service --status-all
```

The output of `service --status-all` lists the state of services controlled by `System V`.

The `+` indicates the service is running, `-` indicates a stopped service. You can see this by running service SERVICENAME status for a + and - service.

Some services are managed by **Upstart**. You can check the status of all Upstart services with sudo initctl list. Any service managed by Upstart will also show in the list provided by service --status-all but will be marked with a ?.

ref: https://askubuntu.com/questions/407075/how-to-read-service-status-all-results

Section 9.2: Systemd service management

Listing services

- `systemctl` To list running services
- `systemctl --failed` To list failed services

Managing Targets (Similar to Runlevels in SysV)

- `systemctl get-default` To find the default target for your system
- `systemctl set-default <target-name>` To set the default target for your system

Managing services at runtime

- `systemctl start [service-name]` To start a service
- `systemctl stop [service-name]` To stop a service
- `systemctl restart [service-name]` To restart a service
- `systemctl reload [service-name]` To request service to reload its configuration
- `systemctl status [service-name]` To show current status of a service

Managing autostart of services

- `systemctl is-enabled [service-name]` To show whether a service is enabled on system boot
- `systemctl is-active [service-name]` To show whether a service is currently active(running)
- `systemctl enable [service-name]` To enable a service on system boot
- `systemctl disable [service-name]` To disable a service on system boot

Masking services

- `systemctl mask [service-name]` To mask a service (Makes it hard to start a service by mistake)
- `systemctl unmask [service-name]` To unmask a service

Restarting systemd

```
systemctl daemon-reload
```

Chapter 10: Managing Services

Section 10.1: Diagnosing a problem with a service

On systems using systemd, such as Fedora => 15, Ubuntu (Server and Desktop) >= 15.04, and RHEL/CentOS >= 7:

```
systemctl status [servicename]
```

...where `[servicename]` is the service in question; for example, `systemctl status sshd`.

This will show basic status information and any recent errors logged.

You can see further errors with `journalctl`. For example,`journalctl -xe` will load the last 1000 logged into a pager (like `less`), jumping to the end. You can also use `journalctl -f`, which will follow log messages as they come in.

To see logs for a particular service, use the `-t` flag, like this:

```
journalctl -f -t sshd
```

Other handy options include `-p` for priority (`-p warnings` to see only warnings and above), `-b` for "since last boot", and `-S` for "since" — putting that together, we might do

```
journalctl -p err -S yesterday
```

to see all items logged as errors since yesterday.

If journalctl is not available, or if you are following application error logs which do not use the system journal, the `tail` command can be used to show the last few lines of a file. A useful flag for tail is `-f` (for "follow"), which causes tail continue showing data as it gets appended to the file. To see messages from most services on the system:

```
tail -f /var/log/messages
```

Or, if the service is privileged, and may log sensitive data:

```
tail -f /var/log/secure
```

Some services have their own log files, a good example is `auditd`, the linux auditing daemon, which has its logs stored in `/var/log/audit/`. If you do not see output from your service in `/var/log/messages` try looking for service specific logs in `/var/log/`

Section 10.2: Starting and Stopping Services

On systems that use the System-V style init scripts, such as RHEL/CentOS 6:

```
service <service> start
```

```
service <service> stop
```

On systems using systemd, such as Ubuntu (Server and Desktop) >= 15.04, and RHEL/CentOS >= 7:

```
systemctl <service> dnsmasq
```

```
systemctl <service> dnsmasq
```

Section 10.3: Getting the status of a service

On systems that use the System-V style init scripts, such as RHEL/CentOS 6:

```
service <service> status
```

On systems using systemd, such as Ubuntu (Server and Desktop) >= 15.04, and RHEL/CentOS >= 7.0:

```
systemctl status <service>
```

Chapter 11: Modifying Users

Parameter	Details
username	The name of the user. Do not use capital letters, do not use dots, do not end it in dash, it must not include colons, no special characters. Cannot start with a number.

Section 11.1: Setting your own password

```
passwd
```

Section 11.2: Setting another user's password

Run the following as root:

```
passwd username
```

Section 11.3: Adding a user

Run the following as root:

```
useradd username
```

Section 11.4: Removing a user

Run the following as root:

```
userdel username
```

Section 11.5: Removing a user and its home folder

Run the following as root:

```
userdel -r username
```

Section 11.6: Listing groups the current user is in

```
groups
```

More detailed information about user and group numerical IDs can be found with the `id` command.

Section 11.7: Listing groups a user is in

```
groups username
```

More detailed information about user and group numerical IDs can be found with `id username`.

Chapter 12: LAMP Stack

LAMP (**L**inux **A**pache **M**ySQL **P**HP) consists of the Linux operating system as development environment, the Apache HTTP Server as web server, the MySQL relational database management system (RDBMS) as DB (**D**ata **B**ase) system, and the PHP programming language as Server side (Back End) programming language.

LAMP is used as a Open Source stack of technologies solution to web development area. Windows version of this stack is called WAMP (**W**indows **A**pache **M**ySQL **P**HP)

Section 12.1: Installing LAMP on Arch Linux

With this line we will install all the necessary packages in one step, and the last update:

```
pacman -Syu apache php php-apache mariadb
```

HTTP

Edit

`/etc/httpd/conf/httpd.conf`

Change `ServerAdmin you@example.com` as you need.

The folder of the WEB Pages by default is `ServerRoot "/etc/httpd"`. Directory must be set to the same folder, so change the line

```
<Directory "/etc/httpd">
```

This folder must have read and execution access, so

```
chmod o+x /etc/httpd
```

Change `AllowOverride from none (default) to All` so .htaccess will works.

Now you need the `~/public_html` folder for each user. (to get the root page of each user as http://localhost/~yourusername/. Unremark this line:

```
Include conf/extra/httpd-userdir.conf
```

Now as root you need to create the `~/public_html` for each user and change the access to (755) of each one.

```
chmod 755 /home
chmod 755 /home/username
chmod 755 /home/username/public_html
```

You can comment out this line if you want to use SSL:

```
LoadModule ssl_module modules/mod_ssl.so
```

If you need to use virtual domains, uncomment the line:

```
Include conf/extra/httpd-vhosts.conf
```

and in `/etc/httpd/conf/extra/httpd-vhosts.conf` you must to add all the virtual domains. (plus into `/etc/hosts` if you want to test those virtuals domains)

Edit `/etc/httpd/conf/extra/httpd-default.conf` and change **ServerSignature** to Off and **ServerToken** to Prod for hiding critical data

PHP

Edit: `/etc/httpd/conf/httpd.conf`

Comment out: `LoadModule mpm_event_module modules/mod_mpm_event.so`

Uncomment: `LoadModule mpm_prefork_module modules/mod_mpm_prefork.so`

As last item in the LoadModule list, add `LoadModule php7_module modules/libphp7.so`

As last item in the include list, add `Include conf/extra/php7_module.conf`

Edit `/etc/php/php.ini`

Uncomment `extension=mysqli.so` and `extension=pdo_mysql.so`

Change the timezone as you need, for example:

```
date.timezone = America/Argentina/Buenos_Aires, date.default_latitude = 0.0, date.default_longitude
= 0.0
```

MySQL

Run as root:

```
mysql_install_db --user=mysql --basedir=/usr --datadir=/var/lib/mysql
```

Now you have the root of the MySQL Server.

Start MySQL daemon:

```
systemctl enable mysqld
systemctl start mysqld
```

At last, run:

```
sh /usr/bin/mysql_secure_installation
```

That all to get a web server ready to be customized as you need.

Section 12.2: Installing LAMP on Ubuntu

Install apache:

```
sudo apt-get install apache2
```

Install MySql:

```
sudo apt-get install mysql-server
```

Install PHP:

```
sudo apt-get install php5 libapache2-mod-php5
```

Restart system:

```
sudo systemctl restart apache2
```

Check PHP installation:

```
php -r 'echo "\n\nYour PHP installation is working fine.\n\n\n";'
```

Section 12.3: Installing LAMP stack on CentoOS

Install Apache Web Server

First step is to install web server Apache.

```
sudo yum -y install httpd
```

Once it is installed, enable (to run on startup) and start Apache web server service.

```
sudo systemctl enable --now httpd
```

Point your browser to:

http://localhost

You will see the default Apache web server page.

Install MariaDB Server

Second step is to install MariaDB:

```
sudo yum -y install mariadb-server
```

Then start and enable (on startup) the MariaDB server:

```
sudo systemctl enable --now mariadb
```

As needed, use **mysql_secure_installation** to secure your database.

This script will allow you to do the following:

- Change the root user's password
- Remove test databases
- Disable remote access

Install PHP

```
sudo yum -y install php php-common
```

Then restart Apache's httpd service.

```
sudo systemctl restart httpd
```

To test PHP, create a file called **index.php** in **/var/www/html**.

Then add the following line to the file:

Then point your browser to:

http://localhost/index.php

You should see information related to your server. If you do not, ensure that php is for sure installed correctly by running the following command:

```
php --version
```

If you receive something like:

```
PHP 5.4.16 (cli) (built: Nov 6 2016 00:29:02) Copyright (c) 1997-2013 The PHP Group
```

Then PHP is installed correctly. If this is the case, please ensure that you've restarted your web server.

Chapter 13: tee command

Options	Description
-a, --append	Append to the given FILEs. Do not overwrite.
-i, --ignore-interrupts	Ignore interrupt signals.
--help	Display a help message, and exit.
--version	Display version information, and exit.

tee - read from standard input and write to standard output and files.

The tee command is named after the T-splitter in plumbing, which splits water into two directions and is shaped like an uppercase T.

tee copies data from standard input to each FILE, and also to standard output. In effect, tee duplicates its input, routing it to multiple outputs at once.

Section 13.1: Write output to stdout, and also to a file

The following command displays output only on the screen (stdout).

```
$ ls
```

The following command writes the output only to the file and not to the screen.

```
$ ls > file
```

The following command (with the help of `tee` command) writes the output both to the screen (stdout) and to the file.

```
$ ls | tee file
```

Section 13.2: Write output from the middle of a pipe chain to a file and pass it back to the pipe

You can also use `tee` command to store the output of a command in a file and redirect the same output to another command.

The following command will write current crontab entries to a file `crontab-backup.txt` and pass the crontab entries to `sed` command, which will do the substituion. After the substitution, it will be added as a new cron job.

```
$ crontab -l | tee crontab-backup.txt | sed 's/old/new/' | crontab –
```

Section 13.3: write the output to multiple files

You can pipe your output to multiple files (including your terminal) by using `tee` like this:

```
$ ls | tee file1 file2 file3
```

Section 13.4: Instruct tee command to append to the file

By default `tee` command overwrites the file. You can instruct `tee` to append to the file using the –a option as shown

below.

```
$ ls | tee –a file
```

Chapter 14: Secure Shell (SSH)

A secure shell is used to remotely access a server from a client over an encrypted connection. OpenSSH is used as an alternative to Telnet connections that achieve remote shell access but are unencrypted. The OpenSSH Client is installed on most GNU/Linux distributions by default and is used to connect to a server. These examples show use how to use the SSH suite to for accept SSH connections and connecting to another host.

Section 14.1: Connecting to a remote server

To connect to a server we must use SSH on the client as follows,

```
# ssh -p port user@server-address
```

- **port** - The listening ssh port of the server (default port 22).
- **user** - Must be an existing user on the server with SSH privileges.
- **server address** - The IP/Domain of the server.

For a real world example lets pretend that you're making a website. The company you chose to host your site tells you that the server is located at **web-servers.com** on a custom port of **2020** and your account name **usr1** has been chosen to create a user on the server with SSH privileges. In this case the SSH command used would be as such

```
# ssh -p 2020 usr1@web-servers.com
```

If account name on the remote system is the same as the one one the local client you may leave the user name off. So if you are `usr1` on both systems then you my simply use `web-servers.com` instead of `usr1@web-servers.com`.

When a server you want to connect to is not directly accessible to you, you can try using ProxyJump switch to connect to it through another server which is accessible to you and can connect to the desired server.

```
# ssh -J usr1@10.0.0.1:2020 usr2@10.0.0.2 -p 2222
```

This will let you connect to the server 10.0.0.2 (running ssh on port 2222) through server at 10.0.0.1 (running ssh on port 2020). You will need to have accounts on both servers of course. Also note that the -J switch is introduced in OpenSSH version 7.3.

Section 14.2: Installing OpenSSH suite

Both connecting to a remove SSH server and accepting SSH connections require installation of `openssh`

Debian:

```
# apt-get install openssh
```

Arch Linux:

```
# pacman -S openssh
```

Yum:

```
# yum install openssh
```

Section 14.3: Configuring an SSH server to accept connections

First we must edit the SSH daemon config file. Though under different Linux distributions this may be located in different directories, usually it is stored under `/etc/ssh/sshd_config`

Use your text editor to change the values set in this file, all lines starting with # are commented out and must have this character removed to take any effect. A list of recommendations follow as such.

```
Port (chose a number between 0 - 65535, normaly greater than four digits)
PasswordAuthentication yes
AllowUsers    user1 user2 ...etc
```

Note that it is preferable to disable password logins all together and use SSH Keys for improved security as explained in this document.

Section 14.4: Passwordless connection (using a key pair)

First of all you'll need to have a key pair. If you don't have one yet, take a look at the 'Generate public and private key topic'.

Your key pair is composed by a private key (id_rsa) and a public key (id_rsa.pub). All you need to do is to copy the public key to the remote host and add its contents to the `~/.ssh/authorized_keys` file.

One simple way to do that is:

```
ssh <user>@<ssh-server> 'cat >> ~/.ssh/authorized_keys' < id_rsa.pub
```

Once the public key is properly placed in your user's home directory, you just need to login using the respective private key:

```
ssh <user>@<ssh-server> -i id_rsa
```

Section 14.5: Generate public and private key

To generate keys for SSH client:

```
ssh-keygen [-t rsa | rsa1 | dsa ] [-C <comment>] [-b bits]
```

For example:

```
ssh-keygen -t rsa -b 4096 - C myemail@email.com
```

Default location is `~/.ssh/id_rsa` for private and `~/.ssh/id_rsa.pub` for public key.

For more info, please visit man.openbsd.org

Section 14.6: Disable ssh service

This will disable the SSH server side service, as if needed this will insure that clients cannot connect via ssh

Ubuntu

```
sudo service ssh stop
```

```
sudo systemctl disable sshd.service
```

Debian

```
sudo /etc/init.d/ssh stop
sudo systemctl disable sshd.service
```

Arch Linux

```
sudo killall sshd
sudo systemctl disable sshd.service
```

Chapter 15: SCP

Section 15.1: Secure Copy

scp command is used to securely copy a file to or from a remote destination. If the file is in current working directly only filename is sufficient else full path is required which included the remote hostname e.g. remote_user@some_server.org:/path/to/file

Copy local file in your CWD to new directory

```
scp localfile.txt /home/friend/share/
```

Copy remote file to you current working directory

```
scp rocky@arena51.net:/home/rocky/game/data.txt ./
```

Copy file from one remote location to another remote location

```
scp mars@universe.org:/beacon/light/bitmap.conf jupiter@universe.org:/beacon/night/
```

To copy directory and sub-directories use '-r' recursive option to scp

```
scp -r user@192.168.0.4:~/project/* ./workspace/
```

Section 15.2: Basic Usage

```
# Copy remote file to local dir
scp user@remotehost.com:/remote/path/to/foobar.md /local/dest

# Copy local file to remote dir
scp foobar.md user@remotehost.com:/remote/dest

# Key files can be used (just like ssh)
scp -i my_key.pem foobar.md user@remotehost.com:/remote/dest
```

Chapter 16: GnuPG (GPG)

GnuPG is a sophisticated key management system which allows for secure signing or encrypting data. GPG is a command-line tool used to create and manipulate GnuPG keys.

GnuPG is most widely used for having SSH (Secure Shell) connections without password or any means of interactive authentication, which improves security level significantly.

Following sections describe ways to create, use, and maintain security of GnuPG keys.

Section 16.1: Exporting your public key

In order for your public-private keypair to be of use, you must make your public key freely available to others. Be sure that you are working with your public key here since you should *never* share your private key. You can export your public key with the following command:

```
gpg —armor —export EMAIL_ADDRESS > public_key.asc
```

where EMAIL_ADDRESS is the email address associated with the key

Alternately, you can upload your public key to a public key server such as keys.gnupg.net so that others can use it. To do so, enter the following in a terminal:

```
gpg —list-keys
```

Then, search for the 8-digit string (the primary ID) associated with the key you want to export. Then, issue the command:

```
gpg —send-keys PRIMARY_ID
```

where PRIMARY_ID is the actual ID of that key.

Now, the public key has been uploaded to the key server and is publicly available.

Section 16.2: Create and use a GnuPG key quickly

Install haveged (example `sudo apt-get install haveged`) to speed up the random byte process. Then:

```
gpg --gen-key
gpg --list-keys
```

outputs:

```
pub   2048R/NNNNNNNN 2016-01-01
uid                  Name <name@example.com>
sub   2048R/xxxxxxxx 2016-01-01
```

Then publish:

```
gpg --keyserver pgp.mit.edu --send-keys NNNNNNNN
```

Then plan to revoke: https://www.hackdiary.com/2004/01/18/revoking-a-gpg-key/

Chapter 17: Network Configuration

This document covers TCP/IP networking, network administration and system configuration basics. Linux can support multiple network devices. The device names are numbered and begin at zero and count upwards. For example, a computer with two NICs will have two devices labeled eth0 and eth1.

Section 17.1: Local DNS resolution

File: `/etc/hosts` contains a list of hosts that are to be resolved locally(not by DNS)

Sample contents of the file:

```
127.0.0.1         your-node-name.your-domain.com  localhost.localdomain  localhost
XXX.XXX.XXX.XXX   node-name
```

The file format for the hosts file is specified by RFC 952

Section 17.2: Configure DNS servers for domain name resolution

File: `/etc/resolv.conf` contains a list of DNS servers for domain name resolution

Sample contents of the file:

```
nameserver 8.8.8.8 # IP address of the primary name server
nameserver 8.8.4.4 # IP address of the secondary name server
```

In case internal DNS server you can validate if this server resolve DNS names properly using `dig` command:

```
$ dig google.com @your.dns.server.com +short
```

Section 17.3: See and manipulate routes

Manipulate the IP routing table using `route`

Display routing table

```
$ route # Displays list or routes and also resolves host names
$ route -n # Displays list of routes without resolving host names for faster results
```

Add/Delete route

Option	Description
`add` or `del`	Add or delete a route
`-host x.x.x.x`	Add route to a single host identified by the IP address
`-net x.x.x.x`	Add route to a network identified by the network address
`gw x.x.x.x`	Specify the network gateway
`netmask x.x.x.x`	Specify the network netmask
`default`	Add a default route

Examples:

- add route to a host `$ route add -host x.x.x.x eth1`
- add route to a network `$ route add -net 2.2.2.0 netmask 255.255.255.0 eth0`
- Alternatively, you could also use cidr format to add a route to network `route add -net 2.2.2.0/24 eth0`
- add default gateway `$ route add default gw 2.2.2.1 eth0`
- delete a route `$ route del -net 2.2.2.0/24`

Manipulate the IP routing table using `ip`

Display routing table

```
$ ip route show # List routing table
```

Add/Delete route

Option	Description
`add` or `del` or `change` or `append` or `replace`	Change a route
`show` or `flush`	the command displays the contents of the routing tables or remove it
`restore`	restore routing table information from stdin
`get`	this command gets a single route to a destination and prints its contents exactly as the kernel sees it

Examples:

- Set default gateway to 1.2.3.254 `$ ip route add default via 1.2.3.254`
- Adds a default route (for all addresses) via the local gateway 192.168.1.1 that can be reached on device eth0 `$ ip route add default via 192.168.1.1 dev eth0`

Section 17.4: Configure a hostname for some other system on your network

You can configure your Linux (or macOS) system in order to tie in an identifier **`<hostname>`** to some other system's IP address in your network. You can configure it:

- Systemwide. You should modify the */etc/hosts* file. You just have to add to that file a new line containing:

    1. the remote system's IP address **`<ip_rem>`**,
    2. one or more blank spaces, and
    3. the identifier **`<hostname>`**.

- For a single user. You should modify the *~/.hosts* file --- you-d have to create it. It is not as simple as for systemwide. Here you can see an explanation.

For instance, you could add this line using the **`cat`** Unix tool. Suppose that you want to make a **`ping`** to a PC in yout local network whose IP address is 192.168.1.44 and you want to refer to that IP address just by `remote_pc`. Then you must write on your shell:

```
$ sudo cat 192.168.1.44 remote_pc
```

Then you can make that ping just by:

```
$ ping remote_pc
```

Section 17.5: Interface details

Ifconfig

List all the interfaces available on the machine

```
$ ifconfig -a
```

List the details of a specific interface

Syntax: `$ ifconfig <interface>`

Example:

```
$ ifconfig eth0
eth0      Link encap:Ethernet  HWaddr xx:xx:xx:xx:xx:xx
          inet addr:x.x.x.x  Bcast:x.x.x.x  Mask:x.x.x.x
          inet6 addr: xxxx::xxx:xxxx:xxxx:xxxx/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:4426618 errors:0 dropped:1124 overruns:0 frame:0
          TX packets:189171 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:382611580 (382.6 MB)  TX bytes:36923665 (36.9 MB)
          Interrupt:16 Memory:fb5e0000-fb600000
```

Ethtool - query the network driver and hardware settings

Syntax: `$ ethtool <interface>`

Example:

```
$ ethtool eth0
```

```
Settings for eth0:
Supported ports: [ TP ]
Supported link modes:   10baseT/Half 10baseT/Full
100baseT/Half 100baseT/Full
1000baseT/Full
Supported pause frame use: No
Supports auto-negotiation: Yes
Advertised link modes:  10baseT/Half 10baseT/Full
100baseT/Half 100baseT/Full
1000baseT/Full
Advertised pause frame use: No
Advertised auto-negotiation: Yes
Speed: 1000Mb/s
Duplex: Full
Port: Twisted Pair
PHYAD: 1
Transceiver: internal
Auto-negotiation: on
MDI-X: on (auto)
Supports Wake-on: pumbg
Wake-on: g
Current message level: 0x00000007 (7)
drv probe link
Link detected: yes
```

ip - show / manipulate routing, devices, policy routing and tunnels

Syntax: `$ ip { link | ... | route | macsec }` (please see `man ip` for full list of objects)

Examples

List network interfaces

```
$ ip link show
```

Rename interface eth0 to wan

```
$ ip link set dev eth0 name wan
```

Bring interface eth0 up (or down)

```
$ ip link set dev eth0 up
```

List addresses for interfaces

```
$ ip addr show
```

Add (or del) ip and mask (255.255.255.0)

```
$ ip addr add 1.2.3.4/24 brd + dev eth0
```

Section 17.6: Adding IP to an interface

An IP address to an interface could be obtained via DHCP or Static assignment

DHCP If you are connected to a network with a DHCP server running, `dhclient` command can get an IP address for your interface

```
$ dhclient <interface>
```

or alternatively, you could make a change to the `/etc/network/interfaces` file for the interface to be brought up on boot and obtain DHCP IP

```
auto eth0
iface eth0 inet dhcp
```

Static configuration(Permanent Change) using `/etc/network/interfaces` file

If you want to statically configure the interface settings(permanent change), you could do so in the `/etc/network/interfaces` file.

Example:

```
auto eth0 # Bring up the interface on boot
iface eth0 inet static
    address 10.10.70.10
    netmask 255.255.0.0
    gateway 10.10.1.1
    dns-nameservers 10.10.1.20
    dns-nameservers 10.10.1.30
```

These changes persist even after system reboot.

Static configuration(Temporary change) using `ifconfig` utility

A static IP address could be added to an interface using the **`ifconfig`** utility as follows

```
$ ifconfig <interface> <ip-address>/<mask> up
```

Example:

```
$ ifconfig eth0 10.10.50.100/16 up
```

Chapter 18: Midnight Commander

Midnight Commander or mc is a console file manager. This topic includes the descripton of it's functionalities and examples and tips of how to use it to it's full potential.

Section 18.1: Midnight Commander function keys in browsing mode

Here is a list of actions which can be triggered in the Midnight Commander filesystem browsing mode by using function keys on your keyboard.

`F1` Displays help

`F2` Opens user menu

`F3` Displays the contents of the selected file

`F4` Opens the selected file in the internal file editor

`F5` Copies the selected file to the directory open in the second panel

`F6` Moves the selected file to the directory open in the second panel

`F7` Makes a new directory in the directory open in the current panel

`F8` Deletes the selected file or directory

`F9` Focuses to the main menu on the top of the screen

`F10` Exits mc

Section 18.2: Midnight Commander function keys in file editing mode

Midnight Commander has a built in editor which is started by `F4` function key when over the desired file in the browse mode. It can also be invoked in standalone mode by executing

```
mcedit <filename>
```

Here is a list of actions which can be triggered in the edit mode.

`F1` Displays help

`F2` Saves current file

`F3` Marks the start of the text selection. Move cursor any direction to select. Second hit marks the end of the selection.

`F4` Brings up the text search/replace dialog

`F5` Copies selected text to the cursor location (copy/paste)

`F6` Moves selected text to the cursor location (cut/paste)

`F7` Brings up the text search dialog

F8 Deletes selected text

F9 Focuses to the main menu on the top of the screen

F10 Exits the editor

Chapter 19: Change root (chroot)

Change root (chroot) is an operation that changes the apparent root directory for the current running process and their children. A program that is run in such a modified environment cannot access files and commands outside that environmental directory tree.

Section 19.1: Requirements

- root privileges
- another working Linux environment,such as Live CD boot or an existing distribution
- matching environment architectures of `chroot` source and destination (check current environment architecture with `uname -m`)
- kernel modules which you may need in `chroot` environment must be loaded (for example, with `modprobe`)

Section 19.2: Manually changing root in a directory

1. Ensure you met all requirements, as per Requirements
2. Mount the temporary API filesystems:

```
cd /location/of/new/root
mount -t proc proc proc/
mount --rbind /sys sys/
mount --rbind /dev dev/
mount --rbind /run run/ (optionally)
```

3. If you need to use an internet connection in the chroot environment, copy over the DNS details:

```
cp /etc/resolv.conf etc/resolv.conf
```

4. Change root into /location/of/new/root, specifying the shell (`/bin/bash` in this example):

```
chroot /location/of/new/root /bin/bash
```

5. After chrooting it may be necessary to load the local bash configuration:

```
source /etc/profile
source ~/.bashrc
```

6. Optionally, create a unique prompt to be able to differentiate your chroot environment:

```
export PS1="(chroot) $PS1"
```

7. When finished with the chroot, you can exit it via:

```
exit
```

8. Unmount the temporary file systems:

```
cd /
umount --recursive /location/of/new/root
```

Section 19.3: Reasons to use chroot

Changing root is commonly done for performing system maintenance on systems where booting and/or logging in is no longer possible.

Common examples are:

- reinstalling the bootloader
- rebuilding the initramfs image
- upgrading or downgrading packages
- resetting a forgotten password
- building software in a clean root environment

Chapter 20: Package Managers

Section 20.1: How to update packages with the apt package manager

The **A**dvanced **P**ackage **T**ool, aptly named the 'apt' package manager can handle the installation and removal of software on the Debian, Slackware, and other Linux Distributions. Below are some simple examples of use:

update
This option retrieves and scans the Packages.gz files, so that information about new and updated packages is available. To do so, enter the following command:

```
sudo apt-get update
```

upgrade
This option is used to install the newest versions of all packages currently installed on the system. Packages currently installed with new versions available are retrieved and upgraded; under no circumstances are currently installed packages removed, or packages not already installed retrieved and installed. To upgrade, enter the following command:

```
sudo apt-get upgrade
```

dist-upgrade
In addition to performing the function of upgrade, dist-upgrade also intelligently handles changing dependencies with new versions of packages. It will attempt to upgrade the most important packages at the expense of less important ones if necessary. To do so, enter the following command:

```
sudo apt-get dist-upgrade
```

Section 20.2: How to install a package with the pacman package manager

In order to search for packages in the databse, searching both in packages' names and descriptions:

```
pacman -Ss string1 string2 ...
```

To install a single package or list of packages (including dependencies), issue the following command:

```
sudo pacman -S package_name1 package_name2 ...
```

source

Section 20.3: How to update packages with the pacman package manager

To update a specific program:

```
sudo pacman -S <programName>
```

To update entire the system:

```
sudo pacman -Syu
```

Section 20.4: How to update packages with yum

Yellowdog **U**pdater, **M**odified, one of the last remaining vestiges of Yellow Dog Linux, is the package manager used by Red Hat, Fedora, and CentOS systems and their derivatives. It can handle the installation and removal of software packaged as **rpms** for these Linux distributions. Below are some simple examples of use:

search
This command will attempt to locate software packages in the configured software repositories that match the given search criteria, and display the name / version / repository location of the matches it finds. To use it, enter the following command:

```
yum search <queryString>
```

install
This command will attempt to locate and install the named software from the configured software repositories, recursively locating and installing any needed prerequisite software as well. To use it, enter the following command:

```
sudo yum install <packageName>
```

update
This option is used to install the newest versions of all packages currently installed on the system. Packages currently installed with new versions available are retrieved and upgraded; new prerequisites are also retrieved and installed as necessary, and replaced or obsoleted packages are removed. To upgrade, enter the following command:

```
sudo yum update
```

Unlike **apt**, most **yum** commands will also automatically check for updates to repository metadata if a check has not been done recently (or if forced to do so) and will retrieve and scan updated metadata so that information about new and updated packages is available before the requested operation is performed.

Chapter 21: Compiling the Linux kernel

Section 21.1: Compilation of Linux Kernel on Ubuntu

> **Warning:** be sure you have at least 15 GB of free disk space.

Compilation in Ubuntu >=13.04

Option A) Use Git

Use git if you want to stay in sync with the latest Ubuntu kernel source. Detailed instructions can be found in the Kernel Git Guide. The git repository does not include necessary control files, so you must build them by:

```
fakeroot debian/rules clean
```

Option B) Download the source archive

Download the source archive - This is for users who want to rebuild the standard Ubuntu packages with additional patches. Use a follow command to install the build dependencies and extract the source (to the current directory):

1. Install the following packages:

```
sudo apt-get build-dep linux-image-`uname -r`
```

Option C) Download the source package and build

This is for users who want to modify, or play around with, the Ubuntu-patched kernel source.

1. Retrieve the latest kernel source from kernel.org.
2. Extract the archive to a directory and `cd` into it:

```
tar xf linux-*.tar.xz
cd linux-*
```

3. Build the ncurses configuration interface:

```
make menuconfig
```

4. To accept the default configuration, press → to highlight `< Exit >` and then Return.
5. Press Return again to save the configuration.
6. Use `make` to build the kernel:

```
make
```

   Note that you can use the `-jem>` flag to compile files in parallel and take advantage of multiple cores.

The compressed kernel image can be found at `arch/[arch]/boot/bzImage`, where `[arch]` is equal to `uname -a`.

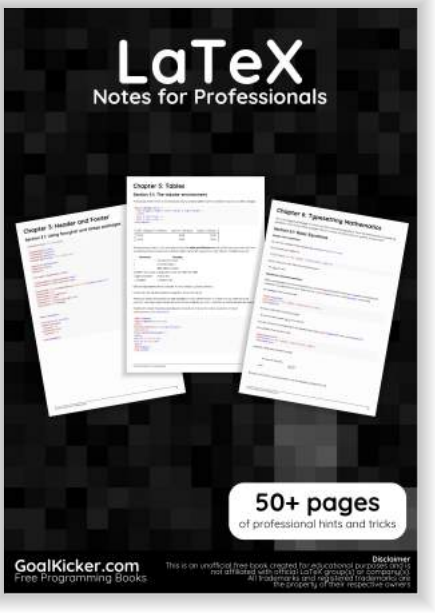
Credits

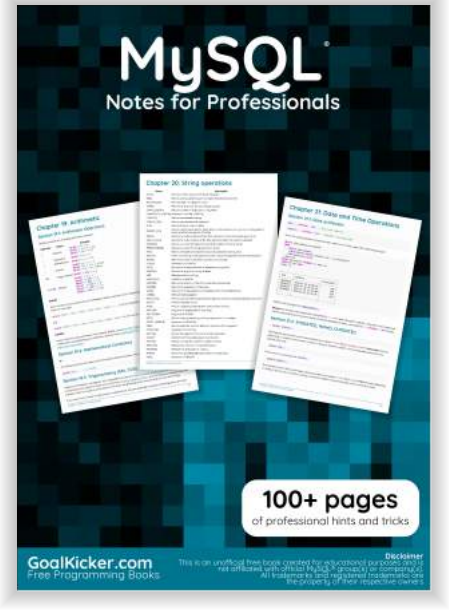
Thank you greatly to all the people from Stack Overflow Documentation who helped provide this content, more changes can be sent to web@petercv.com for new content to be published or updated

7heo.tk	Chapter 1
Aaron Skomra	Chapter 16
Ajay Sangale	Chapters 1 and 9
Anagh Hegde	Chapters 4 and 14
Ani Menon	Chapters 2 and 4
Arden Shackelford	Chapter 12
Armali	Chapter 1
Baard Kopperud	Chapter 8
BrightOne	Chapters 9, 13, 14 and 19
C.W.Holeman II	Chapter 14
caped114	Chapters 1 and 4
colelemonz	Chapter 1
ctafur	Chapter 17
DaveM	Chapter 2
depperm	Chapter 1
e.dan	Chapter 1
embedded	Chapter 8
Emmanuel Mathi	Chapter 4
EsmaeelE	Chapter 1
fdeslaur	Chapter 3
Federico Ponzi	Chapter 9
Filipe	Chapter 14
Flamewires	Chapter 10
FOP	Chapter 12
foxtrot9	Chapter 7
geek1011	Chapter 11
Jarryd	Chapter 1
Jensd	Chapters 1 and 4
KerDam	Chapter 1
Kiran Vemuri	Chapters 6 and 17
kuldeep mishra	Chapter 13
Léo Léopold Hertz 준영	Chapter 21
lardenn	Chapters 2 and 4
leeor	Chapter 9
likewhoa	Chapter 1
manav m	Chapter 15
Manuel	Chapter 14
Marsso	Chapter 8
Mateusz Piotrowski	Chapter 1
mattdm	Chapters 2, 10 and 11
mertyildiran	Chapter 5
Mike P	Chapter 1
Mohammad	Chapter 1
Nathan Osman	Chapter 21
Naveen Chakravarthy	Chapter 1
Nikhil Raj	Chapter 2
Not22	Chapter 8

oznek	Chapter 4
Paradox	Chapters 16 and 20
parkydr	Chapter 7
Philip Kirkbride	Chapters 4, 7, 12 and 20
Quaker	Chapter 4
Rajesh Rengaraj	Chapter 14
Riley Guerin	Chapter 15
Rubio	Chapters 1, 4 and 20
S.Rohit	Chapters 3, 6 and 12
Sava	Chapters 1, 4, 14 and 18
Sergey Stolyarov	Chapter 2
Sudip Bhandari	Chapter 1
Teddy	Chapter 2
Tejus Prasad	Chapter 1
TiansHUo	Chapter 1
Todd	Chapters 1 and 14
vishram0709	Chapter 1
Whoami	Chapter 1
Y4Rv1K	Chapters 6 and 17
Zumo de Vidrio	Chapter 1
zyio	Chapter 5

You may also like





Discounts

1) An article is marked up by 120% above its cost price and then sold for Rs. 1320 after giving 20% discount. Find the cost price of the article.

a) Rs. 600

b) Rs. 750

c) Rs. 960

d) Rs. 840

Correct Choice: b

Solution

Let the cost price of the article be Rs. ‘x’

According to the question,

2.2 × 0.8x = 1320

Or, x = 1320/1.76 = Rs. 750

Hence, option b.

Time & Work

2) Amar is 4 times more efficient than Amish. Both working together can complete the work in 12 days. Find the number of days taken by Amar to complete the work alone.

a) 16 days

b) 15.8 days

c) 12.5 days

d) 14.4 days

Correct Choice: d

Solution

Let the efficiency of Amish be x units/day

Therefore, efficiency of Amar = 4x + x = 5x units/day

Total work = (5x + x) × 12 = 72x units

Time taken by Amar to complete the whole work alone = 72x/5x = 14.4 days

Hence, option d.

Ratios & Proportions

3) In a bag there are coins of Rs. 1, Rs. 2, 25 paise and 50 paise in the ratio 4:2:5:3, respectively. If the total amount in the bag is Rs. 172. Find the difference between the number of Rs. 1 coins and 50 paise coins.

a) 16

b) 12

c) 18

d) 14

Correct Choice: a

Solution

Let the number of coins of Rs. 1, Rs. 2, 25 paise and 50 paise be 4x, 2x, 5x and 3x respectively

According to the question,

$4x + (2 \times 2x) + (5x/4) + (3x/2) = 172$

Or, $16x + 16x + 5x + 6x = 172 \times 4$

Or, $x = (172 \times 4)/43$

Or, $x = 16$

Required difference = $(4x – 3x) = x = 16$

Hence, option a.

Time & Distance

4) Two cyclists ‘A’ and ‘B’ are coming towards each other with a speed of 25 km/hr and 30 km/hr. If both of them meet after 48 minutes after starting and both start at the same time, then find the distance between them at the time they start.

a) 38 km

b) 36 km

c) 42 km

d) 44 km

Correct Choice: d

Solution

Required distance = $(25 + 30) \times (48/60) = 44$ km

Hence, option d.

Algebra

5) If (17/6) + (3x – 14/3) = 5x/2, then find the value of ‘x’.

a) 11/3

b) 16/3

c) 17/6

d) None of these

Correct Choice: a

Solution

According to the question,

{(5x/2) – 3x} = (17/6) – (14/3)

Or, x = 11/3

Hence, option a.

Progressions

6) The 3^{rd} and 7^{th} terms of an arithmetic progression is 143 and 399 respectively. Find its 15^{th} term.

a) 749

b) 865

c) 911

d) 857

Correct Choice: c

Solution

Let the first term and common difference of the series be ‘a’ and ‘d’ respectively

According to the question,

{a + (7 – 1)d} – {a + (3 – 1)d} = 399 – 143

Or, 4d = 256

Or, d = 64

Therefore, a = 143 - 128 = 15

Therefore, 15^{th} term of the series = a + (15 – 1)d = 911

Hence, option c.

Coordinate Geometry

7) Point (2, -1) is midpoint of points A(x, -6) and B(-3, y). Find the value of (x + y).

a) 11

b) -9

c) 15

d) -14

Correct Choice: a

Solution

According to the question,

$\{x + (-3)\}/2 = 2$

Or, $x = 4 + 3 = 7$

Also, $(-6 + y)/2 = -1$

Or, $y = -2 + 6 = 4$

Therefore, $(x + y) = 11$

Hence, option a.

Areas

8) The ratio of the length to breadth of a rectangular field is 7:4, respectively. If the total cost of fencing at the rate of Rs. 2.5/m is Rs. 550, then find the area of the field.

a) 56 dam^2

b) 28 m^2

c) 28 dam^2

d) None of these

Correct Choice: c

Solution

Let the length and breadth of the rectangular field be 7x metres and 4x metres respectively

Therefore, $2(7x + 4x) = 550/2.5$

Or, $x = 220/22 = 10$ metres

Therefore, area of the field = $7x \times 4x = 2800\ m^2 = 28\ dam^2$

Hence, option c.

Mixtures & Allegations

9) A 18 kg alloy ‘A’ of tin and copper contains 12 kg tin and rest copper. Alloy ‘A’ is mixed with alloy ‘B’ of tin and copper having the ratio 3:2 respectively. If the ratio of tin to copper in final mixture is 5:3, then find the quantity of copper in alloy ‘B’.

a) 12 kg

b) 16 kg

c) 10 kg

d) 15 kg

Correct Choice: a

Solution

Let the quantity of tin and copper in alloy ‘B’ be ‘3x’ kg and ‘2x’ kg respectively

According to the question,

$(12 + 3x)/(6 + 2x) = 5/3$

Or, $36 + 9x = 30 + 10x$

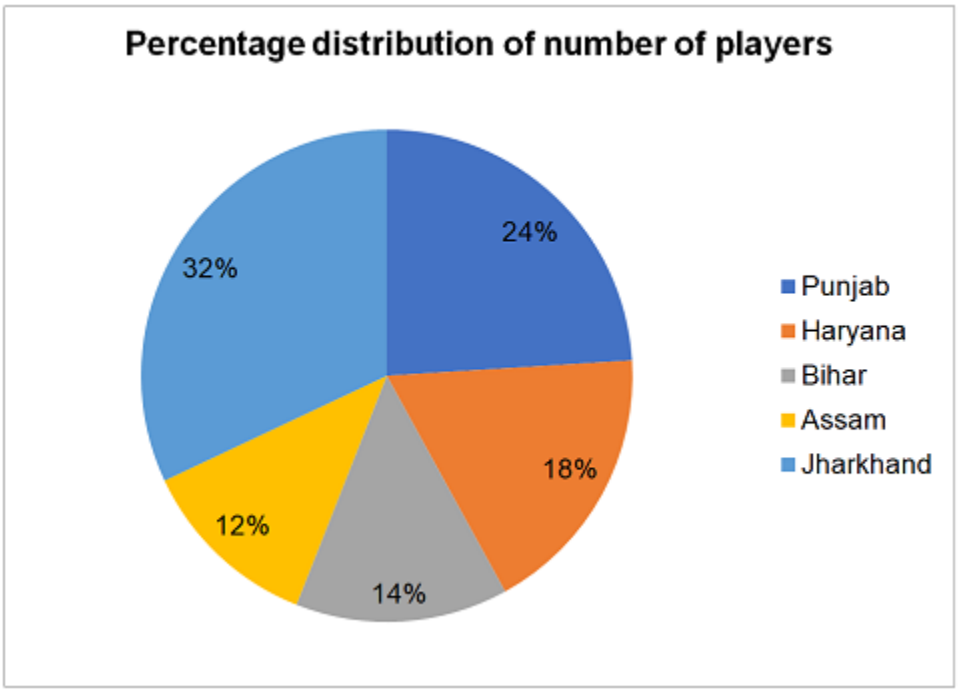
Or, $x = 6$

Therefore, quantity of Copper in alloy ‘B’ = $2x = 12$ kg

Hence, option a.

Data Interpretation (Pie Chart on Percentages)

10) The given pie-chart shows the percentage distribution of 1500 players (boys and girls) from five different states as total number of players from five states together who participated in Khelo India games.



If out of the total number of players from Haryana and Assam 2/3rd and 3/5th were boys respectively then find the difference between the number of girls who participated from given two states.

a) 32

b) 12

c) 24

d) 18

Correct Choice: d

Solution

Required difference = {(1/3) × 0.18 – (2/5) × 0.12} × 1500 = 18

Hence, option d.

Profit & Loss

11) A milk seller purchased 20 litres of milk at the rate of Rs. 25 per litre. While selling the milk, he added 4 litres of water to it and sold the mixture at the same rate. Find the profit/loss percentage of the milk seller.

a) 15%

b) 20%

c) 25%

d) 18%

Correct Choice: b

Solution

Total cost price for the milk seller = (20 × 25) = Rs. 500

Total selling price for the milk seller = (20 + 4) × 25 = Rs. 600

Required profit percent = {(600 – 500)/500} × 100 = 20%

Hence, option b.

Data Interpretation(Pie chart on percentages)

12) The given pie-chart shows the percentage distribution of 1500 players (boys and girls) from five different states as total number of players from five states together who participated in Khelo India games.

If 10%, 20%, 30%, 40% and 50% of the players from Punjab, Haryana, Bihar, Assam and Jharkhand won gold medals in games played by them, then find total number of games held. Only these five states participated.

a) 465

b) 540

c) 395

d) 620

Correct Choice: a

Solution

Required number of games = {(0.10 × 0.24) + (0.20 × 0.18) + (0.30 × 0.14) + (0.40 × 0.12) + (0.50 × 0.32)} × 1500 = 465

Hence, option a.

Problems on Numbers

13) When the numerator and denominator of a fraction is increased by 7 and 13 respectively the fraction becomes 3/5. Find the fraction if the denominator is 2 more than the numerator.

a) 3/5

b) 5/7

c) 11/13

d) 7/9

Correct Choice: b

Solution

Let the numerator of the fraction be ‘a’

According to the question,

{(a + 7)/(a + 2 + 13)} = 3/5

Or, 5a + 35 = 3a + 45

Or, a = 10/2 = 5

Required fraction = 5/7

Hence, option b.

14) 60% of a number is 45% of another number. If the sum of the numbers is 350, then find the difference of the numbers.

a) 50

b) 75

c) 80

d) 45

Correct Choice: a

Solution

Let the numbers be ‘x’ and ‘y’ respectively

According to the question,

0.60x = 0.45y

Or, x/y = 3/4

Therefore, (3 + 4) units = 350

Or, 1 unit = 50 units

Therefore, difference = 4 – 3 = 1 unit = 50

Hence, option a.

Data Interpretation (Tabular Form on percentages)

(15-17) Directions: Answer the questions based on the information given below.

The given table shows the income (in Rs.) of two persons and their percentage expenditure (in accordance to their incomes), in four different years.

	Income of ‘A’	Percentage expenditure of ‘A’	Income of ‘B’	Percentage expenditure of ‘B’
2010	45000	60%	32000	75%

2011	36000	75%	48000	50%
2012	48000	80%	25000	80%
2013	54000	45%	40000	65%

15) Find the ratio of the expenditures of ‘A’ in 2010 and 2011, together to the savings of ‘B’ in 2011.

a) 4:1

b) 9:4

c) 3:2

d) 7:5
Correct Choice: b

Solution

In 2010:

Income of ‘A’ = Rs. 45000

Expenditure of ‘A’ = 0.6 × 45000 = Rs. 27000

Expenditure of ‘A’ = 45000 – 27000 = Rs. 18000

Similarly,

	Income of 'A'	Expenditure of 'A'	Savings of 'A'	Income of 'B'	Expenditure of 'B'	Savings of 'B'
2010	45000	27000	18000	32000	24000	8000
2011	36000	27000	9000	48000	24000	24000
2012	48000	38400	9600	25000	20000	5000
2013	54000	24300	29700	40000	26000	14000

Required ratio = (27000 + 27000):24000 = 9:4

Hence, option b.

16) Find the difference between savings of 'A' in 2013 and expenditures of 'B' in 2011 and 2013, together.

a) Rs. 24300

b) Rs. 19500

c) Rs. 18400

d) Rs. 20300
Correct Choice: d

Solution

In 2010:

Income of ‘A’ = Rs. 45000

Expenditure of ‘A’ = 0.6 × 45000 = Rs. 27000

Expenditure of ‘A’ = 45000 – 27000 = Rs. 18000

Similarly,

	Income of ‘A’	Expenditure of ‘A’	Savings of ‘A’	Income of ‘B’	Expenditure of ‘B’	Savings of ‘B’
2010	45000	27000	18000	32000	24000	8000
2011	36000	27000	9000	48000	24000	24000
2012	48000	38400	9600	25000	20000	5000
2013	54000	24300	29700	40000	26000	14000

Required difference = (24000 + 26000) – 29700 = Rs. 20300

Hence, option e.

17) The expenditure of ‘A’ in 2012 is how much percent of income of ‘B’ in 2010 and 2011, together?

a) 60%

b) 56%

c) 54%

d) 48%

Correct Choice: d

Solution

In 2010:

Income of ‘A’ = Rs. 45000

Expenditure of ‘A’ = 0.6 × 45000 = Rs. 27000

Expenditure of ‘A’ = 45000 – 27000 = Rs. 18000

Similarly,

	Income of ‘A’	Expenditure of ‘A’	Savings of ‘A’	Income of ‘B’	Expenditure of ‘B’	Savings of ‘B’
2010	45000	27000	18000	32000	24000	8000
2011	36000	27000	9000	48000	24000	24000
2012	48000	38400	9600	25000	20000	5000
2013	54000	24300	29700	40000	26000	14000

Required percentage = {38400/(32000 + 48000)} × 100 = 48%

Hence, option d.

Time & work

18) ‘A’ and ‘B’ together working with 75% of their efficiencies can complete a work together in 20 days while ‘A’ alone working with his original efficiency takes 48 days to complete the same work. Find the time taken by ‘B’ alone (working with his original efficiency) to complete 55% of the work.

a) 10 days

b) 18 days

c) 12 days

d) 15 days
Correct Choice: c

Solution

Let the total work be 240 units

75% of efficiency of (A + B) = 240/20 = 12 units/day

Original efficiency of ‘A’ = 240/48 = 5 units/day

Original efficiency of (A + B) = 12/0.75 = 16 units/day

Therefore, original efficiency of ‘B’ = 16 – 5 = 11 units/day

Required time taken = 0.55 × (240/11) = 12 days

Hence, option c.

Mixtures & Allegations

19) A mixture contains paint and oil in the ratio 11:8, respectively. 20% of the mixture is taken out and replaced with ‘y’ litres of paint and ‘2y’ litres of oil such that the ratio of paint to oil in the resultant mixture becomes 8:9. Find the difference between quantity of oil added and the initial quantity of oil, in the mixture.

a) 8 liters

b) 5 liters

c) 10 liters

d) None of these

Correct Choice: d

Solution

Let the initial quantity of paint and oil in the mixture be 11x litres and 8x litres, respectively

According to the question,

{(0.8 × 11x) + y} / {(0.8 × 8x) + 2y} = 8/9

Or, 79.2x + 9y = 51.2x + 16y

Or, 7y = 28x

Or, y = 4x

Or, 2y = 8x

Therefore, quantity of oil added in the mixture = 2y = 8x litres

Initial quantity of the oil in the mixture = 8x litres

Difference = 8x – 8x = 0

Hence, option d.

Partnership

20) ‘A’, ‘B’ and ‘C’ invested Rs. 2400, Rs. 8000 and Rs. 3200, in a business together. After 8 months, ‘B’ left and ‘A’ and ‘C’ added 25% more amount of their initial investments. If the profit received by ‘B’ at the end of the year is Rs. 12000, then find the total profit received by A, B and C together.

a) Rs. 21320

b) Rs. 25650

c) Rs. 14560

d) Rs. 22840
Correct Choice: b

Solution

Ratio of the profits received by ‘A’, ‘B’ and ‘C’

= {(2400 × 8) + (1.25 × 2400 × 4)}:{(8000 × 8)}:{(3200 × 8) + (1.25 × 3200 × 4)} = 39:80:52

Therefore, required profit received = 12000 × (39 + 80 + 52)/80 = Rs. 25650

Hence, option b.

Areas

21) The sum of the perimeters of an equilateral triangle and a square is 120 cm. If their perimeters are equal then find the area of the square.

a) 256 cm^2

b) 225 cm^2

c) 196 cm^2

d) 400 cm^2
Correct Choice: b

Solution

Let each side of square be ‘a’ cm

According to the question,

Perimeter of the square = 120/2 = 60 cm

Or, 4a = 60

Or, a = 15

Therefore, area of the square = 15^2 = 225 cm^2

Hence, option b.

Simple Interest – Compound Interes

22) Rs. (x + 200) when invested at 15% p.a. for 8 years gives a simple interest of Rs. (x + 680). Find the amount received when Rs. (x + 1000) is invested at 50% p.a. compound interest for two years, compounded annually.

a) Rs. 5000

b) Rs. 6400

c) Rs. 7200

d) Rs. 8000

Correct Choice: c

Solution

According to the question,

{(x + 200) × 15 × 8}/100 = (x + 680)

Or, 1.2x – x = 680 – 240

Or, x = 440/0.2 = 2200

Required amount received = $(x + 1000)(1 + 50/100)^2 = 3200(1 + 50/100)^2$ = Rs. 7200

Hence, option c.

Averages

23) The average of three numbers (a, b and c) is 320. ‘b’ is 25% more than ‘a’ and 120 less than ‘c’. Find the average of ‘a’ and ‘c’.

a) 330

b) 300

c) 350

d) 280

Correct Choice: a

Solution

According to the question,

b = 1.25a and c = 1.25a + 120

According to the question,

1.25a + 1.25a + 120 + a = 3 × 320

Or, a = 840/3.5 = 240

Required average = (a + 1.25a + 120)/2 = 660/2 = 330

Hence, option a.

Problems on Ages

24) The ratio of the ages of ‘A’ and ‘B’, 8 years ago from now was 3:2, respectively. If the ratio of their ages 4 years hence form now will be 9:7, respectively, then find the difference between their present ages.

a) 4 years

b) 8 years

c) 2 years

d) 6 years
Correct Choice: b

Solution

Let the ages of ‘A’ and ‘B’, 8 years ago from now be 3x years and 2x years, respectively

According to the question,

(3x + 8 + 4) / (2x + 8 + 4) = 9/7

Or, 21x + 84 = 18x + 108

Or, 3x = 24

Or, x = 8

Required difference = 3x – 2x = x = 8 years

Hence, option b.

Percentages

25) In a group of animals (cows + horses), if 40% of cows left and same number of horses join the group, then the number of horses would have been 50% more than that of cows. Find the original percentage of horses in the group of animals.

a) 11.5%

b) 20%

c) 33.33%

d) 16.66%
Correct Choice: c

Solution

When 40% of cows left and same number of horses join:

Let the number of cows be ‘x’

Therefore, number of horses = 1.5x

Total number of animals = x + 1.5x = 2.5x

Now, original number of cows = x/0.6 = 5x/3

Therefore, original number of horses = 2.5x – (5x/3) = 5x/6

Required percentage = {(5x/6)/2.5x} × 100 = 33.33%

Hence, option c.

Comparison on Boats & Streams - Q1, Q2

26) In the question, two quantities I and II are given. You have to solve both the quantities to establish the correct relation between Quantity-I and Quantity-II and choose the correct option.

Quantity-I: Boat ‘A’ can travel 125 km upstream in 12.5 hours and 340 km downstream in 8.5 hours. The speed of the boat in still water is what percent of the speed of the current.

Quantity-II: A boat can travel 110 km downstream in 5 hours. If the speed of the boat in still water is 6 km/hr more than that of current, then the speed of the boat in still water is what percent of the speed of the current.

https://www.freshersnow.com/placement-papers-download/

a) Quantity-I > Quantity-II

b) Quantity-I < Quantity-II

c) Quantity-I ≤ Quantity-II

d) Quantity-I = Quantity-II or No relation

Correct Choice: b

Solution

Quantity I:

Let the speed of the boat in still water be ‘x’ km/hr and the speed of the current be ‘y’ km/hr

According to the question,

(x - y) = 125 / 12.5 = 10… .. (1)

Also, (x + y) = 34/8.5 = 40….. (2)

On solving equation (1) and (2), we get

Speed of the boat in still water = x = 25 km/hr

Speed of the current = 15 km/hr

Required percentage = 25/15 × 100 = 166.67%

Quantity II:

Let the speed of the current be ‘a’ km/hr

Therefore, speed of the boat in still water = (a + 6) km/hr

According to the question,(a + a + 6) = 110/5 = 22

Or, a = 16/2 = 8

Required percentage = 14/8 × 100 = 175%

Hence, option b.

FRESHERS NOW

TCS NQT PAPER -10

Verbal ability

Reading comprehension

(1-4) Directions: Read the following passage carefully and answer the questions.

The term herd immunity comes from the observation of how a herd of buffaloes forms a circle, with the strong on the outside protecting the weaker and more vulnerable on the inside. This is similar to how herd immunity works in preventing the spread of infectious diseases. Those who are strong enough to get vaccinated directly protect themselves from infection. They also indirectly shield vulnerable people who cannot be vaccinated.

There are various reasons a person may not be able to be successfully vaccinated. People undergoing cancer treatment, and whose immune systems are compromised, for instance, are impaired in their ability to develop protective immunity from all vaccines. Often, people who can't be vaccinated are susceptible to the most serious consequences from being infected. Another vulnerable group are babies. Infants under six months of age are susceptible to serious complications from influenza. Yet they can't be given the flu vaccine as their immune systems are not strong enough.

For a contagious disease to spread, an infectious agent needs to find susceptible (non-immune) people to infect. If it can't, the chain of infection is interrupted and the amount of disease in the population reduces. Another way of thinking about it is that the disease needs susceptible victims to survive in the population. Without these, it effectively starves and dies out.

How many people need to be vaccinated to achieve herd immunity varies from disease to disease. Measles can be transmitted through coughing and sneezing and the virus causing measles can survive outside the body for up to two hours. So it's possible to catch measles just by being in the same room as someone who

is ill if you touch a surface they've coughed or sneezed on. In contrast, Ebola can only be spread by direct contact with infected secretions (blood, faeces or vomit) and therefore requires close contact with an ill person. This makes it much less spreadable.

Measles is one of the most infectious diseases to affect humans. To achieve herd immunity to measles in a population we need 92-95% of the population to be vaccinated. Current data indicates full vaccine coverage for five year olds in Australia is sitting at around the 95% level. However, vaccination rates in some communities have fallen below ideal levels, making them susceptible to measles outbreaks. The overwhelming success of measles vaccinations means many people have no memory of what this disease looks like, and this has resulted in its effects being underestimated. Measles can cause blindness and acute encephalitis (inflammation of the brain), which can result in permanent brain damage.

Herd immunity, or community immunity, as it's sometimes called, is a powerful public health tool. By ensuring those who can be vaccinated do get vaccinated we can achieve herd immunity and prevent the illness and suffering that comes from the spread of infectious diseases.

1.As per the passage, which of the following groups of people may not be able to be successfully vaccinated?

1. People who are undergoing cancer treatment

2. People who are undergoing surgery

3. People whose immune systems are weak

A-Only 1
B-Only 1 and 2
3-Only 1 and 3
D-Only 2 and 3

Solution

In the second para, the author tells us why certain groups of people may not be successfully vaccinated. Both 1 and 3 are mentioned. Refer to the lines: *"There are various reasons a person may not be able to be successfully vaccinated. People*

undergoing cancer treatment, and whose immune systems are compromised, for instance, are impaired in their ability to develop protective immunity from all vaccines." **Thus, (c) is the right answer.**

2 is incorrect as it is not mentioned or implied in the passage.

2. **The passage provides sufficient information to answer which of the following questions?**

1. For which disease has herd immunity been achieved in India?

2. From where did the term herd immunity come from?

3. Until what age can infants not be given the flu vaccine?

A -Only 2
B -Only 1 and 2
C -Only 1 and 3
D -Only 2 and 3

Solution

2 is answered by the information in the first para. Refer to the lines: *"The term herd immunity comes from the observation of how a herd of buffaloes forms a circle, with the strong on the outside protecting the weaker and more vulnerable on the inside."*

3 is answered by the information in the second para. Refer to the lines: *"Infants under six months of age are susceptible to serious complications from influenza. Yet they can't be given the flu vaccine as their immune systems are not strong enough."*

1 is not answered by the passage.

Thus, (d) is the right answer.

3. Which of the following statement(s) is/ are true as per the passage?

1. Ebola is transmitted only through direct contact with infected bodily secretions.

2. Measles can result in permanent brain damage.

3. To achieve herd immunity to measles we need 92-95% of that population to be vaccinated.

A -Only 3
B -Only 1 and 2
C -Only 1 and 3
D -All 1, 2 and 3

Solution

All 1, 2 and 3 are mentioned in the passage. Refer to the lines: *"In contrast, Ebola can only be spread by direct contact with infected secretions (blood, faeces or vomit) and therefore requires close contact with an ill person...To achieve herd immunity to measles in a population we need 92-95% of the population to be vaccinated...Measles can cause blindness and acute encephalitis (inflammation of the brain), which can result in permanent brain damage."* **Thus, (d) is the right answer.**

4. Which of the following best explains herd immunity?

A -Herd immunity means that every single member of the population is vaccinated to a disease.
B -Herd immunity protects a vulnerable section of the population through the stronger immune majority.
C -Herd immunity is achieved only through vaccination in the infancy stage.
D -Herd immunity is achieved only when successive generations of a population are found immune to a disease.

Solution

In the first para of the passage, the author explains the concept of herd immunity. The author tells us how a strong population which is immune to a disease help in

protecting a smaller vulnerable section. Refer to the lines: *"This is similar to how herd immunity works in preventing the spread of infectious diseases. Those who are strong enough to get vaccinated directly protect themselves from infection. They also indirectly shield vulnerable people who cannot be vaccinated."* **Thus, (b) is the right answer**.

None of the other options is mentioned or implied in the passage.

Error location

5. In the following question, one part of the sentence may have an error. Find out which part of the sentence has an error and select the appropriate option. If the sentence is free from error, select 'No Error'.

A -Why do you insist on paying
B -for your school expenses yourself,
C -when your parents are willing
D -to given you financial support?

Solution

In D, replace the past participle form verb GIVEN with GIVE as the infinitive construction TO + VERB will take the base form of the verb. **Thus, (d) is the right answer.**

Error location

6. **In the following question, one part of the sentence may have an error. Find out which part of the sentence has an error and select the appropriate option. If the sentence is free from error, select 'No Error'.**

A -The new terminal will permitted
B -the airport to handle 35 million
C -passengers a year, more than
D -California's entire population.

FRESHERS NOW

Solution

In A, replace the second form verb PERMITTED with PERMIT as the modal verb WILL will take the base form of the main verb that follows it. **Thus, (a) is the right answer.**

Phrase replacement

7. Which of the following phrases (1), (2), and (3) given below each sentence should replace the phrase printed in bold letters to make the sentence grammatically correct? Choose the best option among the five given alternatives that reflect the correct use of phrase in the context of the grammatically correct sentence. If the sentence is correct as it is, mark 'No Improvement' as the answer.

Had war broken out, the Polish navy was prepared to invade the Danish isles.

1. If war had broke out

2. Having broken out war

3. War had broken out

A -Only 1 and 2
B -No Improvement
C -Only 1 and 3
D -Only 2 and 3

Solution

The sentence talks about how the Polish navy was prepared to invade the Danish isles if war broke out. The highlighted phrase is correct, which makes (b) the right answer.

1 is incorrect as the past perfect tense construction HAD + VERB will take the past participle BROKEN and not the second form verb BROKE. 2 is incorrect as it changes the subject to WAR, while the subject of the sentence is the POLISH NAVY. 3 is incorrect as it fails to link itself to the rest of the sentence.

Phrase replacement

8. Which of the following phrases (1), (2), and (3) given below each sentence should replace the phrase printed in bold letters to make the sentence grammatically correct? Choose the best option among the five given alternatives that reflect the correct use of phrase in the context of the grammatically correct sentence. If the sentence is correct as it is, mark 'No Improvement' as the answer.

The disease is caused by the chikungunya virus, which is inhibited mainly by mosquitoes in the Aedes family.

1. Is transmitted mainly

2. Is primarily spread

3. Is transported chiefly

A -No Improvement
B -Only 1 and 2
C- Only 1 and 3
D -Only 2 and 3

Solution

The sentence tells us about the chikungunya virus and the vector that carries it. Both 1 and 2 can replace the highlighted phrase to convey the requisite meaning. **Thus, (b) is the right answer**.

3 is incorrect as TRANSPORTED cannot be used with respect to a disease. The highlighted phrase is incorrect as INHIBITED means restricted, which does not lend meaning to the context.

Para-jumbled sentences

9. Directions: In the question given below, a set of sentences is given, which when properly sequenced, form a coherent paragraph. Arrange the sentences in the correct sequence, and answer the questions.

P. Controlling the spread of flu means dealing with this ongoing evolution.

Q. Every few years, new variants of flu emerge and cause epidemics around the world.

R. Evolution is usually very slow, a process of change that takes thousands or millions of years to see.

S. But for influenza, evolution is fast – and deadly.

T. Moreover, flu viruses change rapidly to escape the body's defenses.

Which of the following statements would be the FIRST after rearrangement?

A -R
B -P
C -S
D –T

Solution

RSQTP is the final order. R begins the sentence by telling us how evolution is a slow process. S follows by giving an exception to this - the influenza virus. QT follow as a pair - Q tells us that new variants of flu emerge every few years, thus confirming the fact mentioned in S. T follows by telling us how flu viruses keep changing rapidly to evolve to the body's defenses. P concludes by telling us how we need to deal with this fast evolution if we are to control the spread of flu. **Thus, (b) is the right answer**.

Cloze test

(11-15) Directions: In the following passage, some of the words have been left out and replaced by a blank represented by a letter. First, read the passage and try to understand what it is about. Then fill in the blanks as per the questions given.

Low-interest rates have traditionally been ___(A)____ as positive for economic growth. But the recent research suggests that this may not be the case. Instead, extremely low-interest rates may ___(B)____ to slower growth by increasing market concentration. It ____(C)___ that reducing interest rates further will not save the global economy from stagnation. The traditional view _____D____ that

when long-term rates fall, the net present value of future cash flows increases, making it more attractive for firms to ___(E)___ in productivity-enhancing technologies. Low-interest rates, therefore, have an expansionary effect on the economy through stronger productivity growth.

11.(A)

A -Allured
B -Accrued
C -Viewed
D –Shrewd

Solution

We need the third form of a verb to fit in the present perfect construction HAVE + BEEN + VERB. VIEWED means 'regarded'. It fits contextually as the sentence talks about the traditional view of low-interest rates. **Thus, (c) is the correct answer.**

Allured — Attracted. Accrued — Accumulated. Shrewd — cunning. Feud — Conflict.

12. (B)

A -Plead
B -Lead
C -Spread
D -Aforesaid

Solution

We need a verb in the base form here to follow the modal verb MAY. LEAD TO means to result in, and it fits contextually as the sentence talks about how extremely low interest rates can influence the growth rate of the economy. **Thus, (b) is the correct answer.**

Plead — implore. Spread — Extend. Aforesaid — Preceding.

13. (C)

A -Implies
B -Defies
C -rectify
D –Suffices

Solution

We need a singular verb to agree with the singular pronoun IT. IMPLIES means 'Indicates'. It fits contextually as the sentence talks about the traditional view and what it suggests. **Thus, (a) is the correct answer.**

Defies — Disobeys. Rectify— Amend. This verb is in the plural form.

Suffices — to be enough. Apprise — notify. This verb is in the plural form.

14. (D)

A -Aggravates
B -Holds
C -Opines
D -Both (b) and (c)

Solution

We need a verb in the singular form to agree with the singular subject VIEW. HOLDS and OPINES means 'to support'. It fits contextually as the sentence talks about the premise on which traditional view is based on. **Thus, (d) is the correct answer.**

Aggravates — making the problem worse. Abrogates — Repeals.

15. (E)

A -Manifest
B -Inquire

C -Divest
D –Invest

Solution

We need a verb in its base form to fit in the infinitive construction TO + VERB. INVEST means 'spend'. It fits contextually as the sentence talks about the net present value of future cash flows and how it enables firms to invest in productivity-enhancing technologies. **Thus, (d) is the correct answer**.

Manifest — Apparent. Inquire — ask. Divest — Deprive. Behest — Request.

Phrasal verbs/idioms

16. In the following question, a sentence is given with a phrase or idiom in brackets. Select the option given below that can replace the bracketed phrase.

The man did not (let me in) the room since a meeting was going on inside.

A -let me off
B -let me out
C -let me down
D -No Improvement

Solution

We need a phrasal verb here that means to allow someone to enter inside a room. LET ME IN will fit here. **Thus, (d) is the right answer.**

LET OFF means to not punish someone for a wrongdoing. LET OUT means to allow one to leave a place. LET DOWN means fail to meet expectations.

Active/passive voice

17. In the following question, a sentence has been given in Active/ Passive Voice. Out of the four alternatives suggested, select the one which best expresses the same sentence in Passive/Active Voice.

I was watching a movie last night.

A -I watched a movie last night.
B -A movie was watched by me last night.
C -A movie was being watched by me last night.
D -A movie had been watched by me last night.

Solution

The sentence is in active voice and past continuous tense. Follow these rules to convert it into passive voice:

1. The subject clause will become the object clause. Here, the subject pronoun 'I' will change to the object of the verb and assume the object form 'me'.

2. The object 'A MOVIE' will become the subject and begin the sentence.

3. To form the passive voice construct, change 'WAS WATCHING' to 'WAS BEING WATCHED'.

4. Add the conjunction 'BY' before 'ME' to link the verb with its object.

Thus, (c) is the correct answer.

antonyms

18. In the following question, choose the word opposite in meaning to the given word.

Undaunted

A –hazy

B –timid

C –livid

D –dense

Solution

(a) is the right answer. The idiom ROOT AND BRANCH means in an extensive and comprehensive manner.

Phrasal verbs/idioms

20. In the following question, a sentence is given with a phrase or idiom in brackets. Select the option given below that can replace the bracketed phrase.

The plane (took out) from Miami two hours ago.

A -took off
B -took on
C -took after
D -No Improvement

Solution

We need a phrasal verb here that means to depart from a place. TOOK OFF will convey the requisite meaning here. **Thus, (a) is the right answer.**

TOOK OUT means to remove or extract. TOOK ON means to hire or engage staff. TOOK AFTER means to inherit traits from someone.

One word substitution

21. In the following question, out of the four alternatives, choose the one which can be substituted for the given words/sentence.

A feel of discomfort whose cause is hard to identify

A -menace
B -malaise
C -debacle
D –magnate

Solution

(b) is the right answer.

Menace – something that causes annoyance.

Debacle – collapse of something.

Magnate - a wealthy and influential businessman.

idioms

22. In the question below, an idiom is given, followed by four options. Select the option that gives the correct meaning of the idiom.

After his father died, he was left to paddle his own canoe and take charge of the family business.

A -to have freedom to do as one wishes
B -to have no one to comfort one
C -to take responsibility at a very young age
D -to act independently and do something oneself

Solution

(d) is the right answer. The idiom PADDLE ONE'S OWN CANOE means to take independent decisions and do a task oneself.

Phrasal verbs/idioms

23. In the following question, a sentence is given with a phrase or idiom in brackets. Select the option given below that can replace the bracketed phrase.

I (wore off) my shoes while running.

A -wore on
B -wore out
C -wore over
D -No Improvement

Solution

We need a phrasal verb that means to become unusable. WORE OUT will fit here. **Thus, (b) is the right answer.**

WORE ON and WORE OVER are grammatically incorrect. WORE OFF refers to the effect of a drug that is no longer effective.

Active/passive voice

24. In the following question, a sentence has been given in Active/ Passive Voice. Out of the four alternatives suggested, select the one which best expresses the same sentence in Passive/Active Voice.

Have you finished your meal?

A -Has your meal been finished by you?
B -Have you finished the meal?
C -Have the homework been completed by you?
D -Passive form is not possible.

Solution

The sentence is in interrogative mood, active voice and present perfect tense. Follow the rules below to convert a sentence in interrogative mood to passive voice:

1. The subject clause will become the object clause. Here, the subject pronoun 'YOU' will change to the object of the verb and assume the object form 'YOU'.

2. The object 'YOUR MEAL' will become the subject and will come at the beginning of the sentence.

3. The helping verb 'HAVE' will change to 'HAS' in accordance with its subject 'YOUR MEAL' and will begin the interrogative sentence.

4. To form the passive voice construct, change 'FINISHED' to 'BEEN FINISHED'.

5. Add the conjunction 'BY' before 'YOU' to link the verb with its object.

Thus, (a) is the correct answer.

Reasoning ability

Alphabet series

25. Select the combination of letters that when sequentially placed in the gaps of the given letter series will complete the series.

p _ q r r _ t t u _ p q r r s _ t u p p _ r r s t _ u

A -psptqu
B -psqtut
C -qspsqu
D –psptqt

Solution

From option (d),

p p q r r s t t u / p p q r r s t t u / p p q r r s t t u

Hence, option d.

Counting the no.of figures

26. How many triangles are there in the following figure?

A -4
B -3
C -5
D -6

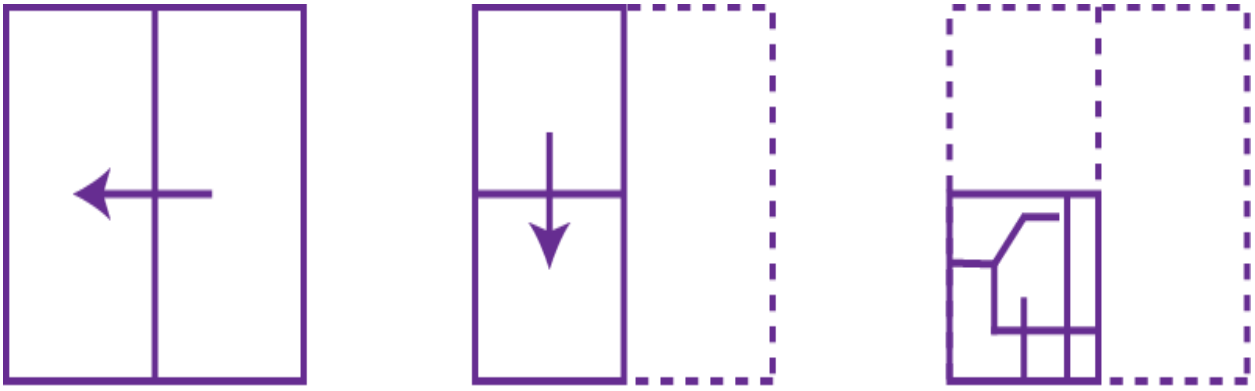
Solution

There are total 5 triangles present in the figure.

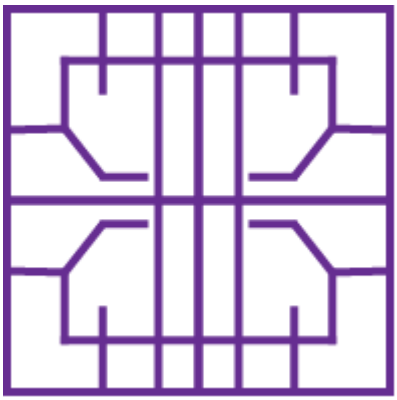
Hence, option c.

Paper folding

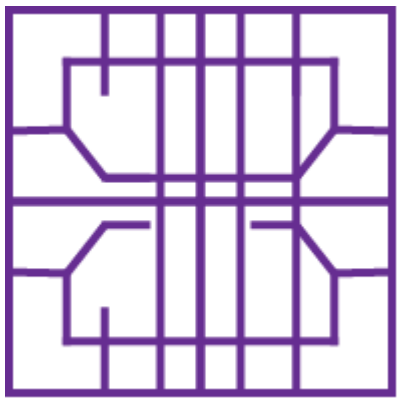
27. A square paper is folded and cut as shown below. How will it appear when unfolded?



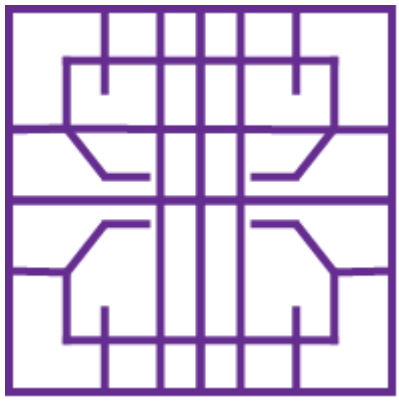
A -



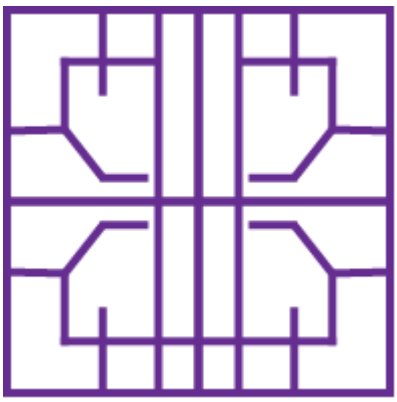
B -



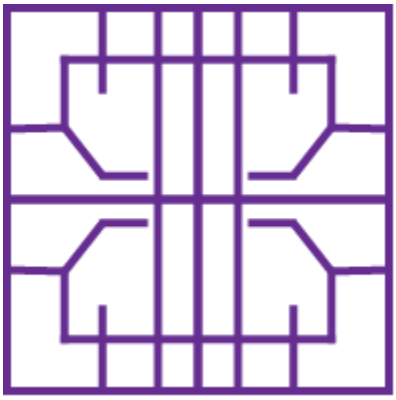
C -



D -



Solution



Hence, option a.

Mathematical operations(interchange of signs&numbers)

28. Which of the following “signs” are placed in place of “symbols” given in equation to make the equation true?

8 @ 3 # 2 – 7 × 4 ÷ 21 $ 2 ÷ 3 = 10

A -×, -, +
B -÷, +, ×
C -×, ÷, -
D -÷, ×, +

Solution

Given equation:

8 @ 3 # 2 – 7 × 4 ÷ 21 $ 2 ÷ 3 = 10

So, if place ×, ÷ and -, we get,

8 × 3 ÷ 2 – 7 × 4 ÷ 21 - 2 ÷ 3 = 10

Hence, option c.

Coding-decoding(matrix coding)

29. In the question, a word is represented by only one set of number as given in any one of the alternatives. The sets of numbers given in the alternatives are represented by 2 classes of alphabets as in two matrices given below. The columns and rows of matrix I are numbers from 0-4 and that of matrix II are numbers from 5-9. A letter from this matrix can be represented 1st by its row and next by its column. Ex- 'A' can be represented by 20, 42 etc. and 'B' can be represented by 22, 43, 76 etc. Similarly, you have to identify the set of word 'VOLCANIC'.

Matrix - I

	0	1	2	3	4
0	P	N	R	C	O
1	W	P	V	S	L
2	A	H	B	C	U
3	Z	I	T	E	M
4	M	R	A	B	W

Matrix - II

	5	6	7	8	9
5	J	P	J	T	S
6	I	J	E	Y	P
7	L	B	C	N	V
8	F	Y	M	P	D
9	M	R	U	V	K

A -98, 04, 14, 32, 42, 01, 65, 77
B -12, 40, 14, 23, 42, 01, 65, 77
C -98, 04, 14, 23, 24, 10, 65, 77
D -98, 04, 14, 23, 42, 01, 65, 77

Solution

As the digits of the numbers represented by columns and rows respectively,

V	O	L	C	A	N	I	C
12, 98, 78	04	14, 75	03, 23, 77	20, 42	01, 78	31, 65	03, 23, 77

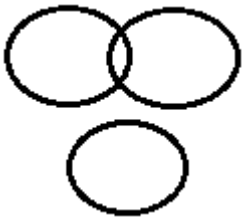
Hence, option d.

Logical venn diagrams type-1

30. Select the Venn diagram that best illustrates the relationship between the following classes:

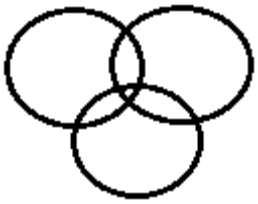
Worker, Boy, Employee

A -



B -

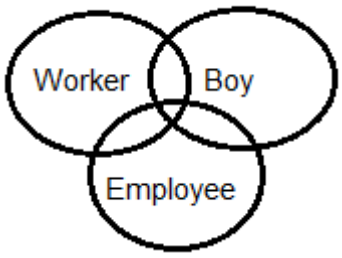
C -



D -

Solution

Following Venn diagram represents the relation between Worker, Boy, Employee



Hence, option c.

Missing no. series

31. Complete the given series.

7.1 ? 11 14.95 20.5

A -8.01
B -8.54
C -7.45
D -8.45

Solution

Given series:

7.1 8.45 11 14.95 20.5

1.35 2.55 3.95 5.55

1.2 1.4 1.6

0.2 0.2

Hence, option d.

Blood relations

32. If brother of L's mother's mother is the father of D's father. How is father-in-law of father of L related to D's mother's father in law?

A -Brother-in-law
B -Son-in-law
C -Brother
D –Uncle

Solution

Father-in-law of father of F is the brother-in-law of D's mother's father-in-law.

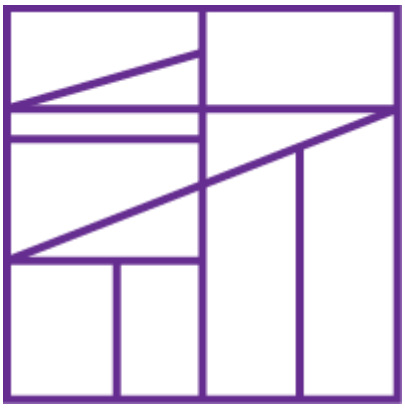
Hence, option a.

Mirror image

33. Identify the mirror image of the following figure when the mirror is placed to the right of the figure.

A -

B -

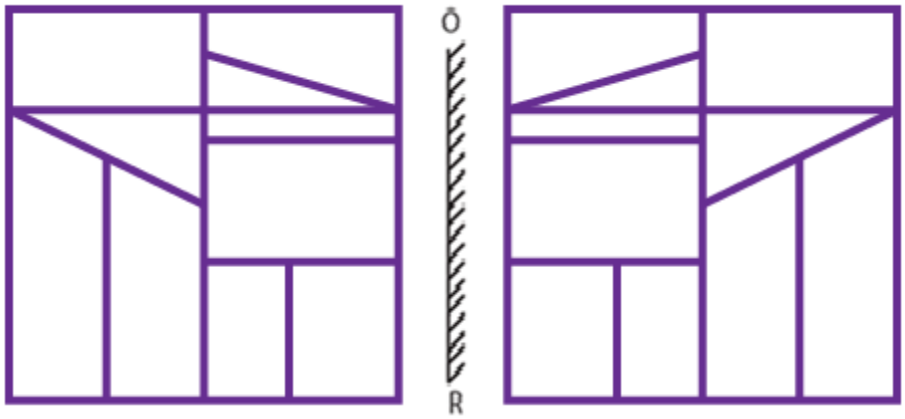


C -

D -

Solution

If mirror is placed to the right of the figure.



Hence, option d.

syllogisms

34. Three statements are given followed by three conclusions numbered I, II, and III assuming the statements to be true, even if they seem to be at variance with commonly known facts. Decide which of conclusion logically follow(s) from the statement.

Statements:

A few suns are earths.

No earths are mars.

All suns are moons.

Conclusions:

I. No mars are moons.

II. Mostly suns are not mars.

III. All earths are suns.

A -Only conclusion I follows

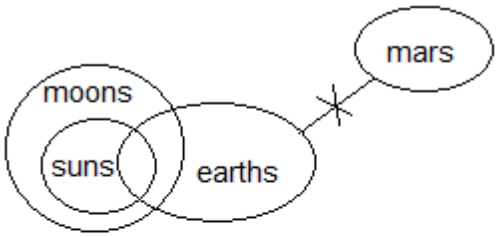
B -Only conclusion II follows

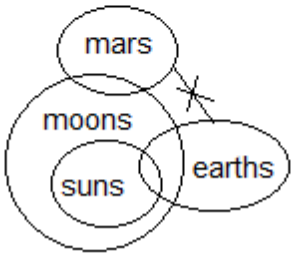
C -Conclusion II and conclusion III follow

D -Conclusion I and conclusion III follow

Solution

Following figure can be formed from the statements.



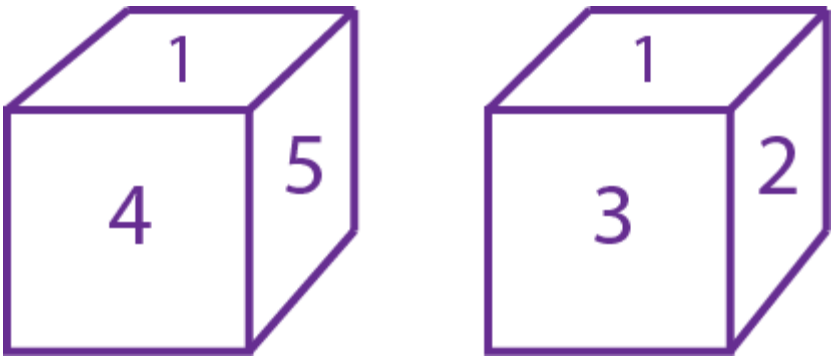


Only conclusion II follows.

Hence, option b.

Cubes&dices

35. Two positions of the same dice are given, which of the following number will be opposite to 1?



A -6
B -5
C -1
D -3

Solution

The number opposite to 1 would be '6'.

Hence, option a.

Classification (choose the odd numeral)

36. Three of the following four numbers are alike in a certain way and one is different. Pick the odd one out.

A -86
B -96
C -39
D -37

Solution

Among the given numbers, only 37 is a prime number.

Hence, option d.

Analogy(complete the analogous pair)

37. Select the option that is related to the third number in the same way as the second number is related to the first number.

290 : 17 :: ? : 19

A -360
B -359
C -326
D -362

Solution

$(290 – 1) = 17^2$

Similarly, $(362 – 1) = 19^2$

Hence, option d.

Coding-decoding(number/symbol coding)

38. 'DRAIN' will be coded as ____, if 'D' is coded as 4 and 'DIG' is coded as 20.

A -47
B -46
C -45
D -42

Solution

If 'D' is coded as 4, 'DIG' is coded as D(4) + I(9) + G(7) = 20, it means the letter are numbered as the alphabetical order, so 'DRAIN' is coded as D(4) + R(18) + A(1) + I(9) + N(14) = 46.

Hence, option b.

Coding-decoding(direct letter coding)

39. If 'KGLEJC' is coded as 'MINGLE' and 'BMLICW' is coded as 'DONKEY', then 'FLIGHTS' will be coded as ___?

A -HNIKJVU
B -HNJIKUV
C -HOKIKVU
D –HNKIJVU

Solution

If 'KGLEJC' is coded as 'MINGLE' and 'BMLICW' is coded as 'DONKEY', which means all the letters of the words are replaced by their second succeeding letters in the alphabetical series, so 'FLIGHTS' will be coded as 'HNKIJVU'.

Hence, option d.

Alphabet test(word formation using letters of a given word)

40. Which of the following word can't be formed from the letters of the word 'MULTIBRANCHED' from the given alternatives by using each letter only once?

A -DRAIN
B -LAMENT
C -MENTION
D –BRUTAL

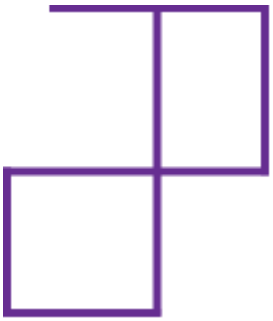
Solution

The word 'MENTION' can't be formed from the word 'MULTIBRANCHED' because there is no 'O' in the given word.

Hence, option c.

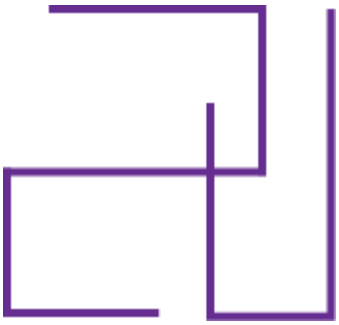
Spotting out the embedded figure

41. Select the figure in which the given figure is embedded.

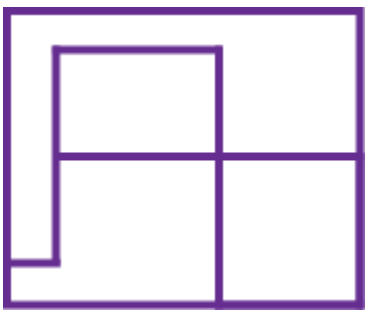


A -

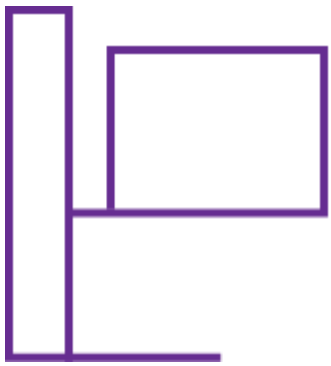
B -



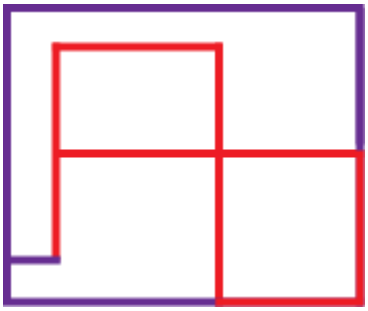
C -



D -



Solution



Hence, option c.

Alphabet test(alphabetical order of words)

42. Arrange the following words in the dictionary order:

1. College
2. Column
3. Cool
4. Character
5. Charm
6. Chair

7. Come

A -6, 4, 5, 1, 2, 7, 3
B -6, 4, 5, 2, 1, 7, 3
C -4, 6, 5, 1, 2, 7, 3
D -6, 4, 5, 1, 2, 3, 7

Solution

In the dictionary order:

6. Chair, 4. Character, 5. Charm, 1. College, 2. Column, 7. Come, 3. Cool

Hence, option a.

Floors based puzzle

(43-47) Directions: Answer the following questions based on the information given below:

Eleven persons A, B, C, D, E, F, G, H, I, J and K live on different floors of a building. Each of them like different colors among red, orange, purple, pink, yellow, magenta, green, white, grey, blue and black. Only one person lives on each floor. The bottommost floor is considered as 1st floor and the floor above it as 2nd floor and so on.

B lives three floors above the one, who likes yellow color.

H lives on bottommost floor and likes red color.

J lives on 8th floor and does not like purple or magenta color.

The one, who lives on 5th floor, likes pink color.

The one, who likes yellow color lives above the one, who likes pink color.

Equal number of persons live above and below C's floor.

F, who likes orange color, lives three floors below C's floor.

G lives just above E's floor but neither of them lives adjacent to B's floor.

K lives above I's floor but not on the topmost floor.

D likes black color but does not live adjacent to A's floor.

A likes green color and lives on an odd numbered floor.

The one, who likes blue color lives just below one, who likes black color but just above one, who likes white color.

43.Who among the following likes Purple color?

A -I
B -E
C -C
D -Cannot be determined

Solution

Now, B lives three floors above one, who likes yellow color. H lives on bottommost floor and likes red color. J lives on 8^{th} floor and does not like purple or magenta color. The one, who lives on 5^{th} floor, likes pink color. The one, who likes yellow color lives above the one, who likes pink color. Equal number of persons live above and below C's floor. F, who likes orange color, lives three floors below C's floor. So, there are three cases,

Floors	Case 1		Case 2		Case 3	
11^{th}	B					
10^{th}			B			
9^{th}					B	
8^{th}	J	Yellow	J	×purple, ×magenta	J	×purple, ×magenta
7^{th}				Yellow		

6th	C		C		C	Yellow
5th		Pink		Pink		Pink
4th						
3rd	F	Orange	F	Orange	F	Orange
2nd						
1st	H	Red	H	Red	H	Red

Now, G lives just above E's floor but neither live adjacent to B's floor. K lives above I's floor but not at the topmost floor. D likes black color but does not live adjacent to A's floor. The one, who likes blue color lives just below one, who likes black color but just above one, who likes white color. So, case 1 is rejected as B and D together can't live on the 11th floor. In case 2 and case 3, the persons, who like Black, Blue and White, live in 11th, 10th and 9th floor respectively. A likes green color and live on an odd numbered floor. So, case 2 is rejected as A cannot live on any floor.

So, the final table is shown below,

Floors	Persons	Colors
11th	D	Black
10th	K	Blue
9th	B	White
8th	J	Grey
7th	A	Green

6th	C	Yellow
5th	G	Pink
4th	E	Purple/Magenta
3rd	F	Orange
2nd	I	Magenta/Purple
1st	H	Red

Either I or E likes purple color.

Hence, option d.

44. Which color does J like?

A -Green
B -Grey
C -Pink
D –Yellow

Solution

Now, B lives three floors above one, who likes yellow color. H lives on bottommost floor and likes red color. J lives on 8th floor and does not like purple or magenta color. The one, who lives on 5th floor, likes pink color. The one, who likes yellow color lives above the one, who likes pink color. Equal number of persons live above and below C's floor. F, who likes orange color, lives three floors below C's floor. So, there are three cases,

Floors	Case 1	Case 2	Case 3

11th	B					
10th			B			
9th					B	
8th	J	Yellow	J	×purple, ×magenta	J	×purple, ×magenta
7th				Yellow		
6th	C		C		C	Yellow
5th		Pink		Pink		Pink
4th						
3rd	F	Orange	F	Orange	F	Orange
2nd						
1st	H	Red	H	Red	H	Red

Now, G lives just above E's floor but neither live adjacent to B's floor. K lives above I's floor but not at the topmost floor. D likes black color but does not live adjacent to A's floor. The one, who likes blue color lives just below one, who likes black color but just above one, who likes white color. So, case 1 is rejected as B and D together can't live on the 11th floor. In case 2 and case 3, the persons, who like Black, Blue and White, live in 11th, 10th and 9th floor respectively. A likes green color and live on an odd numbered floor. So, case 2 is rejected as A cannot live on any floor.

So, the final table is shown below,

Floors	Persons	Colors
11th	D	Black
10th	K	Blue
9th	B	White
8th	J	Grey
7th	A	Green
6th	C	Yellow
5th	G	Pink
4th	E	Purple/Magenta
3rd	F	Orange
2nd	I	Magenta/Purple
1st	H	Red

J likes Grey color.

Hence, option b.

45. How many persons live between the floors of F and A?

A -Ten
B -Three
C -Two
D –Four

Solution

Now, B lives three floors above one, who likes yellow color. H lives on bottommost floor and likes red color. J lives on 8th floor and does not like purple or magenta color. The one, who lives on 5th floor, likes pink color. The one, who likes yellow color lives above the one, who likes pink color. Equal number of persons live above and below C's floor. F, who likes orange color, lives three floors below C's floor. So, there are three cases,

Floors	Case 1		Case 2		Case 3	
11th	B					
10th			B			
9th					B	
8th	J	Yellow	J	×purple, ×magenta	J	×purple, ×magenta
7th				Yellow		
6th	C		C		C	Yellow
5th		Pink		Pink		Pink
4th						
3rd	F	Orange	F	Orange	F	Orange
2nd						
1st	H	Red	H	Red	H	Red

Now, G lives just above E's floor but neither live adjacent to B's floor. K lives above I's floor but not at the topmost floor. D likes black color but does not live adjacent to A's floor. The one, who likes blue color lives just below one, who likes black color but just above one, who likes white color. So, case 1 is rejected as B and D together can't live on the 11th floor. In case 2 and case 3, the persons, who like Black, Blue and White, live in 11th, 10th and 9th floor respectively. A likes green color and live on an odd numbered floor. So, case 2 is rejected as A cannot live on any floor.

So, the final table is shown below,

Floors	Persons	Colors
11th	D	Black
10th	K	Blue
9th	B	White
8th	J	Grey
7th	A	Green
6th	C	Yellow
5th	G	Pink
4th	E	Purple/Magenta
3rd	F	Orange
2nd	I	Magenta/Purple
1st	H	Red

Three persons live between the floors of F and A.

Hence, option b.

46. How many persons live above J's floor?

A -Three
B -Two
C -One
D –Four

Solution

Now, B lives three floors above one, who likes yellow color. H lives on bottommost floor and likes red color. J lives on 8th floor and does not like purple or magenta color. The one, who lives on 5th floor, likes pink color. The one, who likes yellow color lives above the one, who likes pink color. Equal number of persons live above and below C's floor. F, who likes orange color, lives three floors below C's floor. So, there are three cases,

Floors	Case 1		Case 2		Case 3	
11th	B					
10th			B			
9th					B	
8th	J	Yellow	J	×purple, ×magenta	J	×purple, ×magenta
7th				Yellow		
6th	C		C		C	Yellow

5th		Pink		Pink		Pink
4th						
3rd	F	Orange	F	Orange	F	Orange
2nd						
1st	H	Red	H	Red	H	Red

Now, G lives just above E's floor but neither live adjacent to B's floor. K lives above I's floor but not at the topmost floor. D likes black color but does not live adjacent to A's floor. The one, who likes blue color lives just below one, who likes black color but just above one, who likes white color. So, case 1 is rejected as B and D together can't live on the 11th floor. In case 2 and case 3, the persons, who like Black, Blue and White, live in 11th, 10th and 9th floor respectively. A likes green color and live on an odd numbered floor. So, case 2 is rejected as A cannot live on any floor.

So, the final table is shown below,

Floors	Persons	Colors
11th	D	Black
10th	K	Blue
9th	B	White
8th	J	Grey
7th	A	Green
6th	C	Yellow

5th	G	Pink
4th	E	Purple/Magenta
3rd	F	Orange
2nd	I	Magenta/Purple
1st	H	Red

Three persons live above J's floor.

Hence, option a.

47. Who lives just below D's floor?

A -K
B -B
C -G
D –E

Solution

Now, B lives three floors above one, who likes yellow color. H lives on bottommost floor and likes red color. J lives on 8th floor and does not like purple or magenta color. The one, who lives on 5th floor, likes pink color. The one, who likes yellow color lives above the one, who likes pink color. Equal number of persons live above and below C's floor. F, who likes orange color, lives three floors below C's floor. So, there are three cases,

Floors	Case 1		Case 2		Case 3	
11th	B					

10th			B			
9th					B	
8th	J	Yellow	J	×purple, ×magenta	J	×purple, ×magenta
7th				Yellow		
6th	C		C		C	Yellow
5th		Pink		Pink		Pink
4th						
3rd	F	Orange	F	Orange	F	Orange
2nd						
1st	H	Red	H	Red	H	Red

Now, G lives just above E's floor but neither live adjacent to B's floor. K lives above I's floor but not at the topmost floor. D likes black color but does not live adjacent to A's floor. The one, who likes blue color lives just below one, who likes black color but just above one, who likes white color. So, case 1 is rejected as B and D together can't live on the 11th floor. In case 2 and case 3, the persons, who like Black, Blue and White, live in 11th, 10th and 9th floor respectively. A likes green color and live on an odd numbered floor. So, case 2 is rejected as A cannot live on any floor.

So, the final table is shown below,

Floors	Persons	Colors

11th	D	Black
10th	K	Blue
9th	B	White
8th	J	Grey
7th	A	Green
6th	C	Yellow
5th	G	Pink
4th	E	Purple/Magenta
3rd	F	Orange
2nd	I	Magenta/Purple
1st	H	Red

K lives just below D's floor.

Hence, option a.

Pentagon based sitting arrangement

(48-51) Directions: Answer the questions based on the information given below:

Ten persons namely, P, Q, R, S, T, U, V, W, X and Y are sitting around the pentagon table. Two persons sit on each edge. Equal number of persons is facing towards the center and away from the centre. Not more than two adjacent persons face in same direction.

T sits third to the right of S.

W sits to the immediate right of P on the same edge.

V does not face away from the centre.

Two persons sit between S and R.

X sits to the immediate left of R but not on the same edge.

T and P face in same direction.

Y sits to the immediate right of U and both face in same direction.

Y is not adjacent to X or R.

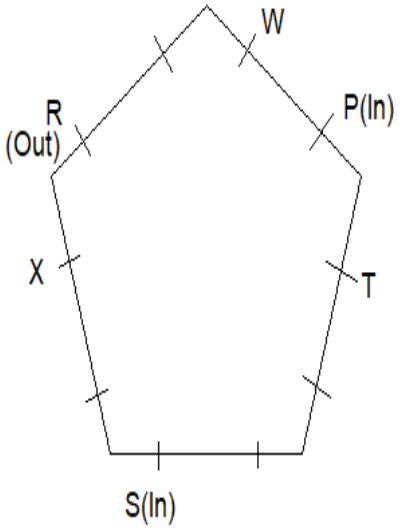
Q is not adjacent to S, who faces in same direction as V.

48.Who among the following sits to the immediate left of P?

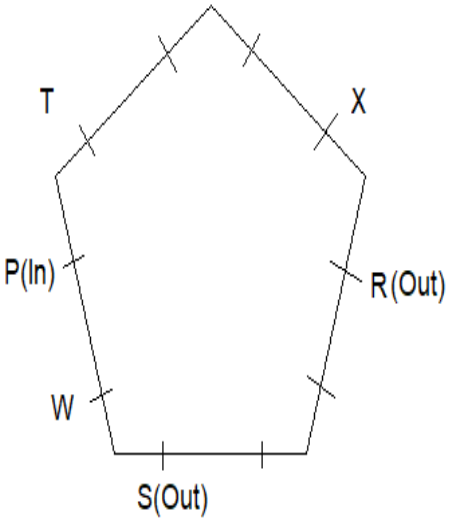
A -U
B -T
C -X
D –V

Solution

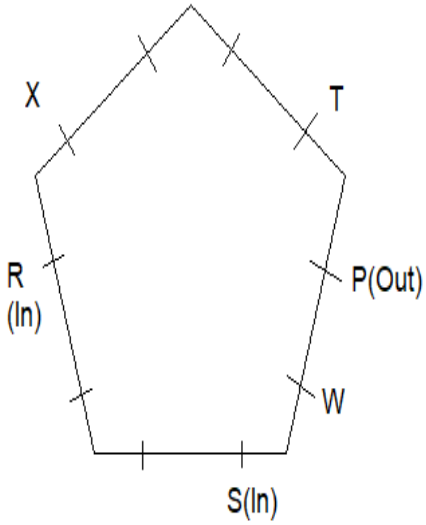
Now, T sits third to the right of S. W sits to the immediate right of P on the same edge. V does not face away from the centre. Two persons sit between S and R. X sits to the immediate left of R but not on the same edge. So, there are four cases,



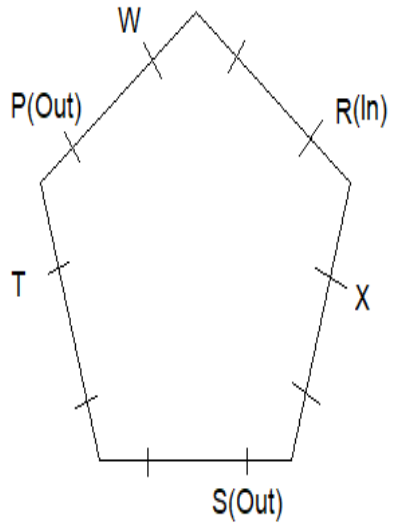
Case 1(a)



Case 1(b)



Case 2(a)

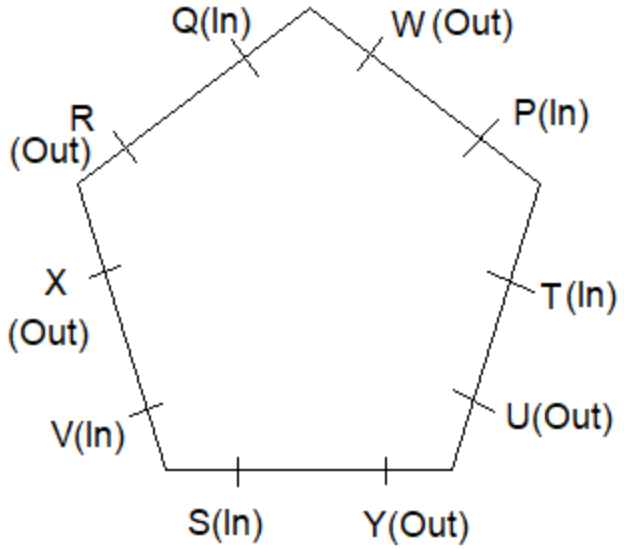


Case 2(b)

--

Now, T and P face in same direction. Y sits immediate right of U and both face in same direction. Y is not adjacent to X or R. So, case 1(b) and 2(a) are rejected. Q is not adjacent to S, who face in same direction as V. So, case 2(b) is rejected.

The final table is shown below,



T sits to the immediate left of V.

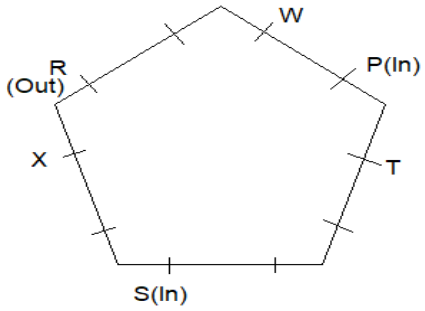
Hence, option b.

49. Who sits third to the left of R?

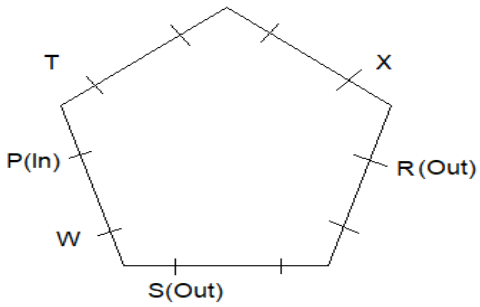
A -Y
B -U
C -S
D –P

Solution

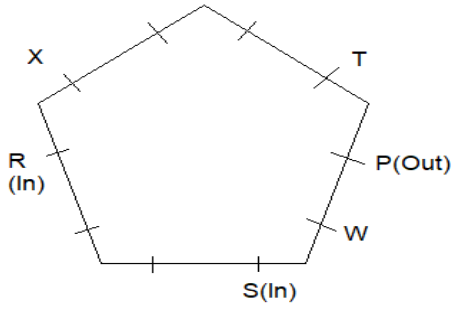
Now, T sits third to the right of S. W sits to the immediate right of P on the same edge. V does not face away from the centre. Two persons sit between S and R. X sits to the immediate left of R but not on the same edge. So, there are four cases,



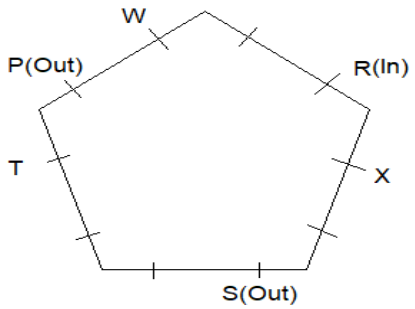
Case 1(a)



Case 1(b)



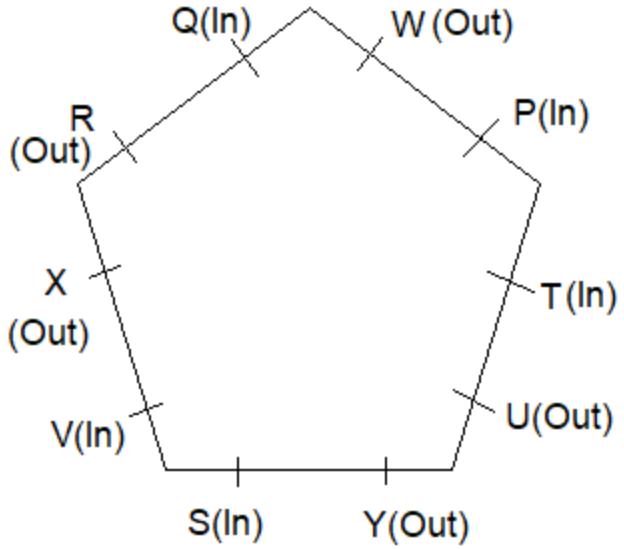
Case 2(a)



Case 2(b)

Now, T and P face in same direction. Y sits immediate right of U and both face in same direction. Y is not adjacent to X or R. So, case 1(b) and 2(a) are rejected. Q is not adjacent to S, who face in same direction as V. So, case 2(b) is rejected.

The final table is shown below,



S sits third to the left of R.

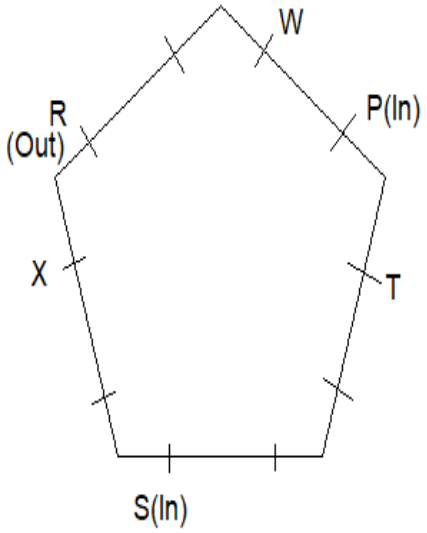
Hence, option c.

50. Find the odd one out.

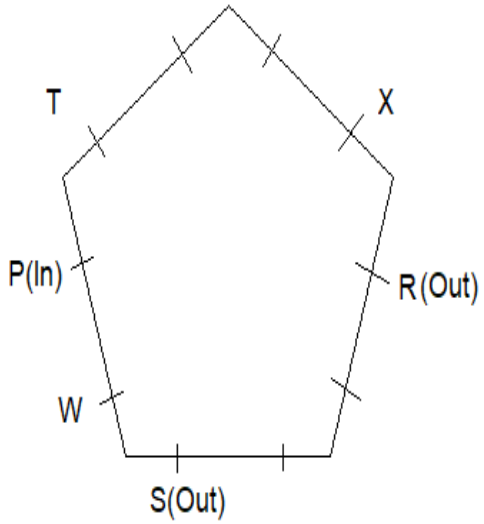
A -U, T
B -R, Q
C -X, V
D -R, P

Solution

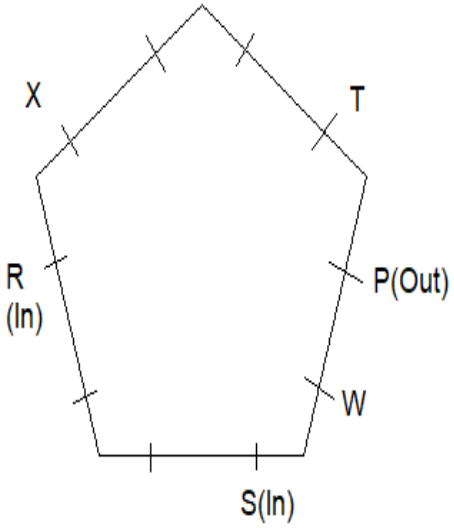
Now, T sits third to the right of S. W sits to the immediate right of P on the same edge. V does not face away from the centre. Two persons sit between S and R. X sits to the immediate left of R but not on the same edge. So, there are four cases,



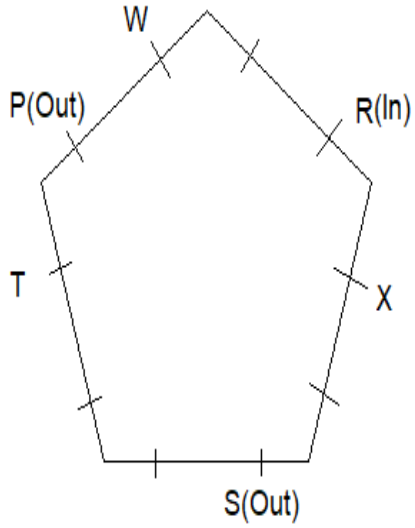
Case 1(a)



Case 1(b)



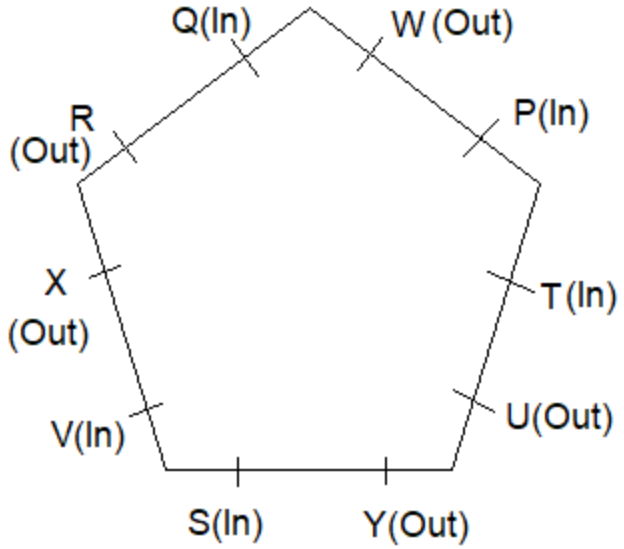
Case 2(a)



Case 2(b)

Now, T and P face in same direction. Y sits immediate right of U and both face in same direction. Y is not adjacent to X or R. So, case 1(b) and 2(a) are rejected. Q is not adjacent to S, who face in same direction as V. So, case 2(b) is rejected.

The final table is shown below,



All pairs sit on the same edge except (R, P).

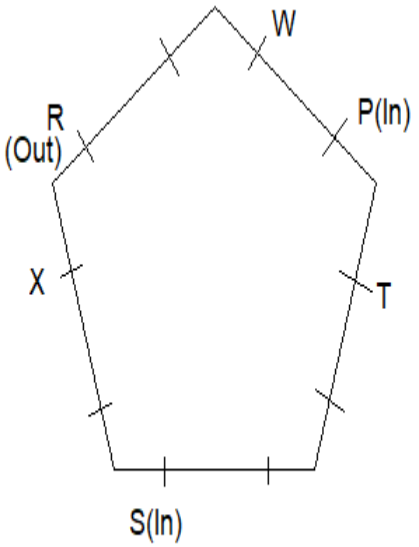
Hence, option d.

51. Who sits to the immediate right of Y?

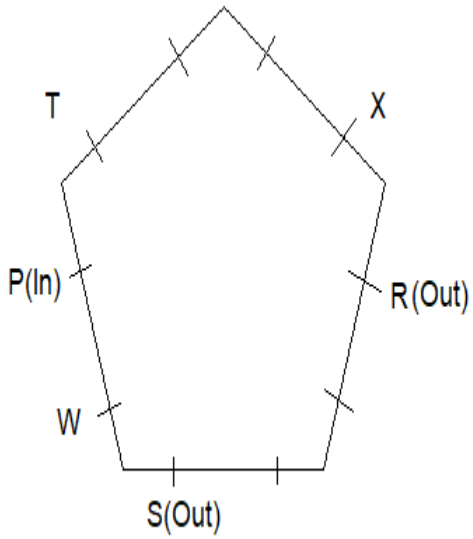
A -V
B -P
C -X
D –S

Solution

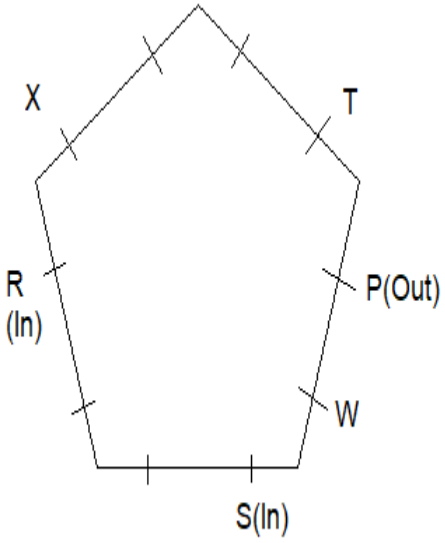
Now, T sits third to the right of S. W sits to the immediate right of P on the same edge. V does not face away from the centre. Two persons sit between S and R. X sits to the immediate left of R but not on the same edge. So, there are four cases,



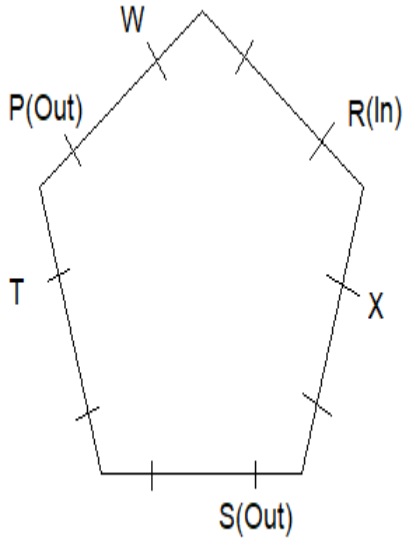
Case 1(a)



Case 1(b)



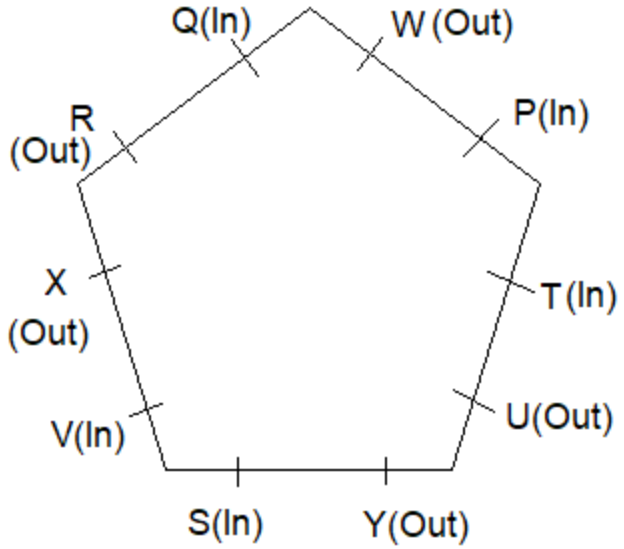
Case 2(a)



Case 2(b)

Now, T and P face in same direction. Y sits immediate right of U and both face in same direction. Y is not adjacent to X or R. So, case 1(b) and 2(a) are rejected. Q is not adjacent to S, who face in same direction as V. So, case 2(b) is rejected.

The final table is shown below,



S sits to the immediate right of Y.

Hence, option d.

Sequential order of things based puzzle

(52-54) Directions: Answer the questions based on the information given below:

Six persons P, Q, R, S, T and U goes from Delhi to Mumbai on a trip on different dates among 4th, 6th, 9th, 19th, 24th and 29th October. Each of them uses different modes among bus, bike, airplane, scooter, train and cycle.

S goes by scooter but not on 4th October.

U goes on 29th October but not by cycle or airplane.

R goes 5 days after the one, who goes by train.

Neither P nor T goes by train.

One person goes on a trip by bus on 6th October.

T does not go by bus.

S goes before T.

52.Who among the following goes by Train?

A -R
B -S
C -Q
D –U

Solution

Now, S goes only by scooter but not on 4th October. U goes on 29th October but not by cycle or airplane. R goes 5 days after the one, who goes by train. Neither P nor T goes by train. So, there are two cases,

Persons	Case 1		Case 2	
P				
Q	Train	19th	Train	4th
R		24th		9th
S	Scooter	× 4th	Scooter	× 4th
T				
U	× cycle, airplane	29th	× cycle, airplane	29th

Now, one person goes on a trip by bus on 6th October. T does not go by bus. S goes before T. So, case 1 is rejected as S does not go on 4th October.

The final table is shown below,

Persons	Modes	Dates
P	Bus	6th
Q	Train	4th
R	Cycle/Airplane	9th
S	Scooter	19th
T	Cycle/Airplane	24th
U	Bike	29th

Q goes by train.

Hence, option c.

53. How many persons go on a trip after S?

A -Three
B -One
C -Two
D –Four

Solution

Now, S goes only by scooter but not on 4th October. U goes on 29th October but not by cycle or airplane. R goes 5 days after the one, who goes by train. Neither P nor T goes by train. So, there are two cases,

Persons	Case 1		Case 2	
P				

Q	Train	19th	Train	4th
R		24th		9th
S	Scooter	× 4th	Scooter	× 4th
T				
U	× cycle, airplane	29th	× cycle, airplane	29th

Now, one person goes on a trip by bus on 6th October. T does not go by bus. S goes before T. So, case 1 is rejected as S does not go on 4th October.

The final table is shown below,

Persons	Modes	Dates
P	Bus	6th
Q	Train	4th
R	Cycle/Airplane	9th
S	Scooter	19th
T	Cycle/Airplane	24th
U	Bike	29th

Two persons go on a trip after S.

Hence, option c.

54. One which date does Q go on a trip?

A -9th
B -24th
C -6th
D -4th

Solution

Now, S goes only by scooter but not on 4th October. U goes on 29th October but not by cycle or airplane. R goes 5 days after the one, who goes by train. Neither P nor T goes by train. So, there are two cases,

Persons	Case 1		Case 2	
P				
Q	Train	19th	Train	4th
R		24th		9th
S	Scooter	× 4th	Scooter	× 4th
T				
U	× cycle, airplane	29th	× cycle, airplane	29th

Now, one person goes on a trip by bus on 6th October. T does not go by bus. S goes before T. So, case 1 is rejected as S does not go on 4th October.

The final table is shown below,

Persons	Modes	Dates
P	Bus	6th
Q	Train	4th

R	Cycle/Airplane	9th
S	Scooter	19th
T	Cycle/Airplane	24th
U	Bike	29th

Q goes on a trip on 4th October.

Hence, option d.

Numerical ability

Caselet on percentages,ratios&proportions

(55-58) Directions: Answer the questions based on the information given below.

Four jewellers namely Mahesh, Suresh, Ramesh and Mukesh sold different number of gold rings and silver rings. Total number of rings (gold and silver) sold by Mahesh is 250 which is (200/3)% more than total number of rings sold by Mukesh. Ratio of number of gold rings to silver rings sold by Mahesh is 2:3. Number of gold rings and silver rings sold by Ramesh is 80% more and 20% less than the same sold by Mahesh. Ratio of the number of gold rings sold by Suresh and Ramesh is 25:36, respectively and the ratio of number of gold and silver rings sold by Suresh is 5:3, respectively. Number of gold rings sold by Mukesh is 15 more than the number of silver rings sold by Suresh.

55.What is the total number of rings sold by all the four jewellers together?

A -800
B -900
C -880
D -920

Solution

Total number of rings sold by Mukesh = 250/(1 + 2/3) = 150

Number of gold rings sold by Mahesh = 2/5 × 250 = 100

Number of Silver rings sold by Mahesh = 250 – 100 = 150

Number of gold rings sold by Ramesh = 1.8 × 100 = 180

Number of silver rings sold by Ramesh = 0.8 × 150 = 120

Number of gold rings sold by Suresh = 25/36 × 180 = 125

Number of Silver rings sold by Suresh = 3/5 × 125 = 75

Number of gold rings sold by Mukesh = 15 + 75 = 90

Number of silver rings sold by Mukesh = 150 – 90 = 60

Jewellers	Number of gold rings sold	Number of silver rings sold	Total number of rings sold
Mahesh	100	150	250
Suresh	125	75	200
Ramesh	180	120	300
Mukesh	90	60	150
Total	495	405	900

Total number of rings sold by all the four jewellers together = 900

Hence, option b.

56. Number of gold rings sold by Mukesh is what percent of total number of rings sold by all four together?

A -10%
B -12%
C -15%
D -20%

Solution

Total number of rings sold by Mukesh = 250/(1 + 2/3) = 150

Number of gold rings sold by Mahesh = 2/5 × 250 = 100

Number of Silver rings sold by Mahesh = 250 – 100 = 150

Number of gold rings sold by Ramesh = 1.8 × 100 = 180

Number of silver rings sold by Ramesh = 0.8 × 150 = 120

Number of gold rings sold by Suresh = 25/36 × 180 = 125

Number of Silver rings sold by Suresh = 3/5 × 125 = 75

Number of gold rings sold by Mukesh = 15 + 75 = 90

Number of silver rings sold by Mukesh = 150 – 90 = 60

Jewellers	Number of gold rings sold	Number of silver rings sold	Total number of rings sold
Mahesh	100	150	250
Suresh	125	75	200
Ramesh	180	120	300
Mukesh	90	60	150
Total	495	405	900

Desired Percentage = 90/900 × 100 = 10%

Hence, option a.

57. What is the total number of rings sold by Suresh?

A -300
B -225
C -250
D -200

Solution

Total number of rings sold by Mukesh = 250/(1 + 2/3) = 150

Number of gold rings sold by Mahesh = 2/5 × 250 = 100

Number of Silver rings sold by Mahesh = 250 – 100 = 150

Number of gold rings sold by Ramesh = 1.8 × 100 = 180

Number of silver rings sold by Ramesh = 0.8 × 150 = 120

Number of gold rings sold by Suresh = 25/36 × 180 = 125

Number of Silver rings sold by Suresh = 3/5 × 125 = 75

Number of gold rings sold by Mukesh = 15 + 75 = 90

Number of silver rings sold by Mukesh = 150 – 90 = 60

Jewellers	Number of gold rings sold	Number of silver rings sold	Total number of rings sold
Mahesh	100	150	250
Suresh	125	75	200
Ramesh	180	120	300

Mukesh	90	60	150
Total	495	405	900

Total number of rings sold by Suresh = 200

Hence, option d.

58. What is the average number of gold rings sold by Mahesh and Ramesh together?

A -150
B -120
C -140
D -130

Solution

Total number of rings sold by Mukesh = 250/(1 + 2/3) = 150

Number of gold rings sold by Mahesh = 2/5 × 250 = 100

Number of Silver rings sold by Mahesh = 250 – 100 = 150

Number of gold rings sold by Ramesh = 1.8 × 100 = 180

Number of silver rings sold by Ramesh = 0.8 × 150 = 120

Number of gold rings sold by Suresh = 25/36 × 180 = 125

Number of Silver rings sold by Suresh = 3/5 × 125 = 75

Number of gold rings sold by Mukesh = 15 + 75 = 90

Number of silver rings sold by Mukesh = 150 – 90 = 60

Jewellers	Number of gold rings sold	Number of silver rings sold	Total number of rings sold

Mahesh	100	150	250
Suresh	125	75	200
Ramesh	180	120	300
Mukesh	90	60	150
Total	495	405	900

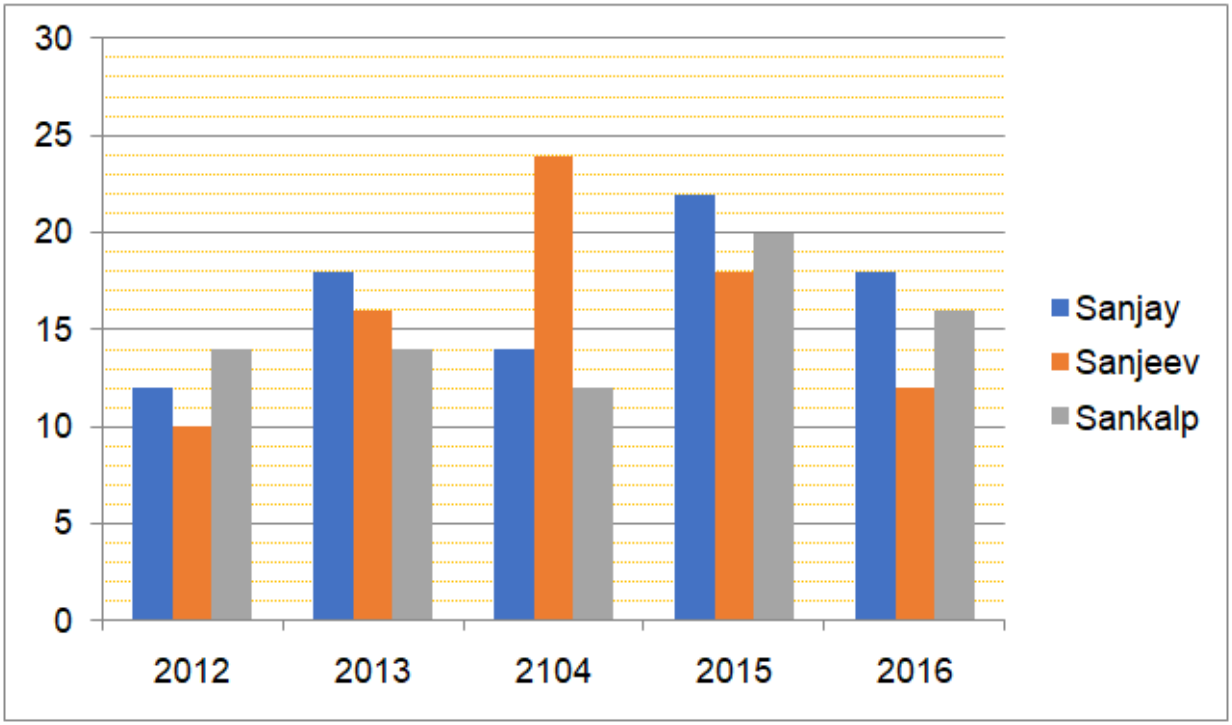
Desires average = (100 + 180)/2 = 140

Hence, option c.

Data interpretation(Bar graphs on absolute values)

(59-60) Directions: Answer the questions based on the information given below.

The bar graph given below shows the monthly income of 3 different persons (in thousands) over five different years.



59.In 2012 and 2013 together, what is the average monthly income of Sanjay, Sanjeev and Sankalp?

A -Rs. 28000
B -Rs. 24000
C -Rs. 26000
D -Rs. 29000

Solution

Average monthly income of Sanjay, Sanjeev and Sankalp in 2012 and 2013 together = (12 + 18 + 10 + 16 + 14 + 14)/3 = 84/3 = 28 × 1000 = RS. 28000

Hence, option a.

60. **Sanjeev saved 23% and 28% of his monthly income in 2014 and 2015, respectively. Find his total savings over these two years?**

A -Rs. 11400
B -Rs. 10200

C -Rs. 10560
D -Rs. 12800

Solution

Total savings of Sanjeev over these two years = 0.23 × 24000 + 0.28 × 18000= 5520 + 5040 = Rs. 10560

Hence, option c.

Time&work

61. 'A' takes 4 days less than 'B' who can complete 1/6th of the work in 4 days. Both of them started the work together and worked for 6 days. What fraction of work is left?

A -6/25
B -7/15
C -9/20
D -2/15

Solution

Time taken by 'B' to complete the work = 4 × 6 = 24 days

Therefore, time taken by 'A' to complete the work = 24 – 4 = 20 days

Let the total work = 120 units

Efficiency of 'A' = 120/20 = 6 units/day

Efficiency of 'B' = 120/24 = 5 units/day

Amount of work completed by them in 6 days = 6 × (5 + 6) = 66 units

Therefore, fraction of work left = 1 – (66/120) = 9/20

Hence, option c.

Ratios&proportions

62. In an interview, the ratio of the number of qualified to disqualified candidates is 12:5. If 40 less candidates had applied and 10 more qualified, then the ratio of qualified to disqualified candidates would have been 5:1. Find the number of candidates who were interviewed.

A -510
B -300
C -340
D -420

Solution

Let the number of candidates qualified and disqualified be 12x and 5x, respectively.

Therefore, total number of candidates interviewed = (12x + 5x) = 17x

According to the question,

Total number of candidates interviewed = (17x – 40)

Number of qualified candidates = (12x + 10)

Number of disqualified candidates = 17x – 40 – 12x – 10 = (5x – 50)

According to the question,

(12x + 10)/(5x – 50) = 5/1

Or, 12x + 10 = 25x – 250

Or, x = 260/13 = 20

Therefore, total number of candidates interviewed = 17x = 340

Hence, option c.

Profit&loss

63. An electric shop owner purchased a fan and a cooler. He sold the cooler at 15% loss and fan at 25% profit. If the shop owner had purchased the fan for Rs. 512 less than the cooler, and the selling price of fan and cooler is same, then find the selling price of the fan.

A -Rs. 1360
B -Rs. 1520
C -Rs. 1640
D -Rs. 1460

Solution

Let the cost price of the cooler be Rs. x

Therefore, selling price of the cooler = Rs. 0.85x

Cost price of the fan = 0.85x/1.25 = Rs. 0.68x

According to the question,

(x – 0.68x) = 512

Or, x = 512/0.32 = Rs. 1600

Therefore, selling price of the fan = 0.85x = Rs. 1360

Hence, option a.

Problems on numbers

64. **Two numbers 'a' and 'b' are 10% and 20% less than the third number 'c'. By how much percent 'b' should be increased to make it equal to 'a'.**

A -20%
B -12.5%
C -15%
D -10%

Solution

Let ‘c’ = ‘x’

Therefore, a = 0.9x

And, b = 0.8x

Required percentage = {(0.9x – 0.8x)/0.8x} × 100 = 12.5%

Hence, option b.

Time&distance

65. Two cars ‘A’ and ‘B’ are travelling from one city to another. The speed of car ‘A’ and ‘B’ is 40 km/hr and 75 km/hr respectively. If car ‘A’ takes 1 hour and 45 minutes more than ‘B’ to reach the destination, then find the distance between the two cities.

A -240 km
B -150 km
C -120 km
D -180 km

Solution

1 hours 45 minutes = 1.75 hours

Let the distance between the two cities be ‘D’ km

According to the question,

(D/40) – (D/75) = 1.75

Or, (15D – 8D) = 1.75 × 600

Or, D = 1050/7 = 150 km

Hence, option b.

Compound interest

66. At what rate of compound interest will be a sum of Rs. 16150 will amounts to Rs. 31654 in 2 years.

A -20% p.a.
B -30% p.a.
C -25% p.a.
D -40% p.a.

Solution

Let the rate of interest be r% p.a.

According to the question,

$16150(1 + r/100)^2 = 31654$

Or, $(1 + r/100)^2 = 49/25$

Or, $1 + r/100 = 7/5$

Or, r = 40% p.a.

Hence, option d.

averages

67. If the average cost of 8 shirts is Rs. 625, then find the new average cost if a shirt whose cost is Rs. 1300 is also purchased.

A -Rs. 690
B -Rs. 630
C -Rs. 720
D -Rs. 700

Solution

Required average cost = {(8 × 625) + 1300}/9 = Rs. 700

Hence, option d.

Time&work

68. ‘A’ can complete a piece of work in 30 days while ‘A’ and ‘B’ together can complete a work in 20 days. In how many days ‘B’ alone can complete the work?

A -60 days
B -40 days
C -45 days
D -50 days

Solution

Let the total work = 120 units

Efficiency of (A + B) = 120/20 = 6 units/day

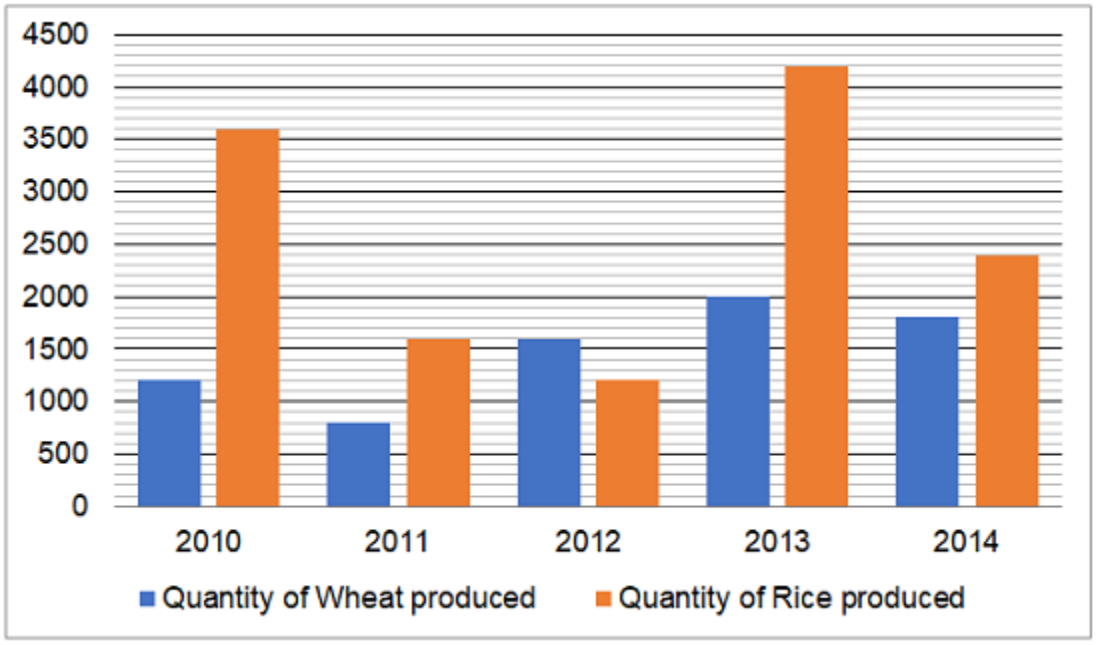
Efficiency of ‘A’ = 120/30 = 4 units/day

Therefore, number of days taken by ‘B’ alone = 120/(6 – 4) = 60 days

Hence, option a.

Bar graphs on absolute values

69. The given bar shows the quantity of wheat and rice produced (in metric tonnes) in five different years.



Find the average quantity of wheat (in metric tonnes) produced in all the five years.

A -1320 metric tonnes
B -1480 metric tonnes
C -1140 metric tonnes
D -1560 metric tonnes

Solution

Required average = (1200 + 800 + 1600 + 2000 + 1800)/5 = 1480 metric tonnes

Hence, option b.

Trigonometry

70. Find the simplified value of √{(1 – sinA)/(1 + sinA)}.

A -cotA - cosA
B -secA - tanA
C -1
D -sinA – tanA

Solution

Given, √(1 – sinA)/(1 + sinA)

= √{(1 – sinA)(1 – sinA)/(1 + sinA)((1 – sinA)

Or, (1 – sinA)/cosA = secA – tanA

Hence, option b.

Problems on numbers

71. 40% of a number when increased by 50, the resultant becomes 20% less than the original number. Find the original number.

A -125
B -140
C -175
D -180

Solution

Let the number be ‘x’

According to the question,

0.40x + 50 = 0.8x

Or, x = 50/0.4

Or, x = 125

Hence, option a.

Mixtures&allegations

72. 25% of a mixture of acetone and water contains 12 litres of water. The quantity of acetone is 12 litres less than that of water in the total mixture. Find the total quantity of the mixture.

A -72 litres
B -84 litres
C -65 litres
D -96 litres

Solution

Quantity of water in the total mixture = 12/0.25 = 48 litres

Quantity of acetone in the total mixture = 48 – 12 = 36 litres

Total quantity of the mixture = 48 + 36 = 84 litres

Hence, option b.

Discounts

73. An article is sold at Rs. 756 after giving two successive discounts of 16% and 25% respectively. If the article is marked up by Rs. 200 above its cost price, then find the cost price of the article.

A -Rs. 1200
B -Rs. 1500
C -Rs. 1000
D -Rs. 800

Solution

Marked price of the article = 756/(0.84 × 0.75) = Rs. 1200

Therefore, cost price of the article = 1200 – 200 = Rs. 1000

Hence, option c.

Problems on ages

74. The ratio of the present ages of Ram and Shyam is 9:5, respectively. The ratio of the age of Ram 3 years ago to the age of Shyam 3 years hence is 3:2. Find the present age of Ram.

A -45 years
B -30 years
C -24 years
D -36 years

Solution

Let the present age of Ram and Shyam be 9x years and 5x years respectively

According to the question,

$(9x – 3)/(5x + 3) = 3/2$

Or, $18x – 6 = 15x + 9$

Or, $x = 15/3 = 5$ years

Therefore, present age of Ram = $9x = 45$ years

Hence, option a.

Simple interest-compound interest

75. A certain sum when invested at 30% p.a. simple interest for 5 years the interest received is Rs. 2100. Find the interest received when the same sum is invested at same rate at compound interest, compounded annually for 2 years.

A -Rs. 966
B -Rs. 814
C -Rs. 720
D -Rs. 680

Solution

Let the sum invested at simple interest be Rs. x

Therefore, (x × 30 × 5)/100 = 2100

Or, x = Rs. 1400

Therefore, interest received at compound interest = $1400(1 + 30/100)^2 – 1400$ = Rs. 966

Hence, option a.

Problems on numbers

76. When a two-digit number is increased by 27, the digits get reversed. The sum of the digits is 9. Find the original number.

A -28
B -42
C -36
D -45

Solution

Let the unit and ten's digit be 'x' and 'y' respectively

Two-digit number = (10y + x)

According to the question,

10y + x – 10x – y = -27

Or, y – x = -3…… (1)

And, x + y = 9……. (2)

On solving equation (1) and (2), we get

x = 6 and y = 3

Therefore, required number = 10y + x = 36

Hence, option c.

Problems on numbers

77. 45% of a number is 3 more than 50% of another number. The sum of the two numbers is 260. Find the larger number.

A -180
B -120
C -140
D -160

Solution

Let the numbers be ‘a’ and ‘b’ respectively

According to the question,

0.45a – 0.5b = 3….. (1)

And, a + b = 260…… (2)

On solving equation (1) and (2), we get

a = 140 and b = 120

So, larger number = 140

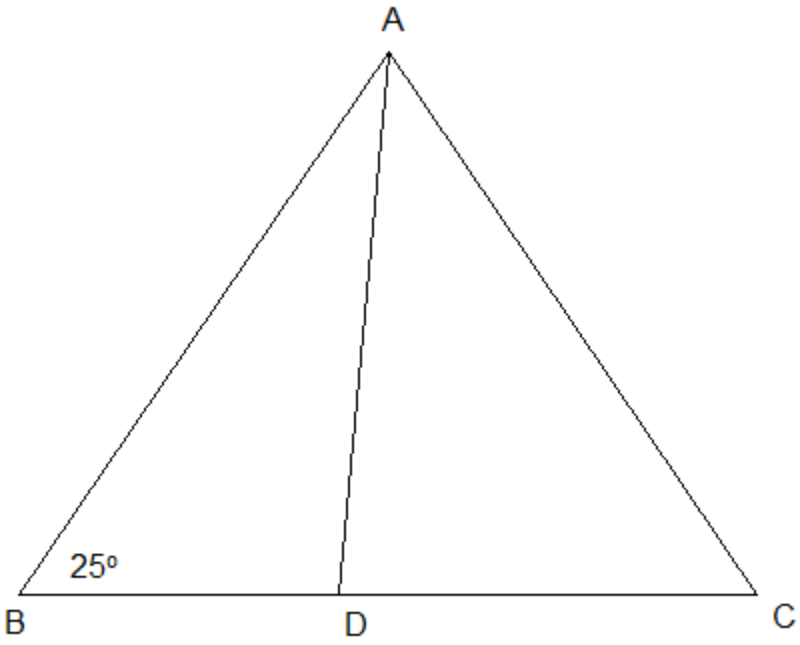
Hence, option c.

Geometry(Triangles)

78. In a ΔABC, D is a point on BC. If ∠ABD = 25° and AD = BD, then find the value of ∠ADB.

A -120°
B -150°
C -130°
D -100°

Solution



In ΔABD,

AD = BD

∠ABD = ∠DAB = 25°

Therefore, ∠ADB = 180° – (25° + 25°) = 130°

Hence, option c.

HCF&LCM of numbers

79. If $(15x^3 + 28x^2 + 24x + k)$ is divisible $(3x + 2)$, then find the value of k.

A -6
B -8
C -10
D -5

Solution

Since, $(15x^3 + 28x^2 + 24x + k)$ is divisible by $(3x + 2)$

Therefore, at $x = (-2/3)$, the value of the given expression will be 0.

Or, $(-2/3)\{15 \times (4/9) + 28 \times (-2/3) + 24\} + k = 0$

Or, $-8 + k = 0$

Or, $k = 8$

Hence, option b.

Trigonometry

80. If sinx = 45/53, then find the value of secx + tanx.

A -7/2
B -5/2
C -8/5
D -None of these

Solution

Given, $\sin x = 45/53 = p/h$

Therefore, $b = \sqrt{(53^2 - 45^2)} = 28$

Or, $\sec x + \tan x = 53/28 + 45/28 = 7/2$

Hence, option a.

FRESHERS NOW

You Can also check:

★★ **Free TCS NQT Mock Tests** ★★

★★ **Free TCS Digital Mock Tests** ★★

★★ **TCS NQT Recruitment** ★★

★ **You Can Also Check** ★

TCS Campus Commune Registration	Reasoning Questions & Answers
TCS NextStep	TCS Off Campus
TCS Syllabus	TCS Recruitment
TCS Future Forward Careers NQT	TCS NQT
TCS Jobs For BCA, B.Sc Freshers Off Campus Drive	TCS CBO Test Results
TCS CBO Careers	TCS NQT Exam Result
TCS NQT Syllabus	TCS Placement Papers
TCS NQT Hall Ticket	TCS CodeVita
TCS HumAin Contest	TCS Verbal Ability Questions

FRESHERS NOW

TCS Aptitude Questions	TCS Coding Programming Questions
TCS Hiring Challenge	TCS Alumni
TCS Webmail Login	TCS Branches In India
TCS E-Mail Writing	TCS Ignite Open Challenge
TCS Webmail Login	TCS Application Form
TCS Zimbra Mail	TCS iON Self Service Learning Hub
TCS Internship	TCS iBegin Login Registration Link
Verbal Ability Questions & Answers	Aptitude Questions & Answers

"PYTHON"

I want to speak with the computer
Language = Python
- Guido van Rossum
- 1991.

Syntax :-

```
import pandas as pd.
```

Programming language application
Gaming, Banking, Machine learning

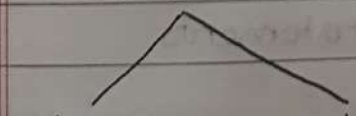
Basic variables.

Data → collection of the facts.
Ex: My name is poorna.

How to store the data
"Poorna" → String
56 → Integer
True/False → Boolean

variables.
Data/values can be stored in temporary storage spaces called variables.

Student = "Poorna"



Name associated & address associated
Student → Name of the variable.

10 = a
20 = b
a+b = 30
a-b = -10
a*b = 200
a/b =
multiple operations on variables

3) Decision Making statements.

if	else
it's raining :	go out and play
sit inside	

if	else
marks > 70 :	give practice test
Get ice-cream	

if --- else pseudo code

```
if(condition){
statements to be executed
}

else{
statements to be executed
}
```

If the "if' condition is true
at that time it will execute the statements inside if
else if it is false
It will execute the statements inside else

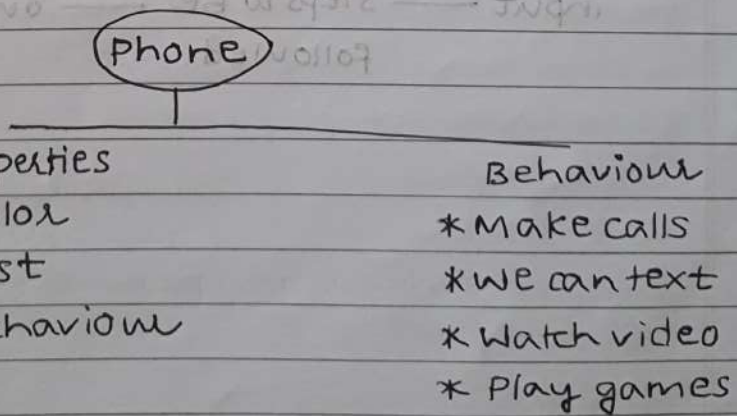
4) looping statements.
are used to repeat a task multiple times.
while, Repeat, each

while loop pseudo code

```
while(TRUE){
keep executing statements
}
```

5) Functions in programming
Function is a block of code which performs a specific task.

6) OOP concepts.
I am surrounded with the objects
Mobile, laptop, bag, bike, dog, cat
object have
→ Property
→ Behaviour
class is a template for real world entities.



classmate
Date
Page

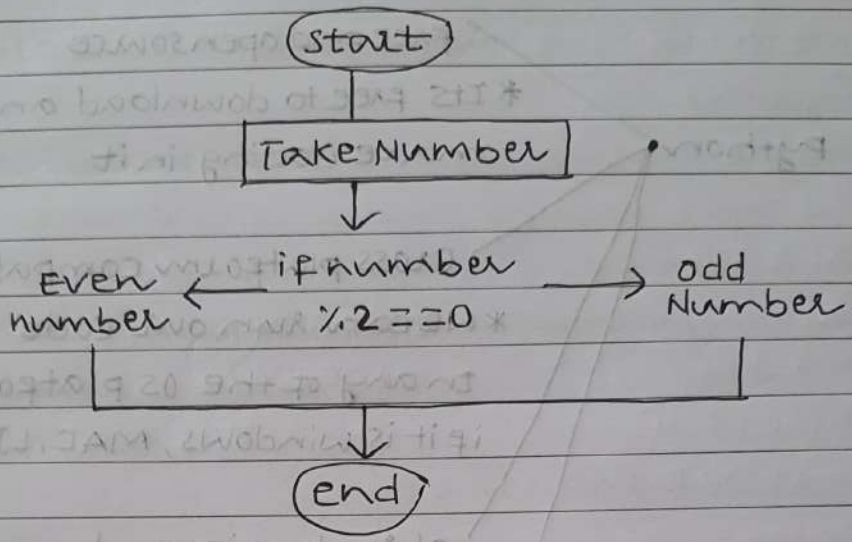
object
It has a specific template
* objects are specific instances of a class

Phone would be the class
specific instances would be the objects
Brands of phone Apple
samsung
Nokia
google pixel
Tesla
Apple is the brand of phone which is object
o/p b/w objects
All having
Different values for properties
Different values for Behaviour

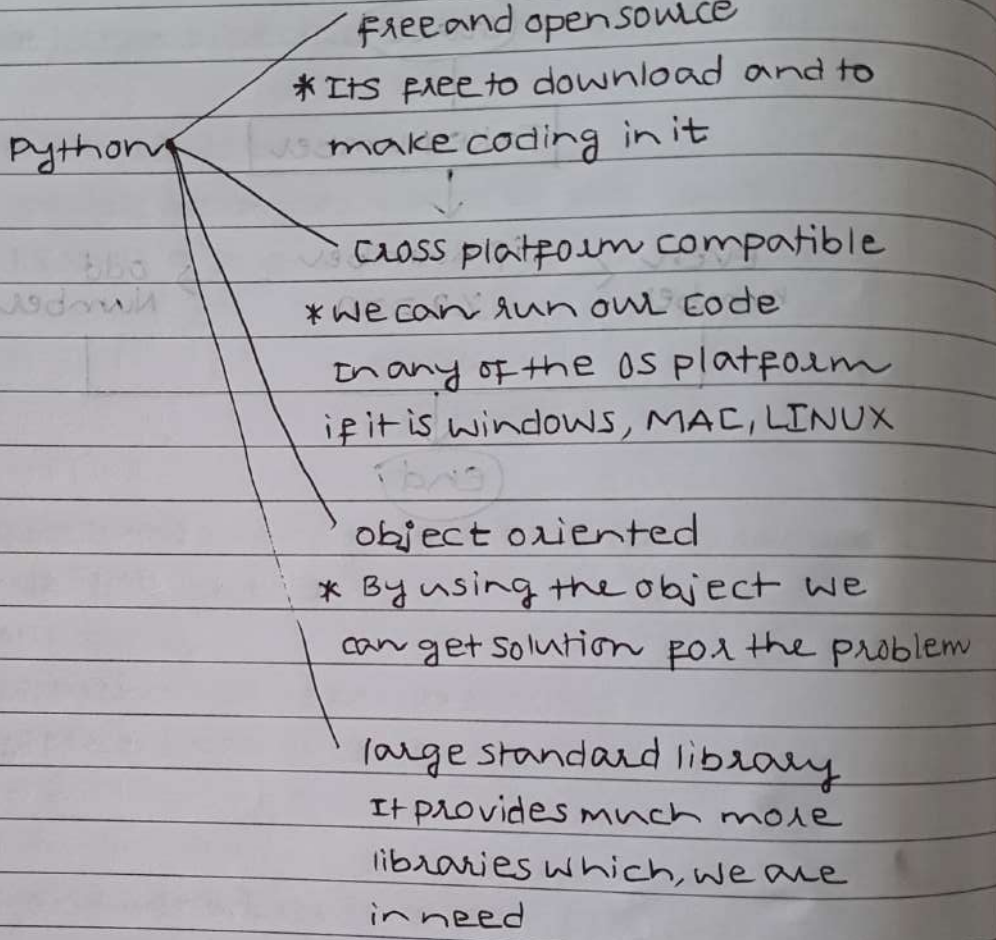
6) Algorithmic approach to solve problem:
step by step approach
to solve a problem is known as Algorithm.

input —— steps to be followed —— output

Algorithm to find if number even/odd.



7 Introduction to python.



IDE - Integrated Development Environment
Python, Pycharm, Anaconda
↓ webpage Distribution

8) Intro to Jupyter notebook.

kernel executes the program.

Print("This is poorna")
This is poorna

9) Variables and Data-types in python.

variables.

a = "Poorna"

a

'a' is a variable

→ Poorna

a = "chandru"

a

→ chandru

a = "Gowda"

a

→ Gowda

Data types in python

every variable is associated with the data

1) Integers (int) Integer values

1, 2, 3, 4, 5, 100, 1000, 999

2) Float →

6.19, 3.14, 9.32, 8.674

3) Boolean →

True, False

4) String (" " mention it inside double quote)

"Poorna" 'chandru' 'Earth'

* a1 = 100
a1
→ 100
type(a1)
→ int

* a1 = 3.14
a1
→ 3.14
type(3.14)
→ Float

a1 = 3.14
type(a1)
→ Float

* a1 = True
a1
→ True
type(a1)
→ Bool

* a1 = "Poorna"
a1
→ 'Poorna'
type(a1)
→ Str

* Complex number
Both Real and Imaginary part
a1 = 6+9j
a1
→ 6+9j
type(a1)
→ Complex

classmate
Date
Page

(10) operators in python:

Arithmetic operator
Relational operator
logical operator
We can perform various types of operations by using above mentioned operators.

Arithmetic operator
a=10
b=20
Arithmetic or mathematic operations
(+, −, *, /)
a+b ⟶ 30 (10+20)
a−b ⟶ −10 (10−20)
b−a ⟶ 10 (20−10)
a*b ⟶ 200 (10*20)
a/b ⟶ 0.5 (10/20) ⟶ we should use forward slash

Relational operators
(>, <, ==, !=)
a=10
b=20
a>b ⟶ False, b>a ⟶ True (20>10)

a<b ⟶ True (10<20)

a==b ⟶ False (10 is not equal to 20)

a!=b ⟶ True

a=100
b=100
a==b → True (100==100, a==b)

Logical operators.
→ logical operator "and", "or" and "not"

→ Bitwise operator "&", "|"

& And operator
a= True (1)
b=False (0)
a&b → False
b&a → False
b&b → False
a&a → True

w.r.t the And operator we get a true value only when both the operands are true

| or operator
It will give use the true result when either of the operands is true or both of the operands are true

we will get a False result w.r.t the or operator only when both the operands are false
a= True (1)
b= False (0)
a|b → True (either one value having true)
b|a → True (a is true)
a|a → True (both the operands are true
b|b → False

(11) Tokens in python

smallest meaningful component in a program

keywords
Identifiers
Literals
operators

(1) keywords

→ are special reserved words

It will give meaningful information for the compiler or interpreter.

False	class	Finally	is	Return
None	continue	For	lambda	Try
True	def	From	Nonlocal	While
and	del	Global	Not	With
as	elif	If	or	yield

(2) Identifiers.

are names used for variables, functions or objects

Rules →

No special character except _ (identifiers)
Identifiers are case sensitive
First letter cannot be a digit

Poorna is his Identifier
Poorna is being Identified by his name

(3) Python literals

→ constants in python (Do not change)

a1 = "poorna" ↳ It is a string literal

↳ a1 is a variable

a1 = 123 → Numeric literal

a1 = True → Boolean literal

(12) strings in python

→ are sequence of characters enclosed within single quotes (' '), double quotes (" ") or triple quotes (''' ''')

'Hello World'

" This is poorna"

''' I am going to kerala tomorrow '''

b1 = 'Hello world'

b1 → 'Hello world'

b1 = " This is poorna"

b1 → ' This is poorna'

b1 = ''' This is a multiline
string
'''

b1 → ' this is a multiline\nstring\n'

Extracting the Individual characters.

Indexing in python starts with 0, not 1.

```
                    0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
My_string = "My_name_is_poorna
                   -17-16-15-14-13-12-11-10-9-8-7-6-5-4-3-2-1
```

My_string[0] → 'M'

My_string[-1] → 'a'

My_string[4] → 'n'

I want to extract name

So the indexing is done like this

n starts with 3

ends with e → 6

— → 7

The first element would be inclusive (n)

The last element would be exclusive (-)

My_string[3:7] → 'name'

* Now I want poorna

My_string[11:17] → 'poorna'

String Functions:

len(My_string) → 17

My_string.upper() → 'MY NAME IS POORNA'

My_string.lower() → 'my name is poorna'

* Replacing a substring

```
my_string.replace("y","a")
→'Ma name is poorna'
```

* Number of occurrences of substring

```
new_string = "hello hello hello world"
new_string.count("hello")
→ 3
```

* Finding the Index of substring

```
s1 = "This is poorna"
s1.find("poorna")
→ 8
```

* Splitting a string

```
fruit = "I like apples, Mangoes, banana, grapes,
         cherries"
fruit.split(", ")
→['I like apples', 'Mangoes', 'banana', 'grapes', 'cherries']
```

(13) Tuples in python.

Data-structures in python

- Tuple
- List
- Dictionary
- set

Tuple

is an ordered collection of elements enclosed within ()

* Tuples are Immutable
  once you create the tuple, you cant change the value inside the tuple
* Heterogenous mixture of different elements.

#Tuple

```
tup1 = (100,"b", True, "c", False)
tup1 ⟶ (100, 'b', True, 'c', False)
type(tup1) ⟶ tuple.
```

Extract Individual elements

```
tup1[0] ⟶ 100
tup1[1] ⟶ 'b'
tup1[-1] ⟶ False
tup1[1:3] ⟶ 'b', True
tup1[2] == "hello"
```

We will get error "tuple" object does not support item assignment.

* Finding length of Tuple

```
tup1 = (100, "b", True, "c", False)
len(tup1) → 5
```

* concatenating tuples

```
tup1 = (1,2,3)
tup2 = (4,5,6)
tup1 + tup2
→ (1,2,3,4,5,6)
```

* Repeating Tuple elements

```
tup1 = ("Poorna", 300)
tup1*3
→ ('Poorna', 300, 'Poorna', 300, 'Poorna', 300)
```

* Reapeating and concatenating

```
tup1 = ("Poorna", 300)
tup2 = (4,5,6)
tup1*3 + tup2
→('Poorna',300,'Poorna',300,'Poorna',300,4,5,6)
```

* Tuple Functions.

minimum value

```
tup1 = (1,2,3,4,5)
min(tup1)
→ 1
```

Maximum value

```
tup1 = (1,2,3,4,5)
max(tup1)
→ 5
```

(14) List in python.

list is an ordered collection of elements enclosed within [ ]

* lists are mutable
  we can add, subtract, multiply values inside the list.

```
LI = [1, "Poorna", 3.14, True, 5+9j]
type(LI) ⟶ list
```

* Extracting individual elements

```
LI[-1] ⟶ 5+9j
LI[1:4] ⟶ ['Poorna', 3.14, True]
```

* Modifying a list

  changing the element of 0th index

```
LI = [1, "a", 2, "b", 3, "c"]
LI[0] ⟶ 100
LI ⟶
  [100, 'a', 2, 'b', 3, 'c']
```

  Appending a new element

```
LI = [1, "a", 2, "b", 3, "c"]
LI.append("poorna")
LI
⟶ [1, 'a', 2, 'b', 3, 'c', 'Poorna']
```

  Popping the last element

```
LI = [1, "a", 2, "b", 3, "c"]
LI.pop()
LI
⟶ [1, 'a', 2, 'b', 3]
```

* Reversing elements of a list

```
L1 = [1, "a", 2, "b", 3, "c"]
L1.reverse()
L1
→ ['c', 3, 'b', 2, 'a', 1]
```

* Inserting element at a specified index

```
L1 = [1, "a", 2, "b", 3, "c"]
L1.insert(1, "poorna")
→ [1, 'poorna', 'a', 2, 'b', 3, 'c']
```

* Sorting a list

```
L1 = ["Mango", "cherry", "banana", "guava"]
L1.sort()
→ ['banana', 'cherry', 'guava', 'Mango']
```

* Concatenating lists

```
L1 = [1, 2, 3]
L2 = ["a", "b", "c"]
L1+L2
→ [1, 2, 3, 'a', 'b', 'c']
```

* Repeating elements

```
L1 = [1, "a", True]
L1*3
→ [1, 'a', True, 1, 'a', True, 1, 'a', True]
```

(15) Dictionary in python:

is an unordered collection of key-value pairs enclosed with {}

* Dictionary is mutable

key ← pair → value

```
fruit = {"apple": 50, "banana": 30, "orange": 40, "peach": 100}
type(fruit) ⟶ dict
```

* Extracting keys

```
fruit = {"apple": 50, "banana": 30, "orange": 40, "guava": 60}
fruit.keys()
⟶
dict_keys(['Apple', 'orange', 'banana', 'guava'])
```

* Extracting values

```
fruit = {"apple": 50, "banana": 30, "orange": 40, "guava": 60}
fruit.values()
⟶
dict_values([50, 30, 40, 60])
```

```
fruit.items()
⟶
dict_items([('apple', 50), ('banana', 30), ('orange', 40), ('guava', 60)])
```

* Adding a new element.

```
fruit={"Apple":10,"orange":20,"Banana":30}
fruit["Mango"]=50
fruit ⟶
{'Apple':10,'orange':20,'Banana':30,'Mango':50}
```

* Changing an existing element

```
fruit={"Apple":10,"orange":20}
fruit["Apple"]=100
fruit ⟶
{'Apple':100,'orange':20}
```

* Update one dictonary's element with another

```
fruit1={"Apple":10,"orange":20}
fruit2={"cherry":30,"Banana":40}

fruit.update(fruit2)
fruit1 ⟶
{'Apple':10,'orange':20,'cherry':30,'Banana':40}
```

* Popping an element (Removing)

```
fruit={"Apple":10,"orange":20,"Banana":30}
fruit.pop("orange")
fruit ⟶
{"Apple':10,'Banana':30}
```

Set in python

Set is an unordered and unindexed collection of elements elclosed with { }

* Duplicates are not allowed in set

```
s1 = {1, "Poorna", "Poorna", 1}
s1 ⟶
{1, "Poorna"}
```

We can see, duplicates are not allowed in set

Set operations.

* Adding a new element

```
s1 = {1, "a", True, 2, "b", False}
s1.add("Hello")
s1 ⟶
{1, 2, False, 'Hello', 'a', 'b'}
```

⟶ There is no proper sequence in set so the indexing is not possible in sets.

* updating multiple elements

```
s1 = {1, "a", True, 2, "b", False}
s1.update([10, 20, 30])
s1 ⟶
{1, 10, 2, 20, 30, False, 'a', 'b'}
```

* Removing an element

```
s1 = {1, "a", True, 2, "b", False}
s1.remove("b")
s1
{1, 2, False, 'a'}
```

set functions.

* union and Intersection

* union

s1 = {1, 2, 3}

s2 = {"a", "b", "c"}

s1.union(s2) ⟶

{1, 2, 3, 'a', 'b', 'c'}

Basically concatenating two sets.

* Intersection

s1 = {1, 2, 3, 4, 5, 6}

s2 = {5, 6, 7, 8, 9}

s1.intersection(s2) ⟶

{5, 6}

To find out the common elements between two sets.

If statement in python

* If statement,

  if
  it's raining
  sit inside

  else
  go out and play

①
```
a=10
b=20
if a>b:
    print("a is greater than b")
if b>a:
    print("b is greater than a")

if a>b:
    print("a is greater than b")
else:
    print("a is not greater than b")
```

Rwn →

```
a is not greater than b
```

elseif → we will use it when
we want to check the multiple statements.

classmate
Date
Page

(2)
```
a=10
b=20
c=30
if(a>b and & a>c):
    print("a is the greatest")
elif (b>a & b>c):
    print("b is the greatest")
else:
    print(" c is greatest")
```
Run → c is the greatest

* Now can see how can we use this Conditional statements with List, Tuple and dictionary.

→ tupl=(1,2,3,4) (with tuple)
```
if 2 in tupl:
    print(" 2 is present in tuple")
```
Run → 2 is present in tuple

→ tupl=(1,2,3,4)
```
if 6 in tupl:
    print(" 6 is present in tuple")
else:
    print("6 is not present in tuple")
```
Run → 6 is not present in tuple

with list if with list

*
```
li = [1,2,3,4,5]
if li[1]==2:
    li[1]=li[1]+100
li ⟶ [1,102,3,4,5]
```

*
```
li=[1,2,3,4,5]
if li[4]==10:
    li[1]=li[1]+100
else:
    li[4]=li[4]+500
li ⟶ [1,102,3,4,505]
```

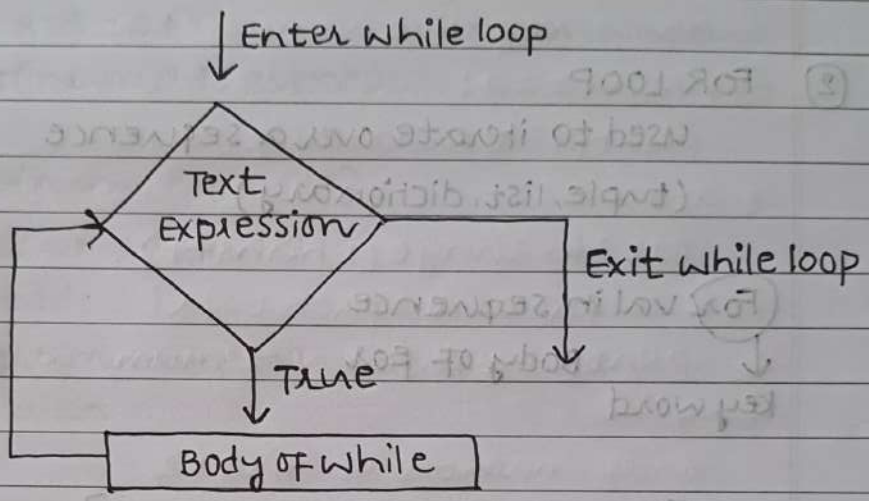
if with dictionary

*
```
dl = {"a":1,"b":2,"c":3}
if dl["b"]==2:
    dl["b"]=dl["b"]+100
dl ⟶ {'a':1,'b':102,'c':3}
```

LOOPING STATEMENTS IN PYTHON:

used to repeat a task multiple times.

① While loop =



Syntax

```
while condition :
        Execute statements
```

*
```
i=1
while i<=10:
    print(i)
    i=i+1
```
→ 1 2 3 4 5 6 7 8 9 10

*
```
i=1
n=2
while i<=10:
    print(n,"*",i,"=",n*i)
    i=i+1
```
→
2*1 = 2
2*2 = 4
2*3 = 6
2*4 = 8
2*5 = 10
2*6 = 12
2*7 = 14
2*8 = 16
2*9 = 18
2*20 = 40
2*10 = 20

* `li = [1,2,3,4,5]` `i=0`

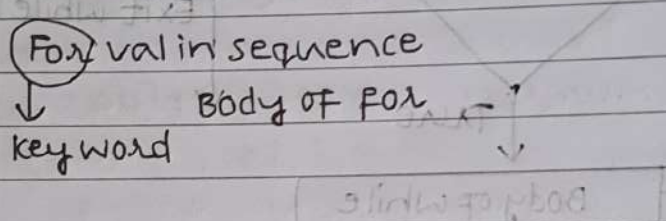
```
while i < len(li):
    li[i] = li[i] + 100
    i = i+1
```

li ⟶ [101, 102, 103, 104, 105]

② FOR LOOP

used to iterate over a sequence

(tuple, list, dictionary)



keyword

*
```
li = ["apple", "Banana", "orange"]
for i in li:
    print(i)
```

⟶ apple
banana
orange

* nested for loop

```
li = ["orange", "blue", "green"]
l2 = ["book", "chair", "phone"]
for i in li:
    for j in l2:
        print(i, j)
```

⟶ orange book
orange chair
orange phone
blue book
blue chair
blue phone
green book
green chair
green phone

Basic problems in python

① check even or odd :

```
num = int(input("Enter a number: "))
if(num%2)==0:          %→Remainder
   print(num," is even")
else:
   print(num," is odd")
```

→ Enter a number: 5
  5 is odd

→ Enter a number: 8
  8 is even

② check positive, negative or 0

```
num = float(input("Enter a number: "))
if num>0:
   print("positive number")
elif num==0:
   print("zero")
else:
   print("Negative number")
```

→ Enter a number: 8
  Positive number

→ 0
  zero

→ Enter a number: -5
  Negative number

③ Factorial of a number.

```
num = int(input("Enter a number: "))
factorial = 1
if num<0:
    print("Sorry, Factorial doesn't exist for
        Negative number")
elif num==0:
    print("The Factorial of 0 is 1")
else:
    for i in range(1,num+1):
        factorial = factorial*i
    print("The factorial of", num, "is", factorial)
```

⟶

Enter a number 4 4X3X2

24

⟶

Enter a number

3628800

classmate
Date
Page

(4) Reversing a number

```
n=int(input("Enter number: "))
rev=0
while(n>0):
    dig=n%10
    rev=rev*10+dig
    n=n//10
print("Reverse of the number:", rev)
```

n = 123 → 321

(5) check if it is a palindrome (12321)

```
n=int(input("Enter a number: "))
temp=n
rev=0
while(n>0):
    dig=n%10
    rev=rev*10+dig
    n=n//10
if(temp==rev):
  print("The number is a palindrome!")
else:
  print("The number is not a palindrome!")
```

121/10 → rem(1)	n	rev
12/10 → 2	121	0*10+1 = 0+1 = 01
	12	1*10+2 = 10+2 = 12
	1	12*10+1 = 120+1 = 121

dig
1
2
1

Enter a number: 121
The number is a palindrome

⑥ Fibonacci 0 1 1 2 3 5 8

```
n=int(input("Enter number: "))
a=0
b=1
if n<0:
    print("Incorrect input")
elif n==0:
    print(a)
elif n==1:
    print(a)
else:
  for i in range(2,n):
      c=a+b
      a=b
      b=c
    print(b)
```

⟶ Enter a number: 7
8

a	b	c
0	1	1
1	1	2
1	2	3
2	3	5

Functions in python.

Block of code which performs a specific task.

Normal function

Lambda function

Normal function syntax

```
def function_name:
        Execute statements
```

Lambda function syntax

```
Lambda arguements : expression
```

* ```
  def hello():
      print("Hello World")
  hello()
  ```

  Hello World

* ```
  def add10(x):
      return x+10
  add(10(10)
  ```

  def — keyword

  x — parameter

  20

```
def even_odd(x):
    if x%2==0:
        print(x,"is even")
    else:
        print(x,"is odd")
even_odd(5)
```

5 is odd

Lambda functions. (filter, map, reduce)

```
g = lambda x: x*x*x
print(g(7))
```

343

Main function of lambda operations

lambda functions with (filter) takes two parameter

```
li=[5,7,22,97,54,62,77,23,73,61]
final_list=list(filter(lambda x:(x%2!=0),li))
print(final_list)
```

1st 2nd

extracting the odd numbers.

→

[5, 7, 97, 77, 23, 73, 61]

#lambda with map

```
li=[5,7,22,97,54,62,77,23,73,61]
final_list = list(map(lambda x:x*2,li))
print(final_list)
```

⟶

```
[10,14,44,194,108,124,154,46,146,122]
```

lambda with reduce

```
from functools import reduce
li=[5,8,10,20,50,100]
sum = reduce((lambda x,y:x+y),li)
print(sum)
```

⟶ 193

Object oriented programming

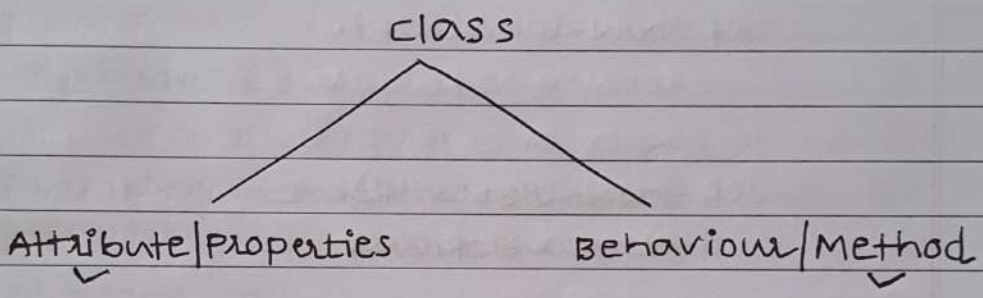
classmate
Date
Page

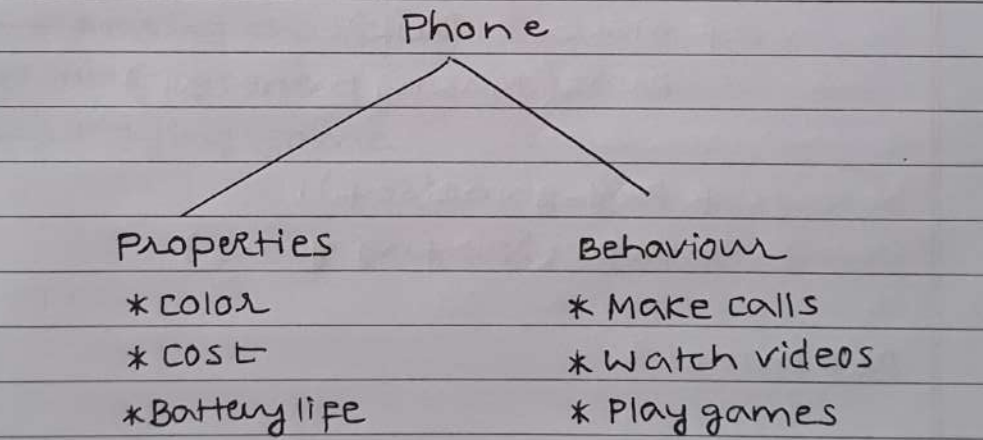
OOPs in python:

we are surrounded with objects
like (mobile, laptop, bike, key, pen, dog, book)

classes:-

class is a template for real world entities

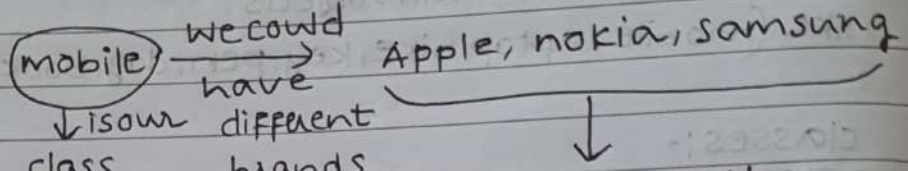




class is a user defined datatype
↳ It is a datatype which we can create by ourselves

objects :-

objects are specific instances of a class.



classmate
Date
Page

creating the first class.

```
#creating a class phone

class phone :
    def make_call(self):
    print("Making phone call")
    def play_game(self):
    print("playing game")

p1=phone()

#invoking methods through object
p1.make_call
Making phone call
p1.play_game
Playing game
```

methods

Referencing call

keyword

name of object

name of object → name of the method

Adding parameters to a class method

```
class Phone:
    def set_color(self, color):
        self.color = color

    def set_cost(self, cost):
        self.cost = cost

    def show_color(self):
        return self.color

    def show_cost(self):
        return self.cost

    def make_call(self):
        print("Making phone call")

    def play_game(self):
        print("playing game")

P2 = Phone()

P2.assign
P2.set_color('blue')
P2.set_cost(500)
P2.show_color() → 'blue'
P2.show_cost() → 500
```

classmate
Date
Page

creating a class with constructor.

```
class Employee:
    def __init__(self, name, age, salary, gender):
        ↳ init method acts as constructor
        ↓ spcl type of function
        self.name = name
        self.age = age
        self.salary = salary
        self.gender = gender

    def employee_details(self):
        print("Name of employee is ", self.name)
        print("Age of employee is ", self.age)
        print("salary of employee is ", self.salary)
        print("Gender of employee is", self.gender)

e1 = Employee("Poorna", 23, 1,00,000, "Male")
↓ instantiating the "e1" object

e1.employee_details() → Invoking 'employee_details' method.
```

Name of employee is poorna
age of employee is 23
salary of employee is 1,00,000
gender of employee is Male

Inheritance in python

With Inheritance one class can derive the properties of the another class.

```
class vehicle:
    def __init__(self,mileage,cost):
        self.mileage=mileage
        self.cost=cost

    def show_details(self):
        print("I am a vehicle")
        print("Mileage of vehicle is", self.mileage)
        print("cost of vehicle is", self.cost)

V1=vehicle(500,500)
V1.show_details()
```

⟶

```
I am a vehicle
Mileage of vehicle is 500
cost of vehicle is 500
```

classmate
Date
Page

creating the child class

```
class car(vehicle):
    def show_car(self):
        print("I am a car")

c1 = car(200,1200)
c1.show_details()
I am a vehicle
Mileage of vehicle is 200
Cost of vehicle is 1200

c1.show_car()
I am a car
```

→ Instantiating the object for child class

→ Invoking the child class method

over-riding init method.

```
class car(vehicle):
    def __init__(self, mileage, cost, tyres, hp):
        super().__init__(mileage, cost)
        self.tyres = tyres
        self.hp = hp

    def show_car_details(self):
        print("I am a car")
        print("Number of tyres are", self.tyres)
        print("value of hp is", self.hp)

#invoking show_details()
 method from parent class

c1 = car(20, 12000, 4, 300)
c1.show_details()
```

⟶

```
I am a vehicle
Mileage of vehicle is 20
cost of vehicle is 12000
```

```
#invoking show_car_details()
 method from child class

c1.show_car_details()
```

⟶

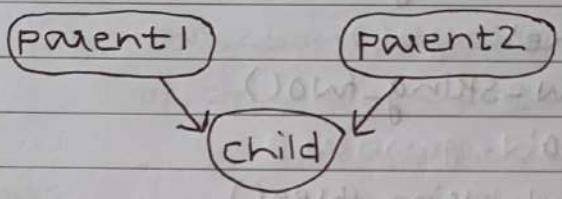
```
I am a car
Number of tyres are 4
value of hp is 300
```

Types of Inheritance

Single Inheritance
Multiple Inheritance
Multi-level Inheritance
hybrid Inheritance

Multiple Inheritance

The child inherits from More than 1 parent class.




```
class parent1():
    def assign_string_one(self,str1):
        self.str1 = str1

    def show_string_one(self):
        return self.str1
```

(Parent class 1)

```
class parent2():
    def assign_string_two(self,str2):
        self.str2 = str2
    def show_string_two(self):
        return self.str2
```

(Parent class 2)

```
class derived(Parent1, parent2):
    def assign_string_three(self,str3):
        self.str3 = str3
    def show_string_three(self):
        return self.str3
```

Creating the object of the child class

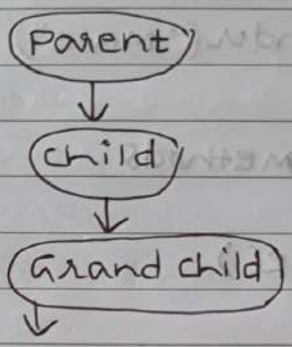
```
d1. Derived
d1.assign-string-one("one")
d1. assign-string-two("two")
d1. assign-string-three("three")
```

Invoking methods

```
d1.show_string-one()
→ 'one'
d1.show-string-two()
→ 'two'
d1.show_string-three()
→ 'three'
```

Multi-level Inheritance

We have parent, child and grandchild Relationship



I have both the properties of child and the parent.

Parent class

```
class parent():
    def assign_name(self,name):
        self.name = name
    def show_name(self):
        return self.name
```

child class:

```
class child(parent):
    def assign_age(self,age):
        self.age = age
    def show_age(self):
        return self.age
```

Grand child class

```
class grandchild(child):
    def assign_gender(self,gender):
        self.gender = gender
    def show_gender(self):
        return self.name
```

gl.ch grandchild()

gl.assign_name("Poorna")
gl.assign-age(25)
gl.assign-gendu("Male")

Invoking class methods

gl.show-name()
⟶ Poorna
gl.show-age()
⟶ 25
gl.show-gendu()
⟶ Male

Continuation of python

(26) **Array in python**

All the value of same type.

if we have int array, we should only have int array

if we have float array, we should only have float array.

```
import array as arr
arr.array
                (or)        i - signed integer (-1 to +1)
from array import *         I - unsigned integer
vals = array('i', [5,9,8,42])
print(vals)
```

* `print(vals.buffer_info())`
  → (Address, size)
* `print(typecode)`
  → i → we are working with integer
*
```
from array import *
vals = array('i', [5, 9, -8, 4, 2])
vals.reverse()
print(vals)
```
→ array('i', [2, 4, -8, 9, 5])

```
vals = array('i', [5,9,-8,4,2])
for i in range(5):
    print(i)
```
→ 5
9
-8
4
2

* When we dont know the range

```
for i in range(len(vals)):
    print(vals[i])
```

```
for e in vals:
    print(e)
```

* Can we work with characters
    U - unicode

* from array import *
```
vals = array('u', ['a', 'e', 'i'])
for e in vals:
    print(e) → a
               e
               i
```

* If I want to create a new array using the old array
```
from array import *
vals = ('i', [5, 9, 8, 4, 2])
newarr = array(vals.typecode, (a for a in vals))
for e in newarr:                    (a*a for a in vals))
 print e
```

---

using while loop

```
While i<len(newarr):
    print(newarr[i])
    i=i+1
```

(27) inserting element in array
searching for element in array
in python

we can create a blank array

```
from array import *
arr=array('i',[])
n=int(input("Enter the length of the array"))
for i in range(5): range(n):
    x=int(input("Enter the next value"))
    arr.append(x)
print(arr)
val=int(input("Enter the value for search"))
k=0
for e in arr:
    if e==val:
        print(k)
        break
    k=k+1
print(arr.index(val))
```

single dimensional
one row and multiple columns

multi dimensional array
multiple rows and multiple columns

Exception Handling

What is

compile time error → a = hello" (Problem in syntax)

Runtime error → L=[1,2,3] print(L[4])

logical error → 4+4|2 (Problem with logic)

⑥

Exceptions:- Runtime errors as exception

There is an error occuring during the execution of the particular task

Exception handling.

keywords -

try:

except:

else: } optional

finally: }

try:-

Contains operations

except:-

what has to be done

write that code here.

Types of error

1) n,x = 5,6,"Poorna"

value error = too many values to unpack

2) L=[1,2,3]

L[4]

index error = list index out of range

python.org

classmate
Date
Page

(3) b=10
a

NameError: name a is not defined.

(4) import poorna

ModuleNotFoundError = no module named 'poorna'

(5) 2+'3'

TypeError: unsupported operand type for +
'int' and 'str'.

Program

(*) import math
#enter negetive number to check what happens

num=int(input("Enter number to comput of
factorial: "))
try:
print(math.factorial(num))
except valueerror:
print("cannot compute the factorial of
negetive numbers")

⊛ Program

```
import math

num = int(input("Enter no to compute factorial of : "))
valid_input = false

while not valid_input:
try:
        print(math.factorial(num))
        valid_input = True
except valueerror
        print("cannot compute the factorial
                of negative nos")
        num = int(input("please re-enter: "))
```

⊛ Program

```
def getmonth():
    month = int(input("Enter current month (1-12): "));
    if month<1 or month>12:
            raise value error
    return month

valid = False
monthname = ("Jan", "Feb", "march", "Ap", "may", "June",
            "July", "Aug", "sept", "Nov", "Dec")
while not valid
    try:
        month = getmonth()
        print("The month you entered is",
            month_name[month-1])
        valid1 = True
    except valueError:
        print("Invalid month enter\n")
```

classmate
Date
Page

(*) program

```
def getMonth():
    Month = int(input("Enter current month (1-12): "))
    if month<1 or month>12:
      raise valueError("Invalid Month value")
    return month

valid=false
month_name=("Jan",
            "Dec")
While not valid:
try:
    month =getMonth()
    print("The month you entered is",
          month_name[month-1])
    valid = True
except valueError as ve_msg:
    print(ve_msg)
          ↳ This an object of value error
```

(*) Program

→ creating my own exception class

```
class UserDefinedException(Exception):
    def __init__(self, message):
        self.message = message
try:
    s=input("Enter name: ")
    if(s==""):
        raise UserDefinedException("No empty string Allowed")
    else:
        print(s)
except UserDefinedException as msg:
    print("Error Message", msg)
```

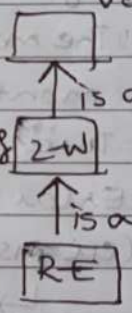
① Guess the output

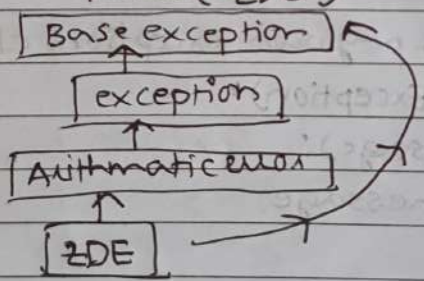
```
try:
    a = 5
    b = 5/0
except ZeroDivisionError:
    print("ZDE")
except BaseException:
    print("BE")
except Exception:
    print("E")
```

```
try:
    a = 5
    b = 5/0 → raise ZDE
except BaseException:
    print("BE")
except Exception:
    print("E")
except ZeroDivisionError:
    print("ZDE")
```

ZDE is a Base exception variable

If tries to matching





ZDE is a Baseexception error

② The else part will executed when no exception has been caught

4)
```
try:
    a=5
    b=5|5
except ZDE:
    print("ZDE")
except BaseException:
    print("BE")
except Exception:
    print("E")
else:
    print("ELSE")
```

5)
```
try:
    a=5
    b=5|0
except(BaseException,Exception,ZeroDivisionError):
```

6)
```
try:
    a=5
    b=5|5
except:
    print("Hi")
else:
    print("Hello")
```

7)
```
try:
    a=5
    b=5|5
else                    → It will give us compile time error
    print("Hello")
```

⑧

```
try:
    a=5
    b=5/0
except:
    print("Hi")
else:
    print("Hello")
finally:
    print("Namasthe")
```

```
try:
    cook-biriyani()
except salt less error
except overcooked error
finally:
    switchoff-stove
```

whether there is exception is executed or not finally will be executed.

—| end |—

of exception handling

(29) Ways of creating Arrays in Numpy

array(), linespace(), logspace(),
arange(), zeros(), ones()

```
from numpy import *
arr = array([1,2,3,4,5])
print(arr)
```

in array all the values should be of same type

* linspace() (start, stop, step)

```
arr = linspace(0,15,16)
```

→ dividing into the parts

* arange()

```
arr = arange(1,15,2)
print(arr)
```

→ 1, 3, 5, 7, 9, 11, 13

* logspace()

```
arr = logspace(1,40,5)
print(arr)
```

---

* zeros() ones() → more efficient

```
arr = ones(5)
print(arr)
```

→ [1. 1. 1. 1. 1.]

(30) Copying an Array in python

* From numpy import *
```
arr = array([1,2,3,4,5])
arr = arr+5
print(arr)
→ [6 7 8 9 10]
```

* vectorized operation
```
from numpy import *
arr1 = array([1,2,3,4,5])
arr2 = array([5,4,3,2,1])
arr3 = arr1+arr2
print(arr3)
→[6,6,6,6,6]
```

* Mathematical operations in numpy
```
from numpy import *
arr1 =array([1,2,3,4,5])
print(arr1)
print(log(arr1))
print(sin(arr1))
print(cos(arr1))
print(sqrt(arr1))
print(sum(arr1))
print(min(arr1))
print(max(arr1))
unique, sort
```

* we can also concatinate
```
from numpy import *
arr1 =([1,2,3,4,5])
arr2 =array([6,1,9,3,2])
print( concatenate([arr1,arr2]))
```

classmate
Date
Page

Copying

```
from numpy import *
arr1 = array([1,2,3,4,5])
arr2 = arr1.view()
print(arr1)
print(arr2)
```

view is a function that creats new array

shallow copy:

The both the array are interlinked

```
from numpy import *
arr1 = array([2,6,8,1,3])
arr2 = arr1.view()
arr1[1] = 7
print(arr1) → 2,7,8,1,3
print(arr2) → 2,7,8,1,3
```

Deep copy

The both the arrays are not interlinked

```
from numpy import *
arr = array([1,2,3,4,5])
arr2 = arr1.copy()
arr1[1] = 7
print(arr1) → 1,7,3,4,5
print(arr2) → 1,2,3,4,5
```

(31) Working with matrix

```
from numpy import *
arr = array([
        [1, 2, 3],
        [4, 5, 6]
        ])
print(arr)
print(arr.ndim) → show the no of dimension
print(arr.shape) → 2 row, 3 columns
print(arr.size) → 6 (counts the size)
```

I want to create 2d array into 1D array

* From numpy import *

```
arr = array([
        [1, 2, 3],
        [4, 5, 6]
        ])
arr2 = arr.flatten()
print(arr2)
[1, 2, 3, 4, 5, 6]
```

arr2 is 1D

* To create 3D array from 1D array

```
from numpy import *
arr = array([
        [1, 2, 3, 6, 2, 9],
        [4, 5, 6, 7, 5, 3],
        ])
arr2 = arr.flatten()
arr3 = arr2.reshape(3, 4)
print(arr3)
```

classmate
Date
Page

matrix operations.

```
from numpy import *
m=matrix("1,2,3; 6,4,5; 1 6 7")
print(diagonal(m)) → [1 4 7]
print(m.min()) → 1
Print(m.max()) → 7
```

```
from numpy import *
m1=matrix('1 2 3; 6 4 5; 1 6 7')
m2= matrix('1 2 3; 6 8 5; 2 6 7')
m3 = m1 * m2 ;
Print(m3)
```

(32) Functions in python:

I want to define a function, later on I want to call that function

func name

```
* def greet():
      print("Hello")
      print("Good Morning")
  greet()
```

sweet

parameter or argument

```
* def add(x,y):
      c=x+y
      print(c)
  add(5,4) ⟶ 9
```

function can return a value

```
* def add(x,y):
      c=x+y
      return c
  result = add(5,4)
  print(result)
```

* from one function we can return multiple values

```
def add_sub(x,y):
    c=x+y
    d=x-y
    return c,d
result1, result2 = add_sub(5,4)
print(result1, result2)
```

classmate
Date
Page

(33) Function arguements in python

How to pass a parameter to function

```
def update(x):
    x=8
    print(x)
update(10)
```

(34) Types of Arguements.

```
def add(a,b):
    c=a+b
    print(c)
add(5,6) ⟶ 11
```

a,b → Formal Arguement

Arguement (Actual Arguement)

Position

```
def person(name, age):
    print(name)
    print(age)
person("Poorna", 23)
```

Keyword

```
Person(age=28, name="Poorna")
```

default

```
* def person(name,age=18):
    print(name)
    print(age)
person("Poorna") ⟶ Poorna.
```

variable length arguement

```
def sum(a,b):
    c=a+b
    print(c)
sum(5,6)
```

```
def sum(a, *b)
    c=a
    for i in b:
        c=c+i
    print(c)
sum(5,6,34,78) ⟶ 123
```

(35) **kwargs

↑function

```
def person():
def person(name, **data):
    print(name)
    print(data)
person("navin", age=28, City="mumbai", mDb=00)
```

(36) Global keywords.

scope

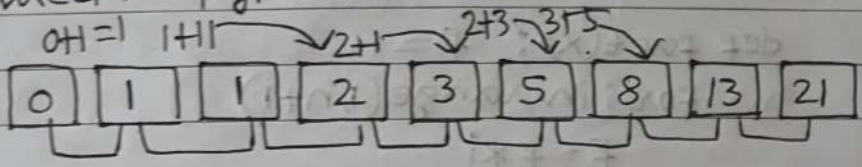
(a=10) → This a this is outside the fun is global

```
def something():
    (a=15) → Inside the fun is local variable
    print(a)  ↳ we cannot use this out side

print(a)
```

(37) Pass list to a function

(38) Fibonacci in python.



We should add 2 numbers to get next number

```
def fib(n):
    a=0
    b=1
    print(a)
    print(b)
    for i in range(2,n):
        c=a+b
        a=b
        b=c
        print(c)
fib(5)
```

(39) Factorial of a number.

$5! = 5*4*3*2*1 = 120$

$4! = 24$

```
def fact(x):  f=1
    for i in range(1,n+1)
        f = f*i
    return f
x=4
result = fact(x)
print(result)
```

(40) Recursion in python

```
def greet():
    print("Hello")

import sys
sys.setrecursionlimit(2000)
print(sys.getrecursionlimit())
i=0
def greet():
    global i
    i+=1
    print("Hello", i)
    greet()
greet()
```

A function calling itself is Recursion.

(41) factorial using recursion.

```
def fact(n):
    if n==0
        return 1
    return n* fact(n-1)
result = fact(5)
print(result) ⟶ 120
```

(42) lambda ⟶ Anonymus function
function without names are called as Anonymus function (or) lambda.
"functions are objects" in python

```
f= lambda a ; a*a
result = f(5)
print(result) ⟶ 25
```

(43) lambda,

(43) lambda
Filter
map
reduce.

```
def is_even(n):
    return n%2==0
nums=[3,2,6,8,4,6,2,9]

evens = filter(
evens = list(filter(is_even,nums))
print(evens)
```

```
nums = [3,2,6,8,4,6,2,9]
evens = list(filter(lambda n: n%2==0,nums))
print(evens)
doubles=list(map(lambda n:n*2,evens))
```

filter=

reduce

```
from functools import reduce

nums=[3,2,6,8,4,6,2,9]
evens=list(filter(lambda n:n%2==0,nums))
doubles=list(map(lambda n:n*2,evens))
print(doubles)
sum=reduce(lambda a,b: a+b,doubles)
print(sum)
```

(44) Decorators:

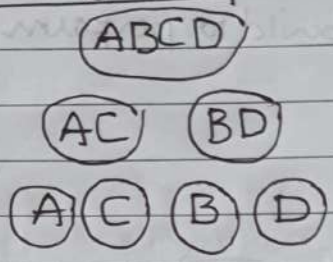
functions are build to perform certain task

```
def div(a,b):
    print(a/b)
def smart_div(func):          # It is taking it as a parameter
    def inner(a,b):
        if a<b:
            a,b = b,a
        return func(a,b)
    return inner

div1 = smart_div(div)
div1(2,4)
```

We can change the behaviour of existing function

45 Modules in python.



1 module will be file

```
a=9
b=7
```

new file

```
def add(a,b):
    return(a+b
def sub(a,b):
    return a-b
def multi(a,b):
    return a*b
def div(a,b):
    return a/b
```

```
import calc
a=9
b=7
c=calc.add(a,b)
```

```
from calc import *
a=9
b=7
c=sub(a,b)
```

→ 2

(46) `__name__ = "__main__"`
special variable `__name__`

```
print(__name__)
```
→ `__main__` → starting point of program

(47) continued part

demo
```
from calc import add
def fun1():
    add()
    print("from fun1")
def fun2():
    print("from fun2")
fun1()
fun2()
def main()
    fun1()
    fun2()
main()
```
→ in calc main
result 1 is __main__
result 2 is

calc
```
def add():
    print("result 1 is", __name__)
def sub():
    print("result 2 is")
def main():
    print("in calc main")
    add()
    sub()
if __name__ == "__main__":
    main()
```

(6) Iterators in python

one value at a time

```
nums = [7, 8, 9, 5]
nums[0] ⟶ 7
nums[3] ⟶ 5
for i in nums:
    print(i) ⟶ 7
                8
                9
                5
```

```
nums = [7, 8, 9, 5]
it = iter(nums)
print(it.__next__()) ⟶ 7
print(next(it)) ⟶ 8
```

lets create own class.

```
class topten:
    def __init__(self):
        self.num = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.num <= 10:
            val = self.num
            self.num += 1
            return val
        else:
            raise StopIteration

values = Topten()
for i in values:
    print(i)
```

(62) Generators.

top10 perfect square

```
def topten():
    n=1
    while n<=10:
        sq = n*n
        yield sq
        n+=1
values = topten()
for i in values:
    print(i)
```

Python for Data Analysts - Quick Summary

Join our official channel for more Interview & Resume Tips: Data Analysts
Follow us on Linkedin: Data Analysts Linkedin Page

This pdf contains important python concepts useful for a data analyst. I'll provide a more concise summary of the key Python concepts and libraries essential for data analysts, with some additional details for each topic:

Python for Data Analysts: Essential Concepts and Libraries

Python is a versatile and widely-used programming language in the field of data analysis due to its simplicity and rich ecosystem of libraries. As a data analyst, you'll need to understand various concepts and libraries to effectively work with data.

1. Basics of Python

Variables and Data Types
Python supports various data types, including integers, floats, strings, lists, and dictionaries. These are the building blocks for data manipulation.

Control Structures
Learn about loops (for and while) and conditional statements (if, else) to control the flow of your code.

Functions
Functions allow you to encapsulate reusable code, making your programs more organized and efficient.

Libraries

Python's power lies in its libraries. Learn how to import and use external libraries/modules to extend Python's capabilities.

2. NumPy

NumPy is a fundamental library for numerical operations in Python.

Arrays
NumPy introduces arrays, N-dimensional structures that allow efficient numerical operations.

Array Manipulation
You can reshape, stack, and slice arrays for various data processing tasks.

Mathematical Functions
NumPy offers a wide range of mathematical functions, making it a cornerstone for data analysis.

3. Pandas

Pandas is a powerful library for data manipulation and analysis.

DataFrames
DataFrames are two-dimensional tabular data structures that enable data cleaning and analysis.

Data Cleaning
Learn techniques to handle missing data, duplicates, and outliers.

Data Selection and Filtering

Methods like indexing, slicing, and filtering help you extract specific data points.

Grouping and Aggregation

You can group data and perform summary statistics using Pandas functions.

4. Matplotlib and Seaborn

Data visualization is essential in data analysis. Matplotlib and Seaborn are popular libraries for creating plots and charts.

Basic Plots

Create line plots, scatter plots, bar charts, and more.

Customization

Customize your visualizations with labels, colors, and styles to convey insights effectively.

Seaborn

Seaborn offers a high-level interface for creating visually appealing statistical plots.

5. Data Cleaning and Preprocessing

Before diving into analysis, you must clean and preprocess data.

Handling Missing Data

Learn options like removal, interpolation, or filling to deal with missing data.

Data Transformation

Techniques like scaling and encoding are used to prepare data for analysis.

Feature Engineering

Create new features from existing data to improve model performance.

6. Statistical Analysis

Statistical analysis helps you draw meaningful conclusions from data.

Descriptive Statistics

Understand mean, median, mode, and measures of variability to summarize data.

Hypothesis Testing

Learn techniques like t-tests and chi-squared tests to make data-driven decisions.

Correlation and Regression

Analyze relationships between variables and make predictions using regression.

7. Machine Learning

Machine learning can enhance your data analysis capabilities.

Supervised Learning

Predictive modeling with labeled data using algorithms like linear regression or decision trees.

Unsupervised Learning

Discover patterns in data through clustering and dimensionality reduction techniques.

Evaluation Metrics

Assess and compare the performance of machine learning models using metrics like accuracy and F1-score.

8. SQL and Databases

Data analysts often work with databases for data retrieval and manipulation.

Structured Query Language (SQL)

Learn to write SQL queries to extract, filter, and aggregate data from relational databases.

Database Connection

Establish connections to databases using Python libraries like SQLAlchemy.

9. Jupyter Notebooks

Jupyter notebooks are interactive documents for data analysis.

Markdown and Code Cells

Combine explanatory text with executable code, making your analysis more accessible.

Data Visualization

Integrate Matplotlib and other plotting libraries to create informative visualizations.

Sharing and Collaboration

Export and share notebooks to communicate your findings and collaborate with others.

10. Data Analysis Projects

Practical experience is crucial for becoming a proficient data analyst.

Identify a Problem

Select a dataset and define a data analysis problem that aligns with your goals.

Data Collection

Gather, clean, and preprocess the data to prepare it for analysis.

Analysis and Visualization

Perform data analysis and create visualizations to gain insights.

Communication

Present your findings effectively to convey the significance of your analysis.

11. Data Ethics and Privacy

Data analysts must be aware of ethical considerations when working with data.

Privacy

Safeguard sensitive information in datasets and follow data protection regulations.

Bias and Fairness

Recognize and mitigate biases in data and algorithms to ensure fairness.

Transparency

Document and explain your data analysis processes for transparency and accountability.

12. Python Libraries for Specialized Tasks

Depending on your specific domain, you may need to explore libraries like SciPy, scikit-learn, Statsmodels, and others for advanced statistical analysis and machine learning tasks.

Join our Telegram channel for more Free Resources: Data Analysts

Arithmetic operators:

An arithmetic operator performs mathematical operations like addition, subtraction, division and multiplication.

Operator	Meaning	Example
+	Add two operands	>>>5+4 9
-	Subtract two operands	>>>6 - 4 2
*	Multiply two operands	>>>2*10 20
/	divide the left oferend by right operand	>>>5/2 2.5
%	which will give us remindar part	>>>5%3 2
//	which will give integer part	>>>5//3 1
**	raised to the power	>>>5**3 125

Comparisons operator

Operator	Description	Example
==	If two operands values are equal, then the condition becomes true.	>>> 5==3 False
!=	values of two operands are not equal, then condition becomes true.	>>>5!=3 True
>	If the value of left operand is greater than the value of right operand, then condition becomes true.	>>>4>3 True
<	If the value of left operand is less than the value of right operand, then condition becomes true.	>>>6<4 False
>=	**greater than equal:** If the value of left operand is greater than or equal to the value of right operand, then condition becomes true.	>>>5>=6 False
<=	**less than e**	>>>4<=6 True

Logical operators

- Logical operators is Boolean expressions such as and, or, not.
- It is just a conditional test that a result is either True or False.

operator	Description	Example
And	it two operands are true it become true	>>> True and True True >>>False and True False
Or	it two operands are non-zero then condition becomes true.	>>>True or False True >>>False or False False
Not	It is used to reverse the logical state of its operand.	>>>not True False >>>not not not True False

Assignment operator:

Operator	Description	Example
=	Assigns values from right side operands to left side operand	>>>x=20
+=	It adds right operand to the left operand and assign the result to left operand	>>>x+=20
-=	It subtracts right operand from the left operand and assign the result to left operand	>>>x-=20
*=	It multiplies right operand with the left operand and assign the result to left operand	>>>x*=50
/=	It divides left operand with the right operand and assign the result to left operand	>>>x/=50
%=	It takes modulus using two operands and assign the result to left operand	>>>x%=50
**=	Performs exponential (power) calculation on operators and assign value to the left operand	>>>x**=50
//=	It performs floor division on operators and assign value to the left operand	>>>x//=50

Membership operator

operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	>>>x="python" >>>'p' in x True
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.	>>>x="python" >>>'z' in x False

Bitwise operators

Operator	Description	Example
&	Operator copies a bit to the result if it exists in both operands	>>>10&20 0
|	It copies a bit if it exists in either operand.	>>>10|20 30
^	It copies the bit if it is set in one operand but not both.	>>>10^20 30
~	It is unary and has the effect of 'flipping' bits.	>>>~20 -21
<<	The left operands value is moved left by the number of bits specified by the right operand.	>>>10<<2 40
>>	The left operands value is moved right by the number of bits specified by the right operand.	>>>10>>2 2

Identity operators

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	>>>x="hello" >>>y="hello" >>>x is y True
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	>>>x="hello" >>>y="hello" >>>x is not y False

URBAN EDGE

Python programming notes by Harry

What is Programming?

Just like we use Hindi or English to communicate with each other, we use a programming language like Python to communicate with the computer.
Programming is a way to instruct the computer to perform various tasks.

What is Python?

Python is a simple and easy to understand language which feels like reading simple english. This Pseudo Code nature of python makes it easy to learn and understandable by beginners

Features of Python

Easy to understand = Less development time
Free and open source
High level language
Portable → Works on Linux / Windows / Mac.
+ Fun to work with

Installation

Python can be easily installed from python.org
When you click on the download button, python can be installed right after you complete the setup by executing the file for your platform

URBAN EDGE

Chapter 1 : Modules, Comments & pip

Lets write our very first python program. Create a file called hello.py and paste the below code in it

```
print ("Hello World")
```
→ print is a function (More later)

Execute this file (.py file) by typing python hello.py and you will see Hello World printed on the screen.

Modules

A module is a file containing code written by somebody else (usually) which can be imported and used in our programs

Pip

Pip is the package manager for python
You can use pip to install a module on your system

↳ pip install flask installs flask module.

Types of modules

There are two types of modules in Python

1) Built in modules → Pre installed in python
2) External modules → Need to install using pip.

Some examples of built in modules are os, abc, etc.
Some examples of external modules are tensorflow, flask etc.

Using Python as a Calculator

We can use python as a calculator by typing "python" + ↵ on the terminal

↳ This opens REPL or Read Evaluate Print Loop

Comments

Comments are used to write something which the programmer does not want to execute.

↳ Can be used to mark author name, date etc.

Types of Comments

There are two types of comments in Python

1. Single line Comments → Written using #
2. Multi line Comments → Written using ''' Comment '''

URBAN EDGE

Chapter 1 - Practice Set

1. Write a program to print Twinkle twinkle little star poem in python.

2. Use REPL and print the table of 5 using it

3. Install an external module and use it to perform an operation of your interest

4. Write a python program to print the contents of a directory using os module. Search online for the function which does that.

5. Label the program written in Problem 4 with Comments.

URBAN EDGE

Chapter 2 - Variables and Datatypes

A variable is the name given to a memory location in a program. for example

```
a = 30
b = "Harry"
c = 71.22
```

→ Variables = Containers to store a value.
→ Keywords = Reserved words in Python
Identifiers = class/function/variable name

Data Types

Primarily there are following data types in Python

1. Integers
2. Floating point numbers
3. Strings
4. Booleans
5. None

Python is a fantastic language that automatically identifies the type of data for us.

```
a = 71          ⇒ Identifies a as class<int>
b = 88.44       ⇒ Identifies b as class <float>
name = "Harry"  ⇒ Identifies name as class <str>
```

Rules for defining a Variable name → Also applies to other Identifiers

→ A variable name can contain alphabets, digits and underscores.
→ A variable name can only start with an alphabet and underscore.
→ A variable name cant start with a digit
→ No white space is allowed to be used inside a variable name.

URBAN EDGE

Examples of a few variable names are :-
harry, one8, seven_, _seven etc.

Operators in Python
Following are some common operators in Python:
1. Arithmetic operators ⇒ +, -, *, / etc.
2. Assignment operators ⇒ =, +=, -= etc.
3. Comparison operators ⇒ ==, >, >=, <, != etc.
4. Logical operators ⇒ and, or, not

type() function and Typecasting
type function is used to find the data type of a given variable in Python.

```
a = 31
type(a) => class <int>

b = "31"
type(b) => class <str>
```

A number can be converted into a string and vice versa (if possible)
There are many functions to convert one data type into another

```
str(31) => "31"      => Integer to String Conversion
int("32") => 32      => String to Integer Conversion
float(32) => 32.0    => Integer to Float Conversion

... and so on
```

Here "31" is a string literal and 31 a numeric literal.

URBAN EDGE

input() function
This function allows the user to take input from the keyboard as a string

a = input("Enter name") ⇒ If a is "harry", the user entered harry

It is important to note that the output of input is always a string (even if the number is entered) ⇒ If a is "34" user entered 34

URBAN EDGE

Chapter 2 - Practice Set

1. Write a Python program to add two numbers

2. Write a Python program to find remainder when a number is divided by 2.

3. Check the type of the variable assigned using input() function.

4. Use Comparison operators to find out whether a given variable 'a' is greater than 'b' or not.
   Take a = 34 and b = 80

5. Write a python program to find average of two numbers entered by the user.

6. Write a python program to calculate square of a number entered by the user.

URBAN EDGE

Chapter 3 - Strings

String is a data type in Python.
String is a sequence of characters enclosed in quotes.
We can primarily, write a string in these three ways

1. Single quoted strings → a = 'Harry'
2. Double quoted strings → b = "Harry"
3. Triple quoted strings → c = '''Harry'''

String Slicing

A String in Python can be sliced for getting a part of the string.
Consider the following string:

name = "H a r r y" ⇒ length = 5

	H	a	r	r	y
	0	1	2	3	4
	(-5)	(-4)	(-3)	(-2)	(-1)

The index in a string starts from 0 to (length-1) in Python. In order to slice a string, we use the following syntax:

sl = name [ind_start : ind_end]
first index included — last index is not included

sl [0:3] returns "Har" → characters from 0 to 3
sl [1:3] returns "ar" → characters from 1 to 3

Negative Indices: Negative Indices can also be used as shown in the figure above. -1 corresponds to the (length-1) index, -2 to (length-2)

URBAN EDGE

Slicing with skip value
We can provide a skip value as a part of our slice like this :

```
word = "amazing"
word [1:6:2] → 'mzn'
```

Other advanced slicing techniques

```
word = "amazing"
word [:7] → word [0:7] → 'amazing'
word [0:] → word [0:7] → 'amazing'
```

String functions
Some of the mostly used functions to perform operations on or manipulate strings are :

1> len() function → This function returns the length of the string

len ("Harry") → returns 5

2> string.endswith ("xxy") → This function tells whether the variable string ends with the string "xxy" or not. if string is "Harry", it returns true for "xxy" since Harry ends with xxy

3> string.count ("c") → Counts the total number of occurence of any character

4> string.capitalize() → This function capitalizes the first character of a given string.

URBAN EDGE

5. String.find(word) - This function finds a word and returns the index of first occurence of that word in the string.

6. String.replace(oldword, newword) - This function replaces the oldword with newword in the entire string.

Escape Sequence Characters
Sequence of characters after backslash '\' → Escape Seq characters

Escape sequence character comprises of more than one characters but represents one character when used within the strings.

Examples \n, \t, \', \\ etc.
(\n → newline, \t → Tab, \' → Single quote, \\ → backslash.)

URBAN EDGE

Chapter 3 - Practice Set

1. Write a Python program to display a user entered name followed by Good Afternoon using input() function

2. Write a program to fill in a letter template given below with name and date.

```
letter = ''' Dear <|NAME|>,
                You are selected!
<|DATE|> '''
```

3. Write a program to detect double spaces in a string

4. Replace the double spaces from Problem 3 with single spaces.

5. Write a program to format the following letter using escape sequence characters.

letter = "Dear Harry, This Python course is nice. Thanks!"

Chapter 4 - Lists and Tuples

Python Lists are containers to store a set of values of any data type

```
friends = ["Apple", "Akash", "Rohan", 7, False]
```

"Apple" → str()
"Akash" → can store value of any datatype
7 → int()
False → bool()

List Indexing

A List can be indexed just like a String

```
L1 = [7, 9, "Harry"]
```

```
L1[0]  => 7
L1[1]  => 9
L1[70] => Error
L1[0:2] => [7, 9]   => List Slicing
```

List Methods

Consider the following list:

```
L1 = [1, 8, 7, 2, 21, 15]
```

1. L1.sort(): updates the list to [1, 2, 7, 8, 15, 21]
2. L1.reverse(): updates the list to [15, 21, 2, 7, 8, 1]
3. L1.append(8): adds 8 at the end of the list
4. L1.insert(3,8): This will add 8 at 3 index

URBAN EDGE

5> L1.pop(2): Will delete element at index 2 and return its value

6> L1.remove(21): Will remove 21 from the list.

Tuples in Python

A tuple is an immutable data type in python.
↳ Cannot change

a = ( ) => Empty tuple
a = (1,) => Tuple with only one element needs a comma
a = (1, 7, 2) => Tuple with more than one element

Once defined a tuple's elements cant be altered or manipulated.

Tuple methods

Consider the following tuple
a = (1, 7, 2)

1> a.count(1): a.count(1) will return number of times 1 occurs in a.

2> a.index(1): a.index(1) will return the index of first occurence of 1 in a.

URBAN EDGE

Chapter 4 - Practice Set

1. Write a program to store seven fruits in a list entered by the user.

2. Write a program to accept marks of 6 students and display them in a sorted manner

3. Check that a tuple cannot be changed in python

4. Write a program to sum a list with 4 numbers.

5. Write a program to count the number of zeros in the following tuple:

$$a = (7, 0, 8, 0, 0, 9)$$

Chapter 5 : Dictionary & Sets

Dictionary is a collection of key-value pairs

Syntax :

```
a = { "key" : "value",
      "harry" : "code",
      "marks" : "100",
      "list" : [1,2,9] }
```

(keys → "key", "harry", "marks", "list"; values → "value", "code", "100", [1,2,9])

a["key"] => Prints "value"
a["list"] => Prints [1,2,9]

Properties of a Python Dictionaries

1. It is unordered
2. It is mutable
3. It is indexed
4. Cannot contain duplicate keys

Dictionary Methodes

Consider the following dictionary

```
a = { "name" : "Harry",
      "from" : "India",
      "marks" : [92,98,96] }
```

1. a.items() : returns a list of (key, value) tuples
2. a.keys() : returns a list containing dictionary's keys
3. a.update({"friend" : "Sam"}) : updates the dictionary with supplied key-value pairs

URBAN EDGE

4. a.get("name"): returns the value of the specified keys (and value is returned eg. "Harry" is returned here)

More methods are available on docs.python.org.

Sets in Python

Set is a collection of non repetitive elements

```
S = Set()                    ⇒ No repetition allowed!
S.add(1)
S.add(2)                     ⇒ or Set = {1, 2}
```

If you are a programming beginner without much knowledge of mathematical operations on sets, you can simply look at sets in python as data types containing unique values.

Properties of Sets

1. Sets are unordered ⇒ Element's order doesn't matter
2. Sets are unindexed ⇒ Cannot access elements by index
3. There is no way to change items in sets.
4. Sets cannot contain duplicate values

Operations on Sets

Consider the following Set:

S = {1, 8, 2, 3}

1. len(s): Returns 4, the length of the set
2. S.remove(8): Updates the set S and removes 8 from S.

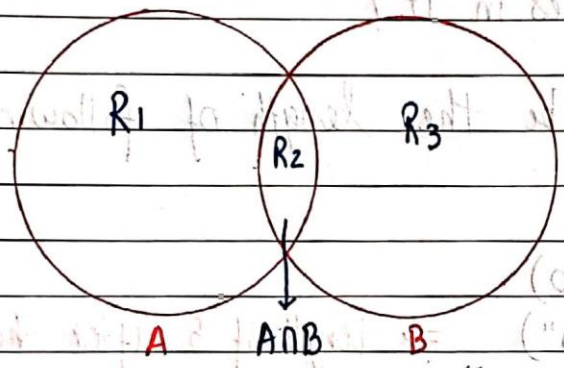
URBAN EDGE

3. S.pop() : Removes an arbitrary element from the set and returns the element removed

4. S.clear() : Empties the set S

5. S.union({8,11}) : Returns a new set with all items from both sets. => {1,8,2,3,11}

6. S.intersection({8,11}) : Returns a set which contains only items in both sets. => {8}



$R_2 \Rightarrow A \cap B$

$R_1 + R_2 + R_3 \Rightarrow A \cup B$

$R_1 + R_3 \Rightarrow A \Delta B$

$R_1 \Rightarrow A - B$

$R_3 \Rightarrow B - A$

URBAN EDGE

Chapter 5: Practice Set

1. Write a program to create a dictionary of Hindi words with values as their english translation. Provide user with an option to look it up!

2. Write a program to input eight numbers from the user and display all the unique numbers (once).

3. Can we have a set with 18 (int) and "18" (str) as a values in it?

4. What will be the length of following set S:
```
S = Set()
S.add(20)
S.add(20.0)
S.add("20")
```
=> Length of S after these operations?

5. S = {}
what is the type of S?

6. Create an empty dictionary. Allow 4 friends to enter their favourite language as values and use keys as their names. Assume that the names are unique

7. If names of 2 friends are same; what will to the program in Problem 6?

8. If languages of two friends are same; what will happen to the program in Problem 6?

9 Can you change the values inside a list which is contained in Set S

$S = \{8, 7, 12, \text{"Harry"}, [1, 2]\}$

Chapter 6 - Conditional Expressions

Sometimes we want to play pubG on our phone if the day is Sunday.
Sometimes we order Icecream online if the day is Sunny.
Sometimes we go hiking if our parents allow.

All these are decisions which depends on a condition being met.
In Python programming too, we must be able to execute instructions on a condition(s) being met. This is what conditionals are for!

If else and elif in Python

If else and elif statements are a multiway decision taken by our program due to certain conditions in our code.

Syntax:

```
if (condition1):                    => if condition 1 is true
    print ("yes")
elif (condition2):                  => if condition 2 is true
    print ("No")
else:                               => otherwise
    print ("May be")
```

Indentation ←

Code example:

```
a = 22
if(a>9):
    print("Greater")
else:
    print("lesser")
```

Quick Quiz: Write a program to print yes when the age entered by the user is greater than or equal to 18.

Relational Operators

Relational operators are used to evaluate conditions inside the if statements. Some examples of relational operators are:

- == → equals
- >= → greater than/equal to
- <=, etc.

Logical operators

In python logical operators operate on conditional statements. Example:

- and → true if both operands are true else false
- or → true if at least one operand is true else false
- not → inverts true to false & false to true

elif clause

elif in python means [else if]. An if statement can be chained together with a lot of these elif statements followed by an else statement

```
if (Condition1):
    # code
elif (condition 2):
    # code
elif (condition 3):
    # code
:
:
else:
    # code
```

⇒ This ladder will stop once a condition in an if or elif is met.

Important notes:

1. There can be any number of elif statements.
2. last else is executed only if all the conditions inside elifs fail.

URBAN EDGE

Chapter 6 - Practice Set

1. Write a program to find greatest of four numbers entered by the user.

2. Write a program to find out whether a student is pass or fail, if it requires total 40% and at least 33% in each subject to pass. Assume 3 subjects and take marks as an input from the user.

3. A spam comment is defined as a text containing following keywords:
"make a lot of money", "buy now", "subscribe this", "click this". Write a program to detect these spams.

4. Write a program to find whether a given username contains less than 10 characters or not.

5. Write a program which finds out whether a given name is present in a list or not.

6. Write a program to calculate the grade of a student from his marks from the following scheme:

90 - 100	→	Ex
80 - 90	→	A
70 - 80	→	B
60 - 70	→	C
50 - 60	-	D
<50	-	F

URBAN EDGE

7. Write a program to find out whether a given post is talking about "Harry" or not.

URBAN EDGE

Chapter 7 - Loops in Python

Sometimes we want to repeat a set of statements in our program. For instance: Print 1 to 1000

Loops make it easy for a programmer to tell the computer, which set of instructions to repeat and how!

Types of loops in Python

Primarily there are two types of loops in Python

1. While loop
2. For loop

We will look into these one by one!

While loop

The syntax of a while loop looks like this:

```
While Condition:
    #Body of the loop
```

⇒ The block keeps executing until the condition is true

In while loops, the condition is checked first. If it evaluates to true, the Body of the loop is executed, otherwise not!
If the loop is entered, the process of [Condition check & execution] is continued until the condition becomes false.

Quick Quiz: Write a program to print 1 to 50 using a while loop.

URBAN EDGE

An Example

```
i = 0
while i < 5:
    print("Harry")
    i = i + 1
```

⇒ Prints "Harry" - 5 times!

Note: If the condition never becomes False, the loop keeps getting executed.

Quick Quiz: Write a program to print the content of a list using while loops.

For loop

A for loop is used to iterate through a sequence like list, tuple or string [Iterables]

The syntax of a for loop looks like this:

```
l = [1, 7, 8]
for item in l:
    print(item)
```

→ print 1, 7 and 8

Range function in Python

The range function in python is used to generate a sequence of numbers.
We can also specify the start, stop and step-size as follows:

```
range(start, stop, step_size)
```

↳ step size is usually not used with range()

An Example demonstrating range() function

```
for i in range(0, 7):
    print(i)
```

→ range(7) can also be used
→ prints 0 to 6

For loop with else

An optional else can be used with a for loop if the code is to be executed when the loop exhausts

Example :

```
l = [1, 7, 8]
for item in l:
    print(item)
else:
    print("Done")
```

→ This is printed when the loop exhausts!

Output :

```
1
7
8
Done
```

The break statement

'break' is used to come out of the loop when encountered
It instructs the program to - Exit the loop now

Example :

```
for i in range(0,80):
    print(i)
    if i==3:
        break
```

→ This will print 0, 1, 2 and 3

URBAN EDGE

The Continue Statement

'Continue' is used to stop the current iteration of the loop and continue with the next one. It instructs the program to "skip this iteration"

Example:

```
for i in range(4):
    print("printing")
    if i == 2:
        continue
    print(i)
```

⇒ if i is 2, the iteration is skipped.

pass statement

pass is a null statement in python. It instructs to "Do nothing"

Example:

```
l = [1, 7, 8]
for item in l:
    pass
```

→ Without pass, the program will throw an error

URBAN EDGE

Chapter 7 - Practice Set

1. Write a program to print multiplication table of a given number using for loop.

2. Write a program to greet all the person names stored in a list l1 and which starts with S

$l_1 = [$"Harry", "Soham", "Sachin", "Rahul"$]$

3. Attempt problem 1 using while loop.

4. Write a program to find whether a given number is prime or not

5. Write a program to find the Sum of first n natural numbers using while loop.

6. Write a program to calculate the factorial of a given number using for loop.

7. Write a program to print the following star pattern

```
    *
  * * *
* * * * *
```
for n = 3

8. Write a program to print the following star pattern:

```
*
* *
* * *
```
for n = 3

URBAN EDGE

9 Write a program to print the following star pattern

```
* * *
*   *
* * *
```

for $n = 3$

10 Write a program to print multiplication table of n using for loop in reversed order.

URBAN EDGE

Chapter 8 - Functions & Recursions

A function is a group of statements performing a specific task.

When a program gets bigger in size and its complexity grows, it gets difficult for a programmer to keep track on which piece of code is doing what!

A function can be reused by the programmer in a given program any number of

Example and Syntax of a function
The syntax of a function looks as follows:

```
def func1():
    print("Hello")
```

This function can be called any number of times, anywhere in the program.

Function Call
Whenever we want to call a function, we put the name of the function followed by parenthesis as follows:

```
func1()
```

→ This is called function call

function definition
The part containing the exact set of instructions which are executed during the function call.

Quick Quiz: Write a program to greet a user with "Good day" using functions.

Types of functions in Python
There are two types of functions in Python:
1. Built in functions → Already present in Python
2. User defined functions → Defined by the user

Examples of built in function includes len(), print(), range() etc.

The funcI() function we defined is an example of user defined function

Functions with arguments
A function can accept some values it can work with. We can put these values in the parentheses
A function can also return values as shown below:

```
def greet(name):
    gr = "Hello " + name
    return gr

a = greet("Harry")
```

→ "Harry" is passed to greet in name

↳ a will now contain "Hello Harry"

Default Parameter Value
We can have a value as default argument in a function.
If we specify name = 'stranger' in the line containing def, this value is used when no argument is passed.

URBAN EDGE

For example:

```
def greet(name = "Stranger"):
    # function body
```

greet() → Name will be "Stranger" in function body (default)
greet("Harry") → Name will be "Harry" in function body (passed)

Recursion

Recursion is a function which calls itself
It is used to directly use a mathematical formula as a function. For example:

factorial(n) = n × factorial(n-1)

This function can be defined as follows:

```
def factorial(n):
    if i==0 or i==1: → Base condition which doesn't call the function any further
        return 1
    else:
        return n * factorial(n-1) → function calling itself
```

This works as follows:

Factorial(4)
↓ ↳ [Function called]
4 × factorial(3)
4 × [3 ↓ × factorial(2)]
4 × 3 × [2 ↓ × factorial(1)]
4 × 3 × 2 × [1] [Function returned]

The programmer need to be extremely careful while working with recursion to ensure that the function doesnt Infinitely keep calling itself.
Recursion is sometimes the most direct way to code an algorithm

Chapter 8 - Practice Set

1. Write a program using function to find greatest of three numbers

2. Write a python program using function to convert Celsius to fahrenheit.

3. How do you prevent a python print() function to print a new line at the end.

4. Write a recursive function to calculate the sum of first n natural numbers.

5. Write a python function to print first n lines of the following pattern:

```
* * *
* *        → For n = 3
*
```

6. Write a python function which converts inches to cms

7. Write a python function to remove a given word from a list and strip it at the same time.

8. Write a python function to print multiplication table of a given number.

URBAN EDGE

Project 1: Snake, Water, Gun Game

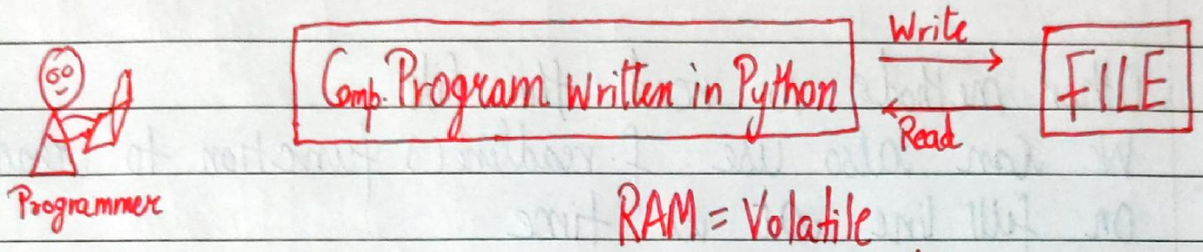
We all have played snake, water gun game in our childhood. If you haven't, google the rules of this game and write a Python program capable of playing this game with the user.

URBAN EDGE

Chapter 9 - File I/O

The random Access memory is volatile and all its contents are lost once a program terminates
In order to persist the data forever, we use files

A file is data stored in a storage device. A Python program can talk to the File by reading content from it and writing content to it



RAM = Volatile
HDD = Non Volatile

Types of Files

There are 2 types of files:

1. Text files (.txt, .c etc)
2. Binary files (.jpg, .dat, etc)

Python has a lot of functions for reading, updating and deleting files.

Opening a file

Python has an open() function for opening files. It takes 2 parameters: filename and mode.

```
open("this.txt", "r")
```

- open is a built-in function
- "this.txt" → Filename
- "r" → mode of opening (read mode)

URBAN EDGE

Reading a file in python

```
f = open("this.txt", "r")  → open the file in r mode
text = f.read()            → Read its contents
print(text)                → Print its contents
f.close()                  → Close the file
```

We can also specify the number of characters in read() function : f.read(2)
↳ Reads first 2 characters

Other methods to read the file
We can also use f.readline() function to read on full line at a time

f.readline() → Reads one line from the file

Modes of opening a file

r → open for reading
w → open for writing
a → open for appending
+ → open for updating

'rb' will open for read in binary mode
'rt' will open for read in text mode

Writing files in Python
In order to write to a file, we first open it in write or append mode after which, we use the python's f.write() method to write to the file!

```
f = open("this.txt", "w")
f.write("This is nice")
f.close()
```

→ Can be called multiple times

With statement

The best way to open and close the file automatically is the with statement

```
with open("this.txt") as f:
    f.read()
```

↳ Dont need to write f.close() as it is done automatically.

URBAN EDGE

Chapter 9 - Practice Set

1. Write a program to read the text from a given file 'poems.txt' and find out whether it contains the word 'twinkle'.

2. The game() function in a program lets a user play a game and returns the score as an integer. You need to read a file 'History.txt' which is either blank or contains the previous Hi-score. You need to write a program to update the Hi-Score whenever game() breaks the Hi-Score.

3. Write a program to generate multiplication tables from 2 to 20 and write it to the different files. Place these files in a folder for a 13-year old.

4. A file contains a word "Donkey" multiple times. You need to write a program which replaces this word with ###### by updating the same file.

5. Repeat program 4 for a list of such words to be censored.

6. Write a program to mine a log file and find out whether it contains 'python'.

7. Write a program to find out the line number where python is present from Ques 6

URBAN EDGE

8 Write a program to make a copy of a text file "this.txt"

9 Write a program to find out whether a file is identical & matches the content of another file.

10 Write a program to wipe out the contents of a file using python

11 Write a python program to rename a file to "renamed_by_python.txt"

URBAN EDGE

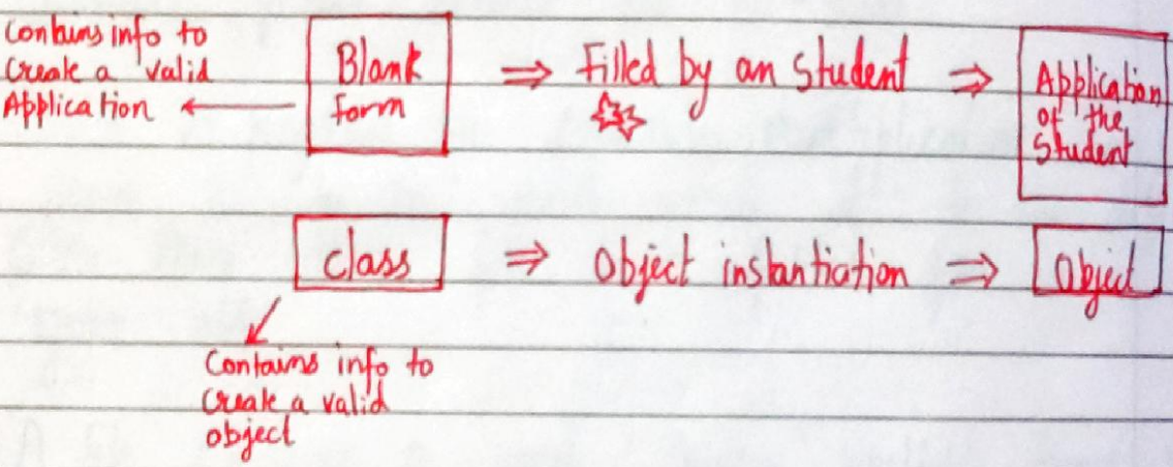
Chapter 10 - Object Oriented Programming

Solving a problem by creating objects is one of the most popular approaches in programming. This is called Object oriented programming.

This concept focuses on using reusable code.
↳ Implements DRY principle

Class

A class is a blueprint for creating objects.



The syntax of a class looks like this:

```
Class Employee:
    # methods & variables
```

[Classname is written in PascalCase]

Object

An Object is an instantiation of a class. When class is defined, a template (info) is defined. Memory is allocated only after object instantiation.

Objects of a given class can invoke the methods available to it without revealing the implementation details to the user. → Abstraction & Encapsulation!

URBAN EDGE

Modelling a problem in OOPs

We identify the following in our problem

Noun → Class → Employee
Adjective → Attributes → name, age, salary
Verbs → Methods → getSalary(), increment()

Class Attributes

An attribute that belongs to the class rather than a particular object.

Example :

```
Class Employee :
    company = "Google"      → [Specific to each class]

harry = Employee()          → object instantiation
harry.company
Employee.company = "YouTube" → changing class attribute
```

Instance Attributes

An attribute that belongs to the Instance (object)
Assuming the class from the previous example :

```
harry.name = "Harry"
harry.salary = "30K"        ⇒ Adding instance attributes
```

Note : Instance attributes take preference over class attributes during assignment & retrieval

harry.attribute1 → ① Is attribute1 present in object ?
② Is attribute1 present in class ?

'self' parameter
self refers to the instance of the class
It is automatically passed with a function call from an object

```
harry.getSalary()
```
→ here self is harry
↳ equivalent to Employee.getSalary(harry)

The function getsalary is defined as:

```
class Employee:
    Company = "Google"
    def getSalary(self):
        print("Salary is not there")
```

Static method
Sometimes we need a function that doesnt use the self parameter. we can define a static method like this:

```
@staticmethod
def greet():
    print("Hello user")
```
→ decorator to mark greet as a static method

__init__() Constructor
__init__() is a special method which is first run as soon as the object is created.
__init__() method is also known as constructor

It takes self argument and can also take further arguments

URBAN EDGE ___/___/___

For Example:

```
Class Employee:
    def __init__(self, name):
        self.name = name

    def getSalary(self):
        ...

harry = Employee("Harry")
```

↳ Object can be instantiated using constructor like this!

URBAN EDGE

Chapter 10 - Practice Set

1. Create a class programmer for storing information of few programmers working at microsoft.

2. Write a class Calculator capable of finding square, cube and squareroot of a number.

3. Create a class with a class attribute a; create an object from it and set a directly using object.a = 0. Does this change the class attribute?

4. Add a static method in problem 2 to greet the user with hello.

5. Write a class Train which has methods to book a ticket, get status (no of seats) and get fare information of trains running under Indian Railways.

6. Can you change the self parameter inside a class to something else (say 'harry'). Try changing self to 'slf' or 'harry' and see the effects.

URBAN EDGE

Chapter 11 - Inheritance & more on OOPs

Inheritance is a way of creating a new class from an existing class

Syntax:

```
Class Employee:                    → Base Class
    # Code
    ...

Class Programmer (Employee):       → Derived or child class
    # Code
```

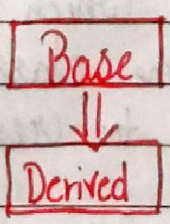
We can use the methods and attributes of Employee in Programmer object.
Also, we can overwrite or add new attributes and methods in Programmer class.

Types of Inheritance

1. Single inheritance
2. Multiple inheritance
3. Multilevel inheritance

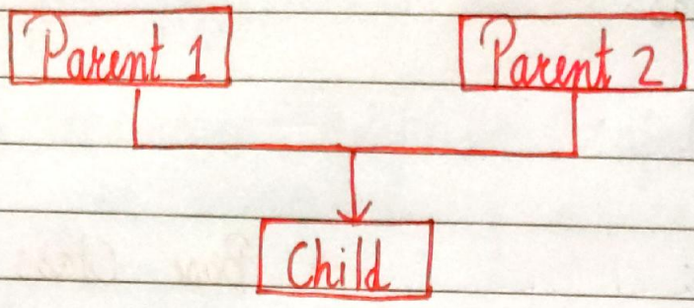
Single Inheritance

Single inheritance occurs when child class inherits only a single parent class



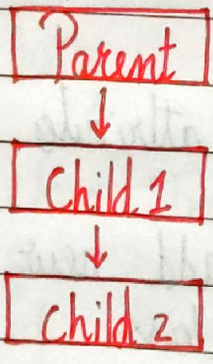
Multiple Inheritance

Multiple inheritance occurs when the child class inherits from more than one parent class.



Multilevel Inheritance

When a child class becomes a parent for another child class



Super() method

Super method is used to access the methods of a super class in the derived class

```
super().__init__()
```

↳ Calls constructor of the base class

Class methods

A class method is a method which is bound to the class and not the object of the class.

@classmethod decorator is used to create a class method

Syntax to create a class method:

```
@classmethod
def (cls, p1, p2):
    ...
```

@property decorators
Consider the following class

```
class Employee:
    @property
    def name(self):
        return self.ename
```

if e = Employee() is an object of class employee, we can print(e.name) to print the ename/call name() function.

@.getters and @.setters
The method name with @property decorator is called getter method
We can define a function + @name.setter decorator like below:

```
@name.setter
def name(self, value):
    self.ename = value
```

Operator overloading in Python
Operators in python can be overloaded using dunder methods.
These methods are called when a given operator is used on the objects.

Operators in python can be overloaded using the following methods:

$p_1 + p_2 \rightarrow p_1.__add__(p_2)$

$p_1 - p_2 \rightarrow p_1.__sub__(p_2)$

$p_1 * p_2 \rightarrow p_1.__mul__(p_2)$

$p_1 / p_2 \rightarrow p_1.__truediv__(p_2)$

$p_1 // p_2 \rightarrow p_1.__floordiv__(p_2)$

Other dunder/magic methods in python

`__str__()` → used to set what gets displayed upon calling `str(obj)`

`__len__()` → used to set what gets displayed upon calling `.__len__()` or `len(obj)`

URBAN EDGE

Chapter 11 - Practice Set

1. Create a class C2dVector and use it to create another class representing a 3-d vector.

2. Create a class pets from a class Animals and further create class Dog from Pets. Add a method bark to class Dog.

3. Create a class Employee and add salary and increment properties to it.
Write a method salaryAfterIncrement method with a @property decorator with a setter which changes the value of increment based on the salary.

4. Write a class Complex to represent complex numbers, along with overloaded operators + and * which adds and multiplies them

5. Write a class vector representing a vector of n dimension. Overload the + and * operator which calculates the sum and the dot product of them.

6. Write __str__() method to print the vector as follows:

$$7\hat{i} + 8\hat{j} + 10\hat{k}$$

Assume vector of dimension 3 for this problem.

URBAN EDGE

7 Override the __len__() method on Vector of problem 5 to display the dimension of the Vector.

URBAN EDGE

Project 2 - The Perfect Guess

We are going to write a program that generates a random number and asks the user to guess it.
If the player's guess is higher than the actual number, the program displays "Lower number please".
Similarly if the user's guess is too low, the program prints "higher number please"
When the user guesses the correct number, the program displays the number of guesses the player used to arrive at the number.

Hint: Use the random module

Chapter 12 - Advanced Python 1

Exception Handling in Python

There are many built-in exceptions which are raised in Python when something goes wrong.
Exceptions in Python can be handled using a try statement. The code that handles the exception is written in the except clause.

```
try:
    # Code                          → Code which might throw Exception
except Exception as e:
    print(e)
```

When the exception is handled, the code flow continues without program interruption.

We can also specify the exceptions to catch like below:

```
try:
    #Code
except ZeroDivisionError:
    # Code
except TypeError:
    # code
except:
    # code          → All other exceptions are handled here.
```

Raising Exceptions

We can raise custom exceptions using the raise keyword in python.

try with else clause
Sometimes we want to run a piece of code when try was successful.

```
try:
    # Some code
except:
    # Some code
else:
    # Code        → This is executed only if the try was successful
```

try with finally
Python offers a finally clause which ensures execution of a piece of code irrespective of the exception.

```
try:
    # Some code
except:
    # Some code
finally:
    # Some code    → executed regardless of error!
```

if __name__ == '__main__' in Python
__name__ evaluates to the name of the module in Python from where the program is ran

If the module is being run directly from the command line, the __name__ is set to string "__main__"
Thus this behaviour is used to check whether the module is run directly or imported to another file.

URBAN EDGE

The global keyword

global keyword is used to modify the variable outside of the current scope.

Enumerate function in Python

The enumerate function adds counter to an iterable and returns it

```
for i, item in list1:
    print(i, item)
```

↳ Prints the items of list 1 with index!

List Comprehensions

list comprehension is an elegant way to create lists based on existing lists

```
list1 = [1, 7, 12, 11, 22]

list2 = [i for item in list1 if item > 8]
```

URBAN EDGE

Chapter 12 - Practice Set

1. Write a program to open three files 1.txt, 2.txt and 3.txt. If any of these files are not present, a message without exiting the program must be printed prompting the same.

2. Write a program to print third, fifth and seventh element from a list using enumerate function

3. Write a list comprehension to print a list which contains the multiplication table of a user entered number.

4. Write a program to display a/b where a and b are integers. If b=0, display Infinite by handling the ZeroDivisionError.

5. Store the multiplication tables generated in Problem 3 in a file named Tables.txt.

URBAN EDGE

Chapter 13 - Advanced Python 2

Virtual Environment

An environment which is same as the System interpretor but is isolated from the other python environments on the system.

Installation

To use virtual environments, we write

```
pip install virtualenv → Install the package
```

We create a new environment using:

```
virtualenv myprojectenv → Creates a new venv
```

The next step after creating the virtual environment is to activate it.
We can now use this virtual environment as a separate python installation.

pip freeze command

pip freeze returns all the packages installed in a given python environment along with the versions

```
" pip freeze > requirements.txt "
```

The above command creats a file named requirements.txt in the same directory containing the output of pip freeze.
We can distribute this file to other users and they can recreate the same environment using:

URBAN EDGE

```
pip install -r requirements.txt
```

Lambda functions

functions created using an expression using lambda keyword

Syntax:

```
lambda arguments : expressions
```

↳ can be used as a normal function

Example:

```
Square = lambda x : x*x
Square(6)   → returns 36

Sum = lambda a,b,c : a+b+c
Sum(1,2,3)  → returns 6
```

.join method (strings)

creates a string from iterable objects

```
l = ["apple", "mango", "banana"]

",and,".join(l)
```

The above line will return "apple,and,mango,and,banana"

format method (strings)

formats the values inside the string into a desired output

```
template.format(p1, p2 ...)
```

↳ arguments

Syntax for format looks like:

"{} is a good {}".format("Harry", "boy") —①

"{1} is a good {0}".format("Harry", "boy") —②

Output for ①
Harry is a good boy

Output for ②
boy is a good Harry

Map, Filter & Reduce
Map applies a function to all the items in an input_list

Syntax:

map(function, input_list)
↳ Can be lambda function

Filter creates a list of items for which the function returns true.

list(filter(function))
↳ Can be a lambda function

Reduce applies a rolling computation to sequential pair of elements

```
from functools import reduce
val = reduce(function, list1)
```
↳ Can be a lambda function

If the function computes sum of two numbers and the

list is [1, 2, 3, 4]

1 2 3 4

3 3 4

6 4

10

=> Sequential Computation

URBAN EDGE

Chapter 13 - Practice Set

1. Create two virtual environments, install few packages in the first one. How do you create a similar environment in the second one?

2. Write a program to input name, marks and phone number of a student and format it using the format function like below:

"The name of the student is Harry, his marks are 72 and phone number is 99999888"

3. A list contains the multiplication table of 7. Write a program to convert it to a vertical string of same numbers (7 / 14 / ⋮)

4. Write a program to filter a list of numbers which are divisible by 5

5. Write a program to find the maximum of the numbers in a list using the reduce function.

6. Run pip freeze for the system interpreter. Take the contents and create a similar virtualenv.

7. Explore the Flask module and create a web server using Flask & Python.

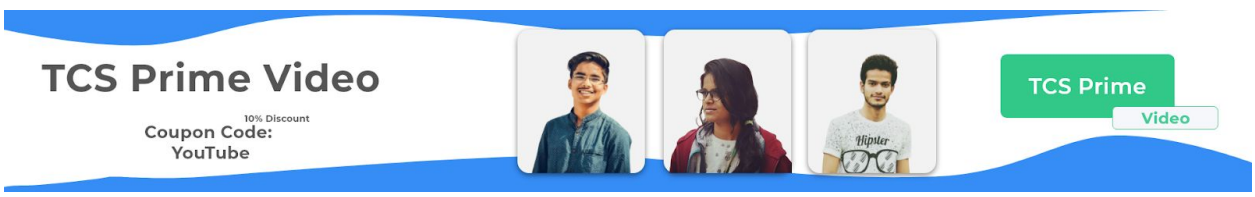
URBAN EDGE

Project 3 - Student Library

Implement a student library system using OOPs where students can borrow a book from the list of books.
Create a separate Library and Student class.
Your program must be menu driven.
You are free to choose methods and attributes of your choice to implement this functionality.

Prepare Instantly

PrepInsta



TCS NQT Hiring 2020-21

TCS NQT is finally here with once again as the India's largest recruitment platform, dates for drive will be Oct 24, 25, and 26 2020. We have came up with TCS NQT Handbook to help you with TCS NQT Registration, Eligibility Criteria, Test Pattern, Syllabus as well as Model Sample Paper for TCS NQT.

Content	Page
1. Introduction to TCS NQT	1
2. Important Dates	1
3. Eligibility Criteria for TCS NQT	1
4. TCS Campus Hiring for 2021 and Eligibility Criteria	2
5. Registration for TCS NQT and TCS Campus Hiring for 2021	2
6. Test Pattern	4
7. Detailed Syllabus	4
8. Model Sample Paper for TCS NQT	5
8.1 Verbal Ability	5
8.2 Reasoning Ability	9
8.3 Numerical Ability	15
8.4 Programming Logic	21
8.5 Coding	26

PrepInsta

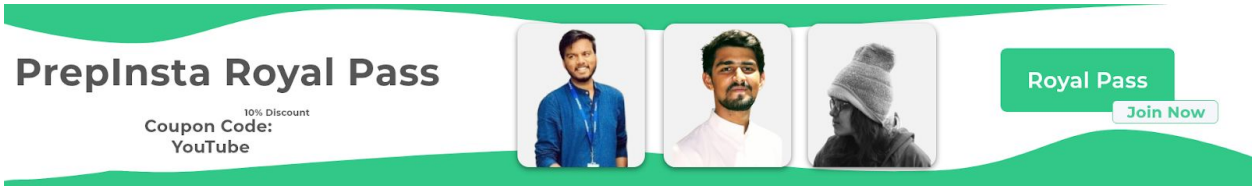
1. About the TCS NQT

The TCS NQT (National Qualifier Test) is a test that's used as a gateway to jobs across various sectors and organizations, the most important one being TCS. Until last year, this test was exclusively open only for final year students and for recruiting only for TCS. This year, however, there are some exciting changes and the scope of the NQT has expanded.

The TCS NQT is a nation-level employability test conducted by TCS iON (an assessment company, also an official subsidiary of TCS). Taking on the test provides you with a score that's valid for 2 years. It helps you apply for entry-level roles in corporate industries such as IT, FMCG, Manufacturing, BFSI, Pharma, etc. Companies such as TCS, Croma, Titan, Godrej, and more TATA Group and non-TATA Group companies are affiliated with TCS iON in recognizing the NQT score as a pathway for hiring candidates. The NQT score is also a gateway for 2021 pass-outs to apply for the TCS Campus Hiring Programme.

2. Important Dates

Registration End Date	17th October 2020
National Qualifier	24/25/26 October



Test Date	2020

3. Broader Eligibility for the TCS NQT

• UG, PG, and Diploma students in their pre-final and final year of study
• Students with any degree and specialization/discipline
• Freshers with less than 2 years of experience
• Age limit - minimum 18 years and maximum of 28 years

This NQT can be taken any number of times and the score provides a gateway to jobs across various sectors.

Note:
Please note that NQT is a test that can be taken as per the above eligibility. TCS will be using the NQT to conduct Campus Hiring for the 2021 Batch. There is separate eligibility for the latter.

4. TCS Campus Hiring YOP 2021 and Eligibility Criteria

TCS is also conducting its flagship Campus Hiring initiative of 2021 pass-outs across the country through the NQT. The primary role that TCS hires for through this Hiring Programme is nicknamed TCS Ninja. Top-performers in the NQT may also be provided an opportunity to apply for TCS Digital (the 7 LPA offer provided by TCS).

To participate in the TCS Campus Hiring of 2021 pass-outs, one must write the NQT and get a score. Here are the eligibility criteria for the TCS Campus Hiring YOP 2021.

DEGREES AND BRANCHES

BE / B.Tech / ME /M.Tech / M.Sc / MCA 2021 pass-outs from any stream.

PERCENTAGE

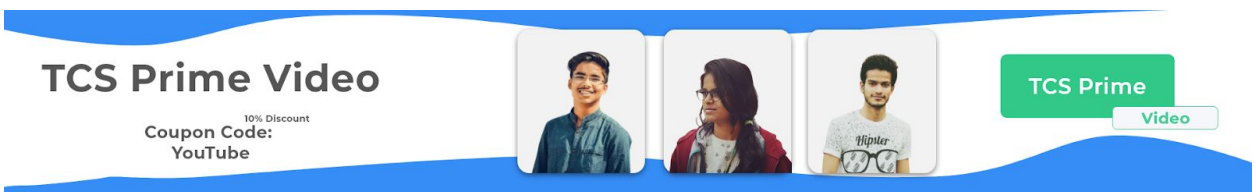
An applicant must have a minimum of 60% or 6 CGPA throughout academics and an aggregate of 60% or 6.00 CGPA in the highest qualification till the semester for which results have been declared.

BACKLOGS / ARREARS / ATKT

The applicant must not have more than 1 active backlog/arrear/ATKT while appearing for the TCS Selection process.
If selected, the student while joining TCS must have cleared all pending backlogs/arrears/ATKT within the course duration as stipulated by the University failing which joining will not be permitted.

GAP / BREAK-IN EDUCATION

It is mandatory to declare academic gaps. The overall break should not exceed 24 months. Any exceptions are permissible only for valid reasons subject to a satisfactory check of the



relevant document(s) at the time of test/interview/onboarding.

AGE

An applicant should be between 18 to 28 years of age to participate in the TCS Campus Hiring process.

COURSE TYPES

Only full-time courses will be considered (part-time / correspondence courses will not be considered).
Applicants with Secondary and/or Senior Secondary course from NIOS (National Institute of Open Schooling) are eligible to apply if the other courses are full time.

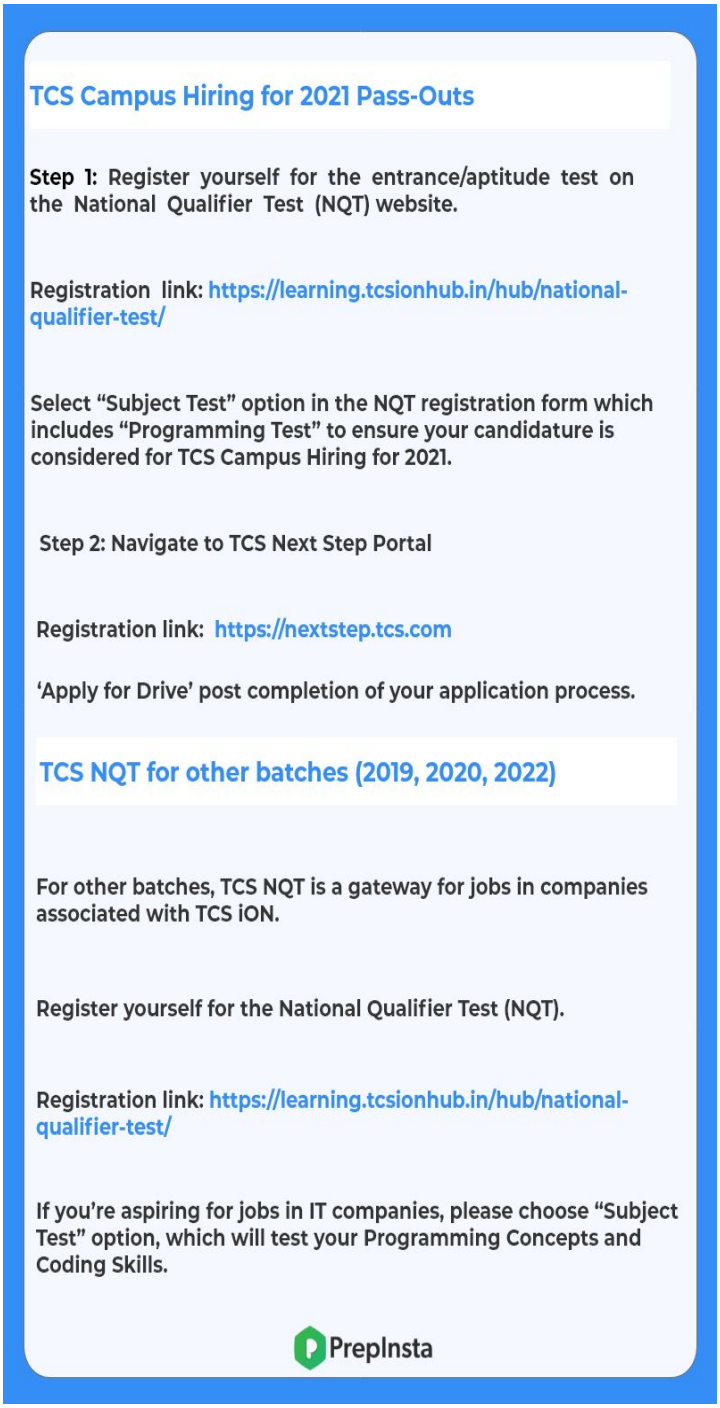
EXTENDED EDUCATION

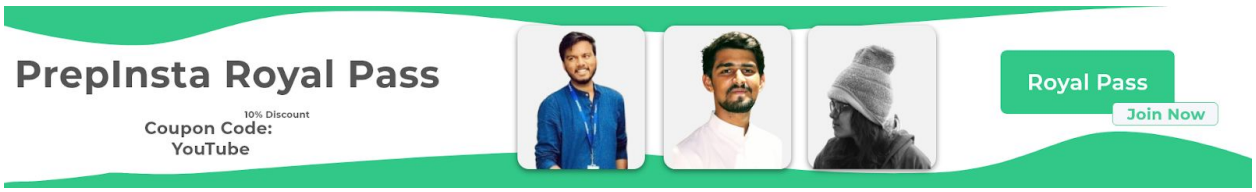
Applicants with extended education in the highest qualification are not eligible. Any exceptions are permissible only for valid reasons subject to the satisfactory check of the relevant document(s) at the time of test/interview/onboarding.

5. Registration for TCS NQT and TCS Campus Hiring YOP 2021

The Registration Process for TCS NQT and TCS Campus Hiring is different.
There are 2 steps for TCS NQT Registration
Step1:
https://learning.tcsionhub.in/hub/national-qualifier-test/

Step 2:
https://nextstep.tcs.com





PrepInsta

TCS NQT

For TCS Campus Hiring 2021

TCS NQT for TCS Campus Hiring 2021 is a online placement exam for just 2021 batch.
Students from all the branches are allowed to give this exam but Coding is compulsary in SQT.

POINT 1

2021 pass-outs from B.E./B.Tech/ M.E/M.Tech/M.Sc/MCA (all stream) are eligible

POINT 2

Hiring is for Flagship TCS Ninja role, candidates with best scores may get opportunity for TCS Digital

POINT 3

2-Step Registration

- Register for NQT on TCS iON Website
- Apply for drive on TCS Campus Commune

POINT 4

Must take up the Subject NQT (Programming Logic and Coding) along with Aptitude, Verbal, and Resoning to be eligible

Information Source:
www.prepinsta.com

PrepInsta

TCS NQT

For jobs in companies associated with TCS iON

TCS NQT for jobs in companies associated with TCS iON is a online placement exam for 2019, 2020, 2021, and 2022.
Students from all the branches are allowed to give this exam and you can select your Stream Section in Subject NQT

POINT 1

- 2019 to 2022 pass-outs from UG/PG/Diploma (any discipline and stream)
- Freshers with less than 2 year of experience

POINT 2

Jobs possible across IT (including TCS), BFSI, manufacturing, Pharma, FMCG, etc sectors through companies affiliated with TCS iON

POINT 3

Register for NQT on TCS iON Website

POINT 4

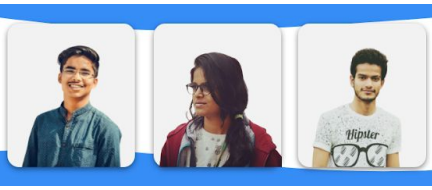
Subject NQT optional depending on weather you target IT jobs or your branch specific core companies

Information Source:
www.prepinsta.com

PrepInsta

TCS Prime Video

10% Discount
Coupon Code:
YouTube



TCS Prime Video

6. Test Pattern

The pattern of the NQT has gone through a change this year. Here is the updated pattern.

TCS NQT 2021 Test Pattern

Section	Number of Question	Duration (minutes)	What's changed from last year
Verbal Ability	24	30	For the first time, the NQT will have a longer Verbal Ability section (until previous year, it was a 10-minute section)
Reasoning Ability	30	50	Brand new section for the NQT
Numerical Ability	26	40	Rebranded from what was earlier called Quantitative Aptitude. Change in no. of Qs and duration.
Programming Logic*	10	15	Reduced duration (until last year, this was a 10 Qs, 20 min section)
Coding*	2	45 (1 Q for 15 mins and 1 Q for 30 mins)	Increase in no. of Qs and duration (Last year, it was 1Q in 30 minutes)

PrepInsta

* The Programming Logic and Coding sections are applicable only for candidates who choose the Subject NQT while registering for the NQT. The Subject NQT is mandatory to take part in the TCS Campus Hiring of 2021 pass-outs.

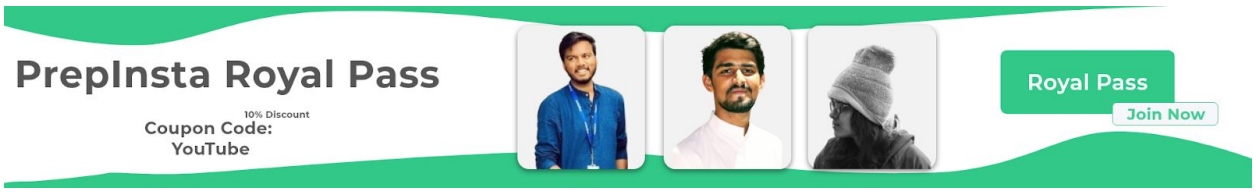
7. Detailed Syllabus

Here's the syllabus for the updated pattern of the NQT.

Verbal Ability

Until 2019, this section majorly had questions on Cloze Passage. With an expansive section this time, here are the topics that you must prepare yourself on.

- Reading Comprehension
- Grammar (Covering topics such as Subject Verb Agreement, Pronoun Agreement, Parallelism, Prepositions, Articles, Tenses, Modifiers, etc.)
- Sentence Completion



- Passage Ordering
- Cloze Passage
- Spelling
- Selecting Words
- Error Identification and Correction

Reasoning Ability

This is a brand-new section for the NQT. Here are the topics that you must focus on.

- Meaningful Word Creation
- Coding, Decoding, Number Series
- Blood Relations
- Odd Man Out
- Data Sufficiency
- Problem on Ages
- Distance and Direction Sense
- Seating Arrangements
- Statement and Conclusions
- Mathematical Operational Arrangements
- Symbols and Notations

Numerical Ability

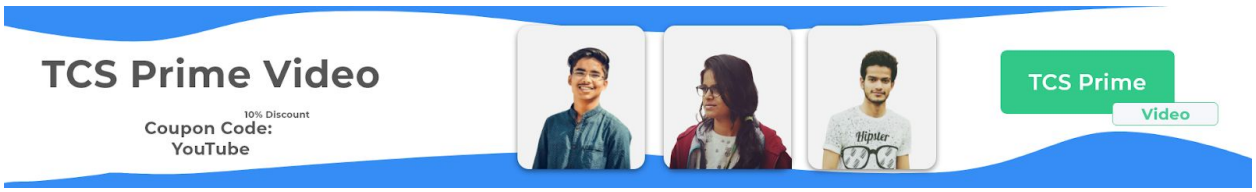
This is one of the toughest sections in the NQT. Here are topics that have been tested in previous years.

- Arrangements and Series
- Permutation and Combinations
- Number System, LCM & HCF
- Percentages
- Allegations and Mixtures
- Probability
- Ratios, Proportion and Averages
- Reasoning
- Work and Time
- Speed Time and Distance
- Geometry
- Divisibility
- Profit and Loss
- Problem on Ages
- Clocks & Calendar
- Series and Progressions
- Equations
- Averages
- Area, Shapes & Perimeter
- Numbers & Decimal Fractions

Programming Logic

Last year, this section had questions from Fundamentals of Programming (Data Types, Operators, Looping and Conditional Statements, Arrays, Strings, Functions) in C, C++, and Java.

Topics you need to prepare are:
- Data Types in C
- Functions and Scope
- Recursion and Iteration
- File Handling
- Array
- Variables and Registers
- Loops
- Command Line Arguments
- Object Oriented Programming (OOPS)
- Input Output
- Stacks and Queue
- Linked List
- Trees
- Graphs



• Dynamic Programming and Greedy
• Hashing
• Searching and Sorting
• Time Complexity

8. Model Sample TCS NQT Papers

8.1. Verbal Ability

Q1 to Q5: Fill in the blanks in the passage given below using the options provided.

India has the potential to become a cybersecurity powerhouse. We must take advantage of the fact that India is entering the demographic dividend phase and provide an adequate training opportunity for the youth. The __1__ the future cybersecurity workforce and it is important to __2__ cybersecurity courses from the ground up in the curriculum of middle school, high school, and college students. Other countries notably Israel, have already taken steps to educate the youth on cyber threats and attacks and effective __3__ against such threats. The National Skills Development Council (NSDC) has several qualification packs in Cyber Security that will create a skilled workforce. As a __4__, a course on Cyber Security covering areas like cryptography, network security, and secure application development __5__ in TCS for 500 fresh Science graduates.

Q1. Fill in the 1st blank.
A. Youths constitute
B. Youth constitutes
C. Youth constitute
D. Youths constitutes

Answer: Option B

Explanation: This can be answered with thorough knowledge of the following things.
• The plural form of youth is youths.
• The subject introduced in the sentence prior to the first blank is youth (singular, i.e. "....adequate training opportunity for the y o u t h")
• As per Subject Verb Agreement, the appropriate verb for the singular subject youth is constitutes. Similar examples include bird flies, boy runs, the movie plays, etc.

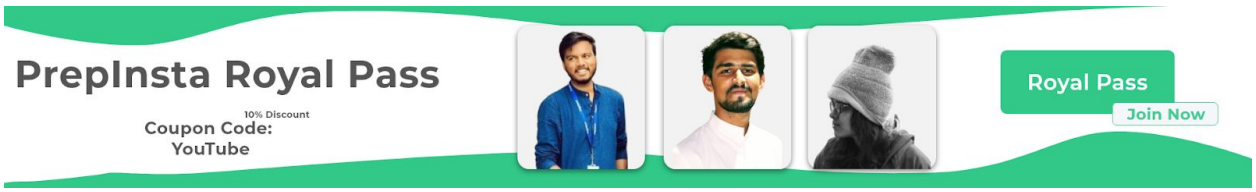
Thus, the answer is option b.

Q2. Fill in the 2nd blank.
A. Embed
B. Instill
C. Attach
D. Extend

Answer: Option A

Explanation: If keywords to help answer this blank are ground up. If one were to proactively solve this blank, we'd go for a word that's similar to incorporate.
Amongst the answer options, Embed means incorporate something, while Instill means inculcate ideas of qualities (not something that's physically tangible, but more used with qualities, i.e. intangible stuff). The other answer options won't fit in anyway, and thus the fight is between options A and B.
Since we're discussing the incorporation of something physically tangible here (i.e. cybersecurity courses), the answer is option A.

Q3. Fill in the 3rd blank.



A. Warfare
B. Countermeasures
C. Softwares
D. Viruses

Answer: Option B

Explanation: The key phrase is against such threats. Countermeasures are what you take against threats. Thus, the answer is option b.

Q4. Fill in the 4th blank.

A. Stopgap arrangement
B. Piece of the puzzle
C. Fight in this stair
D. Step in this direction

Answer: Option d

Explanation: The sentence prior to the blank talks about the NSDC having several qualification packs on Cyber Security. It is in continuation to it that TCS has a course on Cyber Security. Thus, the right answer is option d.
Stopgap arrangement: Temporary arrangement
pieces of the puzzle: An element that helps you solve a mystery

Q5. Fill in the 5th blank.

A. Is in the cockpit
B. Is piloting
C. Is being piloted
D. Are being piloted

Answer: Option C

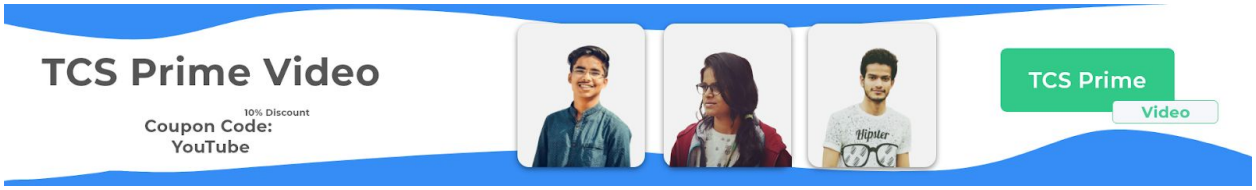
Explanation: Cockpit refers to an aircraft term, unlike piloting (which can also mean testing/ trying for the first time). Thus, option A is eliminated. The sentence here is in the passive voice. Thus, one cannot use option B (in which case, we need to know who is piloting this course). The sentence is not structured to work in the active voice form. Between options C and D, option C is the right answer since the subject (a course on Cyber Security) is singular. We need a singular verb (is being piloted).

Qs 6 to 10: Fill in the blanks in the passage given below using the options provided.
TCS business leader, Ashok Pal, recently blogged: "Too much focus on technology and too little on the operating model will, after initial success, __6__ the enterprise in the middle of a digital dip. Such a digital dip is typically caused by __7__ approach, confusion over KPIs, conflicting __8__ views, and poor user adoption. Operating model changes without adequate technology upgrade will __9__ the efficiency of the enterprise.
Remember that digital success across the breadth and depth of the enterprise is the real objective. It is much easier to succeed with smaller projects. For lasting results, technology transformation must be driven by a comprehensive IT and operation __10__ that leverage agility, analytics automation, and cloud."

Q6. Fill in the 6th blank.

A. Landed
B. Land
C. Lands
D. Left

Answer: Option B



Explanation: Here, the subject is enterprise. The appropriate verb to with the future tense auxiliary verb (will) island. While forming future tense, the syntax is always will + singular present tense form of the verb.

Q7. Fill in the 7th blank.

A. Siloed
B. Singleton
C. Bottom-top
D. Heroic

Answer: Option A

Explanation: The keywords that help you answer this question are "...too much focus on technology and too little focus on the operating model". This means there's focus only on one area, while the other is neglected. We need a word that symbolizes this meaning. That is done by siloed. It's meaning is isolated from others. In this case, it refers to an approach where other options are neglected.

Q8. Fill in the 8th blank.

A. Client
B. Stakeholder
C. Consumer
D. Rival

Answer: Option B

Explanation: This part of the paragraph talks about a dip in an organization. It cannot be caused by the client or consumer or rival views, all of whom are external agents to an organization. However, stakeholder refers to internal decision-makers and parties concerned, who can cause this digital dip. Thus, the answer is option B.

Q9. Fill in the 9th blank.

A. Maximize
B. Sophisticate
C. Constrain
D. Increase

Answer: Option C

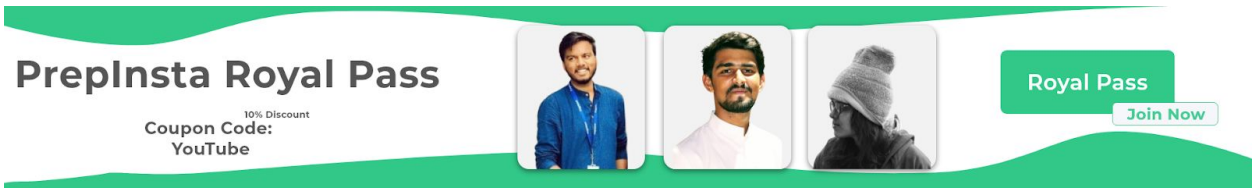
Explanation: We need an answer option that refers to something that will be done to the efficiency of the enterprise when operating model changes are without adequate technology upgrades. The technology upgrade is positive. Thus, without adequate technology upgrades, only something negative can happen to an enterprise's efficiency. The only negative answer option is option c. Constrain, by the way, refers to control or stunt.

Q10. Fill in the 10th blank.

A. Protocol
B. Pact
C. Law
D. Framework

Answer: Option D

Explanation: Protocol: A set of rules and regulations
Pact: An agreement
Framework: A structure or system
In this sentence, the option that best fits is option d, since we're looking for a word that goes with operations. Operations protocol (or) operations pact



(or) operations law does not fit well when talking about something that leverages agility, analytics automation, etc.

Q11. One part of the sentence below may contain an error. Identify the part. If there is no error, choose 'No error'.

Suma told / she would answer / all the calls later.

A. Suma told
B. She would answer
C. All the calls later
D. No error

Answer: Option A

Explanation: Whenever told is used, it must always be followed a person (told X, told Y, told Z, etc.). In this case, that person (told whom?) is missing. Thus, option A is error-strewn.

Q12 to Q14: Please go through the following passage to answer the questions that follow.

In the game of cricket, any side with a pair of bowlers of complementing superior qualities proves invincible. The current generation would know the Bhuvi-Bumrah or Starc-Cummins pairs better than their own parents. So, let me dissect a pair that ruled the cricketing world during the fifties and early sixties. Brian Statham and Fred Trueman of England proved to be a lethal combination that every batsman wanted to avoid playing. Statham was a bowler of immaculate accuracy, nagging length, and lyrical run-up. Oxymoronically, his virtues were the basis for his misfortunes. Knowing that he would bowl in a manner that's unplayable, batsmen were smugger in defending his balls than when playing an erratic bowler.
Trueman, nicknamed as Fiery, had a long delivery stride and made no bones about his intentions to test batsman's physical and mental strength. The ball would flash past the bat in moments of incredibility with the batsman thanking his Maker if the ball missed his limbs and occasionally the stumps.

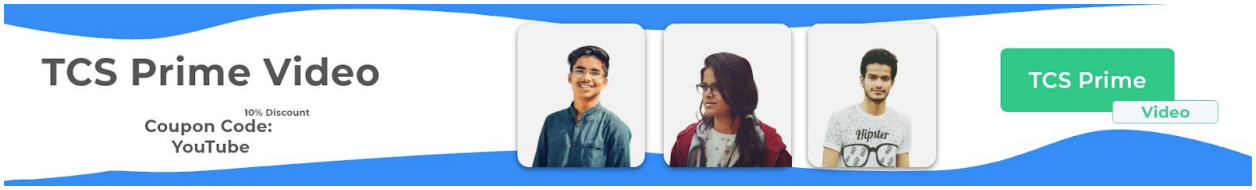
Q12. Which one of the following sentences has an appropriate bold part that can be replaced with lethal?

A. The series of never-ending bubbles rejoiced the children to an end
B. The never-say-die team won the tournament for the tenth time.
C. Mount Vesuvius' eruption proved to be completely ruinous to Pompeii.
D. The life-saving drug administered to the patient brought him back to his elements.

Answer: Option C

Explanation: The meaning of lethal is dangerous. This can be contextually deciphered by going through the sentence "...a lethal combination that every batsman wanted to avoid playing" if you didn't know the meaning of the word.
The answer option that best uses this meaning is option C. Ruinous means disastrous, which is similar to what we need.

Q13. Any team with two bowlers sharing the required traits is:
A. Arrogant to its opponents
B. Vulnerable to choke
C. Hard to defeat
D. Insignificant in its contribution



Answer: Option C

Explanation: This can be answered by going through the very first line of the passage ("In the game of cricket, any side with a pair of bowlers of complementing superior qualities prove invincible.")
Invincible: Hard to defeat

Q14. Oxymoron is a special language device, where

A. A tough idea is simplified by bringing a simple parallel to it
B. Two contrasting elements are placed side by side reinforcing an idea
C. A brilliant object issued in place of a dull entrance
D. An abstract entity is compared to an enactment emphasizing similarity/difference

Answer: Option B

Explanation: You could try answering this question through your knowledge of the word's meaning, which would point to option B. You could also try deciphering it by going through the line where this word is used in the passage ("Oxymoronically, his virtues were the basis for his misfortunes.") The line in the passage goes on to mention that a positive quality (virtue) was also negative (misfortune), thus rhyming with option B.

Here are more examples of oxymorons in the English language.
• Bittersweet
• Act naturally
• Deafening silence
• Definitely maybe

Q15. For the four-sentence (S1-S4) paragraph given below, sentences S1 and S4 are given. From the options P, Q and R, choose appropriate sentences for S2 and S3 respectively.

S1: Can we imagine carrying objects that are ten times heavier than our own body weight?
S2: _____
S3: _____
S4: This is because of their small body size; it enables them to use fewer muscles to carry their own body and more to carry bigger loads.

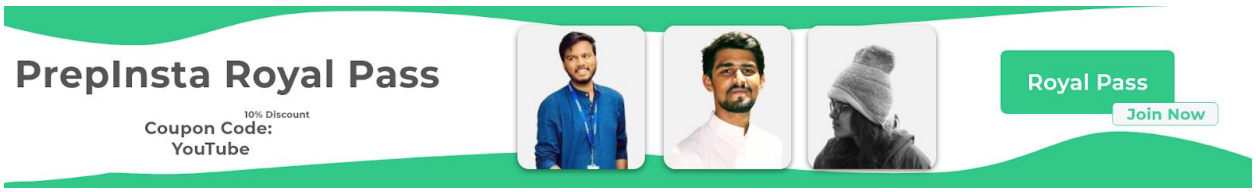
P: Ants, however, are able to carry or pull objects much heavier than their own body weight.
Q: One would assume that they have stronger muscles that enable them to have more strength than humans.
R: Superman probably could, but it does not seem possible for an ordinary man to do so.

A. PR
B. RP
C. QR
D. RQ

Answer: Option b

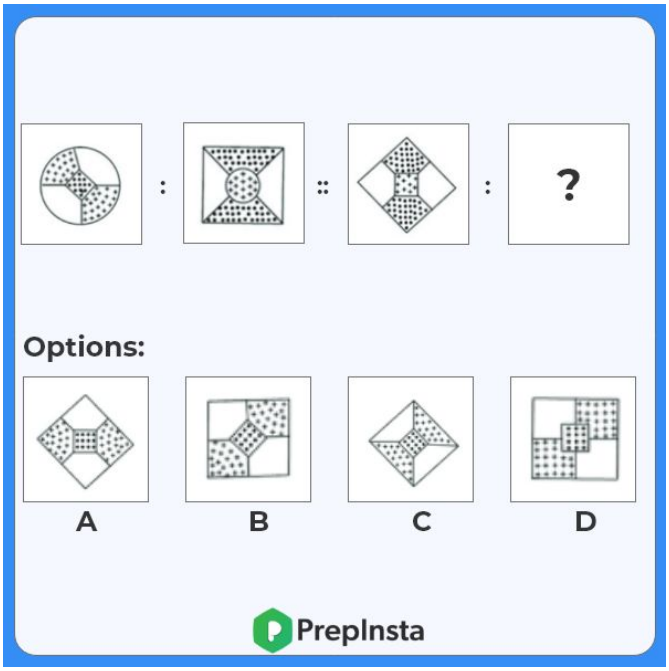
Explanation: S4 begins with "This is because of their small body size". Thus, we need a sentence before S4 that introduces a subject that will have a small body size. That is done by P (Ants is the subject). This means that P has to immediately precede S4.

The only answer option with P at the end is option B.



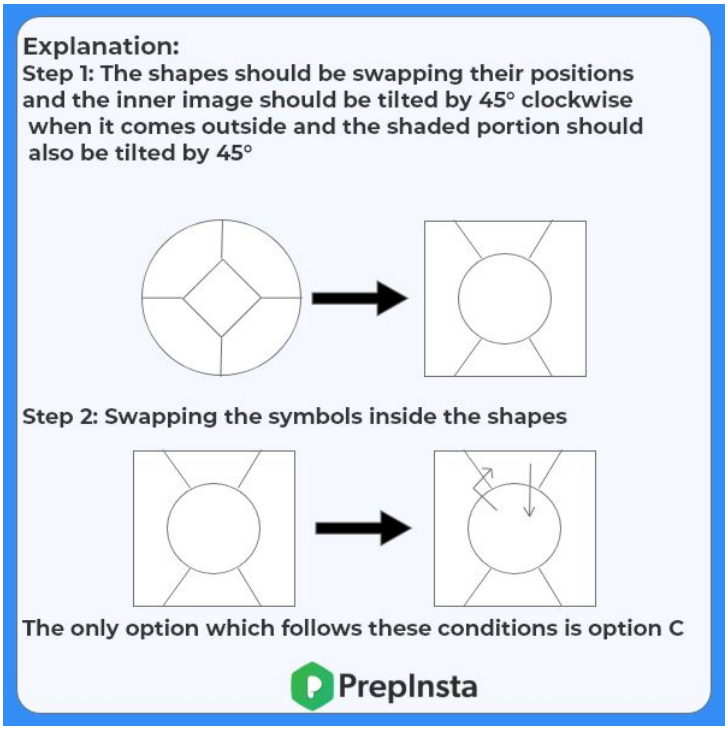
8.2. Reasoning Ability

Q1. The two figures on the left of the sign "::" are related in a certain way. The same relationship holds good for the second pair of figures on the right of the sign "::" of which, one figure is missing. Find the missing figure from the alternatives.



Answer: Option C

Explanation:

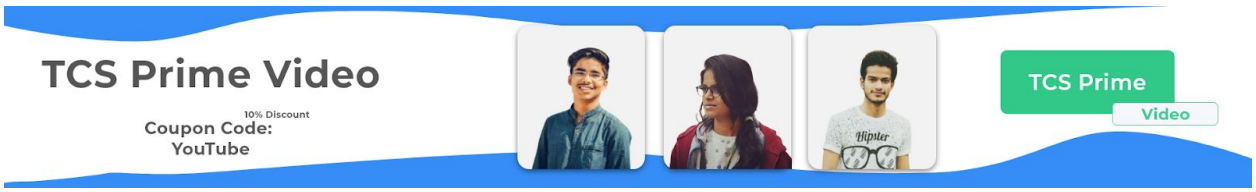


Q2. A medical representative plans to visit six doctors A, B, C, D, E and F - one doctor per day - during a week from Monday to Saturday.
i. He visited D on the day just before the day he visited B
ii. He visited C on Monday
iii. He did not visit F on either Tuesday or Saturday
iv. He visited A on the day before the day he visited E
v. He visited D on some day after he visited A

Which doctor did he visit on Saturday?

A. A
B. B
C. D
D. E

Answer: Option B



Explanation:- He visits B immediately after D (DB should be together)
- C visits on Monday(M)
- He can visit F on Wednesday(W) or Thursday(Th) or Friday(F)
- He visits E immediately after A(AE should be together)
- D should be after A(not immediately)
C AE F DB = M TW Th FS
He visited B on Saturday.

Q3. A statement is followed by two assumptions numbered I and II. Consider the statement and decide which of the given assumption(s) is/are implicit in the statement.
Statement:
In spite of several efforts by the environmentalists, some of the rarest species of birds are on the brink of extinction.

Assumptions:
I. People, in general, do not care for birds and engage in actions that threaten the survival of certain bird species.II. Whether or not environmentalists care, every species would be extinct in course of time.

A. Only I is implicit
B. Only II is implicit
C. Both I and II are implicit
D. Neither I nor II is implicit

Answer: Option D

Explanation:
Meaning of the statement given: Environmentalists put efforts to safeguard the species = But some of the rarest species of birds are still in the brink of extinction.

Assumption I - As per the statement, we do not have any information about people, in general. So this is not implicit from the statement given.
Assumption II - The meaning of the statement given is about the rarest bird species becoming extinct but this assumption is about the entire species becoming extinct, which is not in line with the given statement. So, this assumption does not implicit the statement.Neither I nor II implicate the statement.

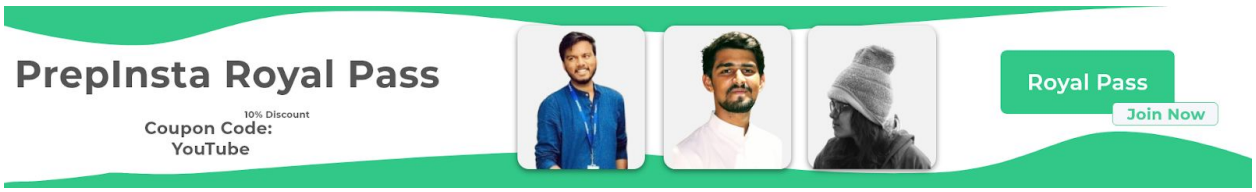
Q4. Three shops A, B and C sold cars in January, February and March. The total number of cars sold in this three-month period by A, B and C were 460, 320 and 500 respectively. Both A and C sold the same number of cars in January. The number of cars sold by A and C in March was also the same. A sold 120 cars in February which was half of the number of cars sold by C in March. Both A and B sold exactly the same number of cars in February. The ratio of the number of cars sold by B in January and March was 1:4

What was the ratio of the total number of cars sold in January, February and March by the three shops?

A. 3:5:8
B. 2:3:5
C. 3:4:5
D. 5:3:7

Answer: Option A

Explanation:
Data given -



Shops/Months	A	B	C
JAN			
FEB			
MAR			
Total	460	320	500

A on January (AJ) = C on January (CJ)
A on March (AM) = C on March (CM)
A on February (AF) = 120
AF = ½(CM)
CM = 120*2 = 240
CM = AM = 240

Shops/Months	A	B	C
JAN			
FEB	120		
MAR	240		240
Total	460	320	500

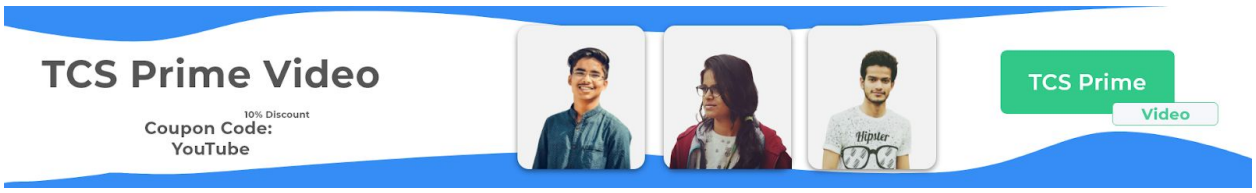
$A_J = 460 - 240 - 120 = 100 = C_J$

Shops/Months	A	B	C
JAN	100		100
FEB	120		
MAR	240		240
Total	460	320	500

$C_F = 500 - 240 - 100 = 160$
$B_F = A_F$

Shops/Months	A	B	C
JAN	100		100
FEB	120	120	160
MAR	240		240
Total	460	320	500

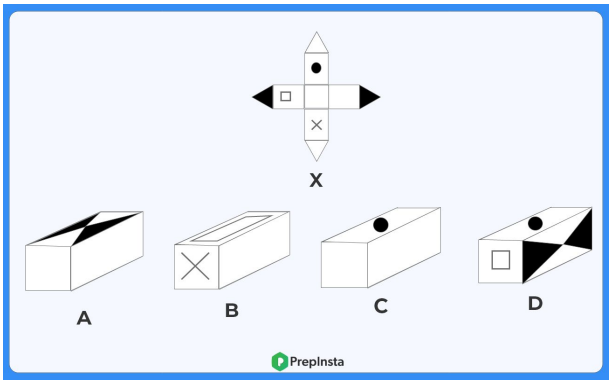
B sold 120 in February. The remaining (320-120) 200 is divided on the ratio 1:4 (Jan:Mar)
200 in 1:4 = 40:160



Shops/Months	A	B	C	Total (Row)
JAN	100	40	100	240
FEB	120	120	160	400
MAR	240	160	240	640
Total	460	320	500	

January: February: March = 240:400:640 = 3:5:8

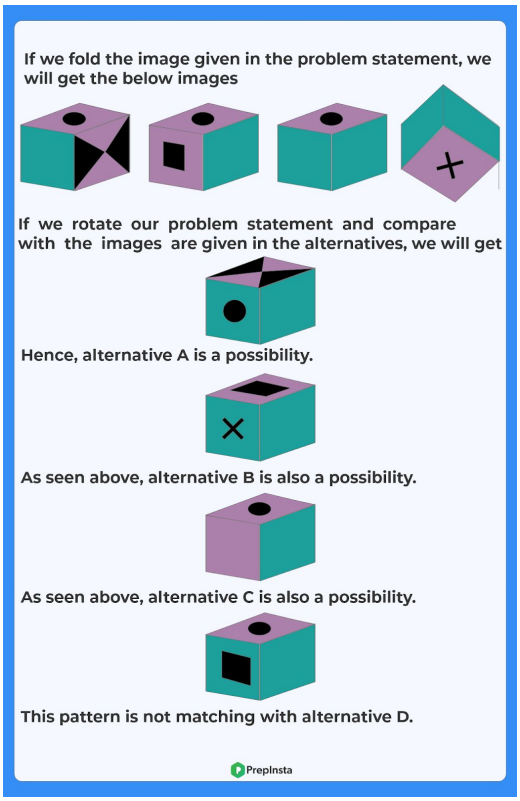
Q5. The figure marked 'X' is folded to form a box. Assuming that the paper used for the box is opaque and the signs, shaded triangles have to be visible on the outer side of the box, four possible box formations - A, B, C and D are given below. Some of which can be formed and some cannot be formed. Choose from the alternatives the boxes that can be formed.



A. A and C only
B. C and D only
C. A, B and C
D. All of A, B, C and D

Answer: Option C

Explanation: Problem statement:



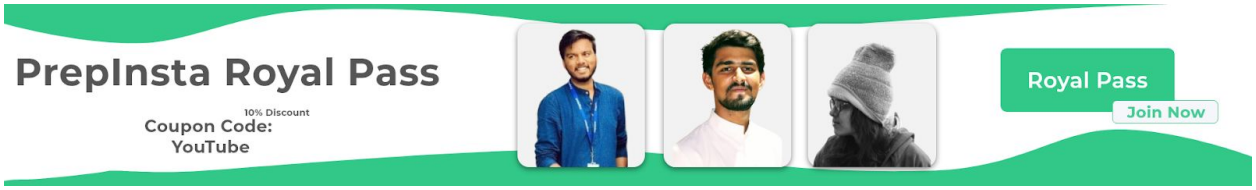
Hence, the three alternatives A, B and C can be formed.Answer is option C.

Q6. 60, 48, 38, 28, 24, 20, 18
What is the wrong number in the sequence?

A. 28
B. 38
C. 60
D. 18

Answer: Option A

Explanation:
Let's observe the pattern from right most number given in the series: 60, 48, 38, 28, 24, 20, 18
18+2 = 20



20+4 = 24
24+4 = 28
28 + 10 = 38
48 + 12 = 60

First the number is getting increased by +2 but after 20 the increment becomes 4(twice). If we observe the last two terms they are getting increased by +2. This gives us a clue that from Right to Left the term in the series should get increased by +2. But it is not happening at term 28, so it is the odd term in the given series.

The correct series would be:
18 + 2 = 20
20 + 4 = 24
24 + 6 = 30
30 + 8 = 38
38 + 12 = 60

Hence the correct answer is Option A.

Q7. A workman starts his work on Monday, works for 8 days and takes every 9th day as his holiday. His 12th holiday will fall on:
A. Monday
B. Wednesday
C. Thursday
D. Tuesday

Answer: Option B

Explanation:
Starting from Monday, he works for 8 days and 9th day will be taken as a holiday. His every holiday cycle contains a total of 9 days. And, in total we have 12 holidays. Hence there should be 12 working cycles.

Thus,
Total no. of days = 12*9 = 108
No. of odd days = 108/7 =Q(15) & R(3)
It starts from Monday and completed 15 weeks(cycles) - Monday to Sunday
We have Remainder of 3(last cycle). We know that the last day of last cycle will be our 12th holiday.
Remainder 1 - Monday
Remainder 2 - Tuesday
Remainder 3 - Wednesday
Hence, the 12th holiday will fall on Wednesday.

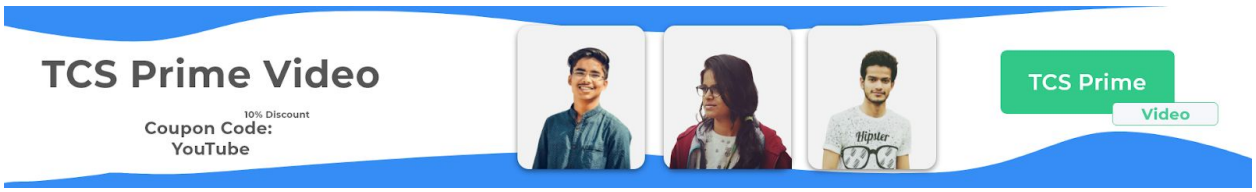
Q8. For any two numbers, we define an operation $ yielding another number X $ Y such that the following condition holds: X $ X = 0.
Also, for all X, X $ (Y $ Z) = X $ Y + Z.
Find the value of 2012 $ 0 + 2012 $ 1912.

A. 2112
B. 100
C. 5936
D. Cannot be determined

Answer: Option A

Explanation:
Given that X $ X = 0 is possible only if $ represents subtraction operation.
Also, x $(y $ z) = x-(y-z) =x – y + z
It can be written as x $ y + z
2012 - 0 + 2012 – 1912 = 2112

Q9. A very special island is inhabited only by knights and knaves. Knights always tell the truth, and knaves always lie. You meet four inhabitants: Usha, Eesha, Nisha and Asha. Usha tells you ‘Eesha is a knave’. Eesha tells you ‘Asha is a knave’. Nisha claims,



'Eesha is a knight and Asha is a knave.'. Asha tells you, 'I and Nisha are different.' Identify the knave(s).

A. Eesha only
B. Usha only
C. Eesha and Nisha
D. Nisha and Asha

Answer: Option C

Explanation:
Let's assume that Usha is a Knight and tells the truth

Said by	Knights (Truth)	Knaves (Lie)
Usha	Usha	Eesha

Eesha tells us that Asha is a knave, which must be a lie as Eesha is a Knave. So, Aasha must be a knight.

Said by	Knights (Truth)	Knaves (Lie)
Usha	Usha	Eesha
Eesha	Asha	

Nisha says Eesha is a knight, which cannot be true as we assumed Usha to be a Knight and she told Eesha is a Knave. So, Nisha must be telling a lie and she must be a knave. Since we got Nisha to be a knave, her second statement must be false. So Asha must be a knight.
Asha(Knight) says, I and Nisha are different - Satisfies the assumption(Nisha is a knave)

Our assumption is correct as none of the statements contradict one another. So the knaves in this problem are Eesha and Nisha.

Q10. The current age difference between 2 persons is 25. 9 years before the elder one's age was 6 times that of the younger one. What is the sum of their current ages?

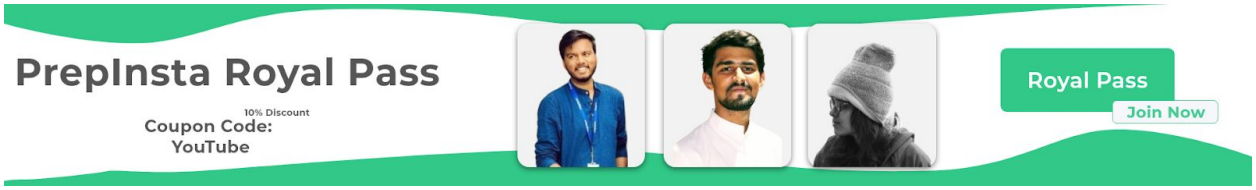
A. 53 years
B. 50 years
C. 47 years
D. 75 years

Answer: Option A

Explanation:
Let X and Y be the ages of the two persons.X – Y = 25X = Y + 25 ------- Eq 1
Before 9 years, X – 9 = 6 (Y - 9) ------ Eq 2
Substituting Eq 1 on 2Y + 16 = 6 (Y - 9)
5Y = 70
Y = 14
X = 39
X + Y = 53

Q11. According to the stock policy of a company, each employee in the technical division is given 15 shares of the company and each employee in the recruitment division is given 10 shares. Employees belonging to both committees get 25 shares each. There are 20 employees in the company, and each one belongs to at least one division. The cost of each share is Rs. 10. If the technical division has 15 employees and the recruitment division has 10 employees, then what is the total cost of the shares given by the company?

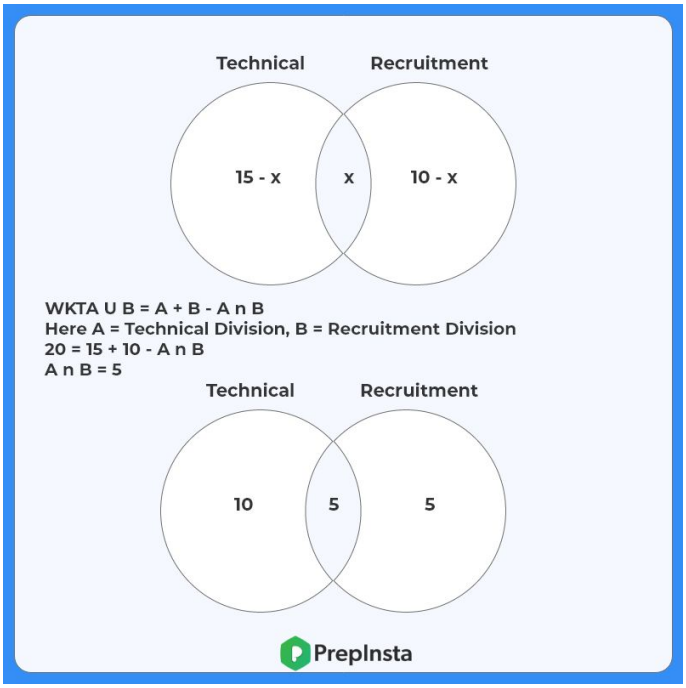
A. Rs. 2650
B. Rs. 3180
C. Rs. 3250



D. Rs. 3120

Answer: Option C

Explanation:



Employee belonging to Technical division only = 15 - 5 = 10
Number of shares = 10*15 = 150 shares
Employee belonging to Recruitment division only = 10 – 5 = 5
Number of shares = 5*10 = 50 shares
Employees belonging to both communities = 5
Number of shares = 5*25 = 125 shares
Total shares = 150 + 50 + 125 = 325 shares
Total cost of the shares = 325*10
= Rs. 3250

Q12. If 5+3+2 = 151022, 9+2+4=183652, 8+6+3 = 482466 and 5+4+5 = 202541, then 7+2+5

A. 143547
B. 132234
C. 2577224
D. 112321

Answer: Option A

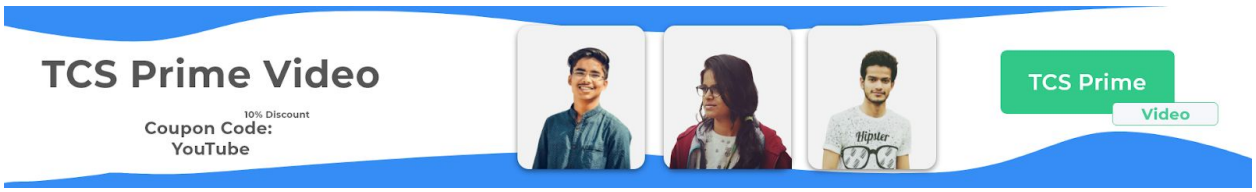
Explanation:
Let's observe the pattern:
5+3+2 =(5*3) | (5*2) | (5*3+5*2-3) = 15 | 10 | 22 = 151022
Pattern: (Leftmost digit * Second Digit) | (Leftmost digit * Right most digit) | (Leftmost digit * Second Digit) + (Leftmost digit * Right most digit) – Second digit)

9+2+4 = (9*2) | (9*4) | (9*2+9*4-2) = 18 | 36 | 52 = 1836528+6+3 = (8*6) | (8*3) | (8*6+8*3-6) = 48 | 24 | 66 = 4824665+4+5 = (5*4) | (5*5) | (5*4+5*5-4) = 20 | 25 | 41 = 202541
So,
7 + 2 + 5 = (7*2) | (7*5) | (7*2+7*5-2) = 14 | 35 | 47

Q13. In an office, at various times during the day the boss gives the secretary a letter to type, each time putting the letter on top of the pile in the secretary's inbox. When there is time, the secretary takes the top letter off the pile and types it. If there are five letters in all, and the boss delivers them in the order 1 2 3 4 5, which of the following could NOT be the order in which the secretary types them?

A. 12345
B. 24351
C. 32415
D. 45231

Answer: Option D



Explanation:
Let's check the option sequence:
1 2 3 4 5: the boss delivers 1, then the secretary does 1, then the boss delivers 2, then the secretary does 2, etc. up to 5. Hence this is a possible case.

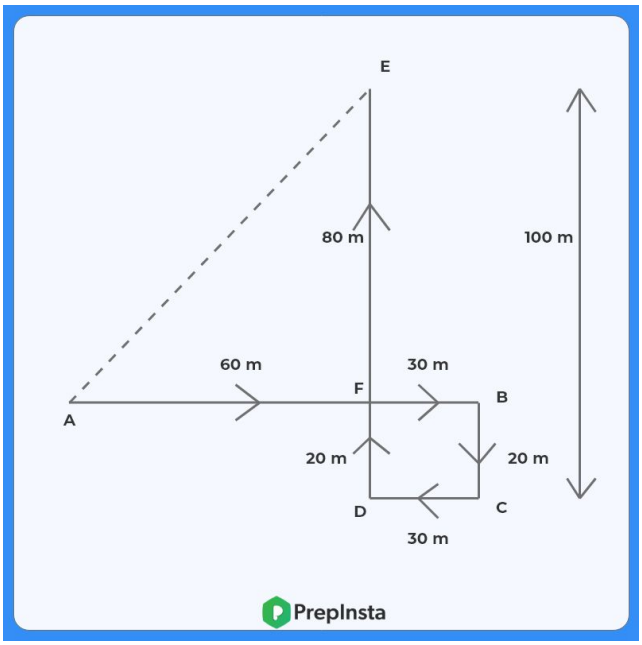
2 4 3 5 1: the boss delivers 1, then delivers 2, then the secretary does 2, then the boss delivers 3 and 4 before the secretary completes 2, after completing 2 the secretary does 4, then the secretary does 3, then the boss delivers 5, then the secretary does 5, then the secretary does 1. Hence, this is a possible case.
3 2 4 1 5: the boss delivers 1, then delivers 2, then delivers 3, then the secretary does 3, then does 2, then the boss delivers 4, then the secretary does 4, then does 1, then the boss delivers 5, then the secretary does 5. Hence this is a possible case.
4 5 2 3 1 : In order for the secretary to start with 4, the boss must deliver 1, 2, 3, and 4, then the secretary does 4. To get 5 next, the boss next delivers 5, then the secretary does 5, but now we can't get 2, 3, 1: the remaining letters must start with 3, which is on the top of the pile.
Hence, 4 5 2 3 1 cannot be the order.So, the correct answer option is d. 4 5 2 3 1

Q14. A child was looking for his father. He went 90 metres in the East before turning to his right. He went 20 metres before turning to his right again to look for his father at his uncle's place 30 metres from this point. His father was not there. From here he went 100 metres to the North before meeting his father in a street. How far did the son meet his father from the starting point?

A. 90
B. 30
C. 80
D. 100

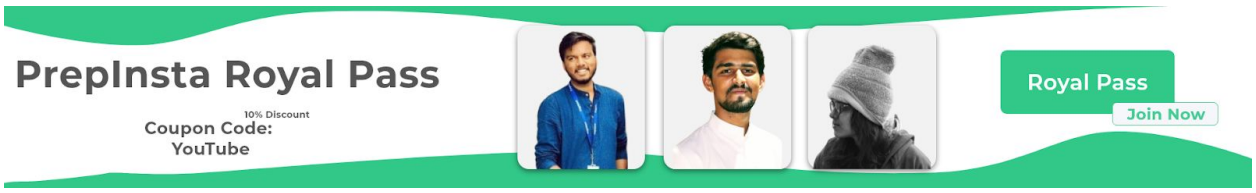
Answer: Option D

Explanation:



Clearly, the child moves from A to B 90 metres eastwards up to B, then turns right and moves 20 metres up to C, then turns right and
moves up to 30 metres up to D. Finally, he turns right and moves up to 100 metres up to E.
So, AB = 90 m, BF = CD = 30 m,
So, AF = AB - BF = 60 metre
Also, DE = 100 metre, DF = BC = 20 metre
So, EF = DE - DF = 80 metre
As we can see in the image that triangle AFE is a right-angled triangle and we are having two sides, we need to calculate the third one.

By applying Pythagoras theorem



$AE^2 = AF^2 + EF^2$
$= 60^2 + 80^2$
$= 3600 + 6400$
$= 10000$
$AE^2 = 100^2$
$AE = 100$

Q15. F, G, H, J, K, L, M and N are 8 people. They need to be grouped into two with the following conditions:
• F and J must be in the same group
• G and N must be in different groups
• H and L must be in the same group
• M and G are not in the same group

Find the correct ordering of groups
A. FJ, KL, MN, GH
B. FH, JL, MN, GK
C. FJ, HL, MN, GK
D. FJ, HL, MN, GH

Answer: Option C
Explanation:
F and J must be in the same group
H and L must be in the same group
If we see the options,
In option A: FJ, KL, MN, GH
H and L are not together
In option B: FH, JL, MN, GK
H and L are not together
In option D: FJ, HL, MN, GH
H and G are together, which is not possible.
In option C: FJ, HL, MN, GK
First condition: F and J are together FJ, HL, MN, GK
Second Condition: G and N are in different group FJ, HL, MN, GK
Third Condition: H and L are in the same group FJ, HL, MN, GK
Fourth Condition: M and G are not in the same group FJ, HL, MN, GK

Option C satisfies all the conditions.

8.3. Numerical Ability

Q1. The values of

$$\frac{(6.43)^2 + 33.3074 + (5.18)^2}{6.43 \times 41.3449 - 10.36 \times 2.59 \times 5.18}$$

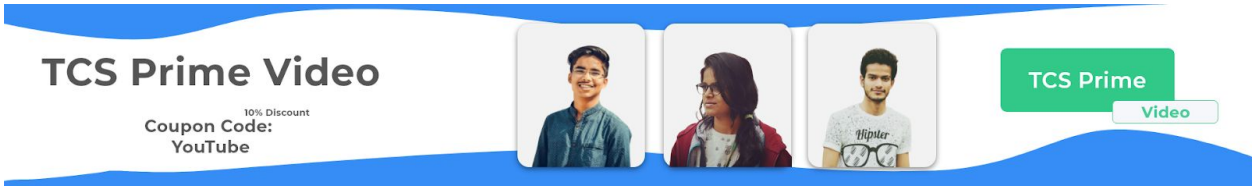
lies between

A. 0.1 and 0.5
B. 0.5 and 1
C. 1 and 1.2
D. 1.2 and 1.5

Answer: Option B

Explanation:
Let's assume A = 6.43 and B = 5.18
Numerator
A. $6.43*6.43 = A^2$
B. $5.18*5.18 = B^2$
C. $33.3074 = A*B$
Numerator = $A^2 + AB + B^2$

As per BODMAS,
Denominator:
A. $6.43 * 41.3449 = 6.43*6.43*6.43 = A^3$
B. $10.36*2.59*5.18 = 5.18*2*2.59*5.18 = B^3$
Denominator = $A^3 - B^3$, this can also be written as $(A^2 + AB + B^2)(A-B)$



Numerator/Denominator = $(A^2 + AB + B^2) / (A^2 + AB + B^2)(A-B) = 1/(A-B) = 1/(6.43 - 5.18) = 1/1.25 = 0.8$
0.8 is between 0.5 and 1.

Q2. A's capital exceeds B's capital by 20.5%. B invests his capital at 20% p.a for 3 years, interest compounded annually. At what rate percentage p.a must A invest his capital at simple interest so that at the end of 3 years, both get the same amount(in INR)? (Correct to one decimal place)

A. 13.8
B. 14.2
C. 14.5
D. 15.2

Answer: Option C

Explanation:
Let's assume B invests Rs. X and A invests Rs. X + 20.5% of X = 1.205 X
B invests his capital at 20% p.a for 3 years(compounded annually)
Amount(C.I) = $P(1+r/100)^n = X(1+20/100)^3 = 1.728X$

A invests his capital(1.205X) at R interest at S.I for 3 years

Amount(S.I) = P + PNR/100 = 1.205X + (1.205X*3*R)/100 = It has been given that amount should be equal.

1.728X = 1.205X (1+ 3R/100)

1.4340 = 1+ 3R/100

1.4340-1 = 3R/100

3R = 43.40

R = 43.40/3

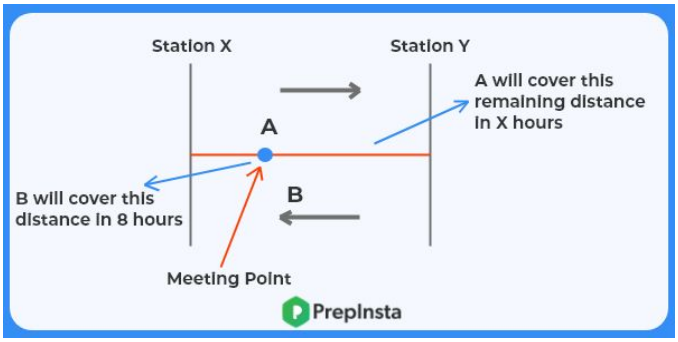
R = 14.46 approximately equals 14.5

Q3. Trains A and B start moving at the same time from stations X and Y, respectively towards each other on parallel tracks. After passing each other, A and B take x hours and 8 hours to reach Y and X, respectively. If the speed of B is 25% more than that of A, then what is the value of x?

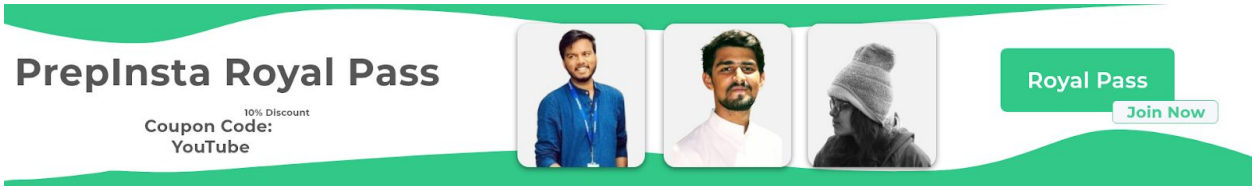
A. 10
B. 10(½)
C. 12(½)
D. 12

Answer: Option C

Explanation:
Since B's speed is more than A, B would have covered more distance than A, when they met.
Let's assume that they meet at this point



Let's assume the distance between Station X and meeting point as D1 and distance between station Y and meeting point as D2.It has been given Speed of B(S_B) = 1.25*Speed of A(S_A)



If speed ratio of S_A:S = 1:1.25, which can also be written as 4:5
then time ratio will be T_A:T_B = 5:4
B took 8 hours to cover D1, using the time ratio(5:4) we can find A covers the same distance in 10 hours. This means that from the time when A started from station X, A travelled for 10 hours to reach the meeting point.

It is clearly given in the problem that both start at the same time, if A starts and takes 10 hours to reach the meeting point, B would have also taken 10 hours to reach the meeting point, which is Station Y to meeting point(D2)
D2 was covered by B in 10 hours and A took X hours to cover D2. With the help of time ratio again(T_A:T_B = 5:4), if B is 10 hours, 4 parts = 10, 1 part = 2.5.
5 parts of A would be 5*2.5 = 12.5 hours.
X = 12.5 hours

Q4. For the data

Marks Obtained	Below 10	Below 20	Below 30	Below 40	Below 50
No. of students	5	16	28	42	72

If x is the class mark of the modal class and y is the upper limit of the median class, then the value of (2x+y) is

A. 95
B. 125
C. 130
D. 135
Answer: Option C
Explanation:
The cumulative number of students has been given in the question, let's find the number of students belonging to each category. i.e number of students scored between 0-10, 10-20, 20-30
For example: To find the number of students in the 10-20 category, we have to subtract those who scored less 10 from the total number of students scored less than 20. i.e 16 - 5 = 11

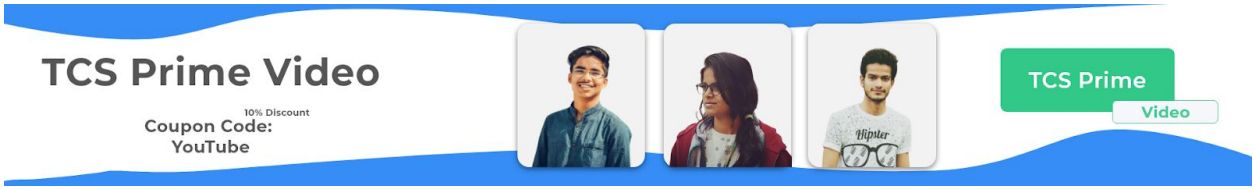
Marks Scored Between	Number of Students	Cumulative number
0 - 10	5	5
10 - 20	11	16
20 - 30	12	28
30 - 40	14	42
40 - 50	30	72

Modal class = Category with maximum number of studentsAs per the data, category 40-50 has 30 students. So our modal class is 40-50

It is given that x is the class mark of modal class.
Class mark = (lower limit + upper limit) / 2
Class mark of the modal class(x) = (40+50)/2 = 45

As per the question statement, y is the upper limit of median class.Median class = class where n/2 is achieved, where n is the total number of students
n/2 = 72/2 = 36

We got the 36th student in the 30-40 category. It is mentioned that y is the upper limit of median class

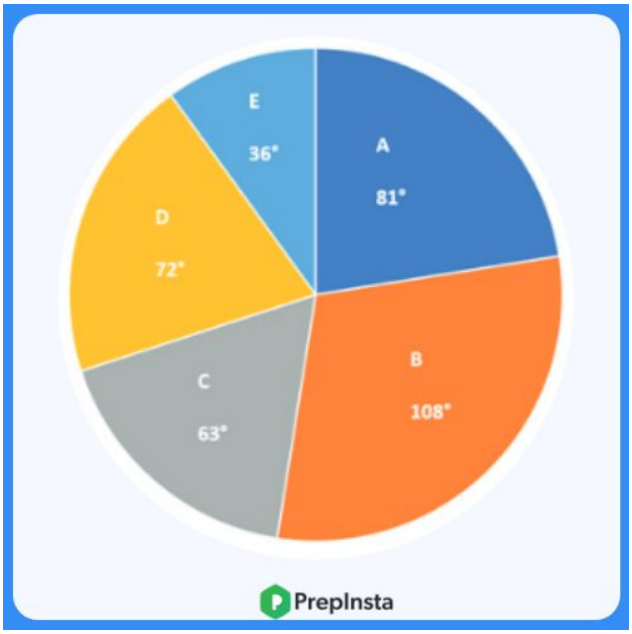


y = upper limit of 30-40 = 40

If x = 45 and y = 40, then 2x+y = 2(45)+40 = 90+40 = 130

Q5. Study the following pie chart and answer the question:

Breakup of the total number of employees of a company in its five offices in city A, B, C, D and E



Total number of employees = 2400If one-third of the total number of employees of the company in offices in cities B and D are shifted to offices in cities C and E in the ratio 2:3, the difference between the number of employees in offices in cities C and E is

A. 160
B. 100
C. 120
D. 90

Answer: Option B

Explanation:

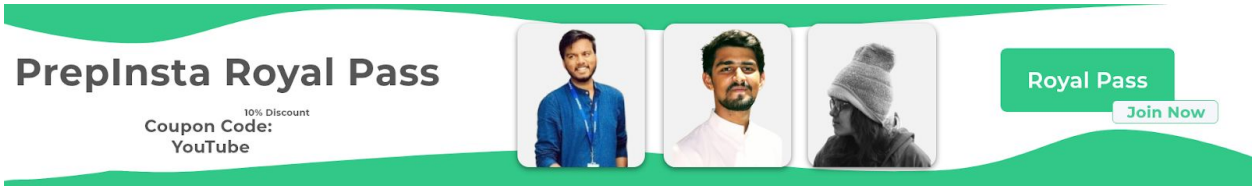
Total number of employees, 360° = 2400
1° = 2400/360
Number of employees in city B = 108° = 108 * 2400/360 = 720
Number of employees in city D = 72° = 480
⅓ of (City B and D) = ⅓ (720+480) = 400
400 employees are shifting to company C and E in the ratio 2:3. 5 parts = 400
1 part = 80, then 2:3 is 160 & 240
Existing employees in city C = 63° = 63*2400/360 = 420
New count = 420 + 160 = 580
Existing employees in City E = 36° = 36*2400/360 = 240
New count = 240 + 240 = 480
Difference between the number of employees in C and E = 580-480 = 100

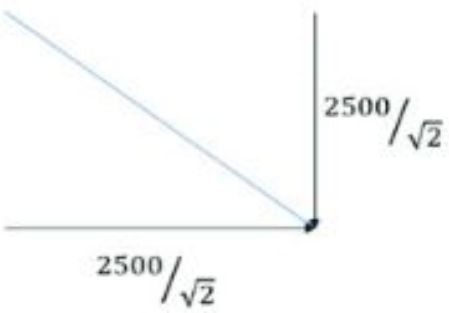
Q6. Whenever a force acts on a body obliquely, it's split into vertical and horizontal components for further analysis. If the force acting on a stone has the same horizontal and vertical components of value 2500/√2 newtons, what's the value of oblique force?

A. 2500/2
B. 2500
C. 2500/4
D. 1500

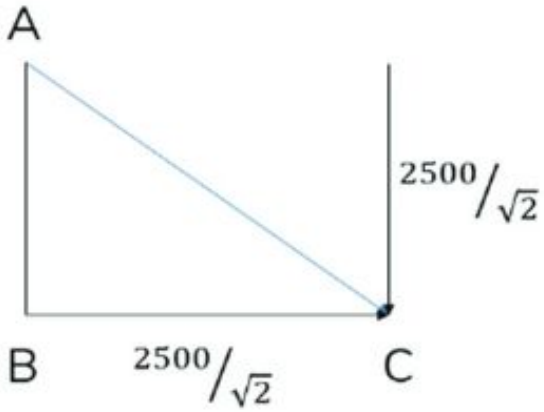
Answer: Option b

Explanation:
Force acting on a stone splits into vertical and horizontal components and has same value 2500/√2





In order to find the value of the oblique force, let's form a right-angled triangle and find the answer.



To find the oblique force, let's find the value of AC
$AC^2 = AB^2 + BC^2$
$AC^2 = (2500/\sqrt{2})^2 + (2500/\sqrt{2})^2$
$AC^2 = 2500^2/2 + 2500^2/2$
AC = 2500

Q7. A shop sells chocolates. It used to sell chocolates for Rs.2 each, but there were no sales at that price. When he reduced the price, all chocolates sold out enabling the shopkeeper to realize Rs. 164.90 from the chocolates alone. If the new price was not less than half the original price quoted. How many chocolates were sold (at the reduced price)?

A. 39
B. 97
C. 37
D. 71

Answer: Option B

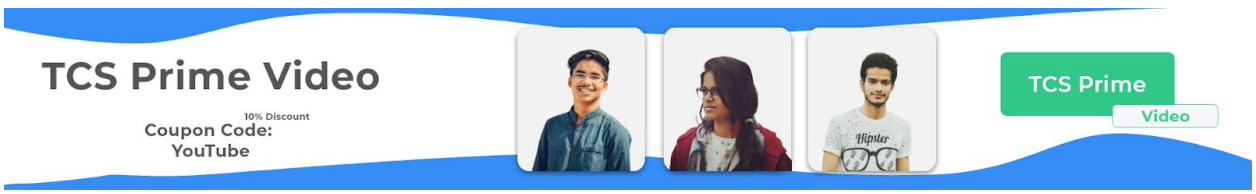
Explanation:

Let the number of chocolates be x and price per chocolate be y
New price of the chocolates is not less than half of the quoted price, which means our answer should be between 1 and 2
Quoted price = Rs. 2.
New price will be not less than Rs. 1.
x*y = Between 1 and 2. The options given are the values of X. Let's substitute and find which option gives y value between 1 and 2,
Options A: y = 164.9/39 = Rs. 4.22
Options B: y = 164.9/97 = Rs. 1.7
Option C: y = 164.9/37 = Rs. 4.45
Option D: y = 164.9/71 = Rs. 2.32
In all the options except 2, the selling price is more than Rs. 2. It should be less than Rs. 2.

So, answer is option B.

Q8. A sum of Rs. 3000 is distributed amongst A, B and C. A gets 2/3 of what B and C got together and C gets 1/3 of what A and B got together. C's share is:

A. Rs. 1200
B. Rs. 2250
C. Rs. 750
D. Rs. 1050



Answer: Option C

Explanation:

A's share + B's share + C's share = 3000 ---- Eq 1
A = 2/3(B+C) ---- Eq 2
Also, C = 1/3(A+B) A + B = 3C ---- Eq 3
Put the value of C in Eq (1)
A+B+C = 30003C + C = 3000
C = Rs. 750
Hence C's share will be Rs. 750

Q9. Assume that f(1) = 0 and f(m+n)=f(m) + f(n)+ 4(9mn-1). for all the natural numbers (integers >0) m & n.What is the value of f(17)?

A. 5436
B. 4831
C. 5508
D. 4832

Answer: Option D

Explanation:
Given that f(1) = 0 and f(m+n)=f(m) + f(n)+ 4(9mn-1).
Where m and n are positive integers, in order to find the value of f(17), we should assume the values of m and n

Let's substitute 16 and 1(for m and n) so that we get m+n to be 17
f(17) = f(16+1) = f(16)+f(1)+4(9*16*1 - 1) = f(16) + 0 + 572

To find the value of f(17), we should know the value of f(16)

Let's substitute m = 8 and n = 8 so that we get as the total as 16
f(16) = f(8+8)=f(8)+f(8)+4(9*8*8 - 1)

To find f(16), we should know f(8)
Let's substitute m = 4 and n = 4
f(8) = f(4+4) = f(4) + f(4) + 4(9*4*4 - 1)

To find f(8), we should know the value of f(4)
Let's substitute m = 2 and n = 2
f(4) = f(2+2)=f(2)+f(2)+4(9*2*2 - 1)
f(2) can be found by substituting 1 for m and n(m = 1; n = 1)
f(2) = f(1+1) = f(1) + f(1) + 4(9*1*1 - 1) = 0 + 0 + 4 * 8 = 32
f(4) = f(2+2) = f(2) + f(2) + 4(9 * 2 * 2 - 1) = 32 + 32 + 4 * 35 = 204
f(8) = f(4+4) = f(4) + f(4) + 4(9 * 4 * 4 - 1) = 204 + 204 +4 * 143 = 980
f(16) = f(8+8) = f(8) + f(8) + 4(9 * 8 * 8 - 1) = 980 + 980 + 4 * 575 = 4260
f(17) = f(16+1) = f(16) + f(1) + 4(9 * 16 * 1 - 1) = 4260 + 0 + 4 * 143 = 4832

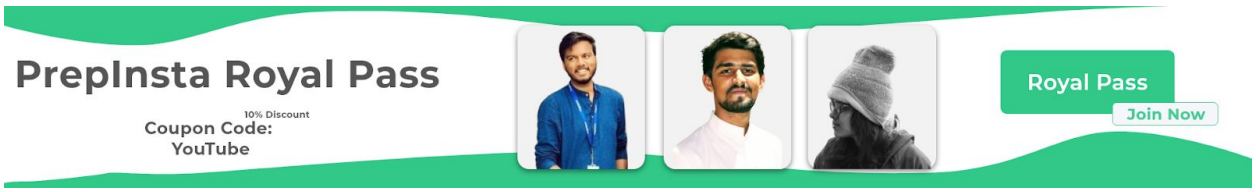
Q10. A number divided by 357 leaves 5 as remainder. If the number is divided by 17, what is the remainder?

A. 9
B. 3
C. 5
D. 7

Answer: Option C

Explanation:

Example: 149/24 = R(5)



Let's take the factors of 24 which can produce remainder 5.
Factors of 24 are 1, 2, 3, 4, 6, 8, 12, 24
6, 8, 12 are the numbers which can produce reminder 5. If we divide 149 with any of these 3 numbers, it will always give us remainder 5(same as the parent number)

In the same way, In this problem, X/357 = R(5), all the eligible factors of 357 will provide the same 5 dividing the X.
Factors of 357 = 1, 3, 7, 17, 21, 51, 119, 357

17 is a multiple of 357, it will also provide the same remainder(5).

Q11. A girl entered a store and bought x flowers for y dollars (x and y are integers). When she was about to leave, the clerk said, "If you buy 10 more flowers I will give you all For $2, and you will save 80 cents a dozen". The values of x and y are:

A. 15, 1
B. 10, 1
C. 5, 1
D. Cannot be determined

Answer: Option C

Explanation:

She bought x flowers for y dollars.
So, 1 flower cost = y/x
12 flower or 1 dozen flower cost = 12y/x
Again, x + 10 = 2 dollars
1 flower cost = 2/(10+x)
12 flower or 1 dozen cost = (2 x 12) / (10 + x) = 24/(10+x)

Given that this new dozen cost is 80 cents.
80 cents = 80/100 dollars = 4/5
We know that: She will save 80 cents a dozen
(12y/x) – [24/(10+x)] = 4/5 ------- Eqn (1)
L.H.S = (12y/x) – [24/(10+x)]
R.H.S. = 4/5

Only option which gives the 4/5 as a value when put in L.H.S. will be the answer.

Now let's check the options:

Option A: x = 15; y = 112/15 – 24/11 ≠ ¾
Option B: x = 10; y = 1(12/10) – (24/25) ≠ ¾
Option C: x = 5; y = 112/5 – 24/15 = ¾
Hence, option C is the correct answer.

Q12. The letters in the word ROADIE are permuted in all possible ways and arranged in alphabetical order. Find the word in the 44th rank.

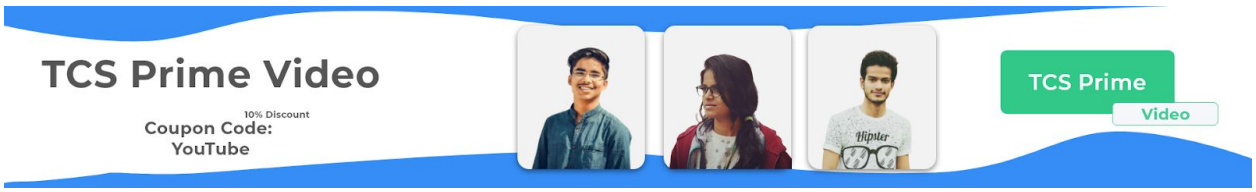
A. AERIOD
B. AERDOI
C. AERODI
D. AEODRI

Answer: Option B

Explanation:

Observe the letters used to frame the word ROADIE. If we try to arrange them in alphabetical order then the first rank should be ADEIOR

To calculate the rank let's take the number of letters which will start from A first.



A _ _ _ _ = A x 5 ways x 4 ways x 3 ways x 2 ways x 1 way (In rank question if nothing is given then we always assuming there is no repetition)

So total letter which will be formed with starting letter A = 120

It means that the 44th rank word will start with A

Now, let's fix the letter A and D
AD _ _ _ _ = A D x 4 ways x 3 ways x 2 ways x 1 way = 4! = 24
AD sequence will get over at 24th rank. But we need 44th rank. So we need to check for the next alphabet. The next sequence will be A E with 24 such words so we will reach at 48 rank. So our answer will start with AE
Since we confirmed it starts with AE, Let's fix 3 letters and narrow down the possibilities

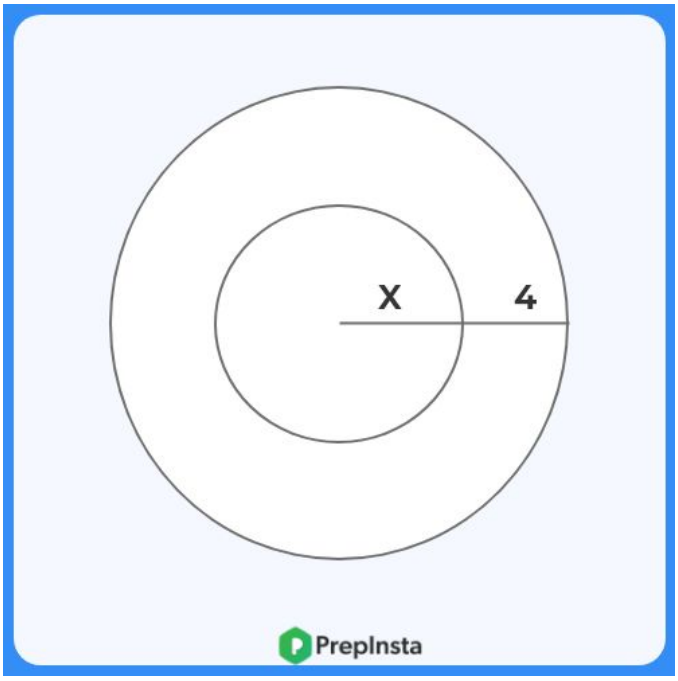
AED--- => 3! = 6 --- 24 ranks (AD) + 6 = 30
AEI--- => 3! = 6 --- 30 ranks + 6 = 36
AEO--- => 3! = 6 --- 36 ranks + 6 = 42
Next will letter after AEO will be AE R sequence:
AERDIO => 43th
AERDOI => 44th

Q13. There is a pool of radius X feet and there is a pathway around the pool with a width of 4 feet. Find the radius of the pool if the path area/pool area = 11/25.

A. 12
B. 20
C. 25
D. 29

Answer: Option B

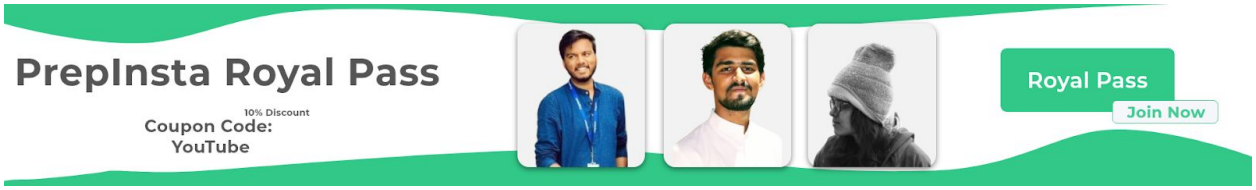
Explanation:



To find the path area, we should subtract the pool area from the total area of the circle(pool+path)
Area of pool/Area of path = 11/25
Since Pi is common in both numerator and denominator, it will get cancelled.
[(x+4)2– x2]/x2 = 11/25
(x+4)2 /x2 = (11/25) +1
[(x+4)/x]2 = 36/25
(x+4)/x = (6/5)2
(x + 4)/x = 6/5
5x + 20 = 6x
x = 20 feet

Q14. 6 tasks and 6 persons. P1 and P2 do not do task T1. T2 is assigned to P3 or P4;. Each person should be assigned with at least 1 task. In how many ways the task can be assigned?

A. 192



B. 360
C. 144
D. 180

Answer: Option C

Explanation:

There are 6 tasks T1, T2, T3, T4, T5, T6
And 6 people P1, P2, P3, P4, P5, P6.
If T2 is assigned to P3(1 way), there are 5 remaining tasks T1, T3, T4, T5, T6 and 5 remaining people P1, P2, P4, P5, P6.
T1 can't be assigned to P1 or P2, so T1 could be assigned to P4, P5, P6 = 3 ways
T3 could be assigned to each of 4 remaining people = 4 ways
T4 could be assigned to each of 3 remaining people = 3 ways
T5 could be assigned to each of 2 remaining people = 2 ways
T6 could be assigned to the only remaining people = 1 way
Hence, the total combination in this case is 3*4*3*2*1 = 72 ways
If T2 is assigned to P4. The result is the same as the first case above. The total combination is 72 ways again

Answer = 72 + 72 = 144 ways

Q15. Car A leaves city C at 5 pm and drives at a speed of 40 kmph. 2 hours later another car B leaves city C and drives in the same direction as car A. In how much time will car B be 9 km ahead of car A if the speed of car B is 60 kmph?

A. 4.25 hours
B. 4.17 hours
C. 4.30 hours
D. 4.45 hours

Answer: Option D

Explanation:

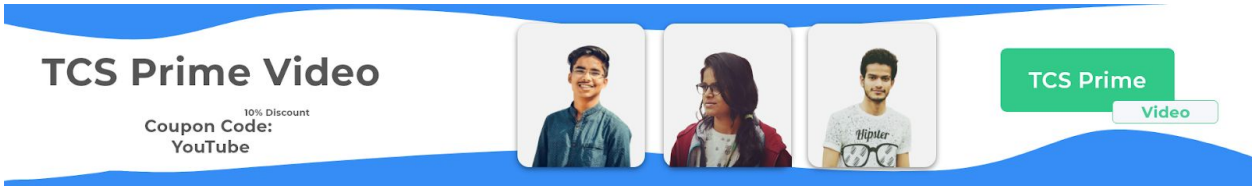
Speed of car A = 40 kmph,
Speed of car B = 60 kmph
Distance travelled by A in 2 hours.
Distance = Speed * Time = 40 * 2 = 80 km
Distance B must travel to be 9 km ahead of A = 80 km + 9 km = 89 km
Relative speed of A and B = 60 - 40 = 20 kmph
Time taken by B to be 9 km ahead of A:
Time = Distance/Speed = 89km/20kmph = 4.45 hours

8.4. Programming Logic

Q1. What is the output of this program?

```
#include < iostream >
using namespace std;
int main( )
{
    char line[100];
    cin.getline( line, 100, 't' );
    cout << line;
    return 0;
}
```

Options
A. 100
B. T
C. It will print whatever strings we stored.



D. None of the mentioned

Answer: Option C
Explanation: The program will store all strings entered and will print them only when the character 't' is encountered
Input >> coding
Input >> is fun
Input >> t
Output:
coding
is fun

Q2. Predict the output

```
#include<iostream>
using namespace std;
int &fun()
{
    static int x = 10;
    return x;
}
int main()
{
    fun() = 30;
    cout << fun();
    return 0;
}
```

Options
A. 30
B. 10
C. Error
D. Segmentation Fault

Answer: Option A

Explanation:

When a function returns by reference, it can be used as lvalue. Since x is a static variable, it is shared among function calls and the initialization line "static int x = 10;" is executed only once.The function call fun() = 30, modifies x to 30. The next call "cout << fun()" returns the modified value.

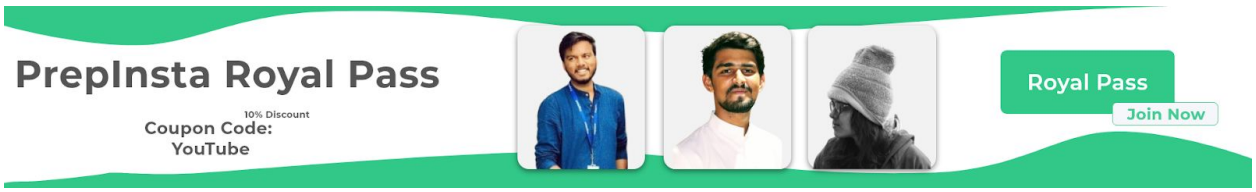
Q3. Predict the output

```
#include <iostream>
using namespace std;
 int main()
 {
     int a = 32, *ptr = &a;
     char ch = 'A', &cho = ch;
     cho += a;
     *ptr += ch;
      cout << a << ", " << ch <<
endl;
     return 0;
 }
```

Options
A. 32, A
B. 32 , a
C. 129 , a
D. 129, A

Answer: Option C

Explanation:



The "ptr" variable is a pointer which holds the address of variable "a". And "*ptr" returns the value of "a" variable. "cho" is a reference variable to "ch". So any change made to "cho" will be reflected to "ch". As such, when "cho" is increased by 32, it adds to the ASCII value of "A"(which is 65), and this results to 97 which is the ASCII value of "a"(from the alphabet). So this "a" gets stored in "ch". As for when "*ptr" is incremented by "ch", it gives value 97+32=129

Q4. What will be the output of the following C++ code?

```
#include <iostream>
using namespace std;
 int main()
 {
     int arr[] = {4, 5, 6, 7};
     int *p = (arr + 1);
     cout << *p;
     return 0;
 }
```

Options
A. 4
B. 5
C. 6
D. 7

Answer: Option B

Explanation:

We are making the pointer point to the next value and are printing it. Thus, the answer will be option b (5)

Q5 What will be the output of the following C++ code?

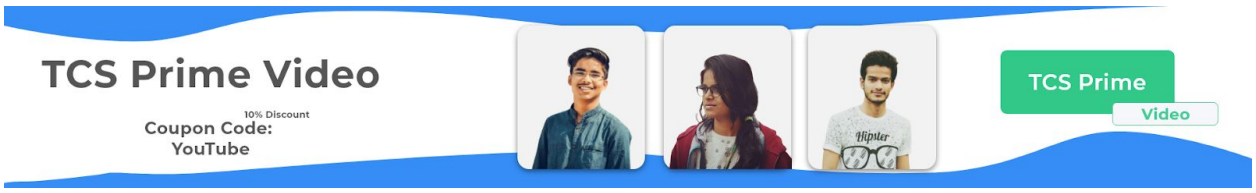
```
#include <iostream>
using namespace std;
 int main()
 {
     int a = 5, b = 10, c = 15;
     int *arr[ ] = {&a, &b, &c};
     cout << arr[1];
     return 0;
 }
```

Options
A. 5
B. 10
C. 15
D. It will return some random number

Answer: Option D

Explanation:
Array element cannot be address of auto variable. It can be address of static or extern variables

Q6. What will be the output of the following C++ code?



```
#include <stdio.h>
#include <iostream>
using namespace std;
 int main()
 {
      int array[] = {0, 2, 4, 6,
7, 5, 3};

     int n, result = 0;
     for (n = 0; n < 8; n++)
     {
        result += array[n];
     }
    cout << result;
    return 0;
 }
```

Options:
A. 25
B. 26
C. 27
D. 21

Answer: Options C

Explanation:

We are adding all the elements in the array and printing it. The total number of elements in the array is 7, but the for loop will go beyond 7 and add a garbage value.

Q7. What will be the output of the following C++ code?

```
#include <stdio.h>
#include <iostream>
using namespace std;
 int main()
 {
    int array[] = {10, 20, 30};
    cout << -2[array];
    return 0;
 }
```

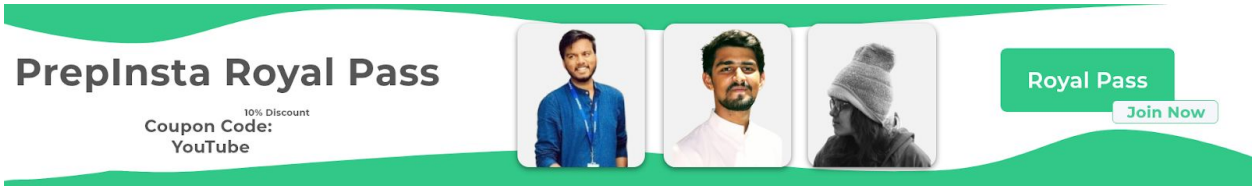
Options
A. -15
B. -30
C. Compile time error
D. Garbage value

Answer: Option b

Explanation: The program will print the negative value of the concerned element. Index value Array[2]=30. So, the answer is -30.

Q8 What is the return value of f(p, p) if the value of p is initialized to 5 before the call? Note that the first parameter is passed by reference, whereas the second parameter is passed by value.

```
int f(int &x, int c)
 {
     c  = c - 1;
     if (c == 0)
     return 1;
     x = x + 1;
     return f(x, c) * x;
}
```



Options

A. 3024
B. 6561
C. 55440
D. 161051

Explanation: Since c is passed by value and x is passed by reference, all functions will have same copy of x, but different copies of
c.f(5, 5) = f(x, 4)*x
= f(x, 3)*x*x = f(x, 2)*x*x*x
= f(x, 1)*x*x*x*x = 1*x*x*x*x = x^4.

Q9. What will be the output of the following C++ code?

```
#include <stdio.h>
#include<iostream>
using namespace std;
int main()
{
    int a = 5, b = 10, c = 15;
    int arr[3] = {&a, &b, &c};
    cout << *arr[*arr[1] - 8];
    return 0;
}
```

Options

A. 15
B. 18
C. Garbage Value
D. Compile time error

Answer: Option d
Explanation: The conversion is invalid in this array. So, it will throw an error. The following compilation error will be raised:cannot convert from ‘int *’ to ‘int’This is because &a, &b and &c represent int* whereas the array defined is of int type

Q10. Which function is used to get the length of a string object?

a. str.length()
b. str.size()
c. str.max_size()
d. Both size() and length() function

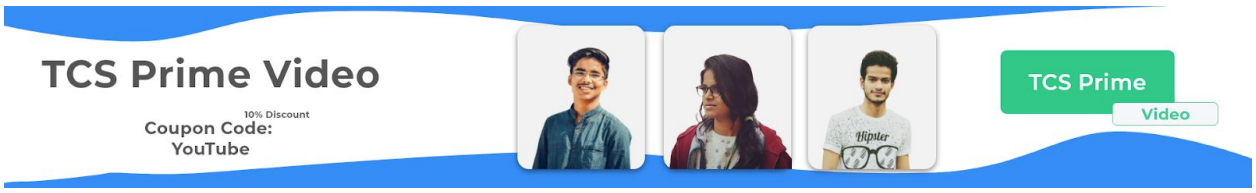
Answer: Option d

Explanation - Both size() and length() are used to get the size of the string objects.

Q11. Which of the following is not a modifier function in string class?

a. operator+=()
b. operator[]()
c. push_back()
d. erase()

Answer: Option b
Explanation
[] operator is used to access one of the characters of the string objects whereas other functions are used to modify the string in some way



Q12. Evaluate the following Java expression, if x=3, y=5, and z=10:++z + y - y + z + x++

Options

A. 24
B. 23
C. 20
D. 25

Answer: Option a

Explanation: In the expression given above, ++z means that the value will first increment by 1, then used. Now, evaluate the statement by putting the values of x, y, and z.
On calculating, the final answer is 24, as shown below.
++z +y -y +z + x++
11 + 5 - 5 + 10 + 3 = 24

Q13. What will be the output of the following program?

```
public class Test
{
    public static void main
(String[]args)
  {
    int count = 1;
    while (count <= 15)
      {
     System.out.println (count %
2 == 1 ? "***" : "+++++");
    ++count;
        }                    // end
while

    }          // end main
}
```

Options

a. 15 times ***
b. 15 times +++++
c. 8 times *** and 7 times +++++
d. Both will print only once

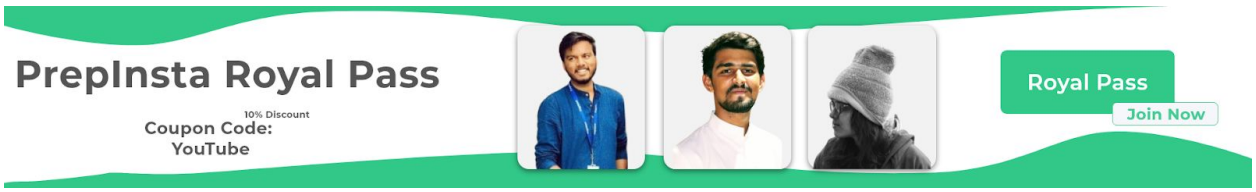
Answer: Option c

Explanation: We have declared count = 1. The value of count will be increased till 14 because of the while (count<=15) statement. If the remainder is equal to 1 on dividing the count by 2, it will print (***) else print (+++++). Therefore, for all odd numbers till 15 (1, 3, 5, 7, 9, 11, 13, 15), it will print (***), and for all even numbers till 14 (2, 4, 6, 8, 10, 12, 14) it will print (+++++).Hence, an asterisk (***) will be printed eight times, and plus (+++++) will be printed seven times

Q14. Which of the following for loop declaration is not valid?

Options

a. for (int i = 99; i >= 0; i / 9)
b. for (int i = 7; i <= 77; i += 7)
c. for (int i = 20; i >= 2; - -i)



d. for (int i = 2; i <= 20; i = 2* i)

Answer: Option a

Explanation: The first option is not a valid declaration as i/9 is not declared correctly. The correct statement will be: for (int i= 99; i>=0; i= i/9)

Then the code would execute. But without assigning the value of i/9 to a variable, it would not execute, and an exception is thrown, as shown below.

Exception in thread "main" java.lang.Error: Unresolved compilation problem:
Syntax error on token "/", invalid AssignmentOperator

The other three statements are valid and will execute. Hence, the correct answer is the option a

Q15. What will be the output of the following Java program?

```
class bitwise_operator
{
   public static void main (String
args[])
  {
    int var1 = 42;
    int var2 = ~var1;
       System.out.print (var1 + " "
+var2);
```

```
  }

}
```

Options

A. 42 42
B. 43 43
C. 42 -43
D. 42 43

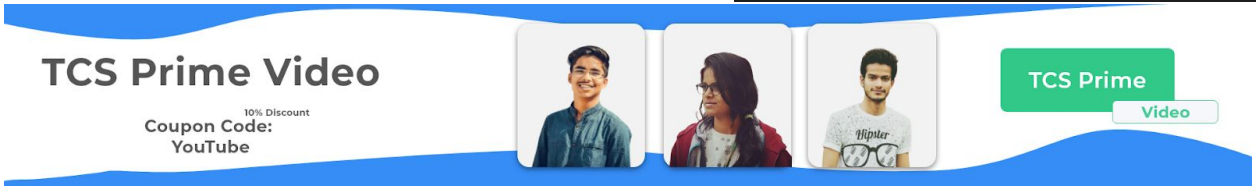
Answer: Option c

Explanation: Unary not operator, ~, inverts all of the bits of its operand. 42 in binary is 00101010 in using ~ operator on var1 and assigning it to var2 we get inverted value of 42 i:e 11010101 which is -43 in decimal.

Q16. What will be the output of the following Java program?

```
class comma_operator
{
          public static void
main(String args[])
     {
          int sum = 0;
           for (int i = 0, j = 0;
i < 5 & j < 5; ++i, j = i + 1)
          sum += i;

System.out.println(sum);
```



```
        }

}
```

Options

A. 5
B. 6
C. 14
D. Compilation error

Answer: Option b

Explanation: Using comma operator, we can include more than one statement in the initialization and iteration portion of the for loop. Therefore both ++i and j = i + 1 is executed i gets the value – 0,1,2,3,4 & j gets the values -0,1,2,3,4,5.

Q17. How many categories are containers divided into?

Options

A. 1
B. 2
C. 3
D. 4

Answer: Option d

Explanation: Containers are divided into 4 categories, namely Sequence Containers, Associative Containers, Unordered Associative Containers and Container Adaptors

Q18. Which of the following is a Non-modifying Sequence Operation?

Options

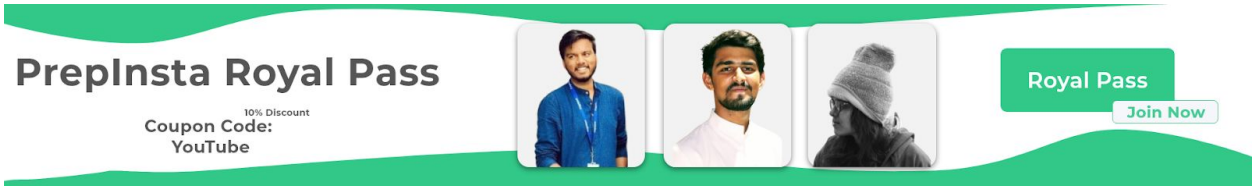
a. swap()
b. transform()
c. remove()
d. search()

Answer: Option d

Explanation: search() is a non-modifying sequence operation because while searching we never change anything whereas swapping, transforming and removing involves modifying the sequence in some way.

Q19. What will be the output of the following program?

```
public class MyFirst
{
    public static void main
(String[]args)
  {
     MyFirst obj = new MyFirst
(n);
  } static int a = 10;
  static int n;
  int b = 5;
  int c;
  public MyFirst (int m)
  {
    System.out.println (a + ","
+b + ","+c + "," +n + "," + m
);
```

```
   }                    // Instance
Block
  {
       b = 30;
       n = 20;

  }    // Static Block
  static
  {
       a = 60;

  }

}
```

Options

A. 10, 5, 0, 20, 0
B. 10, 30, 20
C. 60, 5, 0, 20
D. 60, 30, 0, 20, 0

Answer: Option d

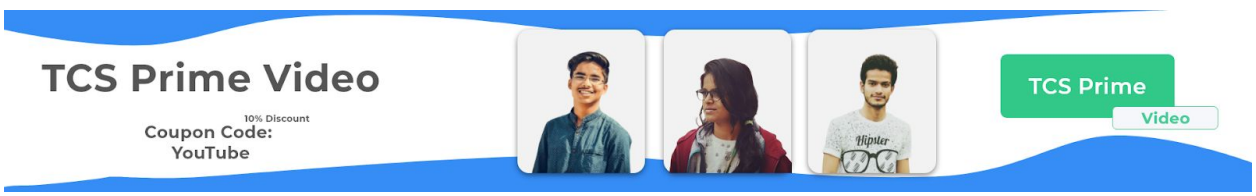
Explanation: There are two values of variable a, i.e., 10 and 60. Similarly, there are two values of variable b, i.e., 5 and 30. But in the output, the values of a and b are 60 and 30, respectively. It is because of the execution order of the program.The execution order of the program is that the static block executes first, then instance block, and then constructor. Hence, the JVM will consider the value of a and b as 60 and 30 concerning the execution order. The value of a = 10 and b = 5 are of no use. And the value of variables c and m is 0 as we have not assigned any value to them.

Q20. Which of these operators is used to allocate memory for an object?

Options

A. malloc
b. alloc
c. new
d. Give

Explanation: The operator new dynamically allocates memory for an object and returns a reference to it. This reference is address in memory of the object allocated by new.



8.5. Coding

Q1 Sweet Seventeen
Given a maximum of four digits to the base 17 (10 -> A, 11 -> B, 12 -> C, 16 -> G) as input, output its decimal value.

Expected Input
1A
Expected Output
27

Sample Input:
23GF
Sample Output:
10980

Code Solution in C++:

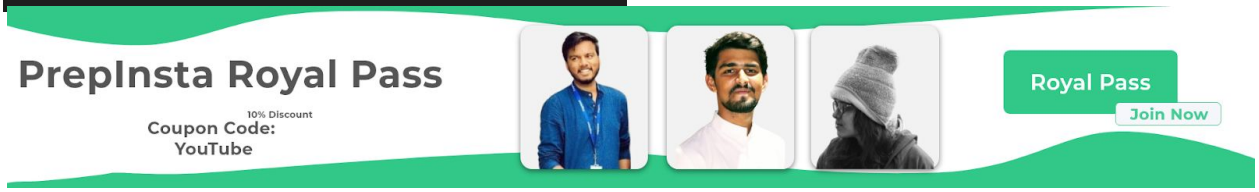
```
#include<iostream>
#define MAX 99999

using namespace std;

void fibonacci(int n)
{
   /* Variable initialization */
   int a = 0, b = 1, next;
   //the below code is for fibonacci series till nth position
   for (int i = 1; i<=n; i++)
   {
      next = a + b;
      a = b;
      b = next;
   }

   //will print a not b or next as they are stored to calculate next  and next to next term
   cout<< a;
}

void prime(int n)
{
   int i, j, flag, count =0;
   //as prime numbers in given question start from 2
   for (i=2; i<=MAX; i++)
   {
      flag = 0;
      //to check if divisible apart from 1 & itself
      //loop starts from 2 to ignore divisibilty by 1 & ends before the number itself
      for (j=2; j<i; j++)
      {
         if(i%j == 0)
         {
            //number is not prime
            flag = 1;
            break;
         }
      }
      //is prime
      if (flag == 0){
         //if found the nth prime number
         if(++count == n)
         {
```



```
            cout<< i;
            break;
        }
    }
  }
}
int main(){
  int n;
  cin >> n;

  /*if n is odd
    nth number in main series will be found at
(n/2 + 1) position
    in fibonacci sub series
  else
    if n is even then it will be found in (n/2)
position in prime sub series */

  if(n%2 == 1)
    fibonacci (n/2 + 1);
  else
    prime(n/2);


  return 0;
}
```

Q2. A Sober Walk

Our hoary culture had several great persons since time immemorial and king vikramaditya's nava ratnas (nine gems) belongs to this ilk.They are named in the following shloka

धनवंतरी क्षषणकाडमरसिंह राडृ चेठालमदृ धटकर्परः कर्मिदाक ख्यति कराहमिहिरि नृम्ते समाभ्यम रत्नति वै क्स्मस्मिनति तिम्द्म्

Among these, Varahamihira was an astrologer of eminence and his book Brihat Jataak is reckoned as the ultimate authority in astrology.

He was once talking with Amarasimha,another gem among the nava ratnas and the author of Sanskrit thesaurus, Amarakosha.

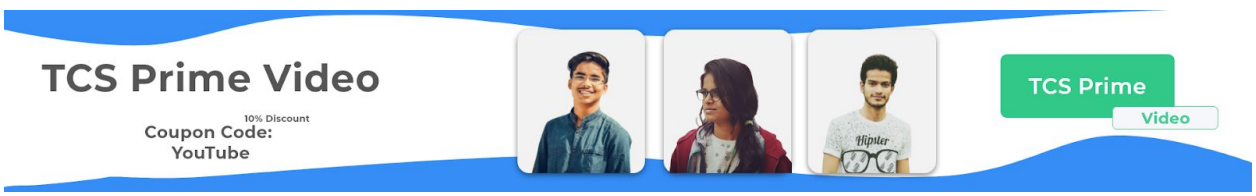
Amarasimha wanted to know the final position of a person, who starts from the origin 0 0 and travels per following scheme.

Constraints:

2<=n<=1000

Scheme

- He first turns and travels 10 units of distance
- His second turn is upward for 20 units
- Third turn is to the left for 30 units
- Fourth turn is the downward for 40 units
- Fifth turn is to the right(again) for 50 units



… And thus he travels, every time increasing the travel distance by 10 units.

Test Cases

Case 1

- Input : 3
- Expected Output :-20 20

Case 2

- Input: 4
- Expected Output: -20 -20

Case 3

- Input : 5
- Expected Output : 30 -20

Case 4

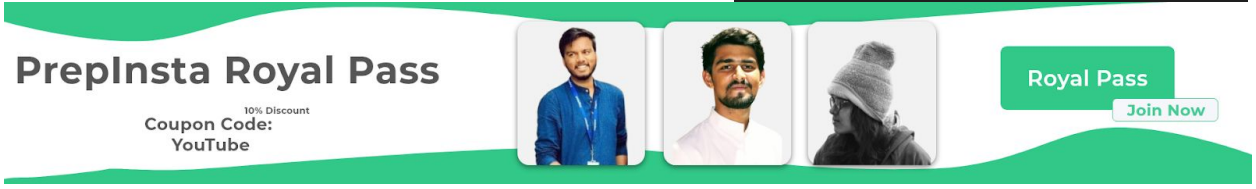
- Input : 7
- Expected Output : 90 -20

Code in C++

```
#include <iostream>
using namespace std;
int main()
{
   int n;
   cin >> n;

   char c = 'R';
   int x = 0, y = 0;
   int distance = 10;

   while(n)
   {
      switch(c)
      {
         case 'R':
         x = x + distance;
         c = 'U';
         distance = distance + 10;
         break;
```



```
        case 'U':
        y = y + distance;
        c = 'L';
        distance = distance + 10;
        break;

        case 'L':
        x = x - distance;
        c = 'D';
        distance = distance + 10;
        break;

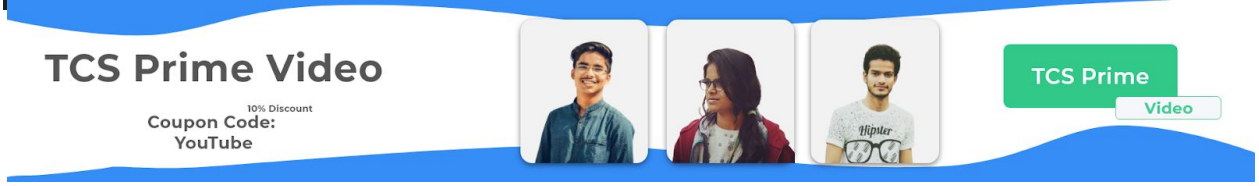
        case 'D':
        y = y - distance;
        c = 'A';
        distance = distance + 10;
        break;

        case 'A':
        x = x + distance;
        c = 'R';
        distance = distance + 10;
        break;
      }
      n--;
    }

    cout << x << " " << y <<endl;
    return 0;
}
```

Code in C

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n;
    scanf("%d", &n);
```



```
    char c = 'R';

    int x = 0, y = 0;

    int distance = 10;

    while(n)

    {

        switch(c)

        {

            case 'R':

            x = x + distance;

            c = 'U';

            distance =
distance + 10;

            break;



            case 'U':

            y = y + distance;

            c = 'L';

            distance =
distance + 10;

            break;



            case 'L':

            x = x - distance;

            c = 'D';

            distance =
distance + 10;

            break;



            case 'D':

            y = y - distance;

            c = 'A';

            distance =
distance + 10;

            break;



            case 'A':

            x = x + distance;

            c = 'R';

            distance =
distance + 10;

            break;

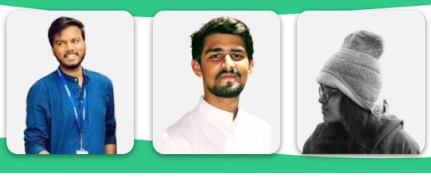
        }

        n--;

    }



    printf("%d %d",x,y);
```

PrepInsta Royal Pass
10% Discount
Coupon Code:
YouTube

Royal Pass
Join Now

```
    return 0;

}
```

Code in JAVA

```
import java.util.Scanner;
public class Main {
  public static void main(String[]
args)
  {
    Scanner sc = new
Scanner(System.in);
    int testCase = sc.nextInt();
    getDistance(testCase);
  }

public static void getDistance(int a)
{
  int distance = 10;
  int x = 0;
  int y = 0;
  char ch = 'R';
  while(a > 0)
  {
    switch(ch)
    {
      case 'R':
      x = x + distance;
      ch = 'U';
      distance = distance+10;
      break;

      case 'U':
      y = y + distance;
      ch = 'L';
      distance = distance + 10;
      break;

      case 'L':
      x = x - distance;
      ch = 'D';
      distance = distance + 10;
      break;

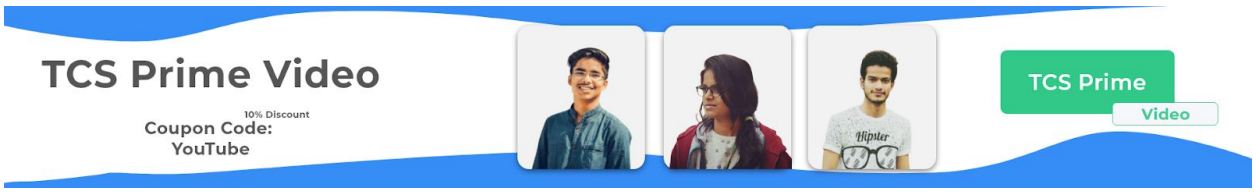
      case 'D':
      y = y - distance;
      ch = 'A';
      distance = distance + 10;
      break;

      case 'A':
      x = x + distance;
      ch = 'R';
      distance = distance + 10;
      break;
    }
    a--;
  }
  System.out.println(x+ " , "+y);
  }

}
```

Code in Python

```
n = int(input())
c = 'R'
dis = 10
```



```
x,y=0,0
for i in range(n):
    if c=='R':
        x=x+dis
        c='U'
        dis=dis+10
    elif c=='U':
        y=y+dis
        c='L'
        dis=dis+10
    elif c=='L':
        x=x-dis
        c='D'
        dis=dis+10
    elif c=='D':
        y=y-dis
        c='A'
        dis=dis+10
    elif c=='A':
        x=x+dis
        c='R'
        dis=dis+10
print(x,y)
```

Q3. Segregate 0s and 1s
Write a program to segregate 0s and 1s. You are given an array of 0s and 1s in random order. Segregate 0s on left side and 1s on right side of the array.

Sample Input:
6
0 1 0 1 1 1
Sample Output:
0 0 1 1 1 1

Code in C++

```
// C++ code to Segregate 0s and 1s in an array
#include <bits/stdc++.h>
using namespace std;

// Function to segregate 0s and 1s
void segregate0and1(int arr[], int n)
{
    int count = 0; // Counts the no of zeros in arr

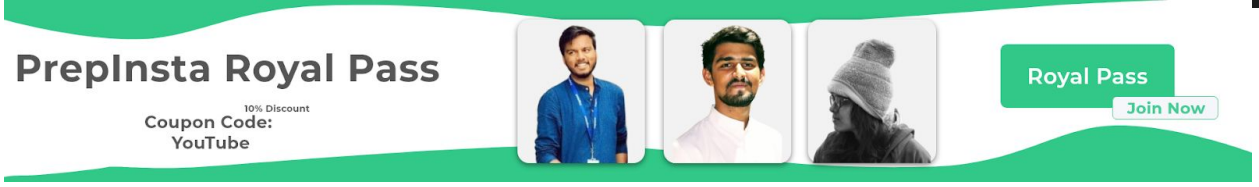
    for (int i = 0; i < n; i++) {
        if (arr[i] == 0)
            count++;
    }

    // Loop fills the arr with 0 until count
    for (int i = 0; i < count; i++)
        arr[i] = 0;

    // Loop fills remaining arr space with 1
    for (int i = count; i < n; i++)
        arr[i] = 1;
}

// Function to print segregated array
void print(int arr[], int n)
{
    cout << "Array after segregation is ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
```



```
}

// Driver function
int main()
{
    int arr[] = { 0, 1, 0, 1, 1, 1 };
    int n = sizeof(arr) / sizeof(arr[0]);

    segregate0and1(arr, n);
    print(arr, n);

    return 0;
}
```

Code in JAVA

```
// Java code to Segregate 0s and 1s in an array
class GFG {

    // function to segregate 0s and 1s
    static void segregate0and1(int arr[], int n)
    {
        int count = 0; // counts the no of zeros in
arr

        for (int i = 0; i < n; i++) {
            if (arr[i] == 0)
                count++;
        }

        // loop fills the arr with 0 until count
        for (int i = 0; i < count; i++)
            arr[i] = 0;

        // loop fills remaining arr space with 1
        for (int i = count; i < n; i++)
            arr[i] = 1;
    }

    // function to print segregated array
    static void print(int arr[], int n)
    {
        System.out.print("Array after segregation
is ");
        for (int i = 0; i < n; i++)
            System.out.print(arr[i] + " ");
    }

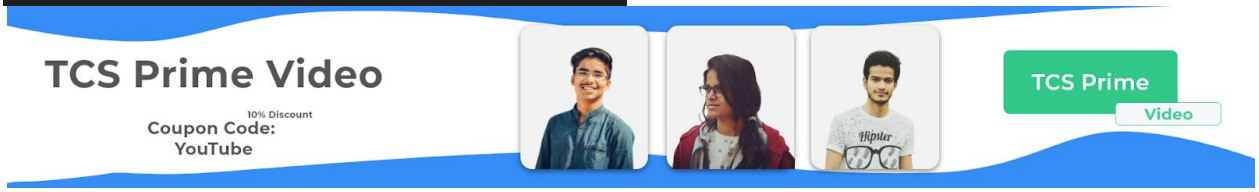
    // driver function
    public static void main(String[] args)
    {
        int arr[] = new int[]{ 0, 1, 0, 1, 1, 1 };
        int n = arr.length;

        segregate0and1(arr, n);
        print(arr, n);

    }
}
```

Q4. Printing Unique Elements in an Array

Write a program to print all the unique elements in the given array.



Input Format:
The first line of input denotes the size of an array(N). The remaining line of input denotes the ‘N’ elements.

Output Format:

Print all the unique elements
Sample Input:
5
1
1
2
2
3

Sample Output:
3

Code in C

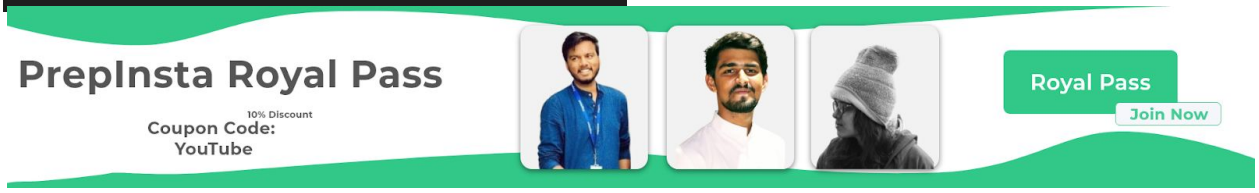
```
#include <stdio.h>
#define MAX_SIZE 50
int
main ()
{
 int a[MAX_SIZE], num[MAX_SIZE];
 int s, i, j, c;
 printf ("Enter length of array: ");
 scanf ("%d", &s);
 printf ("Enter elements of the array array: ");
 for (i = 0; i < s; i++)
   {
    scanf ("%d", &a[i]);
    num[i] = -1;
   }

for (i = 0; i < s; i++)
  {
   c = 1;
   for (j = i + 1; j < s; j++)
  {
   if (a[i] == a[j])
     {
      c++;
      num[j] = 0;
     }
  }

   if (num[i] != 0)
  {
   num[i] = c;
  }
  }

printf ("\nUnique elements in the array are: ");
for (i = 0; i < s; i++)
  {
   if (num[i] == 1)
  {
   printf ("%d ", a[i]);
  }
  }

return 0;
}
```



Code in JAVA

```
import java.util.Scanner;
public class Main
{
    public static void NonRepeating(int a[],int n)
    {
        int count;
        System.out.println("Non Repeating element in array");

       for(int i=0;i<n;i++)
       {
          count=0;
          //initialise the for loop for checking the elenemnt which are not repeated
          for (int j = 0; j < n; j++)
          {
            //check the condition if any number is same so
            //incre
            if(a[i]==a[j] && i!=j)
                count++;
          }
       //if count became zero so print the current element
       if(count==0)
          System.out.print(a[i]+" ");
       }
    }

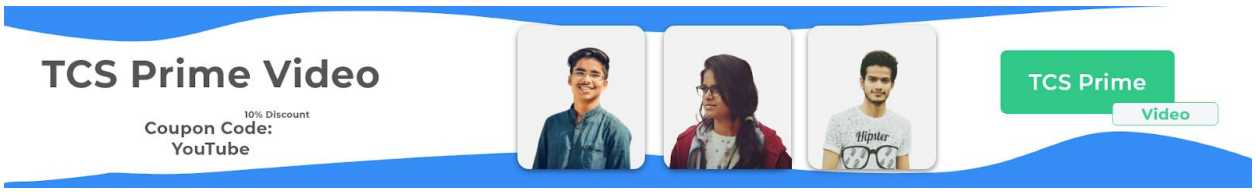
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        int[] a=new int[50];
        System.out.println("enter size of an array");
        int size=sc.nextInt();
        System.out.println("enter elements of an array");
        for(int i=0;i<size;i++)
        {
            a[i]=sc.nextInt();
        }
        //call the NonRepeating function
        NonRepeating(a,size);
    }
}
```

Q5 Binary to Octal

Write a program to convert the given binary number to its equivalent octal value.

Input Format:
Input consists of integer

Output Format:
Refer the sample output format



Sample Input:
10101

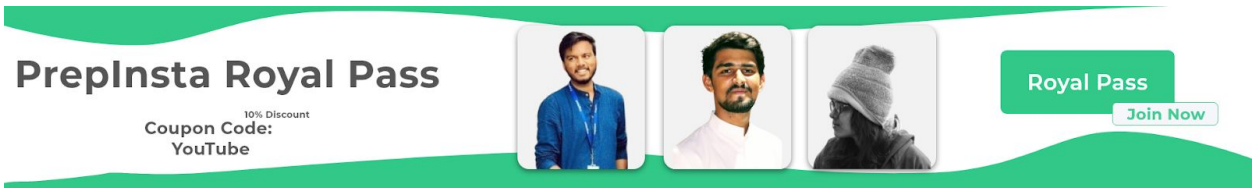
Sample Output:
25

Code Solution in C++:

```
#include <iostream>
#include <math.h>
using namespace std;
//Function to convert binary to octal
int convert(long binary)
{
   int octal = 0, decimal = 0, i = 0,rem;
   //converting binary to decimal
   while(binary != 0)
   {
      rem = binary % 10;
      int res = rem * pow(2,i);
      decimal += res;
      i++;
      binary/=10;
   }
   i = 1;
   //converting decimal to octal
   while (decimal != 0)
   {
      rem = decimal % 8;
      octal += rem * i;
      decimal /= 8;
      i *= 10;
   }
   return octal;
}
//main program
int main()
{
   long binary;
   cout << "Enter a binary number: ";
   //user input
   cin >> binary;
   //calling function
   int octal=convert(binary);
   //printing output
   cout << binary << " in binary = " << octal << " in octal ";
   return 0;
}
```

Code in JAVA

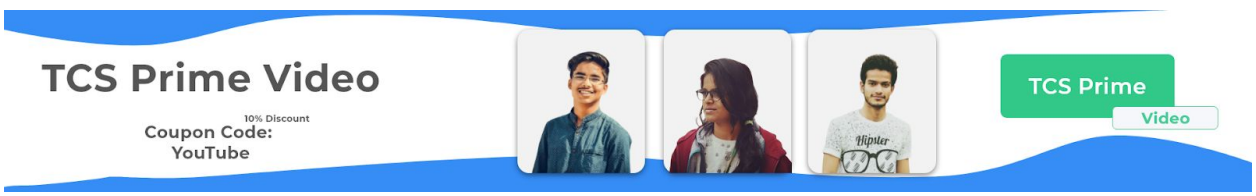
```
import java.util.Scanner;
public class Binary_To_Octal
{
   public static void main(String args[])
   {
      //scanner class object creation
      Scanner sc = new Scanner(System.in);
      //input from user
      System.out.print("Enter a binary number : ");
      int binary = sc.nextInt();
```



```
        //Declaring variable to store decimal
number
        int decimal = 0;
        //Declaring variable to use in power
        int n = 0;
        //writing logic for the conversion from
binary to decimal
        while(binary > 0)
        {
            int temp = binary%10;
            decimal += temp*Math.pow(2, n);
            binary = binary/10;
            n++;
        }
        int octal[] = new int[20];
        int i = 0;
        //writing logic for the conversion from
decimal to octal
        while(decimal > 0)
        {
            int r = decimal % 8;
            octal[i++] = r;
            decimal = decimal / 8;
        }
        //printing result
        System.out.print("Octal number : ");
        for(int j = i-1 ; j >= 0 ; j--)
        System.out.print(octal[j]);
        //closing scanner class(not compulsory, but
good practice)
        sc.close();
    }
}
```

Code in Python

```
Bin_num = 0b10111
#convert using oct() function
Oct_num = oct(Bin_num)
#print number
print('Number after conversion is :' +
str(Oct_num))
```



FACE

FACE Prep's

TCS NQT Solved MOCK Test

This is the PDF version of the solved mock test conducted by FACE Prep on 28th July 2019 for all the NQT aspirants. If you want to learn more or practice unlimited number of questions, you can take a look at our FREE resources –

1. For Aptitude, visit www.faceprep.in/aptipedia
2. For Programming and Coding, visit www.faceprep.in/procoder

If you have any queries, feel free to post it in India's largest Placement Forum www.faceprep.in/discussions

Section 1: English

1) Choose the relevant meaning for the underlined word.

His expression was the most piteous.

a) Joyous
b) Kind
c) Anxious
d) Pathetic

Answer: D

2) Select the correct synonym for the underlined word from the options given below.

His request had been rejected as a battle seemed imminent and every officer was essential.

a) Important
b) Nearing
c) Avoidable
d) Immigrant

Answer: B

FACE

3) Fill in the blank with an appropriate variant of the primary helping verb given in brackets.

___ (be) I the only person who could help you now to complete the project?

a) Is
b) Am
c) Are
d) Was

Answer: B

Explanation: Clue word: 'I'
The subject is the first person singular in the interrogative.so, we have to use helping verb 'am' .

4) Complete the sentences. Use the phrasal verb, which means the same as the verb in brackets.

It would be great if you could _____ this Sunday. (visit me)

a) Come over
b) Came out
c) Come up
d) come across

Answer: A

5) Fill in the blanks with appropriate prepositions.

Their ship came _____ our ship.

a) close to
b) on
c) until
d) since
e) for

Answer: A

Explanation: The ship came close to our boat will be the correct statement. None of the other options fit perfectly in the blank.

Rearrange the following sentences into a coherent paragraph.

1) Mr D Gautam's personality sets him apart from the rest.
2) Nothing is too small for his attention
3) He has a fanatical devotion to detail.
4) This is what makes him a different guy.

6) Which sentence will follow the sentence 3 after rearrangement?

a) 1
b) 2
c) 4
d) None

Answer: B

Explanation: The paragraph should start with statement 1, which introduces the main noun ' Mr. Gautham' .
Statement 3 says' He has a fanatical devotion to detail.'. This is further explained in statement 2, which
says nothing or no detail is too small to be missed by him. Hence 32 link is established. The correct sequence is 1324.

7) After rearrangement which sentence will begin the paragraph?

a) 1
b) 2
c) 3
d) 4

Answer: A

Explanation: The paragraph should start with statement 1, which introduces the main noun ' Mr. Gautham' .
Statement 3 says' He has a fanatical devotion to detail.'. This is further explained in statement 2, which
says nothing or no detail is too small to be missed by him. Hence 32 link is established. The correct sequence is 1324.

Directions: Identify the error(s) in the following sentences.

8) After the detective examined the fingerprints / left at the scene of the crime, / he concluded that his prime suspect / committed the theft.

a) After the detective examined the fingerprints
b) left at the scene of the crime,
c) he concluded that his prime suspect
d) committed the theft

Answer: D

Explanation:
The key insight in this question is seeing that the crime occurred before the police investigated. you need to use the past perfect tense for the earlier action and the simple past tense for the more recent action. In this question, had committed is correct since the crime occurred before the police conducted
the investigation and reached their conclusion. Answer is D

9) Roberts Consultants, a large regional consulting firm /experiencing rapid growth,/recently indicated that they / would expand into new territory.

a) Roberts Consultants, a large regional consulting firm
b) experiencing rapid growth,
c) recently indicated that they
d) would expand into new territory

Answer: C

Explanation:
It is a Pronoun-Antecedent error. Since the sentence speaks about Robert consultants, the pronoun used should be It not they. Hence the error lies in part 3 of the sentence. So the answer is C

10) Rewrite the following sentences after changing the form of speech.

Bella said that she didn't have to pay for that outfit.

a) Bella said, "I don't have to pay for that outfit"
b) Bella said, "I didn't have to pay for that outfit"
c) Bella said, "I don't want to be paying for this outfit"
d) Bella said, "I don't have to pay for this outfit"

Answer: D
Explanation:
Given sentence is in the past tense. Thus, the sentence in direct speech must be in the present tense. The
indirect speech has "that", which is the result of the conversion of "this" to "that". So, indirect speech, 'this'
must have been there.

Direction: In the following passage, there are blanks each of which has been numbered. These numbers
are printed below the passage and against each four words have been suggested, one of which fills the
blanks appropriately.

BRICS leaders are gathered in the Russian town of Ufa for the bloc's annual summit, and Internet governance is high on their ___1___. The summit comes at a ____2____juncture in India's internet diplomacy. Last month in Buenos Aires, at a conference organised by the Internet Corporation for Assigned Names and Numbers (ICANN), IT minister Ravi Shankar Prasad offered an "Indian ___3____ for
the Internet". ICANN is the organisation that manages the Domain Name System, which serves as the
___4____ for all technical and commercial activity in cyberspace. In his recorded message, Mr. Prasad declared India would move away from state-led approaches to ___5____ the Internet, preferring instead a mechanism that co-opts the private sector and civil society into the policymaking process.

11) Find out the appropriate word for blank 1.

a) agenda
b) goal
c) aim
d) objective

Answer: A

Explanation:
Agenda means a list of items to be discussed at a formal meeting and as the context here is of a formal
meeting of world leaders, thus option A is the most appropriate response.

12) Choose the correct option for the blank 2.

a) trivial
b) crucial
c) acute
d) exiguous

Answer: B

Explanation:
Trivial means of little value or importance. Crucial means decisive or critical, especially in the success or
failure of something. Acute means (of an unpleasant or unwelcome situation or phenomenon) present or experienced to a severe or intense degree. Exiguous means very small in size or amount. The context
here is of a meeting between international leaders and hence it is most likely to make some big decisions. Out of the available options, only 'crucial' serves the context.

FACE

13) Choose the correct option for the blank 3.

a) objective
b) goal
c) sight
d) vision

Answer: D

Explanation:
'Compute' is rrelevant in accordance with the context of the passage. The conference being of international importance, it is highly unlikely that a minister will showcase Indian 'objective' or 'goal' as it would be highly narrow approach and would bring diplomatic pressure. 'Vision' is the only option that suits the
context.

14) Choose the correct option for the blank 4.

a) stability
b) anchor
c) harmony
d) backbone

Answer: D

Explanation:
Anchor means a person or thing which provides stability or confidence in an otherwise uncertain situation. Harmony means the state of being in agreement or concord. Posterity means the descendants
of a person. The context here is of showing the importance of domain name system in the technical activities and not show it being the stabilising feature, so 'backbone' will take precedence over 'anchor'.

15) Find the appropriate word for the blank 5.

a) altered
b) chaotic
c) unpredictable
d) bizarre

Answer: C

Explanation:
The monsoon patterns are best described as "unpredictable" because the change in climate results in "unpredictable" monsoon patterns. Hence choice 3 is most appropriate.

FACE

Section 2: Quantitative Aptitude

1) If a student walks from his house to school at 5 kmph, he is late by 30 minutes. However, if he walks at 6 kmph, he is late by 5 minutes only. The distance of his school from his house is :

a) 2.5 km
b) 3.6 km
c) 5.5 km
d) 12.5 km

Answer: D

Explanation:
Let the required distance be x km. Then, x5 – x6 = 2560 [Difference between two times is 25 min]

⇒ 12x – 10x = 25 ⇒ 2x = 25
⇒ x = 12.5 km

2) A man covers a certain distance on scooter. Had he moved 3 kmph faster, he would have taken 40 minutes less. If he had moved 2 kmph slower, he would have taken 40 minutes more. The distance (in km) is :

a) 20
b) 36
c) 37.5
d) 40

Answer: D

Explanation:
Let distance = x km and usual rate = y kmph.
Then,
(x/y) - (x/y+3) = 40/60 => 2y(y+3)=9x ....(i)
And,
(x/y-2) - (x/y) = 40/60 => y(y - 2) = 3x ....(ii)
On dividing (i) by (ii), we get: x = 40.

3) A sum of money is to be divided among P, Q and R in the ratio of 2 : 3 : 5. If the total share of P and R together is Rs 400 more than that of Q, what is R's share in it?

a) Rs. 100
b) Rs. 200
c) Rs. 300
d) Rs. 500

Answer: D

Explanation:
Let P = 2x ,
Q = 3x and R=5x.
Now P+R-Q = 400
2x+5x-3x = 400
hence x =1OO R = 5x = 500.

4) If -2=<x=<2 and 3<=y<=8, which of the following represents the range of all possible values of x-y?
a) 5<=y-x<=6
b) 1<=y-x<=5

c) 1<=y-x<=6
d) 1<=y-x<=10

Answer: D

Explanation:
-2 = x = {-2,-1,0,1,2}
3<=y<=8 => y={3,4,5,6,7,8}
y-x=(3+(-2)) to (8+2)
starting range of y+x to last range of y+x
i,e 1<=y-x<=10

5) A passenger jet took three hours to fly 1800 miles in the direction of the jet stream. The return trip against the jet stream took four hours. What was the jet's speed in still air and the jet stream's speed?

a) 600 mph, 100 mph
b) 525 mph, 75 mph
c) 575 mph, 25 mph
d) 450 mph, 125 mph

Answer: B

Explanation:
In each case, the distance equation will be "(the combined speed) times (the time at that speed) equals (the total distance travelled)"
With the jetstream: (p + w)(3) = 1800
Against the jetstream: (p – w)(4) = 1800
p + w = 600
p – w = 450
Then, by adding down, 2p = 1050 so p = 525, and w must then be 75.
The jet's speed was 525 mph and the jetstream wind speed was 75 mph.

6) The admission fee at a small fair is $1.50 for children and $4.00 for adults. On a certain day, 2200 people enter the fair and $5050 is collected. By fact, the number of children visiting the fair is higher than the number of adults visiting the fair by a margin of 900. Now, on that certain day given above, the number of children exceeded the adults by?

a) 700
b) 800
c) 900
d) 1000

Answer: B

Explanation:
Let adults be denoted as x
Let children be denoted as y
2200 people attend the fair
x + y = 2200 ------ (1)
Child admission fee is $1.50
Adult admission fee is $4.00
The total amount collected is $5050.00
4x + 1.5y = 5050 ------ (2)
solving the equations we get
y = 1500, x = 700
1500 children attended the fair
700 adults attended the fair.
Exceedance is 1500-700 = 800.

7) A schoolyard contains only bicycles and 4 wheeled wagons. On Tuesday, the total number of wheels in the schoolyard was 166. What could the possible number of bicycles?

a) 14
b) 10
c) 12
d) 11

Answer: D

Explanation:
Let no of bicycles=x and wagon=y
so,
2x+4y=166
= x+2y=83
as x and y should be integer.
x=odd no.
the only answer that satisfies the condition is 11.

8) 9 years ago, Andromeda's age was twice Achille's age. 9 years hence, Andromeda's age will be 4/3 times the age of Achilles'. Find Andromeda's present age in binary numbers.

a) 11011
b) 11000
c) 1001
d) 1010

Answer: A

Explanation: let A be the the age of andromeda's, B be the age of Achilles
9 YEARS AGO,

A-9=2(B-9)
A=2B-9
B= (A+9)/2 ----------(1)

9 YEARS HENCE,

A+9=4/3(B+9)
3A+27=4B+36
3A=4B+9 ----------(2)

Substitute (1) in (2).

3A = 4((A+9)/2)+9)
= 2A+9+18
3A =2A+27
A=27

THE BINARY NUMBER OF 27 IS 11011.

9) 21 people meet and shake hands. The maximum number of handshakes possible if there is to be no 'cycle' of handshakes is (A cycle of handshakes is a sequence of people a1, a2, ... , aK such that the pairs (a1, a2), (a2, a3),... , (a (k-1), ak), (ak, a1) shake hands)

a) 17
b) 18
c) 19
d) 20

Answer: D

Explanation:
If handshakes are taken in a random manner then max. handshakes = 21*20/2*1 = 210
If all the persons are there in a straight line then number of handshakes = 21-1 = 20

10) The ticket to Disneyland will cost anywhere from 1p to 63p. You need to produce the exact change at the ticket counter and have with you a 63p coin, So you decide to break this into change but you want to carry with you as few coins as possible. Assuming that coins of all denominations are available, how many coins (denominations) would you split the 63p into?

a) 33
b) 63
c) 6
d) 64

Answer: C

Explanation:
The coins (denominations) we want are- 1, 2, 4, 8, 16 and 32.
using 1 and 2 we can make 3, using 1,2 and 4 we can make all numbers up to 7 and so on...
so, 6 is the answer.

OR
Write the given number to binary form and count the number of ones.
63 can be written in binary as 111111.
Number of 1 is 6.
So, 6 is the answer.

11) Peter is twice as old as Paul was when Peter was as old as Paul is now. The combined age of Peter and Paul is 42 years. How old is Peter now?

a) 21
b) 18
c) 24
d) 26

Answer: C

Explanation:
Let Peter's age be x.
Paul(42-x). Peter was (42-x) before (x-(42-x))=(2x-42)years.
so that time Paul was ((42-x)-(2x-42))=(84-3x).
Now from question, we can say x=2(84-3x)
So x=24

FACE

12) There are two water tanks A and B, A is much smaller than B. While water fills at the rate of one litre every hour in A, it gets filled up like 10, 20, 40, 80, 160....., in-tank B. (At the end of first hour, B has 10 liters, second hour it has 20, and so on) If tank B is 1/16 filled after 17 hours, what is the total durations required to fill it completely?

a) 4 hours
b) 21 hours
c) 22 hours
d) 24 hours

Answer: B

Explanation:
For every hour, water in tank in B is doubled. Let the duration to fill the tank B is x hours. x/16 part of water in tank of B is filled in 17 hours, Next hour it is doubled so, 2*(x/16) part i.e (x/8) part is filled in 18 hours, Similarly (x/4)th part in 19 hours,(x/2)th part is filled in 20 hours, (x/)th part is filled in 21 hours, So answer is 21 hours.

13) The ratio of a two-digit natural number to a number formed by reversing its digits is 4 : 7. Which of the following is the sum of all the numbers of all such pairs? **(advanced aptitude question)**

a) 99
b) 198
c) 330
d) 132

Answer: A
Explanation:
Let the two digit number be 10a + b and the number formed by reversing its digits be 10b + a.
10a + b/10b + a= 4/7
70a + 7b = 40b + 4a
66a = 33b
Therefore,
a/b= 1/2
So, let us list down all possible values for a and b.
a b Number Reversed Number
1 2 12 21
2 4 24 42
3 6 36 63
4 8 48 84
Hence, the sum of all the numbers would be,
12 + 21 + 24 + 42 + 36 + 63 + 48 + 84 = 330.

FACE

14) When 40! is expressed in base 8 form, what is the last non–zero digit in the base 8 expansion? **(advanced aptitude question)**

a) 2
b) 6
c) 4
d) 2 or 6

Answer: C

Explanation:
We need to find the largest power of 8 that divides 40!.
We need to find the largest power of 2 that divides 40!
This is given by (40/2) and then successive division by 2. = 20 + 10 + 5 + 2 + 1 = 38
So, 238 divides 40! Or, (23)12 × 22 divides 40!
(23)12 divides the number, or the base 8 representation ends with 12 zeroes. Now, the base 8 representation of this number will be some (abcd...n)8 × (1000000000000)8. Now, (abcd...n)8 does not end in 0 and is a multiple of 22. The last digit has to be 4.
The last non–zero digit is 4

15) Consider a class of 40 students whose average weight is 40 kgs. m new students join this class whose average weight is n kgs. If it is known that m + n = 50, what is the maximum possible average weight of the class now? **(advanced aptitude question)**

a) 40.18
b) 40.56
c) 40.67
d) 40.49

Answer: B
Explanation:
If the overall average weight has to increase after the new people are added, the average weight of the new entrants has to be higher than 40.
So, n > 40
Consequently, m has to be < 10 (as n + m = 50)
Working with the “differences” approach, we know that the total additional weight added by “m” students would be (n - 40) each, above the already existing average of 40. m(n - 40) is the total extra additional weight added, which is shared amongst 40 + m students. So, m(n – 40) / (m + 40) has to be maximum for the overall average to be maximum.

At this point, use the trial and error approach (or else, go with the answer options) to arrive at the answer. The maximum average occurs when m = 5,and n = 45

And the average is 40 + (45 – 40) * (5 / 45)
= 40 + (5 / 9)
= 40.56 kgs

Section 3: Programming Concepts

1) The below program(sum.c) is run with two command line parameters mentioned below: What will be the output of the below program?

```
#include <stdio.h>
int main(int num, char * argv[])
{
   int sum;
   sum = argv[1] + argv[2];
   printf(&amp;quot;%d&amp;quot;, sum);
   return 0;
}
```

a) 3
b) Error
c) 1
d) Garbage value

Answer: B

Explanation: Here argv[1] and argv[2] char * type.

2) Which one of the following is an application of Queue Data Structure?

a) When a resource is shared among multiple consumers.
b) When data is transferred asynchronously (data not necessarily received at same rate as sent) between two
processes
c) Load Balancing
d) All of the above

Answer: D

Explanation: Queue is used when things don't have to be processed immediately, but have to be processed in First In First Out order like Breadth First Search. This property of Queue makes it also useful in above kind of scenarios.

3) The difference between procedural and object oriented Programming is

a) In Object Oriented Programming, the program is divided into functions and in Procedural oriented Programming, the program is divided into objects.
b) POP does not have any access specifier whereas OOP has access specifier
c) In OOP data can be accessed easily and in POP data cannot move to another function
d) There is no difference

Answer: B

Explanation: OOP has access specifiers such as private, public and protected

4) Which is correct with respect to the size of the datatypes?

a) char > int > float
b) int > char > float
c) char < int < double
d) double > char > int

Answer: C
Explanation: char has lesser bytes than int and int has lesser bytes than double in any system

5) Given an array arr = {5,6,77,88,99} and key = 88; How many iterations are done in binary search until the element is found?

a) 1
b) 3
c) 4
d) 2

Answer: D

Explanation: Iteration1 : mid = 77; Iteration2 : mid = 88;

6) What is the time complexity of the following code:

```
int a = 0;
for (i = 0; i < N; i++) {
    for (j = N; j > i; j--) {
        a = a + i + j;
    }
}
```

a) O(N)
b) O(N*log(N))
c) O(N * Sqrt(N))
d) $O(N^2)$

Answer: D

Explanation: The above code runs total no of times
= N + (N – 1) + (N – 2) + … 1 + 0
= N * (N + 1) / 2
= 1/2 * N^2 + 1/2 * N
O(N^2) times

FACE

7) Which of the following statements are correct about an array?

1: The array int num[26]; can store 26 elements.
2: The expression num[1] designates the very first element in the array.
3: It is necessary to initialize the array at the time of declaration.
4: The declaration num[SIZE] is allowed if SIZE is a macro.

a) 1
b) 1,4
c) 2,3
d) 2,4

Answer: B
Explanation:

- The array int num[26]; can store 26 elements. This statement is true.
- The expression num[1] designates the very first element in the array. This statement is false, because it designates the second element of the array.
- It is necessary to initialize the array at the time of declaration. This statement is false.

- The declaration num[SIZE] is allowed if SIZE is a macro. This statement is true, because the MACRO just replaces the symbol SIZE with given value. Hence the statements '1' and '4' are correct statements.

8) Consider a 13 element hash table for which f(key)=key mod 13 is used with integer keys. Assuming linear probing is used for collision resolution, at which location would the key 103 be inserted, if the keys 661, 182, 24 and 103 are inserted in that order? **(advanced programming concepts question)**

a) 0
b) 1
c) 11
d) 12

Answer: B
Explanation:
661 mod 13 = 11
182 mod 13 = 0
24 mod 13 = 11, already filled, so after linear probing it will get index 12
103 mod 13 = 12, already filled, so after linear probing it will get index 1

9) The height of a binary tree is the maximum number of edges in any root to leaf path. The maximum number of nodes in a binary tree of height h is: **(advanced programming concepts question)**

a) 2^(h) -1
b) 2^(h-1) – 1
c) 2^(h+1) -1
d) 2*(h+1)

Answer: C

Explanation: Maximum number of nodes will be there for a complete tree. Number of nodes in a complete tree of height h = 1 + 2 + 2^2 + 2*3 + ... 2^h = 2^(h+1) – 1

10) Which of the following tree traversal uses a queue data structure? **(advanced programming concepts question)**

a) Preorder
b) Inorder
c) Postorder
d) Level order

Answer: D

Explanation:
Level order traversal uses a queue data structure to visit the nodes level by level.

Section 4: Coding

1) Print the Words Ending with Letter S. This program takes a string as input and print the words ending with letter s.

Input:
this was mine

Output:
this
was

Solution:

```
#include <stdio.h>
#include <string.h>
char str[100];
void main()
{
int i, t, j, len;
scanf("%[^\n]s", str);
len = strlen(str);
str[len] = ' ';
for (t = 0, i = 0; i < strlen(str); i++)
{
if ((str[i] == ' ') && (str[i - 1] == 's'))
{
for (j = t; j < i; j++)
{
printf("%c", str[j]);
}
t = i + 1;
printf("\n");
}
else
{
if (str[i] == ' ')
{
t = i + 1;
}
}
}}}
```

FACE

FACE
Prep

Verbal Ability

FACE Prep

Question

Fill in the blanks with the correct option. (TCS 2022)

Radha walked to school each day. Her school was (A) _____ from her house. She was always punctual and arrived early to school, she believed in the proverb (B) _____. At school, she would usually help her teacher set up the class before students came in. Radha was a brilliant student too. She excelled in her studies and was kind and helpful to all her friends. She never gave anyone (C) _____ even if they did not like (D). _____

FACE Prep

Question

A) (A) at a stony throw (B) the early bird caught all the worms (C) a cold shoulder (D) see each other eye to eye.

B) (A) at a stone’s throw (B) the early bird gets the worm (C) a cold shoulder (D) seeing each other eye to eye.

C) (A) as the crow flew (B) only the early bird gets the worm (C) a rough shoulder (D) seeing eye to eye.

D) (A) at a stone’s throw (B) an early bird catches the worm (C) a crossed shoulder (D) seeing one another in the eye.

FACE Prep

Question

Fill in the blanks with the correct words from the given options.

Investing in wine is becoming a popular investment alternative these days as demands for fine wines is increasing, especially in China. More and more wine brokerage houses are being set up in various places in Asia, (A) _____ keen investors. In facts, Bloomberg has been featuring interviews and (B) _____ about wine investing in recent months, mainly focusing on its positive side as well as its negatives. Because of these. I have been seriously considering such investment to (C) _____ my portfolio. It is therefore (D) _____ for me to study the pros and cons of wine investing.

FACE Prep

Question

A) (A) Wooing (B) features (C) diversify (D) incumbent

B) (A) Attracting (B) traits (C) contrast (D) compulsory

C) (A) Inviting (B) hallmark (C) balance (D) obligatory

D) (A) Courting (B) qualities (C) deviate (D) necessary

FACE Prep

Question

One part of the sentence below may contain an error. Identify the part, if there is no error choose no error

He is so intelligent as he can crack any government examination

A) No error

B) government examination

C) He is so intelligent

D) as he can crack any

FACE Prep

Question

Fill in the blanks with the correct words from the given options. (TCS 2019)

One of the main reasons of corruption in elections today is the (A) _____ of power which (B) _____ the politicians so much that they feel no (C) _____ of conscience in adopting any underhand method to come out successful. The watergate scandal in the U.S.A is a/an (D) _____ example to testify to the fact how even the top level politicians can stoop to the lowest level in order to maintain themselves in power.

FACE Prep

Question

A) allurement (B) shuns (C) pricks (D) ant

B) lure (B) haunts (C) qualms (D) éloquent

C) enticement (B) affects (C) scruples (D) ethereal

D) temptation (B) resorts (C) morals (D) appropriate

FACE Prep

Question

Select the most appropriate option to fill in the blank.

Wisdom is _____ mainly through adversity.

A) pained

B) remained

C) gained

D) contained

E) None of these

FACE Prep

Question

Given below is a paragraph with five sentences, which are presented in a jumbled order. Write down the best order (e.g ABCDE) which would form a meaningful, unified, and coherent paragraph.

A. To provide him or her with good data and interesting problems to solve and
B. Of course with the opportunity to work with other data science and analytics people
C. The most important way to retain a data scientist is
D. Given the strong need for intellectual stimulation and growth

CABD

FACE Prep

Question

Select the most appropriate option to fill in the blank. (TCS 2020)

Speaking in more general terms, English is the native language of the _____ of several countries.

A) populist

B) popular

C) population

D) populate

FACE Prep

Question

Given below is a paragraph with five sentences, which are presented in a jumbled order. Write down the best order (e.g ABCDE) which would form a meaningful, unified, and coherent paragraph. (TCS 2020)

A. In Kusanagi’s cases, not because they annoyed him but
B. Unless he had something specific in mind, for Yukawa’s policy had always been to avoid direct involvement
C. Because of deference to Kusanagi’s position
D. Kusanagi didn’t think Yukawa would just drop in on a suspect in an unsolved case

DBAC

FACE Prep

Question

Read the following passage and choose the option that best expresses the thought given in the passage.

While it is true that most animals and birds can utter cries which indicate emotion, anger, fear and affection, nevertheless, their range of sounds is very limited, so that they are not capable of expressing more complicated emotions. On the other hand, man can express a very wide range of ideas and emotions through speech.

FACE Prep

Question

A) Birds and animals do not possess complex thoughts as man does.

B) Birds and animals have the same emotions as that of man

C) Man has higher speech abilities when compared to animals and birds and can express more complex thoughts and emotions

D) Only a higher speech order ensures possession of complex thoughts and emotions

FACE Prep

Question

One part of the sentence below may contain an error. Identify the part, if there is no error choose no error. (TCS 2019)

If we want to restore better health for our planet then both the rich and the poor nations shall come together to share equal responsibility in combating problems of climate change.

A) and the poor nations shall come together to share equal

B) responsibility in combating problems of climate change

C) No error

D) If we want to restore better health for our planet then both the rich

FACE Prep

Question

One part of the sentence below may contain an error. Identify the part, if there is no error choose no error.

He thought of himself as being poor even though his monthly salary exceeded one lakh.

A) No error

B) even through his monthly

C) salary exceeded one lakh

D) He thought of himself as being poor

FACE Prep

Question

One part of the sentence below may contain an error. Identify the part, if there is no error choose no error. (TCS 2020)

I advised him not to went there without all precautions.

A) No error

B) to went there without

C) all precautions

D) I advised him not

FACE Prep

Question

One part of the sentence below may contain an error. Identify the part, if there is no error choose no error.

2020 was most horrible year in recent human history.

A) year in recent

B) human history

C) 2020 was most horrible

D) No error

FACE Prep

Question

One part of the sentence below may contain an error. Identify the part, if there is no error choose no error.

The celebrated music composers, AR Rahman and Ananya Birla dedicated their cheer song to the athletes participating in the Tokyo Olympics.

A) dedicated their cheer song to the athletes

B) The celebrated music composers, AR Rahman and Ananya

C) Birla

D) participating in the Tokyo Olympics

No error

FACE Prep

Question

One part of the sentence below may contain an error. Identify the part, if there is no error choose no error. (TCS 2022)

He is unable to come to your party tomorrow because he has many work to do.

A) party tomorrow because

B) He is unable to come to your

C) No error

D) he has many work to do

FACE Prep

Question

Which of the following is the most suitable reported speech form of the following sentence?

Chef Kunal Kapoor says, “Ask me anything related to food and I will answer them.”

A) Chef Kunal Kapoor says he will answer anything related to food.

B) Chef Kunal Kapoor said that he will answer anything related to food.

C) Chef Kunal Kapoor says he would answer anything related to food.

D) Chef Kunal Kapoor told them that he would answer any question related to food.

FACE Prep

Question

In the following passage, some words have been deleted. Read the passage carefully and select the most appropriate option to fill in the blanks.

As you well know, we are faced with a whole series of global environmental problems which are
harming the biosphere and human life in (A) _____ ways that may soon become (B) _____.
The great challenge of our time is to create (C) _____ communities; that is, social and cultural environments in which we can satisfy our needs without (D) _____ the chances of future generations.

FACE Prep

Question

A) (A) Terrifying (B) irrecoverable (C) untenable (D) decreasing

B) (A) Composing (B) irreparable (C) supportable (D) glorifying

C) (A) Frightening (B) irredeemable (C) maintainable (D)

D) (A) Alarming (B) irreversible (C) sustainable (D) diminishing

FACE Prep

Question

Read the passage and answer the questions that follow: (TCS 2021)

Rosio Sánchez, a Mexican-American chef who lives in Copenhagen, makes the best tortillas in Scandinavia. That, she admits, isn't necessarily saying much – like laying claim to the best pizza in Indonesia. "It was so much worse," she said, describing the state of Mexican food when she arrived in 2010 to work as a pastry chef at the celebrated restaurant Noma. "Imagine the worst Tex-Mex food in America, and imagine that being passed on like a game of telephone, by people who have no idea what real Mexican food is." That is beginning to change, and not only in Copenhagen, where Sánchez has opened a taqueria with freshly ground, hand-pressed corn tortillas.

FACE Prep

Question

Read the passage and answer the questions that follow:

It goes far beyond tacos and tortillas, though: Mexican cuisine has made the leap to the global stage of fine dining.

Chefs are making house-cured chorizo in Toronto, and Michelin-starred chilaquiles at Punto MX in Madrid. Last week, the Houston chef Hugo Ortega, who began his working life as a shoeshine boy in Mexico City, received the James Beard Foundation’s award for best chef in his region: a
first for a Mexico-born chef.

FACE Prep

Question

Read the passage and answer the questions that follow:

“Everywhere, I see a new respect for Mexican culture,” said Martha Ortiz, a celebrity chef in Mexico who is opening a warmly elegant restaurant, Ella Canta, in the InterContinental London Park Lane hotel this summer. Ten years ago, when a taco in London might easily have contained canned baked beans, the idea of a Mexican restaurant in a posh hotel would have been mystifying.

“Our traditional food has always had a high value at home, and there is a lot of respect for the women who produce it,” she said. “But for people internationally to be excited about it and willing to pay for it? That is new.”

FACE Prep

Question

What is the main idea of the passage?

A) There is more to Mexican food than just tacos and tortillas.

B) There are celebrity chefs in Mexican cuisine nowadays.

C) Mexican food is being globally appreciated now.

D) Mexican restaurants are opening in different countries.

FACE Prep

Question

Why is the Mexican food being compared to a game of telephone?

A) It was like the worst Tex-Mex food being served in the name of Mexican food.

B) It reached people instantly like a telephone message.

C) Its naturally was drastically changed when it reached people in other countries.

D) It was passed from hand before reaching the consumers.

FACE Prep

Question

Which of the following is NOT a Mexican food item?

A) Tortillas

B) Chorizo

C) Chilaquiles

D) Taqueria

FACE Prep

Question

In the context of the given passage, which of the following is closest in meaning to the word ‘mystifying’?

A) Appropriate cuisine

B) Perfectly suitable

C) Impossible to explain

D) A special occurrence

FACE Prep

Question

Read the passage carefully and answer the questions that follow .

Over the past 18months, corporate India and the world of finance has been roiled by a peculiar problem: the promoters of a slew of companies have been eased out of board rooms because they pledged their shares to avail loans and then defaulted. The suicide of Cafe Coffee Day’s V.G.Siddhartha in July this year, the roller coaster ride that Yes Bank Ltd’s stock went through when its former promoter Rana Kapoor dealt with lenders: and even the unfolding financial scandal at Karvy stock broking have one thing in common – the use , or rather, overuse of a humble instrument called LAS(loan against shares).

FACE Prep

Question

The product has been in existence since trading began in Indian markets. It was primarily meant as a tool for stock market operators, who used their existing shares as security to borrow and speculate in the market. It was meant to be short term. And the sums were supposed to be small. Traditional banks even have an upper limit; Rs 20 lakh, if the collateral is purely shares (not applicable to brokerage firms). But mutual funds and non banking financial companies caught in a credit squeeze and a slow down in traditional bank lending, promoters increasingly began to rely on pledged shares to raise funds. In many cases, those funds came in through channels that are relatively less regulated than traditional banks, setting up a perfect storm.

FACE Prep

Question

When economic growth began to fall and “sentiments” began to collapse, lenders inevitably began calling in on the pledge or, in some cases, even selling the equity, reducing company promoters to minor shareholders in their own firms.
The fortunes of at least six big promoters are on the line, including Zee Entertainment Enterprises Ltd.'s Subash Chandra and Reliance Group’s Anil Ambani.
The sparkling diamonds, or shares, which are forever, are usually never meant to be sold. It is the equivalent of bringing out the family silver. “how do you get funding in any market- you borrow , beg or steal,” said Amit Tandon, managing director of Institutional Investors Advisory Services(IIAS), “

FACE Prep

Question

In a bad market such as this, borrowing is not happening because no one will lend. You are too proud to beg. So, you steal from your existing pool which is shares, pledged them, and borrow.”
Perhaps, the promoters never thought they would lose control. Perhaps, corporate India never anticipated a multi-quarter economic slowdown. Perhaps, internal risk-management mechanisms were extremely weak resulting in risky hedges and abuse of LAS. One thing is certain though; The unfolding saga of India Inc.’s troubled alliance with LAS captures, in many ways, the degree of desperation among certain sections of the business community. It is a cautionary tale which could only get worse before it gets any better.

FACE Prep

Question

As per regulations issued by the capital market regulator Securities and Exchange Board of India(SEBI), a company needs to disclose the amount of pledged shares, when the pledge is created, to whom the shares are pledged, and for what reason they have been pledged.

However, an analysis done by Mint shows that despite regulatory tightening, the disclosures are missing. In the case of at least four companies including Yes Bank, firms have not even disclosed to whom the shares have been pledged- a clear violation of disclosure norms.

Adani Group, GMR Group and Vedanta Ltd practice a more innovative strategy. They disclose the name of the trustee but not the pledgee, giving investors no indication regarding the purpose of the pledge.

IFACE Prep

Question

For quite some time, SEBI and RBI have been on the same page regarding the need to crack down on excessive pledging. The capital market regulator had expressed concern particularly about how regulated mutual funds have now assumed the role of lenders, rather than acting on behalf of investors.
Several mutual funds have begun to invest in LAS products in their fixed maturity plan schemes, that is, papers with pledged shares as the underlying collateral. Due to a fear of default, the funds sometimes roll over the maturity date in the hope that a stake sale would result in realization of dues. SEBI has taken one step after another to stem this practice.

FACE Prep

Question

Why did the promoters of many companies not find a seat in the board room?

A) They were interested in dealing in stocks, rather than managing the companies.

B) They pledged their shares for loans, but could not settle the due.

C) The stocks of the companies plummeted and they became minor shareholders.

D) The stocks of the companies plummeted and they became major share holders.

FACE Prep

Question

Which of the following statements are TRUE regarding the financial instrument ‘ Loan against shares’ (LAS)?

A) Traditional banks don’t have any upper limit while issuing loans through LAS.

B) NBFCs do not have the authority to issue loans through LAS.

C) The financial instrument LAS primary was designed to issue loans for long term.

D) None of the above

FACE Prep

Question

What does the author mean by the expression: 'It is a cautionary tale which could only get worse before it gets any better’?

A) The incident can get become worse before showing signs of improvement, and all business men should learn from it.

B) The tale is a proverbial one and some experience is to be gained from it.

C) No promoter should ever take loan through LAS and the situation should be remembered by all.

D) None of the above

FACE Prep

Question

What does the analysis done by Mint show?

A) The analysis shows that regulations issued by SEBI mandate a company to disclose the amount of pledged shares, when the pledge is created, to whom the shares are pledged, and for what reason they have been pledged.

B) The analysis shows that at least four companies are in violation of the regulatory norms issued by SEBI.

C) The analysis shows that many companies disclose the name of the trustee but not the pledgee, giving investors no indication regarding the purpose of the pledge.

D) B and C

FACE Prep

Question

Which of the following is the author’s opinion regarding shares?

A) The author compares ‘share’ with ‘sparling diamond’.

B) The author thinks that usually one should never sell ‘shares’.

C) The author draws an analogy between ‘shares’ and ‘family silver’.

D) A, B and C

FACE Prep

Question

Directions: In this question, a sentence has been divided into four parts marked as I, II, III and IV. You need to find which part/parts does not/do not have any error in terms of grammatical or contextual usage. If the sentence is absolutely correct, mark (D) as your answer.

I. A tradition, said to have been starting by St. Francis of Assisi.
II. In 1223 at Greccio, central Italy, the Nativity scene
III. Emphasizes the spiritual aspects of Christ's birth and advocates
IV. Worship over materialism during the festivity season

(TCS 2019)

FACE Prep

Question

A) Only I, II and III

B) Only I and III

C) Only II and III

D) No error

FACE Prep

Question

Directions: In this question, a sentence has been divided into four parts marked as I, II, III and IV. You need to find which part/parts does not/do not have any error in terms of grammatical or contextual usage. If the sentence is absolutely correct, mark (E) as your answer.

I. The consumer economy scorecard
II. of India appear particularly bleak,
III. with vehicle sales growth losing momentum,
IV. After hinting at a turnaround in the previous month.

FACE Prep

Question

A) Only I and II

B) Only I , III and IV

C) Only I and III

D) Only II

E) No error

FACE Prep

Question

Directions: In this question, a sentence has been divided into four parts marked as I, II, III and IV. You need to find which part/parts does not/do not have any error in terms of grammatical or contextual usage. If the sentence is absolutely correct, mark (E) as your answer.

I. With new tools and new expeditions,
II. Scientists are peering into Antarctica's nooks and crannies
III. And even its subsurface, discovering a world
IV. That seems unimaginable from more temperate climes.

(TCS 2022)

FACE Prep

Question

A) Only I and II

B) Only I and III

C) Only II and III

D) Only II

E) No error

FACE Prep

Question

Choose the most appropriate option to change the voice(active/ passive) form of the given sentence.

I am to do it immediately.

A) It is supposed to be done by me immediately.

B) It has to be done by me immediately.

C) It is to be done by me immediately.

D) It is being done by me immediately.

FACE Prep

Question

Identify the best way to improve the bold part of the given sentence. If there is no improvement required, select ‘ No improvement’.

In today’s time, many private companies **follow the concept** of corporate and social responsibility**.**

A) Follows the concept

B) Follow the concepts

C) Follows the concepts

D) No improvement

FACE Prep

Question

Choose the most appropriate option to change the Narration (direct/indirect) form of the given sentence (TCS 2019)

Meena inquired of me why I had done that.

A) Meena told me, “Why did you do that?”

B) Meena said to me, “Why did you do that?”

C) Meena said to me, “Why did you do this?”

D) Meena asked me, “Why did you do this?”

FACE Prep

Question

Identify the best way to improve the bold part of the given sentence. If there is no improvement required, select ‘ No improvement’

The blogger **asks their followers** to share the feedback on his recent post.

A) Asked their follower

B) Asked his followers

C) Asks his followers

D) No improvement

FACE Prep

Question

Identify the segment in the sentence, which contains the grammatical error.

The egg were fragile, with the thin outer shell protecting its liquid interior until it was put in the boiling water.

A) With the thin outer shell

B) The egg were fragile

C) It was put in the boiling water

D) Protecting its liquid interior until

FACE Prep

Question

Choose the most appropriate option to change the voice (active/passive) form of the given sentence

Somebody built the house last year.

A) The house has been built last year

B) The house was built last year by somebody.

C) The house was being built last year by somebody.

D) The house was built last year.

FACE Prep

Question

Identify the segment in the sentence, which contains the grammatical error.

Once upon a time, a daughter complained to her father that his life was miserable and that she didn’t know how she was going to make it.

A) Life was miserable and that she

B) Complained to her father that his

C) Didn’t know how she was going to make it.

D) Once upon a time, a daughter

FACE Prep

Question

Choose the most appropriate option to change the voice(active/ passive) form of the given sentence (TCS 2022)

A book has been given to me by him last week.

A) He has been given me a book last week.

B) He has given me a book last week.

C) He has gave me a book last week.

D) A book has to be given to me last week.

FACE Prep

Question

Identify the best way to improve the bold part of the given sentence. If there is no improvement required, select ‘ No improvement’

In order to be successful, students must inculcate a **habits of discipline** routine.

A) habit of discipline

B) habit of disciplined

C) habits of disciplined

D) No improvement

FACE Prep

Question

Identify the best way to improve the bold part of the given sentence. If there is no improvement required, select 'No improvement'

Parents should encourage their **child to participating** in recreational activities as well.

A) Children to participate

B) Children for participation

C) Child to participate

D) No improvement

FACE Prep

Question

Given below are four jumbled sentences. Pick the option that gives their correct order.

A. The shift to stringent emission norms, the transition to electrification, and potentially changing consumer preferences with shared mobility and a connected ecosystem.
B. Change may be the only constant, but the pace with which it takes place is not.
C. There are primarily three factors causing this disruption.
D. The automotive industry is seeing a spectrum of disruptions, as we move towards a cleaner, safer and convenient transport eco system.

FACE Prep

Question

A) DACB

B) CABD

C) CDAB

D) BDCA

FACE Prep

Question

Given below are four jumbled sentences. Pick the option that gives their correct order.

A. Especially where the penetration of mobile telephony and social media exceeds that of education and awareness.
B. Also because it may induce people to act on the information.
C. Fake news is a menace because it is usually motivated by an intent to deceive and misinform.
D. This can have grave consequences beyond imagination.

FACE Prep

Question

A) DACB

B) CBDA

C) CDAB

D) DBAC

FACE Prep

Question

Given below are four jumbled sentences. Pick the option that gives their correct order.

A. Banijay’s acquisition of the Endemol shine group globally may still be a work in progress , but the latter is already working on the Dutch show Penoza for India.

B. The Indian arm of Paris based company Banijay group is developing the Indian versions of American reality show Survivor and legal thriller Damages, Belgian TV series The Mole and Turkish drama Public Enemy.

C. Mean while, Sameer Nair- led Applause Entertainment is bringing the Indian versions of Israeli shows your Honour and Fauda.

D. New- age content creators on video streaming services are now looking at popular international formats that they can

FACE Prep

Question

A) DBAC

B) CABD

C) CDAB

D) BDCA

FACE Prep

THANK YOU